Texts in Applied Mathematics 80

Maria Isabel Bueno Cachadina
Javier Perez Alvaro

Matrix Methods in Data Analysis

MOREMEDIA

Springer

Texts in Applied Mathematics

Volume 80

Editors-in-Chief

Anthony Bloch, University of Michigan, Ann Arbor, USA

Charles L. Epstein, University of Pennsylvania, Philadelphia, USA

Alain Goriely, University of Oxford, Oxford, UK

Leslie Greengard, New York University, New York, USA

Series Editors

J. Bell, Lawrence Berkeley National Laboratory, Berkeley, USA

R. Kohn, New York University, New York, USA

P. Newton, University of Southern California, Los Angeles, USA

C. Peskin, New York University, New York, USA

R. Pego, Carnegie Mellon University, Pittsburgh, USA

L. Ryzhik, Stanford University, Stanford, USA

A. Singer, Princeton University, Princeton, USA

A. Stevens, University of Münster, Münster, Germany

A. Stuart, University of Warwick, Coventry, UK

T. Witelski, Duke University, Durham, USA

S. Wright, University of Wisconsin, Madison, USA

The mathematization of all sciences, the fading of traditional scientific boundaries, the impact of computer technology, the growing importance of computer modelling and the necessity of scientific planning all create the need both in education and research for books that are introductory to and abreast of these developments. The aim of this series is to provide such textbooks in applied mathematics for the student scientist. Books should be well illustrated and have clear exposition and sound pedagogy. Large number of examples and exercises at varying levels are recommended. TAM publishes textbooks suitable for advanced undergraduate and beginning graduate courses, and complements the Applied Mathematical Sciences (AMS) series, which focuses on advanced textbooks and research-level monographs.

Maria Isabel Bueno Cachadina ·
Javier Perez Alvaro

Matrix Methods in Data Analysis

Springer

Maria Isabel Bueno Cachadina
Department of Mathematics
and College of Creative Studies
University of California
Santa Barbara, CA, USA

Javier Perez Alvaro
Department of Mathematical Sciences
University of Montana
Missoula, MT, USA

ISSN 0939-2475 ISSN 2196-9949 (electronic)
Texts in Applied Mathematics
ISBN 978-3-032-11313-9 ISBN 978-3-032-11314-6 (eBook)
https://doi.org/10.1007/978-3-032-11314-6

Mathematics Subject Classification: 15-01, 65Fxx

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

Over the past decade, Data Science programs have grown quickly across American universities. This growth has prompted Math, Statistics, and Computer Science departments to offer new courses that cover the skills needed in this field. Machine Learning, a key part of Data Science, uses mathematical models and computational methods to make predictions from data. Over time, it has become clear that Linear Algebra and Optimization are two important foundations of Machine Learning. Because of this, many new math courses for Data Science focus on these two areas.

One of the authors of this book was asked by her department to develop such a course in Matrix Analysis. Understanding the matrix tools behind many Machine Learning algorithms is important for anyone aiming to work in Data Science, where processing, analyzing, and gaining insights from large datasets is essential.

Although students found the material in the first version of the course interesting, they wanted more examples connecting the theory to real-world problems. This showed us the gap between abstract mathematical concepts and their practical uses– an issue that inspired the writing of *Matrix Methods in Data Analysis*.

This book is designed not only to teach matrix theory but also to demonstrate how it directly supports the methods used in Machine Learning and, by extension, Data Science. While many textbooks focus either on the theory of Matrix Analysis or on its algorithmic applications, we believe that having both is important for anyone who wants to succeed in Data Science, Analytics, or Industrial Research. Our goal is to provide a solid theoretical background while showing a range of modern applications relevant to Machine Learning.

Target audience

Matrix Methods in Data Analysis is intended for undergraduate students who have completed a proof-based Linear Algebra course and are ready for more advanced topics. Some experience with a programming language like MATLAB, Julia, R, or Python is helpful. A strong interest in applied mathematics is important, since real-world applications make up a large part of the book.

Structure of the book

This book is designed to be flexible for different teaching approaches and course lengths, making it suitable for both one-semester and year-long classes.

It is divided into eleven parts, each containing five chapters (with the exception of Part I):

- **Chapter 1: Why Should We Care?** Introduces the Machine Learning/Data Science applications that motivate the matrix tools covered in that part of the book.
- **Chapter 2: What You May/Must Have Learned Before**. Provides a short review of Linear Algebra or summarizes results from earlier parts of this book, making sure readers have the necessary background.
- **Chapter 3: Core Topics.** Presents the main theoretical material, including key theorems, proofs, and concepts.
- **Chapter 4: Supplementary Topics.** Offers more advanced ideas beyond the core content, for readers who want a deeper exploration.
- **Chapter 5: From the Classroom to Real Life.** Shows how the theory from Chapter 3 (and sometimes Chapter 4) is used in practical Machine Learning/Data Science problems.

This structure allows for different levels of depth. If you have limited time, you can focus on Chapters 1, 3, and 5 for each part, forming a single-semester course that covers the essentials of Matrix Analysis for Machine Learning (recommended for students with a solid background in upper-division Linear Algebra). For a more in-depth, year-long course, adding Chapters 2 and 4 provides a more thorough exploration of foundational and advanced topics. Chapter 2 helps those who have only taken lower-division math to catch up, while Chapter 4 enriches the overall content.

As a result, the book is accessible to a wide range of students, especially those interested in Data Science or Industrial Research. You can also cover each part on its own, though later chapters build on earlier material, especially when covering algorithms from numerical Linear Algebra (like the power method or QR algorithm) and matrix factorizations (such as LU, QR, or SVD) used in Machine Learning.

Errata and comments

No book of this length is free from mistakes, and despite our best efforts, some errors and typographical mistakes are inevitable. We warmly welcome comments, corrections, and suggestions from readers. If you find an error, notice an omission, or have a suggestion for improving the exposition, please contact us at

`javier.perez-alvaro@mso.umt.edu` or `mbueno@ucsb.edu`.

An up-to-date errata list will be maintained at

https://github.com/um-perez-alvaro/Matrix-Methods-in-Data-Analysis.

We hope this book helps students not only master the mathematical tools they need, but also see how these tools are used in the fast-changing world of Machine Learning. Enjoy exploring Matrix Analysis, and may it lead you to exciting opportunities in Data Science and beyond!

Acknowledgements

The first author wishes to thank a few individuals. First, I acknowledge Gilbert Strang. Although I never met him, discovering his *Introduction to Applied Mathematics* with my friend David as an undergraduate sparked my interest in linear algebra and transformed my view of it from a study of abstract linear functions into a fascinating subject full of practical applications. Next, I thank Froilán, Fernando, Julio, and Juanma at Carlos III. Their teaching during my graduate studies kept the passion for Linear Algebra and Matrix Analysis alive and burning. Finally, I thank my colleague and co-author, Maribel. Several years ago, her suggestion to write a book set this project in motion.

As the second author, I would like to express my sincere gratitude to the individuals who made this work possible. To my co-author, thank you for the countless hours we spent turning an idea into a reality. This book would not exist without your essential skills and knowledge.

To my graduate advisor, Froilan M. Dopico, who first introduced me to the world of Matrix Analysis and Applied Linear Algebra, thank you. Your passion for research and teaching is contagious. To my colleague, Hector Ceniceros, thank you for inspiring this project back when I knew nothing about Machine Learning.

And to my family–my husband Jesus, my daughter Diana, my son David, and our beautiful doggies Bella and Lucky–thank you for your endless support, unconditional love, and appreciation for my work over the years.

Competing Interests The author has no competing interests to declare that are relevant to the content of this manuscript.

Contents

Part I LINEAR ALGEBRA AND MACHINE LEARNING 1

1 I-WHY SHOULD WE CARE? 3
1.1 What is Machine Learning? . . . 3
1.2 Supervised Learning . . . 3
1.2.1 Regression . . . 4
1.2.2 Classification . . . 5
1.2.3 Supervised dimensionality reduction . . . 5
1.3 Unsupervised Learning . . . 6
1.3.1 Clustering . . . 7
1.3.2 Dimensionality reduction . . . 7
1.3.3 Information retrieval . . . 7
1.4 Reinforcement Learning . . . 8
1.5 Linear Algebra is not enough . . . 9
Problems . . . 10

2 I-WHAT YOU MAY HAVE LEARNED BEFORE 11
2.1 Vectors and matrices . . . 11
2.2 Submatrices and the colon notation . . . 13
2.3 Sparse matrices and non-zero patterns . . . 14
Problems . . . 15

3 I-CORE TOPICS 17
3.1 Encoding data into matrices . . . 17
3.2 Tabular data as matrices . . . 17
3.3 Images as matrices . . . 21
3.4 Graphs as matrices . . . 22
3.5 Text documents as matrices . . . 28
3.6 Preferences and ratings as matrices . . . 33
Problems . . . 33

4 I-SUPPLEMENTARY TOPICS 39
4.1 An introduction to Python and Jupyter notebooks . . . 39
4.2 How to start a Jupyter notebook . . . 39
4.3 Python libraries and the book's GitHub repository . . . 39
Problems . . . 41

Part II MATRIX MULTIPLICATION AND PARTITIONED MATRICES 43

5 II-WHY SHOULD WE CARE? 45
5.1 The Erdős number 45
5.2 The most important nodes in a graph 46

6 II-WHAT YOU MAY HAVE LEARNED BEFORE 49
6.1 Addition, scalar multiplication, and transpose of matrices 49
6.2 Matrix multiplication 51
6.3 Invertible matrices and their properties 56
6.4 Linear transformations and their matrix representation 60
Problems 63

7 II-CORE TOPICS 67
7.1 Standard matrix multiplication: new perspectives 67
7.1.1 Matrix-vector and vector-matrix multiplication 67
7.1.2 The product Ax as a linear transformation 69
7.1.3 Matrix-matrix multiplication 72
7.1.4 The product AB as a composition of linear transformations 76
7.1.5 Matrix factorizations 76
7.2 Partitioned/Block matrices 77
7.2.1 Addition, scalar multiplication, and transpose of block matrices 78
7.2.2 Multiplication of block matrices 80
7.2.3 Inverse of block matrices 81
Problems 83

8 II-SUPPLEMENTARY TOPICS 87
8.1 The Hadamard matrix product 87
8.2 The Kronecker matrix product 88
Problems 89

9 II-FROM THE CLASSROOM TO REAL LIFE 93
9.1 The number of walks of a given length in a graph 93
9.2 The Katz centrality measure 97
Problems 101

Part III NORMS, DISTANCES, AND SIMILARITIES 105

10 III-WHY SHOULD WE CARE? 107
10.1 A classification problem: the Palmer Archipelago penguin data . 107
10.2 A clustering problem: Old Faithful eruption data 109
10.3 Information retrieval and query matching 110

11 III-WHAT YOU MAY HAVE LEARNED BEFORE 113
11.1 Real vector spaces 113

11.2 Norm function 114
11.3 Angle between vectors 118
Problems 119

12 III-CORE TOPICS 121
12.1 Examples of norms in $\mathbb{R}^n$ 121
12.1.1 Equivalent norms 124
12.2 Examples of matrix norms 126
12.2.1 Matrix norms via column stacking 126
12.2.2 Induced matrix norms 129
12.2.3 The submultiplicativity property 134
12.3 Distance functions 136
12.3.1 Distance induced by a norm 137
12.3.2 The angular distance 141
12.3.3 Spherical distance 142
12.3.4 The Hamming and Levenshtein distances 143
12.3.5 The Jaccard distance 146
12.3.6 Distance between nodes in undirected graphs 149
12.4 Similarity functions 152
12.4.1 Cosine similarity 152
12.4.2 Pearson's correlation coefficient: a special case of cosine similarity 154
12.4.3 Set-similarity measures 155
Problems 159

13 III-SUPPLEMENTARY TOPICS 169
13.1 Norms and distances in the complex vector space $\mathbb{C}^n$ 169
13.2 Pseudodistances 171
13.3 Distance-based similarities 172
13.4 Building graphs from pairwise distances or similarities 173
Problems 175

14 III-FROM THE CLASSROOM TO REAL LIFE 177
14.1 Feature scaling in distance-based algorithms 177
14.2 The k-Nearest Neighbors (k-NN) method 179
14.2.1 The k-NN method for classification 179
14.2.2 The k-NN method for regression 181
14.3 The k-means method for clustering 182
14.4 Hamming error correcting codes 189
14.5 The closeness centrality measure 194
14.6 Information retrieval 197
14.7 Movie recommender systems 199
Problems 201

Part IV THE FOUR FUNDAMENTAL SUBSPACES OF A MATRIX, AND GRAM-MATRICES 205

15 IV-WHY SHOULD WE CARE? 207
15.1 The regression line and data modeling 207

16 IV-WHAT YOU MAY HAVE LEARNED BEFORE 211
16.1 Subspaces of $\mathbb{R}^n$ 211
16.2 Linear combinations and span 212
16.3 Linearly independent sets, basis, and dimension 212
16.4 The sum of two subspaces 214
16.5 Elementary row and column operations 218
16.6 Row and column equivalence 224
16.7 Echelon form and reduced-row echelon form 226
Problems 227

17 IV-CORE TOPICS 231
17.1 The column and row spaces 231
17.2 The right and left nullspaces 238
17.3 The rank of a matrix 242
17.3.1 The rank-nullity theorem and the fundamental theorem of Linear Algebra 247
17.3.2 The rank of AB and $A + B$ 249
17.3.3 Characterization of rank-k matrices and full-rank factorizations 251
17.4 Gram-matrices 253
17.4.1 Full-rank factorizations revisited 255
17.4.2 Right and left inverses 256
Problems 258

18 IV-SUPPLEMENTARY TOPICS 261
18.1 Column and row factorizations of a matrix 261
18.1.1 The CR Factorization 261
18.1.2 The CUR factorization 265
18.2 Equalities and inequalities involving matrix ranks 270
18.2.1 Rank of 2×2 block matrices 270
18.2.2 Frobenius' and Sylvester's inequalities 275
Problems 276

19 IV-FROM THE CLASSROOM TO REAL LIFE 279
19.1 The regression line 279
19.2 Applications of full rank factorizations 284
19.2.1 Storing a large but low-rank matrix efficiently 285
19.2.2 Efficient matrix-vector multiplication with low-rank matrices 285
19.2.3 Topic modeling 285
Problems 287

Part V THE LU FACTORIZATION OF A MATRIX 289

20 V-WHY SHOULD WE CARE? 291
20.1 Solving linear systems of equations with several right-hand sides 291
20.2 Computing the determinant of a matrix 294
20.3 Label propagation 294

21 V-WHAT YOU MAY HAVE LEARNED BEFORE 297
21.1 Solving linear systems by Gaussian elimination 297
Problems 299

22 V-CORE TOPICS 301
22.1 The LU Factorization of a matrix 301
22.1.1 Solving linear systems of equations using the LU factorization 308
22.1.2 Existence and uniqueness of the LU factorization when A is nonsingular 310
22.2 The LDL^T factorization for symmetric matrices 314
22.3 Gaussian elimination with partial pivoting and the $PA = LU$ factorization 316
22.3.1 Permutation matrices 318
22.3.2 The $PA = LU$ factorization 320
22.3.3 Solving linear systems by using the PLU factorization . 326
Problems 327

23 V-SUPPLEMENTARY TOPICS 331
23.1 Existence and uniqueness of LU factorizations 331
Problems 336

24 V-FROM THE CLASSROOM TO REAL LIFE 339
24.1 Graph Laplacian linear systems in Data Science 339
24.1.1 The graph Laplacian matrix 339
24.1.2 The label propagation problem 341
Problems 347

Part VI ORTHOGONALITY AND THE QR FACTORIZATION 349

25 VI-WHY SHOULD WE CARE? 351
25.1 Applications of orthogonal matrices 351
25.2 Applications of the QR factorization 352

26 VI-WHAT YOU MAY HAVE LEARNED BEFORE 355
26.1 Orthogonal sets and their properties 355
26.2 Turning bases into orthonormal bases: The Gram-Schmidt process 360
Problems 366

27 VI-CORE TOPICS 369
27.1 Orthogonal complements . . . 369
27.1.1 The orthogonality of the four fundamental subspaces . . 374
27.2 Orthogonal matrices and their properties . . . 376
27.3 Givens rotators and Householder reflectors . . . 381
27.3.1 Givens rotators . . . 382
27.3.2 Householder reflectors . . . 387
27.4 The QR factorization . . . 391
27.4.1 The full column rank case . . . 392
27.4.2 The general case . . . 399
27.4.3 The Householder algorithm for computing the QR factorization . . . 401
27.5 QR factorization with column pivoting . . . 406
27.5.1 Householder algorithm with column pivoting . . . 408
Problems . . . 414

28 VI-SUPPLEMENTARY TOPICS 421
28.1 Decomposing orthogonal matrices into products of Givens rotators and Householder reflectors . . . 421
28.2 Updating the QR factorization . . . 423
28.2.1 Adding a row . . . 424
28.2.2 Deleting a row . . . 425
28.2.3 Adding a column . . . 428
28.2.4 Deleting a column . . . 429
28.3 Reducing matrices to simpler forms . . . 430
28.3.1 Hessenberg reduction . . . 430
28.3.2 Bidiagonal reduction . . . 434
Problems . . . 438

29 VI-FROM THE CLASSROOM TO REAL LIFE 441
29.1 Denoising and compressing data: a dimensionality reduction technique . . . 442
29.1.1 The Haar basis and the Haar transform . . . 442
29.1.2 The discrete Haar transform . . . 445
29.1.3 Compressing and removing noise with the Haar transform 451
Problems . . . 454

Part VII ORTHOGONAL PROJECTIONS AND THE LEAST SQUARES PROBLEM 457

30 VII-WHY SHOULD WE CARE? 459
30.1 Dimensionality reduction . . . 459
30.1.1 Eliminating redundant information . . . 459
30.1.2 The intrinsic dimension of data . . . 460
30.1.3 Data visualization . . . 461
30.2 Beyond the regression line . . . 462

30.2.1 Polynomial regression 462
30.2.2 Multilinear regression 463

31 VII-WHAT YOU MAY HAVE LEARNED BEFORE 465
31.1 Orthogonal projections 465
Problems 466

32 VII-CORE TOPICS 469
32.1 Orthogonal projection matrices and their properties 469
32.2 The best approximation theorem 476
32.3 Least squares problems 478
32.4 Solving least squares problems with the QR factorization 480
32.4.1 The full column rank case 480
32.4.2 The general case 481
Problems 483

33 VII-SUPPLEMENTARY TOPICS 487
33.1 Variants of least squares problems 487
33.1.1 Weighted least squares 487
33.1.2 Regularized least squares 489
33.1.3 Linearly constrained least squares 491
33.2 Applications of least squares variants 496
33.2.1 Weighted least squares regression 496
33.2.2 The distortion caused by data transformations 497
33.2.3 Regularized least squares regression 499
33.2.4 Piecewise polynomial regression 501
Problems 503

34 VII-FROM THE CLASSROOM TO REAL LIFE 509
34.1 Least squares data fitting 509
34.1.1 Polynomial least squares regression 509
34.1.2 Generalized least squares regression 511
34.1.3 Multilinear least squares regression 514
34.2 Fitting an implicit equation 517
34.3 Transforming data for least squares regression 519
34.3.1 Exponential model fitting 519
34.3.2 Logistic model fitting 522
34.4 Local least squares regression 524
Problems 526

Part VIII EIGENVALUES, EIGENVECTORS, AND ALGORITHMS 531

35 VIII-WHY SHOULD WE CARE? 533
35.1 Three applications of eigenvalues and eigenvectors in Machine Learning 533
35.1.1 Matrix powers 533

35.1.2 Matrix nearness problems . . . 533
35.1.3 Optimization problems . . . 534
35.2 Corner detection in computer vision . . . 534

36 VIII-WHAT YOU MAY HAVE LEARNED BEFORE 537
36.1 Eigenvalues and eigenvectors . . . 537
36.2 Similar matrices and diagonalizability . . . 546
36.2.1 Characterizations of diagonalizable matrices . . . 549
36.3 Non-diagonalizable matrices and the Jordan canonical form . . . 553
36.4 Invariant subspaces . . . 555
Problems . . . 557

37 VIII-CORE TOPICS 563
37.1 Eigenvalues and eigenvectors of special matrices . . . 563
37.2 Powers of matrices . . . 570
37.2.1 Matrix powers when A is diagonalizable . . . 571
37.2.2 Applying a diagonalizable matrix repeatedly to vectors . 573
37.2.3 Matrix powers when A is not diagonalizable . . . 575
37.2.4 Some applications of matrix powers . . . 578
37.3 The Schur form and the spectral theorem for normal matrices . . 581
37.4 Numerical methods for computing eigenvalues and eigenvectors 585
37.4.1 Using the characteristic polynomial: the naive way . . . 585
37.4.2 The power iteration method . . . 588
37.4.3 The shift-and-invert iteration method . . . 597
37.4.4 The QR algorithm . . . 600
37.4.5 Practical considerations for the QR algorithm . . . 607
37.4.6 The real symmetric QR algorithm . . . 615
Problems . . . 616

38 VIII-SUPPLEMENTARY TOPICS 625
38.1 Localizing the eigenvalues of a matrix: Gershgorin's circle theorem 625
38.2 The Schur factorization and invariant subspaces . . . 629
38.3 More on algorithms for computing eigenvalues . . . 634
38.3.1 Updating the shift: The Rayleigh quotient iteration . . . 634
38.3.2 Orthogonal iteration and the QR algorithm . . . 636
38.3.3 From orthogonal iteration to the QR algorithm . . . 640
38.4 The Arnoldi procedure . . . 641
38.4.1 Partial Hessenberg reduction and the Arnoldi algorithm 641
38.4.2 Krylov subspaces . . . 647
38.4.3 Ritz values and Ritz vectors . . . 649
38.4.4 The Arnoldi process in action: numerical examples . . . 650
Problems . . . 654

39 VIII-FROM THE CLASSROOM TO REAL LIFE 659
39.1 The Harris method for corner detection . . . 659
Problems . . . 664

Part IX SYMMETRIC AND POSITIVE DEFINITE MATRICES 667

40 IX-WHY SHOULD WE CARE? 669
40.1 Clustering when k-means does not work . . . 669
40.2 Dimensionality reduction and class separability . . . 670

41 IX-WHAT YOU MUST HAVE LEARNED BEFORE 673
41.1 Symmetric matrices and the spectral theorem . . . 673
41.2 Change of basis and the geometric interpretation of real symmetric matrices . . . 674
41.3 Projection decomposition of a symmetric matrix . . . 678
Problems . . . 679

42 IX-CORE TOPICS 681
42.1 Symmetric matrices and quadratic forms . . . 681
42.1.1 Classifying quadratic forms . . . 684
42.1.2 Constrained optimization of quadratic forms . . . 687
42.2 Positive (semi)definite matrices and their characterizations . . . 691
42.2.1 Eigenvalue characterization . . . 694
42.2.2 Leading principal minor characterization . . . 695
42.2.3 Pivot characterization . . . 700
42.2.4 Gram characterization and the Cholesky factorization . 702
42.2.5 Square root characterization . . . 705
42.3 Positive (semi)definite matrices in Machine Learning applications 707
42.3.1 The graph Laplacian matrix and its properties . . . 707
42.3.2 The Hessian matrix: minimization and convexity . . . 713
42.3.3 The covariance matrix . . . 715
Problems . . . 720

43 IX-SUPPLEMENTARY TOPICS 729
43.1 Geometric insights on positive definite quadratic forms: The Principal Axes Theorem . . . 729
43.2 Extensions of the Rayleigh-Ritz theorem . . . 732
43.2.1 Optimization of a quadratic form over an ellipsoid . . . 733
43.2.2 Trace optimization problems . . . 735
43.3 The Cholesky algorithm . . . 738
43.4 More characterizations of positive semidefinite matrices . . . 741
43.4.1 Principal minor characterization . . . 741
43.4.2 Gram-matrix characterization and the semi-Cholesky factorization . . . 744
43.4.3 Pivot characterization . . . 746
Problems . . . 748

44 IX-FROM THE CLASSROOM TO REAL LIFE 751
44.1 Graph clustering methods . . . 751
44.1.1 Graph spectral partitioning and the Fiedler vector . . . 751
44.1.2 Spectral clustering . . . 758

44.2 Dimensionality reduction methods . . . 762
44.2.1 Feature selection via class-separation variance . . . 762
44.2.2 Linear discriminant analysis . . . 767
44.2.3 Spectral embedding . . . 775
44.2.4 Multidimensional scaling . . . 780
Problems . . . 784

Part X SINGULAR VALUE DECOMPOSITION 789

45 X-WHY SHOULD WE CARE? 791
45.1 Data compression and low-rank approximations . . . 791
45.1.1 Image compression . . . 792
45.1.2 Storing weather data . . . 792

46 X-WHAT YOU MUST HAVE LEARNED BEFORE 795
Problems . . . 797

47 X-CORE TOPICS 799
47.1 The singular value decomposition . . . 799
47.1.1 Proof of Theorem 47.1: existence of the singular value decomposition . . . 802
47.1.2 Properties of the singular value decomposition . . . 805
47.1.3 When is the singular value decomposition unique? . . . 807
47.1.4 The geometry of the singular value decomposition . . . 809
47.1.5 The reduced singular value decomposition . . . 811
47.2 The singular value decomposition and optimization . . . 813
47.3 The pseudoinverse matrix . . . 815
47.3.1 Definition and characterizations of the pseudoinverse . 817
47.3.2 Pseudoinverse of some special matrices . . . 821
47.3.3 Properties of the pseudoinverse . . . 822
47.3.4 Linear systems and least squares problems revisited: minimum norm solutions . . . 826
Problems . . . 827

48 X-SUPPLEMENTARY TOPICS 833
48.1 Characterization of all the reduced SVDs of a matrix . . . 833
48.2 The Polar decomposition and two distance problems . . . 837
48.3 More characterizations of the pseudoinverse . . . 840
48.4 Algorithms for computing the SVD . . . 843
Problems . . . 847

49 X-FROM THE CLASSROOM TO REAL LIFE 851
49.1 The best low-rank approximation . . . 851
49.2 Principal component analysis . . . 856
49.2.1 The first principal component: finding the line that maximizes data spread . . . 859
49.2.2 Second and higher principal components . . . 864

49.2.3 PCA representations and reconstructions 866
49.2.4 Principal component analysis via the SVD 868
49.2.5 Choosing the number of principal components: the total variance explained 869
49.2.6 Global vs. iterative principal components 874
49.2.7 The least reconstruction error 876
49.2.8 Practical applications of PCA 877
49.2.9 Revisiting the weather data example 879
49.2.10 Principal component analysis vs. linear discriminant analysis 882
Problems 883

Part XI NONNEGATIVE MATRICES AND PERRON THEORY 887

50 XI-WHY SHOULD WE CARE? 889
50.1 Number of walks in a graph 889
50.2 Page ranking for a web search engine 891
50.3 Prediction of change of population 891

51 XI-WHAT YOU MUST HAVE LEARNED BEFORE 893
51.1 Matrix norms and the submultiplicative property 893
51.2 Eigenvalues and eigenvectors 894
51.3 The Jordan canonical form 895
51.4 Graphs and the adjacency matrix 896
51.5 Limit of a sequence of matrices 897

52 XI-CORE TOPICS 899
52.1 Nonnegative matrices: definition and properties 899
52.1.1 Order in the set of matrices 901
52.2 The spectral radius 903
52.3 Spectral properties of square positive matrices 911
52.4 Spectral properties of square nonnegative matrices 913
52.4.1 Characterization of irreducible and primitive nonnegative matrices 917
52.5 Stochastic matrices: a special class of nonnegative matrices . . . 927
52.6 The theory of discrete time Markov chains 930
52.6.1 The definition of Markov chains 934
52.6.2 Irreducible and aperiodic Markov chains 942
52.6.3 Limiting distributions 944
52.6.4 Stationary distributions 950
Problems 958

53 XI-SUPPLEMENTARY TOPICS 967
53.1 Number of walks of length k in a graph 967
Problems 972

54 XI-FROM THE CLASSROOM TO REAL LIFE 975
54.1 Page ranking for a web search engine 975
Problems 989

References 995

Index 999

Part I
LINEAR ALGEBRA AND MACHINE LEARNING

The first part of this book introduces the connections between Linear Algebra and Machine Learning. It starts with an overview of the field of Machine Learning and its three main areas: Supervised, Unsupervised, and Reinforcement Learning. Next, it reviews vectors and matrices and their use in storing and working with data. You will learn how many data types, such as tabular data, images, graphs, and textual data, can be encoded into matrices. This part also introduces Python and Jupyter notebooks, standard tools for hands-on coding, experimentation, and visualization in Machine Learning and Data Science.

Chapter 1
I-WHY SHOULD WE CARE?

1.1 What is Machine Learning?

In today's data-driven world, we collect massive amounts of information, often not knowing which parts will be important in the future. This results in databases filled with vast, unstructured data. The challenge of extracting valuable insights from these databases is the essence of Data Science.

Data Science is an interdisciplinary field that combines ideas and techniques from Computer Science, Statistics, Linear Algebra, and Optimization. It involves various methods and algorithms to uncover hidden patterns in raw data.

Machine Learning, a subset of Data Science, allows computers to learn from data. Instead of being explicitly programmed, Machine Learning models analyze data to learn patterns and make informed decisions.

This textbook's main focus is on Linear Algebra and Matrix Analysis. While these topics have wide-ranging uses in applications, we will concentrate on how we use them in Machine Learning.

In this chapter, we examine different aspects of Machine Learning that we will discuss in later chapters. Broadly speaking, we can divide the field of Machine Learning into three main subfields: Supervised Learning, Unsupervised Learning, and Reinforcement Learning. Figure 1.1 illustrates this division, along with some applications specific to each field. The next sections explain the main goal of each of these fields and present some examples of their applications in more detail.

1.2 Supervised Learning

Put simply, **Supervised Learning** builds a model that predicts an output from a given input. In Supervised Learning, we use a labeled dataset $\{(x_i, y_i)\}_{i=1}^{m}$, where each x_i is an input and y_i its corresponding output (or label). Although x_i is often a numeric vector, it may also be text or an image. The goal is to "learn" a function, or model, f that maps each input x_i to its output y_i. We use this model to predict the outputs of new, unseen inputs.

Based on the type of labels y_i, supervised learning is usually split into regression and classification. Some dimensionality reduction methods can also be treated as supervised learning.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_1

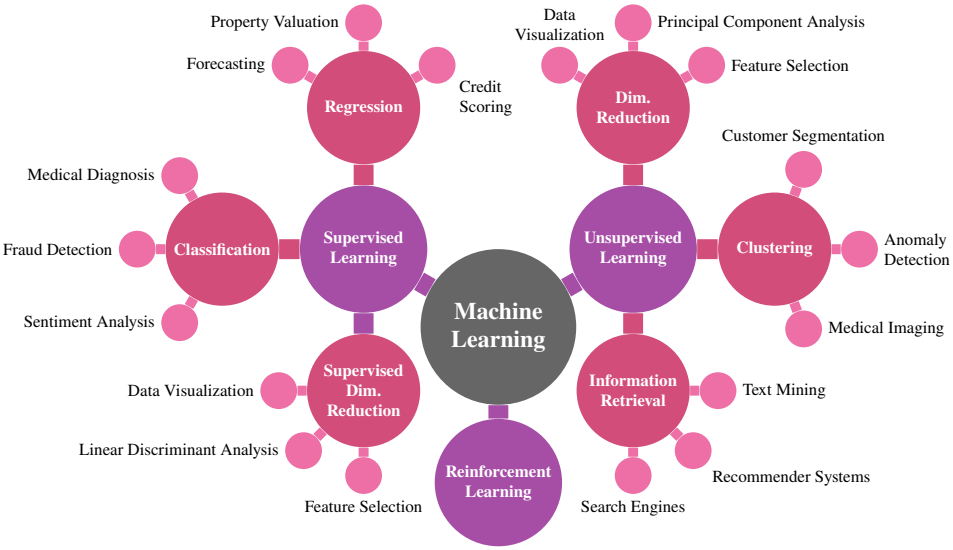


Fig. 1.1: Main machine learning fields (supervised, unsupervised, and reinforcement learning), their subfields (regression, classification, dimensionality reduction, clustering, and information retrieval), and some applications.

1.2.1 Regression

In **regression**, the outputs y_i belong to some interval of real numbers. Thus, they belong to a continuous set.

Example 1.1. (Predicting house prices)

A classic regression task is predicting house prices. The model uses factors such as living-room size, location (e.g., distance to schools), house age, and so on. Each house is represented by a feature vector x that records the numerical values of these features:

$$x = \begin{bmatrix} 200 \\ 5.5 \\ 2.5 \\ \vdots \end{bmatrix} \begin{array}{l} \longleftarrow \text{Size of the living room (in square feet)} \\ \longleftarrow \text{House age (in years)} \\ \longleftarrow \text{Distance from a school (in miles)} \end{array}$$

Our goal is to learn a function f that maps x to the house price y. Because house prices vary over the continuous range $[0, \infty)$, predicting them is a regression task.

Example 1.2. (Predicting cardiovascular disease risk)

Consider another regression problem: estimating a patient's risk of cardiovascular disease. Heart attacks and strokes claim millions of lives each year, many before age 70, so early detection is important. A model uses a feature vector x containing age, sex, blood pressure, cholesterol, blood sugar, ECG results, and similar measures to output a probability $y \in [0, 1]$ that the patient will develop cardiovascular disease. Because y varies continuously between 0 and 1, this is a regression task.

In Chapters 19 and 34, we will introduce Linear Regression along with some variants. This simple yet widely used method is essential to many data-science applications we will explore later.

1.2.2 Classification

The other subfield of Supervised Learning, **classification**, works with discrete outputs. In a classification problem, y_i takes values from a finite, typically small, set of options called categories or classes. The goal is to determine which category a new data point belongs to.

Example 1.3. (Email spam classification)

Consider detecting spam e-mails, a common classification task. To combat this pervasive issue, we can use supervised learning to label each e-mail as spam or not spam (also known as ham).

Each e-mail is mapped to a feature vector x (e.g., keyword counts, sender info). We train a function f that outputs $y = 1$ for spam and $y = 0$ for ham. Because there are only two possible labels, this is a case of **binary classification**.

Example 1.4. (News article categorization)

Imagine automatically sorting news stories into categories such as politics, sports, or technology, helping readers find what interests them. Each article is encoded as a feature vector x, for example, using keyword frequencies. We train a function f that maps x to one of the predefined labels y. Because the label must be chosen from several (but finitely many) categories, this is a case of **multiclass classification**.

Throughout this book we will encounter several classification problems and algorithms. For example, Chapter 14 introduces the widely used k-nearest-neighbors algorithm.

1.2.3 Supervised dimensionality reduction

Many problems involve data sets with a large number of features. Not all of them matter for every task. When the data is labeled, supervised dimensionality reduction

methods reduce the number of features while keeping the information most useful for predicting the labels.

In Chapter 44, we will cover Linear Discriminant Analysis (LDA), which finds new features, formed as linear combinations of the original ones, that best separate the classes in the data. LDA is common in classification because it projects data to a lower-dimensional space while keeping the classes as far apart as possible.

Example 1.5. (Medical diagnosis)

A hospital collects data from hundreds of patients, each with dozens of lab test results (cholesterol, blood pressure, enzyme levels, etc.). Each record is labeled as either "Healthy" or "Has Disease". The goal is to predict health status using as few tests as possible without losing accuracy.

Linear Discriminant Analysis (LDA) uses the labels to find a new variable, a linear combination of the original test results, that best separates the two groups. For example, LDA might produce the new feature

$$\text{LDA variable} = 0.2 \times (\text{cholesterol}) + 0.5 \times (\text{enzyme A level}).$$

Instead of using all the original tests, we can focus on this single variable, which captures the information most useful for telling healthy and diseased patients apart. This helps doctors make faster and more accurate predictions with fewer measurements.

Example 1.6. (Email Spam Detection)

Consider classifying emails as "Spam" or "Not Spam". Suppose each email is described by hundreds of features: keyword presence, number of links, sender's domain, and more. Each email is labeled as spam or not.

With so many features, some may be irrelevant or add noise, making the task harder. Feature selection can find the features most useful for predicting spam.

For example, the presence of the word "free" or the number of hyperlinks may be strong spam indicators, while others, like the subject line length, may not help. By keeping only the most informative features, the spam filter becomes simpler, faster, and more accurate, focusing on what matters most for separating the labeled classes.

1.3 Unsupervised Learning

In **Unsupervised Learning** we work with unlabeled datasets $\{x_i\}_{i=1}^m$. The aim is to uncover hidden patterns and structure in the data. The three main tasks in unsupervised learning are clustering, dimensionality reduction, and information retrieval[1].

[1] These three tasks cover much of unsupervised learning, but not all of it. Other common aims include *generative modelling*, *anomaly detection*, and *feature learning*.

1.3.1 Clustering

Clustering aims to group similar data points. This helps uncover inherent patterns in the data.

Example 1.7. (Customer segmentation)

In business, knowing your customers is key. Companies often have vast amounts of customer data but need a way to make sense of it. Clustering can help by dividing customers into distinct groups based on traits such as buying habits, age, or location.

Consider a retailer with a year of purchase records. By using clustering, the company can identify different customer types: those who regularly buy home appliances, tech enthusiasts focused on electronic gadgets, shoppers who prefer seasonal items, etc. Identifying these groups lets the retailer target its marketing and personalize its services.

This book presents several clustering techniques. Chapter 14 covers the widely used k-means algorithm, while Chapter 44 introduces spectral clustering.

1.3.2 Dimensionality reduction

Dimensionality reduction aims to replace many variables in a data set with a smaller set, simplifying the data while preserving its essential structure.

Example 1.8. (Visualizing COVID-19 research)

During the COVID-19 pandemic, the rapid influx of research papers posed a challenge for health professionals to stay updated. To address this, the White House collaborated with leading research groups to develop the COVID-19 Open Research Dataset (CORD-19), which contained over a million scholarly articles on the virus and related topics [1].

Originally, these articles were represented as high-dimensional vectors based on keyword frequency. A dimensionality reduction technique known as t-SNE was used to map each document into a two-dimensional space, which helped group these documents into clusters, as Figure 1.2 shows. This visualization was further refined through topic modeling techniques to assign thematic labels to each cluster, thus enhancing navigability in this rich research repository [20].

In this book, we will introduce a variety of Linear Algebra-based dimensionality reduction techniques. Chapter 49 introduces the well-known Principal Component Analysis, and Chapter 44 discusses Linear Discriminant Analysis.

1.3.3 Information retrieval

The goal of **Information retrieval** is to locate relevant items (documents, images, records, etc) within a large collection in response to a query and rank them so the most useful appear first.

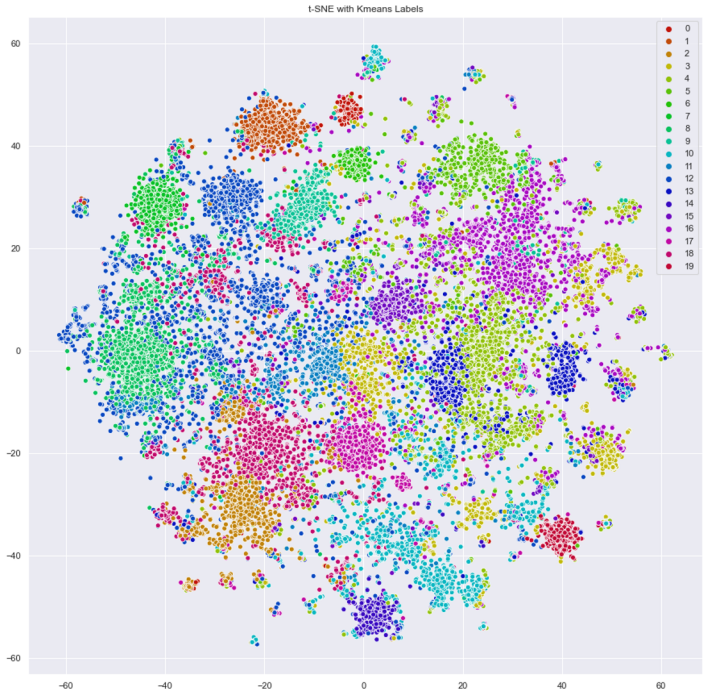


Fig. 1.2: Dimensionality reduction and clustering of the COVID-19 literature.

Example 1.9. (Information retrieval in digital libraries)

Consider the task of searching for relevant articles in digital libraries. With the increasing number of academic publications, finding specific research papers can be very difficult. Information retrieval techniques are important here.

For example, a researcher might look for recent articles on artificial intelligence in healthcare. A good information retrieval system uses algorithms to go through thousands of papers and find the most relevant ones. This not only saves time but also ensures that valuable research does not get lost in the large amount of information.

Information retrieval centers around the idea of similarity, which we will cover in Chapter 14. To determine a document's relevance to a search query, we measure its similarity to the query. The more similar the document, the more relevant it is.

1.4 Reinforcement Learning

Reinforcement Learning is a branch of Machine Learning where an "agent" learns to interact with its environment to achieve a goal or maximize a reward. It adapts through a continuous process of trial and error, refining its strategies based on the outcomes of its previous actions.

Example 1.10. (Self-driving cars)

Self-driving cars, or autonomous vehicles, can navigate roads and handle traffic on their own, without human help. They need to understand their environment, plan the best route, and make accurate moves. Reinforcement Learning is perfect for this purpose, as it lets these cars learn from experience and adjust to new situations.

While this area of Machine Learning is fascinating, we will not cover it in this book. Its techniques often rely more on concepts from Probability Theory and Optimization, rather than Linear Algebra. Thus, our focus will remain on Supervised and Unsupervised Learning problems and algorithms, where Linear Algebra plays a more central role.

1.5 Linear Algebra is not enough

While Linear Algebra lays the foundation for understanding Machine Learning and Data Analysis, it is just the starting point. To truly excel in these fields, one must also master the following domains or areas of expertise:

1. **Probability and Statistics:** Essential for understanding and interpreting data.
2. **Optimization:** Necessary for solving many mathematical problems in Data Analysis, Statistics, and Machine Learning. Mathematical problems in these areas often require maximizing or minimizing a function, possibly under certain constraints.
3. **Programming Proficiency:** A solid understanding of a programming language such as Python, Julia, R, or MATLAB is essential, along with familiarity with libraries for Machine Learning, Deep Learning, Data Analysis, and Statistics.

Here are some of our favorite recommended references for going in-depth into these areas.

Statistics for Machine Learning:

- *An Introduction to Statistical Learning: with Applications in Python*, by Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor [32].
- *The Elements of Statistical Learning*, by Trevor Hastie, Robert Tibshirani and Jerome Friedman [29].

Optimization and Machine Learning:

- *Optimization for Data Analysis*, by Stephen J. Wright, and Benjamin Recht [53].
- *Convex Optimization*, by Stephen Boyd and Lieven Vandenberghe [12].

Python for Machine Learning and Deep Learning:

- *Python for Data Analysis*, by Wes McKinney [40].
- *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, by Aurélien Géron [25].
- *Deep Learning with Python*, by François Chollet [13].

Problems

1.1 Decide whether each scenario is best described as a classification or a regression problem.

(a) You are developing a financial model to predict the future price of a particular stock based on historical stock prices, trading volumes, and other market data.
(b) As part of your social media sentiment analysis project, your objective is to classify user comments into "positive", "negative", or "neutral" sentiments based on their textual content.
(c) In a human resources project, you need to predict the annual salary of employees based on their years of experience, education level, job role, and other relevant factors.
(d) In a telecommunications company, you want to predict whether a customer will "churn" (cancel their subscription) or "stay" based on their usage patterns, billing history, and customer feedback.
(e) You are working for a customer service company, and your goal is to predict customer satisfaction scores based on various customer interactions and feedback. The satisfaction scores range from 1 to 5, with 1 being "very dissatisfied" and 5 being "very satisfied". You want to provide a numerical estimate of customer satisfaction.

1.2 For each scenario, determine whether it falls under Supervised Learning (regression or classification) or Unsupervised Learning (clustering or dimensionality reduction).

(a) A retail company wants to group its customers into segments for targeted marketing efforts. They have a dataset of customer purchase history and demographic information.
(b) You are developing a system that can recognize handwritten digits (0-9) from scanned documents. You have a labeled dataset of handwritten digits for training.
(c) An e-commerce platform wants to recommend products to users based on their past purchase history and the behavior of similar users.
(d) A language translation company aims to enhance the quality of its automated translation system. They have a dataset of translated sentences along with human-rated quality scores for each translation. The quality scores range from 1 (poor) to 5 (excellent) and are based on human assessments of fluency, accuracy, and readability. The goal is to build a model that can predict the translation quality score for new translations automatically.
(e) You are working on a project to group similar social media posts together for trend analysis. The posts are not labeled with specific categories.

Chapter 2
I-WHAT YOU MAY HAVE LEARNED BEFORE

This book assumes you have completed an introductory Linear Algebra course, have a basic understanding of Multivariable Calculus, and are somewhat familiar with complex numbers. While it is a good idea to refresh your knowledge of these topics, we will review the key concepts needed for the material in this book, especially when it helps to look at them from a new perspective.

2.1 Vectors and matrices

Linear Algebra deals with vectors and matrices with entries in some field[1]. In most of the book, we assume the field is the set of real numbers, $\mathbb{R}$, with standard addition and multiplication. In Chapter 37, we will also consider the field of complex numbers, $\mathbb{C}$.

Vectors are mathematical objects that you can scale or add to other vectors of the same size. You are probably most familiar with column vectors. A **column vector** is simply a collection of numbers arranged in a column. When a computer stores a vector, it stores the entries sequentially, in the same order as they appear in the column.

We perform vector operations elementwise:

$$\alpha \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} \alpha v_1 \\ \vdots \\ \alpha v_n \end{bmatrix}, \quad \text{for all scalars } \alpha, \quad \text{and}$$

$$\begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} + \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix} = \begin{bmatrix} v_1 + w_1 \\ \vdots \\ v_n + w_n \end{bmatrix}.$$

In this book, we mostly use column vectors, but row vectors (numbers arranged in a row) are also important. Every row vector is simply the **transpose** of a column vector:

[1] Intuitively, a "field" is a set of numbers where you can add, subtract, multiply, and divide (except by zero) following the usual rules of arithmetic.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_2

$$\begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix} = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix}^T .$$

We often represent vectors with two or three components as an "arrow" in the two- or three-dimensional space, as shown in Figure 2.1.

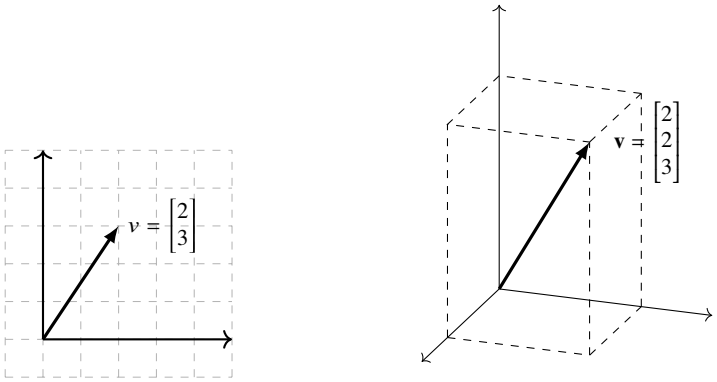


Fig. 2.1: Vector representation in two- and three-dimensional spaces.

We often associate a vector's arrow representation with its endpoint. Because of this, we will use the terms "vector" and "point" interchangeably. When we need to make a clear distinction between them, we will represent points as a sequence of numbers in parentheses:

$$\text{vector} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} \quad \longrightarrow \quad \text{point } (v_1, \ldots, v_n).$$

Another important concept in Linear Algebra is **matrices**, which are a generalization of column vectors. A **matrix** is a two-dimensional array of numbers.

In the matrix

$$A = \begin{bmatrix} A_{11} & \cdots & A_{1n} \\ A_{21} & \cdots & A_{2n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \cdots & A_{mn} \end{bmatrix},$$

there are m rows and n columns, so we call it an $m \times n$ matrix. If the number of rows equals the number of columns, we call the matrix *square*. Often, though, $m \neq n$; then the matrix is *rectangular*. We will often refer to a matrix as "tall-and-skinny" when $m > n$ and "short-and-wide" when $m < n$.

The element A_{ij} at row i and column j is called the (i, j)-th entry (or component) of the matrix A.

Two important matrices are the identity matrix I and the zero matrix 0. The **identity matrix** is square, with ones on the main diagonal and zeros everywhere else. The **zero matrix** can be square or rectangular and contains only zeros.

$$I = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \quad \text{and} \quad 0 = \begin{bmatrix} 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}.$$

We use subscripts to indicate their size: I_n for the $n \times n$ identity matrix and $0_{m\times n}$ for the $m \times n$ zero matrix (or simply 0_n for the $n \times n$ zero matrix).

2.2 Submatrices and the colon notation

Often, we will focus our attention not on a matrix but on a "piece" of a matrix. We call a matrix B a **submatrix** of A when we form B by selecting specific rows and columns from A.

For example, consider the matrix

$$A = \begin{bmatrix} 1 & 0 & 3 & -4 & 2 \\ 2 & 2 & -3 & 0 & 0 \\ 0 & -1 & 2 & 5 & -3 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

We can create a submatrix of A by choosing rows 1 and 3, along with columns 2 and 5, as highlighted in red below:

$$A = \begin{bmatrix} 1 & 0 & 3 & -4 & 2 \\ 2 & 2 & -3 & 0 & 0 \\ 0 & -1 & 2 & 5 & -3 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix} \quad \rightarrow \quad B = \begin{bmatrix} 0 & 2 \\ -1 & -3 \end{bmatrix}.$$

Let us formalize this concept. To create a submatrix of an $m \times n$ matrix A, we select a set of p row indices $r = \{i_1, i_2, \ldots, i_p\}$ where $1 \leqslant i_1 < i_2 < \cdots < i_p \leqslant m$, and a set of q column indices $c = \{j_1, j_2, \ldots, j_q\}$ where $1 \leqslant j_1 < j_2 < \cdots < j_q \leqslant n$. We denote the resulting $p \times q$ submatrix B formed by intersecting rows $i_1, \ldots, i_p$ and columns $j_1, \ldots, j_q$ of A as

$$B = A[r, c].$$

In our earlier example, we can express B as:

$$B = \begin{bmatrix} 0 & 2 \\ -1 & -3 \end{bmatrix} = A[\{1, 3\}, \{2, 5\}].$$

We use the **colon notation** to specify submatrices made up of consecutive rows and/or columns. Suppose we have integers i and j, where $j \geqslant i$. The notation $i\colon j$ represents the set $\{i, i+1, \ldots, j\}$. So, $A[p\colon q, r\colon s]$ refers to the submatrix of A that includes rows p through q and columns r through s, that is,

$$A[p\colon q, r\colon s] = \begin{bmatrix} A_{pr} & A_{p,r+1} & \cdots & A_{ps} \\ A_{p+1,r} & A_{p+1,r+1} & \cdots & A_{p+1,s} \\ \vdots & \vdots & \ddots & \vdots \\ A_{qr} & A_{q,r+1} & \cdots & A_{qs} \end{bmatrix}.$$

In the colon notation, there are two particularly useful special cases. We use $A[\,:\,, r\colon s]$ to mean that we are taking all the rows of a matrix A but only the columns from r to s. Similarly, $A[p\colon q, \,:\,]$ indicates that we are selecting all columns, but only the rows from p to q.

Let us see a few examples of colon notation using the matrix A from earlier:

$$A[1\colon 2, 3\colon 4] = \begin{bmatrix} 3 & -4 \\ -3 & 0 \end{bmatrix},$$

$$A[\,:\,, 2\colon 3] = \begin{bmatrix} 0 & 3 \\ 2 & -3 \\ -1 & 2 \\ 0 & 1 \end{bmatrix}, \quad \text{and}$$

$$A[1\colon 2, \,:\,] = \begin{bmatrix} 1 & 0 & 3 & -4 & 2 \\ 2 & 2 & -3 & 0 & 0 \end{bmatrix}.$$

Entries and submatrices of matrix formulas

Sometimes, we need to refer to entries or submatrices of a matrix formula. In these cases, we will surround the formula with parentheses and use the notation we just introduced. For example, if we have the expression[2] $A + (I_n - \alpha A)^{-1}$, we can denote its (i, j)th entry as $\left(A + (I_n - \alpha A)^{-1}\right)_{ij}$ or $\left(A + (I_n - \alpha A)^{-1}\right)[i, j]$, and a submatrix by using the colon notation, such as $\left(A + (I_n - \alpha A)^{-1}\right)[p\colon q, r\colon s]$.

2.3 Sparse matrices and non-zero patterns

A **sparse matrix** is a matrix predominantly composed of zero entries. While there is not a strict threshold defining how many zeros constitute sparsity, a matrix is typically regarded as sparse if it becomes computationally advantageous to omit storing or explicitly manipulating its zero elements.

We can define important types of sparse matrices by their non-zero patterns. The way these non-zero entries are arranged in a matrix A is what we call the sparsity pattern of A. Here are some important types that will appear in later chapters. We say a matrix A is:

- **diagonal** if $A_{ij} = 0$ for $i \neq j$;
- **upper triangular** if $A_{ij} = 0$ for $i > j$, or **lower triangular** if $A_{ij} = 0$ for $i < j$;
- **upper Hessenberg** if $A_{ij} = 0$ for $i > j + 1$, or **lower Hessenberg** if $A_{ij} = 0$ for $i < j - 1$;

[2] Do not worry if this matrix formula does not make much sense right now, it will become clearer as we progress.

- b**-banded** if $A_{ij} = 0$ for $|i - j| > b$. An important special case is when $b = 1$, in which case we call A a **tridiagonal** matrix.

We often show a matrix's non-zero pattern by marking where non-zero elements are, leaving blank spaces for the zeros. For example:

$$\begin{bmatrix} \times & & & & \\ & \times & & & \\ & & \times & & \\ & & & \times & \\ & & & & \times \end{bmatrix}, \quad \begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \\ & & & & \times \end{bmatrix}, \quad \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}, \quad \text{and} \quad \begin{bmatrix} \times & \times & & & \\ \times & \times & \times & & \\ & \times & \times & \times & \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}$$

show the non-zero patterns of diagonal, upper triangular, upper Hessenberg, and tridiagonal 5×5 matrices, respectively.

The "diag" notation

We frequently need to extract the diagonal entries of a square matrix A and put them into a column vector v. To do this, we use the notation $v = \text{diag}(A)$. For example:

$$\text{if} \qquad A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \qquad \text{then} \qquad v = \text{diag}(A) = \begin{bmatrix} 1 \\ 5 \\ 9 \end{bmatrix}.$$

Sometimes, given a vector v, we want to create a diagonal matrix A with v as its diagonal. For this, we use the notation $A = \text{diag}(v)$. For example:

$$\text{if} \qquad v = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \qquad \text{then} \qquad A = \text{diag}(v) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

Notice that we use the same "diag" notation for both extracting a diagonal and building a diagonal matrix. This is standard practice, and the context should make it clear which one we are referring to.

Problems

2.1 Given vectors $u = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ and $v = \begin{bmatrix} -1 \\ 0 \\ 4 \end{bmatrix}$, compute the following:

(a) $2u + 3v$;
(b) $u - v$;
(c) $4u + v$.

2.2 Determine whether the following matrices are diagonal, upper triangular, lower triangular, upper Hessenberg, lower Hessenberg, tridiagonal, or none:

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 3 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 4 & 3 \\ 0 & 0 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 5 \\ 0 & 6 & 7 \end{bmatrix}.$$

2.3 Let $x = \begin{bmatrix} 4 \\ -3 \\ 2 \end{bmatrix}$. Write the row vector x^T and compute the row vector $y = 2x^T$.

2.4 Given

$$M = \begin{bmatrix} 5 & -1 & 3 & 4 \\ 0 & 2 & 1 & -2 \\ 7 & 0 & 6 & 8 \\ -3 & 4 & 9 & 1 \end{bmatrix},$$

extract the following submatrices:

- $M[1\colon 2, 3\colon 4]$;
- $M[\colon, 2\colon 3]$;
- $M[3\colon 4, \colon]$.

2.5 For the vector $v = \begin{bmatrix} 6 \\ -1 \\ 0 \\ 4 \end{bmatrix}$, form the matrix $\mathrm{diag}(v)$.

2.6 For a matrix A, what is $\mathrm{diag}(\mathrm{diag}(A))$?

2.7 Using matrix $P = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$, express the submatrix $\begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$ using colon notation.

2.8 Provide examples (one for each case) of 4×4 matrices that exhibit the following sparsity patterns:

(a) Upper triangular.
(b) Lower Hessenberg.
(c) Tridiagonal.

2.9 Write explicitly the 3×3 identity matrix I_3 and the 2×4 zero matrix $0_{2\times 4}$.

2.10 Consider the vector $z = \begin{bmatrix} 1+i \\ 2-2i \\ -3i \end{bmatrix}$. Compute the scalar multiplication $(1-i)z$.

2.11 Given $X = A + (I_n - \alpha A)^{-1}$, write down how you would denote the submatrix of X formed by rows 2 to 4 and columns 3 to 5.

2.12 Use your favorite programming language (such as Python or MATLAB) to do the following tasks:

(a) Construct a random 6×5 matrix A with real entries.
(b) Produce the submatrix $A[\{1, 4, 5, 6\}, \{2, 3, 5\}]$.
(c) Produce the submatrix $A[3\colon 6, \colon]$ of A.
(d) Produce the submatrix $A[\colon, 1\colon 3]$ of A.
(e) Construct a 5×5 random upper triangular matrix.
(f) Construct a 6×6 tridiagonal matrix with positive nonzero entries.

Chapter 3
I-CORE TOPICS

3.1 Encoding data into matrices

As noted in Chapter 1, Machine-Learning data are usually vectors $x_1, x_2, \ldots, x_m$. We often call these vectors *data points* or *observations*. All vectors have the same length n. The i-th one is

$$x_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{in} \end{bmatrix}.$$

The entries of these vectors are the inputs discussed earlier. In the House-Price Prediction example (Example 1.1), the entries of x_i for the i-th house record facts such as square footage, and age. We call the individual values x_{ij} *features* or *attributes* (e.g., living-room size, house age, distance to school). Thus x_{ij} is the value of the j-th feature for the i-th observation.

Typically, we arrange the data vectors $x_1, \ldots, x_m$ as rows in an $m \times n$ matrix A. That is,

$$A = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix} = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ x_{21} & \cdots & x_{2n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

We call this type of matrix a **data matrix**. Figure 3.1 illustrates this. Each row represents a data point or observation, and each column represents a feature or attribute. The entries in the matrix are the actual data values.

In the next sections we will see that, whether the data are tables, images, text, or graphs, we can always organize it into a matrix.

3.2 Tabular data as matrices

You have probably seen tabular data in spreadsheets, where information is organized into rows and columns. For example, Table 3.1 shows an example of tabular data

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_3

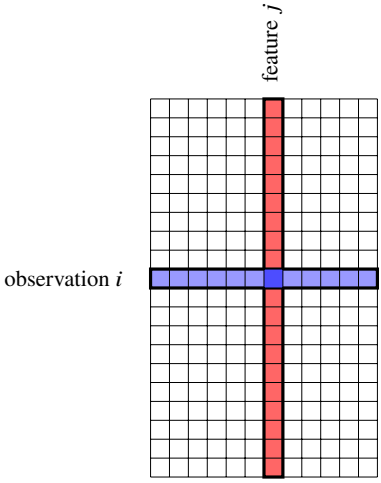


Fig. 3.1: A representation of a data matrix. Each row corresponds to a data point/observation, and each column represents a feature/attribute. The entries contain actual data values.

where all the columns contain numerical values. Rows represent individual houses and columns show different attributes of the houses.

House ID	Price (USD)	Bedrooms	Bathrooms	Square Feet
1	300,000	3	2	1,800
2	450,000	4	3	2,500
3	350,000	3	3	2,000
4	200,000	2	1	1,200
5	600,000	4	4	2,900

Table 3.1: A table of house listings showing different numerical features.

We can turn the entire table into a matrix by taking the numerical values from each row (excluding the "House ID" column and the header row) and arranging them as row vectors. Doing this for all the houses gives us the following matrix:

$$A = \begin{bmatrix} 300,000 & 3 & 2 & 1,800 \\ 450,000 & 4 & 3 & 2,500 \\ 350,000 & 3 & 3 & 2,000 \\ 200,000 & 2 & 1 & 1,200 \\ 600,000 & 4 & 4 & 2,900 \end{bmatrix}.$$

Converting tabular data to a matrix is not always as simple as in the previous example, since tables may contain mixed data types, some numeric, others not. Table 3.2 shows such a mixed-type case.

House ID	Price (USD)	Bedrooms	Bathrooms	Square Feet	House Type	Condition
1	300,000	3	2	1,800	Detached	Good
2	450,000	4	3	2,500	Semi-Detached	Excellent
3	350,000	3	3	2,000	Detached	Fair
4	200,000	2	1	1,200	Townhouse	Poor
5	600,000	4	4	2,900	Detached	Excellent

Table 3.2: House listings showing various attributes, including both numerical and categorical data like type and condition.

In Table 3.2, along with numerical attributes like price and size, we have non-numerical features that describe the house type and condition. To turn this table into a matrix, we first need to convert the non-numerical attributes into numbers. This process is called **encoding**.

Ordinal data and ordinal encoding

Ordinal data refers to data that falls into categories with a specific order. For example, in our house listings table (Table 3.2), the "Condition" column is ordinal because it ranks houses from "Poor" to "Excellent".

To work with ordinal data, we use **ordinal encoding**, where we assign each category a unique integer while keeping the original order. Table 3.3 shows how we can encode the four categories in the "Condition" column.

Condition	Integer Encoding
Poor	0
Fair	1
Good	2
Excellent	3

Table 3.3: Ordinal encoding of house conditions.

Categorical data and one-hot encoding

Categorical data refers to data that falls into distinct groups or categories without a specific order. In our house listings table, the "House Type" column includes categories like "Detached", "Semi-detached", and "Townhouse".

To handle categorical data, we use **one-hot encoding**, which transforms each category into a binary vector. This vector uses 0's and 1's to indicate the presence or absence of each category. Each house type gets its own unique binary representation. For example, the category "Detached" is encoded as

$$\begin{array}{ccc} \text{detached} & \text{semi-detached} & \text{townhouse} \\ \left[\begin{array}{ccc} 1 & 0 & 0 \end{array}\right] \end{array}$$

The "1" indicates the correct house type. Table 3.4 shows how we encode the three categories in the "House Type" column.

House Type	Vector Encoding
Detached	$[1\ 0\ 0]$
Semi-detached	$[0\ 1\ 0]$
Townhouse	$[0\ 0\ 1]$

Table 3.4: One-hot encoding for house types.

After applying ordinal encoding to the "Condition" column and one-hot encoding to the "House Type" column, we transform the data from Table 3.2 into the following data matrix:

$$A = \begin{bmatrix} 300,000 & 3 & 2 & 1,800 & 1 & 0 & 0 & 2 \\ 450,000 & 4 & 3 & 2,500 & 0 & 1 & 0 & 3 \\ 350,000 & 3 & 3 & 2,000 & 1 & 0 & 0 & 1 \\ 200,000 & 2 & 1 & 1,200 & 0 & 0 & 1 & 0 \\ 600,000 & 4 & 4 & 2,900 & 1 & 0 & 0 & 3 \end{bmatrix}.$$

In this matrix, the last column comes from the ordinal encoding of the "Condition" column, and the three columns before it represent the one-hot encoding of the "House Type" column.

In one of the Jupyter notebooks accompanying this book (see Section 4.3), we will walk you through these two preprocessing steps in Python: ordinal and one-hot encoding of non-numerical features.

Keep in mind that every encoding technique has its own advantages and disadvantages. Ordinal encoding is simple, but it treats successive categories as equally spaced, a simplification that may misrepresent real differences and blur important details. For example, the gap between "Poor" and "Fair" could be larger than between "Good" and "Excellent". One-hot encoding avoids that assumption but increases your data size. For example, the "House Type" column becomes three columns of 0's and 1's. This requires more memory, especially with many categories.

In addition to ordinal and one-hot encoding, there are many other encoding techniques, each with its own benefits and uses. However, covering those is beyond the scope of this book, so we encourage you to explore them on your own.

3.3 Images as matrices

We can represent digital images as matrices. For example, a grayscale image can be seen as a matrix where each entry corresponds to a pixel. The value of each element shows the brightness, usually on a scale from 0 to 255[1], where 0 is black, 255 is white, and values in between represent different shades of gray.

Figure 3.2 illustrates this idea. The left panel displays a 16×16 pixel image of a handwritten digit five. The middle panel overlays the pixel grid with the corresponding intensity values. The right panel presents the matrix representation of the image, where each entry corresponds to the intensity of the pixel at that position.

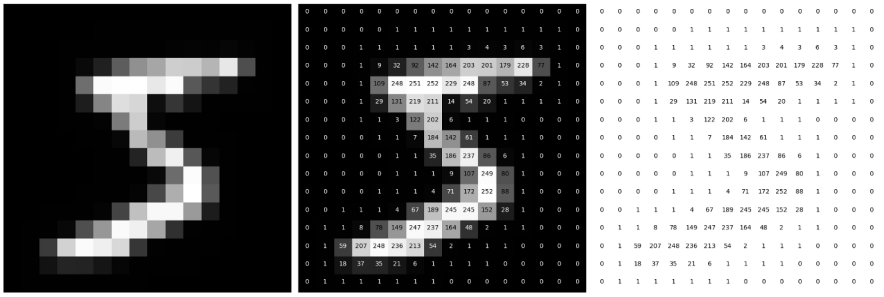


Fig. 3.2: Matrix representation of a grayscale image.

We can flatten a $p \times q$ grayscale image into a vector of length $n = pq$. We start with the top row of pixels and place its intensities in the vector. Then, we append the second row's pixel intensities, followed by each subsequent row, until the vector contains all the pixel values.

Now suppose we have m such images. To organize the data as a matrix, first convert each image to its length-n vector, then stack these vectors as the rows of an $m \times n$ matrix. Each row now represents an image, and each column a particular pixel location (see Figure 3.3, where every row is a handwritten-digit image).

For color images, the representation is more complex than for grayscale images. Typically, we use different color spaces, like the widely-used RGB (Red, Green, Blue) color space[2]. In the RGB color space, we represent an image with three separate matrices, one for each color channel: red, green, and blue. Each matrix holds the intensity values for its respective color.

The left panel of Figure 3.4 shows a 16×16 pixel RGB image of a Christmas tree. The other panels separate the image into its red, green, and blue components, each overlaid by the corresponding color intensities, ranging from 0 to 1.

[1] 256 is the number of integers an 8-bit scale can represent, which is common in digital imaging.

[2] While RGB is common, other systems like HSV (Hue, Saturation, Value) exist and may better match how we perceive colors. However, these alternatives are beyond the scope of this book.

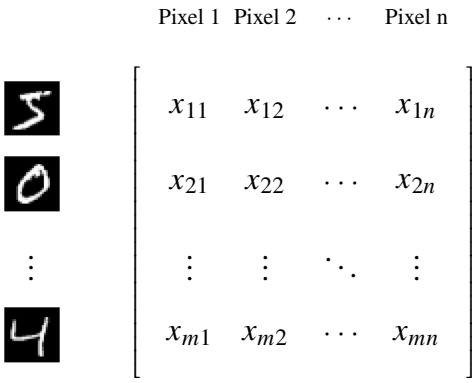


Fig. 3.3: Representing grayscale images as rows of a matrix.

Fig. 3.4: RGB representation and color channels of a 16 × 16 pixel image of a Christmas tree.

Similar to grayscale images, we can arrange a set of m RGB images, each sized $p \times q$ pixels, into a matrix. We transform each image into a row vector of length $3n$ (where $n = pq$), dividing it into three segments: the first n entries for red, the next n for green, and the last n for blue channel intensities. Figure 3.5 illustrates this approach.

3.4 Graphs as matrices

Graphs are one of the most important tools in Applied Mathematics, Machine Learning, and Data Science.

Definition 3.1. (Undirected/directed/simple graph)

A **graph**, often referred to as a network, consists of **nodes** (also called vertices) connected by **edges**.

Edges can be classified as either **undirected**, meaning they represent bidirectional connections shown as line segments, or **directed**, representing unidirectional connections shown with arrows. These distinctions lead to two types of graphs: **undirected graphs** and **directed graphs**.

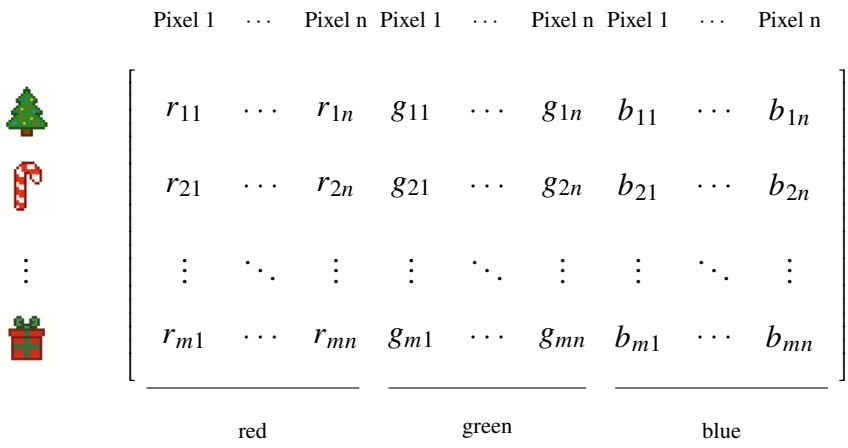


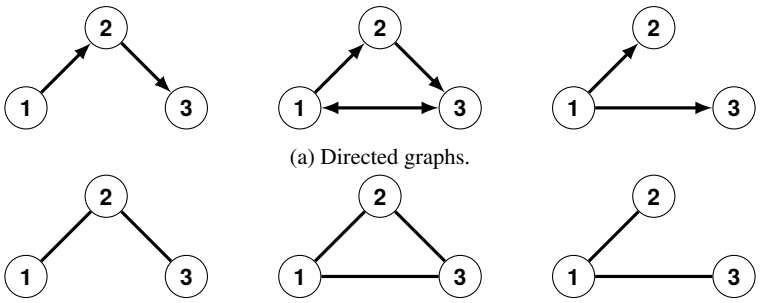
Fig. 3.5: Representing RGB images as rows in a matrix.

A graph is termed **simple** if it contains no loops (edges connecting a node to itself) and has at most one edge between any two distinct nodes.

Directed graphs are also commonly called **digraphs**.

Example 3.2. (Directed/undirected and simple/non-simple graphs)

In Figure 3.6, we display examples of both directed and undirected graphs. Each graph contains three nodes, represented by circles with numbers inside them, linked by different edges. These edges are shown as either arrows (for directed graphs) or line segments (for undirected graphs).

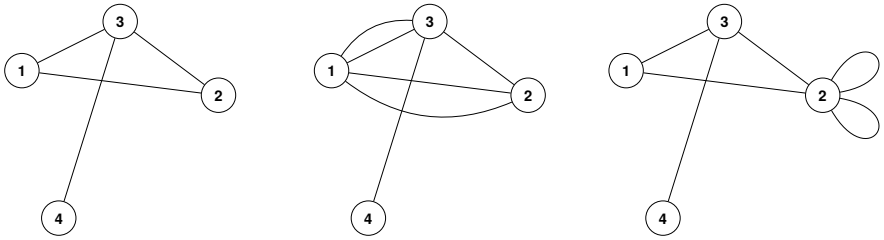


(a) Directed graphs.

(b) Undirected graphs.

Fig. 3.6: Examples of directed and undirected graphs with three nodes.

Figure 3.7 shows examples of both simple and non-simple undirected graphs.



(a) Simple undirected graph.

(b) Non-simple undirected graph with multiple edges between pairs of nodes.

(c) Non-simple undirected graph with loops.

Fig. 3.7: Simple and non-simple undirected graphs.

In many applications, we assign a numerical value, called a weight, to each edge of a graph.

Definition 3.3. (Weighted/unweighted graph)

A **weighted graph** assigns a (generally non-negative) real number, known as a **weight**, to each edge.

For a graph with n nodes labeled from 1 to n, we denote the weight between nodes i and j as w_{ij}. In directed graphs, w_{ij} and w_{ji} might differ, whereas in undirected graphs, w_{ij} always equals w_{ji}.

A graph is **unweighted** if it has no edge weights. However, we can treat an unweighted graph as a special case of a weighted graph by setting $w_{ij} = 1$ if nodes i and j are connected by an edge, and $w_{ij} = 0$ if they are not.

Let us see some examples of weighted graphs.

Example 3.4. (Weighted graphs)

Consider the directed (left) and undirected (right) graphs in Figure 3.8. Both graphs have seven nodes, labeled 1 through 7. Every edge carries a positive weight w_{ij} as indicated in the figure. The directed graph has 9 directed edges, each with its own weight. The undirected graph has 7 undirected edges, so it requires 7 weights.

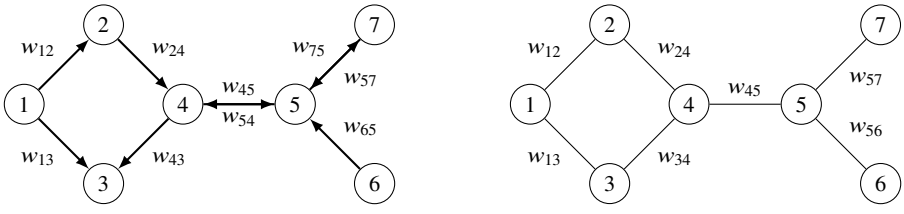


Fig. 3.8: A directed weighted graph (left) and an undirected weighted graph (right).

Notation for graphs

We often use the notation $G = (V, E)$ for an unweighted graph G, where V is the set of vertices and E is the set of edges. For weighted graphs, we write $G = (V, E, w)$, where $w : E \to [0, \infty]$ is the edge-weight function.

To illustrate this, consider the directed and weighted graph in Figure 3.8. For this graph, we have:

$$V = \{1, 2, 3, 4, 5, 6, 7\},$$
$$E = \{(1, 2), (1, 3), (2, 4), (4, 3), (4, 5), (5, 4), (5, 7), (7, 5), (6, 5)\}, \quad \text{and}$$
$$\{w(e) : e \in E\} = \{w_{12}, w_{13}, w_{24}, w_{43}, w_{45}, w_{54}, w_{57}, w_{75}, w_{65}\}.$$

Graphs show how objects are connected, making it easier to model real-world systems. Let us look at some examples.

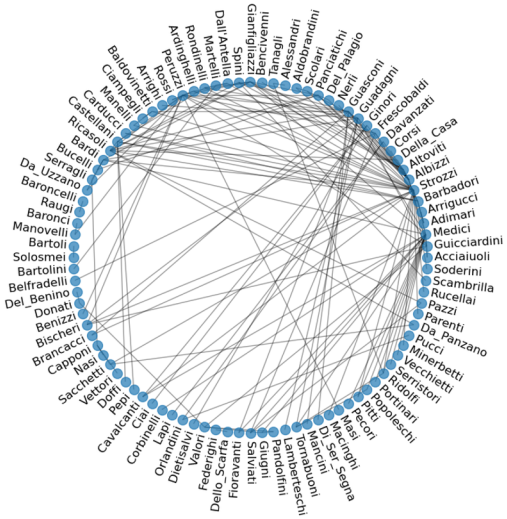
- **Transportation networks:** Nodes represent stations or stops, while edges represent routes connecting them. Weights can show distances, travel times, or costs.
- **Communication networks:** Nodes are devices like computers or cell phones, and edges show how they communicate. Weights can indicate bandwidth, signal strength, or latency.
- **The World Wide Web:** Nodes represent web pages, and edges represent hyperlinks. Weights can show click-through rates or link relevance.
- **Social networks:** Nodes are individuals or groups of people, and edges show relationships or interactions between them. Weights can indicate relationship strength, interaction frequency, or influence.
- **Electric circuits:** Nodes are electrical components, and edges represent the wires connecting them. Weights can show resistance, current capacity, or power loss.
- **Rumor dissemination:** Nodes are individuals or groups of people, and edges show how information spreads. Weights can represent the probability of transmission, influence, or speed of spread.
- **Data organization:** Nodes represent data elements, and edges show their relationships. Weights can indicate hierarchical relationships or access speeds.

Let us look at some examples of real-world graphs that we will explore in upcoming chapters.

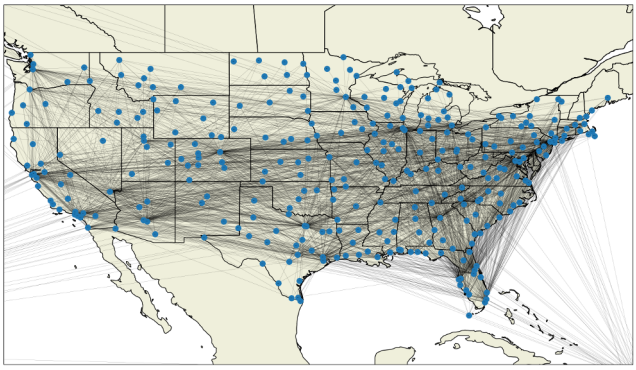
Example 3.5. (Three real-world graphs)

In Figure 3.9, we present three scenarios that use graphs to model real-world situations.

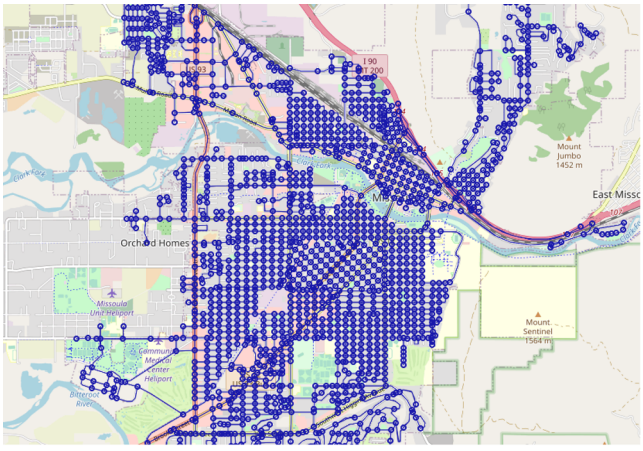
The top panel illustrates a 15th-century social network, constructed from John Padgett's Florentine Families dataset [43]. It shows the marital ties between 96 well-known families of that time, such as the Medici, Strozzi, and Rucellai families. In



(a) 15th-century Florentine social network depicting marital ties (edges) between prominent families (nodes).



(b) US air transportation network illustrating direct flight connections (edges) between airports (nodes).



(c) Street intersection graph of Missoula, Montana, with nodes indicating intersections and edges representing connecting streets.

Fig. 3.9: Three real-world graph examples.

this graph, we represent every family as a node and the marriages between families as edges.

The middle panel shows the US air transportation network [3]. Here, airports are the nodes, and the direct flight paths connecting these airports are the edges.

Lastly, the bottom panel gives us a view of street intersections in Missoula, Montana. In this graph, each intersection is a node, and the streets connecting these intersections are the edges.

Let us return to our main goal: how can we represent a graph as a matrix? There are several ways to do this. One common approach is the adjacency matrix.

Definition 3.6. (Adjacency matrix)

Let $G = (V, E, w)$ be a weighted graph with vertex set $V = \{1, 2, \ldots, n\}$, where $w : E \to [0, \infty)$ assigns a weight to each edge.

For $i, j \in V$, let E_{ij} denote the set of edges directed from node i to node j when G is directed, and the set of edges joining nodes i and j when G is undirected. The *adjacency matrix* of G is the matrix $A = [A_{ij}] \in \mathbb{R}^{n \times n}$ defined by

$$A_{ij} = \sum_{e \in E_{ij}} w(e),$$

where the sum over an empty set is understood to be zero.

For an unweighted graph, we assign weight 1 to every edge. Thus, A_{ij} equals the number of edges directed from node i to node j in a directed graph, and the number of edges joining nodes i and j in an undirected graph.

If G is undirected, then $A_{ij} = A_{ji}$, and hence its adjacency matrix is symmetric.

Remark 3.7. (Simple and non-simple graphs)

For a simple graph, there is at most one edge between any two distinct nodes and there are no loops. Therefore, in the unweighted case,

$$A_{ij} = \begin{cases} 1, & \text{if nodes } i \text{ and } j \text{ are connected by an edge,} \\ 0, & \text{otherwise.} \end{cases}$$

For a non-simple graph with multiple edges between pairs of nodes, the entry A_{ij} is the sum of the weights of all edges from i to j, or, in the unweighted case, the number of such edges.

If loops are allowed, an additional convention is required for the diagonal entries of the adjacency matrix. Unless otherwise stated, we will associate adjacency matrices only with loopless graphs, so that

[3] This graph is constructed using data from the "Airline On-Time" database available at the Bureau of Transportation Statistics website `http://www.bts.gov`.

$$A_{ii} = 0 \qquad \text{for all } i = 1, \ldots, n.$$

Example 3.8. (Adjacency matrices)

Consider the graphs in Figure 3.8. Their adjacency matrices are, respectively,

$$\begin{bmatrix} 0 & w_{12} & w_{13} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_{24} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & w_{43} & 0 & w_{45} & 0 & 0 \\ 0 & 0 & 0 & w_{54} & 0 & 0 & w_{57} \\ 0 & 0 & 0 & 0 & w_{65} & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{75} & 0 & 0 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0 & w_{12} & w_{13} & 0 & 0 & 0 & 0 \\ w_{12} & 0 & 0 & w_{24} & 0 & 0 & 0 \\ w_{13} & 0 & 0 & w_{34} & 0 & 0 & 0 \\ 0 & w_{24} & w_{34} & 0 & w_{45} & 0 & 0 \\ 0 & 0 & 0 & w_{45} & 0 & w_{56} & w_{57} \\ 0 & 0 & 0 & 0 & w_{56} & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{57} & 0 & 0 \end{bmatrix}.$$

The second matrix is symmetric, because its graph is undirected. If the graphs were unweighted, we would replace every weight w_{ij} with 1 in the adjacency matrices.

In later chapters, we will see how to use the adjacency matrix to analyze graphs. In Chapters 9 and 14, we will introduce "graph centrality measures" to identify the most important nodes in a graph. In Chapter 44, we will cover clustering nodes and embedding graphs in lower-dimensional spaces.

3.5 Text documents as matrices

In many applications, we work with datasets consisting of text documents, like emails, scientific papers, letters, or memos. There are several ways to transform these datasets into matrices. One basic method is the "bag-of-words" model, which converts text data into a matrix of word counts or frequencies.

Bag-of-words and document-term matrices

Let us consider a small collection of three documents:

1. "The student solved the equation aided by the professor."
2. "The professor explained the theorem."
3. "The student and the professor discussed the solution."

To create a matrix from these three text documents, we first list all the unique words, ignoring punctuation and case. This gives us our vocabulary:

```
{"the", "student", "solved", "equation", "aided", "by",
"professor", "explained", "theorem", "and", "discussed",
"solution"}.
```

Next, we create Table 3.5, where each row represents a document and each column corresponds to a word from the vocabulary. The entries in the table show how many times each word appears in the documents.

	the	student	solved	equation	aided	by	professor	explained	theorem	and	discussed	solution
Doc 1	3	1	1	1	1	1	1	0	0	0	0	0
Doc 2	2	0	0	0	0	0	1	1	1	0	0	0
Doc 3	3	1	0	0	0	0	1	0	0	1	1	1

Table 3.5: Document-term table representation of text documents.

For example, the number in the "Doc 1" row and the "equation" column in Table 3.5 is 1. This means that the word "equation" appears once in the first document.

We can represent the data in Table 3.5 more compactly using the following **document-term matrix**:

$$A = \begin{bmatrix} 3 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

Stop-words removal and lemmatization

It is important to clean text data before analysis by removing stop words (common words like "the" and "is" that add little value) and applying lemmatization to reduce words to their base form (e.g., "running" to "run").

In our example, words like "the", "by", and "and" are stop words. After removing them, our vocabulary becomes:

```
{"student", "solved", "equation", "aided", "professor",
"explained", "theorem",  "discussed", "solution"}.
```

Lemmatization simplifies words to their base form, such as "aided" to "aid" and "discussed" to "discuss". The final vocabulary is:

```
{"student", "solve", "equation", "aid", "professor",
"explain", "theorem","discuss", "solution"}.
```

With our refined vocabulary, we construct Table 3.6, with each document as a row and each term as a column. Notice the table is smaller now that the vocabulary has been reduced.

	student	solve	equation	aid	professor	explain	theorem	discuss	solution
Doc 1	1	1	1	1	1	0	0	0	0
Doc 2	0	0	0	0	1	1	1	0	0
Doc 3	1	0	0	0	1	0	0	1	1

Table 3.6: Refined vocabulary: post stop-words removal and lemmatization, representing documents with simplified terms.

We can simplify this table into the following document-term matrix :

$$A = \begin{bmatrix} 1\,1\,1\,1\,1\,0\,0\,0\,0 \\ 0\,0\,0\,0\,1\,1\,1\,0\,0 \\ 1\,0\,0\,0\,1\,0\,0\,1\,1 \end{bmatrix}.$$

Term Frequency - Inverse Document Frequency (TF-IDF)

After removing stop words and applying lemmatization, an alternative to the word count method is TF-IDF (Term Frequency-Inverse Document Frequency) [47].

Let us revisit the vocabulary from the previous example:

```
{"student", "solve", "equation", "aid", "professor",
"explain", "theorem",  "discuss", "solution"}.
```

The TF-IDF method has two steps: calculating the term frequency (TF) and the inverse document frequency (IDF) for each word.

First, we compute the term frequency (TF), which measures how often a word w appears in a document d relative to the total number of words in that document. We define it as:

$$\mathrm{TF}(w, d) = \frac{\text{number of times word } w \text{ appears in document } d}{\text{total number of words in document } d}.$$

Simply put, $\mathrm{TF}(w, d)$ shows how common a word is in a specific document.

Using this, we can create a table of TF values for our three documents (Table 3.7).

	student	solve	equation	aid	professor	explain	theorem	discuss	solution
Doc 1	1/5	1/5	1/5	1/5	1/5	0	0	0	0
Doc 2	0	0	0	0	1/3	1/3	1/3	0	0
Doc 3	1/4	0	0	0	1/4	0	0	1/4	1/4

Table 3.7: Term frequency (TF) calculation for three documents.

Next, we calculate the inverse document frequency (IDF), which measures how important a word is across all documents. The IDF of a word w is given by[4]

$$\mathrm{IDF}(w) = \log\left(\frac{\text{total number of documents}}{\text{number of documents containing word } w}\right),$$

where log is the natural logarithm. Table 3.8 measures the IDF values for our vocabulary.

The goal of IDF is to penalize words that appear in many documents by assigning them lower scores. If a word appears in every document, like “professor” in our example, its IDF becomes zero.

[4] There are several variations of the IDF formula. This book uses one of the common versions.

Term	IDF
student	$\log(3/2) \approx 0.4$
solve	$\log(3) \approx 1.1$
equation	$\log(3) \approx 1.1$
aid	$\log(3) \approx 1.1$
professor	$\log(1) \approx 0$
explain	$\log(3) \approx 1.1$
theorem	$\log(3) \approx 1.1$
discuss	$\log(3) \approx 1.1$
solution	$\log(3) \approx 1.1$

Table 3.8: Inverse document frequency (IDF) values for the vocabulary of three documents.

Next, we assign each word w in each document d a TF-IDF score by multiplying the TF and IDF values:

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w).$$

In simple terms, TF-IDF shows how important a word is in a document relative to how often it appears across all documents. A higher score means the word is rarer and more unique to that document.

Let us see how to interpret the number TF-IDF(w, d) in an example.

Example 3.9. (TF-IDF scores)

Imagine that you have a 150-word document in which the word "function" appears 15 times. The TF score of this word is

$$\frac{15}{150} = 0.1.$$

Suppose now that we have a collection of one million mathematical documents and that the word "function" appears in 230,000 of them. Its IDF score is

$$\log\left(\frac{10^6}{230{,}000}\right) = \log\left(\frac{100}{23}\right) \approx 1.47.$$

Therefore, its TF-IDF score is

$$0.1(1.47) \approx 0.147.$$

This relatively low score reflects the fact that the word "function" is common across the collection and is therefore not especially distinctive of this document.

Now consider the word "isomorphism," which appears 10 times in the same 150-word document but occurs in only 200 of the one million documents. Its TF-IDF score is

$$\frac{10}{150}\log\left(\frac{10^6}{200}\right) = \frac{1}{15}\log(5000) \approx 0.568.$$

This higher TF-IDF score indicates that the word "isomorphism" is rarer across the collection and is therefore more informative for distinguishing this document from the others.

Revisiting our three-documents example, we now use the TF-IDF scores to create the following document-term matrix:

$$A = \begin{bmatrix} 0.08 & 0.22 & 0.22 & 0.22 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.37 & 0.37 & 0 & 0 \\ 0.1 & 0 & 0 & 0 & 0 & 0 & 0 & 0.275 & 0.275 \end{bmatrix}.$$

We encourage you to interpret the entries of this matrix in terms of how relevant each word is to its corresponding document.

Regardless of how we build our vocabulary, whether by removing stop words, applying lemmatization, or using other methods, we can always form a document-term matrix. Figure 3.10 shows an example, where each row is a document and each column is a word from the vocabulary. The entries represent the frequency of words, which can be based on word counts, TF-IDF values, or similar metrics to show how important or common each word is in the documents.

$$\begin{array}{c} \\ \text{doc 1} \\ \text{doc 2} \\ \vdots \\ \text{doc m} \end{array} \begin{array}{c} \begin{array}{cccc} \text{word 1} & \text{word 2} & \cdots & \text{word n} \end{array} \\ \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{bmatrix} \end{array}$$

Fig. 3.10: Document-term matrix. Entries indicate word frequencies within documents, using metrics like word counts or TF-IDF values.

In one of the Jupyter notebooks that accompanies this book (see Section 4.3), we will guide you through using Python to remove stop-words, apply lemmatization, and compute word counts and TF-IDF values.

Throughout this book, we will explore various examples involving text data, such as spam detection and information retrieval. Once we convert text data into a document-term matrix using word counts or TF-IDF, we can apply all the Machine Learning methods covered in this book.

3.6 Preferences and ratings as matrices

In everyday activities like online shopping or streaming, we constantly provide data through our ratings and preferences. This data is often organized into a **rating matrix**, where each row is a user and each column is an item, like a product, movie, or song. Our ratings fill the matrix.

Example 3.10. (Movie ratings)

Imagine a streaming service where users rate movies from 1 to 5, with 1 being the lowest and 5 the highest. We can organize these ratings into a matrix, where each row is a user and each column is a movie. Empty cells show where a user has not rated a movie.

	Casablanca	Gone with the Wind	Citizen Kane	12 Angry Men
User 1	5	4		
User 2	4	5	3	4
User 3			4	5
User 4	3	3		
User 5	5	4	2	1
User 6	4	4	5	4
User 7	3	3	5	4
User 8			4	5

Table 3.9: User ratings for movies; a blank space indicates an unrated movie.

Unlike our previous examples, the rating matrix may have blank entries. An important task is to estimate these missing values. This is essential in **recommender systems**, where algorithms predict what a user might like based on past behavior and similar users' preferences. We will explore this topic further in Chapter 14.

Problems

3.1 You are working on building a model to predict the prices of used cars in the market. You have access to a dataset containing various features related to cars. However, the dataset is not well-documented. Categorize the features in Table 3.10 into numerical, ordinal, and categorical types so that you can proceed with the regression analysis.

3.2 Identify a real-world scenario where you would use a regression model. What outcome do you want to predict? What features would you use to make the predictions? Are these features numerical or non-numerical? For non-numerical features, are they categorical or ordinal? Repeat this exercise for a classification problem.

Feature	Type
Car Manufacturer	
Car Model	
Year of Manufacture	
Mileage (in miles)	
Engine Displacement (in liters)	
Fuel Type	
Transmission Type	
Number of Doors	
Condition	
Number of Previous Owners	
Vehicle Color	
Interior Material	
Sunroof Availability	
Navigation System	
Audio System Quality	

Table 3.10: Features for car price prediction in Problem 3.1.

3.3 Select a real-world problem where you would apply a clustering algorithm. What kind of groups or patterns are you looking to identify? List features that could be used for your clustering problem.

3.4 Think of a real-world situation that could be represented as a graph. Is this graph best modeled as directed or undirected? Would it be weighted or unweighted?

3.5 For each part of this problem, encode the given data into a matrix.

(a) Given the following collection of short documents, encode it as a matrix using the bag-of-words model. Before encoding, remove stop-words (“the”, “at”, and “and”) and lemmatize the words.

(doc 1) The cats chased the rats.
(doc 2) Dogs and rats share food.
(doc 3) The dog barked at the cat.

(b) Encode the graph in Figure 3.11 into its adjacency matrix.

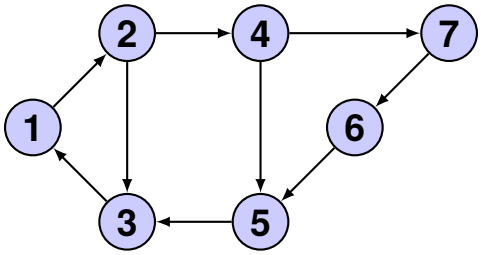


Fig. 3.11: Graph for part (b) in Problem 3.5.

(c) Given the dataset on cars in Table 3.11, encode it into a matrix. The dataset includes various features of cars: a numerical feature (Horsepower), a categorical feature (Type), and an ordinal feature (Condition). The Condition feature has levels: Poor, Fair, Good, Excellent.

Car	Horsepower	Type	Condition
Car A	200	Sedan	Good
Car B	150	SUV	Excellent
Car C	180	Coupe	Fair
Car D	220	Sedan	Excellent
Car E	170	SUV	Good

Table 3.11: Car features for part (c) in Problem 3.5.

3.6 A survey was conducted among 5 people, and they answered 4 questions with numerical scores. The data is tabulated as follows:

Person	Q_1	Q_2	Q_3	Q_4
1	3	4	2	5
2	4	3	5	2
3	5	5	4	3
4	2	4	3	4
5	3	3	4	5

(a) Represent this table as a data matrix A.
(b) Describe what the rows and columns represent.

3.7 A simple undirected graph with 4 vertices and the following edges is given:

$$1\text{–}2, \quad 1\text{–}3, \quad 2\text{–}3, \quad 3\text{–}4.$$

(a) Construct the adjacency matrix A for this graph.
(b) Explain what the rows and columns of A represent.

3.8 Consider a dataset where 5 people are characterized by the presence (1) or absence (0) of 4 features (e.g., owns a car, has a pet, uses social media, and exercises regularly).

(a) Create a 5×4 binary matrix to represent this data.
(b) Explain how each entry of the matrix is interpreted.

3.9 A network of 3 sensors records temperature data every hour. Over a period of 6 hours, each sensor provides a reading. Represent the sensor readings as a 3×6 matrix, where each row corresponds to a sensor and each column to a time stamp.

3.10 Consider a directed social network of 4 individuals where a directed edge from i to j indicates that person i follows person j. Suppose the relationships are:

$$1 \to 2, \quad 1 \to 3, \quad 2 \to 3, \quad 3 \to 4, \quad 4 \to 1.$$

Construct the corresponding adjacency matrix.

3.11 In computer vision, images are often transformed into feature vectors. Consider an image represented by a 5×5 matrix of pixel intensities. Explain how you could "flatten" this matrix into a single 25-dimensional vector.

3.12 You receive the following data from a movie rating website:

- User "Alice" rated "Star Wars" 5/5, "Matrix" 4/5, has not seen "Inception".
- User "Bob" rated "Matrix" 5/5, "Inception" 3/5, has not seen "Star Wars".
- User "Carol" rated "Star Wars" 2/5, "Inception" 4/5, "Matrix" 3/5.

Represent this information as a matrix. Use 0 to represent unseen movies. Label the rows and columns.

3.13 You have collected data from a small social network:

- Alice is friends with Bob and Carol.
- Bob is friends with Alice and David.
- Carol is friends with Alice only.
- David is friends with Bob only.

Create the adjacency matrix representing these friendships. Explain what each entry in your matrix represents.

3.14 You are analyzing black and white pixel art. Below is a 3×3 image where # represents a black pixel and space represents a white pixel:

```
# #
 #
# #
```

Convert this into a matrix where black pixels are represented by 1 and white pixels by 0.

3.15 In a small genetics study, you have collected DNA sequences from three patients:

- Patient 1's DNA sequence is "AGCT".
- Patient 2's DNA sequence is "TCGA".
- Patient 3's DNA sequence is "GCTA".

Create a matrix encoding these sequences. Use any reasonable encoding scheme, but clearly explain your encoding method.

3.16 You have collected three short text documents:

- Document 1: "the cat sat on the mat".
- Document 2: "the dog ran fast".
- Document 3: "the cat and the dog play".

Create a term-frequency matrix where each row represents a document and each column represents a unique word. List the words you used as columns.

3.17 You have collected grade data from a classroom:

- John got 85 in Math, 92 in Physics, 78 in Chemistry.
- Mary got 90 in Math, 88 in Physics, 95 in Chemistry.
- Peter got 75 in Math, 80 in Physics, 85 in Chemistry.

Create a grade matrix where rows represent students and columns represent subjects. Calculate the average score for each subject using your matrix.

3.18 You are analyzing grayscale images where each pixel has a value from 0 (black) to 255 (white). Below is a 2×2 image described in English:

- Top-left pixel is white (255).
- Top-right pixel is light gray (192).
- Bottom-left pixel is dark gray (64).
- Bottom-right pixel is black (0).

Represent this image as a matrix.

Chapter 4
I-SUPPLEMENTARY TOPICS

4.1 An introduction to Python and Jupyter notebooks

In some problems and examples throughout this book, we will need to use a programming language to work with applications. Python currently stands out as one of the most popular languages in the field of Machine Learning. The next section offers a quick-start tutorial on Python.

You can download Python from `www.python.org`. For beginners, we recommend the Anaconda distribution (`www.anaconda.com`), which includes essential libraries like NumPy, SciPy, Matplotlib, scikit-learn, and pandas, as well as Jupyter Notebook, all useful for Data Science and Machine Learning.

Before we get into Python, we will introduce Jupyter notebooks. These notebooks let you write and run Python code directly in your web browser. They are especially useful because you can run parts of your code independently and add comments, making it easier to share your work.

4.2 How to start a Jupyter notebook

Starting a Jupyter Notebook is easy with the Anaconda Navigator (`www.anaconda.com`). Just open the application and click on the box for Jupyter Notebook. Figure 4.1a shows what the Jupyter Notebook interface looks like after launching it.

Click "New" in the upper right corner to start a new Jupyter notebook, as shown in Figure 4.1b. Start by naming your Jupyter notebook by clicking on **Untitled** at the top of the page. Next, import the necessary libraries for your task. Once you have written your code in a cell, click on **Run** to execute it and see the results.[1]

4.3 Python libraries and the book's GitHub repository

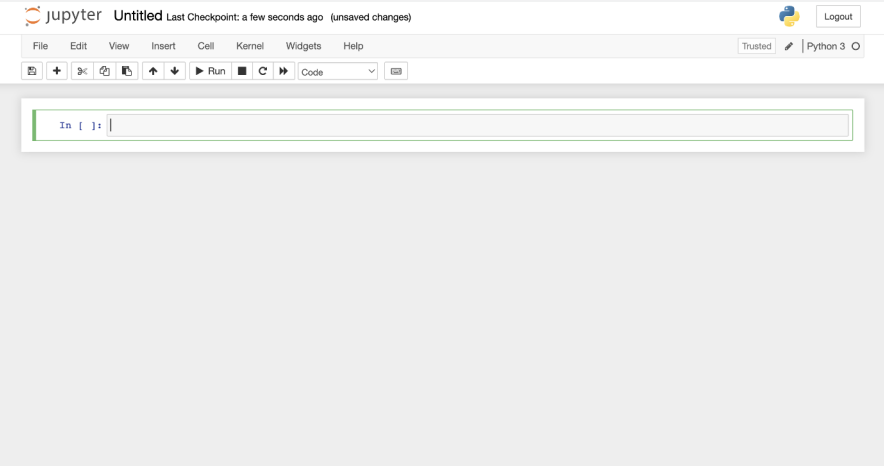
Our textbook focuses on Linear Algebra and Matrix Analysis in Data Science and Machine Learning. At the end of some chapters, we include exercises that use Python.

[1] To speed up your work in Jupyter Notebooks, there are useful keyboard shortcuts. For instance, "Shift + Enter" runs a cell. You can find more shortcuts under the "Help" menu in "Keyboard Shortcuts".

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_4

(a) The Jupyter Notebook interface upon launch.



(b) Creating a new Jupyter Notebook and running code.

Fig. 4.1: Steps for starting and using a Jupyter Notebook in Anaconda.

These exercises involve libraries like NumPy, Matplotlib, and pandas, which are great tools for these fields.

NumPy is a popular Python library for numerical and matrix computations. Its main feature is the ndarray, an n-dimensional array, ideal for working with matrices and vectors.

To use NumPy in Python, first import it using the alias “np”:

```
import numpy as np
```

This gives you access to NumPy’s functions. In our examples, “np” always stands for the NumPy library.

Similarly, you can access the other two key libraries we use:

```
import matplotlib.pyplot as plt
import pandas as pd
```

See Figure 4.2 for how to start these libraries in a Jupyter notebook. Matplotlib, especially its pyplot module, lets you create various plots and charts. The pandas library is great for manipulating structured data efficiently.

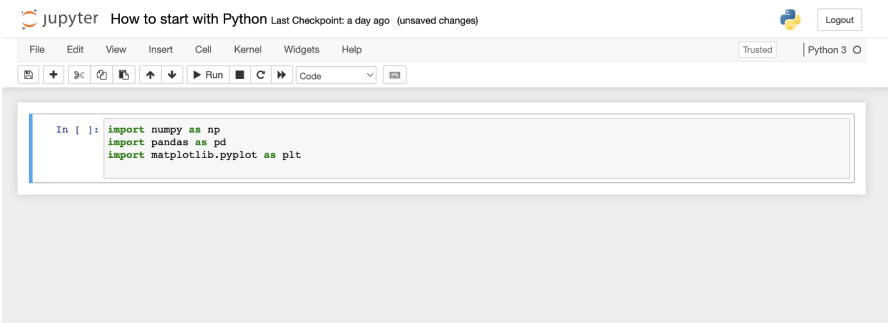


Fig. 4.2: Adding a title and importing libraries.

You can find the accompanying book's GitHub repository at the following URL:

`https://github.com/um-perez-alvaro/Matrix-Methods-in-Data-Analysis`

This repository contains a collection of Jupyter notebooks corresponding to each part of the book, starting with an introduction to Python and the NumPy, Matplotlib, and pandas libraries.

Problems

4.1 Create a Jupyter notebook. Call it "Vectors and matrices". Write the Python code to create the following vectors and matrices:

(a) A vector of length 10 containing random entries.
(b) A matrix of zeros of size 3×2.
(c) A vector of length 5 whose entries are equally spaced and in the interval $[0, 10]$.

4.2 Write a Python function to compute the dot product of two vectors. Assume the vectors are given as lists of numbers. For example, the dot product of $[1, 2, 3]$ and $[4, 5, 6]$ should return $1 \times 4 + 2 \times 5 + 3 \times 6 = 32$.

Hint: You can use a loop or list comprehension. Alternatively, consider using the `NumPy` library.

4.3 (Matrix Multiplication) Write a Python program to multiply two matrices. Represent matrices either as nested lists or by using the `NumPy` library. For example, multiply:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

Part II
MATRIX MULTIPLICATION AND PARTITIONED MATRICES

Matrix operations like addition, scalar multiplication, and matrix multiplication are among the first concepts we learn in Linear Algebra. While matrix addition and scalar multiplication are straightforward extensions of the same operations with real numbers, matrix multiplication is more involved. The standard definition of the product AB involves taking dot products between the rows of A and the columns of B. In this part, we explore an equivalent definition of matrix multiplication that uses the columns of A and the rows of B.

Additionally, when working with matrices, we often focus on their individual entries. However, in both theory and practice, it is often more convenient to view a matrix as an array of submatrices and perform operations on those submatrices. We will show that we can work with arrays of submatrices in much the same way as regular matrices.

Finally, we explore alternative definitions of matrix multiplication, such as the Hadamard and Kronecker products, which have interesting applications in Data Science.

Chapter 5
II-WHY SHOULD WE CARE?

5.1 The Erdős number

Paul Erdős (1913–1996) was a Hungarian mathematician who authored an impressive number of papers with colleagues. He probably holds the record for the most published papers by any mathematician since Euler, with 1525 works and 509 collaborators, as published by J. Grossman in [27], which uses data from 2004. To visualize these collaborative connections, we can use a graph (recall Definition 3.1). We represent each mathematician as a node and draw an edge between two nodes if the corresponding mathematicians have co-authored a paper (i.e., they have written a paper together).

For illustration, Figure 5.1 shows the collaboration graph of Erdős and his coauthors. The large node at the center represents Erdős, and his coauthors appear around it in a circular layout to make the figure easier to read. In principle, there should be an edge between Erdős and each of his coauthors. However, including all those edges would clutter the figure, so we omit them. The edges that appear in the figure connect collaborators among themselves.

We also apply a clustering method to detect different "communities", that is, groups of mathematicians who are more likely to collaborate within the group than with mathematicians outside it. These communities most likely correspond to different mathematical areas. In Chapter 44, we present one method for graph clustering.

In honor of his prolific work, some of his mathematician friends created the Erdős number [27]. How does the Erdős number work? Paul Erdős has an Erdős number of zero. Mathematicians who directly collaborated with Erdős have an Erdős number of one. For everyone else, their Erdős number is determined as $k + 1$, where k is the lowest Erdős number among their collaborators.

Suppose we have the graph of mathematicians and their collaborations, similar to the one in Figure 5.1 but expanded to include all mathematicians. We know that Paul Erdős is the only individual with an Erdős number of zero. His direct collaborators have an Erdős number of one. An interesting question is: How can we determine how many people have an Erdős number equal to some natural number n?

To determine the number of mathematicians with an Erdős number of two, we would need to identify the collaborators of Erdős' collaborators, excluding those with

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_5

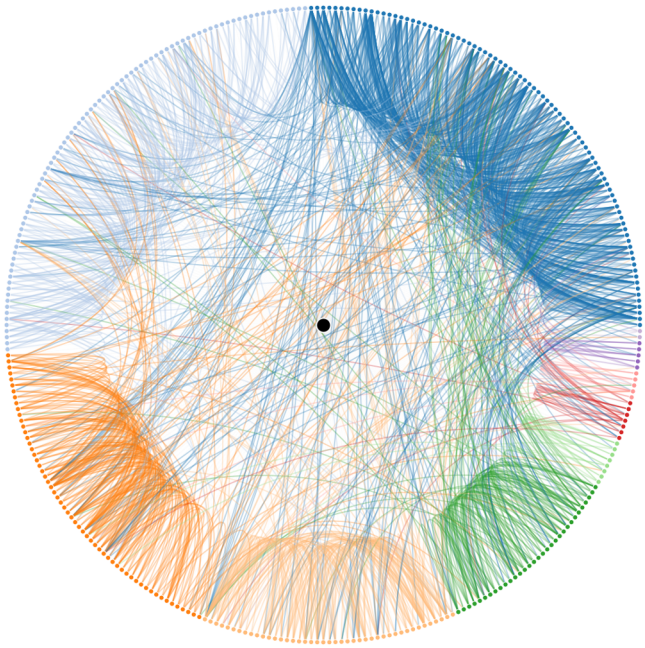


Fig. 5.1: Collaboration graph of Erdős and his coauthors. The central node represents Erdős. The surrounding nodes represent his collaborators. Edges between collaborators are shown, while edges between Erdős and each coauthor are omitted to keep the figure readable. Colors indicate communities detected by a clustering method.

Erdős number one. As one can imagine, this becomes progressively challenging as n increases. However, it turns out that this problem can be addressed using tools from Linear Algebra and Graph Theory, more precisely, matrix multiplication and the adjacency matrix of a graph (recall Definition 3.6).

In Chapter 9 and upcoming chapters, we will explore many interesting applications that arise from the concept of matrix multiplication and adjacency matrices.

By the way, the authors of this book have Erdős numbers 3 and 4, respectively.

5.2 The most important nodes in a graph

Given a graph, sometimes we need to identify its "most important" nodes. For example, let us consider the US airport transportation network.

In this network, each node represents a US airport, and an edge connects two airports if there is a direct flight between them. Figure 5.2 shows the airport network with nodes color-coded according to their "importance", measured through the so-called Katz centrality measure, which we will introduce in Chapter 9. The most important nodes are precisely the big airport hubs (Dallas, Chicago, Denver, etc.),

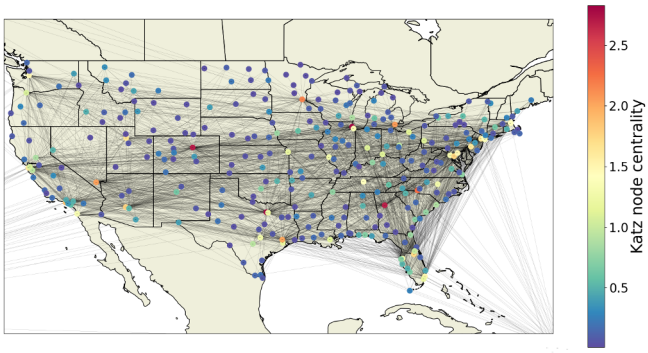


Fig. 5.2: US airport transportation graph. The nodes (airports) have been colored according to their "importance".

that is, airports where many airlines are present, and provide direct flights to many different cities.

The Katz centrality of the nodes of a graph can be obtained by computing powers of the adjacency matrix of the graph. We will examine this concept in more depth in Chapter 9.

Chapter 6
II-WHAT YOU MAY HAVE LEARNED BEFORE

In this chapter we review the most important operations with matrices and their basic properties.

6.1 Addition, scalar multiplication, and transpose of matrices

We begin by reviewing two fundamental operations on matrices: addition and scalar multiplication, both of which are performed componentwise and extend the operations applied to vectors (as a vector is a special case of a matrix). We will also review matrix transposition.

We can add two matrices A and B if they have the same number of rows and columns. The addition $A + B$ is done elementwise by adding corresponding entries:

$$\begin{bmatrix} A_{11} & \cdots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \cdots & A_{mn} \end{bmatrix} + \begin{bmatrix} B_{11} & \cdots & B_{1n} \\ \vdots & \ddots & \vdots \\ B_{m1} & \cdots & B_{mn} \end{bmatrix} = \begin{bmatrix} A_{11} + B_{11} & \cdots & A_{1n} + B_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} + B_{m1} & \cdots & A_{mn} + B_{mn} \end{bmatrix}.$$

Definition 6.1. (Matrix addition)

If $A, B \in \mathbb{R}^{m\times n}$, then $A + B \in \mathbb{R}^{m\times n}$ and

$$(A + B)_{ij} = A_{ij} + B_{ij},$$

for all $i = 1, \ldots, m$ and $j = 1, \ldots, n$.

When we add a real number a with itself, we usually write $a+a = 2a$. It is tempting to use a similar notation when we add a matrix with itself, that is, $A + A = 2A$. The problem is that 2 is a real number while A is a matrix. Thus, what do we mean by multiplying a matrix by a number? Multiplying each element of a matrix A by a scalar α is a process known as **scalar multiplication**. This results in the product αA:

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_6

$$\alpha \begin{bmatrix} A_{11} & \cdots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \cdots & A_{mn} \end{bmatrix} = \begin{bmatrix} \alpha A_{11} & \cdots & \alpha A_{1n} \\ \vdots & \ddots & \vdots \\ \alpha A_{m1} & \cdots & \alpha A_{mn} \end{bmatrix}.$$

Definition 6.2. (Scalar multiplication)

If $\alpha \in \mathbb{R}$ and $A \in \mathbb{R}^{m\times n}$, then $\alpha A \in \mathbb{R}^{m\times n}$ and

$$(\alpha A)_{ij} = \alpha A_{ij},$$

for all $i = 1, \ldots, m$ and $j = 1, \ldots, n$.

We obtain the transpose of a matrix A, denoted as A^T, by switching its rows and columns. That is,

$$\begin{bmatrix} A_{11} & \cdots & A_{1n} \\ A_{21} & \cdots & A_{2n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \cdots & A_{mn} \end{bmatrix}^T = \begin{bmatrix} A_{11} & A_{21} & \cdots & A_{m1} \\ \vdots & \vdots & \ddots & \vdots \\ A_{1n} & A_{2n} & \cdots & A_{mn} \end{bmatrix}.$$

Definition 6.3. (Transpose of a matrix)

If $A \in \mathbb{R}^{m\times n}$, then $A^T \in \mathbb{R}^{n\times m}$ and

$$(A^T)_{ij} = A_{ji},$$

for all $i = 1, \ldots, n$ and $j = 1, \ldots, m$.

Below is a summary of the properties of matrix addition, scalar multiplication, and transposition. It is a good exercise to verify these properties for yourself.

Theorem 6.4. (Properties of matrix addition, scalar multiplication, and transposition)

For any matrices $A, B, C \in \mathbb{R}^{m\times n}$, and scalars α and β, we have

$A + B = B + A$	(commutativity of addition)
$(A + B) + C = A + (B + C)$	(associativity of addition)
$A + 0_{m\times n} = 0_{m\times n} + A = A$	(additive identity)
$A + (-A) = (-A) + A = 0_{m\times n}$	(additive inverse)
$1 \cdot A = A$	(scalar multiplication identity)
$\alpha(\beta A) = (\alpha\beta)A$	(mixed associativity of scalar multiplication)
$\alpha(A + B) = \alpha A + \alpha B$	(distributive property I)
$(\alpha + \beta)A = \alpha A + \beta A$	(distributive property II)
$(A^T)^T = A$	
$(A + B)^T = A^T + B^T$	(linearity of transposition I)
$(\alpha A)^T = \alpha A^T$	(linearity of transposition II)

6.2 Matrix multiplication

Unlike addition and scalar multiplication, matrix multiplication is not performed componentwise. Before reviewing the multiplication of two matrices, let us first revisit the multiplication of a row vector by a column vector, which is known as the standard dot product.

Definition 6.5. (Dot product in $\mathbb{R}^n$)

Consider $u, v \in \mathbb{R}^n$. We call the **dot product** of u and v the scalar given by [1]

$$u^T v = \begin{bmatrix} u_1 & \cdots & u_n \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} := \sum_{i=1}^{n} u_i v_i,$$

where u_i and v_i denote, respectively, the ith entry of u and v.

Example 6.6 Consider the vectors

$$u = \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix} \qquad \text{and} \qquad v = \begin{bmatrix} 0 \\ -3 \\ 1 \end{bmatrix}.$$

Then,

[1] Notice that in the equation below we are using the symbol ":=". We use this symbol to define the expression on the left for the first time, indicating a definition rather than an equation. Conversely, when we use "=:", we are defining the expression on the right. On the other hand, the "=" symbol indicates that the expressions on either side of it are equal.

$$u^T v = \begin{bmatrix} 1 & -2 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ -3 \\ 1 \end{bmatrix} = 1 \cdot 0 + (-2) \cdot (-3) + 3 \cdot 1 = 9.$$

The following proposition states the dot product properties we will use throughout the book.

Proposition 6.7. (Properties of the dot product)

The dot product satisfies the following properties

(a) $u^T v = v^T u$, for all $u, v \in \mathbb{R}^n$.
(b) $u^T (v + w) = u^T v + u^T w$, for all $u, v, w \in \mathbb{R}^n$.
(c) $(\alpha u)^T v = \alpha (u^T v) = u^T (\alpha v)$, for all $u, v \in \mathbb{R}^n$ and $\alpha \in \mathbb{R}$.
(d) $u^T u \geqslant 0$, for all $u \in \mathbb{R}^n$, and $u^T u = 0$ if and only if $u = 0$.

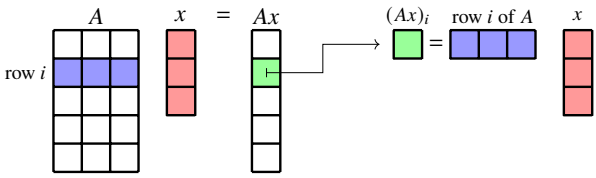
We can define matrix-vector multiplication using dot products by treating A as a set of row vectors.

Definition 6.8. (Matrix-vector product: Ax = rows of A times x)

Consider an $m \times n$ matrix A with rows $r_1^T, r_2^T, \ldots, r_m^T$, and let $x \in \mathbb{R}^n$. Then, the **product of A and x** is given by

$$Ax = \begin{bmatrix} r_1^T \\ r_2^T \\ \vdots \\ r_m^T \end{bmatrix} x := \begin{bmatrix} r_1^T x \\ r_2^T x \\ \vdots \\ r_m^T x \end{bmatrix}.$$

Simply put, the entries of Ax are the dot products of the rows of the matrix A with the vector x. Graphically, we can picture matrix-vector multiplication as follows:



Now finally, we recall how to multiply two generic matrices. If we think of a matrix A as a collection of row vectors and a matrix B as a collection of column vectors, the product AB contains all of the pairwise dot products of rows of A and columns of B.

Definition 6.9. (Matrix multiplication: rows times columns)

Let

$$A = \begin{bmatrix} r_1^T \\ r_2^T \\ \vdots \\ r_m^T \end{bmatrix} \in \mathbb{R}^{m\times n}, \quad \text{and} \quad B = \begin{bmatrix} c_1 & c_2 & \cdots & c_p \end{bmatrix} \in \mathbb{R}^{n\times p},$$

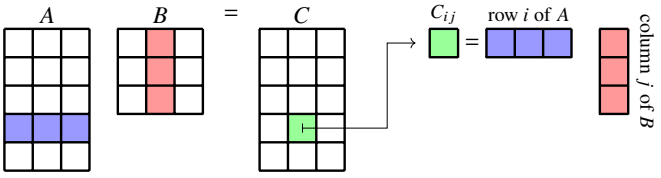
where r_i^T denotes the ith row of A and c_j denotes the jth column of B. Then, the product $AB \in \mathbb{R}^{m\times p}$ is given by

$$AB := \begin{bmatrix} r_1^T c_1 & r_1^T c_2 & \cdots & r_1^T c_p \\ r_2^T c_1 & r_2^T c_2 & \cdots & r_2^T c_p \\ \vdots & \vdots & \ddots & \vdots \\ r_m^T c_1 & r_m^T c_2 & \cdots & r_m^T c_p \end{bmatrix}.$$

Hence, the (i, j)th entry of AB is the dot product of the ith row of A and the jth column of B, that is,

$$(AB)_{ij} = r_i^T c_j = \sum_{k=1}^{n} A_{ik} B_{kj}. \tag{6.1}$$

Graphically,



The dot product and the matrix-times-vector definitions we discussed earlier are special cases of the general matrix product definition.

Example 6.10 Let us consider the matrices

$$A = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{bmatrix}.$$

We find AB by taking dot products of the rows of A with the columns of B:

$$AB = \begin{bmatrix} 1\cdot 0 + 0\cdot 2 + (-1)\cdot 4 & 1\cdot 1 + 0\cdot 3 + (-1)\cdot 5 \\ 2\cdot 0 + 1\cdot 2 + 0\cdot 4 & 2\cdot 1 + 1\cdot 3 + 0\cdot 5 \\ 0\cdot 0 + (-2)\cdot 2 + 1\cdot 4 & 0\cdot 1 + (-2)\cdot 3 + 1\cdot 5 \end{bmatrix} = \begin{bmatrix} -4 & -4 \\ 2 & 5 \\ 0 & -1 \end{bmatrix}. \tag{6.2}$$

Remark 6.11. (Matrix size compatibility for multiplication)

Note that matrix multiplication requires that the number of columns in the first matrix A matches the number of rows in the second matrix B. If this is the case, we say that A and B are **conformable for matrix multiplication**. Moreover, if A is $m \times n$ and B is $n \times p$, then their product AB will be a matrix of size $m \times p$.

Below are the main properties of matrix multiplication. We encourage you to verify that these properties hold.

Theorem 6.12. (Properties of matrix multiplication)

For any matrices A, B, C of appropriate sizes, and scalar $\alpha \in \mathbb{R}$, we have

$A(BC) = (AB)C$	(associative property)
$I_m A = AI_n = A$	(left and right identity for $m \times n$ matrices)
$\alpha(AB) = (\alpha A)B = A(\alpha B)$,	(commutativity property of scalars)
$(A + B)C = AC + BC$	(distributive property I)
$C(A + B) = CA + CB$	(distributive property II)
$(AB)^T = B^T A^T$	(transposition reverses the order)

Using a straightforward induction argument, we can generalize $(AB)^T = B^T A^T$ to the product of an arbitrary number of matrices.

Proposition 6.13. (Transpose of a product of matrices)

If $A_1, A_2, \ldots, A_k$ are matrices such that their product $A_1 A_2 \cdots A_k$ is defined, then

$$(A_1 A_2 \cdots A_k)^T = A_k^T \cdots A_2^T A_1^T. \tag{6.3}$$

We will refer to (6.3) as the **"reverse order law of transposition"**.

Remark 6.14. (The associative property and matrix multiplication performance)

The associative property in matrix multiplication means that the result of $A_1 A_2 \cdots A_k$ stays the same no matter how we group the matrices with parentheses. However, the way we group them can greatly impact the number of arithmetic operations needed. Problems 6.15 and 6.16 show how the order of operations affects performance.

We know that multiplication of real numbers satisfies the following properties:

1. **Closure**: if $a, b \in \mathbb{R}$, then $ab \in \mathbb{R}$.
2. **Associative**: for all $a, b \in \mathbb{R}$, $a(bc) = (ab)c$.
3. **Commutative**: for all $a, b \in \mathbb{R}$, $ab = ba$.
4. **Identity**: for all $a \in \mathbb{R}$, $1 \cdot a = a \cdot 1 = a$.
5. **Multiplicative inverse**: for all $0 \neq a \in \mathbb{R}$, there exists another real number denoted $1/a$ such that $a \cdot (1/a) = (1/a) \cdot a = 1$.
6. **Cancellation law**: for all $0 \neq a \in \mathbb{R}$ and all $b, c \in \mathbb{R}$, if $ab = ac$, then $b = c$.

Matrix multiplication shares some of these properties. For example, matrix multiplication is associative when the products $A(BC)$ and $(AB)C$ are defined, and the identity matrix of the appropriate size has the property: For any $m \times n$ matrix A, we have $AI_n = A$ and $I_m A = A$. However, there are important differences. Many errors in Linear Algebra come from overlooking these differences between matrix multiplication and scalar multiplication. Let us see some examples.

Multiplication is not (generally) closed

Unlike real numbers, not every pair of matrices can be multiplied. For multiplication to be closed, the operation must be restricted to square matrices of a fixed size n.

Matrix multiplication is not commutative

If $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, and $m \neq p$, we can compute AB, but BA is not defined. Thus, the commutative property does not hold.

If $m = p$ but $m \neq n$, then AB and BA are both defined but $AB \in \mathbb{R}^{m \times m}$, while $BA \in \mathbb{R}^{n \times n}$, and again the commutative property does not hold.

Even if we only consider square matrices of size n, in which case AB and BA are both defined and have the same size, often $AB \neq BA$.

The cancellation law does not hold

The cancellation law, which holds for scalars, does not hold for matrices. Even if $A \neq 0$, the equation $AB = AC$ does not necessarily mean $B = C$. For example, if

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, \quad \text{and} \quad C = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix},$$

then $AB = AC = \left[\begin{smallmatrix} 0 & 0 \\ 0 & 0 \end{smallmatrix}\right]$, but clearly $B \neq C$.

What about multiplicative inverses? We will address this in Section 6.3. Unlike nonzero real numbers, not every matrix has a multiplicative inverse.

Problems 6.8–6.13 illustrate more examples where matrices behave very differently from scalars.

Matrix powers

Just like with numbers, we can raise a square matrix to any nonnegative integer power by repeatedly multiplying it by itself. For an $n \times n$ matrix, we have

$$\begin{aligned} A^0 &= I_n, \\ A^1 &= A, \\ A^2 &= AA, \\ &\vdots \\ A^k &= \underbrace{AA \cdots A}_{k \text{ times}}. \end{aligned}$$

Notice it is the associative law that makes A^k well-defined, as the way we group the matrices does not matter. For example, whether we compute $A(AA)$ or $(AA)A$, the result is the same, which we denote by A^3.

The standard rules of exponents apply

$$A^k A^m = A^{k+m} \qquad \text{and} \qquad (A^k)^m = A^{km}.$$

We only define powers for square matrices because non-square matrices cannot be multiplied by themselves due to their dimensions.

6.3 Invertible matrices and their properties

In this section, we review invertible matrices and their inverses. Just as we solve $ax = b$ (with $a \neq 0$) by dividing by a, we will see how to use a matrix's inverse to solve the linear system of equations $Ax = b$ when A is invertible.

Notice that below we only consider the inverse of a square matrix. This is because invertibility makes sense only for square matrices (see Remark 6.24).

Definition 6.15. (The inverse matrix)

A square matrix $A \in \mathbb{R}^{n \times n}$ is **invertible (or nonsingular)** if and only if there exists a matrix $B \in \mathbb{R}^{n \times n}$ such that

$$AB = I_n = BA.$$

In this case, we refer to B as an **inverse** of A. If a matrix does not have an inverse, we call it **singular**.

Remark 6.16. (A single condition for invertibility)

In Definition 6.15, we require both $AB = I_n$ and $BA = I_n$ because matrix multiplication is not commutative. However, for square $n \times n$ matrices A and B, if $AB = I_n$, it automatically implies $BA = I_n$ (see Theorem 17.37 and Problem 17.8). So in practice, we only need to verify one condition, either $AB = I_n$ or $BA = I_n$.

As with real numbers, if an inverse for a matrix exists, it is unique. Therefore, we call the matrix B in Definition 6.15 "the inverse of A" and denote it by A^{-1}.

Theorem 6.17. (Uniqueness of the inverse)

If $A \in \mathbb{R}^{n\times n}$ is an invertible matrix, then there exists a unique matrix B such that $AB = I_n = BA$.

In the scalar case, the only real number without a multiplicative inverse is zero. Similarly, the $n \times n$ zero matrix 0_n is singular because any matrix B multiplied by 0_n results in 0_n. However, there are also nonzero square matrices with no inverse. For example, the matrix $A = \left[\begin{smallmatrix} 0 & 0 \\ 1 & 2 \end{smallmatrix}\right]$ is singular since the first row of AB is zero for every 2×2 matrix B.

How do we tell if a matrix is invertible or singular? Sometimes, it is easy to find a matrix B such that $AB = I_n = BA$, which implies A is invertible and $A^{-1} = B$. But this is not always the case. In Chapter 17, Theorem 17.37 and Problem 17.8 will give us more ways to identify invertible matrices.

The following theorem shows three fundamental properties of the inverse matrix.

Theorem 6.18. (Properties of the inverse matrix)

For $n \times n$ invertible matrices A and B, we have

(a) $\left(A^{-1}\right)^{-1} = A$,
(b) the product AB is invertible with $(AB)^{-1} = B^{-1}A^{-1}$, and
(c) A^T is invertible with $\left(A^T\right)^{-1} = \left(A^{-1}\right)^T$.

Using a straightforward induction argument, we can generalize part (b) in Theorem 6.18 to an arbitrary finite number of matrices.

Proposition 6.19. (Product of invertible matrices)

If $A_1, A_2, \ldots, A_k \in \mathbb{R}^{n\times n}$ are invertible, then their product $A_1A_2 \cdots A_k$ is invertible with its inverse given by

$$(A_1A_2 \cdots A_k)^{-1} = A_k^{-1} \cdots A_2^{-1}A_1^{-1}. \tag{6.4}$$

We will refer to (6.4) as the **"reverse order law of matrix inversion"**.

Let us look at some examples of invertible matrices and their inverses.

Example 6.20. (The inverse of a 2×2 matrix)

The matrix $A = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ is invertible if and only if $ad - bc \neq 0$, with its inverse given by

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

As in the 2×2 case above, there are formulas for the inverse of $n \times n$ matrices using their entries, but these are usually impractical for $n > 2$. Fortunately, we almost never need to form an inverse directly. When we do, Chapter 22 will show a more efficient way to do it.

We can easily invert other types of matrices such as diagonal matrices.

Example 6.21. (The inverse of a diagonal matrix)

The real diagonal matrix

$$D = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}$$

is invertible if and only if all of its diagonal entries d_i (for $i = 1, 2, \ldots, n$) are nonzero. In the former case, its inverse is the diagonal matrix

$$D^{-1} = \begin{bmatrix} \frac{1}{d_1} & 0 & \cdots & 0 \\ 0 & \frac{1}{d_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{d_n} \end{bmatrix}.$$

We can easily determine when a triangular matrix is invertible.

Example 6.22. (Invertible triangular matrices)

The upper triangular matrix

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ 0 & A_{22} & \cdots & A_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & A_{nn} \end{bmatrix}.$$

is invertible if and only if $A_{ii} \neq 0$, for $i = 1, \ldots, n$. Applying part (c) of Theorem 6.18 extends the result to lower triangular matrices as well.

For triangular matrices bigger than 2×2 or 3×3, there is not an easy formula for the inverse using the entries of the matrix. In the 2×2 case, as shown in Example 6.20, if a and d are nonzero, then

$$\begin{bmatrix} a & b \\ 0 & d \end{bmatrix}^{-1} = \begin{bmatrix} 1/a & -b/ad \\ 0 & 1/d \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} a & 0 \\ c & d \end{bmatrix}^{-1} = \begin{bmatrix} 1/a & 0 \\ -c/ad & 1/d \end{bmatrix}. \tag{6.5}$$

In Problem 6.21, you will learn how to derive a (complicated) explicit formula for the inverse of an $n \times n$ triangular matrix.

When a square matrix A is invertible, we can solve linear systems of equations $Ax = b$ by basically "dividing by A".

Theorem 6.23. (Solving a linear system using the inverse matrix)

Given an invertible matrix $A \in \mathbb{R}^{n \times n}$ and a vector $b \in \mathbb{R}^n$, the linear system $Ax = b$ has a unique solution given by

$$x = A^{-1}b.$$

Theorem 6.23 shows how to solve linear systems by finding A^{-1} and then multiplying by b, but we rarely use this method in practice. In Chapter 22, we will see that finding the inverse requires more computations than other methods for solving $Ax = b$.

We have defined invertibility only for square matrices, but what about rectangular matrices? Is there an analogous idea?

Remark 6.24. (Rectangular matrices and one-sided inverses)

For an $m \times n$ matrix A, we call an $n \times m$ matrix B a **right inverse** of A if $AB = I_m$. Similarly, we refer to B as a **left inverse** of A if $BA = I_n$. Later, in Chapter 17, we will discuss how a rectangular matrix can have either a right or a left inverse, but it is impossible for it to have both (see also Problem 6.23). In other words, if $n \neq m$, no rectangular matrix B can fulfill both conditions $AB = I_m$ and $BA = I_n$ simultaneously.

Example 6.25 Consider the 2×3 matrix $A = \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]$. The 3×2 matrix

$$B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

is a right inverse of A since it satisfies $AB = I_2$. However, there is not a 3×2 matrix B for which $BA = I_3$, as the third column of BA would be all zeros, no matter the choice of B. Thus, A has a right inverse, but it does not have a left inverse.

6.4 Linear transformations and their matrix representation

When we talk about a "transformation", we mean a function that maps vectors from one vector space to another. In this section, we focus on a specific type of transformation.

Definition 6.26. (Linear transformation)

A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is called a **linear transformation** if it satisfies the following property:

$$f(\alpha x + \beta y) = \alpha f(x) + \beta f(y), \tag{6.6}$$

for any two vectors $x, y \in \mathbb{R}^n$ and scalars $\alpha, \beta \in \mathbb{R}$.

For any linear transformation f, it follows from (6.6) that $f(0) = 0$. This means that a linear transformation $f : \mathbb{R}^n \to \mathbb{R}^m$ always maps the zero vector in $\mathbb{R}^n$ to the zero vector in $\mathbb{R}^m$.

We can characterize all linear transformations $f : \mathbb{R}^n \to \mathbb{R}^m$ as follows.

Proposition 6.27. (The matrix representation)

Let $f : \mathbb{R}^n \to \mathbb{R}^m$ be a linear transformation. Then, there exists an $m \times n$ matrix A such that $f(x) = Ax$. If c_i denotes the ith column of A, then $c_i = f(e_i)$, where e_i is the ith column of I_n.

The previous result states that a linear transformation is uniquely determined by its action on the vectors e_i, for $i = 1, \ldots, n$.

Example 6.28 Consider the linear transformation f on $\mathbb{R}^2$ that reflects any vector $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ across the x_1-axis. Note that

$$f(e_1) = f\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \text{and} \qquad f(e_2) = f\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

Thus, the matrix representation of f is

$$f(x) = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

In many situations, we need to apply two linear transformations, g and f, one after the other. This leads to the concept of the composition of transformations. The composition $g \circ f$ is defined as:

$$(g \circ f)(x) := g(f(x)).$$

Proposition 6.29. (Composition of linear transformations)

Let $f : \mathbb{R}^n \to \mathbb{R}^m$ and $g : \mathbb{R}^m \to \mathbb{R}^p$ be two linear transformations. Then, the composition $g \circ f : \mathbb{R}^n \to \mathbb{R}^p$ of f and g is also a linear transformation.

Recall from Proposition 6.27 that any linear transformation $\mathbb{R}^n \to \mathbb{R}^m$ can be written as multiplication by a matrix. Which matrix represents the composition $g \circ f$?

If $f(x) = Ax$ and $g(x) = Bx$ with $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times m}$, then

$$g(f(x)) = B(Ax) = (BA)x,$$

where the second equality is a consequence of the associative property of matrix multiplication. Thus, $BA \in \mathbb{R}^{p \times n}$ is the matrix associated with the composition $g \circ f$. The fact that the product of two matrices represents the composition of their linear transformations explains why matrix multiplication is defined in its seemingly "unnatural" way.

Thus, we have the following theorem.

Theorem 6.30. (Matrix multiplication as linear transformation composition)

Let $f(x) = Ax$ and $g(x) = Bx$ be two linear transformations, where $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{p \times m}$. The composition $g \circ f$ is a linear transformation with matrix representation BA.

Example 6.31 Consider the matrices

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \qquad \text{and} \qquad B = \begin{bmatrix} \cos(\pi/6) & -\sin(\pi/6) \\ \sin(\pi/6) & \cos(\pi/6) \end{bmatrix}.$$

Figure 6.1 shows the effect of applying A (a scaling) followed by B (a rotation) on the vector $x = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$. The overall effect is the same as applying to the vector x the matrix

$$C = BA = \begin{bmatrix} 2\cos(\pi/6) & -\sin(\pi/6) \\ 2\sin(\pi/6) & \cos(\pi/6) \end{bmatrix}.$$

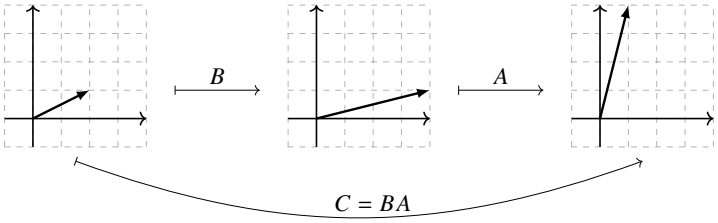


Fig. 6.1: Matrix multiplication computes the composition of two linear transformations.

Next, we consider a particular type of function.

Definition 6.32. (Invertible function)

A function $f : \mathbb{R}^n \to \mathbb{R}^n$ is **invertible** if there exists a function $g : \mathbb{R}^n \to \mathbb{R}^n$ such that $g \circ f = \mathrm{id}_{\mathbb{R}^n}$ and $f \circ g = \mathrm{id}_{\mathbb{R}^n}$[2], that is,

$$g(f(x)) = x \quad \text{and} \quad f(g(y)) = y, \quad \text{for all } x \in \mathbb{R}^n \text{ and } y \in \mathbb{R}^n.$$

We refer to g as the **inverse function** of f.

It can be shown that, if a function f is invertible, it has a unique inverse (Problem 6.29), denoted by f^{-1} .

Proposition 6.33. (Inverse of a linear transformation)

Let $f : \mathbb{R}^n \to \mathbb{R}^n$ be an invertible linear transformation. Then, $f^{-1} : \mathbb{R}^n \to \mathbb{R}^n$ is also a linear transformation.

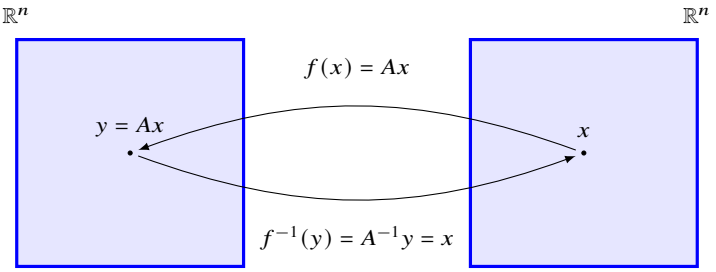
Since f^{-1} is also linear, it must have a matrix representation.

Theorem 6.34. (Invertible matrices characterize invertible linear transformations)

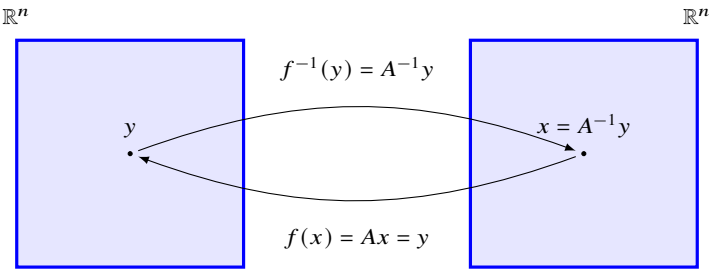
A linear transformation $f(x) = Ax$ with $A \in \mathbb{R}^{n\times n}$ is invertible if and only if A is invertible, in which case $f^{-1}(x) = A^{-1}x$.

[2] The function $\mathrm{id}_{\mathbb{R}^n}$ denotes the "identity function on $\mathbb{R}^n$", defined as $\mathrm{id}_{\mathbb{R}^n}(x) = x$ for all $x \in \mathbb{R}^n$.

Figure 6.2 illustrates how a linear transformation and its inverse operate.



(a) The inverse linear transformation f^{-1} "undoes what f does", that is, f^{-1} takes every $y = f(x)$ back to x since $f^{-1}(f(x)) = x$.



(b) The linear transformation f "undoes what f^{-1} does", that is, f takes every $x = f^{-1}(y)$ back to y since $f(f^{-1}(y)) = y$.

Fig. 6.2: A linear transformation $f(x) = Ax$ and its inverse linear transformation $f^{-1}(x) = A^{-1}x$.

Problems

6.1 Verify all the properties listed in Theorem 6.4.

6.2 Solve for X in $2X + A = B$, given that $A = \begin{bmatrix} 3 & -1 \\ -2 & 1 \\ 1 & 1 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 3 \\ 2 & 1 \\ 3 & 3 \end{bmatrix}$.

6.3 Count the number of arithmetic operations (additions/subtractions and multiplications/divisions) necessary for computing the dot product $u^T v$, where $u, v \in \mathbb{R}^n$.

6.4 Prove Proposition 6.7.

6.5 Prove Theorem 6.12.

6.6 (Multiplication by a diagonal matrix) Verify the following. When you multiply a matrix A on the left by a diagonal matrix D, the rows of A get scaled by the main

diagonal entries of D. Conversely, when A is multiplied by D on the right, its columns get scaled by those same numbers. In other words, if A is an $m \times n$ matrix with rows $r_1^T, \ldots, r_m^T$ and columns $c_1, \ldots, c_n$, then

$$\begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_m \end{bmatrix} \begin{bmatrix} r_1^T \\ \vdots \\ r_m^T \end{bmatrix} = \begin{bmatrix} d_1 r_1^T \\ \vdots \\ d_m r_m^T \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} c_1 & \cdots & c_n \end{bmatrix} \begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{bmatrix} = \begin{bmatrix} d_1 c_1 & \cdots & d_n c_n \end{bmatrix}.$$

6.7 (The trace function) For a square matrix $A \in \mathbb{R}^{n \times n}$, the *trace* is the sum of its diagonal entries:

$$\operatorname{trace}(A) = \sum_{i=1}^{n} A_{ii}.$$

Show that if $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times m}$, then $\operatorname{trace}(AB) = \operatorname{trace}(BA)$ even though $AB \neq BA$ in general.

6.8 Prove that, in general, $(A + B)^2$ is not equal to $A^2 + 2AB + B^2$. Explain why.

6.9 Given the matrix $A = \left[\begin{smallmatrix} 2 & -3 \\ -4 & 6 \end{smallmatrix}\right]$, find two distinct matrices B and C in $\mathbb{R}^{2 \times 2}$ such that $AB = AC$. This means that cancellation law does not hold for matrix multiplication. What extra condition could be imposed on A for this property to hold?

6.10 Find a 2×2 matrix A such that $A^2 = \left[\begin{smallmatrix} -1 & 0 \\ 0 & -1 \end{smallmatrix}\right]$.

6.11 Find a nonzero 3×3 matrix A such that $A^2 = 0$.

6.12 Find two nonzero matrices A and B such that $AB = 0$.

6.13 Find a nonzero 3×3 matrix A such that $A^2 \neq 0$ but $A^3 = 0$.

6.14 (The cost of matrix multiplication) Find the number of arithmetic operations (additions/subtractions and multiplications/divisions) needed to compute the matrix product AB, where $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times q}$.

6.15 Matrix multiplication is associative. In other words, no matter how the product is parenthesized, the result obtained will remain the same. For example, for four matrices A, B, C, and D, we have:

$$\begin{aligned} ABCD = &((AB)C)D = \\ &(A(BC))D = \\ &(AB)(CD) = \\ &A((BC)D) = \\ &A(B(CD)). \end{aligned}$$

Suppose you have four matrices: $A \in \mathbb{R}^{1 \times 100}$, $B \in \mathbb{R}^{100 \times 100}$, $C \in \mathbb{R}^{100 \times 10}$ and $D \in \mathbb{R}^{10 \times 1}$. What would be the most efficient way (in terms of the number of arithmetic operations) of computing the product $ABCD$?

6.16 Consider the computation of the matrix $B = xy^T A$, where x and y are column vectors in $\mathbb{R}^n$, and A is an $n \times n$ matrix. The associative law provides two options for the order of multiplication:

(a) compute $(xy^T)A$, that is, compute xy^T first, or
(b) compute $x(y^T A)$, that is, compute $y^T A$ first.

The result is the same but the number of arithmetic operations differs significantly. Count the number of arithmetic operations necessary for computing $B = (xy^T)A$, and $B = x(y^T A)$.

6.17 Show that if A is invertible, then the equation $Ax = 0$ or $x^T A = 0$ has only the solution $x = 0$.

6.18 Using Problem 6.17, show that the square matrix A is singular in each of the following situations:

(a) A has a row of zeros.
(b) A has two identical columns.
(c) A has one column that is a multiple of another column.

6.19 Consider the 2×2 matrix $A = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$. Prove that A is singular if $ad - bc = 0$.

Hint: If A is the zero matrix, it is clearly singular. Assuming A is nonzero, find a nonzero vector x such that $Ax = 0$. Use Problem 6.17 to conclude that A is singular.

6.20 For an upper or lower triangular matrix A, prove that A is singular if any of its diagonal elements are zero.

6.21 (The inverse of an upper triangular matrix) Consider an $n \times n$ upper triangular matrix A with nonzero diagonal entries. Write A as $A = D + U$, where D is diagonal with A's diagonal entries, and U is upper triangular with A's off-diagonal entries and zeros on the diagonal. Then, $A = D(I_n + D^{-1}U)$.

(a) Show that $(D^{-1}U)^n = 0$.

Hint: Look at the patterns in the nonzero entries when computing powers of $D^{-1}U$.

(b) Show that $I_n + D^{-1}U$ is invertible with $(I_n + D^{-1}U)^{-1} = \sum_{k=0}^{n-1} (-D^{-1}U)^k$.
(c) Conclude that

$$A^{-1} = \left(\sum_{k=0}^{n-1} (-D^{-1}U)^k \right) D^{-1}.$$

6.22 Use Problem 6.21 to derive an explicit formula for the inverse of a 3×3 upper triangular matrix using its entries.

6.23 Applying the result from Problem 6.7, prove that for matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times m}$, if $AB = I_m$ and $BA = I_n$, it must follow that $m = n$.

6.24 Consider $n \times n$ matrices A and B. Prove that if their product $C = AB$ is invertible, then both A and B must also be invertible.

Hint: Try to find expressions for A^{-1} and B^{-1} using C^{-1}.

6.25 Prove that the definition of a linear transformation in (6.6) is equivalent to f being linear if

$$f(\alpha x + y) = \alpha f(x) + f(y),$$

for any scalar $\alpha \in \mathbb{R}$ and for any vectors $x, y \in \mathbb{R}^n$.

6.26 Which of the following transformations are linear? If it is, find the associated matrix A.

(a) $f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} x_3 \\ x_2 \\ x_1 \end{bmatrix}$ (b) $f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 3x_1 - x_2 \\ 0 \\ x_3^2 \end{bmatrix}$ (c) $f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ e^{x_1} \end{bmatrix}$

(d) $f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} x_1 + x_2 \\ x_1 - x_2 \\ 2x_1 \end{bmatrix}$ (e) $f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} x_2 \\ x_1 x_2 \end{bmatrix}$ (f) $f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = x_1 - x_2$

(g) $f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} x_2 \\ x_1 \\ 1 \end{bmatrix}$ (h) $f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ (i) $f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ |x_1| \end{bmatrix}$

6.27 Show that the composition $F(x) = g(f(x))$ of two linear transformations f and g is also a linear transformation. That is, verify that F satisfies (6.6).

6.28 In $\mathbb{R}^3$, find the linear transformation which reflects about the x_3-axis, then rotates by 180 degrees around the x_1-axis, then reflects about the x_1x_2-plane.

6.29 Prove that if an inverse function exists, then it is unique.

6.30 Prove that if a linear transformation f is invertible, then its inverse f^{-1} is also a linear transformation; that is, verify that whenever f^{-1} exists, it satisfies (6.6).

Chapter 7
II-CORE TOPICS

In this chapter we examine several equivalent ways to define standard matrix multiplication. Later, in Chapter 8, we will consider non-standard products such as the Hadamard and Kronecker products.

7.1 Standard matrix multiplication: new perspectives

As noted in the previous chapter, standard matrix multiplication, unlike addition or scalar multiplication, is not performed componentwise. This section presents several equivalent ways to express the product AB of two matrices A and B.

7.1.1 Matrix-vector and vector-matrix multiplication

According to Definition 6.8, the product Ax (a matrix A multiplied by a vector x) is a vector, where each entry is the dot product of the rows of A with x. An equally useful way to view Ax is in terms of the columns of A rather than its rows.

Proposition 7.1. (Ax = **linear combination of the columns of** A)

Let $A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}$ be an $m \times n$ matrix with columns $a_1, a_2, \ldots, a_n$, and let $x = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}^T$ be a column vector. Then, Ax is a linear combination[1] of the columns of A with coefficients the entries of x, that is,

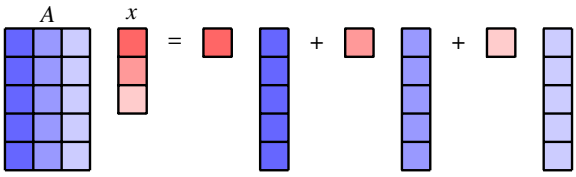
$$Ax = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = x_1 a_1 + x_2 a_2 + \cdots + x_n a_n.$$

[1] If you do not remember what a linear combination of vectors is, see Definition 16.4.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_7

Graphically,



Proof By the "rows of A times x" definition (Definition 6.8) of matrix-vector multiplication, we have

$$\begin{aligned} Ax &= \begin{bmatrix} A_{11}x_1 + A_{12}x_2 + \cdots + A_{1n}x_n \\ A_{21}x_1 + A_{22}x_2 + \cdots + A_{2n}x_n \\ \vdots \\ A_{m1}x_1 + A_{m2}x_2 + \cdots + A_{mn}x_n \end{bmatrix} = \begin{bmatrix} A_{11}x_1 \\ A_{21}x_1 \\ \vdots \\ A_{m1}x_1 \end{bmatrix} + \begin{bmatrix} A_{12}x_2 \\ A_{22}x_2 \\ \vdots \\ A_{m2}x_2 \end{bmatrix} + \cdots + \begin{bmatrix} A_{1n}x_n \\ A_{2n}x_n \\ \vdots \\ A_{mn}x_n \end{bmatrix} \\ &= x_1 \begin{bmatrix} A_{11} \\ A_{21} \\ \vdots \\ A_{m1} \end{bmatrix} + x_2 \begin{bmatrix} A_{12} \\ A_{22} \\ \vdots \\ A_{m2} \end{bmatrix} + \cdots + x_n \begin{bmatrix} A_{1n} \\ A_{2n} \\ \vdots \\ A_{mn} \end{bmatrix} \\ &= x_1 a_1 + x_2 a_2 + \cdots + x_n a_n, \end{aligned}$$

where the second equality follows from the definition of vector addition, and the third equality is by definition of scalar multiplication of vectors. □

Example 7.2 Consider the matrix $A = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$ and the vector $x = \left[\begin{smallmatrix} -3 \\ 2 \end{smallmatrix}\right]$. Then, Ax is the linear combination

$$Ax = -3 \begin{bmatrix} 1 \\ 3 \end{bmatrix} + 2 \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Using the definition of standard matrix multiplication, we can express the product of a row vector y^T with a matrix A in terms of the columns of A.

Proposition 7.3. (Vector-matrix product: $y^T A = y^T$ times columns of A)

Consider a matrix $A \in \mathbb{R}^{m \times n}$ with columns $c_1, c_2, \ldots, c_n \in \mathbb{R}^m$, and let $y \in \mathbb{R}^m$. Then, the product of y^T and A is

$$y^T A = y^T \begin{bmatrix} c_1 & c_2 & \cdots & c_n \end{bmatrix} = \begin{bmatrix} y^T c_1 & y^T c_2 & \cdots & y^T c_n \end{bmatrix}.$$

Simply put, $y^T A$ is a row vector whose entries are the dot products of the vector y with the columns of A.

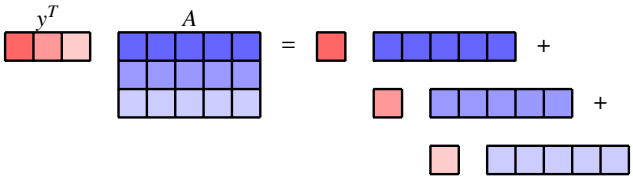
We can also interpret the product $y^T A$ as a linear combination of the rows of A, just as we did with Ax.

Proposition 7.4. ($y^T A$ = **linear combination of the rows of** A)

Consider a matrix $A \in \mathbb{R}^{m\times n}$ with rows $r_1^T, \ldots, r_m^T$, and let $y = \begin{bmatrix} y_1 & \cdots & y_m \end{bmatrix}^T \in \mathbb{R}^m$. Then, $y^T A$ is a linear combination of the rows of A with coefficients the entries of y, that is,

$$y^T A = \begin{bmatrix} y_1 & \cdots & y_m \end{bmatrix} \begin{bmatrix} r_1^T \\ \vdots \\ r_m^T \end{bmatrix} = y_1 r_1^T + \cdots + y_m r_m^T.$$

Graphically,



Proof Note that the transpose of the rows of A are the columns of A^T. Thus, by Proposition 7.1, we have $A^T y = y_1 r_1 + \cdots + y_m r_m$. Using the linearity of the transpose operation, we get

$$y^T A = (A^T y)^T = (y_1 r_1 + \cdots + y_m r_m)^T = y_1 r_1^T + \cdots + y_m r_m^T,$$

as desired. □

7.1.2 The product Ax as a linear transformation

As mentioned in Proposition 6.27, any linear transformation $f : \mathbb{R}^n \to \mathbb{R}^m$ can be represented by an $m \times n$ matrix A such that $f(x) = Ax$. In other words, we can think of every matrix as a linear transformation, and Ax as how that transformation acts on x.

Application: The geometry of linear systems

Solving linear systems is a central problem in Linear Algebra. Using matrix multiplication, we can express any linear system as $Ax = b$, where A is the matrix, x is the vector of unknowns, and b is the vector of right-hand sides.

For example, let us consider the following linear system of equations with two variables and two equations:

$$\begin{aligned} 2x_1 + 3x_2 &= 9 \\ -x_1 + 2x_2 &= -1 \end{aligned} \tag{7.1}$$

whose only solution is $x_1 = 3$ and $x_2 = 1$. We can express (7.1) as

$$\underbrace{\begin{bmatrix} 2 & 3 \\ -1 & 2 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 9 \\ -1 \end{bmatrix}}_{b}. \tag{7.2}$$

Next, we will look at three ways to interpret the solution of a linear system, based on the different approaches to matrix-vector multiplication we have discussed. We will also interpret these approaches geometrically to better understand the solution visually.

To illustrate these interpretations, we will use the system in (7.2).

The row picture: The linear system (7.1), expressed as two linear equations, is equivalent to computing Ax as the dot products of the rows of A and x in (7.2). Geometrically, the solution set of each individual equation forms a line in $\mathbb{R}^2$. The system's solution, $\left[\begin{smallmatrix} x_1 \\ x_2 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 3 \\ 1 \end{smallmatrix}\right]$, is where these two lines intersect. See Figure 7.1.

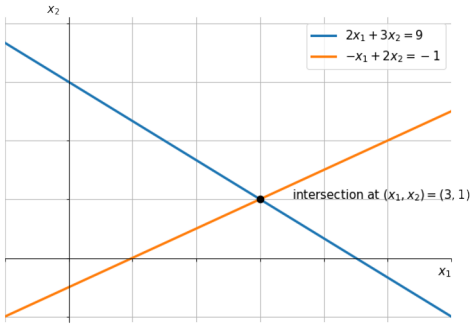


Fig. 7.1: The row picture of the linear system (7.1): the solution $(x_1, x_2) = (3, 1)$ is the intersection of the two lines $2x_1 + 3x_2 = 9$ and $-x_1 + 2x_2 = -1$.

The column picture: By considering matrix-vector multiplication as a linear combination of the columns of A in (7.2) we seek scalars x_1 and x_2 such that

$$x_1 \begin{bmatrix} 2 \\ -1 \end{bmatrix} + x_2 \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 9 \\ -1 \end{bmatrix}.$$

This results in a single vector equation instead of the two scalar equations in (7.1). The solution, $x_1 = 3$ and $x_2 = 1$, comes from the coefficients of the linear combination of the two columns $a_1 = \left[\begin{smallmatrix} 2 \\ -1 \end{smallmatrix}\right]$ and $a_2 = \left[\begin{smallmatrix} 3 \\ 2 \end{smallmatrix}\right]$ of the matrix A that produce the vector $b = \left[\begin{smallmatrix} 9 \\ -1 \end{smallmatrix}\right]$. See Figure 7.2.

The linear transformation picture: The left-hand side of $Ax = b$ can be seen as a linear transformation. Thus, the solution is a vector that the transformation

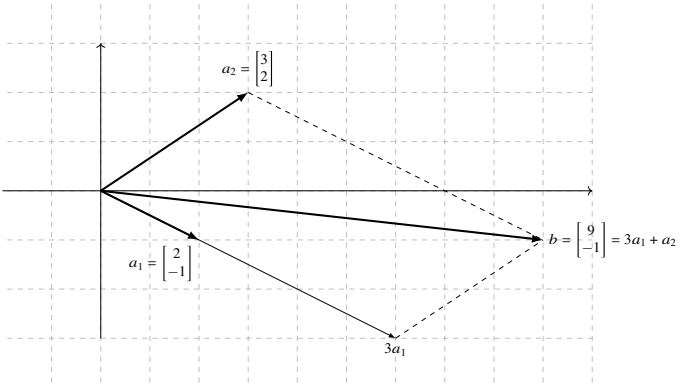


Fig. 7.2: The column picture of the linear system (7.1): the solution is the linear combination of the columns of A that produces the vector b.

$f(x) = Ax$ maps to b. In our example, the solution $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ to the system (7.2) is the vector that the linear transformation $f(x) = Ax$ maps to $\begin{bmatrix} 9 \\ -1 \end{bmatrix}$. See Figure 7.3.

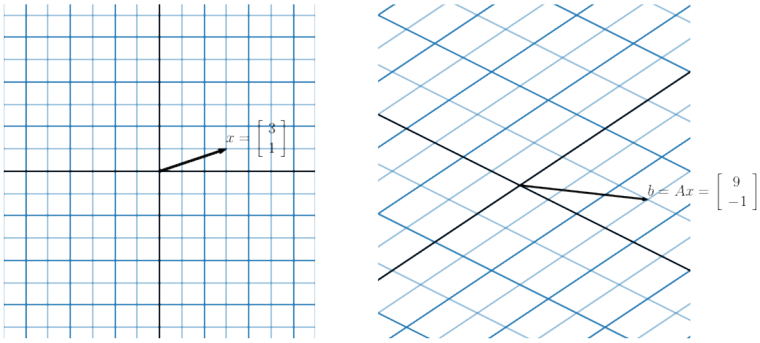


Fig. 7.3: The matrix picture of the linear system (7.1): the solution $x = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$ is the vector that is transformed to $\begin{bmatrix} 9 \\ -1 \end{bmatrix}$ by the linear transformation $f(x) = Ax = \begin{bmatrix} 2 & 3 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$.

For matrices with two columns, the geometry of these three approaches may seem quite similar. But as the problem size increases to a general n, the visual interpretation becomes harder. Consider a 3×3 matrix A, for example. The "row picture" involves finding the intersection of three planes in 3D space, which is difficult to visualize. On the other hand, the "column picture" asks how to combine three vectors (the columns

of A) to get the vector b. The column picture often gives a clearer understanding of the solution to a linear system.

7.1.3 Matrix-matrix multiplication

We have looked at several ways to multiply a vector by a matrix, starting from the standard definition of matrix-vector multiplication. Now we present equivalent ways to describe the product of two matrices.

In Definition 6.9, we compute the entries of AB by taking the dot products of rows from A with columns from B. We can rearrange these operations in different ways, giving us various perspectives on matrix multiplication.

We can express the product AB using the columns of B.

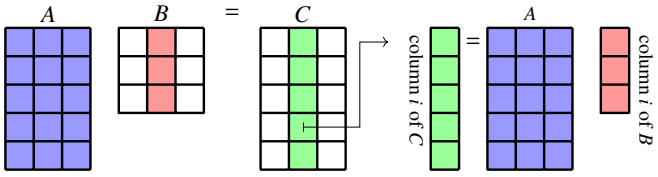
Proposition 7.5. (The columns of AB)

The ith column of AB equals A times the ith column of B. In other words, we have

$$AB = A\begin{bmatrix} c_1 & c_2 & \cdots & c_p \end{bmatrix} = \begin{bmatrix} Ac_1 & Ac_2 & \cdots & Ac_p \end{bmatrix},$$

where c_i denotes the ith column of B.

Graphically,



Example 7.6 For the matrices

$$A = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{bmatrix},$$

we can compute AB column by column:

$$AB = \left[\begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \\ 4 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \\ 0 & -2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} \right] = \begin{bmatrix} -4 & -4 \\ 2 & 5 \\ 0 & -1 \end{bmatrix}.$$

We can also express the product AB in terms of the rows of the matrix A.

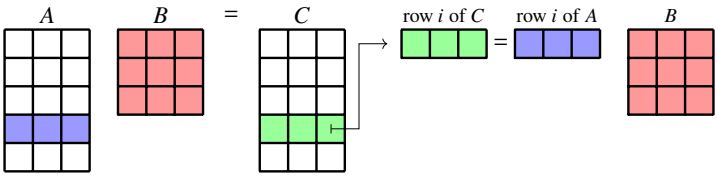
Proposition 7.7. (The rows of AB)

The ith row of AB equals the ith row of A times B. In other words, we have

$$AB = \begin{bmatrix} r_1^T \\ r_2^T \\ \vdots \\ r_m^T \end{bmatrix} B = \begin{bmatrix} r_1^T B \\ r_2^T B \\ \vdots \\ r_m^T B \end{bmatrix},$$

where r_i^T denotes the ith row of A.

Graphically,



Example 7.8 For the matrices A and B in Example 7.6, we can also compute AB row by row:

$$AB = \begin{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{bmatrix} \\ \begin{bmatrix} 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{bmatrix} \\ \begin{bmatrix} 0 & -2 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 2 & 3 \\ 4 & 5 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} -4 & -4 \\ 2 & 5 \\ 0 & -1 \end{bmatrix}.$$

Another way to compute the product AB is by using the columns of A and the rows of B.

Proposition 7.9. (Matrix multiplication: columns times rows)

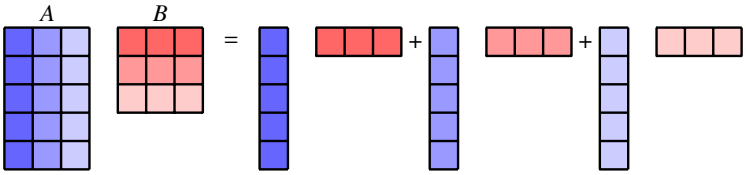
Consider two matrices

$$A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \in \mathbb{R}^{m \times n} \qquad \text{and} \qquad B = \begin{bmatrix} b_1^T \\ b_2^T \\ \vdots \\ b_n^T \end{bmatrix} \in \mathbb{R}^{n \times p},$$

where a_i denotes the ith column of A, and b_i^T denotes the ith row of B. Then,

$$AB = a_1 b_1^T + a_2 b_2^T + \cdots + a_n b_n^T.$$

Graphically,



Proof First notice that each matrix $a_i b_i^T$ has size $m \times p$, the size of AB. We want to show that $(AB)_{ij} = (a_1 b_1^T + \cdots + a_n b_n^T)_{ij}$, for $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, p$. We have

$$\begin{aligned} (a_1 b_1^T + \cdots + a_n b_n^T)_{ij} &= (a_1 b_1^T)_{ij} + \cdots + (a_n b_n^T)_{ij} \\ &= A_{i1} B_{1j} + \cdots + A_{in} B_{nj} \\ &= (AB)_{ij}, \end{aligned}$$

where the first equality follows from the definition of matrix addition and the second equality follows from the definition of matrix multiplication. □

Example 7.10 For the matrices A and B from Example 7.6, we compute AB as follows:

$$\begin{aligned} AB &= \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ -2 \end{bmatrix} \begin{bmatrix} 2 & 3 \end{bmatrix} + \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 4 & 5 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 \\ 0 & 2 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 2 & 3 \\ -4 & -6 \end{bmatrix} + \begin{bmatrix} -4 & -5 \\ 0 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} -4 & -4 \\ 2 & 5 \\ 0 & -1 \end{bmatrix}, \end{aligned}$$

matching the previous results we obtained when we computed this product using two other approaches.

Rank-one matrices and outer products

Using the "columns-times-rows" view of matrix multiplication, we can express AB as a sum of matrices in the form uv^T, where u and v are column vectors. When u and v are nonzero, the matrix uv^T is called a **rank-one matrix** (we will explain this term in Chapter 17). It is also known as the **outer product** of u and v. We will study these matrices in later chapters, but for now, let us look at some of their properties.

Consider two vectors

$$u = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix} \qquad \text{and} \qquad v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}.$$

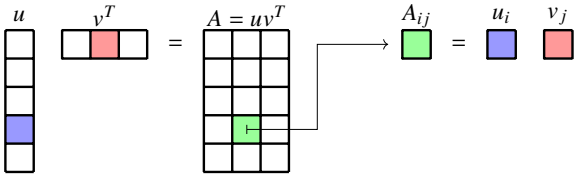
We can express the $m \times n$ matrix $A = uv^T$ in three distinct but equivalent ways:

$$\begin{aligned} A = uv^T &= \begin{bmatrix} uv_1 & uv_2 & \cdots & uv_n \end{bmatrix} && \text{(columns of } A\text{)} \\ &= \begin{bmatrix} u_1 v^T \\ u_2 v^T \\ \vdots \\ u_m v^T \end{bmatrix} && \text{(rows of } A\text{)} \\ &= \begin{bmatrix} u_1v_1 & u_1v_2 & \cdots & u_1v_n \\ u_2v_1 & u_2v_2 & \cdots & u_2v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_mv_1 & u_mv_2 & \cdots & u_mv_n \end{bmatrix} && \text{(entries of } A\text{)}. \end{aligned}$$

Although we rarely need to form an outer product matrix explicitly (see, for example, Problem 6.16), it is important to keep in mind the following facts about the matrix $A = uv^T$:

(i) each row of A is a multiple of v^T;
(ii) each column of A is a multiple of u; and
(iii) the element at the (i, j)th position of A is the product of the ith entry of u and the jth entry of v, that is, $A_{ij} = (uv^T)_{ij} = u_i v_j$.

Graphically,



7.1.4 The product AB as a composition of linear transformations

As recalled in Theorem 6.30, given two linear transformations $f : \mathbb{R}^n \to \mathbb{R}^m$ and $g : \mathbb{R}^m \to \mathbb{R}^p$, with matrix representations A and B, respectively, we have

$$g(f(x)) = BAx,$$

that is, the matrix representation of the composition $g \circ f$ is the product of the matrix representations. Thus, we can see the product of two matrices as the composition of two linear transformations and the product BAx as the action of those concatenated transformations on x.

7.1.5 Matrix factorizations

So far, we have seen how to multiply two matrices. In many Machine Learning applications, it is just as important to reverse the process and write a matrix A as a product of other matrices. This process is called **factorization**.

In upcoming chapters, we will introduce different matrix factorization methods, which help extract important information about the matrix or how it acts on vectors. These include, for example, **LU**, **QR**, **eigenvalue**, and **SVD** factorizations. These factorizations are useful for tasks like dimensionality reduction, solving least squares problems, linear systems, feature extraction, and recommender systems.

Some types of matrix factorizations can simplify solving certain problems.

Example 7.11 Consider solving the linear system $Ax = b$, where

$$A = \begin{bmatrix} 1 & -2 & 3 \\ -2 & 6 & -7 \\ 3 & -2 & 10 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}.$$

If we factor A as follows [2]:

$$A = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 3 & 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & -2 & 3 \\ 0 & 2 & -1 \\ 0 & 0 & 3 \end{bmatrix},$$

we can solve $Ax = b$ by breaking it into two simpler triangular systems:

$$\begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 3 & 2 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 1 & -2 & 3 \\ 0 & 2 & -1 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}.$$

From a geometric perspective, factoring a matrix into simpler matrices helps us understand its action on vectors.

[2] This is the LU factorization of A, which will be introduced in Chapter 22.

Example 7.12 Consider 2×2 matrix

$$A = \sqrt{2}\begin{bmatrix} -3/2 & 3/2 \\ 1 & 1 \end{bmatrix},$$

which defines the linear transformation $f(x) = Ax$. To understand how A acts on vectors, we can factor A as:

$$A = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}\begin{bmatrix} \sqrt{2}/2 & -\sqrt{2}/2 \\ \sqrt{2}/2 & \sqrt{2}/2 \end{bmatrix} =: A_3A_2A_1.$$

For $x \in \mathbb{R}^2$, we compute Ax as $A_3(A_2(A_1x))$, using matrix multiplication's associative property. First, A_1 rotates x by $\pi/4$ radians counterclockwise. Then, A_2 stretches the first component by 3 and the second by 2. Finally, A_3 reflects the result across the x_2-axis.

7.2 Partitioned/Block matrices

In certain applications or for numerical reasons, it is sometimes convenient to partition a matrix into submatrices (also known as blocks), before doing operations with them.

Example 7.13. (Block matrices)

We say that the matrix

$$A = \left[\begin{array}{ccc|cc|c} 3 & 0 & -1 & 5 & 9 & -2 \\ -5 & 2 & 4 & 0 & -3 & 1 \\ \hline -8 & -6 & 3 & 1 & 7 & -4 \end{array}\right]$$

is a 2×3 **block (or partitioned) matrix**.

We can represent a generic 2×3 **block matrix** as

$$A = \left[\begin{array}{c|c|c} A_{11} & A_{12} & A_{13} \\ \hline A_{21} & A_{22} & A_{23} \end{array}\right].$$

In our example above, we have that the **block entries** A_{ij} are

$$A_{11} = \begin{bmatrix} 3 & 0 & -1 \\ -5 & 2 & 4 \end{bmatrix}, \qquad A_{12} = \begin{bmatrix} 5 & 9 \\ 0 & -3 \end{bmatrix}, \qquad A_{13} = \begin{bmatrix} -2 \\ 1 \end{bmatrix},$$
$$A_{21} = \begin{bmatrix} -8 & -6 & 3 \end{bmatrix}, \qquad A_{22} = \begin{bmatrix} 1 & 7 \end{bmatrix}, \quad \text{and} \qquad A_{23} = \begin{bmatrix} -4 \end{bmatrix}.$$

We can partition the same matrix in several different ways. For example, we could have also partitioned the matrix A as

$$A = \left[\begin{array}{cc|cc|cc} 3 & 0 & -1 & 5 & 9 & -2 \\ \hline -5 & 2 & 4 & 0 & -3 & 1 \\ -8 & -6 & 3 & 1 & 7 & -4 \end{array}\right] \quad \text{or} \quad A = \left[\begin{array}{cc|cc|cc} 3 & 0 & -1 & 5 & 9 & -2 \\ -5 & 2 & 4 & 0 & -3 & 1 \\ -8 & -6 & 3 & 1 & 7 & -4 \end{array}\right].$$

Definition 7.14. (Block (or partitioned) matrices)

When we partition a matrix as shown below

$$A = \left[\begin{array}{c|c|c|c} A_{11} & A_{12} & \cdots & A_{1n} \\ \hline A_{21} & A_{22} & \cdots & A_{2n} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline A_{m1} & A_{m2} & \cdots & A_{mn} \end{array}\right],$$

we refer to A as an $m \times n$ **block matrix** and call the submatrices A_{ij} its **block entries**. Moreover, for $i = 1, \ldots, m$ and $j = 1, \ldots, n$, we call the $1 \times n$ and $m \times 1$ block matrices

$$\left[\begin{array}{c|c|c|c} A_{i1} & A_{i2} & \cdots & A_{in} \end{array}\right] \qquad \text{and} \qquad \left[\begin{array}{c} A_{1j} \\ \hline A_{2j} \\ \hline \vdots \\ \hline A_{mj} \end{array}\right]$$

the ith **block row** and the jth **block column** of the block matrix A, respectively.

Note that all block entries in the same block row must have the same number of rows, and all block entries in the same block column must have the same number of columns.

We often skip the partition lines in block matrices, letting the context clarify whether A_{ij} is a block (matrix) or an entry (scalar).

7.2.1 Addition, scalar multiplication, and transpose of block matrices

In the next two theorems, we show that block matrices can be manipulated just like regular matrices. Problem 7.8 asks you to verify these results.

If two matrices A and B have the same dimensions and are partitioned identically, we can apply that partition to their sum $A + B$. Each block of $A + B$ is then the sum of the corresponding blocks of A and B. Similarly, we perform scalar multiplication block by block. The following theorem summarizes these facts.

Theorem 7.15. (Addition and scalar multiplication of block matrices)

Consider the following $m \times n$ block matrices

$$A = \left[\begin{array}{c|c|c|c} A_{11} & A_{12} & \cdots & A_{1n} \\ \hline A_{21} & A_{22} & \cdots & A_{2n} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline A_{m1} & A_{m2} & \cdots & A_{mn} \end{array}\right] \quad \text{and} \quad B = \left[\begin{array}{c|c|c|c} B_{11} & B_{12} & \cdots & B_{1n} \\ \hline B_{21} & B_{22} & \cdots & B_{2n} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline B_{m1} & B_{m2} & \cdots & B_{mn} \end{array}\right].$$

If the block entries A_{ij} and B_{ij} have the same size, for all $i = 1, \ldots, m$ and $j = 1, \ldots, n$, then

$$A + B = \left[\begin{array}{c|c|c|c} A_{11} + B_{11} & A_{12} + B_{12} & \cdots & A_{1n} + B_{1n} \\ \hline A_{21} + B_{21} & A_{22} + B_{22} & \cdots & A_{2n} + B_{2n} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline A_{m1} + B_{m1} & A_{m2} + B_{m2} & \cdots & A_{mn} + B_{mn} \end{array}\right].$$

If $\alpha \in \mathbb{R}$ is a scalar, then

$$\alpha A = \left[\begin{array}{c|c|c|c} \alpha A_{11} & \alpha A_{12} & \cdots & \alpha A_{1n} \\ \hline \alpha A_{21} & \alpha A_{22} & \cdots & \alpha A_{2n} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline \alpha A_{m1} & \alpha A_{m2} & \cdots & \alpha A_{mn} \end{array}\right].$$

We can also express the transpose of a block matrix in terms of its block entries.

Theorem 7.16. (The transpose of a block matrix)

Consider the following $m \times n$ block matrix

$$A = \left[\begin{array}{c|c|c} A_{11} & \cdots & A_{1n} \\ \hline A_{21} & \cdots & A_{2n} \\ \hline \vdots & \ddots & \vdots \\ \hline A_{m1} & \cdots & A_{mn} \end{array}\right].$$

The transpose of A is the $n \times m$ block matrix

$$A^T = \left[\begin{array}{c|c|c} A_{11} & \cdots & A_{1n} \\ \hline A_{21} & \cdots & A_{2n} \\ \hline \vdots & \ddots & \vdots \\ \hline A_{m1} & \cdots & A_{mn} \end{array}\right]^T = \left[\begin{array}{c|c|c|c} A_{11}^T & A_{21}^T & \cdots & A_{m1}^T \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline A_{1n}^T & A_{2n}^T & \cdots & A_{mn}^T \end{array}\right].$$

Example 7.17 The transpose of the 2×2 block matrix

$$A = \left[\begin{array}{ccc|cc} 2 & -3 & 1 & 0 & -4 \\ 1 & 5 & -2 & 3 & -1 \\ \hline -1 & -4 & -2 & 7 & -1 \end{array}\right] =: \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right]$$

is given by

$$A^T = \left[\begin{array}{c|c} A_{11}^T & A_{21}^T \\ \hline A_{12}^T & A_{22}^T \end{array}\right] = \left[\begin{array}{cc|c} 2 & 1 & -1 \\ -3 & 5 & -4 \\ 1 & -2 & -2 \\ \hline 0 & 3 & 7 \\ -4 & -1 & -1 \end{array}\right].$$

7.2.2 Multiplication of block matrices

Recall that two matrices must be *conformable* to multiply: the number of columns of A must equal the number of rows of B for AB to be defined. A similar rule applies to block matrices. Using the usual "rows-times-columns" rule, now treating whole blocks as single entries, we can form AB when the number of block columns in A matches the number of block rows in B, and each block pair (A_{ik}, B_{kj}) is itself conformable. When these conditions hold, we say that A and B are **conformable for block multiplication**.

Theorem 7.18. (Block matrix multiplication)

Let A and B be the $m \times n$ and $n \times p$ block matrices

$$A = \left[\begin{array}{c|c|c|c} A_{11} & A_{12} & \cdots & A_{1n} \\ \hline A_{21} & A_{22} & \cdots & A_{2n} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline A_{m1} & A_{m2} & \cdots & A_{mn} \end{array}\right] \quad \text{and} \quad B = \left[\begin{array}{c|c|c|c} B_{11} & B_{12} & \cdots & B_{1p} \\ \hline B_{21} & B_{22} & \cdots & B_{2p} \\ \hline \vdots & \vdots & \ddots & \vdots \\ \hline B_{n1} & B_{n2} & \cdots & B_{np} \end{array}\right].$$

If A and B are conformable for block multiplication, then the matrix AB is an $m \times p$ block matrix whose (i, j)th block entry is given by the product of the ith block row of A and the jth block column of B, that is,

$$(AB)_{ij} = \begin{bmatrix} A_{i1} & A_{i2} & \cdots & A_{in} \end{bmatrix} \begin{bmatrix} B_{1j} \\ B_{2j} \\ \vdots \\ B_{nj} \end{bmatrix} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

Proof See Problem 7.18. □

Example 7.19. (2×2 block matrix multiplication)

Let us consider the 2×2 block matrices

$$A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right] \qquad \text{and} \qquad B = \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array}\right],$$

and assume they are conformable for block matrix multiplication. The product $C = AB$ is the block matrix

$$C = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array}\right] \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array}\right] = \left[\begin{array}{c|c} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ \hline A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{array}\right].$$

For a numerical example, let us partition the matrices A and B from Example 6.10 as follows

$$A = \left[\begin{array}{cc|c} 1 & 0 & -1 \\ 2 & 1 & 0 \\ \hline 0 & -2 & 1 \end{array}\right] \quad \text{and} \quad B = \left[\begin{array}{c|c} 0 & 1 \\ 2 & 3 \\ \hline 4 & 5 \end{array}\right].$$

We can compute their product as the block matrix

$$AB = \left[\begin{array}{c|c} \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 0 \\ 2 \end{bmatrix} + \begin{bmatrix} -1 \\ 0 \end{bmatrix}\begin{bmatrix} 4 \end{bmatrix} & \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 3 \end{bmatrix} + \begin{bmatrix} -1 \\ 0 \end{bmatrix}\begin{bmatrix} 5 \end{bmatrix} \\ \hline \begin{bmatrix} 0 & -2 \end{bmatrix}\begin{bmatrix} 0 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \end{bmatrix}\begin{bmatrix} 4 \end{bmatrix} & \begin{bmatrix} 0 & -2 \end{bmatrix}\begin{bmatrix} 1 \\ 3 \end{bmatrix} + \begin{bmatrix} 1 \end{bmatrix}\begin{bmatrix} 5 \end{bmatrix} \end{array}\right] = \left[\begin{array}{c|c} -4 & -4 \\ 2 & 5 \\ \hline 0 & -1 \end{array}\right],$$

which matches the result we got from standard matrix multiplication in (6.2).

The four perspectives on matrix multiplication that we have studied are special cases of the general block matrix multiplication formula in Theorem 7.18.

7.2.3 Inverse of block matrices

Can we determine if a square block matrix A is invertible based on whether its block entries are invertible? In general, the answer to this question is not straightforward. However, for certain block matrix types, we can obtain some conclusions.

The results that follow extend Examples 6.21 and 6.22 to the block matrix case.

Example 7.20. (The inverse of a block diagonal matrix)

The block diagonal matrix

$$A = \begin{bmatrix} D_1 & 0 & \cdots & 0 \\ 0 & D_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & D_k \end{bmatrix},$$

with square block diagonal entries D_i is invertible if and only if every diagonal block D_i, for $i = 1, 2, \ldots, k$, is invertible. In the former case, its inverse is the block diagonal matrix

$$A^{-1} = \begin{bmatrix} D_1^{-1} & 0 & \cdots & 0 \\ 0 & D_2^{-1} & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & D_k^{-1} \end{bmatrix}.$$

Example 7.21. (Invertible block triangular matrices)

The upper and lower $k \times k$ block triangular matrices

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1k} \\ 0 & A_{22} & \cdots & A_{2k} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & A_{kk} \end{bmatrix}, \quad \text{and} \quad A = \begin{bmatrix} A_{11} & 0 & \cdots & 0 \\ A_{21} & A_{22} & \ddots & \vdots \\ \vdots & \vdots & \ddots & 0 \\ A_{k1} & A_{k2} & \cdots & A_{kk} \end{bmatrix},$$

where each A_{ii} is a square matrix, for $i = 1, \ldots, k$, are invertible if and only if all diagonal blocks A_{ii} are invertible. This result can be proven by induction on k.

Although it is easy to characterize the nonsingular block triangular matrices, computing their inverse is not so trivial. For a 2×2 block triangular matrix, we have an explicit inverse formula. Example 7.22 extends the (6.5) formulas to 2×2 block triangular matrices.

Example 7.22. (The inverse of a 2×2 block triangular matrix)

Given matrices $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times m}$, $C \in \mathbb{R}^{m\times n}$, and $D \in \mathbb{R}^{m\times m}$, if A and D are invertible, then

$$\begin{bmatrix} A & B \\ 0 & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & -A^{-1}BD^{-1} \\ 0 & D^{-1} \end{bmatrix}, \quad \text{and}$$

$$\begin{bmatrix} A & 0 \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & 0 \\ -D^{-1}CA^{-1} & D^{-1} \end{bmatrix}.$$

What about finding the inverse of a general 2×2 block matrix $\left[\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right]$, where $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times m}$, $C \in \mathbb{R}^{m\times n}$ and $D \in \mathbb{R}^{m\times m}$? Can we determine whether this matrix is invertible based on its blocks? See Problems 16.30 and 16.31 for some results.

Problems

7.1 Consider the following linear system

$$\begin{aligned} 4x_1 + \ \ x_2 &= 9 \\ 2x_1 - \ \ x_2 &= 3 \\ 5x_1 - 3x_2 &= 7 \end{aligned}$$

What are its row, column, and linear transformation pictures?

7.2 The following linear system

$$\begin{aligned} x_1 + 2x_2 &= 4 \\ 2x_1 + 4x_2 &= 8 \end{aligned}$$

has infinitely many solutions. How do you interpret this from the row, column, and linear transformation pictures point of view?

7.3 Show that if for two $m \times n$ matrices A and B, we have $Ax = Bx$ for every $x \in \mathbb{R}^n$, then A equals B.

7.4 Compute the product of the following two matrices in four different ways: the standard rows-times-columns method from Chapter 6 and the three methods introduced in Section 7.1.3.

$$A = \begin{bmatrix} 1 & 2 \\ 2 & -3 \\ 0 & 1 \end{bmatrix}, \quad \text{and} \quad B = \begin{bmatrix} 1 & 0 & 1 \\ 2 & -1 & 0 \end{bmatrix}.$$

7.5 Show that for two matrices A and B, where the product AB is defined, each column of AB is a linear combination of the columns of A. Also, prove that every row of AB is a linear combination of the rows of B.

7.6 Consider three matrices A, B, and C. Determine the minimum information we need about these matrices (entries, rows, and columns) to compute the following:

(a) The last row of AB.
(b) The first column of AB.
(c) The entry located at position $(2, 3)$ in AB.
(d) The second row of ABC.
(e) The last column of ABC.
(f) The entry situated at position $(1, 5)$ in ABC.

7.7 Describe how the following changes to the matrices A and B influence the matrix product AB.

(a) Swap the first and last rows of A.
(b) Make the second column of B all zeros.
(c) Replace row 1 of A with the sum of rows 2 and 3.
(d) Multiply row 3 of A by a scalar α.

(e) Add a row of zeros to A between rows 1 and 2.
(f) Append a column of zeros to B as its last column.

7.8 Prove Theorems 7.15 and 7.16.

7.9 For the partitioned matrix $X = \begin{bmatrix} X_1 & X_2 \end{bmatrix}$, write $X^T X$ and XX^T in terms of X_1 and X_2.

7.10 (To be solved in a computer) Suppose A is a 20×30 matrix. Write code that accomplishes the following tasks.

(a) Display the submatrix of A from rows 15 to 20 and columns 5 to 10.
(b) Insert a 5×10 matrix B into A, beginning at row 10 and column 20.
(c) Create a 50×50 matrix of the form

$$B = \begin{bmatrix} A & 0 \\ 0 & A^T \end{bmatrix}.$$

7.11 (To be solved in a computer) Suppose memory or size restrictions prevent your computer from working with matrices having more than 32 rows and 32 columns, and suppose some project involves 50×50 matrices A and B. Describe the code you would write to accomplish the following tasks:

(a) Compute $A + B$.
(b) Compute AB.
(c) Solve $Ax = b$ for some vector b of length 50, assuming that A can be partitioned into a 2×2 block matrix $[A_{ij}]$ with A_{11} an invertible 20×20 matrix, A_{22} an invertible 30×30 matrix, and A_{12} a zero matrix.

Hint: Describe appropriate smaller systems to solve, without using any matrix inverses.

7.12 Given all blocks are $n \times n$ matrices, compute the following using block multiplication.

(a) $\begin{bmatrix} I_n & -A \\ B & I_n \end{bmatrix} \begin{bmatrix} A & I_n \\ I_n & 0 \end{bmatrix}$.

(b) $\begin{bmatrix} -I_n & 0 \\ A & I_n \end{bmatrix}^n$, for $n = 1, 2, \ldots$.

(c) $\begin{bmatrix} 0 & I_n \\ A & 0 \end{bmatrix}^n$, for $n = 1, 2, \ldots$.

7.13 Use partitioned matrices to prove, by induction, that the product of two lower triangular matrices A and B is also lower triangular. What are the diagonal entries of AB?

7.14 Determine if the columns-times-rows definition of matrix multiplication can be extended to block matrices. Prove or give a counterexample.

7.15 Assuming that the following matrices are partitioned conformably for block multiplication, compute the product

$$\begin{bmatrix} I_n & 0 \\ -X & I_m \end{bmatrix} \begin{bmatrix} A & B \\ C & D \end{bmatrix}.$$

7.16 Find formulas for X, Y and Z in terms of A, B. Justify your calculations. If necessary, you may need to make assumptions about the size of a matrix in order to produce a formula.

$$\begin{bmatrix} A & B \\ 0 & I \end{bmatrix} \begin{bmatrix} X & Y & Z \\ 0 & 0 & I \end{bmatrix} = \begin{bmatrix} I & 0 & 0 \\ 0 & 0 & I \end{bmatrix}.$$

7.17 Is the following statement true or false. Justify your answer. "The definition of matrix-vector product Ax is a special case of block multiplication".

7.18 (Block multiplication formula) This problem guides you in proving the block-matrix multiplication formula of Theorem 7.18. First we will establish the identity in a few simpler cases, then use those results to derive the general formula.

Assuming that the matrices below are partitioned conformably for block multiplication, prove the following formulas:

(a) $$A \left[\, B_1 \middle| B_2 \middle| \cdots \middle| B_p \,\right] = \left[\, AB_1 \middle| AB_2 \middle| \cdots \middle| AB_p \,\right],$$

(b) $$\begin{bmatrix} A_1 \\ \hline A_2 \\ \hline \vdots \\ \hline A_m \end{bmatrix} B = \begin{bmatrix} A_1B \\ \hline A_2B \\ \hline \vdots \\ \hline A_mB \end{bmatrix}, \quad \text{and}$$

(c) $$\left[\, A_1 \middle| A_2 \middle| \cdots \middle| A_n \,\right] \begin{bmatrix} B_1 \\ \hline B_2 \\ \hline \vdots \\ \hline B_n \end{bmatrix} = A_1B_1 + A_2B_2 + \cdots + A_nB_n.$$

After establishing parts (a), (b), and (c), use these results to prove the general formula in Theorem 7.18.

7.19 NASA launched a new communication satellite recently. In order to keep it in the right orbit, corrections to its trajectory may be necessary several times a day. NASA receives information about the position of the satellite and compares it with what should be the correct position. Let $X_k = \begin{bmatrix} x_1 & \cdots & x_k \end{bmatrix}$ be a matrix with k columns. Each vector x_i gives information about the real position of the satellite compared with the ideal position at time i. For the different necessary calculations, the matrix $H_k = X_k X_k^T$ is required. The problem is that when NASA collects new information, say x_{k+1}, a new matrix H_{k+1} must be computed. When many data vectors arrive the same day, the calculation of the new matrix requires a lot of computational effort. Show how matrix multiplication of partitioned matrices can help in this task by using the columns-times-rows expansion (Proposition 7.9) of H_k and H_{k+1}. What must be computed to obtain H_{k+1} from H_k?

7.20 Find the inverse of

$$A = \begin{bmatrix} 1 & 2 & 0 & 0 & 0 \\ 3 & 5 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 7 & 8 \\ 0 & 0 & 0 & 5 & 6 \end{bmatrix}$$

using an adequate partitioning of A.

7.21 Find the inverse (if it exists) of the block matrix

$$A = \begin{bmatrix} 0 & I_n \\ I_m & Y \end{bmatrix}.$$

7.22 Prove that if the block matrix

$$\begin{bmatrix} A & B \\ 0 & D \end{bmatrix},$$

where A and D are square matrices, is invertible, then both A and D must be invertible.

7.23 Consider the matrix equation

$$\begin{bmatrix} A & B \\ 0 & D \end{bmatrix} \begin{bmatrix} Z & 0 \\ X & Y \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ I & 0 \end{bmatrix},$$

where all block entries A, B, D, X, Y, Z are $n \times n$ matrices. Assume that the block matrix

$$\begin{bmatrix} A & B \\ 0 & D \end{bmatrix}$$

is invertible. Find formulas for X, Y, Z.

7.24 Use partitioned matrices to prove by induction that the inverse of an upper triangular matrix A is also upper triangular. What are the diagonal entries of A^{-1}?

Chapter 8
II-SUPPLEMENTARY TOPICS

Although most Linear Algebra textbooks usually focus on the standard matrix multiplication, there exist other alternative ways of multiplying matrices that are very useful in applications.

The Hadamard and Kronecker products are two unconventional methods of matrix multiplication, each with various practical uses.

8.1 The Hadamard matrix product

You may have considered entrywise multiplication as the intuitive way to multiply matrices. That is exactly what the Hadamard product accomplishes.

Definition 8.1. (The Hadamard product)

Let A and B be two $m \times n$ matrices. The Hadamard product of A and B, denoted by $A \odot B$, is the entrywise product of A and B. In other words $(A \odot B)_{ij} = A_{ij}B_{ij}$.

Note that for $A \odot B$ to be defined, both A and B must be of the same size, although they do not necessarily need to be square matrices.

Example 8.2 Consider the matrices

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \end{bmatrix} \qquad \text{and} \qquad B = \begin{bmatrix} 3 & 2 & 1 \\ 6 & 5 & 4 \end{bmatrix}.$$

Then, the Hadamard product of A and B is

$$A \odot B = \begin{bmatrix} 1\cdot 3 & 2\cdot 2 & 3\cdot 1 \\ 3\cdot 6 & 2\cdot 5 & 1\cdot 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 & 3 \\ 18 & 10 & 4 \end{bmatrix}.$$

The Hadamard product shares several algebraic properties with standard matrix multiplication, including associativity and distributivity. In addition, it has the prop-

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_8

erty of commutativity; that is, $A \odot B = B \odot A$. There is also an identity element for Hadamard multiplication: the matrix whose entries are all ones. However, as with standard matrix multiplication, not all matrices have a multiplicative inverse under this operation. In particular, any matrix with at least one zero entry is not invertible with respect to the Hadamard product.

The Hadamard product equals standard matrix multiplication when the matrices are diagonal, that is, $A \odot B = AB$ if both A and B are diagonal but the converse is not true (see Problem 8.2). Proposition 8.3 further illustrates the interaction between diagonal matrices and both standard and Hadamard products. We ask you to prove this result in Problem 8.3.

Proposition 8.3. (The Hadamard product and diagonal matrices)

Consider two matrices $A, B \in \mathbb{R}^{m\times n}$ and diagonal matrices $D_1 \in \mathbb{R}^{m\times m}$ and $D_2 \in \mathbb{R}^{n\times n}$. Then,

$$\begin{aligned} D_1(A \odot B)D_2 &= (D_1 A D_2) \odot B \\ &= A \odot (D_1 B D_2) \\ &= (AD_2) \odot (D_1 B) \\ &= (D_1 A) \odot (B D_2). \end{aligned}$$

8.2 The Kronecker matrix product

The Kronecker product is an operation on two matrices of any size, resulting in a block matrix.

Definition 8.4. (The Kronecker product)

The Kronecker product of $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{p\times q}$, denoted by $A \otimes B$, is the $m \times n$ block matrix

$$A \otimes B = \left[\begin{array}{c|c|c} A_{11}B & \cdots & A_{1n}B \\ \hline \vdots & \ddots & \vdots \\ \hline A_{m1}B & \cdots & A_{mn}B \end{array}\right] \in \mathbb{R}^{mp\times nq}.$$

Example 8.5 Consider the matrices

$$A = \begin{bmatrix} 1 & 2 \\ 0 & -3 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

Then,

$$A\otimes B = \left[\begin{array}{c|c} 1\cdot\begin{bmatrix}1\\-1\\0\end{bmatrix} & 2\cdot\begin{bmatrix}1\\-1\\0\end{bmatrix} \\ \hline 0\cdot\begin{bmatrix}1\\-1\\0\end{bmatrix} & -3\cdot\begin{bmatrix}1\\-1\\0\end{bmatrix}\end{array}\right] = \left[\begin{array}{cc|cc} 1 & 2 & \multicolumn{2}{c}{\multirow{3}{*}{$\begin{matrix}0&-3\\0&3\\0&0\end{matrix}$}} \\ -1 & -2 & & \\ 0 & 0 & & \end{array}\right] \quad \text{and} \quad B\otimes A = \left[\begin{array}{c} 1\cdot\begin{bmatrix}1&2\\0&-3\end{bmatrix} \\ \hline -1\cdot\begin{bmatrix}1&2\\0&-3\end{bmatrix} \\ \hline 0\cdot\begin{bmatrix}1&2\\0&-3\end{bmatrix}\end{array}\right] = \left[\begin{array}{cc} 1 & 2 \\ 0 & -3 \\ \hline -1 & -2 \\ 0 & 3 \\ \hline 0 & 0 \\ 0 & 0 \end{array}\right].$$

The Kronecker product is associative and distributive with respect to addition (Problem 8.7). Like standard matrix multiplication, the Kronecker product is not commutative. That is, generally, $A \otimes B$ does not equal $B \otimes A$ (the previous example illustrates this claim). Nevertheless, all possible products of entries in A with entries in B occur in both $A \otimes B$ and $B \otimes A$, but in different positions. In the example, note that the rows of $B \otimes A$ are the same as those of $A \otimes B$ but in a different order. In fact, for any matrices A and B, there exist permutation matrices P and Q such that $B \otimes A = P(A \otimes B)Q$ (see Problem 8.8). In the example, $Q = I_2$ and

$$P = \begin{bmatrix} 1&0&0&0&0&0 \\ 0&0&0&1&0&0 \\ 0&1&0&0&0&0 \\ 0&0&0&0&1&0 \\ 0&0&1&0&0&0 \\ 0&0&0&0&0&1 \end{bmatrix}.$$

The following proposition shows how the Kronecker product interacts with standard and Hadamard matrix products.

Proposition 8.6. (The mixed-product property)

Consider matrices A, B, C and D such that the products AC and BD are defined. Then,

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD).$$

Moreover, if A and C have the same size, and B and D have the same size, then

$$(A \otimes B) \odot (C \otimes D) = (A \odot C) \otimes (B \odot D).$$

Problem 8.12 asks you to prove Proposition 8.6, and Problem 8.13 shows two ways in which Proposition 8.6 can be generalized.

Problems

8.1 Prove that, for all $A, B, C \in \mathbb{R}^{m\times n}$ and any scalar $\alpha \in \mathbb{R}$, the Hadamard product satisfies the following properties.

(a) $A \odot B = B \odot A$,
(b) $A \odot (B \odot C) = (A \odot B) \odot C$,
(c) $A \odot (B + C) = A \odot B + A \odot C$, and
(d) $(\alpha A) \odot B = A \odot (\alpha B) = \alpha(A \odot B)$.

8.2 Show that if A and B are both diagonal matrices, then $A \odot B = AB$. Then give an example of two non-diagonal matrices A and B for which $A \odot B = AB$.

8.3 Prove Proposition 8.3.

8.4 Consider a matrix $A \in \mathbb{R}^{n\times n}$ and a column vector $x \in \mathbb{R}^n$, and let $D = \text{diag}(x)$ be the diagonal matrix with the vector x as its main diagonal. Show that $A \odot (xx^T) = DAD$.

8.5 Show that $I_n \otimes I_m = I_{nm}$, that is, the Kronecker product of two identity matrices is an identity matrix as well.

8.6 Show that if $A \otimes B = 0$, then either $A = 0$ or $B = 0$.

8.7 Prove that, for all matrices A, B, C of appropriate size, and scalar α, the Kronecker product satisfies the following properties.

(a) $A \otimes (B + C) = A \otimes B + A \otimes C$.
(b) $(B + C) \otimes A = B \otimes A + C \otimes A$.
(c) $(\alpha A) \otimes B = A \otimes (\alpha B) = \alpha(A \otimes B)$.
(d) $(A \otimes B) \otimes C = A \otimes (B \otimes C)$.

8.8 Consider the 2×2 matrices $A = \left[\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right]$ and $B = \left[\begin{smallmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{smallmatrix}\right]$. Find permutation matrices P and Q such that $B \otimes A = P(A \otimes B)Q$.

Note: this result can be generalized to arbitrary matrices A and B. For any matrices A and B, there exist permutation matrices P and Q such that $B \otimes A = P(A \otimes B)Q$. The matrices P and Q belong to the class of matrices known as *perfect-shuffle permutations* and they only depend on the size of A and B and not on their particular entries.

8.9 If two matrices, A and B, of the same size have the same entries but arranged differently, can you always find permutation matrices P and Q such that $A = PBQ$?

8.10 Show that $(A \otimes B)^T = A^T \otimes B^T$. So transposing does not reverse the order of the Kronecker product, unlike standard matrix multiplication.

8.11 Show that if A and B are invertible matrices, then $A \otimes B$ is also invertible and $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$.

8.12 Prove Proposition 8.6.

8.13 Using the mixed-product property of the Kronecker product, prove by induction the following equalities

$$(A_1 \otimes A_2 \otimes \cdots \otimes A_n)(B_1 \otimes B_2 \otimes \cdots \otimes B_n) = (A_1B_1) \otimes (A_2B_2) \otimes \cdots \otimes (A_nB_n)$$

and

$$(A_1 \otimes B_1)(A_2 \otimes B_2) \cdots (A_n \otimes B_n) = (A_1A_2 \cdots A_n) \otimes (B_1B_2 \cdots B_n).$$

8.14 For a square matrix A, show that $(A \otimes I_n)^k = A^k \otimes I_n$ and $(I_n \otimes A)^k = I_n \otimes A^k$.

Chapter 9
II-FROM THE CLASSROOM TO REAL LIFE

In this chapter, we illustrate how matrix multiplication arises in Data Science applications. All examples focus on standard matrix multiplication. Hadamard and Kronecker products are also important but require additional background, so we do not cover them here; instead, we briefly note where each is used.

Hadamard matrix products in applications

The Hadamard product is used in compression algorithms such as JPEG, in image processing for enhancing, suppressing, or masking image regions, and in Machine Learning. For example, it describes the architecture of recurrent neural networks such as gated recurrence units (GRUs) or long short-term memory units (LSTMs), and plays a role in the gradient descent method for training neural networks.

Kronecker matrix products in applications

The Kronecker product is useful in solving some modeling problems in two- and three-dimensional settings. It lets us rewrite certain image-processing operations as large matrix–vector products and turns matrix equations arising from partial-differential-equation models into standard linear systems.

For applications of the standard matrix product, all upcoming examples will center on graphs. You can revisit the key definitions and concepts related to graphs in Chapter 3 for a quick refresher.

9.1 The number of walks of a given length in a graph

A walk is a fundamental concept in Graph Theory. When we start at a node and move to another node along an edge, we are starting a walk in the graph. In directed graphs, we must walk following the direction of the edges.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

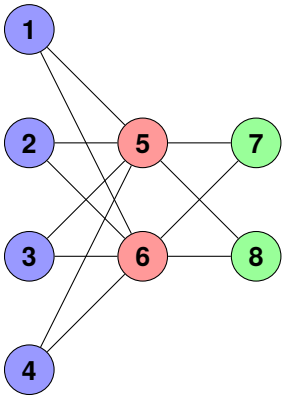
M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_9

Definition 9.1. (Walks of length k and paths)

In an unweighted graph (directed or undirected), we define a **walk of length** k as a sequence of k edges that connect consecutive nodes. This sequence can include repeated nodes and edges. In particular, the starting and end point of the walk can be the same node. If a walk does not use the same vertex more than once, then we call it a **path**.

Example 9.2. (Walks in a graph)

Consider the undirected graph below and its corresponding adjacency matrix A, which we have partitioned according to the color of the nodes.

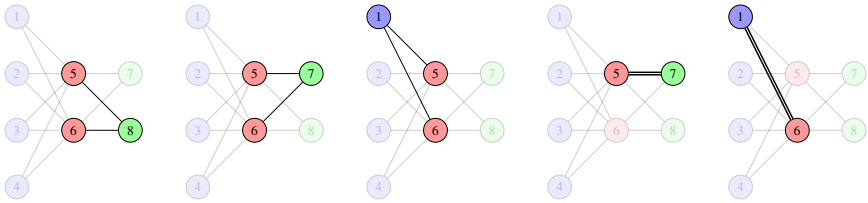


$$A = \left[\begin{array}{cccc|cc|cc} 0&0&0&0&1&1&0&0 \\ 0&0&0&0&1&1&0&0 \\ 0&0&0&0&1&1&0&0 \\ 0&0&0&0&1&1&0&0 \\ \hline 1&1&1&1&0&0&1&1 \\ 1&1&1&1&0&0&1&1 \\ \hline 0&0&0&0&1&1&0&0 \\ 0&0&0&0&1&1&0&0 \end{array}\right].$$

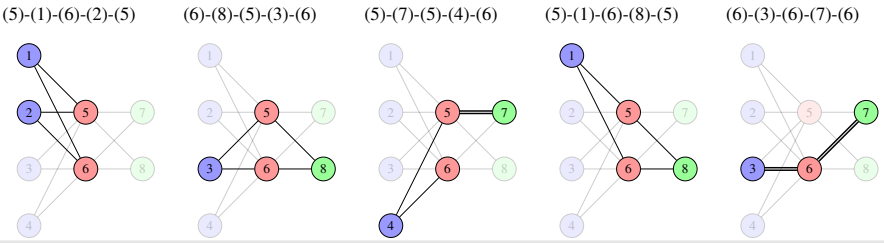
If we view this graph as a communication network, where each edge is a road and each node is a city, for example, we can think of different ways to connect cities. Each walk in the graph then becomes a unique route for traveling from one city to another. An example of a walk of length 3 would be given by the sequence of edges $(1)-(6), (6)-(8), (8)-(5)$. We can simply represent this path as $(1)-(6)-(8)-(5)$.

Consider the problem of counting walks of positive length at most 4 that start and end at a red node. It is evident that no walks of length 1 exist. There are numerous walks of length 2, with some examples provided below.

(5)-(8)-(6) (5)-(7)-(6) (5)-(1)-(6) (5)-(7)-(5) (6)-(1)-(6)



No walks of length 3 exist, but there are many walks of length 4, as some examples shown below illustrate.



Counting walks by hand is tedious. The next theorem shows that the adjacency matrix A of an unweighted graph and its powers A^k contain the information we need.

Theorem 9.3. (Powers of the adjacency matrix and number of walks)

Let A be the adjacency matrix of an unweighted graph. The (i, j)th entry of A^k counts the number of walks from node i to node j of length k.

Proof We will prove the result by induction on the exponent k. The case $k = 1$ is clear, since by construction, $A_{ij} = 1$ if there is an edge from node i to node j, and $A_{ij} = 0$ otherwise. Thus, the entries of A count walks of length 1.

Assume the result is true for $k - 1$. Notice that we can express the number of walks of length k between nodes i and j as the number of walks of length $k - 1$ whose end point is connected with the jth node:

$$\sum_{t \text{ such that } A_{tj}=1} \text{number of length-}(k-1)\text{ walks between nodes } i \text{ and } t. \tag{9.1}$$

By the induction hypothesis, we have that, for any node t, the number of length-$(k - 1)$ walks from node i to node t equals $(A^{k-1})_{it}$, and the number of walks of length one from node t to node j is A_{tj}. Hence, (9.1) becomes

$$\sum_t (A^{k-1})_{it} A_{tj} = (A^k)_{ij},$$

as we wanted to show. □

Let us consider again the problem of counting the number of walks of length at most 4 that start and end at a red node in Example 9.2. We need to count the walks that start at node 5 and end at 5 or 6, and the walks that start at node 6 and end at 5 or 6. Theorem 9.3 says that we can determine the number of these walks simply by computing the kth powers, with $k = 1, 2, 3, 4$, of the adjacency matrix A. These powers are

$$A^2 = \left[\begin{array}{cccc|cc|cc} 2&2&2&2&0&0&2&2\\ 2&2&2&2&0&0&2&2\\ 2&2&2&2&0&0&2&2\\ 2&2&2&2&0&0&2&2\\ \hline 0&0&0&0&6&6&0&0\\ 0&0&0&0&6&6&0&0\\ \hline 2&2&2&2&0&0&2&2\\ 2&2&2&2&0&0&2&2 \end{array}\right], \quad A^3 = \left[\begin{array}{cccc|cc|cc} 0&0&0&0&12&12&0&0\\ 0&0&0&0&12&12&0&0\\ 0&0&0&0&12&12&0&0\\ 0&0&0&0&12&12&0&0\\ \hline 12&12&12&12&0&0&12&12\\ 12&12&12&12&0&0&12&12\\ \hline 0&0&0&0&12&12&0&0\\ 0&0&0&0&12&12&0&0 \end{array}\right], \quad \text{and}$$

$$A^4 = \left[\begin{array}{cccc|cc|cc} 24&24&24&24&0&0&24&24\\ 24&24&24&24&0&0&24&24\\ 24&24&24&24&0&0&24&24\\ 24&24&24&24&0&0&24&24\\ \hline 0&0&0&0&72&72&0&0\\ 0&0&0&0&72&72&0&0\\ \hline 24&24&24&24&0&0&24&24\\ 24&24&24&24&0&0&24&24 \end{array}\right].$$

To count the number of walks of length at most 4 between red nodes, we just add the entries of the adjacency matrix A, and its powers A^2, A^3 and A^4 in the block entry at position $(2, 2)$. Doing so gives:

$$\begin{aligned} \text{number of walks of length at most 4} =& 0+0+0+0 && \text{(length-1 walks)}\\ &+6+6+6+6 && \text{(length-2 walks)}\\ &+0+0+0+0 && \text{(length-3 walks)}\\ &+72+72+72+72 && \text{(length-4 walks)}\\ =& 312 \end{aligned}$$

walks in total. Imagine counting 312 walks by hand!

Now we can revisit the example that motivated this chapter. We want to compute, for any natural number n, the number of authors with Erdős number n. Recall that the collaboration graph has a node for each mathematician, with an edge between two nodes exactly when those mathematicians coauthored a paper. The Erdős number of a mathematician is the length of the shortest walk from Paul Erdős's node to that mathematician's node. In other words, our goal is to count mathematicians, excluding Erdős himself, for whom there is a walk of length n from Erdős in the collaboration graph, but no walk of length less than n.

Below, we present one way to approach this problem. It may not be the most efficient method, but it shows how a few Linear Algebra tools can solve a seemingly complex problem.

The Erdős number revisited

Consider the mathematicians' collaboration graph, where nodes represent mathematicians, and edges indicate joint publications.

Let A be the adjacency matrix of the collaboration graph, and assume that Erdős corresponds to the first node. Let

$$e_1 = [1, 0, \ldots, 0]^T.$$

To determine the number of mathematicians with Erdős number r, proceed as follows.

1. Compute
$$s_{r-1} = Ae_1 + A^2e_1 + \cdots + A^{r-1}e_1,$$
where $s_0 = 0$ when $r = 1$.
2. Replace every nonzero entry of s_{r-1} by 1, and remove the first entry, corresponding to Erdős. Call the resulting vector w. Thus, w has a 1 precisely at the positions corresponding to mathematicians whose Erdős number is less than r.
3. Compute
$$s_r = s_{r-1} + A^r e_1.$$
Replace every nonzero entry of s_r by 1, and remove the first entry. Call the resulting vector u. Thus, u has a 1 precisely at the positions corresponding to mathematicians whose Erdős number is at most r.
4. Compute $u - w$. This vector has a 1 precisely at the positions corresponding to mathematicians whose Erdős number is exactly r. Therefore, the number of its entries equal to 1 is the number of mathematicians with Erdős number r.

9.2 The Katz centrality measure

Given a graph, we often want to identify its most "important" nodes. One way to do this is by assigning each node a numerical value, called its centrality score, and then ranking nodes by that score.

Different centrality measures capture different aspects of "importance". In this section, we focus on Katz centrality and briefly mention other measures at the end.

Katz centrality, introduced by Katz in 1953 [33] and later extended by Bonacich in 1972 and 1987 [10, 11], assigns a score to each node based on all walks leading to it. In essence, a node's Katz centrality is high if many other nodes can reach it via short walks.

Formally, Katz centrality for node i in an unweighted graph sums contributions from all walks ending at i, weighting each walk of length k by α^k, where $0 < \alpha < 1$. Immediate neighbors (walks of length 1) thus carry more weight than nodes connected by longer walks.

Definition 9.4. (Katz centrality)

Given an unweighted graph $G = (V, E)$, the Katz centrality of node i, denoted by c_i, is defined as

$$c_i = \sum_{k=1}^{\infty} \alpha^k \times (\text{number of length-}k \text{ walks that end at node } i)\,. \tag{9.2}$$

The parameter α (with $0 < \alpha < 1$) penalizes longer walks and must be chosen sufficiently small to ensure that the series converges (see Theorem 9.6). Each term α^k down-weights the contribution of length-k walks, so shorter walks contribute more to c_i.

We can express Katz centrality using powers of the adjacency matrix A of the graph, as we show in the following theorem.

Theorem 9.5. (Katz centrality formula - Part I)

Consider an unweighted graph $G = (V, E)$ with n nodes and an adjacency matrix $A \in \mathbb{R}^{n\times n}$. Let $c_{\text{Katz}} = \begin{bmatrix} c_1 & c_2 & \cdots & c_n \end{bmatrix}$ be a row vector, where c_i is the Katz centrality of node i as defined in (9.2). Then, we can express the Katz centrality vector c_{Katz} as

$$c_{\text{Katz}} = \sum_{k=1}^{\infty} e^T (\alpha A)^k, \tag{9.3}$$

where $e \in \mathbb{R}^n$ is the vector of all ones.

Proof From Theorem 9.3, the (i, j)th entry of A^k gives the number of length-k walks from node i to node j. Therefore, we can express the total number of length-k walks leading to node i as

$$\sum_{j=1}^{n} (A^k)_{ji} = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} (A^k)_{1i} \\ (A^k)_{2i} \\ \vdots \\ (A^k)_{ni} \end{bmatrix} = e^T A^k[:, i].$$

Consequently, the Katz centrality of node i is

$$c_i = \sum_{k=1}^{\infty} \alpha^k \sum_{j=1}^{n} (A^k)_{ji} = \sum_{k=1}^{\infty} \alpha^k e^T A^k[:, i] = \sum_{k=1}^{\infty} e^T (\alpha A)^k[:, i].$$

By expressing all node centrality values c_i using a single equation, we obtain (9.3).□

The formula for Katz centrality (9.3) closely resembles the scalar geometric series $\sum_{k=1}^{\infty} r^k$. In this analogy, the matrix αA plays the role of the common ratio r. Recall that, for $|r| < 1$,

$$\sum_{k=1}^{\infty} r^k = \sum_{k=0}^{\infty} r^k - 1 = \frac{1}{1-r} - 1.$$

Similarly, under an appropriate convergence condition, the matrix geometric series satisfies $\sum_{k=1}^{\infty} (\alpha A)^k = (I_n - \alpha A)^{-1} - I_n$. Multiplying this identity on the left by e^T gives a closed formula for the Katz centrality vector. The precise convergence

condition involves the spectral radius of αA, a concept that will be introduced in Chapter 52. We therefore state the following theorem without proof.

Theorem 9.6. (Katz centrality formula - Part II)

Consider an unweighted graph $G = (V, E)$ with n nodes and an adjacency matrix $A \in \mathbb{R}^{n \times n}$. Let $c_{\text{Katz}} = \begin{bmatrix} c_1 & c_2 & \cdots & c_n \end{bmatrix}$ be a row vector, where c_i is the Katz centrality of node i as defined in (9.2), and let $e \in \mathbb{R}^n$ be the vector of all ones. Then, we can express the Katz centrality vector c_{Katz} as

$$c_{\text{Katz}} = e^T \left((I_n - \alpha A)^{-1} - I_n \right), \tag{9.4}$$

provided that $\alpha > 0$ and $\alpha \rho(A) < 1$, where $\rho(A)$ is the so-called spectral radius of A. [1]

Example 9.7 Let us compute the Katz centrality for the nodes in the graph presented in Figure 9.1.

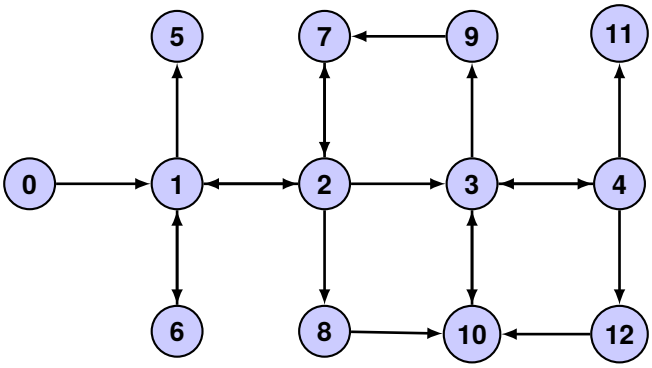


Fig. 9.1: Graph in Example 9.7.

Table 9.1 shows the Katz centrality values (using $\alpha = 0.1$) for all thirteen nodes in the graph. Node 0 has a score of zero since it is unreachable. Node 3 has the highest score, highlighting its central role in the graph.

[1] We will introduce the spectral radius in Chapter 52.

Node	Katz centrality	Node	Katz centrality
node 0	0	node 7	0.24
node 1	0.339	node 8	0.126
node 2	0.258	node 9	0.138
node 3	0.376	node 10	0.362
node 4	0.138	node 11	0.114
node 5	0.134	node 12	0.114
node 6	0.134		

Table 9.1: Katz centralities for the nodes in the graph in Figure 9.1.

Example 9.8. (US airport transportation network)

In this real-world example, we examine the Katz centrality values of nodes in the US airport transportation network. In this network, each node represents a US airport, and an edge connects two airports if there is a direct flight between them.

Figure 9.2 shows the airport network with nodes color-coded according to their Katz centrality values (using $\alpha = 0.01$). We have also listed the top-10 and bottom-10 airports by their Katz centrality scores in Table 9.2. As would intuitively be expected, the top-10 airports are mostly major US hubs, whereas the bottom-10 are smaller, regional airports.

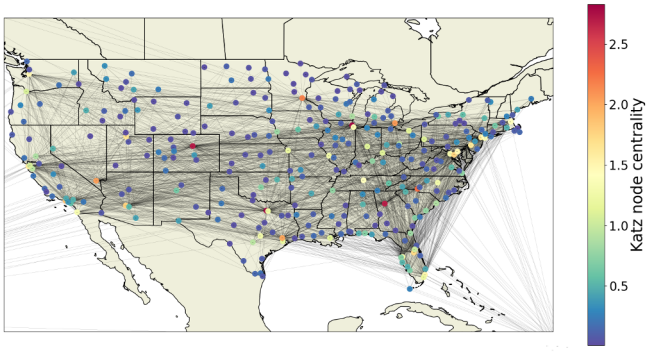


Fig. 9.2: US airport transportation graph. The nodes (airports) have been colored according to their Katz centrality.

Katz centrality is just one example of a node centrality measure. There are other centrality measures, and each measures node importance differently. Some of the other measures include:

- **Closeness centrality** evaluates how "quickly" a node can reach all other nodes in the graph by measuring its average distance to all other nodes.

Top-10		Bottom-10	
Airport name	**Katz centrality**	**Airport name**	**Katz centrality**
Dallas/Fort Worth International	2.83	Pago Pago International	0.02
Chicago O'Hare International	2.79	Bethel Airport	0.02
Denver International	2.70	King Salmon Airport	0.02
Hartsfield-Jackson Atlanta International	2.68	Dillingham Airport	0.02
Charlotte Douglas International	2.26	Kodiak Airport	0.02
Minneapolis-St Paul International	2.16	Adak	0.02
Detroit Metro Wayne County	2.06	Ogden-Hinckley	0.02
Harry Reid International Airport/Las Vegas	2.04	St. Cloud Regional	0.02
George Bush Intercontinental/Houston	2.00	Gustavus Airport	0.01
Los Angeles International	1.89	Saipan International	0.01

Table 9.2: Top- and bottom-10 Katz centralities for the US airport transportation network.

- **Betweenness centrality** measures how often a node appears on shortest paths between other nodes, identifying those that serve as "bridges" in the network.
- **Eigenvector centrality** assigns higher importance to nodes that are connected to other well-connected nodes, and is typically applied to undirected graphs.
- **PageRank centrality**, popularized by Google, extends the eigenvector-centrality idea to directed graphs.

Chapter 14 introduces Closeness centrality, and Chapter 54 introduces PageRank centrality.

Problems

9.1 Consider an $n \times n$ adjacency matrix A of an unweighted directed graph, and let e be the vector of all ones in $\mathbb{R}^n$. Describe what the entries of the vectors Ae and $e^T A$ represent. What do these entries tell us about the graph?

9.2 What is the computational cost, in terms of the number of additions/subtractions and multiplications/divisions, of the algorithm for counting mathematicians with Erdős number n described at the end of Section 9.1?

9.3 The graph in Figure 9.3 represents a road network connecting 13 cities (numbered 1–13).

There will be a truck driver strike and you need to transport your goods from city 8 to city 3. You want to have a list of all your options. For economical reasons, you would like to get from one city to the other passing by no more than 2 cities other than 8 and 3. How many such routes are there?

Note: you can use a computer to do the computations, but your solution should be supported by the results presented in this chapter.

9.4 The graph in Figure 9.4 represents the collaboration network of author 1, that is, the researchers that author 1 and his collaborators work with. Each node represents a researcher and an edge joins nodes i and j if authors i and j have published a paper together.

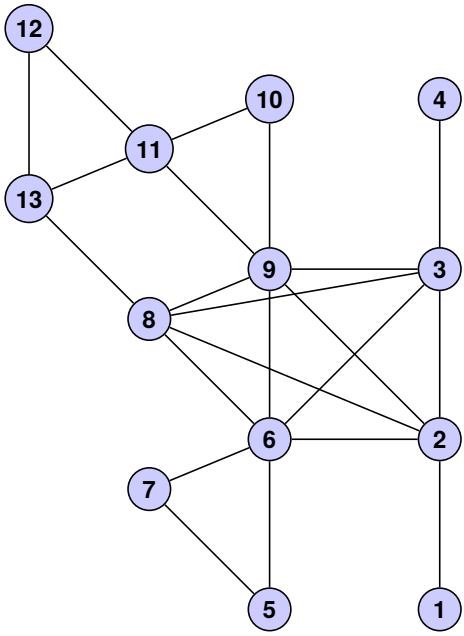


Fig. 9.3: Road network connecting 13 cities in Problem 9.3.

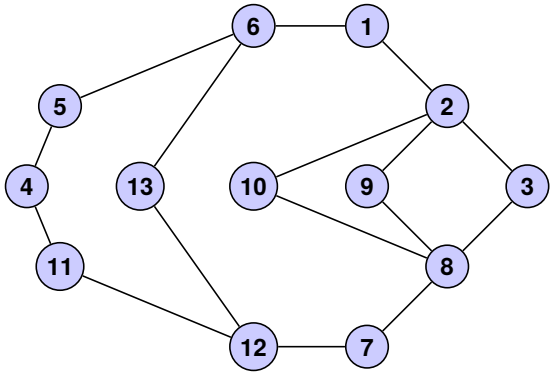


Fig. 9.4: Collaboration network in Problem 9.4.

(a) How many authors have written a paper with author 1?
(b) How many authors have written a paper with the direct collaborators of author 1?
(c) Assume that author 1 was Erdős. How many authors in this toy collaboration network have Erdős number 3?

9.5 Give an example of a directed graph with 5 nodes whose adjacency matrix A is nilpotent (i.e., $A^p = 0$ for some positive integer p). Compute the Katz centrality vector of this graph using $\alpha = 0.1$.

9.6 Consider the light-rail system shown in Figure 9.5, which has six stops and their connecting tracks.

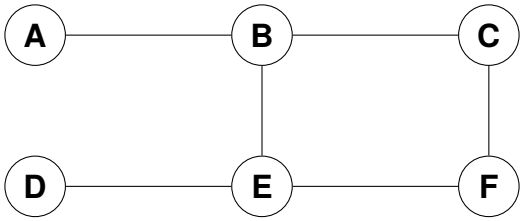


Fig. 9.5: Stops and tracks of the light-rail system in Problem 9.6.

(a) List every ordered pair (i, j) of distinct stops that can be reached with exactly one transfer (two legs). How many such origin-destination pairs are there?
(b) Let $B = A + A^2 + A^3$, where A denotes the adjacency matrix of the graph above. Explain the transportation meaning of a non-zero entry B_{ij}. Which pair of stops first becomes connected only when the third power A^3 is included?
(c) City planners consider adding a direct shuttle between D and A (an edge $D \leftrightarrow A$). Without recomputing the entire matrix A^2 from scratch, describe how you would update the number of one-transfer trips that *start* at D.

9.7 Consider a directed graph in which every edge is pointed toward a single central node. Compute the Katz centrality vector in terms of the α parameter.

9.8 Table 9.3 lists the edges of an undirected graph constructed from data collected by Wayne Zachary on the members of a university karate club [54]. Each node represents a club member, and each edge indicates a relationship outside the studio. Compute the Katz centrality vector for this graph and interpret the results. Can you identify which node corresponds to the instructor?

i	$j > i$ such that $(i, j) \in E$
1	2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 18, 20, 22, 32
2	3, 4, 8, 14, 18, 20, 22, 31
3	4, 8, 9, 10, 14, 28, 29, 33
4	8, 13, 14
5	7, 11
6	7, 11, 17
7	17
9	31, 33, 34
10	34
15	33, 34
16	33, 34
19	33, 34
21	33, 34
23	33, 34
24	26, 28, 30, 32, 33, 34
25	26, 28, 32
26	32
27	30, 34
28	34
29	32, 34
30	33, 34
31	33, 34
32	33, 34
33	34

Table 9.3: Adjacency list of the Zachary Karate Club graph. For each vertex i, the second column lists all vertices $j > i$ such that $(i, j) \in E$; hence, each edge appears exactly once.

Part III
NORMS, DISTANCES, AND SIMILARITIES

This part of the book introduces norms, distances, and similarity functions, important concepts in Machine Learning and Data Science. Norms measure the size of data points or data matrices. Distances compare data points or data matrices. We use distances to find patterns, classify data, group similar items, and measure approximation quality. Similarity functions measure how alike objects are. We apply similarity to tasks like recommendations and information retrieval.

Chapter 10
III-WHY SHOULD WE CARE?

As we mentioned in Chapter 1, two common tasks in Machine Learning are classification and clustering. In classification, we determine which category an object belongs to. In clustering, we group similar objects together. In Chapter 14, we introduce two popular methods: k-nearest neighbors (for classification and regression) and k-means (for clustering). Both rely on the concept of "distance between data points" to measure similarity.

We will also consider information retrieval: searching datasets or collections for specific items. Similarity plays a key role in this process. To decide how relevant a document or item is to a query, we measure its similarity to that query. The higher the similarity, the higher it ranks in the results.

10.1 A classification problem: the Palmer Archipelago penguin data

To illustrate a classification problem, we will use the Palmer Archipelago penguin data [31]. This dataset contains details on 344 penguins belonging to three distinct species: Adélie, Gentoo, and Chinstrap. These penguins were observed on three different islands within the Palmer Archipelago in Antarctica. In Figure 10.1, we display a randomly chosen subset of five rows of data.

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
134	Adelie	Dream	38.1	17.6	187.0	3425.0	FEMALE
122	Adelie	Torgersen	40.2	17.0	176.0	3450.0	FEMALE
265	Gentoo	Biscoe	43.6	13.9	217.0	4900.0	FEMALE
105	Adelie	Biscoe	39.7	18.9	184.0	3550.0	MALE
156	Chinstrap	Dream	52.7	19.8	197.0	3725.0	MALE

Fig. 10.1: Palmer Archipelago penguin sample data.

Table 10.1 provides a description of each column in the Palmer Archipelago penguin dataset.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_10

Variables	Description
species	penguin species (Chinstrap, Adélie, or Gentoo)
island	island name (Dream, Torgersen, or Biscoe) in the Palmer Archipelago (Antarctica)
culmen_length_mm	culmen length in millimeters
culmen_depth_mm	culmen depth in millimeters
flipper_length_mm	flipper length in millimeters
body_mass_g	penguin body mass in grams
sex	penguin sex

The culmen of a bird is essentially the top of the beak, from the base to the tip.

Table 10.1: Palmer Archipelago penguin data variables.

Let us consider the problem of predicting a penguin's species based on two physical measurements: the length and depth of its culmen. In the left side of Figure 10.2, we plotted the culmen's length against its depth for the entire dataset. Each data point is color-coded by species. You can see that each species occupies a distinct area in this two-dimensional space of culmen length and depth. This suggests we might be able to predict a penguin's species using just these two features.

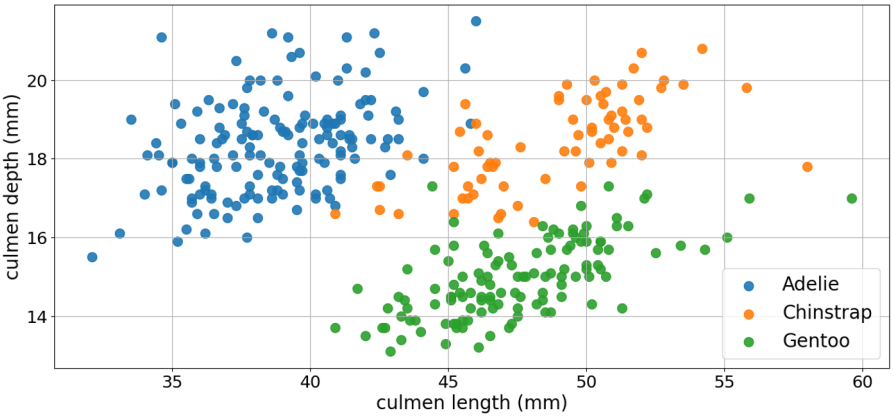


Fig. 10.2: Palmer Archipelago penguin data.

Imagine we take physical measurements of a newly encountered penguin. The measurements show that the culmen is 45 mm long and 16 mm deep. In Figure 10.3, we represent this new data point with a black dot. This view is a zoomed-in section of the plot shown in Figure 10.2.

With this data, how do we determine the species of the new penguin? One approach is to look at the closest neighbors of this data point and assign it the species that occurs most frequently among them. In our case, the majority species is Gentoo. This idea is the basis of the k-nearest neighbors (kNN) method, which we

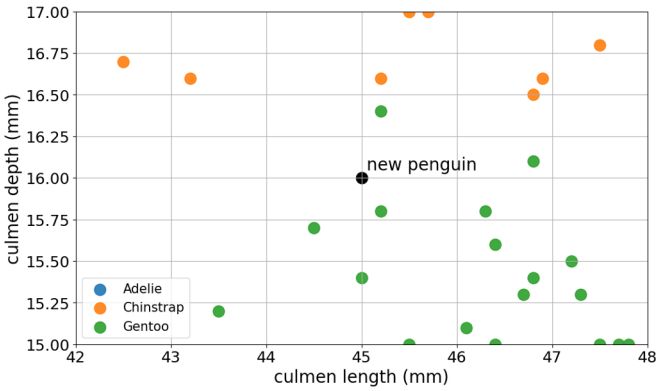


Fig. 10.3: We receive new physical measurements of a penguin represented by the black dot. Our goal is to predict the species of the penguin based on the data we already have.

will cover in detail in Chapter 14. Before we discuss that, we introduce vector norms and distances in Chapter 12, to better understand what a neighbor is.

10.2 A clustering problem: Old Faithful eruption data

Let us look at a clustering problem using the Old Faithful eruption data as an example [7]. This dataset contains information about the waiting times between eruptions and the duration of eruptions in minutes for the Old Faithful geyser in Yellowstone National Park, Wyoming, USA. Figure 10.4 shows a plot of the data points.

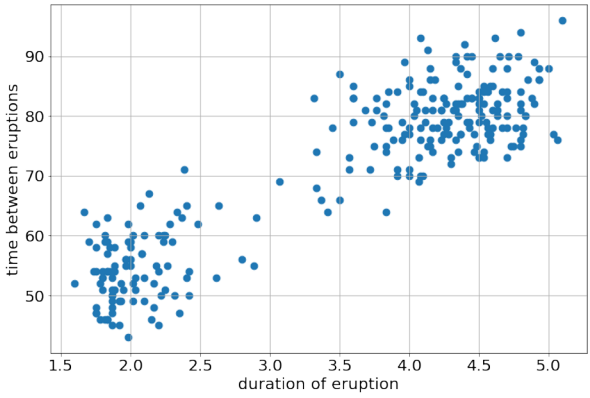


Fig. 10.4: Old Faithful eruption data.

The Old Faithful eruption data shows two distinct groups of eruptions (the two blobs of points in Figure 10.4). One cluster corresponds to shorter eruptions that occur more frequently (shorter intervals between eruptions), while the other represents longer eruptions with longer intervals between them. Can we separate this data into clusters without plotting it? The k-means method provides a simple, effective answer. We will cover it in Chapter 14. As with the nearest neighbors method, measuring distance is essential to k-means.

10.3 Information retrieval and query matching

In some applications, we need to determine the similarity between objects. For instance, in information retrieval, our datasets may include websites, emails, text messages, social media posts, etc. As we discussed in Chapter 3, we can transform this data into vectors. When a query is made, our goal is to measure its similarity to the vectors in our database and find the most relevant documents.

Let us consider a simple example where the documents are just sentences.

Example 10.1. (Query matching)

Imagine you are building a search feature for a math resource website. The site has brief descriptions of three math articles:

- Document 1: "A **similarity measure** is a **function** that **quantifies** the **similarity** between two **objects**."
- Document 2: "A **similarity measure** can be constructed from a **distance** in a space whose **dimensions** represent features of the **objects.**"
- Document 3: "**Similarity** is the property of **sharing** the same **shape**."

A student enters the search query: "similarity measure". The goal is to identify the article most relevant to this query.

The process begins by representing both the documents and the query as vectors. As discussed in Chapter 3, a common approach is to count how frequently specific terms appear in each document. First, we create a vocabulary from the bolded terms (converted to lowercase), and then build a term-frequency vector for each document based on this vocabulary.

Here is the resulting term-frequency matrix:

	Doc 1	**Doc 2**	**Doc 3**	**Query**
similarity	2	1	1	1
measure	1	1	0	1
function	1	0	0	0
quantifies	1	0	0	0
objects	1	1	0	0
distance	0	1	0	0
dimensions	0	1	0	0
features	0	1	0	0
sharing	0	0	1	0
shape	0	0	1	0

We can now represent each document and the query as a vector of length 10:

$$\text{doc}_1 = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \text{doc}_2 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \text{doc}_3 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \quad \text{and} \quad \text{query} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

In Chapter 12, we will cover different similarity measures to determine which document is most relevant to the student's query.

Chapter 11
III-WHAT YOU MAY HAVE LEARNED BEFORE

11.1 Real vector spaces

One of the central topics in Linear Algebra is the concept of vector space. We will focus specifically on vector spaces over the real numbers.

Definition 11.1. (Real vector space)

A real **vector space** consists of:

(i) A nonempty set V of objects called **vectors**.
(ii) The set of real numbers called **scalars**.
(iii) A rule for **vector addition**: $x + y \in V$ for all $x, y \in V$.
(iv) A rule for **scalar multiplication**: $\alpha x \in V$ for all $\alpha \in \mathbb{R}$ and $x \in V$.

Moreover, the following properties must hold:

- **Associative:** $x + (y + z) = (x + y) + z$ for all $x, y, z \in V$.
- **Commutative:** $x + y = y + x$ for all $x, y \in V$.
- **Additive Identity:** There exists an element $y \in V$ such that $x + y = x$ for all $x \in V$. The vector y is usually denoted by 0.
- **Additive Inverse:** For each $x \in V$, there exists $y \in V$ such that $x + y = 0$. The vector y is usually denoted by $-x$.
- **Scalar Identity:** $1 \cdot x = x$ for all $x \in V$.
- **Distributive with respect to the addition of vectors:** $\alpha(x + y) = \alpha x + \alpha y$ for all $\alpha \in \mathbb{R}$ and $x, y \in V$.
- **Distributive with respect to the addition of scalars:** $(\alpha + \beta)x = \alpha x + \beta x$ for all $\alpha, \beta \in \mathbb{R}$ and $x \in V$.
- **Mixed associative property:** $\alpha(\beta x) = (\alpha\beta)x$ for all $\alpha, \beta \in \mathbb{R}$ and $x \in V$.

We are particularly interested in two vector spaces:

- The real vector space $\mathbb{R}^n$, that is, the set of column vectors of length n with real entries, along with the standard operations of addition and scalar multiplication discussed in Chapter 2.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_11

- The real vector space $\mathbb{R}^{m\times n}$, that is, the set of matrices of size $m \times n$ with real entries, with the operations of addition and scalar multiplication presented in Chapter 6.

In some chapters, we will need to consider vector spaces over the complex numbers. The definition is the same, but we replace the reals with the complex numbers as scalars.

11.2 Norm function

In this section, we review the concept of a vector norm, which, informally, measures the size of a vector. Formally, a norm is a real-valued function defined on a vector space. It must satisfy certain rules, which we present next.

Definition 11.2. (Norm function)

A **norm** on a real vector space V is a function $\|\cdot\| : V \to \mathbb{R}$ that satisfies the following four properties:

(i) $\|v\| \geqslant 0$, for all vectors $v \in V$;
(ii) $\|v\| = 0$ if and only if $v = 0$;
(iii) $\|\alpha v\| = |\alpha|\,\|v\|$, for all vectors $v \in V$ and scalars $\alpha \in \mathbb{R}$; and
(iv) $\|u + v\| \leqslant \|u\| + \|v\|$ for all vectors $u, v \in V$.

We typically use the "double bar" symbol to represent vector norms in this book. If we use two different norms, we might add a subscript, like $\|\cdot\|_{\#}$ or $\|\cdot\|_{*}$, to tell them apart.

We call conditions (i)–(iv) in Definition 11.2 the **norm axioms**. Property (iv) is known as the **triangle inequality**, which states that the norm of the sum of two vectors is never greater than the sum of their individual norms. This mirrors the geometric fact that, in any triangle, the length of one side is at most the sum of the lengths of the other two sides.

By repeatedly applying the triangle inequality, we can extend property (iv) to the sum of k vectors $v_1, v_2, \ldots, v_k$ as follows:

$$\begin{aligned}\|v_1 + v_2 + v_3 + \cdots + v_k\| &\leqslant \|v_1\| + \|v_2 + v_3 + \cdots + v_k\| \\ &\leqslant \|v_1\| + \|v_2\| + \|v_3 + \cdots + v_k\| \\ &\;\;\vdots \\ &\leqslant \|v_1\| + \|v_2\| + \|v_3\| + \cdots + \|v_k\|.\end{aligned}$$

The following example illustrates an intuitive and familiar norm.

Example 11.3. (The absolute value)

The absolute value is a norm on the set of real numbers $\mathbb{R}$ as it satisfies all the norm axioms.

It is easy to check that the first three norm axioms are satisfied by the absolute value function. In Problem 11.8, we ask you to verify that it also satisfies the triangle inequality.

The most familiar norm on $\mathbb{R}^n$ is the Euclidean norm, which generalizes the usual notion of length from $\mathbb{R}^2$ and $\mathbb{R}^3$ to higher dimensions.

Definition 11.4. (The Euclidean norm)

The **Euclidean norm** of a vector $v = \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix}^T \in \mathbb{R}^n$ is

$$\|v\|_2 := \left(\sum_{i=1}^{n} v_i^2 \right)^{1/2}. \tag{11.1}$$

The Euclidean norm is also called the **2-norm**, which explains the subscript "2" in $\| \cdot \|_2$. You will understand this name better once we introduce the concept of p-norm in the next chapter (see Definition 12.1).

The Euclidean norm and dot products

The Euclidean norm is closely related to the dot product of vectors in $\mathbb{R}^n$ (recall Definition 6.5). We can compute the Euclidean norm of a vector $v \in \mathbb{R}^n$ as $\|v\|_2 = \left(v^T v\right)^{1/2}$.

Theorem 11.6 shows that the Euclidean norm $\| \cdot \|_2$ satisfies the four norm axioms. To establish this, we will first prove the well-known Cauchy-Schwarz inequality.

Theorem 11.5. (Cauchy-Schwarz inequality)

Let u and v be vectors in $\mathbb{R}^n$. The dot product of u and v satisfies the inequality

$$|u^T v| \leqslant \|u\|_2 \|v\|_2.$$

Moreover, equality holds if and only if one of u or v is a scalar multiple of the other.

Proof Because the square function is strictly increasing on $[0, \infty)$, the Cauchy-Schwarz inequality is equivalent to

$$(u^T v)^2 \leqslant \|u\|_2^2 \|v\|_2^2.$$

In other words, it is enough to show that, for $u = \begin{bmatrix} u_1 & \cdots & u_n \end{bmatrix}^T$ and $v = \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix}^T$, we have

$$\left(\sum_{i=1}^{n} u_i v_i\right)^2 \leqslant \left(\sum_{i=1}^{n} u_i^2\right)\left(\sum_{j=1}^{n} v_j^2\right).$$

Observe that

$$\begin{aligned} 2\left[\left(\sum_{i=1}^{n} u_i^2\right)\left(\sum_{j=1}^{n} v_j^2\right) - \left(\sum_{i=1}^{n} u_i v_i\right)^2\right] &= \sum_{i=1}^{n} u_i^2 \sum_{j=1}^{n} v_j^2 + \sum_{i=1}^{n} v_i^2 \sum_{j=1}^{n} u_j^2 - 2\sum_{i=1}^{n} u_i v_i \sum_{j=1}^{n} u_j v_j \\ &= \sum_{i=1}^{n}\sum_{j=1}^{n} (u_i^2 v_j^2 + u_j^2 v_i^2 - 2u_i v_j u_j v_i) \\ &= \sum_{i=1}^{n}\sum_{j=1}^{n} (u_i v_j - u_j v_i)^2 \geqslant 0, \end{aligned} \tag{11.2}$$

which is equivalent to the Cauchy-Schwarz inequality.

To show the second part of the theorem, assume first that one of u or v is a scalar multiple of the other. Without loss of generality, assume that $v = \alpha u$ for some scalar α. Then,

$$|u^T v| = |u^T(\alpha u)| = |\alpha||u^T u| = |\alpha|\|u\|_2^2 = \|u\|_2 \, \|\alpha u\|_2 = \|u\|_2 \, \|v\|_2,$$

that is, equality in Cauchy-Schwarz inequality holds.

Conversely, suppose that equality holds, that is, $|u^T v| = \|u\|_2 \|v\|_2$. This is equivalent to the inequality in (11.2) being an equality. This means that $u_i v_j - u_j v_i = 0$ for all i and j such that $i \neq j$, which implies that one of u or v is a scalar multiple of the other as we see next.

If $v = 0$, then clearly $v = 0 \cdot u$. Assume that $v \neq 0$. Then, there exists $i \in \{1, 2, \ldots, n\}$ such that $v_i \neq 0$ and, by assumption, for all $j \neq i$,

$$u_j v_i = u_i v_j.$$

This implies

$$u_j = \frac{u_i}{v_i} v_j,$$

or equivalently,

$$u = \frac{u_i}{v_i} v.$$

Hence, u is a multiple of v. □

Theorem 11.6. (The Euclidean norm satisfies the norm axioms)

The Euclidean norm is a norm function.

Proof To prove properties (i) and (ii), we use the expression for the Euclidean norm in (11.1). Since every term v_i^2 is nonnegative, the sum (and its square root) is also nonnegative. Hence, $\|v\|_2 \geqslant 0$.

If $\|v\|_2 = 0$, then each term v_i^2 must be zero. This means $v_i = 0$ for all components, or equivalently, v is the zero vector. Conversely, if v is the zero vector, then $\|v\|_2^2 = 0$. Therefore, $\|v\|_2 \geqslant 0$ and is zero if and only if v is the zero vector. This establishes properties (i) and (ii).

To prove property (iii), we compute the norm of the scaled vector αv as

$$\|\alpha v\|_2 = \left(\sum_{i=1}^{n} (\alpha v_i)^2\right)^{1/2} = \left(\sum_{i=1}^{n} \alpha^2 v_i^2\right)^{1/2} = \left(\alpha^2\right)^{1/2} \left(\sum_{i=1}^{n} v_i^2\right)^{1/2} = |\alpha| \, \|v\|_2.$$

To prove property (iv), we express the Euclidean norm using the dot product and then apply properties of transposes and the dot product. We get

$$\begin{aligned} \|u+v\|_2^2 &= (u+v)^T (u+v) = (u^T + v^T)(u+v) \\ &= \|u\|_2^2 + 2u^T v + \|v\|_2^2 \\ &\leqslant \|u\|_2^2 + 2\|u\|_2\|v\|_2 + \|v\|_2^2 = (\|u\|_2 + \|v\|_2)^2, \end{aligned}$$

where the inequality follows from the Cauchy-Schwarz inequality. Taking the square root of both sides gives the triangle inequality. □

In many applications, vectors that have a norm of one are important, so we give them a special name.

Definition 11.7. (Unit vectors)

We say that a vector v in $\mathbb{R}^n$ is a **unit vector** with respect to a norm $\|\cdot\|$ if $\|v\| = 1$. If we know which norm is being used, we simply refer to v as a unit vector.

Example 11.8 The **canonical basis vectors** for $\mathbb{R}^n$:

$$e_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}, \quad e_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}, \quad \ldots, \quad e_n = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}, \tag{11.3}$$

are unit vectors with respect to the Euclidean norm.

Sometimes, we need a unit vector that lies on the same line as a given non-zero vector v. The following result follows directly from property (iii) of vector norms, which you can verify in Problem 11.15.

Proposition 11.9. (Unit vectors)

Given a non-zero vector $v \in \mathbb{R}^n$ and a norm $\|\cdot\|$, the vectors $\frac{v}{\|v\|}$ and $-\frac{v}{\|v\|}$ are the only unit vectors parallel to v.

11.3 Angle between vectors

When we study geometry, we define an angle as the union of two rays (half lines) OA and OB sharing the same endpoint O. This angle is denoted by $\angle AOB$, and the point O is called the **vertex of the angle**. When we talk about the angle defined by vectors, we mean the angle defined by the rays containing those vectors.

To every angle $\angle AOB$, we can assign a real number $m(\angle AOB)$ called the **measure of the angle** with the following properties:

1. $0 \leqslant m(\angle AOB) \leqslant \pi$;
2. If OA is a ray on the edge of a half-plane, for every $r \in [0, \pi]$, there is exactly one ray OB with B in that half-plane and such that $m(\angle AOB) = r$.
3. If C is a point in the interior of the angle $\angle AOB$, then

$$m(\angle AOB) = m(\angle AOC) + m(\angle COB).$$

Notice that, in practice, we usually call angle to its measure and do not make a distinction between these two concepts.

Definition 11.10 formalizes the notion of the angle between two vectors in $\mathbb{R}^n$. This definition is based on the following observation: for any two nonzero vectors u and v, the Cauchy-Schwarz inequality implies

$$-\|u\|_2\|v\|_2 \leqslant u^T v \leqslant \|u\|_2\|v\|_2.$$

We can rewrite this as

$$-1 \leqslant \frac{u^T v}{\|u\|_2\|v\|_2} \leqslant 1.$$

From these inequalities, we obtain that there is a unique angle θ in the range $[0, \pi]$ such that

$$\cos(\theta) = \frac{u^T v}{\|u\|_2\|v\|_2}.$$

Definition 11.10. (Angle between two vectors in $\mathbb{R}^n$)

For two nonzero vectors u and v in $\mathbb{R}^n$, the scalar

$$\theta(u, v) = \arccos\left(\frac{u^T v}{\|u\|_2\|v\|_2}\right) \tag{11.4}$$

is the **angle between u and v**, with arccos denoting the inverse cosine, constrained to the interval $[0, \pi]$.

We can show that this definition of angle meets the standard properties that angle measures should have. However, we will not include that proof here since it is beyond the scope of this book.

Problems

11.1 While $\mathbb{R}^n$ (column vectors) and $\mathbb{R}^{m\times n}$ (matrices) are central examples of real vector spaces, many others arise in mathematics. Can you think of other real vector spaces you have seen in previous courses?

11.2 Determine if the following functions $f, g : \mathbb{R}^2 \to \mathbb{R}$ are norms on $\mathbb{R}^2$ or not:

$$f\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = |x| + 3|x - y|, \quad \text{and}$$
$$g\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \sqrt{x^2 - xy + y^2}.$$

Hint: To simplify the analysis for the function g, write $x^2 - xy + y^2$ as a "sum of squares".

11.3 Show that the norm axiom (iii) implies that the norm of the zero vector is zero.

11.4 Show that for any positive constant c, if $\|\cdot\|$ is a norm on $\mathbb{R}^n$, then $c\|\cdot\|$ is also a norm on $\mathbb{R}^n$.

11.5 Consider two norms $\|\cdot\|_\star$ and $\|\cdot\|_*$ defined on $\mathbb{R}^n$ and $\mathbb{R}^m$, respectively. For any vector $u = \left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right] \in \mathbb{R}^{m+n}$, where $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$, define

$$\|u\| = \|x\|_\star + \|y\|_*.$$

Show that $\|\cdot\|$ satisfies the norm axioms. Can you think of a different way of creating a new norm based on the $\|\cdot\|_\star$ and $\|\cdot\|_*$ norms?

11.6 Consider two norms $\|\cdot\|_\star$ and $\|\cdot\|_*$ defined on $\mathbb{R}^n$, and let $\alpha \in [0, 1]$. For any vector $u \in \mathbb{R}^n$, define

$$\|u\| = \alpha\|u\|_\star + (1 - \alpha)\|u\|_*.$$

Show that $\|\cdot\|$ satisfies the norm axioms.

11.7 (The reverse triangle inequality) For any vector norm on $\mathbb{R}^n$ and vectors $u, v \in \mathbb{R}^n$ prove that $\|u - v\| \geqslant |\|u\| - \|v\||$.

Hint: Apply the triangle inequality to $u = (u - v) + v$ and $v = (v - u) + u$.

11.8 Prove that the absolute value satisfies $|x + y| \leqslant |x| + |y|$ for all real numbers x and y.

11.9 On $\mathbb{R}$, the absolute value $|x|$ is a norm. By Problem 11.4, any positive multiple $c\,|x|$ (with $c > 0$) is also a norm on $\mathbb{R}$. Can you think of any other norms on $\mathbb{R}$?

11.10 Use the Cauchy-Schwarz inequality to show that

$$\left(\frac{a+b}{2}\right)^2 \leqslant \frac{a^2+b^2}{2},$$

for any $a, b \in \mathbb{R}$.

11.11 Use the Cauchy-Schwarz inequality to show that

$$\sqrt{ab} \leqslant \left(\frac{a+b}{2}\right),$$

for any non-negative $a, b \in \mathbb{R}$. This inequality is known as the AM-GM (arithmetic mean-geometric mean) inequality.

11.12 True or false. Justify your answer.

(a) The norm of two parallel vectors in $\mathbb{R}^n$ is the same.
(b) $\|u+v\|_2 = \|u\|_2 + \|v\|_2$ if u and v form an angle of $\pi/2$ radians.
(c) For any value of the angles θ and ϕ, the vector

$$u = \begin{bmatrix} \cos(\theta)\cos(\phi) \\ \cos(\theta)\sin(\phi) \\ \sin(\theta) \end{bmatrix}$$

is a unit vector in the Euclidean norm.

11.13 Prove the parallelogram law:

$$\|x+y\|_2^2 + \|x-y\|_2^2 = 2\|x\|_2^2 + 2\|y\|_2^2, \quad \text{for all } x, y \in \mathbb{R}^n.$$

Why do you think this identity is called the "parallelogram identity".

11.14 Show that, for all $x, y \in \mathbb{R}^n$,

$$x^T y = \frac{1}{4}\left(\|x+y\|_2^2 - \|x-y\|_2^2\right),$$

that is, the dot product of two vectors can be recovered from the norm of their sum and difference.

11.15 Prove Proposition 11.9.

11.16 Give an example of two different bases of $\mathbb{R}^3$ whose vectors are unit vectors (with respect to the Euclidean norm).

11.17 This problem shows an alternative proof for the Cauchy-Schwarz inequality $|u^T v| \leqslant \|u\|_2\|v\|_2$ given in Theorem 11.5. Assume both u and v are nonzero vectors.

(a) Assume $\|u\|_2 = \|v\|_2 = 1$. Use the nonnegativity of $\|u+v\|_2^2$ and $\|u-v\|_2^2$ to conclude $|u^T v| \leqslant 1$.
(b) Use part (a) to prove the Cauchy-Schwarz inequality for any nonzero vectors u and v.

Chapter 12
III-CORE TOPICS

In Chapter 10, we discussed some examples of classification and clustering problems where measuring the distance between vectors representing data points is important. In this chapter, we present the formal definition of a distance function and introduce some important examples.

Because many distances derive from norms (see Definition 11.2), the next two sections introduce important norms on $\mathbb{R}^n$ and $\mathbb{R}^{m\times n}$.

12.1 Examples of norms in $\mathbb{R}^n$

We have already covered the Euclidean norm (or 2-norm) on $\mathbb{R}^n$ (see Definition 11.4). This norm belongs to a family of vector norms called p-norms, which we introduce next.

Definition 12.1. (p-norm)

For a given $p \in [1, \infty)$ and vector $u = \begin{bmatrix} u_1 & \cdots & u_n \end{bmatrix}^T \in \mathbb{R}^n$, the **$p$-norm** (or **Minkowski norm**) of u is defined by

$$\|u\|_p := \left(\sum_{i=1}^{n} |u_i|^p \right)^{1/p}. \tag{12.1}$$

It is easy to verify that the function $\|\cdot\|_p$ with $p \in [1, \infty)$ satisfies the first three norm axioms. However, proving the triangle inequality, also known as the Minkowski's inequality, for any $p \in [1, \infty)$ is beyond our current discussion.

Among the family of p-norms, the 1-norm is also often used in applications alongside the 2-norm (Euclidean norm).

Definition 12.2. (The 1-norm or Taxicab norm)

The 1-norm of a vector $u = \begin{bmatrix} u_1 & \cdots & u_n \end{bmatrix}^T \in \mathbb{R}^n$ is given by

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_12

$$\|u\|_1 = \sum_{i=1}^{n} |u_i|.$$

Problem 12.1 asks you to verify that the 1-norm satisfies the norm axioms.

Known as the **Taxicab norm** or **Manhattan norm**, the 1-norm measures distance between two points on a grid, like navigating city blocks. Figure 12.1 compares the 1-norm and 2-norm of a vector u. The length of the blue path gives the 2-norm, while the length of the two red paths give the 1-norm. In other words, the 1-norm equals the distance from the origin to the endpoint of u when restricted to grid-aligned moves.

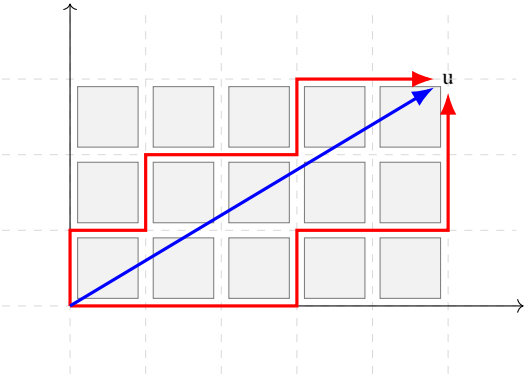


Fig. 12.1: Euclidean norm (2-norm) vs Taxicab norm (1-norm). The length of the blue path equals the 2-norm of u. The lengths of the two red paths both equal the 1-norm of u.

In Chapter 52, when studying Markov chains, we will work with vectors such as $v = \begin{bmatrix} 1/3 & 2/3 & 0 \end{bmatrix}^T$, which have nonnegative entries and a 1-norm equal to 1. We now introduce a name for this type of vector.

Definition 12.3. (Probability vectors)

A vector $v \in \mathbb{R}^n$ is called a **probability vector** if all its entries are nonnegative and $\|v\|_1 = 1$.

Note that probability vectors are unit vectors with respect to the 1-norm.

Taking the limit of the p-norm $\|v\|_p$ as $p \to \infty$ gives another important vector norm on $\mathbb{R}^n$.

Definition 12.4. (Infinity-norm or max norm)

The **infinity norm** of a vector $v = \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix}^T \in \mathbb{R}^n$ is given by

$$\|v\|_\infty := \lim_{p\to\infty} \|v\|_p = \max\{|v_1|, \ldots, |v_n|\}.$$

Problem 12.2 asks you to verify that the infinity-norm satisfies the norm axioms.

Example 12.5 Consider the vector $v = \begin{bmatrix}-1/3 & 2/3 & 0\end{bmatrix}^T$. Then, we have

$$\begin{aligned}
\|v\|_2 &= \sqrt{\frac{1}{9} + \frac{4}{9}} = \frac{\sqrt{5}}{3}, \\
\|v\|_1 &= \frac{1}{3} + \frac{2}{3} = 1, \qquad \text{and} \\
\|v\|_\infty &= \max\left\{\frac{1}{3}, \frac{2}{3}, 0\right\} = \frac{2}{3}.
\end{aligned}$$

A fun and insightful way to visualize the differences between various norms in $\mathbb{R}^n$ is to look at the shape formed by the unit vectors under each norm.

Definition 12.6. (Unit sphere)

For a given norm $\|\cdot\|$, the **unit sphere** in $\mathbb{R}^n$ is the set of all unit vectors in that norm, that is, $\{v \mid \|v\| = 1\}$.

Figure 12.2 displays the unit spheres in $\mathbb{R}^2$ for the 2-, 1-, and infinity- norms.

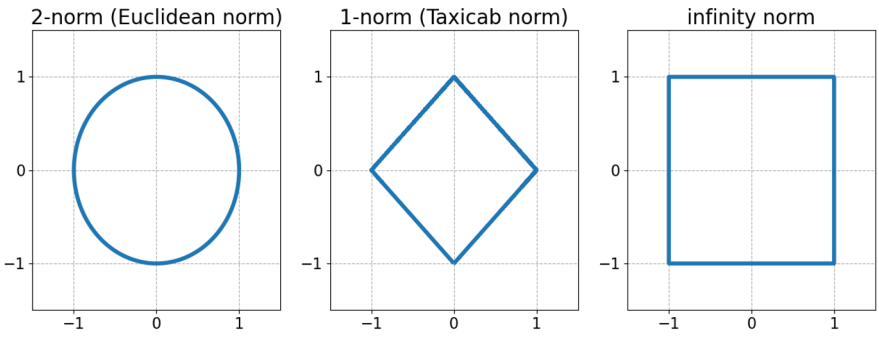


Fig. 12.2: Unit spheres in $\mathbb{R}^2$ for the 1, 2 and infinity norms.

While the expression $\|\cdot\|_p$ is well-defined for $0 < p < 1$, it does not define a norm because it fails the triangle inequality (see Problem 12.5). Such a function is sometimes called a quasi-norm. However, it is interesting to see how the shape of the "unit sphere" changes as p varies over $0 < p < \infty$, as Figure 12.3 shows.

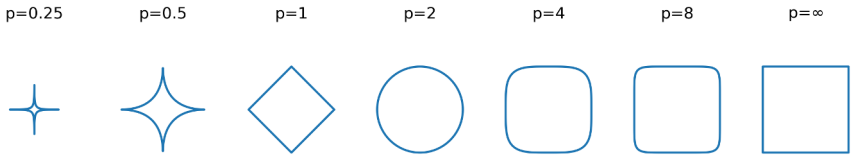


Fig. 12.3: "Unit spheres" for different values of p.

12.1.1 Equivalent norms

As we know, the norm of a vector measures its "size". Different norms give us different ways of measuring vectors. But can we expect those values to be wildly different? It turns out they cannot, at least in the settings we care about most.

Definition 12.7. (Equivalent norms)

Two norms $\|\cdot\|_a$ and $\|\cdot\|_b$ on a vector space V are said to be *equivalent* if there exist constants $c, C > 0$ such that for all $x \in V$

$$c\|x\|_a \leqslant \|x\|_b \leqslant C\|x\|_a.$$

The left inequality says that $\|x\|_b$ cannot be too small compared to $\|x\|_a$ while the right one says that $\|x\|_b$ cannot be too large compared to $\|x\|_a$. In other words, the two norms always measure vector sizes within constant multiples of each other.

The concept of equivalent norms is particularly important in Analysis since equivalent norms define the same notion of

1. **convergence**: a sequence converges in $\|\cdot\|_a$ if and only if it converges in $\|\cdot\|_b$;
2. **continuity**: a function is continuous in one norm if and only if it is continuous in the other;
3. **topology**: the two norms generate the same collection of open sets. Thus, equivalent norms generate the same topology and therefore have the same compact and connected sets. They also determine the same bounded sets.

Thus, for the purpose of Analysis, two equivalent norms are "the same".

Next we show that all norms of interest to us are equivalent when defined on the same vector space. The proof uses Analysis tools.

Theorem 12.8. (All norms are equivalent)

On a finite-dimensional vector space, all norms are equivalent.

Proof If the vector space $V = \{0\}$, the result is immediate. Assume that $\dim(V) = n \geqslant 1$, and choose a basis $\mathcal{B} = \{e_1, \ldots, e_n\}$ of V. For arbitrary $x \in V$, we have

$$x = x_1 e_1 + \cdots + x_n e_n$$

for some scalars $x_1, \ldots, x_n$. Define the coordinate norm of x by

$$\|x\|_{\mathcal{B}} := \sum_{i=1}^{n} |x_i|.$$

We first show that any norm $\|\cdot\|$ on V is equivalent to $\|\cdot\|_{\mathcal{B}}$. Let $M := \max_{1 \leqslant i \leqslant n} \|e_i\|$. By the triangle inequality and homogeneity of the norm,

$$\|x\| = \left\| \sum_{i=1}^{n} x_i e_i \right\| \leqslant \sum_{i=1}^{n} |x_i| \, \|e_i\| \leqslant M \sum_{i=1}^{n} |x_i| = M \|x\|_{\mathcal{B}}.$$

Thus,

$$\|x\| \leqslant M \|x\|_{\mathcal{B}} \qquad \text{for all } x \in V.$$

This inequality, together with the reverse triangle inequality, shows that the function $x \mapsto \|x\|$ is continuous with respect to the coordinate norm. Indeed

$$\big| \|x\| - \|y\| \big| \leqslant \|x - y\| \leqslant M \|x - y\|_{\mathcal{B}}.$$

Consider the coordinate unit sphere $S_{\mathcal{B}} = \{x \in V : \|x\|_{\mathcal{B}} = 1\}$. Under the coordinate identification of V with $\mathbb{R}^n$, this set corresponds to

$$\left\{ (x_1, \ldots, x_n) \in \mathbb{R}^n : \sum_{i=1}^{n} |x_i| = 1 \right\},$$

which is closed and bounded, and hence compact. Since $x \mapsto \|x\|$ is continuous with respect to the coordinate norm $\|\cdot\|_{\mathcal{B}}$, it attains a minimum m on $S_{\mathcal{B}}$. Moreover, $m > 0$, because every vector in $S_{\mathcal{B}}$ is nonzero and a norm is positive on nonzero vectors.

For any nonzero $x \in V$, the vector $y = \frac{x}{\|x\|_{\mathcal{B}}}$ belongs to $S_{\mathcal{B}}$. Therefore,

$$\|x\| = \|x\|_{\mathcal{B}} \|y\| \geqslant m \|x\|_{\mathcal{B}}.$$

The same inequality is trivially true when $x = 0$. Consequently,

$$m \|x\|_{\mathcal{B}} \leqslant \|x\| \leqslant M \|x\|_{\mathcal{B}} \qquad \text{for all } x \in V.$$

Apply this result to the two norms $\|\cdot\|_a$ and $\|\cdot\|_b$. There exist positive constants m_a, M_a, m_b, M_b such that

$$m_a \|x\|_{\mathcal{B}} \leqslant \|x\|_a \leqslant M_a \|x\|_{\mathcal{B}}$$

and

$$m_b \|x\|_{\mathcal{B}} \leqslant \|x\|_b \leqslant M_b \|x\|_{\mathcal{B}}.$$

It follows that

$$\|x\|_b \geqslant m_b \|x\|_{\mathcal{B}} \geqslant \frac{m_b}{M_a} \|x\|_a$$

and

$$\|x\|_b \leqslant M_b\|x\|_{\mathcal{B}} \leqslant \frac{M_b}{m_a}\|x\|_a.$$

Hence,

$$\frac{m_b}{M_a}\|x\|_a \leqslant \|x\|_b \leqslant \frac{M_b}{m_a}\|x\|_a \qquad \text{for all } x \in V.$$

Thus, $\|\cdot\|_a$ and $\|\cdot\|_b$ are equivalent. □

The table below provides the sharpest constants c and C for the inequality

$$c\|x\|_a \leqslant \|x\|_b \leqslant C\|x\|_a$$

comparing the most commonly used p-norms on $\mathbb{R}^n$.

Inequality	**Constants** (c, C)
$\|x\|_\infty \leqslant \|x\|_2 \leqslant \sqrt{n}\,\|x\|_\infty$	$(1,\ \sqrt{n})$
$\frac{1}{\sqrt{n}}\|x\|_1 \leqslant \|x\|_2 \leqslant \|x\|_1$	$(\frac{1}{\sqrt{n}},\ 1)$
$\|x\|_\infty \leqslant \|x\|_1 \leqslant n\,\|x\|_\infty$	$(1,\ n)$

12.2 Examples of matrix norms

In Chapter 49, we will need to measure how different two matrices are in size or how close one is to another, especially when approximating matrices. To do this, we need a notion of distance. As with vectors, many matrix distances arise from norms. In this section, we introduce several examples of matrix norms.

Just like with vector norms, we typically use the "double bar" notation for most matrix norms. However, for certain specific norms, we will use the less common "triple bar" notation $|||\cdot|||$.

In the next two subsections, we introduce two approaches to defining matrix norms. The first approach measures the overall "size" of the matrix entries, while the second captures how much the matrix can "stretch" a vector.

12.2.1 Matrix norms via column stacking

While we typically see matrices as two-dimensional objects, we can also think of them as vectors by stacking their columns in some predetermined order. This process turns an $m \times n$ matrix A into a vector in $\mathbb{R}^{mn}$.

Definition 12.9. (The vectorization of a matrix)

For a matrix $A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \in \mathbb{R}^{m\times n}$, with a_i denoting its ith column, we define the **vectorization** of A as the vector obtained by stacking the columns of the matrix on top of each other as follows:

$$\text{vec}(A) = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \in \mathbb{R}^{mn}.$$

Example 12.10 Consider the 2×2 matrix $A = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$. The vectorization of A is

$$\text{vec}(A) = \begin{bmatrix} a \\ c \\ b \\ d \end{bmatrix}.$$

Through the vectorization operation, we can take any vector norm $\|\cdot\|$ on $\mathbb{R}^{mn}$ and use it to define a matrix norm on $\mathbb{R}^{m\times n}$. This resulting matrix norm is

$$|||A||| := \|\text{vec}(A)\|. \tag{12.2}$$

We use the "triple bar" notation here, reserving the "double bar" notation for a different matrix norm we will study in the next section.

Using a p-norm in $\mathbb{R}^{mn}$, we derive from (12.2) the matrix norm

$$|||A|||_p := \left(\sum_{i=1}^{m} \sum_{j=1}^{n} |A_{ij}|^p \right)^{\frac{1}{p}}. \tag{12.3}$$

We will refer to (12.3) as the **vectorized matrix p-norm**. In particular, when we use the 1- and infinity- vector norms, we get the matrix norms

$$|||A|||_1 = \sum_{i=1}^{m} \sum_{j=1}^{n} |A_{ij}|, \quad \text{and}$$
$$|||A|||_\infty = \max_{\substack{1\leqslant i\leqslant m \\ 1\leqslant j\leqslant n}} |A_{ij}|.$$

The most commonly used matrix norm derived from (12.3) is the Frobenius norm. It is based on the vector 2-norm and is denoted with the special notation $\|\cdot\|_F$, rather than $|||A|||_2$.

Definition 12.11. (Frobenius norm)

The **Frobenius norm** of a matrix $A \in \mathbb{R}^{m\times n}$ is given by

$$\|A\|_F := \|\|A\|\|_2 = \|\text{vec}(A)\|_2 = \left(\sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}^2\right)^{1/2}.$$

There are different, but equivalent, ways to define the Frobenius norm $\|\cdot\|_F$. One way is by taking advantage of the fact that any $m \times n$ matrix A can be partitioned either by columns or rows, that is,

$$A = \begin{bmatrix} c_1 & \cdots & c_n \end{bmatrix} = \begin{bmatrix} r_1^T \\ \vdots \\ r_m^T \end{bmatrix},$$

where c_i is the i-th column of A and r_i^T is the i-th row of A. Using the 2-norms of these columns or rows, we can compute the Frobenius norm of A as

$$\|A\|_F = \left(\sum_{i=1}^{n} \|c_i\|_2^2\right)^{1/2} = \left(\sum_{i=1}^{m} \|r_i\|_2^2\right)^{1/2}.$$

The next proposition gives another way to express the Frobenius norm using the trace operation.

Proposition 12.12. (Frobenius norm and trace)

For any matrix A, we have

$$\|A\|_F = (\text{trace}(AA^T))^{1/2},$$

where $\text{trace}(M) = \sum_{i=1}^{n} M_{ii}$ for all $M \in \mathbb{R}^{n\times n}$.

Proof We can see that

$$\text{trace}(AA^T) = \sum_{i=1}^{m}(AA^T)_{ii} = \sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}A_{ji}^T = \sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}^2 = \|A\|_F^2,$$

as we wanted to show. □

Remark 12.13. (The family of matrix norms $\|\|\cdot\|\|_p$)

When we fix the vector p-norm but allow the matrix dimensions m and n to vary, equation (12.3) defines a different norm for each combination of m and n. However, despite the changing dimensions, all these norms are defined by the same formula. For this reason, we can think of equation (12.3) as defining a **family of matrix norms**. This is why we use the same notation $\|\|\cdot\|\|_p$ for every norm in the family, omitting m and n.

Apart from the Frobenius norm, norms defined by vectorizing a matrix see little practical use. In the next section, we introduce more commonly used matrix norms.

12.2.2 Induced matrix norms

The matrix norms we discussed in the previous section focus on the "size" of the matrix entries. In this section, we will treat matrices as linear transformations and discuss norms that measure how much a matrix stretches or amplifies a vector's norm.

Definition 12.14. (Induced matrix norm)

Let $A \in \mathbb{R}^{m\times n}$, and suppose that $\|\cdot\|$ denotes a family of vector norms, with a corresponding norm defined on $\mathbb{R}^d$ for each positive integer d. The matrix norm induced by this family is defined by

$$\|A\| := \max_{0\neq x\in\mathbb{R}^n} \frac{\|Ax\|}{\|x\|}, \quad \text{or equivalently,} \quad \|A\| := \max_{\|x\|=1} \|Ax\|. \tag{12.4}$$

Here, the norm in the denominator is the norm on $\mathbb{R}^n$, while the norm in the numerator is the corresponding norm on $\mathbb{R}^m$.

Remark 12.15. (Induced matrix norms are well-defined)

There are two questions about the definition of induced matrix norm that we need to discuss:

1. Why are the two expressions in (12.4) equivalent?
2. Is this norm well-defined since it involves computing the maximum of an infinite set?

The equivalence of the two definitions in (12.4) follows from the third vector norm axiom. We ask you to show this equivalence in Problem 12.14.

Regarding the second question, since the induced matrix norm involves finding the maximum of an infinite set of numbers, we might wonder if this maximum is always attained. A fundamental theorem in Real Analysis states that any continuous function reaches both a maximum and a minimum on a compact set. Since vector norms are continuous functions and unit spheres are compact sets, the function $\|Ax\|$ reaches its maximum for some vector x within the unit sphere $\{x \in \mathbb{R}^n \mid \|x\| = 1\}$. Therefore, there always exists a vector x such that $\|x\| = 1$ and $\|A\| = \|Ax\|$. Thus, induced matrix norms are well-defined.

In simple terms, the induced matrix norm (12.4) measures how much a matrix A can stretch vectors in the unit sphere. Figure 12.4 illustrates this in $\mathbb{R}^2$ with three common vector norms. It shows how A transforms the unit spheres for the 2-norm

(left), 1-norm (center), and infinity-norm (right). In these images, the unit vector x is stretched the most by A, so $\|Ax\| = \|A\|$.

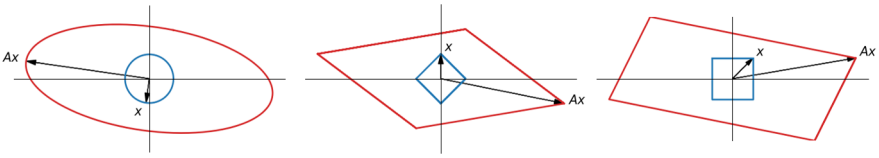


Fig. 12.4: In blue, unit spheres in $\mathbb{R}^2$ for the norms $\|\cdot\|_2$, $\|\cdot\|_1$ and $\|\cdot\|_\infty$. In red, the images of these unit spheres under the action of a matrix A. The vector x is the unit vector that is most amplified by A.

In Theorem 12.22, we will show that (12.4) satisfies all the properties of a norm. But first, let us look at some examples of induced matrix norms.

One important set comes from the p-vector norms:

$$\|A\|_p := \max_{0 \neq x \in \mathbb{R}^n} \frac{\|Ax\|_p}{\|x\|_p} = \max_{\|x\|_p = 1} \|Ax\|_p. \tag{12.5}$$

We refer to these as **matrix p-norms**. Another important matrix norm is derived from the infinity-vector norm.

Remark 12.16. (The family of matrix norms $\|\cdot\|_p$)

Just like we noted in Remark 12.13 in the previous section, we can apply the same idea to the matrix p-norm. For a fixed p, the formula (12.5) defines a family of matrix norms, with one norm for each matrix size $m \times n$. We use the same notation $\|\cdot\|_p$ for all the norms in this family.

Next, we will derive specific formulas for the 1-norm and infinity-norm of a matrix based on its entries. We will cover the formula for the 2-norm later in Chapter 47 when we discuss singular values of a matrix. For other values of p, there is no known direct formula for $\|A\|_p$.

Theorem 12.17. (Matrix 1- and infinity-norms formulas)

For a matrix $A \in \mathbb{R}^{m \times n}$, we have

$$\|A\|_1 = \max_{1 \leqslant j \leqslant n} \sum_{i=1}^{m} |A_{ij}| = \max_{1 \leqslant j \leqslant n} \|c_j\|_1, \quad \text{and}$$

$$\|A\|_\infty = \max_{1 \leqslant i \leqslant m} \sum_{j=1}^{n} |A_{ij}| = \max_{1 \leqslant i \leqslant m} \|r_i\|_1,$$

where c_j and r_i denote, respectively, the jth column and the ith row of A.

Proof We present the proof for the 1-norm formula. We encourage you to try proving the infinity-norm formula on your own in Problem 12.18.

Let us denote the columns of the matrix A by $c_1, c_2, \ldots, c_n$, and let $x \in \mathbb{R}^n$ with $\|x\|_1 = 1$. Since the matrix-vector multiplication Ax produces a linear combination of the columns of A, we have

$$\|Ax\|_1 = \left\| \sum_{j=1}^{n} x_j c_j \right\|_1 \leqslant \sum_{j=1}^{n} |x_j| \, \|c_j\|_1$$
$$\leqslant \max_{1 \leqslant j \leqslant n} \|c_j\|_1 \sum_{j=1}^{n} |x_j| = \max_{1 \leqslant j \leqslant n} \|c_j\|_1,$$

where the first inequality follows from properties (iii) (homogeneity) and (iv) (the triangle inequality) of a norm, and the final equality holds because $\|x\|_1 = 1$. Taking into account the definition of induced norm, the inequality above shows that $\|A\|_1 = \max_{\|x\|_1=1} \|Ax\|_1 \leqslant \max_{1 \leqslant j \leqslant n} \|c_j\|_1$.

To complete the proof, we need to find a vector x satisfying $\|x\|_1 = 1$ and $\|Ax\|_1 = \max_{1 \leqslant j \leqslant n} \|c_j\|_1$. To find this vector, let $k \in \{1, 2, \ldots, n\}$ be the column index where $\max_{1 \leqslant j \leqslant n} \|c_j\|_1$ is attained. In other words, $\max_{1 \leqslant j \leqslant n} \|c_j\|_1 = \|c_k\|_1$. If we take the vector $x = e_k$, we get

$$\|Ae_k\|_1 = \|c_k\|_1 = \max_{1 \leqslant j \leqslant n} \|c_j\|_1.$$

Therefore, $\|A\|_1 = \max_{1 \leqslant j \leqslant n} \|c_j\|_1$, proving our claim. □

The 1-vertical, ∞-horizontal trick

Theorem 12.17 tells us that the 1-norm of a matrix A is the largest of the 1-norms of its columns, and the infinity-norm of A is the largest of the 1-norms of its rows. A helpful trick to remember these formulas is to notice that the number 1 resembles a vertical line (for columns), while ∞ resembles a horizontal symbol (for rows).

Example 12.18 Consider the matrix

$$A = \begin{bmatrix} 2 & 5 \\ 2 & -1 \end{bmatrix}.$$

We use Theorem 12.17 to compute the 1-norm and the infinity-norm of A. We have $\|A\|_1 = \max\{4, 6\} = 6$ and $\|A\|_\infty = \max\{7, 3\} = 7$.

Remark 12.19. (The cost of computing matrix p-norms)

An advantage of the 1- and infinity-matrix norms is their computational efficiency. By Theorem 12.17, either norm of a dense matrix $A \in \mathbb{R}^{m\times n}$ can be computed in roughly mn operations, which is relatively inexpensive.

In Chapter 47, we will present the formula for the matrix 2-norm in terms of the singular values of A. For a dense matrix $A \in \mathbb{R}^{m\times n}$, computing the 2-norm through a singular value decomposition generally requires on the order of $mn\min\{m, n\}$ operations, making it significantly more expensive than computing the 1- and infinity-norms.

In the previous section we defined equivalent norms and showed that all norms in a finite-dimensional vector space are equivalent. The space of $m \times n$ real matrices is a finite-dimensional vector space, so as a direct consequence, all matrix norms are equivalent.

The following table provides the sharpest positive constants c and C in the inequality

$$c\|A\|_a \leqslant \|A\|_b \leqslant C\|A\|_a,$$

comparing the standard induced p-norms for $A \in \mathbb{R}^{m\times n}$.

Inequality	**Constants** (c, C)
$\frac{1}{\sqrt{m}}\|A\|_1 \leqslant \|A\|_2 \leqslant \sqrt{n}\,\|A\|_1$	$(\frac{1}{\sqrt{m}}, \sqrt{n})$
$\frac{1}{\sqrt{n}}\|A\|_\infty \leqslant \|A\|_2 \leqslant \sqrt{m}\,\|A\|_\infty$	$(\frac{1}{\sqrt{n}}, \sqrt{m})$
$\frac{1}{n}\|A\|_\infty \leqslant \|A\|_1 \leqslant m\|A\|_\infty$	$(\frac{1}{n}, m)$

Another interesting inequality comparing the standard matrix norms is the following:

$$\|A\|_2 \leqslant \sqrt{\|A\|_1\,\|A\|_\infty}.$$

We can generalize the induced matrix norm in Definition 12.14 by allowing different norms for x and Ax.

Definition 12.20. (Induced matrix norm II)

Given an $m \times n$ matrix A, consider a vector norm $\|\cdot\|_\#$ on $\mathbb{R}^m$ and a vector norm $\|\cdot\|_*$ on $\mathbb{R}^n$. We define the **induced matrix norm** $\|\cdot\|_{\#,*}$ as

$$\|A\|_{\#,*} := \max_{0\neq x\in\mathbb{R}^n} \frac{\|Ax\|_\#}{\|x\|_*}, \quad \text{or equivalently,} \quad \|A\|_{\#,*} := \max_{\|x\|_*=1} \|Ax\|_\#. \tag{12.6}$$

Using different vector p-norms for the numerator and denominator in (12.6), we can define the more general matrix norm

$$\|A\|_{q,p} := \max_{0\neq x\in\mathbb{R}^n} \frac{\|Ax\|_q}{\|x\|_p} = \max_{\|x\|_p=1} \|Ax\|_q.$$

We call these **mixed matrix p-norms**. Some mixed matrix norms have straightforward expressions based on the entries of matrix A (see Problem 12.23). While these norms have applications in Statistics, we will not use them in this book.

We will now show that the induced matrix norm $\|\cdot\|_{\#,*}$ in (12.6) (and thus, in (12.4)) satisfies the four norm axioms. To begin, we need the following result.

Proposition 12.21. (A key property of induced matrix norms)

For any $A \in \mathbb{R}^{m\times n}$ and any vector norms $\|\cdot\|_{\#} : \mathbb{R}^m \to \mathbb{R}$ and $\|\cdot\|_{*} : \mathbb{R}^n \to \mathbb{R}$, we have

$$\|Ax\|_{\#} \leqslant \|A\|_{\#,*}\, \|x\|_{*},$$

for all vectors $x \in \mathbb{R}^n$.

Proof By definition of $\|A\|_{\#,*}$, for any nonzero vector x we have $\frac{\|Ax\|_{\#}}{\|x\|_{*}} \leqslant \|A\|_{\#,*}$, which implies $\|Ax\|_{\#} \leqslant \|A\|_{\#,*}\, \|x\|_{*}$ for all nonzero x. But this inequality remains valid even when $x = 0$, as both sides are equal to zero. Thus $\|Ax\|_{\#} \leqslant \|A\|_{\#,*}\, \|x\|_{*}$ holds for all vectors x. □

Theorem 12.22. (Induced matrix norms satisfy the norm axioms)

For any two vector norms $\|\cdot\|_{\#} : \mathbb{R}^m \to \mathbb{R}$ and $\|\cdot\|_{*} : \mathbb{R}^n \to \mathbb{R}$, the function $\|\cdot\|_{\#,*}\ \mathbb{R}^{m\times n} \to \mathbb{R}$ defined in (12.6) is a norm on $\mathbb{R}^{m\times n}$.

Proof Given that $\|\cdot\|_{\#}$ and $\|\cdot\|_{*}$ are vector norms, for any nonzero vector $x \in \mathbb{R}^n$ and matrix $A \in \mathbb{R}^{m\times n}$, we have $\|Ax\|_{\#} \geqslant 0$, and $\|x\|_{*} > 0$. Consequently, $\frac{\|Ax\|_{\#}}{\|x\|_{*}} \geqslant 0$. This implies that $\|A\|_{\#,*} \geqslant 0$ for all matrices $A \in \mathbb{R}^{m\times n}$.

If $A = 0$ (the $m \times n$ zero matrix), then $Ax = 0$ for any vector x, which implies $\|Ax\|_{\#} = 0$. Consequently, $\frac{\|Ax\|_{\#}}{\|x\|_{*}} = 0$ for all $0 \neq x \in \mathbb{R}^n$, and thus, $\|A\|_{\#,*} = 0$. Conversely, assume $\|A\|_{\#,*} = 0$. Since, by Proposition 12.21,

$$0 \leqslant \|Ax\|_{\#} \leqslant \|A\|_{\#,*}\|x\|_{*} = 0,$$

we deduce $\|Ax\|_{\#} = 0$ (and therefore, $Ax = 0$) for all $x \in \mathbb{R}^n$, which implies $A = 0$.

Consider $\alpha \in \mathbb{R}$. We have

$$\|\alpha A\|_{\#,*} = \max_{x\in\mathbb{R}^n} \frac{\|\alpha Ax\|_{\#}}{\|x\|_{*}} = \max_{x\in\mathbb{R}^n} \frac{|\alpha|\, \|Ax\|_{\#}}{\|x\|_{*}} = |\alpha| \max_{x\in\mathbb{R}^n} \frac{\|Ax\|_{\#}}{\|x\|_{*}} = |\alpha|\, \|A\|_{\#,*}.$$

This confirms property (iii).

Finally, for any matrices $A, B \in \mathbb{R}^{m\times n}$ and nonzero vector $x \in \mathbb{R}^n$, we have

$$\begin{aligned}\frac{\|(A+B)x\|_{\#}}{\|x\|_{*}} = \frac{\|Ax+Bx\|_{\#}}{\|x\|_{*}} &\leqslant \frac{\|Ax\|_{\#} + \|Bx\|_{\#}}{\|x\|_{*}} \\ &= \frac{\|Ax\|_{\#}}{\|x\|_{*}} + \frac{\|Bx\|_{\#}}{\|x\|_{*}} \leqslant \|A\|_{\#,*} + \|B\|_{\#,*}.\end{aligned}$$

In the first inequality, we applied the triangle inequality for the vector norm $\|\cdot\|_{\#}$, and for the second inequality, we used the definitions of $\|A\|_{\#,*}$ and $\|B\|_{\#,*}$ Taking the maximum over all nonzero vectors x, we finally conclude $\|A+B\|_{\#,*} \leqslant \|A\|_{\#,*} + \|B\|_{\#,*}$, which is the triangle inequality. □

12.2.3 The submultiplicativity property

Norms are defined on vector spaces, where we can add vectors and scale them. But matrices can also be multiplied (provided their dimensions are compatible). So, when defining norms on matrices, it is natural to hope that the norm interacts well with matrix multiplication. The next definition makes this idea precise.

Definition 12.23. (The submultiplicative property)

A matrix norm $\|\cdot\|$ is **submultiplicative** (also called **consistent**) if $\|AB\| \leqslant \|A\|\,\|B\|$ whenever the product AB is defined.

Simply put, Definition 12.23 says the norm of a product should not exceed the product of the matrix norms. This concept is analogous to the triangle inequality, but for matrix multiplication.

Proposition 12.24. (Properties of submultiplicative matrix norms)

For any submultiplicative matrix norm $\|\cdot\|$ on $\mathbb{R}^{n\times n}$, the following hold:

(a) $\|A^k\| \leqslant \|A\|^k$, for all positive integers k and any matrix $A \in \mathbb{R}^{n\times n}$; and
(b) $\|I_n\| \geqslant 1$.

Proof We prove part (a) by induction on the exponent k. The property holds clearly for $k = 1$. Assume it is true for a fixed k. Then,

$$\|A^{k+1}\| = \|AA^k\| \leqslant \|A\|\|A^k\| \leqslant \|A\|\|A\|^k = \|A\|^{k+1},$$

where the inequalities follow from the submultiplicative property and the inductive hypothesis, respectively.

In particular,

$$\|I_n\| = \|I_n^2\| \leqslant \|I_n\|^2. \tag{12.7}$$

Since $I_n \neq 0$, we have $\|I_n\| > 0$. Hence, dividing (12.7) by $\|I_n\|$, part (b) follows. □

Theorem 12.25 establishes that the induced matrix norm (12.4) is submultiplicative for any vector norm $\|\cdot\|$. In particular, it follows that every matrix p-norm $\|\cdot\|_p$ is submultiplicative for $p \geqslant 1$.

Theorem 12.25. (Induced matrix norms are submultiplicative)

For a given vector norm $\|\cdot\|$, the induced matrix norm (12.4) is submultiplicative.

Proof For any matrices $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$, our goal is to show that $\|AB\| \leqslant \|A\|\,\|B\|$.

Let y be a nonzero vector where the value of $\max_{x\neq 0} \frac{\|ABx\|}{\|x\|}$ is achieved. Then, by Proposition 12.21, we have

$$\begin{aligned}\|AB\| = \frac{\|ABy\|}{\|y\|} &\leqslant \|A\|\frac{\|By\|}{\|y\|} \\ &\leqslant \|A\| \max_{x\neq 0} \frac{\|Bx\|}{\|x\|} = \|A\|\,\|B\|,\end{aligned}$$

which concludes the proof. □

What about the vectorized p-norms $|||\cdot|||_p$ in (12.3)? Are they submultiplicative? It turns out that for some values of p they are, and for others they are not. Here, we consider only the cases $p = 1, 2, \infty$, which are the norms of most interest.

Theorem 12.26. (Submultiplicativity of some vectorized matrix norms)

The matrix norm $|||\cdot|||_p$ defined in (12.3) is submultiplicative for $p = 1$ and $p = 2$, but the infinite-norm is not submultiplicative.

Proof Let $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times q}$. For $p = 1$, we have

$$\begin{aligned}|||AB|||_1 &= \sum_{i=1}^{m}\sum_{j=1}^{q} |(AB)_{ij}| = \sum_{i=1}^{m}\sum_{j=1}^{q}\left|\sum_{k=1}^{n} A_{ik}B_{kj}\right| \\ &\leqslant \sum_{i=1}^{m}\sum_{j=1}^{q}\sum_{k=1}^{n} |A_{ik}||B_{kj}| \leqslant \sum_{i=1}^{m}\sum_{j=1}^{q}\sum_{k=1}^{n}\sum_{\ell=1}^{n} |A_{ik}|\,|B_{\ell j}| \\ &= \sum_{i=1}^{m}\sum_{k=1}^{n} |A_{ik}| \sum_{\ell=1}^{n}\sum_{j=1}^{q} |B_{\ell k}| = |||A|||_1\,|||B|||_1.\end{aligned}$$

In the first inequality, we use the absolute value triangle inequality, and in the second inequality, we bound $|B_{kj}|$ with $|B_{1j}| + \cdots + |B_{nj}|$.

Now, let us prove submultiplicativity for $p = 2$ (i.e., the Frobenius norm). We have

$$\begin{aligned}|||AB|||_2 = \|AB\|_F &= \left(\sum_{i=1}^{m}\sum_{j=1}^{q} |(AB)_{ij}|^2\right)^{1/2} = \left(\sum_{i=1}^{m}\sum_{j=1}^{q}\left|\sum_{k=1}^{n} A_{ik}B_{kj}\right|^2\right)^{1/2} \\ &\leqslant \left(\sum_{i=1}^{m}\sum_{j=1}^{q}\left(\sum_{k=1}^{n} |A_{ik}|^2\right)\left(\sum_{\ell=1}^{n} |B_{\ell j}|^2\right)\right)^{1/2} \\ &= \left(\sum_{i=1}^{m}\sum_{k=1}^{n} |A_{ik}|^2 \sum_{\ell=1}^{n}\sum_{j=1}^{q} |B_{\ell j}|^2\right)^{1/2} = \|A\|_F\|B\|_F,\end{aligned}$$

where in the first inequality, we use the Cauchy-Schwarz inequality.

Lastly, for $p = \infty$, we prove that submultiplicativity does not hold. Consider the matrices

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

The product of these matrices is $AB = \left[\begin{smallmatrix} 0 & 2 \\ 0 & 0 \end{smallmatrix}\right]$. Therefore, $|||AB|||_\infty = 2$. However, $|||A|||_\infty = |||B|||_\infty = 1$. Hence, the submultiplicativity property does not hold in this case. □

Despite the fact that the matrix norm $|||\cdot|||_\infty$ is not submultiplicative, it still exhibits a quasi-submultiplicative property (as you will see in Problem 12.26).

The more general induced matrix norm in (12.6) is typically not submultiplicative. However, in Problem 12.28, you will find some properties of induced matrix norms that resemble submultiplicativity.

12.3 Distance functions

In everyday life, we often think of distance as the space between two points, like on a map or in three-dimensional space. In mathematics, we generalize this idea to more abstract settings. In this section, we introduce the formal definition of a distance function and present several important examples. Unlike norms, which apply only to vector spaces, a distance function can be defined on any nonempty set. For instance, we will consider distances between graph nodes, between strings of symbols, and between collections of sets.

Definition 12.27. (Distance function on a set X)

Let X be a nonempty set. A function $d : X \times X \to \mathbb{R}$ is called a **distance** if it satisfies the following properties for all points $x, y, z \in X$:

(i) $d(x, y) = d(y, x)$ (symmetry);
(ii) $d(x, y) \geqslant 0$, and $d(x, y) = 0$ if and only if $x = y$;
(iii) $d(x, y) \leqslant d(x, z) + d(z, y)$ (triangle inequality).

We refer to conditions (i)-(iii) in Definition 12.27 as the **distance axioms**.

The third condition, known as the **triangle inequality**, states that traveling from point x to point y through another point z will always cover at least as much distance as going directly from x to y. This principle applies even when you have multiple stops. For example, if you travel from x to y via points $z_1, z_2, \ldots, z_n$, we have

$$\begin{aligned} d(x, y) &\leqslant d(x, z_1) + d(z_1, y) \\ &\leqslant d(x, z_1) + d(z_1, z_2) + d(z_2, y) \\ &\leqslant d(x, z_1) + d(z_1, z_2) + d(z_2, z_3) + d(z_3, y) \\ &\;\;\vdots \\ &\leqslant d(x, z_1) + d(z_1, z_2) + d(z_2, z_3) + \cdots + d(z_n, y). \end{aligned}$$

12.3.1 Distance induced by a norm

In this section, we use norms to define distances between vectors or matrices. Theorem 12.29 shows that indeed every norm defines a distance function.

Definition 12.28. (Distance induced by a norm)

Given a norm $\|\cdot\|$, the function

$$d(u, v) := \|u - v\|$$

is called the **distance induced by the norm** .

Using the norm axioms, we can prove that the distance function induced by a norm is, in fact, a distance function.

Theorem 12.29. (The distance induced by a norm satisfies the distance axioms)

For any norm $\|\cdot\|$, the function $d(u, v) = \|u - v\|$ is a distance function.

Proof Let V be a real vector space. Using property (iii) of a norm, for any $u, v \in V$ we have

$$\begin{aligned} d(u, v) &= \|u - v\| \\ &= \|(-1)(v - u)\| \\ &= |-1|\,\|v - u\| = d(v, u), \end{aligned}$$

which shows the symmetry property.

The nonnegativity of the distance comes directly from property (i) of a norm. Also, from property (ii) of a norm, $d(u, v) = \|u - v\|$ is zero if and only if $u - v = 0$, which is the same as saying $u = v$.

Finally, for any $w \in V$, we have

$$\begin{aligned} d(u, v) &= \|u - v\| \\ &= \|u - w + w - v\| \\ &\leqslant \|u - w\| + \|w - v\| = d(u, w) + d(w, v), \end{aligned}$$

where the inequality follows from the triangle inequality for the norm $\|\cdot\|$. □

Let us look at some examples of distances induced by vector norms.

Example 12.30. (Euclidean and Taxicab distances)

Using the 1- and 2-vector norms, we can define the Taxicab (or Manhattan) and Euclidean distances, respectively, as

$$d_1(u,v) = \|u-v\|_1 = \sum_{i=1}^{n} |u_i - v_i|, \quad \text{and}$$
$$d_2(u,v) = \|u-v\|_2 = \left(\sum_{i=1}^{n} (u_i - v_i)^2\right)^{1/2},$$

where $u, v \in \mathbb{R}^n$. Here, u_i and v_i represent the ith entry of vectors u and v.

Example 12.31 Consider the vectors $u = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and $v = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$. To find the Taxicab and Euclidean distances between u and v, we compute the 1- and 2- norms of the difference $u - v = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$:

$$\|u-v\|_1 = |-2| + |2| = 4, \quad \text{and}$$
$$\|u-v\|_2 = \sqrt{(-2)^2 + 2^2} = 2\sqrt{2}.$$

The Euclidean distance matches our natural sense of how far apart two points are in space. In Figure 12.5, we see how the Euclidean distance between two vectors u and v in $\mathbb{R}^2$ corresponds to the length of the vector $u - v$.

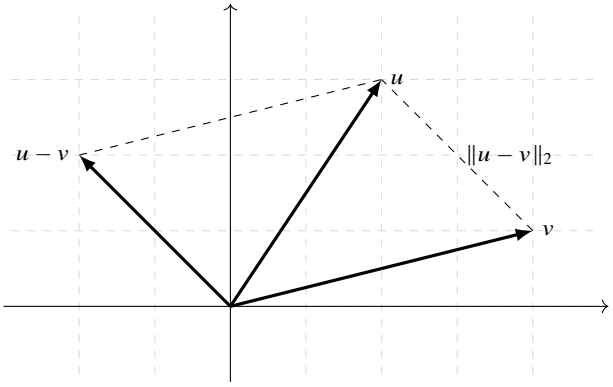


Fig. 12.5: Visualizing the Euclidean distance between vectors u and v using the parallelogram law in $\mathbb{R}^2$. Vectors u and v form two sides of the parallelogram, and their difference $u - v$ is shown as its diagonal.

The Taxicab distance is particularly useful for measuring distances within a grid-based city layout, like the paths pedestrians might take when walking. Given that walking straight through buildings is not an option, the true distance is determined by the length of the streets traversed between two points.

In the following example, we compare the use of the Euclidean and Taxicab distances.

Booking a hotel: Euclidean distance vs Taxicab distance

Suppose first that your family and your best friend's family are visiting a city together, and there are three hotels downtown. They are represented as the points A, B, C in the scaled version of the city presented in Figure 12.6.

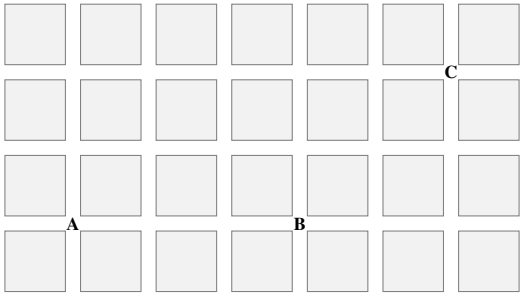


Fig. 12.6: Scaled version of a city and the position of three hotels represented by $A = (1, 1)$, $B = (4, 1)$ and $C = (6, 3)$.

None of the hotels has two available rooms, so each family needs to book a room at a different hotel. Logically, we want the two hotels to be the closest possible. What hotels should you book?

One possible way of measuring the distance between the hotels is using a ruler to measure the distance between pairs of points, that is, to use the Euclidean distance. By doing this, we obtain the following results:

Pairs of hotels	Euclidean distance
Hotel B, Hotel C	$\sqrt{8} \approx 2.83$
Hotel A, Hotel B	3
Hotel A, Hotel C	$\sqrt{29} \approx 5.38$

Thus, using the Euclidean measure, Hotels B and C are the closest. In practice, if we want to measure distances between sites in a city, the Euclidean distance is not ideal since the existence of buildings may prevent us from walking in a straight line. Thus, it is more appropriate to use the Taxicab distance. Using this distance, we obtain the following results:

Pairs of hotels	Taxicab distance
Hotel B, Hotel C	4
Hotel A, Hotel B	3
Hotel A, Hotel C	7

We find that Hotels A and B are the closest pair under the 1-norm, which differs from the result obtained using the Euclidean distance.

While many distance functions in $\mathbb{R}^n$ come from vector norms, not all do (see Problems 12.30, 12.38 and 12.39 for an example of a non-norm-based distance and a characterization of the distances that are induced by norms.).

Similarly to how we have defined distances in $\mathbb{R}^n$ induced by vector norms, we can define distances in $\mathbb{R}^{m\times n}$ induced by matrix norms.

Example 12.32. (Comparing images with matrix norms)

Suppose you are building a simple image recognition or clustering system using grayscale images of the same size (for instance, 28×28 pixel images like those in the MNIST dataset [37]).

In Chapter 3, we saw how grayscale images can be represented as matrices, where each entry gives the intensity of a pixel. To compare two images A and B, you can measure the distance between them using a matrix norm:

$$d(A, B) = \|A - B\|.$$

Different matrix norms capture different ways of measuring similarity:

(i) **Frobenius norm:**

$$\|A - B\|_F = \left(\sum_{i,j} |A_{ij} - B_{ij}|^2\right)^{1/2}$$

This measures the overall pixel-by-pixel difference, treating the image as a flat list of values. It is often used in tasks like denoising, reconstruction, or compression.

(ii) **Infinity-norm:**

$$\|A - B\|_\infty = \max_i \sum_j |A_{ij} - B_{ij}|$$

This looks at the row (a horizontal scanline) with the biggest total difference. It can be useful when horizontal structure matters.

(iii) **1-norm:**

$$\|A - B\|_1 = \max_j \sum_i |A_{ij} - B_{ij}|$$

This focuses on the column (a vertical slice) with the biggest total difference. It helps when vertical patterns are important.

To summarize:

Norm	What it focuses on	When it is useful
Frobenius	Overall pixel-by-pixel difference	General similarity
Infinity-norm	Row with the largest difference	Detecting horizontal features
1-norm	Column with the largest difference	Detecting vertical features

12.3.2 The angular distance

The angular distance measures the separation between intersecting lines, rays, and vectors sharing a common origin in $\mathbb{R}^n$. Unlike distances from norms, angular distance is based on the angle between vectors and is not induced by a norm.

To compute the angular distance, we consider the angle between two unit vectors pointing in the same directions as the given lines, rays, or vectors.

Definition 12.33. (Angular distance between unit vectors)

We define the **angular distance** between two unit vectors u and v in $\mathbb{R}^n$ as the angle between them, that is,

$$\theta(u, v) = \arccos\left(u^T v\right).$$

It is straightforward to verify that the angular distance satisfies the first two distance axioms (i) $\theta(u, v) = \theta(v, u)$ and (ii) $\theta(u, v) \geqslant 0$, and $\theta(u, v) = 0$ if and only if $u = v$. The fact that angles between vectors in $\mathbb{R}^2$ and $\mathbb{R}^3$ satisfy the triangle inequality (axiom (iii)) can be traced back to Euclid's Proposition XI.20 in "The Elements", and it also holds in $\mathbb{R}^n$. Problem 12.41 guides you through proving that angles satisfy the triangle inequality in the simplest case of unit vectors in $\mathbb{R}^2$.

Next we present an example that compares the use of the Euclidean distance and the angular distance in a "real life" situation.

Boats in the night: Euclidean distance vs angular distance

On a dark night at sea, you spot the lights of three distant ships. From your perspective, which pair appears closest together?

Imagine your boat is located at the origin in$\mathbb{R}^2$, and the three ships are represented as colored points in Figure 12.7, labeled Ship A, Ship B, and Ship C.

At night, it's difficult to judge how far away the ships are; they all appear to lie on a circle centered around your position. What stands out is not their distance, but their direction relative to a reference point. What you notice is how much you need to turn to see each ship in relation to that reference.

In this situation, it is better to use angular distance instead of Euclidean distance. We measure the distance between points based on the angle formed by each pair of vectors. The table below shows both the Euclidean and angular distances:

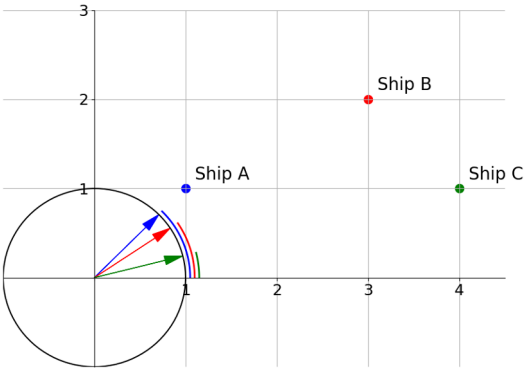


Fig. 12.7: Points on the circle are the perceived positions of the ships in the dark.

Pairs of ships	Angular distance	Euclidean distance
Ship B, Ship A	0.1974	$\sqrt{5}$
Ship C, Ship B	0.3430	$\sqrt{2}$
Ship A, Ship C	0.5404	3

Ships A and B are closest when using angular distance. This is different from the closest pair found using Euclidean distance, which is ships B and C. This highlights how different situations require different methods for measuring distance between vectors.

12.3.3 Spherical distance

The spherical distance measures how far apart two points on a sphere in $\mathbb{R}^3$ are along its surface.

Imagine you have two points, u and v, on the surface of a sphere with radius R, as shown in Figure 12.8. The **spherical distance**, also known as the **great-circle distance**, is the shortest path between u and v along the sphere's surface, rather than a straight line through the sphere.

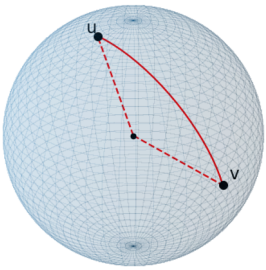


Fig. 12.8: The spherical distance between two points u and v on a sphere.

A sphere's great circle is the intersection of a plane passing through the sphere's center and its boundary. The spherical distance between points u and v is the length of the shortest arc on this great circle connecting them.

You can calculate this length using the sphere's radius and the angle $\theta(u, v)$ between the vectors u and v:

$$R\,\theta(u, v) = R \arccos\left(\frac{u^T v}{\|u\|_2 \|v\|_2}\right) = R \arccos\left(\frac{u^T v}{R^2}\right).$$

When $R = 1$, the spherical distance equals the angular distance.

The spherical distance is often used in navigation and to calculate flight distances between countries. Using Euclidean distance (straight-line distances) in these cases would misrepresent the path, as it ignores the Earth's curvature. Although the Earth is not a perfect sphere, this method still gives a good estimate.

12.3.4 The Hamming and Levenshtein distances

In some cases, we need to measure the distance between two strings of numbers, letters, or symbols. For example, to compare the words "camping" and "bumping", we can count how many letters differ in the same positions. Both words have seven letters, and they differ in two positions: 1 and 2. This concept leads us to the following definition.

Definition 12.34. (The Hamming distance [28])

Given two strings $u = (u_1, u_2, \ldots, u_n)$ and $v = (v_1, v_2, \ldots, v_n)$ of the same length n, the **Hamming distance** between u and v is defined to be the number of positions at which the corresponding symbols differ, that is,

$$d_H(u, v) = \sum_{i=1}^{n} [u_i \neq v_i],$$

where $[\cdot]$ denotes the Iverson bracket, which takes value 1 if its argument is true, and 0 otherwise.

Example 12.35 Consider the two binary strings $u = (1, 1, 0, 0, 0, 0, 1)$ and $v = (1, 1, 1, 0, 0, 0, 1)$, both of length seven. Their Hamming distance is 1, as they differ only at the third position.

Next, we will show that the Hamming distance for strings of fixed length n satisfies the three distance axioms.

Theorem 12.36. (The Hamming distance satisfies the distance axioms)

The Hamming distance on the set of strings of length $n \geqslant 1$ is a distance function.

Proof Let d_H denote the Hamming distance function. From the definition, it is clear that the Hamming distance is symmetric, that is, $d_H(u, v) = d_H(v, u)$. Moreover, since the Hamming distance counts the differing entries between two vectors, its value is always a nonnegative integer. Additionally, d_H is zero precisely when both strings match completely.

To prove the triangle inequality for the Hamming distance, consider three strings u, v and w of the same length n. We aim to show that $d_H(u, v) \leqslant d_H(u, w) + d_H(w, v)$. Whenever u and v have a mismatch at a certain position i, it means that either u and w mismatch at that position, or w and v do. This implies that the count of mismatches between u and v cannot exceed the combined count of mismatches between u and w and between w and v. Thus, $d_H(u, v) \leqslant d_H(u, w) + d_H(w, v)$. □

The Hamming distance is simple and useful in many applications, but it has limitations. Let us consider the following situation.

Example 12.37. (Handling typos in police complaint records)

After New York State repealed the law that kept police disciplinary records secret, ProPublica obtained data from New York City's Civilian Complaint Review Board, which investigates public complaints against NYPD officers. The resulting database contains over 12,000 civilian complaints involving roughly 4,000 unique officer names [45]. Inevitably, typos and minor variations appear in these records, so we need a way to identify when two slightly different entries actually refer to the same person.

Each officer's name is stored as a string of characters, letters, plus a comma separating last and first names. For example, the database might list both "Gonzalez,Marcos" and "Gonzalez,Marco" as separate entries. Because these two strings have different lengths, the Hamming distance (which compares only equally long strings) cannot detect that they are nearly identical.

The example above shows that some applications require a more flexible string distance function, one that can handle strings of different lengths.

Definition 12.38. (Levenshtein distance [38])

Given two strings $u = (u_1, u_2, \ldots, u_n)$ and $v = (v_1, v_2, \ldots, v_m)$ of possibly different lengths, the **Levenshtein distance** between u and v, denoted by $d_L(u, v)$, is the minimum number of edits needed to transform one string into the other. An edit can be: (1) changing one character to another single character; (2) deleting one character from the given string; (3) inserting a single character into the given string.

Example 12.39 Going back to the example about similar names, the Levenshtein distance between the strings "Gonzalez, Marcos" and "Gonzalez, Marco" is 1 since, by deleting the letter "s" in the first string, we obtain the second one.

Example 12.40 Consider the two strings "rain" and "shine". To transform "rain" into "shine", we can replace "r" with "s", replace "a" with "h", and insert "e". No sequence of edits (substitutions, insertions, or deletions) can achieve this in fewer than three steps, so the Levenshtein distance is 3.

Computing the Levenshtein distance by hand is difficult because there are many possible edit sequences. Fortunately, efficient algorithms exist (e.g., Wagner-Fischer [49]), and most languages offer libraries or built-in functions to compute it automatically.

Let us now show that the Levenshtein distance is indeed a distance function.

Theorem 12.41. (The Levenshtein distance satisfies the distance axioms)

The Levenshtein distance on the set of strings of any length is a distance function.

Proof The Levenshtein distance $d_L(u, v)$ between two strings u and v is always nonnegative because the total number of edits required to transform u into v cannot be negative. Furthermore, $d_L(u, v) = 0$ if and only if u and v are identical, as no edits are needed to transform a string into itself. This establishes that the Levenshtein distance satisfies the distance axioms (i) and (ii).

To prove the triangle inequality, consider three strings u, v and w. The Levenshtein distance $d_L(u, w)$ is the minimum number of edits required to transform u into w. Transforming u into v takes $d_L(u, v)$ edits, and transforming v into w takes $d_L(v, w)$ edits. Thus, $d_L(u, w)$ is at most $d_L(u, v) + d_L(v, w)$, since combining these transformations provides a valid, though possibly not minimal, way to transform u to w. Hence, $d_L(u, w) \leqslant d_L(u, v) + d_L(v, w)$. □

Remark 12.42. (Hamming vs Levenshtein)

When two strings have the same length, you can use either the Hamming or the Levenshtein distance. The Hamming distance is always an upper bound on the Levenshtein distance. For example, consider the words "card" and "arcs". Their Hamming distance is 4, while the Levenshtein distance is 3: delete the first letter of "card", change "d" to "c", and add "s".

Example 12.43. (Handling typos in police complaint records revisited)

Returning to the officer complaints data in Example 12.37, we computed the Levenshtein distance for every pair of names. Table 12.1 lists all name pairs whose Levenshtein distance equals 1.

Name 1	Name 2	Name 1	Name 2
Wilson, James	Gilson, James	Martinez, Louis	Martinez, Luis
Gonzalez, Marcos	Gonzalez, Marco	Vasquez, Angel	Vazquez, Angel
Gomez, Alvin	Gomez, Elvin	Garcia, Roberto	Garcia, Robert
Barresi, Anthony	Baresi, Anthony	Campbell, Dewan	Campbell, Dean
Fernandez, Antonio	Fernandes, Antonio	Ortiz, Erick	Ortiz, Eric
Cohan, Steven	Cohn, Steven	Raso, Michael	Riso, Michael
Leek, Daniel	Lee, Daniel	King, Darrien	King, Darren
Rodriguez, Andres	Rodriguez, Andrew	Ramos, Adrian	Ramon, Adrian
Ross, Joseph	Foss, Joseph	Colon, Angel	Rolon, Angel
Park, Michael	Parks, Michael	Thomas, Juby	Thomas, Joby

Table 12.1: Name pairs with the smallest nonzero Levenshtein distance.

Using the Levenshtein distance, we found similar names. However, we cannot determine if they are typos or correctly but similarly spelled names. A human should double-check all these matches.

There are other methods for measuring distances between strings, like the Damerau-Levenshtein distance, Jaro distance, and Jaro-Winkler distance. We encourage you to look into these methods to learn when to use each one.

12.3.5 The Jaccard distance

In this section, we introduce the Jaccard distance, a measure of how different two sets are.

First, let us review some key definitions related to sets.

Definition 12.44. (Size, union and intersection of sets)

Given two finite sets A and B, then

(i) $|A|$ denotes the number of elements in the set A;
(ii) $A \cap B$ denotes the intersection of A and B, that is, $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$; and
(iii) $A \cup B$ denotes the union of A and B, that is, $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.

The Jaccard distance measures how different two sets are. Unlike Hamming and Levenshtein distances, which look at ordered strings, Jaccard distance works with sets where order does not matter.

Definition 12.45. (Jaccard distance)

Given two finite sets A and B, the **Jaccard distance** between A and B is defined as

$$d_j(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|},$$

if at least one of A or B is nonempty, and $d_j(\varnothing, \varnothing) = 0$, where $\varnothing$ denotes the empty set.

The Jaccard distance measures the similarity between two sets based on their shared elements compared to their total unique elements. It ranges from 0 to 1: a distance of 0 indicates the sets are identical, while a distance of 1 means they have no elements in common. For any nonempty set A and the empty set, the Jaccard distance is always 1.

For example, the Jaccard distance is useful for comparing lists.

Example 12.46. (The shopping list)

Imagine that you have prepared a shopping list for the week and one of your roommates goes shopping but forgets to take it with them. Here is the list that you prepared and the list of items that your roommate ended up buying:

Your list: {milk, chicken, potatoes, bananas, oranges, almonds, carrots, lettuce, paper towels, napkins, lemon juice}.

What your roommate bought: {milk, chips, Cheetos, pizza, chicken, peanuts, ice-cream, beer, paper towels}.

It looks like you made healthier choices than your roommate! Anyways, how similar is their list compared to yours? Let us call A your list, and B your roommate's. Then, the Jaccard distance between the two lists is

$$d_j(A, B) = 1 - \frac{3}{17} \approx 0.82,$$

which is close to 1. Thus, the two lists are not very similar. Your roommate did a horrible job!

Is the Jaccard distance a distance function? It clearly satisfies the distance axioms (i) and (ii). In Proposition 12.47, we prove it also satisfies the triangle inequality. While the proof is elemental, it is not immediately obvious. To establish this, we will use the following two inequalities (see Problem 12.55):

$$|A \cap C|\,|B \cup C| + |A \cup C|\,|B \cap C| \leqslant |C|\,(|A| + |B|), \quad \text{and} \tag{12.8}$$

$$|A \cap B|\,|A \cup B| \leqslant |A|\,|B|. \tag{12.9}$$

These inequalities hold for all finite sets A, B, and C. Another important property we will need is that $|A| \leqslant |B|$ whenever $A \subseteq B$, with A and B finite sets.

Proposition 12.47. (The Jaccard distance satisfies the triangle inequality)

For any finite sets A, B, and C, we have

$$d_j(A,B) \leqslant d_j(A,C) + d_j(C,B).$$

Proof When at least one of the sets A, B, or C is the empty set, checking that the Jaccard distance satisfies the triangular inequality is straightforward since $d_j(S,\varnothing) = 1$ for all sets S. So let us assume that the three sets A, B and C are nonempty. Then, we have

$$\begin{aligned}
\frac{|A\cap C|}{|A\cup C|} + \frac{|B\cap C|}{|B\cup C|} &= \frac{|A\cap C|\,|B\cup C| + |A\cup C|\,|B\cap C|}{|A\cup C|\,|B\cup C|} \\
&\leqslant \frac{|C|(|A|+|B|)}{|A\cup C|\,|B\cup C|} \\
&\quad \text{(by (12.8))} \\
&\leqslant \frac{|C|(|A|+|B|)}{|(A\cup C)\cap(B\cup C)|\,|(A\cup C)\cup(B\cup C)|} \\
&\quad \text{(by (12.9))} \\
&\leqslant \frac{|C|(|A|+|B|)}{|(A\cap B)\cup C|\,|A\cup B|} \\
&\quad \text{(since } A\cup B\subseteq A\cup B\cup C\text{)} \\
&\leqslant \frac{|C|(|A|+|B|)}{|C|\,|A\cup B|} \\
&\quad \text{(since } C\subseteq (A\cap B)\cup C\text{)} \\
&= \frac{|A|+|B|}{|A\cup B|}.
\end{aligned}$$

The above inequality implies

$$\begin{aligned}
d_j(A,C) + d_j(C,B) &= 2 - \frac{|A\cap C|}{|A\cup C|} - \frac{|B\cap C|}{|B\cup C|} \\
&\geqslant 2 - \frac{|A|+|B|}{|A\cup B|} \\
&= 1 + \frac{|A\cup B|}{|A\cup B|} - \frac{|A|+|B|}{|A\cup B|} \\
&= 1 + \frac{|A\cap B|}{|A\cup B|} = d_j(A,B),
\end{aligned}$$

which is the triangle inequality. □

12.3.6 Distance between nodes in undirected graphs

In a graph, we define a notion of "distance" to quantify how closely nodes are connected. Before defining a distance between nodes, let us review a related idea: the length of a walk.

In Part II, we measured walk length in unweighted graphs by counting edges. Now we extend that concept to weighted graphs, where length depends on both the number of edges and their weights.

Definition 12.48. (The length of a walk)

Given a weighted graph $G(V, E, w)$ and a walk from node u to node v defined by the vertex sequence

$$u = v_0, v_1, \ldots, v_\ell = v,$$

with corresponding edges $e_j = (v_{j-1}, v_j)$ and weights $w(e_j)$, the length of the walk is defined as

$$\sum_{j=1}^{\ell} w(e_j).$$

Example 12.49 In Figure 12.9, we have a weighted graph with generic weights. Consider the walk from vertex 1 to vertex 5, highlighted in the figure, which follows the sequence of vertices $1, 2, 4, 5$. The length of this walk is given by the sum of the edge weights: $w_{12} + w_{24} + w_{45}$.

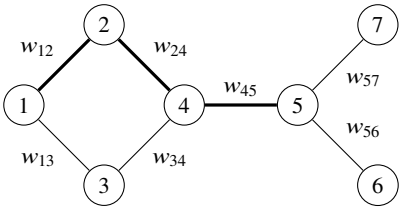


Fig. 12.9: A weighted graph.

Note that Definition 12.48 agrees with the concept of walk length in an unweighted graph from Part II, since an unweighted graph is just a weighted graph with all edge-weights equal to 1. In that case, walk length reduces to the number of edges.

A path is a particular case of a walk, that is, a walk with no repeated nodes. Thus, the definition of length of a walk also applies to paths.

Also, if there is a walk between nodes, then there is also a path connecting them.

Now that we have defined the length of a walk (and hence of a path), we define the distance between nodes in a weighted undirected connected graph. A graph is called **connected** if, for every pair of nodes $u, v \in V$, there exists a path connecting u and v.

Definition 12.50. (Distance between nodes in a weighted undirected connected graph)

Let $G = (V, E, w)$ be a weighted undirected connected graph, where the edge weights are positive. For any two nodes $u, v \in V$, we define the distance between u and v, denoted by $d_G(u, v)$, as the minimum of the lengths of all paths connecting u and v.

Remark 12.51 Since the trivial path from a node u to itself has length 0, we have $d_G(u, u) = 0$ for every $u \in V$.

If G is not connected, then there may exist nodes u and v for which no path connects u to v. In that case, one often defines $d_G(u, v) = \infty$. However, with this convention, d_G is no longer a distance in our sense, since it does not take values in $\mathbb{R}$.

You might ask why Definition 12.50 applies only to undirected graphs. In undirected graphs, distance satisfies all three axioms, including symmetry. In a directed graph, however, the shortest path from u to v may differ from the shortest path from v to u, or one or both paths might not exist, so symmetry can fail.

Example 12.52 Consider the unweighted, undirected graph G shown in the left panel of Figure 12.10. The shortest path from node 1 to node 5 is highlighted in the right panel. Since each edge in an unweighted graph has a unit weight, the distance between nodes 1 and 5 is $d_G(1, 5) = 3$.

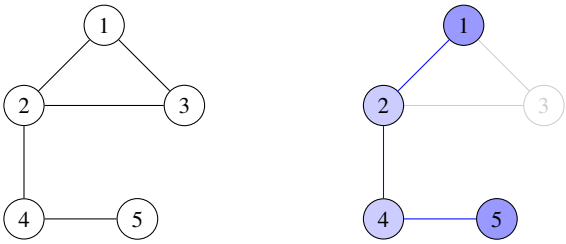


Fig. 12.10: A toy graph with five nodes (left), and the shortest path between nodes 1 and 5 (right).

There are efficient algorithms (Dijkstra's [16], Bellman–Ford [8], and Floyd-Warshall [24]) that find shortest paths in a graph. While we will not cover these

methods here, the next theorem shows that, for an unweighted, undirected, connected graph, powers of the adjacency matrix alone can determine the shortest-path length between two nodes.

Theorem 12.53. (The adjacency matrix and distance between nodes)

Consider an unweighted, undirected, connected graph $G = (V, E)$ with adjacency matrix A. Let v_i denote the ith node of G. The distance between nodes v_i and v_j is given by

$$d_G(v_i, v_j) = \min\{k \mid \text{the } (i,j)\text{th entry of } A^k \text{ is nonzero}\}.$$

Proof By Theorem 9.3, the (i, j)th entry of A^k counts the number of walks of length k between nodes v_i and v_j. Thus, $(A^k)_{ij} \neq 0$ if and only if there exists a walk of length k from v_i to v_j.

Let

$$m = \min\{k \mid (A^k)_{ij} \neq 0\}.$$

Then there exists a walk of length m from v_i to v_j. Since every walk between two nodes contains a path connecting the same nodes, there exists a path from v_i to v_j of length at most m. Hence,

$$d_G(v_i, v_j) \leqslant m.$$

On the other hand, if $d_G(v_i, v_j) = r$, then there exists a path of length r from v_i to v_j. A path is, in particular, a walk, so $(A^r)_{ij} \neq 0$. Therefore,

$$m \leqslant r = d_G(v_i, v_j).$$

We conclude that

$$d_G(v_i, v_j) = m = \min\{k \mid (A^k)_{ij} \neq 0\}.$$

Example 12.54 Consider the graph shown in the left panel of Figure 12.10. Its adjacency matrix A and some of its powers are

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \quad A^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 0 \\ 1 & 3 & 1 & 0 & 1 \\ 1 & 1 & 2 & 1 & 0 \\ 1 & 0 & 1 & 2 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{bmatrix}, \quad \text{and} \quad A^3 = \begin{bmatrix} 2 & 4 & 3 & 1 & 1 \\ 4 & 2 & 4 & 4 & 0 \\ 3 & 4 & 2 & 1 & 1 \\ 1 & 4 & 1 & 0 & 2 \\ 1 & 0 & 1 & 2 & 0 \end{bmatrix}.$$

In each matrix, the $(1, 5)$ entry is highlighted in gray. Notice that this entry is zero in A and A^2, but becomes nonzero in A^3. This indicates that there are no walks of length 1 or 2 between nodes 1 and 5, while there is at least one walk of length 3. Since the shortest walk between two nodes is always a path, the shortest path between nodes 1 and 5 has a length of 3.

12.4 Similarity functions

In many applications, we need a quantitative way to compare how alike two objects are. In this section, we introduce similarity functions for this purpose.

Like distance functions, similarity functions can be defined not only for vectors but also for other mathematical objects.

Definition 12.55. (Similarity function on a set X)

A **similarity** $s : X \times X \to \mathbb{R}$ is a function that satisfies the following properties for all points $x, y, z \in X$:

(i) $s(x, x) = s(y, y)$;
(ii) $s(x, y) < s(x, x)$;
(iii) $s(x, y) = s(y, x)$; and
(iv) if $s(x, y)$ and $s(y, z)$ are both "high", then $s(x, z)$ should also be "high".

Property (iv) is somewhat vague, but it hints at how similarity should behave. It implies transitivity: if x is similar to y and y is similar to z, then x should be similar to z.

As with distance, there are multiple approaches to defining a similarity function. In Section 13.3, we show how to construct a similarity function from a distance function. In the next subsections, we examine different types of similarity measures, such as cosine similarity and Jaccard similarity.

12.4.1 Cosine similarity

Cosine similarity, widely used in information retrieval, image processing, and natural language processing, measures how similar two vectors are focusing on direction over magnitude.

Definition 12.56. (Cosine similarity)

Given two nonzero vectors u and v in $\mathbb{R}^n$, the **cosine similarity** between them is defined as

$$s_{\cos}(u, v) := \cos(\theta(u, v)) = \frac{u^T v}{\|u\|_2 \|v\|_2}, \tag{12.10}$$

where $\theta(u, v)$ is the angle between u and v (as defined in (11.4)).

Cosine similarity ranges from -1 to 1. A value of 1 means the vectors point in the same direction, 0 means they are perpendicular, and -1 means they point in

opposite directions. It relates to angular distance but is simpler since it does not require computing the arccosine.

Cosine similarity clearly satisfies properties (i) and (iii). To check property (iv), we can show that

$$s_{\cos}(u,v) \geqslant s_{\cos}(u,w)s_{\cos}(w,v) - \left(1-(s_{\cos}(u,w))^2\right)^{1/2}\left(1-(s_{\cos}(w,v))^2\right)^{1/2},$$

for any three vectors $u, v, w \in \mathbb{R}^n$. This means that if both $s_{\cos}(u,w)$ and $s_{\cos}(w,v)$ are close to 1, then $s_{\cos}(u,v)$ is also close to 1.

However, cosine similarity does not satisfy property (ii) since distinct vectors in the same direction have cosine similarity 1, so it is technically not a similarity measure. Nonetheless, it remains widely used[1].

One main application of cosine similarity is measuring the similarity between documents.

Example 12.57. (Resume matching for job posting)

Imagine you are on the HR team at a tech company receiving many software developer resumes. To make the process easier, you decide to use a document similarity technique to find candidates who closely match the job requirements.

Job Description:

- "We are seeking a software developer with a good understanding of **Python**. The candidate should be proficient in working with **Databases** and have a foundational knowledge of **Machine Learning**."

Sample Resumes:

- Resume A: "I am a developer experienced in **Python** programming. I have worked with various **Databases** and have completed a project in **Machine Learning**."
- Resume B: As a seasoned software developer, I have consistently employed **Python** in numerous undertakings, including a recent project built entirely in **Python**. I have designed and overseen **Databases** using SQL. With a strong foundation in **Machine Learning**, I have integrated **Machine Learning** techniques into my last two projects.
- Resume C: "In my role as a lead software engineer, **Python** has been my primary programming language. I have developed multiple applications using **Python** frameworks and libraries. My most notable project involved creating a **Machine Learning** prediction model using **Python**. I have handled **Databases** in various capacities, ranging from design to optimization. Furthermore, my expertise extends to implementing **Machine Learning** solutions integrated with **Databases**, making processes more efficient."

[1] Ironically, the most popular "similarity" measure does not meet the formal definition!

Using the bag-of-words approach, we can represent each document as a numerical vector. Focusing on the terms “Python”, “Databases”, and “Machine Learning” (in bold), these vectors are

$$\text{job} = \begin{bmatrix}1\\1\\1\end{bmatrix}, \quad \text{resume A} = \begin{bmatrix}1\\1\\1\end{bmatrix}, \quad \text{resume B} = \begin{bmatrix}2\\1\\2\end{bmatrix}, \quad \text{and} \quad \text{resume C} = \begin{bmatrix}3\\2\\2\end{bmatrix}.$$

To measure similarity between the job description and resumes A, B, and C, we must select a similarity function. One simple choice is to use the Euclidean distance, interpreting a smaller distance as greater similarity, and we obtain the following results:

Resume	Euclidean Distance
A	0
B	$\sqrt{2}$
C	$\sqrt{6}$

Resume A ranks as most similar, then B, with C as least similar. However, this can be skewed by resume length and repeated terms. For example, a higher count of “Python” does not necessarily mean a worse fit: Resume C mentions “Python” three times more than Resume A, but it is also three times longer.

We need a similarity measure that is not skewed by document length. Cosine similarity works well because it compares only the angle between vectors, ignoring their norms.

Using cosine similarity, we get:

Resume	Cosine Similarity
A	1
B	0.96
C	0.98

The cosine similarity scores indicate that all three resumes are strong matches to the job posting, which aligns with a quick review of their content.

12.4.2 Pearson’s correlation coefficient: a special case of cosine similarity

In statistics, when we record different measurements for each member of a sample, we often want to know if there is a relationship between them. For example, if we measure the height and weight of a group of people, we might wonder whether taller people tend to weigh more.

A common way to assess this relationship is using Pearson’s correlation coefficient. This coefficient measures the strength of the linear relationship between two data sets by showing how closely the data points follow a straight line.

Definition 12.58. (Pearson's correlation coefficient)

Given a set of pairs of real numbers $\{(x_i, y_i),\ i = 1, 2, \ldots, n\}$, we define their **Pearson's correlation coefficient** as

$$r = \frac{n \sum_{i=1}^{n} x_i y_i - \sum_{i=1}^{n} x_i \sum_{j=1}^{n} y_j}{\sqrt{\left(n \sum_{i=1}^{n} x_i^2 - (\sum_{i=1}^{n} x_i)^2\right)\left(n \sum_{i=1}^{n} y_i^2 - (\sum_{i=1}^{n} y_i)^2\right)}}$$

Pearson's correlation coefficient can be viewed as a cosine similarity. Let x and y be vectors with entries x_i and y_i, and let $\overline{x}$ and $\overline{y}$ denote the mean of the entries of x and the mean of the entries of y, respectively. Then, we form the "mean-centered" vectors $x - \overline{x}$ and $y - \overline{y}$ by subtracting each vector's entry mean from its entries; that is, $x - \overline{x}$ subtracts $\overline{x}$ from every entry of x, and similarly for $y - \overline{y}$. Thus, we can write

$$r = \frac{(x - \overline{x})^T (y - \overline{y})}{\|x - \overline{x}\|_2 \|y - \overline{y}\|_2} = \cos(\theta(x - \overline{x}, y - \overline{y})). \tag{12.11}$$

In other words, Pearson's correlation coefficient r is the cosine similarity of the vectors $x - \overline{x}$ and $y - \overline{y}$.

Equation (12.11) implies that Pearson's correlation coefficient r ranges from -1 to 1. A coefficient close to 1 indicates a strong positive correlation, with points near a straight line with a positive slope (see top left panel in Figure 12.11). A coefficient near 0 indicates a weak linear relationship, where x provides little information about y's values (see bottom left panel in Figure 12.11). A negative coefficient indicates a strong negative correlation, with points near a straight line with a negative slope (see bottom right panel in Figure 12.11).

Pearson's correlation coefficient has an advantage over cosine similarity because it takes into account the mean values of the variables. This helps show linear relationships based on how much values differ from the average. This is especially useful when comparing variables with different averages and scales, like when users rate items differently in recommender systems. We will discuss an example in Chapter 14. Essentially, Pearson's correlation coefficient is the cosine similarity of the mean-centered vectors $x - \overline{x}\mathbf{1}$ and $y - \overline{y}\mathbf{1}$, where $\mathbf{1}$ denotes the vector of all ones of appropriate size.

12.4.3 Set-similarity measures

Cosine similarity works for numeric vectors but not for sets. Here we introduce two set-specific measures: Jaccard similarity and overlap similarity.

Definition 12.59. (Jaccard similarity)

Given two nonempty finite sets A and B, the **Jaccard similarity** between A and B is defined as

$$\mathrm{j}(A, B) := \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

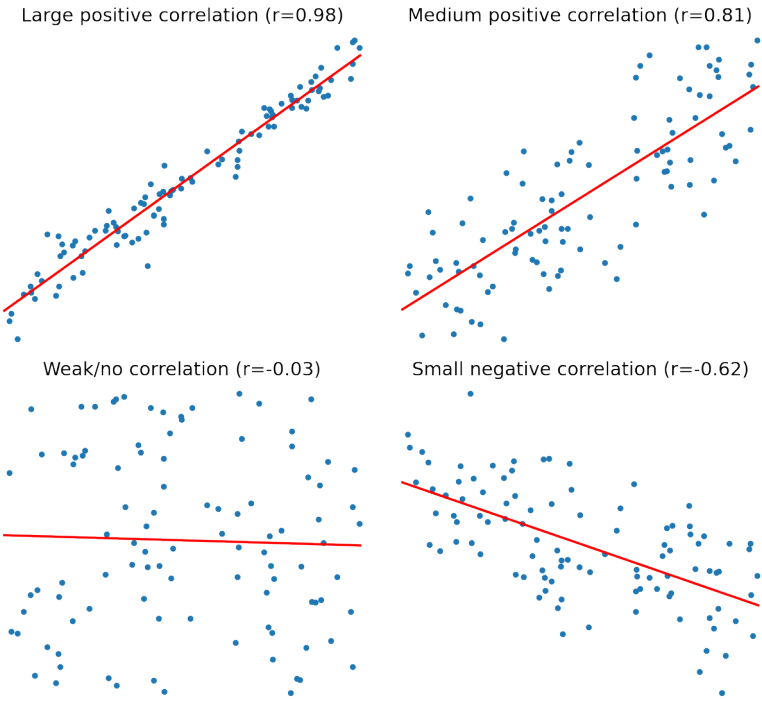


Fig. 12.11: Pearson's correlation coefficient. Positive values indicate that as x increases, y also tends to increase; negative values indicate that as x increases, y tends to decrease. Values near zero imply no linear relationship between x and y.

The Jaccard similarity measures the ratio of the size of the intersection $A \cap B$ to the size of their union $A \cup B$. Its values lie between 0 and 1. A value of 0 indicates no overlap between the sets, while a value of 1 means identical sets. It is related to the Jaccard distance through the equation

$$\mathrm{j}(A, B) = 1 - d_{\mathrm{j}}(A, B). \tag{12.12}$$

The Jaccard similarity clearly satisfies the similarity axioms (i), (ii), and (iii). Regarding axiom (iv), we have

$$\mathrm{j}(A, B) \geqslant \mathrm{j}(A, C) + \mathrm{j}(B, C) - 1, \tag{12.13}$$

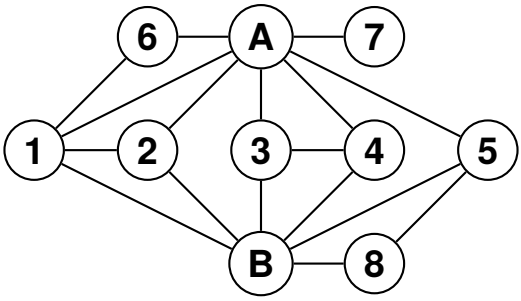
for any three sets A, B and C, which follows from (12.12) and the triangle inequality for the Jaccard distance (Proposition 12.47).

Example 12.60. (Node similarity and edge prediction)

We can use Jaccard similarity to measure how similar two nodes, A and B, are by comparing their neighbors. Two nodes are neighbors if they share an edge. For each

node, let N_A and N_B be the sets of its neighbors. The Jaccard similarity between N_A and N_B quantifies how alike those neighbor sets are.

Consider the graph below.



To find the Jaccard similarity between nodes A and B, we consider their neighbors, i.e., those directly connected to them by edges. For node A, the neighbors are $N_A = \{1, 2, 3, 4, 5, 6, 7\}$, and for node B, they are $N_B = \{1, 2, 3, 4, 5, 8\}$. The Jaccard similarity between these two sets is $\mathrm{j}(N_A, N_B) = \frac{5}{8} = 0.625$.

In Data Science, we frequently use node similarity, especially for edge prediction. But what does edge prediction mean? Here are two examples:

(a) **Predicting new friendships:** Imagine a social network where nodes are users, and an edge shows mutual following. If two users not connected by an edge have a high Jaccard similarity, the platform might suggest they follow each other.
(b) **Predicting drug interactions:** In this case, nodes represent drugs, and an edge indicates an interaction. With so many drugs, we can not test every possible interaction. If two drugs that are not currently connected by an edge have a high Jaccard similarity, they might interact, even if we have not discovered that yet.

These two scenarios show how we predict if an edge may exist between two nodes in a network. A simple way to do this is by using a threshold, like 0.75. If the Jaccard similarity between two nodes (such as A and B, which are not directly connected) is above this threshold, we predict that an edge will form between them.

Another measure of set similarity is overlap similarity.

Definition 12.61. (Overlap similarity)

Given two nonempty finite sets A and B, the **overlap similarity** between A and B is defined as

$$\mathrm{o}(A, B) := \frac{|A \cap B|}{\min\{|A|, |B|\}}.$$

Remark 12.62 Notice that $1 - o(A, B)$ does not define a distance, as happened with the Jaccard similarity. You can check that the triangle inequality does not hold with

$$A = \{1, 2, 3, 4\}, \quad B = \{1, 2, 3, 4, 5\}, \quad C = \{1, 5, 6, 7\}.$$

Jaccard and overlap similarity functions are quite similar. They differ only by their denominators: Jaccard divides by the size of the union, while overlap divides by the size of the smaller set. Why have both? Let us compare them on a recommender-system example to see which works better.

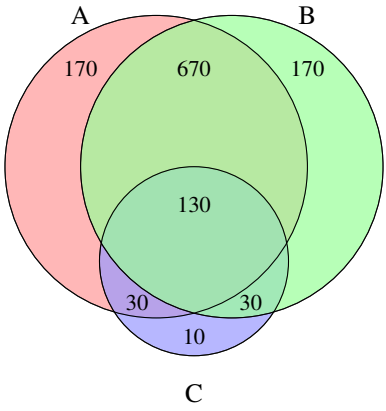
Example 12.63. (A movie recommender system: Jaccard vs overlap similarity)

Imagine you work as a data scientist for a movie streaming service like Netflix. Your job is to recommend movies based on a new user's first pick. To do this, you create a list of users for each movie available on the platform. For example:

$$\begin{aligned}&\text{Jurassic Park} = \{\text{user 3}, \text{user 21}, \text{user 24}, \text{user 58}, \ldots\}\\&\text{Blade Runner} = \{\text{user 1}, \text{user 11}, \text{user 45}, \text{user 78}, \ldots\}\\&\text{The Truman Show} = \{\text{user 15}, \text{user 33}, \text{user 37}, \text{user 45}, \ldots\}\\&\qquad\vdots\end{aligned}$$

When a new user watches movie A, the system checks how similar the audience of movie A is to those of other movies. It then suggests the top 10 most similar movies based on viewer lists. To do this, you can use either Jaccard or overlap similarity. Which one works better?

Look at the illustration below showing the viewership of movies A, B, and C on our streaming platform. The circle sizes show how many users watched each movie. Movies A and B, with 1000 viewers each, are the platform's most popular, while movie C has 200 viewers.



If a new user has just watched movie A, which movie should we recommend next: B or C? Using Jaccard similarity, we get

$$\begin{aligned}\mathrm{j}(A,B) &= \frac{800}{1200} \approx 0.67, \quad \text{and}\\ \mathrm{j}(A,C) &= \frac{160}{1040} \approx 0.15.\end{aligned}$$

By this measure, B is more similar to A. Yet B is already very popular, so the user may already know it. Recommending the lesser-known C could be more valuable.

For overlap similarity, we find

$$\mathrm{o}(A, B) = \frac{800}{1000} = 0.8, \quad \text{and}$$
$$\mathrm{o}(A, C) = \frac{160}{200} = 0.8.$$

Both movies score equally, so this measure would recommend B and C alike. Unlike Jaccard, overlap similarity does not favor the already-popular movie B, making it easier to highlight lesser-known titles. We will explore this property further in Chapter 14.

Problems

12.1 Show that the vector 1-norm $\|\cdot\|_1$ satisfies the norm axioms.

12.2 Show that the vector infinity norm $\|\cdot\|_\infty$ satisfies the norm axioms.

12.3 For any vector $v \in \mathbb{R}^n$ and positive scalars $w_1, w_2, \ldots, w_n > 0$, define the weighted p-norm of v as

$$\|v\|_{p,w} = (w_1|v_1|^p + w_2|v_2|^p + \cdots + w_n|v_n|^p)^{1/p}.$$

Given that $\|\cdot\|_p$ is a norm, show that $\|\cdot\|_{p,w}$ satisfies the norm axioms.

12.4 The "0-norm" of a vector v is defined as

$$\|v\|_0 = \text{number of nonzero entries of } v.$$

For example, the 0-norm of the vector $v = \left[\begin{smallmatrix} 10 \\ 0 \end{smallmatrix}\right]$ is $\|v\|_0 = 1$, because v has one nonzero entry. Show that $\|v\|_0$ is actually not a norm.

12.5 Show that the function $\|\cdot\|_p$ is not a vector norm when $0 < p < 1$.

Hint: Consider the canonical basis vectors e_1 and e_2, and show that $\|e_1 + e_2\|_p > \|e_1\|_p + \|e_2\|_p$.

12.6 Let $A \in \mathbb{R}^{n\times n}$ be an invertible matrix, show that $\|Ax\|_p$ defines a vector norm on $\mathbb{R}^n$ for any p-norm $\|\cdot\|_p$.

Hint: A matrix A is invertible if and only if $Ax = 0$ implies $x = 0$.

12.7 Problem 12.6 is how we built the norm examples in Problem 11.2. Can you guess the matrix A and the value of p that we used for each example?

12.8 Find a unit vector in the 1-norm in the direction of $u = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}^T$.

12.9 Prove that the canonical basis vectors e_i defined in (11.3) are unit vectors for any p-norm.

12.10 Show that, given a vector x with nonnegative entries, we can construct a probability vector u pointing in the same direction as x.

12.11 Show that the unit spheres in $\mathbb{R}^2$ for the 2-norm, 1-norm and ∞-norm are the curves sketched in Figure 12.2.

12.12 Compute by hand the 2-norm of the matrix $\left[\begin{smallmatrix} 1 & -2 \\ -1 & 2 \end{smallmatrix}\right]$.

12.13 Show that $\|A\|_F = \|A^T\|_F$ for any matrix $A \in \mathbb{R}^{m\times n}$.

12.14 Given a matrix $A \in \mathbb{R}^{m\times n}$ and two vector norms $\|\cdot\|_\# : \mathbb{R}^m \to \mathbb{R}$ and $\|\cdot\|_* : \mathbb{R}^n \to \mathbb{R}$, show that

$$\max_{0\neq x\in\mathbb{R}^n} \frac{\|Ax\|_\#}{\|x\|_*} = \max_{\|x\|_*=1} \|Ax\|_\#.$$

12.15 Given a vector norm $\|\cdot\| : \mathbb{R}^m \to \mathbb{R}$ and its induced matrix norm $\|A\| = \max_{\|x\|=1} \|Ax\|$, show that the norm of the $n \times n$ identity matrix is equal to one.

12.16 Show that there does not exist a vector norm $\|\cdot\|$ such that $\|A\|_F = \max_{\|x\|=1} \|Ax\|$.

Hint: Consider the Frobenius norm of the identity matrix.

12.17 Consider a vector norm $\|\cdot\|$ and its induced matrix norm $\|A\| = \max_{\|x\|=1} \|Ax\|$. For any invertible matrix A, prove that

$$\|A^{-1}\| = \frac{1}{\min_{\|x\|=1} \|Ax\|}.$$

12.18 For any matrix $A \in \mathbb{R}^{m\times n}$, show that

$$\|A\|_\infty = \max_{1\leqslant i\leqslant m} \sum_{j=1}^{n} |A_{ij}| = \max_{1\leqslant i\leqslant m} \|A[i,:]^T\|_1.$$

12.19 For any matrix $A \in \mathbb{R}^{n\times n}$, show that $\|A\|_2 \leqslant \sqrt{n}\|A\|_\infty$.

12.20 What is the matrix p-norm of a permutation matrix P?

12.21 If P is a permutation matrix, show that $\|A\|_p = \|PA\|_p$. In other words, show that matrix p-norms do not depend on the row order of the matrix.

12.22 For any matrix A, prove that

(a) $\|A\|_2 = \max_{\|x\|_2=1=\|y\|_2} \|y^T Ax\|_2$,
(b) $\|A^T\|_2 = \|A\|_2$, and
(c) $\|A^T A\|_2 = \|AA^T\|_2 = \|A\|_2$.

Hint: to show part (a), you will need the Cauchy-Schwarz inequality.

12.23 For any matrix $A \in \mathbb{R}^{m\times n}$, show that

$$\|A\|_{1,\infty} = \max_{\substack{1\leqslant i\leqslant m\\ 1\leqslant j\leqslant n}} |A_{ij}| \quad \left(= |||A|||_\infty\right),$$
$$\|A\|_{2,\infty} = \max_{1\leqslant i\leqslant m} \|A[i,:]^T\|_2, \quad \text{and}$$
$$\|A\|_{1,2} = \max_{1\leqslant j\leqslant n} \|A[:,j]\|_2.$$

12.24 Show that the Frobenius norm on $\mathbb{R}^{n\times n}$ is a submultiplicative norm.

12.25 Show that, if max is replaced by min in the definition of matrix norm induced by a vector norm, that is,

$$\|A\|_{\min} := \min_{\|x\|_m=1} \|Ax\|,$$

then a norm is not obtained anymore.

12.26 We have seen in Theorem 12.26 that the matrix norm $|||\cdot|||_\infty$ is not submultiplicative. Nonetheless, prove for any matrices $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$ that $|||AB|||_\infty \leqslant n|||A|||_\infty |||B|||_\infty$. Can you find A and B for which equality holds, i.e. $|||AB|||_\infty = n\,|||A|||_\infty\,|||B|||_\infty$?

12.27 Show that the function $\|\cdot\| : \mathbb{R}^{n\times n} \to \mathbb{R}$ given by

$$\|A\| := n|||A|||_\infty$$

is a submultiplicative matrix norm.

12.28 Given two vector norms $\|\cdot\|_\# : \mathbb{R}^m \to \mathbb{R}$ and $\|\cdot\|_* : \mathbb{R}^p \to \mathbb{R}$, prove for any two matrices $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$ that

$$\|AB\|_{\#,*} \leqslant \|A\|_{\#,+}\|B\|_{+,*},$$

for any third vector norm $\|\cdot\|_+ : \mathbb{R}^n \to \mathbb{R}$.

12.29 Suppose $A \in \mathbb{R}^{m\times n}$ and let the matrix B be any submatrix of A. Show that $\|B\|_p \leqslant \|A\|_p$.

12.30 For any $u, v \in \mathbb{R}^n$, define δ by

$$\delta(u,v) = \begin{cases} 1 & \text{if } u \neq v,\\ 0 & \text{if } u = v. \end{cases}$$

Show that the function δ satisfies the distance axioms.

12.31 For any $u, v \in \mathbb{R}^2$, define the "lift distance" d as

$$d(u,v) = \begin{cases} |u_1 - v_1| & \text{if } u_2 = v_2,\\ |u_1| + |u_2 - v_2| + |v_1| & \text{if } u_2 \neq v_2. \end{cases}$$

Show that d satisfies the distance axioms. Can you visualize what this distance means geometrically?

12.32 Consider a distance d on $\mathbb{R}^n$, and let $a \in \mathbb{R}^n$ be fixed. Show that

$$\delta(u, v) = \frac{2d(u, v)}{d(u, a) + d(v, a) + d(u, v)}$$

satisfies the distance axioms.

Hint: To show that δ satisfies the triangle inequality, use that for any $p, q > 0$ and $r \geqslant 0$ such that $p \leqslant q$, then $\frac{p}{q} \leqslant \frac{p+r}{q+r}$.

12.33 Consider the vectors in $\mathbb{R}^3$ given by $[1, 2, 0]^T$, $[0, 1, 1]^T$ and $[0, 0, 3]^T$. Which of these vectors is closest to $u = [1, -2, 1]^T$ using the Euclidean distance?

12.34 Let $u = [1, 2, 0]^T$ and $v = [-3, 1, 2]^T$ be vectors in $\mathbb{R}^3$.

(a) Compute $\|u\|_2$, the Euclidean distance between u and v, and the angle between these two vectors.
(b) Compute $\|u\|_1$ and the distance induced by this norm between u and v, that is, $d_1(u, v) = \|u - v\|_1$.
(c) Show with a picture two different paths between u and v whose length gives the Euclidean distance between u and v and the distance induced by the 1-norm between these two points.

12.35 The squared Euclidean distance is defined as

$$d(u, v) = (u_1 - v_1)^2 + (u_2 - v_2)^2 + \cdots + (u_n - v_n)^2.$$

Show that it breaks the triangle inequality.

12.36 For any vectors $u, v \in \mathbb{R}^3$, define their distance as

$$d(u, v) = \sqrt{(u_1 - v_1)^2 + (u_2 - v_2)^2} + |u_3 - v_3|.$$

Show that d satisfies the distance axioms. Can you visualize what this distance means geometrically? Is d induced by a norm? If so, identify that norm.

12.37 Given a norm $\|\cdot\|$ in $\mathbb{R}^n$ and the distance $d(u, v) = \|u - v\|$ induced by this norm, find the maximum distance between two unit vectors.

12.38 Show that a distance function $d(u, v)$ is a distance induced by a norm if and only if d is **homogeneous**:

$$d(\alpha u, \alpha v) = |\alpha| d(u, v),$$

and **translation invariant**:

$$d(u + w, v + w) = d(u, v).$$

Hint: Define $\|x\| = d(x, 0)$ and check that it is a norm when d is homogeneous and translation invariant.

12.39 Using the result in Problem 12.38, show that the distance introduced in Problem 12.30 is not induced by a norm.

12.40 Let u and v be two nonzero vectors in $\mathbb{R}^n$. Then, the scalar

$$d_{\cos}(u, v) = 1 - \cos(\theta(u, v))$$

is called the **cosine distance** between u and v.

(a) If we consider that two vectors are "equal" if one of them is a positive scalar of the other, show that the cosine distance satisfies properties (i) and (ii) in the definition of a distance function.
(b) Show that the cosine distance does not satisfy the triangle inequality.

12.41 In this problem, you will show that in $\mathbb{R}^2$, $\theta(x, y)$, the angle between vectors x and y, satisfies the triangle inequality

$$\theta(x, z) \leqslant \theta(x, y) + \theta(y, z),$$

for any three vectors $x, y, z \in \mathbb{R}^2$. Use Figure 12.12 as your guide.

(a) Show that you can write $x = \cos(\theta(x, y))y + \sin(\theta(x, y))u$, where u is a unit vector perpendicular to y. Similarly, show that you can write $z = \cos(\theta(y, z))y + \sin(\theta(y, z))v$, where $v = \pm u$.
(b) Show that $\cos\theta(x, z) = x^T z = \cos\theta(x, y)\cos\theta(y, z) + \sin\theta(x, y)\sin\theta(y, z)u^T v$.
(c) Show $\cos\theta(x, z) \geqslant \cos\theta(x, y)\cos\theta(y, z) - \sin\theta(x, y)\sin\theta(y, z) = \cos(\theta(x, y) + \theta(y, z))$.
(d) Conclude that $\theta(x, z) \leqslant \theta(x, y) + \theta(y, z)$.

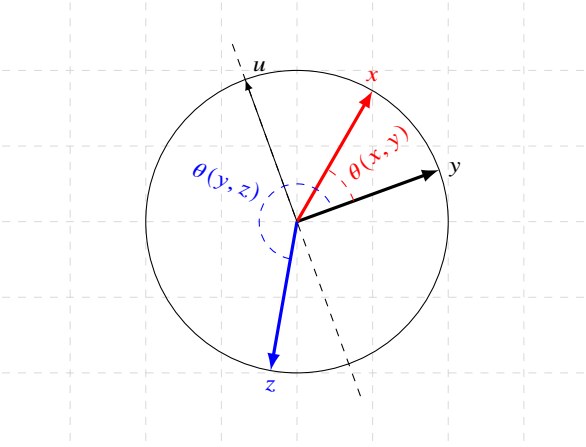


Fig. 12.12: An illustration for proving the triangle inequality for angles between vectors in $\mathbb{R}^2$ in Problem 12.41.

12.42 Compute the Hamming and Levenshtein distances between the two strings "developer" and "dissector".

12.43 Suppose you are working with a team on a new method for digital data storage. Due to the experimental nature of the storage medium, it can sometimes alter bits randomly. You decide to use the Hamming distance to measure the error rate. Table 12.2 shows a few recorded instances of original data strings and the corresponding strings read from the experimental storage.

Instance	Original Data	Read Data
1	1011011	1011011
2	1100110	1101100
3	1111000	1111010
4	1010101	1010111
5	1001110	1001111

Table 12.2: Comparison of original and read data in Problem 12.43.

(a) Compute the Hamming distance for each instance between the original and the read data.
(b) Which instance has the highest error?
(c) On average, how reliable is the experimental storage medium? (Based on the Hamming distances).

12.44 Suppose you are working for an e-commerce platform, analyzing shopping cart patterns to enhance the user experience. The platform has identified M unique items. Each shopping cart is represented as a binary string of length M, where a "1" in position i indicates the presence of item i in the cart, and a "0" indicates its absence. You wish to group similar shopping carts to potentially suggest items to users based on what others with similar carts are purchasing.

Use the data in Table 12.3 to complete the following tasks.

(a) Compute the Hamming distance between Cart 1 and the other shopping carts.
(b) Identify the cart most similar to Cart 1 using Hamming distance.
(c) Which items would you recommend to the user with Cart 1?

12.45 Imagine you are developing an autocorrect feature for a text messaging app. Users often make typos, and the app needs to suggest the correct word. Consider the dictionary: "apple", "banana", "orange", "grape", "mango", "blueberry". Suppose a user types "appel" instead of "apple". The app must find the closest match. Compute the Levenshtein distance between "appel" and each word in the dictionary, identify the word with the smallest distance, and suggest the correct word based on your calculations.

12.46 Prove that the Levenshtein distance d_L satisfies the following inequalities

$$| |u| - |v| | \leqslant d_L(u, v) \leqslant \max\{|u|, |v|\},$$

where $|u|$ and $|v|$ denote the lengths of the strings u and v.

Items	Cart 1	Cart 2	Cart 3	Cart 4	Cart 5	Cart 6
Smartphone	1	1	1	1	1	1
Laptop	0	0	1	0	1	0
Wireless earbuds	0	0	1	0	1	1
E-book	1	1	0	0	1	0
Gaming console	1	1	1	0	1	1
Camera	0	0	0	0	0	1
Smartwatch	0	0	0	1	0	0
Backpack	1	1	1	1	0	1
Desk lamp	0	0	0	0	0	0
Coffee machine	0	0	0	0	0	0
Keyboard	0	0	0	0	0	0
Mouse	0	0	0	0	0	0
Notebook	0	1	0	0	0	0
Pen	0	0	0	0	0	1
Sunglasses	0	1	0	0	0	0

Table 12.3: Binary representation of items in shopping carts in Problem 12.44.

12.47 The **Damerau-Levenshtein distance** between two strings is the minimum number of edits (insertions, deletions, substitutions, or transpositions of two adjacent characters) needed to transform one string into the other.

(a) Consider the two words "example" and "samples". Compute both the Damerau-Levenshtein distance and the Levenshtein distance between these two words.
(b) Does the Damerau-Levenshtein distance satisfy the distance axioms in Definition 12.27?

12.48 The Damerau-Levenshtein distance in Problem 12.47 allows adjacent transpositions anywhere in the string, even if the characters have already been edited. This makes it more flexible but also more computationally demanding. To simplify things, some people use a restricted version called the optimal string alignment distance. The **optimal string alignment distance** between two strings is the smallest number of insertions, deletions, substitutions, and adjacent transpositions needed to turn one string into the other, *with the rule that no substring can be edited more than once*. Does the optimal string alignment distance satisfy the distance axioms?

12.49 Consider the block vectors

$$v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} \qquad \text{and} \qquad u = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix},$$

where $v_1, w_1 \in \mathbb{R}^n$ and $v_2, w_2 \in \mathbb{R}^m$. Show

$$\cos(\theta(v, w)) \leqslant \max\{\cos(\theta(v_1, w_1)), \cos(\theta(v_2, w_2))\}.$$

12.50 You are developing a recommendation system for a library. A user liked the book "Introduction to Space", and you want to recommend another book from the library's database that is thematically similar based on the content description.

Sample Book Descriptions:

1. **Introduction to Space**: "A comprehensive introduction to space exploration. Learn about planets, stars, galaxies, and the universe. Understand the history and future of space travel."
2. **Book A**: "An in-depth look into galaxies and stars. Explore the mysteries of the universe and learn about space missions."
3. **Book B**: "Discover the wonders of the Earth's ecosystems. Dive into oceans, forests, and deserts to understand our planet's biodiversity."
4. **Book C**: "A beginner's guide to space. Understand the basics of planets, space missions, and the history of astronomy."

Complete the tasks below:

(a) First, remove stop words from all book descriptions. Stop words include common terms like "a", "and", "the", "of", etc. that might not significantly impact the thematic meaning of a description.
(b) Convert the book descriptions into vectors using word counts.
(c) Compute the cosine similarity between the vector for "Introduction to Space" and each book description vector.

12.51 You are part of a team building a movie recommendation system. Given a target user, the goal is to find the user with the most similar movie preferences from a database and then recommend movies that this similar user enjoyed but the target user has not seen yet. Table 12.4 shows ratings for various movies under the genres: action, drama, comedy, tragicomedy, and kid films. Ratings are in the range $[0, 10]$.

Movie	User A	User B	User C	User D	User E
Space War	8	3	9	2	7
Battlefront	7	5	7	3	8
Heart's Echo	9	2	8	8	9
Lost Dreams	8	1	7	8	8
Laugh Burst	8	2	5	2	8
Jester's Play	7	3	7	1	7
Tears of Joy	9	4	8	10	9
Bitter Smile	10	5	7	8	9
Adventure Tales	6	1	5	1	8
Magic Friends	7	2	6	1	7

Table 12.4: User ratings for various movies in Problem 12.51.

Given **User A** as the target, identify the user with the highest correlation coefficient (12.11) as the most similar user.

12.52 Imagine you are working with a movie streaming service, and you want to recommend movies to users based on their watch history. Two users, User A and User B, have watched the following movies:

User A: {Titanic, Inception, Avatar, Shawshank Redemption}

User B: {Avatar, Shawshank Redemption, Jurassic Park, The Matrix}.

Calculate the Jaccard and overlap similarities to estimate how similar their movie tastes are.

12.53 You are a data scientist at an e-commerce company. The company wants to improve its product recommendation system by finding similarities between products based on the customers who bought them. The idea is, if two products have been bought by a very similar set of customers, then they can likely be recommended together.

A customer has just bought Product X, and the system needs to decide which of the two other products, Product Y or Product Z, should be recommended to the customer based on historical purchase data. Table 12.5 shows the list of customers who bought Products X, Y, and Z.

Product	Customers
Product X	Alice, Bob, Carol, Dave, Ellen, Frank
Product Y	Carol, Dave, Ellen, George, Helen
Product Z	Alice, Frank, Ian, Jack, Karen

Table 12.5: Customers for products X, Y, and Z in Problem 12.55.

Determine the Jaccard Similarity between Product X's customer set with both Product Y's and Product Z's customer sets. Recommend the product with the higher Jaccard Similarity to Product X.

12.54 Imagine a small social media platform designed for a university campus called CampusConnect. Each user on CampusConnect can send friend requests to other users, and once the request is accepted, they become friends on the platform. The goal is to determine how similar two users' friendship circles are. This can help in friend recommendation systems, for instance.

Figure 12.13 shows the friendship connections among seven users of the CampusConnect platform. For each pair of users, compute the Jaccard similarity of their friend sets. Based on these similarities, which friends should the platform suggest to these users?

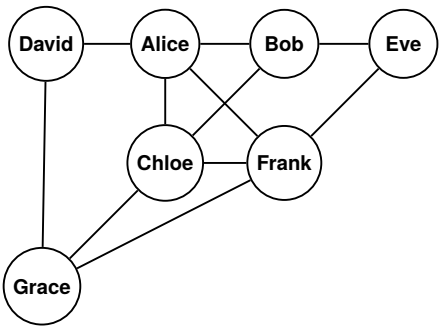


Fig. 12.13: Friendship network on CampusConnect in Problem 12.54.

12.55 Let $A, B, C \subseteq \mathcal{X}$ be finite sets.

(a) Show that $|A \cap C|\,|B \cup C| \leqslant |C|\ (|B| + |A \cap C| - |B \cap C|)$.

Hint: Use $|X \cap Y| + |X \cup Y| = |X| + |Y|$, for any two finite sets, and $|Y| \leqslant |X|$ for any two finite sets such that $Y \subseteq X$.

(b) Show that $|A \cap C|\,|B \cup C| + |A \cup C|\,|B \cap C| \leqslant |C|\ (|A| + |B|)$.
(c) Use the inequality in part (b) to show that $|A \cap B|\,|A \cup B| \leqslant |A|\,|B|$.
(d) Finally, show that $\mathrm{j}(A, B) \geqslant \mathrm{j}(A, C) + \mathrm{j}(B, C) - 1$, where j is the Jaccard similarity in Definition 12.59.

12.56 The **Dice-Sørensen coefficient** between two finite sets A and B is given by

$$S(A, B) = \frac{2|A \cap B|}{|A| + |B|}.$$

Determine whether the Dice-Sørensen coefficient satisfies all the properties of a similarity function. Compare and contrast the Dice-Sørensen coefficient with the Jaccard similarity.

Chapter 13
III-SUPPLEMENTARY TOPICS

In this chapter, we cover a few supplementary topics related to the core concepts of norms, distances, and similarity measures introduced in the previous chapter.

13.1 Norms and distances in the complex vector space $\mathbb{C}^n$

Throughout most of this book, we work with real vectors and matrices, but in some chapters we consider vectors and matrices with complex entries. Moving to complex spaces changes how we define norms and distances. We review those adjustments here.

Recall that a complex number z is written as $z = a + bi$, where a and b are real numbers, and $i = \sqrt{-1}$ represents the imaginary unit. The set of all complex numbers is denoted by $\mathbb{C}$.

The concept of modulus of a complex number, that we introduce next, is the equivalent to the absolute value for real numbers.

Definition 13.1. (Modulus of a complex number)

Given a complex number $z = a + bi \in \mathbb{C}$, we define the **modulus** of z as

$$|z| := \sqrt{\overline{z}z} = \sqrt{a^2 + b^2},$$

where $\overline{z} := a - bi$ is the **conjugate** of z.

Note that the modulus of a complex number is a real number.

If we view a complex number $z = a+bi$ as the point (a, b) in $\mathbb{R}^2$, then its modulus $|z|$ is the Euclidean distance from (a, b) to the origin, just as the absolute value $|x|$ gives the distance of a real number x from zero. In particular, when $b = 0$, $|z|$ reduces exactly to the usual absolute value of a.

The modulus has certain algebraic properties similar to those of the absolute value in $\mathbb{R}$ such as

(i) $|zw| = |z||w|$,
(ii) $|z/w| = |z|/|w|$, where $w \neq 0$,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_13

(iii) $|z + w| \leqslant |z| + |w|$,

where z and w represent arbitrary complex numbers.

The complex vector space $\mathbb{C}^n$ consists of all n-dimensional vectors with complex entries, that is,

$$\mathbb{C}^n := \left\{ \begin{bmatrix} z_1 \\ \vdots \\ z_n \end{bmatrix} \mid z_1, \ldots, z_n \in \mathbb{C} \right\}.$$

For a vector $z \in \mathbb{C}^n$, its **conjugate transpose**, denoted z^*, is the row vector obtained by transposing z and then taking the complex conjugate of each entry:

$$\begin{bmatrix} z_1 \\ \vdots \\ z_n \end{bmatrix}^* = \begin{bmatrix} \overline{z_1} & \cdots & \overline{z_n} \end{bmatrix}.$$

The product $z^*w = \overline{z_1}w_1 + \cdots + \overline{z_n}w_n$ is the analogue for complex vectors of the dot product.

For complex vectors, we can extend the definition of p-norm for real vectors by simply replacing the absolute value with the modulus in the expression for a p-norm. This small change allows us to use norm-based concepts of distance and similarity in complex vector spaces as well.

Definition 13.2. (The p- and infinity-norms in $\mathbb{C}^n$)

For $1 \leqslant p < \infty$, the p-norm of $z = \begin{bmatrix} z_1 & \cdots & z_n \end{bmatrix}^T \in \mathbb{C}^n$ is

$$\|z\|_p = \left(\sum_{i=1}^{n} |z_i|^p \right)^{1/p}.$$

The infinity-norm of z is

$$\|z\|_\infty = \max\{|z_1|, \ldots, |z_n|\}.$$

Notice we can express the 2-norm of a vector $z \in \mathbb{C}^n$ as $\|z\| = (z^*z)^{1/2}$.

The p- and infinity-norm defined on $\mathbb{C}^n$ satisfy the norm axioms.

Example 13.3 Consider the vector $z = \begin{bmatrix} 2 + i & 3 - i & 4i \end{bmatrix}^T$. Then,

$$\|z\|_2 = (z^*z)^{1/2} = \left(\begin{bmatrix} 2 - i & 3 + i & -4i \end{bmatrix} \begin{bmatrix} 2 + i \\ 3 - i \\ 4i \end{bmatrix} \right)^{1/2} = \sqrt{5 + 10 + 16} = \sqrt{31}.$$

Also,

$$\|z\|_1 = \sqrt{5} + \sqrt{10} + 4, \quad \text{and}$$
$$\|z\|_\infty = \max\{\sqrt{5}, \sqrt{10}, 4\} = 4.$$

We extend the norm-induced distance to $\mathbb{C}^n$ exactly as in the real case. Given any vector norm $\|\cdot\|$ on $\mathbb{C}^n$, define

$$d(z, w) = \| z - w \| \quad \text{for } z, w \in \mathbb{C}^n.$$

It is easy to see that d is a distance function on $\mathbb{C}^n$.

These concepts extend to matrices with complex entries. A complex matrix has entries in $\mathbb{C}$, and $\mathbb{C}^{n\times n}$ denotes the set of all $n \times n$ complex matrices. One way to define a norm on $\mathbb{C}^{n\times n}$ is to "vectorize" A by stacking its columns into a single vector in $\mathbb{C}^{n^2}$ and then apply any complex vector norm. Alternatively, given a vector norm $\|\cdot\|$ on $\mathbb{C}^n$, we can define the induced matrix norm by

$$\|A\| = \max_{z\neq 0} \frac{\|Az\|}{\|z\|} = \max_{\|z\|=1} \|Az\|.$$

13.2 Pseudodistances

In Section 12.3.2 we saw that the angle between two unit vectors in $\mathbb{R}^n$ is a distance function. What happens if we drop the unit-length requirement and measure angles between arbitrary nonzero vectors? The resulting function fails to satisfy one of the distance axioms: the angle between v and any positive multiple αv is zero, even though $v \neq \alpha v$. Because it satisfies all other distance properties, we call it a **pseudodistance**.

Definition 13.4. (Pseudodistance on a set X)

A function $d(\cdot,\cdot) : X \times X \to \mathbb{R}$ is a **pseudodistance** if, for any $x, y, z \in X$, it satisfies the following three properties

(i) $d(x, y) = d(y, x)$,
(ii) $d(x, y) \geqslant 0$ and $d(x, x) = 0$, and
(iii) $d(x, y) \leqslant d(x, z) + d(z, y)$.

As we mentioned, the main distinction between a distance and a pseudodistance function is that the latter can assign a distance of zero to pairs x and y even when they are not equal.

Here is an important example of a pseudodistance on $\mathbb{R}^n$.

Definition 13.5. (Angular distance)

The **angular distance**, between two vectors $u, v \in \mathbb{R}^n$ is given by

$$\theta(u, v) = \arccos\left(\frac{u^T v}{\|u\|_2 \|v\|_2}\right) \in [0, \pi].$$

Problem 13.6 asks you to show that the angular distance satisfies the three properties that define a pseudodistance function.

In many applications, we use cosine similarity (12.10) instead of angular distance because it captures the same angle interpretation but is simpler to compute. For vectors u and v, cosine similarity is defined as

$$\frac{u^T v}{\|u\|_2 \, \|v\|_2}.$$

We get this value directly with basic linear algebra operations: one dot product and two norm calculations.

Cosine similarity is commonly used in information retrieval and other Machine Learning applications where vector comparisons are needed.

13.3 Distance-based similarities

In Section 36.2, we introduced several types of similarity functions commonly used in applications to Data Science. Here we show how to construct a family of similarity functions using distance functions.

It is possible to construct a similarity function using any given distance function d. A common approach is to define a function $s(x, y)$ such that it decreases as the distance $d(x, y)$ increases. For instance, to make the similarity decay exponentially with distance, we can use

$$s_{\exp}(x, y) := \exp(-d(x, y)). \tag{13.1}$$

This particular similarity function yields values within the range $(0, 1]$. As the distance between x and y decreases, the similarity $s_{\exp}(x, y)$ approaches one.

From (13.1), we can easily see that $s_{\exp}(x, y)$ satisfies properties (i), (ii), and (iii) of similarity functions. For property (iv), we have

$$\begin{aligned} s_{\exp}(x, z) &= \exp(-d(x, z)) \\ &\geqslant \exp(-(d(x, y) - d(y, z)) \\ &= \exp(-d(x, y)) \exp(-d(y, z)) \\ &= s_{\exp}(x, y) \, s_{\exp}(y, z), \end{aligned}$$

where the inequality follows from the triangle inequality for the distance function d, and the fact that $f(t) = \exp(-t)$ is a decreasing function of t. Therefore, if both $s_{\exp}(x, y)$ and $s_{\exp}(y, z)$ are close to 1, $s_{\exp}(x, z)$ will also be near to one.

Another widely-used approach defines the similarity function s to decrease in inverse proportion to the distance:

$$s_{\text{inv}}(x, y) = \frac{1}{1 + d(x, y)}. \tag{13.2}$$

We include the summand 1 in the denominator to prevent $s_{\text{inv}}(x, y)$ from becoming infinite as the distance nears zero. Similar to $s_{\text{exp}}(x, y)$ in (13.1), the similarity function (13.2) yields values in the range $(0, 1]$. The closer the points, the more the similarity value approaches one.

From (13.2), it is clear that the similarity properties (i), (ii), and (iii) hold. For property (iv), we have (as we ask you to show in Problem 13.7)

$$s_{\text{inv}}(x, z) \geqslant s_{\text{inv}}(x, y)s_{\text{inv}}(y, z).$$

Therefore, if both $s_{\text{inv}}(x, y)$ and $s_{\text{inv}}(y, z)$ are close to one, $s_{\text{inv}}(x, z)$ is too.

13.4 Building graphs from pairwise distances or similarities

In this section, we introduce some popular methods for creating an undirected weighted graph from a numerical dataset. Each node in the graph represents a data point, but the way we connect these nodes and assign weights to the connections can vary depending on what we want to learn about the data.

For our examples, we will use the Euclidean distance, $d(x, y) = \|x - y\|_2$. Of course, you can also use any of the other distance or similarity measures we discussed earlier.

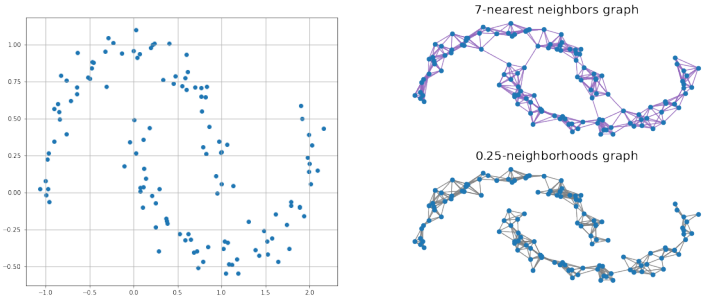
Suppose we have data points $x_1, x_2, \ldots, x_m \in \mathbb{R}^n$. Our goal is to build a graph that captures the underlying structure of the data. Each node corresponds to a data point, so the first step is to decide which nodes should be connected (i.e., choose the edges) and what weights to assign to these connections.

How should we choose the edges? Three common approaches are:

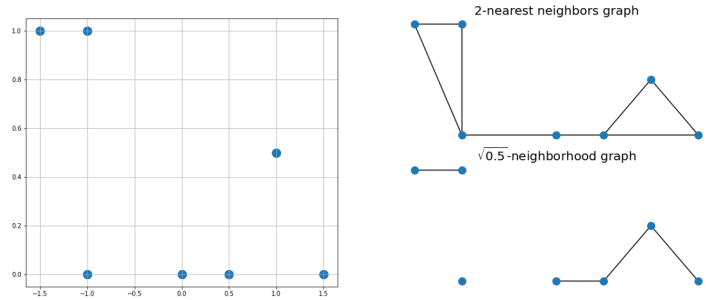
1. **Fully connected graph:** Link every pair of points. Here the choice of edge weights is very important. If we give all edges the same weight, the graph tells us nothing about the data's structure. Later, we will describe several common approaches for assigning those weights.
2. **k-nearest neighbors graph:** Choose a positive integer k. Compute distances (or similarities) between all point pairs. For each point x, find its k closest points, its "nearest neighbors", and connect x to each of them. Every node is connected to k edges, but the graph may be directed: x can list y as a neighbor without y listing x. To make the graph undirected, symmetrize it in one of two ways: add an edge between x and y if either point lists the other as a nearest neighbor, or add an edge only if both list each other.
3. **ϵ-neighborhood graph:** Rather than fixing the number of neighbors, pick a distance (or similarity) threshold ε. For each pair of points, draw an edge if their distance is below ε (or their similarity above ε). This method creates a graph where the connectivity reflects the density of the data: points in dense regions have many connections, while isolated points may have few or none.

Selecting the right parameter can be challenging. In a k-nearest neighbors graph, too small a k fragments the network, while too large a k makes it overly connected. Likewise, in an ϵ-neighborhood graph, a small ϵ leaves many nodes isolated, whereas a large ϵ produces an excessively dense graph.

Figures 13.1a and 13.1b illustrate two toy examples of graphs constructed using the k-nearest neighbors and ϵ-neighborhood approaches.



(a) A numerical dataset (left) and its associated 7-nearest neighbors graph (top-right) and 0.25-neighborhood graph (bottom-right).



(b) A numerical dataset (left) and its associated 2-nearest neighbors graph (top-right) and $\sqrt{0.5}$-neighborhood graph (bottom-right).

Fig. 13.1: Transforming data sets into graphs.

Once we decide which edges to include, we must assign weights. Three common approaches are:

1. **Binary weights:** Give each edge a weight of 1 (and 0 for non-edges). This simply records whether a connection exists, treating all edges equally.
2. **Distance-based weights:** We assign weights using the distances between points, choosing a function that decreases as distance grows. For example, we can assign the weight $1/d(x_i, x_j)$ to the edge connecting x_i and x_j, so closer points receive larger weights and more distant points receive smaller ones.
3. **Similarity-based weights:** Instead of distances, we use a similarity measure to assign edge weights. For each pair of points, we set the edge weight equal to their similarity score, so the more similar two points are, the higher the weight.

One common similarity-based approach uses Gaussian weights, which give high values to nearby points and low values to distant ones. For a formal definition, see Definition 13.6.

Definition 13.6. (Gaussian weights)

For connected vertices x_i and x_j in a graph, the **Gaussian weight** w_{ij} is defined as

$$w_{ij} = \exp\left(-\frac{d(x_i, x_j)^2}{2\sigma^2}\right), \tag{13.3}$$

where $d(x_i, x_j)$ is the distance between data points x_i and x_j, and σ is a user-specified parameter.

The exponential term in the Gaussian weights (13.3) guarantees that weights decrease as the distance between data points increases. The parameter σ controls how quickly this decay happens: a smaller σ makes the weights drop off faster, while a larger σ slows the decay. Choosing the right σ can be difficult and often depends on the specific application.

Gaussian weights are commonly used in graphs constructed using either the ϵ-neighborhood or fully connected approaches.

Problems

13.1 For the complex vectors

$$v_1 = \begin{bmatrix} 1+i \\ 1-i \\ 0 \end{bmatrix}, \quad v_2 = \begin{bmatrix} -i \\ 0 \\ 2-i \end{bmatrix}, \quad \text{and} \quad v_3 = \begin{bmatrix} 2+i \\ 1-3i \\ 2i \end{bmatrix},$$

compute the following quantities.

(a) The dot product between v_1 and v_2.
(b) The dot product between v_2 and v_3.
(c) The 1-norm of v_1.
(d) The 2-norm of v_2.
(e) The infinity norm of v_3.
(f) The Euclidean distance between v_1 and v_3.

13.2 Given a complex number $z = a + ib$, where a is the real part and b is the imaginary part, its modulus is given by $|z| = \sqrt{a^2 + b^2}$. For any complex numbers z and w, show that $|z + w| \leqslant |z| + |w|$.

13.3 Given a complex vector $v \in \mathbb{C}^n$, explain why defining its norm as $\|v\| = v^T v$ is problematic. What should the transpose be replaced with to correctly define the norm of v?

13.4 For any two vectors $u = \left[\begin{smallmatrix} u_1 \\ u_2 \end{smallmatrix}\right]$ and $v = \left[\begin{smallmatrix} v_1 \\ v_2 \end{smallmatrix}\right]$, define their "distance" as $d(u, v) = |u_1 - v_1|$. Show that $d(u, v)$ is a pseudodistance function.

13.5 Let $f : \mathbb{R} \to \mathbb{R}$ be any function, and define

$$d(x, y) \;=\; \big|f(x) - f(y)\big| \quad \text{for } x, y \in \mathbb{R}.$$

(a) Prove that d satisfies nonnegativity, symmetry, and the triangle inequality, and thus is a pseudodistance on $\mathbb{R}$.
(b) Give an explicit example of a function f for which there exist distinct $x, y \in \mathbb{R}$ with $d(x, y) = 0$.

13.6 Show that the angular distance is a pseudodistance function.

Hint: Properties (i) and (ii) are immediate. That angles between vectors in $\mathbb{R}^3$ satisfy the triangle inequality (property (iii)) has been known at least since Euclid (Proposition XI.20 of Euclid's The Elements). Argue that if the triangle inequality holds in $\mathbb{R}^3$, then it also holds in $\mathbb{R}^n$.

13.7 For any given distance d on $\mathbb{R}^n$, show that the similarity function $s_{\text{inv}}(u, v) = 1/(d(u, v) + 1)$ satisfies $s_{\text{inv}}(u, v) \geqslant s_{\text{inv}}(u, w)s_{\text{inv}}(w, v)$.

13.8 For a distance function $d(u, v)$, is the function $s(u, v) = 1 - d(u, v)$ a similarity function?

Chapter 14
III-FROM THE CLASSROOM TO REAL LIFE

We begin this chapter with two important algorithms: k-nearest neighbors and k-means. Both are based on the idea of distance. We then show how distance functions are used in error-correcting codes, graph analysis, information retrieval, and movie recommendation systems.

Before we begin, we need to see how the scale of our data can affect distance.

14.1 Feature scaling in distance-based algorithms

When working with data, different features often have different units and ranges. For example, age might go from 0 to 100, while income might range from thousands to millions. These differences can cause problems for distance-based algorithms.

Imagine computing distance using age and income. Since income values are much larger, they can dominate the result. A small change in income might outweigh a big difference in age. That is why we need to scale the features.

We can prevent any one feature from dominating by putting all features on a similar scale. This is usually done in one of two ways:

1. **Standardization:** This method centers each feature by subtracting its mean and then scales it by dividing by its standard deviation. For a feature vector x, we replace it with
$$x' = \frac{x - \overline{x}}{\sigma(x)}, \tag{14.1}$$
where $\overline{x}$ is a vector with all entries equal to the mean of x, and $\sigma(x)$ is the standard deviation of the entries of x. After this, the entries of x' will have a mean of 0 and a standard deviation of 1.
2. **Normalization:** This method scales features to a specific range, usually between 0 and 1 or between -1 and 1. For each feature, we can replace the vector x by
$$x' = \frac{x - \min\{x\}}{\max\{x\} - \min\{x\}} \quad \text{or} \quad x' = \frac{x}{\max\{|x|\}}, \tag{14.2}$$
where $\max\{x\}$ and $\min\{x\}$ are the vectors whose entries are all equal to the maximum and minimum values of the entries of x, respectively, and $|x|$ is the

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_14

vector of absolute values of the entries of x. The first formula scales the entries of x' to the range of $[0, 1]$, while the second formula scales them to $[-1, 1]$.

For example, consider the data in Table 14.1. It shows four groups: young people with high earnings, young people with low earnings, older people with high earnings, and older people with low earnings.

Data point	Age	Income ($)	Data point	Age	Income ($)
A	22	29,000	G	25	70,000
B	25	27,000	H	24	65,000
C	20	25,000	I	27	62,000
D	55	30,000	J	61	75,000
E	57	28,000	K	60	68,000
F	62	32,000	L	58	71,000

Table 14.1: Mock income-age data.

On the left side of Figure 14.1, we plot the raw data. This is what a distance-based algorithm would see. It is hard to tell apart the four groups: young high earners, young low earners, old high earners, and old low earners. The algorithm might just split the data into two groups: low and high earners. To fix this, we scale each feature by dividing by its largest value (age and income). As shown on the right side of Figure 14.1, the four groups are now much easier to see.

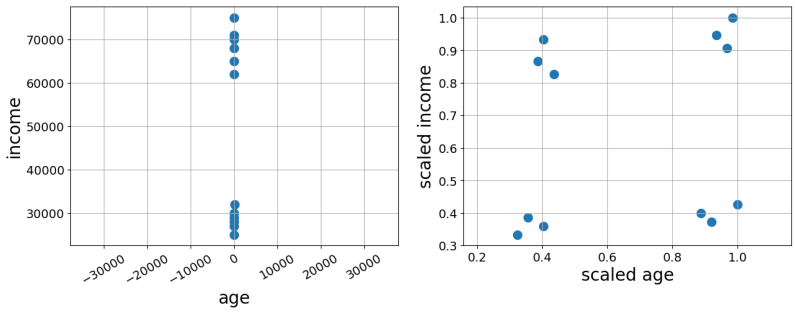


Fig. 14.1: Comparison of raw and scaled data for age and income. The left panel shows the unscaled data, which makes it hard to see the four distinct groups. In the right panel, after scaling, the four groups are much clearer.

In short, it is almost always important to scale features before using distance-based algorithms. This simple step can make a big difference. By putting features on the same scale, the algorithm can focus on real patterns instead of being misled by the raw magnitudes of the features.

14.2 The k-Nearest Neighbors (k-NN) method

The k-Nearest Neighbors method works for both classification and regression. We will start with classification, then see how to adjust it for regression problems.

14.2.1 The k-NN method for classification

In Chapter 1, we discussed classification problems, which involve labeled data points $(x_i, y_i)_{i=1}^m$. Each x_i is a vector in $\mathbb{R}^n$, and each label $y_i \in 1, 2, \ldots, \ell$ tells us the class. For example, in Chapter 10, we saw penguin data with traits like beak size and body mass, and labels for species.

The goal of a classification algorithm is to use the labeled dataset $(x_i, y_i)_{i=1}^m$ to build a function that predicts the class of a new point $x \in \mathbb{R}^n$. For example, given the features of an unknown penguin, the algorithm uses the training data to guess its species. The function f takes the input x and returns the predicted label $y = f(x)$.

The k-Nearest Neighbors (k-NN) method is a simple and popular classification technique in Machine Learning. It is easy to use and gives clear results, which makes it a common choice in practice.

The core ideas behind k-NN are simple:

1. Use a distance function, usually Euclidean distance, to measure how close data points are.
2. Assume that nearby points in the feature space likely belong to the same class.

To classify a new data point, k-NN looks at the k closest labeled points and assigns the most common class among them[1].

For our first example, we will use the Palmer Archipelago penguin data from Section 10.1. We use two features, culmen depth and culmen length, which are already on a similar scale, so we do not need to scale them.

Penguin Species Classification Using k-NN

Figure 14.2 shows how the k-NN algorithm classifies a new data point with $k = 10$. In the top-left, each labeled point is colored by its class. We add a new point at $x = (52, 18)$ (top-right), and the algorithm finds its 10 nearest neighbors (bottom-left). Of these, 8 are "Chinstrap", 2 are "Gentoo", and none are "Adelie". Based on these counts, the algorithm estimates the conditional probabilities (i.e., the chance of each class given the input x). The class probabilities for the new point are:

$$
\begin{aligned}
&p(\text{Adelie} \mid x) = 0,\\
&p(\text{Chinstrap} \mid x) = 8/10 = 4/5,\\
&p(\text{Gentoo} \mid x) = 2/10 = 1/5.
\end{aligned}
$$

Since "Chinstrap" has the highest probability, k-NN classifies the new point as "Chinstrap" (bottom-right of the figure).

[1] Some versions give more weight to closer neighbors, so nearer points count more in the prediction. We will not cover those ideas in this book. We will focus on the simpler version of k-NN.

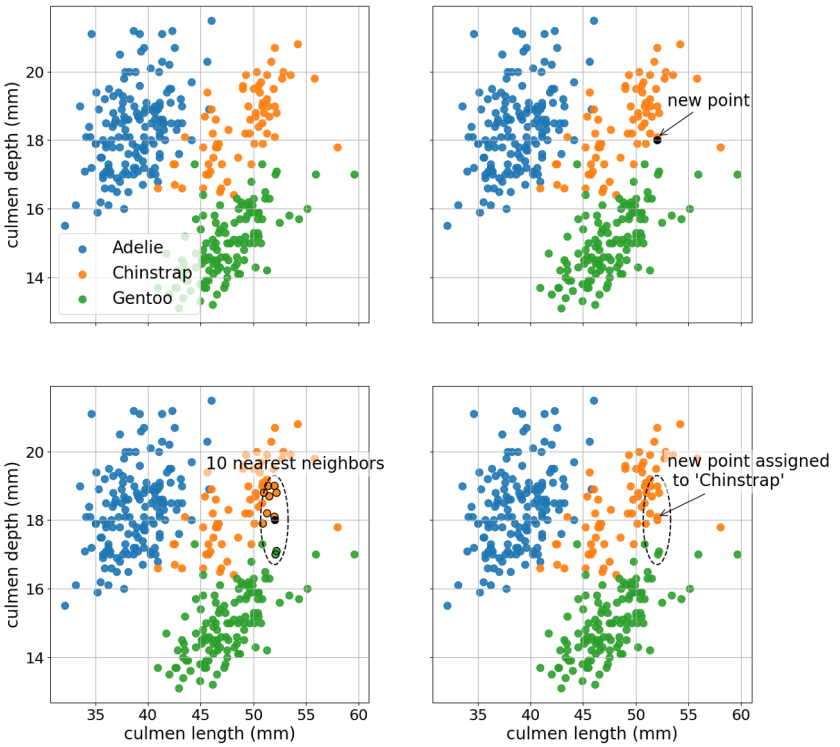


Fig. 14.2: Steps of the k-NN algorithm applied to a new data point in the Palmer Archipelago penguin dataset.

The number of neighbors, k, has a big impact on k-NN predictions. If k is too small, the model depends on very few points and can give unstable results. If k is too large, it might smooth over real patterns and just predict the most frequent label.

How k affects penguin classification

Figure 14.3 shows how k-NN divides the feature space $\mathbb{R}^2$ into classification regions, depending on the value of k. A classification region is a part of the space where any new point would be given the same class label. The shape and size of these regions change as k changes.

The image shows how k-NN classifies penguins for different values of k. With $k = 1$, the boundaries are sharp and follow the data closely. At $k = 10$, the boundaries are smoother, making the species groups easier to see. With $k = 100$, the regions grow larger, and the model tends to favor the most common species. At $k = 200$, the regions are even broader, which can hide important differences between the species.

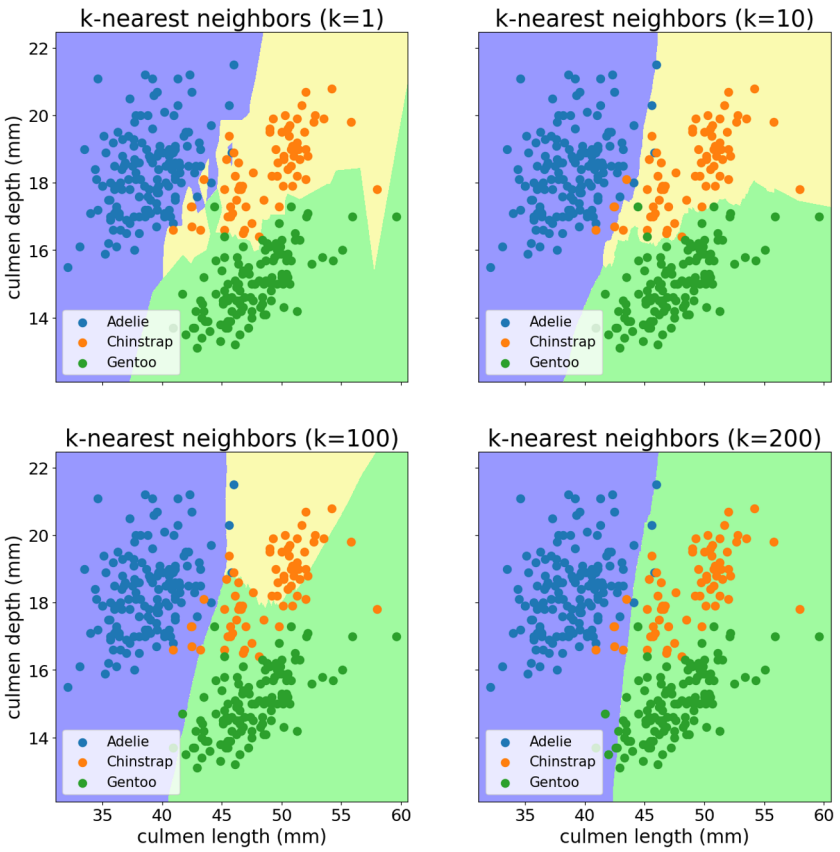


Fig. 14.3: Classification regions of the k-NN method for different values of k.

14.2.2 The k-NN method for regression

The k-NN algorithm also works for regression, with a small change in how it makes predictions. In regression, we still use labeled data points $(x_i, y_i)_{i=1}^m$, but the y_i values are continuous instead of class labels.

As in classification, k-NN finds the k closest neighbors of a new data point. But instead of assigning a class, it predicts a continuous value by averaging the neighbors' y values (or using another summary like the median or quartiles).

Predicting car fuel consumption with k-NN

Imagine you work at a car insurance company and need to predict a car's fuel consumption in miles per gallon (mpg) based on features like the car's weight (in pounds) and model year. You gather data on these features along with each car's

mpg[2]. The left panel of Figure 14.4 shows this data, with each point representing a car and color-coded by mpg value.

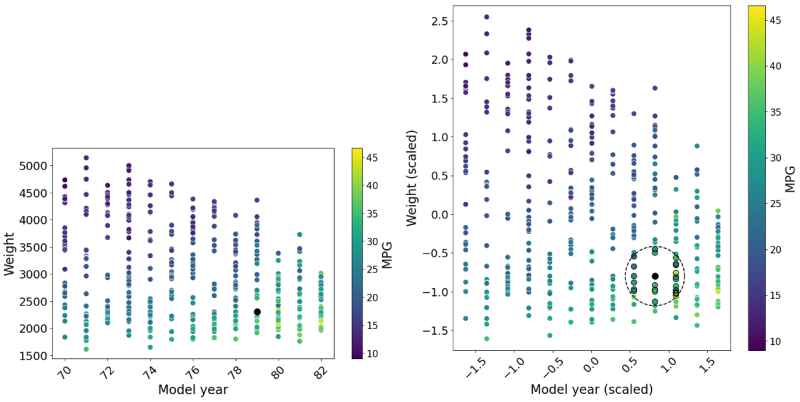
Now, imagine you have a new data point (shown in black in Figure 14.4):

$$x = \begin{bmatrix} \text{model year} \\ \text{weight} \end{bmatrix} = \begin{bmatrix} 79 \\ 2300 \end{bmatrix}.$$

You want to predict its fuel consumption, and we can use the k-NN algorithm to do that. We look at the 20 closest neighbors and take the average of their fuel consumption values.

But first, we need to scale the features, as weight and model year are on very different scales. If we use the raw data, k-NN might give too much importance to weight because its values are much larger.

The right panel of Figure 14.4 shows the scaled data using (14.1) and highlights the 20 nearest neighbors. Based on these, we predict the car's fuel consumption to be about 30.4 miles per gallon



(a) Unscaled features. (b) Scaled features.

Fig. 14.4: Each data point represents a car, color-coded by its mpg (miles per gallon) value. The left panel shows the original unscaled data, while the right panel displays the scaled data with the nearest 20 neighbors highlighted. The black dot in both panels represents a new car, for which we aim to predict the mpg value.

14.3 The k-means method for clustering

As we saw in Chapter 1, clustering is an unsupervised learning problem that finds patterns in data without using class labels. The main goal is to group items so that

[2] The data for this example comes from the Auto MPG dataset found at the UCI Machine Learning Repository [46].

those in the same group are more similar (or closer) to each other than to those in other groups.

The k-means algorithm is a widely used method for clustering. It forms clusters by minimizing a measure called inertia.

Definition 14.1. (Inertia of a partition)

Consider an unlabeled dataset $\{x_i\}_{i=1}^n$ divided into k groups. Let C_j be the set of points in the jth group, and let m_j be its mean (also called the center or centroid):

$$m_j = \frac{1}{|C_j|} \sum_{x \in C_j} x,$$

where $|C_j|$ is the number of points in group j.

The **inertia of the partition** is the average squared distance from each point to the mean of its group:

$$\text{inertia} = \frac{1}{n} \sum_{j=1}^{k} \sum_{x \in C_j} \|x - m_j\|_2^2. \tag{14.3}$$

In its most basic implementation, the k-means method works as follows. First, we choose the value of k, which is the number of clusters we want to find. Choosing the right k can be difficult, as it has a big impact on the results. We will discuss this shortly. Next, we randomly select k data points from the dataset to serve as the initial cluster means [3].

Then:

Step 1: Assign each data point to the nearest cluster mean using Euclidean distance, forming k clusters.

Step 2: Update each cluster mean by averaging the points assigned to it.

Repeat these steps until the means stop changing or a maximum number of iterations is reached.

Let us illustrate the k-means method with the Old Faithful geyser dataset from Chapter 10.

Clustering the Old Faithful dataset

We apply the k-means method with $k = 2$ to the Old Faithful geyser dataset. First, we plot the data in Figure 14.5-(a). After scaling the features, we run the algorithm.

In Figure 14.5-(b), top-left, the green and red circles show the randomly chosen initial cluster centers. We happened to pick both centers in the top-right blob, unlucky, but the method does not know that. Next, we assign each point to the nearest center

[3] Random initialization is the simplest way to start k-means, but it can sometimes lead to poor results. The k-means++ method [5] improves this by spreading out the initial centers, though we will not cover it here.

(green or red). In the top-right plot, we color each point by its assigned center. Notice that the green cluster contains points from both blobs due to the poor initialization, but this will improve in the next steps.

We then update the centers by averaging the points in each cluster. The middle-left plot shows the updated centers. Note how the green center shifts toward the bottom-left blob. The algorithm keeps repeating these two steps until the centers stop changing, which happens after three iterations.

In the Old Faithful geyser example, it was clear from the plot that there were two clusters, so we used $k = 2$ in the k-means method. But what if we do not know the number of clusters ahead of time? Choosing the right k can be difficult, and a poor choice can lead to worse results. Figure 14.6 shows this using a toy dataset with four true clusters (top-left), and the results for $k = 3$, $k = 4$, and $k = 6$.

When k is too small, like $k = 3$, separate clusters can merge. In the top-right figure, the two upper clusters (orange) combine into one. On the other hand, when k is too large, like $k = 6$, individual clusters can split into smaller ones, as seen in the bottom-right figure. The ideal number of clusters for this example is $k = 4$ (bottom-left figure).

The simplest way to choose k is *the elbow method*. We run the k-means algorithm for different values of k and plot the inertia (14.3) against k, as shown in Figure 14.7.

Inertia measures how spread out the points are within each cluster, so lower values mean tighter, more compact clusters. When k is small, increasing it helps reduce the spread because we add more centers, so inertia drops quickly. But after a certain point, adding more clusters does not help much. New clusters just split the real clusters in the data, and the curve flattens, forming an "elbow". We usually choose the number of clusters at this elbow point, since going beyond it gives only small improvements[4].

In the following examples, we will see how clustering and the k-means method can reveal interesting patterns in data.

Example 14.2. (Fremont bridge bicycle traffic)

For this example, we use hourly bicycle counts from Seattle's Fremont Bridge, available on the city's open data website [5].

Figure 14.8-(a) shows the daily traffic from 2012 to 2019[6], with each line representing a single day. Each day is treated as a vector in $\mathbb{R}^{24}$, where each component indicates the bicycle count for each hour.

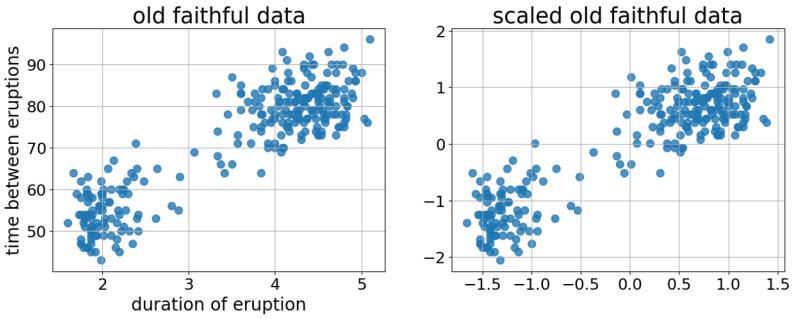
Before applying the k-means method to these traffic vectors, we standardize each one by subtracting the mean and dividing by the standard deviation, as we explained in Section 14.1. The scaled data is shown in Figure 14.8-(b).

Next, we run the k-means method with $k = 2$ to create two clusters of days. Figure 14.9 displays the results, highlighting the cluster means in red along with the day

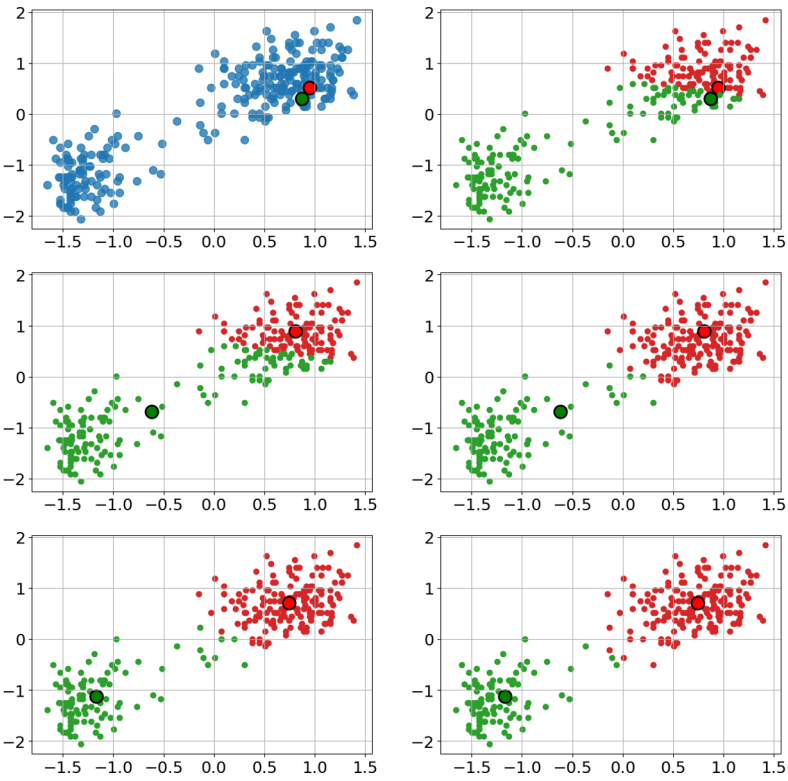
[4] Other methods, like the Silhouette score and Gap statistic, can also help determine k, but we will not cover them in this book.

[5] You can download it from https://data.seattle.gov/

[6] We limit the data to 2019 due to disruptions from the COVID-19 pandemic.



(a) Old Faithful geyser dataset.



(b) k-means process.

Fig. 14.5: Steps of the k-means method applied to the Old Faithful geyser dataset with $k = 2$.

vectors for each cluster. The first cluster shows a bimodal trend, peaking around 8 AM and 5 PM, typical for workday commutes. The second cluster shows a single peak around 2 PM, suggesting that people likely use bikes for recreation, especially on weekends or holidays.

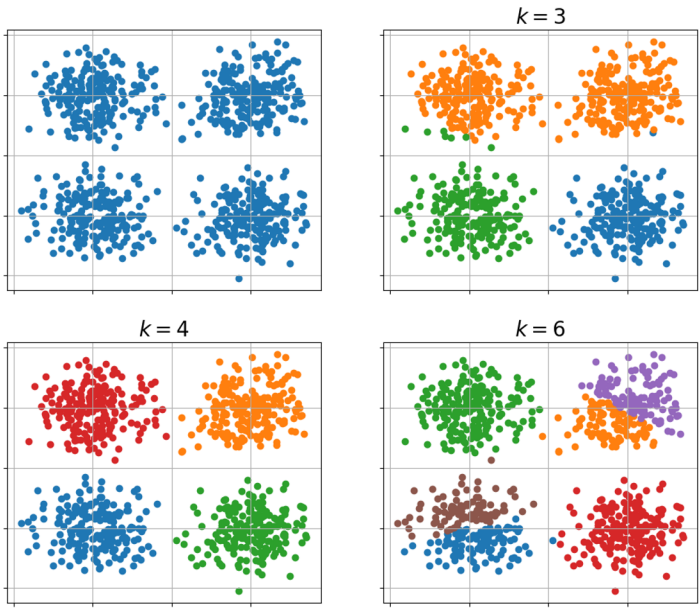


Fig. 14.6: k-means clusters for different vales of k.

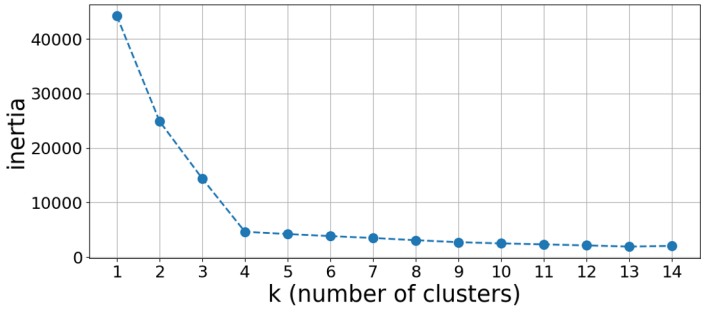


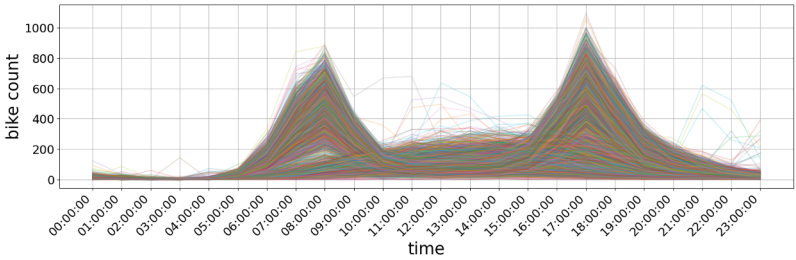
Fig. 14.7: Inertia as a function of the number of clusters k.

We can use the k-means method for other problems, such as color quantization, one of its earliest applications.

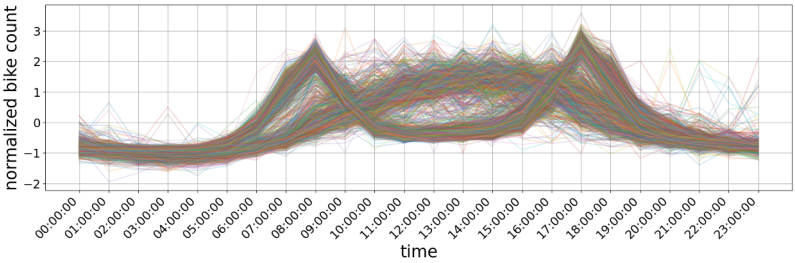
Example 14.3. (Clustering for color quantization)

Ax we explained in Chapter 3, an RGB image represents each pixel's color by combining red, green, and blue intensities. These intensities range from 0 (no intensity) to 1 (full intensity). An $m \times n$ pixel RGB image is typically stored as three $m \times n$ matrices, one for each color.

Figure 14.10 shows a 2×2 RGB image on the left, along with its red, green, and blue intensities.



(a) Hourly bicycle traffic.



(b) Standardize hourly traffic.

Fig. 14.8: Daily bicycle traffic data on the Fremont Bridge.

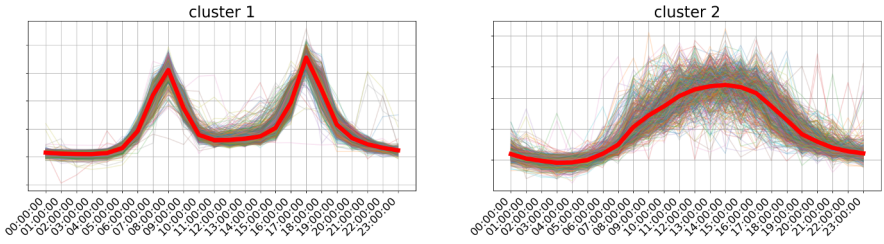


Fig. 14.9: Clustered Fremont bridge hourly bicycle traffic.

We can think of the pixels in an RGB image as vectors in $\mathbb{R}^3$, where each vector represents the color components for a pixel. For the image in Figure 14.10, the color intensities are stored in the following matrix:

$$\begin{bmatrix} 0.8 & 0.4 & 0.3 \\ 0.3 & 0.0 & 1.0 \\ 0.1 & 0.8 & 0.1 \\ 1.0 & 0.15 & 0.7 \end{bmatrix},$$

Each row corresponds to a pixel (top-left, bottom-left, top-right, and bottom-right), and the columns correspond to the colors (red, green, and blue). For example, the third row $\begin{bmatrix} 0.1 & 0.8 & 0.1 \end{bmatrix}$ indicates that the top-right pixel has color intensities of 0.1 for red, 0.8 for green, and 0.1 for blue.

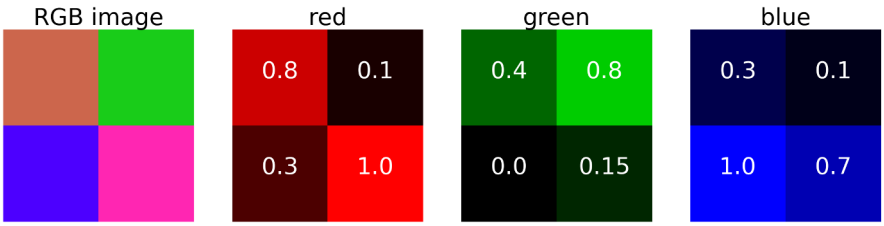


Fig. 14.10: A 2 × 2 pixels RGB image (left) and its red, green and blue pixel intensities.

The RGB image in this example uses just four colors, one for each pixel, made from different combinations of red, green, and blue. That is a tiny number. In a real photo, though, there are usually millions of color combinations. But do we really need that many? Probably not. Many pixel colors are so close that the human eye cannot tell the difference.

The goal of color quantization is to reduce the number of colors in an image to a fixed number, k. For example, Figure 14.11 shows a 1000 × 1500 RGB image of the University of Montana's Main Hall. It uses about 125,000 unique colors, each one a different mix of red, green, and blue intensities. The goal is to shrink this huge set to a much smaller one while maintaining the image's overall quality.

Fig. 14.11: University of Montana Main Hall.

We can apply the k-means algorithm for color quantization by mapping pixels into the three-dimensional space and finding k distinct color clusters. Pixels in the same cluster have similar colors. Figure 14.12 shows how we replace each pixel with the average color of its cluster for different values of k. The value of k determines the number of unique colors in the image. We can achieve a close approximation of the original image using just 64 colors!

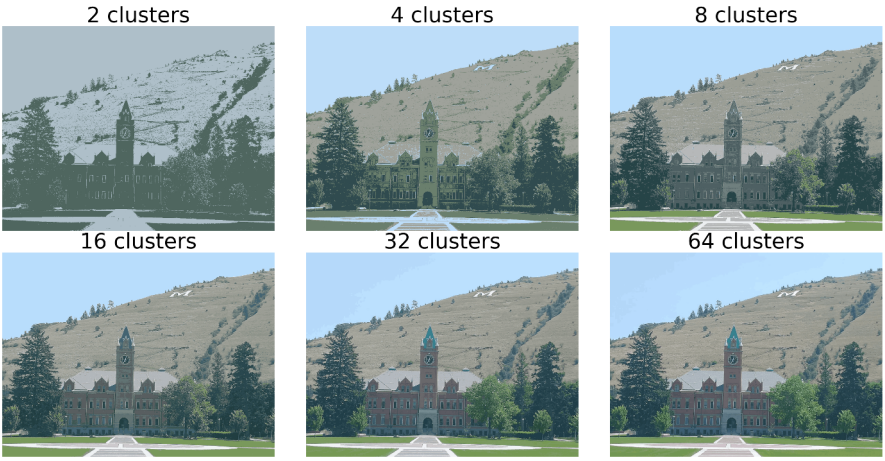


Fig. 14.12: Color quantization of the University of Montana's Main Hall image using k-means clustering with different values of k.

14.4 Hamming error correcting codes

In this section, we consider a new application of distance that we have not discussed yet in the book.

Imagine sending a message encoded as a string of zeros and ones from your computer to another. Noise, like power surges or electrical interference, can flip some zeros to ones or vice versa, creating errors. A similar problem happens with physical storage devices like CDs, DVDs, or Blu-Rays, where scratches can damage data.

Can we recover the original data from a possibly corrupted version? That is the goal of error-correcting codes: to detect and, if possible, correct errors in messages sent over noisy or unreliable channels.

The main idea behind error detection and correction is to send extra, redundant data. The more redundancy we add, the more errors we can detect and fix. However, adding too much redundancy increases the message size, which can slow things down or waste storage space. So the challenge is to strike a balance between error correction and efficiency.

Let us consider a simple way to add redundancy to a message.

Example 14.4. (Adding redundancy for error correction)

Suppose you need to send the following binary message:

0	0	1	1	1	0	1	0	1

To handle potential errors, you send three copies of the message:

original message

0	0	1	1	1	0	1	0	1
0	0	1	1	1	0	1	0	1
0	0	1	1	1	0	1	0	1

redundant messages

The receiver compares the three messages they receive and picks the majority bit in case of a mismatch. For example, if there is an error in the third bit, the receiver can identify it and correct it by choosing the bit that appears twice:

0	0	1	1	1	0	1	0	1
0	0	0	1	1	0	1	0	1
0	0	1	1	1	0	1	0	1

However, this method uses two-thirds of our channel just to make copies. Moreover, even with redundancy, it can not guarantee recovery if more than one bit flips in the same position across different copies.

For example, assume the messages received are:

0	0	0	1	1	0	1	0	1
0	0	0	1	1	0	1	0	1
0	0	1	1	1	0	1	0	1

Using the “majority rule”, the receiver would think the correct message is:

0	0	0	1	1	0	1	0	1

This result is clearly incorrect. Sending the message multiple times could help, but it would also waste a lot of channel capacity just for extra copies.

So let us try a different approach. The goal is to correct errors while adding as little extra redundancy as possible. The main idea now is to define a subset of messages as valid. If a message gets altered, the receiver corrects it by finding the nearest valid one.

In Figure 14.13, each dot is a possible message, and the blue dots are the valid ones. When a valid message gets changed, the receiver looks for the closest blue dot to fix it. To do this, we use Hamming distance between strings. The greater the Hamming distance between valid messages, the more errors we can detect and fix.

In Example 14.4, where we send the message three times, the valid messages are those where all three copies match. The minimum Hamming distance between any two valid messages is 3, since changing one bit means changing it in all three copies.

The fact that the minimum Hamming distance between valid messages is 3 means that any single-bit change will still leave the corrupted message closer to the original than to any other valid message.

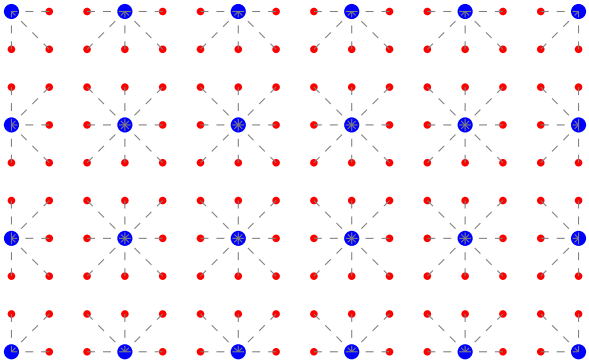


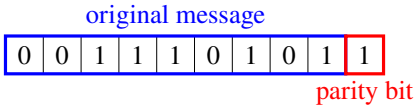
Fig. 14.13: All potential messages are represented by dots, with valid messages as blue dots. The error correction process involves mapping received messages to the nearest valid message.

So if one bit flips, the corrupted message is just 1 unit away from the original valid message, but at least 2 units away from any other valid message. This is why we can both detect and correct a single-bit error: the original message is still the closest valid one.

Can we do better than sending three copies to detect and correct an error? Is it possible to keep a Hamming distance of 3 while using less redundancy? Hamming[7] found a clever solution. The idea is to add **parity bits**.

Example 14.5. (Using a parity bit for error detection)

Imagine we need to send the binary string 001110101 across a channel that might introduce errors, similar to Example 14.4. To help the receiver detect any errors, we add a **parity bit** to the end of our message. The purpose of the parity bit is to make the total count of 1's in the complete message (including the parity bit) even. To do this, we count the 1's in the message. If the count is odd, we set the parity bit to 1; if it is even, we set it to 0. For our example, the transmitted sequence would be



A valid message will have an even total number of 1's, including the parity bit. Adding a parity bit increases the Hamming distance between valid messages to at least 2.

If a single bit flips during transmission, the total number of 1s becomes odd, so the message is flagged as invalid. The receiver can detect that an error occurred, but it cannot correct it.

[7] Richard Hamming was an American mathematician and computer scientist known for his work on error-correcting codes and numerical methods.

If two bits flip (which is less likely), the number of 1s stays even. In that case, the message looks valid, and the error goes undetected.

Using a single parity bit gives valid messages a minimum Hamming distance of 2. With distance 2, a corrupted message is 1 bit away from the original, but possibly also 1 bit away from another valid message. So there is not a unique closest valid message, and the receiver cannot tell which bit flipped. This lets us detect single-bit errors, but not correct them (while two-bit errors may go undetected).

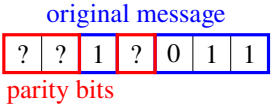
Repeating the message three times increases the minimum distance to 3. In this case, a valid message has three identical copies of the original. This allows us to correct any one-bit error and detect (but not always correct) two-bit errors[8]. The trade-off is that we need twice as much space for redundancy.

By adding and arranging parity bits in a clever way, we can create unique error patterns that let us correct one-bit errors and detect two-bit errors, without needing as much redundancy as sending the message three times.

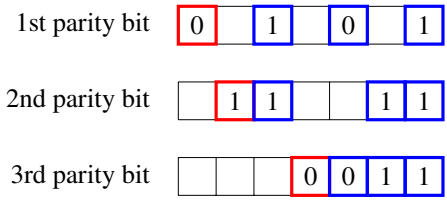
Example 14.6. (Detecting and correcting a single error)

Suppose we are sending a seven-bit message (the reason for this length will become clear by the end of the section). We can detect and correct a single error by using three bits for error-checking and four for the message itself. The error-checking bits are placed in positions that are powers of two: 1, 2 and 4. This leaves us with positions 3, 5, 6, and 7 to carry the actual message. This means we are sending shorter messages than in our earlier examples. But soon, we will see how to extend this idea to longer messages.

For our example, we will send the message 1011 as follows:



The "?" marks show where the parity bits will go. These bits make sure that the count of 1s is even in certain subsets of the message bits. The parity bit in position 1 covers positions 1, 3, 5, and 7. The bit in position 2 covers positions 2, 3, 6, and 7. The parity bit in position 4 covers positions 4 through 7.



The final message we will send, with the added parity bits to make sure there are an even number of 1s in their respective subsets, is

[8] If the two flipped bits are in different positions across the copies, the receiver may still detect and fix them.

0	1	1	0	0	1	1

To understand the placement of parity bits, consider the binary representation of their positions. Instead of thinking in terms of $1, 2, 3, \ldots, 7$, use their binary representation $000, 001, 010, \ldots, 111$. The parity bits are placed at positions with only one binary digit as 1: 001 (position 1), 010 (position 2), and 100 (position 4). The first parity bit covers positions with binary forms ending in 1: 001 (position 1), 011 (position 3), 101 (position 5), and 111 (position 7). The second parity bit covers positions with the middle binary digit as 1: 010 (position 2), 011 (position 3), 110 (position 6), and 111 (position 7). The third parity bit covers positions with binary forms starting with 1: 100 (position 4), 101 (position 5), 110 (position 6), and 111 (position 7).

The receiver can detect and correct a single error by checking the received message. For example, if the bit at position 6 flips, here is how the receiver identifies and corrects the error:

0	1	1	0	0	0	1

Assuming there was only one error, the receiver inspects the parity bits to identify potential errors. The first parity bit is fine, suggesting no error in the bits it checks. If there is an error, its binary location ends with a 0. The second parity bit indicates an error because the total number of 1's in positions 2, 3, 6, and 7 is odd, which means the middle digit of the error's binary position is 1. The third parity bit also shows an error, indicating that the first digit of the binary error position is 1. Putting these together, the error is found at the binary position 110, which corresponds to the 6th bit.

Of course, this worked because we assumed there was only one error. The received message could have more than one error, though that is less likely. For example, if two errors happen in the bits checked by the same parity bit, it might still appear correct. But multiple errors are less common than a single error, since the chances of two or more bits flipping at once are usually much lower.

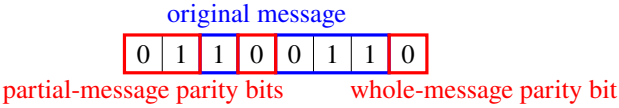
We can use the same idea for longer messages too. With m parity bits, we can handle messages up to $2^m - 1$ bits long. After we account for the parity bits, we have $2^m - m - 1$ data bits left. This is called the $(2^m - 1, 2^m - m - 1)$ Hamming code[9]. For example, with $m = 3$, we get the (7,4)-Hamming code used earlier. With $m = 9$, we can send 511-bit messages, using only 9 parity bits to detect and correct a single error.

What if the message has more than one error? With a small adjustment, we can detect, but not correct, two-bit errors. Adding one extra parity bit allows us to do that.

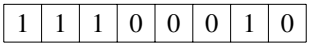
[9] In this context, "code" refers to a systematic way of encoding information with redundancy for error detection and correction.

Example 14.7. (Detecting two-bit errors)

To detect two-bit errors, we add one extra parity bit that checks the entire message. For example, if we want to send the message 1011, we now use four parity bits: the same three as before, plus one more to make sure the total number of 1s is even.



Now, suppose the bits at positions 1 and 6 are flipped. The receiver then gets this message:



The whole-message parity bit looks correct (the number of 1s is even). This means there must be an even number of errors. However, all the partial-message parity bits are incorrect. This indicates that the message is corrupted. The receiver can detect the two-bit errors but cannot tell which bits were flipped.

Hamming codes, created by Richard Hamming in the late 1940s, are an early type of error-correcting code. They established the groundwork for this field, which has evolved significantly since then. Today, modern error-correcting codes, such as Reed-Solomon codes, are used in many technologies, including DVDs, Blu-rays, QR codes, and modems.

14.5 The closeness centrality measure

In Section 9.2, we introduced node centrality and gave one example: Katz centrality. Here, we introduce closeness centrality, which is based on the distances between nodes.

Closeness centrality measures a node's "importance" as the inverse of the average distance from that node to all others in the graph. In simple terms, a node that is close to many others gets a higher score. Since it depends on distances between nodes, we define it for undirected graphs[10].

We first define closeness centrality for undirected connected graphs. Recall that a graph is *connected* if there is a path between every pair of nodes. This ensures that all distances are finite, so the centrality measure is well-defined.

Definition 14.8. (Closeness centrality for a connected graph)

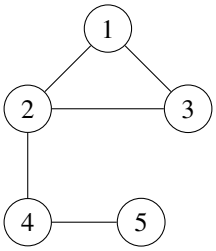
For any connected undirected graph $G = (E, V)$, weighted or unweighted, the **closeness centrality** of a node $u \in V$ is given by

$$c(u) = \frac{n-1}{\sum_{v \in V} d(u,v)},$$

[10] It can be extended to directed graphs, but we will not do that here.

where $d(u, v)$ is the distance between nodes u and v (see Definition 12.50), and n is the number of vertices of G.

Example 14.9 Consider the connected, undirected, and unweighted graph shown below



The closeness centrality of each node is

$$\begin{aligned} c(1) &= \frac{4}{1+1+2+3} = \frac{4}{7} \approx 0.57, \\ c(2) &= \frac{4}{1+1+1+2} = \frac{4}{5} \approx 0.8, \\ c(3) &= \frac{4}{1+1+2+3} = \frac{4}{7} \approx 0.57, \\ c(4) &= \frac{4}{2+1+2+1} = \frac{2}{3} \approx 0.67, \quad \text{and} \\ c(5) &= \frac{4}{3+2+3+1} = \frac{4}{9} \approx 0.44. \end{aligned}$$

Thus, based on closeness centrality, node 2 is the most central, while node 5 is the least central in the graph.

When a graph has multiple connected components, we compute a node's closeness centrality using only the nodes in its own component. To account for the size of the component, we scale the result by the fraction of total nodes it contains. This means nodes in smaller components get lower closeness scores, since they are close to fewer nodes in the graph.

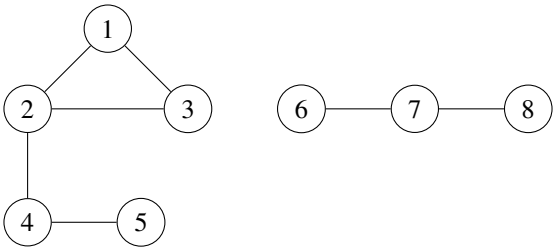
Definition 14.10. (Closeness centrality for disconnected graphs)

For any undirected graph $G = (E, V)$, weighted or unweighted, with $k \geqslant 1$ connected components $G_i = (E_i, V_i)$, for $i = 1, \ldots, k$, the closeness centrality of a node $u \in V_i$ is

$$c(u) = \frac{n_i - 1}{n - 1} \frac{n_i - 1}{\sum_{v \in V_i} d(u, v)},$$

where n_i is the number of nodes of the ith connected component G_i, and $n = n_1 + \cdots + n_k$ is the total number of nodes of G.

Example 14.11 Consider the connected, undirected, and unweighted graph with two connected components shown below



The closeness centrality scores for the nodes in the left component are

$$\begin{aligned}
c(1) &= \frac{4}{7}\frac{4}{1+1+2+3} = \frac{16}{49} \approx 0.33,\\
c(2) &= \frac{4}{7}\frac{4}{1+1+1+2} = \frac{16}{35} \approx 0.46,\\
c(3) &= \frac{4}{7}\frac{4}{1+1+2+3} = \frac{16}{49} \approx 0.33,\\
c(4) &= \frac{4}{7}\frac{4}{2+1+2+1} = \frac{8}{21} \approx 0.38, \quad \text{and}\\
c(5) &= \frac{4}{7}\frac{4}{3+2+3+1} = \frac{16}{63} \approx 0.25.
\end{aligned}$$

For the nodes in the right component, the scores are

$$\begin{aligned}
c(6) &= \frac{2}{7}\frac{2}{1+2} = \frac{4}{21} \approx 0.19,\\
c(7) &= \frac{2}{7}\frac{2}{1+1} = \frac{2}{7} \approx 0.29, \quad \text{and}\\
c(8) &= \frac{2}{7}\frac{2}{2+1} = \frac{4}{21} \approx 0.19.
\end{aligned}$$

Thus, node 2 is the most important according to closeness centrality, while nodes 6 and 8 are the least important.

Closeness centrality is useful in social networks because it shows how easily a node can reach the rest of the network. A node with high closeness centrality can spread information quickly, since it has short paths to many others.

Example 14.12. (A 15th century social network: the Florentine families graph)

The data for our next example is the marriage relationships among prominent Florentine families in 15th-century Italy from Example 3.5. Figure 3.9b shows a circular layout of the family marriage graph, where an edge connects two families if a child from one family married a child from another.

The graph consists of 96 nodes (families) and four connected components: one large component with 90 nodes and three small components with 2 nodes each.

In Figure 14.14, families are colored by their closeness centrality scores. Table 14.15 shows the top 10 and bottom 10 families. Not surprisingly, the Medici and Strozzi, major political players of the time, are among the top three. At the other end, the six families with the lowest scores come from the three small, isolated components of the network.

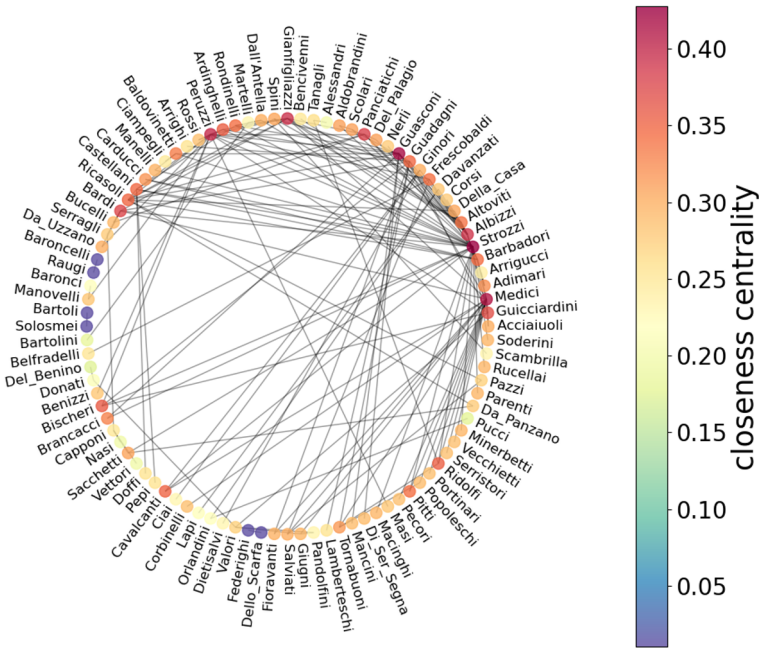


Fig. 14.14: Graph of 15th century Florence marriage relationships of prominent Florentine families. The nodes have been colored according to their closeness centrality.

14.6 Information retrieval

In Chapters 12 and 13, we introduced several similarity functions. Now, let us return to Example 10.1, where the goal was to find the most relevant article for the student's query "similarity measure", and see how we can use similarity measures to make that choice.

Top-10		Bottom-10	
Family	**Closeness centrality**	**Family**	**Closeness centrality**
Strozzi	0.427584	Nasi	0.197113
Guasconi	0.418990	Bartolini	0.184467
Medici	0.410734	Pucci	0.177402
Peruzzi	0.397043	Del Benino	0.173706
Gianfigliazzi	0.393297	Dello Scarfa	0.010526
Albizzi	0.389621	Federighi	0.010526
Panciatichi	0.378995	Baroncelli	0.010526
Bardi	0.375581	Raugi	0.010526
Guicciardini	0.368933	Bartoli	0.010526
Bischeri	0.357850	Solosmei	0.010526

Fig. 14.15: Top- and bottom-10 closeness centralities of 15th century Florence families.

We have calculated the pairwise cosine similarities between the query and each document below.

	Doc 1	**Doc 2**	**Doc 3**	**Query**
Doc 1	1.00	0.58	0.4082	0.75
Doc 2	0.58	1.00	0.24	0.58
Doc 3	0.41	0.24	1.00	0.41
Query	0.75	0.58	0.41	1.00

The 1s on the main diagonal show that each document is identical to itself. The last row (and column) gives the similarity between the query and each of the three documents. Higher cosine similarity means the document is more relevant to the query. In this case, the first document, “A similarity measure is a function that quantifies the similarity between two objects”, has the highest score, so it is the most relevant.

Why is cosine similarity commonly used in information retrieval? When documents are represented as word-count vectors, only words appearing in both documents contribute to the dot product in the numerator. Words appearing in only one document do not contribute to the numerator, but they increase the norm of that document and can therefore lower the cosine similarity. Cosine similarity is useful because it normalizes by the vector lengths and compares the relative distributions of terms rather than their raw counts. In particular, multiplying all word counts in one document by the same positive constant does not change the cosine similarity. Consequently, two documents can have high cosine similarity even when one is longer than the other, provided that their relative term frequencies are similar.

For example, the table below shows the Euclidean distances between the documents and the query. Smaller distances indicate greater similarity. Based on this measure, document 3 appears to be the most relevant. However, it does not actually discuss “similarity measure”.

	Doc 1	Doc 2	Doc 3	Query
Doc 1	0.00	2.45	2.65	2.00
Doc 2	2.45	0.00	2.65	2.00
Doc 3	2.65	2.65	0.00	1.73
Query	2.00	2.00	1.73	0.00

14.7 Movie recommender systems

Imagine you have just started working at a streaming platform, and your job is to create a movie recommender system. In this section, we will introduce some basic ideas to help you with that task.

For our examples, we will use the "MovieLens 10M" dataset, which contains around 10 million ratings from 72,000 users for 9,000 movies[11]. Figure 14.16 shows the first five rows of the dataset, with ratings ranging from 0.5 to 5 in 0.5 increments.

	userId	movieId	rating	timestamp	title
0	4	1	4.0	1113765937	Toy Story (1995)
1	4	2	4.0	1113767306	Jumanji (1995)
2	4	5	2.0	1123990453	Father of the Bride Part II (1995)
3	4	6	4.5	1113767242	Heat (1995)
4	4	10	4.0	1113765995	GoldenEye (1995)

Fig. 14.16: Top rows of the "MovieLens 10M" dataset.

The techniques we will cover fall under the category of *collaborative filtering*, which recommends items by finding patterns or similarities between users or movies[12].

The first problem we will look at is the *cold start problem*. When a new user, with no prior data, watches a movie, the platform needs to recommend more films for them to watch.

Suppose a new user watches "Dark City"[13]. To recommend more films, we need to find movies that are similar to "Dark City" using our existing data. We will look at two strategies. The first method, based on rating counts, was discussed earlier in Example 12.63.

Recommendations based on rating counts

For each film on the streaming service, we start by identifying the set of users who have watched it. Using these user sets, we measure the similarity of other movies to "Dark City" with metrics like Jaccard and overlap.

[11] This data was collected by GroupLens Research and is available on the MovieLens website https://movielens.org.

[12] In contrast to *content-based filtering*, which recommends movies based on their content and features.

[13] Dark City is an adventure, sci-fi movie, somewhat similar to "The Matrix".

Table 14.17 lists the top 10 movie recommendations and their Jaccard and overlap similarities to “Dark City”. While the Jaccard recommendations tend to favor popular blockbusters, the overlap suggestions highlight some lesser-known hidden gems.

Jaccard		Overlap	
movie	**similarity**	**movie**	**similarity**
Gattaca	0.379	eXistenZ	0.647
Starship Troopers	0.350	The City of Lost Children	0.583
Twelve Monkeys	0.349	Event Horizon	0.574
Total Recall	0.345	Cube	0.567
Blade Runner	0.343	Strange Days	0.563
The Fifth Element	0.341	Sphere	0.556
Army of Darkness	0.331	Ghost in the Shell	0.556
Pi	0.330	Spawn	0.552
Aliens	0.330	Jacob’s Ladder	0.552
Contact	0.328	Pi	0.547

Fig. 14.17: Top-10 recommendations for a user that have just watched “Dark City” movie.

The recommendations above only use information about who watched each movie. Adding user ratings could make the suggestions more accurate and helpful.

Recommendations based on ratings

Now, we represent each movie as a vector, where the length of the vector is the total number of users in the dataset. Each entry in the vector corresponds to the rating a specific user gave the movie. By comparing these vectors with measures like cosine similarity, we can find movies that are similar based on user ratings.

One issue with this approach is that not every user rates every movie, leaving some entries in the movie vectors empty. To handle this, we can either set missing values to zero or compare movies using only the mutual ratings.

Table 14.2 shows the top 10 recommendations for a new user who just watched “Dark City”, along with their cosine similarities.

If you are familiar with movies, you will notice that all 10 recommendations are in the same or a similar genre as Dark City, which is pretty good!

Some time has passed, and our new user has rated several dozen movies. Now, we can find users with similar tastes and offer personalized movie recommendations.

Recommendations based on user similarities

We created a fictional user who loves sci-fi and action movies. This user has rated 285 films and gave “Dark City” a score of 5. Our goal is to predict how this user would rate the Matrix series movies: “The Matrix”, “The Matrix Reloaded”, and “The Matrix Revolutions”.

Movie	Cosine similarity
Twelve Monkeys	0.560
Gattaca	0.566
Blade Runner	0.552
The Fifth Element	0.539
The Matrix	0.536
Alien	0.529
Aliens	0.526
Terminator 2	0.523
Total Recall	0.519
The Terminator	0.511

Table 14.2: Top 10 movie suggestions using cosine similarity for a user who recently viewed “Dark City”.

Now, we represent users as vectors, with each entry corresponding to a movie rating. By comparing these vectors, we can identify other users with similar tastes. However, people tend to have different rating habits, some consistently give higher or lower ratings, even when they have similar preferences. Pearson correlation 12.11 accounts for this by computing similarity after adjusting for the mean and scaling by the standard deviation of the ratings. This makes Pearson correlation typically better at capturing true shared preferences than cosine similarity.

To predict ratings for the Matrix movies, we first calculate the Pearson correlation between our user and all other users. For “The Matrix”, “The Matrix Reloaded”, and “The Matrix Revolutions”, we identify the top 10 users most similar to our user. We then average the ratings these 10 users gave to each movie, using that as our prediction. The results are shown in Table 14.3.

Movie	Predicted rating
The Matrix	4.25
The Matrix Reloaded	3.9
The Matrix Revolution	4.2

Table 14.3: Predicted ratings for the Matrix movies based on user similarity.

These three methods are just a starting point for building recommendation systems, and there is plenty of room to improve them. For more on this topic, we recommend the book “Recommender Systems” by Charu C. Aggarwal [4].

Problems

14.1 During the COVID pandemic you started an online business to sell clothes. Since people cannot try the clothes on, you have decided to use machine learning to advice clients on what size to buy based on their weight (in kilograms) and height (in centimeters). The table below shows some data that you have collected.

S means small, M means medium, and L means large.

Weight (in kg)	Height (in cm)	Size
55	160	S
58	165	S
60	160	M
65	166	M
70	168	M
68	171	M
75	175	L
75	180	L
80	187	L
90	190	L

(a) Plot the data points using excel or any other computer program.
(b) The weight and height of a new customer are, respectively, 69 Kg and 170 cm. Use the kNN algorithm with $k = 3$ to give a recommendation on what size the client should buy. You can use excel or any other computer program to do the necessary computations.

14.2 Imagine a small town where the population has recently declined. There are five coffee shops, but the franchise owner believes there are too many and that they are not well placed. The company now wants to find two optimal locations for new coffee shops, using the current ones as a starting point.

Assume we represent locations on a grid (like a map). The units could be blocks or any unit of distance. Here are the coordinates of the five coffee shops:

$$A : (1, 3)$$
$$B : (2, 5)$$
$$C : (6, 4)$$
$$D : (7, 2)$$
$$E : (5, 6)$$

Using k-means clustering with $k = 2$ and starting with initial centers A and B, identify the two locations for the new coffee shops.

14.3 A mobile phone company is launching a new smartphone and wants to create a targeted marketing campaign. They surveyed 10 people to understand how much time they spend on their current phone for various activities and their interest in upgrading to a new phone. The company wants to group these individuals to tailor its marketing strategy.

Imagine the data collected has two dimensions: (i) Hours spent on the phone daily (this is a mix of activities like calling, texting, apps, browsing, etc.) (ii) Interest in upgrading (on a scale of 1 to 10): Where 1 is not interested at all, and 10 is extremely interested. Table 14.4 shows the survey data.

The goal is to segment the audience to better understand how phone usage might relate to interest in upgrading. Use k-means clustering with $k = 2$ to group these individuals.

Individual	Hours spent on the phone	Interest in upgrading
Alice	2	8
Bob	4	7
Charlie	1	3
Dana	5	9
Eli	3	5
Faye	2	2
George	6	10
Hannah	4	6
Ian	1.5	2
Jane	5.5	9

Table 14.4: Smartphone usage and interest in upgrading in Problem 14.3.

Part IV
THE FOUR FUNDAMENTAL SUBSPACES OF A MATRIX, AND GRAM-MATRICES

For any $m \times n$ matrix A, we can identify four fundamental subspaces: two in $\mathbb{R}^n$ (righ nullspace and row space) and two in $\mathbb{R}^m$ (left nullspace and column space). These subspaces reveal important properties of A and how it acts on vectors. In this part of the book, we introduce and study these four subspaces, discuss the concept of rank of a matrix, and examine Gram matrices and their role in linear regression.

Chapter 15
IV-WHY SHOULD WE CARE?

In this part of the book, we further strengthen our Linear Algebra skills. While most of the material is theoretical and may not seem immediately applicable, it sets the groundwork for important topics we will cover later. Nevertheless, we start with a practical application of a key idea introduced in this part: Gram matrices.

15.1 The regression line and data modeling

Linear regression is a useful tool for data modeling. It helps us predict the value of one variable using one or more other variables.

For example, imagine you are a data scientist at a real estate company. Your task is to help your boss understand the local housing market by analyzing recent home sale prices. You have collected data on 1,450 house sales[1]. Figure 15.1 shows the sale price of each house as a blue dot.

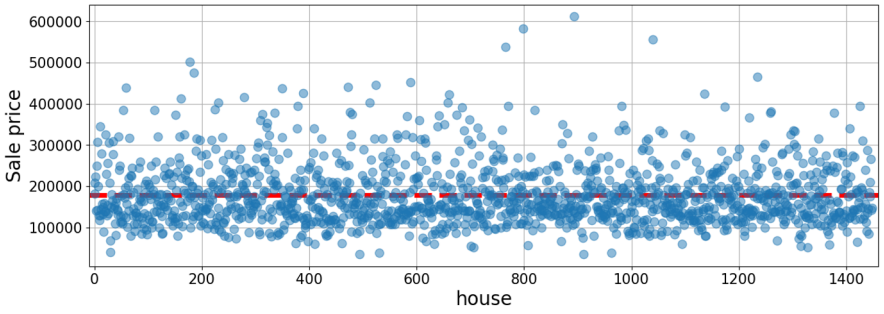


Fig. 15.1: House prices of 1450 houses.

Your boss asks for an estimate of a "representative value" of a house based on the data in Figure 15.1. The first idea that comes to mind is to summarize the overall house prices by calculating the average, which is given by

[1] This data is from the "Ames Residential Home Sales dataset" [14].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_15

$$\mathrm{m} = \frac{1}{1450}\left(\mathrm{price}_1 + \mathrm{price}_2 + \cdots + \mathrm{price}_{1450}\right). \tag{15.1}$$

Let us see how computing this average is the same as finding a horizontal line $y = m$ that "best fits the data". But what does "best fit" mean here? That depends on how we choose to define it. A common choice is to minimize the Euclidean distance between the actual prices and a vector where every entry equals m. Minimizing this distance is the same as minimizing the sum of squared differences between the actual prices and m:

$$f(m) = \sum_{i=1}^{1450} (\mathrm{price}_i - m)^2, \tag{15.2}$$

where price_i is the price of the i-th house.

As we learned in Multivariable Calculus, to minimize (15.2) with respect to m, we take the derivative and set it equal to zero. The derivative of $f(m)$ with respect to m is

$$\frac{\mathrm{d}f}{\mathrm{d}m} = -2 \sum_{i=1}^{1450} (\mathrm{price}_i - m).$$

Setting this equal to zero and solving for m gives the formula for the average in (15.1).

In other words, minimizing (15.2) leads us to the mean value. The horizontal dashed line in Figure 15.1 represents the average house price from the data.

Now, suppose you also have the total area of each of the 1,450 houses in your dataset. Figure 15.2 shows the sale price (in dollars) and area (in square feet) for each house, with each (price, area) pair represented by a dot. The dashed red line in the plot represents the mean price (15.1).

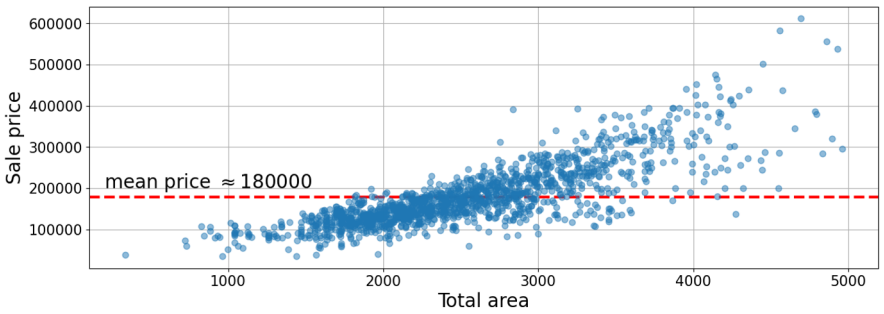


Fig. 15.2: House prices vs. area, with mean price indicated.

Figure 15.2 clearly shows that larger houses tend to have higher prices. For instance, Figure 15.3 highlights two vertical strips. The left strip represents houses with areas between 1,900 and 2,100 square feet, with an average price of $180,000. The right strip shows houses with areas between 3,900 and 4,100 square feet, with an average price of $300,000.

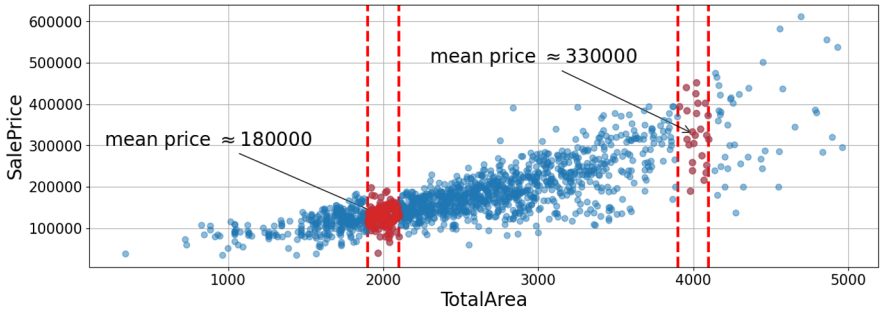


Fig. 15.3: House prices vs. area with vertical strips showing average prices for selected ranges.

To estimate house prices based on their areas, we can use a simple linear model:

$$\text{price} = c_0 + c_1 \cdot \text{area}.$$

This equation says that the price of a house depends on its area in a straight-line way. The number c_1 tells us how much the price increases for each additional unit of area (like dollars per square foot), and c_0 is the starting value when the area is zero. This kind of equation is called a **linear model**. It is one of the most common ways to describe the relationship between two variables.

Just like we found the mean price in (15.1), we can find the model's coefficients c_0 and c_1 by minimizing the sum of squared differences between the actual prices and the prices predicted by the linear model:

$$f(c_0, c_1) := \sum_{i=1}^{1450} \left(\text{price}_i - (c_0 + c_1 \cdot \text{area}_i)\right)^2. \tag{15.3}$$

Minimizing this function leads to the concept of the "regression line", a fundamental tool in Data Science, Statistics, and Machine Learning.

Figure 15.4 shows the regression line in red, which approximates the average price based on house area. In Chapter 19, we will explain how to calculate the regression line, and we will explore it further in Chapters 33 and 34.

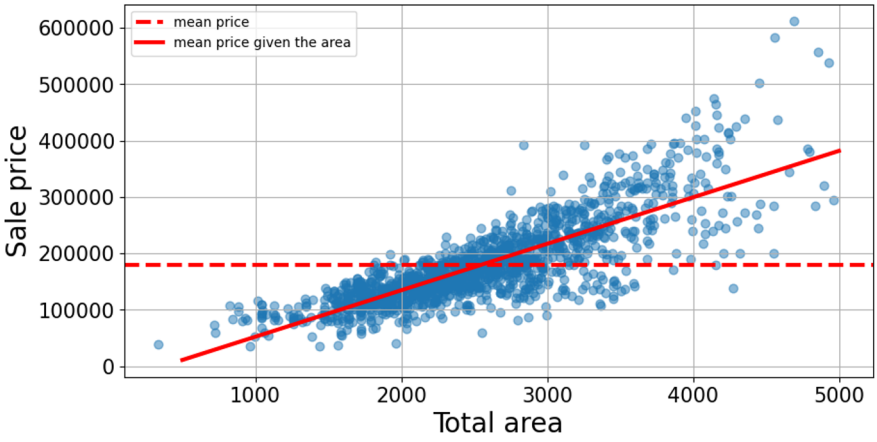


Fig. 15.4: House prices with regression line showing the relationship between price and area.

Chapter 16
IV-WHAT YOU MAY HAVE LEARNED BEFORE

In this chapter, we review some key concepts and results that you should be familiar with from your introductory Linear Algebra course. We begin by recalling the definitions of subspace of $\mathbb{R}^n$, linear combination, and span.

16.1 Subspaces of $\mathbb{R}^n$

In Chapter 11, we reviewed the definition of a real vector space. Now, we focus on certain subsets of $\mathbb{R}^n$ that are also vector spaces, using the same addition and scalar multiplication as in $\mathbb{R}^n$.

Definition 16.1. (Subspace of $\mathbb{R}^n$)

A **subspace** of $\mathbb{R}^n$ is any collection $\mathcal{W}$ of vectors in $\mathbb{R}^n$ such that

(i) the zero vector is in $\mathcal{W}$,
(ii) for any $x, y \in \mathcal{W}$, $x + y \in \mathcal{W}$ (that is, $\mathcal{W}$ is *closed under addition in* $\mathbb{R}^n$), and
(iii) for any $x \in \mathcal{W}$ and any scalar $\alpha \in \mathbb{R}$, $\alpha x \in \mathcal{W}$ (that is, $\mathcal{W}$ is *closed under scalar multiplication in* $\mathbb{R}^n$).

Notice that property (i) guarantees that a subspace $\mathcal{W}$ is never empty. It always includes the zero vector.

Example 16.2 The whole space $\mathbb{R}^n$ is a subspace of itself. The set $\{0\}$, which contains only the zero vector, is also a subspace of $\mathbb{R}^n$.

We call $\{0\}$ the **trivial subspace** of $\mathbb{R}^n$. We call a subspace of $\mathbb{R}^n$ **proper** if it is neither the whole space $\mathbb{R}^n$ nor just the zero vector $\{0\}$.

Example 16.3 The subspaces of $\mathbb{R}^3$ other than the trivial subspace and the whole space are lines and planes that pass through the origin.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_16

16.2 Linear combinations and span

Every vector space has two operations: addition and scalar multiplication. By combining these, we can build new vectors by scaling and adding existing ones. This idea leads to two key concepts: linear combination and span.

Definition 16.4. (Linear combination of vectors)

A **linear combination** of vectors $v_1, v_2, \ldots, v_k$ in $\mathbb{R}^n$ is an expression of the form $c_1v_1 + c_2v_2 + \cdots + c_kv_k$, where $c_1, c_2, \ldots, c_k$ are real numbers.

We say a vector $v \in \mathbb{R}^n$ is a linear combination of a set of vectors $\{v_1, \ldots, v_k\}$ if there exist scalars $c_1, \ldots, c_k \in \mathbb{R}$ such that

$$v = c_1v_1 + \cdots + c_kv_k.$$

A natural question to ask is: What kind of set results from taking all linear combinations of a given set of vectors?

Definition 16.5. (The span of a set of vectors)

The **span** of a set of vectors $\beta = \{v_1, v_2, \ldots, v_k\}$ in $\mathbb{R}^n$ is the set of all possible linear combinations of these vectors:

$$\text{span}(\beta) = \text{span}(v_1, v_2, \ldots, v_k) := \{c_1v_1 + c_2v_2 + \cdots + c_kv_k \mid c_i \in \mathbb{R}\}.$$

For any set of vectors $\beta = \{v_1, v_2, \ldots, v_k\}$ from $\mathbb{R}^n$, the set $\mathcal{W} = \text{span}(v_1, v_2, \ldots, v_k)$ is a subspace of $\mathbb{R}^n$. We call this the **subspace spanned (or generated) by** β. We also say that the set of vectors $\{v_1, v_2, \ldots, v_k\}$ **spans (or generates)** $\mathcal{W}$.

Thus, every finite set of vectors in $\mathbb{R}^n$ generates a subspace of $\mathbb{R}^n$. Next, we ask the converse: Does every subspace of $\mathbb{R}^n$ have a generating set? If so, which of these sets are minimal in the sense that they cannot be reduced to a smaller generating set?

16.3 Linearly independent sets, basis, and dimension

Definition 16.6. (Linearly independent sets)

The empty set is linearly independent. A nonempty set of vectors $\{v_1, v_2, \ldots, v_k\}$ in $\mathbb{R}^n$ is **linearly independent** if the only solution to the equation

$$c_1v_1 + c_2v_2 + \cdots + c_kv_k = 0, \quad c_i \in \mathbb{R},$$

is the trivial solution $c_1 = c_2 = \cdots = c_k = 0$.

If a set of vectors is not linearly independent, we say it is **linearly dependent**.

Some important facts to remember are:

(a) If a set of vectors contains the zero vector, it is linearly dependent.
(b) A set of at least two nonzero vectors is linearly dependent if and only at least one of the vectors in the set can be expressed as a linear combination of the others.
(c) If $m > n$, then any set of m vectors in $\mathbb{R}^n$ is linearly dependent.

Theorem 16.7. (Invertible matrices preserve linear independence)

Consider a nonempty linearly independent set $\{v_1, \ldots, v_k\}$ in $\mathbb{R}^n$. If $S \in \mathbb{R}^{n\times n}$ is an invertible matrix, then the set of vectors $\{Sv_1, \ldots, Sv_k\}$ is also linearly independent.

Proof Suppose $c_1 Sv_1 + \cdots + c_k Sv_k = 0$ for some scalars $c_i \in \mathbb{R}$. Then, multiplying both sides of this equation by S^{-1} (which is possible since S is invertible), we have $c_1 v_1 + \cdots + c_k v_k = 0$. Since $\{v_1, \ldots, v_k\}$ is linearly independent, we deduce that $c_1 = \cdots = c_k = 0$, and therefore $\{Sv_1, \ldots, Sv_k\}$ is also linearly independent. □

Definition 16.8. (A basis for a subspace)

A **basis** for a subspace $\mathcal{W}$ of $\mathbb{R}^n$ is a linearly independent set $\{v_1, v_2, \ldots, v_k\}$ that spans $\mathcal{W}$.

Bases are the minimal generating sets we have been looking for. A basis is just the right size: big enough to generate the subspace, but without any extra, unnecessary vectors (see Problem 16.6).

Theorem 16.9. (The basis theorem)

Any nontrivial subspace $\mathcal{W}$ of $\mathbb{R}^n$ has a basis, and the empty set is said to be a basis for the trivial subspace. In the nontrivial case, every subspace has infinitely many bases. Any two bases for $\mathcal{W}$ have the same number of vectors.

The fact that all the bases for a subspace have the same number of vectors leads to the following definition.

Definition 16.10. (Dimension)

The **dimension** of the subspace $\mathcal{W} \subseteq \mathbb{R}^n$ is equal to the number of vectors in any basis for $\mathcal{W}$.

The following lemmas are two useful results that we will use throughout the book.

Lemma 16.11. (The subspace dimension lemma)

Consider two subspaces $\mathcal{W}$ and $\mathcal{V}$ of $\mathbb{R}^n$. If one subspace is contained in another, say $\mathcal{W} \subseteq \mathcal{V}$, then $\dim(\mathcal{W}) \leqslant \dim(\mathcal{V})$. If both $\mathcal{W} \subseteq \mathcal{V}$ and $\dim(\mathcal{W}) = \dim(\mathcal{V})$, then $\mathcal{W} = \mathcal{V}$.

Lemma 16.12. (The basis extension lemma)

Consider two subspaces $\mathcal{W}$ and $\mathcal{V}$ of $\mathbb{R}^n$ such that $\mathcal{W} \subset \mathcal{V}$, and let $d = \dim(\mathcal{W})$ and $p = \dim(\mathcal{V})$. Any basis $\{w_1, \ldots, w_d\}$ for $\mathcal{W}$ can be extended to a basis $\{w_1, \ldots, w_d, w_{d+1}, \ldots, w_p\}$ for $\mathcal{V}$.

In particular, Lemma 16.12 implies that every linearly independent set of vectors in $\mathbb{R}^n$ can be extended to a basis of $\mathbb{R}^n$.

16.4 The sum of two subspaces

Given two subsets $\mathcal{W}_1$ and $\mathcal{W}_2$ of $\mathbb{R}^n$, two fundamental operations are the **union** and the **intersection**:

$$\mathcal{W}_1 \cup \mathcal{W}_2 = \{x \in \mathbb{R}^n \mid x \in \mathcal{W}_1 \text{ or } x \in \mathcal{W}_2\}, \quad \text{and}$$
$$\mathcal{W}_1 \cap \mathcal{W}_2 = \{x \in \mathbb{R}^n \mid x \in \mathcal{W}_1 \text{ and } x \in \mathcal{W}_2\}.$$

When $\mathcal{W}_1$ and $\mathcal{W}_2$ are both subspaces of $\mathbb{R}^n$, it is natural to ask whether their union and intersection are also subspaces.

The intersection $\mathcal{W}_1 \cap \mathcal{W}_2$ is always a subspace of $\mathbb{R}^n$ (see Problem 16.10). In fact, it is the largest subspace contained in both $\mathcal{W}_1$ and $\mathcal{W}_2$.

What about the union $\mathcal{W}_1 \cup \mathcal{W}_2$? In general, this is not a subspace. While the union is closed under scalar multiplication, it is not closed under addition (unless one subspace is entirely contained in the other).

A natural question then arises: what is the smallest subspace that contains both $\mathcal{W}_1$ and $\mathcal{W}_2$?

Definition 16.13. (The sum of two subspaces)

Suppose $\mathcal{W}_1$ and $\mathcal{W}_2$ are subspaces of $\mathbb{R}^n$. We define their **sum** as

$$\mathcal{W}_1 + \mathcal{W}_2 := \{w_1 + w_2 \mid w_1 \in \mathcal{W}_1 \text{ and } w_2 \in \mathcal{W}_2\}.$$

In simple terms, the sum $\mathcal{W}_1 + \mathcal{W}_2$ consists of all the vectors that can be written as a vector from $\mathcal{W}_1$ plus a vector from $\mathcal{W}_2$ (see Problems 16.13 and 16.14).

Proposition 16.14. (Smallest subspace containing $\mathcal{W}_1$ and $\mathcal{W}_2$)

The sum $\mathcal{W}_1 + \mathcal{W}_2$ of two subspaces $\mathcal{W}_1$ and $\mathcal{W}_2$ is the smallest subspace that contains both $\mathcal{W}_1$ and $\mathcal{W}_2$.

For any vector $u \in \mathcal{W}_1 + \mathcal{W}_2$, there exist $w_1 \in \mathcal{W}_1$ and $w_2 \in \mathcal{W}_2$ such that $u = w_1 + w_2$. But is this decomposition unique? In general, no.

Consider the two-dimensional subspaces $\mathcal{S}$ and $\mathcal{T}$ of $\mathbb{R}^3$ shown in the left panel of Figure 16.1. These subspaces are planes that go through the origin. Their intersection is a one-dimensional subspace, the line through the origin where the planes meet.

The sum of these subspaces is the entire space $\mathbb{R}^3$. This means every vector $b \in \mathbb{R}^3$ can be written as $b = s + t$, where $s \in \mathcal{S}$ and $t \in \mathcal{T}$. This representation is not unique; there are many ways to choose s and t that add up to b. The figure on the right illustrates this by showing two different ways to express b as a combination of vectors from $\mathcal{S}$ and $\mathcal{T}$.

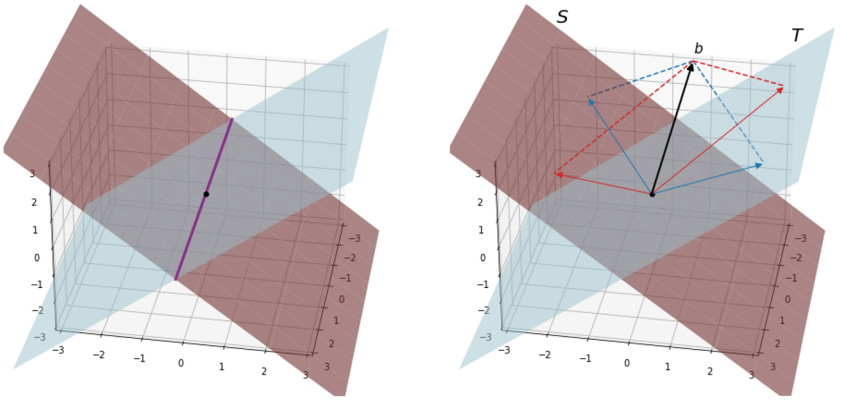


Fig. 16.1: Illustration of the intersection and sum of two 2-dimensional subspaces of $\mathbb{R}^3$.

If every $u \in \mathcal{W}_1 + \mathcal{W}_2$ can be uniquely written as $u = w_1 + w_2$, with $w_1 \in \mathcal{W}_1$ and $w_2 \in \mathcal{W}_2$, then we say that $\mathcal{W}_1 + \mathcal{W}_2$ is the direct sum of $\mathcal{W}_1$ and $\mathcal{W}_2$.

Definition 16.15. (The direct sum of two subspaces)

Let $\mathcal{W}_1$ and $\mathcal{W}_2$ be two subspaces of $\mathbb{R}^n$. Suppose we can represent every $u \in \mathcal{W}_1 + \mathcal{W}_2$ uniquely as $u = w_1 + w_2$, where $w_1 \in \mathcal{W}_1$ and $w_2 \in \mathcal{W}_2$. Then, we call the subspace $\mathcal{W}_1 + \mathcal{W}_2$ the **direct sum** of $\mathcal{W}_1$ and $\mathcal{W}_2$, and denote it as $\mathcal{W}_1 \oplus \mathcal{W}_2$.

Example 16.16 Consider the following three subspaces

$$\begin{aligned}\mathcal{W}_1 &= \{\begin{bmatrix}x_1 & x_2 & x_3\end{bmatrix}^T \in \mathbb{R}^3 \mid x_1 = 0\},\\ \mathcal{W}_2 &= \{\begin{bmatrix}x_1 & x_2 & x_3\end{bmatrix}^T \in \mathbb{R}^3 \mid x_1 = 0, x_2 = 0\}, \quad \text{and}\\ \mathcal{W}_3 &= \{\begin{bmatrix}x_1 & x_2 & x_3\end{bmatrix}^T \in \mathbb{R}^3 \mid x_3 = 0\}.\end{aligned}$$

We have $\mathbb{R}^3 = \mathcal{W}_2 \oplus \mathcal{W}_3$ since we can write any vector $u \in \mathbb{R}^3$ as

$$\begin{bmatrix}u_1\\u_2\\u_3\end{bmatrix} = \begin{bmatrix}0\\0\\u_3\end{bmatrix} + \begin{bmatrix}u_1\\u_2\\0\end{bmatrix} \in \mathcal{W}_2 + \mathcal{W}_3,$$

and that decomposition is unique.

We also have $\mathbb{R}^3 = \mathcal{W}_1 + \mathcal{W}_3$. But $\mathbb{R}^3$ is not the direct sum of $\mathcal{W}_1$ and $\mathcal{W}_3$ since we can write

$$\begin{aligned}\begin{bmatrix}u_1\\u_2\\u_3\end{bmatrix} &= \begin{bmatrix}0\\0\\u_3\end{bmatrix} + \begin{bmatrix}u_1\\u_2\\0\end{bmatrix} \in \mathcal{W}_1 + \mathcal{W}_3, \quad \text{and}\\ &= \begin{bmatrix}0\\u_2\\u_3\end{bmatrix} + \begin{bmatrix}u_1\\0\\0\end{bmatrix} \in \mathcal{W}_1 + \mathcal{W}_3.\end{aligned}$$

Thus, the decomposition is not unique.

Direct sums play an important role in future chapters. They will be important when we discuss orthogonal projections in Chapter 32 and eigenspaces in Chapter 37.

The sum of two subspaces is a direct sum precisely when their only common vector is the zero vector.

Theorem 16.17. (Characterization of direct sums)

Suppose $\mathcal{W}_1$ and $\mathcal{W}_2$ are two subspaces of $\mathbb{R}^n$, and let $\mathcal{U} = \mathcal{W}_1 + \mathcal{W}_2$. Then, $\mathcal{U} = \mathcal{W}_1 \oplus \mathcal{W}_2$ if and only if $\mathcal{W}_1 \cap \mathcal{W}_2 = \{0\}$.

Proof Suppose $\mathcal{U} = \mathcal{W}_1 \oplus \mathcal{W}_2$, and let $u \in \mathcal{W}_1 \cap \mathcal{W}_2$. Since u is in both $\mathcal{W}_1$ and $\mathcal{W}_2$, we can write u as the sum of elements from $\mathcal{W}_1$ and $\mathcal{W}_2$ in two different ways: $u = u + 0$ and $u = 0 + u$. By the definition of direct sum, this is only possible if $u = 0$. Thus, $\mathcal{W}_1 \cap \mathcal{W}_2 = \{0\}$.

Conversely, assume $\mathcal{W}_1 \cap \mathcal{W}_2 = \{0\}$, and let $u \in \mathcal{U} = \mathcal{W}_1 + \mathcal{W}_2$. Suppose we can represent u as a sum of elements from $\mathcal{W}_1$ and $\mathcal{W}_2$ in two different ways:

$$u = w_1 + w_2 = \widetilde{w}_1 + \widetilde{w}_2,$$

where $w_1, \widetilde{w}_1 \in \mathcal{W}_1$ and $w_2, \widetilde{w}_2 \in \mathcal{W}_2$. Then, $w_1 - \widetilde{w}_1 = \widetilde{w}_2 - w_2$. Since $w_1 - \widetilde{w}_1$ is in $\mathcal{W}_1$ and $w_2 - \widetilde{w}_2$ is in $\mathcal{W}_2$, we have that both $w_1 - \widetilde{w}_1$ and $w_2 - \widetilde{w}_2$ are in $\mathcal{W}_1 \cap \mathcal{W}_2 = \{0\}$. Hence, $w_1 = \widetilde{w}_1$ and $w_2 = \widetilde{w}_2$, which implies that $\mathcal{U} = \mathcal{W}_1 \oplus \mathcal{W}_2$.□

The dimension of a direct sum is the sum of the dimensions of the individual subspaces. See Problem 16.15 for a proof of this result.

Theorem 16.18. (The dimension of a direct sum)

Consider the sum $\mathcal{U} = \mathcal{W}_1 + \mathcal{W}_2$ of two subspaces $\mathcal{W}_1$ and $\mathcal{W}_2$ of $\mathbb{R}^n$. Then, $\mathcal{U} = \mathcal{W}_1 \oplus \mathcal{W}_2$ if and only if $\dim(\mathcal{U}) = \dim(\mathcal{W}_1) + \dim(\mathcal{W}_2)$.

The concept of a direct sum extends naturally from two subspaces to any finite collection of subspaces.

Definition 16.19. (Sum of a finite collection of subspaces)

Let $\mathcal{W}_1, \ldots, \mathcal{W}_k \subseteq \mathbb{R}^n$. If every vector in the sum $\mathcal{W}_1 + \cdots + \mathcal{W}_k$ can be expressed *uniquely* as

$$u = w_1 + \cdots + w_k, \qquad w_i \in \mathcal{W}_i,$$

then the sum is called a **direct sum** and is denoted $\mathcal{W}_1 \oplus \cdots \oplus \mathcal{W}_k$.

The characterizations we have seen for the direct sum of two subspaces extend naturally to any finite collection. The following theorem summarizes several equivalent conditions for a sum to be direct.

Theorem 16.20. (Characterizations of direct sums of subspaces)

Let $\mathcal{W}_1, \ldots, \mathcal{W}_k \subseteq \mathbb{R}^n$ be subspaces and set $\mathcal{W} = \mathcal{W}_1 + \cdots + \mathcal{W}_k$. The following statements are equivalent.

(a) $\mathcal{W} = \mathcal{W}_1 \oplus \cdots \oplus \mathcal{W}_k$, that is, every vector $u \in \mathcal{W}$ admits a *unique* decomposition $u = w_1 + \cdots + w_k$ with $w_i \in \mathcal{W}_i$.

(b) For each i, the intersection of $\mathcal{W}_i$ with the sum of the remaining subspaces is the zero vector:

$$\mathcal{W}_i \cap \left(\mathcal{W}_1 + \cdots + \mathcal{W}_{i-1} + \mathcal{W}_{i+1} + \cdots + \mathcal{W}_k\right) = \{0\}.$$

In particular, when $k = 2$ this reduces to $\mathcal{W}_1 \cap \mathcal{W}_2 = \{0\}$.

(c) The only way to write the zero vector as $w_1 + \cdots + w_k$ with $w_i \in \mathcal{W}_i$ is to take $w_1 = \cdots = w_k = 0$.

(d) $\dim(\mathcal{W}_1 + \cdots + \mathcal{W}_k) = \dim(\mathcal{W}_1) + \cdots + \dim(\mathcal{W}_k)$.

In the second part of this chapter, we review the concepts of elementary row and column operations as well as the concepts of elementary matrix, and row and column equivalence. These concepts will be used in Chapter 18.

16.5 Elementary row and column operations

In this section, we review transformations you can apply to the rows and columns of a matrix to simplify it while preserving certain properties.

Definition 16.21. (Elementary row/column operations)

The **elementary row/column operations** are:

- Type I: multiplication of a row/column by a nonzero scalar;
- Type II: interchange of two rows/columns;
- Type III: addition to a row/column of a nonzero multiple of another row/column.

Example 16.22 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}.$$

Below, we apply some elementary row operations to A and show the resulting matrices.

- Multiply the first row of A by 2:

$$\begin{bmatrix} 2 & 4 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}.$$

- Interchange the two top rows of A:

$$\begin{bmatrix} -3 & 1 & 2 \\ 1 & 2 & 0 \\ 0 & 1 & -1 \end{bmatrix}.$$

- Add to the second row of A the first row multiplied by 3:

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 7 & 2 \\ 0 & 1 & -1 \end{bmatrix}.$$

When we apply a row or column operation to a matrix in $\mathbb{R}^{m\times n}$, we can think of it as a function $A \to e(A)$. This function transforms A into another matrix $e(A)$ of the same size. We can represent these functions as matrix multiplications. To explain this, we introduce the following definition.

Definition 16.23. (Elementary matrix)

An $m \times m$ matrix E is called an **elementary matrix** if we can obtain E from the identity matrix I_m by applying exactly one row (or column) operation e. In other words, $E = e(I_m)$.

Example 16.24 The following are examples of 3×3 elementary matrices:

$$E_1 = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad E_2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \text{and} \qquad E_3 = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

We obtain E_1 by multiplying the first row of I_3 by 2. We obtain E_2 by swapping the first two rows of I_3. We obtain E_3 by adding three times the first row to the second row of I_3.

Applying elementary row or column operations is the same as multiplying by elementary matrices. This is covered in basic Linear Algebra courses, so we will not include the proof here.

Theorem 16.25. (Elementary operations as matrix multiplications)

If e is an elementary row operation on $\mathbb{R}^{m\times n}$ and $E = e(I_m)$ is the associated elementary matrix, then

$$e(A) = EA,$$

for all $A \in \mathbb{R}^{m\times n}$. Similarly, if e is an elementary column operation on $\mathbb{R}^{m\times n}$ and $E = e(I_n)$ is the associated elementary matrix, then

$$e(A) = AE,$$

for all $A \in \mathbb{R}^{m\times n}$.

Example 16.26 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}.$$

Multiplying the second row by a constant c is the same as premultiplying A by an elementary matrix E, which we get by applying this row operation to I_3. The result is

$$EA = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 \\ -3c & c & 2c \\ 0 & 1 & -1 \end{bmatrix}.$$

We classify elementary matrices based on the type of operations they perform. For example, the matrix E in Example 16.26 is called a **type I elementary matrix**, Notice that it is invertible precisely when $c \neq 0$.

Definition 16.27. (Type I elementary matrix)

A **Type I elementary matrix** is a matrix of the form

$$M_i(c) := \operatorname{diag}(1, \ldots, 1, c, 1, \ldots, 1), \tag{16.1}$$

where $M_i(c)$ has the entry c in the ith position on the main diagonal, and all other diagonal entries are 1.

Multiplying a matrix A by $M_i(c)$ on the left scales row i of A by c. Multiplying A by $M_i(c)$ on the right scales column i of A by c.

Proposition 16.28. (Type I elementary matrix invertibility)

A type I elementary matrix $M_i(c)$ as in (16.1) is invertible if and only if $c \neq 0$. Its inverse is given by

$$M_i(1/c) = \operatorname{diag}\left(1, \ldots, 1, \frac{1}{c}, 1, \ldots, 1\right).$$

Thus, the inverse of a type I elementary matrix is also a type I elementary matrix.

We can combine multiple type I row (or column) operations into a single matrix multiplication. Multiplying row i of an $m \times n$ matrix A by the scalar c_i, for $i = 1, \ldots, m$, is equivalent to the left multiplication

$$MA = \operatorname{diag}(c_1, \ldots, c_m)A.$$

The matrix $M = M_1(c_2)M_2(c_2) \cdots M_m(c_m)$ is invertible exactly when all the diagonal entries c_i are nonzero, and its inverse is given by $M^{-1} = \operatorname{diag}(c_1^{-1}, \ldots, c_m^{-1})$.

Now, let us apply Theorem 16.25 using type II row operations.

Example 16.29 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}.$$

If we swap rows 2 and 3 of A, it is the same as premultiplying A by the elementary matrix P, obtained by swapping these rows in I_3. The result is

$$PA = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & -1 \\ -3 & 1 & 2 \end{bmatrix}.$$

The matrix P is called a **type II elementary matrix**, and it is invertible with $P^{-1} = P$.

Definition 16.30. (Type II elementary matrix)

A **type II elementary matrix** is formed by swapping two rows of the identity matrix. It takes the form:

$$P_{ij} = \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \begin{array}{c} \begin{array}{ccccccccccc} & & & \text{col } i & & & & & \text{col } j & & \\ \end{array} \\ \begin{bmatrix} 1 & & & & & & & & & & \\ & \ddots & & & & & & & & & \\ & & 1 & & & & & & & & \\ & & & 0 & & & & & 1 & & \\ & & & & 1 & & & & & & \\ & & & & & \ddots & & & & & \\ & & & & & & 1 & & & & \\ & & & 1 & & & & & 0 & & \\ & & & & & & & 1 & & & \\ & & & & & & & & \ddots & & \\ & & & & & & & & & 1 & \end{bmatrix} \end{array} \begin{array}{l} \\ \\ \\ \\ \text{row i} \\ \\ \\ \\ \text{row j} \\ \\ \\ \end{array} . \tag{16.2}$$

Type II elementary matrices are a special type of permutation matrix, which is why we use P (for "permutation") instead of E (for "elementary"). We also call them **elementary permutation matrices**.

Multiplying A by P_{ij} on the left swaps rows i and j of A, while multiplying on the right swaps columns i and j.

If $i = j$, P_{ij} is simply the identity matrix.

Proposition 16.31. (Type II elementary matrix invertibility)

Type II elementary matrices are invertible. If P_{ij} is a type II elementary matrix, then $P_{ij}^{-1} = P_{ij}$. Therefore, the inverse of a type II elementary matrix is also a type II elementary matrix.

As mentioned above, Type II elementary matrices are permutation matrices. Before moving on to Type III, let us quickly recall what a permutation matrix is.

Definition 16.32. (Permutation matrix)

A **permutation matrix** is a matrix with exactly one entry equal to 1 in each row and each column, and 0's elsewhere. In other words, a permutation matrix is any matrix obtained from the identity matrix of the same size by reordering its rows and/or columns.

Example 16.33 There are two permutation matrices of size 2×2:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

There are six permutation matrices of size 3×3:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad \text{and} \quad \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

Among these, the first, second, third, and sixth matrices are type II elementary matrices. The fourth and fifth matrices are not since they are not symmetric.

In general, there are $n!$ permutation matrices of size $n \times n$. We will review permutation matrices and their properties in more depth in Chapter 22.

Finally, let us apply Theorem 16.25 to type III row operations, where a nonzero multiple of one row is added to another row.

Example 16.34 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix}.$$

If we add ℓ times row 1 to row 3 of A, we obtain the same result as premultiplying A by the elementary matrix L, which is derived from I_3 by applying this row operation.

This results in

$$LA = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \ell & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ 0 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 \\ -3 & 1 & 2 \\ \ell & 1+2\ell & -1 \end{bmatrix}.$$

The matrix L in Example 16.34 is called a **type III elementary matrix**, and it is invertible since it is a unit lower triangular matrix[1].

Definition 16.35. (Type III elementary matrix)

A **type III elementary matrix** is a matrix of the form

$$L_{ij}(\ell) = I_n + \ell\, e_j e_i^T \tag{16.3}$$

for some $i, j \in \{1, 2, \ldots, n\}$ with $i \neq j$, and $\ell \in \mathbb{R}$.

Left multiplication LA adds ℓ times row i of A to row j of A. Right multiplication AL adds ℓ times column i of A to column j of A.

Type III elementary matrices are unit lower triangular, which is why we use the notation L (for "lower") instead of E (for "elementary").

Proposition 16.36. (Type III matrix invertibility)

A type III elementary matrix $L_{ij}(\ell)$ as in (16.3) is invertible with

$$(L_{ij}(\ell))^{-1} = L_{ij}(-\ell) = I_n - \ell\, e_j e_i^T.$$

So, the inverse of a type III elementary matrix is also a type III matrix, but with ℓ replaced with $-\ell$.

We can combine multiple type III row or column operations into one matrix multiplication. For example, adding ℓ_j times row i to row j for all $j \neq i$ is equivalent to the left multiplication LA with

$$L = \begin{bmatrix} 1 & & \ell_1 & & \\ & \ddots & \vdots & & \\ & & 1 & \ell_{i-1} & \\ & & & 1 & \\ & & \ell_{i+1} & & 1 \\ & & \vdots & & \ddots \\ & & \ell_n & & & 1 \end{bmatrix}.$$

[1] "unit" means that the main diagonal consists entirely of ones.

The entries of L that are not shown are zero, and the entries $\ell_1, \ldots, \ell_{i-1}, \ell_{i+1}, \ldots, \ell_n$ are in column i. If we define the vector $\ell := [\, \ell_1 \ \cdots \ \ell_{i-1} \ 0 \ \ell_{i+1} \ \cdots \ \ell_n \,]^T$, we can express L as $L = I_n + \ell e_i^T$.

The matrix L is always invertible. To find its inverse, we simply flip the signs of the ℓ_i's:

$$L^{-1} = L - \ell e_i^T = \begin{bmatrix} 1 & & -\ell_1 & & \\ & \ddots & \vdots & & \\ & & 1 & -\ell_{i-1} & \\ & & & 1 & \\ & & -\ell_{i+1} & 1 & \\ & & \vdots & & \ddots \\ & & -\ell_n & & 1 \end{bmatrix},$$

which you can verify by multiplying the matrices.

We can also apply a sequence of row and column operations to a matrix A by multiplying it on both sides by elementary matrices.

16.6 Row and column equivalence

When solving a linear system $Ax = b$ using Gaussian elimination, we apply row operations to the augmented matrix $[A \mid b]$. This gives a new matrix $[U \mid c]$, which, as we will see in Section 21.1, is the augmented matrix of a linear system of equations $Ux = c$ with the same solution set as $Ax = b$. We say that the two systems are equivalent. In particular, we say that A and U are row equivalent. This defines an equivalence relation on the set of $m \times n$ matrices [2].

As mentioned earlier, each elementary operation acts as a function that transforms a matrix A into another matrix of the same size.

Definition 16.37. (Row and column equivalence)

Two matrices $A, B \in \mathbb{R}^{m \times n}$ are **row-equivalent** (denoted $A \sim_{\mathrm{r}} B$) if we can transform A into B using a finite sequence of elementary row operations. That is, $A \sim_{\mathrm{r}} B$ if there exist elementary matrices $E_1, \ldots, E_k$ such that

$$B = E_k \cdots E_1 A.$$

Similarly, B is **column-equivalent** to A (denoted by $A \sim_{\mathrm{c}} B$) if we can obtain B from A using a finite sequence of elementary column operations.

Sometimes, we will use the symbol $\sim$ to represent either row or column equivalence when it is clear from the context which type of operation is being applied.

[2] An equivalence relation groups objects that are considered "equivalent" by satisfying three properties: reflexive (everything is equivalent to itself), symmetric (if one thing is equivalent to another, then the converse is also true), and transitive (if one thing is equivalent to a second, and the second to a third, then the first is equivalent to the third).

Example 16.38 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 \\ -1 & -3 & 2 \\ 2 & 1 & 1 \end{bmatrix}.$$

By performing three type III row operations on A, adding the first row to the second, subtracting 2 times the first row from the third, and subtracting 3 times the second row from the third, we get

$$A \sim_{\mathrm{r}} \begin{bmatrix} 1 & 2 & 0 \\ 0 & -1 & 2 \\ 2 & 1 & 1 \end{bmatrix} \sim_{\mathrm{r}} \begin{bmatrix} 1 & 2 & 0 \\ 0 & -1 & 2 \\ 0 & -3 & 1 \end{bmatrix} \sim_{\mathrm{r}} \begin{bmatrix} 1 & 2 & 0 \\ 0 & -1 & 2 \\ 0 & 0 & -5 \end{bmatrix} =: B.$$

Thus, A is row equivalent to the upper triangular matrix B.

It is easy to see that "being row equivalent" or "being column equivalent" are equivalence relations:

- $A \sim A$, since the identity matrix is an elementary matrix;
- •• if $A \sim B$, then $B \sim A$, taking into account that elementary matrices are invertible and their inverses are also elementary matrices;
- •• if $A \sim B$ and $B \sim C$, then $A \sim C$, since A can be transformed into C by first transforming A into B and then transforming B into C.

We will frequently use the following result in upcoming sections and chapters.

Theorem 16.39. (Row/column equivalence and invertible matrices)

Let $A, B \in \mathbb{R}^{m\times n}$. Then, B is row-equivalent to A if and only if there exists a nonsingular matrix S such that $B = SA$. Similarly, B is column-equivalent to A if and only if there exists a nonsingular matrix S such that $B = AS$.

Proof We provide a brief outline of the proof since this result is usually covered in Linear Algebra courses.

Assume that B is row-equivalent to A. By definition, $B = E_1E_2\cdots E_kA$, where each E_i is an elementary matrix. Elementary matrices are nonsingular, and the product of nonsingular matrices is nonsingular. Hence, $B = SA$ with $S = E_1E_2\cdots E_k$, and S is nonsingular.

Conversely, assume that $B = SA$ for some nonsingular matrix S. Any nonsingular matrix can be reduced to the identity matrix through a sequence of elementary row operations. Thus, $S = E_1E_2\cdots E_k$ for some elementary matrices E_i. Therefore, $B = SA = E_1E_2\cdots E_kA$, which shows that B is row-equivalent to A. □

16.7 Echelon form and reduced-row echelon form

Using row operations, we can transform any matrix into two simpler forms: echelon form and reduced-row echelon form. In the next section, we will use these forms to solve linear systems.

Theorem 16.40. (Row-echelon form)

Any $m \times n$ matrix A is row equivalent to a matrix B in row-echelon form. A matrix in **row-echelon form** has the following properties:

(i) All zero rows are at the bottom of the matrix.
(ii) If the matrix has r nonzero rows and $k_1, k_2, \ldots, k_r$ are the column indices of the leading entries (first nonzero entry from left to right) in rows $1, 2, \ldots, r$, then $k_1 < k_2 < \cdots < k_r$.

A matrix in row-echelon form is a special type of (rectangular) upper triangular matrix.

Example 16.41 Consider the matrices

$$A = \begin{bmatrix} 0 & 1 & 2 & 4 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 2 & 4 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Both matrices are upper triangular, but only A is in echelon form. In B, the leading entry of the third row is in column 4, which is not to the right of the leading entry in the second row. So, B is not in echelon form.

Theorem 16.42. (The reduced-row echelon form)

A matrix is in **reduced row echelon form** when it satisfies all three conditions below:

(i) It is in row-echelon form.
(ii) Its leading entries (called **pivots**) in each nonzero row are all equal to 1.
(iii) In the **pivot columns** (columns containing a pivot), the pivot entry is the only nonzero entry.

For every matrix $A \in \mathbb{R}^{m \times n}$, there is a unique matrix $\operatorname{rref}(A)$ in reduced row echelon form such that A and $\operatorname{rref}(A)$ are row-equivalent.

Example 16.43 Consider the matrices

$$A = \begin{bmatrix} 0 & 1 & 2 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 2 & 4 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The matrix A is in reduced-row echelon form. Its pivot columns are the second and fourth columns. The matrix B is not in reduced-row echelon form because the fourth column, which is a pivot column, has another nonzero entry besides the pivot.

Problems

16.1 Determine if the following sets are subspaces of $\mathbb{R}^2$.

(i) The line passing through $(0, -1)$ and $(0, 1)$.
(ii) The set of points (x_1, x_2) such that $x_1 x_2 = 0$.
(iii) The set of points (x_1, x_2) such that $x_1 + x_2 = 0$.
(iv) The set of points (x_1, x_2) such that $x_1 - x_2 = 1$.
(v) The line passing through $(1, 1)$ and $(-1, -1)$.

16.2 Verify that the set

$$\mathcal{S} = \left\{ \begin{bmatrix} a+b \\ a+c \\ b+c \end{bmatrix} \mid a, b, c \in \mathbb{R} \right\}$$

is a subspace of $\mathbb{R}^3$ and find a generating set for $\mathcal{S}$.

16.3 Determine whether or not the following set is linearly independent.

$$\left\{ \begin{bmatrix} 1 \\ 4 \\ 7 \end{bmatrix}, \begin{bmatrix} 2 \\ 5 \\ 8 \end{bmatrix}, \begin{bmatrix} 3 \\ 6 \\ 9 \end{bmatrix} \right\}$$

16.4 Consider the subspace $\mathcal{W}$ of $\mathbb{R}^3$ defined by the equation $x_1 + x_2 + x_3 = 0$. Find two different bases for $\mathcal{W}$.

16.5 Consider a linearly independent set of vectors β in $\mathbb{R}^n$ that spans a proper subspace of $\mathbb{R}^n$, and let $w \in \mathbb{R}^n \backslash \text{span}(\beta)$. Show that the set $\beta \cup \{w\}$ is also linearly independent.

16.6 Prove that the definition of basis for a subspace in Definition 16.8 is equivalent to each of the following two definitions.

(a) **Minimal spanning set:** A basis for a subspace $\mathcal{W}$ of $\mathbb{R}^n$ is a set of vectors in $\mathcal{W}$ that spans $\mathcal{W}$ and has the property that if any one of the vectors is removed, the resulting set no longer spans $\mathcal{W}$.

(b) **Maximal linearly independent subset:** A basis for a subspace $\mathcal{W}$ of $\mathbb{R}^n$ is a set of vectors in $\mathcal{W}$ with the property that adding any vector from $\mathcal{W}$ to this set will make it linearly dependent.

16.7 (Vector representation relative to a basis) Consider a basis $\{v_1, v_2, \ldots, v_k\}$ for a subspace $\mathcal{W}$ of $\mathbb{R}^n$. Let w be any vector in $\mathcal{W}$. Show that there exist unique scalars $c_1, c_2, \ldots, c_k$ such that $w = c_1v_1 + c_2v_2 + \cdots + c_kv_k$. These scalars c_i are called the **coordinates of** v with respect to the basis $\{v_1, \ldots, v_k\}$.

16.8 Carefully formulate the converse of the result in Problem 16.7 and provide a proof.

16.9 Use Problem 16.5 to prove the subspace extension lemma (Lemma 16.12).

16.10 (The intersection of two subspaces) Consider two subspaces $\mathcal{W}_1$ and $\mathcal{W}_2$ of $\mathbb{R}^n$. Show that their intersection

$$\mathcal{W}_1 \cap \mathcal{W}_2 := \{x \in \mathbb{R}^n \mid x \in \mathcal{W}_1 \text{ and } x \in \mathcal{W}_2\}$$

is also a subspace of $\mathbb{R}^n$.

16.11 Consider two subspaces $\mathcal{W}_1$ and $\mathcal{W}_2$ of $\mathbb{R}^n$. Show that $\mathcal{W}_1 \cap \mathcal{W}_2$ is the largest subspace contained in both $\mathcal{W}_1$ and $\mathcal{W}_2$.

16.12 (The image of a subspace under a linear transformation) Consider a subspace $\mathcal{W}$ of $\mathbb{R}^n$, and let $A \in \mathbb{R}^{m\times n}$. The **image** $A(\mathcal{W})$ **of** $\mathcal{W}$ **under** A is defined as the set of all vectors in $\mathbb{R}^m$ that can be obtained by applying A to vectors in $\mathcal{W}$:

$$A(\mathcal{W}) := \{Ax \mid x \in \mathcal{W}\}.$$

Show that $A(\mathcal{W})$ is a subspace of $\mathbb{R}^m$.

16.13 Verify that the sum $\mathcal{W}_1 + \mathcal{W}_2$ of two subspaces $\mathcal{W}_1$ and $\mathcal{W}_2$ of $\mathbb{R}^n$ is itself a subspace of $\mathbb{R}^n$.

16.14 Consider two subspaces $\mathcal{W}_1$ and $\mathcal{W}_2$ of $\mathbb{R}^n$, and let U be a subspace of $\mathbb{R}^n$ that contains both $\mathcal{W}_1$ and $\mathcal{W}_2$. Show that $\mathcal{W}_1 + \mathcal{W}_2 \subseteq U$, that is, $\mathcal{W}_1 + \mathcal{W}_2$ is the smallest subspace containing both W_1 and W_2.

16.15 (Dimension of a sum lemma) Consider the sum $\mathcal{W}_1 + \mathcal{W}_2$ of two subspaces $\mathcal{W}_1$ and $\mathcal{W}_2$ of $\mathbb{R}^n$. By Problem 16.10, we know that $\mathcal{W}_1 \cap \mathcal{W}_2$ is a subspace of $\mathbb{R}^n$. Let $\{z_1, \ldots, z_t\}$ be a basis for $\mathcal{W}_1 \cap \mathcal{W}_2$. Since $\mathcal{W}_1 \cap \mathcal{W}_2 \subseteq \mathcal{W}_1$ and $\mathcal{W}_1 \cap \mathcal{W}_2 \subseteq \mathcal{W}_2$, we can extend $\{z_1, \ldots, z_t\}$ to a basis $\{z_1, \ldots, z_t, v_1, \ldots, v_m\}$ of $\mathcal{W}_1$ and to a basis $\{z_1, \ldots, z_t, w_1, \ldots, w_n\}$ of $\mathcal{W}_2$. Show that $\{z_1, \ldots, z_t, v_1, \ldots, v_m, w_1, \ldots, w_n\}$ is a basis for $\mathcal{W}_1 + \mathcal{W}_2$. Conclude that

$$\dim(\mathcal{W}_1 + \mathcal{W}_2) = \dim(\mathcal{W}_1) + \dim(\mathcal{W}_2) - \dim(\mathcal{W}_1 \cap \mathcal{W}_2).$$

16.16 Use Problem 16.15 to prove Theorem 16.18.

16.17 (**Direct sum characterization**) Let $\mathcal{W}_1$ and $\mathcal{W}_2$ be subspaces of $\mathbb{R}^n$, and define $\mathcal{U} = \mathcal{W}_1 + \mathcal{W}_2$. Prove that $\mathcal{U} = \mathcal{W}_1 \oplus \mathcal{W}_2$ if and only if the condition $w_1 + w_2 = 0$, where $w_1 \in \mathcal{W}_1$ and $w_2 \in \mathcal{W}_2$, implies $w_1 = w_2 = 0$.

16.18 Prove Theorem 16.20.

16.19 Assume that a matrix B is obtained from A by the application of a row operation. Show that B^T can be obtained from A^T by applying a corresponding column operation. Which operation is this?

16.20 Prove Theorem 16.25.

16.21 Show that every invertible matrix is a product of elementary matrices.

16.22 Prove that row-equivalence, as defined in Definition 16.37, is an equivalence relation. That is, show that it satisfies:

(a) $A \sim_{\mathrm{r}} A$,
(b) if $A \sim_{\mathrm{r}} B$, then $B \sim_{\mathrm{r}} A$, and
(b) if $A \sim_{\mathrm{r}} B$ and $B \sim_{\mathrm{r}} C$, then $A \sim_{\mathrm{r}} C$.

16.23 Assume that a matrix B is obtained from a matrix A by the application of a finite number of row and column operations. Show that there exist two invertible matrices S and T such that $B = SAT$.

16.24 Prove that every matrix is row-equivalent to a matrix in echelon form. In other words, prove Theorem 16.40.

16.25 Prove that any matrix can be transformed into echelon form using only type II and III row operations. Can you transform a matrix A into echelon form using only type III row operations? Prove your answer or provide a counterexample.

16.26 Define a matrix in "column-echelon form" similarly to how we define echelon form for rows. Provide an example to illustrate your definition.

16.27 What is the reduced row echelon form of an invertible matrix A?

16.28 Assume that A is row-equivalent to a matrix in echelon form U. Show that there is a permutation matrix P such that

$$UP = \begin{bmatrix} U_r & M \\ 0 & 0 \end{bmatrix},$$

where U_r is a nonsingular upper triangular matrix and $r = \operatorname{rank}(A)$. What is U_r when A is reduced to reduced row echelon form?

16.29 Consider a 2×2 matrix

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.$$

Assume that $a_{11} \neq 0$. In order to find an upper-triangular matrix row equivalent to A we apply the row operation "add to the second row the first one multiplied by $-a_{11}^{-1}a_{21}$", and we get

$$\begin{bmatrix} a_{11} & a_{12} \\ 0 & a_{22} - a_{12}a_{11}^{-1}a_{21} \end{bmatrix}.$$

Consider now a 2×2 block matrix

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}.$$

Define a block row elementary operation to transform A into a 2×2 block upper triangular matrix. What condition on A_{11} would be necessary for this operation to be applied?

16.30 Let

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

be a 2×2 block matrix such that A_{11} is invertible. Show that A is invertible if and only if $S = A_{22} - A_{21}A_{11}^{-1}A_{12}$ (called the Schur complement of A_{11}) is invertible. How about if we assume that A_{22} is invertible instead? What would be necessary and sufficient conditions for A to be invertible?

16.31 Let

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

where A_{11} is invertible. Show that A can be written as the product of a block lower triangular matrix and a block upper triangular matrix. Then use this factorization to find a formula for A^{-1} in terms of the block entries of A.

Chapter 17
IV-CORE TOPICS

This chapter introduces the four fundamental subspaces of a matrix: the column space, the row space, the (right) nullspace, and the left nullspace. Understanding these subspaces is important for solving many Linear Algebra problems and studying matrix factorizations.

17.1 The column and row spaces

Let us begin by introducing the column and row spaces of a matrix.

Definition 17.1. (The column and row spaces)

Consider an $m \times n$ real matrix A. The **column space** of A is the subspace of $\mathbb{R}^m$ spanned by the columns of A. The **row space** of A is the subspace of $\mathbb{R}^n$ spanned by the transpose of the rows of A. We denote the column space of A by $\text{col}(A)$. Thus, the row space of A is $\text{col}(A^T)$.

Example 17.2 Consider the matrix $A = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 6 \end{smallmatrix}\right]$. The column space of A is

$$\text{col}(A) = \text{span}\left\{\begin{bmatrix} 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 6 \end{bmatrix}\right\} = \text{span}\left\{\begin{bmatrix} 1 \\ 3 \end{bmatrix}\right\}.$$

Geometrically, this is the line in $\mathbb{R}^2$ passing through the origin along the vector $\left[\begin{smallmatrix} 1 \\ 3 \end{smallmatrix}\right]$. Similarly, the row space of A is

$$\text{col}(A^T) = \text{span}\left\{\begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ 6 \end{bmatrix}\right\} = \text{span}\left\{\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right\}.$$

This is also a line in $\mathbb{R}^2$, passing through the origin along the vector $\left[\begin{smallmatrix} 1 \\ 2 \end{smallmatrix}\right]$.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_17

Recall from Proposition 7.1 that a matrix-vector product Ax is a linear combination of the columns of A. This lets us express the column space of a matrix as follows.

Proposition 17.3. (The column space)

For any matrix $A \in \mathbb{R}^{m\times n}$, we have $\text{col}(A) = \{Ax \mid x \in \mathbb{R}^n\}$.

Proof Let $a_1, a_2, \ldots, a_n \in \mathbb{R}^m$ denote the columns of A. We can write any linear combination of these columns as

$$x_1a_1 + x_2a_2 + \cdots + x_na_n = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = Ax.$$

In other words, the set of all products of the form Ax is precisely the subspace spanned by the columns of A. □

This rewriting of the column space connects it with systems of linear equations. Asking if a linear system $Ax = b$ has a solution is the same as asking if b is in the column space of A.

Theorem 17.4. (The column space $\text{col}(A)$ and the solvability of $Ax = b$)

Consider a matrix $A \in \mathbb{R}^{m\times n}$ and a vector $b \in \mathbb{R}^m$. Then the linear system $Ax = b$ has a solution if and only if $b \in \text{col}(A)$.

In Figure 17.1, we illustrate the geometric interpretation of the column space and its relationship to the solvability of $Ax = b$. On the left, we see that b is not in the column space of A, as it is not contained in the plane generated by the columns of A. On the right, we see an example of a vector b that is in the column space of A, and therefore, $Ax = b$ has a solution.

Example 17.5 Consider the following vector and matrix

$$b = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \qquad \text{and} \qquad A = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 2 \\ 0 & 1 & 1 \end{bmatrix}.$$

We want to determine whether b is in the column space of A. By Theorem 17.4, we can answer this by examining the solvability of the linear system $Ax = b$. First, we form the augmented matrix $\left[\, A \middle| b \,\right]$ and use Gaussian Elimination until A is in reduced row echelon form:

$$\left[\begin{array}{ccc|c} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & -1 \end{array}\right].$$

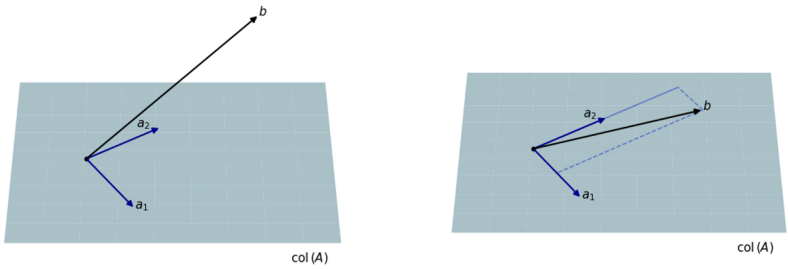


Fig. 17.1: Is b in the column space of A?

This augmented matrix represents an inconsistent system since the last row corresponds to the equation $0x_1 + 0x_2 + 0x_3 = -1$, which has no solutions. Thus, the vector b is not in the column space of A.

We can also think about the column space using the language of functions and linear transformations. To do that, let us start with a definition.

Definition 17.6. (The range of a linear transformation)

The **range** of a linear transformation $f : \mathbb{R}^n \to \mathbb{R}^m$ is defined as

$$R(f) = \{f(x) \mid x \in \mathbb{R}^n\} \subseteq \mathbb{R}^m.$$

Recall that f is **onto** if $R(f) = \mathbb{R}^m$. In other words, for every $y \in \mathbb{R}^m$, there exists some $x \in \mathbb{R}^n$ such that $f(x) = y$.

We will now show that the column space of a matrix characterizes the range of a linear transformation, and the dimension of $\text{col}(A)$ determines whether a linear transformation $f(x) = Ax$ is onto.

Theorem 17.7. (Characterization of the range of a linear transformation)

Consider the linear transformation $f(x) = Ax$, with $A \in \mathbb{R}^{m \times n}$. Then, $R(f) = \text{col}(A)$. Moreover, f is onto if and only if $\dim(\text{col}(A)) = m$.

Proof First, by Proposition 17.3, we have

$$R(f) = \{f(x) \mid x \in \mathbb{R}^n\} = \{Ax \mid x \in \mathbb{R}^n\} = \text{col}(A),$$

which proves the first claim.

To prove the second claim, recall that f is onto if, for all $b \in \mathbb{R}^m$, there exists $x \in \mathbb{R}^n$ such that $b = f(x) = Ax$. According to Theorem 17.4, this happens if and only if $b \in \text{col}(A)$ for all b. Thus, f is onto precisely when $\text{col}(A) = \mathbb{R}^m$ or equivalently, when $\dim(\text{col}(A)) = m$. □

It is interesting to see how the operations of addition and multiplication of matrices affect the column and row spaces.

Proposition 17.8. (The column and row spaces of a matrix sum)

Consider two matrices $A, B \in \mathbb{R}^{m\times n}$. Then,

(a) $\text{col}(A + B) \subseteq \text{col}(A) + \text{col}(B)$, and $\dim(\text{col}(A + B)) \leqslant \dim(\text{col}(A)) + \dim(\text{col}(B))$.
(b) $\text{col}((A+B)^T) \subseteq \text{col}(A^T)+\text{col}(B^T)$, and $\dim\left(\text{col}((A + B)^T)\right) \leqslant \dim\left(\text{col}(A^T)\right)+ \dim\left(\text{col}(B^T)\right)$.

Proof We prove part (a). Part (b) follows by applying part (a) to $(A+B)^T = A^T+B^T$.

Suppose $b \in \text{col}(A + B)$. Then $b = (A + B)x$, for some vector x. Expanding this expression, we get $b = Ax + Bx$. Since $Ax \in \text{col}(A)$ and $Bx \in \text{col}(B)$, we have $b \in \text{col}(A) + \text{col}(B)$. Hence, $\text{col}(A + B) \subseteq \text{col}(A) + \text{col}(B)$.

To show the dimension inequality, we use the dimension-of-the-sum result in Problem 16.15. We have

$$\begin{aligned}\dim(\text{col}(A + B)) &\leqslant \dim(\text{col}(A) + \text{col}(B))\\ &= \dim(\text{col}(A)) + \dim(\text{col}(B)) - \dim(\text{col}(A) \cap \text{col}(B))\\ &\leqslant \dim(\text{col}(A)) + \dim(\text{col}(B)),\end{aligned}$$

as we claimed. □

Proposition 17.9. (The column and row spaces of a matrix product)

Consider two matrices $A \in \mathbb{R}^{m\times k}$ and $B \in \mathbb{R}^{k\times n}$. Then,

(a) $\text{col}(AB) \subseteq \text{col}(A)$, and $\dim(\text{col}(AB)) \leqslant k$; and
(b) $\text{col}((AB)^T) \subseteq \text{col}(B^T)$, and $\dim\left(\text{col}((AB)^T)\right) \leqslant k$.

Proof We prove part (a) for the column space. Part (b) follows by applying part (a) to $(AB)^T = B^T A^T$.

Suppose $b \in \text{col}(AB)$. Then $b = (AB)x$, for some vector x. So, we have $b = (AB)x = A(Bx) = Ay$, with $y = Bx$, which shows that $b \in \text{col}(A)$. Hence, we have $\text{col}(AB) \subseteq \text{col}(A)$, and thus $\dim(\text{col}(AB)) \leqslant \dim(\text{col}(A)) \leqslant k$. □

Example 17.10 Consider the matrices

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \end{bmatrix}, \qquad \text{and} \qquad AB = \begin{bmatrix} 1 & 3 & 5 \\ 1 & 3 & 5 \\ 1 & 3 & 5 \end{bmatrix}.$$

We have

$$\operatorname{col}(AB) = \operatorname{span}\left\{\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}\right\} = \operatorname{col}(A), \quad \text{and}$$

$$\operatorname{col}((AB)^T) = \operatorname{span}\left\{\begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}\right\} \subset \operatorname{span}\left\{\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}\right\} = \operatorname{col}(B^T).$$

We now study how multiplication by an invertible matrix affects the column space.

Theorem 17.11. (Invertible right multiplication preserves the column space)

For any matrix $A \in \mathbb{R}^{m\times n}$ and invertible matrices $S \in \mathbb{R}^{n\times n}$ and $H \in \mathbb{R}^{m\times m}$, we have

$$\operatorname{col}(A) = \operatorname{col}(AS) \quad \text{and} \quad \operatorname{col}(A^T) = \operatorname{col}((HA)^T).$$

Proof By Proposition 17.9-(a), we have $\operatorname{col}(AS) \subseteq \operatorname{col}(A)$.

Suppose now $b \in \operatorname{col}(A)$. Then, $b = Ax$, for some vector $x \in \mathbb{R}^n$. Since S is invertible, we can write $b = Ax = ASS^{-1}x = AS(S^{-1}x) = ASy$, where $y = S^{-1}x$. This shows $b \in \operatorname{col}(AS)$, and thus $\operatorname{col}(A) \subseteq \operatorname{col}(AS)$. Therefore, $\operatorname{col}(A) = \operatorname{col}(AS)$.

For the second claim, notice that H^T is invertible if H is. Thus, by the first part of this proof, we have $\operatorname{col}(A^T) = \operatorname{col}(A^T H^T) = \operatorname{col}((HA)^T)$. □

The previous result shows that multiplying A by an invertible matrix on the right keeps the column space the same, and multiplying on the left keeps the row space the same. But does multiplying on the left preserve the column space, or multiplying on the right preserve the row space? The next example shows that, in general, it does not.

Example 17.12 Consider the matrix $A = \left[\begin{smallmatrix} 1 & 1 \\ 2 & 2 \end{smallmatrix}\right]$ and invertible matrix $S = \left[\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right]$. Then, we have

$$\operatorname{col}(A) = \operatorname{span}\left\{\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right\}, \quad \text{and} \quad \operatorname{col}(SA) = \operatorname{span}\left\{\begin{bmatrix} 1 \\ 3 \end{bmatrix}\right\},$$

and

$$\text{col}(A^T) = \text{span}\left\{\begin{bmatrix}1\\1\end{bmatrix}\right\}, \quad \text{and} \quad \text{col}((AS)^T) = \text{span}\left\{\begin{bmatrix}2\\1\end{bmatrix}\right\}.$$

Thus, multiplying A by an invertible matrix on the left does not preserve its column space, and multiplying A by an invertible matrix on the right does not preserve its row space. However, the dimensions of these spaces stay the same (in the example, both remain one). This is not a coincidence. In Section 17.3, we will see that multiplication by an invertible matrix (either from the left or the right) preserves the dimensions of the row and column spaces.

We end this section with two important insights. A basis for either the column space or the row space of a matrix A produces a factorization of A.

Proposition 17.13. (A basis for $\text{col}(A)$ produces a factorization of A)

Consider a matrix $A \in \mathbb{R}^{m\times n}$. Let $k = \dim(\text{col}(A))$. Suppose $\{u_1, u_2, \ldots, u_k\}$ is a basis for $\text{col}(A)$, and let $U = \begin{bmatrix}u_1 & u_2 & \cdots & u_k\end{bmatrix} \in \mathbb{R}^{m\times k}$ be the matrix with these basis vectors as columns. Then, $A = UV^T$, for some matrix $V \in \mathbb{R}^{n\times k}$.

Proof Let $a_1, a_2, \ldots, a_n$ be the columns of A. Since $\{u_1, u_2, \ldots, u_k\}$ is a basis for $\text{col}(A)$, each column of A is a linear combination of these basis vectors. Thus, for $i = 1, \ldots, n$, we can write

$$a_i = v_{i1}u_1 + v_{i2}u_2 + \cdots + v_{ik}u_k = \begin{bmatrix}u_1 & u_2 & \cdots & u_k\end{bmatrix}\begin{bmatrix}v_{i1}\\v_{i2}\\\vdots\\v_{ik}\end{bmatrix},$$

for some $v_{ij} \in \mathbb{R}$. Putting together all the columns of A, we obtain the following factorization

$$\begin{bmatrix}a_1 & a_2 & \cdots & a_n\end{bmatrix} = \begin{bmatrix}u_1 & u_2 & \cdots & u_k\end{bmatrix}\begin{bmatrix}v_{11} & v_{21} & \cdots & v_{n1}\\v_{12} & v_{22} & \cdots & v_{n2}\\\vdots & \vdots & \ddots & \vdots\\v_{1k} & v_{2k} & \cdots & v_{nk}\end{bmatrix} =: UV^T,$$

as we wanted to show. □

Figure 17.2 illustrates Proposition 17.13. The picture shows the three columns a_1, a_2 and a_3 of a matrix A (the column vectors in blue). These columns are linear combinations of two vectors u_1 and u_2 (the column vectors in red). Writing these linear combinations as a single matrix equation produces a factorization of A.

Is the factorization of A based on a basis for $\text{col}(A)$ unique? In general, no, because there are infinitely many bases for $\text{col}(A)$. Each choice gives a different factorization, as we show in the following example.

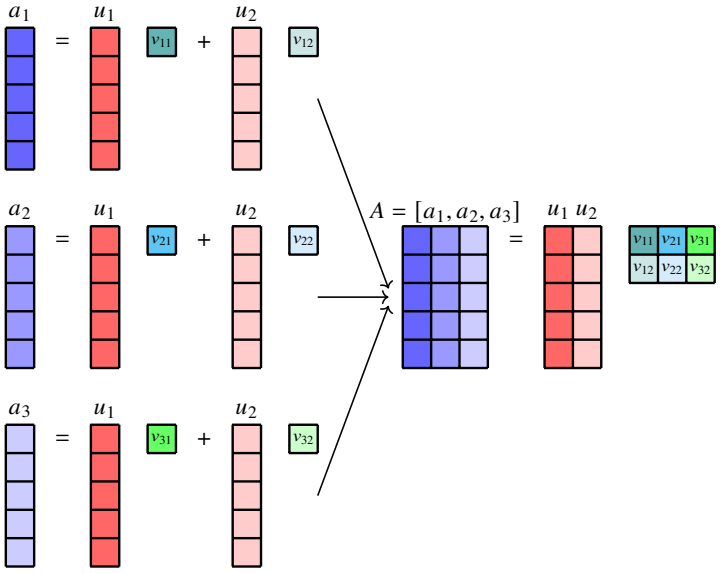


Fig. 17.2: Illustration of matrix factorization using a basis for $\mathrm{col}(A)$.

Example 17.14 Consider the matrix

$$A = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 1 & 0 & 2 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}.$$

Its column space has dimension two (the first two columns are linearly independent, and the other two are linear combinations of the first two). Let us see how different bases for the column space produce different factorizations.

Using the first two columns as a basis, we can express the other two columns of A as

$$\begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} = 2\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

This produces the factorization

$$A = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 1 & 0 & 2 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & -1 & -1 \end{bmatrix} =: U_1 V_1^T.$$

If, instead, we use the second and fourth columns of A as a basis for its column space, we get the factorization

$$A = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 1 & 0 & 2 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 2 & 1 \end{bmatrix} =: U_2 V_2^T.$$

We can even find a factorization of A without using its columns directly by choosing any set of two linearly independent vectors that span the column space of A. Using the vectors $\begin{bmatrix}3 & 3 & 2\end{bmatrix}^T$ and $\begin{bmatrix}2 & 2 & 2\end{bmatrix}^T$, we get

$$A = \begin{bmatrix}1 & 0 & 2 & 1\\ 1 & 0 & 2 & 1\\ 1 & 1 & 1 & 0\end{bmatrix} = \begin{bmatrix}3 & 2\\ 3 & 2\\ 2 & 2\end{bmatrix} \begin{bmatrix} 0 & -1 & 1 & 1\\ 1/2 & 3/2 & -1/2 & -1\end{bmatrix} =: U_3 V_3^T.$$

In Section 17.3.3, we will call these types of factorizations **full-rank factorizations** for reasons that will become clearer later. In Chapters 18 (CUR factorization), 22 (LU factorization), 27 (QR factorization), and 47 (SVD factorization), we will introduce some important methods to produce these factorizations.

As mentioned before, a basis of the row space can also be used to produce a factorization of a matrix A as we show next.

Proposition 17.15. (A basis for $\mathrm{col}(A^T)$ produces a factorization of A)

Consider a matrix $A \in \mathbb{R}^{m\times n}$. Suppose $\{v_1, v_2, \ldots, v_k\}$ is a basis for $\mathrm{col}(A^T)$, and let $V = \begin{bmatrix}v_1 & v_2 & \cdots & v_k\end{bmatrix} \in \mathbb{R}^{n\times k}$ be the matrix with these basis vectors as columns. Then, $A = UV^T$, for some matrix $U \in \mathbb{R}^{m\times k}$.

Proof Just apply Proposition 17.13 to A^T. □

17.2 The right and left nullspaces

In addition to the column and row spaces, a matrix has two other important subspaces: the (right) nullspace and the left nullspace.

Definition 17.16. (The right and left nullspaces)

Consider an $m \times n$ matrix A. We call the **right nullspace** (or, simply, the **nullspace**) of A, and denote it by $\mathrm{null}(A)$, the solution set of the homogeneous linear system of equations $Ax = 0$. In other words,

$$\mathrm{null}(A) := \{x \in \mathbb{R}^n \mid Ax = 0\}.$$

The **left nullspace** of A is the nullspace of A^T:

$$\mathrm{null}(A^T) = \{y \in \mathbb{R}^m \mid y^T A = 0\}.$$

It is clear from the definitions that $\mathrm{null}(A)$ is a subspace of $\mathbb{R}^n$, and $\mathrm{null}(A^T)$ is a subspace of $\mathbb{R}^m$.

Example 17.17 Consider the matrix $A = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 6 \end{smallmatrix}\right]$. We find the nullspace of A by solving the homogeneous system

$$Ax = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 + 2x_2 \\ 3x_1 + 6x_2 \end{bmatrix} = \begin{bmatrix} x_1 + 2x_2 \\ 3(x_1 + 2x_2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Solving this system, we get $x_1 = -2x_2$. Hence, the nullspace of A is

$$\text{null}(A) = \left\{ \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \mid x_1 = -2x_2 \right\} = \text{span} \left\{ \begin{bmatrix} -2 \\ 1 \end{bmatrix} \right\},$$

which is a one-dimensional (line) subspace of $\mathbb{R}^2$.

In Theorem 17.4, we showed that the system $Ax = b$ has a solution if and only if b lies in the column space of A. Now, we will see that when the system is consistent (i.e., it has solutions), the nullspace of A characterizes all possible solutions to $Ax = b$.

Theorem 17.18. (The nullspace of A and the general solution of $Ax = b$)

Consider a consistent linear system of equations $Ax = b$, where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Let x_p be any particular solution to $Ax = b$. Then, we can write any other solution to $Ax = b$ as

$$x = x_p + x_n, \quad \text{for some } x_n \in \text{null}(A).$$

In other words, the solution set $\mathcal{S}$ of $Ax = b$ is given by

$$\mathcal{S} = x_p + \text{null}(A) := \{x_p + x_n \mid x_n \in \text{null}(A)\}.$$

Another way to state the above result is that the difference between any two solutions of $Ax = b$ lies in the nullspace of A. That is, if x_1 and x_2 are two solutions to $Ax = b$, then $x_1 - x_2 \in \text{null}(A)$.

Example 17.19 Consider the linear system of equations $Ax = b$ with

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \qquad \text{and} \qquad b = \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

By inspection, we see that the vector $x_p = \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$ is a particular solution of $Ax = b$. To find the nullspace of A, we row-reduce the augmented matrix $\left[\begin{smallmatrix} A & 0 \end{smallmatrix}\right]$ and obtain

$$\left[\begin{array}{cc|c} 1 & 2 & 0 \\ 0 & 0 & 0 \end{array}\right].$$

Thus, the nullspace of A is span $\left(\begin{bmatrix} 2 \\ -1 \end{bmatrix}\right)$, which is a line passing through the origin in $\mathbb{R}^2$. Hence, any other solution to $Ax = b$ must be of the form $x = x_p + x_n$, where $x_n \in \text{null}(A)$. That is, the solution set $\mathcal{S}$ of $Ax = b$ is given by

$$\mathcal{S} = x_p + \text{null}(A) = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix} + t \begin{bmatrix} -2 \\ 1 \end{bmatrix} \mid t \in \mathbb{R} \right\}.$$

Geometrically, the solution set $\mathcal{S}$ is a translation of null(A) by the particular solution x_p. In this case, $\mathcal{S}$ is the line in $\mathbb{R}^2$, which passes through the point $x_p = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and has direction vector $\begin{bmatrix} -2 \\ 1 \end{bmatrix}$, as shown in Figure 17.3.

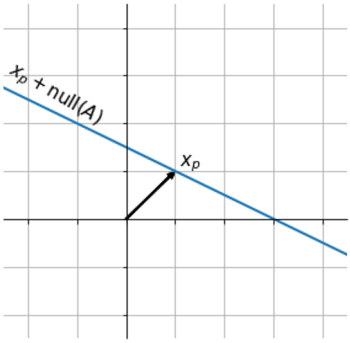


Fig. 17.3: The line through $x_p = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ in the direction of $\begin{bmatrix} 2 \\ -1 \end{bmatrix}$ contains all the solutions of the linear system $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$.

In Theorem 17.7, we saw that the column space characterizes the range of a linear transformation and tells us when it is onto. Similarly, the nullspace characterizes the kernel of a linear transformation and determines when it is one-to-one.

Definition 17.20. (One-to-one linear transformation and kernel)

The **kernel** of a linear transformation $f : \mathbb{R}^n \to \mathbb{R}^m$ is the set

$$\ker(f) = \{x \in \mathbb{R}^n \mid f(x) = 0\}.$$

The function f is called **one-to-one** if $f(x) = f(y)$ implies $x = y$.

Theorem 17.21. (Characterization of the kernel of a linear transformation)

Consider the linear transformation $f(x) = Ax$, with $A \in \mathbb{R}^{m\times n}$. Then, $\ker(f) = \text{null}(A)$. Moreover, f is one-to-one if and only if $\text{null}(A) = \{0\}$.

Proof By definition, the kernel of f is given by

$$\ker(f) = \{x \mid f(x) = 0\} = \{x \mid Ax = 0\} = \text{null}(A),$$

as we claimed.

Suppose f is one-to-one and let $x \in \text{null}(A)$. By definition of nullspace, $f(x) = Ax = 0$. For any linear transformation we always have $f(0) = 0$. This implies that $f(x) = 0 = f(0)$. Since f is one-to-one, it must be that $x = 0$. Therefore, $\text{null}(A) = \{0\}$.

Now suppose that $\text{null}(A) = \{0\}$. Let $x, y \in \mathbb{R}^n$ such that $f(x) = f(y)$. Then, $f(x) - f(y) = Ax - Ay = A(x - y) = 0$. This implies that $x - y$ is in $\text{null}(A)$. Hence, $x - y = 0$ or $x = y$. Therefore, f is one-to-one. □

Just as we did with the column and row spaces, we can study how addition and multiplication affect nullspaces.

Proposition 17.22. (The nullspaces of a matrix sum)

Consider two matrices $A, B \in \mathbb{R}^{m\times n}$. Then,

(a) $\text{null}(A) \cap \text{null}(B) \subseteq \text{null}(A + B)$, and
(b) $\text{null}(A^T) \cap \text{null}(B^T) \subseteq \text{null}((A + B)^T)$.

Proof Suppose $x \in \text{null}(A) \cap \text{null}(B)$. Then, we have $Ax = 0$ and $Bx = 0$. Adding these equations, we obtain $Ax+Bx = (A+B)x = 0$, which shows that $x \in \text{null}(A+B)$. Therefore, $\text{null}(A) \cap \text{null}(B) \subseteq \text{null}(A + B)$, proving part (a). Part (b) follows from applying part (a) to $(A + B)^T = A^T + B^T$. □

Proposition 17.23. (The nullspaces of a matrix product)

Consider two matrices $A \in \mathbb{R}^{n\times k}$ and $B \in \mathbb{R}^{k\times n}$. Then,

(a) $\text{null}(B) \subseteq \text{null}(AB)$, and
(b) $\text{null}(A^T) \subseteq \text{null}((AB)^T)$.

Proof Suppose $x \in \text{null}(B)$. Then, we have $Bx = 0$. Thus, $ABx = 0$, which implies that $x \in \text{null}(AB)$. Therefore, $\text{null}(B) \subseteq \text{null}(AB)$. Part (b) follows by applying part (a) to $(AB)^T = B^T A^T$. □

Next, we show how multiplication by an invertible matrix affects the nullspace.

Theorem 17.24. (Invertible left multiplication preserves the nullspace)

For any matrix $A \in \mathbb{R}^{m\times n}$ and invertible matrices $S \in \mathbb{R}^{m\times m}$ and $H \in \mathbb{R}^{n\times n}$, we have

$$\text{null}(A) = \text{null}(SA) \quad \text{and} \quad \text{null}(A^T) = \text{null}((AH)^T).$$

Proof By Proposition 17.23-(a), we have $\text{null}(A) \subseteq \text{null}(SA)$.

Suppose $x \in \text{null}(SA)$. Then, $SAx = 0$. Since S is invertible, $SAx = 0$ implies $Ax = 0$. Hence, $x \in \text{null}(A)$, and thus $\text{null}(SA) \subseteq \text{null}(A)$. Therefore, we have shown that $\text{null}(A) = \text{null}(SA)$.

The second claim follows from the first one applied to A^T and H^T. □

Similar to the column and row space cases, we wonder if multiplying A by an invertible matrix S on the right (or left) also preserves the nullspace (or left nullspace). Let us illustrate this with an example.

Example 17.25 As in Example 17.12, consider the matrix $A = \left[\begin{smallmatrix} 1 & 1 \\ 2 & 2 \end{smallmatrix}\right]$ and invertible matrix $S = \left[\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right]$. Then,

$$\text{null}(A) = \text{span}\left\{\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right\} \quad \text{and} \quad \text{null}(AS) = \text{span}\left\{\begin{bmatrix} -1 \\ 2 \end{bmatrix}\right\}.$$

Also,

$$\text{null}(A^T) = \text{span}\left\{\begin{bmatrix} -2 \\ 1 \end{bmatrix}\right\} \quad \text{and} \quad \text{null}((SA)^T) = \text{span}\left\{\begin{bmatrix} -3 \\ 1 \end{bmatrix}\right\}.$$

Thus, multiplying A by an invertible matrix on the right does not preserve its nullspace, and multiplying A by an invertible matrix on the left does not preserve its left nullspace. However, note that the dimensions of the nullspaces remain unchanged. This is not a coincidence. In Section 17.3, we will prove that multiplication by an invertible matrix, either from the left or the right, preserves these dimensions.

17.3 The rank of a matrix

As we have seen, every $m \times n$ matrix A leads to four subspaces that we call fundamental subspaces of A. The next definition summarizes them.

Definition 17.26. (The four fundamental subspaces)

The **four fundamental subspaces** associated with a matrix $A \in \mathbb{R}^{m\times n}$ are

the column space:	$\text{col}(A) = \{Ax \mid x \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$,
the row space:	$\text{col}(A^T) = \{A^T y \mid y \in \mathbb{R}^m\} \subseteq \mathbb{R}^n$,
the (right) nullspace:	$\text{null}(A) = \{x \in \mathbb{R}^n \mid Ax = 0\} \subseteq \mathbb{R}^n$, and
the left nullspace:	$\text{null}(A^T) = \{y \in \mathbb{R}^m \mid A^T y = 0\} \subseteq \mathbb{R}^m$.

If we think of $A \in \mathbb{R}^{m\times n}$ as a linear transformation, we can represent its four fundamental subspaces as shown in Figure 17.4. At this point, we only know where these subspaces live: $\text{col}(A^T)$ and $\text{null}(A)$ are subspaces of $\mathbb{R}^n$, while $\text{col}(A)$ and $\text{null}(A^T)$ are subspaces of $\mathbb{R}^m$. As we move forward, we will refine this picture and see why these spaces are called "fundamental".

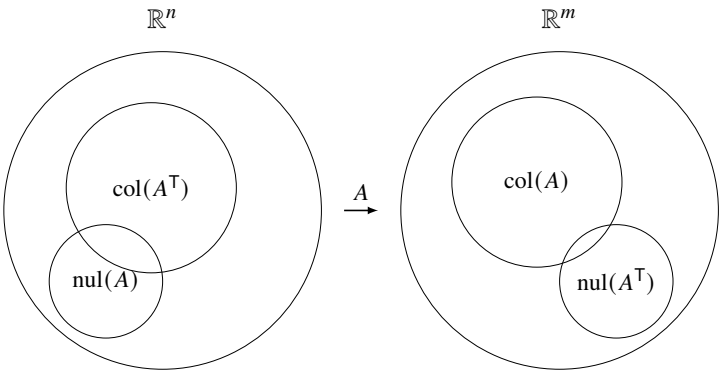


Fig. 17.4: Representation of the four fundamental subspaces of an $m \times n$ matrix A when seen as a linear transformation: $\text{col}(A^T)$ and $\text{null}(A)$ are subspaces of $\mathbb{R}^n$, while $\text{col}(A)$ and $\text{null}(A^T)$ are subspaces of $\mathbb{R}^m$

In this section and the one that follows, we will see how the dimensions of the four fundamental subspaces are connected. Let us start with an example.

Example 17.27 Consider the matrix

$$A = \begin{bmatrix} 1 & 0 \\ 1 & -1 \\ 2 & 0 \end{bmatrix}.$$

Then, the four fundamental subspaces of A are

$$\operatorname{col}(A) = \operatorname{span}\left\{\begin{bmatrix}1\\1\\2\end{bmatrix}, \begin{bmatrix}0\\-1\\0\end{bmatrix}\right\}, \qquad \operatorname{null}(A^T) = \operatorname{span}\left\{\begin{bmatrix}-2\\0\\1\end{bmatrix}\right\},$$
$$\operatorname{col}(A^T) = \operatorname{span}\left\{\begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}1\\-1\end{bmatrix}\right\}, \quad \text{and} \quad \operatorname{null}(A) = \left\{\begin{bmatrix}0\\0\end{bmatrix}\right\}.$$

Note that the column space and the left nullspace of A are subspaces of $\mathbb{R}^3$, while the row space and the nullspace of A are subspaces of $\mathbb{R}^2$. Also, observe that

$$\begin{aligned}&\dim(\operatorname{col}(A)) = \dim(\operatorname{col}(A^T)) = 2,\\ &\dim(\operatorname{col}(A)) + \dim(\operatorname{null}(A^T)) = \dim(\mathbb{R}^3) = 3, \quad \text{and}\\ &\dim(\operatorname{col}(A^T)) + \dim(\operatorname{null}(A)) = \dim(\mathbb{R}^2) = 2.\end{aligned}$$

It is no coincidence that the column and row spaces have the same dimension, or that the dimensions of the column space and left nullspace add up to the number of rows, just as the row space and nullspace dimensions add up to the number of columns. Proving these relationships is one of our main goals in this chapter.

Before we proceed, let us introduce some concepts.

Definition 17.28. (The column and row rank of a matrix)

Consider a matrix $A \in \mathbb{R}^{m\times n}$. The **column rank** of A, denoted by $\operatorname{colrank}(A)$, is the dimension of its column space $\operatorname{col}(A)$. Similarly, the **row rank** of A, denoted by $\operatorname{rowrank}(A)$, is the dimension of its row space $\operatorname{col}(A^T)$.

Example 17.29 Let us revisit Example 17.27. Note that the matrix

$$A = \begin{bmatrix}1 & 0\\ 1 & -1\\ 2 & 0\end{bmatrix}$$

has a column space that is spanned by two linearly independent vectors, so $\operatorname{colrank}(A) = 2$. Similarly, the row space of A is spanned by two linearly independent vectors, so $\operatorname{rowrank}(A) = 2$ as well. Hence, even though $\operatorname{col}(A)$ and $\operatorname{col}(A^T)$ "live" in different spaces ($\mathbb{R}^3$ and $\mathbb{R}^2$, respectively), they have the same dimension.

The equality of the column rank and row rank in the previous example is not a coincidence.

Theorem 17.30. (The column rank equals the row rank)

For any matrix $A \in \mathbb{R}^{m\times n}$, we have $\text{rowrank}(A) = \text{colrank}(A)$.

Proof If A is the zero matrix, the result holds since both column and row ranks are zero. Now, assume $A \neq 0$.

Let k denote the column rank of A, and let $\{u_1, \ldots, u_k\}$ be a basis for $\text{col}(A)$. Place these basis vectors as columns of a matrix $U = \begin{bmatrix} u_1 \cdots u_k \end{bmatrix} \in \mathbb{R}^{m\times k}$. By Proposition 17.13, we can factor A as $A = UV^T$, for some matrix $V \in \mathbb{R}^{n\times k}$.

Let $v_1, \ldots, v_k$ denote the columns of V. We can write any vector $A^T y$ in the row space of A as

$$A^T y = VU^T y = \begin{bmatrix} v_1 \cdots v_k \end{bmatrix} \begin{bmatrix} u_1^T \\ \vdots \\ u_k^T \end{bmatrix} y = \begin{bmatrix} v_1 \cdots v_k \end{bmatrix} \begin{bmatrix} u_1^T y \\ \vdots \\ u_k^T y \end{bmatrix} = \sum_{i=1}^{k} (u_i^T y) v_i.$$

This means that every vector in the row space of A is a linear combination of the vectors $v_1, \ldots, v_k$. Therefore, $\text{col}(A^T) = \text{span}(v_1, \ldots, v_k)$, which implies

$$\text{rowrank}(A) \leqslant k = \text{colrank}(A). \tag{17.1}$$

If we swap the roles of A and A^T in the above argument, we obtain

$$\text{rowrank}(A^T) \leqslant \text{colrank}(A^T). \tag{17.2}$$

Combining (17.1) and (17.2), we get

$$\begin{aligned} \text{rowrank}(A) &\leqslant \text{colrank}(A) = \text{rowrank}(A^T) \\ &\leqslant \text{colrank}(A^T) = \text{rowrank}(A). \end{aligned}$$

Therefore, $\text{rowrank}(A) = \text{colrank}(A)$. □

Definition 17.31. (Matrix rank and full rank matrices)

Since the column rank and the row rank of a matrix are the same, from now on we refer to that common number as the **rank** of A, and denote it by $\text{rank}(A)$.

We call an $m \times n$ matrix A **full column rank** if $\text{rank}(A) = n$ and **full row rank** if $\text{rank}(A) = m$. We say A has **full rank** if

$$\text{rank}(A) = \min\{m, n\},$$

that is, the smaller of the number of rows or columns.

Next, we present two immediate properties of matrix rank.

Proposition 17.32. (Basic properties of the rank)

For any $m \times n$ matrix A, we have

(a) $\text{rank}(A) = \text{rank}(A^T)$, and
(b) $\text{rank}(A) \leqslant \min\{m, n\}$.

Proof Part (a) is just a restatement of Theorem 17.30. For part (b), an $m \times n$ matrix A has at most m linearly independent rows and at most n linearly independent columns. Therefore, the rank of A cannot exceed the smaller of these two values. □

Example 17.33 Consider a matrix A of size $3 \times n$. The rank of A is at most 3, since the number of linearly independent columns cannot exceed the number of rows (by Proposition 17.32-part (b)). We can interpret the column space $\text{col}(A)$ as a subspace of $\mathbb{R}^3$ in terms of its dimension, which is equal to the rank of A. We have

(i) If $\text{rank}(A) = 0$, then $A = 0$ and $\text{col}(A) = 0$, which corresponds to a single point at the origin of $\mathbb{R}^3$.
(ii) If $\text{rank}(A) = 1$, then $\text{col}(A)$ is a line in $\mathbb{R}^3$ passing through zero.
(iii) If $\text{rank}(A) = 2$, then $\text{col}(A)$ is a plane in $\mathbb{R}^3$ passing through zero.
(iv) If $\text{rank}(A) = 3$, then $\text{col}(A) = \mathbb{R}^3$, which corresponds to the entire three-dimensional space.

The following theorem shows an important property of matrix rank: it is preserved under multiplication by invertible matrices.

Theorem 17.34. (Invariance of rank under multiplication by invertible matrices)

For any matrix $A \in \mathbb{R}^{m \times n}$ and invertible matrices $S \in \mathbb{R}^{m \times m}$ and $H \in \mathbb{R}^{n \times n}$, we have

$$\text{rank}(A) = \text{rank}(AH) = \text{rank}(SA) = \text{rank}(SAH).$$

Proof From Theorem 17.11, we have $\text{col}(A) = \text{col}(AH)$, which implies $\text{rank}(A) = \text{rank}(AH)$. We also have $\text{rank}(SA) = \text{rank}((SA)^T) = \text{rank}(A^T S^T) = \text{rank}(A^T) = \text{rank}(A)$, because rank is preserved by transposition and S^T is also invertible. Finally, if we combine the two results we just proved, we get $\text{rank}(SAH) = \text{rank}(S(AH)) = \text{rank}(AH) = \text{rank}(A)$. □

17.3.1 The rank-nullity theorem and the fundamental theorem of Linear Algebra

There is a trade-off between the dimension of the nullspace of a matrix A and its rank.[1]

Theorem 17.35. (The rank-nullity theorem)

For any matrix $A \in \mathbb{R}^{m\times n}$, we have

$$\text{rank}(A) + \dim(\text{null}(A)) = n.$$

Proof We will outline the main ideas of the proof, and you will complete the details in Problem 17.11.

Suppose $k = \dim(\text{null}(A))$ and let $\{n_1, \ldots, n_k\}$ be a basis for $\text{null}(A)$. Since $\text{null}(A) \subseteq \mathbb{R}^n$, we have $k \leqslant n$ by the Subspace Dimension Lemma (Lemma 16.11). Extend $\{n_1, \ldots, n_k\}$ to a basis $\{n_1, \ldots, n_k, v_1, \ldots, v_{n-k}\}$ of $\mathbb{R}^n$. We claim that the set of vectors $\{Av_1, \ldots, Av_{n-k}\}$ forms a basis for $\text{col}(A)$ (this is the part you need to prove). Therefore, $\text{rank}(A) = \dim(\text{col}(A)) = n - k = n - \dim(\text{null}(A))$. □

The rank-nullity theorem relates the dimensions of the four fundamental subspaces of a matrix to its rank:

$$\begin{aligned}
&\dim(\text{col}(A)) = \text{rank}(A),\\
&\dim(\text{col}(A^T)) = \text{rank}(A),\\
&\dim(\text{null}(A)) = n - \text{rank}(A),\\
&\dim(\text{null}(A^T)) = m - \text{rank}(A).
\end{aligned} \tag{17.3}$$

The first result is simply the definition of rank, not a result of the rank-nullity theorem, but it is included to give the dimensions of all four subspaces.

But there is more to these subspaces. They let us express $\mathbb{R}^n$ and $\mathbb{R}^m$ as direct sums of pairs of fundamental subspaces. Recall that a direct sum means every vector in the space can be written uniquely as the sum of one vector from each subspace.

Theorem 17.36. (The fundamental theorem of Linear Algebra - Part I)

For a matrix $A \in \mathbb{R}^{m\times n}$, we have

$$\begin{aligned}
\mathbb{R}^n &= \text{col}(A^T) \oplus \text{null}(A), \qquad \text{and}\\
\mathbb{R}^m &= \text{col}(A) \oplus \text{null}(A^T).
\end{aligned}$$

This means every vector in $\mathbb{R}^n$ can be written uniquely as the sum of a vector in $\text{null}(A)$ and a vector in $\text{col}(A^T)$, and similarly for $\mathbb{R}^m$ with $\text{col}(A)$ and $\text{null}(A^T)$.

[1] Some books refer to the dimension of the nullspace as "nullity", which is why the theorem is called the rank-nullity theorem.

Proof We will prove the first direct sum, $\mathbb{R}^n = \text{col}(A^T) \oplus \text{null}(A)$. The second one follows in the same way by replacing A with A^T.

First, we show that the intersection of $\text{null}(A)$ and $\text{col}(A^T)$ contains only the zero vector. Suppose $v \in \text{null}(A) \cap \text{col}(A^T)$. Then there exists a vector x such that $v = A^T x$, and since $v \in \text{null}(A)$, we also have $Av = 0$.

We have

$$\|v\|_2^2 = v^T v = (A^T x)^T v = x^T A v = x^T A v = 0.$$

So $\|v\|_2^2 = 0$, which implies $v = 0$. Therefore, $\text{null}(A) \cap \text{col}(A^T) = \{0\}$.

Next, we use the dimension formula for the sum of two subspaces (see Problem 16.15):

$$\dim\left(\text{null}(A) + \text{col}(A^T)\right) = \dim\left(\text{null}(A)\right) + \dim\left(\text{col}(A^T)\right) - \dim\left(\text{null}(A) \cap \text{col}(A^T)\right).$$

We just proved that the intersection is $\{0\}$, so the last term is 0. Using the dimension formulas from (17.3), we get

$$\dim\left(\text{null}(A) + \text{col}(A^T)\right) = (n - \text{rank}(A)) + \text{rank}(A) = n.$$

Hence, $\mathbb{R}^n = \text{null}(A) + \text{col}(A^T)$. And since the intersection is trivial, this sum is direct by Theorem 16.17. So we have $\mathbb{R}^n = \text{null}(A) \oplus \text{col}(A^T)$. □

We can now return to Figure 17.4 and refine our visual understanding of the fundamental subspaces using our new result. The updated diagram appears in Figure 47.10. It shows not only where each subspace lives, but also highlights that each pair intersects only at the zero vector, reflecting the fact that they form direct sums.

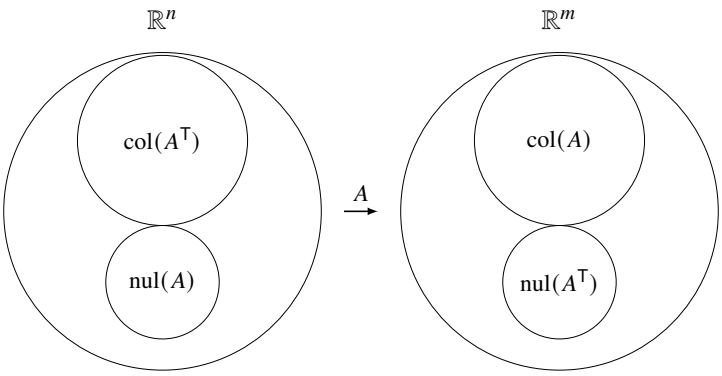


Fig. 17.5: Given an $m \times n$ matrix A, the spaces $\mathbb{R}^n$ and $\mathbb{R}^m$ can be expressed as the direct sum of two of the four fundamental subspace of A.

Still, Figure 47.10 falls short in conveying how these subspaces together add up to all of $\mathbb{R}^n$ and $\mathbb{R}^m$ as direct sums. That part is much harder to represent visually! In Chapter 27, when we introduce the second part of the Fundamental Theorem of Linear Algebra, we will revisit this and try to give a more complete picture.

Another consequence of Theorem 17.36, together with the dimension formulas in (17.3), is a characterization of invertible matrices and linear transformations.

Proposition 17.37. (Characterization of invertible matrices in terms of fundamental subspaces)

A matrix $A \in \mathbb{R}^{n\times n}$ is invertible if and only if

$$\text{null}(A) = \{0\}, \quad \text{col}(A^T) = \mathbb{R}^n, \quad \text{null}(A^T) = \{0\}, \quad \text{and} \quad \text{col}(A) = \mathbb{R}^n.$$

Note that the four conditions in Proposition 17.37 are not independent. By Theorem 17.36, together with (17.3), if any one of them holds, the other three do as well. So to check whether a matrix is invertible, it is enough to verify just one of these four subspace conditions.

17.3.2 The rank of AB and $A + B$

How does the rank behave under matrix addition and multiplication? How much can it increase or decrease? The following two theorems provide estimates for the ranks of matrix products and sums based on the ranks of the individual matrices.

Theorem 17.38. (The rank of a sum)

For any matrices $A, B \in \mathbb{R}^{m\times n}$, we have

$$|\text{rank}(A) - \text{rank}(B)| \leqslant \text{rank}(A + B) \leqslant \text{rank}(A) + \text{rank}(B).$$

Proof The right inequality follows from

$$\begin{aligned} \text{rank}(A + B) &= \dim(\text{col}(A + B)) && \text{(Definition 17.31)} \\ &\leqslant \dim(\text{col}(A)) + \dim(\text{col}(B)) && \text{(Proposition 17.8)} \\ &= \text{rank}(A) + \text{rank}(B). \end{aligned}$$

To prove the left inequality, let $C = A + B$. This implies $A = C + (-B)$ and $B = C + (-A)$. Using the right inequality we just proved, we have

$$\begin{aligned} \text{rank}(A) &= \text{rank}(C + (-B)) \\ &\leqslant \text{rank}(C) + \text{rank}(-B) \\ &= \text{rank}(C) + \text{rank}(B). \end{aligned}$$

Rearranging, we get

$$\text{rank}(A) - \text{rank}(B) \leqslant \text{rank}(C) = \text{rank}(A + B). \tag{17.4}$$

Similarly,

$$\begin{aligned}\operatorname{rank}(B) &= \operatorname{rank}(C + (-A)) \\ &\leqslant \operatorname{rank}(C) + \operatorname{rank}(-A) \\ &= \operatorname{rank}(C) + \operatorname{rank}(A).\end{aligned}$$

Rearranging, we get

$$\operatorname{rank}(B) - \operatorname{rank}(A) \leqslant \operatorname{rank}(C) = \operatorname{rank}(A + B). \tag{17.5}$$

Combining the inequalities (17.4) and (17.5), we obtain the desired result. □

Theorem 17.39. (The rank of a matrix product)

For any matrices $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$, we have

$$\operatorname{rank}(A) + \operatorname{rank}(B) - n \leqslant \operatorname{rank}(AB) \leqslant \min\{\operatorname{rank}(A), \operatorname{rank}(B)\}.$$

Proof Proposition 17.9 gives $\operatorname{col}(AB) \subseteq \operatorname{col}(A)$. By the Subspace Dimension Lemma (Lemma 16.11), this implies $\dim(\operatorname{col}(AB)) \leqslant \dim(\operatorname{col}(A))$, so $\operatorname{rank}(AB) \leqslant \operatorname{rank}(A)$. Taking the transpose, which preserves the rank, we get $\operatorname{rank}(AB) = \operatorname{rank}((AB)^T) \leqslant \operatorname{rank}(B^T) = \operatorname{rank}(B)$. Combining these inequalities gives the right inequality.

Proving the left inequality, known as the Sylvester's inequality, is more challenging. To establish it, we need to analyze the dimensions of the nullspaces of the matrices A, B and AB.

Applying the rank-nullity theorem (Theorem 17.35) to A, B and AB, we obtain

$$\begin{aligned} n &= \operatorname{rank}(A) + \dim(\operatorname{null}(A)), \\ p &= \operatorname{rank}(B) + \dim(\operatorname{null}(B)), \qquad \text{and} \\ p &= \operatorname{rank}(AB) + \dim(\operatorname{null}(AB)).\end{aligned}$$

Thus,

$$\begin{aligned} n + p &= \operatorname{rank}(A) + \dim(\operatorname{null}(A)) + \operatorname{rank}(B) + \dim(\operatorname{null}(B)) \\ &= n + \operatorname{rank}(AB) + \dim(\operatorname{null}(AB)).\end{aligned}$$

Rearranging, we get

$$\operatorname{rank}(AB) - \operatorname{rank}(A) - \operatorname{rank}(B) + n = \dim(\operatorname{null}(A)) + \dim(\operatorname{null}(B)) - \dim(\operatorname{null}(AB)).$$

To finish the proof, it suffices to show that $\dim(\operatorname{null}(AB)) \leqslant \dim(\operatorname{null}(A)) + \dim(\operatorname{null}(B))$.

We will outline the proof, and you will complete the details. Let $k = \dim(\operatorname{null}(B))$ and $t = \dim(\operatorname{null}(AB))$, and consider a basis $\{n_1, \ldots, n_k\}$ for $\operatorname{null}(B)$. By Propo-

sition 17.9, $\text{null}(B) \subseteq \text{null}(AB)$, so $k \leqslant t$. By Lemma 16.12, we can extend $\{n_1, \ldots, n_k\}$ to a basis $\{n_1, \ldots, n_k, n_{k+1}, \ldots, n_t\}$ for $\text{null}(AB)$.

We claim that the $t - k$ vectors $Bn_{k+1}, \ldots, Bn_t$ are linearly independent vectors in $\text{null}(A)$ (this is the part you need to prove). Hence, $\dim(\text{null}(A)) \geqslant t - k = \dim(\text{null}(AB)) - \dim(\text{null}(B))$, completing the proof. □

In Chapter 18, you will find an alternative proof of Sylvester's inequality using block matrices.

17.3.3 Characterization of rank-k matrices and full-rank factorizations

In many applications, it is important to work with matrices of a specific rank. Rank-one matrices, in particular, will play a key role in several methods we will introduce later. In this section, we will see how to characterize matrices with a fixed rank.

Let us begin with rank-one matrices, our main tool.

Theorem 17.40. (Rank-one matrices)

A matrix $A \in \mathbb{R}^{m \times n}$ has $\text{rank}(A) = 1$ if and only if $A = uv^T$, for some nonzero vectors $u \in \mathbb{R}^m$ and $v \in \mathbb{R}^n$.

Proof Assume first that $A = uv^T$, for some nonzero vectors $u \in \mathbb{R}^m$ and $v \in \mathbb{R}^n$. Then all the columns of A are multiples of u, so $\text{col}(A) = \text{span}(u)$. Therefore, $\text{rank}(A) = \dim(\text{col}(A)) = 1$.

Conversely, assume $\text{rank}(A) = 1$. Since the column space of A is one dimensional, it is spanned by a nonzero vector u. Let $a_1, \ldots, a_n$ be the columns of A. Each column must be a multiple of u, so for $i = 1, \ldots, n$, we have $a_i = v_i u$, for some scalar v_i. Thus, $A = uv^T$, where v is the vector with entries v_i. □

We have shown that a rank-one matrix has the form $A = uv^T$, for some nonzero vectors u and v. Each column of A is a multiple of u. A sum of k rank-one matrices can be written as

$$u_1 v_1^T + \cdots + u_k v_k^T = \begin{bmatrix} u_1 & \cdots & u_k \end{bmatrix} \begin{bmatrix} v_1^T \\ \vdots \\ v_k^T \end{bmatrix} =: UV^T.$$

Now observe that

$$\text{rank}(A) = \text{rank}(UV^T) \leqslant \text{rank}(U) \leqslant k.$$

The next theorem tells us exactly when we get equality in this rank inequality.

Theorem 17.41. (Rank-k matrices)

Let $A \in \mathbb{R}^{m \times n}$. Then, A has $\text{rank}(A) = k$ if and only if we can factor A as $A = UV^T$, for some matrices $U \in \mathbb{R}^{m \times k}$ and $V \in \mathbb{R}^{n \times k}$ with $\text{rank}(U) = \text{rank}(V) = k$.

Equivalently, A has $\text{rank}(A) = k$ if and only if we can write A as the sum of k rank-one matrices $A = u_1 v_1^T + \cdots + u_k v_k^T$, where $\{u_1, \ldots, u_k\}$ and $\{v_1, \ldots, v_k\}$ are both linearly independent sets.

Proof Suppose $\text{rank}(A) = k$. Consider a basis $\{u_1, \ldots, u_k\}$ for $\text{col}(A)$ and let $U = \begin{bmatrix} u_1 & \cdots & u_k \end{bmatrix} \in \mathbb{R}^{m \times k}$. The matrix U has rank k because its k columns are linearly independent. By Proposition 17.13, we can factor A as $A = UV^T$, for some matrix $V \in \mathbb{R}^{k \times n}$.

Denote the columns of V by $v_1, \ldots, v_k$. The vectors $v_1, \ldots, v_k$ must be linearly independent. Otherwise, the row space of A would be spanned by fewer than k vectors, since $\text{col}(A^T) \subseteq \text{col}(V)$, violating the column-rank-equals-row-rank theorem (Theorem 17.30). Thus, V has rank k.

Conversely, assume $A = UV^T$, for some matrices $U \in \mathbb{R}^{m \times k}$ and $V \in \mathbb{R}^{n \times k}$ with $\text{rank}(U) = \text{rank}(V) = k$. Since $\text{rank}(V) = \text{rank}(V^T)$, Theorem 17.39 gives

$$\text{rank}(U) + \text{rank}(V) - k = k \leqslant \text{rank}(A) \leqslant \min\{\text{rank}(U), \text{rank}(V)\} = k.$$

Thus, $\text{rank}(A) = k$.

The equivalence of the second statement to the first follows directly from the column-by-row view of matrix multiplication. □

The factorization $A = UV^T$, where $U \in \mathbb{R}^{m \times k}$ and $V \in \mathbb{R}^{n \times k}$ have full column rank, is called a **full-rank factorization of** A. Figure 17.6 shows a typical example of this factorization. With a full-rank factorization, we can determine the rank of A from the dimensions of U and V.

However, finding a full-rank factorization of A is not straightforward. In Propositions 17.13 and 17.15, we learned that we need a basis for the column space or row space of A. We will cover techniques for this in Chapters 18 (CUR factorization), 22 (LU factorization), 27 (QR factorization), and 47 (SVD factorization).

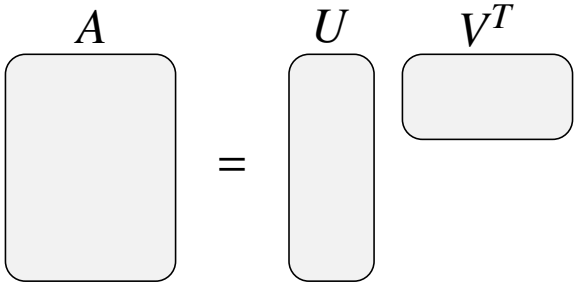


Fig. 17.6: Full-rank factorization.

The following result can be deduced directly from the proof of Theorem 17.41.

Proposition 17.42. (The columns of V form a basis for the row space of UV^T)

Let $A = UV^T$ be a factorization of A such that $\text{col}(U) = \text{col}(A)$. Then, the columns of V form a basis for the row space of A.

Example 17.43 Let us revisit Example 17.14. Let us show that the columns of V_1, for example, form a basis for the row space of A. Note first that V_1 has rank 2, as A has. Moreover, the first row of V_1 equals the first row of A, and the second row of A is the sum of the two rows of V_1. Thus, the row space of A is a subspace of the row space of V_1^T and both have the same dimension, which implies they are the same.

A full-rank factorization of an $m \times n$ matrix A with rank k captures all the essential information in a more compact form. Since each column and each row of A can be written as a linear combination of at most k others, the rest are redundant. When k is much smaller than m or n, this works like data compression. For more on this idea, see Chapter 19.

17.4 Gram-matrices

Gram matrices are a special type of square matrix that appear in fields like Numerical Analysis, Statistics, Data Science, and Signal Processing. We will see them again when we solve the least squares problem in Chapters 19 and 32.

Definition 17.44. (Gram matrix)

Consider a matrix $A \in \mathbb{R}^{m\times n}$ with columns $a_1, a_2, \ldots, a_n$. The **Gram matrix** associated with A is the $n \times n$ matrix

$$A^TA = \begin{bmatrix} a_1^Ta_1 & a_1^Ta_2 & \cdots & a_1^Ta_n \\ a_2^Ta_1 & a_2^Ta_2 & \cdots & a_2^Ta_n \\ \vdots & \vdots & \ddots & \vdots \\ a_n^Ta_1 & a_n^Ta_2 & \cdots & a_n^Ta_n \end{bmatrix},$$

which contains the dot products of all pairs of columns of A. Note that the matrix AA^T is the Gram matrix associated with A^T and contains all the dot products of pairs of rows of A.

The column space, nullspace and rank of the Gram matrices AA^T and A^TA are directly related to those of A and A^T, as we show in the following theorem.

Theorem 17.45. (The four fundamental subspaces of a Gram matrix)

For any matrix $A \in \mathbb{R}^{m\times n}$, we have

$$\begin{aligned} &\operatorname{col}(A^TA) = \operatorname{col}((A^TA)^T) = \operatorname{col}(A^T), && \text{(column and row spaces)} \\ &\operatorname{null}(A^TA) = \operatorname{null}((A^TA)^T) = \operatorname{null}(A) && \text{(right and left nullspaces).} \end{aligned}$$

Moreover,

$$\operatorname{rank}(A) = \operatorname{rank}(A^T) = \operatorname{rank}(A^TA) = \operatorname{rank}(AA^T).$$

Proof First we show that $\operatorname{null}(A^TA) = \operatorname{null}(A)$. We have $\operatorname{null}(A) \subseteq \operatorname{null}(A^TA)$ by Proposition 17.23. Now, assume $x \in \operatorname{null}(A^TA)$. Then, $A^TAx = 0$. Multiplying both sides by x^T on the left gives $0 = x^TA^TAx = (Ax)^T(Ax) = \|Ax\|_2^2$. Therefore, $Ax = 0$, and hence, $x \in \operatorname{null}(A)$. Thus, $\operatorname{null}(A^TA) = \operatorname{null}(A)$. Replacing A by A^T, we have $\operatorname{null}(A^T) = \operatorname{null}(AA^T)$.

Next, applying the rank-nullity theorem (Theorem 17.35) to A^TA and A gives

$$\begin{aligned} \operatorname{rank}(A^TA) &= n - \dim(\operatorname{null}(A^TA)) \\ &= n - \dim(\operatorname{null}(A)) = \operatorname{rank}(A). \end{aligned}$$

Thus,

$$\operatorname{rank}(A) = \operatorname{rank}(A^TA).$$

Applying the rank-nullity theorem to AA^T and A^T, we obtain

$$\operatorname{rank}(A^T) = \operatorname{rank}(AA^T).$$

Since $\operatorname{rank}(A) = \operatorname{rank}(A^T)$, the rank statement in the theorem holds.

To show $\operatorname{col}(A^T) = \operatorname{col}(A^TA)$, first note that $\operatorname{col}(A^TA) \subseteq \operatorname{col}(A^T)$ by Proposition 17.9. Since $\dim(\operatorname{col}(A^TA)) = \operatorname{rank}(A^TA) = \operatorname{rank}(A^T) = \dim(\operatorname{col}(A^T))$, we conclude by the Subspace Dimension Lemma (Lemma 16.11) that $\operatorname{col}(A^TA) = \operatorname{col}(A^T)$.

Finally, notice that the Gram matrix A^TA satisfies $(A^TA)^T = A^TA$. We will call this property *symmetric* in Part IX. This property implies that the row and column spaces of A^TA are the same, as are its right and left nullspace, completing the proof of the theorem.

The following theorem characterizes invertible Gram matrices.

Theorem 17.46. (Invertible Gram matrices)

Consider a matrix $A \in \mathbb{R}^{m \times n}$. Then,

(a) The matrix A has full column rank if and only if the Gram-matrix A^TA is invertible.
(b) The matrix A has full row rank if and only if the Gram-matrix AA^T is invertible.

Proof By Theorem 17.45, $\operatorname{rank}(A^TA) = \operatorname{rank}(A)$ and $\operatorname{rank}(AA^T) = \operatorname{rank}(A)$. By Theorem 17.37, the matrix A^TA is invertible if and only if $\operatorname{rank}(A^TA) = n$, which is equivalent to $\operatorname{rank}(A) = n$, i.e., A has full column rank. Similarly, the matrix AA^T

is invertible if and only if $\text{rank}(AA^T) = m$, which is equivalent to $\text{rank}(A) = m$, i.e., A has full row rank. □

Gram matrices appear frequently in Linear Algebra. One important application is in least squares problems, which we will introduce in Section 19.1 and study in detail in Chapter 32. In the next two sections, we will look at some other important uses of Gram matrices.

17.4.1 Full-rank factorizations revisited

In Propositions 17.13 and 17.15, we saw how to factor a matrix using a basis for either the column space or the row space. In Section 17.3.3, we referred to this as a full-rank factorization.

But what if we have bases for both the column and row spaces? Can we use both to factor A in a single expression? The answer is yes. Instead of writing $A = UV^T$, we factor it as

$$A = UMV^T,$$

where M is an invertible matrix called the **mixing matrix**. Figure 17.7 illustrates this type of factorization.

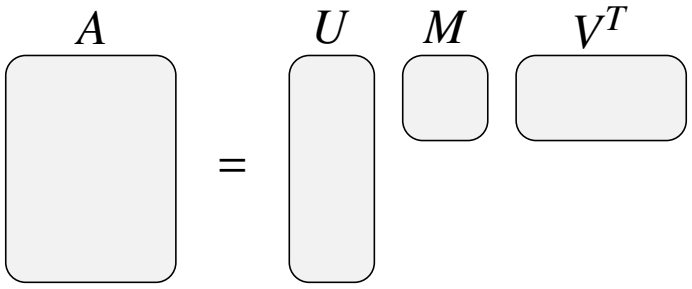


Fig. 17.7: Full-rank factorization with mixing matrix M of a matrix A.

Theorem 17.47. (Full-rank factorization with a mixing matrix)

Consider a matrix $A \in \mathbb{R}^{m\times n}$ with $\text{rank}(A) = k$. Suppose $\{u_1, \ldots, u_k\}$ and $\{v_1, \ldots, v_k\}$ are bases for the column and row spaces of A, respectively. Let $U = \begin{bmatrix} u_1 & \cdots & u_k \end{bmatrix}$ and $V = \begin{bmatrix} v_1 & \cdots & v_k \end{bmatrix}$. Then,

$$A = UMV^T,$$

with $M = (U^TU)^{-1}U^TAV(V^TV)^{-1} \in \mathbb{R}^{k\times k}$ invertible.

Proof By Theorem 17.46, the $k \times k$ matrices U^TU and V^TV are both invertible. We need to show that the matrix

$$M = (U^T U)^{-1} U^T A V (V^T V)^{-1}$$

satisfies $A = UMV^T$ and is invertible. To see this, by Proposition 17.13, notice that we can write $A = UW^T$, for some matrix $W \in \mathbb{R}^{n \times k}$. Then, we have

$$U(U^T U)^{-1} U^T A = U(U^T U)^{-1} U^T U W^T = UW^T = A.$$

Similarly, we can show $AV(V^T V)^{-1} V^T = A$. Thus, we have

$$\begin{aligned} UMV^T = U(U^T U)^{-1} U^T A V (V^T V)^{-1} V^T &= \left(U(U^T U)^{-1} U^T A\right) V (V^T V)^{-1} V^T \\ &= AV(V^T V)^{-1} V^T = A, \end{aligned}$$

as we wanted to show.

Now we show that M is invertible. Assume $\text{rank}(M) = t < k$. Then,

$$\text{rank}(A) \leqslant \min\{\text{rank}(U), \text{rank}(MV^T)\} \leqslant \min\{\text{rank}(U), \text{rank}(M), \text{rank}(V)\} = t < k,$$

a contradiction since $\text{rank}(A) = k$. □

17.4.2 Right and left inverses

Another application of Gram matrices is constructing left and right inverses of matrices, as defined below.

Definition 17.48. (Left and right inverses)

Consider a matrix $A \in \mathbb{R}^{m \times n}$. We say that A has a **left inverse** if there exists an $n \times m$ matrix B such that

$$BA = I_n, \tag{17.6}$$

and we say that B is a left inverse of A. We say that A has a **right inverse** if there exists an $n \times m$ matrix B such that

$$AB = I_m, \tag{17.7}$$

and we say that B is a right inverse of A.

For matrices with full column or full row rank, it is easy to verify the existence of right or left inverses and give an explicit formula.

Theorem 17.49. (Right and left inverse formulas for full-rank matrices)

Consider a matrix $A \in \mathbb{R}^{m \times n}$.

(a) If A has full column rank, then the matrix

$$A_L = (A^T A)^{-1} A^T \tag{17.8}$$

is a left inverse of A.

(b) If A has full row rank, then the matrix

$$A_R = A^T(AA^T)^{-1} \tag{17.9}$$

is a right inverse of A.

Proof If A has full column rank, then A^TA is invertible by Theorem 17.46. Using the associative property of matrix multiplication, we have $A_LA = ((A^TA)^{-1}A^T)A = (A^TA)^{-1})(A^TA) = I_n$, which proves that A_L is a left inverse of A. The proof of the second claim is similar. □

Next, we show that, in fact, matrices with full column rank or full row rank are the only ones that can have a left or right inverse.

Theorem 17.50. (Existence of left and right inverses)

Consider a matrix $A \in \mathbb{R}^{m\times n}$. Then,

(a) A has a left inverse if and only if A has full column rank, and
(b) A has a right inverse if and only if A has full row rank.

Proof We prove that full column rank and full row rank are necessary conditions for A to have a left and right inverse, respectively. The sufficiency of these conditions follows from Theorem 17.49, where we provide explicit formulas for constructing left and right inverses of matrices satisfying these rank conditions.

Assume A has a left inverse, that is, there is a matrix B such that $BA = I_n$. Let us show that $\text{null}(A) = \{0\}$. Let $x \in \text{null}(A)$. Multiplying $BA = I_n$ by x gives $BAx = x$, which implies $0 = x$. Thus, the nullspace of A only contains the zero vector. By the rank-nullity theorem (Theorem 17.35), we have $\text{rank}(A) = n - \dim(\text{null}(A)) = n - 0 = n$. Hence, A has full column rank.

Assume that A has a right inverse, that is, there is a matrix B such that $AB = I_m$. This implies that we can write any vector $b \in \mathbb{R}^m$ as $b = A(Bb) = Ax$. Hence $\text{col}(A) = \mathbb{R}^m$, or equivalently, $\text{rank}(A) = m$, which shows that A has full row rank.□

Unlike the inverse matrix, left and right inverses are not necessarily unique. Theorem 17.49 provides formulas for constructing left and right inverses. While there may be other left and right inverses, the ones in Theorem 17.49 are the most important for this book.

What are left and right inverses useful for?

Linear systems and one-sided inverses

We know that a central problem in Linear Algebra is solving the linear system $Ax = b$. When A has full column rank, the system has either one solution or is inconsistent (i.e., no solution exists). Multiplying both sides of $Ax = b$ by a left inverse A_L gives $A_LAx = x = A_Lb$. So, if the system is consistent, $x = A_Lb$ is the unique solution.

On the other hand, when A has full row rank and $m \neq n$, the system has infinitely many solutions. In this case, if A_R is a right inverse of A, then $x = A_R b$ is a solution, since $Ax = AA_R b = b$.

Problems

17.1 Find the column space and the null space of $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$.

17.2 Give the four fundamental subspaces associated with the following matrix A and their respective dimensions.

$$A = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 1 & 1 \\ 1 & 4 \end{bmatrix}.$$

17.3 Express the column space of the block matrix $C = \begin{bmatrix} A & B \end{bmatrix}$ in terms of the column spaces of A and B.

17.4 Let $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{p\times n}$, and define $C = \left[\begin{smallmatrix} A \\ B \end{smallmatrix}\right]$ Characterize the nullspace of C in terms of the nullspaces of A and B. Prove your characterization.

17.5 Consider two matrices $A, B \in \mathbb{R}^{m\times n}$, and let $C = A + B$.

(a) Show that if $\operatorname{rank}(C) = \operatorname{rank}(A) + \operatorname{rank}(B)$, then $\operatorname{col}(C) = \operatorname{col}(A) \oplus \operatorname{col}(B)$. In particular, prove that $\operatorname{col}(A) \cap \operatorname{col}(B) = \{0\}$.
(b) Is the condition $\operatorname{col}(A) \cap \operatorname{col}(B) = \{0\}$ sufficient to conclude that $\operatorname{col}(C) = \operatorname{col}(A) + \operatorname{col}(B)$? Give a proof or a counterexample.

Hint: Recall that

$$\operatorname{col}(A + B) \subseteq \operatorname{col}(A) + \operatorname{col}(B)$$

and use the dimension formula for the sum of two subspaces.

17.6 A matrix A is said to be symmetric if $A = A^T$. It is obvious that for symmetric matrices, $\operatorname{col}(A) = \operatorname{col}(A^T)$ and $\operatorname{null}(A) = \operatorname{null}(A^T)$. Is the converse true? That is, is it true that if $\operatorname{col}(A) = \operatorname{col}(A^T)$ and $\operatorname{null}(A) = \operatorname{null}(A^T)$, then A is symmetric?

17.7 Suppose two matrices A and B have the same column space.

(a) Show that their row spaces can be different.
(b) What number will be the same for A and B?

17.8 For a square matrix $A \in \mathbb{R}^{n\times n}$, prove the following statements are equivalent.

(a) A is invertible.
(b) There is a matrix $B \in \mathbb{R}^{n\times n}$ such that $AB = I_n$.
(c) There is a matrix $B \in \mathbb{R}^{n\times n}$ such that $BA = I_n$.
(d) $\operatorname{col}(A) = \mathbb{R}^n$ or equivalently $\operatorname{rank}(A) = n$.
(e) $\operatorname{null}(A) = \{0\}$.

(f) The columns and the rows of A are linearly independent.

17.9 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 2 & 4 & 0 & 1 \end{bmatrix}.$$

Find a decomposition $A = UV^T$ such that U contains a basis for the column space of A.

17.10 Consider square matrices where consecutive integers starting from 1 are placed in order:

$$A_2 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad A_3 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad A_4 = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad \dots$$

(a) What is the rank of A_n?
(b) Find a factorization $A_n = U_n V_n^T$ such that U contains a basis for $\mathrm{col}(A_n)$.
(c) Find a factorization $A_n = U_n V_n^T$ such that V contains a basis for $\mathrm{col}(A^T)$.

17.11 Complete the missing steps in the proof of Theorem 17.35.

17.12 Express the matrix

$$A = \begin{bmatrix} 1 & 3 & 8 \\ 1 & 2 & 6 \\ 0 & 1 & 2 \end{bmatrix}$$

as a sum of rank-one matrices.

17.13 Prove that $A^T A = 0$ implies $A = 0$.

17.14 Find all the right inverses of the matrix

$$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}.$$

17.15 Find a full rank factorization with a mixing matrix (as in Theorem 17.47) of the matrix

$$A_4 = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}.$$

Chapter 18
IV-SUPPLEMENTARY TOPICS

18.1 Column and row factorizations of a matrix

In the previous chapter, we introduced the idea of *full-rank* factorization. The key ingredients for building such a factorization of a matrix A are a basis for its column space and/or its row space. In this section, we show how to construct these factorizations using bases formed directly from the actual columns and rows of the matrix.

These factorizations have gained importance in recent years. However, they do not play as central a role in this book as other full-rank factorizations like the QR and SVD. That is why we include them in this supplementary chapter instead of the core chapters. Still, we will highlight some of the advantages of the column and row factorizations compared to the other two.

In this section, we use a few techniques reviewed in Chapter 16, such as row operations, the reduced row echelon form and permutation matrices. If you remember your introductory linear algebra course, these concepts should be familiar.

18.1.1 The CR Factorization

We have already seen how to construct a full-rank factorization $A = UV^T$ for a matrix A. The first step is to find a basis for the column space of A. Placing these basis vectors as columns in a matrix U gives us the first factor. The matrix V^T then contains the coefficients needed to write each column of A as a linear combination of the columns of U.

There are many ways to find a basis for $\text{col}(A)$. Here, we focus on extracting a basis directly from the columns of A. Since the columns of A span $\text{col}(A)$, they contain a basis for it.

We begin with two examples.

Example 18.1 Consider the matrix

$$A = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 11 \\ 1 & 2 & 8 \\ 0 & 1 & 3 \end{bmatrix},$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_18

where a_1, a_2, a_3 denote the columns of A. We want to find a basis for $\text{col}(A)$ using columns from A itself. To do this, we move from left to right: we include a column if it cannot be written as a linear combination of the previous ones. Otherwise, we skip it. We will collect the linearly independent columns of A chosen in this manner into a new matrix C.

In our example, the first column of A is nonzero, so it is linearly independent. We include it as the first column of C.

The second column of A is not a scalar multiple of the first, so the first two columns are linearly independent. We include the second column of A as the second column of C.

The third column of A is a linear combination of the first two: $a_3 = 2a_1 + 3a_2$. So, we do not include it in C.

Following this process, we build the matrix

$$C = \begin{bmatrix} 1 & 3 \\ 1 & 2 \\ 0 & 1 \end{bmatrix},$$

whose columns form a basis for $\text{col}(A)$. This tells us that $\text{rank}(A) = 2$.

The matrix C relates to A through a third matrix R, such that $A = CR$. The columns of R give the coordinates of each column of A expressed in terms of the basis formed by the columns of C. In this case,

$$A = CR = \begin{bmatrix} 1 & 3 \\ 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \end{bmatrix}.$$

This is a full-rank factorization of A whose left factor is built from the columns of A.

Note that in this example, we have $\text{rank}(A) = \text{rank}(C)$, since C is built from a basis of $\text{col}(A)$.

We can turn the approach from this example into a general procedure for extracting a basis for $\text{col}(A)$ using columns of A. The following algorithm summarizes the steps for constructing the matrix C:

1. Start with the first nonzero column a_1 of A and set it as the first column c_1 of C.
2. Move from left to right through the columns of A. At each step, add a column c_j to C if $\{c_1, c_2, \ldots, c_j\}$ is linearly independent.
3. Stop when no more columns can be added without losing linear independence.

This algorithm outputs a matrix C whose columns form a basis for $\text{col}(A)$.

There is no special reason to start building C from the first column of A. We could just as well move from right to left, starting with the last nonzero column, or even select columns in a random order. This might produce a different matrix C, but it would still contain a basis for $\text{col}(A)$, and the number of columns in C would still equal the rank of A.

As the size of A grows, using the above procedure to find a basis for $\mathrm{col}(A)$ by inspecting its columns becomes increasingly impractical. We need a more efficient method. Fortunately, there is one, and you may already be familiar with the technique it uses: the *reduced row echelon form* (rref) of A. Let us see how this works with an example.

Example 18.2 Consider the 4×5 matrix

$$A = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 2 & -1 & 0 \\ 2 & 6 & 1 & 4 & 3 \\ -1 & -3 & -3 & 3 & 1 \\ 3 & 9 & 8 & -7 & 2 \end{bmatrix},$$

where we denote by a_i the ith column of A. Performing row operations, we find the reduced row echelon form of A to be

$$\mathrm{rref}(A) = \begin{bmatrix} 1 & 3 & 0 & 3 & 0 \\ 0 & 0 & 1 & -2 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} =: \begin{bmatrix} R \\ 0 \end{bmatrix},$$

where R contains the nonzero rows of $\mathrm{rref}(A)$.

As you may recall, row operations preserve linear dependence and independence among the columns. The reduced row echelon form clearly shows which columns are linearly independent: the first, third, and fifth columns. These are called pivot columns. It also reveals how the dependent columns can be written as combinations of the pivot ones:

$$\begin{aligned} a_2 &= 3a_1, \\ a_4 &= 3a_1 - 2a_3. \end{aligned}$$

Therefore, the first, third, and fifth columns of A form a basis for $\mathrm{col}(A)$, and $\mathrm{rank}(A) = 3$.

After removing the zero row from $\mathrm{rref}(A)$, we get the following factorization

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 3 \\ -1 & -3 & 1 \\ 3 & 8 & 2 \end{bmatrix} \begin{bmatrix} 1 & 3 & 0 & 3 & 0 \\ 0 & 0 & 1 & -2 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} = CR,$$

where $\mathrm{col}(A) = \mathrm{col}(C)$, and the matrix C consists of columns of A.

The example above shows that the reduced row echelon form does more than just identifying columns of A that form a basis for $\mathrm{col}(A)$. It also gives a full-rank factorization of A. Let us now prove this result.

Theorem 18.3. (A CR factorization from the reduced row echelon form)

Let A be an $m \times n$ matrix of rank r. Let C be the matrix formed by the pivot columns of A, and let R be the matrix obtained by removing all zero rows from $\text{rref}(A)$; that is,

$$\text{rref}(A) = \begin{bmatrix} R \\ 0 \end{bmatrix}.$$

Then $A = CR$ is a CR factorization of A.

Proof Let $\text{rref}(A)$ be the reduced row echelon form of A. Since A has rank r, there are exactly r pivot columns in $\text{rref}(A)$, which correspond to r linearly independent columns in A.

We can use a permutation matrix P to rearrange the columns of A so that the pivot columns come first. After reordering, the reduced row echelon form of A takes the form

$$\text{rref}(A)P = \begin{bmatrix} R \\ 0 \end{bmatrix} P = \begin{bmatrix} I_r & F \\ 0 & 0 \end{bmatrix},$$

where I_r is the $r \times r$ identity matrix and F is an $r \times (n - r)$ matrix that contains the coefficients to obtain the non-pivot columns of A as linear combinations of the pivot columns.

Now let C be the matrix formed by the first r columns of AP. Then, we can write

$$AP = \begin{bmatrix} C & CF \end{bmatrix} = C \begin{bmatrix} I_r & F \end{bmatrix}.$$

Hence,

$$A = C[I_r \ F]P^T = C\text{rref}(A) = C \begin{bmatrix} R \\ 0 \end{bmatrix} = CR.$$

This gives the desired factorization $A = CR$ of A. □

So far, we have shown how to produce a full rank factorization $A = CR$ in which the columns of C are columns of A. But what if we want to have a factorization that instead preserves some rows of A? Next we will see that it is possible to factor A as $A = CR$, where the rows of R are rows of A. To find such a factorization, we can apply the method presented above to the transpose A^T. The following example illustrates this approach.

Example 18.4 Consider the matrix

$$A = \begin{bmatrix} 1 & 4 & 6 \\ 2 & 5 & 7 \\ 3 & 6 & 8 \\ 4 & 7 & 9 \end{bmatrix}.$$

Let us compute the reduced row echelon form of its transpose:

$$\operatorname{rref}(A^T) = \begin{bmatrix} 1 & 0 & -1 & -2 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

By dropping the zero rows from $\operatorname{rref}(A^T)$, we get the factorization

$$A^T = \begin{bmatrix} 1 & 2 \\ 4 & 5 \\ 6 & 7 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 & -2 \\ 0 & 1 & 2 & 3 \end{bmatrix}.$$

Taking the transpose, we get the factorization

$$A = CR = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 2 \\ -2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 4 & 6 \\ 2 & 5 & 7 \end{bmatrix}.$$

Note that the rows of R are rows of the original matrix A.

18.1.2 The *CUR* factorization

In this section, we construct a factorization of a matrix A consisting of three factors CUR with the property that the columns of C form a basis for $\operatorname{col}(A)$ consisting of columns of A, and the columns of R^T form a basis for $\operatorname{col}(A^T)$ consisting of columns of A^T.

One possible way to find such C and R is computing $\operatorname{rref}(A)$ and $\operatorname{rref}(A^T)$, and then identifying the pivot columns of A and A^T. But given these two matrices, is there always a matrix U such that $A = CUR$?

By Theorem 17.47, there exists an invertible matrix U, called the *mixing matrix*[1], with such property. Moreover, Theorem 17.47 gives a (somewhat complicated) formula for the mixing matrix:

$$U = (C^T C)^{-1} C^T A R^T (R R^T)^{-1}.$$

But as we will see, we do not actually need this formula. We can write U directly in terms of the entries of A.

We will start with an example.

Example 18.5 Consider the rank-2 matrix

$$A = \begin{bmatrix} 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 1 \\ 1 & 3 & 6 & 3 \end{bmatrix}.$$

[1] In Theorem 17.47, we used M for the mixing matrix. But when using actual columns and rows of A, it is common to use U instead. We will follow that convention.

Since $\text{rank}(A) = 2$, the matrix has two linearly independent columns and two linearly independent rows that span its column and row spaces.

For example, we can pick the first and third columns and the first and third rows of A. These are highlighted in red below:

$$A = \begin{bmatrix} 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 1 \\ 1 & 3 & 6 & 3 \end{bmatrix}$$

These selected columns and rows form the C and R factors in the factorization $A = CUR$. But how do we find the mixing matrix U?

We build U using the 2×2 submatrix formed by the intersection of the chosen rows and columns, that is, the entries of A that appear both in the selected columns and in the selected rows. In our case, this submatrix is:

$$A[\{1,3\},\{1,3\}] = \begin{bmatrix} 1 & 4 \\ 1 & 6 \end{bmatrix}.$$

Then, the mixing matrix is the inverse of this intersection:

$$U = (A[\{1,3\},\{1,3\}])^{-1} = \begin{bmatrix} 1 & 4 \\ 1 & 6 \end{bmatrix}^{-1} = \begin{bmatrix} 3 & -2 \\ -1/2 & 1/2 \end{bmatrix}.$$

Putting everything together, we get the factorization:

$$A = CUR = \begin{bmatrix} 1 & 4 \\ 0 & 2 \\ 1 & 6 \end{bmatrix} \begin{bmatrix} 3 & -2 \\ -1/2 & 1/2 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 & 2 \\ 1 & 3 & 6 & 3 \end{bmatrix}.$$

There is nothing unique about the specific columns and rows we chose in the previous example. If we had selected a different pair of linearly independent columns and rows (i.e., different bases for the column and row spaces in Example 18.5), the factorization would still work the same way. We would simply get a different CUR factorization of the matrix.

Example 18.6 In Example 18.5, we could instead use the first two columns and the first two rows as bases for the column and row spaces. These are highlighted below:

$$A = \begin{bmatrix} 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 1 \\ 1 & 3 & 6 & 3 \end{bmatrix}.$$

With this new choice of columns and rows, we get the CUR factorization

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 1 \end{bmatrix}.$$

We could also have used columns 1 and 4, and rows 2 and 3, as shown below:

$$A = \begin{bmatrix} 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 1 \\ 1 & 3 & 6 & 3 \end{bmatrix}.$$

With this selection, we get the CUR factorization:

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 3 \end{bmatrix}^{-1} \begin{bmatrix} 0 & 1 & 2 & 1 \\ 1 & 3 & 6 & 3 \end{bmatrix}.$$

Let us now state the general result behind the CUR factorizations we have constructed in the previous examples.

Theorem 18.7. (CUR factorization)

Let $A \in \mathbb{R}^{m \times n}$ be a matrix of rank r. Suppose we select r linearly independent columns of A to form a matrix C, and r linearly independent rows of A to form a matrix R. Let U be the inverse of the $r \times r$ submatrix of A formed by taking the intersection of the selected rows and columns of A. Then, we have the factorization $A = CUR$.

Proof Let W be the submatrix formed by the intersection of the selected columns and rows of A that form C and R, respectively. We begin by permuting the rows and columns of A so that this submatrix W appears in the top-left corner. That is, there exist permutation matrices P_1 and P_2 such that

$$P_1 A P_2 = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} W & A_{12} \\ A_{21} & A_{22} \end{bmatrix}.$$

Since permutation matrices are invertible, they preserve linear independence. So in the permuted matrix $P_1 A P_2$, the first r rows and the first r columns remain linearly independent. The remaining rows and columns are linear combinations of the first r. This means there exist matrices S and T such that

$$\begin{bmatrix} A_{12} \\ A_{22} \end{bmatrix} = \begin{bmatrix} W \\ A_{21} \end{bmatrix} S, \qquad \text{and} \qquad \begin{bmatrix} A_{21} & A_{22} \end{bmatrix} = T \begin{bmatrix} W & A_{12} \end{bmatrix}.$$

Hence, we can write

$$P_1 A P_2 = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} W & WS \\ TW & TWS \end{bmatrix}.$$

The expression above implies that the matrix W must be invertible. If W were singular, there would exist a nonzero vector x such that $Wx = 0$. Then,

$$\begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix} x = \begin{bmatrix} Wx \\ TWx \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

which contradicts the assumption that the first r columns are linearly independent. We may therefore define $U := W^{-1}$. We can thus factor $P_1 A P_2$ as

$$P_1 A P_2 = \begin{bmatrix} W & WS \\ TW & TWS \end{bmatrix} = \begin{bmatrix} W \\ TW \end{bmatrix} W^{-1} \begin{bmatrix} W & WS \end{bmatrix} = \begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix} U \begin{bmatrix} A_{11} & A_{12} \end{bmatrix}.$$

Permutation matrices are invertible, and their inverses are equal to their transposes. Using this, we can write

$$A = \left(P_1^T \begin{bmatrix} A_{11} \\ A_{21} \end{bmatrix} \right) U \left(\begin{bmatrix} A_{11} & A_{12} \end{bmatrix} P_2^T \right). = CUR,$$

where we have used that multiplying by P_1^T and P_2^T undoes the row and column permutations we applied at the start. □

The CUR factorization is one of many possible full-rank factorizations of a matrix. We will encounter others in later chapters, but it is worth pausing to highlight a few advantages of the CUR factorization compared to the others.

The first main advantage is *interpretability*. If A is a data matrix, its rows represent data points, and its columns represent attributes or features. In a CUR factorization, the matrix C is formed from actual columns of A, so its columns correspond to features in the dataset. Similarly, the rows of R come from actual rows of A, so they are data points. This means we are expressing all data points using a small number of representative ones (the rows of R), and all features in terms of a few selected features (the columns of C). This makes the factorization easier to interpret compared to others that use abstract combinations of the original rows and columns.

The second advantage is that the factors C and R inherit structural properties from A. For instance, if A has only nonnegative entries, then so do C and R, since they are made up of actual columns and rows of A. Likewise, if A is sparse, that is, it contains many zeros, then C and R are likely to be sparse as well. The mixing matrix U may not share these properties, but it is typically small (of size equal to the rank of A), so this is usually not a concern.

Below are some examples in data science where the CUR factorization is useful.

Example 18.8. (Genomics and medical research)

In genetics, researchers often work with vast datasets where rows represent different patients and columns capture the activity levels of thousands of genes. Analyzing this data to uncover links between genes and diseases can be very complex. CUR factorization can simplify the task by identifying a small subset of "marker" genes (columns) and a key group of patients (rows) that best represent the variations in the entire dataset.

For example, a research team studying a specific type of cancer can apply CUR factorization to gene expression data. This helps them pinpoint a small set of genes whose activity is most strongly linked to the disease. They can then focus their research and lab work on these genes rather than the entire genome, saving time

and resources. The results are also easier to interpret, since they connect directly to actual genes and patient groups.

Example 18.9. (Text analysis and document summarization)

Consider a large collection of documents, such as news articles or customer reviews. This can be written as a matrix with rows as documents and columns as words, where entries show how often each word appears.

CUR factorization can simplify this term-document matrix by selecting a small set of key documents (rows) and important words (columns). The chosen documents capture the main topics, and the chosen words highlight the terms most useful for understanding those topics.

For example, a news aggregator may want to sort thousands of articles into topics like sports, politics, and technology. With CUR factorization, the system can pick the most representative articles and the most discriminative words for each category. This makes automatic categorization faster and more accurate. It can also generate summaries by focusing on the selected key documents and important words.

Example 18.10. (Image compression)

A digital image can be seen as a matrix of pixels, where each entry gives the color and intensity of a pixel.

CUR factorization can break down this image matrix by selecting a subset of the most informative pixel rows and columns. With these and a small weighting matrix, the original image can be reconstructed with high accuracy.

For example, when you upload a photo to a social media platform, it is often compressed to save space and improve loading speed. A CUR-like approach can shrink the file size by keeping only the most important pixel information. This preserves the main features of the image while removing redundant data, producing a smaller file that still looks good to the human eye.

In data science applications, instead of finding an exact CUR factorization of A, it is common to find a matrix CUR that approximates A. The columns of C are still linearly independent columns of A, the rows of R are still linearly independent rows of A but the rank of C and R is much smaller than the rank of A. Thus, we produce a low-rank approximation of A instead of an exact reconstruction. We want to capture the most important patterns in the data using only a few columns and rows, rather than all of them.

The goal of a CUR algorithm is to choose the columns and rows for the matrices C and U in the approximation, and construct the U matrix, in a way that minimizes the difference $A - CUR$. This difference is often measured using a matrix norm, such as the Frobenius norm ($\|A - CUR\|_F$). A smaller norm indicates a better

approximation. So, while A and CUR will not be identical, they are designed to be "close" in some measurable sense.

18.2 Equalities and inequalities involving matrix ranks

In Chapter 17, we discussed upper and lower bounds for the rank of a sum or product of two matrices based on their ranks. In this section, we present more equalities and inequalities involving matrix ranks.

18.2.1 Rank of 2×2 block matrices

This section presents some useful equalities and inequalities that relate the rank of a 2×2 block matrix to the ranks of its individual blocks.

We start with the simplest case: a block diagonal matrix. The next theorem gives the rank of a 2×2 block diagonal matrix in terms of the ranks of its diagonal blocks. The proof is straightforward, so we omit it.

Theorem 18.11. (Rank of a 2×2 block diagonal matrix)

Let A be the block diagonal matrix

$$A = \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix},$$

where A_{11} and A_{22} are not necessarily square. Then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}).$$

Theorem 18.11 can be extended to block diagonal matrices with more than two blocks using induction (see Problem 18.1 for details).

Example 18.12 Let

$$A = \left[\begin{array}{c|cccc} 2 & 0 & 0 & 0 & 0 \\ \hline 0 & 1 & -2 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{array}\right] =: \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix}.$$

By Theorem 18.11, we have

$$\operatorname{rank}(A) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}) = 1 + \operatorname{rank}(A_{22}).$$

Now, notice that A_{11} is also a block diagonal matrix

$$A_{22} = \left[\begin{array}{cc|cc} 1 & -2 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ \hline 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{array}\right] =: \begin{bmatrix} B_{11} & 0 \\ 0 & B_{22} \end{bmatrix}.$$

Applying the theorem again, we find

$$\operatorname{rank}(A_{22}) = \operatorname{rank}(B_{11}) + \operatorname{rank}(B_{22}) = 2 + 1.$$

Substituting this back, we get

$$\operatorname{rank}(A) = 1 + \operatorname{rank}(A_{22}) = 1 + 2 + 1 = 4.$$

Next, we study the rank of 2×2 block matrices that contains a zero block. We begin with the important case of block upper triangular matrices.

Theorem 18.13. (Rank of a 2×2 block triangular matrix)

Consider a 2×2 block matrix of the form

$$A = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix} \qquad \text{or} \qquad A = \begin{bmatrix} A_{11} & 0 \\ A_{12} & A_{22} \end{bmatrix}.$$

Then,

$$\operatorname{rank}(A) \geqslant \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}).$$

Moreover, if either A_{11} or A_{22} is nonsingular, then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}).$$

Proof We prove the result for a block upper triangular matrix. The proof for the block lower triangular case is similar and is left as an exercise.

Let $s = \operatorname{rank}(A_{11})$ and $t = \operatorname{rank}(A_{22})$. To prove the inequality $\operatorname{rank}(A) \geqslant \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22})$, we start by rearranging the columns of A_{11} and A_{22} using permutation matrices to place the independent columns first.

Let P be a permutation matrix such that

$$A_{11}P = \begin{bmatrix} B_1 & B_2 \end{bmatrix},$$

where B_1 consists of s linearly independent columns of A_{11}. Similarly, let Q be a permutation matrix such that

$$\begin{bmatrix} A_{12} \\ A_{22} \end{bmatrix} Q = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix},$$

where M_{21} contains t linearly independent columns of A_{22}. Using these permutations, we can write

$$A \begin{bmatrix} P & 0 \\ 0 & Q \end{bmatrix} = \begin{bmatrix} B_1 & B_2 & M_{11} & M_{12} \\ 0 & 0 & M_{21} & M_{22} \end{bmatrix}.$$

Since multiplying A by an invertible matrix does not change its rank, we have

$$\operatorname{rank}(A) = \operatorname{rank}\left(\begin{bmatrix} B_1 & B_2 & M_{11} & M_{12} \\ 0 & 0 & M_{21} & M_{22} \end{bmatrix} \right).$$

To finish, consider the submatrix

$$\widetilde{A} = \begin{bmatrix} B_1 & M_{11} \\ 0 & M_{21} \end{bmatrix}.$$

Removing columns does not increase rank, so $\operatorname{rank}(A) \geqslant \operatorname{rank}(\widetilde{A})$. Moreover, $\widetilde{A}$ has linearly independent columns, so its rank is $\operatorname{rank}(\widetilde{A}) = s + t$. Thus, we have

$$\operatorname{rank}(A) \geqslant \operatorname{rank}(\widetilde{A}) = s + t = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}).$$

Now, if A_{11} is nonsingular, we can factor A as

$$A = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix} \begin{bmatrix} I & A_{11}^{-1} A_{12} \\ 0 & I \end{bmatrix},$$

as we wanted to show.

The first factor is a block diagonal matrix, and the second factor is an invertible block upper triangular matrix. Since multiplying by an invertible matrix does not change the rank, we have

$$\operatorname{rank}(A) = \operatorname{rank}\left(\begin{bmatrix} A_{11} & 0 \\ 0 & A_{22} \end{bmatrix} \right).$$

By Theorem 18.11, the rank of a block diagonal matrix is the sum of the ranks of its diagonal blocks:

$$\operatorname{rank}(A) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}).$$

The case where A_{22} is nonsingular is similar, so we leave the proof as an exercise.□

Example 18.14 Consider the block upper triangular matrix

$$A = \left[\begin{array}{cc|ccc} 1 & 1 & 2 & 0 & 1 \\ 2 & 1 & 3 & -2 & 2 \\ \hline 0 & 0 & 1 & 4 & -1 \\ 0 & 0 & -1 & -4 & 1 \end{array}\right].$$

Since the block $A[1:2, 1:2]$ is nonsingular, we have

$$\operatorname{rank}(A) = \operatorname{rank}\left(\begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}\right) + \operatorname{rank}\left(\begin{bmatrix} 1 & 4 & -1 \\ -1 & -4 & 1 \end{bmatrix}\right) = 2 + 1 = 3.$$

Example 18.15 Consider the 2×2 block upper triangular matrix

$$M = \left[\begin{array}{cc|ccc} 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 3 & -2 & 2 \\ \hline 0 & 0 & 1 & 4 & -1 \\ 0 & 0 & -1 & -4 & 1 \end{array}\right] =: \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{21} \end{bmatrix}.$$

In this case, we have

$$3 = \operatorname{rank}(A) > \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}) = 1 + 1 = 2.$$

Next, we consider the rank of 2×2 block matrices with a zero block that are not upper or lower triangular.

Theorem 18.16. (Rank of a 2×2 block anti-triangular matrix)

Consider a 2×2 block matrix of the form

$$A = \begin{bmatrix} 0 & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \qquad \text{or} \qquad A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & 0 \end{bmatrix}.$$

(a) We have

$$\operatorname{rank}(A) \geqslant \operatorname{rank}(A_{12}) + \operatorname{rank}(A_{21}).$$

Moreover, if either A_{12} or A_{21} is nonsingular, then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{12}) + \operatorname{rank}(A_{21}).$$

(b) In the first case, a block lower anti-triangular matrix, if A_{22} is nonsingular, then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{22}) + \operatorname{rank}(A_{12}A_{22}^{-1}A_{21}).$$

(c) In the second case, a block upper anti-triangular matrix, if A_{11} is nonsingular, then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{21}A_{11}^{-1}A_{12}).$$

Proof We prove the result for the block unti lower triangular matrix (first case). The proof for the block unti upper triangular matrix (second case) is similar and is left as an exercise.

Observe that

$$\begin{bmatrix} 0 & I \\ I & 0 \end{bmatrix} \begin{bmatrix} 0 & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{21} & A_{22} \\ 0 & A_{12} \end{bmatrix}.$$

Since multiplying by an invertible matrix does not change the rank, and $\left[\begin{smallmatrix} 0 & I \\ I & 0 \end{smallmatrix}\right]$ is invertible, part (a) follows directly from Theorem 18.13.

To prove part (b), observe that we can write

$$\begin{bmatrix} I & -A_{12}A_{22}^{-1} \\ 0 & I \end{bmatrix} \begin{bmatrix} 0 & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} -A_{12}A_{22}^{-1}A_{21} & 0 \\ A_{21} & A_{22} \end{bmatrix}.$$

The first matrix is invertible, so multiplying by it does not change the rank. Since A_{22} is invertible, the result follows directly from the block lower triangular version of Theorem 18.13. □

We conclude this section with a result for general 2×2 block matrices.

Theorem 18.17. (The rank of a 2×2 block matrix)

Consider a 2×2 block matrix

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

where either A_{11} or A_{22} is nonsingular.

(a) If A_{11} is nonsingular, then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22} - A_{21}A_{11}^{-1}A_{12}).$$

(b) If A_{22} is nonsingular, then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{22}) + \operatorname{rank}(A_{11} - A_{12}A_{22}^{-1}A_{21}).$$

Proof We prove the theorem for the case where A_{11} is nonsingular. The case where A_{22} is nonsingular is left as an exercise (see Problem 18.9).

We can write

$$\begin{bmatrix} I & 0 \\ -A_{21}A_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}.$$

The matrix $\left[\begin{smallmatrix} I & 0 \\ -A_{21}A_{11}^{-1} & I \end{smallmatrix}\right]$ is nonsingular, so multiplying A by it does not change its rank. Therefore, by Theorem 18.13,

$$\operatorname{rank}(A) = \operatorname{rank}\left(\begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}\right) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22} - A_{21}A_{11}^{-1}A_{12}),$$

as we wanted to show. □

Example 18.18 Consider the 2×2 block matrix

$$A = \left[\begin{array}{cc|cc} 1 & 0 & 2 & 0 \\ 0 & 1 & 3 & -2 \\ \hline 3 & 2 & 1 & 4 \\ 1 & -1 & -1 & 2 \end{array}\right] =: \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}.$$

Note that $A_{11} = I_2$ and, therefore, is nonsingular. Hence,

$$\begin{aligned} \operatorname{rank}(A) &= \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22} - A_{21}A_{11}^{-1}A_{12}) \\ &= \operatorname{rank}\left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right) + \operatorname{rank}\left(\begin{bmatrix} 1 & 4 \\ -1 & 2 \end{bmatrix} - \begin{bmatrix} 3 & 2 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 2 & 0 \\ 3 & -2 \end{bmatrix}\right) \\ &= 2 + \operatorname{rank}\left(\begin{bmatrix} -11 & 8 \\ 0 & 0 \end{bmatrix}\right) = 3. \end{aligned}$$

18.2.2 Frobenius' and Sylvester's inequalities

We now turn to a classic rank inequality that relates the rank of a matrix product ABC to the ranks of the individual matrices. This result, known as Frobenius' inequality, is a useful tool that leads to several other interesting and important results.

Theorem 18.19. (Frobenius' inequality)

For $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times p}$, and $C \in \mathbb{R}^{p\times q}$, we have

$$\operatorname{rank}(ABC) \geqslant \operatorname{rank}(AB) + \operatorname{rank}(BC) - \operatorname{rank}(B).$$

Proof We prove the inequality using properties of block matrices. Define the following 2×2 block matrices

$$S = \begin{bmatrix} I_m & -A \\ 0 & I_n \end{bmatrix}, \quad M = \begin{bmatrix} 0 & AB \\ BC & B \end{bmatrix}, \quad \text{and} \quad T = \begin{bmatrix} I_q & 0 \\ -C & I_p \end{bmatrix}.$$

Then,

$$SMT = \begin{bmatrix} I_m & -A \\ 0 & I_n \end{bmatrix}\begin{bmatrix} 0 & AB \\ BC & B \end{bmatrix}\begin{bmatrix} I_q & 0 \\ -C & I_p \end{bmatrix} = \begin{bmatrix} -ABC & 0 \\ 0 & B \end{bmatrix}.$$

The matrix S is a unit upper triangular matrix and T is a unit lower triangular matrix, so both are nonsingular. Since multiplying by nonsingular matrices preserves the rank, we have

$$\operatorname{rank}(M) = \operatorname{rank}(SMT).$$

The matrix SMT is block diagonal, so

$$\operatorname{rank}(SMT) = \operatorname{rank}(ABC) + \operatorname{rank}(B).$$

By Theorem 18.16, M satisfies

$$\operatorname{rank}(M) \geqslant \operatorname{rank}(BC) + \operatorname{rank}(AB).$$

Combining these results, we get

$$\operatorname{rank}(ABC) + \operatorname{rank}(B) \geqslant \operatorname{rank}(AB) + \operatorname{rank}(BC).$$

This completes the proof. □

Using Frobenius' inequality, we can easily prove Sylvester's inequality. Although we already proved it in Chapter 17 using dimension arguments, this version follows directly as a simple consequence of Frobenius' inequality.

Corollary 18.20. (Sylvester's inequality)

For $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$, we have

$$\operatorname{rank}(AB) \geqslant \operatorname{rank}(A) + \operatorname{rank}(B) - n.$$

Proof Apply Theorem 18.19 with $B = I$ and $C = B$. □

Problems

18.1 Let D be the block diagonal matrix

$$D = \begin{bmatrix} D_1 & & & \\ & D_2 & & \\ & & \ddots & \\ & & & D_k \end{bmatrix},$$

where the main diagonal blocks D_i are not necessarily square matrices. Show that

$$\operatorname{rank}(D) = \operatorname{rank}(D_1) + \operatorname{rank}(D_2) + \cdots + \operatorname{rank}(D_k).$$

18.2 Find a block matrix

$$A = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix}$$

such that $\operatorname{rank}(A) > \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22})$.

18.3 Consider the block matrix

$$A = \begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}.$$

Show that

$$\operatorname{rank}(A) \geqslant \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22}).$$

Furthermore, prove that if either A_{11} or A_{22} is nonsingular, then equality holds.

18.4 Show that

$$\operatorname{rank}\left(\begin{bmatrix} 0 & A_{12} \\ A_{21} & 0 \end{bmatrix}\right) = \operatorname{rank}(A_{12}) + \operatorname{rank}(A_{21}).$$

18.5 Consider the block matrix

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & 0 \end{bmatrix}.$$

Show that

$$\operatorname{rank}(A) \geqslant \operatorname{rank}(A_{12}) + \operatorname{rank}(A_{21}).$$

Furthermore, prove that if either A_{12} or A_{21} is nonsingular, then equality holds. Finally, show that if A_{11} is nonsingular, then

$$\operatorname{rank}(A) = \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{21}A_{11}^{-1}A_{12}).$$

18.6 Let

$$A = \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix}$$

be a 2×2 block upper triangular matrix. Assume that A_{12} is nonsingular. Prove that

$$\operatorname{rank}(A) = \operatorname{rank}(A_{12}) + \operatorname{rank}(A_{22}A_{12}^{-1}A_{11}).$$

18.7 State and prove a result similar to Theorem 18.13 for $n \times n$ block triangular matrices.

18.8 Let

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}.$$

Is it always true that

$$\operatorname{rank}(A) \geqslant \operatorname{rank}(A_{11}) + \operatorname{rank}(A_{22})?$$

Either prove the inequality or provide a counterexample.

18.9 Prove Theorem 18.17 when A_{22} is nonsingular.

18.10 Using what you have learned about the rank of block matrices, prove that for any matrices A and B (of the same size), the inequality

$$\operatorname{rank}(A + B) \leqslant \operatorname{rank}(A) + \operatorname{rank}(B)$$

holds.

18.11 Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. Use Sylvester's inequality to show that if $AB = 0$, then

$$\operatorname{rank}(A) + \operatorname{rank}(B) \leqslant n.$$

18.12 (CUR approximation algorithms) Read the following two papers:

1. *CUR Matrix Decompositions for Improved Data Analysis* by Mahoney and Drineas (PNAS, 2009).
2. *A DEIM Induced CUR Factorization* by Sorensen and Embree (SIAM J. Sci. Comput., 2016).

(a) Summarize the main ideas of CUR factorizations from the Mahoney & Drineas paper. Focus on why CUR is useful and how it differs from other low-rank approximations.
(b) Summarize the DEIM-based approach described by Sorensen & Embree. Explain how DEIM is used to choose rows and columns.
(c) Compare the two approaches (randomized vs. DEIM) in terms of algorithmic ideas and potential advantages.
(d) Implement a small example of CUR factorization (randomized selection or DEIM-based). Apply it to a dataset of your choice (e.g., a text collection or a grayscale image).
(e) Report on the reconstruction error and interpretability of your results.

Chapter 19
IV-FROM THE CLASSROOM TO REAL LIFE

19.1 The regression line

Suppose we have a set of m data points in $\mathbb{R}^2$, $(t_1, y_1), (t_2, y_2), \ldots, (t_m, y_m)$, and we want to find the line

$$y = f(t) = c_0 + c_1 t$$

that best represents this data, where c_0 is the y-intercept and c_1 is the slope of the line. A single line usually cannot pass through all the data points unless they are collinear. For example, in the left panel of Figure 19.1, no line passes through all the data points. Instead, we seek to find a line that "best fits" the data.

But what do we mean by the "best fit line"? One way to measure the goodness of fit is by using the "sum of squared errors", which is the square of the Euclidean distance between the vector $y = \begin{bmatrix} y_1 & y_2 & \cdots & y_m \end{bmatrix}^T$ and the vector $f(t) = \begin{bmatrix} f(t_1) & f(t_2) & \cdots & f(t_m) \end{bmatrix}^T$,

$$\|y - f(t)\|_2^2 = \sum_{i=1}^{m} (y_i - f(t_i))^2.$$

Each term in the sum represents the square of the error when the data point (t_i, y_i) is approximated by $(t_i, f(t_i))$, that is,

$$e_i^2 = (y_i - f(t_i))^2 = (y_i - c_0 - c_1 t_i)^2.$$

Geometrically, e_i^2 is the square of the vertical distance from the point (t_i, y_i) to the line $y = c_0 + c_1 t$, as shown in the right panel of Figure 19.1.

Our goal is to find the coefficients c_0 and c_1 that minimize the sum of squared errors

$$\sum_{i=1}^{m} e_i^2 = \sum_{i=1}^{m} (y_i - c_0 - c_1 t_i)^2, \tag{19.1}$$

or equivalently, minimize $\|y - f(t)\|_2$.

We could have also defined the goodness of fit as the sum of absolute values

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_19

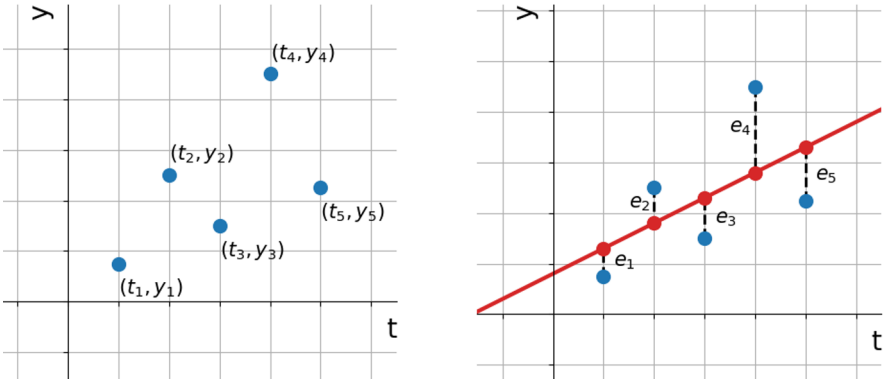


Fig. 19.1: Left panel: Data points. No single line passes through all points. Right panel: Errors between data points and the fitted line.

$$\sum_{i=1}^{m} |e_i| = \sum_{i=1}^{m} |y_i - c_0 - c_1 t_i|, \tag{19.2}$$

in which case we would minimize $\|y - f(t)\|_1$; or, alternatively, define the goodness of fit as the maximum of absolute values

$$\max_{i=1,\ldots,m} |e_i| = \max_{i=1,\ldots,m} |y_i - c_0 - c_1 t_i|, \tag{19.3}$$

which corresponds to minimizing $\|y - f(t)\|_\infty$. So, why do we use the 2-norm? We use the 2-norm because minimizing the sum of squared errors has a closed-form solution.[1] Other p-norms need more complex, iterative methods, which we will not cover here. In Chapter 32, we will see how this closed-form solution lets us use Linear Algebra to efficiently find the solution.

We can express the sum of squared errors (19.1) using vectors and matrices. Define the following

$$e = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_m \end{bmatrix}, \quad b = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \quad A = \begin{bmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix}, \quad \text{and} \quad x = \begin{bmatrix} c_0 \\ c_1 \end{bmatrix}.$$

We can now write the sum of squared errors (19.1) as $\|e\|_2^2 = \|b - Ax\|_2^2$.

The fact that no line passes through all the data points means that the system $Ax = b$ has no solution. Since we cannot make Ax exactly equal to b, we instead look for the vector x that makes $\|b - Ax\|_2^2$ as small as possible. This is called a **least squares problem**, which we will study in detail in Chapter 32. The line $y = c_0 + c_1 t$ is known as the **regression line**.

[1] Another reason, which you will recognize if you have taken statistics, is that minimizing the sum of squared errors matches maximum likelihood estimation with normally distributed errors.

Here, we note that its solution involves the Gram matrix $A^T A$, and is given by

$$x = (A^T A)^{-1} A^T b. \tag{19.4}$$

In Problem 19.1, you will show that $A^T A$ is invertible for this particular problem. Chapter 32 will explain why this is the solution. For now, note that we find the solution x by multiplying the left inverse $(A^T A)^{-1} A^T$ of A by b.

For the data points in Figure 19.1, the figure also shows the resulting regression line.

Once we have computed the coefficients c_0 and c_1 for the regression line, we can use them to predict the output y for a new input value t by substituting t into the equation $f(t) = c_0 + c_1 t$. Figure 19.2 illustrates this process.

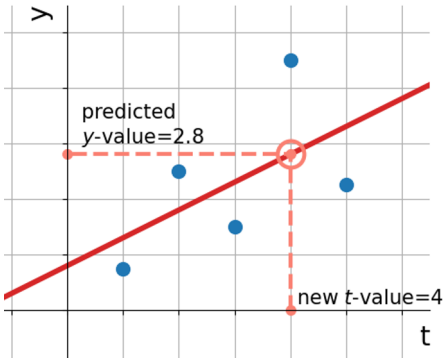


Fig. 19.2: Predictions using the regression line.

One of the main uses of the regression line is to model the relationship between two variables. In the next example, we will use it to predict weekly COVID-19 deaths based on Google search trends.

Example 19.1. (Forecasting weekly COVID-19 deaths)

COVID-19, caused by a coronavirus, first appeared in Wuhan, China, in December 2019. It quickly spread worldwide, causing a devastating pandemic. Figure 19.3 shows the weekly count of new reported COVID-19 deaths in the U.S. from October 2020 to October 2021.

One challenge in managing a pandemic is predicting the number of deaths in advance. Accurate forecasting is difficult, but we can use a simple method with readily available data: the number of Google searches for "COVID symptoms" [2].

Figure 19.4 compares the weekly count of COVID-19 deaths with the number of Google searches for "COVID symptoms". Google Trends normalizes these counts by dividing each query's weekly count by the total weekly search queries in that location, adjusting the values to range from 0 to 100.

[2] We obtained the search data from Google Trends [2].

Fig. 19.3: Weekly number of new reported COVID-19 deaths in the US from October 2020 to October 2021.

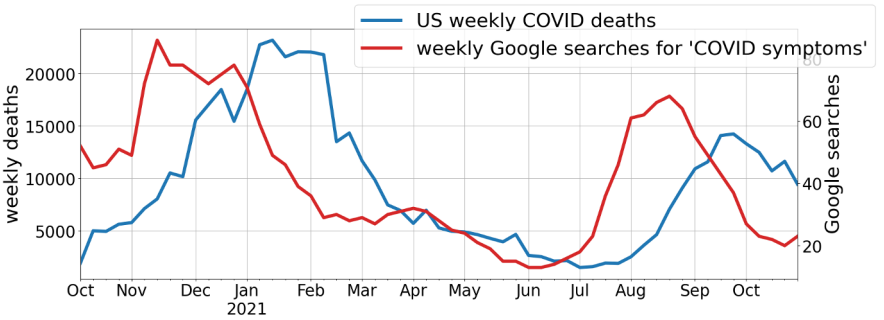


Fig. 19.4: Weekly number of new reported COVID-19 deaths, and Google searches of the term "COVID symptoms" in the US from October 2020 to October 2021.

Figure 19.4 shows a clear similarity between COVID-19 deaths and Google searches for COVID symptoms, with deaths usually lagging a few weeks behind searches. To confirm this, we shifted the search counts forward by six weeks and plotted the new curves in Figure 19.5. As expected, the two curves now overlap closely.

To better show the link between COVID-19 deaths and Google searches, Figure 19.6 plots weekly deaths against the search counts, shifted by six weeks.

The plot in Figure 19.6 shows an approximate linear relationship between the death and search variables. This suggests we can express the relationship as

$$\text{weekly deaths} = c_0 + c_1 \cdot (\text{search counts 6 weeks ago}), \tag{19.5}$$

where c_0 and c_1 are numbers we need to determine.

Equation (19.5) is a **linear model** because it shows "weekly deaths" as a linear function of "search counts 6 weeks ago". The model's coefficients are c_0 and c_1. To fit the observed data, we determine these coefficients by finding the regression line.

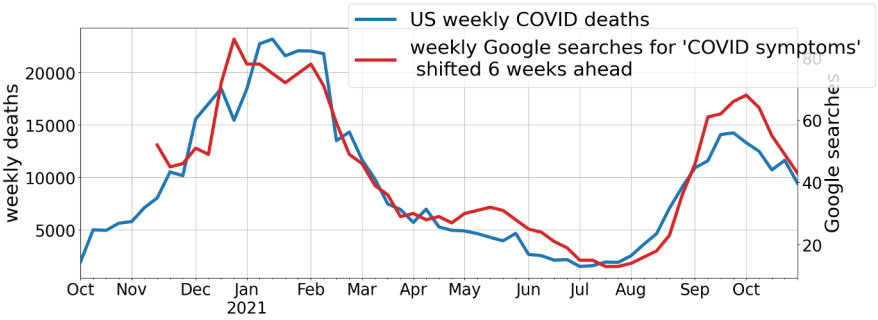


Fig. 19.5: Weekly number of new reported COVID-19 deaths and corresponding Google searches for the term "COVID symptoms" (with a 6-week shift).

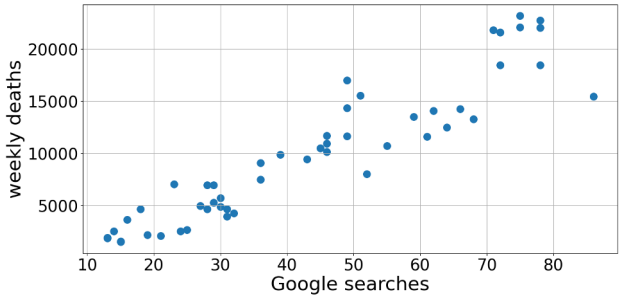


Fig. 19.6: Weekly number of new reported COVID-19 deaths against Google searches of the term "COVID symptoms" (shifted 6 weeks ahead).

Once we have c_0 and c_1, we can use the linear model to predict weekly COVID-19 deaths six weeks ahead. Figure 19.7 compares the model's predictions from Google Trends data with the actual numbers. The predictions closely match the real data.

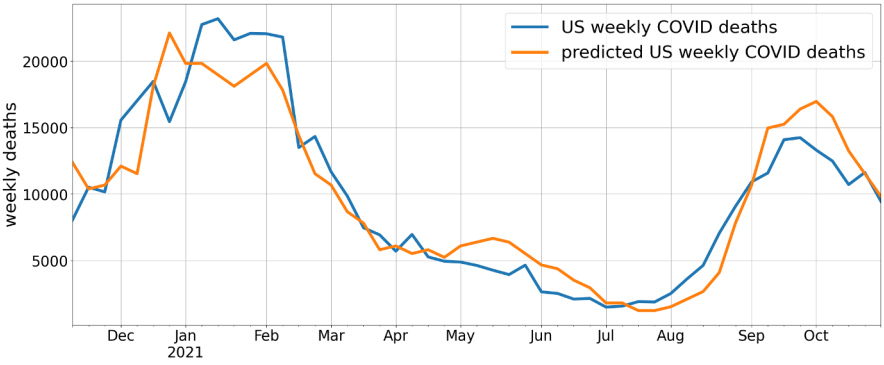


Fig. 19.7: Weekly counts of reported COVID-19 deaths. Comparison of actual and predicted numbers.

As a fun note, one of the authors used this model throughout much of the pandemic. At the start of each week, they downloaded the latest Google Trends data and used the model to forecast COVID-19 deaths several weeks ahead. For a while, the predictions were remarkably accurate. Eventually, though, the model's performance began to decline over time.

19.2 Applications of full rank factorizations

Many matrices in Data Science are large but have low rank [3]. This low rank results from correlations and redundancies across rows and columns.

We now present two examples that show why data matrices often have low rank.

Example 19.2. (Redundancies in rating matrices)

As we discussed in Section 3.6, rating matrices have rows representing users and columns representing items (like movies). The entries are the ratings given by users to these items. These matrices often contain redundant data.

For example, in movie ratings, redundancy comes from genres. Movies in the same genre tend to receive similar ratings because users who enjoy a particular genre, like action, usually rate those movies in a similar way. This creates similar patterns along the columns. Similarly, users with similar tastes give similar ratings, which introduces redundancy across the rows.

Example 19.3. (Redundancies in document-term matrices)

In Section 3.5, we introduced document-term matrices whose rows represent text documents, whose columns represent words, and whose entries contain word counts or TF-IDF values. Redundancies typically arise from document topics. Documents covering similar topics, like politics, often share a common vocabulary, so their corresponding rows exhibit similar patterns, producing redundancy across the rows. Likewise, words that have similar meanings or that tend to occur in the same types of documents have related column patterns, producing redundancy across the columns. For example, terms such as "car" and "auto" may have similar distributions across a collection of documents.

Later, in Section 19.2.3, we will show a toy example of a document-term matrix to illustrate how a full-rank factorization can uncover hidden "topics" in the data.

Imagine an $m \times n$ matrix A with rank $r \ll \min\{m, n\}$. As we saw in Chapter 17, we can write $A = UV^T$, where $U \in \mathbb{R}^{m \times r}$, $V \in \mathbb{R}^{n \times r}$, and both have full column rank. This gives us a way to represent A using smaller, full-rank matrices.

[3] In Chapter 49, we will see that it is more precise to say that matrices from applications have low numerical rank.

Let us see a few advantages of having a full-rank factorization of a low-rank matrix.

19.2.1 Storing a large but low-rank matrix efficiently

For a large matrix $A \in \mathbb{R}^{m\times n}$, an immediate consequence of having low rank r is that storing the factors $U \in \mathbb{R}^{m\times r}$ and $V \in \mathbb{R}^{n\times r}$ in a full rank factorization of A requires less memory than storing the matrix itself.

Storing the matrix A requires storing mn entries, while storing the factors U and V requires storing $r(m+n)$ entries. Hence, if the rank r of A is much smaller than m and n, storing U and V instead of A requires much less storage space.

As an example, suppose A is a $1,000 \times 1,000$ matrix of rank 10. Storing the factors requires storing $2 \cdot 10^4$ entries in total, two orders of magnitude less than storing the 10^6 entries of A.

19.2.2 Efficient matrix-vector multiplication with low-rank matrices

Low-rank matrices are not only efficient to store, but they also allow for inexpensive matrix-vector multiplications in terms of arithmetic operations.

Multiplying an $m \times n$ matrix A by an n-component vector x requires $m(2n-1)$ arithmetic operations. This is because, for each of the m rows in the matrix, we need to compute a dot product, which requires n multiplications and $n-1$ additions.

On the other hand, with a full rank factorization $A = UV^T$, where $U \in \mathbb{R}^{m\times r}$ and $V \in \mathbb{R}^{n\times r}$, we can compute Ax in two steps. First, compute the vector V^Tx, which takes $r(2n-1)$ operations. Then, compute $U(V^Tx)$, which takes $m(2r-1)$ operations. In total, this requires $r(2n+2m-1)-m$ operations. When $r \ll \min\{m,n\}$, this is much smaller than $m(2n-1)$.

As an illustration, consider again a $1,000 \times 1,000$ matrix A with rank 10. Multiplying Ax directly requires about $2 \cdot 10^6$ arithmetic operations. Using a full rank factorization, it requires only $3.9 \cdot 10^4$ operations, which is two orders of magnitude less.

19.2.3 Topic modeling

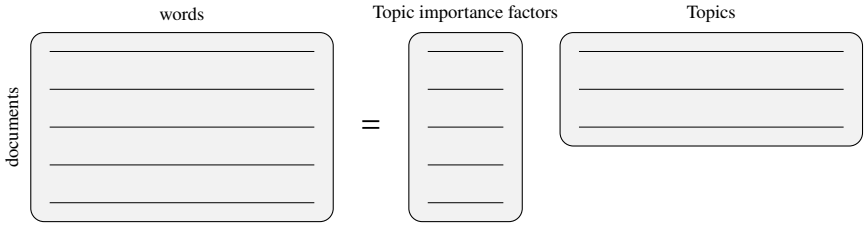
Another important application of full rank matrix factorizations is topic modeling.

In topic modeling, we start with a term-document matrix. As we explained in Chapter 3, each row represents a document, each column represents a word, and the (i,j)th value indicates the presence, count, or frequency of word j in document i. The goal of topic modeling is to extract the main topics from a collection of documents.

For example, consider the following toy term-document matrix associated with a collection of seven documents. The matrix values indicate whether certain words (linear, matrix, convex, etc.) appear in the documents (1 for presence, 0 for absence).

	linear	**matrix**	**convex**	**minimum**	**derivative**	**constrains**	**factorization**	**dimension**
doc 1	1	1	0	0	0	0	1	1
doc 2	1	1	1	1	1	1	1	1
doc 3	0	0	1	1	1	1	0	0
doc 4	1	1	1	1	1	1	1	1
doc 5	0	0	1	1	1	1	0	0
doc 6	1	1	0	0	0	0	1	1
doc 7	1	1	0	0	0	0	1	1

Notice that when we factor the term-document matrix A into $A = UV^T$, we express the rows of A (the documents) as linear combinations of the rows of V^T. This means we can interpret the rows of V^T as different topics, each represented by a set of words. The rows of U then show the importance of each topic in the documents.



For example, we can factor the term-document matrix in our example above as

$$\begin{array}{c} \\ \text{doc 1} \\ \text{doc 2} \\ \text{doc 3} \\ \text{doc 4} \\ \text{doc 5} \\ \text{doc 6} \\ \text{doc 7} \end{array}\!\!\begin{array}{c} \begin{array}{cccccccc} \text{linear} & \text{matrix} & \text{convex} & \text{minimum} & \text{derivative} & \text{constrains} & \text{factorization} & \text{dimension} \end{array} \\ \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} \end{array} = \begin{array}{c} \\ \text{doc 1} \\ \text{doc 2} \\ \text{doc 3} \\ \text{doc 4} \\ \text{doc 5} \\ \text{doc 6} \\ \text{doc 7} \end{array}\!\!\begin{array}{c} \begin{array}{cc} \text{topic 1 factors} & \text{topic 2 factors} \end{array} \\ \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix} \end{array} \begin{array}{c} \begin{array}{cccccccc} \text{linear} & \text{matrix} & \text{convex} & \text{minimum} & \text{derivative} & \text{constrains} & \text{factorization} & \text{dimension} \end{array} \\ \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \end{array} \begin{array}{l} \\ \text{topic 1} \\ \text{topic 2} \end{array}$$

This matrix factorization reveals two main topics. One topic includes the words "linear", "matrix", "factorization", and "dimension" (likely a Linear Algebra topic). You can see this by looking at the nonzero entries in the first row of V^T. The other topic includes "convex", "minimum", "derivative", and "constraints" (likely an Optimization topic). You can see this by looking at the nonzero entries in the second row of V^T.

By examining the rows of U, we see that documents 1, 6, and 7 deal only with topic 1, documents 3 and 5 deal only with topic 2, and documents 2 and 4 combine both topics (perhaps discussing Machine Learning).

In practice, one of the most used algorithms for topic modeling is nonnegative matrix factorization (NMF). NMF approximates A by the product WH of two low-rank matrices W and H with nonnegative entries. Another option is to use a truncated SVD, which we will study in Chapter 47, or to apply the CR or CUR factorizations. The CR factorization is the approach we use to factor the document-term matrix. Notice that the columns and rows of the factors in the factorization are actual columns and rows of A.

Problems

19.1 Show that the matrix

$$A = \begin{bmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix},$$

with $m \geqslant 2$, has full column rank if the numbers $t_1, \ldots, t_m$ are distinct.

19.2 There is an explicit formula for the coefficient vector $x = \left[\begin{smallmatrix} c_0 \\ c_1 \end{smallmatrix}\right]$ in the regression line problem discussed in Section 19.1. Given data points $(t_1, y_1), \ldots, (t_m, y_m)$, use the solution formula (19.4) to show that the coefficients of the regression line are

$$\begin{aligned} c_1 &= \frac{\sum_{i=1}^{m}(t_i - \overline{t})(y_i - \overline{y})}{\sum_{i=1}^{m}(t_i - \overline{t})^2}, \quad \text{and} \\ c_0 &= \overline{y} - c_1\overline{t}, \end{aligned}$$

where $\overline{t} = (t_1 + \cdots + t_m)/m$ and $\overline{y} = (y_1 + \cdots + y_m)/m$.

19.3 Table 19.1 below shows sales (in millions of yen) and advertising expenses (in millions of yen) for the Japanese fashion company "Nihon Fashion" over nine years. We want to predict sales based on advertising expenses.

(a) Plot the data. Is there a linear trend between sales and advertising?
(b) Find the equation of the regression line. Plot the line together with the data points.
(c) Predict sales if 70 million yen were spent on advertising.

Year	Sales (millions of yen)	Advertising (millions of yen)
1	680	25
2	790	28
3	880	32
4	1100	36
5	1225	45
6	1340	50
7	1460	55
8	1490	60
9	1575	62

Table 19.1: Sales and advertising data for Nihon Fashion.

Part V
THE LU FACTORIZATION OF A MATRIX

In this part of the book, we focus on the first and probably most fundamental problem in Linear Algebra: solving linear systems $Ax = b$. We all learned to solve these using Gaussian elimination. We will now see that this process often leads to a factorization of the matrix A. When possible, we write $A = LU$, where L is unit lower triangular and U is upper triangular. This factorization makes solving $Ax = b$ easier, especially if we need to solve several systems with the same matrix A. Not every matrix has an LU factorization. But when that happens, we can always permute the rows of A to get one. This leads to Gaussian elimination with partial pivoting, and the LU factorization with partial pivoting. While Chapter 7 explored geometric approaches to the solution of linear systems of equations, this part focuses on algebraic methods.

Chapter 20
V-WHY SHOULD WE CARE?

The LU factorization of a matrix is an important tool in Machine Learning. It helps solve various mathematical problems that often arise in applications. Here are a few examples.

20.1 Solving linear systems of equations with several right-hand sides

Suppose we have an $n \times n$ matrix A, and we need to solve $Ax = b_i$ for many different right-hand sides b_i. If we use Gaussian elimination each time, we repeat a lot of the same work since A does not change. Can we avoid this? One idea is to apply Gaussian elimination once to the augmented matrix

$$\left[\, A \middle| b_1 \; b_2 \cdots b_k \,\right],$$

and solve all k systems at once. But in practice, we often do not have all the b_i's upfront. They might arrive at different times, and we may need to act as soon as a new b_i comes in. Or maybe the next b_i depends on the solution to the previous system $Ax = b_{i-1}$. What can we do in situations like this? We will see in Chapter 22 that the LU factorization of a matrix helps with this problem.

Let us look at a few situations and example applications where we often need to solve linear systems with multiple right-hand sides.

Interpolation

When fitting a curve $y = f(t)$ such as a polynomial to a dataset $(t_1, y_1), \ldots, (t_m, y_m)$, the model's parameters are usually estimated by solving a linear system. If we have additional datasets, we can end up with the same coefficient matrix A but different right-hand sides b.

Example 20.1 Suppose we have several datasets, each with points in $\mathbb{R}^2$. We want to fit a polynomial of degree $m - 1$ to each dataset, so the curve passes through all the points. For each dataset i, we need to find coefficients $c_{i0}, c_{i1}, \ldots, c_{i,m-1}$ for the

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_20

polynomial $c_{i0}+c_{i1}t+\cdots+c_{i,m-1}t^{m-1}$ that best fits the points $(t_1, y_{i1}), \ldots, (t_m, y_{im})$. Note the t-values are the same for all datasets, but the y-values may differ.

To fit the data, we solve a linear system for each dataset:

$$\begin{bmatrix} 1 & t_1 & t_1^2 & \ldots & t_1^{m-1} \\ 1 & t_2 & t_2^2 & \ldots & t_2^{m-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_m & t_m^2 & \ldots & t_m^{m-1} \end{bmatrix} \begin{bmatrix} c_{i0} \\ c_{i1} \\ \vdots \\ c_{i,m-1} \end{bmatrix} = \begin{bmatrix} y_{i1} \\ y_{i2} \\ \vdots \\ y_{im} \end{bmatrix}.$$

The matrix on the left is the Vandermonde matrix for $t_1, t_2, \ldots, t_m$, and the right-hand side contains the y-values for the dataset. Notice that the matrix is the same for all datasets.

Modeling physical/biological systems

As Galileo said, mathematics is the language of nature. We use mathematics to model physical and biological systems, from atoms to the entire Earth. Tools like partial differential equations help us create these models. To solve the related mathematical problems on a computer, we often "discretize" the problem, which means transforming it into a linear system.

Sometimes, we want to study the system under different scenarios. Each scenario or condition being simulated requires we solve the same system of equations with different right-hand sides.

Example 20.2 In climate modeling, understanding how different conditions affect rainfall is important.

Imagine a climate model for the Southwestern United States, which often faces droughts and water scarcity issues. The model involves solving a system of partial differential equations for atmospheric moisture and precipitation. This results in a system of linear equations with a coefficient matrix A.

Climate scientists explore several scenarios to understand how different factors influence regional precipitation:

Scenario 1: Current Climate (Historical Baseline). Simulates precipitation patterns based on historical data and natural climate variability. This helps to understand the baseline conditions without significant human impact.

Scenario 2: Moderate Climate Change. Assumes moderate greenhouse gas emission reductions or stabilization, leading to a modest increase in global temperatures. This scenario helps to assess the impact of achievable mitigation efforts.

Scenario 3: High Climate Change. Assumes continued high emissions of greenhouse gases, resulting in significant global temperature rise and altered atmospheric circulation patterns. This scenario predicts the worst-case future conditions without effective climate action.

For each scenario, we solve the system $Ax = b_i$. Here, x is the solution vector representing variables like precipitation rates. The right-hand side b_i varies for each scenario, reflecting different boundary conditions and climate forcings specific to each situation. By solving these systems, we can compare the impacts of different climate conditions on regional precipitation.

Iterative methods

Many mathematical problems do not have a closed-form solution or cannot be solved in a finite number of steps. For these problems, we use iterative algorithms, where each iteration brings us closer to the solution.

In many cases, each iteration involves solving a linear system with a fixed matrix A but a changing right-hand side vector b. This is common in Optimization and Linear Algebra problems. We will see an example of this when we introduce algorithms for computing eigenvalues and eigenvectors of matrices in Chapters 37 and 38.

The previous examples show situations where we need to solve linear systems with the same matrix A but different right-hand side vectors b. This problem arises in many areas of Applied Mathematics, Statistics, and Data Science, including engineering design, imaging and signal processing (e.g., CT scans), portfolio analysis, Gaussian processes, and Bayesian inference.

Next, we consider a more theoretical problem where we need to solve $Ax = b$ for different right-hand sides b.

Finding the inverse of a matrix

While we rarely need to compute the inverse of an $n \times n$ matrix A, it is good to know an efficient method when we do. The inverse matrix A^{-1} satisfies

$$AA^{-1} = I_n.$$

If we let x_i be the i-th column of A^{-1}, and note that the i-th column of AA^{-1} is Ax_i, we see that we need to solve n linear systems:

$$Ax_i = e_i \qquad \text{for } i = 1, \ldots, n,$$

where e_i is the ith column of I_n. So, to find A^{-1}, we solve n linear systems with the same matrix A but different right-hand sides e_i.

20.2 Computing the determinant of a matrix

Most of us learned the "big formula" for the determinant of an $n \times n$ matrix A in our introductory Linear Algebra course[1]:

$$\det(A) = \sum_{\sigma \in S_n} \left(\operatorname{sgn}(\sigma) \prod_{i=1}^{n} A_{i,\sigma(i)} \right), \tag{20.1}$$

where S_n is the set of all permutations of $\{1, 2, \ldots, n\}$, σ is a permutation in S_n, and $\operatorname{sgn}(\sigma)$ is the sign (or parity) of the permutation σ.

There are $n! = n(n-1) \cdots 2 \cdot 1$ permutations in S_n. This becomes an extremely large number even for modest values of n. For instance, when $n = 1000$, the determinant formula would have around 4×10^{2567} terms, far more than any computer can handle! Yet, computers compute determinants quickly. How do they do it? They use the LU factorization.

LU factorization breaks the matrix A into two simpler matrices: L (lower triangular) and U (upper triangular). Once we have these, finding the determinant is much easier because

$$\det(A) = \det(LU) = \det(L)\ \det(U).$$

Since L and U are triangular, their determinants are just the products of their diagonal entries. This makes the process much faster and easier for computers. Computing the LU factorization of an $n \times n$ matrix takes about n^3 operations, which is much more practical than computing $n!$ permutations. So instead of handling an enormous number of permutations, we use LU factorization to compute the determinant efficiently.

Example 20.3 Consider the matrix below and its LU factorization (we will show how to find it in Chapter 22):

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 4 & 3 & 10 \\ 6 & 4 & 15 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 1 & 1 \end{bmatrix}}_{=:L} \underbrace{\begin{bmatrix} 2 & 1 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 2 \end{bmatrix}}_{=:U}.$$

Then, $\det(A) = \det(L)\det(U) = (1 \cdot 1 \cdot 1) \cdot (2 \cdot 1 \cdot 2) = 4$.

20.3 Label propagation

Suppose you work for a social network and you want to make good movie recommendations to keep users engaged. Some users have already told you whether they like or dislike a certain movie. Many others have not given any feedback yet.

---

[1] Do not worry if this formula looks confusing; it is rarely used in practice.

You also have another piece of information: the friendship graph. Each user is a node, and edges connect friends. Friends often influence each other, so their tastes are not independent. If you are friends with many people who liked the movie, chances are that you will like it too. If most of your friends disliked it, then you probably will not enjoy it either.

The question is how to use the opinions of the labeled users, the ones who already said "like" or "dislike", to guess the opinions of everyone else. The approach is to spread these labels across the network in a way that respects the connections in the graph. This is the label propagation problem. The basic idea is that information flows through the links of the graph, so users who are connected are more likely to end up with similar labels.

What makes this problem especially interesting for us is that the "spreading" process can be written as a linear system of equations. The matrix that shows up is not just any matrix, but a very special one: the graph Laplacian matrix. We will see exactly how this works in Section 24.1.

Chapter 21
V-WHAT YOU MAY HAVE LEARNED BEFORE

When we need to solve a linear system $Ax = b$, we use Gaussian elimination to transform the augmented matrix $\left[\, A \middle| b \,\right]$ into $\left[\, U \middle| c \,\right]$, where U is upper triangular. The linear systems $Ux = c$ and $Ax = b$ have the same solutions, but $Ux = c$ is easier to solve because U is upper triangular.

Gaussian elimination relies on elementary row operations, which we reviewed in Chapter 16.

21.1 Solving linear systems by Gaussian elimination

Given a matrix A and a vector b, we want to find a vector x such that $Ax = b$. If such a vector exists, we call it a **solution** of the system. A system can have one solution, many solutions, or none. The set of all solutions is called the **solution set**.

To solve $Ax = b$, we use Gaussian elimination. This method transforms A into an upper triangular matrix U using elementary row operations. We apply the same operations to b, resulting in a new vector c.

This process turns the augmented matrix $[A \mid b]$ into $[U \mid c]$ using a finite sequence of elementary row operations. The new system $Ux = c$ is easier to solve and has the same solution set as the original.

Theorem 21.1. (Elementary row operations preserve the solution set)

The solution set of $Ax = b$ is the same as the solution set of $Ux = c$, where $Ux = c$ is obtained from $Ax = b$ by a finite sequence of elementary row operations.

Proof The augmented matrix $[A \mid b]$ is row equivalent to $[U \mid c]$. By Theorem 16.39, there exists an invertible matrix S (a product of elementary matrices) such that $S[A \mid b] = [U \mid c]$. This implies $U = SA$ and $c = Sb$.

Assume x_0 is a solution of $Ax = b$. Then, $Ax_0 = b$, which implies $SAx_0 = Sb$. Hence, x_0 is also a solution of $Ux = c$.

Now assume x_0 is a solution of $Ux = c$. Then $SAx_0 = Sb$. Since S is invertible, multiplying both sides by S^{-1} gives $Ax_0 = b$. Thus, x_0 is a solution of $Ax = b$. □

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_21

When A is a square and invertible matrix, Gaussian elimination is the main method used by scientific computing libraries to solve linear systems. It transforms the system $Ax = b$ into $Ux = c$, where U is upper triangular. We can then solve $Ux = c$ easily using a method called **backward substitution**.

Example 21.2 Consider the linear system

$$\begin{aligned} 2x_1 + \ x_2 - \ x_3 + 2x_4 &= 5 \\ 4x_1 + \ 5x_2 - 3x_3 + 6x_4 &= 9 \\ -2x_1 + \ 5x_2 - 2x_3 + 6x_4 &= 2 \\ 4x_1 + 11x_2 - 4x_3 + 8x_4 &= 2 \end{aligned}$$

or in matrix-times-vector form

$$, \underbrace{\begin{bmatrix} 2 & 1 & -1 & 2 \\ 4 & 5 & -3 & 6 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 5 \\ 9 \\ 2 \\ 2 \end{bmatrix}}_{b} .$$

First, we form the **augmented matrix**:

$$\left[\, A \middle| b \,\right] = \left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 5 \\ 4 & 5 & -3 & 6 & 9 \\ -2 & 5 & -2 & 6 & 2 \\ 4 & 11 & -4 & 8 & 2 \end{array}\right].$$

We apply row operations to the matrix $[A|b]$ until A is in row-echelon form. Recall that in Gaussian elimination, we use three types of transformations: multiplying a row by a nonzero scalar, swapping two rows, and subtracting a multiple of one row from another. The goal is to create zeros below the main diagonal, working through each column one at a time.

Step 1: To introduce zeros below the (1,1) entry of A, we first subtract twice the first row from the second row. Then, we add the first row to the third row, and finally, subtract twice the first row from the fourth row. This results in

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 5 \\ 0 & 3 & -1 & 2 & -1 \\ 0 & 6 & -3 & 8 & 7 \\ 0 & 9 & -2 & 4 & -8 \end{array}\right].$$

Step 2: Next, we subtract twice the second row from the third row and three times the second row from the fourth row to introduce zeros below the $(2, 2)$ entry. This gives

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 5 \\ 0 & 3 & -1 & 2 & -1 \\ 0 & 0 & -1 & 4 & 9 \\ 0 & 0 & 1 & -2 & -5 \end{array}\right].$$

Step 3: Finally, we add row 3 to row 4 to create a zero below the $(3,3)$ entry. This gives

$$\left[\begin{array}{cccc|c} 2 & 1 & -1 & 2 & 5 \\ 0 & 3 & -1 & 2 & -1 \\ 0 & 0 & -1 & 4 & 9 \\ 0 & 0 & 0 & 2 & 4 \end{array}\right] =: \left[\, U \,|\, c \,\right],$$

where the matrix U is upper triangular.

Gaussian elimination gives the upper-triangular system $Ux = c$:

$$\begin{aligned} 2x_1 + \ x_2 - x_3 + 2x_4 &= \ \ 5 \\ 3x_2 - x_3 + 2x_4 &= -1 \\ -x_3 + 4x_4 &= \ \ 9 \\ 2x_4 &= \ \ 4 \end{aligned}$$

We solve this system by backward substitution (from the bottom up):

$$\begin{aligned} x_4 &= 4/2 = 2, \\ x_3 &= -9 + 4x_4 = -1, \\ x_2 &= \frac{1}{3}(-1 + x_3 - 2x_4) = -2, \quad \text{and} \\ x_1 &= \frac{1}{2}(5 - x_2 + x_3 - 2x_4) = 1. \end{aligned}$$

Note that in this example, we reduced A to row-echelon form without using any row interchanges (Type II elementary row operations).

By Theorem 16.42, we could also transform A into its reduced-row echelon form $B = \operatorname{rref}(A)$. So for a system $Ax = b$, there exists a nonsingular matrix S such that

$$S[A \,|\, b] = [B \,|\, c]$$

for some vector c. The systems $Ax = b$ and $Bx = c$ have the same solution set. This process is called **Gauss-Jordan elimination**. However, in this part of the book, we will focus on Gaussian elimination.

Problems

21.1 Assume the linear system of equations $Ax = b$, where A is an $m \times n$ matrix, is consistent (i.e., it has at least one solution). Use Exercise 16.28 to explicitly describe the solution set of this system.

21.2 Consider the linear system of equations $Ax = b$ and assume that

$$\left[\, A \middle| b \,\right] \sim_{\mathrm{r}} \left[\, \mathrm{rref}(A) \middle| c \,\right],$$

where $R = \mathrm{rref}(A)$ is the matrix in reduced-row echelon form of A. Show that $Ax = b$ and $Rx = c$ have the same solution set.

21.3 Consider the linear system $Lx = b$, where L is an $n \times n$ invertible lower triangular matrix. This type of system is solved using **forward substitution**: we first solve $L_{11}x_1 = b_1$, then use x_1 to solve for x_2 in the second equation, and so on, working down the rows. Determine how many arithmetic operations are needed to find the unique solution using forward substitution.

21.4 Consider the following system of linear equations, where k is a real constant:

$$\begin{cases} x + y + kz = 1 \\ x + ky + z = 1 \\ kx + y + z = -2 \end{cases}$$

Determine the values of k for which the system has:

(a) No solution
(b) A unique solution
(c) Infinitely many solutions

For the case with infinitely many solutions, provide the general solution in vector form.

21.5 In machine learning, fitting a model to data is a fundamental task. One basic model is polynomial regression. Given a set of data points (x_i, y_i), we want to find the coefficients of a polynomial $P(x) = c_0 + c_1x + c_2x^2$ that passes through them.

Find the coefficients c_0, c_1, c_2 of the quadratic polynomial that passes through the points (-1, 0), (1, 8), and (2, 9) by setting up a system of linear equations and solving it using Gauss-Jordan elimination.

21.6 The process of Gauss-Jordan elimination to find A^{-1} is equivalent to finding a sequence of elementary matrices $E_k, \ldots, E_2, E_1$ such that $(E_k \cdots E_2E_1)A = I$. The product of these elementary matrices is then A^{-1}.

Consider the matrix:

$$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 2 & 1 & 6 \end{bmatrix}$$

(a) Find the inverse of A using the standard Gauss-Jordan elimination method on the block matrix $[A|I]$.
(b) For each row operation you performed in part (a), write down the corresponding elementary matrix.
(c) Show explicitly that the product of these elementary matrices in the correct order equals the inverse A^{-1} you found.

Chapter 22
V-CORE TOPICS

22.1 The LU Factorization of a matrix

In this chapter, we show how to factor a square matrix A as the product of a lower triangular matrix L and an upper triangular matrix U. This is called an **LU factorization** of A.

As we reviewed in the previous chapter, Gaussian elimination transforms a square matrix $A \in \mathbb{R}^{n\times n}$ into an upper triangular matrix $U \in \mathbb{R}^{n\times n}$ using elementary row operations. These operations are multiplying a row by a nonzero scalar (type I), interchanging rows (type II), and row replacements (type III)[1].

In this chapter, we show that if we can apply Gaussian elimination to A without row interchanges, we can view the process as multiplying A on the left by a sequence of $n-1$ matrices:

$$L_{n-1}L_{n-2}\cdots L_2L_1A = U.$$

Each matrix L_i is the product of type III elementary matrices and is **unit lower triangular**, which means they are lower triangular with ones on the main diagonal. These matrices are always invertible because their diagonal entries are all nonzero. So, we can write

$$A = \underbrace{L_1^{-1}\cdots L_{n-1}^{-1}}_{=:L} U = LU.$$

The inverse of a unit lower triangular matrix is also unit lower triangular, and the product of these matrices remains unit lower triangular. Therefore, L is unit lower triangular.

So, when we apply Gaussian elimination without interchanges, we factor the matrix A into the product of a unit lower triangular matrix L and an upper triangular matrix U.

Definition 22.1. (LU factorization)

For an $n \times n$ matrix A, we call $A = LU$ an **LU factorization** of A if

[1] For a refresher on elementary row operations and elementary matrices, see Definitions 16.21 and 16.23

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_22

(i) the matrix $L \in \mathbb{R}^{n\times n}$ is unit lower triangular, and
(ii) the matrix $U \in \mathbb{R}^{n\times n}$ is upper triangular.

We will address the following three questions:

1. Which matrices have an LU factorization?
2. How can we compute it?
3. Are the factors L and U unique?

Let us start with how to compute it and leave the other two questions for Section 22.1.2 and Chapter 23.

We begin with a small example.

Example 22.2 Consider the matrix

$$A = \begin{bmatrix} 2 & 1 & -1 & 2 \\ 4 & 5 & -3 & 6 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix}.$$

This is the matrix we applied Gaussian elimination to in Example 21.2. We will run the Gaussian elimination process again, but this time, we will show the product by elementary matrices associated with each row operation at each step.

Step 1: Subtract twice the first row from the second row, add the first row to the third row, and subtract twice the first row from the fourth row. This is equivalent to the matrix multiplication

$$L_1 A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ -2 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & -1 & 2 \\ 4 & 5 & -3 & 6 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 1 & -1 & 2 \\ 0 & 3 & -1 & 2 \\ 0 & 6 & -3 & 8 \\ 0 & 9 & -2 & 4 \end{bmatrix}.$$

The matrix L_1 is the product of the elementary matrices associated with the three row operations described above. Note that L_1 is unit lower triangular. Flipping the signs of the entries in column 1 below the diagonal gives 2, -1, 2. These are called **multipliers**. The multipliers are the fractions of row 1 subtracted from rows 2, 3, and 4 to produce zeros in column 1.

Step 2: Subtract twice the second row from the third row and three times the second row from the fourth row. This is equivalent to the matrix multiplication

$$L_2(L_1 A) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -2 & 1 & 0 \\ 0 & -3 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & -1 & 2 \\ 0 & 3 & -1 & 2 \\ 0 & 6 & -3 & 8 \\ 0 & 9 & -2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 1 & -1 & 2 \\ 0 & 3 & -1 & 2 \\ 0 & 0 & -1 & 4 \\ 0 & 0 & 1 & -2 \end{bmatrix}.$$

The matrix L_2 is the product of the elementary matrices associated with the two row operations described above. As L_1, it is unit lower triangular. Flipping the signs of the entries below the diagonal in column 2 gives 2 and 3, which are the next two multipliers.

Step 3: Add row 3 to row 4. This is equivalent to the matrix multiplication

$$L_3(L_2L_1A) = \begin{bmatrix} 1&0&0&0\\ 0&1&0&0\\ 0&0&1&0\\ 0&0&1&1 \end{bmatrix} \begin{bmatrix} 2&1&-1&2\\ 0&3&-1&2\\ 0&0&-1&4\\ 0&0&1&-2 \end{bmatrix} = \begin{bmatrix} 2&1&-1&2\\ 0&3&-1&2\\ 0&0&-1&4\\ 0&0&0&2 \end{bmatrix} =: U.$$

The matrix L_3 is the elementary matrix for the single row operation applied in this case. It is unit lower triangular. Flipping the sign of the entry below the diagonal in column 3 gives the last multiplier, -1.

Putting everything together, we have $L_3L_2L_1A = U$, so $A = L_1^{-1}L_2^{-1}L_3^{-1}U$. Computing the inverses of L_1, L_2, and L_3, and their product, turns out to be surprisingly simple.

First, to find L_1^{-1}, L_2^{-1} and L_3^{-1}, we just flip the signs of their subdiagonal entries:

$$L_1^{-1} = \begin{bmatrix} 1&0&0&0\\ -2&1&0&0\\ 1&0&1&0\\ -2&0&0&1 \end{bmatrix}^{-1} = \begin{bmatrix} 1&0&0&0\\ 2&1&0&0\\ -1&0&1&0\\ 2&0&0&1 \end{bmatrix}, \qquad L_2^{-1} = \begin{bmatrix} 1&0&0&0\\ 0&1&0&0\\ 0&-2&1&0\\ 0&-3&0&1 \end{bmatrix}^{-1} = \begin{bmatrix} 1&0&0&0\\ 0&1&0&0\\ 0&2&1&0\\ 0&3&0&1 \end{bmatrix}, \quad \text{and}$$

$$L_3^{-1} = \begin{bmatrix} 1&0&0&0\\ 0&1&0&0\\ 0&0&1&0\\ 0&0&1&1 \end{bmatrix}^{-1} = \begin{bmatrix} 1&0&0&0\\ 0&1&0&0\\ 0&0&1&0\\ 0&0&-1&1 \end{bmatrix}.$$

So, in each matrix L_i^{-1} the step-i multipliers appear in column i below the diagonal.

Second, the product $L_1^{-1}L_2^{-1}L_3^{-1}$ is a unit lower triangular matrix. Its nonzero subdiagonal entries come from L_1^{-1}, L_2^{-1}, and L_3^{-1}, placed in their corresponding positions. That is,

$$L_1^{-1}L_2^{-1}L_3^{-1} = \begin{bmatrix} 1&0&0&0\\ 2&1&0&0\\ -1&2&1&0\\ 2&3&-1&1 \end{bmatrix}.$$

Putting it all together, we have factored the matrix A as a unit lower triangular matrix times an upper triangular matrix:

$$\begin{bmatrix} 2&1&-1&2\\ 4&5&-3&6\\ -2&5&-2&6\\ 4&11&-4&8 \end{bmatrix} = \begin{bmatrix} 1&0&0&0\\ 2&1&0&0\\ -1&2&1&0\\ 2&3&-1&1 \end{bmatrix} \begin{bmatrix} 2&1&-1&2\\ 0&3&-1&2\\ 0&0&-1&4\\ 0&0&0&2 \end{bmatrix}.$$

The upper triangular factor U is the end product of Gaussian elimination. The unit lower triangular factor L contains the multipliers we used during the process. It holds the information needed to reconstruct all the operations we performed.

Now, we want to formalize all the ideas from Example 22.2. Let us start with the following definition.

Definition 22.3. (Core unit lower triangular matrices)

An $n \times n$ **core unit lower triangular matrix** is a matrix of the form

$$L_k = \begin{bmatrix} & & \text{col } k & & \\ 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -\ell_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -\ell_{nk} & & 1 \end{bmatrix}. \tag{22.1}$$

All the entries that are not depicted are zero. The subdiagonal entries $-\ell_{k+1,k}, \ldots, -\ell_{n,k}$ are in column k of L_k.

Notice that at each step of Gaussian elimination in Example 22.2, we used core unit lower triangular matrices. The matrix L_k is formed by the product $n - k$ type III elementary matrices, which we use to create zeros below position (k, k).

We can write the matrix L_k in (22.1) as a rank-1 modification of the $n \times n$ identity matrix. Define the vector

$$\ell_k := \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \ell_{k+1,k} \\ \vdots \\ \ell_{nk} \end{bmatrix}.$$

Then, we can write

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & \ddots \\ & & & & & 1 \end{bmatrix} - \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \ell_{k,k+1} \\ \vdots \\ \ell_{k,n} \end{bmatrix} \begin{bmatrix} 0 \cdots 0 & 1 & 0 \cdots 0 \end{bmatrix} = I_n - \ell_k e_k^T.$$

Expressing (22.1) as a rank-1 modification of the identity matrix helps us prove several important properties of core unit lower triangular matrices.

Theorem 22.4. (Basic properties of core unit lower triangular matrices)

Consider a core unit lower triangular matrix L_k as in (22.1). The following statements hold:

(a) The inverse of L_k is the core unit lower triangular matrix obtained by flipping the signs of the subdiagonal entries of L_k. In other words, we have

$$L_k^{-1} = (I_n - \ell_k e_k^T)^{-1} = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & \ell_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & \ell_{nk} & & 1 \end{bmatrix} = I_n + \ell_k e_k^T.$$

(b) We have $L_k L_{k+1} \cdots L_{k+r} = I_n - \ell_k e_k^T - \ell_{k+1} e_{k+1}^T - \cdots - \ell_{k+r} e_{k+r}^T$, for $0 \leqslant r \leqslant n-1-k$.

Proof We start by proving part (a). The zero patterns of the vectors ℓ_k and e_k imply that $e_k^T \ell_k = 0$. Thus, we have

$$(I_n - \ell_k e_k^T)(I_n + \ell_k e_k^T) = I_n + \ell_k e_k^T - \ell_k e_k^T - \ell_k (e_k^T \ell_k) e_k^T = I_n.$$

Therefore $(I_n - \ell_k e_k^T)^{-1} = I_n + \ell_k e_k^T$.

We now prove part (b) using induction on r. For $r = 0$, the condition in part (b) holds by definition. Assume it is true for some r.

The zero patterns of the vectors ℓ_i and e_j imply that $e_i^T \ell_j = 0$ for $i \leqslant j$. Then, by the induction hypothesis, we have

$$\begin{aligned} \prod_{i=k}^{k+r+1} L_i = \left(\prod_{i=k}^{k+r} L_i \right) \cdot L_{k+r+1} &= (I_n - \ell_k e_k^T - \cdots - \ell_{k+r} e_{k+r}^T)(I_n - \ell_{k+r+1} e_{k+r+1}^T) \\ &= I_n - \ell_k e_k^T - \cdots - \ell_{k+r} e_{k+r}^T - \ell_{k+r+1} e_{k+r+1}^T, \end{aligned}$$

as we wanted to show. □

We are now ready to describe the steps of Gaussian elimination using core unit lower triangular matrices, assuming only type III row operations are needed to reach upper triangular form. This viewpoint is the key to understanding Gaussian elimination as a matrix factorization. We will address the case with row interchanges in Section 22.3.

At step k, after zeroing out the subdiagonal entries in columns 1 through $k-1$, we have reduced A to a matrix of the form

$$L_{k-1}\cdots L_1 A = \begin{bmatrix} * & \cdots & \cdots & * & x_{1k} & * & \cdots & * \\ 0 & * & \cdots & * & x_{2k} & * & \cdots & * \\ \vdots & \ddots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & * & x_{k-1,k} & * & \cdots & * \\ 0 & \cdots & 0 & 0 & x_{kk} & * & \cdots & * \\ 0 & \cdots & 0 & 0 & x_{k+1,k} & * & \cdots & * \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & x_{nk} & * & \cdots & * \end{bmatrix}.$$

Let the vector $x_k := \begin{bmatrix} x_{1k} & \cdots & x_{nk} \end{bmatrix}^T$ denote the kth column of the matrix $L_{k-1}\cdots L_1 A$. At step k of Gaussian elimination, we need to introduce zeros in entries $k+1, \ldots, n$ of the vector x_k, unless they are already zero:

$$\begin{bmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ x_{k+1,k} \\ \vdots \\ x_{nk} \end{bmatrix} \longrightarrow \begin{bmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Since we do not need type II row operations (row interchanges), we can deduce that $x_{kk} \neq 0$. We create zeros below this entry by subtracting multiples of row k from rows $k+1, k+2, \ldots, n$.

First, we compute the **multipliers**

$$\ell_{ik} = \frac{x_{ik}}{x_{kk}} \quad \text{for } k+1 \leqslant i \leqslant n.$$

Then, we subtract ℓ_{ik} times row k from row i, for $i = k+1, \ldots, n$. We can express all these row operations as the matrix multiplication

$$L_k x_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -\ell_{k+1,k} & 1 & \\ & & \vdots & & \ddots \\ & & -\ell_{nk} & & 1 \end{bmatrix} \begin{bmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ x_{k+1,k} \\ \vdots \\ x_{nk} \end{bmatrix} = \begin{bmatrix} x_{1k} \\ \vdots \\ x_{kk} \\ 0 \\ \vdots \\ 0 \end{bmatrix},$$

where the multipliers $\ell_{k+1,k}, \ldots, \ell_{nk}$ are placed in column k of the core unit lower triangular matrix L_k.

Hence, at step k of Gaussian Elimination, we reduce A to

$$L_k L_{k-1} \cdots L_1 A = \begin{bmatrix} * & \cdots & * & x_{1k} & * & \cdots & * \\ 0 & * & \cdots & * & x_{2k} & * & \cdots & * \\ \vdots & \ddots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & * & x_{k-1,k} & * & \cdots & * \\ 0 & \cdots & 0 & 0 & x_{kk} & * & \cdots & * \\ 0 & \cdots & 0 & 0 & 0 & * & \cdots & * \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & 0 & * & \cdots & * \end{bmatrix}.$$

As we have seen, the entry x_{kk} is important in the kth step of Gaussian elimination because it is used to zero out all entries below it. This entry is called the k**th pivot**.

The following theorem formalizes the idea that Gaussian elimination without interchanges computes an LU factorization for any matrix A.

Theorem 22.5. (When Gaussian elimination computes an LU factorization)

If Gaussian elimination transforms an $n \times n$ matrix A into an upper triangular matrix U without using interchanges (so all the pivots are nonzero), then

$$A = LU,$$

where L is the unit lower triangular matrix

$$L = \begin{bmatrix} 1 & 0 & \cdots & \cdots & 0 \\ \ell_{21} & 1 & \ddots & & \vdots \\ \ell_{31} & \ell_{32} & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{n,n-1} & 1 \end{bmatrix}$$

with the entries ℓ_{ij} corresponding to the Gaussian elimination multipliers.

Proof Gaussian elimination without interchanges is equivalent to multiplying A on the left by a sequence of $n-1$ unit lower triangular elementary matrices, some of which may be the identity matrix if all entries below a pivot are already zero. We can express this as

$$L_{n-1} \cdots L_2 L_1 A = U \quad \text{with} \quad L_k = I_n - \ell_k e_k^T \ (1 \leqslant k \leqslant n-1),$$

where the vector ℓ_k contains the multipliers at step k. The matrix U is upper triangular, the final result of Gaussian elimination.

Since all the L_i matrices are invertible because they are unit lower triangular, we can write

$$A = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} U = LU.$$

By part (a) of Theorem 22.4, L_k^{-1} is a core unit lower triangular matrix given by $L_k^{-1} = I_n + \ell_k e_k^T$. Using part (b) of the same theorem, we get

$$L = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} = I_n + \ell_1 e_1^T + \ell_2 e_2^T + \cdots + \ell_{n-1} e_{n-1}^T.$$

Using the columns-times-rows interpretation of matrix multiplication (recall Proposition 7.9), we can write

$$L = I_n + \begin{bmatrix} \ell_1 & \ell_2 & \cdots & \ell_{n-1} \end{bmatrix} \begin{bmatrix} e_1^T \\ e_2^T \\ \vdots \\ e_{n-1}^T \end{bmatrix} = I_n + \begin{bmatrix} 0 & 0 & \cdots & 0 \\ \ell_{21} & 0 & \ddots & \vdots \\ \ell_{31} & \ell_{32} & \ddots & \vdots \\ \vdots & \vdots & \ddots & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{n,n-1} \end{bmatrix} \begin{bmatrix} 1 & 0 & \cdots & \cdots & 0 \\ 0 & 1 & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & 0 \end{bmatrix} =$$

$$I_n + \begin{bmatrix} 0 & 0 & \cdots & \cdots & 0 \\ \ell_{21} & 0 & \ddots & & \vdots \\ \ell_{31} & \ell_{32} & 0 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{n,n-1} & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & \cdots & 0 \\ \ell_{21} & 1 & \ddots & & \vdots \\ \ell_{31} & \ell_{32} & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{n,n-1} & 1 \end{bmatrix}.$$

This shows that L is unit lower triangular, with the Gaussian elimination multipliers as its entries, as desired. □

22.1.1 Solving linear systems of equations using the LU factorization

After factoring a matrix A as $A = LU$, solving linear systems with A as the coefficient matrix becomes straightforward.

Consider the linear system $Ax = b$ with A invertible, so there is a unique solution $x = A^{-1}b$. If we have an LU factorization of A, then both L and U are invertible, with all diagonal entries of U nonzero.

We can rewrite $Ax = b$ as $L(Ux) = b$. By setting $y := Ux$, we break the system into two triangular systems:

$$\begin{aligned} Ly &= b \quad \text{(lower triangular system)} \\ Ux &= y \quad \text{(upper triangular system).} \end{aligned}$$

To solve the lower triangular linear system

$$\begin{bmatrix} 1 & 0 & \cdots & \cdots & 0 \\ \ell_{21} & 1 & \ddots & & \vdots \\ \ell_{31} & \ell_{32} & 1 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{n,n-1} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix}$$

we use **forward substitution**, working from the top down:

$$\begin{aligned}
y_1 &= b_1 \\
y_2 &= b_2 - \ell_{21} y_1 \\
y_3 &= b_3 - \ell_{31} y_1 - \ell_{32} y_2 \\
&\vdots \\
y_i &= b_i - \ell_{i1} y_1 - \ell_{i2} y_2 - \cdots - \ell_{i,i-1} y_{i-1} \\
&\vdots \\
y_n &= b_n - \ell_{n1} y_1 - \ell_{n2} y_2 - \cdots - \ell_{n,n-2} y_{n-2} - \ell_{n,n-1} y_{n-1}.
\end{aligned} \tag{22.2}$$

To solve the upper triangular system

$$\begin{bmatrix}
u_{11} & \cdots & \cdots & u_{1,n-1} & u_{1n} \\
0 & \ddots & & \vdots & \vdots \\
\vdots & \ddots & u_{n-2,n-2} & u_{n-2,n-1} & u_{n-2,n} \\
\vdots & & 0 & u_{n-1,n-1} & u_{n-1,n} \\
0 & \cdots & \cdots & 0 & u_{nn}
\end{bmatrix}
\begin{bmatrix} x_1 \\ \vdots \\ x_{n-2} \\ x_{n-1} \\ x_n \end{bmatrix} =
\begin{bmatrix} y_1 \\ \vdots \\ y_{n-2} \\ y_{n-1} \\ y_n \end{bmatrix}$$

we use **backward substitution**, working from the bottom up:

$$\begin{aligned}
x_n &= \frac{y_n}{u_{nn}} \\
x_{n-1} &= \frac{y_{n-1} - u_{n-1,n} x_n}{u_{n-1,n-1}} \\
x_{n-2} &= \frac{y_{n-2} - u_{n-2,n-1} x_{n-1} - u_{n-2,n} x_n}{u_{n-2,n-2}} \\
&\vdots \\
x_i &= \frac{y_i - u_{i,i+1} x_{i+1} - \cdots - u_{i,n-1} x_{n-1} - u_{in} x_n}{u_{ii}} \\
&\vdots \\
x_1 &= \frac{y_1 - u_{12} x_2 - u_{13} x_3 - \cdots - u_{1,n-1} x_{n-1} - u_{1n} x_n}{u_{11}}.
\end{aligned} \tag{22.3}$$

Example 22.6 Let us solve the linear system $Ax = b$ from Example 21.2 again, but this time using the LU factorization of A from Example 22.2.

We first solve $Ly = b$ using forward substitution:

$$\begin{aligned} y_1 &= 5 \\ y_2 &= 9 - 2y_1 = -1 \\ y_3 &= 2 + y_1 - 2y_2 = 9 \\ y_4 &= 2 - 2y_1 - 3y_2 + y_3 = 4. \end{aligned}$$

We then solve $Ux = y$ using backward substitution:

$$\begin{aligned} x_4 &= y_4/2 = 2 \\ x_3 &= -(y_3 - 4x_4) = -1 \\ x_2 &= (y_2 + x_3 - 2x_4)/3 = -2 \\ x_1 &= (y_1 - x_2 + x_3 - 2x_4)/2 = 1, \end{aligned}$$

which matches the result from Example 21.2.

Remark 22.7. (Why use the LU factorization instead of Gaussian elimination?)

You might wonder why we need LU factorization if we can just apply Gaussian elimination to solve a linear system. As we saw in Chapter 20, there are situations where we need to solve many systems with the same matrix A but different right-hand sides b. Instead of starting from scratch each time, it is more efficient to factor A once as $A = LU$ and then use forward and backward substitution to solve each new system.

If A is an $n \times n$ matrix, solving $Ax = b$ directly using Gaussian elimination takes about n^3 operations. Computing the LU factorization of A also requires around n^3 operations. However, once the factorization is available, solving the two triangular systems $Ly = b$ and $Ux = y$ only takes about n^2 operations. For large n, as is common with data matrices, using the LU factorization is far more efficient when solving many systems with different right-hand sides but the same matrix A (see Problems 22.17 and 22.18).

As we will see later in Section 22.3, if A does not have an LU factorization, we can always rearrange the rows of A to create a matrix that does. In other words, there is a permutation matrix P (a product of elementary permutation matrices) such that $PA = LU$.

22.1.2 Existence and uniqueness of the LU factorization when A is nonsingular

We still need to answer two questions:

1. Which matrices have an LU factorization? and
2. If an LU factorization exists, is it unique?

In this section, we give necessary and sufficient conditions for a nonsingular square matrix to have an LU factorization, which is the most important case in applications. We will treat the singular and rectangular cases in Chapter 23.

First, let us start with a definition.

Definition 22.8. (Leading principal submatrices)

Let A be an $n \times n$ matrix and let $1 \leqslant k \leqslant n$ be an integer. We call the **k-th leading principal submatrix of** A the $k \times k$ submatrix of A formed by the entries in the first k rows and the first k columns of A. Using the colon notation introduced in Chapter 2, the kth leading principal submatrix of A is $A[1:k, 1:k]$.

Example 22.9 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & 3 \\ 4 & -1 & 0 \end{bmatrix}.$$

The first leading principal submatrix of A is the 1×1 matrix $[1]$, which is the entry in position $(1, 1)$.

The second leading principal submatrix is the 2×2 matrix $\left[\begin{smallmatrix} 1 & 2 \\ 2 & 1 \end{smallmatrix}\right]$, formed by the entries in the first two rows and the first two columns of A.

The third leading principal submatrix is the entire matrix A.

Theorem 22.10. (Existence of the LU factorization in the nonsingular case)

An $n \times n$ nonsingular matrix has an LU factorization if and only if its first $n - 1$ leading principal submatrices are invertible.

Proof Assume that A has an LU factorization. Since A is invertible, the upper triangular matrix U must also be invertible. This implies that the diagonal entries of U are nonzero.

Let $1 \leqslant k < n$ and let us partition A, L and U as follows

$$A = \begin{bmatrix} A_k & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{11}^{(k)} & 0 \\ L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} U_{11}^{(k)} & U_{12} \\ 0 & U_{22} \end{bmatrix},$$

where A_k, $L_{11}^{(k)}$ and $U_{11}^{(k)}$ are the k-th leading principal submatrices of A, L and U, respectively. The submatrices $L_{11}^{(k)}$ and $U_{11}^{(k)}$ are invertible because they are triangular matrices with nonzero diagonal entries.

The partition of A, L and U above shows that $A_k = L_{11}^{(k)} U_{11}^{(k)}$. This implies that the k-th leading principal submatrix of A is nonsingular, since both $L_{11}^{(k)}$ and $U_{11}^{(k)}$ are invertible.

Now, let us prove the converse by showing that each leading principal submatrix of A has an LU factorization, including A itself, which is the n-th leading principal submatrix. We will prove this by induction on k, the size of the leading principal submatrix.

Let us denote the k-th leading principal submatrix of A as A_k. Suppose all the first $n-1$ leading principal submatrices of A are invertible. For $k = 1$, the (1,1) entry A_{11} (the first leading principal submatrix A_1) is nonzero because, by assumption, it is nonsingular. Therefore, we can write $A_{11} = 1 \cdot A_{11}$, which is an LU factorization of the scalar A_{11}.

Let $1 < k \leqslant n$, and assume that the $(k-1)$-th leading principal submatrix of A has an LU factorization, so we have $A_{k-1} = L_{11}^{(k-1)} U_{11}^{(k-1)}$ for some unit lower triangular matrix $L_{11}^{(k-1)}$ and upper triangular matrix $U_{11}^{(k-1)}$. Since A_{k-1} is invertible, both $L_{11}^{(k-1)}$ and $U_{11}^{(k-1)}$ are also invertible.

Let us partition the k-th principal submatrix of A as

$$A_k = \begin{bmatrix} A_{k-1} & b \\ c^T & A_{kk} \end{bmatrix},$$

where b and c are vectors, and A_{kk} is the (k,k) entry of A. We will now find an LU factorization of A_k based on the LU factorization of A_{k-1}.

Let us show that an LU factorization of A_k of the form

$$A_k = \begin{bmatrix} A_{k-1} & b \\ c^T & A_{kk} \end{bmatrix} = \begin{bmatrix} L_{11}^{(k-1)} & 0 \\ \ell^T & 1 \end{bmatrix} \begin{bmatrix} U_{11}^{(k-1)} & u \\ 0 & U_{kk} \end{bmatrix},$$

exists for some vectors $\ell, u \in \mathbb{R}^{k-1}$, and scalar U_{kk}. This requires solving the following system of equations

$$\begin{aligned} &L_{11}^{(k-1)} u = b, \\ &\ell^T U_{11}^{(k-1)} = c^T, \quad \text{and} \\ &\ell^T u + U_{kk} = A_{kk}. \end{aligned}$$

Since the matrices $L_{11}^{(k-1)}$ and $U_{11}^{(k-1)}$ are invertible, this system of equations has the following solutions

$$\begin{aligned} &u = (L_{11}^{(k-1)})^{-1} b, \\ &\ell^T = c^T (U_{11}^{(k-1)})^{-1}, \quad \text{and} \\ &U_{kk} = A_{kk} - c^T (U_{11}^{(k-1)})^{-1} (L_{11}^{(k-1)})^{-1} b = A_{kk} - c^T A_{k-1}^{-1} b. \end{aligned}$$

This shows that A_k has an LU factorization, completing the induction step. □

Theorem 22.11. (Uniqueness of the LU factorization in the nonsingular case)

If A is a nonsingular matrix and has an LU factorization, then that factorization is unique.

Proof Assume that A has two different LU factorizations

$$A = L_1U_1 = L_2U_2.$$

Since both U_1 and L_2 are invertible, we can write

$$L_2^{-1}L_1 = U_2U_1^{-1}.$$

The inverse of a unit lower triangular matrix is also unit lower triangular, and the product of unit lower triangular matrices remains unit lower triangular (see Problem 22.1). Therefore, $L_2^{-1}L_1$ is a unit lower triangular matrix. Similarly, the inverse of an upper triangular matrix is upper triangular, and the product of upper triangular matrices remains upper triangular (see Problem 22.1). So, $U_2U_1^{-1}$ is upper-triangular.

Since $L_2^{-1}L_1 = U_2U_1^{-1}$, the resulting matrix is both unit lower triangular and upper triangular. The only matrix with both properties is the identity, so the expression must equal I_n. This implies $L_1 = L_2$ and $U_1 = U_2$ as desired. □

If you read the proof of Theorem 22.10, you may have noticed that we did not actually require the matrix A itself to be nonsingular in order to compute an LU factorization. The only requirement was that the first $n-1$ leading principal submatrices be nonsingular. However, since LU factorization is most commonly used to solve linear systems where the coefficient matrix A is nonsingular, that is the assumption we made in the theorem. But what about singular matrices? As long as the first $n-1$ leading principal submatrices are nonsingular, a singular matrix has an LU factorization, as the next example illustrates.

Example 22.12 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & -2 \\ 2 & 3 & 1 \end{bmatrix}.$$

This matrix is singular (its third row is the sum of the first two), but its first two leading principal submatrices are nonsingular, so A has an LU factorization. By applying Gaussian elimination to A, we get the factorization

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & -5 \\ 0 & 0 & 0 \end{bmatrix} =: LU.$$

So, we factor A as the product of a unit lower triangular matrix L and an upper triangular matrix U. However, note that the bottom-right entry of U is zero, making U singular.

Just like we used the LU factorization of a nonsingular matrix to solve linear systems, we can also use the LU factorization of a singular matrix to solve consistent linear systems.

Example 22.13 Take, for example, the linear system $Ax = b$ with

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & -2 \\ 2 & 3 & 1 \end{bmatrix} \qquad \text{and} \qquad b = \begin{bmatrix} 3 \\ 2 \\ 5 \end{bmatrix}.$$

This system is consistent – one solution is $x = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}^T$. Now, let us use the LU factorization of A from the previous example to find all the solutions to $Ax = b$.

Using forward substitution, we solve $Ly = b$, which gives $y = \begin{bmatrix} 3 & -1 & 0 \end{bmatrix}^T$. Next, we solve the system $Ux = y$:

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & -5 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \\ 0 \end{bmatrix}.$$

Even though the system involves a singular matrix, backward substitution still works, but the solution will depend on a free variable, x_3. Solving the system, we get:

$$\begin{aligned} x_2 &= -(y_2 + 5x_3) = 1 - 5x_3 \\ x_1 &= y_1 - 2x_2 - 3x_3 = 1 + 7x_3. \end{aligned}$$

Thus, the system has infinitely many solutions, given by

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} 7 \\ -5 \\ 1 \end{bmatrix},$$

where x_3 is a free parameter.

You can find general results about the existence and uniqueness of the LU factorization for singular matrices in Chapter 23. The proofs for those results are a bit more complex.

22.2 The LDL^T factorization for symmetric matrices

A square matrix A is symmetric if $A^T = A$. If a nonsingular symmetric matrix A has an LU factorization, we can use it to find a factorization that takes the symmetry of A into account. This leads to the LDL^T **factorization**, where the factorization naturally reflects the symmetry of A.

Let us begin with an example.

Example 22.14 Consider the following symmetric matrix

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 0 & 3 \\ -1 & 3 & 2 \end{bmatrix}.$$

You can easily verify that all the leading principal submatrices of A are nonsingular. Therefore, by Theorem 22.10, A has a (unique) LU factorization. This factorization is

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -5/4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & -1 \\ 0 & -4 & 5 \\ 0 & 0 & 29/4 \end{bmatrix} =: LU.$$

If we were only given the L and U factors, we would not immediately know that A is symmetric without multiplying them. Can we factor A in a way that shows its symmetry? Yes. Since the diagonal entries of U (the pivots from Gaussian elimination) are nonzero, we can factor U as $U = D\widehat{U}$, where D contains the diagonal entries of U (the pivots), and $\widehat{U}$ is a unit upper triangular matrix formed by dividing each row of U by its pivot.

In our example, we have

$$U = \begin{bmatrix} 1 & 2 & -1 \\ 0 & -4 & 5 \\ 0 & 0 & 29/4 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & -4 & 0 \\ 0 & 0 & 29/4 \end{bmatrix}}_{=:D} \underbrace{\begin{bmatrix} 1 & 2 & -1 \\ 0 & 1 & -5/4 \\ 0 & 0 & 1 \end{bmatrix}}_{=:\widehat{U}}.$$

Notice the remarkable result that $\widehat{U} = L^T$. This means the symmetric matrix A has the following factorization

$$A = LU = LD\widehat{U} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -5/4 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -4 & 0 \\ 0 & 0 & 29/4 \end{bmatrix} \begin{bmatrix} 1 & 2 & -1 \\ 0 & 1 & -5/4 \\ 0 & 0 & 1 \end{bmatrix} = LDL^T.$$

Now, if we know that a matrix is factored as $A = LDL^T$, we can immediately tell that A is symmetric, because LDL^T is always symmetric.

The result in the previous example was not just a coincidence.

Theorem 22.15. (The LDL^T factorization)

If A is an $n \times n$ symmetric (i.e., $A^T = A$), nonsingular matrix with an LU factorization, then we can factor A as

$$A = LDL^T,$$

where L is a unit lower triangular matrix and D is a nonsingular diagonal matrix.

Proof By assumption, A has an LU factorization, so we can write $A = LU$. Since A is invertible, U is also invertible, and its diagonal entries are all nonzero. We can then factor U as $U = D\widehat{U}$, where $\widehat{U}$ is unit upper triangular, and D is diagonal and invertible. Thus, we have

$$A = LU = LD\widehat{U}.$$

Since A is symmetric, we also have

$$A = A^T = \widehat{U}^T(DL^T).$$

This gives us an LU factorization of A with $\widehat{U}^T$ as the unit lower triangular factor and DL^T as the upper-triangular factor. Because the LU factorization is unique (Theorem 22.11), it follows that $L = \widehat{U}^T$. This implies $A = LDL^T$, as we wanted to show. □

22.3 Gaussian elimination with partial pivoting and the $PA = LU$ factorization

Not every matrix has an LU factorization. For example, consider the matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}.$$

Gaussian elimination cannot find an LU factorization here because the first pivot A_{11} is zero. However, since $A_{21} \neq 0$, we can fix this by swapping the rows. After the swap, the matrix becomes upper triangular:

$$\begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}.$$

In this case, we need row interchanges, in addition to replacements, to get an upper triangular matrix. Whenever a pivot is zero, row interchanges are required.

Consider another example in which Gaussian elimination requires row interchanges.

Example 22.16 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 4 & 1 \\ 1 & -1 & 1 \end{bmatrix}.$$

Does A have an LU factorization? Notice that $A_{11} \neq 0$. Thus, we do not need an interchange to apply the first step of Gaussian elimination. If we subtract 2 times row 1 from row 2, and row 1 from row 3, we get

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 0 & 1 \\ 0 & -3 & 1 \end{bmatrix}.$$

Now the entry in the pivot position $(2, 2)$ is zero and, therefore, we need an interchange to proceed with Gaussian elimination. If we swap rows 2 and 3, we finally obtain the matrix

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & -3 & 1 \\ 0 & 0 & 1 \end{bmatrix},$$

which is in upper triangular form.

In this section, we show that when row interchanges are used in Gaussian elimination, we can no longer write $A = LU$. However, we can always find a permutation matrix P such that $PA = LU$.

Another important reason to use row interchanges is to reduce numerical errors during computations, even when the matrix A already has an LU factorization, as the following example shows.

Example 22.17 Consider the linear system of equations

$$\begin{aligned} 0.01x_1 + 1.5x_2 &= 30.1 \\ 1.1x_1 + 0.7x_2 &= 25. \end{aligned}$$

The solution to the system is

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 10 \\ 20 \end{bmatrix}.$$

Suppose we are using an old calculator that only provides 3 digits of accuracy. By "3 digits of accuracy", we mean that the calculator rounds and displays only the first three digits in scientific notation. For example, 2.33436×10^2 would show as 2.33×10^2, and 2.3785×10^{-1} would round to 2.38×10^{-1}. Although modern calculators are more accurate, this example shows how small rounding errors can still cause large inaccuracies.

The augmented matrix for the system is

$$\left[\begin{array}{cc|c} 0.01 & 1.5 & 30.1 \\ 1.1 & 0.7 & 25 \end{array}\right].$$

If we choose 0.01 as the first pivot and perform row operations to eliminate the entry below it, we get

$$\left[\begin{array}{cc|c} 0.01 & 1.5 & 30.1 \\ 0 & -164.3 & -3286 \end{array}\right].$$

However, with our calculator's 3-digit accuracy, the second row is rounded to

$$\left[\begin{array}{cc|c} 0.01 & 1.5 & 30.1 \\ 0 & -164 & -3290 \end{array}\right].$$

Using backward substitution, we find the solution

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} -5 \\ 20.1 \end{bmatrix}.$$

Compare the computed solution $\left[\begin{smallmatrix} -5 \\ 20.1 \end{smallmatrix}\right]$ to the actual solution $\left[\begin{smallmatrix} 10 \\ 20 \end{smallmatrix}\right]$. Notice how small rounding errors in the first step caused a huge error in the final result, especially in the first component, which even has the wrong sign!

To prevent the error amplification we saw in the previous example, good computer algorithms select the largest number in each column as the pivot to reduce errors. This is particularly important when solving large systems of equations on a computer. This approach is called **Gaussian elimination with partial pivoting**, which we will introduce later in this section. But first, we review permutation matrices and their properties.

22.3.1 Permutation matrices

To understand Gaussian elimination with row interchanges as a sequence of matrix multiplications, we first need to understand permutation matrices.

A permutation matrix has exactly one nonzero entry in each row and column, and each of those nonzero entries is equal to one (see Definition 16.32).

We begin with the properties of elementary permutation matrices (Definition 16.30), the simplest kind of permutation matrices.

Theorem 22.18. (Properties of elementary permutation matrices)

Consider an elementary permutation matrix P as in (16.2). Then, the following properties hold:

(a) P is symmetric, that is, $P^T = P$.
(b) P is its own inverse, that is, $P^{-1} = P$.
(c) $\det(P) = -1$.

Proof Parts (a) and (b) follow directly from Proposition 16.31. To prove part (c), recall that we obtain an elementary permutation matrix P by exchanging rows i and j of the $n \times n$ identity matrix. Since $\det(I_n) = 1$ and the determinant changes sign when two rows are swapped, we have $\det(P) = -1$. □

Every permutation matrix can be expressed as a product of elementary permutation matrices.

Theorem 22.19. (Permutation matrices as product of elementary permutation matrices)

Every $n \times n$ permutation matrix P, with $n \geqslant 2$, is either an elementary permutation matrix or the product of elementary permutation matrices.

Proof Observe that the theorem holds for the identity matrix since it is an elementary matrix.

Let P be an $n \times n$ permutation matrix that is not the identity matrix. We prove the result by induction on n.

Notice that the only 2×2 permutation matrix that is not the identity is $P = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$, which is an elementary permutation matrix. So the result holds for $n = 2$.

Let $n \geqslant 3$. Consider a permutation matrix P of size n that is not the identity and assume the result holds for permutation matrices of size $n - 1$. Since P has the rows of the identity matrix in some order, one of its rows is e_1^T. Let i be the index of this row in P. We will consider two cases: $i = 1$ and $i \neq 1$.

When $i = 1$, the permutation matrix P has the form

$$P = \begin{bmatrix} 1 & 0 \\ 0 & \widetilde{P} \end{bmatrix},$$

where $\widetilde{P}$ is a permutation matrix of size $n-1$. Since P is not the identity, $\widetilde{P}$ cannot be the identity matrix either. By the induction hypothesis, we can write $\widetilde{P}$ as a product of elementary permutation matrices:

$$\widetilde{P} = \widetilde{P}_1 \widetilde{P}_2 \cdots \widetilde{P}_k,$$

for some $k \geqslant 1$. Hence, we have

$$P = \begin{bmatrix} 1 & 0 \\ 0 & \widetilde{P}_1 \widetilde{P}_2 \cdots \widetilde{P}_k \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & \widetilde{P}_1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \widetilde{P}_2 \end{bmatrix} \cdots \begin{bmatrix} 1 & 0 \\ 0 & \widetilde{P}_k \end{bmatrix} = P_1 P_2 \cdots P_k,$$

where $P_1, \ldots, P_k$ are elementary permutation matrices of size n, as we wanted to show.

When $i \neq 1$, let P_1 be the elementary permutation matrix that swaps rows i and 1. Then, we have

$$P_1 P = \begin{bmatrix} 1 & 0 \\ 0 & \widetilde{P} \end{bmatrix},$$

for some permutation matrix $\widetilde{P}$ of size $n - 1$. If $\widetilde{P}$ is the identity, we are done. If not, by the argument above, we can write $P_1 P = P_2 P_3 \cdots P_k$ for some elementary matrices $P_2, \ldots, P_k$. Since $P_1^{-1} = P_1$, we obtain $P = P_1 P_2 \cdots P_k$, as we wanted to show. □

With Theorem 22.19, we are ready to prove some important properties of permutation matrices.

Theorem 22.20. (Basic properties of permutation matrices)

The following properties hold:

(a) Every permutation matrix P is invertible, and $P^{-1} = P^T$.
(b) $\det(P) = \pm 1$ for every permutation matrix P.
(c) The product of permutation matrices $P_1 P_2 \cdots P_k$ is also a permutation matrix.

Proof Let P be a permutation matrix. By Theorem 22.19, we can write $P = P_1 \cdots P_k$ for some elementary permutation matrices P_i. Since elementary permutation matrices are invertible, and the product of invertible matrices is also invertible, P is invertible. Moreover, we have

$$P^{-1} = (P_1 \cdots P_k)^{-1} = P_k^{-1} \cdots P_1^{-1} = P_k^T \cdots P_1^T = (P_1 \cdots P_k)^T = P^T,$$

because $P_i^{-1} = P_i^T$ by Theorem 22.18.

Since P is a product of elementary permutation matrices, each of which has determinant -1, the determinant of P is equal to $(-1)^k$, where k is the number of such matrices in the product. If k is even, then $\det(P) = 1$. If k is odd, then $\det(P) = -1$. Therefore, $\det(P) = \pm 1$.

Part (c) follows from the fact that each permutation matrix is a product of elementary permutation matrices. Therefore, the product of permutation matrices is also a product of elementary permutation matrices, which means it is a permutation matrix as well. □

22.3.2 The $PA = LU$ factorization

In this section, we introduce Gaussian elimination with partial pivoting (GEPP) and show that applying GEPP to a matrix A results in an LU factorization of PA, where P is a permutation matrix.

Definition 22.21. (The PLU factorization of a matrix)

For an $n \times n$ matrix A, a PLU factorization of A is a factorization of the form $A = PLU$, where

(i) P is a permutation matrix,
(ii) L is unit lower triangular, and
(iii) U is upper triangular.

Since permutation matrices are invertible and their inverse is also a permutation matrix, if a matrix A has a PLU factorization, we can write $PA = LU$, for some permutation matrix[2] P. This tells us that a row-permuted version of A (that is, PA) has an LU factorization. Let us begin with an example.

[2] Here, P is the transpose of the one in $A = PLU$

Example 22.22 Consider the matrix from Example 22.2:

$$A = \begin{bmatrix} 2 & 1 & -1 & 2 \\ 4 & 5 & -3 & 6 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix}.$$

We will apply Gaussian elimination with partial pivoting to A and explain each step using elementary matrices. At each step, we will choose the largest entry in absolute value as the next pivot.

Step 1: We select the entry with the largest absolute value in the first column of A as the first pivot. In this case, we choose A_{21} (though A_{41} would also work). Next, we swap rows 1 and 2 by left-multiplying A by a permutation matrix P_1:

$$\underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{=:P_1} \begin{bmatrix} 2 & 1 & -1 & 2 \\ 4 & 5 & -3 & 6 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix} = \begin{bmatrix} 4 & 5 & -3 & 6 \\ 2 & 1 & -1 & 2 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix}.$$

Now, we perform type III row operations to create zeros below the first pivot. The three multipliers are:

$$\ell_{21} = \frac{2}{4} = \frac{1}{2},$$
$$\ell_{31} = \frac{-2}{4} = -\frac{1}{2}, \quad \text{and}$$
$$\ell_{41} = \frac{4}{4} = 1.$$

Notice that all these multipliers, in absolute value, are less than or equal to 1 because we chose the "largest" entry in column 1 as the pivot.

Next, we subtract $\frac{1}{2}$ of row 1 from row 2, add $\frac{1}{2}$ of row 1 to row 3, and subtract row 1 from row 4. This is done by left-multiplying by a core unit lower triangular matrix L_1:

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ -1/2 & 1 & 0 & 0 \\ 1/2 & 0 & 1 & 0 \\ -1 & 0 & 0 & 1 \end{bmatrix}}_{=:L_1} \begin{bmatrix} 4 & 5 & -3 & 6 \\ 2 & 1 & -1 & 2 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix} = \begin{bmatrix} 4 & 5 & -3 & 6 \\ 0 & -3/2 & 1/2 & -1 \\ 0 & 15/2 & -7/2 & 9 \\ 0 & 6 & -1 & 2 \end{bmatrix}.$$

Step 2: In column 2, rows 2 through 4, the entry with largest absolute value is $15/2$ located in row 3. We use this entry as the second pivot. To do this, we swap rows 2 and 3 by left-multiplying with a permutation matrix P_2:

$$\underbrace{\begin{bmatrix}1&0&0&0\\0&0&1&0\\0&1&0&0\\0&0&0&1\end{bmatrix}}_{=:P_2}\begin{bmatrix}4&5&-3&6\\0&-3/2&1/2&-1\\0&15/2&-7/2&9\\0&6&-1&2\end{bmatrix}=\begin{bmatrix}4&5&-3&6\\0&15/2&-7/2&9\\0&-3/2&1/2&-1\\0&6&-1&2\end{bmatrix}.$$

Now, we create zeros below the pivot. The two multipliers are

$$\ell_{32}=\frac{-3/2}{15/2}=-\frac{1}{5},\quad\text{and}$$
$$\ell_{42}=\frac{6}{15/2}=\frac{4}{5}.$$

Again, the multipliers are less than or equal to 1 in absolute value.

Next, we add $\frac{1}{5}$ of row 2 to row 3, and subtract $\frac{4}{5}$ of row 2 from row 4. This is done by left-multiplying with a core unit lower triangular matrix L_2:

$$\underbrace{\begin{bmatrix}1&0&0&0\\0&1&0&0\\0&1/5&1&0\\0&-4/5&0&1\end{bmatrix}}_{=:L_2}\begin{bmatrix}4&5&-3&6\\0&15/2&-7/2&9\\0&-3/2&1/2&-1\\0&6&-1&2\end{bmatrix}=\begin{bmatrix}4&5&-3&6\\0&15/2&-7/2&9\\0&0&-1/5&4/5\\0&0&9/5&-26/5\end{bmatrix}.$$

Step 3: In column 3, rows 3 and 4, the entry with largest absolute value is $9/5$, located in row 4. We use this entry as the third pivot. To do so, we swap rows 3 and 4 by left-multiplying with a permutation matrix P_3:

$$\underbrace{\begin{bmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&1&0\end{bmatrix}}_{=:P_3}\begin{bmatrix}4&5&-3&6\\0&15/2&-7/2&9\\0&0&-1/5&4/5\\0&0&9/5&-26/5\end{bmatrix}=\begin{bmatrix}4&5&-3&6\\0&15/2&-7/2&9\\0&0&9/5&-26/5\\0&0&-1/5&4/5\end{bmatrix}.$$

The last multiplier is

$$\ell_{43}=-\frac{1/5}{9/5}=-\frac{1}{9},$$

which is less than or equal to 1 in absolute value.

Finally, we add $\frac{1}{9}$ of row 3 to row 4 by left-multiplying with an elementary matrix L_3:

$$\underbrace{\begin{bmatrix}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&1/9&1\end{bmatrix}}_{=:L_3}\begin{bmatrix}4&5&-3&6\\0&15/2&-7/2&9\\0&0&9/5&-26/5\\0&0&-1/5&4/5\end{bmatrix}=\begin{bmatrix}4&5&-3&6\\0&15/2&-7/2&9\\0&0&9/5&-26/5\\0&0&0&2/9\end{bmatrix}.$$

Putting it all together, the elimination process is equivalent to multiplying A on the left by a sequence of unit lower triangular matrices L_i and elementary permutation

matrices P_i. This gives us

$$L_3P_3L_2P_2L_1P_1A = U. \tag{22.4}$$

The elementary matrices L_i are of the form $L_i = I_n - \ell_i e_i^T$, where the ℓ_i vectors contain the multipliers:

$$\ell_1 = \begin{bmatrix} 0 \\ \ell_{21} \\ \ell_{31} \\ \ell_{41} \end{bmatrix} = \begin{bmatrix} 0 \\ 1/2 \\ -1/2 \\ 1 \end{bmatrix}, \quad \ell_2 = \begin{bmatrix} 0 \\ 0 \\ \ell_{32} \\ \ell_{42} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -1/5 \\ 4/5 \end{bmatrix}, \quad \text{and} \quad \ell_3 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \ell_{43} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -1/9 \end{bmatrix}.$$

Since each elementary permutation matrix P_i is its own inverse, i.e., $P_iP_i = I_n$, we can rewrite the expression (22.4) as

$$\begin{aligned} U &= L_3P_3L_2P_2L_1P_1A \\ &= L_3(P_3L_2P_3)(P_3P_2L_1P_2P_3)(P_3P_2P_1)A \\ &= \widehat{L}_3\widehat{L}_2\widehat{L}_1PA. \end{aligned}$$

Notice that $P = P_3P_2P_1$ is a permutation matrix since it is the product of permutation matrices. In fact, we have

$$P = P_3P_2P_1 = \begin{bmatrix} 0\,1\,0\,0 \\ 0\,0\,1\,0 \\ 0\,0\,0\,1 \\ 1\,0\,0\,0 \end{bmatrix} \begin{bmatrix} 1\,0\,0\,0 \\ 0\,0\,1\,0 \\ 0\,1\,0\,0 \\ 0\,0\,0\,1 \end{bmatrix} \begin{bmatrix} 0\,1\,0\,0 \\ 1\,0\,0\,0 \\ 0\,0\,1\,0 \\ 0\,0\,0\,1 \end{bmatrix} = \begin{bmatrix} 0\,1\,0\,0 \\ 0\,0\,1\,0 \\ 0\,0\,0\,1 \\ 1\,0\,0\,0 \end{bmatrix}.$$

Now we have a key observation about the $\widehat{L}_i$ matrices. Let us begin with $\widehat{L}_1$. This matrix is given by

$$\begin{aligned} \widehat{L}_1 =& P_3P_2L_1P_2P_3 = P_3P_2(I_n - \ell_1 e_1^T)P_2P_3 = \\ & I_n - (P_3P_2\ell_1)(e_1^TP_2P_3). \end{aligned}$$

The permutation matrices P_2 and P_3 only affect entries 2, 3 and 4 of the vector e_1^T, but since the only nonzero entry of e_1^T is the first, we get $e_1^TP_2P_3 = e_1^T$, and thus

$$\widehat{L}_1 = I_n - (P_3P_2\ell_1)e_1^T = I_n - \widehat{\ell}_1 e_1^T,$$

where

$$\widehat{\ell}_1 = P_3P_2\ell_1 = \begin{bmatrix} 1\,0\,0\,0 \\ 0\,0\,1\,0 \\ 0\,0\,0\,1 \\ 0\,1\,0\,0 \end{bmatrix} \begin{bmatrix} 0 \\ \ell_{21} \\ \ell_{31} \\ \ell_{41} \end{bmatrix} = \begin{bmatrix} 0 \\ \ell_{31} \\ \ell_{41} \\ \ell_{21} \end{bmatrix} = \begin{bmatrix} 0 \\ -1/2 \\ 1 \\ 1/2 \end{bmatrix}.$$

In conclusion, the matrix $\widehat{L}_1$ is also a core unit lower triangular matrix, just like L_1, but with the multipliers rearranged.

Similarly, we find that the matrices $\widehat{L}_2$ and $\widehat{L}_3$ are given by

$$\widehat{L}_2 = P_3(I_n - \ell_2 e_2^T)P_3 = I_n - (P_3\ell_2)(e_2^T P_3) = I_n - \widehat{\ell}_2 e_2^T, \quad \text{and}$$
$$\widehat{L}_3 = I_n - \ell_3 e_3^T = I_n - \widehat{\ell}_3 e_3^T,$$

where

$$\widehat{\ell}_2 = P_3\ell_2 = \begin{bmatrix} 1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&1&0 \end{bmatrix}\begin{bmatrix} 0\\0\\\ell_{32}\\\ell_{42} \end{bmatrix} = \begin{bmatrix} 0\\0\\\ell_{42}\\\ell_{32} \end{bmatrix} = \begin{bmatrix} 0\\0\\4/5\\-1/5 \end{bmatrix}, \quad \text{and}$$
$$\widehat{\ell}_3 = \ell_3 = \begin{bmatrix} 0\\0\\0\\\ell_{43} \end{bmatrix} = \begin{bmatrix} 0\\0\\0\\-1/9 \end{bmatrix},$$

Both $\widehat{L}_2$ and $\widehat{L}_3$ are core unit lower triangular matrices.

Writing

$$L = (\widehat{L}_3\widehat{L}_2\widehat{L}_1)^{-1} = \widehat{L}_1^{-1}\widehat{L}_2^{-1}\widehat{L}_3^{-1} = \begin{bmatrix} 1&0&0&0\\\ell_{31}&1&0&0\\\ell_{41}&\ell_{42}&1&0\\\ell_{21}&\ell_{32}&\ell_{43}&1 \end{bmatrix} = \begin{bmatrix} 1&0&0&0\\-1/2&1&0&0\\1&4/5&1&0\\1/2&-1/5&-1/9&1 \end{bmatrix},$$

we get $PA = LU$, where L is unit lower triangular and U is upper triangular. So, the result of Gaussian elimination with partial pivoting is an LU factorization, but not of A itself. Instead, it is an LU factorization of a row-permuted version of A:

$$\underbrace{\begin{bmatrix} 0&1&0&0\\0&0&1&0\\0&0&0&1\\1&0&0&0 \end{bmatrix}}_{P}\underbrace{\begin{bmatrix} 2&1&-1&2\\4&5&-3&6\\-2&5&-2&6\\4&11&-4&8 \end{bmatrix}}_{A} = \underbrace{\begin{bmatrix} 1&0&0&0\\-1/2&1&0&0\\1&4/5&1&0\\1/2&-1/5&-1/9&1 \end{bmatrix}}_{L}\underbrace{\begin{bmatrix} 4&5&-3&6\\0&15/2&-7/2&9\\0&0&9/5&-26/5\\0&0&0&2/9 \end{bmatrix}}_{U}.$$

Notice that U, as in the usual Gaussian elimination, is the final product of the process.

Since at each step we chose the pivot so that all the multipliers are less than or equal to 1 in absolute value, all entries of the factor L are bounded above by 1 in absolute value.

One final note. In this example, A is invertible, so it has a pivot in every column. If A were singular, some columns would not have pivots. That is fine. GEPP still works. We simply skip those columns and continue. The process will still transform A into an upper triangular matrix U, though U will be singular.

Theorem 22.23 formalizes our observations from Example 22.22: GEPP results in a PLU factorization of A, where all nonzero entries of L have an absolute value less than or equal to 1. This factorization exists for any square matrix, unlike the standard LU factorization.

Theorem 22.23. (Gaussian elimination with partial pivoting factors $PA = LU$)

For any $n \times n$ matrix A, Gaussian elimination with partial pivoting produces a PLU factorization, where L is unit lower triangular with all entries bounded by 1 in absolute value. Thus, every square matrix has a PLU factorization.

Proof By Theorem 16.40, we can row-reduce any square matrix A to an upper triangular matrix U. We can achieve this through a series of row interchanges and replacements, in particular, by applying GEPP.

GEPP multiplies A on the left by a sequence of core unit lower triangular matrices and elementary permutation matrices:

$$L_{n-1}P_{n-1}L_{n-2}P_{n-2}\cdots L_2P_2L_1P_1A = U, \tag{22.5}$$

where $L_k = I_n - \ell_k e_k^T$, with

$$\ell_k = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ \ell_{k+1,k} \\ \vdots \\ \ell_{nk} \end{bmatrix},$$

and $\ell_{k+1,k}, \cdots, \ell_{nk}$ are the multipliers at step k. The permutation P_k swaps rows k and r, for some $r \geqslant k$. Note that P_k or L_k could be the identity matrix[3].

Since each elementary permutation matrix P_k is its own inverse (that is, $P_kP_k = I_n$), we can "push" all the permutation matrices to the right and rewrite (22.5) as:

$$\widehat{L}_{n-1}\widehat{L}_{n-2}\cdots\widehat{L}_2\widehat{L}_1(P_{n-1}P_{n-2}\cdots P_2P_1A) = U,$$

where $\widehat{L}_k = P_{n-1}\cdots P_{k+1}L_kP_{k+1}\cdots P_{n-1}$. Then, we have

$$\begin{aligned}\widehat{L}_k &= P_{n-1}\cdots P_{k+1}(I_n - \ell_k e_k^T)P_{k+1}\cdots P_{n-1} \\ &= I_n - (P_{n-1}\cdots P_{k+1}\ell_k)(e_k^TP_{k+1}\cdots P_{n-1}) \\ &= I_n - P_{n-1}\cdots P_{k+1}\ell_k e_k^T \\ &= I_n - \widehat{\ell}_k e_k^T.\end{aligned}$$

The equality $e_k^TP_{k+1}\cdots P_{n-1} = e_k^T$ holds because the permutation matrices $P_{k+1}, \ldots, P_{n-1}$ only affect the entries in positions $k+1, \ldots, n$ of the vector e_k, which are all zero. Thus, $\widehat{L}_k$ is a core unit lower triangular matrix, identical to L_k except with rearranged multipliers.

Writing $L = (\widehat{L}_{n-1}\cdots\widehat{L}_1)^{-1}$ and $P = P_{n-1}\cdots P_1$, we have

$$PA = LU.$$

[3] If no row swap is needed, P_k is the identity matrix. Similarly, if the entries below the pivot are already zero, L_k will be the identity matrix.

The matrix $L = \widehat{L}_1^{-1} \cdots \widehat{L}_{n-1}^{-1}$ is unit lower triangular, as each $\widehat{L}_k$ is a unit lower triangular matrix (by Theorem 22.4), with the multipliers as its subdiagonal entries. Finally, since GEPP chooses pivots so that $|\ell_{ij}| \leqslant 1$, all entries of L are bounded by 1 in absolute value. □

22.3.3 Solving linear systems by using the *PLU* factorization

Just like with the LU factorization, once we have the factorization $PA = LU$, solving linear systems of equations becomes straightforward.

Let us consider a consistent linear system $Ax = b$, and let $PA = LU$ be a PLU factorization of A. Since permutation matrices are invertible, the linear systems

$$Ax = b \quad \text{and} \quad PAx = Pb$$

have the same solution set.

We can rewrite $PAx = Pb$ as $L(Ux) = Pb$. Setting $y = Ux$, solving the linear system $Ax = b$ is equivalent to the following three steps:

$$\begin{aligned} \widetilde{b} &= Pb && \text{permute the vector } b \\ Ly &= \widetilde{b} && \text{solve a lower triangular system} \\ Ux &= y && \text{solve an upper triangular system.} \end{aligned}$$

The matrix L is always invertible because it has ones on the main diagonal, so the system $Ly = \widetilde{b}$ has a unique solution, solved by forward substitution (22.2). If A is invertible, U is also invertible, and we solve $Ux = y$ by backward substitution (22.3).

Example 22.24 Consider the linear system $Ax = b$, where

$$A = \begin{bmatrix} 2 & 1 & -1 & 2 \\ 4 & 5 & -3 & 6 \\ -2 & 5 & -2 & 6 \\ 4 & 11 & -4 & 8 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 4 \\ 12 \\ 7 \\ 19 \end{bmatrix}.$$

In Example 22.22, we found a PLU factorization of A, which we will now use to solve this system.

The first step is to solve $Ly = Pb$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -1/2 & 1 & 0 & 0 \\ 1 & 4/5 & 1 & 0 \\ 1/2 & -1/5 & -1/9 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 12 \\ 7 \\ 19 \\ 4 \end{bmatrix}.$$

Using forward substitution, we find

$$y = \begin{bmatrix} 12 \\ 13 \\ -17/5 \\ 10/45 \end{bmatrix}.$$

Next, we solve the system $Ux = y$:

$$\begin{bmatrix} 4 & 5 & -3 & 6 \\ 0 & 15/2 & -7/2 & 9 \\ 0 & 0 & 9/5 & -26/5 \\ 0 & 0 & 0 & 2/9 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 12 \\ 13 \\ -17/5 \\ 10/45 \end{bmatrix}.$$

Using backward substitution, we get

$$x = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

Problems

22.1 Show that the product of lower (resp. upper) triangular matrices is lower (resp. upper) triangular. In particular, if the matrices are unit lower (resp. upper) matrices (i.e., the main diagonal entries are all ones), then the product is also a unit lower (resp. upper) matrix.

22.2 Consider the matrix

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 4 & 2 \\ 6 & 3 & 5 \end{bmatrix}.$$

Find a lower triangular matrix E that transforms A into an upper triangular matrix U (use what you know about elementary matrices). Thus, $EA = U$. Use this to find an LU factorization of A.

22.3 Find the LU factorization of the following matrices if possible:

$$A = \begin{bmatrix} 2 & 1 \\ 6 & 7 \end{bmatrix}, \qquad \text{and} \qquad B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

22.4 When zero appears in a pivot position, the LU factorization of a matrix A is not possible. Show without doing any row operations why the LU factorization of the following matrices does not exist.

$$A = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} \qquad \text{and} \qquad B = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

22.5 Provide examples, if possible, for the following cases:

(a) A 2×2 singular matrix A where $A_{11} \neq 0$ and A has an LU factorization.
(b) A 2×2 singular matrix A where $A_{11} \neq 0$ but A does not have an LU factorization.

(c) A 3×3 matrix A whose 2×2 leading principal submatrix is singular, but A still has an LU factorization.

22.6 Consider a matrix A with an LU factorization, $A = LU$. Do either L or U, or both, contain a basis for any of the fundamental subspaces of A? Explain your reasoning. What changes, if anything, if $PA = LU$ for some permutation matrix P?

22.7 Solve the following linear system of equations by finding the LU factorization of the matrix of coefficients (if it exists) and then solving two triangular systems of equations:

$$\begin{aligned} x_1 - 2x_2 + \ x_3 &= \ 0 \\ 2x_2 - 8x_3 &= \ 8 \\ -4x_1 + 5x_2 + 9x_3 &= -9 \end{aligned}$$

22.8 Find the equation of a parabola $y = ax^2 + bx + c$ that passes through the points $(1, 6)$, $(2, 4)$ and $(3, 0)$. Solve the corresponding linear system of equations using an LU factorization.

22.9 Use the LU factorization of the matrix

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 1 & 3 & 2 \\ 0 & 1 & 2 \end{bmatrix}$$

to find its inverse by solving three systems of equations.

22.10 Write a program in Python (or your preferred programming language) to check if a matrix has an LU factorization. If it does, compute and display the factorization.

22.11 Explain why if a matrix has an LU factorization, it can be obtained without using elementary row operations of type I (multiplication of a row by a nonzero entry).

22.12 Find the LU and LDL^T factorizations of the tridiagonal matrix

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 3 \end{bmatrix}.$$

22.13 You might have experienced how easy it was to make small mistakes when computing inverses by hand, especially when fractions were involved. That is why teachers often give matrices with integer entries whose inverses also have integer entries. In this problem, we will reveal the secret method behind finding such matrices.

(a) If A is a matrix with integer entries and all its Gaussian elimination pivots are 1, explain why the inverse of A must also contain only integer entries.

(b) You can use part (a) to generate random integer matrices with integer inverses by randomly creating integer lower and upper triangular matrices L and U with ones on their diagonals, and then computing the product $A = LU$. Write a function (in Python or your preferred programming language) that generates random integer matrices with integer inverses.

22.14 If P is the $n \times n$ elementary permutation matrix that swaps rows i and j, show that $P = I_n - uu^T$ with $u = e_i - e_j$, where e_i denotes the ith column of the identity matrix. Use this fact to establish that $P^2 = I_n$.

22.15 Let A be a singular square matrix. Give examples (different from the one in the chapter) of:

(a) A singular square matrix that has an LU factorization without pivoting.
(b) A singular square matrix that does not have an LU factorization without row exchanges. Explain why a zero pivot prevents the factorization.

22.16 Find the PLU factorization (LU factorization with partial pivoting) of the matrix

$$A = \begin{bmatrix} 0 & 2 & 4 \\ 0 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix}$$

and use it to describe the complete solution set of the linear system

$$\begin{bmatrix} 0 & 2 & 4 \\ 0 & 1 & 2 \\ 3 & 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}.$$

22.17 Let A be an invertible $n \times n$ matrix. Count the number of arithmetic operations required to:

(a) Solve the system $Ax = b$ using Gaussian elimination.
(b) Compute the LU factorization of A.
(c) Solve $Ax = b$ using the LU factorization by first solving $Ly = b$ and then $Ux = y$.

22.18 Let A be an invertible $n \times n$ matrix. Suppose you need to solve k linear systems with the same matrix A but different right-hand-side vectors b. Compare the total number of arithmetic operations required by:

(a) Solving each system separately using Gaussian elimination.
(b) Computing the LU factorization of A once, then solving each system using forward and backward substitution.

22.19 Consider an invertible $n \times n$ matrix A. Compare the number of operations required to compute the inverse of A using the following two methods:

(a) Gauss-Jordan elimination: Use elementary row operations to reduce the augmented matrix $\left[\, A \middle| I_n \,\right]$ to $\left[\, I_n \middle| A^{-1} \,\right]$.
(b) First factor A as $PA = LU$, then solve n linear systems $Ax_i = e_i$, where e_i is the i-th column of the identity matrix.

22.20 An $n \times n$ matrix A is said to be strictly diagonally dominant if for $i = 1, \ldots, n$,

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}|.$$

Prove that A has an LU factorization.

22.21 (Nodal analysis and multiple load cases) In circuit design, engineers often analyze the steady-state voltages in a purely resistive network using a method called *nodal analysis*. The setup leads to a linear system

$$Gv = i, \qquad v \in \mathbb{R}^n,$$

where the matrix $G \in \mathbb{R}^{n \times n}$ is symmetric and represents how the components are connected (it is called the *conductance matrix*), and the vector $i \in \mathbb{R}^n$ contains the net currents injected at each node. The unknowns are the node voltages v.

In practice, the circuit topology, and hence the matrix G, is fixed, but engineers want to test multiple *load cases* (different current patterns) to ensure the design works under many scenarios. Rather than solving each system from scratch, it is much more efficient to factor the matrix once as $G = LU$ and then re-use the factors to solve $Gv^{(k)} = i^{(k)}$ for each new load case.

Consider a five-node electrical network that models a small on-board power distribution grid in a sensor package. The conductance matrix for this network is

$$G = \begin{bmatrix} 3 & -1 & 0 & 0 & 0 \\ -1 & 4 & -1 & -1 & 0 \\ 0 & -1 & 3 & 0 & -1 \\ 0 & -1 & 0 & 2 & -1 \\ 0 & 0 & -1 & -1 & 3 \end{bmatrix}.$$

Engineers want to compute the node voltages for three operating conditions, each corresponding to a different pattern of injected currents:

$$\begin{aligned} i^{(1)} &= [\,2,\ 0,\ 0,\ 0,\ 0\,]^T && \text{(2 A injected at node 1),} \\ i^{(2)} &= [\,0,\ 1,\ -1,\ 0,\ 0\,]^T && \text{(1 A from node 2 to node 3),} \\ i^{(3)} &= [\,0,\ 0,\ 0,\ 2,\ -2\,]^T && \text{(2 A from node 4 to node 5).} \end{aligned}$$

(a) Compute the LU factorization $G = LU$ of the matrix G. (You may assume that pivoting is not required.)

(b) Using the same L and U, solve for the voltage vectors $v^{(1)}, v^{(2)}, v^{(3)}$ corresponding to the three current injection patterns.

(c) Estimate the number of floating-point operations (flops) required to:

- apply Gaussian elimination from scratch three times (once for each right-hand side),
- compute a single LU factorization and then solve the three systems using forward and backward substitution.

Which method is more efficient, and by approximately what factor?

Chapter 23
V-SUPPLEMENTARY TOPICS

In Chapter 22, we learned that every $n \times n$ matrix has a PLU factorization. From an application's perspective, this result is exactly what we need to solve linear systems, so we could stop here and be satisfied. But math has its own fun, and sometimes we just want to explore more deeply!

Let us go back to the LU factorization itself. We have seen that not every matrix has an LU factorization. So, which matrices actually do have an LU factorization? And, if an LU factorization exists, when is it unique? Also, can rectangular matrices have an LU factorization?

In this section, we will answer these questions with a series of results from three papers [18, 36, 42]. We will not go through the proofs here–they are a bit technical–but if you are curious, take a look at those papers.

23.1 Existence and uniqueness of LU factorizations

In Definition 22.1, we introduced LU factorization for square matrices. This definition extends easily to rectangular matrices.

Definition 23.1. (LU factorization for rectangular matrices)

For an $m \times n$ matrix A, we call $A = LU$ an **LU factorization** of A if $L \in \mathbb{R}^{m\times m}$ is unit lower triangular, and $U \in \mathbb{R}^{m\times n}$ is upper triangular ($U_{ij} = 0$ for $i > j$).

Notice that L is always square, and U has the same shape as A.

Our first result identifies which matrices, square or rectangular, have an LU factorization. The result depends on row spaces, and we will use the notation $\text{row}(A)$ for the row space of A, rather than $\text{col}(A^T)$, as we have used previously.

Theorem 23.2. (Existence of LU factorizations - the general case)

A matrix $A \in \mathbb{R}^{m\times n}$ has an LU factorization if and only if

$$A[i, 1:k] \in \text{row}\left(A[1:k, 1:k]\right), \tag{23.1}$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_23

for $k = 1, \ldots, \min\{m, n\}$ and $i = k + 1, \ldots, m$.

Example 23.3 Consider the rectangular matrix

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 2 & -1 \\ 0 & 3 & -1 \end{bmatrix}.$$

It is easy to verify that A satisfies the conditions of Theorem 23.2, so A has an LU factorization. One possible LU factorization is

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 2 & -1 \\ 0 & 3 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 3 & 4/3 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & -3 \\ 0 & 0 & 0 \end{bmatrix}.$$

From Theorem 23.2, we can derive some easy to check sufficient conditions for an LU factorization to exist.

First, observe that our existence theorem for square $n \times n$ matrices (Theorem 22.10) follows directly from Theorem 23.2. A nonsingular leading principal submatrix $A[1 : k, 1 : k]$ implies $\text{row}(A[1 : k, 1 : k]) = \mathbb{R}^k$, so $A[i, 1 : k] \in \text{row}(A[1 : k, 1 : k])$ for $i = k + 1, \ldots, n$. Moreover, when $k = n$, condition (23.1) is empty, so there is nothing to check, and we can omit the nonsingularity requirement.

Corollary 23.4. (Sufficient conditions for LU factorization of a square matrix)

A square matrix $A \in \mathbb{R}^{n \times n}$ has an LU factorization if its first $n - 1$ leading principal submatrices are nonsingular.

If an $n \times n$ matrix A has $\text{rank}(A) = r < n$, then its leading principal submatrices of orders $r + 1, r + 2, \ldots, n$ are all singular, so Theorem 22.10 does not apply. However, we can extend Theorem 22.10 to cover certain cases of square singular matrices. The following result is another consequence of Theorem 23.2.

Corollary 23.5. (More sufficient conditions for an LU factorization)

A square matrix $A \in \mathbb{R}^{n \times n}$ of rank r has an LU factorization if its first r leading principal submatrices are nonsingular.

Example 23.6 Consider the matrix

$$A = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 2 & 1 & -1 & 0 \\ 3 & 1 & 0 & 2 \\ 2 & 0 & 2 & 4 \end{bmatrix}.$$

This matrix has rank 2 because the first two rows are linearly independent, while rows 3 and 4 are linear combinations of rows 1 and 2. The matrix A satisfies the conditions of Corollary 23.5 since its first two leading principal submatrices are nonsingular. Therefore, it has an LU factorization. One possible LU factorization is

$$A = \begin{bmatrix} 1 & 0 & 1 & 2 \\ 2 & 1 & -1 & 0 \\ 3 & 1 & 0 & 2 \\ 2 & 0 & 2 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 3 & 1 & 1 & 0 \\ 2 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & -3 & -4 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The conditions in Corollary 23.5 are straightforward to verify, at least for small matrices. However, the converse does not hold, as shown in the following example. Some matrices satisfy the conditions of Theorem 23.2 and thus have an LU factorization, but they do not satisfy the conditions of Corollary 23.5.

Example 23.7 Consider the rank-2 matrices

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 7 \\ 0 & 0 & -1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 2 \end{bmatrix}.$$

Neither matrix satisfies the conditions of Corollary 23.5. For A, the second leading principal submatrix is singular, and for B, both the first and second leading principal submatrices are singular. However, both matrices do have an LU factorization, as they satisfy the conditions of Theorem 23.2. Possible LU factorizations of A and B are, respectively,

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 7 \\ 0 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad \text{and}$$

$$B = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -2 & 2 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{bmatrix}.$$

Now, let us consider the question of uniqueness: which matrices have a unique LU factorization? To start, here is a simple example showing that if a matrix has an LU factorization, it may not be unique.

Example 23.8 Consider the matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

This matrix satisfies the conditions of Theorem 23.2, so it has an LU factorization. In fact, it has infinitely many! For any scalar x, we can write

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & x & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Notice that a nonsingular matrix with all its leading principal submatrices nonsingular not only has an LU factorization (Theorem 22.10) but this factorization is unique (Theorem 22.11).

If we drop the nonsingularity condition, we know that a square matrix with its first $n-1$ leading principal submatrices nonsingular also has an LU factorization (Corollary 23.4). But is this factorization unique, as it is in the nonsingular case? Let us look at an example, first.

Example 23.9 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 0 & 1 \\ 2 & -1 & 1 \end{bmatrix},$$

which is singular but has nonsingular first and second leading principal submatrices. Does this matrix have a unique LU factorization?

Suppose we write

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 0 & 1 \\ 2 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}.$$

The first row of U must equal the first row of A, so we get

$$u_{11} = 1, \quad u_{12} = 2, \quad \text{and} \quad u_{13} = 3.$$

For the second row of A, we have $1 = \ell_{21}u_{11}$, which uniquely determines $\ell_{21} = 1$. Continuing the matrix multiplication, we find that $u_{22} = -2$ and $u_{23} = -2$. Finally, from the third row of A, we solve for $\ell_{31} = 2$, $\ell_{32} = \frac{5}{2}$, and $u_{33} = 0$.

Since all entries in L and U are fully determined by the entries in A, we have no freedom in choosing values. Thus, the LU factorization is unique!

The previous example illustrates the following result.

Theorem 23.10. (Uniqueness of the LU factorization of a square matrix)

An $n \times n$ singular matrix A has a unique LU factorization if and only if its first $n-1$ leading principal submatrices are nonsingular.

What if A is singular and at least one of its first $n-1$ leading principal submatrices is also singular? And what about rectangular matrices? Which matrices have a unique LU factorization? The following theorem gives the conditions for a matrix to have a unique LU factorization.

Theorem 23.11. (Uniqueness of the LU factorization - general case)

Let $A \in \mathbb{R}^{m \times n}$. If $m \leqslant n+1$, A has a unique LU factorization if and only if the first $m-1$ leading principal submatrices are nonsingular. If $m > n+1$, an LU factorization of A cannot be unique.

Our previous uniqueness theorem (Theorem 22.11) actually follows from Theorems 23.2 and 23.11, though this might not be immediately obvious.

In all our work on LU factorizations so far, we have required L to be unit lower triangular. But what happens if we relax this condition and only ask L to be lower triangular?

The results we will discuss apply only to square matrices, so we will focus on this case here.

Let us start with a definition.

Definition 23.12. (Generalized LU factorization)

For an $n \times n$ matrix A, we call $A = LU$ a **generalized LU factorization** if $L \in \mathbb{R}^{n \times n}$ is lower triangular, and $U \in \mathbb{R}^{n \times n}$ is upper triangular.

The only difference between an LU factorization and a generalized LU factorization is that in the generalized version, the factor L does not need to have ones on its main diagonal.

The following theorem provides necessary and sufficient conditions for any square matrix to have a generalized LU factorization.

Theorem 23.13. (Existence of generalized LU factorizations)

A matrix $A \in \mathbb{R}^{n\times n}$ has a generalized LU factorization if and only if

$$\text{rank}\,(A[1:k,1:k]) \geqslant \text{rank}\,(A[1:k,1:n]) + \text{rank}\,(A[1:n,1:k]) - k,$$

for each $k = 1, 2, \ldots, n$.

The conditions in Theorem 23.2 automatically satisfy those in Theorem 23.13, since we have $\text{rank}(A[1:m,1:k]) = \text{rank}(A[1:k,1:k])$ and $\text{rank}(A[1:k,1:n]) \leqslant k$. This result is expected, as the generalized LU factorization includes the standard LU factorization as a special case.

Example 23.14 Consider the matrix

$$A = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 2 \\ 2 & -1 & 1 & 2 \end{bmatrix}.$$

This matrix satisfies the conditions of Theorem 23.2 for $k = 1, 2$, but fails for $k = 3$. Therefore, it does not have an LU factorization with L as a unit lower triangular matrix. However, it does satisfy the conditions in Theorem 23.13, so a generalized LU factorization exists. One possible factorization is

$$A = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 2 \\ 2 & -1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 2 & -1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Problems

23.1 For each matrix below, determine whether it has

(a) A unique LU factorization. If so, find the factorization.
(b) Multiple LU factorizations. If so, provide at least two factorizations.
(c) No LU factorization at all. If so, determine if the matrix has a generalized LU factorization and, if it does, find one.

$$\begin{bmatrix} 4 & 2 & 3 \\ 6 & 3 & 6 \\ 5 & 7 & 9 \end{bmatrix}, \quad \begin{bmatrix} 0 & 2 & 3 \\ 0 & 4 & 6 \\ 0 & 6 & 9 \end{bmatrix}, \quad \begin{bmatrix} 20 & 5 & 9 \\ 16 & 4 & 7 \\ 4 & 1 & 3 \end{bmatrix}, \quad \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad \begin{bmatrix} 1 & 5 & 1 \\ 1 & 4 & 2 \\ 4 & 10 & 2 \end{bmatrix}, \quad \begin{bmatrix} 8 & 4 & 1 \\ 4 & 2 & 1 \\ 4 & 2 & 1 \end{bmatrix}.$$

23.2 (LU Factorization for a rectangular full-rank matrix) Let A be an $m \times n$ matrix with $m > n$ and $\text{rank}(A) = n$. Determine when A can be factored as

$$A = LU,$$

where L is an $m \times m$ lower triangular matrix (with ones on its main diagonal) and U is an $m \times n$ upper triangular matrix. Illustrate your answer with an explicit example where $m = 4$ and $n = 3$.

How about if L is an $m \times n$ lower triangular matrix and U is an $n \times n$ upper triangular matrix with ones in the main diagonal?

23.3 Let A be an $m \times n$ matrix with $m > n$ and $\text{rank}(A) < n$. Can A have an LU factorization of the form $A = LU$, where L is an $m \times m$ lower triangular matrix and U is an $m \times n$ upper triangular matrix? Explain your reasoning and give an example, if possible.

23.4 We have discussed that a matrix can sometimes have multiple LU factorizations. Provide an example of a rectangular (not square) matrix A that admits more than one LU factorization.

23.5 Provide an example of a singular matrix where one or more leading principal minors are zero, and comment on the existence of its LU factorization.

Chapter 24
V-FROM THE CLASSROOM TO REAL LIFE

24.1 Graph Laplacian linear systems in Data Science

One of the main ways linear systems appear in modern data analysis is through graph-based methods. Many datasets can be described as graphs: users connected by friendships, documents linked by citations, or pixels connected by similarity. In these situations, a special matrix called the *graph Laplacian* plays a central role. In this section, we will define this matrix and see how it is used.

Solving linear systems that involve the Laplacian is a basic step in many algorithms. These applications are everywhere: recommendation and ranking problems, bioinformatics for studying protein similarities, and image processing tasks such as filling in missing data or classifying images. More recently, Laplacians have also been combined with deep learning methods, for example in graph convolutional networks, which are now widely used in data science.

Among the most elegant of these applications is *label propagation*, where we use the structure of the graph to spread a small set of known labels across all the nodes. This gives us a way to make predictions for the unlabeled data by solving a linear system tied to the graph Laplacian.

24.1.1 The graph Laplacian matrix

This section uses graph concepts from Chapter 3. You may want to review that chapter, especially the definition of the adjacency matrix (Definition 3.6).

Suppose a dataset is represented as an undirected weighted graph $G = (V, E, w)$ with n vertices: the set V contains the nodes (each node is a data point such as a user, an image, or a document); the set E contains the edges (edges link nodes that are related in some way); and the weights w measure how strong or similar those relationships are. If two nodes are not connected, we can think of the edge weight as zero. For unweighted graphs, every edge has weight one.

We now introduce the *(graph) Laplacian matrix*.

Definition 24.1. (Degree and graph Laplacian matrices)

Let G be an undirected weighted graph.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_24

(a) The **degree of a vertex** i, written d_i, is the sum of the weights of all edges connected to i:

$$d_i := \sum_{j=1}^{n} w_{ij}, \quad i = 1, \ldots, n.$$

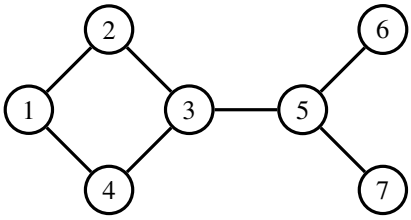
The **degree matrix** D is the diagonal matrix with entries $d_1, \ldots, d_n$.

(b) The **graph Laplacian matrix** L is defined as

$$L := D - A,$$

where A is the adjacency matrix of G.

Example 24.2 Consider the following undirected graph.



Since the graph is unweighted, we assume that every edge has a weight of 1. The degree matrix D, adjacency matrix A, and graph Laplacian matrix $L = D - A$ for this graph are, respectively,

$$D = \begin{bmatrix} 2&0&0&0&0&0&0 \\ 0&2&0&0&0&0&0 \\ 0&0&3&0&0&0&0 \\ 0&0&0&2&0&0&0 \\ 0&0&0&0&3&0&0 \\ 0&0&0&0&0&1&0 \\ 0&0&0&0&0&0&1 \end{bmatrix} \quad A = \begin{bmatrix} 0&1&0&1&0&0&0 \\ 1&0&1&0&0&0&0 \\ 0&1&0&1&1&0&0 \\ 1&0&1&0&0&0&0 \\ 0&0&1&0&0&1&1 \\ 0&0&0&0&1&0&0 \\ 0&0&0&0&1&0&0 \end{bmatrix}, \quad \text{and} \quad L = \begin{bmatrix} 2&-1&0&-1&0&0&0 \\ -1&2&-1&0&0&0&0 \\ 0&-1&3&-1&-1&0&0 \\ -1&0&-1&2&0&0&0 \\ 0&0&-1&0&3&-1&-1 \\ 0&0&0&0&-1&1&0 \\ 0&0&0&0&-1&0&1 \end{bmatrix}.$$

The Laplacian matrix, which we will study in detail in Section 42.3.1, has a very useful property: it measures how values vary across a graph. To see this, take a vector $x \in \mathbb{R}^n$, where each entry x_i is the value at node i. For example, x_i could be the rating that user i gives to a movie, or the temperature recorded by sensor i. Then, we have

$$x^\top L x \;=\; \frac{1}{2} \sum_{i,j=1}^{n} w_{ij}(x_i - x_j)^2.$$

This expression tells us how "smooth" or "rough" the values are across the edges. If two nodes are strongly connected (large w_{ij}) but their values x_i and x_j differ a

lot, the contribution to the sum is large. If connected nodes have similar values, the contribution is small.

So, a small value of $x^T Lx$ means the function defined by x varies smoothly along the graph. In other words, connected nodes tend to have similar values. This smoothness property is the reason the Laplacian shows up in many data analysis problems, and we will soon see concrete examples of how it is used.

24.1.2 The label propagation problem

We now return to the problem introduced in the motivation: some nodes of a graph come with labels, and many others do not. The goal is to assign labels to the unlabeled nodes in a way that is consistent with the known ones and varies smoothly across the graph.

Formally, let $G = (V, E, w)$ be an undirected weighted graph with n nodes. We divide the nodes into two groups. The set L contains the labeled nodes, where each node $i \in L$ has a fixed label value y_i. The set U contains the unlabeled nodes, for which we want to infer labels. We represent all labels with a vector $f \in \mathbb{R}^n$, where $f_i = y_i$ if $i \in L$, and f_i is unknown if $i \in U$.

The principle is that the function f should be smooth across the graph. A natural way to capture this is to minimize the Laplacian quadratic form $f^T Lf$. This leads to the minimization problem

$$\min_f \; f^T Lf = \min_f \frac{1}{2} \sum_{i,j=1}^{n} w_{ij}(f_i - f_j)^2 \quad \text{subject to } f_i = y_i \;\; \text{for all } i \in L. \tag{24.1}$$

To solve this minimization problem, it helps to reorder the nodes so that the labeled ones come first and the unlabeled ones come second. With this ordering, we can write the vector f in block form as

$$f = \begin{bmatrix} f_L \\ f_U \end{bmatrix},$$

where f_L contains the fixed labels and f_U contains the unknown ones. The Laplacian matrix can be written in a matching block form:

$$L = \begin{bmatrix} L_{LL} & L_{LU} \\ L_{UL} & L_{UU} \end{bmatrix}.$$

Substituting the block structure of f and L into the quadratic form gives

$$\begin{aligned} f^T Lf &= \begin{bmatrix} f_L^T & f_U^T \end{bmatrix} \begin{bmatrix} L_{LL} & L_{LU} \\ L_{UL} & L_{UU} \end{bmatrix} \begin{bmatrix} f_L \\ f_U \end{bmatrix} \\ &= f_L^T L_{LL} f_L + f_L^T L_{LU} f_U + f_U^T L_{UL} f_L + f_U^T L_{UU} f_U \\ &= f_L^T L_{LL} f_L + 2 f_U^T L_{UL} f_L + f_U^T L_{UU} f_U. \end{aligned}$$

where we have used that L is symmetric, so $L_{UL} = L_{LU}^T$, and the middle two terms combine.

Notice that the first term, $f_L^T L_{LL} f_L$, depends only on the fixed labels f_L. Since those labels are given and do not change, this term is a constant. Constants do not influence the location of the minimizer, so we can safely ignore it when solving the problem. The problem now reduces to minimizing

$$\Phi(f_U) = f_U^T L_{UU} f_U + 2 f_U^T L_{UL} f_L.$$

As you (hopefully) remember from Multivariable Calculus, we find the minimizer of a function by setting its gradient to zero. In our case, the gradient with respect to f_U is (see Problem 24.1)

$$\nabla_{f_U} \Phi = 2 L_{UU} f_U + 2 L_{UL} f_L.$$

Setting this equal to zero gives the optimality condition

$$L_{UU} f_U = -L_{UL} f_L.$$

This linear system determines the unknown labels f_U on the unlabeled nodes. The matrix L_{UU} is the part of the Laplacian that reflects the connections among the unlabeled nodes, while the vector $L_{UL} f_L$ captures the influence of the labeled nodes on their unlabeled neighbors.

Now, how do we solve the linear system of equations $L_{UU} f_U = b$? The matrix L_{UU} has two important properties:

1. It is *symmetric*. Since the original graph is undirected ($w_{ij} = w_{ji}$), the full Laplacian L is symmetric, and so is its principal submatrix L_{UU}.
2. Under reasonable conditions (see Problem 42.60), it is *positive definite*, a property we will study in depth in Chapter 42. In particular, this implies it is invertible.

For a symmetric positive definite matrix, the standard LU decomposition can be replaced by a more efficient and numerically stable method called *Cholesky factorization*, which we will also study in Chapter 42. Instead of writing $A = LU$, Cholesky finds a unique upper triangular matrix R such that

$$A = R^T R.$$

This factorization is about twice as fast as LU and requires only half the storage, since we only need to compute and store R.

The solution of $L_{UU} f_U = b$ with Cholesky factorization is a two-step process:

1. **Factorize**. Compute the Cholesky factorization $L_{UU} = R^\top R$ (see Chapter 43 for details on the algorithm).
2. **Solve**. Use the factorization to solve the system in two triangular steps:
   - First solve $R^T z = b$ for z by *forward substitution*.
   - Then solve $R\, f_U = z$ for y_U by *backward substitution*.

Example 24.3. (Predicting movie preferences in a social network)

Imagine a small social network of five friends, where each edge represents a strong friendship tie. Alice (1) is friends with Bob (2) and Carol (3). Bob (2) is friends

with Alice (1), Carol (3), and David (4). Carol (3) is friends with Alice (1), Bob (2), and Eve (5). David (4) is friends only with Bob (2), and Eve (5) is friends only with Carol (3). Figure 24.1 illustrates this social network.

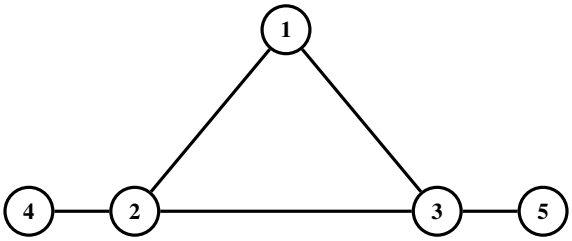


Fig. 24.1: Five friends in a social network. Alice (1) likes the movie and David (4) dislikes it; we want to predict what Bob (2), Carol (3), and Eve (5) might think.

Now suppose a few of them reveal whether they like a new movie. Alice (1) says "like" (+1), David (4) says "dislike" (−1). Using label propagation, we want to predict the preferences of the remaining friends: Bob, Carol, and Eve.

The adjacency and degree matrices of the social network are

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}, \quad \text{and} \quad D = \operatorname{diag}(2, 3, 3, 1, 1).$$

The Laplacian matrix is then

$$L = D - A = \begin{bmatrix} 2 & -1 & -1 & 0 & 0 \\ -1 & 3 & -1 & -1 & 0 \\ -1 & -1 & 3 & 0 & -1 \\ 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \end{bmatrix}.$$

We look for a vector $y \in \mathbb{R}^5$ that matches the given labels and stays smooth across the graph. This gives the minimization problem

$$\min_y \; y^T L y \quad \text{subject to } y_1 = +1, \; y_4 = -1.$$

Solving this is equivalent to working with the reduced Laplacian system for the unknown entries y_2, y_3, and y_5.

By separating the variables into knowns (y_1, y_4) and unknowns (y_2, y_3, y_5), we arrive at the linear system

$$\widetilde{L}\, u = b,$$

with

$$\widetilde{L} = \begin{bmatrix} 3 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 1 \end{bmatrix}, \qquad b = -L_{UL}\, y_L = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix},$$

where $u = [y_2, y_3, y_5]^T$.

The Cholesky factorization of the reduced Laplacian $\widetilde{L}$ is

$$\widetilde{L} = R^T R, \quad \text{where} \quad R = \begin{bmatrix} \sqrt{3} & -\sqrt{3}/3 & 0 \\ 0 & 2\sqrt{6}/3 & -\sqrt{6}/4 \\ 0 & 0 & \sqrt{10}/4 \end{bmatrix}.$$

Solving $R^T R u = b$ gives approximately

$$u \approx \begin{bmatrix} 0.2 \\ 0.6 \\ 0.6 \end{bmatrix}.$$

The predicted movie preferences are:

- Bob (2): +0.2 (leans toward not liking),
- Carol (3): +0.6 (leans towards liking), and
- Eve (5): +0.6 (leans toward liking).

Nodes close to Alice inherit her "like", but the effect is moderated by David's "dislike".

The example above was small, but in real applications the graphs can have millions of nodes. The same principle applies, but solving the corresponding linear system can be a real challenge.

There are two broad ways to solve a large linear system: *direct* and *iterative* methods. Iterative methods produce a sequence of approximations that (hopefully) converge to the true solution. Direct methods, which are variants of Gaussian elimination, produce an exact solution in a finite number of steps. For symmetric, positive definite matrices like L_{UU}, the standard direct approach is the *Cholesky factorization*. Direct methods can be expensive for very large graphs, but they are robust and effective when the set of unlabeled nodes is not too large.

At first glance, it may seem impossible to solve such huge systems directly. After all, an $n \times n$ Laplacian matrix with millions of nodes is enormous. The trick lies in a preprocessing step called *matrix reordering*.

Laplacian matrices are usually very *sparse*, meaning most of their entries are zero. But when we perform Cholesky factorization, the triangular factor R often becomes much denser than L. This creation of new nonzeros is called **fill-in**. Too much fill-in can use huge amounts of memory and slow the computation dramatically. The amount of fill-in depends on the order in which rows and columns are eliminated.

By reordering the nodes in the graph, we can reduce fill-in. Instead of solving $Lx = b$, we introduce a *permutation matrix* P and solve

$$(PLP^T)y = Pb, \quad \text{with } x = P^T y.$$

The matrix PLP^T is just a row and column permutation of L. The goal is to choose P so that the Cholesky factor of PLP^T stays sparse.

Two popular reordering strategies are:

1. **Approximate Minimum Degree (AMD)**. A greedy algorithm that always eliminates the node with the fewest neighbors first. By removing the "lonely" nodes early, it avoids creating too many new connections. AMD is fast and works well in many cases.
2. **Nested Dissection**. A divide-and-conquer method that finds a small set of nodes (a **separator**) whose removal splits the graph into two parts. The separator nodes are placed last in the ordering, which confines fill-in to within each part. This works especially well for graphs coming from grids, such as those in image processing.

In practice, high-quality sparse direct solvers automatically perform reordering (usually AMD or something similar) before computing a Cholesky factorization. This reordering step is what makes Cholesky practical for the large, sparse systems that often arise in data science.

Let us now see more situations where Laplacian systems naturally arise.

Example 24.4. (Interactive image segmentation)

This technique is widely used for separating a primary object (the foreground) from the rest of an image (the background) with very little user effort.

We begin by turning the image into a graph. Each pixel (i, j) becomes a node. Neighboring pixels are connected by edges, and the weight of an edge reflects how similar the two pixels are. A common choice is a Gaussian function of the color difference:

$$W_{uv} = \exp\left(-\frac{\|I_u - I_v\|^2}{2\sigma^2}\right),$$

where I_u and I_v are the colors of pixels u and v.

A user then provides a few scribbles: some pixels are marked as foreground (+1) and others as background (-1). These are the labeled nodes. Every other pixel is treated as unlabeled.

To assign labels to the unlabeled pixels, we use the label propagation method. The solution gives each unlabeled pixel a soft label, which can be interpreted as a confidence score. By applying a simple threshold, for example, classifying pixels with $y_i > 0$ as foreground, we obtain a segmentation of the image.

Example 24.5. (Classifying documents in a corpus)

This is a classic example of label propagation. Imagine an archive of millions of news articles, where only a handful are tagged with topics such as "Technology" or "Finance".

We begin by turning the collection into a graph. Each document becomes a node, and we represent documents as numerical vectors using a method like TF-IDF. The

weight of an edge between two documents is given by their cosine similarity, so that highly similar articles are strongly connected.

A small number of documents are manually labeled, for example, assigning +1 to "Technology" and -1 to "Not Technology". These are the labeled nodes. The vast majority of articles remain unlabeled.

To extend the labels, we use label propagation. Each unlabeled document then receives a score that reflects its likelihood of belonging to the "Technology" category. By ranking documents according to this score, the known labels spread naturally through the entire corpus.

Example 24.6. (Graph-based image denoising)

A common task in image processing is to remove noise from an image while keeping important features such as edges and textures. Graph-based methods give a natural way to do this by encouraging nearby pixels with similar colors to stay close in value.

We represent the image as a graph. Each pixel is a node, and edges connect nearby pixels. The weight of an edge is high when the two pixels have similar colors. This favors smoothness within regions while still preserving sharp edges where colors differ.

Suppose the noisy image is given by a vector of pixel values x_{noisy}. We want to find a cleaner image x that is close to the noisy input but also smooth on the graph. This balance is expressed by the minimization problem

$$\min_x \ \|x - x_{\text{noisy}}\|_2^2 \ + \ \lambda\, x^T L x.$$

The first term, $\|x - x_{\text{noisy}}\|_2^2$, keeps the result close to the observed image. The second term, $\lambda x^\top L x$, enforces smoothness by penalizing differences between connected pixels. The parameter λ controls how strongly we smooth: larger λ removes more noise but can blur details.

By setting the gradient to zero (as you may recall from calculus), this problem turns into a linear system

$$(I_n + \lambda L)x = x_{\text{noisy}}.$$

Solving this system gives the denoised image x.

Example 24.7. (Ranking and influence propagation on a network)

In many networks, we may want to find the most "important" or "influential" nodes relative to a specific starting point. For example, in a social network, who is most influenced by a particular user? One way to model this is as a diffusion process on the graph.

Think of injecting "heat" at some source nodes and letting it spread through the network. Nodes that become hotter are considered more influenced or more relevant. The steady-state heat distribution gives a natural ranking of the nodes.

Mathematically, this leads to the linear system

$$(I_n + \alpha L)r = s,$$

where s is a vector marking the sources ($s_i = 1$ if node i is a source, and $s_i = 0$ otherwise), and r is the vector of influence scores. The parameter $\alpha > 0$ controls how far the influence spreads: small α keeps it local, while larger α lets it propagate more widely.

Solving this system gives each node a score in r. This idea appears in recommendation systems, personalized search, and even in ranking genes or proteins in biological networks.

Problems

24.1 Let $f : \mathbb{R}^n \to \mathbb{R}$ be the quadratic function

$$f(x) = x^T Ax + x^T b + c,$$

where $A \in \mathbb{R}^{n \times n}$ is symmetric, $b \in \mathbb{R}^n$ and $c \in \mathbb{R}$. Show that

$$\nabla_x f := \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} = 2Ax + b.$$

24.2 A **path graph** is a graph where the nodes are arranged in a line, and each node is connected only to its immediate neighbors. Consider the path graph with $n = 5$ nodes, where the first and last nodes are labeled: node 1 has label +1 and node 5 has label -1. Formulate the label propagation problem for this graph and solve for the labels of the three unlabeled nodes.

24.3 Consider a social network of six people, where an edge means two people interact in person. Anna (1) interacts with Ben (2) and Clara (3). Ben (2) interacts with Anna (1), Clara (3), and Dan (4). Clara (3) interacts with Anna (1), Ben (2), Dan (4), and Frank (6). Dan (4) interacts with Ben (2), Clara (3), and Emma (5). Emma (5) interacts with Dan (4). Frank (6) interacts only with Clara (3).

Suppose Anna (1) is known to be infected with the flu (label +1) and Emma (5) is known to be not infected (label -1). We think of the label as the *health status* of each person: +1 means infected, -1 means not infected. Formulate the label propagation problem for this network and compute the predicted health status of Ben (2), Clara (3), Dan (4), and Frank (6).

Part VI
ORTHOGONALITY AND THE QR FACTORIZATION

In this part, we cover two important topics: orthogonal matrices and the QR factorization.

We begin with orthogonal matrices. They are easy to work with because their inverse is just the transpose. This makes them useful for data transformations like the Fourier and wavelet transforms. They also play a big role in numerical algorithms, where they help turn matrices into simpler forms like upper triangular, Hessenberg, or bidiagonal.

Next, we introduce the QR factorization. This factorization is a key tool for solving least squares and eigenvalue problems. We also introduce the Householder algorithm and some of its variants, which compute the QR factorization efficiently and accurately.

Chapter 25
VI-WHY SHOULD WE CARE?

In this part of the book, we study orthogonality and orthogonal matrices, which are key ideas in Linear Algebra. Orthogonality simplifies many problems, as we will see throughout this and later chapters. We also cover the QR factorization, a useful tool with many applications in Machine Learning.

25.1 Applications of orthogonal matrices

Orthogonal matrices are widely used in signal processing, especially for denoising and data compression.

Denoising and compressing data with orthogonal bases

A *time series* is a set of data points collected at regular intervals over time. It can track things like temperature changes, rainfall, stock prices, interest rates, or heart rate during activities. While time series data shows trends or patterns, it often includes noise from measurement errors or random events, which makes it harder to see the real behavior.

Heart rate data is a good example of a noisy time series. Figure 25.1 shows one of the author's heart rates over a day, with activities like exercise, walking, and sleep marked. The data has 720 entries (one every 2 minutes), and fluctuations happen for many reasons.

To remove noise from the data, we want to filter out fast, sharp changes (high-frequency components) and keep the slower, meaningful trends (low-frequency components). Low-frequency signals change slowly and produce smooth graphs with gentle curves. High-frequency signals change quickly and create spiky graphs with lots of peaks and drops. For example, in heart rate data, we try to remove the fast fluctuations and focus on the smoother, long-term trends.

As mentioned, Figure 25.1 shows the signal over time. To remove noise, we switch from the time view to the frequency view, which shows how fast the signal changes. This lets us see how much energy is in each frequency, making it easier to filter out the noise. We can do this using the Fourier or Haar transform (both use orthogonal bases). We will focus on the Haar transform in Chapter 29, since the Fourier transform is already well known. After filtering, we switch back to the time

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_25

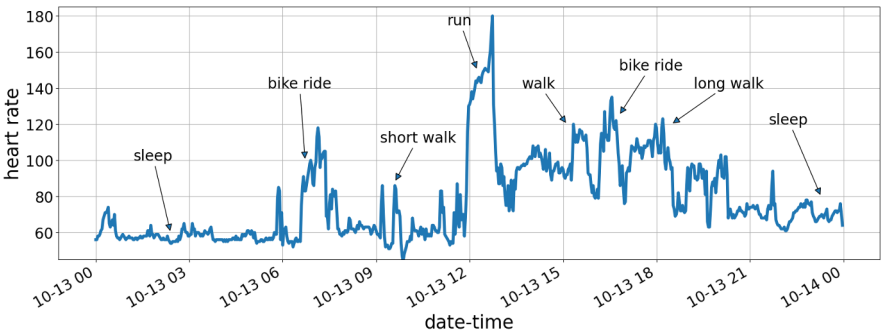


Fig. 25.1: Heart rate time series data over the course of one day.

view using the inverse transform. Orthogonal bases make this whole process fast and easy.

Although this example focuses on denoising, the same ideas also apply to data compression, which is a type of dimensionality reduction. We will cover this in more detail in Chapter 29.

25.2 Applications of the QR factorization

This section shows how the QR factorization is used in Machine Learning. We will revisit and expand on some of these applications in later chapters.

The QR factorization and least squares problems

Many problems lead to linear systems of equations $Ax = b$, where A is "tall and skinny". Often, these systems do not have a solution. So, what is the closest we can get to solving this unsolvable problem? In Linear Algebra, we find a vector x that minimizes $\|b - Ax\|_2$. This is known as the **least squares problem**, which we will cover in detail in Chapter 32.

In Chapter 22, we used the LU factorization to solve linear systems by breaking $Ax = b$ into two simpler problems: $Ly = b$ and $Ux = y$. For least squares problems, we will use the QR factorization. We will factor $A \in \mathbb{R}^{m \times n}$ as $A = QR$, where $Q \in \mathbb{R}^{m \times n}$ satisfies $Q^T Q = I_n$ and $R \in \mathbb{R}^{n \times n}$ is upper triangular. This turns the least squares problem into a linear system $Rx = Q^T b$, where R is nonsingular. The unique solution to this system is also the solution to the least squares problem, as we will prove in Chapter 32.

QR and eigenvalue computations

The QR factorization appears in surprising places in Linear Algebra. You might remember calculating eigenvalues for small matrices in your introductory Linear

Algebra course (how small depending on how tough your professor was!). And all those tricky determinants you carefully computed, with hardly any mistakes, of course (wink, wink).

Well, that is not how computers do these computations. To find eigenvalues, they start with a QR factorization of the matrix $A = QR$. Then they swap the factors to form the matrix RQ and repeat: factor into QR, swap to RQ. Over time, the matrix approaches a diagonal matrix, with the eigenvalues neatly appearing on the diagonal. We will study this algorithm in Chapter 37.

QR, SVD, and PCA

In addition to the QR factorization, there are other important factorizations where orthogonality plays a key role. One of them is the Singular Value Decomposition (SVD), which we will cover in Chapter 47. A major application of the SVD is Principal Component Analysis (PCA), a widely used method for dimensionality reduction, which we will introduce in Chapter 49.

From a computational standpoint, the SVD requires more operations than the QR factorization. We will also see how to use QR to perform PCA more efficiently in certain situations.

Feature selection

In modern applications, datasets often have thousands of features, but many of them may be irrelevant or redundant. This makes analysis more complex and slower. Feature selection helps by identifying the most important features.

In Linear Algebra, features are redundant when the columns of the data matrix A are nearly linearly dependent. This means some columns can be approximately expressed as combinations of others. The column-pivoted QR algorithm chooses columns that are "as independent as possible". At each step, it selects the column least predicted by the ones already chosen. This results in the pivoted QR factorization:

$$AP = QR,$$

where P is a permutation matrix, and R is an upper triangular matrix with diagonal entries that decrease in magnitude.

The permutation matrix P ranks the columns of A by their importance. The "most independent" columns appear first. To select features, we choose the top k columns based on this order, as they represent the most relevant and independent features in our dataset.

Chapter 26
VI-WHAT YOU MAY HAVE LEARNED BEFORE

26.1 Orthogonal sets and their properties

When we think of orthogonal vectors, we usually imagine two vectors in $\mathbb{R}^2$ forming a right angle, as shown in Figure 26.1. The angles α and β that vectors u and v form with the horizontal axis satisfy

$$\beta = \alpha \pm \frac{\pi}{2}.$$

This gives

$$\tan\beta = \frac{-1}{\tan\alpha}.$$

Since $\tan\alpha = \frac{u_2}{u_1}$ and $\tan\beta = \frac{v_2}{v_1}$, we get

$$u_1v_1 + u_2v_2 = 0.$$

In other words, in $\mathbb{R}^2$, vectors are orthogonal when their dot product is zero [1]. We use this same idea to define orthogonality in higher dimensions.

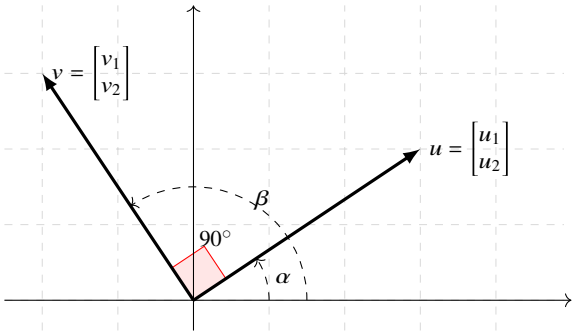


Fig. 26.1: Two orthogonal vectors in $\mathbb{R}^2$.

[1] Keep in mind that we also need to account for cases where u_1 or v_1 are zero. However, the condition that the dot product equals zero still holds and covers these special cases.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_26

Definition 26.1. (Orthogonal vectors)

We say that two vectors $u, v \in \mathbb{R}^m$ are **orthogonal** (denoted by $u \perp v$) if their dot product $u^T v$ is zero.

Example 26.2 The vectors $u = \begin{bmatrix} 2 & 3 & -1 \end{bmatrix}^T$ and $v = \begin{bmatrix} 0 & 1 & 3 \end{bmatrix}^T$ are orthogonal since $u^T v = 2 \cdot 0 + 3 \cdot 1 + (-1) \cdot 3 = 0$.

Notice that the definition of orthogonality makes the m-dimensional zero vector orthogonal to every other vector in $\mathbb{R}^m$.

Our intuition suggests that if a vector u is orthogonal to two nonzero vectors v_1 and v_2, then u should also be orthogonal to the entire plane spanned by v_1 and v_2. The next result confirms that this holds not just for two vectors but for any number of vectors.

Proposition 26.3. (Orthogonality, linear combinations, and span)

If a vector u is orthogonal to vectors $v_1, v_2, \ldots, v_k$, then u is also orthogonal to any vector in the subspace $\text{span}\{v_1, v_2, \ldots, v_k\}$.

Proof We want to show that if u is orthogonal to $v_1, v_2, \ldots, v_k$, then it is orthogonal to any linear combination of them. Using the distributive property of the dot product, we have

$$u^T(\alpha_1 v_1 + \cdots + \alpha_k v_k) = \alpha_1 u^T v_1 + \cdots + \alpha_k u^T v_k = 0,$$

for any α_i's. □

Orthogonality and right angles

We define orthogonal vectors in $\mathbb{R}^m$ using the dot product, but this also connects to a geometric idea about angles. Back in Chapter 11, we defined the angle $\theta(u, v)$ between two nonzero vectors u and v in $\mathbb{R}^n$ as

$$\theta(u, v) = \arccos\left(\frac{u^T v}{\|u\|_2 \|v\|_2}\right) \in [0, \pi].$$

Thus, $u^T v = 0$ precisely when $\theta = \frac{\pi}{2}$ (or 90°). So, even in higher dimensions where we cannot visualize angles, orthogonality (a zero dot product) means the vectors form right angles.

We can extend the idea of orthogonality from two vectors to sets of vectors.

Definition 26.4. (Orthogonal and orthonormal sets of vectors)

Let $S \subseteq \mathbb{R}^m$. The set S is **orthogonal** if, for all distinct vectors $u, v \in S$, the dot product $u^T v = 0$. The set S is **orthonormal** if it is orthogonal and each vector in S is a unit vector with respect to the 2-norm[2], that is, $\|u\|_2 = 1$ for every $u \in S$.

Example 26.5 The standard basis for $\mathbb{R}^3$

$$\beta = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\},$$

is both an orthogonal and an orthonormal set. Similarly, the standard basis $\{e_1, e_2, \ldots, e_m\}$ for $\mathbb{R}^m$ is an orthonormal set for every positive m.

Example 26.6 The set

$$\left\{ \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} \right\}$$

is orthogonal, but not orthonormal, since the vectors are not unit length. To make the set orthonormal, we divide each vector by its 2-norm. Doing this gives the orthonormal set

$$\left\{ \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}, \frac{1}{2} \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix}, \frac{1}{2} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} \right\}.$$

Orthogonality (for nonzero vectors) is a stronger condition than linear independence, as we will show next.

Proposition 26.7. (Nonzero orthogonality implies linear independence)

If $S = \{v_1, v_2, \ldots, v_k\} \subseteq \mathbb{R}^m$ is an orthogonal set of nonzero vectors, then S is linearly independent.

[2] In theory, we could define unit vectors using other norms, but this is rarely done in practice. Sticking to the 2-norm leads to many useful properties.

Proof Assume $\alpha_1 v_1 + \cdots + \alpha_k v_k = 0$ for some scalars $\alpha_1, \ldots, \alpha_k \in \mathbb{R}$. We need to show that all α_i are zero. Taking the dot product of both sides with v_i for each $i = 1, \ldots, k$, we get

$$0 = v_i^T(\alpha_1 v_1 + \cdots + \alpha_k v_k) = \alpha_1 v_i^T v_1 + \cdots + \alpha_k v_i^T v_k = \alpha_i v_i^T v_i,$$

since S is orthogonal.

Because $v_i^T v_i = \|v_i\|_2^2 \neq 0$ (as v_i is nonzero), it follows that $\alpha_i = 0$ for all $i = 1, \ldots, k$, which proves that S is linearly independent. □

Proposition 26.7 shows that a set of orthonormal vectors always forms a basis for the subspace they span. As we will see, this type of basis is the best because it simplifies many Linear Algebra operations and problems.

Let us update our notation. From now on, when we talk about "orthogonal" or "orthonormal" vectors, we will use the more standard notation q for the vectors, instead of other letters like u, v, or w.

Definition 26.8. (Orthogonal and orthonormal bases)

We call a basis $\beta = \{q_1, q_2, \ldots, q_n\}$ for a subspace $\mathcal{S} \subseteq \mathbb{R}^m$ **orthogonal** when β is an orthogonal set, and **orthonormal** when it is an orthonormal set.

Orthogonal and orthonormal bases simplify the process of representing vectors in different bases, as the following proposition shows.

Proposition 26.9. (Vector expansion in an orthogonal or orthonormal basis)

Consider a subspace $\mathcal{S} \subseteq \mathbb{R}^m$ and a vector $v \in \mathcal{S}$. Assume $\beta = \{q_1, \ldots, q_n\}$ is a basis for $\mathcal{S}$.

(a) If β is orthogonal, the expansion of v as a linear combination of the basis vectors is

$$v = \left(\frac{q_1^T v}{\|q_1\|_2^2}\right) q_1 + \left(\frac{q_2^T v}{\|q_2\|_2^2}\right) q_2 + \cdots + \left(\frac{q_n^T v}{\|q_n\|_2^2}\right) q_n.$$

(b) If β is orthonormal, the expansion of v as a linear combination of the basis vectors is

$$v = \left(q_1^T v\right) q_1 + \left(q_2^T v\right) q_2 + \cdots + \left(q_n^T v\right) q_n.$$

Proof Since $\{q_1, \ldots, q_n\}$ is a basis for $\mathcal{S}$, we can express v as a linear combination of the q_i's:

$$v = \alpha_1 q_1 + \alpha_2 q_2 + \cdots + \alpha_n q_n,$$

where $\alpha_1, \ldots, \alpha_n$ are scalars. Taking the dot product with each q_i gives

$$q_i^T v = \alpha_1 q_i^T q_1 + \cdots + \alpha_i q_i^T q_i + \cdots + \alpha_n q_i^T q_n.$$

If $\{q_1, \ldots, q_n\}$ is orthogonal, then $q_i^T q_j = 0$ for $i \neq j$, so the above equation simplifies to

$$q_i^T v = \alpha_i q_i^T q_i = \alpha_i \|q_i\|_2^2.$$

Solving for α_i, we get $\alpha_i = \frac{q_i^T v}{\|q_i\|_2^2}$, confirming part (a).

If $\{q_1, \ldots, q_n\}$ is orthonormal, each q_i is a unit vector, so $\|q_i\|_2^2 = 1$. This simplifies α_i to $\alpha_i = q_i^T v$, confirming part (b). □

Example 26.10 Consider the vector $v = \begin{bmatrix} 1 & 2 & 2 & 1 \end{bmatrix}^T$ and the orthonormal basis from Example 26.6. Using Proposition 26.9, we can find the coordinates of v in terms of this basis by computing the dot product of v with each basis vector:

$$\begin{aligned}
\alpha_1 &= q_1^T v = \frac{1}{2}(1 + 2 + 2 + 1) = 3, \\
\alpha_2 &= q_2^T v = \frac{1}{2}(1 + 2 - 2 - 1) = 0, \\
\alpha_3 &= q_3^T v = \frac{1}{2}(1 - 2 - 2 + 1) = -1, \qquad \text{and} \\
\alpha_4 &= q_4^T v = \frac{1}{2}(1 - 2 + 2 - 1) = 0.
\end{aligned}$$

Now, we can represent v as a linear combination of the basis vectors:

$$\begin{bmatrix} 1 \\ 2 \\ 2 \\ 1 \end{bmatrix} = 3 \begin{bmatrix} 1/2 \\ 1/2 \\ 1/2 \\ 1/2 \end{bmatrix} + 0 \begin{bmatrix} 1/2 \\ 1/2 \\ -1/2 \\ -1/2 \end{bmatrix} - \begin{bmatrix} 1/2 \\ -1/2 \\ -1/2 \\ 1/2 \end{bmatrix} + 0 \begin{bmatrix} 1/2 \\ -1/2 \\ 1/2 \\ -1/2 \end{bmatrix}.$$

We can express the result in Proposition 26.9 in matrix form. First, we build the matrix

$$Q := \begin{bmatrix} q_1 & \cdots & q_n \end{bmatrix},$$

with the basis vectors $q_1, \ldots, q_n$ as its columns. We can write any vector v in the subspace $\mathcal{S} = \text{span}\{q_1, \ldots, q_n\}$ as

$$v = \alpha_1 q_1 + \cdots + \alpha_n q_n = \begin{bmatrix} q_1 & \cdots & q_n \end{bmatrix} \underbrace{\begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix}}_{=:c} = Qc,$$

where c is the vector of coordinates α_i in the basis $\{q_1, \ldots, q_n\}$. Since the columns of Q are orthonormal, we have $Q^T Q = I_n$. Therefore, we can find c simply by computing

$$c = Q^T v = \begin{bmatrix} q_1^T \\ q_2^T \\ \vdots \\ q_n^T \end{bmatrix} v = \begin{bmatrix} q_1^T v \\ q_2^T v \\ \vdots \\ q_n^T v \end{bmatrix},$$

as shown in Proposition 26.9.

Example 26.11 Let us revisit Example 26.10. We compute the coordinates of the vector v in the basis $\{q_1, q_2, q_3, q_4\}$ using matrix-vector multiplication. We have,

$$Q^T v = \begin{bmatrix} 1/2 & 1/2 & 1/2 & 1/2 \\ 1/2 & 1/2 & -1/2 & -1/2 \\ 1/2 & -1/2 & -1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 & -1/2 \end{bmatrix}^T \begin{bmatrix} 1 \\ 2 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ -1 \\ 0 \end{bmatrix},$$

which matches the coordinates we found earlier.

26.2 Turning bases into orthonormal bases: The Gram-Schmidt process

A natural question is: does every subspace of $\mathbb{R}^m$ have an orthonormal basis? If so, how do we find one?

In Theorem 26.12, we show how to transform any linearly independent set of vectors β in $\mathbb{R}^m$ into an orthonormal set with the same span, using the **Gram-Schmidt process**.

The Gram-Schmidt process starts with linearly independent vectors $w_1, w_2, \ldots, w_n$ and produces orthonormal vectors $q_1, q_2, \ldots, q_n$ such that

$$\text{span}\{q_1, \ldots, q_i\} = \text{span}\{w_1, \ldots, w_i\}, \quad \text{for } i = 1, \ldots, n.$$

This process guarantees that we can always find an orthonormal basis for any subspace of $\mathbb{R}^m$.

Before introducing the Gram–Schmidt process, we will start with some algebraic intuition behind the algorithm. We will also provide a geometric intuition in Chapter 32.

Gram-Schmidt process with three vectors

Consider three linearly independent vectors $w_1, w_2, w_3 \in \mathbb{R}^m$. Our goal is to find an orthonormal set $\{q_1, q_2, q_3\}$ such that

$$\begin{aligned} &\text{span}(q_1) = \text{span}(w_1), \\ &\text{span}(q_1, q_2) = \text{span}(w_1, w_2), \quad \text{and} \\ &\text{span}(q_1, q_2, q_3) = \text{span}(w_1, w_2, w_3). \end{aligned}$$

Since the vectors w_1, w_2, w_3 are linearly independent, we know that $w_1 \neq 0$. We can now define the vector

$$q_1 := \frac{w_1}{\|w_1\|_2}.$$

This is a unit vector that spans the same subspace as w_1, as both q_1 and w_1 lie on the same line.

Next, we need to find a unit vector q_2 that is orthogonal to q_1. We begin by finding a nonzero vector v_2 orthogonal to q_1 so that $\text{span}(q_1, v_2) = \text{span}(w_1, w_2)$. As shown in Figure 26.2, we can construct v_2 by subtracting a multiple of q_1 from w_2. So, let us consider a vector of the form $v_2 = w_2 - \alpha_1 q_1$, for some $\alpha_1 \in \mathbb{R}$.

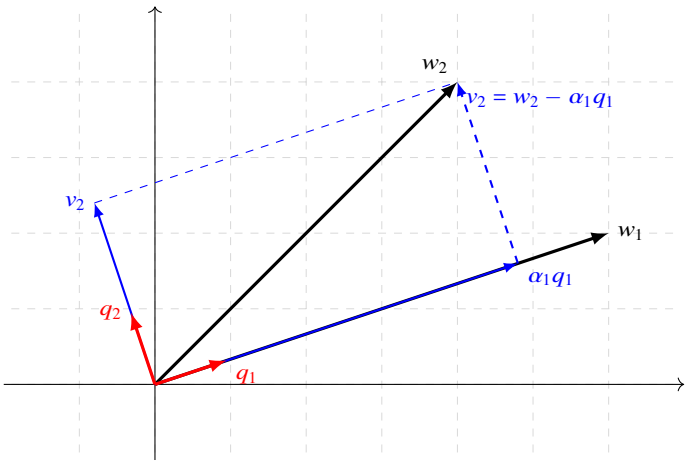


Fig. 26.2: Gram-Schmidt process with two vectors.

We need to determine the scalar α_1 that makes v_2 orthogonal to q_1. The orthogonality condition is given by

$$\begin{aligned} 0 = q_1^T v_2 &= q_1^T (w_2 - \alpha_1 q_1) \\ &= q_1^T w_2 - \alpha_1 q_1^T q_1 \\ &= q_1^T w_2 - \alpha_1. \end{aligned}$$

Thus, we find

$$\alpha_1 = q_1^T w_2,$$

and therefore

$$v_2 = w_2 - (q_1^T w_2) q_1.$$

Notice that $v_2 \neq 0$; otherwise, it would mean $w_2 \in \text{span}(q_1) = \text{span}(w_1)$, which contradicts the fact that w_1 and w_2 are linearly independent.

Finally, we normalize v_2 to obtain

$$q_2 := \frac{v_2}{\|v_2\|_2}.$$

Also, note that $v_2 \in \text{span}\{q_1, w_2\} = \text{span}\{w_1, w_2\}$. Thus, q_1 and q_2 both lie in $\text{span}\{w_1, w_2\}$, which implies $\text{span}\{q_1, q_2\} \subseteq \text{span}\{w_1, w_2\}$. Since both subspaces have the same dimension, they must be equal.

With this, q_1 and q_2 form the required orthonormal basis for the subspace spanned by w_1 and w_2.

Finally, we need a vector q_3 that is orthogonal to both q_1 and q_2. To find this vector, we start with a vector v_3 that is orthogonal to both q_1 and q_2 and lies in $\text{span}(w_1, w_2, w_3)$.

Similar to what we did to find v_2, we can find v_3 by subtracting appropriate multiples of q_1 and q_2 from w_3. So, we consider a vector of the form

$$v_3 = w_3 - \beta_1 q_1 - \beta_2 q_2, \quad \text{for some } \beta_1, \beta_2 \in \mathbb{R}.$$

Using the orthogonality conditions $q_1^T v_3 = q_2^T v_3 = 0$, we get

$$\begin{aligned} 0 = q_1^T v_3 &= q_1^T w_3 - \beta_1, \quad \text{and} \\ 0 = q_2^T v_3 &= q_2^T w_3 - \beta_2, \end{aligned}$$

which gives us $\beta_1 = q_1^T w_3$ and $\beta_2 = q_2^T w_3$.

Thus, we have

$$v_3 = w_3 - (q_1^T w_3) q_1 - (q_2^T w_3) q_2.$$

Note that $v_3 \neq 0$, because if it were, we would have $w_3 \in \text{span}(q_1, q_2) = \text{span}(w_1, w_2)$, which contradicts the fact that w_1, w_2, w_3 are linearly independent.

Finally, we normalize v_3 to get

$$q_3 := \frac{v_3}{\|v_3\|_2}.$$

This gives us the desired orthonormal set $\{q_1, q_2, q_3\}$.

Based on the calculations above, here are the formulas to create an orthonormal set $\{q_1, q_2, q_3\}$ that spans the same subspace as the three linearly independent vectors $\{w_1, w_2, w_3\}$:

$$\begin{aligned} v_1 &:= w_1, & q_1 &:= \frac{v_1}{\|v_1\|_2} \\ v_2 &:= w_2 - (q_1^T w_2) q_1, & q_2 &:= \frac{v_2}{\|v_2\|_2} \\ v_3 &:= w_3 - (q_1^T w_3) q_1 - (q_2^T w_3) q_2, & q_3 &:= \frac{v_3}{\|v_3\|_2}. \end{aligned}$$

We can extend this process to a larger set of vectors using induction.

Theorem 26.12. (Gram-Schmidt process)

Let $\{w_1, \ldots, w_n\} \subseteq \mathbb{R}^m$ be a linearly independent set. Define the vectors

$$q_i := \frac{v_i}{\|v_i\|_2},$$

where

$$v_1 := w_1 \qquad \text{and} \qquad v_i := w_i - \sum_{j=1}^{i-1} (q_j^T w_i) q_j, \quad \text{for i} = 2, \ldots, n.$$

Then, $\{q_1, \ldots, q_n\}$ is an orthonormal set with the property

$$\text{span}(q_1, \ldots, q_i) = \text{span}(w_1, \ldots, w_i), \quad \text{for } i = 1, \ldots, n.$$

Proof We prove the result by induction on the number of vectors n.

The base case $n = 1$ is straightforward since q_1 is a unit vector in the same direction as w_1. Therefore, $\{q_1\}$ is an orthonormal basis for $\text{span}(w_1)$.

Now, assume that the statement holds for sets with less than n vectors and consider a linearly independent set $\{w_1, \ldots, w_n\}$, with n vectors. By the inductive hypothesis applied to $\{w_1, \ldots, w_{n-1}\}$, there exists an orthonormal set $\{q_1, \ldots, q_{n-1}\}$ with the property

$$\text{span}\{q_1, \ldots, q_i\} = \text{span}\{w_1, \ldots, w_i\}, \quad \text{for } i = 1, \ldots, n-1.$$

Let

$$v_n := w_n - \sum_{j=1}^{n-1} (q_j^T w_n) q_j.$$

Thus, we have

$$v_n \in \text{span}(q_1, \ldots, q_{n-1}, w_n) = \text{span}(w_1, \ldots, w_{n-1}, w_n),$$

where the equality follows from the inductive hypothesis. Moreover, we have $v_n \neq 0$, otherwise we would have $w_n = \sum_{j=1}^{n-1} (q_j^T w_n) q_j \in \text{span}(q_1, \ldots, q_{n-1}) = \text{span}(w_1, \ldots, w_{n-1})$, contradicting the fact that $\{w_1, \ldots, w_n\}$ is a linearly independent set. Hence, $\|v_n\|_2 \neq 0$ and we can define

$$0 \neq q_n := \frac{v_n}{\|v_n\|_2}.$$

Note that $\|q_n\|_2 = 1$. To show that $\{q_1, \ldots, q_n\}$ is orthonormal, we need to show that the vector q_n is orthogonal to each of $q_1, \ldots, q_{n-1}$. For $t = 1, \ldots, n-1$, we have

$$\begin{aligned} q_t^T q_n &= \frac{q_t^T}{\|v_n\|_2}\left(w_n - \sum_{j=1}^{n-1}(q_j^T w_n)q_j\right) \\ &= \frac{1}{\|v_n\|_2}\left(q_t^T w_n - \sum_{j=1}^{n-1}(q_j^T w_n)q_t^T q_j\right) \\ &= \frac{1}{\|v_n\|_2}\left(q_t^T w_n - (q_t^T w_n)q_t^T q_t\right) = 0, \end{aligned}$$

where the last equality follows from the fact that q_t is a unit vector, i.e., $q_t^T q_t = 1$.

Finally, we show that $\{q_1, \ldots, q_n\}$ is an orthonormal basis for the subspace $\mathcal{S} := \text{span}(w_1, \ldots, w_n)$. First, we observe that $\text{span}(q_1, \ldots, q_n) \subseteq \mathcal{S}$ since each vector q_i is in S. By Lemma 26.7, an orthonormal set is linearly independent. Therefore, $\dim(\text{span}(q_1, \ldots, q_n)) = n = \dim(\mathcal{S})$, which implies that $\{q_1, \ldots, q_n\}$ is a basis for S by the Subspace-Dimension-Lemma (Lemma 16.11). Hence, $\{q_1, \ldots, q_n\}$ is an orthonormal basis for S. This completes the induction step and the proof. □

Example 26.13 Consider the following three linearly independent vectors

$$w_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad w_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \quad \text{and} \quad w_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}.$$

Let us use the Gram-Schmidt process to find an orthonormal basis for the subspace they span. We obtain

$$v_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \qquad q_1 = \frac{v_1}{\|v_1\|_2} = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$v_2 = w_2 - (q_1^T w_2)q_1 = \begin{bmatrix} -1/2 \\ 1/2 \\ 1 \\ 0 \end{bmatrix} \qquad q_2 = \frac{v_2}{\|v_2\|_2} = \frac{1}{\sqrt{6}}\begin{bmatrix} -1 \\ 1 \\ 2 \\ 0 \end{bmatrix}$$

$$v_3 = w_3 - (q_1^T w_3)q_1 - (q_2^T w_3)q_2 = \begin{bmatrix} 1/3 \\ -1/3 \\ 1/3 \\ 1 \end{bmatrix} \qquad q_3 = \frac{v_3}{\|v_3\|_2} = \frac{1}{\sqrt{12}}\begin{bmatrix} 1 \\ -1 \\ 1 \\ 3 \end{bmatrix}.$$

We can rewrite the Gram-Schmidt process using matrices.

The Gram-Schmidt process in matrix form

Consider linearly independent vectors $w_1, \ldots, w_n \in \mathbb{R}^m$. The Gram-Schmidt process generates the vectors

$$q_1 = \frac{w_1}{\|w_1\|_2}, \qquad \text{and} \qquad q_i = \frac{w_i - \sum_{j=1}^{i-1}(q_j^T w_i)q_j}{\|w_i - \sum_{j=1}^{i-1}(q_j^T w_i)q_j\|_2}, \quad \text{for i} = 2, \ldots, n.$$

To put this in matrix form, we define matrices $Q_1 := 0_{m\times 1}$ and $Q_i := \begin{bmatrix} q_1 & q_2 & \cdots & q_{i-1} \end{bmatrix}$, for $i = 2, \ldots, n$. Then, we have

$$Q_i^T w_i = \begin{bmatrix} q_1 & q_2 & \cdots & q_{i-1} \end{bmatrix}^T w_i = \begin{bmatrix} q_1^T \\ q_2^T \\ \vdots \\ q_{i-1}^T \end{bmatrix} w_i = \begin{bmatrix} q_1^T w_i \\ q_2^T w_i \\ \vdots \\ q_{i-1}^T w_i \end{bmatrix}$$

and

$$Q_i Q_i^T w_i = \begin{bmatrix} q_1 & q_2 & \cdots & q_{i-1} \end{bmatrix} \begin{bmatrix} q_1^T w_i \\ q_2^T w_i \\ \vdots \\ q_{i-1}^T w_i \end{bmatrix} = \sum_{j=1}^{i-1} (q_j^T w_i) q_j.$$

This simplifies the Gram-Schmidt calculation for the v_i vector to

$$v_i = w_i - \sum_{j=1}^{i-1} (q_j^T w_i) q_j = w_i - Q_i Q_i^T w_i = (I_m - Q_i Q_i^T) w_i.$$

Thus, the vectors from the Gram-Schmidt process are

$$v_i = (I_m - Q_i Q_i^T) w_i \qquad \text{and} \qquad q_i = \frac{(I_m - Q_i Q_i^T) w_i}{\|(I_m - Q_i Q_i^T) w_i\|_2}, \tag{26.1}$$

for $i = 1, 2, \ldots, n$. In Chapter 32, we will interpret these formulas as orthogonal projections.

We can now answer the question we began the section with. The following result directly follows from Theorem 26.12.

Corollary 26.14. (Subspaces and orthonormal bases)

Every nontrivial subspace $\mathcal{W}$ of $\mathbb{R}^m$ has an orthonormal basis.

Proof Let $\mathcal{W}$ be a nontrivial subspace of $\mathbb{R}^m$ with a basis $\{w_1, \ldots, w_n\}$. By Theorem 26.12, we can apply the Gram-Schmidt process to these vectors to get an orthonormal set $\{q_1, \ldots, q_n\}$ that still spans $\mathcal{W}$. Since orthonormal vectors are also linearly independent, $\{q_1, \ldots, q_n\}$ is an orthonormal basis for $\mathcal{W}$. □

Remark 26.15. (Gram-Schmidt and numerical algorithms)

While the Gram-Schmidt process is a useful theoretical tool, it does not work well as a numerical algorithm. When implemented on a computer, it can produce vectors that are not truly orthonormal (see Problem 26.18). In Problem 26.16, we introduce a modified version of the process that is more numerically stable. In the next chapter, we present an even better method for computing orthonormal bases.

Problems

26.1 Let $u_1 = \begin{bmatrix}1 & 2 & 0 & -2\end{bmatrix}^T$ and $u_2 = \begin{bmatrix}0 & 1 & 0 & -2\end{bmatrix}^T$. Find a vector v which is orthogonal to both u_1 and u_2. Show that v is also orthogonal to all the vectors in $\text{span}(u_1, u_2)$.

26.2 Let $A \in \mathbb{R}^{n\times n}$. Give sufficient conditions on A so that it preserves orthogonality, that is, $(Ax)^T(Ay) = 0$ whenever $x^T y = 0$.

26.3 Let $\{v_i\}_{i=1,\ldots,n}$ be an orthogonal set. Show that $\{v_i/\|v_i\|_2\}_{i=1,\ldots,n}$ is an orthonormal set.

26.4 Let $\{v_1, \ldots, v_n\}$ be a basis for $\mathbb{R}^n$ and let $x \in \mathbb{R}^n$. Show that if x is orthogonal to all vectors v_i, then $x = 0$.

26.5 (Pythagorean Theorem) Given an orthogonal set of vectors $\{q_1, q_2, \ldots, q_n\}$, prove that $\|q_1 + q_2 + \cdots + q_n\|_2^2 = \|q_1\|_2^2 + \|q_2\|_2^2 + \cdots + \|q_n\|_2^2$.

26.6 (**Parseval's identity**) Let $\{q_1, \ldots, q_n\}$ be an orthonormal basis in $\mathbb{R}^n$. Prove that, for all $v, w \in \mathbb{R}^n$,

$$v^T w = \sum_{i=1}^{n} (v^T q_i)(q_i^T w).$$

26.7 (**Bessel's identity**) Let $\{q_1, \ldots, q_n\}$ be an orthonormal basis in $\mathbb{R}^n$. Show that, for all $v \in \mathbb{R}^n$,

$$\|v\|_2^2 = \sum_{i=1}^{n} |v^T q_i|^2.$$

26.8 (**Bessel's inequality**) Consider an orthonormal set of vectors $\{q_1, q_2, \ldots, q_n\}$ in $\mathbb{R}^m$. Prove that if $x \in \mathbb{R}^m$ and $\alpha_i = q_i^T x$, for $i = 1, \ldots, n$, then

$$\sum_{i=1}^{n} \alpha_i^2 \leqslant \|x\|_2^2.$$

What condition must x meet for this inequality to become an equality?

Hint: Consider $\|x - \sum_{i=1}^{n} \alpha_i q_i\|_2^2$.

26.9 Let $\{q_1, \ldots, q_m\}$ be an orthonormal set in $\mathbb{R}^n$. Show that

(a) $m \leqslant n$;
(b) if $m < n$, then there exist vectors $q_{m+1}, \ldots, q_n$ such that $\{q_1, \ldots, q_n\}$ is an orthonormal basis for $\mathbb{R}^n$.

26.10 (**Orthogonal complex vectors**) Let $u, v \in \mathbb{C}^n$. We say that u and v are orthogonal if $u^* v = 0$.

(a) Show that $\|u + v\|_2^2 = \|u\|_2^2 + \|v\|_2^2$.
(b) Is it true that if $\|u + v\|_2^2 = \|u\|_2^2 + \|v\|_2^2$, then u and v are orthogonal? Prove your answer or provide a counterexample.

26.11 Why do we define two complex vectors $u, v \in \mathbb{C}^n$ as orthogonal when $u^* v = 0$ instead of using $u^T v = 0$, as we have done in previous problem?

26.12 Let $A \in \mathbb{C}^{n \times n}$. Prove that $AA^* = I_n$ if and only if the row of A form an orthonormal basis for $\mathbb{C}^n$.

26.13 Use the Gram-Schmidt process to construct an orthonormal basis for the subspace spanned by the vectors

$$w_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \qquad w_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \qquad \text{and} \qquad w_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}.$$

26.14 Find an orthonormal basis for the plane $x_1 + x_2 + x_4 = 0$ in $\mathbb{R}^4$.

26.15 Suppose a clustering algorithm produces cluster centers $c_1, \ldots, c_k$. To maximize cluster separation, the centers are sometimes encouraged to be orthogonal. Given the cluster centers

$$c_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \qquad c_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \qquad \text{and} \qquad c_3 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix},$$

use the Gram-Schmidt process to produce orthogonal clusters.

26.16 (Modified Gram-Schmidt process) Given linearly independent vectors $w_1, \ldots, w_n \in \mathbb{R}^m$, recall that the vectors $q_1, \ldots, q_n$ produced by the Gram-Schmidt process can be written as

$$q_i = \frac{(I_m - Q_i Q_i^T) w_i}{\|(I_m - Q_i Q_i^T) w_i\|_2}, \quad \text{where} \quad Q_1 = 0_{m \times 1} \text{ and } Q_i = \begin{bmatrix} q_1 & \cdots & q_{i-1} \end{bmatrix}.$$

(a) Define the matrices $E_1 = I_m$ and $E_j = I_m - q_{j-1}q_{j-1}^T$, for $j > 1$. Show that the Gram-Schmidt vectors can be expressed as

$$q_i = \frac{E_i \cdots E_2 E_1 w_i}{\|E_i \cdots E_2 E_1 w_i\|_2}, \quad \text{for } i = 1, 2, \ldots, n. \tag{26.2}$$

(b) Show that the Gram-Schmidt vectors can be generated by the following algorithm

$$\begin{array}{ll}
\text{normalize 1st vector} & \{q_1 = \dfrac{w_1}{\|w_1\|_2}, w_2, w_3, \ldots, w_n\} \\
\text{apply } E_2 & \{q_1, E_2 w_2, E_2 w_3, \ldots, E_2 w_3\} \\
\text{normalize 2nd vector} & \{q_1, q_2 = \dfrac{E_2 w_2}{\|E_2 w_2\|_2}, E_2 w_3, \ldots, E_2 w_n\} \\
\text{apply } E_3 & \{q_1, q_2, E_3 E_2 w_3, \ldots, E_3 E_2 w_n\} \\
\vdots &
\end{array}$$

26.17 Use the modified Gram-Schmidt process introduced in Problem 26.16 on the vectors from Problem 26.13. Verify that the resulting vectors are the same.

26.18 (To be solved in a computer) Although there is no theoretical difference between the Gram-Schmidt process and the modified Gram-Schmidt process in Problem 26.16, the latter is more stable computationally. To explore this, apply both the Gram-Schmidt and the modified Gram-Schmidt processes to the vectors:

$$w_1 = \begin{bmatrix} 1 \\ 2 \\ 3 \\ \vdots \\ 10 \end{bmatrix} \qquad \text{and} \qquad w_2 = w_1 + 10^{-15} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

Check if the resulting vectors are orthogonal.

26.19 Let $\{v_1, \ldots, v_m\}$ be a set of nonzero orthogonal vectors in $\mathbb{R}^n$. Show that, if we apply Gram-Schmidt to this set is equivalent to dividing each v_i by its norm.

26.20 Explain what happens if you apply the Gram-Schmidt process to a set of linearly dependent vectors.

26.21 You have a set of vectors $\{w_1, \ldots, w_m\}$ that are linearly dependent. How can you modify the Gram-Schmidt process to create an orthonormal basis for $\text{span}(w_1, \ldots, w_m)$?

26.22 (Computational complexity of the Gram-Schmidt process) Show that applying the Gram-Schmidt process (either the classical or modified version from Problem 26.16) to n linearly independent vectors $w_1, \ldots, w_n \in \mathbb{R}^m$ requires approximately $2mn^2$ arithmetic operations.

Chapter 27
VI-CORE TOPICS

In the previous chapter, we used the word "orthogonal" in a few different ways. For example, we introduced:

- orthogonal vectors: $u^T v = 0$; and
- orthogonal set $\{v_1, \ldots, v_n\}$, where $v_i^T v_j = 0$ for $i \neq j$.

In this chapter, we will discuss the concept of "orthogonal" as it applies to pairs of subspaces and specific types of matrices.

27.1 Orthogonal complements

In Chapter 26, we reviewed the concept of an orthogonal set of vectors, collections of vectors that are all mutually orthogonal. Now we extend the idea of "orthogonality" to apply between two different sets of vectors.

For example, in Figure 27.1, the subspace $\mathcal{P}$ (represented by the blue plane) is orthogonal to the line $\mathcal{L}$ (represented by the red line). The line $\mathcal{L}$ is orthogonal to every vector in $\mathcal{P}$, including the three vectors v_1, v_2, v_3 shown in the figure.

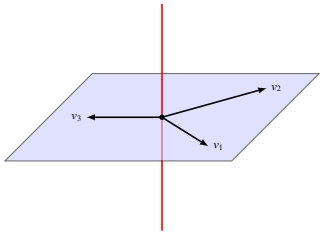


Fig. 27.1: Illustration of two orthogonal subspaces: a plane (in blue) and a line (in red). Every vector in the plane is orthogonal to every vector in the line and vice versa.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_27

Let us start with the formal definition of orthogonal sets.

Definition 27.1. (Orthogonal sets)

Consider two sets $S, T \subseteq \mathbb{R}^m$. We say that S and T are **orthogonal** (denoted $S \perp T$) if every vector in S is orthogonal to every vector in T. In other words, for all $s \in S$ and $t \in T$, we have $s^T t = 0$.

Example 27.2 Consider the plane $\mathcal{P}$ and the line $\mathcal{L}$ in $\mathbb{R}^3$, both passing through the origin, given by

$$\mathcal{P} = \left\{ \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^3 \mid 2x_1 + 3x_2 - x_3 = 0 \right\}, \qquad \text{and}$$

$$\mathcal{L} = \text{span}\left(\begin{bmatrix} 2 \\ 3 \\ -1 \end{bmatrix} \right) = \left\{ \alpha \begin{bmatrix} 2 \\ 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 2\alpha \\ 3\alpha \\ -\alpha \end{bmatrix} \mid \alpha \in \mathbb{R} \right\}.$$

For every vector $\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}^T \in \mathcal{P}$ and every vector $\begin{bmatrix} 2\alpha & 3\alpha & -\alpha \end{bmatrix}^T \in \mathcal{L}$, we have

$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 2\alpha \\ 3\alpha \\ -\alpha \end{bmatrix} = 2\alpha x_1 + 3\alpha x_2 - \alpha x_3 = \alpha(2x_1 + 3x_2 - x_3) = 0,$$

where the last equality follows from the definition of $\mathcal{P}$. Thus $\mathcal{P} \perp \mathcal{L}$.

In many situations, we need to find the largest subset of $\mathbb{R}^m$ that is orthogonal to a given subset $U \subseteq \mathbb{R}^m$. This leads to the following definition.

Definition 27.3. (Orthogonal complement)

Given a subset U of $\mathbb{R}^m$, the **orthogonal complement** of U (denoted $U^\perp$) is the set of all vectors in $\mathbb{R}^m$ that are orthogonal to every vector in U. That is,

$$U^\perp = \{x \in \mathbb{R}^m \mid x \perp u, \text{ for all } u \in U\}.$$

Example 27.4 The orthogonal complement of the line $\mathcal{L} = \text{span}\left(\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}^T\right)$ in $\mathbb{R}^3$ is the plane

$$\mathcal{L}^\perp = \left\{ \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}^T \mid x_1 + x_3 = 0 \right\},$$

because a vector $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$ is orthogonal to $\mathcal{L}$ if and only if $\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = x_1 + x_3 = 0$.

Example 27.5 The orthogonal complement of $\mathbb{R}^m$ is $\{0\}$, because the zero vector is the only vector that is orthogonal to every vector in $\mathbb{R}^m$.

The following result shows that the orthogonal complement of a set $U \subseteq \mathbb{R}^m$ is a subspace of $\mathbb{R}^m$, even if U itself is not a subspace.

Theorem 27.6. (The orthogonal complement is a subspace)

For every subset $U \subseteq \mathbb{R}^m$, the orthogonal complement $U^\perp$ is a subspace of $\mathbb{R}^m$.

Proof To show that $U^\perp$ is a subspace, we check if it is closed under addition and scalar multiplication.

Consider two vectors $w_1, w_2 \in U^\perp$ and any scalars $\alpha, \beta \in \mathbb{R}$. We want to show that $\alpha w_1 + \beta w_2$ also belongs to $U^\perp$. For any vector $u \in U$, we have

$$u^T(\alpha w_1 + \beta w_2) = \alpha u^T w_1 + \beta u^T w_2 = 0,$$

since w_1 and w_2 are orthogonal to all vectors in U. Thus, $\alpha w_1 + \beta w_2$ is orthogonal to every vector in U, so it belongs to $U^\perp$. The zero vector is also in $U^\perp$, so $U^\perp$ is nonempty. Therefore, $U^\perp$ is a subspace of $\mathbb{R}^m$. □

The most important case of an orthogonal complement is that of a subspace. The next theorem shows how to find it by computing the left nullspace of a matrix whose columns form a basis for the given subspace.

Theorem 27.7. (The orthogonal complement as a nullspace)

Consider a subspace $\mathcal{W} \subseteq \mathbb{R}^m$ with a basis $\{w_1, w_2, \ldots, w_n\}$. If we form the matrix $W = \begin{bmatrix} w_1 & w_2 & \cdots & w_n \end{bmatrix}$, then

$$\mathcal{W}^\perp = \text{null}(W^T).$$

Proof Let $v \in \text{null}(W^T)$, and let b be any vector in $\mathcal{W}$. Since b is in $\mathcal{W}$, we can write $b = Wx$, for some vector $x \in \mathbb{R}^n$ because $\mathcal{W} = \text{col}(W)$. Then, $b^T v = x^T W^T v = x^T 0 = 0$. This shows that v is orthogonal to all the vectors in $\mathcal{W}$, so it is in $\mathcal{W}^\perp$. Conversely, if $v \in \mathcal{W}^\perp$, then since all columns of W are vectors in $\mathcal{W}$, we have $W^T v = 0$. This shows that $v \in \text{null}(W^T)$. Therefore, $\mathcal{W}^\perp = \text{null}(W^T)$. □

Example 27.8 Let us use Theorem 27.7 to find the orthogonal complement of the subspace

$$\mathcal{W} = \text{span}\left(\begin{bmatrix}1\\0\\1\end{bmatrix}, \begin{bmatrix}0\\1\\-1\end{bmatrix}\right) = \text{col}\left(\begin{bmatrix}1 & 0\\0 & 1\\1 & -1\end{bmatrix}\right).$$

By Theorem 27.7, we have

$$\mathcal{W}^{\perp} = \text{null}\left(\begin{bmatrix}1 & 0 & 1\\0 & 1 & -1\end{bmatrix}\right) = \text{span}\left(\begin{bmatrix}-1\\1\\1\end{bmatrix}\right).$$

In Example 27.8, $\mathcal{W}$ is a plane in $\mathbb{R}^3$, and its orthogonal complement is a line perpendicular to that plane. So, $\dim(\mathcal{W}^{\perp}) = \dim(\mathbb{R}^3) - \dim(\mathcal{W})$. This relationship between dimensions holds for any subspace, as we will see in the next theorem.

Theorem 27.9. (Properties of the orthogonal complement of a subspace)

Consider a subspace $\mathcal{W} \subseteq \mathbb{R}^m$. Then,

(a) $\dim(\mathcal{W}^{\perp}) = m - \dim(\mathcal{W})$, and
(b) $(\mathcal{W}^{\perp})^{\perp} = \mathcal{W}$.

Proof Let $n = \dim(\mathcal{W})$, and let $w_1, \ldots, w_n$ be a basis for $\mathcal{W}$. Form the matrix $W = \begin{bmatrix} w_1 & \cdots & w_n \end{bmatrix} \in \mathbb{R}^{m \times n}$. By Theorem 27.7, we know that $\mathcal{W}^{\perp} = \text{null}(W^T)$. By the rank-nullity theorem (Theorem 17.35), we get

$$\begin{aligned}\dim(\mathcal{W}^{\perp}) &= \dim(\text{null}(W^T))\\ &= m - \text{rank}(W^T)\\ &= m - \text{rank}(W)\\ &= m - \dim(\mathcal{W}),\end{aligned}$$

where the third equality holds because the row rank and column rank of a matrix are the same (Theorem 17.30).

For part (b), if we replace $\mathcal{W}$ with $\mathcal{W}^{\perp}$ in part (a), we get

$$\begin{aligned}\dim\left((\mathcal{W}^{\perp})^{\perp}\right) &= m - \dim(\mathcal{W}^{\perp})\\ &= m - (m - \dim(\mathcal{W}))\\ &= \dim(\mathcal{W}).\end{aligned}$$

Now, consider any vector $w \in \mathcal{W}$. Since w is orthogonal to all vectors in $\mathcal{W}^{\perp}$, we have $w \in (\mathcal{W}^{\perp})^{\perp}$. Thus, $\mathcal{W} \subseteq (\mathcal{W}^{\perp})^{\perp}$. Since their dimensions are equal, by the Subspace-Dimension-Lemma (Lemma 16.11), we conclude that $\mathcal{W} = (\mathcal{W}^{\perp})^{\perp}$. □

In Chapter 16, we defined complementary subspaces as two subspaces $\mathcal{S}, \mathcal{T} \subseteq \mathbb{R}^m$ such that $\mathbb{R}^m = \mathcal{S} \oplus \mathcal{T}$ (see Definition 16.15). We call $\mathcal{T}$ a complement of $\mathcal{S}$ in $\mathbb{R}^m$, and vice versa. The following result shows that every subspace $\mathcal{W}$ has a unique complement in $\mathbb{R}^m$ that is orthogonal to it, which is precisely its orthogonal complement $\mathcal{W}^\perp$.

Theorem 27.10. (Orthogonal complementary subspaces)

Let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$. Then,

$$\mathbb{R}^m = \mathcal{W} \oplus \mathcal{W}^\perp,$$

that is, $\mathcal{W}$ and $\mathcal{W}^\perp$ are complementary subspaces. Moreover, if $\mathcal{N} \subseteq \mathbb{R}^m$ is a subspace such that $\mathcal{N} \perp \mathcal{W}$ and $\mathbb{R}^m = \mathcal{W} \oplus \mathcal{N}$, then $\mathcal{N} = \mathcal{W}^\perp$.

Proof To show that $\mathcal{W}$ and $\mathcal{W}^\perp$ are complementary subspaces, we need to prove two things: first, that $\dim(\mathcal{W} + \mathcal{W}^\perp) = m$, so their sum is all of $\mathbb{R}^m$; and second, that $\mathcal{W} \cap \mathcal{W}^\perp = \{0\}$, which means the sum is direct.

First, we show that $\mathcal{W} \cap \mathcal{W}^\perp = \{0\}$. Let $x \in \mathcal{W} \cap \mathcal{W}^\perp$. Then x must be orthogonal to itself, since it belongs to both $\mathcal{W}$ and $\mathcal{W}^\perp$. Hence, $x^T x = \|x\|_2^2 = 0$, which implies $x = 0$ since the only vector with zero norm is the zero vector.

Second, we apply the Dimension-of-a-Sum-Lemma (Exercise 16.15) and Theorem 27.9 to get

$$\begin{aligned}\dim(\mathcal{W} + \mathcal{W}^\perp) &= \dim(\mathcal{W}) + \dim(\mathcal{W}^\perp) - \dim(\mathcal{W} \cap \mathcal{W}^\perp) \\ &= \dim(\mathcal{W}) + \dim(\mathcal{W}^\perp) \\ &= \dim(\mathcal{W}) + m - \dim(\mathcal{W}) \\ &= m = \dim(\mathbb{R}^m).\end{aligned}$$

Therefore, $\mathbb{R}^m = \mathcal{W} + \mathcal{W}^\perp$, and it follows that $\mathbb{R}^m = \mathcal{W} \oplus \mathcal{W}^\perp$ by Theorem 16.17.

Finally, let $\mathcal{N}$ be a subspace of $\mathbb{R}^m$ such that $\mathcal{N} \perp \mathcal{W}$ and $\mathbb{R}^m = \mathcal{W} \oplus \mathcal{N}$. Because $\mathcal{N} \perp \mathcal{W}$, we know that $\mathcal{N} \subseteq \mathcal{W}^\perp$. And since $\mathcal{W} \oplus \mathcal{W}^\perp = \mathbb{R}^m = \mathcal{W} \oplus \mathcal{N}$, Theorem 16.18 implies that

$$\dim(\mathcal{W}) + \dim(\mathcal{W}^\perp) = m = \dim(\mathcal{W}) + \dim(\mathcal{N}).$$

Therefore, $\dim(\mathcal{W}^\perp) = \dim(\mathcal{N})$, and the Subspace-Dimension-Lemma (Lemma 16.11) implies $\mathcal{N} = \mathcal{W}^\perp$. □

The previous result leads to the following definition.

Definition 27.11. (Orthogonal decomposition)

Given a subspace $\mathcal{W}$ of $\mathbb{R}^m$, we call $\mathbb{R}^m = \mathcal{W} \oplus \mathcal{W}^\perp$ an **orthogonal decomposition** of $\mathbb{R}^m$.

By the properties of the direct sum, we can uniquely decompose any vector $x \in \mathbb{R}^m$ as $x = w + w_\perp$, where $w \in \mathcal{W}$ and $w_\perp \in \mathcal{W}^\perp$.

27.1.1 The orthogonality of the four fundamental subspaces

In this section, we revisit the four fundamental subspaces of a matrix from a new angle. The Fundamental Theorem of Linear Algebra (Theorem 17.36) tells us that

$$\mathbb{R}^n = \text{col}(A^T) \oplus \text{null}(A), \quad \text{and} \quad \mathbb{R}^m = \text{col}(A) \oplus \text{null}(A^T).$$

The next theorem shows an even stronger connection: in each pair, one subspace is the orthogonal complement of the other.

Theorem 27.12. (The fundamental theorem of Linear Algebra - Part II)

For any $m \times n$ matrix A, we have

$$\begin{aligned} \text{null}(A) &= \text{col}(A^T)^\perp, \quad \text{and} \\ \text{null}(A^T) &= \text{col}(A)^\perp. \end{aligned}$$

Proof To show that $\text{null}(A) = \text{col}(A^T)^\perp$, let $x \in \mathbb{R}^n$ be any vector in $\text{null}(A)$ and $b \in \text{col}(A^T)$. Then, $b = A^T y$ for some $y \in \mathbb{R}^m$. Hence,

$$b^T x = y^T A x = 0,$$

which means that $x \in \text{col}(A^T)^\perp$. Therefore, $\text{null}(A) \subseteq \text{col}(A^T)^\perp$. Moreover, by part-(i) of Theorem 27.9, we have $\dim(\text{col}(A^T)^\perp) = n - \dim(\text{col}(A^T)) = n - \text{rank}(A) = \dim(\text{null}(A))$. By the Subspace-Dimension-Lemma (Lemma 16.11), we conclude that $\text{null}(A) = \text{col}(A^T)^\perp$.

The proof of the second claim follows from the first one when we replace A by A^T. □

By combining both parts of the Fundamental Theorem of Linear Algebra, we get orthogonal decompositions of $\mathbb{R}^m$ and $\mathbb{R}^n$ for any $m \times n$ matrix A:

$$\begin{aligned} \mathbb{R}^m &= \text{col}(A) \oplus \text{null}(A^T), && \text{with} \quad \text{col}(A)^\perp = \text{null}(A^T) \quad \text{and} \\ \mathbb{R}^n &= \text{col}(A^T) \oplus \text{null}(A), && \text{with} \quad \text{col}(A^T)^\perp = \text{null}(A). \end{aligned}$$

These orthogonal decompositions imply that we can uniquely express any vector in $\mathbb{R}^m$ as the sum of a vector from the column space of A and a vector from the left nullspace of A. Similarly, we can write any vector in $\mathbb{R}^n$ as the sum of a vector from the row space of A and a vector from the nullspace of A.

We can now refine the picture of the four fundamental subspaces we have seen in earlier chapters. Figure 27.2 depicts the relationships among the four fundamental

subspaces associated with the transformation $y = Ax$, where A is an $m \times n$ matrix[1]. The figure aims to highlight several key points:

(i) the row space and nullspace of A live in $\mathbb{R}^n$ while the column space and the left nullspace of A live in $\mathbb{R}^m$;
(ii) the column and row spaces have the same dimensions;
(iii) the nullspace is the orthogonal complement of the row space, and the left nullspace is the orthogonal complement of the column space;
(iv) every vector $x \in \mathbb{R}^n$ can be decomposed (uniquely) into $x = x_r + x_n$, where $x_r \in \text{col}(A^T)$ and $x_n \in \text{null}(A)$; and
(v) the matrix A sends the vector x_r to $b = Ax \in \text{col}(A)$ and the vector x_n to the zero vector.

In a later chapter, we will refine our language and say that x_r and x_n are the *orthogonal projections* onto $\text{col}(A^T)$ and $\text{null}(A)$, respectively.

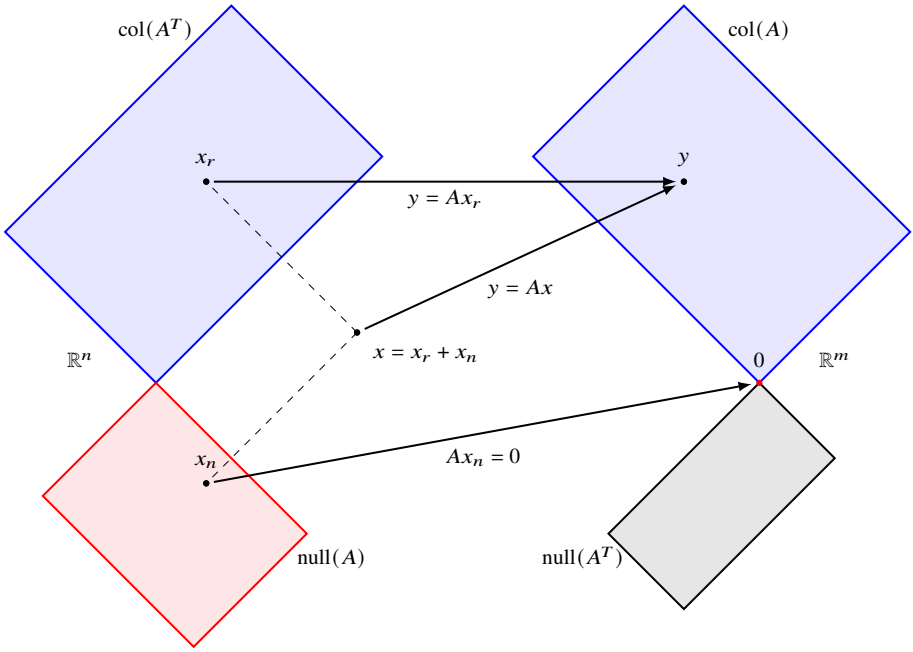


Fig. 27.2: The orthogonality of the four fundamental subspaces of a matrix $A \in \mathbb{R}^{m \times n}$.

[1] This image was popularized by the Linear Algebra maestro, Gilbert Strang.

27.2 Orthogonal matrices and their properties

In this section we focus on a special class of matrices which will be very useful in many numerical algorithms. We start by considering matrices whose columns are orthonormal.

Example 27.13 The following three matrices have orthonormal columns

$$\frac{1}{3}\begin{bmatrix} 2 \\ 2 \\ -1 \end{bmatrix}, \quad \frac{1}{3}\begin{bmatrix} 2 & 2 \\ 2 & -1 \\ -1 & 2 \end{bmatrix}, \quad \text{and} \quad \frac{1}{3}\begin{bmatrix} 2 & 2 & -1 \\ 2 & -1 & 2 \\ -1 & 2 & 2 \end{bmatrix}.$$

The key property of matrices with orthonormal columns

An $m \times n$ matrix Q with orthonormal columns satisfies

$$Q^T Q = I_n,$$

This follows directly from the (row-column) definition of matrix multiplication and the definition of orthonormality. If $Q = \begin{bmatrix} q_1 & \cdots & q_n \end{bmatrix}$, where the columns $q_1, \ldots, q_n$ are orthonormal, then

$$Q^T Q = \begin{bmatrix} q_1^T \\ q_2^T \\ \vdots \\ q_n^T \end{bmatrix} \begin{bmatrix} q_1 & q_2 & \ldots & q_n \end{bmatrix} = \begin{bmatrix} q_1^T q_1 & q_1^T q_2 & \cdots & q_1^T q_n \\ q_2^T q_1 & q_2^T q_2 & \cdots & q_2^T q_n \\ \vdots & \vdots & \ddots & \vdots \\ q_n^T q_1 & q_n^T q_2 & \cdots & q_n^T q_n \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix},$$

because $q_i^T q_j = 0$ for $i \neq j$, and $q_i^T q_i = 1$ for $i = 1, \ldots, n$.

Note that QQ^T is generally not equal to I_m. A matrix with orthonormal columns does not necessarily have orthonormal rows. We will discuss when a matrix has both properties later.

Matrices with orthonormal columns have an important geometric property: they preserve the 2-norm of vectors. Because of this, linear transformations of the form $f(x) = Qx$, where Q has orthonormal columns, are called **linear isometries**[2]. The reverse is also true: every linear isometry corresponds to a matrix with orthonormal columns, as we will show next.

Theorem 27.14. (Linear isometries)

A matrix $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns if and only if Q preserves the 2-norm of vectors, that is,

[2] The word comes from Greek: "iso" means same, and "metron" means measure.

$$\|Qx\|_2 = \|x\|_2, \quad \text{for all vectors } x \in \mathbb{R}^n.$$

Proof First, assume that $Q \in \mathbb{R}^{m\times n}$ has orthonormal columns. Then, for any vector $x \in \mathbb{R}^n$, we have

$$\|Qx\|_2^2 = (Qx)^T Qx = x^T Q^T Qx = x^T x = \|x\|_2^2,$$

where the third equality follows from $Q^T Q = I_n$. Therefore, the linear transformation $f(x) = Qx$ preserves the 2-norm and is a linear isometry.

Next, assume $f(x) = Qx$ is a linear isometry, that is, $\|f(x)\|_2 = \|Qx\|_2 = \|x\|_2$ for all vectors $x \in \mathbb{R}^n$. Let $q_1, q_2, \ldots, q_n$ be the columns of Q. To show these columns are orthonormal, note that for any i we have

$$1 = \|e_i\|_2^2 = \|Qe_i\|_2^2 = (e_i^T Q^T)(Qe_i) = q_i^T q_i = \|q_i\|_2^2,$$

where e_i is the ith standard basis vector, and the second equality follows from Q preserving 2-norms. Hence, each column q_i is a unit vector.

To show the columns are orthogonal, let i and j be distinct indices. Then

$$\begin{aligned} 2 = \|e_i + e_j\|_2^2 = \|Q(e_i + e_j)\|_2^2 &= (Q(e_i + e_j))^T Q(e_i + e_j) \\ &= (q_i^T + q_j^T)(q_i + q_j) \\ &= q_i^T q_i + q_i^T q_j + q_j^T q_i + q_j^T q_j \\ &= 1 + q_i^T q_j + q_j^T q_i + 1 \\ &= 2 + 2q_i^T q_j. \end{aligned}$$

Thus, $q_i^T q_j = 0$, and we conclude that the columns $q_1, q_2, \ldots, q_n$ of Q are orthonormal. □

Example 27.15 Consider the linear transformation $T : \mathbb{R}^2 \mapsto \mathbb{R}^2$ that reflects vectors across the x_1-axis. This transformation is given by

$$T\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Notice that the matrix representation of T has orthonormal columns. It is also clear that T is a linear isometry since reflections preserve the length of vectors.

When does a matrix $Q \in \mathbb{R}^{m\times n}$ with orthonormal columns also have orthonormal rows? In other words, under what conditions does $Q^T Q = I_n$ also imply $QQ^T = I_m$? The following proposition provides the answer.

Proposition 27.16. (Matrices with orthonormal columns and rows)

Let $Q \in \mathbb{R}^{m\times n}$ be a matrix with orthonormal columns. Then, Q has orthonormal rows if and only if Q is square. Moreover, if Q has orthonormal rows, then Q is invertible, and $Q^{-1} = Q^T$.

Proof First, assume that Q has orthonormal rows and columns. Then, Q must have both full column rank and full row rank, since orthonormal sets are linearly independent. Because column rank equals row rank, we have

$$n = \dim(\text{col}(Q)) = \dim(\text{col}(Q^T)) = m,$$

which means Q is a full-rank square matrix and therefore invertible. Moreover, since $QQ^T = Q^TQ = I_n$, we conclude that $Q^{-1} = Q^T$.

Next, assume that Q is a square $n \times n$ matrix with orthonormal columns. Since orthonormal columns are linearly independent, Q has full column rank and is therefore invertible. Since $Q^TQ = I_n$, it follows that $Q^{-1} = Q^T$ (see Problem 17.8). This also implies that $QQ^T = I_n$, which implies that the rows of Q are orthonormal. □

The proposition above leads to the following definition.

Definition 27.17. (Orthogonal matrix)

We call a matrix Q an **orthogonal** if

$$QQ^T = I_n = Q^TQ.$$

Notice that, according to Proposition 27.16, to determine if a matrix $Q \in \mathbb{R}^{n\times n}$ is orthogonal, you only need to check one of the following: it has orthogonal columns, $Q^TQ = I_n$, or it has orthogonal rows, $QQ^T = I_n$.

Let us look at a few examples of orthogonal matrices. You will find more examples in Section 27.3, where we introduce Householder rotators and Givens reflectors, as well as in the Problems section at the end of the chapter.

Example 27.18 The 3×3 matrix

$$Q = \frac{1}{3}\begin{bmatrix} 2 & 2 & -1 \\ 2 & -1 & 2 \\ -1 & 2 & 2 \end{bmatrix}$$

is orthogonal. To confirm this, we just need to check that $Q^TQ = I_3$, since Q is square.

Example 27.19 The n columns of the identity matrix I_n are orthonormal, so I_n is an orthogonal matrix. More generally, any $n \times n$ permutation matrix P is also orthogonal, since it has the same columns as the identity matrix, just in a different order.

Example 27.20 The matrix

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

is an orthogonal matrix. This matrix represents a counterclockwise rotation by $\frac{\pi}{2}$.

The following theorem provides several important characterizations of orthogonal matrices.

Theorem 27.21. (Characterizations of orthogonal matrices)

For a matrix $Q \in \mathbb{R}^{n \times n}$, the following statements are equivalent.

(a) Q is orthogonal, that is, $Q^T Q = I_n = QQ^T$.
(b) Q^T is orthogonal
(c) Q is invertible with $Q^{-1} = Q^T$.
(d) The columns of Q form an orthonormal set, that is, $Q^T Q = I_n$.
(e) The rows of Q form an orthonormal set, that is, $QQ^T = I_n$.
(f) For each $x \in \mathbb{R}^n$, $\|x\|_2 = \|Qx\|_2$, that is, Q preserves the 2-norm and, thus, the Euclidean distance.
(g) For each $x, y \in \mathbb{R}^n$, $(Qx)^T(Qy) = x^T y$, that is, Q preserves dot products and, thus, angles.

Proof That (a) and (b) are equivalent is clear.

The equivalence of (a) and (c) follows directly from the definitions of orthogonal matrix and inverse.

Both (d) and (e) are equivalent to (c) by the results from Problem 17.8.

That (a) implies (f) follows from Theorem 27.14.

To show that (f) implies (g), we will use the "Polarization Identity" (see Problem 27.6):

$$x^T y = \frac{1}{4}\|x + y\|^2 - \frac{1}{4}\|x - y\|^2.$$

Now, we have

$$\begin{aligned}(Qx)^T(Qy) &= \frac{1}{4}\|Qx+Qy\|^2 - \frac{1}{4}\|Qx-Qy\|^2 \\ &= \frac{1}{4}\|Q(x+y)\|^2 - \frac{1}{4}\|Q(x-y)\|^2 \\ &= \frac{1}{4}\|x+y\|^2 - \frac{1}{4}\|x-y\|^2 = x^T y.\end{aligned}$$

Thus, Q preserves dot products.

Finally, let us show that (g) implies (a). Assume that $(Qx)^T(Qy) = x^T y$ for all $x, y \in \mathbb{R}^n$. This is equivalent to $x^T Q^T Q y = x^T y$ for all x, y. In particular, if e_i denotes the ith column of the identity matrix, we obtain $(Q^T Q)_{ij} = e_i^T (Q^T Q) e_j = e_i^T e_j = \delta_{ij}$.[3] This gives us $Q^T Q = I_n$, so by Proposition 27.16, Q is orthogonal. □

The product of orthogonal matrices remains orthogonal, as we show next.

Theorem 27.22. (Products of orthogonal matrices)

The product $Q = Q_1 Q_2 \cdots Q_k$ of any finite number of orthogonal matrices Q_i is also an orthogonal matrix.

Proof The product of $n \times n$ orthogonal matrices $Q_1, Q_2, \ldots, Q_k$ is itself an $n \times n$ matrix. We prove that $Q^T Q = I_n$ by induction on k.

For $k = 1$, the result is clear. Assume the statement is true for the product of $k-1$ orthogonal matrices. Now, assume $Q_1, \ldots, Q_k$ are orthogonal matrices and let $Q = Q_1 \cdots Q_k$. Then,

$$\begin{aligned}Q^T Q &= (Q_1 Q_2 \cdots Q_k)^T (Q_1 Q_2 \cdots Q_k) \\ &= Q_k^T (Q_1 \cdots Q_{k-1})^T (Q_1 \cdots Q_{k-1}) Q_k \\ &= Q_k^T Q_k = I_n,\end{aligned}$$

where the third equality uses the inductive hypothesis, and the final equality follows from Q_k being orthogonal. □

Proposition 27.23. (The determinant of an orthogonal matrix)

For any orthogonal matrix Q,

$$\det(Q) = \pm 1.$$

Proof Let Q be an orthogonal matrix. Since $Q^T Q = I_n$, using the multiplicative property of the determinant, we obtain $\det(Q^T Q) = \det(Q^T)\det(Q) = \det(Q)^2 = 1$, which implies $\det(Q) = \pm 1$. □

[3] δ_{ij} is the Kronecker delta, which equals 1 if $i = j$ and 0 otherwise.

Based on Proposition 27.23, orthogonal matrices fall into two groups: those with determinant +1, called **proper orthogonal matrices**, and those with determinant −1. We can turn any proper orthogonal matrix into a non-proper one, and vice versa, by multiplying any of its columns by −1.

We can extend the concept of a (real) orthogonal matrix to complex matrices.

Definition 27.24. (Unitary matrices)

A matrix $U \in \mathbb{C}^{n\times n}$ is a **unitary matrix** if $U^*U = I_n$, where U^* denotes the complex conjugate transpose of U.

Unitary matrices play an important role in Chapter 37, where we discuss eigenvalues and eigenvectors.

27.3 Givens rotators and Householder reflectors

In this section, we introduce two important types of orthogonal matrices that we will use extensively in this and future chapters: **Givens rotators**, also known as plane rotation matrices, and **Householder reflectors**, also called elementary reflection matrices.[4]

Let us start with the following question: what do 2×2 orthogonal matrices Q look like? Let us denote by q_1 and q_2 the columns of such a matrix.

We know that q_1 is a unit vector in $\mathbb{R}^2$, and any unit vector in $\mathbb{R}^2$ lies on the unit circle centered at the origin. Hence, we must have

$$q_1 = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix} \qquad \text{for some } 0 \leqslant \theta < 2\pi.$$

The second column of Q is a unit vector orthogonal to the first column, so we have two choices for it, either

$$q_2 = \begin{bmatrix} \cos(\theta + \pi/2) \\ \sin(\theta + \pi/2) \end{bmatrix} = \begin{bmatrix} -\sin(\theta) \\ \cos(\theta) \end{bmatrix} \qquad \text{or} \qquad q_2 = \begin{bmatrix} \cos(\theta - \pi/2) \\ \sin(\theta - \pi/2) \end{bmatrix} = \begin{bmatrix} \sin(\theta) \\ -\cos(\theta) \end{bmatrix}.$$

In other words, 2×2 orthogonal matrices are of the form

$$Q = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \qquad \text{or} \qquad Q = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ \sin(\theta) & -\cos(\theta) \end{bmatrix}. \tag{27.1}$$

Both matrices in (27.1) have a geometric interpretation. The first rotates vectors counterclockwise by an angle θ (see Problem 27.13). The second reflects vectors across a line through the origin that forms an angle of $\theta/2$ with the x_1-axis (see Problem 27.14). You can tell them apart by their determinants: the first has determinant +1, so it is a proper orthogonal matrix, while the second has determinant −1.

[4] These matrices are named after Wallace Givens and Alston Householder, who pioneered their use in Numerical Linear Algebra.

A 2×2 orthogonal matrix with determinant -1 represents a linear transformation that *reverses the orientation of space*. Imagine, for example, that A is a $2 \times n$ data matrix and the scatter plot of the n columns of A depicts a right hand, as in Figure 27.3 in blue. Then, the scatter plot of the $2 \times n$ matrix QA will depict a left hand (in red in Figure 27.3).

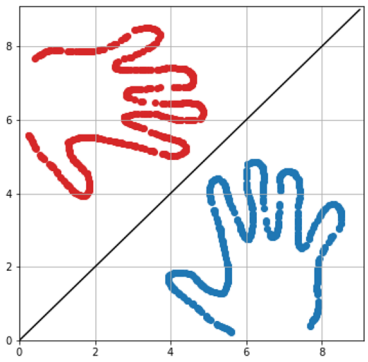


Fig. 27.3: Orthogonal matrices with negative determinant reverse the orientation of space as linear transformations. For example, when a right hand (in blue) is reflected across a line, it produces a left hand (in red).

In the next two sections, we will extend the two orthogonal matrices from (27.1) to any dimension n and study their properties.

27.3.1 Givens rotators

We can generalize the first orthogonal matrix in (27.1) to any dimension n as follows.

Definition 27.25. (Givens rotators)

A **Givens rotator** is an $n \times n$ matrix of the form

$$
\begin{array}{c}
\qquad\qquad\quad \text{col } i \qquad\qquad\quad \text{col } j \\
\begin{bmatrix}
1 & & & & & & & \\
& \ddots & & & & & & \\
& & 1 & & & & & \\
& & & \cos(\theta) & & -\sin(\theta) & & \text{row i} \\
& & & & 1 & & & \\
& & & & & \ddots & & \\
& & & & & & 1 & \\
& & & \sin(\theta) & & \cos(\theta) & & \text{row j} \\
& & & & & & & 1 \\
& & & & & & & & \ddots \\
& & & & & & & & & 1
\end{bmatrix}
\end{array}, \tag{27.2}
$$

for some $\theta \in [0, 2\pi)$, where all the entries that have not been filled in are zeros.

Multiplying the Givens rotator (27.2) by a vector x rotates the i-th and j-th components of x counterclockwise by θ radians in the (x_i, x_j)-plane, leaving the other components unchanged. This explains why Givens matrices are also called "plane rotators". The example below illustrates this fact for a 3×3 Givens rotator.

Example 27.26. (Three dimensional Givens rotators)

When $n = 3$, there are three types of Givens rotators:

$$
\begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} \cos(\theta) & 0 & -\sin(\theta) \\ 0 & 1 & 0 \\ \sin(\theta) & 0 & \cos(\theta) \end{bmatrix}.
$$

Figure 27.4 shows how these Givens rotators act on a cloud of points. The first type rotates the (x_1, x_2)-plane while keeping the x_3 component fixed (left panel). The second type rotates the (x_2, x_3)-plane while keeping the x_1 component fixed (middle panel). The third type rotates the (x_1, x_3)-plane while keeping the x_2 component fixed (right panel).

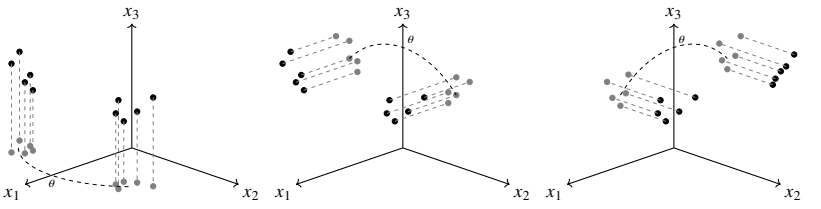


Fig. 27.4: The action of the three types of 3×3 Givens rotators on a cloud of five points.

The following theorem presents some basic properties of Givens rotators. The proof is left as an exercise.

Theorem 27.27. (Basic properties of Givens rotators)

Consider a Givens rotator Q as in (27.2). Then,

(a) Q is orthogonal, that is, $Q^T Q = I_n = QQ^T$, and
(b) $\det(Q) = 1$.

One of the main uses of Givens rotators is to introduce zeros into vectors and matrices. Let us start with the two-dimensional case.

Proposition 27.28. (Producing zeros in 2-dimensional vectors with Givens rotators)

For any vector $x \in \mathbb{R}^2$, there exists a Givens rotator Q such that

$$Q\,x = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ 0 \end{bmatrix} \quad \text{with } y_1 = \pm\|x\|_2. \tag{27.3}$$

Proof If $x = 0$, any Givens rotator Q satisfies (27.3). Hence, let us assume $x \neq 0$. Then, (27.3) requires

$$x_1 \sin(\theta) + x_2 \cos(\theta) = 0. \tag{27.4}$$

First, let us consider the case where $x_1 = 0$. In this case, $x_2 \neq 0$, and from (27.4), we get $\cos(\theta) = 0$. Thus, $\sin(\theta) = \pm 1$, and the Givens rotator becomes

$$Q = \begin{bmatrix} 0 & \pm 1 \\ \mp 1 & 0 \end{bmatrix}.$$

Now observe that

$$Q\,x = \begin{bmatrix} \pm x_2 \\ 0 \end{bmatrix} = \begin{bmatrix} \pm\|x\|_2 \\ 0 \end{bmatrix},$$

as we wanted.

Now, let us consider the case where $x_1 \neq 0$. From (27.4), we have

$$\theta = \arctan\left(-\frac{x_2}{x_1}\right).$$

Thus,

$$\cos(\theta) = \pm\frac{x_1}{\|x\|_2} \quad \text{and} \quad \sin(\theta) = \mp\frac{x_2}{\|x\|_2}.$$

One choice of signs gives the Givens rotator

$$Q = \frac{1}{\|x\|_2}\begin{bmatrix} x_1 & x_2 \\ -x_2 & x_1 \end{bmatrix},$$

which satisfies

$$Qx = \frac{1}{\|x\|_2}\begin{bmatrix} x_1 & x_2 \\ -x_2 & x_1 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \|x\|_2 \\ 0 \end{bmatrix}.$$

The other choice of signs gives the Givens rotator

$$Q = \frac{1}{\|x\|_2}\begin{bmatrix} -x_1 & -x_2 \\ x_2 & -x_1 \end{bmatrix},$$

which satisfies

$$Qx = \frac{1}{\|x\|_2}\begin{bmatrix} -x_1 & -x_2 \\ x_2 & -x_1 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} -\|x\|_2 \\ 0 \end{bmatrix}.$$

In both cases, we get the desired result. □

By the preceding result, you can always rotate any vector in $\mathbb{R}^2$ so that it lines up with the x_1-axis. Because rotations preserve lengths, the rotated vector has the same 2-norm as the original one.

Example 27.29 Consider the vector $x = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$. According to Proposition 27.28, the two Givens rotators that zero out the second component of x are

$$\frac{1}{\sqrt{13}}\begin{bmatrix} 2 & -3 \\ 3 & 2 \end{bmatrix} \quad \text{and} \quad \frac{1}{\sqrt{13}}\begin{bmatrix} -2 & 3 \\ -3 & -2 \end{bmatrix}.$$

Multiplying x by each of these Givens rotators gives

$$\frac{1}{\sqrt{13}}\begin{bmatrix} 2 & -3 \\ 3 & 2 \end{bmatrix}\begin{bmatrix} 2 \\ -3 \end{bmatrix} = \begin{bmatrix} \sqrt{13} \\ 0 \end{bmatrix} \quad \text{and} \quad \frac{1}{\sqrt{13}}\begin{bmatrix} -2 & 3 \\ -3 & -2 \end{bmatrix}\begin{bmatrix} 2 \\ -3 \end{bmatrix} = \begin{bmatrix} -\sqrt{13} \\ 0 \end{bmatrix}.$$

We can use a sequence of Givens rotators to introduce zeros in vectors of any length.

Theorem 27.30. (Introducing zeros into vectors with Givens rotators)

For any vector $x \in \mathbb{R}^n$, there is a sequence of $n-1$ Givens rotators $Q_1, Q_2, \ldots, Q_{n-1}$ such that the vector

$$Q_{n-1}\cdots Q_2 Q_1 x = Q_{n-1}\cdots Q_2 Q_1 \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} \pm\|x\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Proof From Proposition 27.28, we know there is an angle θ_1 such that

$$\begin{bmatrix} \cos(\theta_1) & -\sin(\theta_1) \\ \sin(\theta_1) & \cos(\theta_1) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ 0 \end{bmatrix} \qquad \text{for some } y_1.$$

We can then build a Givens rotator Q_1 acting on the (x_1, x_2)-plane to create a zero in the second component of x as follows:

$$\begin{bmatrix} \cos(\theta_1) & -\sin(\theta_1) & & & & \\ \sin(\theta_1) & \cos(\theta_1) & & & & \\ & & 1 & & & \\ & & & 1 & & \\ & & & & \ddots & \\ & & & & & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ 0 \\ x_3 \\ x_4 \\ \vdots \\ x_n \end{bmatrix}.$$

Similarly, there exists an angle θ_2 such that

$$\begin{bmatrix} \cos(\theta_2) & -\sin(\theta_2) \\ \sin(\theta_2) & \cos(\theta_2) \end{bmatrix} \begin{bmatrix} y_1 \\ x_3 \end{bmatrix} = \begin{bmatrix} z_1 \\ 0 \end{bmatrix} \qquad \text{for some } z_1.$$

We then build a second Givens rotator Q_2 acting on the (x_1, x_3)-plane to produce a zero in the third component of $Q_1 x$ while preserving the zero created by Q_1:

$$\begin{bmatrix} \cos(\theta_2) & 0 & -\sin(\theta_2) & & & \\ 0 & 1 & 0 & & & \\ \sin(\theta_2) & 0 & \cos(\theta_2) & & & \\ & & & 1 & & \\ & & & & \ddots & \\ & & & & & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ 0 \\ x_3 \\ x_4 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} z_1 \\ 0 \\ 0 \\ x_4 \\ \vdots \\ x_n \end{bmatrix}.$$

We continue this process, constructing a sequence of Givens rotators $Q_1, Q_2, \ldots, Q_{n-1}$, each zeroing an entry while preserving the previously created zeros. We have

$$Q_{n-1} \cdots Q_3 Q_2 Q_1 \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_n \end{bmatrix} = Q_{n-1} \cdots Q_3 Q_2 \begin{bmatrix} \star \\ 0 \\ \star \\ \star \\ \vdots \\ \star \end{bmatrix} =$$

$$Q_{n-1} \cdots Q_3 \begin{bmatrix} \star \\ 0 \\ 0 \\ \star \\ \vdots \\ \star \end{bmatrix} = \cdots = Q_{n-1} \begin{bmatrix} \star \\ 0 \\ 0 \\ \vdots \\ 0 \\ \star \end{bmatrix} = \begin{bmatrix} \star \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix},$$

as desired.

Since each Givens rotator is an orthogonal matrix, the 2-norm of the vector is preserved at every step. Therefore, the resulting vector $Q_{n-1} \cdots Q_1 x$ has the same 2-norm as x, which implies its top entry $\pm\|x\|_2$. □

27.3.2 Householder reflectors

We can generalize the second orthogonal matrix in (27.1) to any dimension n as follows.

Definition 27.31. (Householder reflectors)

A **Householder reflector** is an $n \times n$ matrix of the form

$$Q = I_n - 2\tau uu^T, \quad \text{with} \quad \tau = \frac{1}{\|u\|_2^2},$$

for some nonzero vector u or, equivalently,

$$Q = I_n - 2uu^T, \tag{27.5}$$

when u is a unit vector.

At first glance, it might not be clear how a Householder reflector generalizes the 2×2 orthogonal matrix

$$Q = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ \sin(\theta) & -\cos(\theta) \end{bmatrix}.$$

However, using the trigonometric identities

$$\begin{aligned} \sin(\theta) &= 2\sin(\theta/2)\cos(\theta/2), \quad \text{and} \\ \cos(\theta) &= 1 - 2\sin^2(\theta/2) = 2\cos^2(\theta/2) - 1, \end{aligned}$$

we can rewrite Q as

$$\begin{aligned} Q = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ \sin(\theta) & -\cos(\theta) \end{bmatrix} &= \begin{bmatrix} 1 - 2\sin^2(\theta/2) & 2\sin(\theta/2)\cos(\theta/2) \\ 2\sin(\theta/2)\cos(\theta/2) & 1 - 2\cos^2(\theta/2) \end{bmatrix} = \\ &\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 2\begin{bmatrix} \sin^2(\theta/2) & -\sin(\theta/2)\cos(\theta/2) \\ -\sin(\theta/2)\cos(\theta/2) & \cos^2(\theta/2) \end{bmatrix} = \\ &\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 2\begin{bmatrix} -\sin(\theta/2) \\ \cos(\theta/2) \end{bmatrix}\begin{bmatrix} -\sin(\theta/2) & \cos(\theta/2) \end{bmatrix} = \\ &I_2 - 2uu^T. \end{aligned}$$

Since the vector $u = \begin{bmatrix} -\sin(\theta/2) \\ \cos(\theta/2) \end{bmatrix}$ has unit norm, this shows that Q is a Householder reflector.

The following theorem presents some basic properties of Householder reflectors.

Theorem 27.32. (Basic properties of Householder reflectors)

Consider a Householder reflector $Q = I_n - 2uu^T$, where u is a unit vector. Then,

(a) Q is symmetric, that is, $Q^T = Q$,
(b) Q is orthogonal, that is, $Q^TQ = I_n = QQ^T$,
(c) Q is its own inverse, that is, $Q^2 = I_n$, and
(d) $\det(Q) = -1$.

Proof The transpose of the Householder reflector Q is $Q^T = (I_n - 2uu^T)^T = I_n - 2(uu^T)^T = I_n - 2uu^T = Q$. Hence, Q is a symmetric matrix (part (a)). Moreover, since u is a unit vector (i.e., $u^Tu = 1$) and Q is symmetric, we have

$$\begin{aligned} Q^TQ = QQ^T = Q^2 = (I_n - 2uu^T)(I_n - 2uu^T) = & I_n - 4uu^T + 4uu^Tuu^T = \\ & I_n - 4uu^T + 4u(u^Tu)u^T = \\ & I_n - 4uu^T + 4uu^T = I_n, \end{aligned}$$

which establishes, first, that Q is orthogonal (part (b)), and, second, that Q is its own inverse (part (c)).

Finally, part (d) follows from the rank-one-update determinant formula $\det(I_n + tab^T) = 1 + tb^Ta$, for any two vectors $a, b \in \mathbb{R}^n$ and scalar $t \in \mathbb{R}$ (see Problem 27.25 for a proof of this). □

We can interpret the geometric action of a Householder reflector $Q = I_n - 2uu^T$ as follows: The product Qx reflects x across the hyperplane $\text{span}(u)^\perp$ (the orthogonal complement of $\text{span}(u)$), as we show below.

The geometric action of a Householder reflector

Let u be a unit vector in $\mathbb{R}^n$ and let $\text{span}(u)^\perp$ be the orthogonal complement of $\text{span}(u)$, the subspace of dimension $n-1$ containing all the vectors in $\mathbb{R}^n$ orthogonal to u. In Figure 27.5, $\text{span}(u)^\perp$ is represented as the line orthogonal to u passing through the origin.

Let x be a vector in $\mathbb{R}^n$. Since $\mathbb{R}^n = \text{span}(u) \oplus \text{span}(u)^\perp$, we can decompose the vector x as the sum of two vectors $x = p_1 + p_2$, where $p_1 \in \text{span}(u)$ and $p_2 \in \text{span}(u)^\perp$.

Since $p_1 \in \text{span}(u)$, we have $p_1 = \alpha u$ for some scalar α. Thus, since u and p_2 are orthogonal,

$$u^Tx = u^T(\alpha u) + u^Tp_2 = \alpha u^Tu = \alpha.$$

This implies

$$p_1 = (u^Tx)u = uu^Tx \qquad \text{and} \qquad p_2 = x - p_1 = x - uu^Tx.$$

Now we calculate the reflection y of $x = p_1 + p_2$ across the subspace $\text{span}(u)^\perp$. To reflect x, we keep the component p_2 unchanged and flip the sign of p_1. This gives us

$$y = p_2 - p_1 = x - (uu^T)x - (uu^T)x = x - 2uu^Tx = (I_n - 2uu^T)x.$$

In other words, the vector y is the product of the Householder reflector $Q = I_n - 2uu^T$ times the vector x.

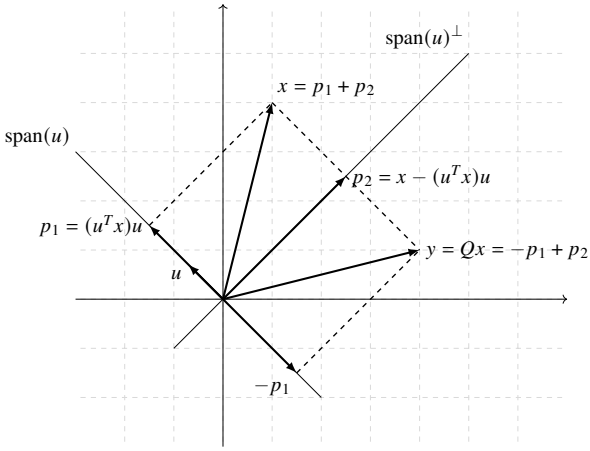


Fig. 27.5: The action of a Householder reflector on a vector x: every vector x can be decomposed as $x = p_1 + p_2$, where $p_1 \in \text{span}(u)$ and $p_2 = \text{span}(u)^\perp$. The Householder reflector $Q = I_n - 2uu^T$ leaves p_2 unchanged and flips the sign of p_1.

Like Givens rotators, one of the main uses of Householder reflectors is to introduce zeros into vectors and matrices. Before explaining how this is achieved, we need the following result.

Theorem 27.33. (Householder reflector theorem)

For any two nonzero vectors $x, y \in \mathbb{R}^n$ with $\|x\|_2 = \|y\|_2$, there is a unique Householder reflector Q such that $y = Qx$.

To build some intuition for the proof of Theorem 27.33, let us first look at the two-dimensional case. Figure 27.6 suggests that we can transform the vector x into y by reflecting x across the line that bisects the angle between them. We find a unit vector orthogonal to this bisector by normalizing the difference between x and y, i.e., $u = (x - y)/\|x - y\|_2$.

Proof (of Theorem 27.33) Consider the unit vector $u = (x - y)/\|x - y\|_2$. First, note

$$\|x - y\|_2^2 = (x - y)^T(x - y) = x^T x - 2y^T x + y^T y = 2(x^T x + y^T x),$$

where we used $\|x\|_2^2 = x^T x = \|y\|_2^2 = y^T y$. Now, using $u = (x - y)/\|x - y\|_2$, we build the Householder reflector $Q = I_n - 2uu^T$. Then, we have,

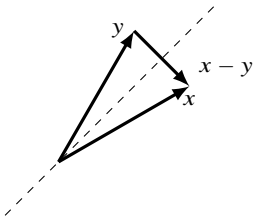


Fig. 27.6: Reflecting the vector x across the line that bisects the angle between x and y transforms x into y.

$$\begin{aligned} Qx = (I_n - 2uu^T)x &= x - 2uu^T x \\ &= x - 2u(u^T x) \\ &= x - 2\frac{x-y}{\|x-y\|_2}\left(\frac{x-y}{\|x-y\|_2}\right)^T x \\ &= x - \frac{2}{\|x-y\|_2^2}(x-y)(x-y)^T x \\ &= x - \frac{2}{\|x-y\|_2^2}(x-y)(x^T x - y^T x) \\ &= x - (x-y) = y. \end{aligned}$$

This shows that the Householder reflector works as intended.

To prove the uniqueness of the Householder reflector, assume there are two reflectors, $Q = I_n - 2uu^T$ and $\widehat{Q} = I_n - 2vv^T$, where u and v are unit vectors, such that $Qx = y = \widehat{Q}x$. Then we can write

$$\begin{aligned} 0 &= (Q - \widehat{Q})x \\ &= (-2uu^T + 2vv^T)x \\ &= -2(u^T x)u + 2(v^T x)v. \end{aligned}$$

This simplifies to $(u^T x)u = (v^T x)v$. Since u and v are unit vectors, the only way this equation holds is if v is a multiple of u, because

$$|u^T x| = \|(u^T x)u\|_2 = \|(v^T x)v\|_2 = |v^T x|,$$

which implies

$$v = \frac{u^T x}{v^T x}u = \pm u.$$

Thus, we can write $\widehat{Q} = I_n - 2vv^T = I_n - 2(\pm u)(\pm u)^T = I_n - 2uu^T = Q$. This proves that the Householder reflector is unique. □

Unlike Givens rotators, a single Householder reflector can zero out all but the first entry of a vector.

Theorem 27.34. (Producing zeros in vectors with Householder reflectors)

For any vector $x \in \mathbb{R}^n$, there exists a Householder reflector Q such that

$$Qx = \begin{bmatrix} \pm\|x\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Proof If $x = 0$, any Householder reflector works. For $x \neq 0$, apply Theorem 27.33 with $y = \pm\|x\|_2 e_1$. □

Example 27.35 Let $x = \begin{bmatrix} 2 & 1 & 0 & 2 \end{bmatrix}^T \in \mathbb{R}^4$. We aim to find a Householder reflector that takes x to $y = \|x\|_2 e_1 = 3e_1$. Consider the vector

$$u = \frac{x - y}{\|x - y\|_2} = \frac{1}{\sqrt{6}} \begin{bmatrix} -1 \\ 1 \\ 0 \\ 2 \end{bmatrix}.$$

Let

$$Q = I_4 - 2uu^T = \begin{bmatrix} 2/3 & 1/3 & 0 & 2/3 \\ 1/3 & 2/3 & 0 & -2/3 \\ 0 & 0 & 1 & 0 \\ 2/3 & -2/3 & 0 & -1/3 \end{bmatrix}.$$

A simple computation shows that $Qx = y$.

In Section 28.3, we will see how to use Theorem 27.34 to reduce an arbitrary matrix to a matrix in either upper Hessenberg or bidiagonal form. These transformations significantly decrease the computational complexity of algorithms for computing eigenvalues (Chapter 37) and the singular value decomposition (Chapter 47) of a matrix.

27.4 The QR factorization

In this section, we will see how to factor any matrix $A \in \mathbb{R}^{m \times n}$ as $A = QR$, where Q is an orthogonal matrix and R is upper triangular (i.e. $R_{ij} = 0$ for $i > j$). This is the **QR factorization**, one of the most useful matrix factorizations. We will use it to solve least squares problems and compute eigenvalues in Chapters 32 and 37, respectively.

27.4.1 The full column rank case

We will first consider a tall-and-skinny $m \times n$ matrix A (more rows than columns) with linearly independent columns, so it has full column rank[5]. The first goal is to factor A as $A = QR$, where Q has orthonormal columns and R is upper triangular and invertible. Notice that this factorization is slightly different than the one described at the beginning of the section since Q is not assumed to be square but R is.

Definition 27.36. (Reduced QR factorization)

Consider an $m \times n$ matrix A with full column rank. We say that $A = QR$, with $Q \in \mathbb{R}^{m\times n}$ and $R \in \mathbb{R}^{n\times n}$, is a **reduced QR factorization** of A if

(i) Q has orthonormal columns, and
(ii) R is upper triangular.

The reason for calling this factorization "reduced" will become clearer later when we introduce the full QR factorization.

Example 27.37 Consider the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

A reduced QR factorization of A is

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} = \underbrace{\begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{6} \\ 1/\sqrt{2} & -1/\sqrt{6} \\ 0 & 2/\sqrt{6} \end{bmatrix}}_{=:Q} \underbrace{\begin{bmatrix} \sqrt{2} & 1/\sqrt{2} \\ 0 & 3/\sqrt{6} \end{bmatrix}}_{=:R} = QR,$$

because the columns of Q are orthonormal and R is upper triangular.

Notice that the upper triangular matrix R in Example 27.40 is invertible because its diagonal entries are nonzero. This is not a coincidence. The next theorem shows that in any reduced QR factorization of a full column rank matrix A, the matrix R must be invertible.

[5] This case is especially important for least squares problems, as we will see in Chapter 32.

Theorem 27.38. (Properties of a reduced QR factorization of a full column rank matrix)

Let $A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \in \mathbb{R}^{m \times n}$ be a matrix with linearly independent columns $a_1, a_2, \ldots, a_n$, and let $A = QR$ be a reduced QR factorization of A. Then, the following statements hold:

(a) The matrix R is invertible.
(b) If $Q = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}$, then

$$\operatorname{span}(q_1, q_2, \ldots, q_i) = \operatorname{span}(a_1, a_2, \ldots, a_i), \quad i = 1, \ldots, n.$$

In particular, $\operatorname{col}(Q) = \operatorname{col}(A)$.

Proof Assume R is singular. Since A has full column rank, by Theorem 17.39,

$$n = \operatorname{rank}(A) = \operatorname{rank}(QR) \leqslant \operatorname{rank}(R) < n,$$

a contradiction.

The invertibility of R implies that all its main diagonal elements are nonzero. From the first i columns of $A = QR$, where $1 \leqslant i \leqslant n$, we have

$$A[:, 1:i] = Q[:, 1:i]R[1:i, 1:i].$$

Since $R[1:i, 1:i]$ is upper triangular with nonzero diagonal elements, it is invertible. By Theorem 17.11, we have

$$\operatorname{span}(a_1, a_2, \ldots, a_i) = \operatorname{col}(A[:, 1:i]) = \operatorname{col}(Q[:, 1:i]) = \operatorname{span}(q_1, q_2, \ldots, q_i),$$

as we wanted to show. □

Part (b) of Theorem 27.38 should remind you of the Gram-Schmidt process from Chapter 26. In fact, we can use Gram-Schmidt, which turns a linearly independent set into an orthonormal one, to show that QR factorizations exist, as we will do in the next proof.

Theorem 27.39. (Gram-Schmidt finds a reduced QR factorization)

Let $A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \in \mathbb{R}^{m \times n}$ be a matrix with full column rank. Suppose the Gram-Schmidt process transforms the linearly independent columns $\{a_1, a_2, \ldots, a_n\}$ into the orthonormal set $\{q_1, q_2, \ldots, q_n\}$, and define $r_{ij} := q_i^T a_j$. Then,

$$A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & r_{nn} \end{bmatrix} =: QR \tag{27.6}$$

is a reduced QR factorization of A. Moreover, all the diagonal entries $r_{11}, \ldots, r_{nn}$ of the upper triangular factor R are positive.

Proof Recall that the Gram-Schmidt process computes the orthonormal vectors $q_1, \ldots, q_n$ as follows:

$$\begin{aligned}
v_1 &= a_1 && q_1 = \frac{v_1}{\|v_1\|_2} \\
v_2 &= a_2 - (q_1^T a_2) q_1 && q_2 = \frac{v_2}{\|v_2\|_2} \\
v_3 &= a_3 - (q_1^T a_3) q_1 - (q_2^T a_3) q_2 && q_3 = \frac{v_3}{\|v_3\|_2} \\
&\vdots \\
v_n &= a_n - (q_1^T a_n) q_1 - \cdots - (q_{n-1}^T a_n) q_{n-1} && q_n = \frac{v_n}{\|v_n\|_2}.
\end{aligned}$$

Moreover, since $\|v_i\|_2 \neq 0$, for $i = 1, \ldots, n$, and the set $\{q_1, \ldots, q_n\}$ is orthonormal, we get

$$r_{ii} = q_i^T a_i = q_i^T \left(v_i + (q_1^T a_1) q_1 + \cdots + (q_{i-1}^T a_i) q_{i-1} \right) = q_i^T v_i = \|v_i\|_2 > 0.$$

Now, we express the columns of A as combinations of the orthonormal vectors $q_1, \ldots, q_n$ from the Gram-Schmidt process:

$$\begin{aligned}
a_1 &= r_{11} q_1 \\
a_2 &= r_{12} q_1 + r_{22} q_2 \\
a_3 &= r_{13} q_1 + r_{23} q_2 + r_{33} q_3 \\
&\vdots \\
a_n &= r_{1n} q_1 + r_{2n} q_2 + \cdots + r_{nn} q_n,
\end{aligned}$$

which we write in matrix form as (27.6). The matrix Q has orthonormal columns by construction, and the matrix R is upper triangular, so (27.6) gives a reduced QR factorization of A.

Finally, the diagonal entries of R are $r_{ii} = \|v_i\|_2 > 0$, for $i = 1, \ldots, n$. □

Example 27.40 Let us use the Gram-Schmidt process to find a reduced QR factorization of the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

To get the first column q_1 of Q, we normalize the first column a_1 of A:

$$q_1 = \frac{a_1}{\|a_1\|_2} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

Next, we compute

$$r_{11} = \|v_1\|_2 = \|a_1\|_2 = \sqrt{2}, \quad \text{and}$$
$$r_{12} = q_1^T a_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}.$$

For the second column q_2 of Q, we first calculate

$$v_2 = a_2 - r_{12} q_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1/2 \\ -1/2 \\ 1 \end{bmatrix},$$

and then normalize it to get

$$q_2 = \frac{v_2}{\|v_2\|_2} = \frac{1}{\sqrt{6}} \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix}.$$

Finally, we compute

$$r_{22} = \|v_2\|_2 = \sqrt{\frac{3}{2}}.$$

Hence, the Q and R factors are

$$Q = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{6} \\ 1/\sqrt{2} & -1/\sqrt{6} \\ 0 & 2/\sqrt{6} \end{bmatrix} \qquad \text{and} \qquad R = \begin{bmatrix} \sqrt{2} & 1/\sqrt{2} \\ 0 & \sqrt{3/2} \end{bmatrix}.$$

The reduced QR factorization of a full column rank matrix A is not unique. For example, we can multiply any column of Q and the corresponding row of R by -1 and still get a valid QR factorization. The following example shows this.

Example 27.41 The following are reduced QR factorizations of the matrix A from Example 27.40. The first is the one we found earlier. The others are created by multiplying one or two columns of Q and the corresponding rows of R by -1.

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{6} \\ 1/\sqrt{2} & -1/\sqrt{6} \\ 0 & 2/\sqrt{6} \end{bmatrix} \begin{bmatrix} \sqrt{2} & 1/\sqrt{2} \\ 0 & \sqrt{3/2} \end{bmatrix} = \begin{bmatrix} -1/\sqrt{2} & 1/\sqrt{6} \\ -1/\sqrt{2} & -1/\sqrt{6} \\ 0 & 2/\sqrt{6} \end{bmatrix} \begin{bmatrix} -\sqrt{2} & -1/\sqrt{2} \\ 0 & \sqrt{3/2} \end{bmatrix}$$
$$= \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{6} \\ 1/\sqrt{2} & 1/\sqrt{6} \\ 0 & -2/\sqrt{6} \end{bmatrix} \begin{bmatrix} \sqrt{2} & 1/\sqrt{2} \\ 0 & -\sqrt{3/2} \end{bmatrix} = \begin{bmatrix} -1/\sqrt{2} & -1/\sqrt{6} \\ -1/\sqrt{2} & 1/\sqrt{6} \\ 0 & -2/\sqrt{6} \end{bmatrix} \begin{bmatrix} -\sqrt{2} & -1/\sqrt{2} \\ 0 & -\sqrt{3/2} \end{bmatrix}.$$

The following theorem shows that flipping the signs of columns and rows is the only freedom we have to produce distinct QR factorizations of a matrix.

Theorem 27.42. (Characterizations of all reduced QR factorizations)

Let A be a full column rank $m \times n$ matrix. If $A = Q_1R_1 = Q_2R_2$ are two reduced QR factorizations of A, then $Q_2 = Q_1D$ and $R_2 = DR_1$, where D is a diagonal matrix with diagonal entries equal to ± 1.

Proof Recall that the upper triangular factor in a reduced QR factorization must be invertible (see Theorem 27.38), so both R_1 and R_2 are invertible.

Multiplying $Q_1R_1 = Q_2R_2$ on the right by R_2^{-1} and on the left by Q_1^T, we get

$$R_1R_2^{-1} = Q_1^TQ_2,$$

which shows that $Q_1^TQ_2$ is upper triangular because both R_1 and R_2^{-1} are upper triangular. Similarly, multiplying $Q_1R_1 = Q_2R_2$ on the right by R_1^{-1} and on the left by Q_2^T, we get

$$Q_1^TQ_2 = R_1^{-T}R_2^T,$$

which shows that $Q_1^TQ_2$ is also lower triangular. Since $Q_1^TQ_2$ is both upper and lower triangular, it must be diagonal. So, $Q_1^TQ_2 = D = \text{diag}(d_1, d_2, \ldots, d_n)$ or equivalently

$$Q_2 = Q_1 \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \ddots & 0 \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & d_n \end{bmatrix} = Q_1D.$$

Since the columns of both Q_1 and Q_2 are unit vectors, we must have $d_i = \pm 1$, for $i = 1, \ldots, n$.

Finally, since $R_1R_2^{-1} = D$, we have $R_1 = DR_2$, or equivalently, $R_2 = DR_1$, since D is its own inverse. □

We can make the reduced QR factorization of a full column rank matrix A unique by requiring the diagonal entries of R to be positive. This is the factorization produced by the Gram-Schmidt process.

Corollary 27.43. (Unique reduced QR factorization)

A full column rank matrix A has a unique reduced QR factorization $A = QR$ where the diagonal entries of R are all positive.

Proof Let $A = Q_1R_1 = Q_2R_2$ be two reduced QR factorizations of A, where both R_1 and R_2 have positive diagonal entries. From Theorem 27.42, we have $Q_2 = Q_1D$ and $R_2 = DR_1$, for some diagonal matrix $D = \text{diag}(d_1, \ldots, d_n)$ with $d_i = \pm 1$.

From $R_2 = DR_1$, we get $(R_2)_{ii} = d_i(R_1)_{ii}$. Since both $(R_1)_{ii}$ and $(R_2)_{ii}$ are positive, it follows that $d_i = 1$ for $i = 1, \ldots, n$.

Therefore, $D = I_n$, which implies $Q_1 = Q_2$ and $R_1 = R_2$, as claimed. □

If we have a reduced QR factorization $A = Q_1R_1$ for a full column rank matrix $A \in \mathbb{R}^{m\times n}$, the columns $q_1, \ldots, q_n$ of Q_1 form an orthonormal set in $\mathbb{R}^m$. Since any orthonormal set is linearly independent, we can extend it to a full basis $\{q_1, \ldots, q_n, y_{n+1}, \ldots, y_m\}$ for $\mathbb{R}^m$ (Lemma 16.12). Applying the Gram-Schmidt process to this set gives an orthonormal basis $\{q_1, \ldots, q_n, q_{n+1}, \ldots, q_m\}$ for $\mathbb{R}^m$.

We can then put these vectors into the matrix

$$Q = \begin{bmatrix} q_1 \cdots q_n \; q_{n+1} \cdots q_m \end{bmatrix} =: \begin{bmatrix} Q_1 \; Q_2 \end{bmatrix} \in \mathbb{R}^{m\times m}.$$

Now we can write

$$A = Q_1R_1 = \begin{bmatrix} Q_1 \; Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} =: QR,$$

The matrix Q is an $m \times m$ orthogonal matrix. To get R from R_1, we add rows of zeros, keeping R upper triangular ($R_{ij} = 0$ for $i > j$).

This brings us to the following definition.

Definition 27.44. (Full QR factorization)

Let A be an $m \times n$ matrix. We call $A = QR$, with $Q \in \mathbb{R}^{m\times m}$ and $R \in \mathbb{R}^{m\times n}$, a **full QR factorization** of A if

(i) Q is orthogonal, and
(ii) R is (rectangular) upper triangular (i.e., $R_{ij} = 0$ for $i > j$).

See Figure 27.7 for a visual comparison of a reduced and full QR factorizations of a matrix $A \in \mathbb{R}^{m\times n}$, with $m \geqslant n$. To get a reduced QR factorization from a full one, we just drop the $m - n$ zero rows from the bottom of R and remove the last $m - n$ columns from Q. Thus, every full QR factorization also "contains" a reduced QR factorization.

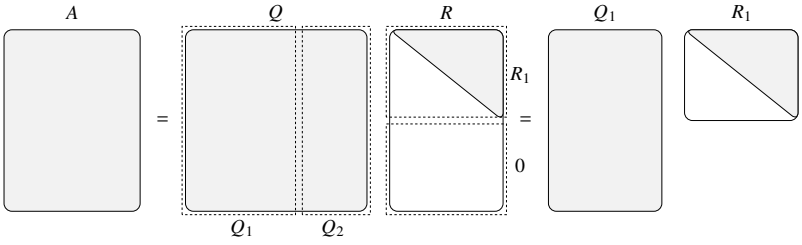


Fig. 27.7: Reduced QR factorization $A = Q_1R_1$ and full QR factorization $A = QR$ of a matrix A.

Theorem 27.45. (Properties of a full QR factorization of a full column rank matrix)

Let $A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \in \mathbb{R}^{m \times n}$ be a matrix with linearly independent columns $a_1, a_2, \ldots, a_n$, and let

$$A = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix}$$

be a full QR factorization of A. Then, the following statements hold:

(a) The matrix R_1 is invertible.
(b) If $Q_1 = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}$, then

$$\operatorname{span}(q_1, q_2, \ldots, q_i) = \operatorname{span}(a_1, a_2, \ldots, a_i),$$

for $i = 1, \ldots, n$. In particular, $\operatorname{col}(Q_1) = \operatorname{col}(A)$.
(c) $\operatorname{col}(Q_2) = \operatorname{null}(A^T)$.

Proof Parts (a) and (b) follow from Theorem 27.38 because $A = Q_1 R_1$ is a reduced QR factorization of A. To prove part (c), recall from Theorem 27.12 that the orthogonal complement of $\operatorname{col}(A)$ is $\operatorname{null}(A^T)$. Then, since Q is an orthogonal matrix, we have $\operatorname{col}(Q_2) = \operatorname{col}(Q_1)^\perp = \operatorname{col}(A)^\perp = \operatorname{null}(A^T)$, as desired. □

Any full column rank matrix A admits a full QR factorization. We can first use the Gram-Schmidt process to get a reduced QR factorization $A = Q_1 R_1$, then extend Q_1 to a full orthogonal matrix Q. But is this factorization unique? Just as we did with the reduced QR factorization, the next theorem shows how unique the full QR factorization is.

Theorem 27.46. (Characterizations of all full QR factorizations)

Let A be a full column rank $m \times n$ matrix. If

$$A = \begin{bmatrix} Q_1 & U_1 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = \begin{bmatrix} Q_2 & U_2 \end{bmatrix} \begin{bmatrix} R_2 \\ 0 \end{bmatrix}$$

are two full QR factorizations of A, then $Q_2 = Q_1 D$, $R_2 = D R_1$, where D is a diagonal matrix with entries ± 1, and $U_2 = U_1 S$ for some orthogonal matrix S.

Proof Since $A = Q_1 R_1 = Q_2 R_2$ are two reduced QR factorizations of A, Theorem 27.42 tells us that

$$Q_2 = Q_1 D, \quad R_2 = D R_1,$$

for some diagonal matrix D with entries ± 1.

From part (c) of Theorem 27.45, we also know that

$$\operatorname{col}(U_1) = \operatorname{null}(A^T) = \operatorname{col}(U_2).$$

So the columns of U_1 and U_2 are orthonormal bases for the same subspace. This means there exists an invertible matrix S such that $U_2 = U_1 S$. Since both U_1 and U_2 have orthonormal columns, we have

$$I = U_2^T U_2 = (U_1 S)^T (U_1 S) = S^T U_1^T U_1 S = S^T S,$$

which shows that S is orthogonal. □

But what if the matrix A does not have full column rank? Can we still find a QR factorization? To answer that, we need new tools, since the Gram-Schmidt process, as we have presented it, will not work in this case (though it can be modified to handle rank-deficient matrices; see Problem 27.35). This is where Householder reflectors, introduced in Section 27.3, come in.

27.4.2 The general case

Here, we will show that any matrix $A \in \mathbb{R}^{m\times n}$ has a full QR factorization $A = QR$, where $Q \in \mathbb{R}^{m\times m}$ is orthogonal, and $R \in \mathbb{R}^{m\times n}$ is upper triangular, regardless of its rank.

Theorem 27.47. (Existence of full QR factorizations)

Every matrix $A \in \mathbb{R}^{m\times n}$ has a full QR factorization.

Proof If A is the zero matrix, we can simply take $Q = I_m$ and $R = A$, and we are done. If $m = 1$ (i.e., A has just one row), we can take $Q = 1$ and $R = A$.

Now, assume $A \neq 0$ and $m \geqslant 2$.

We will prove the existence of full QR factorizations by induction on n, the number of columns of the matrix A.

For the base case, let $n = 1$. By Theorem 27.34, there is a Householder reflector Q that transforms A so that all entries below the first become zero. Since Q is symmetric and satisfies $Q = Q^T = Q^{-1}$, we have

$$A = Q(QA) = QR, \quad \text{with } R := QA$$

which gives a full QR factorization of A.

Assume the result holds for matrices with $n-1$ columns. Now let $A \in \mathbb{R}^{m\times n}$. By Theorem 27.34, there exists a Householder reflector Q_1 such that the first column of $Q_1 A$ has zeros below the first entry. In other words, we have

$$Q_1 A = \left[\begin{array}{c|ccc} \star & \star & \cdots & \star \\ \hline 0 & & & \\ \vdots & & \widehat{A} & \\ 0 & & & \end{array}\right], \quad \text{for some } (m-1)\times(n-1) \text{ matrix } \widehat{A}.$$

By the induction hypothesis, $\widehat{A}$ has a full QR factorization $\widehat{A} = \widehat{Q}\widehat{R}$. We can then write

$$A = Q_1^T \left[\begin{array}{c|ccc} \star & \star & \cdots & \star \\ \hline 0 & & & \\ \vdots & & \widehat{Q}\widehat{R} & \\ 0 & & & \end{array}\right] = \underbrace{Q_1^T \left[\begin{array}{c|ccc} 1 & 0 & \cdots & 0 \\ \hline 0 & & & \\ \vdots & & \widehat{Q} & \\ 0 & & & \end{array}\right]}_{=:Q} \underbrace{\left[\begin{array}{c|ccc} \star & \star & \cdots & \star \\ \hline 0 & & & \\ \vdots & & \widehat{R} & \\ 0 & & & \end{array}\right]}_{=:R} = QR.$$

The matrix R is upper triangular because $\widehat{R}$ is upper triangular, and Q is orthogonal because both Q_1^T and $\operatorname{diag}(1, \widehat{Q})$ are orthogonal, and the product of orthogonal matrices is also orthogonal. We conclude that A has a full QR factorization. □

Since the factor R in a full QR factorization is upper triangular, we can write it as

$$R = \begin{bmatrix} R_1 \\ 0 \end{bmatrix} \begin{matrix} n \\ m-n \end{matrix} \qquad \text{or} \qquad R = \begin{bmatrix} \overset{m}{R_1} & \overset{n-m}{R_2} \end{bmatrix}$$

depending on whether A has more rows than columns ($m \geqslant n$) or more columns than rows ($n \geqslant m$), where R_1 is a square, upper triangular matrix. Figure 27.8 illustrates the full QR factorization for both tall-and-skinny ($m \geqslant n$) and short-and-wide ($m \leqslant n$) matrices. As in the full column rank case, when $m \geqslant n$, Figure 27.7-(a) shows that from a full QR factorization of A,

$$A = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix},$$

we can obtain a reduced QR factorization $A = Q_1 R_1$, where $Q_1 \in \mathbb{R}^{m \times n}$ has orthonormal columns and $R_1 \in \mathbb{R}^{n \times n}$ is upper triangular. However, if A does not have full column rank, R_1 will not be invertible.

Next are two examples of full QR factorizations: one for a square matrix without full column rank, and one for a matrix with more columns than rows.

Example 27.48 Consider the following matrix

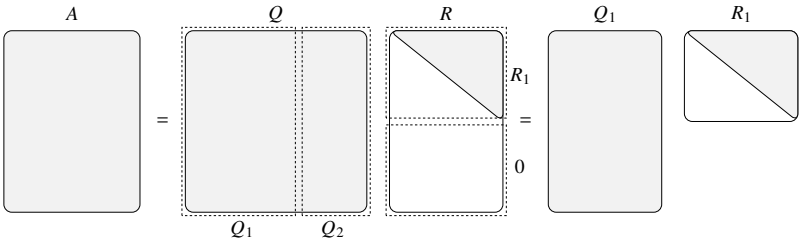
$$A = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 2 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}.$$

Notice that the columns a_1, a_3 and a_4 are linearly independent, but $a_2 = a_1$. Hence, $\operatorname{col}(A)$ has dimension 3, and thus A does not have full column rank.

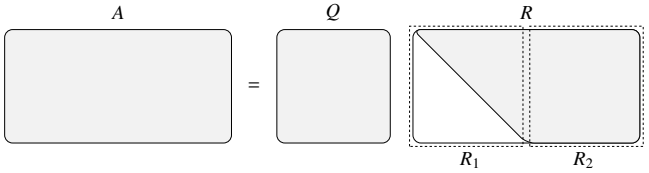
A full QR factorization of A is

$$A = QR = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 \end{bmatrix} R = \begin{bmatrix} 1/2 & 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 & -1/2 \\ 1/2 & 1/2 & -1/2 & -1/2 \\ 1/2 & -1/2 & -1/2 & 1/2 \end{bmatrix} \begin{bmatrix} 2 & 2 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

There is something unsatisfactory about this factorization. Unlike a QR factorization for a full column rank matrix, $\operatorname{col}(A)$ does not equal $\operatorname{span}\{q_1, q_2, q_3\}$. In fact,



(a) Full and reduced QR factorizations of a tall-and-skinny matrix. Unlike in the full column rank case, the matrix R_1 may not be invertible.



(b) Full QR factorization of a wide-and-short matrix.

Fig. 27.8: Full QR factorizations of a tall-and-skinny and short-and-wide matrices. The gray shaded area represents all the entries that are (possibly) nonzero.

no combination of three columns from Q spans $\mathrm{col}(A)$. We will address this issue in Section 27.5.

Example 27.49 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 2 & 5 & 3 \\ 2 & 3 & 1 & 3 & 2 \\ 2 & 2 & 1 & 2 & 1 \end{bmatrix}.$$

A full QR factorization of A is

$$A = QR = \begin{bmatrix} 1/3 & 2/3 & 2/3 \\ 2/3 & 1/3 & -2/3 \\ 2/3 & -2/3 & 1/3 \end{bmatrix} \begin{bmatrix} 3 & 4 & 2 & 5 & 3 \\ 0 & 2 & 1 & 3 & 2 \\ 0 & 0 & 1 & 2 & 1 \end{bmatrix}.$$

27.4.3 The Householder algorithm for computing the QR factorization

The proof of Theorem 27.47 gives the main idea for the algorithm we introduce in this section: using Householder reflectors to eliminate subdiagonal entries.

Given an $m \times n$ matrix A, the Householder algorithm computes an orthogonal matrix Q that transforms A into an upper triangular matrix. It achieves this by applying a sequence of orthogonal matrices to A.

For simplicity, we will assume $m \geqslant n$, as this case is the most common in applications. However, the same ideas can be extended to the case where $m < n$.

In the case where $m \geqslant n$, the Householder algorithm transforms A into upper triangular form by applying a sequence of up to n orthogonal matrices $Q_1, \ldots, Q_n$. The result is

$$Q_n Q_{n-1} \cdots Q_2 Q_1 A = \begin{bmatrix} R_1 \\ 0 \end{bmatrix} =: R,$$

where R_1 is upper triangular. The main tool for the algorithm is the Householder reflector, which the algorithm uses to systematically zero out entries in A.

The Householder algorithm is similar to Gaussian elimination, as it zeros out entries below the main diagonal one column at a time. The key difference is that the Householder algorithm uses orthogonal matrices for these transformations, while Gaussian elimination does not.

To see how the process works, let us consider a small example with a 6×4 matrix A. After each step, the matrices $Q_i \cdots Q_1 A$ take the following form:

$$A = \begin{bmatrix} \star & \star & \star & \star \\ \star & \star & \star & \star \\ \star & \star & \star & \star \\ \star & \star & \star & \star \\ \star & \star & \star & \star \end{bmatrix} \to \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & \star & \star & \star \end{bmatrix} \to \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & \star & \star \end{bmatrix} \to \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & \star \end{bmatrix} \to \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The first step of the Householder algorithm

We begin with an $m \times n$ matrix A, where $\star$ represents the (possibly) nonzero entries of A. Let a_1 be the first column of A (shaded in blue below).

$$A = \begin{bmatrix} \star & \star & \cdots & \star \\ \star & \star & \cdots & \star \\ \star & \star & \cdots & \star \\ \vdots & \vdots & \ddots & \vdots \\ \star & \star & \cdots & \star \end{bmatrix}.$$

In the first step of the Householder algorithm, we construct an $m \times m$ Householder reflector $Q_1 = I_m - 2u_1 u_1^T$ so that $Q_1 a_1 = \pm \|a_1\|_2 e_1$. This matrix zeros out the elements below the main diagonal in the first column of A, giving us the transformed matrix

$$A_1 := Q_1 A = Q_1 \begin{bmatrix} \star & \star & \cdots & \star \\ \star & \star & \cdots & \star \\ \star & \star & \cdots & \star \\ \vdots & \vdots & \ddots & \vdots \\ \star & \star & \cdots & \star \end{bmatrix} = \begin{bmatrix} \times & \times & \times & \cdots & \times \\ 0 & \times & \times & \cdots & \times \\ 0 & \times & \times & \cdots & \times \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \times & \times & \cdots & \times \end{bmatrix},$$

where $\times$ represents elements that may have changed during the transformation and are (possibly) nonzero.

If all entries below the top entry of a_1 are already zero, we just set $Q_1 = I_m$.

The Householder algorithm works recursively. After the first step, we transform the matrix A into $A_1 = Q_1 A$, whose first column is $\pm\|a_1\|_2 e_1$. Next, we apply the same process to the submatrix $A_1[2:m, 2:n]$ and repeat the steps.

The second step of the Householder algorithm

Let a_2 be the first column of $A_1[2:m, 2:n]$, shown shaded in blue below:

$$A_1 = \begin{bmatrix} \star & \star & \star & \cdots & \star \\ 0 & \star & \star & \cdots & \star \\ 0 & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \star & \cdots & \star \end{bmatrix}.$$

Next, we construct an $(m-1) \times (m-1)$ Householder reflector $I_{m-1} - 2u_2 u_2^T$ such that $(I_{m-1} - 2u_2 u_2^T)a_2 = \pm\|a_2\| e_1$. We then embed this into an $m \times m$ orthogonal matrix

$$Q_2 = \begin{bmatrix} 1 & 0 \\ 0 & I_{m-1} - 2u_2 u_2^T \end{bmatrix}.$$

Multiplying A_1 by Q_2 on the left only affects rows 2 through m, so the zeros from the first step are preserved. The orthogonal matrix Q_2 zeros out the elements below the diagonal in the second column of $Q_1 A$. This results in the matrix

$$A_2 := Q_2 A_1 = Q_2 \begin{bmatrix} \star & \star & \star & \cdots & \star \\ 0 & \star & \star & \cdots & \star \\ 0 & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \star & \cdots & \star \end{bmatrix} = \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \times & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \times & \times & \times \end{bmatrix}.$$

In the matrix $A_2 = Q_2 A_1$, the $\times$ marks the elements that might change during the transformation and could be nonzero, while the $\star$ marks the elements that remain unchanged.

If all entries below the top entry of a_2 are already zero, we just set $Q_2 = I_m$.

Next, we apply step 1 to the submatrix $A_2[3:m, 3:n]$ to get the orthogonal matrix Q_3, and we keep repeating this process, applying up to n orthogonal matrices.

Step i of the Householder algorithm

At the start of step i in the Householder algorithm, the matrix $A_{i-1} = Q_{i-1} \cdots Q_1 A$ has the following form:

$$A_{i-1} = \begin{array}{c} \text{col } i \\ \left[\begin{array}{ccc|cc} \star & \star & \cdots \star & \star & \star \cdots \star \\ 0 & \star & \ddots \vdots & \vdots & \vdots \ddots \vdots \\ \vdots & \ddots & \ddots \star & \star & \star \cdots \star \\ 0 & \cdots & 0 \ \star & \star & \star \cdots \star \\ \hline & & & \star & \star \cdots \star \\ & 0 & & \star & \star \cdots \star \\ & & & \vdots & \vdots \ddots \vdots \\ & & & \star & \star \cdots \star \end{array}\right] \end{array} \begin{array}{l} \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array}.$$

Let a_i be the first column of $A_{i-1}[i:m, i:n]$, shown shaded in blue in the matrix above. In the ith step of the Householder algorithm, we construct an $(m-i+1) \times (m-i+1)$ Householder reflector $I_{m-i+1} - 2u_i u_i^T$, which satisfies $(I_{m-i+1} - 2u_i u_i^T)a_i = \pm\|a_i\|e_1$. We then embed this reflector matrix into an $m \times m$ orthogonal matrix Q_i of the form

$$Q_i = \begin{bmatrix} I_{i-1} & 0 \\ 0 & I_{m-i+1} - 2u_i u_i^T \end{bmatrix}.$$

Multiplying A_{i-1} by Q_i on the left only affects rows $i, i+1, \ldots, m$, so the zeros from the previous $i-1$ steps are preserved. The reflector matrix Q_i zeros out the elements below the diagonal in the ith column of A_{i-1}. This results in the matrix

$$A_i := Q_i A_{i-1} = Q_i \left[\begin{array}{ccc|ccc} \star & \star & \cdots \star & \star & \star & \cdots \star \\ 0 & \star & \ddots \vdots & \vdots & \vdots & \ddots \vdots \\ \vdots & \ddots & \ddots \star & \star & \star & \cdots \star \\ 0 & \cdots & 0 \ \star & \star & \star & \cdots \star \\ \hline & & & \star & \star & \cdots \star \\ & 0 & & \star & \star & \cdots \star \\ & & & \vdots & \vdots & \ddots \vdots \\ & & & \star & \star & \cdots \star \end{array}\right] = \left[\begin{array}{ccc|ccc} \star & \star & \cdots \star & \star & \star & \cdots \star \\ 0 & \star & \ddots \vdots & \vdots & \vdots & \ddots \vdots \\ \vdots & \ddots & \ddots \star & \star & \star & \cdots \star \\ 0 & \cdots & 0 \ \star & \star & \star & \cdots \star \\ \hline & & & \times & \times & \cdots \times \\ & 0 & & 0 & \times & \cdots \times \\ & & & \vdots & \vdots & \ddots \vdots \\ & & & 0 & \times & \cdots \times \end{array}\right],$$

In the matrix A_i, the elements marked with $\times$ have changed during the transformation and may be nonzero, while the elements marked with $\star$ remain unchanged in this step.

If all the entries below the top entry of a_i are already zero, we just set $Q_i = I_m$.

After n steps, the Householder algorithm reduces A to an upper triangular matrix R. We can summarize the sequence of orthogonal transformations as

$$Q_n \cdots Q_2 Q_1 A = R.$$

Since each orthogonal matrix Q_i is its own inverse (a property of Householder reflectors; see Proposition 27.32), we get

$$A = \underbrace{Q_1 Q_2 \cdots Q_n}_{=:Q} R = QR.$$

This gives a full QR factorization of A because Q, being the product of n orthogonal matrices, is also orthogonal.

Remark 27.50. (Choosing the Householder reflector)

At each step of the Householder algorithm, the reflector that introduces the zeros is not unique. Given a vector v, we can transform v into either $\|v\|_2 e_1$ or $-\|v\|_2 e_1$. Both choices are mathematically valid.

However, for numerical purposes, it is important to choose carefully. Without going into too much detail, reflecting v to the vector that is farthest from v helps reduce round-off errors during computations. We do this by transforming v into $-\text{sign}(v_1)\|v\|e_1$, where

$$\text{sign}(v_1) = \begin{cases} +1 & \text{if } v_1 \geqslant 0, \\ -1 & \text{if } v_1 < 0, \end{cases}$$

and v_1 is first entry of v.

Example 27.51 Let us use the Householder algorithm to compute the QR factorization of the matrix

$$A = \begin{bmatrix} -1 & -1 & 1 \\ 1 & 3 & 3 \\ -1 & -1 & 5 \\ 1 & 3 & 7 \end{bmatrix}.$$

We will compute three orthogonal matrices, Q_1, Q_2, and Q_3, to make A upper triangular.

Recall from the proof of Theorem 27.33 that to transform a vector x into y using a Householder reflector $I_n - 2uu^T$, we choose $u = \frac{x-y}{\|x-y\|_2}$.

The first Householder reflector, Q_1, is then

$$Q_1 = I_4 - 2\frac{(a_1 - \|a_1\|_2 e_1)(a_1 - \|a_1\|_2 e_1)^T}{\|a_1 - \|a_1\|_2 e_1\|_2^2} = \frac{1}{6}\begin{bmatrix} -3 & 3 & -3 & 3 \\ 3 & 5 & 1 & -1 \\ -3 & 1 & 5 & 1 \\ 3 & -1 & 1 & 5 \end{bmatrix},$$

where $a_1 = \begin{bmatrix} -1 & 1 & -1 & 1 \end{bmatrix}^T$ is the first column of A. We transform a_1 into $\|a_1\|_2 e_1$ because the first entry of a_1 is negative.

We get

$$Q_1 A = \begin{bmatrix} 2 & 4 & 2 \\ 0 & 4/3 & 8/3 \\ 0 & 2/3 & 16/3 \\ 0 & 4/3 & 20/3 \end{bmatrix}.$$

The second Householder reflector is

$$\widetilde{Q}_2 = I_3 - 2\frac{(a_2 - \|a_2\|_2 e_1)(a_2 - \|a_2\|_2 e_1)^T}{\|a_2 - \|a_2\|_2 e_1\|_2^2} = \frac{1}{15}\begin{bmatrix} -10 & -5 & -10 \\ -5 & 14 & -2 \\ -10 & -2 & 11 \end{bmatrix},$$

where $a_2 = \begin{bmatrix} 4/3 & 2/3 & 4/3 \end{bmatrix}^T$ is the first column of $(Q_1 A)[2:4, 2:3]$. Here, we transform a_2 into $-\|a_2\|_2 e_1$ because its first entry is positive. Set $Q_2 = \text{diag}(1, \widetilde{Q}_2)$, and we get

$$Q_2(Q_1 A) = \begin{bmatrix} 2 & 4 & 2 \\ 0 & -2 & 8 \\ 0 & 0 & 16/5 \\ 0 & 0 & 12/5 \end{bmatrix}.$$

The third Householder reflector is

$$\widetilde{Q}_3 = I_2 - 2\frac{(a_3 - \|a_3\|_2 e_1)(a_3 - \|a_3\|_2 e_1)^T}{\|a_3 - \|a_3\|_2 e_1\|_2^2} = \frac{1}{5}\begin{bmatrix} -4 & -3 \\ -3 & 4 \end{bmatrix},$$

where $a_3 = \begin{bmatrix} 16/5 & 12/5 \end{bmatrix}^T$ comes from $(Q_2 Q_1 A)[3:4, 3]$. Again, we transform a_3 into $-\|a_3\|_2 e_1$ since its first entry is positive. Setting $Q_3 = \text{diag}(I_2, \widetilde{Q}_3)$, we get

$$Q_3(Q_2 Q_1 A) = \begin{bmatrix} 2 & 4 & 2 \\ 0 & -2 & -8 \\ 0 & 0 & -4 \\ 0 & 0 & 0 \end{bmatrix},$$

which is upper triangular.

27.5 QR factorization with column pivoting

In many applications, like the least squares problem in Chapter 32, our $m \times n$ matrix A may be rank-deficient ($r = \text{rank}(A) < \min\{m, n\}$). Sometimes the rank r is known, but often it is not. Even when A has full column rank, it can still be close to a rank-deficient matrix.

For now, let us assume we know the rank of A. We will cover how to handle the case when the rank is unknown in Chapters 47 and 49.

One key difference in the QR factorization of rank-deficient matrices is that the upper triangular submatrix R_1 in the R factor,

$$A = Q\begin{bmatrix} R_1 \\ 0 \end{bmatrix} \quad \text{or} \quad A = Q\begin{bmatrix} R_1 & R_2 \end{bmatrix},$$

depending on whether $m \geqslant n$ or $n \geqslant m$, is not invertible. Also, the columns of Q might not form an orthonormal basis for $\text{col}(A)$, as we saw in Example 27.48. To

fix this, we use column permutations during the QR factorization. This way, when $A \neq 0$, we can easily extract an orthonormal basis for $\text{col}(A)$ from Q.

Theorem 27.52. (QR factorization with column pivoting)

Let $A \in \mathbb{R}^{m\times n}$ with $r = \text{rank}(A)$. Then, there exists a permutation matrix $P \in \mathbb{R}^{n\times n}$ and an orthogonal matrix $Q = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \in \mathbb{R}^{m\times m}$, where $Q_1 \in \mathbb{R}^{m\times r}$ and $Q_2 \in \mathbb{R}^{m\times(m-r)}$, such that

$$AP = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} \begin{matrix} r \\ m-r \end{matrix} \qquad \begin{matrix} r & n-r \end{matrix} \tag{27.7}$$

where $\text{col}(A) = \text{col}(Q_1)$ and $R_{11} \in \mathbb{R}^{r\times r}$ is upper triangular and invertible. This factorization is known as a **QR factorization with column pivoting**.

Proof We will go through the details for the case $m \geqslant n$; the case $n > m$ is similar and left to you.

If $\text{rank}(A) = 0$, then $A = 0$. In this case, we can simply set $Q = I_m$, $P = I_n$, and $R = A = 0$, giving us the desired factorization.

Assume $\text{rank}(A) = r > 0$, so A has r linearly independent columns. Let P be an $n \times n$ permutation matrix that reorders the columns of A, placing the r independent columns first. After applying the permutation, we get $AP = \begin{bmatrix} A_1 & A_2 \end{bmatrix}$, where $A_1 \in \mathbb{R}^{m\times r}$ contains the independent columns, and $A_2 \in \mathbb{R}^{m\times(n-r)}$ contains the dependent columns, which are linear combinations of the columns in A_1.

A full QR factorization of AP gives

$$AP = QR = Q \begin{bmatrix} R_1 \\ 0 \end{bmatrix} \begin{matrix} n \\ m-n \end{matrix},$$

where R_1 is upper triangular. Now, let us partition R_1 as follows

$$R_1 = \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix} \begin{matrix} r \\ n-r \end{matrix} \qquad \begin{matrix} r & n-r \end{matrix},$$

where $R_{11} \in \mathbb{R}^{r\times r}$ and $R_{22} \in \mathbb{R}^{(n-r)\times(n-r)}$ are upper triangular. To complete the proof, we just need to show that R_{11} is invertible, $R_{22} = 0$ and $\text{col}(Q_1) = \text{col}(A)$.

Partitioning Q as $Q = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix}$, where $Q_1 \in \mathbb{R}^{m\times r}$ and $Q_2 \in \mathbb{R}^{m\times(m-r)}$, we have

$$AP = \begin{bmatrix} A_1 & A_2 \end{bmatrix} = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_{11} & R_{21} \\ 0 & R_{22} \\ 0 & 0 \end{bmatrix},$$

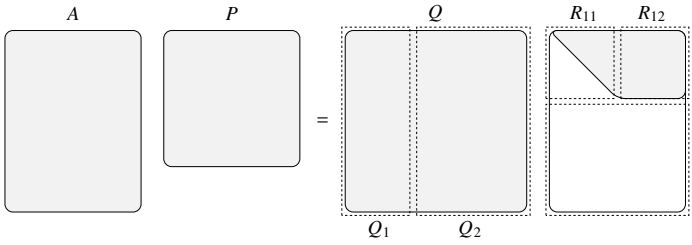
which gives

$$A_1 = Q_1 R_{11} \qquad \text{and} \qquad A_2 = Q_1 R_{21} + Q_2 \begin{bmatrix} R_{22} \\ 0 \end{bmatrix}.$$

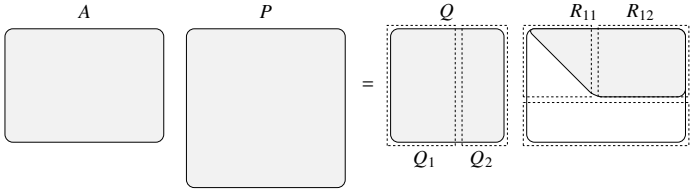
The equality on the left, is a reduced QR factorization of A_1. Thus, from Theorem 27.38, we deduce that $\operatorname{col}(A) = \operatorname{col}(A_1) = \operatorname{col}(Q_1)$ and R_{11} is invertible.

To show that $R_{22} = 0$, notice that $A_2 = Q_1 R_{12} + Q_2 \begin{bmatrix} R_{22} \\ 0 \end{bmatrix}$ expresses each column of A_2 as a sum of a vector in $\operatorname{col}(Q_1)$ and a vector in $\operatorname{col}(Q_2) = \operatorname{col}(Q_1)^{\perp}$. But since the columns of A_2 are linear combinations of the columns of A_1, we have $\operatorname{col}(A_2) \subseteq \operatorname{col}(A_1) = \operatorname{col}(Q_1)$. This implies every column of A_2 is a linear combination of the columns of Q_1. Because the expression of a vector as a combination of columns $Q = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix}$ is unique, we conclude $R_{22} = 0$. □

Figure 27.9 illustrates the structure of a QR factorization with column pivoting for a tall-and-skinny (top) or short-and-wide (bottom) rank-deficient matrix A.



(a) QR factorization with column pivoting of a tall-and-skinny rank-deficient matrix A.



(b) Full QR factorization with column pivoting of a short-and-wide rank-deficient matrix A.

Fig. 27.9: QR factorization with column pivoting of a tall-and-skinny and short-and-wide matrices. The gray shaded area represents all the entres that are (possibly) nonzero.

27.5.1 Householder algorithm with column pivoting

We can modify the Householder algorithm to handle rank-deficient matrices. In fact, the standard Householder algorithm works for any matrix, even if it does not have full rank. Adding column pivoting simply helps reorder the columns so that dependent or nearly dependent columns are processed last.

Let us assume $A \neq 0$ because, if $A = 0$, the QR factorization is simply $A = I_m \cdot 0$. We focus on matrices $A \in \mathbb{R}^{m \times n}$ with $m \geqslant n$, as this is the most common case in practice.

The algorithm starts with a tall-and-skinny $m \times n$ matrix A with $\operatorname{rank}(A) = r < n$. It applies a sequence of at most r orthogonal matrices $Q_1, \ldots, Q_r$ and permutation matrices $P_1, \ldots, P_r$, transforming A into the form

$$\underbrace{Q_r Q_{r-1} \cdots Q_1}_{=:Q} A \underbrace{P_1 \cdots P_{r-1} P_r}_{=:P} = QAP = R = \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix},$$

where $R_{11} \in \mathbb{R}^{r \times r}$ is upper triangular and invertible.

The first r columns of AP form a basis for $\mathrm{col}(A)$. There is usually more than one way to choose these r independent columns from A. The column-pivoted Householder algorithm picks columns that are *as linearly independent as possible*.

The first step of the column-pivoted Householder algorithm

Let $A \neq 0$ be a rank-deficient $m \times n$ matrix. The first step is to find the column of A with the largest 2-norm, which we call the **first pivot column**.

Next, we construct a permutation matrix P_1 to move this pivot column into the first position of AP_1. We then apply a Householder reflector $Q_1 = I_n - 2u_1 u_1^T$, just as in the standard Householder algorithm, to get the matrix $A_1 := Q_1 A P_1$, which has zeros below the diagonal in the first column, as shown below:

$$A_1 = Q_1 A P_1 = \begin{bmatrix} r_{11} & \star & \cdots & \star \\ 0 & \star & \cdots & \star \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \cdots & \star \end{bmatrix}. \tag{27.8}$$

Note that $r_{11} \neq 0$ because the first column of A_1 has the same 2-norm as the corresponding column in A.

The column-pivoted Householder algorithm works recursively. After the first step, the same process is applied to the submatrix $A_1[2:m, 2:n]$, selecting the next pivot column and building the corresponding permutation and reflector. This continues one column at a time, using smaller and smaller submatrices.

Step i of the column-pivoted Householder algorithm

At the start of step i, the matrix $A_{i-1} := (Q_{i-1} \cdots Q_1) A (P_1 \cdots P_{i-1})$ has the following form

$$A_{i-1} = \left[\begin{array}{c|c} B_{i-1} & C_{i-1} \\ \hline 0 & D_{i-1} \end{array}\right] := \left[\begin{array}{cccc|c} r_{11} & \star & \cdots & \star & \star \; \star \cdots \star \\ 0 & r_{22} & \ddots & \vdots & \vdots \; \vdots \; \ddots \; \vdots \\ \vdots & \ddots & \ddots & \star & \star \; \star \cdots \star \\ 0 & \cdots & 0 & r_{i-1,i-1} & \star \; \star \cdots \star \\ \hline & & & & \star \; \star \cdots \star \\ & & & & \star \; \star \cdots \star \\ & & 0 & & \vdots \; \vdots \; \ddots \; \vdots \\ & & & & \star \; \star \cdots \star \end{array}\right] \begin{array}{l} \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array}, \tag{27.9}$$

where the column i is the first column of the right block.

where $B_{i-1} \in \mathbb{R}^{(i-1)\times(i-1)}$ is upper triangular and invertible. If $D_{i-1} := A_{i-1}[i : m, i : n] = 0$ (shaded in red in the matrix above), the algorithm stops[6].

If $D_{i-1} \neq 0$, we first find the column of D_{i-1} with the largest 2-norm, which becomes **the ith pivot column**. Next, we build a permutation matrix P_i that moves the pivot column to the ith column of $A_{i-1}P_i$. Finally, we construct a Householder reflector and embed it into the orthogonal matrix

$$Q_i = \begin{bmatrix} I_{i-1} & 0 \\ 0 & I_{m-i+1} - 2u_i u_i^T \end{bmatrix}$$

so that

$$A_i := Q_i A_{i-1} P_i = \left[\begin{array}{cccc|c|c} r_{11} & \star & \cdots & \star & \star & \star \cdots \star \\ 0 & r_{22} & \ddots & \vdots & \vdots & \vdots \ddots \vdots \\ \vdots & \ddots & \ddots & \star & \star & \star \cdots \star \\ 0 & \cdots & 0 & r_{i-1,i-1} & \star & \star \cdots \star \\ \hline & & & & r_{ii} & \star \cdots \star \\ & & & & 0 & \star \cdots \star \\ & & 0 & & \vdots & \vdots \cdots \vdots \\ & & & & 0 & \star \cdots \star \end{array}\right] \begin{array}{c} \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array} \qquad (27.10)$$

(col i)

has zeros in column i below the main diagonal. Notice that $r_{ii} \neq 0$.

The following theorem states that the column-pivoted Householder algorithm produces a permutation matrix P and an orthogonal matrix Q such that $AP = QR$, where R is an upper triangular matrix that satisfies certain inequalities. In particular, the diagonal entries of R are non-increasing in magnitude, and if the rank of A is less than n, R has the block structure shown in (27.7).

Theorem 27.53. (The QR factorization with column pivoting)

Let $A \in \mathbb{R}^{m\times n}$ with $m \geqslant n$. The column-pivoted Householder algorithm produces a permutation matrix $P \in \mathbb{R}^{n\times n}$ and an orthogonal matrix $Q \in \mathbb{R}^{m\times m}$ such that

$$AP = QR$$

where $R \in \mathbb{R}^{m\times n}$ is upper triangular and satisfies the inequalities

$$|r_{kk}|^2 \geqslant \sum_{t=k}^{m} |r_{tj}|^2 \qquad (27.11)$$

for $k = 1, \ldots, n$ and $j = k+1, \ldots, n$. In particular, we have $|r_{11}| \geqslant |r_{22}| \geqslant \cdots \geqslant |r_{nn}|$. Moreover, if $\operatorname{rank}(A) = r < n$, then R has the form

[6] In practice, the algorithm stops when the norm of D_{i-1} is considered "small enough".

$$R = \begin{matrix} \\ r \\ m-r \end{matrix} \overset{\begin{matrix} r & n-r \end{matrix}}{\begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix}},$$

where $R_{11} \in \mathbb{R}^{r \times r}$ is upper triangular and invertible.

Proof After the r steps of the column-pivoted Householder algorithm, we have transformed the matrix A into an upper triangular matrix $R = Q_r \cdots Q_1 A P_1 \cdots P_r$. So, we have

$$A \underbrace{P_1 \cdots P_r}_{=P} = AP = \underbrace{Q_1 \cdots Q_r}_{=Q} R = QR$$

where Q is orthogonal, and P is a permutation matrix.

We want to show that the matrix $R = A_r$, from the r steps of the column-pivoted Householder algorithm, satisfies the inequalities in (27.11). To do this, we will use induction on the number of steps. We will prove that after i steps, the matrix

$$A_i := Q_i \cdots Q_1 A P_1 \cdots P_i$$

satisfies the inequalities

$$\|A_i[k:m,k]\|_2^2 = |r_{kk}|^2 \geqslant \|A_i[k:m,j]\|_2^2, \tag{27.12}$$

for $k = 1, \ldots, i$ and $j = k+1, \ldots, n$.

We begin with the case when $i = 1$. After the first step of the column-pivoted Householder algorithm, we get a matrix $A_1 = Q_1 A P_1$ in the form of (27.8). By construction, the first column of AP_1 is the column of A with the largest 2-norm. Since the Householder reflector Q_1 preserves the norms of the columns of AP_1, we have:

$$\|A_1[:,1]\|_2^2 = |r_{11}|^2 \geqslant \|A_1[:,j]\|_2^2,$$

for $j = 2, \ldots, n$, which is the inequality (27.12) with $i = 1$. This completes the base case of the induction.

Assuming $i \geqslant 2$, we want to show that if the matrix A_{i-1} satisfies the inequalities

$$\|A_{i-1}[k:m,k]\|_2^2 = |r_{kk}|^2 \geqslant \|A_{i-1}[k:m,j]\|_2^2, \tag{27.13}$$

for $k = 1, \ldots, i-1$ and $j = k+1, \ldots, n$, then after applying the next step of the column-pivoted Householder algorithm to get $A_i = Q_i A_{i-1} P_i$, the resulting matrix A_i will satisfy the inequalities (27.12).

Recall that the matrices A_{i-1} and A_i are of the forms (27.9) and (27.10), respectively. We can write the orthogonal matrix Q_i as

$$Q_i = \begin{bmatrix} I_{i-1} & 0 \\ 0 & I_{m-i+1} - 2uu^T \end{bmatrix} = \begin{bmatrix} I_{k-1} & 0 & 0 \\ 0 & I_{i-k} & 0 \\ 0 & 0 & I_{m-i+1} - 2uu^T \end{bmatrix}, \qquad \text{for k} = 1, \ldots, i.$$

The matrix $\widehat{Q}_i = \operatorname{diag}(I_{i-k}, I_{m-i+1} - 2uu^T)$ is also orthogonal. Thus, we have

$$A_i[k:m,:] = \begin{bmatrix} I_{i-k} & 0 \\ 0 & I_{m-i+1} - 2uu^T \end{bmatrix} (A_{i-1}P_i)[k:m,:],$$

and therefore

$$\|A_i[k:m,j]\|_2 = \|(A_{i-1}P_i)[k:m,j]\|_2,$$

for $k = 1,\ldots,i$ and $j = k,\ldots,n$, because the orthogonal matrix $\widehat{Q}_i$ preserves the 2-norms of the columns of the submatrix $(A_{i-1}P_i)[k:m,:]$. Moreover, the permutation P_i only affects columns i through n. Hence, for $k = 1,\ldots,i-1$ and $j = k+1,\ldots,n$, we have

$$(A_{i-1}P_i)[k:m,j] = A_{i-1}[k:m,\widetilde{j}]$$

for some $k+1 \leqslant \widetilde{j} \leqslant n$.

By the induction hypothesis inequalities (27.13), we get for $k = 1,\ldots,i-1$ and $j = k+1,\ldots,n$ that

$$\|A_i[k:m,j]\|_2^2 = \|(A_{i-1}P_i)[k:m,j]\|_2^2 = \|A_{i-1}[k:m,\widetilde{j}]\|_2^2 \leqslant |r_{kk}|^2.$$

Thus, the matrix A_i satisfies the inequality (27.12) for $k = 1,\ldots,i-1$ and $j = k+1,\ldots,n$.

The remaining inequality ($k = i$) comes from the fact that the permutation P_i is chosen so that $\|(A_{i-1}P_i)[i:m,i]\|_2 \geqslant \|(A_{i-1}P_i)[i:m,j]\|_2$, for $j = i+1,\ldots,n$. Therefore,

$$\begin{aligned} |r_{ii}|^2 = \|A_i[i:m,i]\|_2^2 = &\|(A_{i-1}P_i)[i:m,i]\|_2^2 \geqslant \\ &\|(A_{i-1}P_i)[i:m,j]\|_2^2 = \|A_i[i:m,j]\|_2^2, \end{aligned}$$

for $j = i+1,\ldots,n$. This shows that the inequality (27.12) holds for $k = i$ as well. □

Example 27.54 Let us find the pivoted QR factorization of the matrix

$$A = \begin{bmatrix} 1 & 3 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

Note that the third column is the second minus the first, so A is *rank deficient*.

We start with the permutation matrix as the identity:

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The squared norms of the columns are

$$\begin{aligned}\|a_1\|^2 &= 1^2 + 1^2 + 1^2 = 3,\\ \|a_2\|^2 &= 3^2 + 2^2 + 1^2 = 14, \quad \text{and}\\ \|a_3\|^2 &= 2^2 + 1^2 + 0^2 = 5.\end{aligned}$$

The largest is column 2, so we swap columns 1 and 2 in both A and P:

$$A \to A = \begin{bmatrix}3 & 1 & 2\\ 2 & 1 & 1\\ 1 & 1 & 0\end{bmatrix}, \quad P \to P = \begin{bmatrix}0 & 1 & 0\\ 1 & 0 & 0\\ 0 & 0 & 1\end{bmatrix}.$$

The first Householder reflector is (rounded to three decimals)

$$Q = \begin{bmatrix}0.802 & -0.535 & -0.267\\ -0.535 & 0.841 & -0.079\\ -0.267 & -0.079 & 0.96\end{bmatrix}.$$

Multiplying A by Q produces

$$A_1 = QA = \begin{bmatrix}-3.742 & -1.604 & -2.138\\ 0 & 0.228 & -0.228\\ 0 & 0.614 & -0.614\end{bmatrix}.$$

Now we look at the 2×2 block in the bottom right:

$$A_1[2:3, 2:3] = \begin{bmatrix}0.228 & -0.228\\ 0.614 & -0.614\end{bmatrix}.$$

Both columns have the same norm, so no swap is needed. The next reflector is (rounded to three decimals)

$$Q_2 = \begin{bmatrix}1 & 0 & 0\\ 0 & 0.348 & -0.938\\ 0 & -0.938 & 0.348\end{bmatrix}.$$

Multiplying A_1 by Q_2 gives

$$Q_2A_1 = Q_2Q_1A = \begin{bmatrix}-3.742 & -1.604 & -2.138\\ 0 & 0.655 & 0.655\\ 0 & 0 & 0\end{bmatrix}.$$

So the factorization with column pivoting is (rounded to three decimals):

$$\underbrace{\begin{bmatrix}1 & 3 & 2\\ 1 & 2 & 1\\ 1 & 1 & 0\end{bmatrix}}_{=A}\underbrace{\begin{bmatrix}0 & 1 & 0\\ 1 & 0 & 0\\ 0 & 0 & 1\end{bmatrix}}_{=P} = \underbrace{\begin{bmatrix}-0.802 & 0.436 & 0.408\\ -0.535 & -0.218 & -0.816\\ -0.267 & -0.873 & 0.408\end{bmatrix}}_{=Q_1Q_2=:Q}\underbrace{\begin{bmatrix}-3.742 & -1.604 & -2.138\\ 0 & 0.655 & 0.655\\ 0 & 0 & 0\end{bmatrix}}_{=:R}.$$

Problems

27.1 Find an orthonormal basis for the orthogonal complement of the span of the following two vectors:

$$v_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad \text{and} \quad v_2 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}.$$

27.2 Let $\mathcal{W} = \{\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}^T \mid 2x_1 + x_2 - x_3 = 0\}$. Find an orthogonal complementary subspace of $\mathcal{W}$ in $\mathbb{R}^3$.

27.3 Let $U_1, U_2 \subseteq \mathbb{R}^m$ such that $U_1 \subseteq U_2$. Show that $U_2^\perp \subseteq U_1^\perp$.

27.4 Let $S \subseteq \mathbb{R}^m$ be a nonempty finite set of vectors. Show that $S^\perp = \text{span}(S)^\perp$.

27.5 Let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$, and let $\beta = \{q_1, \ldots, q_k\}$ be an orthonormal basis for $\mathcal{W}$. Show that if we extend β to an orthonormal basis $\{q_1, \ldots, q_m\}$ for $\mathbb{R}^m$, then $\{q_{k+1}, \ldots, q_m\}$ is an orthonormal basis for $\mathcal{W}^\perp$.

27.6 Show that for any vectors $x, y \in \mathbb{R}^n$, we have

$$x^T y = \frac{1}{4}\|x + y\|_2^2 - \frac{1}{4}\|x - y\|_2^2.$$

27.7 A linear transformation $f(x) = Ax$, where $A \in \mathbb{R}^{n \times n}$, is called "rigid" if it preserves the Euclidean distance between every pair of vectors. Show that $f(x) = Ax$ is a rigid linear transformation if and only if the matrix A is orthogonal.

27.8 Show that if $Q \in \mathbb{R}^{n \times n}$ is orthogonal and upper triangular, then Q is diagonal and its main diagonal entries are ± 1.

27.9 (Hadamard matrices) A Hadamard matrix[7] H is a square matrix whose entries are just 1 and -1, and whose columns and rows are orthogonal (but not unit vectors). Show that

(a) an $n \times n$ Hadamard matrix satisfies the equation $H^T H = nI_n$,
(b) H is invertible with $H^{-1} = H^T/n$, and
(c) the matrix $H/\sqrt{n}$ is an orthogonal matrix.

27.10 Give examples of Hadamard matrices of sizes 1, 2, and 4. Is it possible to find a Hadamard matrix of size 3?

27.11 (Sylvester's construction of Hadamard matrices) Sylvester (in 1867) was the first mathematician to construct examples of Hadamard matrices[8].

Define the matrices

[7] Hadamard matrices have many applications in various fields, including quantum information theory, signal processing, error detection and correction codes, statistics, and more.

[8] James Sylvester was a British mathematician known for his work in Matrix Theory and other areas of mathematics.

$$H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad \text{and} \quad H_{2^k} = \begin{bmatrix} H_{2^{k-1}} & H_{2^{k-1}} \\ H_{2^{k-1}} & -H_{2^{k-1}} \end{bmatrix}, \quad \text{for } k \geqslant 2.$$

Show (by induction for example) that the matrices H_{2^k}

(a) are Hadamard matrices,
(a) are symmetric,
(b) have trace zero, and
(c) have all the entries equal to 1 in their first row and column.

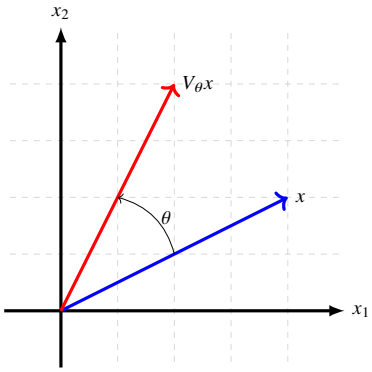
27.12 Show that the Kronecker product $H_1 \otimes H_2$ of two Hadamard matrices H_1 and H_2 is also a Hadamard matrix.

27.13 Fix an angle $\theta \in [0, 2\pi)$, and let V_θ be the matrix of the linear transformation that rotates 2-dimensional vectors about the origin through an angle θ (counter-clockwise). If the vectors

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

are rotated by an angle θ, what are their new coordinates? Conclude that

$$V_\theta = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}.$$

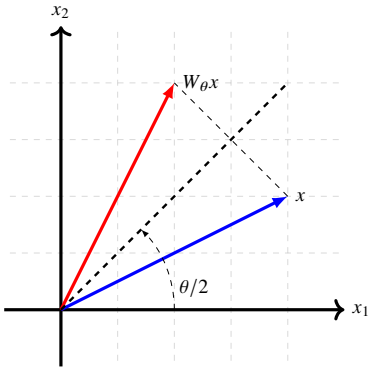


27.14 Let W_θ be the matrix of the linear transformation that reflects 2-dimensional vectors across the line passing through the origin and forming an angle $\theta/2$ with the x_1-axis. If the vectors

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

are reflected across this line, what are their new coordinates? Conclude that

$$W_\theta = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ \sin(\theta) & -\cos(\theta) \end{bmatrix}.$$



27.15 In the two dimensional space, show that if you compose a reflection through a line that forms an angle $\theta/2$ with the x_1-axis, with a reflection through a line that forms an angle $\phi/2$ with the same axis, you obtain a rotation through an angle $\theta + \phi$. Try to prove this fact in two different ways: i) geometrically (that is, with pictures), and ii) algebraically (that is, with matrices). Use this result to show that one can factor a 2×2 Givens rotator matrix as the product of two Householder reflector matrices in infinitely many ways.

27.16 In the two dimensional space, show that a Givens rotator followed by a Householder reflector, and a Householder reflector followed by a Givens rotator are both Householder reflectors.

27.17 Prove Theorem 27.27.

27.18 (Computational complexity of Givens rotator multiplication) Consider an $m \times n$ matrix A.

(a) If $Q \in \mathbb{R}^{m\times m}$ is a Givens rotator, show that calculating QA requires $6n$ arithmetic operations.
(b) If $Q \in \mathbb{R}^{n\times n}$ is a Givens rotator, show that calculating AQ requires $6m$ arithmetic operations.

27.19 (Complex Givens rotators) Consider a nonzero complex vector $x = \left[\begin{smallmatrix} x_1 \\ x_2 \end{smallmatrix}\right] \in \mathbb{C}^2$. Find a 2×2 unitary matrix of the form

$$U = \begin{bmatrix} c & -s \\ \bar{s} & \bar{c} \end{bmatrix}$$

such that

$$U \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \star \\ 0 \end{bmatrix}.$$

27.20 (Hyperbolic matrices) A hyperbolic matrix is a 2×2 matrix of the form

$$Q = \begin{bmatrix} \cosh(t) & \sinh(t) \\ \sinh(t) & \cosh(t) \end{bmatrix}, \quad t \in (-\infty, \infty),$$

where $\sinh(t)$ and $\cosh(t)$ are, respectively, the hyperbolic sine and the hyperbolic cosine of t.

(a) Show that hyperbolic matrices are invertible. What is Q^{-1}? Is Q^{-1} also a hyperbolic matrix?
(b) If $|x_1| > |x_2|$, show that there is a hyperbolic matrix such that

$$\begin{bmatrix} \cosh(t) & \sinh(t) \\ \sinh(t) & \cosh(t) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \sqrt{x_1^2 - x_2^2} \\ 0 \end{bmatrix}.$$

Find formulas for $\sinh(t)$ and $\cosh(t)$ in terms of x_1 and x_2.

27.21 Let $e \in \mathbb{R}^n$ be the vector of all ones.

(a) Find the Householder reflector Q such that $Qe = \|e\|_2 e_1$.
(b) Find the Householder reflector Q such that $Qe = -\|e\|_2 e_1$.

27.22 (Computational complexity of Householder reflector multiplication) Suppose we have an $m \times n$ matrix A and a Householder reflector matrix Q. Show that calculating the product QA requires approximately $4mn$ arithmetic operations. How many arithmetic operations does it take to compute QA if one first forms the matrix Q and then multiplies QA.

27.23 (Computational complexity of multiplying Householder reflectors) Consider $m \times m$ orthogonal matrices of the form

$$Q_i = \begin{bmatrix} I_{i-1} & 0 \\ 0 & I_{m-i+1} - 2uu^T \end{bmatrix}, \quad \text{for } i = 1, \ldots, n.$$

Let $Q = Q_1 Q_2 \cdots Q_n$. Two possible algorithms for computing the matrix Q are the "forward accumulation" and "backward accumulation" methods below.

a Forward accumulation	**b** Backward accumulation
Initialize:	**Initialize:**
$Q = I_m$	$Q = I_m$
for $i = 1$ to n **do**	**for** $i = n$ to 1 **do**
$Q \leftarrow QQ_i$	$Q \leftarrow Q_i Q$
end for	**end for**

Suppose you only need the first k columns of the matrix Q, where $1 \leqslant k \leqslant m$. Explain why "backward accumulation" should be preferred over "forward accumulation". Show that "backward accumulation" requires about $4mnk - 2(m+k)n^2 + \frac{4}{3}n^3$ arithmetic operations to compute the first k columns of Q.

27.24 (Complex Householder reflectors) In analogy with the real case, we can define an $n \times n$ complex Householder reflector as

$$P = I_n - 2uu^*, \quad \text{where } \|u\|_2 = 1. \tag{27.14}$$

(a) Show that P is Hermitian ($P^* = P$).
(b) Show that P is unitary ($P^*P = I_n$).

Let $x, y \in \mathbb{C}^n$ be two nonzero complex vectors such that $\|x\|_2 = \|y\|_2 > 0$. As in the real case, consider the complex Householder reflector

$$P = I_n - 2uu^*, \quad \text{with } u = \frac{x - y}{\|x - y\|_2}. \tag{27.15}$$

(c) Show that if $Px = y$, then we must have $\mathrm{im}(y^*x) = 0$.
(d) Find two distinct complex vectors $x, y \in \mathbb{C}^2$ such that there is no reflector of the form (27.15) satisfying $Px = y$.

As an alternative to (27.15), consider the $n \times n$ complex matrix

$$U = \mathrm{sign}(x^*y)(I_n - 2uu^*), \quad \text{with } u = \frac{\mathrm{sign}(x^*y)x - y}{\|\mathrm{sign}(x^*y)x - y\|_2},$$

where the sign function is defined as

$$\mathrm{sign}(z) = \begin{cases} z/|z| & \text{if } z \neq 0, \quad \text{and} \\ 0 & \text{if } z = 0. \end{cases}$$

(e) Show that U is unitary, that is, $U^*U = I_n$.
(f) Show that $Ux = y$.

27.25 (The determinant of a rank-one perturbation of the identity) For any $t \in \mathbb{R}$ and $a, b \in \mathbb{R}^n$, show

$$\det(I_n + tab^T) = 1 + tb^T a.$$

Hint:

$$\begin{bmatrix} I_n & 0 \\ b^T & 1 \end{bmatrix} \begin{bmatrix} I_n + tab^T & ta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} I_n & 0 \\ -b^T & 1 \end{bmatrix} = \begin{bmatrix} I_n & ta \\ 0 & 1 + tb^T a \end{bmatrix}.$$

27.26 Compute by hand the QR factorization of the matrix

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & -2 & -1 \\ 1 & -2 & 2 \end{bmatrix}.$$

27.27 (The QR factorization of an upper-Hessenberg matrix) Let A be an $n \times n$ upper Hessenberg matrix. Describe an algorithm to compute the QR decomposition of A that requires $O(n^2)$ arithmetic operations.

Hint: use a sequence of $n - 1$ Givens rotators to reduce A to triangular form.

27.28 (Hadamard's inequality) Let $A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}$ be an $n \times n$ matrix, where a_i denotes the ith column of A. Show that

$$|\det(A)| \leqslant \prod_{i=1}^{n} \|a_i\|_2.$$

Hint: use the QR factorization of A.

27.29 Use Hadamard's inequality in Problem 27.28 to show that an $n \times n$ matrix A with ± 1 entries satisfies $|\det(A)| \leqslant n^{n/2}$. Conclude that Hadamard matrices (see Problem 27.9 for their definition) have the largest possible determinant among all matrices with ± 1 entries.

27.30 Apply the Householder algorithm to compute QR factorizations of the matrices from Examples 27.48 and 27.49.

27.31 (Computational complexity of the Householder algorithm) Show that applying the Householder algorithm to an $m \times n$ matrix requires approximately:

(a) $2mn^2 - \frac{2}{3}n^3$ arithmetic operations if we do not need to form the orthogonal matrix $Q = Q_1 \cdots Q_n$.
(b) $4m^2n - 2mn^2 + \frac{2}{3}n^3$ if we need to form Q explicitly.

Hint: Use Problem 27.22 to estimate the cost of computing R, and use Problem 27.23 to estimate the cost of explicitly computing Q.

27.32 Show that using the Householder algorithm to compute an orthonormal basis for the column space of a full column rank matrix A requires about $4mn^2 - \frac{2}{3}n^3$ operations to obtain the matrix Q in factor form, plus an additional $2mn^2 - \frac{2}{3}n^3$ operations to compute the first n columns of Q.

27.33 Count the number of arithmetic operations required to compute the QR decomposition of an $m \times n$ matrix if one uses Givens rotators instead of Householder reflectors.

27.34 (Gram-Schmidt process as triangular orthogonalization) The Gram-Schmidt process and the Householder algorithm both turn a matrix into one with orthonormal columns, but they do it in different ways. The Householder method uses orthogonal matrices on the left to make A upper triangular. In this problem, you will show that the Gram-Schmidt process does the opposite: it applies a sequence of n upper triangular matrices $R_1, R_2, \ldots, R_n$ to the right of A, gradually transforming its columns into an orthonormal set. This gives $AR_1R_2 \cdots R_n = Q$, where Q has orthonormal columns.

If $A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix} \in \mathbb{R}^{m \times n}$, show that the Gram-Schmidt process produces a matrix $Q = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}$ with orthonormal columns $q_1, \ldots, q_n$ such that

$$A \begin{bmatrix} 1/r_{11} & & & & & \\ & 1 & & & & \\ & & 1 & & & \\ & & & \ddots & & \\ & & & & 1 & \\ & & & & & 1 \end{bmatrix} \begin{bmatrix} 1 & -r_{11}/r_{22} & & & & \\ & 1/r_{22} & & & & \\ & & 1 & & & \\ & & & \ddots & & \\ & & & & 1 & \\ & & & & & 1 \end{bmatrix} \cdots \begin{bmatrix} 1 & & & -r_{1n}/r_{nn} \\ & 1 & & -r_{2n}/r_{nn} \\ & & 1 & -r_{3n}/r_{nn} \\ & & & \ddots \quad \vdots \\ & & & 1 \quad -r_{n-1,n}/r_{nn} \\ & & & 1/r_{nn} \end{bmatrix} = Q,$$

where $r_{ij} = q_i^T a_j$.

27.35 Let $S = \{w_1, \ldots, w_n\}$ be a linearly dependent set in $\mathbb{R}^m$. Modify the Gram-Schmidt process so that this modified version transforms S into an orthonormal set $\widetilde{S} = \{q_1, \ldots, q_r\}$ with the property $\operatorname{span}(\widetilde{S}) = \operatorname{span}(S)$.

27.36 Compute the QR factorization with column pivoting for the matrix from Example 27.48.

Chapter 28
VI-SUPPLEMENTARY TOPICS

28.1 Decomposing orthogonal matrices into products of Givens rotators and Householder reflectors

In two dimensions, we have seen that orthogonal matrices are either rotation or reflection matrices (see Equation (27.1)). In higher dimensions, orthogonal matrices come in more varieties. However, every orthogonal matrix can be expressed as a product of Givens rotators and Householder reflectors. The proof of this is in Theorem 28.2, but first, we need to establish the following lemma.

Lemma 28.1. (Embedded orthogonal, Householder reflector, or Givens rotator matrices)

Let $Q \in \mathbb{R}^{n\times n}$ be an orthogonal matrix, Householder reflector, or Givens rotator. Then, the matrix

$$\widehat{Q} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & Q & 0 \\ 0 & 0 & I_t \end{bmatrix} \in \mathbb{R}^{(n+s+t)\times(n+s+t)}$$

is also an orthogonal matrix, Householder reflector, or Givens rotator, respectively.

Proof First, notice that

$$\widehat{Q}^T\widehat{Q} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & Q^TQ & 0 \\ 0 & 0 & I_t \end{bmatrix},$$

which shows that $\widehat{Q}$ is orthogonal if and only if Q is orthogonal.

Next, suppose Q is a Householder reflector, so $Q = I_n - 2uu^T$, where $u \in \mathbb{R}^n$ is a unit vector. We can express $\widehat{Q}$ as

$$\widehat{Q} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & I_n - 2uu^T & 0 \\ 0 & 0 & I_t \end{bmatrix} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & I_n & 0 \\ 0 & 0 & I_t \end{bmatrix} - 2\begin{bmatrix} 0 \\ u \\ 0 \end{bmatrix}\begin{bmatrix} 0 & u^T & 0 \end{bmatrix} = I_{s+n+t} - 2vv^T,$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_28

where $v = \begin{bmatrix} 0 & u & 0 \end{bmatrix}^T$ is a unit vector. This shows that $\widehat{Q}$ is also a Householder reflector.

Conversely, assume that $\widehat{Q} = \text{diag}(I_s, Q, I_t)$ is a Householder reflector. So, $\widehat{Q} = I_{n+s+t} - 2vv^T$ for some unit vector $v \in \mathbb{R}^{n+s+t}$. Then, we have

$$\widehat{Q} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & Q & 0 \\ 0 & 0 & I_t \end{bmatrix} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & I_n & 0 \\ 0 & 0 & I_t \end{bmatrix} - 2 \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} \begin{bmatrix} v_1^T & v_2^T & v_1^T \end{bmatrix},$$

where $v_1 \in \mathbb{R}^s$, $v_2 \in \mathbb{R}^n$, and $v_3 \in \mathbb{R}^t$. Expanding this gives

$$\begin{bmatrix} I_s & 0 & 0 \\ 0 & Q & 0 \\ 0 & 0 & I_t \end{bmatrix} = \begin{bmatrix} I_s - 2v_1v_1^T & -2v_1v_2^T & -2v_1v_3^T \\ -2v_2v_1^T & I_n - 2v_2v_2^T & -2v_2v_3^T \\ -2v_3v_1^T & -2v_3v_2^T & I_t - 2v_3v_3^T \end{bmatrix}.$$

From this, we see that $v_1v_1^T = 0$ and $v_3v_3^T = 0$, which means v_1 and v_3 must be zero vectors. Therefore, we have

$$\widehat{Q} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & Q & 0 \\ 0 & 0 & I_t \end{bmatrix} = \begin{bmatrix} I_s & 0 & 0 \\ 0 & I_n & 0 \\ 0 & 0 & I_t \end{bmatrix} - 2 \begin{bmatrix} 0 \\ v_2 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & v_2^T & 0 \end{bmatrix}.$$

Hence, $Q = I_n - 2v_2v_2^T$, with $\|v_2\|_2 = \|v\|_2 = 1$, confirming Q is also a Householder reflector.

Finally, the result for Givens rotators follows directly from their definition. □

Theorem 28.2. (Orthogonal matrix decomposition)

Let $Q \in \mathbb{R}^{n \times n}$ be an orthogonal matrix. Then, the following statements hold:

(a) Q is the product of at most $n - 1$ plane reflector matrices.
(b) If $\det(Q) = 1$, then Q is the product of at most $n(n-1)/2$ Givens rotators.
(c) If $\det(Q) = -1$, then Q is the product of at most $n(n-1)/2$ Givens rotators followed by a single Householder reflector.

Proof We will prove part (c) by induction on the size n of the matrix Q. The proofs of parts (a) and (b) are left as exercises, as they follow similar reasoning.

The base case is $n = 2$. Recall that an orthogonal matrix Q has determinant $\det(Q) = \pm 1$, and when it is -1, Q is a reflector. In that case, we can write Q as

$$Q = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ \sin(\theta) & -\cos(\theta) \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

This expresses Q as the product of a rotation matrix and a reflection matrix. So, the result holds for 2×2 orthogonal matrices.

Assuming the result holds for $(n-1) \times (n-1)$ orthogonal matrices, we now aim to prove it for $n \times n$ orthogonal matrices. By Theorem 27.30, there exists a sequence of $n - 1$ Givens rotators $Q_1, \ldots, Q_{n-1}$ such that

$$Q_{n-1}\cdots Q_1 Q = \left[\begin{array}{c|c} \widehat{q}_{11} & \widehat{q}_{12}\ \cdots\ \widehat{q}_{1n} \\ \hline 0 & \\ \vdots & \widehat{Q} \\ 0 & \end{array}\right],$$

for some scalars $\widehat{q}_{1i}$ and a matrix $\widehat{Q} \in \mathbb{R}^{(n-1)\times(n-1)}$.

Since the product of orthogonal matrices is also orthogonal, the matrix $Q_{n-1}\cdots Q_1 Q$ is orthogonal. From the fact that the rows and columns of orthogonal matrices have unit norm, we conclude $\widehat{q}_{11} = 1$ and $\widehat{q}_{12} = \cdots = \widehat{q}_{1n} = 0$. Thus, $Q_{n-1}\cdots Q_1 Q$ is of the form $\mathrm{diag}(1, \widehat{Q})$, where $\widehat{Q}$ is an $(n-1)\times(n-1)$ orthogonal matrix by Lemma 28.1.

We also have

$$\det(\widehat{Q}) = \det\left(\mathrm{diag}(1, \widehat{Q})\right) = \det(Q_{n-1})\cdots\det(Q_1)\det(Q) = \det(Q) = -1,$$

because rotator matrices have determinant +1. By the induction hypothesis, we can expressed $\widehat{Q}$ as the product of at most $(n-1)(n-2)/2$ Givens rotators followed by a Householder reflector. So we have

$$\widehat{Q} = \widehat{Q}_{\frac{(n-1)(n-2)}{2}}\cdots\widehat{Q}_1\widehat{Q_0},$$

where $\widehat{Q_1}, \ldots, \widehat{Q}_{\frac{(n-1)(n-2)}{2}}$ are Givens rotators, and $\widehat{Q}_0$ is a Householder reflector. Thus, we can write

$$\begin{aligned} Q =& Q_1^T\cdots Q_{n-1}^T \begin{bmatrix} 1 & 0 \\ 0 & \widehat{Q} \end{bmatrix} = \\ & Q_1^T\cdots Q_{n-1}^T \begin{bmatrix} 1 & & 0 \\ 0 & \widehat{Q}_{\frac{(n-1)(n-2)}{2}} & \cdots\widehat{Q}_1\widehat{Q_0} \end{bmatrix} = \\ & Q_1^T\cdots Q_{n-1}^T \begin{bmatrix} 1 & & 0 \\ 0 & \widehat{Q}_{\frac{(n-1)(n-2)}{2}} & \end{bmatrix}\cdots\begin{bmatrix} 1 & 0 \\ 0 & \widehat{Q}_1 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & \widehat{Q}_0 \end{bmatrix}. \end{aligned}$$

The matrices $Q_1^T, \ldots, Q_{n-1}^T$ are Givens rotators. By Lemma 28.1, the matrices $\mathrm{diag}(1, \widehat{Q}_i)$, for $i = 1, \ldots, \frac{(n-1)(n-2)}{2}$, are also Givens rotators, and the matrix $\mathrm{diag}(1, \widehat{Q}_0)$ is a Householder reflector. Therefore, we have expressed the orthogonal matrix Q as the product of at most $n - 1 + \frac{(n-1)(n-2)}{2} = \frac{n(n-1)}{2}$ Givens rotators followed by a Householder reflectors, as required. □

28.2 Updating the QR factorization

Suppose we have already computed the QR factorization of a matrix A. In practice, we may need to update this factorization when the matrix changes. For example, when adding or removing rows (new data points or old ones discarded) or columns (adding or dropping features). The goal is to update the QR factorization of the

modified matrix without starting from scratch, reusing previous work to save time and computation.

In the next subsections, we will see how to efficiently update the full QR factorization of an $m \times n$ matrix $A = QR$ when adding or removing a row or column. We will focus on the case $m \geqslant n$, which is most relevant for the applications discussed later. We will outline the steps for updating the factorization, and in Problem 28.5, you will estimate the cost and compare it to recomputing the QR factorization from scratch.

Our main tool is the Givens rotator (27.2), which we will use to zero out entries one at a time. Proposition 27.28 shows how to eliminate a single entry in a 2-component vector:

$$Qx = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \pm\|x\|_2 \\ 0 \end{bmatrix}, \qquad \text{or}$$
$$x^T Q^T = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} = \begin{bmatrix} \pm\|x\|_2 & 0 \end{bmatrix}.$$

Throughout this section, we will use $\star$ to represent potentially nonzero entries, while the shaded entries will indicate the elements used to construct the Givens rotators.

28.2.1 Adding a row

Here, we show how to update the QR factorization $A = QR$ when a new row is added to A. For now, let us assume the new row is added to the bottom of the matrix. After this, we will explain how to handle cases where the new row is inserted elsewhere. The updated matrix can be written as

$$\widehat{A} = \begin{bmatrix} A \\ r^T \end{bmatrix},$$

where r^T is the newly added row. Our goal is to find the QR factorization of the matrix $\widehat{A}$ by using the QR factorization of A.

We start by expressing $\widehat{A}$ in terms of $A = QR$ and the added row r^T. We have

$$\widehat{A} = \begin{bmatrix} A \\ r^T \end{bmatrix} = \begin{bmatrix} QR \\ r^T \end{bmatrix} = \begin{bmatrix} Q & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R \\ r^T \end{bmatrix}.$$

This is almost a QR factorization. The $(m+1) \times (m+1)$ matrix $\operatorname{diag}(Q, 1)$ is orthogonal, as follows from Lemma 28.1. However, the $(m+1) \times n$ matrix $\left[\begin{smallmatrix} R \\ r^T \end{smallmatrix}\right]$ is not upper triangular due to the added row. For example, when $m = 5$ and $n = 4$, the pattern of the (possibly) nonzero entries in this matrix is

$$\begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ \star & \star & \star & \star \end{bmatrix}.$$

We will apply a sequence of n Givens rotators $Q_1, \ldots, Q_n$ to gradually transform the matrix $\begin{bmatrix} R \\ r^T \end{bmatrix}$ into upper triangular form. For our example, the intermediate matrices have the following structure

$$\begin{bmatrix} \boxed{\star} & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ \boxed{\star} & \star & \star & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \boxed{\star} & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & \boxed{\star} & \star & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \boxed{\star} & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{\star} & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \boxed{\star} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{\star} \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

For $i = 1, \ldots, n$, the i rotator matrix Q_i acts on rows i and $m + 1$, and produces a zero in position $(m + 1, i)$ without destroying any of the zeros that were already present.

At the end of the process, we have that the matrix

$$Q_n \cdots Q_2 Q_1 \begin{bmatrix} R \\ r^T \end{bmatrix} = \widehat{R}$$

is upper triangular. Hence,

$$\widehat{A} = \begin{bmatrix} Q & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R \\ r^T \end{bmatrix} = \underbrace{\begin{bmatrix} Q & 0 \\ 0 & 1 \end{bmatrix} (Q_1^T \cdots Q_n^T)}_{=\widehat{Q}} \underbrace{(Q_n \cdots Q_1) \begin{bmatrix} R \\ r^T \end{bmatrix}}_{=\widehat{R}} = \widehat{Q}\widehat{R}$$

is the QR factorization of $\widehat{A}$.

When the new row is inserted into a position other than the bottom, we can first use a permutation matrix P to move the new row to the bottom. This reduces the problem to the case we already discussed:

$$P\widehat{A} = \begin{bmatrix} A \\ r^T \end{bmatrix} = \widehat{Q}\widehat{R}.$$

Reversing the permutation gives the updated QR factorization $\widehat{A} = (P^T\widehat{Q})\widehat{R}$, where $P^T\widehat{Q}$ becomes the orthogonal factor.

28.2.2 Deleting a row

In this section, we show how to update the QR factorization of $A = QR$ when deleting a row from A.

Let us assume we are removing the k-th row. To do this, we use the $m \times m$ permutation matrix

$$P = \begin{bmatrix} I_{k-1} & 0 & 0 \\ 0 & 0 & I_{m-k} \\ 0 & 1 & 0 \end{bmatrix},$$

which moves the kth row to the bottom. This gives us

$$PA = \begin{bmatrix} \widehat{A} \\ r^T \end{bmatrix},$$

where r^T is the row being removed. Our goal is to update the QR factorization of A to obtain the QR factorization of $\widehat{A}$.

In the previous section, we discussed how to update the QR factorization when deleting a row. To factor $\widehat{A}$, we can reverse that process. Starting with $A = QR$, we have $PA = (PQ)R$. Then, we apply a sequence of $m-1$ Givens rotators $Q_1, \ldots, Q_{m-1}$ to the right of PQ, gradually transforming PQ into $\mathrm{diag}(\widehat{Q}, \pm 1)$, where $\widehat{Q}$ is an $(m-1) \times (m-1)$ orthogonal matrix. So, we need Givens rotators that satisfy

$$(PQ)Q_1 \cdots Q_{m-1} = \begin{bmatrix} \widehat{Q} & 0 \\ 0 & \pm 1 \end{bmatrix}.$$

Let us look at an example where $m = 6$. In this case, the sequence of matrices $(PQ)Q_1 \ldots Q_i$, for $i = 1, \ldots, m-1$, follows this structure:

$$PQ = \begin{bmatrix} \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & 0 & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & 0 & 0 & \star \end{bmatrix} \rightarrow$$

$$\begin{bmatrix} \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & 0 & 0 & 0 & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & \star & \star & \star & \star & \star \\ \star & 0 & 0 & 0 & 0 & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star & 0 \\ \star & \star & \star & \star & \star & 0 \\ \star & \star & \star & \star & \star & 0 \\ \star & \star & \star & \star & \star & 0 \\ \star & \star & \star & \star & \star & 0 \\ 0 & 0 & 0 & 0 & 0 & \pm 1 \end{bmatrix}.$$

The zeros in the last column appear at the end of the process because the final column has e unit norm.

In general, for $i = 1, \ldots, m-1$, the i-th Givens rotator Q_i acts on columns m and $m-i$, producing a zero at position $(m, m-i)$ without affecting any of the previously created zeros. By the end of this process, we obtain

$$(PQ)Q_1Q_2 \cdots Q_{m-1} = \begin{bmatrix} \widehat{Q} & 0 \\ 0 & \pm 1 \end{bmatrix},$$

where $\widehat{Q}$ is an $(m-1) \times (m-1)$ orthogonal matrix. The last entry of the bottom row is ± 1 because the matrix $(PQ)Q_1Q_2 \cdots Q_{m-1}$ is orthogonal, so its rows are unit vectors. For the same reason, the last column has zeros in every position above the bottom entry.

Using the QR factorization of the matrix A, we can write

$$\begin{aligned} PA = & PQR = \\ & (PQQ_1 \cdots Q_{m-1})(Q_{m-1}^T \cdots Q_1^T R) = \\ & \begin{bmatrix} \widehat{Q} & 0 \\ 0 & \pm 1 \end{bmatrix} (Q_{m-1}^T \cdots Q_1^T R). \end{aligned}$$

The $m \times n$ matrix R is initially upper triangular, but multiplying it on the left by the Givens rotators $Q_{m-1}^T, \ldots, Q_1^T$ disrupts this triangular structure. For example, with $m = 6$ and $n = 4$, the successive matrices $Q_i^T \cdots Q_1^T R$, for $i = 1, \ldots, m-1$, have the following structure

$$R = \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \star & \star \end{bmatrix} \rightarrow$$

$$\begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & \star & \star & \star \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star \\ 0 & \star & \star & \star \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ \star & \star & \star & \star \end{bmatrix}.$$

The blue-shaded rows show where each plane rotator acts. The first $m - n - 1$ rotators $Q_1^T, \ldots, Q_{m-n-1}^T$ have no effect since they act on rows that are already zero. For $i = m - n, \ldots, m - 1$, the i-th rotator Q_i^T acts on rows m and $m - i$, potentially introducing a nonzero entry at position $(m, m - i)$. Thus, we have

$$Q_{m-1}^T \cdots Q_1^T R = \begin{bmatrix} \widehat{R} \\ \widehat{r}^T \end{bmatrix},$$

where $\widehat{R}$ is a $(m-1) \times n$ upper triangular matrix and $\widehat{r}^T$ is a row vector. Finally, we obtain

$$PA = \begin{bmatrix} \widehat{A} \\ r^T \end{bmatrix} = \begin{bmatrix} \widehat{Q} & 0 \\ 0 & \pm 1 \end{bmatrix} \begin{bmatrix} \widehat{R} \\ \widehat{r}^T \end{bmatrix} = \begin{bmatrix} \widehat{Q}\widehat{R} \\ \pm \widehat{r}^T \end{bmatrix},$$

which gives us the QR factorization $\widehat{A} = \widehat{Q}\widehat{R}$.

28.2.3 Adding a column

In this section, we explain how to update the QR factorization when we add a new column to A.

Suppose we have $A = \begin{bmatrix} A_1 & A_2 \end{bmatrix}$ and want to add the column vector c. The updated matrix is

$$\widehat{A} = \begin{bmatrix} A_1 & c & A_2 \end{bmatrix} \in \mathbb{R}^{m \times (n+1)},$$

where c is the newly added column. The task is to compute the QR factorization of $\widehat{A}$ without recomputing it from scratch.

If we partition the upper triangular matrix $R = \begin{bmatrix} R_1 & R_2 \end{bmatrix}$ conformably with the partition of A, then we can write

$$A = \begin{bmatrix} A_1 & A_2 \end{bmatrix} = Q \begin{bmatrix} R_1 & R_2 \end{bmatrix} = \begin{bmatrix} QR_1 & QR_2 \end{bmatrix}.$$

Hence, we have

$$\widehat{A} = \begin{bmatrix} A_1 & c & A_2 \end{bmatrix} = \begin{bmatrix} QR_1 & c & QR_2 \end{bmatrix} = Q \begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix}.$$

The matrix Q is orthogonal, but the matrix $\begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix}$ is not upper triangular. For example, with $m = 6$ and $n = 4$ and the new column introduced at column 3, this matrix has the following structure

$$\begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix} = \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & \star & 0 & \star \\ 0 & 0 & \star & 0 & 0 \\ 0 & 0 & \star & 0 & 0 \end{bmatrix}.$$

We apply a sequence of Givens rotators $Q_1, Q_2, \ldots$ to $\begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix}$ to restore its upper triangular form. If c is in the k-th column of $\widehat{A}$, we need $m - k$ rotators to zero out the $m - k$ entries below the diagonal in $Q^T c$. For example, if $m = 6$, $n = 4$, and $k = 3$, the successive matrices $Q_i \cdots Q_1 \begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix}$ take the following form

$$\begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix} = \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & \star & 0 & \star \\ 0 & 0 & \star & 0 & 0 \\ 0 & 0 & \star & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & \star & 0 & \star \\ 0 & 0 & \star & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & \star & 0 & \star \\ 0 & 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

For $i = 1, \ldots, m - k$, each Givens rotator Q_i acts on rows $m + 1 - i$ and $m - i$, creating a zero in position $(m + 1 - i, k)$. While the rotators may introduce non-zero elements in some previously zero positions (shaded in red), the final matrix remains upper triangular.

After applying the $m - k$ Givens rotators, the matrix

$$Q_{m-k} \cdots Q_2 Q_1 \begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix}$$

becomes upper triangular. Therefore, we can write

$$\widehat{A} = \begin{bmatrix} A_1 & c & A_2 \end{bmatrix} = \underbrace{(Q Q_1^T Q_2^T \cdots Q_{m-k}^T)}_{=\widehat{Q}} \underbrace{(Q_{m-k} \cdots Q_2 Q_1 \begin{bmatrix} R_1 & Q^T c & R_2 \end{bmatrix})}_{=\widehat{R}} = \widehat{Q}\widehat{R},$$

which gives the QR factorization of the matrix $\widehat{A}$.

28.2.4 Deleting a column

In this section, we show how to update the QR factorization when a column is removed from the matrix.

Suppose we have a matrix A partitioned as $A = \begin{bmatrix} A_1 & a_k & A_2 \end{bmatrix}$, where a_k is the k-th column that we want to delete. Our goal is to find the QR factorization of the updated matrix

$$\widehat{A} = \begin{bmatrix} A_1 & A_2 \end{bmatrix},$$

which has dimensions $m \times (n-1)$.

We start by partitioning the upper triangular factor R to match the partition of A. We partition R as $R = \begin{bmatrix} R_1 & r_k & R_2 \end{bmatrix}$. Then, the QR factorization of A becomes $A = Q \begin{bmatrix} R_1 & r_k & R_2 \end{bmatrix}$. Therefore, we can write the updated matrix as

$$\widehat{A} = Q \begin{bmatrix} R_1 & R_2 \end{bmatrix}.$$

Notice that when $k \neq n$, the matrix $\begin{bmatrix} R_1 & R_2 \end{bmatrix}$ is no longer upper triangular. For instance, when $m = 7$, $n = 6$, and $k = 3$, this matrix has a lower trapezoidal shape with nonzero subdiagonal entries, as highlighted below:

$$\begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & \boxed{\star} & \star & \star \\ 0 & 0 & \boxed{0} & \boxed{\star} & \star \\ 0 & 0 & 0 & \boxed{0} & \boxed{\star} \\ 0 & 0 & 0 & 0 & \boxed{0} \end{bmatrix}.$$

We multiply from the left by a sequence of $n-k$ Givens rotators $Q_1, \ldots, Q_{n-k}$ to $\begin{bmatrix} R_1 & R_2 \end{bmatrix}$, selecting the rotators to eliminate the subdiagonal elements. The product $Q_{n-k} \cdots Q_1 \begin{bmatrix} R_1 & R_2 \end{bmatrix}$ restores the upper triangular form we need. For example, with $m = 7$, $n = 6$, and $k = 3$, the successive matrices $Q_i \cdots Q_1 \begin{bmatrix} R_1 & R_2 \end{bmatrix}$ have the following structure:

$$\begin{bmatrix} R_1 & R_2 \end{bmatrix} = \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \boxed{\star} & \star & \star \\ 0 & 0 & \boxed{\star} & \star & \star \\ 0 & 0 & 0 & \star & \star \\ 0 & 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & 0 & \boxed{\star} & \star \\ 0 & 0 & 0 & \boxed{\star} & \star \\ 0 & 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & 0 & \star & \star \\ 0 & 0 & 0 & 0 & \boxed{\star} \\ 0 & 0 & 0 & 0 & \boxed{\star} \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \star & \star \\ 0 & 0 & \star & \star & \star \\ 0 & 0 & 0 & \star & \star \\ 0 & 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

For $i = 1, \ldots, n-k$, the ith rotator matrix Q_i acts on rows $k+i-1$ and $k+i$, eliminating the subdiagonal entry at position $(k+i, k+i-1)$. After applying the sequence of plane rotators, the matrix

$$Q_{n-k} \cdots Q_2 Q_1 \begin{bmatrix} R_1 & R_2 \end{bmatrix}$$

becomes upper triangular. This gives us

$$\widehat{A} = Q \begin{bmatrix} R_1 & R_2 \end{bmatrix} = \underbrace{(Q Q_1^T Q_2^T \cdots Q_{n-k}^T)}_{=:\widehat{Q}} \underbrace{(Q_{n-k} \cdots Q_2 Q_1) \begin{bmatrix} R_1 & R_2 \end{bmatrix}}_{=:\widehat{R}} = \widehat{Q}\widehat{R},$$

which is the QR factorization of $\widehat{A}$.

28.3 Reducing matrices to simpler forms

In many applications, working with arbitrary matrices can be computationally expensive. To make these computations more efficient, we can reduce a matrix to a simpler form while preserving the key properties needed for the application. For example, in Chapters 16 and 22, we used row operations to reduce a matrix to upper triangular form when solving linear systems. This made the system easier to solve without changing the solution. Triangular form is not the only useful reduction. This section introduces two others: Hessenberg and bidiagonal forms, both constructed using orthogonal matrices.

28.3.1 Hessenberg reduction

Recall that an upper Hessenberg matrix has zeros below the first subdiagonal. For example,

$$A = \begin{bmatrix} 1 & 3 & 0 & 4 \\ 2 & 1 & 1 & -1 \\ 0 & 3 & 4 & -1 \\ 0 & 0 & 2 & 2 \end{bmatrix}$$

is a 4×4 upper Hessenberg matrix. In this section, we will refer to upper Hessenberg matrices simply as "Hessenberg" for convenience.

The Hessenberg reduction process reduces an $n \times n$ matrix A to Hessenberg form using orthogonal transformations. The goal is to find a Hessenberg matrix H that is orthogonally similar[1] to A. In other words, we want $H = QAQ^T$, where H is a Hessenberg matrix and Q is orthogonal. To do this, we apply a sequence of $n-2$ Householder reflectors, $Q_1, Q_2, \ldots, Q_{n-2}$, to A, gradually transforming it into the desired Hessenberg form . At the end, since Householder reflectors are symmetric and invertible (i.e., $Q_i^T = Q_i = Q_i^{-1}$), we get

$$\underbrace{Q_{n-2} \cdots Q_2 Q_1}_{=Q} A \underbrace{Q_1 Q_2 \cdots Q_{n-2}}_{=Q^T} = H,$$

where H is the resulting Hessenberg matrix.

[1] If you need a refresher, we cover similarity transformations in Chapter 36.

As we will see in Chapter 37, Hessenberg reduction is an important step in computing eigenvalues of a matrix.

The first step of the Hessenberg reduction

We begin with an $n \times n$ matrix A, as we show below, where $\star$ represents the (possibly) nonzero entries of A. Let $a_1 := A[2:n, 1]$ be the portion of the first column of A shaded in blue below.

$$A = \begin{bmatrix} \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \star & \star & \star & \cdots & \star \end{bmatrix}.$$

To perform the Hessenberg reduction, we first build an $(n-1) \times (n-1)$ Householder reflector $I_{n-1} - 2u_1 u_1^T$, choosing it so that $(I_{n-1} - 2u_1 u_1^T)a_1 = \pm\|a_1\|e_1$. Next, we embed this reflector into an $n \times n$ orthogonal matrix Q_1, given by

$$Q_1 := \begin{bmatrix} 1 & 0 \\ 0 & I_{n-1} - 2uu^T \end{bmatrix}.$$

Multiplying A from the left by Q_1 affects rows 2 through n, zeroing out the elements below the first subdiagonal in column 1. This gives the transformed matrix

$$Q_1 A = Q_1 \begin{bmatrix} \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \star & \star & \star & \cdots & \star \end{bmatrix} = \begin{bmatrix} \star & \star & \star & \star & \star \\ \times & \times & \times & \cdots & \times \\ 0 & \times & \times & \cdots & \times \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \times & \times & \cdots & \times \end{bmatrix},$$

where $\times$ denotes elements that may have changed in the transformation and are (possibly) nonzero. We then right-multiply $Q_1 A$ by Q_1, which only affects columns 2 through n, preserving all the zeros introduced in the previous step. This results in

$$A_1 := Q_1 A Q_1 = \begin{bmatrix} \star & \star & \star & \star & \star \\ \star & \star & \star & \cdots & \star \\ 0 & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \star & \cdots & \star \end{bmatrix} Q_1 = \begin{bmatrix} \star & \times & \times & \cdots & \times \\ \star & \times & \times & \cdots & \times \\ 0 & \times & \times & \cdots & \times \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \times & \times & \cdots & \times \end{bmatrix},$$

where, again, $\times$ represents elements that may have changed during the transformation are (possibly) nonzero.

Like the Householder algorithm for the QR factorization, Hessenberg reduction is a recursive process. After applying the first step to the first column of A, we repeat the process on the submatrix $A_1[2:n, 2:n]$. This gives us a orthogonal matrix Q_2

that zeros out the entries below the sub-diagonal in column 2. We continue this way, generating $n-2$ orthogonal matrices $Q_1, Q_2, \ldots, Q_{n-2}$ to zero out entries below the subdiagonal in each column. By applying these transformations, we turn A into a Hessenberg matrix.

Step i of the Hessenberg reduction

At the beginning of step i of the Hessenberg reduction process, the matrix $A_{i-1} = Q_{i-1} \cdots Q_1 A Q_1 \cdots Q_{i-1}$ takes the following form:

$$A_{i-1} = \begin{array}{c} \\ \\ \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array} \overset{\text{col } i}{\left[\begin{array}{cccccc|ccccc} \star & \star & \cdots & \star & \star & & \star & \star & \cdots & \star \\ \star & \star & \cdots & \star & \star & & \star & \star & \cdots & \star \\ 0 & \star & \ddots & \vdots & \vdots & & \vdots & \vdots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \star & \star & & \star & \star & \cdots & \star \\ 0 & \cdots & 0 & \star & \star & & \star & \star & \cdots & \star \\ \hline & & & & \star & & \star & \star & \cdots & \star \\ & & & & & & \star & \star & \cdots & \star \\ & & & 0 & & & \star & \star & \cdots & \star \\ & & & & & & \vdots & \vdots & \ddots & \vdots \\ & & & & & & \star & \star & \cdots & \star \end{array}\right]}$$

Let $a_i := A_{i-1}[i+1 : n, i]$ be the portion of the first column of $A_{i-1}[i : n, i : n]$, which is shaded in blue in the matrix above. In the ith step of the process, we construct an $(n-i) \times (n-i)$ Householder reflector $I_{n-i} - 2u_i u_i^T$ such that $(I_{n-i} - 2u_i u_i^T)a_i = \pm\|a_i\|e_1$. This Householder reflector is then embedded into an $n \times n$ orthogonal matrix Q_i, given by

$$Q_i = \begin{bmatrix} I_i & 0 \\ 0 & I_{n-i} - 2u_i u_i^T \end{bmatrix}.$$

When we multiply A_{i-1} by Q_i on the left, we affect only rows $i+1, \ldots, n$, so all the zeros that were produced in the previous steps 1 to $i-1$ are preserved. The orthogonal matrix Q_i zeros out the entries below the subdiagonal in the ith column of A_{i-1}, producing the matrix

$$Q_i A_{i-1} = \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \overset{\text{col } i}{\left[\begin{array}{ccccc|ccccc} \star & \star & \cdots & \star & \star & \star & \star & \cdots & \star \\ \star & \star & \cdots & \star & \star & \star & \star & \cdots & \star \\ 0 & \star & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \star & \star & \star & \star & \cdots & \star \\ 0 & \cdots & 0 & \star & \star & \star & \star & \cdots & \star \\ \hline & & & & \star & \star & \star & \cdots & \star \\ & & & & & \times & \times & \cdots & \times \\ & & 0 & & & 0 & \times & \cdots & \times \\ & & & & & \vdots & \vdots & \ddots & \vdots \\ & & & & & 0 & \times & \cdots & \times \end{array}\right]} \begin{array}{c} \\ \\ \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array},$$

where the elements denoted by $\times$'s have changed due to the transformation and may be nonzero, while the elements denoted by $\star$'s are unchanged.

Next, we right-multiply $Q_i A_{i-1}$ by Q_i, which affects only columns $i+1$ through n, as shown in the matrix $Q_i A_{i-1} Q_i$ below:

$$Q_i A_{i-1} Q_i = \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \overset{\text{col } i}{\left[\begin{array}{ccccc|ccccc} \star & \star & \cdots & \star & \star & \star & \times & \cdots & \times \\ \star & \star & \cdots & \star & \star & \star & \times & \cdots & \times \\ 0 & \star & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \star & \star & \star & \times & \cdots & \times \\ 0 & \cdots & 0 & \star & \star & \star & \times & \cdots & \times \\ \hline & & & & \star & \star & \times & \cdots & \times \\ & & & & & \star & \times & \cdots & \times \\ & & 0 & & & 0 & \times & \cdots & \times \\ & & & & & \vdots & \vdots & \ddots & \vdots \\ & & & & & 0 & \times & \cdots & \times \end{array}\right]} \begin{array}{c} \\ \\ \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array},$$

where, again, $\times$ denotes elements that may have changed in the transformation and are (possibly) nonzero.

Remark 28.3. (Why do we need to apply similarities?)

You might wonder why we use similarity transformations when we could get a Hessenberg matrix by applying Householder reflectors just on the left side of A. The reason is that we are preparing to compute eigenvalues. To do that, the transformation must preserve the eigenvalues, which is exactly what similarity transformations do. They leave the eigenvalues unchanged, as you may recall.

28.3.2 Bidiagonal reduction

Recall that an upper bidiagonal matrix has nonzero entries only on the diagonal and the first superdiagonal. Below are some examples:

$$\begin{bmatrix} \star & \star & 0 & 0 \\ 0 & \star & \star & 0 \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \end{bmatrix}, \quad \begin{bmatrix} \star & \star & 0 & 0 & 0 & 0 & 0 \\ 0 & \star & \star & 0 & 0 & 0 & 0 \\ 0 & 0 & \star & \star & 0 & 0 & 0 \\ 0 & 0 & 0 & \star & \star & 0 & 0 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} \star & \star & 0 & 0 \\ 0 & \star & \star & 0 \\ 0 & 0 & \star & \star \\ 0 & 0 & 0 & \star \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The first is a 4×4 square upper bidiagonal matrix, the second is a 4×7 wide-and-short bidiagonal matrix, and the third is a 6×4 tall-and-skinny bidiagonal matrix. The $\star$ symbols represent the entries that can be nonzero.

For simplicity, in this section, we will refer to upper bidiagonal matrices simply as bidiagonal matrices.

The bidiagonal reduction process transforms an $m \times n$ matrix A into a bidiagonal matrix using orthogonal transformations. The goal is to find a bidiagonal matrix orthogonally equivalent[2] to A by finding orthogonal matrices $Q \in \mathbb{R}^{m \times m}$ and $P \in \mathbb{R}^{n \times n}$ such that $B = QAP$.

To do this, we apply a sequence of Householder reflectors $Q_1, Q_2, \ldots, Q_s$ on the left and $P_1, P_2, \ldots, P_t$ on the right, where $s, t \leqslant \min\{m, n\}$. These reflectors gradually transform A into bidiagonal form.

At the end, we have

$$\underbrace{Q_s \cdots Q_2 Q_1}_{=:Q} A \underbrace{P_1 P_2 \cdots P_t}_{=:P} = B,$$

where B is bidiagonal. We obtain the orthogonal matrices Q and P as product of the Householder reflectors.

This reduction is an important step in computing the singular value decomposition (SVD), as we will cover in Chapters 47 and 48.

The first step of the bidiagonal reduction

We begin with an $m \times n$ matrix A, shown below, where $\star$ represents the (possibly) nonzero entries. Let $a_1 = A[1 : n, 1]$ be the first column of A, shaded in blue:

$$A = \begin{bmatrix} \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \star & \star & \star & \cdots & \star \end{bmatrix}.$$

[2] Two matrices A and B are orthogonally equivalent if there exist orthogonal matrices Q and P such that $B = QAP$. This equivalence is important because it preserves singular values, which we will study in Chapter 47.

To begin the bidiagonal reduction, we construct an $m \times m$ Householder reflector $Q_1 = I_m - 2u_1 u_1^T$. We choose this reflector so that $Q_1 a_1 = \pm\|a_1\|_2 e_1$. This reflector eliminates the elements below the main diagonal in the first column of A, resulting in the transformed matrix:

$$Q_1 A = Q_1 \begin{bmatrix} \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \star & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \star & \star & \star & \cdots & \star \end{bmatrix} = \begin{bmatrix} \star & \times & \times & \times & \times \\ 0 & \times & \times & \cdots & \times \\ 0 & \times & \times & \cdots & \times \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \times & \times & \cdots & \times \end{bmatrix},$$

where $\times$ denotes elements that may have changed in the transformation and are (possibly) nonzero.

Next, let $b_1 = (Q_1 A)[1, 2 : n]$ be the portion of the first row of $Q_1 A$ shaded in red below:

$$Q_1 A = \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \cdots & \star \\ 0 & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \star & \cdots & \star \end{bmatrix},$$

We now construct an $(n-1) \times (n-1)$ Householder reflector $I_{n-1} - 2v_1 v_1^T$ such that $(I_{n-1} - 2v_1 v_1^T) b_1^T = \pm\|b_1\|_2 e_1$. Next, we embed this into an $n \times n$ orthogonal matrix

$$P_1 = \begin{bmatrix} 1 & 0 \\ 0 & I_{n-1} - 2v_1 v_1^T \end{bmatrix}.$$

We then multiply $Q_1 A$ on the right by P_1, which affects only columns 2 through n and leaves all the previous zeros untouched. This also zeros out the entries to the right of the main superdiagonal in the first row, resulting in the following transformed matrix:

$$A_1 := Q_1 A P_1 = \begin{bmatrix} \star & \star & \star & \star & \star \\ 0 & \star & \star & \cdots & \star \\ 0 & \star & \star & \cdots & \star \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \star & \cdots & \star \end{bmatrix} P_1 = \begin{bmatrix} \star & \times & 0 & \cdots & 0 \\ 0 & \times & \times & \cdots & \times \\ 0 & \times & \times & \cdots & \times \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \times & \times & \cdots & \times \end{bmatrix},$$

where $\times$ denotes elements that may have changed in the transformation and are possibly nonzero.

The bidiagonal reduction algorithm, like the Householder algorithm and Hessenberg reduction, works recursively. After applying the first step to the matrix A, we repeat the process on the submatrix $A_1[2 : n, 2 : m]$. This continues with smaller submatrices until we obtain a bidiagonal matrix.

Step i of the bidiagonal reduction

At the start of step i in the bidiagonal reduction process, the matrix A_{i-1} has the form shown below. The shaded blue entries represent the first column a_i of the submatrix $A_{i-1}[i:m, i:n]$:

$$A_{i-1} = \left[\begin{array}{cc|cc} \star & \star\ 0 \cdots 0\ 0 & 0 & 0\ 0 \cdots 0 \\ 0 & \star\ \star \cdots 0\ 0 & 0 & 0\ 0 \cdots 0 \\ 0\ 0 & \star \cdots 0\ 0 & 0 & 0\ 0 \cdots 0 \\ \vdots & \vdots\ \vdots\ \ddots\ \vdots & \vdots & \vdots\ \vdots\ \ddots\ \vdots \\ 0\ 0\ 0 & \cdots \star\ \star & 0 & 0\ 0 \cdots 0 \\ 0\ 0\ 0 & \cdots 0\ \star & \star & 0\ 0 \cdots 0 \\ \hline & & \star & \star\ \star \cdots \star \\ & 0 & \star & \star\ \star \cdots \star \\ & & \vdots & \vdots\ \vdots\ \ddots\ \vdots \\ & & \star & \star\ \star \cdots \star \end{array}\right] \begin{array}{c} \\ \\ \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array} \quad \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array}$$

(col i)

In the ith step of the bidiagonal reduction process, we construct a Householder reflector $I_{m-i+1} - 2u_i u_i^T$ of size $(m-i+1) \times (m-i+1)$ that satisfies $(I_{m-i+1} - 2u_i u_i^T)a_i = \pm\|a_i\|_2 e_1$. We then embed this reflector into an $m \times m$ block matrix

$$Q_i = \begin{bmatrix} I_{i-1} & 0 \\ 0 & I_{m-i+1} - 2u_i u_i^T \end{bmatrix}.$$

Multiplying A_{i-1} by Q_i on the left only affects rows $i, i+1, \ldots, m$, keeping all the zeros from the previous $i-1$ steps intact. The matrix Q_i zeros out the entries below the main diagonal in the ith column of A_{i-1}, producing the matrix

$$Q_i A_{i-1} = \left[\begin{array}{cc|cc} \star & \star\ 0 \cdots 0 & 0 & 0\ 0 \cdots 0 \\ 0 & \star\ \star \cdots 0 & 0 & 0\ 0 \cdots 0 \\ 0\ 0 & \star \cdots 0 & \vdots & \vdots\ \vdots\ \ddots\ \vdots \\ \vdots & \vdots\ \ddots\ \ddots\ \star & 0 & 0\ 0 \cdots 0 \\ 0\ 0 & \cdots 0\ \star & \star & 0\ 0 \cdots 0 \\ \hline & & \star & \times\ \times \cdots \times \\ & 0 & 0 & \times\ \times \cdots \times \\ & & \vdots & \vdots\ \vdots\ \ddots\ \vdots \\ & & 0 & \times\ \times \cdots \times \end{array}\right] \begin{array}{c} \\ \\ \\ \\ \\ \\ \text{row } i \\ \\ \\ \end{array} \quad \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array}$$

(col i)

where $\times$ denotes elements that may have changed in the transformation and are (possibly) nonzero.

If a_i has only one entry, there are no entries below the diagonal to zero out. In this case, we do not need to construct the Householder reflector $I_{m-i+1} - 2u_i u_i^T$ or multiply A_{i-1} by Q_i.

Next, let $b_i = (Q_i A_{i-1})[i, i{+}1 : n]$, which represents the entries in the red-shaded portion of the submatrix $(Q_i A_{i-1})[i : m, i : n]$, as shown below:

$$Q_i A_{i-1} = \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array} \begin{array}{c} \text{col } i \\ \left[\begin{array}{cccc|cccc} \star & \star & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & \star & \star & \ddots & \vdots & 0 & 0 & \cdots & 0 \\ 0 & 0 & \star & \ddots & 0 & \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & \star & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & \star & \star & 0 & \cdots & 0 \\ \hline & & & & & \star & \star & \cdots & \star \\ & & & & & 0 & \star & \cdots & \star \\ & & 0 & & & \vdots & \vdots & \ddots & \vdots \\ & & & & & 0 & \star & \cdots & \star \end{array}\right] \end{array} .$$

We next construct an $(n-i) \times (n-i)$ Householder reflector $I_{n-i} - 2v_i v_i^T$ such that $(I_{n-i} - 2v_i v_i^T) b_i^T = \pm \|b_i\|_2 e_1$. We then embed this reflector into the $n \times n$ orthogonal matrix

$$P_i = \begin{bmatrix} I_i & 0 \\ 0 & I_{n-i} - 2v_i v_i^T \end{bmatrix}.$$

Multiplying $Q_i A_{i-1}$ on the right by P_i affects only columns $i+1$ through n, and keeps all previously produced zeros intact. It also zeros out the elements to the right of the main superdiagonal in row i. The resulting matrix is

$$Q_i A_{i-1} P_i = \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \text{row } i \\ \\ \\ \\ \end{array} \begin{array}{c} \text{col } i \\ \left[\begin{array}{cccc|cccc} \star & \star & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & \star & \star & \ddots & \vdots & 0 & 0 & \cdots & 0 \\ 0 & 0 & \star & \ddots & 0 & \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & \star & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & \star & \star & 0 & \cdots & 0 \\ \hline & & & & & \star & \times & \cdots & 0 \\ & & & & & 0 & \times & \cdots & \times \\ & & 0 & & & \vdots & \vdots & \ddots & \vdots \\ & & & & & 0 & \times & \cdots & \times \end{array}\right] \end{array} ,$$

where the symbol $\times$ denotes elements that may have been altered in the transformation and could be nonzero.

If b_i has only one entry, there are no entries to the right of the superdiagonal in row i to zero out. So, there is no need to construct the Householder matrix $I_{n-i} - 2v_i v_i^T$ or multiply $A_{i-1} Q_i$ by P_i.

Problems

28.1 (Rank-1 updates) Consider an $n \times n$ matrix of the form $A = R + uv^T$, where $R \in \mathbb{R}^{n \times n}$ is an upper triangular matrix and $u, v \in \mathbb{R}^n$ are (column) vectors. Describe an algorithm to compute the QR decomposition of A that requires $O(n^2)$ arithmetic operations.

28.2 (Updating for streaming data) In some applications, data points $x_1, x_2, \ldots$ arrive sequentially over time (e.g., x_1 at time 1, x_2 at time 2, and so on). As each new data point arrives, it is added as a new bottom row to the data matrix A. Describe a method to efficiently update the QR factorization of A as new data points arrive, rather than recomputing it from scratch at each step.

28.3 Prove parts (a) and (b) of Theorem 28.2.

28.4 Show that an $n \times n$ Hessenberg and orthogonal matrix Q is the product of at most $n - 1$ Givens rotators and (possibly) a Householder reflector.

28.5 Verify that the estimated operation counts in Table 28.1 for adding or deleting a row or column to an $m \times n$ matrix A are correct. Discuss when it makes sense to compute the QR factorization from scratch rather than updating the factors.

Action	**Cost to update R**	**Cost to update Q**
Adding a row	$3n^2$	$6nm$
Deleting a row	$3n^2$	$6m^2$
Adding a column (at position k)	$2m^2 + 3n^2 - 6nk + 3k^2$	$3(m-k)^2$
Deleting a column (at position k)	$3(n-k)^2$	$6m(n-k)$

Table 28.1: Costs of updating the QR factorization of an $m \times n$ matrix.

28.6 Show that reducing an $n \times n$ matrix to Hessenberg form requires approximately $\frac{10}{3}n^3$ arithmetic operations.

28.7 Show that reducing an $m \times n$ matrix to bidiagonal form requires approximately

$$\begin{cases} 4mn^2 - \frac{4}{3}n^3 & \text{if } m \geqslant n, \\ 4nm^2 - \frac{4}{3}m^3 & \text{if } m < n. \end{cases}$$

28.8 In the programming language of your choice, implement the procedures discussed in this chapter: updating QR factorizations, Hessenberg reduction, and bidiagonal reduction.

28.9 Let A and B be matrices with the same number of columns, where A has full column rank. Consider the QR factorizations:

$$\begin{aligned} A &= Q_A R_A, \\ \begin{bmatrix} R_A \\ B \end{bmatrix} &= Q_B R_B, \quad \text{and} \\ \begin{bmatrix} A \\ B \end{bmatrix} &= QR, \end{aligned}$$

where the diagonal entries of R, R_A, and R_B are all positive. Prove that $R = R_B$.

Chapter 29
VI-FROM THE CLASSROOM TO REAL LIFE

In this chapter, we show how to denoise or compress a time series. Before getting into the details, we will review some key ideas about orthogonal matrices and change of basis, tools that are useful well beyond signal processing.

Orthogonality and change of bases

In some situations, it is helpful to look at a problem from different perspectives. From the Linear Algebra point of view, this can sometimes be understood as a change of basis. For example, if we have data in $\mathbb{R}^n$ and want to remove some unwanted information (like noise), changing the basis might make it easier to identify what to remove. After that, we apply the inverse change of basis to bring the simplified data back to its original form. It is important to be able to change and reverse bases easily and efficiently.

Suppose $\beta = \{w_1, w_2, \ldots, w_n\}$ is a basis for $\mathbb{R}^n$. The matrix that changes coordinates from the basis β to the canonical basis $\{e_1, e_2, \ldots, e_n\}$ is the matrix $W = \begin{bmatrix} w_1 & w_2 & \cdots & w_n \end{bmatrix}$, whose columns are the basis vectors. Thus, if c is a vector in $\mathbb{R}^n$ expressed in the basis β, the vector

$$v = Wc$$

gives its coordinates in the canonical basis. Conversely, to find the coordinates c of a vector v in the new basis, we need to compute

$$c = W^{-1}v.$$

If the vectors in β are orthogonal (or more specifically, orthonormal, which we will review in Chapter 26), the last calculation simplifies. Instead of computing $c = W^{-1}v$, you can use the simpler formula $c = W^T v$ (in the orthonormal case) or $c = D^{-1}W^T v$ for some diagonal matrix D (in the orthogonal case). So, instead of using the inverse, you just use the transpose. Not a bad trade-off for having orthonormal vectors!

This idea is key for signal denoising and compression, like in video and audio. We can quickly remove noise or compress long signals into smaller ones while keeping enough information so that there is no noticeable loss in quality. First, we express

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_29

the signal v in a special basis by computing $c = W^{-1}v$. To reconstruct the signal, we calculate $v = Wc$.

29.1 Denoising and compressing data: a dimensionality reduction technique

In this section, we return to the heart rate data example from Chapter 25, where the goal was to remove noise from a time series. Before applying the method to the data, we first review the discrete Haar transform.

29.1.1 The Haar basis and the Haar transform

Haar functions[1] are defined on the interval $[0, 1]$. The two main functions are the Haar scaling function ψ (the father function) and the mother Haar function ϕ. All other Haar functions are generated by shifting and scaling the mother function.

Definition 29.1. (Haar functions)

The **father Haar or scaling function** ψ is the constant function 1 over the interval $[0, 1]$. The **mother Haar function** is given by:

$$\phi(x) := \begin{cases} 1 \text{ if } 0 \leqslant x < \frac{1}{2}, \\ -1 \text{ if } \frac{1}{2} \leqslant x < 1, \text{and} \\ 0 \text{ if } x = 1. \end{cases}$$

For a nonnegative integer j and an integer k such that $0 \leqslant k \leqslant 2^j - 1$, we define the (j, k)**-Haar function** as

$$\phi_{jk}(x) := \phi(2^j x - k), \quad x \in [0, 1].$$

We call $\{\psi, \phi_{00}, \phi_{10}, \phi_{11}, \ldots\}$ the set of **Haar functions** or the **Haar basis**[2].

Note that the Haar function ϕ_{00} is the same as the mother Haar function ϕ.

Figure 29.1 shows the first eight Haar functions over the interval $[0, 1]$. We create each Haar function ϕ_{jk} by scaling and shifting the mother function ϕ. The index j controls the frequency. As j increases, the functions oscillate more rapidly. The index k shifts them along the interval.

The Haar basis is useful for representing a certain class of functions, especially in applications like time series analysis. We will define that class of functions next.

[1] Named after the Hungarian mathematician Alfréd Haar, who introduced them as part of his work on orthogonal functions.

[2] Haar functions are the simplest example of wavelets, which are wave-like oscillating functions used to extract information from various types of data. While Haar functions are not commonly used in practice due to certain limitations beyond the scope of this book, they are a fantastic academic example because they capture all the key concepts of wavelets.

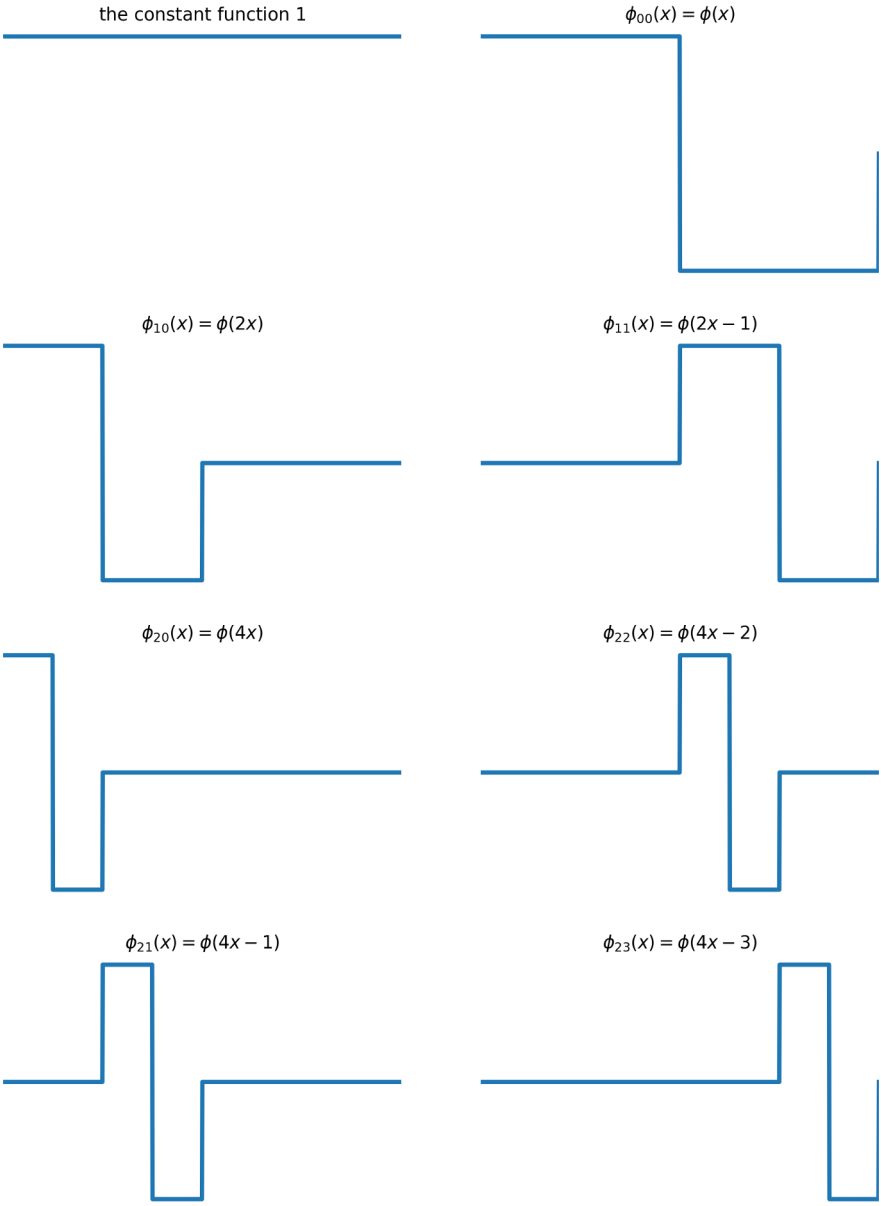


Fig. 29.1: The graphs of the first eight Haar functions over the interval $[0, 1]$.

Definition 29.2. (Square integrable functions)

The space $L^2([0,1])$ consists of all real functions $f : [0,1] \mapsto \mathbb{R}$ such that

$$\int_0^1 |f(x)|^2 \, dx$$

exists and is finite. We say a function $f(x)$ is **square-integrable** over $[0,1]$ if it belongs to $L^2([0,1])$.

Theorem 29.3. (The Haar expansion)

The family

$$\{\psi\} \cup \{\phi_{jk} : j = 0, 1, \ldots, k = 0, 1, \ldots, 2^j - 1\}$$

is an orthogonal basis for $L^2([0,1])$. Consequently, for every $f \in L^2([0,1])$, there exist unique coefficients c_0 and c_{jk} such that

$$f = c_0 \psi + \sum_{j=0}^{\infty} \sum_{k=0}^{2^j-1} c_{jk} \phi_{jk}, \tag{29.1}$$

where the series converges to f in the L^2-norm. We call the coefficients c_0 and c_{jk} the Haar coefficients of f, and their collection is called the Haar transform of f.

The Haar expansion (29.1) is similar to the Fourier expansion, if you are familiar with it. In a Fourier expansion, we break down a function into a sum of cosines and sines with different frequencies:

$$f(x) = a_0 + \sum_{k=1}^{\infty} \left(a_k \cos(2\pi k x) + b_k \sin(2\pi k x)\right).$$

We use the Fourier transform to split a signal into its high-frequency details (quick changes) and low-frequency trends (slow changes). The Haar expansion (29.1) does something similar, but with an important difference. As you can see in Figure 29.1, Haar functions not only capture different frequencies, but they also focus on specific parts of the interval $[0,1]$. This means the Haar expansion captures both how fast things are changing and where those changes happen.

Just like with the Fourier transform, we can approximate a signal using the Haar transform in two ways: keep only the largest coefficients to capture key details, or focus on the low-frequency components to represent the broader trends. Both approaches reduce the number of terms needed for the approximation.

In practice, we work with discrete data, that is, with vectors rather than continuous functions, so we use the discrete version of the Haar transform.

29.1.2 The discrete Haar transform

The Haar transform assumes we have a function f defined over the interval $[0, 1]$. In practice, we often work with discrete data, given as a vector. To apply the Haar transform to a vector, we need to discretize the Haar functions by approximating them with a set of values, so they can also be represented as vectors.

To discretize a Haar function, we choose a finite set of points on the interval $[0, 1]$ and sample the function at those points. A common method uses **dyadic sampling**, which divides the interval into sections with lengths that are powers of 2. We then sample the function at the midpoint of each section.

Example 29.4. (Level-0 discretization of the Haar functions)

In level-0 discretization, we divide the interval $[0, 1]$ into two equal parts of length $\frac{1}{2}$ and sample the scaling function ψ and the Haar function ϕ_{00} at the midpoint of each part, as shown in Figure 29.2.

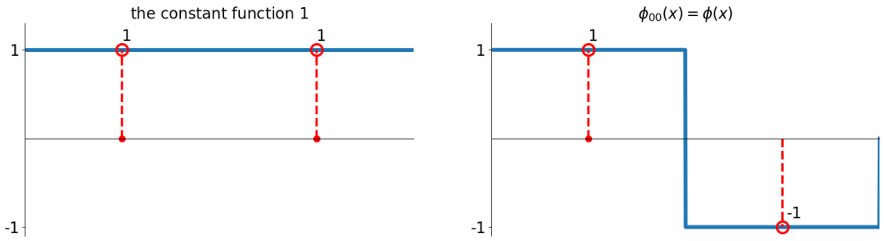


Fig. 29.2: Level-0 discrete Haar functions.

This gives us the following discrete representation of the scaling and Haar mother functions ψ and ϕ_{00}:

$$\psi \to \begin{bmatrix} \psi(1/4) \\ \psi(3/4) \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \qquad \phi_{00} \to \begin{bmatrix} \phi_{00}(1/4) \\ \phi_{00}(3/4) \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

The discretization in Example 29.4 produces the two vectors

$$\{w_1, w_2\} = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\},$$

which form an orthogonal basis for $\mathbb{R}^2$. We can express any vector $f = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix}$ as a linear combination of w_1 and w_2, that is, there exist constants c_1 and c_2 such that $f = c_1 w_1 + c_2 w_2$. Since w_1 and w_2 are orthogonal, we find the coefficients by

$$c_1 = \frac{w_1^T f}{w_1^T w_1} = \frac{f_1 + f_2}{2}, \quad \text{and}$$
$$c_2 = \frac{w_2^T f}{w_2^T w_2} = \frac{f_1 - f_2}{2}.$$

So we can express f as

$$f = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix} = \left(\frac{f_1 + f_2}{2}\right) w_1 + \left(\frac{f_1 - f_2}{2}\right) w_2.$$

From the formulas above, we can interpret both coefficients. The coefficient c_1 gives the average of the function values, while c_2 captures the difference between them.

Example 29.5. (Level-1 discretization of the Haar functions)

At level-1 of the Haar transform, we divide the interval $[0, 1]$ into four subintervals, each of length $\frac{1}{4}$, as shown in Figure 29.3. We then sample the functions ψ, ϕ_{00}, ϕ_{10}, and ϕ_{11} at the midpoints of these subintervals.

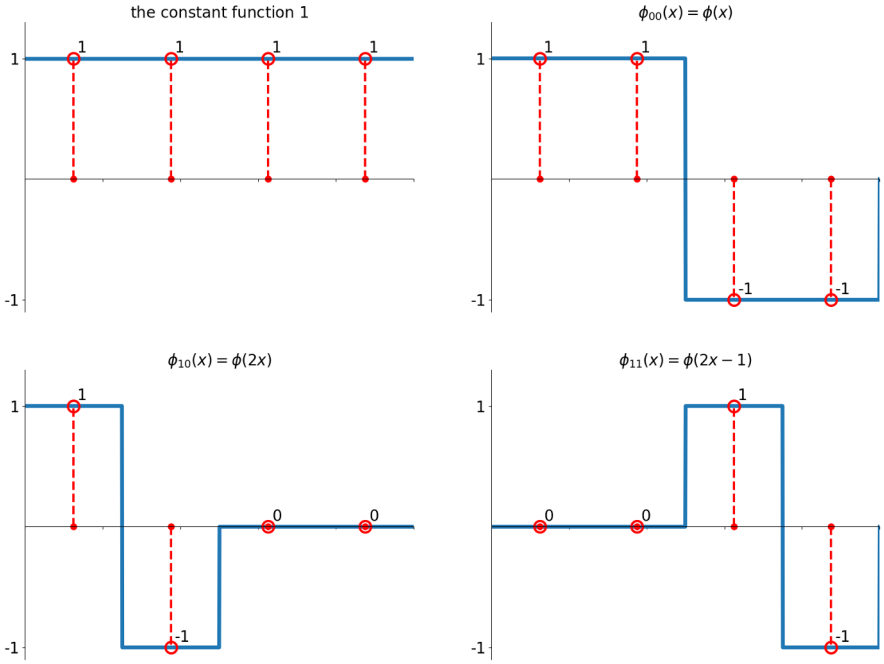


Fig. 29.3: Level-1 discrete Haar functions.

For each function $h \in \{\psi, \phi_{00}, \phi_{10}, \phi_{11}\}$, we get a discrete representation

$$h \to \begin{bmatrix} h(1/8) \\ h(3/8) \\ h(5/8) \\ h(7/8) \end{bmatrix}.$$

Each function is now represented as a vector in $\mathbb{R}^4$, instead of $\mathbb{R}^2$ like in level 0.

The discretization procedure in this example produces the following set of four vectors

$$\{w_1, w_2, w_3, w_4\} = \left\{ \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ -1 \end{bmatrix} \right\},$$

which forms an orthogonal basis for $\mathbb{R}^4$. We can express any vector $f = \begin{bmatrix} f_1 & f_2 & f_3 & f_4 \end{bmatrix}^T$ as a linear combination $f = c_1 w_1 + c_2 w_2 + c_3 w_3 + c_4 w_4$. Since the vectors are orthogonal, the coefficients are calculated as

$$\begin{aligned} c_1 &= \frac{w_1^T f}{w_1^T w_1} = \frac{f_1 + f_2 + f_3 + f_4}{4}, \\ c_2 &= \frac{w_2^T f}{w_2^T w_2} = \frac{f_1 + f_2 - f_3 - f_4}{4}, \\ c_3 &= \frac{w_3^T f}{w_3^T w_3} = \frac{f_1 - f_2}{2}, \quad \text{and} \\ c_4 &= \frac{w_4^T f}{w_4^T w_4} = \frac{f_3 - f_4}{2}. \end{aligned}$$

Notice that c_1 is the average of the function values across all four subintervals, c_2 is the difference between the averages of the first two and the last two subintervals, c_3 is the difference between the values in the first and second subintervals, and c_4 is the difference between the values in the third and fourth. In short, c_1 gives a broad summary of f, c_2 compares the two halves, and c_3 and c_4 capture finer details within each half.

We extend the discretization to higher levels by dividing the interval $[0, 1]$ into smaller subintervals and computing the coefficients using the same steps as before.

Level-ℓ discretization of the Haar functions

At level ℓ, we divide the interval $[0, 1]$ into $2^{\ell+1}$ equal subintervals of length $\frac{1}{2^{\ell+1}}$. We then evaluate the functions ψ and ϕ_{jk}, for $j = 0, \ldots, \ell$ and $k = 0, 1, \ldots, 2^j - 1$, at the midpoint of each subinterval.

For each function $h \in \{\psi\} \cup \{\phi_{jk} \mid j = 0, \ldots, \ell \text{ and } k = 0, 1, \ldots, 2^j - 1\}$, we get the discretized representation

$$h \to \begin{bmatrix} h\left(\frac{1}{2^{\ell+2}}\right) \\ h\left(\frac{3}{2^{\ell+2}}\right) \\ \vdots \\ h\left(\frac{2^{\ell+2}-1}{2^{\ell+2}}\right) \end{bmatrix}.$$

At this level, each function is represented by $2^{\ell+1}$ coefficients, compared to 2 at level 0 or 4 at level 1.

The level-ℓ discretization produces a set of $2^{\ell+1}$ vectors

$$\{w_1, w_2, \ldots, w_{2^{\ell+1}}\},$$

which, as we will show in Theorem 29.8, form an orthogonal (but not orthonormal) basis for $\mathbb{R}^{2^{\ell+1}}$. If we arrange the vectors w_i as the columns of a matrix, we get the **Haar matrix of order** ℓ:

$$W_\ell := \begin{bmatrix} w_1 & w_2 & \cdots & w_{2^{\ell+1}} \end{bmatrix} \in \mathbb{R}^{(2^{\ell+1})\times(2^{\ell+1})}.$$

Example 29.6 The Haar matrices of orders 0, 1 and 2 are, respectively,

$$W_0 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix},$$

$$W_1 = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & -1 & 0 \\ 1 & -1 & 0 & 1 \\ 1 & -1 & 0 & -1 \end{bmatrix}, \quad \text{and}$$

$$W_2 = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & -1 & 0 & 0 & 0 \\ 1 & 1 & -1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & -1 & 0 & 0 & -1 & 0 & 0 \\ 1 & -1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & -1 & 0 & 1 & 0 & 0 & -1 & 0 \\ 1 & -1 & 0 & -1 & 0 & 0 & 0 & 1 \\ 1 & -1 & 0 & -1 & 0 & 0 & 0 & -1 \end{bmatrix}.$$

We can construct the Haar matrix of order ℓ from the Haar matrix of order $\ell - 1$ To see how this works, let us first see how to obtain W_1 from W_0.

To build the Haar matrix of order 1 from the matrix of order 0, we divide each interval of length $1/2$ into two smaller intervals of length $1/4$ (see Figures 29.2 and 29.3). This step doubles the number of sample points, repeating the function

values from the original Haar functions ψ and ϕ_{00} over the finer grid. As a result, the first two columns of W_1 simply repeat the values in W_0, stretched to fit the finer resolution:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix} = W_0 \otimes \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

where $\otimes$ denotes the Kronecker product (see Definition 8.4). Next, the functions ϕ_{10} and ϕ_{11} are scaled and shifted versions of ϕ_{00}, supported on $[0, \frac{1}{2}]$ and $[\frac{1}{2}, 1]$, respectively. Their sampled values form the last two columns of W_1:

$$\begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix} = I_2 \otimes \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Putting everything together, we get the Haar matrix of order 1:

$$W_1 = \begin{bmatrix} W_0 \otimes \begin{bmatrix} 1 \\ 1 \end{bmatrix} & I_2 \otimes \begin{bmatrix} 1 \\ -1 \end{bmatrix} \end{bmatrix}.$$

We can extend this method of constructing the Haar matrix W_1 from W_0 to any order ℓ. This gives us a recursive formula for W_ℓ.

Theorem 29.7. (Haar recurrence)

The Haar matrix of order ℓ, denoted by W_ℓ, can be expressed as

$$W_\ell = \begin{bmatrix} W_{\ell-1} \otimes \begin{bmatrix} 1 \\ 1 \end{bmatrix} & I_{2^\ell} \otimes \begin{bmatrix} 1 \\ -1 \end{bmatrix} \end{bmatrix}. \tag{29.2}$$

Using the Haar recurrence formula (29.2), we can construct the Haar matrix for any order ℓ. This formula also helps prove important properties of the Haar matrices.

Theorem 29.8. (Properties of Haar matrices)

Let W_ℓ be the Haar matrix of order ℓ. Let $D_0 := 2I_2 = \left[\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right]$ and, for $\ell \geqslant 1$, define

$$D_\ell := 2 \begin{bmatrix} D_{\ell-1} & 0 \\ 0 & I_{2^\ell} \end{bmatrix}.$$

Then, for all $\ell \geqslant 0$, we have the following properties:

(a) The matrix D_ℓ is diagonal and invertible,
(b) $W_\ell^T W_\ell = D_\ell$,
(c) W_ℓ is invertible with $W_\ell^{-1} = D_\ell^{-1} W_\ell^T$, and
(d) the columns of W_ℓ are pairwise orthogonal.

Proof We will leave the proof of Theorem 29.8 for you to work through (see Problem 29.3). □

Theorem 29.8 highlights a key feature of the Haar matrix W_ℓ. Finding its inverse is straightforward. It is essentially just the transpose, since D_ℓ is diagonal.

In Problem 29.4, you will work out the explicit formula for D_ℓ^{-1}.

Since the Haar matrix W_ℓ is invertible, we can express any vector $f \in \mathbb{R}^{2^{\ell+1}}$ as a linear combination of the vectors $w_1, w_2, \ldots, w_{2^{\ell+1}}$ using the Haar matrix:

$$f = c_1 w_1 + \cdots + c_{2^{\ell+1}} w_{2^{\ell+1}} = W_\ell c,$$

where $c = \begin{bmatrix} c_1 & c_2 & \cdots & c_{2^{\ell+1}} \end{bmatrix}^T$. This leads to the following definition.

Definition 29.9. (The discrete Haar transform)

For a vector $f \in \mathbb{R}^{2^{\ell+1}}$, we define

$$c = \begin{bmatrix} c_1 \\ \vdots \\ c_{2^{\ell+1}} \end{bmatrix} := W_\ell^{-1} f \in \mathbb{R}^{2^{\ell+1}}$$

as the **level-ℓ discrete Haar transform of** f, or simply the Haar transform of f when the level is clear from context.

The discrete Haar transform is defined for vectors whose length is a power of two. If a vector f does not meet this condition, we pad it with zeros until its length is the next power of two before applying the transform.

By Theorem 29.8, calculating the Haar transform $c = W_\ell^{-1} f$ takes $O(n^2)$ operations, where $n = 2^{\ell+1}$ is the size of the vector f. This is much faster than the general case for matrices, where finding the inverse or solving $Wc = f$ using the LU factorization requires $O(n^3)$ operations [3].

We can reconstruct f from its Haar transform c by computing $f = W_\ell c$.

The Haar transform decomposes the information in the vector f into different levels of detail. The first coefficients show the broad, overall pattern of the entries. The last coefficients focus on finer variations between the entries. Let us see an example to understand this better.

Example 29.10 Consider the vector

$$f = \begin{bmatrix} 3.0 & -1.8 & 1.5 & 2.3 & 9.8 & 9.2 & 5.7 & 15.3 \end{bmatrix}^T.$$

[3] This efficiency is why the Haar basis and similar transforms are widely used for fast audio and video compression. In fact, with some smart techniques, we can even compute the Haar transform in $O(n)$ operations!

To compute its discrete Haar transform, we use the level-2 Haar matrix W_2. From Definition 29.9, we calculate the Haar transform coefficients as

$$c = W_2^{-1} f = \begin{bmatrix} 5.6 & -4.4 & -0.6 & -0.5 & 2.4 & -0.4 & 0.3 & -4.8 \end{bmatrix}^T.$$

Figure 29.4 shows both the original vector f and its Haar transform coefficients c. Notice that c has four large coefficients (in magnitude) and four small ones. The larger ones are c_1, c_2, c_5, and c_8, each capturing a different aspect of f. The coefficients c_1 and c_2 focus on the broader patterns:

- c_1 captures the average of the entries of f.
- c_2 captures the difference between the first and second halves of f.

On the other hand, the coefficients c_5 and c_8 focus on finer details:

- c_5 captures the difference between the first and second entries of f.
- c_8 captures the difference between the seventh and eighth entries of f.

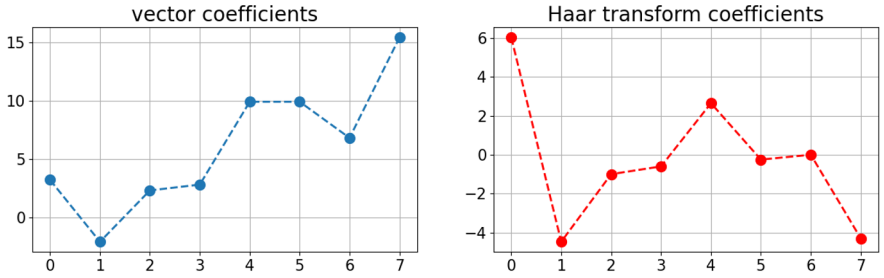


Fig. 29.4: The vector f and its Haar transform c in Example 29.10.

29.1.3 Compressing and removing noise with the Haar transform

We can use the Haar transform to compress data by expressing the vector f as a combination of fewer Haar vectors. The idea is to set the smaller coefficients in c to zero, reducing the data needed to represent f, which is a form of dimensionality reduction. More precisely, we can compress f by either keeping only the k largest coefficients or those above a chosen threshold in the Haar transform c, and setting the rest to zero. This gives us an approximate Haar transform $\widetilde{c}$. We then approximate f by computing $\widetilde{f} = W_\ell \widetilde{c}$.

A smaller k or a larger threshold gives a rougher approximation with less data, while a larger k or a smaller threshold provides a better approximation but requires more data since we need to keep more coefficients.

Example 29.11 Consider the vector f and its Haar transform c from Example 29.10. We can reduce the amount of data needed to store f by compressing it. To

do this, we set the smallest coefficients of c to zero. For example, by setting the four smallest coefficients of c to zero, we get the approximate Haar transform

$$\widetilde{c} = \begin{bmatrix} 5.6 & -4.4 & 0 & 0 & 2.4 & 0 & 0 & -4.8 \end{bmatrix}^T .$$

The corresponding approximate vector $\widetilde{f}$ is

$$\widetilde{f} = W_\ell \widetilde{c} = \begin{bmatrix} 3.6 & -1.2 & 1.2 & 1.2 & 10.0 & 10.0 & 5.2 & 14.8 \end{bmatrix}^T .$$

Figure 29.5 shows both the eight entries of the original vector f and its approximation $\widetilde{f}$. The approximate vector is visually similar to the original, but requires storing only four coefficients instead of eight.

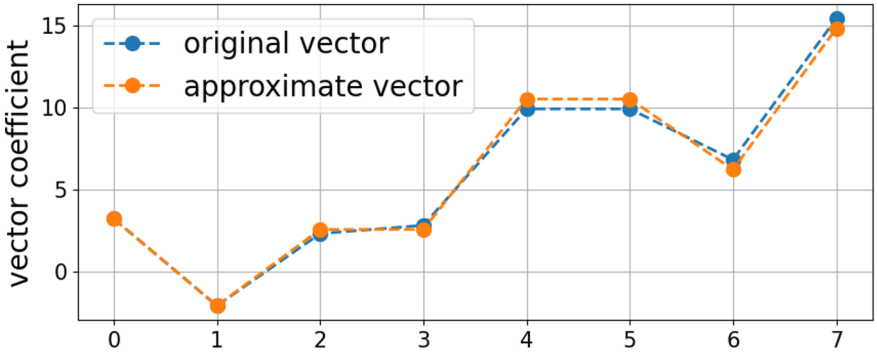


Fig. 29.5: The vector f in Example 29.11 and its approximation $\widetilde{f}$ obtained by setting the four smallest Haar transform coefficients of f to zero.

Example 29.12. (Compressing the heart rate data)

Let us revisit the heart rate data from Chapter 25 and use the Haar transform to compress it. Since the length of the heart rate data vector f is not a power of 2, we first add 306 zeros to the end, making it a length of 1024, which is 2^{10}. We compute the Haar transform $c = W_\ell^{-1} f$ with $\ell = 9$, then set to zero all coefficients with absolute value less than 8. The threshold was chosen by examining the sizes of the coefficients and selecting the largest value that still gave a good approximation. This sets 986 coefficients to zero, significantly reducing the data size. Finally, we reconstruct the compressed data from the remaining nonzero coefficients. Figure 29.6 compares the original data (top) to the reconstructed data (bottom). The reconstructed version closely resembles the original. Important details, like the times of physical activity, remain clear.

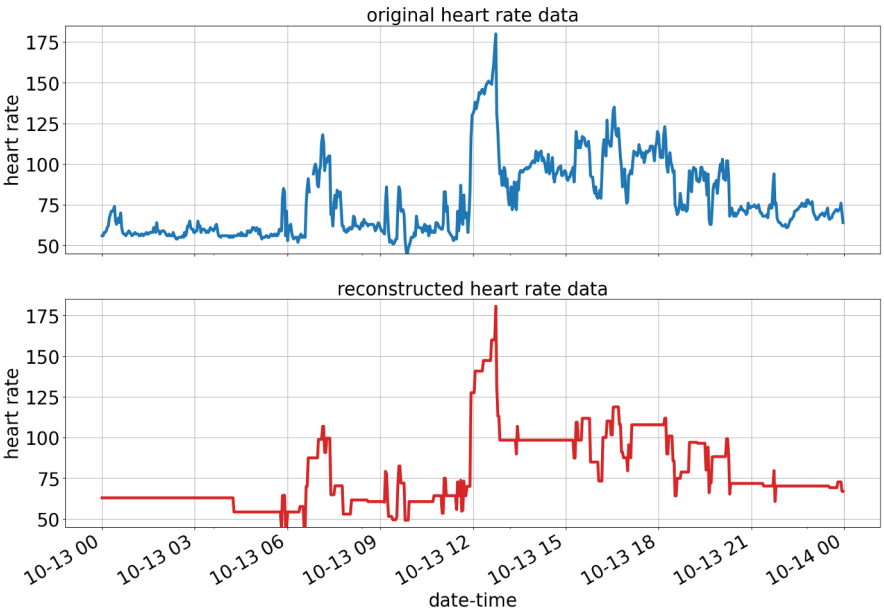


Fig. 29.6: Heart rate time series data over the course of one day and its Haar reconstruction.

We can also use the Haar transform to remove noise from our data by focusing on the low-frequency components. To do this, we keep only the first $k < n$ coefficients of the Haar transform c and set the rest to zero. This gives us an approximate Haar transform $\widetilde{c}$, which we can use to generate the approximate vector $\widetilde{f} = W_\ell \widetilde{c}$.

Example 29.13. (Removing noise from the heart rate data)

We now apply the Haar transform to the heart rate data from Chapter 25 to remove noise and better detect changes in heart rate throughout the day. After computing the transform, we filter out the high-frequency components and keep the low-frequency ones. We then reconstruct the signal using the remaining coefficients.

In Figure 29.7, the reconstructed data (in red) is shown for different numbers k of kept frequencies (coefficients): $k = 10$ (top-left), $k = 25$ (top-right), $k = 50$ (bottom-left), and $k = 100$ (bottom-right), overlaid on the original data (in blue). The fewer frequencies we keep, the smoother the reconstruction. The more frequencies we keep, the closer it is to the original data. With $k = 100$, we keep the key trends, like walking, biking, and sleeping. The data goes from 720 values down to just 100, while still keeping the important information.

The Haar transform is just one method for compressing or denoising signals. Other common options include the discrete cosine transform (DCT) and the discrete Fourier transform (DFT). Which one works best depends on the signal and your goal.

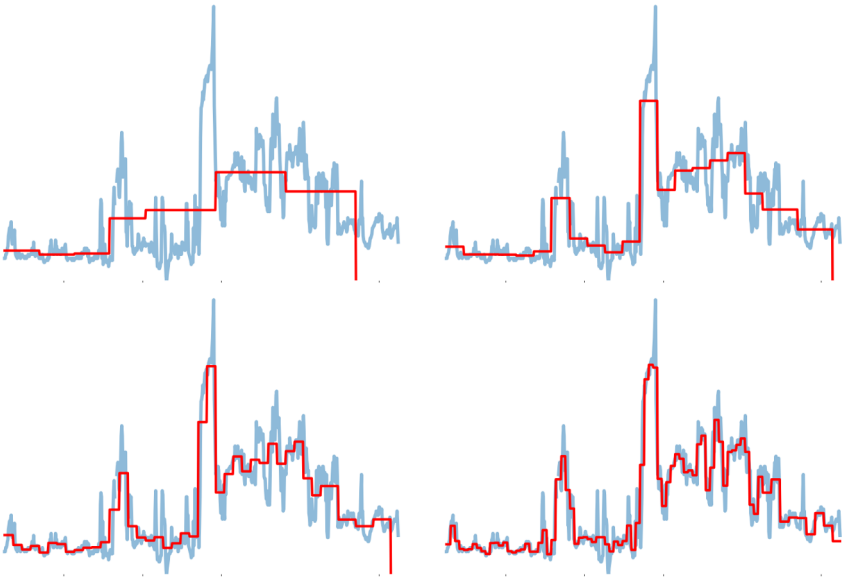


Fig. 29.7: Reconstructed heart rate data (in red) using the first k coefficients: $k = 10$ (top-left), $k = 25$ (top-right), $k = 50$ (bottom-left), and $k = 100$ (bottom-right), compared to the original data (in blue). The more coefficients, the closer the reconstruction is to the original.

Problems

29.1 In this chapter, we applied the Haar transform to a heart rate time series. Think of other time series where the Haar transform could be useful for noise removal and explain why.

29.2 (Orthogonality of Haar functions) Prove that the Haar functions ϕ_{jk} satisfy the following properties:

(a) For all j and $k = 0, 1, \ldots, 2^j - 1$,

$$\int_0^1 \phi_{jk}(x)dx = 0.$$

(b) For all $(j,k) \neq (i,t)$,

$$\int_0^1 \phi_{jk}(x)\phi_{it}(x)dx = 0.$$

29.3 By using induction on the order ℓ of Haar matrices, prove Theorem 29.8.

29.4 Let D_ℓ be the matrix defined in Theorem 29.8. Show that

$$D_\ell^{-1} = \operatorname{diag}\Big(2^{-(\ell+1)}, \underbrace{2^{-(\ell+1)}}_{2^0 \text{ times}}, \underbrace{2^{-\ell}, 2^{-\ell}}_{2^1 \text{ times}}, \underbrace{2^{-\ell-1}, \ldots, 2^{-\ell-1}}_{2^2 \text{ times}}, \ldots, \underbrace{2^{-1}, \ldots, 2^{-1}}_{2^\ell \text{ times}}\Big).$$

29.5 Compute the Haar transform of the vector $f = \begin{bmatrix}-2 & -8 & 3 & 7\end{bmatrix}^T$ and interpret the values of the resulting Haar transform coefficients.

29.6 (The fast Haar transform) In this problem, we will show how to compute the Haar transform $W_\ell^{-1}x$ efficiently, starting with small cases.

(a) Consider a vector $x = \begin{bmatrix}x_1 & x_2 & x_3 & x_4\end{bmatrix}^T \in \mathbb{R}^4$. Define the averages a_i and differences d_i as

$$a_1 = \frac{x_1 + x_2}{2}, \quad a_2 = \frac{x_3 + x_4}{2},$$
$$d_1 = \frac{x_1 - x_2}{2}, \quad d_2 = \frac{x_3 - x_4}{2}.$$

Form the vectors $a = \begin{bmatrix}a_1\\a_2\end{bmatrix}$ and $d = \begin{bmatrix}d_1\\d_2\end{bmatrix}$. Show that the Haar transform of x can be written as

$$W_1^{-1}x = \begin{bmatrix}W_0^{-1}a\\d\end{bmatrix}.$$

(b) Now, consider a vector $x = \begin{bmatrix}x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8\end{bmatrix}^T \in \mathbb{R}^8$. Define the averages a_i and differences d_i as

$$a_1 = \frac{x_1 + x_2}{2}, \quad a_2 = \frac{x_3 + x_4}{2}, \quad a_3 = \frac{x_5 + x_6}{2}, \quad a_4 = \frac{x_7 + x_8}{2},$$
$$d_1 = \frac{x_1 - x_2}{2}, \quad d_2 = \frac{x_3 - x_4}{2}, \quad d_3 = \frac{x_5 - x_6}{2}, \quad d_4 = \frac{x_7 - x_8}{2}.$$

Form the vectors

$$a = \begin{bmatrix}a_1 & a_2 & a_3 & a_4\end{bmatrix}^T, \quad d = \begin{bmatrix}d_1 & d_2 & d_3 & d_4\end{bmatrix}^T.$$

Show that the Haar transform of x can be written as

$$W_2^{-1}x = \begin{bmatrix}W_1^{-1}a\\d\end{bmatrix}.$$

(c) In general, given a vector $x \in \mathbb{R}^n$ with $n = 2^{\ell+1}$, define the averages a_i and differences d_i as

$$a_i = \frac{x_{2i-1} + x_{2i}}{2}, \quad d_i = \frac{x_{2i-1} - x_{2i}}{2}, \quad \text{for } i = 1, 2, \ldots, 2^\ell.$$

Form the vectors

$$a = \begin{bmatrix}a_1 & a_2 & \cdots & a_{2^\ell}\end{bmatrix}^T, \quad d = \begin{bmatrix}d_1 & d_2 & \cdots & d_{2^\ell}\end{bmatrix}^T.$$

Show that the Haar transform of x can be written as

$$W_\ell^{-1}x = \begin{bmatrix}W_{\ell-1}^{-1}a\\d\end{bmatrix}.$$

Thus, computing the Haar transform of x reduces to computing the Haar transform of a vector of half the size. Applying this recursively, we ultimately reduce the problem to computing the Haar transform of 2-component vectors.

29.7 Consider the vector

$$x = \begin{bmatrix} 8 & 6 & 7 & 5 & 3 & 0 & 9 & 4 \end{bmatrix}^T.$$

Using Problem 29.6, compute its Haar transform $W_2^{-1}x$ without explicitly forming the matrix W_2^{-1}. How many arithmetic operations are required to perform this computation?

29.8 Another important transform in time series analysis is the **Fourier transform**. The analogue of the Haar matrix is the **Discrete Fourier Transform (DFT) matrix**, given by

$$W_n = \frac{1}{\sqrt{n}} \begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \cdots & \omega^{2(n-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \cdots & \omega^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \omega^{3(n-1)} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix} \in \mathbb{C}^{n\times n},$$

where $\omega = e^{-2\pi i/n}$ is a complex nth root of unity.

(a) Write down the DFT matrices for $n = 2, 3$, and 4, expressing all entries in the form $a + ib$.

(b) Show that $W_n^* W_n = I_n$, that is, show that W_n is a unitary matrix.

Part VII
ORTHOGONAL PROJECTIONS AND THE LEAST SQUARES PROBLEM

This part begins with orthogonal projections and projection matrices. These are important in Machine Learning for tasks like regression, dimensionality reduction, and data compression. We introduce projection matrices, study their properties, and use them to understand the least squares problem and its variations. We will see how projections help solve least squares problems and explore several applications of least squares.

Chapter 30
VII-WHY SHOULD WE CARE?

30.1 Dimensionality reduction

As we saw in Chapter 1, dimensionality reduction is an important part of Unsupervised Learning.

In Machine Learning, we often face the "curse of dimensionality". As the number of features grows, data becomes sparse, models become harder to train, and algorithms slow down. Dimensionality reduction helps by keeping only the most important features. This simplifies the data, removes noise, and eliminates irrelevant details, making the problem easier to handle. We saw a good example of this in Chapter 29.

In this introductory chapter, we will see three ways dimensionality reduction can help: it removes redundant information, identifies the minimum number of features needed, and makes data easier to visualize.

30.1.1 Eliminating redundant information

Dimensionality reduction removes extra features that do not add useful information. These features can add noise, hide patterns, and make computations slower.

Example 30.1. (Projections and minimal reconstruction error)

A lab wants to estimate body fat percentage using simple body measurements. Two teams, one in the US and one in Canada, collected data from about 250 people. Both teams measured weight, one in pounds and the other in kilograms.

The left panel of Figure 30.1 shows pounds versus kilograms. This information is redundant because kilograms can be converted to pounds by multiplying by 2.205. As a result, all points lie on the red line $y = 2.205x$, spanned by $w = \begin{bmatrix} 1 \\ 2.205 \end{bmatrix}$. This two-dimensional dataset is actually one-dimensional since we only need one weight measurement per person.

In reality, data is not as "clean" as shown in the left panel of Figure 30.1. The right panel shows the actual weights recorded by the two teams, where the points do not line up perfectly on a straight line. These small errors could be due to rounding, measurement inaccuracies, or other factors.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_30

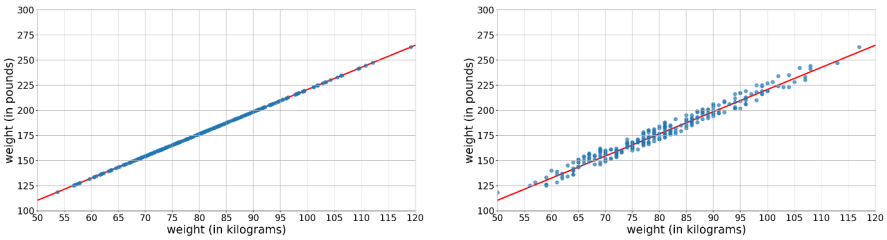


Fig. 30.1: Left panel shows ideal weights in pounds versus kilograms, perfectly aligned on a line. Right panel shows actual recorded weights.

The data in the right panel of Figure 30.1 still follows a mostly linear trend. Most points lie close to the line spanned by $w = \begin{bmatrix} 1 \\ 2.205 \end{bmatrix}$. So we can approximate each point x as $x \approx zw$ for some scalar z. This means we only need to store one number, z, instead of two values for each data point.

But how do we find the best scalar z for a given point x? We want the point zw on the line that is closest to x.

Figure 30.2 zooms in near the point $x = \begin{bmatrix} 82 \\ 188 \end{bmatrix}$ and compares two possible approximations of x on the line spanned by w. Which one is better?

Our intuition says the closest point is where the dotted red line from x to zw is perpendicular to the line. This point is called the **orthogonal projection** of x onto the subspace span(w).

In Chapter 32, we will show that this intuition is correct.

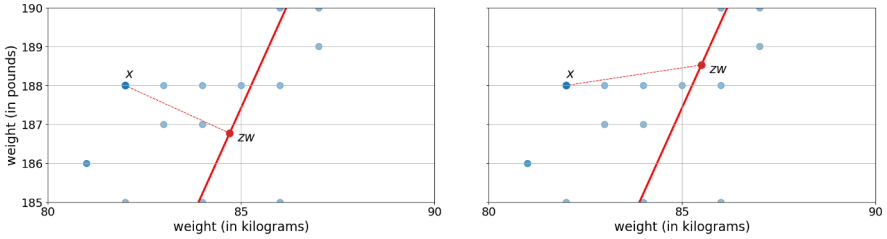


Fig. 30.2: Comparing two approximations of a data point x along the line spanned by w (shown in red). The best approximation happens when the red dotted line from x to zw is perpendicular to the line spanned by w.

30.1.2 The intrinsic dimension of data

The lab example in Example 30.1 might seem simple since we know we do not need both pounds and kilograms to record weight. But it highlights a key idea: data can look high-dimensional while actually being low-dimensional.

Dimensionality reduction helps by finding the "intrinsic dimension" of the data, that is, the number of features needed to preserve the parts that matter most for understanding or using the data effectively. This makes the data easier to work with, speeds up computations, and reduces the impact of random noise.

Example 30.2. (Three-dimensional data that is intrinsically two-dimensional)

The left panel of Figure 30.3 shows data points in 3D space. Notice that all the points lie close to a flat plane. This means the data is essentially two-dimensional, even though it is in 3D space. By projecting the points onto the plane, we reduce the data from three dimensions to two, while keeping the overall shape and structure of the data, as shown in the right panel.

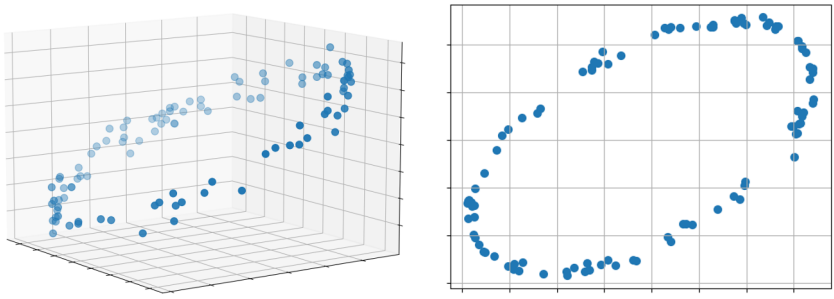


Fig. 30.3: Left panel shows three-dimensional data points close to a plane, revealing the dataset's true two-dimensional nature. The right panel depicts the result of projecting the points onto a plane, reducing dimensionality from three to two.

In Chapter 32, we will cover how to do the "projections" part. Finding the plane to project onto will be covered in Chapter 47.

30.1.3 Data visualization

Plots are a great way to visualize data. For example, a scatter plot can show the relationship between age and height. But if we have 1,000 features, plotting every pair would mean creating about half a million plots, which is far too many to be useful.

Dimensionality reduction helps by keeping just a few features, like two or three, that still capture most of the important structure in the data. This allows us to create simple plots that reveal clusters, patterns, and outliers, making the data easier to see and understand.

The dimensionality reduction methods based on orthogonal projections that we will cover in Chapters 44 and 49, Linear Discriminant Analysis and Singular Value Decomposition, can be used for data visualization. In recent years, however, newer and more powerful dimensionality reduction techniques have been developed that are not based on projections.

To illustrate the potential of data visualization through dimensionality reduction, the next example uses one of these newer methods: UMAP. Although UMAP is not based on orthogonal projections, it often produces striking visualization.

Example 30.3. (Visualizing text documents)

For this example, we will use a subset of the "20 Newsgroups" dataset [35]. It has 8,708 forum posts organized into nine topics: atheism, Christianity, PC hardware, Mac hardware, Windows 10, autos, motorcycles, baseball, and hockey.

We start by preprocessing the data. This includes removing stop words, lemmatizing words, and removing punctuation. Then, we calculate TF-IDF values for each document, turning them into numerical vectors (see Chapter 3). This gives 8,708 vectors, each with 10,699 entries (the vocabulary size). Finally, we use UMAP (Uniform Manifold Approximation and Projection) [39] to reduce the data from 10,699 dimensions to just two.

Reducing the data to two dimensions gives us a clear, big-picture view of the patterns in our dataset.

Figure 30.4 shows the dataset in two dimensions. Notice how posts on similar topics, like baseball and hockey or PC hardware, Mac hardware, and Windows 10, naturally group together. This is remarkable because the algorithm figured out these relationships from the data itself, without knowing the topics in advance.

This view is just the start. The two-dimensional plot helps us find insights and hidden patterns in the data. We can zoom in on specific clusters, examine individual documents, or investigate unusual points that stand out.

30.2 Beyond the regression line

The linear regression line we discussed in Chapter 19 is a powerful modeling tool, but it has limitations. It is great for predicting one variable based on another when we expect a linear relationship. But sometimes, we need to go beyond this simple model. The following examples show why more advanced methods are sometimes necessary.

30.2.1 Polynomial regression

Let us revisit the problem from Chapter 15 on predicting a house's sale price based on its total area.

We started by assuming a linear relationship between area and price, fitting a line

$$\text{price} = c_0 + c_1 \cdot \text{area}$$

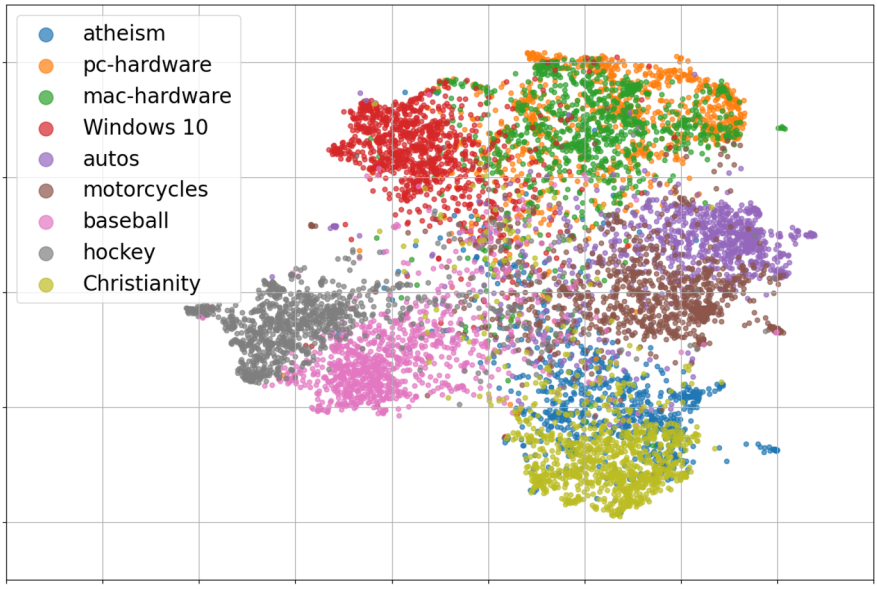


Fig. 30.4: Two-dimensional visualization of forum posts from the 20 newsgroups dataset.

to our data points. But what if the relationship is not that simple? Maybe the price increases faster than linearly with area. In that case, a more flexible model might work better. For example, we might try fitting a quadratic polynomial:

$$\text{price} = c_0 + c_1 \cdot \text{area} + c_2 \cdot \text{area}^2.$$

In Chapter 34, we will learn how to fit this quadratic model, and even higher-degree polynomials, using a method similar to the one used in Chapter 19 to fit lines.

Figure 30.5 shows the quadratic fit applied to our data.

30.2.2 Multilinear regression

In October 2012, the bike counter at Seattle's Fremont Bridge began recording the number of bikes crossing the pedestrian/bike paths. Imagine you have been hired by the city of Seattle to predict bicycle traffic to help plan bike lanes, manage peak times, and support safe, sustainable transportation.

Figure 30.6 shows the daily bicycle trip count across the Fremont Bridge in 2019.

You identify several factors that might affect daily bike traffic: weekends vs. weekdays (commuters may increase traffic), weather (fewer people bike on rainy or very cold days), and daylight hours (people might avoid biking at night), among others. You suspect that no single factor can fully explain the traffic patterns, so you decide to use multiple features in your model. This approach, called **multilinear**

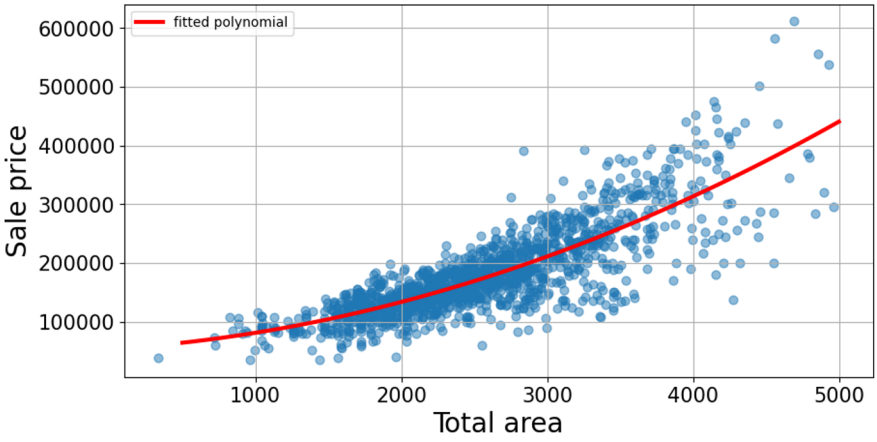


Fig. 30.5: Quadratic fit of house prices based on area.

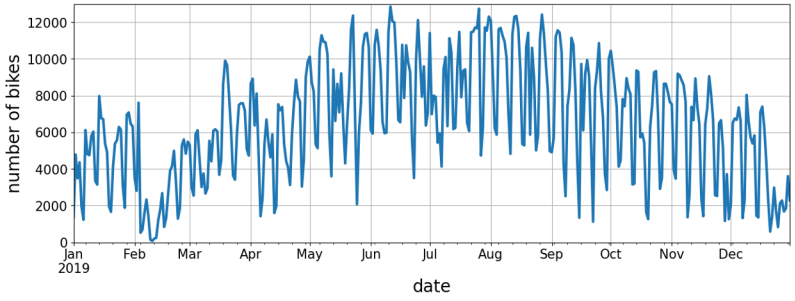


Fig. 30.6: Daily bicycle trips across Seattle's Fremont bridge (January 1 - December 31, 2019).

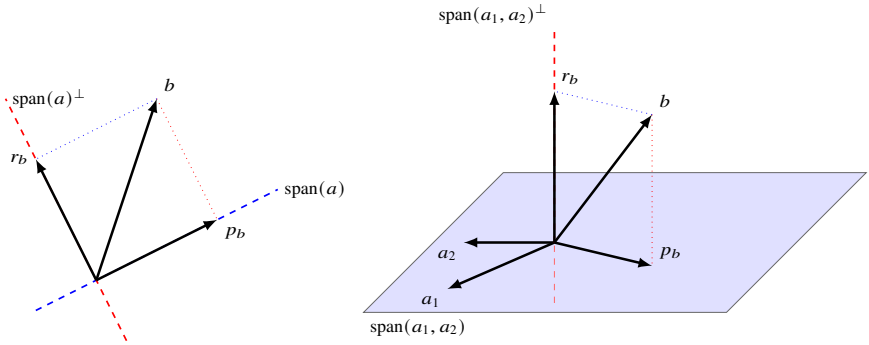
regression, will be covered in Chapter 34, where we will learn how to fit a multilinear regression model using least squares.

Chapter 31
VII-WHAT YOU MAY HAVE LEARNED BEFORE

31.1 Orthogonal projections

In this chapter, we review orthogonal projections onto a subspace $\mathcal{W}$, an important concept you may remember from your introductory Linear Algebra course.

Figure 31.1 illustrates the idea of an orthogonal projection. In the left panel, $\mathcal{W} = \text{span}(a)$ is a line in $\mathbb{R}^2$, and in the right panel, $\mathcal{W} = \text{span}(a_1, a_2)$ is a plane in $\mathbb{R}^3$. The vector b is any vector in the corresponding vector space, and p_b is the **orthogonal projection** of b onto $\mathcal{W}$. This happens when the vector $r_b = b - p_b$, called the **residual**, is orthogonal to $\mathcal{W}$.



(a) Orthogonal projection onto a line. (b) Orthogonal projection onto a plane.

Fig. 31.1: Orthogonal projection examples.

We can also understand orthogonal projection in a more abstract way, using the idea of orthogonal complementary subspaces. Before getting into the details, recall from Definition 27.11 and Theorem 27.10 that for any subspace $\mathcal{W}$ of $\mathbb{R}^m$, we have the decomposition

$$\mathbb{R}^m = \mathcal{W} \oplus \mathcal{W}^\perp.$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_31

This means every vector in $\mathbb{R}^m$ can be written uniquely as the sum of one vector from $\mathcal{W}$ and one from $\mathcal{W}^\perp$, the orthogonal complement of $\mathcal{W}$.

Definition 31.1. (Orthogonal projection onto a subspace)

Let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$, and let $b \in \mathbb{R}^m$ be any vector. We can uniquely decompose b as $b = p_b + r_b$, where $p_b \in \mathcal{W}$ and $r_b \in \mathcal{W}^\perp$. The **orthogonal projection onto** $\mathcal{W}$ is the linear transformation that maps every vector b to p_b. We denote the orthogonal projection of b onto $\mathcal{W}$ as $\text{proj}_{\mathcal{W}}(b) = p_b$.

Since $(\mathcal{W}^\perp)^\perp = \mathcal{W}$, the roles of $\mathcal{W}$ and $\mathcal{W}^\perp$ can be reversed. So if we write any vector b as $b = p_b + r_b$, where p_b lies in $\mathcal{W}$ and r_b lies in $\mathcal{W}^\perp$, then r_b is also the orthogonal projection of b onto $\mathcal{W}^\perp$.

Some properties of orthogonal projections come directly from the definition.

Proposition 31.2. (Basic properties of orthogonal projections)

For any subspace $\mathcal{W}$ of $\mathbb{R}^n$ and its orthogonal projection $f(x) = \text{proj}_{\mathcal{W}}(x)$, we have

(a) $f(x) = x$ if and only if $x \in \mathcal{W}$,
(b) $f(x) = 0$ if and only if $x \in \mathcal{W}^\perp$.
(c) $f \circ f = f$.

The result in Proposition 31.2 gives an intuitive picture of how orthogonal projections work. If x is already in $\mathcal{W}$, the projection leaves it unchanged. If x lies in $\mathcal{W}^\perp$, the projection sends it to zero because it has no component in $\mathcal{W}$. And if you project x more than once, nothing changes. The first projection moves it into $\mathcal{W}$, and once it is there, projecting again has no effect.

The orthogonal projection $\text{proj}_{\mathcal{W}}$ is a linear transformation (see Problem 31.1), so it has a matrix representation. In Chapter 32, we will study the matrix representation of orthogonal projections and their properties. We will also introduce a key result: the orthogonal projection of a vector onto a subspace is the closest point to that vector within the subspace. This makes orthogonal projections especially useful in many applications.

Problems

31.1 Prove that the orthogonal projection $\text{proj}_{\mathcal{W}}$ is a linear transformation.

31.2 Prove Proposition 31.2.

31.3 Show that the range of the orthogonal projection $\text{proj}_{\mathcal{W}}$ is $\mathcal{W}$.

31.4 Let $\mathcal{V}$ and $\mathcal{W}$ be subspaces of $\mathbb{R}^m$, and define

$$U = V \cap W, \quad V_0 = V \cap U^\perp, \quad W_0 = W \cap U^\perp.$$

Assume, in addition, that V_0 and W_0 are orthogonal subspaces. We use the notation $\oplus^\perp$ to denote orthogonal direct sum.

(a) Show that

$$V = U \oplus^\perp V_0 \quad \text{and} \quad W = U \oplus^\perp W_0.$$

(b) Use the assumption $V_0 \perp W_0$ to show that

$$V + W = U \oplus^\perp V_0 \oplus^\perp W_0.$$

(c) Show that

$$\text{proj}_V(x) = \text{proj}_U(x) + \text{proj}_{V_0}(x)$$

and

$$\text{proj}_W(x) = \text{proj}_U(x) + \text{proj}_{W_0}(x).$$

Then, prove that

$$\text{proj}_{\mathcal{V}}(x) + \text{proj}_{\mathcal{W}}(x) - \text{proj}_{\mathcal{V}\cap\mathcal{W}}(x) = \text{proj}_{\mathcal{V}+\mathcal{W}}(x).$$

(d) Show that the formula in part (c) may fail without the assumption $V_0 \perp W_0$.

31.5 (Affine spaces) Let $v \in \mathbb{R}^m$, and let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$. Define the set

$$\mathcal{A} = \{v + w \mid w \in \mathcal{W}\},$$

which is called an **affine space** in $\mathbb{R}^m$.

Affine spaces are not subspaces in general, since they do not contain the origin when $v \neq 0$. However, you can think of them as subspaces that have been shifted by a fixed vector[1]. In this case, $\mathcal{A}$ is the result of translating the subspace $\mathcal{W}$ by the vector v. Many ideas, such as projection onto $\mathcal{W}$, extend naturally to affine spaces.

How would you define the orthogonal projection of a vector onto an affine space? Describe how to compute the orthogonal projection of a vector b onto the affine space $\mathcal{A}$.

[1] We have seen an affine space earlier in the book. Can you remember where?

Chapter 32
VII-CORE TOPICS

32.1 Orthogonal projection matrices and their properties

We know that every linear transformation $f : \mathbb{R}^m \mapsto \mathbb{R}^n$ has a matrix representation. Since an orthogonal projection $f(x) = \text{proj}_{\mathcal{W}}(x)$ is a linear transformation[1], it also has a matrix representation. For any vector $x \in \mathbb{R}^m$, we can express the projection of x onto $\mathcal{W}$ as $\text{proj}_{\mathcal{W}}(x) = Px$, where P is the matrix representing the projection. The following theorem gives two formulas for the matrix P.

Theorem 32.1. (Orthogonal projection matrix formulas)

Suppose $\mathcal{W}$ is a subspace of $\mathbb{R}^m$ with dimension r.

If $M \in \mathbb{R}^{m\times r}$ is a matrix whose columns form a basis for $\mathcal{W}$, then the matrix

$$P = M(M^T M)^{-1} M^T \tag{32.1}$$

is the unique matrix that satisfies $Pb = \text{proj}_{\mathcal{W}}(b)$ for every $b \in \mathbb{R}^m$. We call this matrix the **orthogonal projection matrix** onto $\mathcal{W}$.

If $Q \in \mathbb{R}^{m\times r}$ is a matrix whose columns form an orthonormal basis for $\mathcal{W}$, then the projection matrix simplifies to

$$P = QQ^T. \tag{32.2}$$

Proof To derive (32.1), we express $b \in \mathbb{R}^m$ as the sum of its orthogonal projections onto $\mathcal{W}$ and $\mathcal{W}^\perp$. We write $b = p_b + r_b$, where p_b is the projection onto $\mathcal{W}$ and r_b is the projection onto $\mathcal{W}^\perp$.

Since p_b is in $\mathcal{W}$, we can represent it as $p_b = Mx$ for some $x \in \mathbb{R}^r$. The vector $r_b = b - p_b$ is orthogonal to $\mathcal{W}$, so we have $M^T r_b = 0$.

Combining these, we get

$$0 = M^T r_b = M^T(b - p_b) = M^T b - M^T Mx.$$

[1] You can review its definition in Chapter 31.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_32

Since M has full column rank, M^TM is invertible (see Theorem 17.46). Solving for x, we find $x = (M^TM)^{-1}M^Tb$.

Now, substituting back, we have

$$p_b = Mx = M(M^TM)^{-1}M^Tb.$$

This holds for all $b \in \mathbb{R}^m$. Hence, $P = M(M^TM)^{-1}M^T$.

If Q is a matrix whose columns form an orthonormal basis for $\mathcal{W}$, then $Q^TQ = I_r$. This simplifies the orthogonal projection matrix to $P = QQ^T$.

Finally, we need to show that P is unique, that is, P does not depend on the choice of basis for $\mathcal{W}$. Let M_1 and M_2 be two matrices whose columns form different bases for $\mathcal{W}$. Define

$$P_1 := M_1(M_1^TM_1)^{-1}M_1^T \quad \text{and} \quad P_2 := M_2(M_2^TM_2)^{-1}M_2^T.$$

Let N be any matrix whose columns form a basis for $\mathcal{W}^\perp$. The columns of $S = \begin{bmatrix} M_1 & N \end{bmatrix}$ form a basis for $\mathbb{R}^m$. Since $P_2S = \begin{bmatrix} M_1 & 0 \end{bmatrix}$ and $P_1S = \begin{bmatrix} M_1 & 0 \end{bmatrix}$, we have $P_2S = P_1S$. If two linear transformations agree on the vectors of a basis, they must be the same. Therefore, $P_1 = P_2$. □

Example 32.2 Consider the following subspace of $\mathbb{R}^3$ (geometrically speaking, a plane)

$$\mathcal{W} = \operatorname{span}\left(\begin{bmatrix}1\\1\\0\end{bmatrix}, \begin{bmatrix}0\\1\\1\end{bmatrix}\right).$$

Our goal is to find the matrix representation of the orthogonal projection onto $\mathcal{W}$ and use it to project the following vectors

$$v_1 = \begin{bmatrix}0\\1\\-2\end{bmatrix}, \quad v_2 = \begin{bmatrix}1\\2\\1\end{bmatrix} \quad \text{and} \quad v_3 = \begin{bmatrix}1\\-1\\1\end{bmatrix}.$$

To find the orthogonal projection matrix P, we start with the basis for $\mathcal{W}$ and construct the matrix

$$M = \begin{bmatrix}1 & 0\\1 & 1\\0 & 1\end{bmatrix},$$

where the columns are the basis vectors. Now we can calculate the orthogonal projection matrix onto $\mathcal{W}$ using (32.1). This gives us

$$P = M(M^TM)^{-1}M^T = \begin{bmatrix}2/3 & 1/3 & -1/3\\1/3 & 2/3 & 1/3\\-1/3 & 1/3 & 2/3\end{bmatrix}.$$

Next, we use P to find the orthogonal projections of v_1, v_2, and v_3 onto $\mathcal{W}$. We get

$$Pv_1 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}, \quad Pv_2 = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = v_2, \quad \text{and} \quad Pv_3 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Figure 32.1 shows these projections.

We could have predicted the results for v_2 and v_3. Since v_2 is in $\mathcal{W}$ (it is the sum of the basis vectors), its projection onto $\mathcal{W}$ is just v_2 itself. On the other hand, v_3 is orthogonal to $\mathcal{W}$ because it is orthogonal to both columns of M, so its projection onto $\mathcal{W}$ is the zero vector.

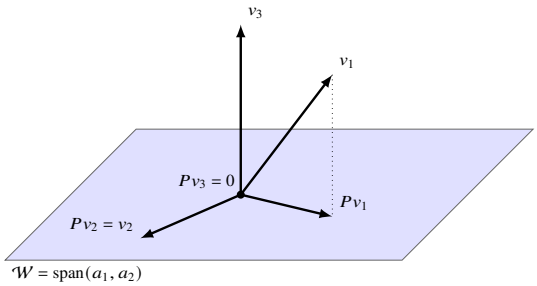


Fig. 32.1: The orthogonal projection of three vectors onto a plane $\mathcal{W} = \text{span}(a_1, a_2)$. The vector v_2 is already in $\mathcal{W}$, so $Pv_2 = v_2$. The vector v_3 is orthogonal to $\mathcal{W}$, so $Pv_3 = 0$.

Computing the projection matrix is easier when $\mathcal{W}$ has an orthonormal basis, because we do not need to compute a matrix inverse.

Example 32.3 Let us revisit Example 32.2, where we found the orthogonal projection matrix onto the subspace $\mathcal{W}$ of $\mathbb{R}^3$ spanned by the vectors

$$a_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \quad \text{and} \quad a_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}.$$

This time, we will use an orthonormal basis for $\mathcal{W}$. By applying the Gram-Schmidt process to a_1 and a_2, we find the following orthonormal basis

$$\mathcal{W} = \text{span}\left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \frac{1}{\sqrt{6}} \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} \right).$$

Let Q be the matrix with these vectors as columns:

$$Q = \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{6} \\ 1/\sqrt{2} & 1/\sqrt{6} \\ 0 & 2/\sqrt{6} \end{bmatrix}.$$

By Theorem 32.1, the orthogonal projection matrix onto $\mathcal{W}$ is QQ^T. Therefore, we have

$$QQ^T = \begin{bmatrix} 2/3 & 1/3 & -1/3 \\ 1/3 & 2/3 & 1/3 \\ -1/3 & 1/3 & 2/3 \end{bmatrix},$$

which matches the matrix we obtained in Example 32.2.

When projecting onto a one-dimensional subspace, that is, onto a line, the projection matrix has a simple form.

Projections onto a line

Let a be a nonzero vector. The orthogonal projection matrix onto the subspace $\mathcal{W} = \text{span}(a)$ is the rank-1 matrix

$$P = \left(\frac{1}{a^T a}\right) aa^T. \tag{32.3}$$

If a is a unit vector (so $a^T a = \|a\|_2^2 = 1$), then the projection matrix (32.3) simplifies to

$$P = aa^T.$$

In general, the projection onto a subspace $\mathcal{W} = \text{span}(a_1, \ldots, a_r)$ is not simply the sum of the projections onto the lines spanned by each basis vector a_i. However, when the basis is orthonormal, equation (32.2) shows that this property holds. We now explain why this is the case.

Decomposing a projection matrix into basis vector projections

Suppose $\mathcal{W}$ is a subspace of $\mathbb{R}^m$ with dimension r, and let Q be an $m \times r$ matrix whose columns $q_1, \ldots, q_r$ form an orthonormal basis for $\mathcal{W}$. According to Theorem 32.1, the orthogonal projection matrix onto $\mathcal{W}$ is $P = QQ^T$.

Using the columns-times-rows formula for matrix multiplication (Proposition 7.9), we can write P as a sum of rank-1 matrices:

$$QQ^T = \begin{bmatrix} q_1 & \cdots & q_r \end{bmatrix} \begin{bmatrix} q_1^T \\ \vdots \\ q_r^T \end{bmatrix} = q_1 q_1^T + \cdots + q_r q_r^T.$$

Each matrix $q_i q_i^T$ is a projection matrix that projects onto the line spanned by the unit vector q_i (see 32.3). Thus, $P = QQ^T$ is the sum of the r orthogonal projections onto these basis vectors. Figure 32.2 illustrates this.

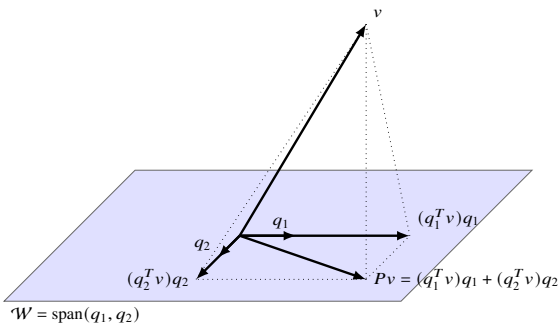


Fig. 32.2: The projection of the vector v onto the two-dimensional subspace $\mathcal{W}$ is the sum of the projections of v onto the orthonormal basis vectors q_1 and q_2.

Furthermore, for any vector $v \in \mathbb{R}^m$, the orthogonal projection onto $\mathcal{W}$ is

$$QQ^T v = q_1 q_1^T v + \cdots + q_r q_r^T v = (q_1^T v)q_1 + \cdots + (q_r^T v)q_r.$$

This shows that the vector

$$Q^T v = \begin{bmatrix} q_1^T \\ \vdots \\ q_r^T \end{bmatrix} v = \begin{bmatrix} q_1^T v \\ \vdots \\ q_r^T v \end{bmatrix}$$

contains the coordinates of the projection of v onto $\mathcal{W}$ in the basis $\{q_1, \ldots, q_r\}$.

This decomposition property holds for orthonormal bases, and also for orthogonal bases. If the basis is not orthonormal or orthogonal, the projection matrix becomes $P = M(M^T M)^{-1} M^T$, and the projections of the basis vectors do not combine as neatly.

Once we have the orthogonal projection matrix for a subspace, we can easily find the projection matrix for its orthogonal complement because these matrices are closely related.

Theorem 32.4. (The projection matrix onto the orthogonal complement)

Let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$. If P is the orthogonal projection matrix onto $\mathcal{W}$, then $I_m - P$ is the orthogonal projection matrix onto $\mathcal{W}^\perp$.

Proof Let $x \in \mathbb{R}^m$ be any vector. Then, $x = p_x + r_x$, where $p_x \in \mathcal{W}$ is the orthogonal projection of x onto $\mathcal{W}$ and $r_x \in \mathcal{W}^\perp$ is the orthogonal projection onto $\mathcal{W}^\perp$. Thus,

$$r_x = x - Px = (I_m - P)x.$$

Example 32.5 Let us revisit Example 32.2 to find the orthogonal projection matrix onto $\mathcal{W}^\perp$ using two different methods.

First, we can apply Theorem 32.4. Starting from P, the orthogonal projection matrix onto $\mathcal{W}$ that we found earlier, we calculate the projection onto $\mathcal{W}^\perp$:

$$I_3 - P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 2/3 & 1/3 & -1/3 \\ 1/3 & 2/3 & 1/3 \\ -1/3 & 1/3 & 2/3 \end{bmatrix} = \begin{bmatrix} 1/3 & -1/3 & 1/3 \\ -1/3 & 1/3 & -1/3 \\ 1/3 & -1/3 & 1/3 \end{bmatrix}.$$

Alternatively, we can find an orthonormal basis for $\mathcal{W}^\perp$ and then use Theorem 32.1. By Theorem 27.7, we have

$$\operatorname{span}\left(\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}\right)^\perp = \operatorname{col}\left(\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}\right)^\perp = \operatorname{null}\left(\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}\right).$$

This means that if $x \in \mathcal{W}^\perp$, it must satisfy

$$\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} x_1 + x_2 \\ x_2 + x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

From this, we find $x_1 = -x_2$ and $x_3 = -x_2$. This allows us to express x as

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = x_2 \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}.$$

This tells us that

$$\mathcal{W}^\perp = \operatorname{span}\left(\begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}\right) = \operatorname{span}\left(\begin{bmatrix} -1/\sqrt{3} \\ 1/\sqrt{3} \\ -1/\sqrt{3} \end{bmatrix}\right).$$

The vector $u = \begin{bmatrix} -\frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{3}} \end{bmatrix}$ forms an orthonormal basis for $\mathcal{W}^\perp$. Thus, the orthogonal projection matrix onto $\mathcal{W}^\perp$ is

$$P = \begin{bmatrix} -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{3}} \end{bmatrix} \begin{bmatrix} -\frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{3}} \end{bmatrix} = \begin{bmatrix} 1/3 & -1/3 & 1/3 \\ -1/3 & 1/3 & -1/3 \\ 1/3 & -1/3 & 1/3 \end{bmatrix},$$

which matches the matrix we found using Theorem 32.4.

Theorem 32.4 provides another perspective on the Gram-Schmidt process (introduced in Chapter 26) by connecting it to orthogonal projections. We can think of the Gram-Schmidt process as a series of projections onto the orthogonal complements of the spans of the previous basis vectors.

Gram-Schmidt steps as orthogonal projections

At the i-th step, with $i \geqslant 2$, the Gram-Schmidt process computes the vector (see (26.1))

$$v_i = (I_m - Q_i Q_i^T) w_i,$$

where the columns of $Q_i = \begin{bmatrix} q_1 & q_2 & \cdots & q_{i-1} \end{bmatrix}$ are the orthonormal vectors obtained from the previous $i-1$ steps. From Theorem 32.4, we see that the Gram-Schmidt process finds v_i by projecting w_i onto the orthogonal complement of $\text{span}(q_1, q_2, \ldots, q_{i-1})$.

Theorem 32.6 provides several important properties of orthogonal projection matrices.

Theorem 32.6. (Properties of orthogonal projection matrices)

Let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$, and let $P \in \mathbb{R}^{m \times m}$ be the orthogonal projection matrix onto $\mathcal{W}$. The following properties hold:

(a) P is symmetric, that is, $P = P^T$.
(b) P is idempotent, that is, $P^2 = P$.
(c) $\text{col}(P) = \mathcal{W}$, and thus $\text{rank}(P) = \dim(\mathcal{W})$.
(d) $\text{null}(P) = \mathcal{W}^\perp$.

Proof Let the columns of $Q \in \mathbb{R}^{m \times r}$, where $r = \dim(\mathcal{W})$, form an orthonormal basis for $\mathcal{W}$. By Theorem 32.1, we have $P = QQ^T$.

To show that P is symmetric, we calculate

$$P^T = (QQ^T)^T = QQ^T = P.$$

So, P is symmetric. Next, we check idempotency:

$$P^2 = QQ^T QQ^T = QQ^T,$$

where we used $Q^T Q = I_r$ because the columns of Q are orthonormal. Thus, $P^2 = P$, confirming that P is idempotent.

Now, let us prove part (c). If $v \in \text{col}(P)$, then there exists $x \in \mathbb{R}^m$ such that $v = Px \in \mathcal{W}$ since P is the orthogonal projection matrix onto $\mathcal{W}$. This shows $\text{col}(P) \subseteq \mathcal{W}$.

Conversely, let $v \in \mathcal{W}$. Then, $v = Pv \in \text{col}(P)$. This means $v \in \text{col}(P)$, which shows $\mathcal{W} \subseteq \text{col}(P)$. Combining these results, we get $\text{col}(P) = \mathcal{W}$, and so $\text{rank}(P) = \dim(\text{col}(P)) = \dim(\mathcal{W})$.

Finally, by the Fundamental Theorem of Linear Algebra (Theorem 27.12), we know $\text{null}(P) = \text{col}(P^T)^\perp = \text{col}(P)^\perp$ since P is symmetric. From part (c), we have $\text{col}(P) = \mathcal{W}$, thus

$$\text{null}(P) = \mathcal{W}^\perp.$$

This completes the proof.

Theorem 32.1 tells us that every orthogonal projection matrix is both symmetric and idempotent. In fact, the reverse is also true: any matrix that is symmetric and idempotent must be an orthogonal projection matrix, as we now show.

Theorem 32.7. (Characterization of orthogonal projection matrices)

A matrix $P \in \mathbb{R}^{m\times m}$ is an orthogonal projection matrix if and only if it is symmetric ($P^T = P$) and idempotent ($P^2 = P$).

Proof Theorem 32.6 tells us that an orthogonal projection matrix is symmetric and idempotent, so these conditions are necessary.

To prove sufficiency, we will show that if P is symmetric and idempotent, then P is the orthogonal projection matrix onto its column space, $\text{col}(P)$. To do this, we will show that we can express any vector $x \in \mathbb{R}^m$ as $x = Px + r$, where $Px \in \text{col}(P)$ and $r \in \text{col}(P)^\perp$.

We can write any vector $x \in \mathbb{R}^m$ as

$$x = Px + (I_m - P)x.$$

The vector Px is in $\text{col}(P)$. We need to show that $(I_m - P)x \in \text{col}(P)^\perp = \text{null}(P^T)$. Since P is symmetric and idempotent, we have

$$\begin{aligned} P^T(I_m - P)x &= P(I_m - P)x \\ &= (P - P^2)x \\ &= (P - P)x = 0, \end{aligned}$$

which implies that $(I_m - P)x$ is in $\text{col}(P)^\perp$, as we wanted to prove. □

32.2 The best approximation theorem

Many dimensionality reduction methods begin by finding a subspace that captures the main patterns or structure in the data. Once we have this "good" subspace $\mathcal{W}$, the next step is to approximate a data point x using this subspace. Orthogonal projection provides a natural way to do this.

An important result we will prove below is that the orthogonal projection p of a vector b onto a subspace $\mathcal{W}$ is the closest point to b within that subspace. In other words, among all vectors in $\mathcal{W}$, p minimizes the Euclidean distance to b. This makes orthogonal projection the best approximation of b using vectors from $\mathcal{W}$, in terms of distance.

Figure 32.3 illustrates this best-approximation property for a subspace in $\mathbb{R}^3$.

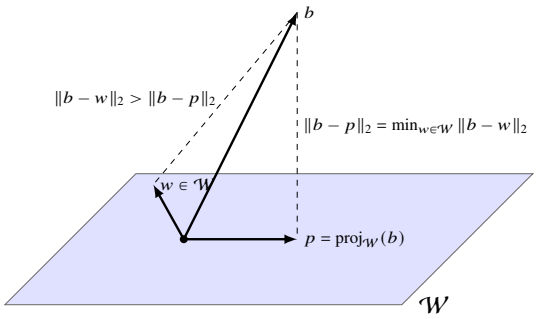


Fig. 32.3: The projection p of the vector b onto $\mathcal{W}$ is the closest vector in $\mathcal{W}$ to b. The distance from any other vector $w \in \mathcal{W}$ to b is always larger than the distance from b to p.

Theorem 32.8. (Vector in a subspace closest to a point)

Let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$, and let $b \in \mathbb{R}^m$. Then, the orthogonal projection $\text{proj}_{\mathcal{W}}(b)$ of b onto $\mathcal{W}$ is the vector in $\mathcal{W}$ closest to b in terms of the Euclidean distance. In other words, we have

$$\|b - \text{proj}_{\mathcal{W}}(b)\|_2 \leqslant \|b - w\|_2, \quad \text{for all } w \in \mathcal{W}.$$

Moreover, the equality holds only when $w = \text{proj}_{\mathcal{W}}(b)$.

Proof Let $p = \text{proj}_{\mathcal{W}}(b)$ and $w \in \mathcal{W}$. We can express $b - w$ as

$$b - w = b - p + p - w.$$

Note that $p - w \in \mathcal{W}$ since both p and w lie in $\mathcal{W}$. Also, $b - p \in \mathcal{W}^{\perp}$ because p is the orthogonal projection of b onto $\mathcal{W}$. Since the two vectors are orthogonal, the Pythagorean theorem gives

$$\|b - w\|_2^2 = \|b - p + p - w\|_2^2 = \|b - p\|_2^2 + \|p - w\|_2^2.$$

This shows

$$\|b - w\|_2^2 \geqslant \|b - p\|_2^2,$$

so the distance from b to any $w \in \mathcal{W}$ is always at least as large as the distance from b to p.

For the inequality to hold as an equality, we must have $\|p - w\|_2 = 0$, which implies $w = p = \text{proj}_{\mathcal{W}}(b)$. In other words, p is the unique point in $\mathcal{W}$ that is closest to b. □

In this and the following chapters, we will see three important applications of Theorem 32.8: least squares, Linear Discriminant Analysis (Chapter 44), and Principal Component Analysis (Chapter 49). The main difference is that in least squares,

the subspace is given in advance. In Linear Discriminant Analysis and Principal Component Analysis, we must first identify a suitable subspace before projecting.

32.3 Least squares problems

Consider a linear system $Ax = b$, where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. If b is not in the column space of A, the system has no solution (see Theorem 17.4). This often happens when A has more rows than columns.

When $Ax = b$ has no solution, we instead look for a vector Ax that is as close as possible to b. We saw an example of this in Chapter 19, where we tried to fit a line to several data points. While no single line passes through all the points, we can still find the line that fits them best.

Definition 32.9. (The least squares problem)

Given $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$, the **least squares problem** consists in finding the vectors $x \in \mathbb{R}^n$ that minimize $\|b - Ax\|_2^2$. We can express this problem as

$$\min_{x \in \mathbb{R}^n} \|b - Ax\|_2^2. \tag{32.4}$$

Any solution that minimizes $\|b - Ax\|_2^2$ is called a **least squares solution**.

You might wonder why we minimize the squared norm $\|b - Ax\|_2^2$ instead of the norm $\|b - Ax\|_2$. The reason is that both lead to the same solution. Squaring does not change the location of the minimum since the square function is increasing for nonnegative values. Minimizing the square simplifies the math by removing the square root from the norm, which makes the calculations easier to work with.

Let us first connect least squares problems with orthogonal projections. Figure 32.4 shows the main idea: the vectors of the form Ax fill the column space of A, so solving the least squares problem (32.4) means finding the vector in $\text{col}(A)$ that is closest to b. And we already know which one that is. By Theorem 32.8, it is the orthogonal projection of b onto $\text{col}(A)$.

The next proposition makes this precise.

Proposition 32.10. (Least squares and projections)

Given $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$, a vector $x \in \mathbb{R}^n$ minimizes $\|b - Ax\|_2^2$ if and only if $Ax = \text{proj}_{\text{col}(A)}(b)$.

We can now characterize all the solutions to the least squares problem (32.4) and determine when the solution is unique.

Theorem 32.11. (Least squares solutions)

Consider the least squares problem $\min_x \|b - Ax\|_2^2$, where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Then, the following statements hold:

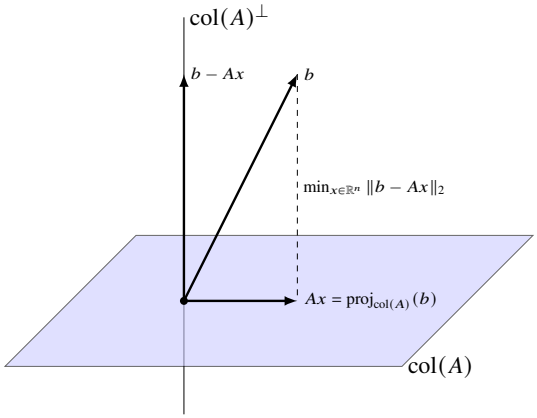


Fig. 32.4: The geometry of the least squares problem $\min_x \|b - Ax\|_2^2$. The vector Ax is the vector in $\mathrm{col}(A)$ closest to b, that is, Ax is the orthogonal projection of b onto $\mathrm{col}(A)$.

(a) The set of all least squares solutions corresponds to the solutions of the linear system

$$A^T Ax = A^T b. \tag{32.5}$$

This linear system, known as the **normal equations**, is always solvable.

(b) The least squares solution is unique if and only if A has full column rank. In this case, the least squares solution is given by the formula

$$x = (A^T A)^{-1} A^T b. \tag{32.6}$$

Proof First, observe that $A^T b \in \mathrm{col}(A^T) = \mathrm{col}(A^T A)$, so the normal equations (32.5) always have a solution.

Let us now show that a vector x solves the least squares problem (32.4) if and only if it satisfies the normal equations (32.5).

By Proposition 32.10, a vector x minimizes $\|b - Ax\|_2^2$ exactly when $Ax = \mathrm{proj}_{\mathrm{col}(A)}(b)$. We can then establish the result through the following chain of equivalent statements

$$\begin{aligned} Ax = \mathrm{proj}_{\mathrm{col}(A)}(b) \quad & \text{if and only if} \quad b - Ax \in \mathrm{col}(A)^\perp \\ & \text{if and only if} \quad b - Ax \in \mathrm{null}(A^T) \\ & \text{if and only if} \quad A^T(b - Ax) = 0 \\ & \text{if and only if} \quad A^T Ax = A^T b. \end{aligned}$$

The first equivalence follows from the definition of the orthogonal projection onto $\mathrm{col}(A)$, the residual lies in the orthogonal complement. The second uses the

Fundamental Theorem of Linear Algebra (Theorem 27.12), which states that the orthogonal complement of $\text{col}(A)$ is $\text{null}(A^T)$. The third comes from the definition of a matrix's nullspace, and the fourth is a straightforward rearrangement of terms.

Finally, the normal equations $A^TAx = A^Tb$ have a unique solution if and only if A^TA is invertible. This happens precisely when A has full column rank, by Theorem 17.45. When A^TA is invertible, the unique solution to the normal equations is $x = (A^TA)^{-1}A^Tb$. □

Theorem 32.11 shows how to solve least squares problems: just solve the normal equations. Chapter done? Not quite. While the normal equations do give the solution, they are not always the best approach. Using A^TA can amplify rounding errors and make calculations less accurate (see Problems 32.18 and 32.19).

In the next sections, we will learn a better method using the QR factorization, which avoids computing A^TA.

32.4 Solving least squares problems with the QR factorization

A common theme in Linear Algebra is transforming a problem into a simpler one without changing its solution. You have seen this idea before: when solving a linear system, we apply invertible transformations (the elementary row operations from Chapter 21) to turn a complicated system into an easier one, such as an upper triangular system.

We can apply the same idea to least squares problems. But what kind of transformation should we use? Since we are minimizing a 2-norm, and 2-norms stay the same when multiplied by orthogonal matrices (see Theorem 27.14), orthogonal matrices are a natural choice. Why not use them to transform A into an upper triangular form? That is exactly what the QR factorization does!

In the next two sections, we will see how to solve least squares problems using the QR factorization and the QR factorization with column pivoting.

32.4.1 The full column rank case

The most common case in applications is when $A \in \mathbb{R}^{m\times n}$ is tall and skinny ($m \geqslant n$) and has full column rank, that is, its columns are linearly independent. In this case, the least squares problem $\min_x \|b - Ax\|_2^2$ always has a unique solution.

Recall from Chapter 27 that if $A \in \mathbb{R}^{m\times n}$ has full column rank, its QR factorization is

$$A = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1R_1, \tag{32.7}$$

where $Q \in \mathbb{R}^{m\times m}$ is orthogonal and $R_1 \in \mathbb{R}^{n\times n}$ is an invertible upper triangular matrix. The first form is the full QR factorization, and the second is the reduced QR factorization.

Theorem 32.12. (Full column rank least squares solution)

Let $A \in \mathbb{R}^{m\times n}$ be a full column rank matrix, and let $b \in \mathbb{R}^m$. Given the QR factorization of A as in (32.7), the unique vector $x \in \mathbb{R}^n$ that solves the least squares problem $\min_x \|b - Ax\|_2^2$ is the unique solution to the upper triangular system

$$R_1 x = Q_1^T b. \tag{32.8}$$

Proof The main idea for the proof is that multiplying a vector by an orthogonal matrix does not change its 2-norm. Thus, we have

$$\begin{aligned}
\min_{x\in\mathbb{R}^n} \|b - Ax\|_2^2 &= \min_{x\in\mathbb{R}^n} \|b - QRx\|_2^2 = \\
&\min_{x\in\mathbb{R}^n} \|Q(Q^T b - Rx)\|_2^2 = \min_{x\in\mathbb{R}^n} \|Q^T b - Rx\|_2^2 = \\
&\min_{x\in\mathbb{R}^n} \left\| \begin{bmatrix} Q_1^T \\ Q_2^T \end{bmatrix} b - \begin{bmatrix} R_1 \\ 0 \end{bmatrix} x \right\|_2^2 = \min_{x\in\mathbb{R}^n} \left\| \begin{bmatrix} Q_1^T b - R_1 x \\ Q_2^T b \end{bmatrix} \right\|_2^2 = \\
&\min_{x\in\mathbb{R}^n} \left(\|Q_1^T b - R_1 x\|_2^2 + \|Q_2^T b\|_2^2 \right).
\end{aligned}$$

The term $\|Q_2^T b\|_2^2$ does not depend on x, so minimizing $\|b - Ax\|_2^2$ is the same as minimizing $\|Q_1^T b - R_1 x\|_2^2$. Since norms are always nonnegative, the smallest possible value of $\|Q_1^T b - R_1 x\|_2^2$ is zero. And we can reach this minimum: because R_1 is invertible, we can solve $R_1 x = Q_1^T b$ for a unique x, which makes the norm zero. □

Theorem 32.12 shows that solving a full column rank least squares problem only requires a reduced QR factorization of A. Once we have it, the solution follows two simple steps: compute $Q_1^T b$, then solve the upper triangular system $R_1 x = Q_1^T b$ using back substitution.

32.4.2 The general case

In this section, we show how to solve a least squares problem for a general matrix $A \in \mathbb{R}^{m\times n}$, no matter its size or rank.

When A does not have full column rank, the least squares problem has infinitely many solutions, and we will characterize all of them. To do this, we will use the QR factorization with column pivoting, given by

$$AP = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{array}{c} \begin{matrix} r & n-r \end{matrix} \\ \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} \begin{matrix} r \\ m-r \end{matrix} \end{array}, \tag{32.9}$$

where $P \in \mathbb{R}^{n\times n}$ is a permutation matrix, $Q \in \mathbb{R}^{m\times m}$ is orthogonal, and $R_{11} \in \mathbb{R}^{r\times r}$ is a nonsingular upper triangular matrix.

Theorem 32.13. (The column-rank deficient least squares solution)

Let $A \in \mathbb{R}^{m\times n}$ be a rank-r matrix, and let $b \in \mathbb{R}^m$. Given the QR factorization with column pivoting of A as in (32.9), the solutions of the least squares problem $\min_x \|b - Ax\|_2^2$ are of the form

$$x = P \begin{bmatrix} y_1 \\ y_2 \end{bmatrix},$$

where $y_2 \in \mathbb{R}^{n-r}$ is arbitrary, and $y_1 \in \mathbb{R}^r$ is the unique solution of the upper triangular linear system

$$R_{11} y_1 = Q_1^T b - R_{12} y_2.$$

Proof We begin with the same idea used to prove Theorem 32.12: the 2-norm does not change when multiplied by an orthogonal matrix. Therefore, we have

$$\min_{x \in \mathbb{R}^n} \|b - Ax\|_2^2 = \min_{x \in \mathbb{R}^n} \|Q^T(b - Ax)\|_2^2 = \min_{x \in \mathbb{R}^n} \|Q^T b - Q^T A P P^T x\|_2^2 = \min_{x \in \mathbb{R}^n} \|Q^T b - R P^T x\|_2^2.$$

Now, let us partition the vector $y := P^T x = \left[\begin{smallmatrix} y_1 \\ y_2 \end{smallmatrix} \right]$ conformably with the partitioning of the factor R. This gives us

$$\min_{x \in \mathbb{R}^n} \|b - Ax\|_2^2 = \min_{x \in \mathbb{R}^n} \|Q^T b - R P^T x\|_2^2 = \min_{x \in \mathbb{R}^n} \left\| \begin{bmatrix} Q_1^T \\ Q_2^T \end{bmatrix} b - \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \right\|_2^2 =$$

$$\min_{x \in \mathbb{R}^n} \left\| \begin{bmatrix} Q_1^T b - R_1 y_1 - R_2 y_2 \\ Q_2^T b \end{bmatrix} \right\|_2^2 =$$

$$\min_{x \in \mathbb{R}^n} \left(\|Q_1^T b - R_{11} y_1 - R_{12} y_2\|_2^2 + \|Q_2^T b\|_2^2 \right).$$

The term $\|Q_2^T b\|_2^2$ does not depend on the choice of x. On the other hand, the term $\|Q_1^T b - R_{11} y_1 - R_{12} y_2\|_2^2$ is always non-negative. Can we make it zero? Yes, we can set

$$\|Q_1^T b - R_{11} y_1 - R_{12} y_2\|_2^2 = 0,$$

by requiring that y_1 and y_2 satisfy the equation

$$R_{11} y_1 = Q_1^T b - R_{12} y_2.$$

Since R_{11} is invertible, we can solve this equation by choosing y_2 arbitrarily and then solving for y_1. Thus, the vectors that solve the least squares problem must be of the form

$$x = P \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = P \begin{bmatrix} R_{11}^{-1}(Q_1^T b - R_{12} y_2) \\ y_2 \end{bmatrix}, \tag{32.10}$$

as we wanted to prove. □

Remark 32.14. (Choosing a solution)

A column rank deficient least squares problem has infinitely many solutions because y_2 in (32.10) is arbitrary. One simple and reasonable choice is to set $y_2 = 0$. This gives the least squares solution $P \left[\begin{smallmatrix} y_1 \\ 0 \end{smallmatrix} \right]$, where $y_1 \in \mathbb{R}^r$ is the unique solution of the

upper triangular system $R_{11}y_1 = Q_1^T b$. In Chapter 47, we will introduce a different method that chooses, among all possible solutions, the one with the smallest 2-norm. This approach is often more useful in practice.

Problems

32.1 Find the orthogonal projection matrix onto the subspace

$$\mathcal{W} = \text{span}\left(\left\{\begin{bmatrix} 1 \\ -2 \\ 2 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 2 \\ 2 \end{bmatrix}\right\}\right).$$

32.2 Find the orthogonal projection matrix onto the subspace $\mathcal{V} \subset \mathbb{R}^4$ defined as the set of all solutions to

$$\begin{aligned} x_1 + x_2 + x_3 + x_4 &= 0 \\ x_1 + 2x_2 + 3x_3 + 4x_4 &= 0 \\ 2x_1 + 3x_2 + 4x_3 + 5x_4 &= 0 \end{aligned}$$

32.3 For any orthogonal projection matrix $P \in \mathbb{R}^{n\times n}$, show

$$\text{col}(P) = \{x \in \mathbb{R}^n \mid Px = x\}.$$

32.4 (Product of orthogonal projection matrices) Let $P, Q \in \mathbb{R}^{n\times n}$ be orthogonal projection matrices. Prove that PQ is an orthogonal projection matrix if and only if $PQ = QP$. Additionally, show that in this case, $\text{col}(PQ) = \text{col}(P) \cap \text{col}(Q)$ and $\text{null}(PQ) = \text{null}(P) + \text{null}(Q)$.

32.5 (Sum of orthogonal projection matrices) Let $P, Q \in \mathbb{R}^{n\times n}$ be orthogonal projection matrices. Prove that the following statements are equivalent.

(a) $P + Q$ is an orthogonal projection matrix.
(b) $\text{col}(P) \perp \text{col}(Q)$.
(c) $\text{col}(P) \subseteq \text{null}(Q)$.
(d) $\text{col}(Q) \subseteq \text{null}(P)$.
(e) $PQ = QP = 0$.

If $P + Q$ is an orthogonal projection matrix, what is $\text{col}(P + Q)$?

32.6 (2-norm of orthogonal projection matrices) Show that $\|P\|_2 = 1$ for any orthogonal projection matrix P.

32.7 (Frobenius norm of orthogonal projection matrices) Show that $\|P\|_F^2 = \text{trace}(P) = \dim(\text{col}(P))$ for any orthogonal projection matrix P.

32.8 (Orthogonal projections and left inverses) Let $\mathcal{W}$ be a subspace of $\mathbb{R}^m$, with dimension r, and let $M \in \mathbb{R}^{m\times r}$ be a matrix whose columns form a basis for $\mathcal{W}$. Since M has full column rank, it has a left inverse M_{L} such that $M_{\text{L}}M = I_r$ (see

Theorem 17.50). Prove that the matrix MM_{L} is the orthogonal projection matrix onto $\mathrm{col}(M) = \mathcal{W}$.

32.9 Find the least squares solution of the following linear systems:

$$\text{(a)}\quad \begin{aligned} x_1 + x_2 &= 9 \\ 2x_1 - 2x_2 &= 7 \\ 3x_1 + x_2 &= 11 \end{aligned} \qquad \text{(b)}\quad \begin{aligned} x_1 + 2x_2 + x_3 &= 1 \\ 2x_1 + x_2 + x_3 &= 1 \\ x_1 + 2x_2 + 3x_3 &= 1 \\ 2x_1 + x_2 + 3x_3 &= 1 \end{aligned}$$

32.10 Find all solutions to the following rank-deficient least squares problem $\min_x \|b - Ax\|_2^2$, where

$$\text{(a)}\quad A = \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 6 \end{bmatrix} \qquad \text{(b)}\quad A = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix} \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}.$$

32.11 Solve the least squares problem for

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix}$$

using a QR factorization of A.

32.12 Solve the least squares problem for

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & -1 \\ 1 & 2 & -3 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 6 \\ 0 \\ 0 \end{bmatrix}$$

using a QR factorization with column pivoting of A.

32.13 (Alternative proof of Theorem 32.12) Show that the unique solution of the normal equations $A^T Ax = A^T b$ is $x = R_1^{-1} Q_1^T b$, where $A = Q_1 R_1$ is a reduced QR factorization of A.

32.14 (Alternative derivation of the normal equations) Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. In your multivariable calculus class, you learned that a minimizer of $\rho(x) = \|b - Ax\|_2^2$ must satisfy

$$\frac{\partial \rho(x)}{\partial x_k} = 0, \quad \text{for } k = 1, \ldots, n.$$

Show that these conditions are equivalent to the normal equations $A^T Ax = A^T b$.

32.15 Let A be a full column rank matrix with the full QR decomposition

$$A = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix}.$$

Let x be the solution to the least squares problem $\min_x \|b - Ax\|_2^2$. Prove that the residual vector $r = b - Ax$ can be written as

$$r = (I_m - Q_1 Q_1^T)b = Q_2 Q_2^T b.$$

Interpret these results in terms of orthogonal projections.

32.16 (Effect of adding a column on the least squares residual) Let $A \in \mathbb{R}^{m \times n}$ be a full column rank matrix and $b \in \mathbb{R}^m$. Suppose we add a new column $a_{n+1} \in \mathbb{R}^m$ to A, forming the matrix

$$\widetilde{A} = \begin{bmatrix} A & a_{n+1} \end{bmatrix} \in \mathbb{R}^{m \times (n+1)},$$

and assume $\widetilde{A}$ also has full column rank. Let the reduced QR factorizations of A and $\widetilde{A}$ be given by

$$A = Q_1 R_1 \quad \text{and} \quad \widetilde{A} = \widetilde{Q}\widetilde{R}_1 = \begin{bmatrix} Q_1 & q_{n+1} \end{bmatrix} \begin{bmatrix} R_1 & r_{1,n+1} \\ 0 & r_{n+1,n+1} \end{bmatrix},$$

where $q_{n+1} \in \mathbb{R}^m$ is a unit vector orthogonal to the columns of Q_1. Let $x = \operatorname{argmin}_x \|b - Ax\|_2^2$ and $\widetilde{x} = \operatorname{argmin}_{\widetilde{x}} \|b - \widetilde{A}\widetilde{x}\|_2^2$, and let $r = b - Ax$ and $\widetilde{r} = b - \widetilde{A}\widetilde{x}$ be the residuals for these least squares problems. Prove

$$\|r\|_2^2 = \|\widetilde{r}\|_2^2 + (q_{n+1}^T b)^2.$$

Conclude that adding a new column to a full column rank matrix (while maintaining full column rank) reduces the least squares residual norm or leaves it unchanged, with equality if and only if $q_{n+1}^T b = 0$.

32.17 (Least norm least squares solution) Let $A \in \mathbb{R}^{m \times n}$ be a matrix with $\operatorname{rank}(A) = r < n \leqslant m$. By Theorem 32.11, the least squares problem $\min_x \|b - Ax\|_2^2$ has infinitely many solutions. Let x_{ln} denote the solution with the smallest 2-norm. Use the expression given in (32.10) to reformulate the problem of finding x_{lm} as a full column rank least squares problem.

32.18 Consider the matrix

$$A = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \end{bmatrix},$$

where ϵ is a very small number, so small that a computer rounds $1 + \epsilon^2$ to 1. What happens if you use a computer to calculate $A^T A$ and solve a least squares problem using the normal equations? Now try solving the same problem using the QR factorization of A. Compare the results and explain the difference.

32.19 The following Python code generates a least squares matrix A and vector b with a known solution x.

```
t = np.arange(0,4,0.1)
n = len(t)
```

```
A = np.zeros((n,3))
A[:,0]=1
A[:,1]=np.sin(t)**2
A[:,2]=np.cos(t+ 1e-10*np.random.randn(n))**2
x = [1,1,1]
b = A.dot(x)
```

Use the normal equations to solve the least squares problem for x, and compare your computed solution to the actual solution. How close are they, and what might explain any differences?

32.20 Let ℓ be the line in $\mathbb{R}^3$ that passes through the point p in the direction of the vector $v \neq 0$, and let $y \in \mathbb{R}^3$. Consider the problem of finding the point on ℓ closest to the point y:

$$\min_{x \in \ell} \|x - y\|_2.$$

Explain how to frame this problem as a least squares problem. Find a closed-form solution. What is the shortest distance between y and a point $x \in \ell$?

32.21 Let ℓ_1 be the line in $\mathbb{R}^3$ that passes through the point p in the direction of the vector $v \neq 0$, and let ℓ_2 be the line in $\mathbb{R}^3$ that passes through the point q in the direction of the vector $w \neq 0$. Consider the problem of finding the shortest distance between a point on ℓ_1 and a point on ℓ_2:

$$\min_{x_1 \in \ell_1,\, x_2 \in \ell_2} \|x_1 - x_2\|_2.$$

Explain how to frame this problem as a least squares problem.

32.22 Let P be the plane in $\mathbb{R}^3$ that passes through three (non collinear) points p_1, p_2, p_3, and let $p \in \mathbb{R}^3$. Consider the problem of finding the point on the plane P closest to the point p:

$$\min_{x \in P} \|x - p\|_2.$$

Explain how to frame this problem as a least squares problem.

32.23 Suppose that you want to find the heights $x_1, \ldots, x_m$ at m specified points. You know that $x_1 = 0$, and have estimates for the height differences $d_{ij} \approx x_i - x_j$. To compute $x_2, \ldots, x_m$ you wish to solve the following problem

$$\min_{x_1,\ldots,x_m} \sum_{i=1}^{m-1} \sum_{j=i+1}^{m} (x_i - x_j - d_{ij})^2$$

subject to $x_1 = 0$. Explain how to frame this problem as a least squares problem.

32.24 Let $A \in \mathbb{R}^{m \times n}$ with $m < n$ and full row rank. Consider the least squares problem $\min_x \|Ax - b\|_2^2$. When x is a least squares solution, how small can the residual norm $\|Ax - b\|_2$ be?

Chapter 33
VII-SUPPLEMENTARY TOPICS

Many variants of least squares problems appear in Statistics, Applied Mathematics, and Machine Learning. In this chapter, we introduce some common variants, show how to solve them using tools we have already learned, and highlight a few of their applications.

33.1 Variants of least squares problems

This section presents several common variants of the least squares problem and how to solve them.

33.1.1 Weighted least squares

In a standard least squares problem, we have a matrix $A \in \mathbb{R}^{m\times n}$ and a vector $b \in \mathbb{R}^m$. The goal is to minimize

$$\|b - Ax\|_2^2 = \sum_{i=1}^{m} (b_i - (Ax)_i)^2.$$

Each term in the sum represents the squared error for one equation, $(Ax)_i = b_i$, and we assume all errors are equally important.

But this is not always true. Sometimes, some errors matter more than others, perhaps because they represent more important measurements or carry more significance in the problem. In such cases, we can adjust the problem to reflect these differences by using weighted least squares, which assigns different weights to the errors.

Definition 33.1. (Weighted least squares problem)

Given $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$ and positive numbers $w_1, \ldots, w_m$, the **weighted least squares problem** consists in finding a vector $x \in \mathbb{R}^n$ that minimizes

$$\sum_{i=1}^{m} w_i (b_i - (Ax)_i)^2. \tag{33.1}$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_33

Any vector x that minimizes (33.1) is called a **weighted least squares solution**.

The positive numbers $w_1, w_2, \ldots, w_m$ are weights that show how much each error matters. Larger weights mean those errors are more important, so the solution focuses more on reducing them. Smaller weights mean the errors matter less and can be a bit bigger. This lets us decide which errors to prioritize.

The following theorem characterizes all the solutions of a weighted least squares problem and shows when the solution is unique.

Theorem 33.2. (Weighted least squares problem solution)

Consider a weighted least squares problem of the form (33.1), where $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$, and w_i are some positive numbers. Let $W := \operatorname{diag}(w_1, \ldots, w_m)$. Then, the following statements hold:

(a) The set of all weighted least squares solutions is the set of solutions of the linear system

$$A^T W A x = A^T W b. \tag{33.2}$$

This linear system is always solvable.

(b) The weighted least squares solution is unique if and only if A has full column rank. In this case, the weighted least squares solution is given by the formula

$$x = (A^T W A)^{-1} A^T W b. \tag{33.3}$$

Proof The key idea behind proving the results is to rewrite the weighted sum of squared errors in (33.1) as

$$\begin{aligned}\sum_{i=1}^{m} w_i (b_i - (Ax)_i)^2 &= \|\operatorname{diag}(w_1^{1/2}, \ldots, w_m^{1/2})(b - Ax)\|_2^2 \\ &= \|W^{1/2}(b - Ax)\|_2^2 = \|W^{1/2}b - W^{1/2}Ax\|_2^2,\end{aligned}$$

where $W^{1/2} := \operatorname{diag}(w_1^{1/2}, \ldots, w_m^{1/2})$ is a diagonal matrix with diagonal entries $w_i^{1/2}$. Thus, we can express the weighted least squares problem as the least squares problem

$$\min_{x\in\mathbb{R}^n} \|(W^{1/2}b - W^{1/2}Ax)\|_2^2.$$

By defining $\widetilde{b} = W^{1/2}b$ and $\widetilde{A} = W^{1/2}A$, we can apply Theorem 32.11 to the least squares problem $\min_x \|\widetilde{b} - \widetilde{A}x\|_2^2$ to prove all the statements. □

Solving weighted least squares problems

The proof of Theorem 33.2 highlights an important property of the weighted least squares problem (33.1). We can rewrite it as a standard least squares problem

$$\min_{x\in\mathbb{R}^n} \|\widetilde{b} - \widetilde{A}x\|_2^2,$$

where $\widetilde{b} = W^{1/2}b$ and $\widetilde{A} = W^{1/2}A$. This allows us to use the QR factorization methods we studied in Chapter 32 directly to solve weighted least squares problems without needing to form the matrix A^TWA.

33.1.2 Regularized least squares

In regularized least squares problems, we add an extra term to the least squares problem. This term helps shrink the entries of the solution, pushing them closer to zero.

Definition 33.3. (Regularized least squares problem)

Given a matrix $A \in \mathbb{R}^{m\times n}$, a vector $b \in \mathbb{R}^m$, and a scalar $\lambda \geqslant 0$, the **regularized least squares problem** is to find a vector $x \in \mathbb{R}^n$ that solves the following problem

$$\min_{x\in\mathbb{R}^n} \left(\|b - Ax\|_2^2 + \lambda\|x\|_2^2\right). \tag{33.4}$$

The nonnegative scalar λ is known as the **regularization parameter**.

The term $\lambda\|x\|_2^2$ adds a penalty that grows with the size of the entries in x. This encourages the solution to have smaller entries, while still aiming to make $Ax \approx b$. The regularization parameter λ controls how strong this effect is. When $\lambda = 0$, this additional term has no effect, and the solution is simply the least squares solution. However, if λ is very large, the added term becomes more important, pushing the entries of x toward zero.

Theorem 33.4. (Regularized least squares solution)

Given $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$, and a scalar $\lambda > 0$ the regularized least squares problem (33.4) has a unique solution that satisfies the linear system

$$(A^TA + \lambda I_n)x = A^Tb.$$

Proof We can rewrite the regularized least squares problem (33.4) as a (larger) least squares problem. We have

$$\min_{x\in\mathbb{R}^n} \|b - Ax\|_2^2 + \lambda\|x\|_2^2 = \min_{x\in\mathbb{R}^n} \left\| \begin{bmatrix} b - Ax \\ \sqrt{\lambda}x \end{bmatrix} \right\|_2^2 =$$
$$\min_{x\in\mathbb{R}^n} \left\| \begin{bmatrix} b \\ 0 \end{bmatrix} - \begin{bmatrix} A \\ \sqrt{\lambda}I_n \end{bmatrix} x \right\|_2^2 = \min_{\mathbb{R}^n} \|\widehat{b} - \widehat{A}x\|_2^2,$$

where $\widehat{b} := \begin{bmatrix} b \\ 0 \end{bmatrix}$ and $\widehat{A} := \begin{bmatrix} A \\ \sqrt{\lambda} I_n \end{bmatrix}$. The normal equations for this least squares problem are

$$\widehat{A}^T \widehat{A} x = \begin{bmatrix} A^T & \sqrt{\lambda} I_n \end{bmatrix} \begin{bmatrix} A \\ \sqrt{\lambda} I_n \end{bmatrix} x = (A^T A + \lambda I_n) x = \widehat{A}^T \widehat{b} = \begin{bmatrix} A^T & \sqrt{\lambda} I_n \end{bmatrix} \begin{bmatrix} b \\ 0 \end{bmatrix} = A^T b.$$

So, any solution to the regularized least squares problem satisfies the equation

$$(A^T A + \lambda I_n) x = A^T b.$$

By Theorem 32.11, the solution is unique if $\widetilde{A}$ has full column rank. So, we need to show that $\widehat{A}$ has this property

Suppose $\widehat{A} x = 0$ for some vector x. Then, we have

$$\widehat{A} x = \begin{bmatrix} A \\ \sqrt{\lambda} I_n \end{bmatrix} x = \begin{bmatrix} Ax \\ \sqrt{\lambda} x \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

This implies $Ax = 0$ and $\sqrt{\lambda} x = 0$. Since $\sqrt{\lambda} > 0$, we conclude that x must be zero. Therefore $\widehat{A}$ has full column rank. □

Solving regularized least squares problems

The proof of Theorem 33.4 shows that the regularized least squares problem (33.4) is equivalent to the larger standard least squares problem

$$\min_{x \in \mathbb{R}^n} \| \begin{bmatrix} A \\ \sqrt{\lambda} I_n \end{bmatrix} x - \begin{bmatrix} b \\ 0 \end{bmatrix} \|_2^2. \tag{33.5}$$

The matrix $\begin{bmatrix} A \\ \sqrt{\lambda} I_n \end{bmatrix}$ is a tall-and-skinny matrix with full column rank. To solve (33.5), we start by computing a reduced QR factorization

$$\begin{bmatrix} A \\ \sqrt{\lambda} I_n \end{bmatrix} = QR = \begin{matrix} m \\ n \end{matrix} \begin{bmatrix} Q_1 \\ Q_2 \end{bmatrix} R,$$

where $R \in \mathbb{R}^{n \times n}$ is a nonsingular upper triangular matrix, and $Q \in \mathbb{R}^{(m+n) \times n}$ is a matrix with orthonormal columns. We have partitioned Q so that its top m rows are in Q_1 and its bottom n rows are in Q_2. From Theorem 32.12, the solution to (33.5) satisfies the upper triangular system

$$Rx = \begin{bmatrix} Q_1^T & Q_2^T \end{bmatrix} \begin{bmatrix} b \\ 0 \end{bmatrix} = Q_1^T b.$$

Since R is upper triangular, we solve this system using backward substitution.

33.1.3 Linearly constrained least squares

In a linearly constrained least squares problem, we minimize $\|b - Ax\|_2^2$ while requiring that x satisfies additional conditions, also known as "constraints", given as a linear system of equations.

Definition 33.5. (Linearly constrained least squares problem)

Given matrices $A \in \mathbb{R}^{m\times n}$ and $C \in \mathbb{R}^{p\times n}$, and vectors $b \in \mathbb{R}^m$ and $d \in \mathbb{R}^p$, the **linearly constrained least squares problem** is to find a vector $x \in \mathbb{R}^n$ that solves the following problem

$$\begin{aligned} &\text{minimize} && \|b - Ax\|_2^2 \\ &\text{subject to} && Cx = d. \end{aligned} \tag{33.6}$$

This type of problem arises, for example, in curve fitting when we use different functions to fit the data over various subintervals. We will see a simple example of this in Section 33.2.

The following theorem characterizes all the solutions to a linearly constrained least squares problem and identifies the conditions for uniqueness of the solution.

Theorem 33.6. (Linearly constrained least squares solutions)

Let $A \in \mathbb{R}^{m\times n}$, $C \in \mathbb{R}^{p\times n}$, $b \in \mathbb{R}^m$ and $d \in \mathbb{R}^p$. Then, the following statements hold:

(a) The set of all solutions x of the constrained least squares problem (33.6) corresponds to the solutions of the linear system

$$\begin{bmatrix} 2A^TA & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} = \begin{bmatrix} 2A^Tb \\ d \end{bmatrix}, \tag{33.7}$$

known as the **Karush-Kuhn-Tucker (KKT)**[1] system of equations.

(b) The solution to the constrained least squares problem (33.6) is unique if and only if C has full row rank, and $\begin{bmatrix} A \\ C \end{bmatrix}$ has full column rank.

Proof First, we show that if x solves the constrained least squares problem (33.6), then x must satisfy the KKT equations (33.7), for some $z \in \mathbb{R}^p$. So the KKT equations are necessary. To do this, we will use the method of Lagrange multipliers.

To apply the method of Lagrange multipliers, we first construct the Lagrangian function

$$L(x, z) = \|Ax - b\|_2^2 + z^T(Cx - d),$$

[1] The Karush-Kuhn-Tucker (KKT) system is named after the mathematicians William Karush, Harold W. Kuhn, and Albert W. Tucker, who developed these conditions to solve optimization problems with constraints.

where $z \in \mathbb{R}^p$ is the vector of Lagrange multipliers corresponding to each constraint (i.e., each row of $Cx = d$). According to the method of Lagrange multipliers, if x solves the constrained least squares problem (33.6), then there exists a vector z of Lagrange multipliers that satisfies the optimality conditions

$$\begin{aligned} \frac{\partial L}{\partial x_k} &= 0, \quad \text{for } k = 1, \ldots, n, \quad \text{and} \\ \frac{\partial L}{\partial z_k} &= 0, \quad \text{for } k = 1, \ldots, p. \end{aligned} \tag{33.8}$$

Our goal now is to show that the optimality conditions in (33.8) are equivalent to the KKT conditions (33.7).

We can write the Lagrangian function $L(x, z)$ as

$$L(x, z) = \sum_{i=1}^{m} \left(b_i - \sum_{j=1}^{n} A_{ij} x_j \right)^2 + \sum_{i=1}^{p} z_i \left(\sum_{j=1}^{n} C_{ij} x_j - d_i \right).$$

Taking partial derivatives of L with respect to x_k, for $k = 1, \ldots, n$, and z_k, for $k = 1, \ldots, p$, yields

$$\frac{\partial L}{\partial x_k} = 2 \sum_{i=1}^{m} \left(b_i - \sum_{j=1}^{n} A_{ij} x_j \right) (-A_{ik}) + \sum_{i=1}^{p} z_i C_{ik}, \quad \text{and} \tag{33.9}$$

$$\frac{\partial L}{\partial z_k} = \sum_{j=1}^{n} C_{kj} x_j - d_k. \tag{33.10}$$

Moreover, we can simplify the expressions (33.9) and (33.10) by using matrix notation. We can write

$$\begin{aligned} \sum_{i=1}^{m} A_{ik} b_i &= 2(A^T b)_k, \\ \sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij} A_{ik} x_j &= 2(A^T A x)_k, \\ \sum_{i=1}^{p} C_{ik} z_i &= (C^T z)_k, \end{aligned}$$

and

$$\sum_{j=1}^{n} C_{kj} x_j - d_k = (Cx - d)_k.$$

As a result, we can simplify the partial derivatives in (33.9) and (33.10) to

$$\frac{\partial L}{\partial x_k} = (-2A^T b + 2A^T Ax + C^T z)_k, \quad \text{and}$$
$$\frac{\partial L}{\partial z_k} = (Cx)_k.$$

Therefore, we can express the optimality conditions (33.8) as the system of equations

$$2A^T Ax + C^T z = 2A^T b, \quad \text{and}$$
$$Cx = d,$$

which corresponds to the KKT conditions (33.7).

Next, we show that any x solving the KKT equations (33.7) minimizes $\|b - Ax\|_2^2$ among all vectors that satisfy the constrains. Thus, the KKT equations are sufficient.

Let x and z satisfy the KKT conditions $2A^T Ax + C^T z = 2A^T b$ and $Cx = d$, and let y be a vector that satisfies $Cy = d$. Using the fact that $\|a+b\|_2^2 = \|a\|_2^2 + \|b\|_2^2 + 2b^T a$, we have

$$\begin{aligned}\|b - Ay\|_2^2 =& \|b - Ax - A(y - x)\|_2^2 = \\ & \|b - Ax\|_2^2 + \|A(y - x)\|_2^2 - 2(A(y - x))^T (b - Ax).\end{aligned}$$

Expanding the last term, we obtain

$$\begin{aligned}2(A(y - x))^T (b - Ax) =& 2(y - x)^T A^T (b - Ax) = \\ & (x - y)^T C^T z = \\ & (Cy - Cx)^T z = 0,\end{aligned}$$

where the second equality follows from the KKT condition $2A^T Ax + C^T z = 2A^T b$, and the last equality holds because $Cx = d = Cy$. Therefore, we have

$$\|b - Ay\|_2^2 = \|b - Ax\|_2^2 + \|A(y - x)\|_2^2 \geqslant \|b - Ax\|_2^2,$$

as we wanted to show.

To prove part (b), we show that the $(n + p) \times (n + p)$ matrix

$$\begin{bmatrix} 2A^T A & C^T \\ C & 0 \end{bmatrix}, \tag{33.11}$$

known as the **KKT matrix**, is invertible if and only if C has full row rank, and $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right]$ has full column rank.

First, let us assume that C has full row rank and $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right]$ has full column rank, and show that the nullspace of the KKT matrix (33.11) consists only of the zero vector, which implies that the KKT matrix is invertible. Let $\left[\begin{smallmatrix} x \\ z \end{smallmatrix}\right]$ be a vector in the nullspace of the KKT matrix (33.11). Then, we have

$$\begin{bmatrix} 2A^T A & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \text{or, equivalently,} \quad \begin{aligned} &2A^T Ax + C^T z = 0, \\ &Cx = 0. \end{aligned} \tag{33.12}$$

We left-multiply the top equation of (33.12) by x^T and use the fact that $Cx = 0$ to get

$$2x^T A^T Ax + x^T C^T x = 2x^T A^T Ax = 2\|Ax\|_2^2 = 0.$$

Thus, $Ax = 0$. Since $Cx = 0$ as well, we have $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right] x = 0$. Since $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right]$ has full column rank, we conclude that $x = 0$. Substituting $x = 0$ into the top equation of (33.12), we get $C^T z = 0$. Since C has full row rank (and thus, C^T has full column rank), we have $z = 0$ as well. Therefore, the nullspace of the KKT matrix contains only the zero vector, and the KKT matrix is invertible.

Next, we show that if C does not have full row rank or if $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right]$ does not have full column rank, then the KKT matrix in (33.11) is singular.

First, suppose that C does not have full row rank. This implies that there is a nonzero vector v such that $v^T C = 0$. Then, we have

$$\begin{bmatrix} 0 & v^T \end{bmatrix} \begin{bmatrix} 2A^T A & C^T \\ C & 0 \end{bmatrix} = \begin{bmatrix} v^T C & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix}.$$

Therefore, the KKT matrix does not have full row rank, and hence, it is not invertible.

Second, suppose that $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right]$ does not have full column rank. Then, there exists a nonzero vector x such that $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right] x = \left[\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}\right]$. Then, we can write

$$\begin{bmatrix} 2A^T A & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} x \\ 0 \end{bmatrix} = \begin{bmatrix} 2A^T Ax \\ Cx \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

It follows that the KKT matrix does not have full column rank, and therefore, it is not invertible.

In conclusion, we have shown that the KKT matrix is invertible, and thus the solution to the KKT system of equations (33.7) is unique, if and only if C has full row rank, and $\left[\begin{smallmatrix} A \\ C \end{smallmatrix}\right]$ has full column rank. □

We can solve a constrained least squares problem (33.6) by solving the KKT equations (33.7). However, just like with standard least squares problems, there are alternative methods that avoid forming the matrix $A^T A$.

First, let us see how to remove the constraints and reduce the problem to a standard least squares problem.

Transforming constrained into unconstrained least squares problems

By Theorem 17.18, we can express the set of all solutions to $Cx = d$ as

$$\{x = x_0 + Nz \mid z \in \mathbb{R}^k\},$$

where x_0 is a particular solution to $Cx = d$, k is the dimension of null(C), and $N \in \mathbb{R}^{n \times k}$ is a matrix whose columns form a basis for null(C). Using this representation, we can rewrite the least squares error term

$$\|b - Ax\|_2^2 = \|b - Ax_0 - ANz\|_2^2 = \|\widetilde{b} - \widetilde{A}z\|_2^2,$$

where $\widetilde{b} = b - Ax_0 \in \mathbb{R}^m$ and $\widetilde{A} = AN \in \mathbb{R}^{m\times k}$. This reduces the constrained problem to the (unconstrained) least squares problem

$$\min_{z\in\mathbb{R}^k} \|\widetilde{b} - \widetilde{A}z\|_2^2.$$

To eliminate the constraints in (33.6) and convert the problem into an unconstrained least squares problem, we need a particular solution of $Cx = d$ and a basis for null(C). A QR factorization of C^T provides both.

Solving constrained least squares problems

For simplicity, we assume that C has full row rank. We start by computing the QR factorization of the full column rank matrix C^T. We have

$$C^T = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix},$$

where $R_1 \in \mathbb{R}^{p\times p}$ is invertible. By Theorem 27.45, the columns of $Q_2 \in \mathbb{R}^{n\times(n-p)}$ are an orthonormal basis for null(C). Define

$$y = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} := Q^T x = \begin{bmatrix} Q_1^T x \\ Q_2^T x \end{bmatrix},$$

and substitute this into $Cx = d$ to get

$$d = Cx = R^T Q^T x = \begin{bmatrix} R_1^T & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = R_1^T y_1.$$

Since R_1^T is invertible (because R_1 is invertible), we solve $Cx = d$ by setting y_1 as the unique solution to $R_1^T y_1 = d$ and choosing y_2 arbitrarily. This gives the solution

$$x = Qy = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = Q_1 y_1 + Q_2 y_2,$$

where $x_0 := Q_1 y_1$ is a particular solution to $Cx = d$, and $Q_2 y_2$, with y_2 arbitrary, represents all vectors in null(C). This is precisely the form of the complete solution to $Cx = d$. With this, the error term in the constrained least squares problem (33.6) becomes

$$\|b - Ax\|_2^2 = \|b - AQy\|_2^2 = \|b - AQ_1 y_1 - AQ_2 y_2\|_2^2 = \|\widetilde{b} - \widetilde{A}y_2\|_2^2,$$

where $\widetilde{b} = b - AQ_1 y_1$ and $\widetilde{A} = AQ_2$.

Let us summarize the steps to find the solution x to (33.6):

1. Factor $C^T = QR$.
2. Compute $AQ = \begin{bmatrix} AQ_1 & AQ_2 \end{bmatrix}$.
3. Solve the lower triangular linear system $R_1^T y_1 = d$ by forward substitution.

4. Solve the least squares problem $\min_{y_2} \|(b - AQ_1y_1) - AQ_2y_2\|_2^2$.
5. Compute $x = Q \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$.

33.2 Applications of least squares variants

This section presents several applications of the least squares problems introduced earlier.

33.2.1 Weighted least squares regression

Imagine you work for a car manufacturer studying how fuel efficiency, measured in miles per gallon (mpg), changes with vehicle speed (mph). Suppose the average relationship follows this equation:

$$\text{mpg} = 60 - 0.2 \times \text{mph}.$$

However, as speed increases, factors like aerodynamic drag and engine efficiency cause the data to deviate more from this true relationship[2]. Figure 33.1 shows the data alongside the true relationship (dashed green line).

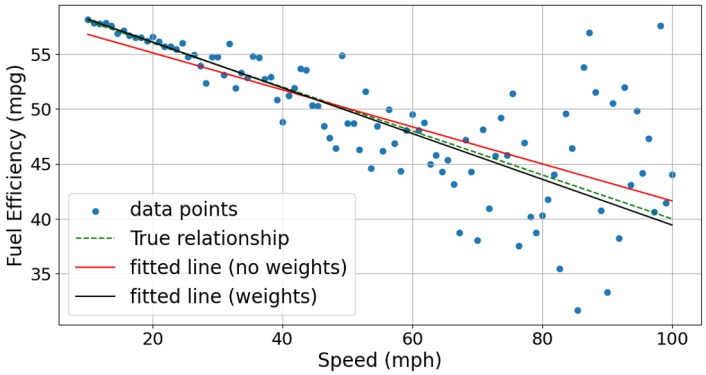


Fig. 33.1: Comparison of fitted lines using least squares and weighted least squares. The least squares line (red) and the weighted least squares line (black) are plotted against the true relationship (dashed green line).

In the previous chapter, we learned to fit a linear model $y = f(t) = c_0 + c_1t$ to data $\{(t_i, y_i)\}_{i=1}^m$ (in our example, t is speed and y is fuel efficiency). We did this by minimizing the sum of squared errors

[2] Statisticians call this phenomenon "heteroskedasticity", a fancy term meaning that the spread of the data points, or the variability, changes across different values of the predictor variable (in this case, speed).

$$\sum_{i=1}^{m}(y_i - f(t_i))^2 = \sum_{i=1}^{m}(y_i - (c_0 + c_1 t_i))^2.$$

Figure 33.1 shows the least squares fitted line in red, which differs slightly from the true relationship (the dashed green line). This happens because the least squares method gives equal importance to all points, allowing the higher-speed points with larger spread to pull the fit away from the true trend.

To account for the increasing spread in fuel efficiency at higher speeds, we assign lower weights to points with higher variability, reducing their influence on the fit. We then minimize a weighted sum of squared errors:

$$\sum_{i=1}^{m} w_i(y_i - (c_0 + c_1 t_i))^2,$$

where each weight $w_i > 0$ reflects how much influence the point (t_i, y_i) has on the fit. Points with larger weights pull the line more, while those with smaller weights have less effect.

In Figure 33.1, we used smaller weights for the higher-speed points, which have more spread. In this toy model, the variance grows like $0.001 \times \text{mph}^2$, so we chose weights proportional to the reciprocal of that. This adjustment reduces the impact of high-speed points and produces a fitted line that stays closer to the true relationship.

The hard part of a weighted least squares problem is choosing the right weights. In our example, we improved the fit because we knew how the spread increased with speed. But in practice, this kind of information is rarely available. Choosing good weights is a deeper topic, and we will not cover it in this book.

In the next subsection, we present an application of weighted least squares where the spread is known, allowing us to choose the weights accordingly.

33.2.2 The distortion caused by data transformations

In Chapter 34, we saw how we can use data transformations to fit certain nonlinear models with least squares regression. However, these transformations can also introduce complications. One common issue, which explains the poor fits in Examples 34.7 and 34.9, is how they can magnify errors already present in the data.

Let us look at an example to understand this better.

Returning to the data from Example 34.8, we added small errors to the y-values of the data points to simulate measurement errors. Figure 33.2 shows the contaminated data, the true model $y = e^{-t}$ (dashed red line), and the fitted model (green line).

The error-contaminated data looks almost identical to the original data (compare the plots in Figures 34.8 and 33.2). However, notice how far the fitted model is from the true model, especially near $t = 0$, where it overestimates y. But why does this happen?

The difference between the true and fitted models happens because the logarithmic transformation $f(x) = \log(x)$ we used to make the model linear. This transformation amplifies small errors in the data, and the amount of amplification depends on the size of y.

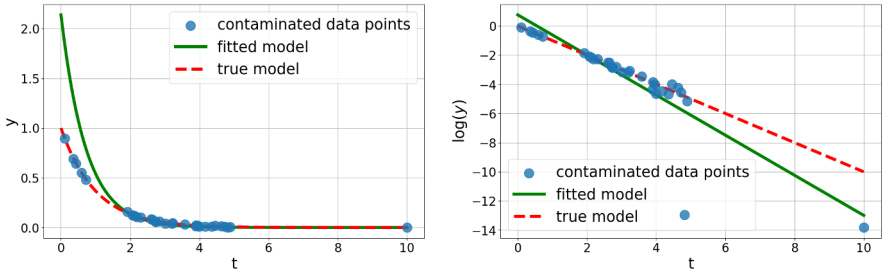


Fig. 33.2: Impact of data errors on model fitting.

To understand how the log transformation affects the errors, suppose we have data points $(t_i, y_i + \Delta y_i)$, where Δy_i represents the difference between the observed and true y-values. After applying the log transformation, these points become $(t_i, \log(y_i + \Delta y_i))$. Using a linear approximation for the log function, we can write

$$\log(y_i + \Delta y_i) \approx \log(y_i) + \frac{\Delta y_i}{y_i}.$$

Now we can better understand Figure 33.2. The transformation amplifies the error Δy_i by a factor of $1/y_i$. Smaller y-values result in greater error amplification. In the right panel of the figure, points with smaller y (on the right) show much larger errors, they deviate more from the true model, compared to points with larger y (on the left). In particular, the large error at the rightmost point strongly influences the slope of the fitted line, pulling it down and affecting the estimated parameter b in the exponential model $y = ae^{bt}$.

In general, when we apply a data transformation $f(x)$ we can be amplifying errors Δy_i by a factor of $f'(y_i)$. How can we minimize this effect? One way is to assign weights to the data points that offset the amplification. Since least squares involves squared terms, we choose weights inversely proportional to the square of the amplification factor:

$$w_i = \left(\frac{1}{|f'(y_i)|}\right)^2.$$

This gives smaller weights to points with large $f'(y_i)$, reducing their impact on the fitted model.

Example 33.7. (Weighted exponential model fitting)

Let us consider the daily Covid-19 deaths data in Example 34.7. Our goal is to fit an exponential model $y = ae^{bt}$ to the first 22 days of data and use it to predict deaths for the next six days.

To fit the model, we apply the logarithmic transformation $f(x) = \log(x)$. This time, to reduce the error amplification caused by the transformation, we solve a weighted least squares problem. The weights are given by

$$w_i = \left(\frac{1}{|f'(y_i)|}\right)^2 = |y_i|^2,$$

where y_i represents the daily death counts.

Figure 33.3 shows the data and the fitted model using weighted least squares. Notice that this model predicts future deaths more accurately compared to the unweighted fit in Example 34.7.

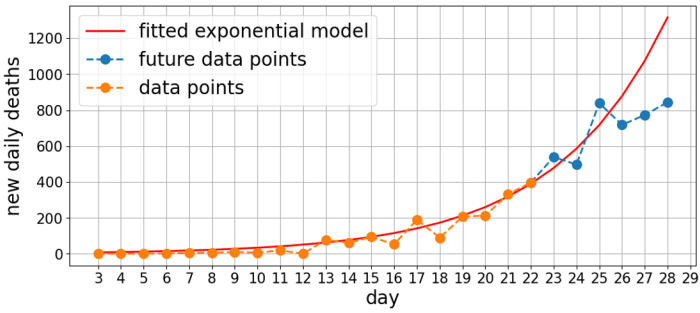


Fig. 33.3: Weighted exponential model fit to Covid-19 deaths data from Example 34.7.

Example 33.8. (Weighted logistic model fitting)

We now consider the influenza outbreak data in Example 34.9. Our goal is to fit a logistic model of the form $y = \frac{1}{1+ae^{-bt}}$ to the first 9 days of data and use it to predict how the virus spreads among the students.

To fit the model, we use the logit transformation $f(x) = \log\left(\frac{x}{1-x}\right)$. To counteract error amplification, we solve a weighted least squares problem with weights given by

$$w_i = \left(\frac{1}{|f'(y_i)|}\right)^2 = |y_i(1-y_i)|^2,$$

where y_i represents the proportion of infected students.

Figure 33.4 shows the data along with the model fitted using weighted least squares. You can see that this approach gives better predictions for the later days than the unweighted fit we used in the earlier example.

33.2.3 Regularized least squares regression

Overfitting happens when a model is too complex for the data it is trying to fit. In Section 34.1.1, we saw how high-degree polynomials can overfit. While they may

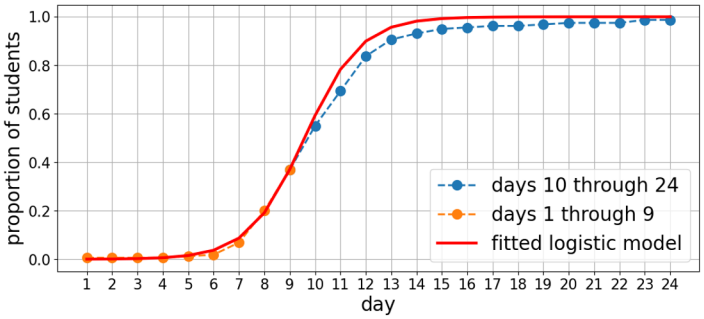


Fig. 33.4: Weighted logistic model fit to the influenza outbreak data from Example 33.8.

fit the given data perfectly, they often fail to generalize to new, unseen data because they capture noise rather than the underlying pattern.

One way to avoid overfitting is to gather more data, though this is not always possible–you work with what you have. Another approach is to simplify the model, such as using a low-degree polynomial. However, sometimes we want to keep the flexibility of a higher-degree polynomial because it can capture useful patterns. In these cases, regularization offers a way to control overfitting without limiting the polynomial's degree.

Recall from Section 33.1.2 that regularization adds a penalty term to the least squares problem, which pushes the model coefficients closer to zero. Instead of minimizing $\|b - Ax\|_2^2$, we now solve

$$\min_{x \in \mathbb{R}^n} \left(\|b - Ax\|_2^2 + \lambda \|x\|_2^2 \right), \quad \lambda > 0.$$

This minimization problem tries to balance fitting the data (minimizing $\|b-Ax\|_2^2$) with keeping the model simple by limiting the size of the coefficients.

As an example, consider the data points in Figure 33.5, generated using the quadratic polynomial $y = 2x^2$. To fit a degree-15 polynomial to these points, we solve a regularized least squares problem with $\lambda = 0$ (no regularization), 0.01 (moderate regularization), and 100 (strong regularization). The left panel shows the data, and the right panel shows the three fitted polynomials for these values of λ.

When $\lambda = 0$, the polynomial overfits the data. It tries to pass through every data point, resulting in a highly oscillating curve that does not generalize well to new data. When $\lambda = 100$, the regularization effect is too strong, forcing the coefficients to be very small. This leads to underfitting, where the model is too simple to capture the pattern in the data. For this example, $\lambda = 0.01$ strikes a good balance. The fitted polynomial captures the data well without overfitting. It is flexible enough to follow the main trend in the data, but not so complex that it starts chasing random noise.

In the example above, we picked a good value for the regularization constant λ by trial and error. There are more advanced methods for this, but that is a story for another book.

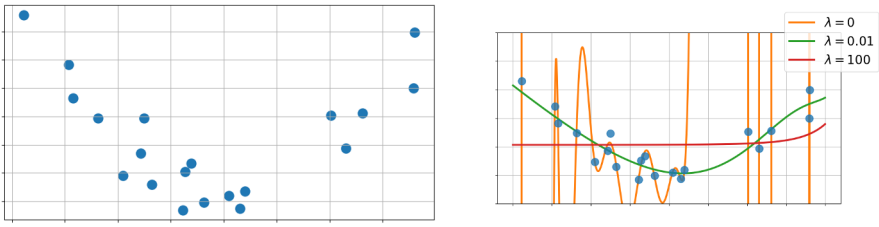


Fig. 33.5: Data points generated from the quadratic polynomial $y = 2x^2$ (left panel), and three fitted degree-15 polynomials (right panel) with different values of the regularization parameter λ.

33.2.4 Piecewise polynomial regression

Another way to prevent overfitting, besides regularized least squares, is piecewise polynomial regression. This method fits separate, usually low-degree polynomials, to different parts of the data instead of relying on a single higher-degree polynomial for the entire dataset.

To see how this works, let us consider the dataset in Figure 33.6, which includes 180 data points $\{(t_i, y_i)\}$. We generated the points to the left of $t = 0$ using a quartic polynomial and the points to the right of $t = 0$ using a quadratic polynomial.

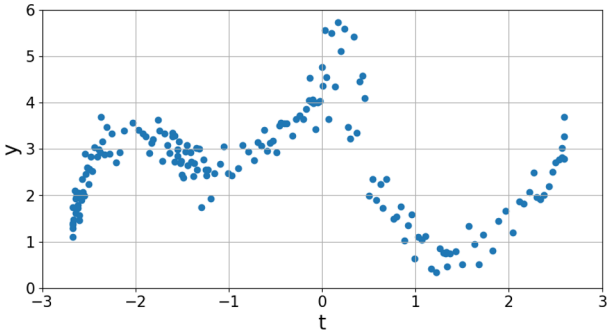


Fig. 33.6: Scatter plot of a toy dataset with 180 points.

We could fit a high-degree polynomial that follows all the ups and downs in the data. But instead of using a single complex model for the entire dataset, we will fit simpler polynomials to different regions. We will use a fourth-degree polynomial for $t \leqslant 0$ and a quadratic for $t > 0$:

$$y = f(t) = \begin{cases} p(t) = \alpha_4 t^4 + \alpha_3 t^3 + \alpha_2 t^2 + \alpha_1 t + \alpha_0 & \text{if } t \leqslant 0, \quad \text{and} \\ q(t) = \beta_2 t^2 + \beta_1 t + \beta_0 & \text{if } t > 0. \end{cases} \tag{33.13}$$

To fit equation (33.13), we use least squares to fit $p(t)$ to the data points left of $t = 0$ and $q(t)$ to the points on the right. The left panel of Figure 33.7 shows the

resulting piecewise polynomial. It follows the data well, but there is a clear issue: it is not continuous at $t = 0$.

To make the curve both continuous and smooth (differentiable) at $t = 0$, we add two constraints:

$$p(0) = q(0) \quad \text{and} \quad p'(0) = q'(0).$$

The right panel of Figure 33.7 shows the piecewise polynomial after applying these constraints. Now, the curve transitions smoothly at $t = 0$, creating a more natural and visually appealing result. We found this polynomial by solving a constrained least squares problem. Let us walk through the process.

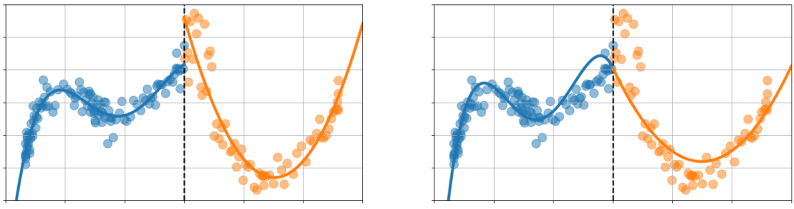


Fig. 33.7: Left: Fitted piecewise polynomial without continuity constraints. Right: Fitted piecewise polynomial with continuity and smoothness at $t = 0$.

To fit the piecewise polynomial $f(t)$ from (33.13), we minimize the sum of squared errors

$$\sum_{i=1}^{m} (y_i - f(t_i))^2.$$

Assuming the data points are ordered such that $t_1, t_2, \ldots, t_M \leqslant 0$ and $t_{M+1}, t_{M+2}, \ldots, t_m > 0$, this becomes

$$\sum_{i=1}^{M} \left(y_i - (\alpha_4 t_i^4 + \alpha_3 t_i^3 + \alpha_2 t_i^2 + \alpha_1 t_i + \alpha_0) \right)^2 + \sum_{i=M+1}^{m} \left(y_i - (\beta_2 t_i^2 + \beta_1 t_i + \beta_0) \right)^2.$$

We can express this minimization as a least squares problem $\min_x \|b - Ax\|_2^2$, where

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & t_1^3 & t_1^4 & 0 & \cdots & 0 \\ 1 & t_2 & t_2^2 & t_2^3 & t_2^4 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_M & t_M^2 & t_M^3 & t_M^4 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & 0 & 1 & t_{M+1} & t_{M+1}^2 \\ 0 & 0 & 0 & \cdots & 0 & 1 & t_{M+2} & t_{M+2}^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 & 1 & t_m & t_m^2 \end{bmatrix}, \quad b = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_M \\ y_{M+1} \\ y_{M+2} \\ \vdots \\ y_m \end{bmatrix}, \quad \text{and} \quad x = \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}.$$

To guarantee continuity and differentiability at $t = 0$, we impose two additional constraints: $p(0) = q(0)$ and $p'(0) = q'(0)$. In terms of the coefficients $\alpha_0, \alpha_1, \alpha_2, \alpha_3, \alpha_4$ and $\beta_0, \beta_1, \beta_2$, these constraints become

$$\begin{aligned}
&\alpha_4 t^4 + \alpha_3 t^3 + \alpha_2 t^2 + \alpha_1 t + \alpha_0 = \beta_2 t^2 + \beta_1 t + \beta_0 \quad \text{at } t = 0, \quad \text{and} \\
&4\alpha_4 t^3 + 3\alpha_3 t^2 + 2\alpha_2 t + \alpha_1 = 2\beta_2 t + \beta_1 \quad \text{at } t = 0,
\end{aligned}$$

which simplifies to $\alpha_0 = \beta_0$ and $\alpha_1 = \beta_1$. We can express these constraints as a linear system $Cx = d$. In our example, this linear system is

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

In conclusion, finding the polynomial coefficients that minimize the sum of squared errors while satisfying $p(0) = q(0)$ and $p'(0) = q'(0)$ requires solving a problem of the form

$$\begin{aligned}
&\text{minimize} && \|b - Ax\|_2^2 \\
&\text{subject to} && Cx = d.
\end{aligned}$$

This is a constrained least squares problem, which we have studied and learned how to solve in Section 33.1.3.

Though this example may look simple, the smooth shape of the fitted polynomial in the right panel of Figure 33.7 is surprisingly satisfying. It captures the idea behind a powerful regression method called splines. We will not cover splines in this book, but they are definitely worth learning about.

Problems

33.1 Solve the following linearly constrained least squares problem

$$\begin{aligned}
&\min_{x \in \mathbb{R}^3} \left\| \begin{bmatrix} x_1 + x_2 - 1 \\ x_1 + x_2 - 3 \\ x_3 - 2 \end{bmatrix} \right\|_2^2 \\
&\text{subject to} \quad x_1 = x_2.
\end{aligned}$$

33.2 (Distance from a point to a line) Let $\ell = \left\{ x = \left[\begin{smallmatrix} x_1 \\ x_2 \end{smallmatrix} \right] : ax_1 + bx_2 + c = 0 \right\}$, where $\left[\begin{smallmatrix} a \\ b \end{smallmatrix} \right] \neq 0$, be a line in $\mathbb{R}^2$, and let $p = \left[\begin{smallmatrix} p_1 \\ p_2 \end{smallmatrix} \right]$. Define the distance from the point p to the line ℓ as the shortest distance from a point $z = \left[\begin{smallmatrix} z_1 \\ z_2 \end{smallmatrix} \right]$ in ℓ to p, that is,

$$\operatorname{dist}(\ell, p) := \min_{z \in \ell} \|z - p\|_2.$$

(a) Express the problem of finding $\text{dist}(\ell, p)$ as a linearly constrained least squares problem.
(b) Show that the point $z \in \ell$ closest to p is

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} p_1 \\ p_2 \end{bmatrix} - \left(\frac{ap_1 + bp_2 + c}{a^2 + b^2} \right) \begin{bmatrix} a \\ b \end{bmatrix}.$$

(c) Conclude that

$$\text{dist}(\ell, p) = \frac{|ap_1 + bp_2 + c|}{\sqrt{a^2 + b^2}}.$$

33.3 (Least squares ranking of teams) In a local Ultimate Frisbee league, six teams (Team A through Team F) compete against each other. The outcomes of their matches are recorded in Table 33.1.

Winning team	Losing team	Scores
Team A	Team D	15-9
Team B	Team E	15-12
Team C	Team F	15-8
Team A	Team B	15-14
Team C	Team D	15-11
Team E	Team F	14-12
Team A	Team C	15-12
Team B	Team F	15-10
Team E	Team D	15-14

Table 33.1: Results of matches between teams A through F in a local Ultimate Frisbee league.

After these matches, you want to rank the six teams, that is, assign each team a number that reflects its overall performance. A common method is to assign ranking points $r_A, r_B, \ldots, r_F$ so that the difference in ranking points between two teams is close to the score difference in their game. For example, if Team A beats Team D with a score of 15-9, then we expect $r_A - r_D \approx 15 - 9 = 6$. Once we have the ranking points, we can sort the teams accordingly. So the goal is to find ranking points such that:

$$
\begin{aligned}
r_A - r_D &\approx 6,\\
r_B - r_E &\approx 3,\\
r_C - r_F &\approx 7,\\
r_A - r_B &\approx 1,\\
r_C - r_D &\approx 4,\\
r_E - r_F &\approx 2,\\
r_A - r_C &\approx 3,\\
r_B - r_F &\approx 5, \quad \text{and}\\
r_E - r_D &\approx 1.
\end{aligned}
$$

In addition, you fix the total number of ranking points $r_A + r_B + \cdots + r_F = 100$. Find the ranking points $r_A, \ldots, r_F$ by solving a linearly constrained least squares problem, and based on your results, rank the six Ultimate Frisbee teams.

33.4 Given a matrix $A \in \mathbb{R}^{m\times n}$, with $m > n$, vectors $b \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$, and a scalar $d \in \mathbb{R}$, explain how the constrained least squares problem

$$
\begin{aligned}
&\text{minimize} && \|b - Ax\|_2^2\\
&\text{subject to} && c^T x = d
\end{aligned}
$$

can be transformed into an unconstrained least squares problem.

Hint: Find a particular solution to $c^T x = d$, and explain how to find a basis for $\text{null}(c^T)$.

33.5 (Explicit solution to linearly constrained least squares problems) Consider the linearly constrained least squares problem (33.6). If $A^T A$ is invertible and C has full row rank, show that the problem has a unique solution given by

$$
x = (A^T A)^{-1}\left(A^T b - C^T (C(A^T A)^{-1} C^T)^{-1}(C(A^T A)^{-1} A^T b - d)\right).
$$

33.6 (Least norm solution of underdetermined linear systems) Given a matrix $A \in \mathbb{R}^{m\times n}$ with $m < n$, a vector $b \in \mathbb{R}^m$, and assuming A has full row rank, the linear system $Ax = b$ is always solvable and has infinitely many solutions. The **least norm solution** x_{ln} is defined as the solution of the following problem

$$
\begin{aligned}
&\text{minimize} && \|x\|_2\\
&\text{subject to} && Ax = b.
\end{aligned}
\tag{33.14}
$$

In words, among all possible solutions to $Ax = b$, x_{ln} has the smallest 2-norm. Write (33.14) as a linearly constrained least squares problem (33.6), and show that $x_{\text{ln}} = A^T (AA^T)^{-1} b$.

33.7 Find the least norm solution $x = \left[\begin{smallmatrix} x_1 \\ x_2 \end{smallmatrix}\right]$ of the linear system $x_1 + x_2 = 1$.

33.8 (Direct verification of the least norm property) Consider a full row rank matrix $A \in \mathbb{R}^{m\times n}$ and a vector $b \in \mathbb{R}^m$. We aim to show that $x_{\text{ln}} = A^T(AA^T)^{-1}b$ is the least norm solution to the linear system $Ax = b$.

(a) Verify that x_{ln} satisfies $Ax_{\text{ln}} = b$.
(b) For any solution x to $Ax = b$, prove that $x - x_{\text{ln}}$ is orthogonal to x_{ln}.
(c) Show $\|x\|_2^2 = \|x_{\text{ln}}\|_2^2 + \|x - x_{\text{ln}}\|_2^2$ and deduce that $\|x\|_2 \geqslant \|x_{\text{ln}}\|_2$.

33.9 Given a full row rank matrix $A \in \mathbb{R}^{m\times n}$ and a vector $b \in \mathbb{R}^m$, prove that a solution x to the linear system $Ax = b$ is the least norm solution if and only if $x \in \text{null}(A)^{\perp} = \text{col}(A^T)$.

33.10 Explore the impact of weighting on linear fitting by generating the following data in Python:

```
n = 20
t = np.random.rand(n)
y = 3*t
s = t**2
y += np.random.normal(0, s)
```

Use both least squares and weighted least squares with weights $w_i = (1/s_i)^2$ to fit a line to this data. Plot the data points along with both fitted lines. Determine which fitted line is closer to the true relationship $y = 3t$.

33.11 Let $\ell_i = \left\{x = \left[\begin{smallmatrix} x_1 \\ x_2 \end{smallmatrix}\right] \,:\, a_i x_1 + b_i x_2 + c_i = 0\right\}$, where $\left[\begin{smallmatrix} a_i \\ b_i \end{smallmatrix}\right] \neq 0$, for $i = 1, \ldots, m$, be m lines in $\mathbb{R}^2$. Consider the problem of finding the point $p \in \mathbb{R}^2$ that minimizes the following sum-of-squares

$$\min_{p\in\mathbb{R}^2} \sum_{i=1}^{m} \text{dist}(\ell_i, p)^2,$$

where $\text{dist}(\ell_i, p)$ is the distance from the point p to the line ℓ_i (see Problem 33.2). Show that the problem of finding p can be framed as a weighted least squares problem.

33.12 The following Python code generates a set of data points (t, y) based on the equation $y = \sin(2\pi t)$:

```
n = 10
t = np.linspace(0,1, num=n)
y = np.sin(2*np.pi*t) + 0.5*np.random.randn(n)
```

Fit a degree-9 polynomial to this data by solving a regularized least squares problem. Experiment with different values of the regularization constant λ. Figure 33.8 shows examples of polynomial fits for various λ values to help guide your experiments.

33.13 (Push-through identity) Consider a matrix $A \in \mathbb{R}^{m\times n}$ and a scalar $\lambda > 0$.

(a) Recall from the proof of Theorem 33.4 that the matrix $A^TA + \lambda I_n$ is invertible. Prove that the matrix $AA^T + \lambda I_m$ is also invertible.

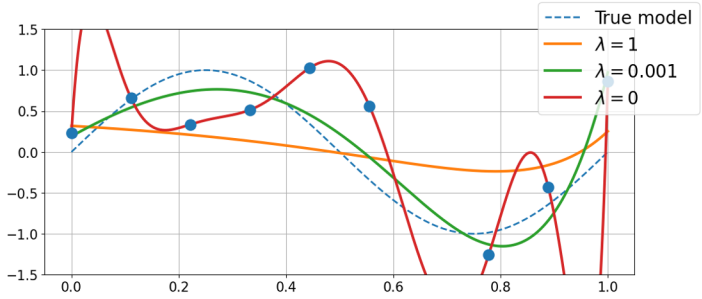


Fig. 33.8: Polynomial fits of degree 9 to the data generated from $y = \sin(2\pi t)$ with added noise, using various values of the regularization constant λ.

(b) Show that $(A^TA + \lambda I_n)^{-1}A^T = A^T(AA^T + \lambda I_m)^{-1}$. This is called the **push-through identity**, as it shows how the matrix A^T can be "pushed through" the inverse.
(c) Conclude that $x = A^T(AA^T + \lambda I_m)^{-1}b$ is an alternative formula for the solution of the regularized least squares problem (33.4).

33.14 Consider a full row rank matrix $A \in \mathbb{R}^{m\times n}$, a vector $b \in \mathbb{R}^m$, and $x_{\text{ln}} = A^T(AA^T)^{-1}b$, the least norm solution to $Ax = b$ (recall Problem 33.6). Prove that

$$x_{\text{ln}} = \lim_{\lambda\to 0^+} (A^TA + \lambda I_n)^{-1}A^Tb.$$

In other words, as the constant $\lambda > 0$ tends to zero, the regularized least squares solution approaches the least norm solution.

33.15 (Regularized least squares solution as a least norm solution) Consider a matrix $A \in \mathbb{R}^{m\times n}$, a vector $b \in \mathbb{R}^m$, and a scalar $\lambda > 0$. Define the matrix $\widehat{A} = \begin{bmatrix} A & \sqrt{\lambda} I_m \end{bmatrix} \in \mathbb{R}^{m\times(n+m)}$. Consider the problem

$$\begin{array}{ll} \text{minimize} & \|v\|_2 \\ \text{subject to} & \widehat{A}v = b. \end{array}$$

If the solution v of this problem is partitioned as $v = \begin{bmatrix} x \\ y \end{bmatrix}$, where $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$, prove that x is the solution to the regularized least squares problem (33.4).

33.16 (Regularized least squares variant I) Given matrices $A \in \mathbb{R}^{m\times n}$ and $W \in \mathbb{R}^{p\times n}$, and a vector $b \in \mathbb{R}^m$, prove that the regularized least squares problem

$$\min_{x\in\mathbb{R}^n} \left(\|b - Ax\|_2^2 + \|Wx\|_2^2 \right) \tag{33.15}$$

has a unique solution if and only if $\text{null}(A) \cap \text{null}(W) = \{0\}$. Show that this unique solution is given by $x = (A^TA + W^TW)^{-1}A^Tb$.

33.17 (Total squared variation denoising) Suppose we have a vector $y \in \mathbb{R}^n$ contaminated with errors. Total squared variation denoising seeks a "smoother"

approximation x to y. Mathematically, we aim to solve

$$\min_{x\in\mathbb{R}^n}\left(\|x-y\|_2^2+\lambda\sum_{i=1}^{n-1}(x_{i+1}-x_i)^2\right), \tag{33.16}$$

where the regularization constant $\lambda > 0$ controls the trade-off between approximating y and keeping x smooth.

(a) Express (33.16) as a regularized least squares problem (33.15) for an appropriate matrix W, and find a formula for its solution.
(b) Consider the following vector y generated in Python

```
y = 2*np.ones(200)
y[75:125] = 0
y += 0.5*np.random.randn(len(y)) # add random noise
```

Apply total squared variation denoising with $\lambda = 1, 10, 100$ to find smoother approximations to y. Plot the entries of the vector y and the approximations (in different colors) for comparison. See Figure 33.9 for an example of the expected plot, where the entries of y are plotted in blue.

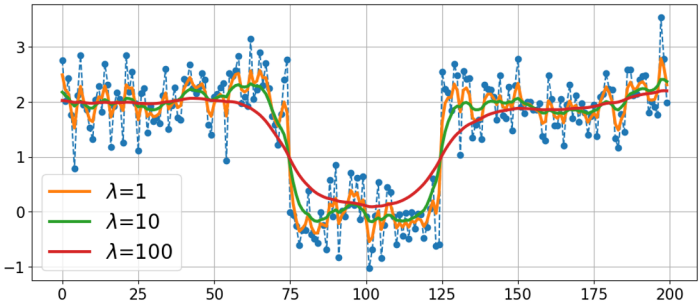


Fig. 33.9: Total squared variation denoising with different λ values.

33.18 (Regularized least squares variant II) Given matrices $A \in \mathbb{R}^{m\times n}$, $W_1 \in \mathbb{R}^{p\times m}$, and $W_2 \in \mathbb{R}^{q\times n}$, along with vectors $b \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$, consider the optimization problem

$$\min_{x\in\mathbb{R}^n}\left(\|W_1(b-Ax)\|_2^2+\|W_2(x-c)\|_2^2\right). \tag{33.17}$$

(a) Reformulate (33.17) as a standard least squares problem $\min_x \|Cx-d\|_2^2$, using appropriate choices for the matrix C and the vector d.
(b) Identify the conditions that guarantee a unique solution for (33.17).
(c) When the solution is unique, derive a formula for it.

Chapter 34
VII-FROM THE CLASSROOM TO REAL LIFE

34.1 Least squares data fitting

In Chapter 19, we saw how linear regression can model the relationship between two variables, y and t. But not all relationships are linear. Sometimes a straight line does not fit the data well. In those cases, we can use more flexible models that capture the trend more accurately.

34.1.1 Polynomial least squares regression

In this section, we will learn how to use least squares to find the best-fitting polynomial for a set of data points.

For example, consider the data points in the left panel of Figure 34.1. They do not follow a straight line. We generated them using the quadratic equation

$$y = 2t^2 + \text{random noise}, \tag{34.1}$$

where the "random noise" is a small variation we added to each y-value, causing the points to deviate slightly from the parabola $y = 2t^2$. A regression line clearly will not fit these points well.

Suppose we have data points $\{(t_i, y_i) \mid i = 1, \ldots, m\}$ and want to fit a polynomial of degree $d > 1$:

$$y = f(t) = c_0 + c_1 t + c_2 t^2 + \cdots + c_d t^d.$$

Just like with a regression line, we find the polynomial coefficients $c_0, c_1, \ldots, c_d$ by minimizing the sum of squared errors

$$\sum_{i=1}^{m} e_i^2 = \sum_{i=1}^{m} (y_i - f(t_i))^2,$$

which is the same as minimizing $\|b - Ax\|_2^2$, where

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_34

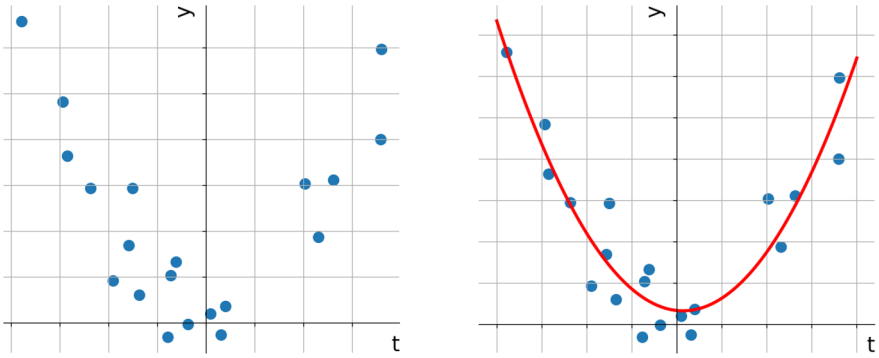


Fig. 34.1: Polynomial fitting. The left panel shows some data points generated based on the quadratic equation $y = 2x^2$. The right panel shows a fitted quadratic polynomial.

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & \cdots & t_1^d \\ 1 & t_2 & t_2^2 & \cdots & t_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_m & t_m^2 & \cdots & t_m^d \end{bmatrix}, \quad b = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \quad \text{and} \quad x = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_d \end{bmatrix}.$$

Fitting a polynomial by minimizing the sum of squared errors leads to a least squares problem. We can solve this using the QR factorization of A. In fact, if the t_i values are distinct and $d \leqslant m - 1$, the matrix A has full column rank (see Exercise 34.3). This allows us to find x using Theorem 32.12.

The right panel of Figure 34.1 shows the quadratic polynomial fitted to the original data points based on (34.1).

Overfitting

In polynomial regression, we fit a degree-d polynomial to data by minimizing the sum of squared errors. As the degree increases, the polynomial fits the data better, and the sum of squared errors goes down. But if we increase the degree too much, we can end up with **overfitting**, where the polynomial captures both the real pattern and the random noise in the data.

To see why, consider again the data generated by (34.1), shown in the left panel of Figure 34.1. A degree-2 polynomial fits this data well. But a degree-14 polynomial (right panel of Figure 34.2) fits the points almost perfectly, with an even smaller sum of squared errors: 10.5 for the quadratic vs. 4.7 for the degree-14 polynomial.

Does the smaller sum of squared errors mean the degree-14 polynomial is a better fit? Not necessarily. The degree-14 polynomial follows the data too closely, capturing both the trend and the noise. This can make it less reliable for predictions. For instance, at $t = 0.75$, the true value is $y = 2(0.75)^2 = 1.125$. The degree-2 polynomial gives $y \approx 1.079$, which is close. But the degree-14 polynomial predicts $y \approx 16.402$, far from the true value.

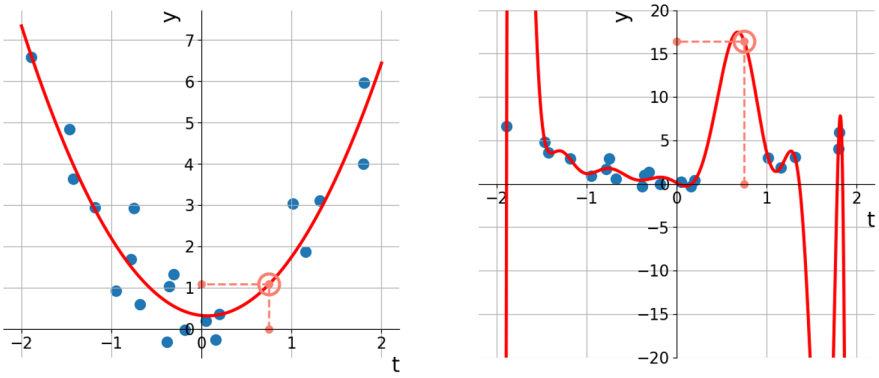


Fig. 34.2: Polynomial fitting: degree 2 (left) and degree 14 (right).

It is important to balance flexibility and simplicity when fitting a model. A model that is too simple may miss important patterns, while one that is too complex may overfit the data and perform poorly on new inputs. Although we will not discuss how to detect overfitting in this book, Chapter 33 introduces a few ways to help prevent it.

34.1.2 Generalized least squares regression

Not all relationships between variables can be captured well by linear or polynomial models. In these cases, we can use a more flexible approach with a function of the form

$$y = f(t) = c_0\phi_0(t) + c_1\phi_1(t) + \cdots + c_n\phi_n(t), \tag{34.2}$$

where the $\phi_i(t)$ functions are chosen based on the specific problem. These functions might come from prior knowledge, domain expertise, or even trial and error. The coefficients $c_0, \ldots, c_n$ are the model's coefficients that we need to determine.

To find the model's coefficients $c_0, \ldots, c_n$ that give the best fit for a set of data points $\{(t_i, y_i) \mid i = 1, \ldots, m\}$, we minimize the sum of squared errors

$$\sum_{i=1}^{m} (y_i - f(t_i))^2 = \sum_{i=1}^{m} (y_i - c_0\phi_0(t_i) - c_1\phi_1(t_i) - \cdots - c_n\phi_n(t_i))^2. \tag{34.3}$$

Since the function $f(t)$ in equation (34.2) is linear in the coefficients $c_0, \ldots, c_n$, we can write this problem in matrix form. The sum of squared errors becomes the 2-norm of the vector $b - Ax$, where

$$A = \begin{bmatrix} \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_n(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & \cdots & \phi_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_0(x_m) & \phi_1(x_m) & \cdots & \phi_n(x_m) \end{bmatrix}, \quad b = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \quad \text{and} \quad x = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

Minimizing the sum of squared errors (34.3) becomes a least squares problem. Solving this gives the best fit function $f(t)$ for the data.

A polynomial is a special case of (34.2), where $\phi_i(t) = t^i$. By choosing other types of functions for the ϕ_i, we can move beyond linear and polynomial regression to better capture different data patterns. Let us see a couple of examples.

Example 34.1. (Atmospheric CO_2 concentration)

Carbon dioxide (CO_2) is a greenhouse gas that plays a major role in climate change. Figure 34.3a shows the monthly average CO_2 concentration measured at Mauna Loa Observatory, Hawaii, from January 2015 to March 2023. The data reveals a steady increase, rising from 400 parts per million (ppm) in 2015 to 420 ppm in 2023. On top of this upward trend, there is a seasonal cycle, with concentrations fluctuating throughout the year.

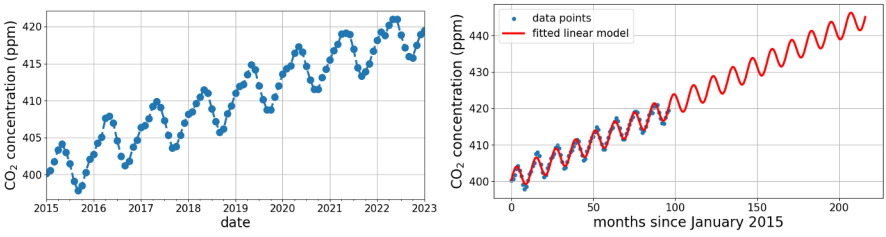
To model CO_2 concentration over time, we use the function

$$y = f(t) = c_0 + c_1 t + c_2 \sin\left(\frac{\pi}{6} t\right),$$

where y represents the CO_2 concentration measured at time t (in months since January 2015). The term $\pi/6$ makes the sine function repeat every 12 months, matching the yearly cycle.

This function is a specific case of (34.2), where $n = 2$, $\phi_0(t) = 1$, $\phi_1(t) = t$ and $\phi_2(t) = \sin\left(\frac{\pi}{6} t\right)$. We find the parameters c_0, c_1, and c_2 by solving a least squares problem, as explained earlier.

Figure 34.3b shows the fitted model, which we then use to make a 10-year projection. If the trend continues, CO_2 levels could rise to around 450 ppm in that time. Many scientists and policymakers consider this a critical threshold, as it increases the risk of severe and irreversible climate impacts.



(a) Monthly mean CO_2 concentration at Mauna Loa Observatory, Hawaii.

(b) Fitted CO_2 concentration model and 10-year projection.

Fig. 34.3: $C0_2$ concentration (2015–2023).

Example 34.2. (Life expectancy in the US)

In most developed countries, life expectancy has steadily increased since the early 20th century. This trend is evident in the average life expectancy of Americans from 1900 to 2010, shown in Table 34.1 and the left panel of Figure 34.4.

year	life expectancy	year	life expectancy
1900	47.3	1960	69.7
1910	50.6	1970	70.8
1920	54.1	1980	73.7
1930	61.6	1990	75.4
1940	64.8	2000	76.8
1950	68.2	2010	78.7

Table 34.1: Average life expectancy by decade in the US (1900–2010).

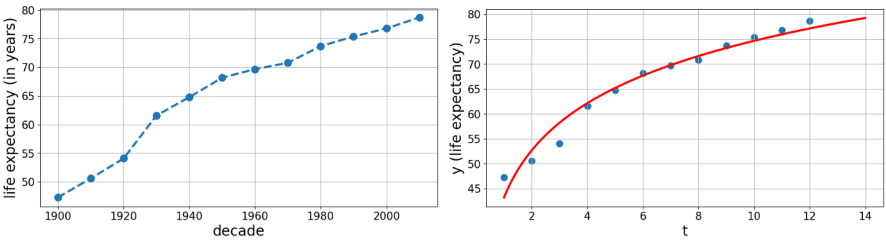
To model this data, we use a logarithmic growth function

$$y = f(t) = c_0 + c_1 \log(t),$$

where t represents time in decades, starting with $t = 1$ for the year 1900, and y represents life expectancy at time t.

This function is a specific case of (34.2), where $n = 1$, $\phi_0(t) = 1$ and $\phi_1(t) = \log(t)$. We estimate the parameters c_0 and c_1 using least squares.

The right panel of Figure 34.4 shows the fitted model, along with a 10-year projection. The model predicted a life expectancy of about 78.2 years for 2020 [1].



(a) Average life expectancy in the US (1900–2010).

(b) Fitted logarithmic growth model with a 2020 projection.

Fig. 34.4: Life expectancy data and model for the US (1900–2020).

[1] However, the actual estimate for 2020 was 77.28 years, about a year lower. This difference is likely due to the unexpected impact of the Covid-19 pandemic.

34.1.3 Multilinear least squares regression

In the last three sections, we learned how to fit lines, polynomials, and general functions (34.2) to data to model the relationship between two variables, t and y. But what if one variable is not enough to predict y? Sometimes, we need several variables, $t_1, t_2, \ldots, t_n$, to make better predictions. This is called **multilinear regression**.

In multilinear regression, we assume y can be expressed as a linear combination of the predictor variables $t_1, t_2, \ldots, t_n$. This means we model y as

$$y = f(t_1, t_2, \ldots, t_n) = c_0 + c_1 t_1 + c_2 t_2 + \cdots + c_n t_n. \tag{34.4}$$

This is known as a **linear model** because y is expressed as a linear combination of $1, t_1, \ldots, t_n$, with coefficients $c_0, c_1, \ldots, c_n$.

Suppose we have m data points. Each point includes a target variable y_i and predictor variables $t_{i1}, t_{i2}, \ldots, t_{in}$, where t_{ij} represents the value of the j-th predictor variable for the i-th data point ($i = 1, 2, \ldots, m$). To find the coefficients $c_0, c_1, \ldots, c_n$ in the linear model (34.4), we minimize the sum of squared errors

$$\sum_{i=1}^{m} e_i^2 = \sum_{i=1}^{m} (y_i - f(t_{i1}, t_{i2}, \ldots, t_{in}))^2.$$

In matrix form, this becomes a least squares problem $\min_x \|b - Ax\|_2^2$, where

$$A = \begin{bmatrix} 1 & t_{11} & t_{12} & \cdots & t_{1n} \\ 1 & t_{21} & t_{22} & \cdots & t_{2n} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & t_{m1} & t_{m2} & \cdots & t_{mn} \end{bmatrix}, \quad b = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \quad \text{and} \quad x = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

We can solve this least squares problem using the QR factorization of A, as we discussed in Chapter 32.

Example 34.3. (Predicting bicycle traffic)

Let us revisit the problem of predicting bicycle traffic across the Fremont bridge from Section 30.2.2. We will use a linear model to predict the daily number of bicycle trips. Table 34.2 lists the variables we included in our model.

Variable	Description
Weekday	1 for Monday to Friday, 0 for Saturday and Sunday
Holiday	1 if the day is a U.S. national holiday, 0 otherwise
Hours of Daylight	Duration of daylight (hours from sunrise to sunset)
Average Temperature	Average temperature (°F)
Precipitation	Total precipitation (inches)
Average Wind Speed	Average wind speed (mph)
Snowfall	Total snowfall (inches)

Table 34.2: Features used to predict daily bicycle trips.

We found the linear model coefficients by solving a least squares problem, as we explained earlier. Figure 34.5 compares the actual and predicted daily bicycle trips across the Fremont bridge, with a zoomed-in view of late November and early December.

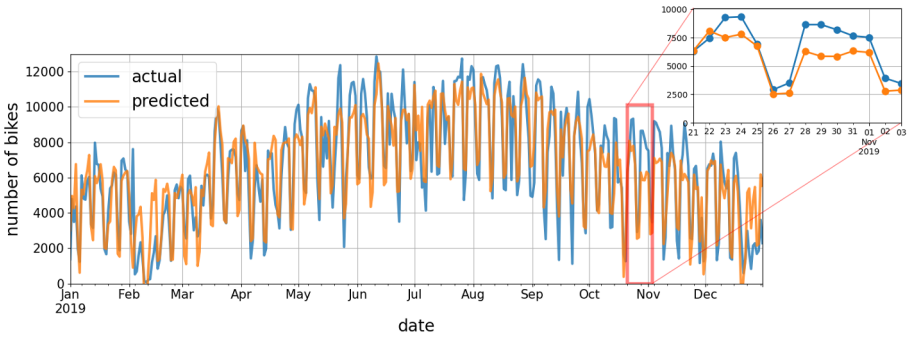


Fig. 34.5: Actual and predicted daily number of bicycle trips across the Seattle's Fremont bridge (from 2019-01-01 to 2019-12-31).

Although the model is not perfect, it still provides useful insights. By examining the coefficients, we can understand how each variable influences the daily bicycle count. Table 34.3 shows the coefficient values for each feature in the linear model.

Variable	**Coefficient**
Weekday	4239
Holiday	-2597
Hours of daylight	100
Average Temperature	161
Precipitation	-2651
Average wind speed	-76
Snowfall	-106

Table 34.3: Coefficients of the linear model for bicycle traffic prediction.

For example, the number of rides is much higher on weekdays than on weekends and holidays. Weekends see about 4,200 fewer trips and holidays roughly 2,600 fewer than an average workday. Longer daylight hours also encourage more rides, each additional hour brings around 100 more riders. Similarly, higher temperatures have a positive effect, with about 160 more riders for every one-degree Fahrenheit increase. Precipitation, however, has a significant negative impact, with each inch of rain reducing the count by about 2,600 riders. Snowfall and wind speed also decrease ridership, though their effects are more modest.

The predictions in Example 34.3 are not perfect, but they do capture the general traffic pattern. However, the model tends to slightly underestimate traffic. This might be because it misses important factors that influence cycling, or because it does not capture nonlinear effects. For instance, people might only avoid biking in the rain when it is also cold and windy.

One way to improve the predictions is by adding polynomial features. These are new features formed by combining existing ones in nonlinear ways. For example, adding temperature^2 can help model how temperature affects ridership in a nonlinear way. We can also include interaction terms like temperature $\times$ precipitation to capture how the impact of rain depends on the temperature.

Now we introduce the formal definition of polynomial features.

Adding polynomial features

When working with n predictor variables $t_1, t_2, \ldots, t_n$, adding degree-d polynomial features means creating new features by taking products of the original variables raised to powers, with the total degree up to d.

For example, if $d = 2$, you would include terms like t_1^2, t_2^2, and $t_1 t_2$. In general, a polynomial feature of degree d takes the form

$$t_1^{d_1} t_2^{d_2} \cdots t_n^{d_n},$$

where $d_1 + d_2 + \cdots + d_n = d$.

These additional features, called **polynomial features**, allow the model to capture more complex relationships between the variables that a simple linear model might miss.

Adding polynomial features can improve your model's predictions, but it comes at a cost, especially in multilinear regression. The number of features grows quickly with the degree and the number of original variables. This can make the model overly complex and lead to overfitting, where the model captures noise in the data instead of the underlying pattern.

A combinatorial explosion of polynomial features

The total number of polynomial features for n variables and degree d is given by

$$\binom{n+d-1}{d} = \frac{(n+d-1)!}{d!(n-1)!}. \tag{34.5}$$

This formula comes from counting the number of ways to distribute the degree d across n variables. Mathematically, this is equivalent to finding the number of nonnegative integer solutions to the equation

$$d_1 + d_2 + \cdots + d_n = d,$$

where d_i represents the exponent of t_i in a monomial. Table 34.4 lists the number of monomials (polynomial features) for different degrees d when $d = 0, 1, 2, 3$.

degree	polynomial variables	number of polynomial variables
0	1	1
1	$t_1, t_2, \ldots, t_n$	n
2	$t_1^2, t_1t_2, t_1t_3, \ldots, t_n^2$	$n(n+1)/2$
3	$t_1^3, t_1^2t_2, t_1^2t_3, \ldots, t_n^3$	$n(n+1)(n+2)/6$

Table 34.4: Number of polynomial features for degrees $d = 1, 2, 3$.

For large n or d, the number of polynomial features increases very quickly. To illustrate this, Figure 34.6 shows how the number of polynomial features grows with the degree d when there are $n = 20$ variables. For example, there are about $2 \cdot 10^7$ features when $d = 10$. This rapid growth can cause problems like overfitting in regression models, even if d is not very large.

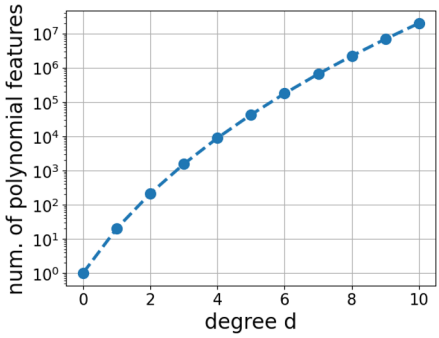


Fig. 34.6: A combinatorial explosion: number of polynomial features (34.5) as a function of the degree d for $n = 20$ variables.

34.2 Fitting an implicit equation

In the previous sections, we learned how to fit a function $y = f(t)$ to a set of data $\{(t_i, y_i)\}_{i=1}^m$. We saw how using least squares allows us to fit lines, polynomials, or more generally, functions of the form $f(t) = \sum_{j=1}^n c_j \phi_j(t)$. However, not all fitting problems fit[2] into this framework.

For instance, not every relationship between variables t and y can be directly expressed as $y = f(t)$. Sometimes, the relationship is defined implicitly by an equation $F(t, y) = c$, where c is a constant. In this section, we show how to fit an implicit equation to data when the function F is of the form

[2] We could not resist the pun.

$$F(t, y) = \sum_{j=1}^{n} c_j \, \phi_j(t, y), \tag{34.6}$$

for some constants c_j and functions ϕ_j.

The following example illustrates an important case of a function of the form (34.6).

Example 34.4. (Conic sections)

An important class of implicit equations $F(t, y) = c$ with a function F of the form (34.6) are quadratic equations. A quadratic equation is given by

$$F(t, y) = c_1 \, t^2 + c_2 \, y^2 + c_3 \, ty + c_4 \, t + c_5 \, y = 1, \tag{34.7}$$

where c_1, c_2, c_3, c_4, and c_5 are constants.

Equation (34.7) describes conic sections, this is lines, circles, ellipses, parabolas, and hyperbolas. Here, $n = 5$ and the ϕ_j functions are $\phi_1(t, y) = t^2$, $\phi_2(t, y) = y^2$, $\phi_3(t, y) = ty$, $\phi_4(t, y) = t$ and $\phi_5(t, y) = y$, with the constant $c = 1$.

Given a set of m data points $\{(t_i, y_i)\}_{i=1}^m$ and a curve described by the implicit equation $F(t, y) = c$, with F of the form (34.6), we can measure the "goodness of fit" between the data and the curve by calculating the **sum of squared algebraic distances**

$$\sum_{i=1}^{m} (F(t_i, y_i) - c)^2 = \sum_{i=1}^{m} \left(\sum_{j=1}^{n} c_j \, \phi_j(t_i, y_i) - c \right)^2, \tag{34.8}$$

where $F(t_i, y_i) - c$, known as the **algebraic distance**, measures how "far" each point (t_i, y_i) is from lying on the curve $F(t, y) = c$. This distance is zero only if the point lies exactly on the curve.

To fit a curve $F(t, y) = c$ to data points, we find the coefficients c_j that minimize the sum of squared algebraic distances (34.8). Given that the equation (34.6) is linear with respect to the coefficients c_j, we can express (34.8) as $\|b - Ax\|_2^2$, where

$$A = \begin{bmatrix} \phi_1(t_1, y_1) & \phi_2(t_1, y_1) & \cdots & \phi_n(t_1, y_1) \\ \phi_1(t_2, y_2) & \phi_2(t_2, y_2) & \cdots & \phi_n(t_2, y_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(t_m, y_m) & \phi_2(t_m, y_m) & \cdots & \phi_n(t_m, y_m) \end{bmatrix}, \quad b = \begin{bmatrix} c \\ c \\ \vdots \\ c \end{bmatrix} \quad \text{and} \quad x = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}.$$

This method allows us to use least squares to find the best-fitting implicit equation for the data. Let us work through an example.

Example 34.5. (Fitting an ellipse)

Fitting data points to an ellipse is a frequent task in image recognition. Given m data points $\{(t_i, y_i)\}_{i=1}^m$ and an ellipse described by the equation (34.7), we can fit

the ellipse by minimizing $\|b - Ax\|_2^2$, where

$$A = \begin{bmatrix} t_1^2 & y_1^2 & t_1 y_1 & t_1 & y_1 \\ t_2^2 & y_2^2 & t_2 y_2 & t_2 & y_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ t_m^2 & y_m^2 & t_m y_m & t_m & y_m \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \quad \text{and} \quad x = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \end{bmatrix}.$$

For example, suppose we have eight data points: $(1, 7)$, $(2, 6)$, $(5, 8)$, $(7, 7)$, $(9, 5)$, $(6, 7)$, $(3, 2)$, and $(8, 4)$. Using the least squares method, we fit an ellipse to these points. Figure 34.7 shows the data points and the fitted ellipse.

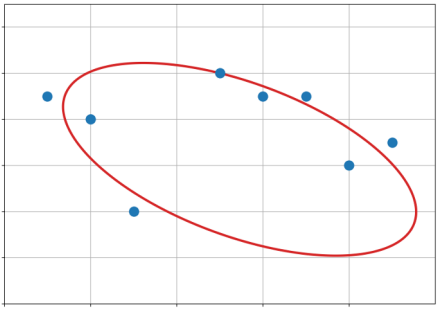


Fig. 34.7: Ellipse fitted to the data points using the least squares method.

34.3 Transforming data for least squares regression

In least squares regression, we fit models to data to understand the relationships between variables. This method requires that the coefficients appear in a linear way, as multiplier of known functions $\phi_j(t)$. If the coefficients enter the model nonlinearly, we cannot apply least squares directly. In some cases, though, we can transform the variables to rewrite the model in a linear form.

In this section, we show two examples where transforming the data lets us use least squares regression with nonlinear models.

34.3.1 Exponential model fitting

Exponential models are commonly used to describe growth or decay in fields like finance, biology, epidemiology, and physics. In this type of model, the variables t and y are related by

$$y = ae^{bt}, \tag{34.9}$$

where a and b are the model parameters.

Suppose we have data points $\{(t_i, y_i) \mid i = 1, \ldots, m\}$ that we want to fit to this exponential model. However, (34.9) depends nonlinearly on the parameter b, so

we cannot directly apply least squares. To work around this, we use a logarithmic transformation.

Taking the natural logarithm of both sides of (34.9), we get

$$\log(y) = \log(ae^{bt}) = \log(a) + bt. \tag{34.10}$$

This transformation makes the relationship between $\log(y)$ and t linear. We can now fit a linear model $\log(y) = c_0 + c_1 t$ to the transformed data $\{(t_i, \log(y_i)) \mid i = 1, \ldots, m\}$. Once we have the parameters c_0 and c_1, we recover the original parameters of the exponential model as $a = e^{c_0}$ and $b = c_1$.

Example 34.6. (Fitting an exponential model to data)

In this example, we generated 20 data points (t_i, y_i) based on the exponential function $y = e^{-t}$, shown in the left panel of Figure 34.8.

To fit an exponential model $y = ae^{bt}$, we applied the logarithmic transformation $f(x) = \log(x)$. The right panel of Figure 34.8 shows the transformed points $(t_i, \log(y_i))$. Notice that the transformed data now lie along a straight line.

We then fitted a linear model $\log(y) = c_0 + c_1 t$ to the transformed data. Using the linear model's parameters, we recovered the original exponential model's parameters, as explained earlier. The left panel shows the fitted exponential model.

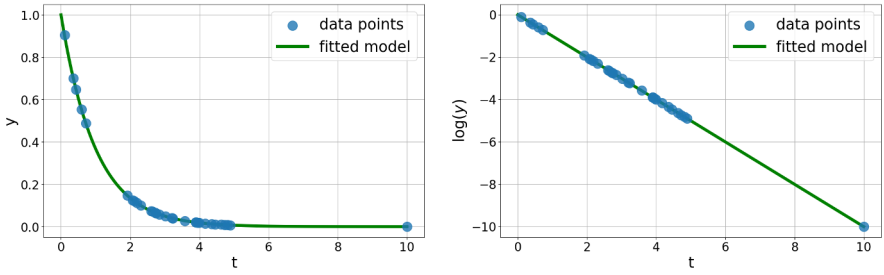


Fig. 34.8: Exponential model fitting. Left: Original data and fitted exponential model. Right: Transformed data and fitted linear model.

A good example of data well-suited for fitting an exponential model is the early days of a pandemic, as the initial spread of the virus often follows an exponential pattern.

Example 34.7. (Covid-19 deaths in Spain)

Spain was one of the first countries hit hard by the Covid-19 pandemic. If you were another country, you might have looked at Spain's early outbreak to help prepare for what was to come.

Table 34.5 shows the daily number of new Covid-19 deaths in Spain from March 3 to March 28, 2020.

Day	Deaths	Day	Deaths
3	1	13	78
4	1	14	62
5	1	15	94
6	2	16	53
7	5	17	191
8	7	18	90
9	11	19	207
10	7	20	213
11	19	21	332
12	1	22	397
23	539	26	718
24	497	27	773
25	839	28	884

Table 34.5: Covid-19 deaths in Spain from March 3 to March 28, 2020.

We fitted an exponential model to the data from March 3 to March 22 and plotted it along with the data points in Figure 34.9. To see how well the model works, we extended it six days into the future and compared the predictions with the actual reported deaths from March 23 to March 28, 2020.

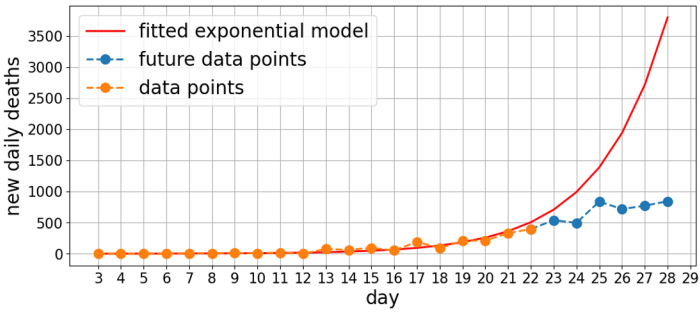


Fig. 34.9: Fitted exponential model for daily Covid-19 deaths in Spain.

The fitted exponential model works well for the first data points but, unfortunately, overestimates the number of deaths in the last six days, the ones we aimed to forecast. The predictions grow too fast.

Why does this happen? In Chapter 33, we saw that the model gives more weight to earlier data, which makes its predictions less reliable for later days. You can see this by how well the model fits the early data, and how the fit gets worse over time.

34.3.2 Logistic model fitting

The logistic model is commonly used in biology, medicine, ecology, economics, and the social sciences. It describes the relationship between t and y with the equation

$$y = f(t) = \frac{1}{1 + ae^{-bt}}, \tag{34.11}$$

where a and b are the model's parameters. Note that y depends on both parameters in a nonlinear way.

To fit a logistic model to a set of data points $\{(t_i, y_i) \mid i = 1, \ldots, m\}$, we use a data transformation, similar to what we did with exponential models. Here, we apply the logit function $f(x) = \log\left(\frac{x}{1-x}\right)$ to both sides of (34.11), giving

$$\log\left(\frac{y}{1-y}\right) = bt - \log(a).$$

This transformation creates a linear relationship between t and the transformed variable $\log\left(\frac{y}{1-y}\right)$. We can then fit a linear model

$$\log\left(\frac{y}{1-y}\right) = c_0 + c_1 t$$

to the transformed data

$$\left\{\left(t_i, \log\left(\frac{y_i}{1-y_i}\right)\right) \mid i = 1, \ldots, m\right\}.$$

Once we have the parameters c_0 and c_1 of the fitted linear model, we recover the logistic model's parameters using $a = e^{-c_0}$ and $b = c_1$.

Example 34.8. (Fitting a logistic model)

In this example, we generated 30 data points (t_i, y_i) using the logistic function $y = \frac{1}{1+e^{-t}}$, shown in the left panel of Figure 34.10. To fit a logistic model, we applied the logit transformation as described earlier. The right panel shows the transformed data points $\left(t_i, \log\left(\frac{y_i}{1-y_i}\right)\right)$.

We fitted a linear model to the transformed data. The right panel of Figure 34.10 shows the transformed points along with the fitted linear model $\log\left(\frac{y}{1-y}\right) = c_0 + c_1 t$. From the linear model's parameters, we recovered the original logistic model's parameters. The left panel displays the fitted logistic model.

Pandemics often begin with rapid exponential growth but eventually slow down as more people become infected or recover. This type of behavior, fast growth followed by a gradual leveling off, is well described by a logistic model.

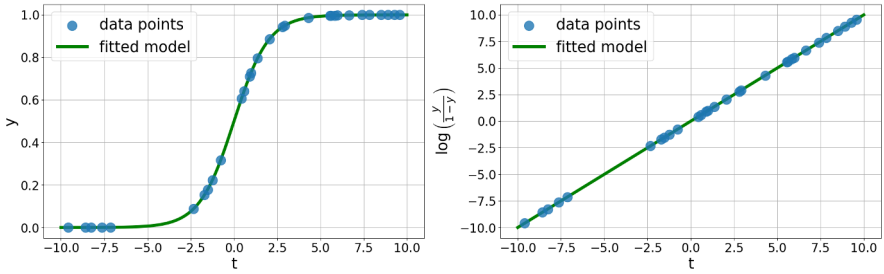


Fig. 34.10: Logistic model fitting. Left: Original logistic data and fitted logistic model. Right: Transformed data and fitted linear model.

Example 34.9. (An influenza outbreak)

An influenza outbreak occurred in a boarding school with 160 students. To contain the virus, the students were isolated from the outside world. During this period, the data in Table 34.6 was collected, showing the number of students who contracted the flu[3].

Day	New Cases	Cumulative Cases	Day	New Cases	Cumulative Cases
1	1	1	13	11	145
2	0	1	14	3	148
3	0	1	15	3	151
4	0	1	16	1	152
5	1	2	17	0	152
6	1	3	18	1	153
7	8	11	19	0	153
8	21	32	20	2	155
9	27	59	21	0	155
10	29	88	22	0	155
11	23	111	23	2	157
12	23	134	24	0	157

Table 34.6: Number of students contracting influenza during an outbreak.

Suppose it is day 9 of the outbreak. To predict the spread and identify the peak, we fitted a logistic model using the data from days 1 through 9. Figure 34.11 shows the fitted logistic model and the proportion of infected students over time.

The model fits the early days of the outbreak reasonably well. However, when we compare its predictions to the actual data from days 10 through 24, it underestimates the number of infected students, particularly during the middle stages of the outbreak. This is similar to what we saw with the exponential model for COVID-19 deaths. The issue arises because the fitting process gives more weight to certain data points. In Chapter 33, we learned why this happens and how weighted least squares can improve the fit.

[3] Do not worry, this data is entirely fictional. The authors promise that no students were injured or locked away in the name of mathematical modeling!

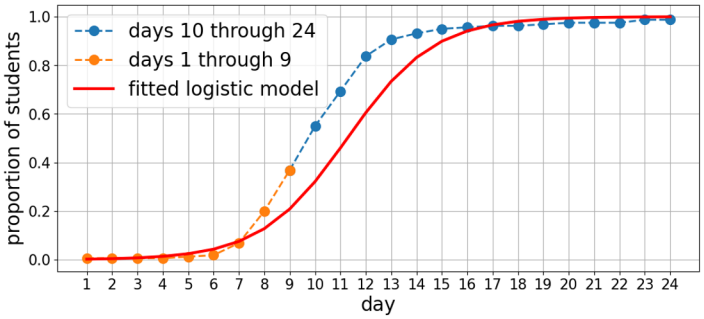


Fig. 34.11: Influenza outbreak: proportion of the 160 students that have been infected, and fitted logistic model.

Exponential and logistic models are examples of nonlinear models that become linear with the right data transformations. Power-law models $y = at^b$ can also be transformed in this way. You will see more examples in Problems 34.10–34.13.

However, not all models can be made linear through transformations. For those, we need to solve nonlinear least squares problems, using other methods. These topics, though important, are beyond the scope of this book.

34.4 Local least squares regression

In Section 14.2, we learned a simple way to estimate the y-value for a new input t, using a dataset $(t_i, y_i)_{i=1}^m$. The idea was to find the nearest neighbors of t and use their y-values to make a prediction. We used the average y-value of those neighbors. Now we will build on that idea and see how to improve it.

In Chapter 15, we saw that averaging is like fitting a constant (a degree-0 polynomial) to the neighbors. A natural next step is to fit something more flexible, like a line or a low-degree polynomial, to better capture the local pattern in the data.

To see how this works, consider the dataset in Figure 34.12. It was generated using the equation $y = \sin(t)$, with some added noise and a few outliers, points that do not follow the general trend.

After choosing the number of neighbors (k) to use, the **local least squares regression** method predicts a value at a specific point t_0 by following these three steps:

1. **Identify the k nearest neighbors**: Find the k data points $\{(t_i, y_i)\}$ whose t-values are closest to t_0. These neighbors will be used to make the prediction.
2. **Fit a regression line or polynomial**: Use the k nearest neighbors to fit a regression line $y = at + b$ or, if needed, a low-degree polynomial. The goal is to capture the local trend around t_0 based on the selected neighbors.
3. **Make the prediction**: Use the fitted regression line (or polynomial) to estimate the y-value at t_0. For a regression line, this gives the prediction $y_0 = at_0 + b$.

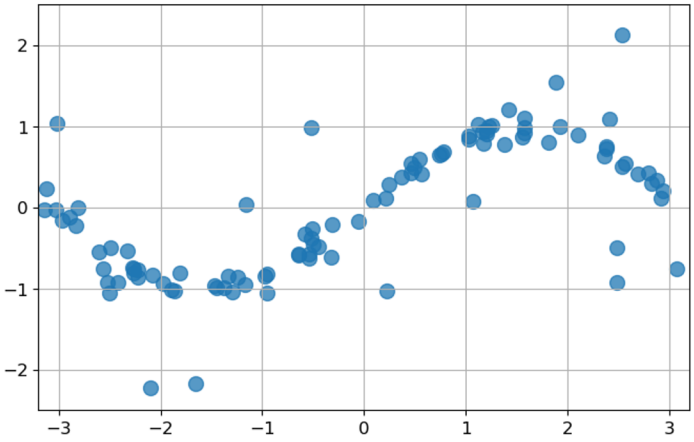


Fig. 34.12: Dataset based on $y = \sin(t)$ with added outliers.

Figure 34.13 shows how the local least squares algorithm works. It gives predictions for y at three points: $t_0 = -2$, 0, and 2, using the $k = 20$ nearest neighbors (highlighted by the red-shaded areas). For each t_0, we fit a regression line (in red) to those neighbors and use it to make a prediction. Each line is different because the set of neighbors changes with t_0.

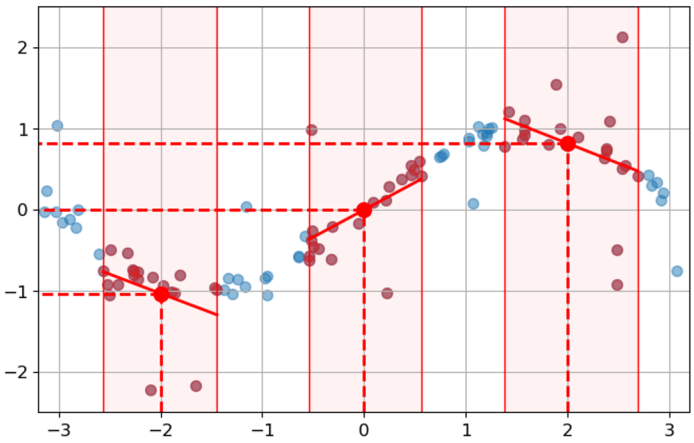


Fig. 34.13: Local least squares predictions at $t_0 = -2, 0, 2$ using 20 nearest neighbors, with regression lines and predictions.

In local least squares regression, the number of neighbors k is important because it controls the "flexibility" of the fitted function. Smaller k values make the function very flexible and local, which can lead to overfitting, where the model captures noise

instead of the overall pattern. Larger k values create a more rigid, global function that might underfit the data, failing to capture the true pattern.

Figure 34.14 illustrates this with fitted functions for $k = 3$ (top), $k = 20$ (middle), and $k = 100$ (bottom). The function with $k = 3$ clearly overfits the data, $k = 100$ underfits, and $k = 20$ finds a good balance, resulting in a well-fitted model.

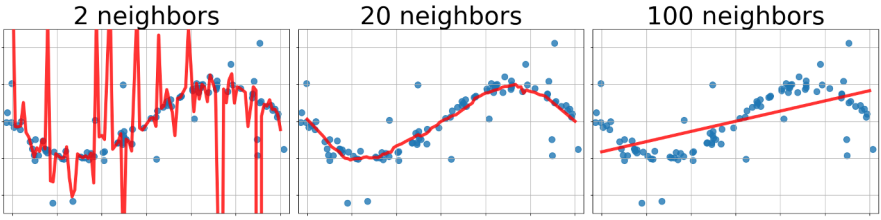


Fig. 34.14: Local least squares regression functions with $k = 2$ neighbors (top), $k = 20$ neighbors (middle), and $k = 100$ neighbors (bottom).

Problems

34.1 Find the parabola that best fits the following dataset in the least squares sense: $(-1, 1)$, $(1, -1)$, $(2, 0)$, $(3, 2)$.

34.2 Why does the sum of squared errors $\|b - Ax\|_2^2$ in polynomial regression always stay the same or get smaller when we increase the degree of the fitted polynomial?

34.3 Prove that the matrix

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & \cdots & t_1^d \\ 1 & t_2 & t_2^2 & \cdots & t_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_m & t_m^2 & \cdots & t_m^d \end{bmatrix}$$

has full column rank if the t_i values are distinct and $d \leqslant m - 1$.

Hint: Think of $Ax = 0$ as evaluating a polynomial $p(t) = x_0 + x_1 t + x_2 t^2 + \cdots + x_d t^d$ at the points $t_1, t_2, \ldots, t_m$. Use the fact that a nonzero polynomial of degree d can have at most d distinct roots.

34.4 Consider a polynomial regression problem in which the fitted polynomial includes a constant term. Let

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 & \cdots & t_1^d \\ 1 & t_2 & t_2^2 & \cdots & t_2^d \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & t_m & t_m^2 & \cdots & t_m^d \end{bmatrix}, \qquad b \in \mathbb{R}^m,$$

and let x be a solution of

$$\min_z \|b - Az\|_2^2.$$

Prove that the entries of the residual vector $r = b - Ax$ sum to zero.

34.5 Find the ellipse that best fits the following data points: $(-1, -1)$, $(2, 1)$, $(-1, -2)$, $(1, -1)$, $(-3, 1)$ and $(0, 2)$.

34.6 Consider the implicit equation

$$(t - t_0)^2 + (y - y_0)^2 = r^2,$$

which describes a circle in $\mathbb{R}^2$ of radius r with center at (t_0, y_0). Show how to fit this circle to a set of data points $\{(t_i, y_i)\}_{i=1}^m$ by solving a least squares problem.

34.7 Consider the ellipse fitting method described in Section 34.2. One drawback of this approach is that Equation (34.7) does not always define an ellipse. It can also represent other conic sections, such as parabolas or hyperbolas. To see this, replace the point at $(5, 8)$ with $(5, 10)$. Check if the minimized sum of squared algebraic distances yields an ellipse or another conic section like a hyperbola, as Figure 34.15 shows.

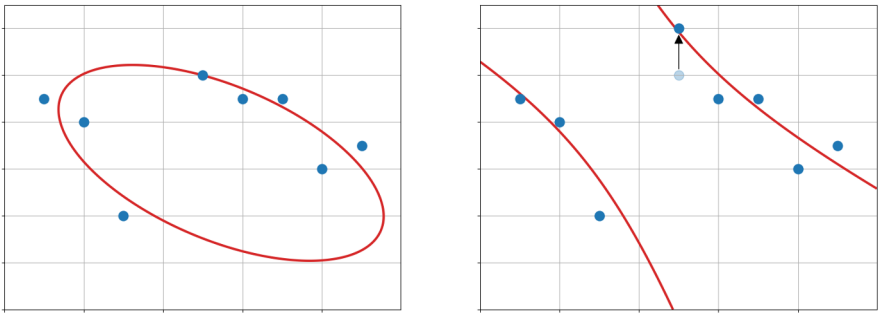


Fig. 34.15: The best-fitting conic section in a least squares sense may be a hyperbola.

34.8 Generate data points in Python using the following code

```
theta = 2*np.pi*np.random.rand(50)
t = (np.sqrt(2) * np.cos(theta)) / (1 + np.sin(theta)**2)
y = (np.sqrt(2) * np.cos(theta) * np.sin(theta)) / (1 + np.sin(theta)**2)
y +=  0.05*np.random.randn(50)
```

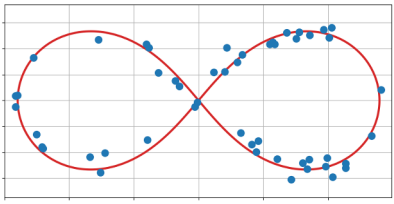
Your task is to fit these data points (t, y) to a Cassini oval $(t^2 + y^2)^2 = at^2 + by^2$. Figure 34.16a illustrates the data points (in blue) and the oval that best fits them (in red).

34.9 Generate data points in Python using the following code

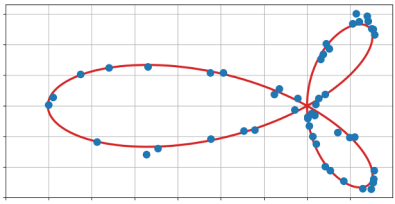
```
theta = 2*np.pi*np.random.rand(50)
r = -3*np.cos(theta)+8*np.cos(theta)*np.sin(theta)**2
```

```
t = r*np.cos(theta)
y = r*np.sin(theta)
y += 0.07*np.random.randn(50)
```

Your task is to fit these data points (t, y) to a trifolium, a three-leaf shaped curve, $(t^2 + y^2)(y^2 + t(t + b)) = 4aty^2$. Figure 34.16b shows the data points (in blue) and the trifolium that best fits them (in red).



(a) Data points and their best fit using a Cassini oval.



(b) Data points and their best fit using a trifolium curve.

Fig. 34.16: Fitted curves and data points for Problems 34.8 and 34.9.

34.10 The **power-law model** is commonly used to describe various natural phenomena, such as the distribution of city sizes, the frequency of earthquakes, and the distribution of word frequencies in a language. The model takes the form

$$y = at^b,$$

where a and b are the parameters of the model.

(a) What data transformation would convert the power-law model into a linear model?
(b) Consider Table 34.7, which contains data on the number of pageviews for websites and their rankings. The rank column indicates the relative popularity of each website, where the website with the highest number of pageviews is ranked 1, the next highest ranked 2, and so on. Fit a power-law model to this data.

34.11 The **Michaelis-Menten** model is a fundamental mathematical model used to describe enzyme-catalyzed reactions. It expresses the reaction rate y as a function of the substrate concentration t using the equation

$$y = \frac{at}{b + t},$$

where a and b are the parameters of the model.

(a) What data transformation would convert the Michaelis-Menten model into a linear model?
(b) Table 34.8 contains data on the reaction rate of an enzyme at different substrate concentrations. Fit a Michaelis-Menten model to this data.

Rank	Pageviews
1	100000
2	50000
3	33333
4	25000
5	20000
6	16667
7	14286
8	12500
9	11111
10	10000

Table 34.7: Pageview data and rankings for websites, with the most viewed website ranked as 1, followed by decreasing order of pageviews.

Concentration	Reaction rate
0.1	0.01
0.2	0.02
0.3	0.04
0.4	0.06
0.5	0.07
0.6	0.08
0.7	0.08
0.8	0.09

Table 34.8: Reaction rates of an enzyme at various substrate concentrations.

34.12 The **Hill model**, developed by Archibald Hill in 1910, describes the cooperative binding of ligands to a receptor or enzyme, such as the binding of oxygen to hemoglobin. In this model, y represents the fraction of occupied receptor or enzyme sites, and t represents the ligand concentration. The model has the form

$$y = \frac{t^n}{k + t^n},$$

where n and k are the parameters of the model.

(a) What data transformation would convert the Hill model into a linear model?
(b) Table 34.9 contains data on the binding of a ligand to a receptor at various concentrations of the ligand. Fit a Hill model to this data.

Concentration of ligand	Fraction of occupied receptor sites
0.1	0.05
0.5	0.2
1	0.4
2	0.7
5	0.9
10	0.95

Table 34.9: Ligand concentrations and corresponding fractions of occupied receptor sites.

34.13 The **Gaussian normal distribution** is given by

$$y = \frac{A}{\sigma\sqrt{2\pi}} e^{-(t-\mu)^2/(2\sigma^2)}, \tag{34.12}$$

where A, σ and μ are the distribution's parameters. Consider the task of fitting a Gaussian normal distribution to a set of data points $\{(t_i, y_i)\}_{i=1}^m$.

(a) What data transformation would convert (34.12) into a linear model?
(b) Describe how to extract the parameters A, σ and μ from the linearized model's parameters.

Part VIII
EIGENVALUES, EIGENVECTORS, AND ALGORITHMS

In this part, we review three core concepts from Linear Algebra: eigenvalues, eigenvectors, and invariant subspaces. We revisit their definitions and properties, and discuss why they are important. The main focus, however, is on practical algorithms for computing eigenvalues and eigenvectors, such as the Power Method, Rayleigh Quotient Iteration, and the QR Algorithm. Later chapters will show how these ideas are central to many Machine Learning applications.

Chapter 35
VIII-WHY SHOULD WE CARE?

35.1 Three applications of eigenvalues and eigenvectors in Machine Learning

Eigenvalues and eigenvectors play an important role in Machine Learning, as we will see in this and future chapters. We start with three important examples.

35.1.1 Matrix powers

Matrix powers appear in many applications. A key example is in studying Markov Chains, which we will cover in Chapter 52. Markov Chains play an important role in many Machine Learning applications, such as predicting the next word in a sequence for natural language processing or ranking web pages with algorithms like Google's PageRank (see Chapter 54).

In these applications, we often study how the sequence of vectors evolves:

$$x, Ax, A^2x, \ldots, A^kx.$$

Some of the questions we will explore include:

- Does A^kx converge as k increases? Does it depend on the choice of x?
- If it diverges, how quickly do the entries of A^kx grow?

These questions are connected to the eigenvalues and eigenvectors of the matrix A. In Chapter 37, we will see how diagonalizing A makes it easier to compute matrix powers. This will also help us understand how A^k and A^kx behave as k increases.

35.1.2 Matrix nearness problems

Matrix nearness problems ask us to find a matrix B with a specific property that is as close as possible to a given matrix A. Formally, we solve:

$$\min \{\|A - B\| \mid B \text{ satisfies property } \mathcal{P}\},$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_35

where $\mathcal{P}$ defines the desired property of B. Common examples include sparsity (certain zero patterns), low rank, orthogonality ($B^TB = I_n$), and rotationality ($B^TB = I_n$ and $\det(B) = 1$).

The norm $\|A - B\|$ measures how far A is from B. The Frobenius norm and 2-norm are the most common choices because they often lead to closed-form solutions. In Chapter 49, we will see examples of these problems and how they are used in Machine Learning and Statistics.

35.1.3 Optimization problems

Optimization plays a big role in Machine Learning. Many problems involve finding values that minimize or maximize a function. In Chapter 32, we saw one of the most common examples: the least squares problem. There, we solved

$$\min_{x \in \mathbb{R}^n} \|b - Ax\|_2^2$$

using the QR factorization. But optimization does not stop at least squares. Some important problems lead to eigenvalue problems instead.

For example, let A and B be $n \times n$ matrices. In some problems, we need to solve

$$\begin{aligned} &\text{optimize} \quad x^TAx \\ &\text{such that} \quad x^TBx = 1, \end{aligned}$$

where "optimize" means either minimize or maximize x^TAx, depending on the context. A more general version asks for an $n \times p$ matrix $X = \begin{bmatrix} x_1 & \cdots & x_p \end{bmatrix}$ that solves

$$\begin{aligned} &\text{optimize} \quad x_1^TAx_1 + x_2^TAx_2 + \cdots + x_p^TAx_p \\ &\text{such that} \quad X^TBX = I_p. \end{aligned}$$

We will see how eigenvalues and eigenvectors help solve problems like these in Chapters 42 and 47, including applications in Machine Learning.

These optimization problems appear in many practical settings. To give you a sense of how they are used, the next section focuses on one example.

35.2 Corner detection in computer vision

Figure 35.1 shows two photos of the same chess set, taken from slightly different angles and with different lighting. It is obvious to us that both images show the same object. But how can we teach a computer to recognize that?

Another example appears in Figure 35.2, which shows three images of the same object taken from different angles[1]. A human can easily tell it is the same object, but how can we teach a computer to recognize that different views represent the same thing?

A common approach in Computer Vision is to find features that stay consistent when an image changes in position, orientation, or lighting. These features help match parts of different images based on how they look locally. This is especially

[1] Captured by one of the authors. Let us just say photography is not their strong suit.

Fig. 35.1: Two photos of the same chess set.

Fig. 35.2: Three pictures of the same object taken from different perspectives.

helpful when objects appear different because of their position. The goal is to find distinct points, called **interest points**, that stand out in their surroundings. Corners are an important example. They are easy to recognize and useful for matching objects across images.

Defining what a "corner" is in a way that a computer can detect is a tricky problem. A popular method turns it into an optimization problem, like the ones in Section 35.1.3. We will go into the details in Chapter 39.

Chapter 36
VIII-WHAT YOU MAY HAVE LEARNED BEFORE

This book focuses mainly on real matrices, but eigenvalue problems are more naturally formulated in the complex setting. Even real matrices can have complex eigenvalues and eigenvectors. So, unless stated otherwise, we will now assume that matrices, vectors, and scalars are complex, and we will use the complex versions of some familiar operations. For example:

1. For a complex number $z = a + bi$, the analog of absolute value is its **magnitude**, defined as $|z| = \sqrt{a^2 + b^2}$.
2. The transpose of a matrix A is replaced by the **conjugate transpose**, denoted A^*, where
$$A^*(i, j) = \overline{A(j, i)}.$$
That is, we transpose A and then take the complex conjugate of each entry, with $\overline{a + bi} = a - bi$. The order does not matter as transposing and conjugating commute. In this setting, orthogonal matrices become unitary matrices, which satisfy $U^*U = UU^* = I_n$.
3. The dot product becomes the **complex dot product**, u^*v. Two vectors $u, v \in \mathbb{C}^n$ are orthogonal if $u^*v = 0$.

Vector and matrix norms extend naturally to the complex setting by using magnitudes instead of absolute values. For example, the 2-norm of a complex vector v is $\|v\|_2 = \sqrt{v^*v}$.

We will also use the complex versions of linear combinations, span, and subspaces throughout this part. To keep the notation simple, we will not distinguish between real and complex subspaces. The context will make it clear.

36.1 Eigenvalues and eigenvectors

Square matrices represent linear transformations that map vectors to other vectors in the same space. In this chapter, we focus on vectors that are mapped to scalar multiples of themselves. They are called eigenvectors, and the scalars that scale them are the eigenvalues.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_36

Definition 36.1. (Eigenvalues and eigenvectors)

Let $A \in \mathbb{C}^{n\times n}$. We say that a nonzero vector $x \in \mathbb{C}^n$ is an **eigenvector** of A if there exists a (possibly non real) scalar λ such that $Ax = \lambda x$. The complex number λ is called an **eigenvalue** of A, and we say that x is an eigenvector of A with associated eigenvalue λ.

If λ is an eigenvalue of A with corresponding eigenvector x, we call the pair (λ, x) an **eigenpair** of A.

Example 36.2 Every non-zero vector in $\mathbb{C}^n$ is an eigenvector of the identity matrix I_n with eigenvalue 1. This is because multiplying any non-zero vector x by I_n returns the same vector, i.e., $I_n x = x$.

Example 36.3 The vectors $x_1 = \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$ and $x_2 = \left[\begin{smallmatrix} 1 \\ -1 \end{smallmatrix}\right]$ are eigenvectors of the matrix $\left[\begin{smallmatrix} 1 & 2 \\ 2 & 1 \end{smallmatrix}\right]$ with eigenvalues $\lambda_1 = 3$ and $\lambda_2 = -1$, respectively, since they are both nonzero vectors and we have

$$Ax_1 = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \lambda_1 x_1, \quad \text{and}$$
$$Ax_2 = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = -1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \lambda_2 x_2.$$

Example 36.4 Consider the (real) matrix

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

This matrix has no real eigenvalues or eigenvectors in $\mathbb{R}^2$ (see Problem 36.2). However, in the complex setting, we find

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ i \end{bmatrix} = \begin{bmatrix} -i \\ 1 \end{bmatrix} = \begin{bmatrix} -i \\ -i^2 \end{bmatrix} = -i \begin{bmatrix} 1 \\ i \end{bmatrix} \quad \text{and}$$
$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -i \end{bmatrix} = \begin{bmatrix} i \\ 1 \end{bmatrix} = \begin{bmatrix} i \\ -i^2 \end{bmatrix} = i \begin{bmatrix} 1 \\ -i \end{bmatrix}.$$

In other words, the (complex) vectors $x_1 = \left[\begin{smallmatrix} 1 \\ i \end{smallmatrix}\right]$ and $x_2 = \left[\begin{smallmatrix} 1 \\ -i \end{smallmatrix}\right]$ are eigenvectors of A with (complex) eigenvalues $\lambda_1 = -i$ and $\lambda_2 = i$, respectively.

When working with real matrices, we often focus on eigenvalues that are real numbers. Eigenvectors corresponding to real eigenvalues are only stretched or compressed. In other words, the result lies along the same line as the original vector, as shown in Figure 36.1. These eigenvectors point to directions where the matrix A stretches or compresses space.

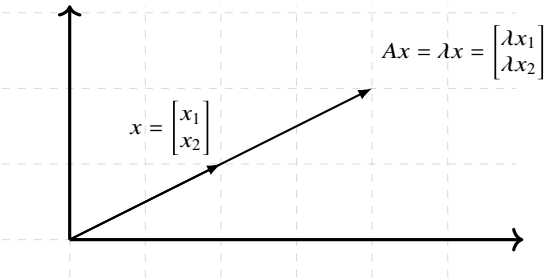


Fig. 36.1: Eigenvectors of a real matrix with a real eigenvalue maintain their direction under transformation by A.

A natural question to ask is: How many eigenvalues does a matrix have? To answer this, observe that a complex number λ is an eigenvalue of a matrix A if there exists a nonzero vector x such that $(\lambda I_n - A)x = 0$. This happens when the matrix $\lambda I_n - A$ is singular, which occurs when its determinant is zero. This leads to the definition of the characteristic polynomial and the characteristic equation of A.

Definition 36.5. (The characteristic polynomial and the characteristic equation of a matrix)

Given an $n \times n$ matrix A, the function $p_A(\lambda) := \det(\lambda I_n - A)$ is a degree-n polynomial in the variable λ known as the **characteristic polynomial** of A. The equation $p_A(\lambda) = 0$ is referred to as the **characteristic equation** of A.

The roots of the characteristic polynomial $p_A(\lambda)$ are precisely the eigenvalues of A. In other words, a complex number λ_0 is an eigenvalue of A if and only if $p_A(\lambda_0) = 0$. If λ_0 is an eigenvalue, then the characteristic polynomial can be factored as

$$p_A(\lambda) = (\lambda - \lambda_0)^k q(\lambda),$$

where $k \geqslant 1$ and $q(\lambda)$ is a polynomial such that $q(\lambda_0) \neq 0$. The exponent k leads to the following definition.

Definition 36.6. (Algebraic multiplicity of an eigenvalue)

Let λ be an eigenvalue of a matrix A. The **algebraic multiplicity** of λ, denoted by $\text{alg}_A(\lambda)$, is the number of times the eigenvalue λ appears as a root of the characteristic polynomial of A.

Example 36.7 The eigenvalues of the matrix A in Example 36.3 have both algebraic multiplicity 1. On the other hand, the matrix $\left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ has only one eigenvalue, $\lambda = 0$, with algebraic multiplicity 2.

Since the characteristic polynomial of an $n \times n$ matrix has degree n, we get the following result.

Theorem 36.8. (Sum of algebraic multiplicities)

Let $A \in \mathbb{C}^{n\times n}$. The sum of the algebraic multiplicities of the eigenvalues of A is n. As a result, a complex $n \times n$ matrix has at least one eigenvalue and at most n distinct eigenvalues.

Proof The characteristic polynomial of A, $p_A(\lambda) = \det(\lambda I_n - A)$, has degree n (see Problem 36.7). The Fundamental Theorem of Algebra tells us that a polynomial of degree n has exactly n roots, counting multiplicities. For $p_A(\lambda)$, these roots are the eigenvalues of A, and their multiplicities are the algebraic multiplicities. Therefore, the sum of the algebraic multiplicities of the eigenvalues of A is n the degree of $p_A(\lambda)$. □

For small matrices, we can find eigenvalues by solving the characteristic equation $p_A(\lambda) = 0$ by hand.

Example 36.9 Consider the 2×2 matrices

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}, \quad \text{and} \quad C = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

To find the eigenvalues of A, B and C, we compute their characteristic polynomials. We have

$$\det(\lambda I_2 - A) = \det \begin{bmatrix} \lambda - 1 & -2 \\ -2 & \lambda - 1 \end{bmatrix} = \lambda^2 - 2\lambda - 3 = (\lambda - 3)(\lambda + 1),$$

$$\det(\lambda I_2 - B) = \det \begin{bmatrix} \lambda - 2 & -1 \\ 0 & \lambda - 2 \end{bmatrix} = (\lambda - 2)^2, \quad \text{and}$$

$$\det(\lambda I_2 - C) = \det \begin{bmatrix} \lambda & 1 \\ -1 & \lambda \end{bmatrix} = \lambda^2 + 1.$$

The eigenvalues of A are $\lambda_1 = 3$ and $\lambda_2 = -1$, which are distinct and real. The only eigenvalue of B is $\lambda = 2$, which has algebraic multiplicity 2. The eigenvalues of C are $\lambda_1 = i$ and $\lambda_2 = -i$.

Solving $p_A(\lambda) = 0$ works fine for the small matrices we often use in class examples, but it becomes impractical for larger ones. For bigger matrices, we use better methods, which we will cover in Chapters 37 and 38.

As we saw in Example 36.9, real matrices can have complex eigenvalues with nonzero imaginary parts. When this happens, the complex eigenvalues always come in conjugate pairs.

Proposition 36.10. (The complex eigenvalues of a real matrix)

If $\lambda_0 = a + ib$ is a complex eigenvalue of a matrix $A \in \mathbb{R}^{n\times n}$, then so is its complex conjugate $\overline{\lambda_0} = a - ib$.

Definition 36.11. (The spectrum of a matrix)

The **spectrum** of a matrix A is the set of all its distinct eigenvalues, denoted by $\sigma(A)$. Formally, the spectrum is defined as

$$\begin{aligned}\sigma(A) =&\{\lambda \in \mathbb{C} \mid (\lambda I_n - A)x = 0 \text{ has a nonzero solution } x \in \mathbb{C}^n\}\\ =&\{\lambda \in \mathbb{C} \mid \det(\lambda I_n - A) = 0\}.\end{aligned}$$

Example 36.12 For the matrices A, B and C in Example 36.9, we have

$$\begin{aligned}\sigma(A) &= \{3, -1\},\\ \sigma(B) &= \{2\}, \quad \text{and}\\ \sigma(C) &= \{i, -i\}.\end{aligned}$$

If x is an eigenvector of a matrix A with eigenvalue λ, then any nonzero scalar multiple cx is also an eigenvector with the same eigenvalue. In fact, any linear combination of eigenvectors associated with λ is again an eigenvector with eigenvalue λ. So, each eigenvalue has infinitely many eigenvectors. This idea leads to the following definition.

Definition 36.13. (Eigenspace associated with an eigenvalue)

The **eigenspace** of a matrix A associated with an eigenvalue λ is the set of all eigenvectors of A corresponding to λ, along with the zero vector. Formally, the eigenspace associated with λ is defined as

$$\text{eig}_A(\lambda) := \{x \in \mathbb{C}^n \mid Ax = \lambda x\}.$$

Any nonzero vector in $\text{eig}_A(\lambda)$ is an eigenvector of A with eigenvalue λ. The zero vector is also in $\text{eig}_A(\lambda)$. In fact, eigenspaces are always subspaces of $\mathbb{C}^n$.

Theorem 36.14. (Eigenspace as a nullspace)

Let $A \in \mathbb{C}^{n\times n}$ and let λ be an eigenvalue of A. The eigenspace of A associated with λ is a subspace of $\mathbb{C}^n$ and can be expressed as the nullspace of $\lambda I_n - A$ over the complex numbers:

$$\text{eig}_A(\lambda) = \text{null}(\lambda I_n - A) = \{x \in \mathbb{C}^n \mid (\lambda I_n - A)x = 0\}.$$

To find the eigenspace associated with an eigenvalue λ of a matrix $A \in \mathbb{C}^{n\times n}$ "by hand", we solve the homogeneous system $(\lambda I_n - A)x = 0$.

Example 36.15 Let us find the eigenspaces of the matrices A, B and C in Example 36.9. The eigenspaces of the matrix A are

$$\begin{aligned}\text{eig}_A(3) &= \{x \in \mathbb{C}^n \mid (3I_2 - A)x = 0\} \quad \text{and}\\ \text{eig}_A(-1) &= \{x \in \mathbb{C}^n \mid (-I_2 - A)x = 0\}.\end{aligned}$$

To compute these eigenspaces, we need to solve the following systems of equations

$$\begin{aligned}(3I_2 - A)x = 0 \quad &\Leftrightarrow \quad \begin{bmatrix}2 & -2\\ 2 & -2\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix} = \begin{bmatrix}0\\ 0\end{bmatrix}, \quad \text{and}\\ (-I_2 - A)x = 0 \quad &\Leftrightarrow \quad \begin{bmatrix}-2 & -2\\ -2 & -2\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix} = \begin{bmatrix}0\\ 0\end{bmatrix}.\end{aligned}$$

The first system can be reduced to $x_1 - x_2 = 0$, and thus we have

$$\text{eig}_A(3) = \left\{\begin{bmatrix}x_1\\ x_1\end{bmatrix} \mid x_1 \in \mathbb{C}\right\} = \text{span}\left(\begin{bmatrix}1\\ 1\end{bmatrix}\right).$$

The second system can be reduced to $x_1 + x_2 = 0$, and thus we have

$$\text{eig}_A(-1) = \left\{\begin{bmatrix}x_1\\ -x_1\end{bmatrix} \mid x_1 \in \mathbb{C}\right\} = \text{span}\left(\begin{bmatrix}1\\ -1\end{bmatrix}\right).$$

For the matrix B, its only eigenspace is

$$\text{eig}_B(2) = \left\{\begin{bmatrix}x_1\\ 0\end{bmatrix} \mid x_1 \in \mathbb{C}\right\} = \text{span}\left(\begin{bmatrix}1\\ 0\end{bmatrix}\right).$$

Finally, the matrix C has eigenvalues $\lambda_1 = i$ and $\lambda_2 = -i$. Using similar calculations to those used for A, we have

$$\operatorname{eig}_C(i) = \left\{ \begin{bmatrix} x_1 \\ ix_1 \end{bmatrix} \mid x_1 \in \mathbb{C} \right\} = \operatorname{span}\left(\begin{bmatrix} 1 \\ i \end{bmatrix} \right), \quad \text{and}$$
$$\operatorname{eig}_C(-i) = \left\{ \begin{bmatrix} x_1 \\ -ix_1 \end{bmatrix} \mid x_1 \in \mathbb{C} \right\} = \operatorname{span}\left(\begin{bmatrix} 1 \\ -i \end{bmatrix} \right).$$

A matrix A has infinitely many eigenvectors for each eigenvalue, since multiplying any eigenvector by a nonzero scalar gives another eigenvector. However, the number of linearly independent eigenvectors associated with an eigenvalue λ is equal to the dimension of the eigenspace $\operatorname{eig}_A(\lambda)$, which is at most n for an $n \times n$ matrix. This motivates the following definition.

Definition 36.16. (Geometric multiplicity of an eigenvalue)

The **geometric multiplicity** of an eigenvalue λ of a matrix A, denoted by $\operatorname{geom}_A(\lambda)$, is the dimension of the eigenspace $\operatorname{eig}_A(\lambda)$. In other words, it is the maximum number of linearly independent eigenvectors of A corresponding to λ.

Example 36.17 Consider the matrix A from Example 36.9. As shown in Example 36.15, every eigenvector with eigenvalue $\lambda = -1$ is a multiple of $\left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix} \right]$. This means the eigenspace $\operatorname{eig}_A(-1)$ is one-dimensional, so there is only one linearly independent eigenvector for $\lambda = -1$. As a result, $\lambda = -1$ has both algebraic and geometric multiplicity equal to 1.

Example 36.18 Now consider the eigenvalue $\lambda = 2$ of the matrix B from Example 36.9. While $\lambda = 2$ has algebraic multiplicity 2, the eigenspace $\operatorname{eig}_B(2)$ is one-dimensional, so its geometric multiplicity is 1.

The difference between an eigenvalue's algebraic and geometric multiplicities will play an important role later. For now we prove that an eigenvalue's geometric multiplicity never exceeds its algebraic multiplicity.

Theorem 36.19. (Multiplicity inequality)

For any eigenvalue λ of an $n \times n$ matrix A, we have

$$1 \leqslant \operatorname{geom}_A(\lambda) \leqslant \operatorname{alg}_A(\lambda).$$

Proof Let $\lambda_i \in \sigma(A)$, and let m be its geometric multiplicity. This means there are m linearly independent eigenvectors $x_1, x_2, \ldots, x_m$ of A associated with λ_i.

Now extend $\{x_1, x_2, \ldots, x_m\}$ to a basis $\{x_1, x_2, \ldots, x_m, x_{m+1}, \ldots, x_n\}$ of $\mathbb{C}^n$, and form the matrix

$$S = \begin{bmatrix} x_1 & x_2 & \cdots & x_m & x_{m+1} & \cdots & x_n \end{bmatrix}.$$

Since the columns of S form a basis for $\mathbb{C}^n$, S is invertible. Let us partition S as

$$S = \begin{bmatrix} S_1 & S_2 \end{bmatrix},$$

where S_1 is the $n \times m$ submatrix consisting of the eigenvectors $x_1, x_2, \ldots, x_m$, and S_2 is the $n \times (n-m)$ submatrix consisting of the remaining basis vectors. Define $B := S^{-1}AS$, and partition B as

$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix},$$

where B_{11} is an $m \times m$ block. Since $Ax_j = \lambda_i x_j$ for $j = 1, \ldots, m$, we can write $AS_1 = S_1 D$, where $D = \operatorname{diag}(\lambda_i, \ldots, \lambda_i)$. From $SB = AS$, we see that

$$S_1 B_{11} + S_2 B_{21} = AS_1 = S_1 D.$$

Rewriting this expression, we get

$$0 = S_1(B_{11} - D) + S_2 B_{21} = S \begin{bmatrix} B_{11} - D \\ B_{21} \end{bmatrix}.$$

Since S is invertible, this equation implies that $B_{11} = D$ and $B_{21} = 0$. Hence, we have

$$B = \begin{bmatrix} D & B_{12} \\ 0 & B_{21} \end{bmatrix}.$$

The characteristic polynomial of B is the same as that of A because the transformation $B = S^{-1}AS$ preserves it[1]. Expanding the determinant of $\lambda I_n - B$, we get

$$p_A(\lambda) = p_B(\lambda) = \det \begin{bmatrix} \lambda I_m - D & -B_{12} \\ 0 & \lambda I_{n-m} - B_{22} \end{bmatrix} = (\lambda - \lambda_i)^m p_{B_{22}}(\lambda),$$

where we used the fact that the determinant of a block upper triangular matrix equals the product of the determinants of its diagonal blocks.

Thus, the algebraic multiplicity of λ_i, which is the power of $(\lambda - \lambda_i)$ in $p_A(\lambda)$, satisfies $\operatorname{alg}_A(\lambda_i) \geqslant m = \operatorname{geom}_A(\lambda_i)$, since we do not know whether more factors of $(\lambda - \lambda_i)$ can be factored out from $p_{B_{22}}(\lambda)$. □

Those eigenvalues for which the inequality in Theorem 36.19 becomes an equality are important enough to receive a name.

[1] This transformation is called *similarity*. We review it in Section 36.2.

Definition 36.20. (Regular eigenvalue)

For a complex square matrix A, an eigenvalue λ is called **regular** when its algebraic and geometric multiplicities are equal.

When a complex matrix A is real, the eigenvectors associated with a real eigenvalue can be chosen to be real since the linear system of equations $(\lambda I_n - A)x = 0$ has real solutions. This means the eigenspace for a real eigenvalue can be defined as a subspace over the real numbers

$$\text{eig}_A(\lambda) = \{x \in \mathbb{R}^n \mid Ax = \lambda x\} \subseteq \mathbb{R}^n.$$

It can also be described as the nullspace of the matrix $\lambda I_n - A$ over the reals:

$$\text{eig}_A(\lambda) = \text{null}(\lambda I_n - A) = \{x \in \mathbb{R}^n \mid (\lambda I_n - A)x = 0\}.$$

Additionally, the geometric multiplicity of a real eigenvalue λ is the same whether we work in $\mathbb{R}^n$ or $\mathbb{C}^n$. This is because the geometric multiplicity depends only on the number of linearly independent solutions to $(\lambda I_n - A)x = 0$, which is determined by the number of pivot columns in $\lambda I_n - A$.

Example 36.21 The matrices A and B from Example 36.9 are real and have real eigenvalues. This means their eigenspaces can be defined as subspaces over the real numbers. We have

$$\begin{aligned}
\text{eig}_A(3) &= \left\{ \begin{bmatrix} x_1 \\ x_1 \end{bmatrix} \mid x_1 \in \mathbb{R} \right\} = \text{span}\left(\begin{bmatrix} 1 \\ 1 \end{bmatrix} \right), \\
\text{eig}_A(-1) &= \left\{ \begin{bmatrix} x_1 \\ -x_1 \end{bmatrix} \mid x_1 \in \mathbb{R} \right\} = \text{span}\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix} \right), \quad \text{and} \\
\text{eig}_B(2) &= \left\{ \begin{bmatrix} x_1 \\ 0 \end{bmatrix} \mid x_1 \in \mathbb{R} \right\} = \text{span}\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} \right),
\end{aligned}$$

which are all subspaces of $\mathbb{R}^2$.

We could use subscripts like $\text{span}_{\mathbb{R}}$ or $\text{span}_{\mathbb{C}}$ to specify whether we mean real or complex linear combinations. However, we will skip the subscripts and rely on context to make it clear.

Definition 36.22. (Real spectrum)

A real $n \times n$ matrix A is said to have a **real spectrum** if all its eigenvalues are real.

Example 36.23 The matrices A and B in Example 36.9 both have a real spectrum.

We end this section with two important facts about invertible matrices: eigenvalues indicate if a matrix is invertible, and the eigenvalues and eigenvectors of A are related to those of A^{-1}.

Proposition 36.24. (Eigenvalues, invertibility, and the inverse matrix)

Let $A \in \mathbb{C}^{n\times n}$. Then:

(a) A is invertible if and only if zero is not an eigenvalue of A.
(b) If A is invertible, x is an eigenvector of A with eigenvalue λ if and only if x is an eigenvector of A^{-1} with eigenvalue $\frac{1}{\lambda}$.

36.2 Similar matrices and diagonalizability

One of the main applications of eigenvalues and eigenvectors is to simplify matrix computations using a process called similarity.

Definition 36.25. (Similar matrices)

We say that two $n \times n$ matrices A and B are **similar** if there exists a nonsingular matrix S such that $B = SAS^{-1}$, where SAS^{-1} is called a **similarity transformation** of the matrix A.

If we want to highlight the role of the matrix S in the relationship between A and B, we say "A is similar to B via S". It does not matter if we say "A is similar to B" or "B is similar to A" because $B = SAS^{-1}$ is equivalent to $A = S^{-1}BS$. This relationship is actually an equivalence relation (see Problem 36.12).

If the similarity transformation uses an orthogonal matrix Q (so $Q^{-1} = Q^T$), then $B = QAQ^T$ is called an **orthogonal similarity**. If S is a unitary matrix instead, we call it a **unitary similarity**.

Similarity preserves many important properties of the original matrix A, including the rank, trace, and determinant.

Proposition 36.26. (Similarity preserves ranks, traces and determinants)

Let A and B be two $n \times n$ similar matrices, i.e., $B = SAS^{-1}$, for some nonsingular matrix S. Then $\text{rank}(A) = \text{rank}(B)$, $\text{trace}(A) = \text{trace}(B)$, and $\det(A) = \det(B)$.

Proof The preservation of the rank follows from the fact that multiplication by nonsingular matrices preserves the rank, as we showed in Theorem 17.34.

To show that the trace and determinant are preserved, we use their invariance under cyclic permutations: for any square matrices A and B, $\operatorname{trace}(AB) = \operatorname{trace}(BA)$ and $\det(AB) = \det(BA)$. Thus, we have

$$\begin{aligned}
\operatorname{trace}(B) &= \operatorname{trace}(SAS^{-1}) && \text{(by assumption)} \\
&= \operatorname{trace}(S(AS^{-1})) && \text{(associativity of matrix multiplication)} \\
&= \operatorname{trace}(AS^{-1}S) && \text{(invariance under cyclic permutations)} \\
&= \operatorname{trace}(A).
\end{aligned}$$

Similarly, we have

$$\begin{aligned}
\det(B) &= \det(SAS^{-1}) && \text{(by assumption)} \\
&= \det(S(AS^{-1})) && \text{(associativity of matrix multiplication)} \\
&= \det(AS^{-1}S) && \text{(invariance under cyclic permutations)} \\
&= \det(A).
\end{aligned}$$

Therefore, the proposition is proved. □

Similarity transformations preserve not only the rank, trace, and determinant of a matrix but also its spectrum. They also modify the eigenspaces (i.e., eigenvectors) in a predictable manner.

Proposition 36.27. (Eigenvalues and eigenvectors under similarity)

Let A and B be two $n \times n$ similar matrices, with $B = SAS^{-1}$ for some nonsingular matrix S. Then:

(a) The spectra of A and B are the same, i.e., $\sigma(A) = \sigma(B)$.
(b) The eigenvectors of B for an eigenvalue λ are obtained by multiplying the eigenvectors of A by S, i.e.,

$$\operatorname{eig}_B(\lambda) = \{Sv \mid v \in \operatorname{eig}_A(\lambda)\}.$$

Proof The characteristic polynomial of B satisfies

$$\begin{aligned}
p_B(\lambda) = \det(\lambda I_n - B) &= \det(\lambda I_n - SAS^{-1}) \\
&= \det(\lambda SS^{-1} - SAS^{-1}) \\
&= \det(S(\lambda I_n - A)S^{-1}) \\
&= \det(\lambda I_n - A) = p_A(\lambda).
\end{aligned}$$

This shows that A and B have the same characteristic polynomial, and therefore the same spectrum.

To prove the claim about eigenvectors, let $\lambda \in \sigma(B) = \sigma(A)$. If v is an eigenvector of A with eigenvalue λ, then:

$$B(Sv) = SAS^{-1}(Sv) = SAv = S(\lambda v) = \lambda(Sv).$$

Since S is invertible, $Sv \neq 0$ whenever $v \neq 0$. This shows that Sv is an eigenvector of B corresponding to λ. Thus, we have $\{Sv \mid v \in \text{eig}_A(\lambda)\} \subseteq \text{eig}_B(\lambda)$.

For the reverse inclusion, let w be an eigenvector of B with eigenvalue λ. Since the columns of S form a basis for $\mathbb{C}^n$, we can write $w = Sv$ for some nonzero vector v. Moreover,

$$Av = S^{-1}BSv = S^{-1}Bw = \lambda S^{-1}w = \lambda v.$$

This shows that v is an eigenvector of A with eigenvalue λ, and hence $w = Sv$ for some $v \in \text{eig}_A(\lambda)$. In conclusion, $\text{eig}_B(\lambda) = \{Sv \mid v \in \text{eig}_A(\lambda)\}$. □

As a corollary of the result above, both the algebraic and geometric multiplicities of an eigenvalue are preserved under similarity transformations.

While similar matrices share the same eigenvalues, the reverse is not always true. Two matrices with the same eigenvalues are not necessarily similar. This is illustrated by the following example.

Example 36.28 Consider the matrices $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ and $B = \left[\begin{smallmatrix} 0 & 0 \\ 0 & 0 \end{smallmatrix}\right]$. Both have the same characteristic polynomial, $p_A(\lambda) = p_B(\lambda) = \lambda^2$, and therefore share the same eigenvalues, $\lambda_1 = \lambda_2 = 0$.

However, A and B are not similar. To see this, suppose A and B were similar. Then, there would exist a nonsingular matrix S such that

$$A = SBS^{-1} = 0,$$

which is clearly not true.

Similarity transformations can often simplify a matrix, and the diagonal form is the simplest and most useful case. This motivates the following definition.

Definition 36.29. (Diagonalizable matrices)

A matrix A is **diagonalizable** if it is similar to a diagonal matrix. That means there exists an invertible matrix S such that $A = SDS^{-1}$, where D is a diagonal matrix.

If A, S, and D are all real matrices, we say A is **diagonalizable over** $\mathbb{R}$.

Since similarity transformations preserve eigenvalues, if A is similar to a diagonal matrix D, then the diagonal entries of D are the eigenvalues of A.

Wouldn't it be nice if all matrices were diagonalizable? Life in Linear Algebra would be much easier. As we will see, diagonalizable matrices are easier to work with and help solve many problems. But not every matrix can be turned into diagonal form through a similarity transformation, as the next example shows.

Example 36.30 Consider the matrix $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$. Let us show that it is not diagonalizable. Suppose A were diagonalizable. Then there would exist an invertible matrix

S such that $A = SDS^{-1}$, where D is diagonal. The eigenvalues of A are $\lambda_1 = \lambda_2 = 0$, so $D = 0$. Substituting into $A = SDS^{-1}$, we get $A = 0$, which is false. Thus, A is not diagonalizable.

36.2.1 Characterizations of diagonalizable matrices

In this section, we aim to describe when a matrix is diagonalizable. We will present two equivalent characterizations.

The following theorem characterizes diagonalizable matrices based on their eigenvectors.

Theorem 36.31. (First characterization of diagonalizable matrices)

An $n \times n$ matrix A is diagonalizable if and only if there exists a basis of $\mathbb{C}^n$ consisting of eigenvectors of A.

Proof Assume A is diagonalizable. Then $A = SDS^{-1}$, where S is invertible, and D is a diagonal matrix with eigenvalues $\lambda_1, \ldots, \lambda_n$ on its main diagonal. Let $x_1, x_2, \ldots, x_n$ be the columns of S. Since S is invertible, these columns are nonzero. From $AS = SD$, we have $Ax_i = \lambda_i x_i$ for $i = 1, 2, \ldots, n$, so the columns of S are eigenvectors of A. Because S is invertible, these eigenvectors are linearly independent and form a basis for $\mathbb{C}^n$.

Conversely, assume $\{x_1, \ldots, x_n\}$ is a basis for $\mathbb{C}^n$ consisting of eigenvectors of A, and let λ_i be the eigenvalue corresponding to x_i, so $Ax_i = \lambda_i x_i$, for $i = 1, 2, \ldots, n$. Then

$$\begin{aligned} A \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix} &= \begin{bmatrix} Ax_1 & \cdots & Ax_n \end{bmatrix} \\ &= \begin{bmatrix} \lambda_1 x_1 & \cdots & \lambda_n x_n \end{bmatrix} \\ &= \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix} \operatorname{diag}(\lambda_1, \ldots, \lambda_n). \end{aligned}$$

Let $S := \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}$ and $D := \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$. Then $AS = SD$. Since $\{x_1, \ldots, x_n\}$ are linearly independent, S is invertible, and we can write $A = SDS^{-1}$. Thus, A is diagonalizable. □

The proof of Theorem 36.31 shows how to construct the matrix S to diagonalize A. To do this, find n linearly independent eigenvectors of A and use them as the columns of S. Then, $S^{-1}AS$ will be diagonal, with the eigenvalues of A as its main diagonal entries, ordered to match the eigenvectors in S.

Example 36.32 Consider the matrix $A = \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix}$. Its characteristic polynomial is $p_A(\lambda) = \lambda^2 - 4\lambda - 5 = (\lambda + 1)(\lambda - 5)$, so the eigenvalues of A are $\lambda_1 = -1$ and $\lambda_2 = 5$. Corresponding eigenvectors are $x_1 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$ and $x_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Since x_1 and x_2 are linearly independent, A is diagonalizable. We construct S using these eigenvectors as its columns:

$$S = \begin{bmatrix} x_1 & x_2 \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 1 & 1 \end{bmatrix}.$$

Now we compute

$$S^{-1}AS = \frac{1}{3}\begin{bmatrix} -1 & 1 \\ 1 & 2 \end{bmatrix}\begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix}\begin{bmatrix} -2 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 5 \end{bmatrix}.$$

This shows that A is diagonalizable.

We can use Theorem 36.31 to see why the matrix $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ from Example 36.30 is not diagonalizable.

Example 36.33 The matrix $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ has a single eigenvalue, $\lambda = 0$, with algebraic multiplicity 2. The eigenspace for $\lambda = 0$ is

$$\text{eig}_A(0) = \text{span}\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right).$$

Since $\text{eig}_A(0)$ is one-dimensional, we cannot find two linearly independent eigenvectors to form a basis of $\mathbb{C}^2$. So, A is not diagonalizable.

Theorem 36.31 shows that to diagonalize a matrix, we need a basis of $\mathbb{C}^n$ made up of eigenvectors of A. But as the example above shows, not every matrix has enough linearly independent eigenvectors to do this. This leads to a natural question: *How many linearly independent eigenvectors does a matrix have?* To begin answering this, we will prove some important properties of eigenvectors corresponding to distinct eigenvalues.

Theorem 36.34. (Eigenvectors with distinct eigenvalues are linearly independent)

Let $A \in \mathbb{C}^{n\times n}$ be a matrix, and let $x_1, x_2, \ldots, x_k$ be eigenvectors of A with corresponding eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k$. If the eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k$ are distinct, then the set $\{x_1, x_2, \ldots, x_k\}$ is linearly independent.

Furthermore, if β_i is a set of linearly independent eigenvectors associated with λ_i, then the combined set $\beta_1 \cup \beta_2 \cup \cdots \cup \beta_k$ is also linearly independent.

Proof We prove the first claim and leave the second as an exercise (see Problem 36.20). The proof uses induction on the number k of eigenvectors.

When $k = 1$, we have a single eigenvector x_1 with eigenvalue λ_1. Since eigenvectors are nonzero by definition, $\{x_1\}$ is linearly independent.

Assume the result holds for $k = i - 1$, where $i - 1 \geqslant 1$. Now consider a set of i eigenvectors $\{x_1, x_2, \ldots, x_i\}$ of A, with distinct eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_i$. We will show that $\{x_1, x_2, \ldots, x_i\}$ is linearly independent.

Suppose

$$c_1 x_1 + c_2 x_2 + \cdots + c_i x_i = 0, \tag{36.1}$$

for some scalars $c_1, c_2, \ldots, c_i$. We aim to prove $c_1 = c_2 = \cdots = c_i = 0$.

Multiplying (36.1) by A gives

$$\lambda_1 c_1 x_1 + \lambda_2 c_2 x_2 + \cdots + \lambda_i c_i x_i = 0, \tag{36.2}$$

where we used $Ax_j = \lambda_j x_j$, for $j = 1, 2, \ldots, i$. Multiplying (36.1) by λ_1 gives:

$$\lambda_1 c_1 x_1 + \lambda_1 c_2 x_2 + \cdots + \lambda_1 c_i x_i = 0. \tag{36.3}$$

Subtracting (36.3) from (36.2) gives

$$(\lambda_2 - \lambda_1) c_2 x_2 + (\lambda_3 - \lambda_1) c_3 x_3 + \cdots + (\lambda_i - \lambda_1) c_i x_i = 0.$$

By the induction hypothesis, the set $\{x_2, x_3, \ldots, x_i\}$ is linearly independent. This implies

$$(\lambda_2 - \lambda_1) c_2 = (\lambda_3 - \lambda_1) c_3 = \cdots = (\lambda_i - \lambda_1) c_i = 0.$$

Since the eigenvalues $\lambda_2, \lambda_3, \ldots, \lambda_i$ are distinct from λ_1, we conclude $c_2 = c_3 = \cdots = c_i = 0$. Substituting $c_2 = c_3 = \cdots = c_i = 0$ into (36.1), we get $c_1 x_1 = 0$. Since $x_1 \neq 0$, it follows that $c_1 = 0$.

Thus, $c_1 = c_2 = \cdots = c_i = 0$, proving that $\{x_1, x_2, \ldots, x_i\}$ is linearly independent. □

Corollary 36.35. (Distinct eigenvalues imply diagonalizability)

An $n \times n$ matrix with n distinct eigenvalues is diagonalizable.

For the following theorem, it will be helpful to revisit the idea of a direct sum of subspaces in Chapter 16.

Theorem 36.36. (Direct sum of eigenspaces)

Let $A \in \mathbb{C}^{n \times n}$, and suppose $\lambda_1, \ldots, \lambda_k$ are distinct eigenvalues of A. Then, the sum of the corresponding eigenspaces is a direct sum:

$$\operatorname{eig}_A(\lambda_1) + \cdots + \operatorname{eig}_A(\lambda_k) = \operatorname{eig}_A(\lambda_1) \oplus \cdots \oplus \operatorname{eig}_A(\lambda_k).$$

Proof Let $v_i \in \operatorname{eig}_A(\lambda_i)$, for $i = 1, \ldots, k$, and assume that $v_1 + \cdots + v_k = 0$. We need to show that $v_1 = \cdots = v_k = 0$ to prove that the sum of the eigenspaces is a direct sum (see Theorem 16.20).

Each v_i is either zero or an eigenvector of A with eigenvalue λ_i (and hence nonzero). If one or more v_i were nonzero, the equation $v_1 + \cdots + v_k = 0$ would represent a linear combination of eigenvectors producing the zero vector. However, eigenvectors corresponding to distinct eigenvalues are linearly independent (Theorem 36.34). It follows that $v_1 = \cdots = v_k = 0$. Thus, $\text{eig}_A(\lambda_1) + \cdots + \text{eig}_A(\lambda_k)$ is a direct sum. □

As we saw in Example 36.32, an $n \times n$ matrix with fewer than n distinct eigenvalues may or may not be diagonalizable. It depends on how many linearly independent eigenvectors the matrix has. The next theorem determines the maximum number of linearly independent eigenvectors a matrix can have.

Theorem 36.37. (Maximum number of independent eigenvectors)

Let A be an $n \times n$ matrix with distinct eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k$. The maximum number of linearly independent eigenvectors of A is

$$\text{geom}_A(\lambda_1) + \text{geom}_A(\lambda_2) + \cdots + \text{geom}_A(\lambda_k).$$

Proof Let A be an $n \times n$ matrix. Define $g := \text{geom}_A(\lambda_1) + \cdots + \text{geom}_A(\lambda_k)$, and let m be the maximum number of linearly independent eigenvectors of A. We aim to show that $m = g$.

First, we show that $m \geqslant g$. We build a set of g linearly independent eigenvectors by taking a basis for each eigenspace $\text{eig}_A(\lambda_i)$ and combining them into one set. By Theorem 36.34, this combined set is also linearly independent, so $m \geqslant g$.

Next, we show that $m \leqslant g$. Let $\{v_1, v_2, \ldots, v_m\}$ be a set of m linearly independent eigenvectors of A. Each v_i belongs to one of the eigenspaces $\text{eig}_A(\lambda_1), \ldots, \text{eig}_A(\lambda_k)$, so $\{v_1, v_2, \ldots, v_m\}$ is a linearly independent subset of the subspace $\text{eig}_A(\lambda_1) + \cdots + \text{eig}_A(\lambda_k)$, which is a direct sum by Theorem 36.36. Since the dimension of this space is g, we have $m \leqslant g$.

Combining the inequalities $m \geqslant g$ and $m \leqslant g$, we conclude that $m = g$. □

We now introduce a second characterization of diagonalizable matrices, this time in terms of geometric and algebraic multiplicities.

Theorem 36.38. (Second characterization of diagonalizable matrices)

An $n \times n$ matrix A is diagonalizable if and only if the geometric multiplicity of each eigenvalue λ equals its algebraic multiplicity. In other words,

$$\text{geom}_A(\lambda) = \text{alg}_A(\lambda) \quad \text{for every } \lambda \in \sigma(A).$$

Proof First, assume that the geometric multiplicity of each eigenvalue λ of A equals its algebraic multiplicity, i.e., $\text{geom}_A(\lambda) = \text{alg}_A(\lambda)$ for every $\lambda \in \sigma(A)$. Using Theorems 36.8 and 36.37, we know that the total number of linearly independent

eigenvectors of A is

$$\sum_{\lambda\in\sigma(A)} \operatorname{geom}_A(\lambda) = \sum_{\lambda\in\sigma(A)} \operatorname{alg}_A(\lambda) = n.$$

By Theorem 36.31, this implies A is diagonalizable.

Now, suppose A is diagonalizable. From Theorems 36.31 and 36.37, we know that

$$\sum_{\lambda\in\sigma(A)} \operatorname{geom}_A(\lambda) = n.$$

Using Theorem 36.19, we also have

$$n = \sum_{\lambda\in\sigma(A)} \operatorname{geom}_A(\lambda) \leqslant \sum_{\lambda\in\sigma(A)} \operatorname{alg}_A(\lambda) = n.$$

Since the inequality is an equality, we conclude that $\operatorname{geom}_A(\lambda) = \operatorname{alg}_A(\lambda)$ for every $\lambda \in \sigma(A)$. □

36.3 Non-diagonalizable matrices and the Jordan canonical form

Diagonalization simplifies many matrix operations by turning a matrix into diagonal form. But not all matrices are diagonalizable. So what can we do with those that are not?

The Jordan canonical form gives us a way forward. It transforms any square matrix into a block diagonal form, where each block is almost diagonal, with just a few ones above the diagonal.

Definition 36.39. (Jordan block)

A Jordan block of size $k \times k$ associated with a scalar μ, denoted $J_k(\mu)$, is the matrix given by

$$J_k(\mu) := \begin{bmatrix} \mu & 1 & & & \\ & \mu & 1 & & \\ & & \ddots & \ddots & \\ & & & \mu & 1 \\ & & & & \mu \end{bmatrix} \in \mathbb{C}^{k\times k}. \tag{36.4}$$

The following proposition states some basic properties of a Jordan block. We ask your to prove these properties in Problem 36.33.

Proposition 36.40. (Properties of a Jordan block)

Let $J_k(\mu)$ be the Jordan block in (36.4). Then, the following statements hold.

(a) $\sigma(J_k(\mu)) = \{\mu\}$;
(b) $\operatorname{alg}_{J_k(\mu)}(\mu) = k$;
(c) $\operatorname{eig}_{J_k(\mu)}(\mu) = \operatorname{span}(e_1)$;

(d) $\text{geom}_{J_k(\mu)}(\mu) = 1$.

The next theorem is the main result of this section. It shows that any complex matrix can be transformed into a block diagonal form, where each block is upper bidiagonal: the diagonal entries are eigenvalues, the entries just above the diagonal are 1, and all other entries are zero.

Theorem 36.41. (The Jordan canonical form)

Let $A \in \mathbb{C}^{n\times n}$. Then, there exists an invertible matrix S such that $J := S^{-1}AS$ is a block diagonal matrix, where each block is a Jordan block. Moreover, the number of Jordan blocks corresponding to an eigenvalue λ_i of A equals $\text{geom}_A(\lambda_i)$. The matrix J is called **a Jordan canonical form** of A.

The Jordan canonical form of a matrix is not unique unless the order of the diagonal blocks is specified.

The proof of Theorem 36.41 is beyond the scope of this book. If you are curious about it, you can find it in [6].

The Jordan canonical form is especially useful for proving theoretical results. We will use it, for example, in our study of Markov Chains in Chapter 52.

Example 36.42 The following is a Jordan matrix

$$J = \left[\begin{array}{c|ccc|cc} 2 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 1 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & 3 & 1 \\ 0 & 0 & 0 & 0 & 0 & 3 \end{array}\right].$$

The eigenvalues of J are $\lambda_1 = 2$ and $\lambda_2 = 3$, with algebraic multiplicities 4 and 2, respectively. There are two Jordan blocks associated with $\lambda_1 = 2$. Thus, the geometric multiplicity of λ_1 is 2. Similarly, there is only one Jordan block for $\lambda_2 = 3$, so its geometric multiplicity is 1.

Recall that an eigenvalue is *regular* when its algebraic and geometric multiplicities are equal (Definition 36.20). In the Jordan canonical form this happens precisely when every Jordan block corresponding to that eigenvalue is 1×1.

If a matrix is *diagonalizable*, its Jordan canonical form coincides with its diagonal form: all Jordan blocks are 1×1 matrices, each containing an eigenvalue of the matrix. Hence a matrix is diagonalizable precisely when all its eigenvalues are regular.

Remark 36.43. (The fragility of Jordan blocks)

The Jordan canonical form is a great theoretical tool for understanding matrices, but it is not very practical from a computational point of view. Jordan blocks are highly sensitive to small changes in a matrix. Even tiny perturbations in the entries can transform its Jordan form into a diagonal form. This means that not only must the matrix be known exactly, without any rounding or errors, but also that computations must avoid introducing even small rounding errors. For this reason, computing the Jordan form reliably using floating-point arithmetic is nearly impossible.

The following example shows how even small changes in a matrix's entries can drastically affect its Jordan form.

Example 36.44 Consider the 2×2 Jordan block $J_2(0) = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$. For any $\epsilon > 0$, no matter how small, the perturbed matrix

$$\begin{bmatrix} 0 & 1 \\ \epsilon & 0 \end{bmatrix}$$

has two distinct eigenvalues, $\lambda = \pm\sqrt{\epsilon}$, and its Jordan form becomes

$$\begin{bmatrix} \sqrt{\epsilon} & 0 \\ 0 & -\sqrt{\epsilon} \end{bmatrix}.$$

36.4 Invariant subspaces

In this section, we introduce a special type of subspace related to matrices, called an invariant subspace. These subspaces generalize the idea of eigenvectors and eigenspaces. Just as eigenvectors help diagonalize matrices, invariant subspaces are useful because they allow us to transform a matrix into upper triangular or block upper triangular form using similarity transformations.

Before defining invariant subspaces, recall that the image of a subspace $\mathcal{V} \subseteq \mathbb{C}^n$ under a matrix A is the subspace obtained by applying A to all vectors in $\mathcal{V}$, that is,

$$A(\mathcal{V}) := \{Av \mid v \in \mathcal{V}\}.$$

Definition 36.45. (Invariant subspace)

An **invariant subspace** $\mathcal{V}$ of a matrix $A \in \mathbb{C}^{n\times n}$ is a subspace of $\mathbb{C}^n$ such that $A(\mathcal{V}) \subseteq \mathcal{V}$. In other words, the linear transformation A maps $\mathcal{V}$ onto itself.

Put simply, $\mathcal{V}$ is an invariant subspace of A if multiplying any vector v in $\mathcal{V}$ by A results in another vector that is still in $\mathcal{V}$.

Example 36.46. (Examples of invariant subspaces)

The following are examples of invariant subspaces associated with a matrix A:

(a) The trivial subspace $\{0\}$.
(b) The entire space $\mathbb{C}^n$.
(c) The column space of A.
(d) The nullspace of A.

In Problem 36.35, we ask you to show that each of these is an invariant subspace of A.

An additional important example of an invariant subspace of a matrix is the subspace spanned by one of its eigenvectors.

Proposition 36.47. (The span of an eigenvector is an invariant subspace)

Let $A \in \mathbb{C}^{n\times n}$, and let $x \in \mathbb{C}^n$ be an eigenvector of A with eigenvalue λ. Then, the subspace $\mathcal{V} = \text{span}(x)$ is an invariant subspace of A.

Proof Let $v \in \text{span}(x)$. Then $v = cx$ for some scalar c. Since $Ax = \lambda x$, we have $A(cx) = c(Ax) = c(\lambda x) = (\lambda c)x \in \text{span}(x)$. Thus, $Av \in \text{span}(x)$, which shows that $\text{span}(x)$ is an invariant subspace of A. □

In fact, the span of an eigenvector is the only type of one-dimensional invariant subspace.

Proposition 36.48. (One-dimensional invariant subspaces characterization)

Let $A \in \mathbb{C}^{n\times n}$ and let $x \in \mathbb{C}^n$ be a nonzero vector. The one-dimensional subspace $\text{span}(x)$ is an invariant subspace of A if and only if x is an eigenvector of A.

Proof Proposition 36.47 shows that if x is an eigenvector of A, then $\text{span}(x)$ is an invariant subspace of A.

Now, suppose $\mathcal{V} := \text{span}(x)$ is a one-dimensional invariant subspace of A. Then, x is a nonnzero vector. Since $\mathcal{V}$ is invariant under A, we have $Ax \in \text{span}(x)$. This gives $Ax = cx$ for some scalar c. Thus, x is an eigenvector of A with eigenvalue c.□

We can construct higher-dimensional invariant subspaces by taking the span of multiple eigenvectors. Note that, in the following proposition we are not assuming that the eigenvalues are distinct.

Proposition 36.49. (Invariant subspaces spanned by eigenvectors)

Let $A \in \mathbb{C}^{n\times n}$, and let $\{x_1, \ldots, x_p\}$ be a set of linearly independent eigenvectors of A with eigenvalues $\lambda_1, \ldots, \lambda_p$. Then, the subspace $\mathcal{V} := \text{span}(x_1, \ldots, x_p)$ is a p-dimensional invariant subspace of A.

Proof Let $v \in \mathcal{V} = \text{span}\{x_1, \ldots, x_p\}$. Then we can write $v = c_1x_1 + \cdots + c_px_p$ for some scalars $c_1, \ldots, c_p \in \mathbb{C}$. Multiplying v by A, we get

$$Av = A(c_1x_1 + \cdots + c_px_p) = c_1Ax_1 + \cdots + c_pAx_p.$$

Using $Ax_i = \lambda_i x_i$ for each eigenvector x_i, this expression becomes

$$Av = (c_1\lambda_1)x_1 + \cdots + (c_p\lambda_p)x_p \in \text{span}\{x_1, \ldots, x_p\}.$$

It follows that $\mathcal{V}$ is an invariant subspace of A. □

Proposition 36.49 shows that the eigenspaces of a matrix A and their sums are invariant subspaces of A.

Every one-dimensional invariant subspace of a matrix A is the span of an eigenvector, but this is not always true for higher-dimensional invariant subspaces. Some matrices have p-dimensional invariant subspaces that cannot be spanned by p linearly independent eigenvectors. To see this, consider the following example.

Example 36.50 Consider the matrix $A = \left[\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right]$. This matrix has a single eigenvalue $\lambda = 1$ with eigenspace

$$\text{eig}_A(\lambda) = \text{span}\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right).$$

Now, let $x_1 = \left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right]$ and $x_2 = \left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right]$. The two-dimensional subspace $\mathcal{V} = \text{span}(x_1, x_2) = \mathbb{R}^2$, which is invariant under A. However, $\mathcal{V}$ cannot be spanned by eigenvectors of A, since A does not have two linearly independent eigenvectors.

For a diagonalizable matrix A, every p-dimensional invariant subspace of A is spanned by p eigenvectors of A (see Problem 36.41).

Problems

36.1 Find by hand the eigenvalues and associated eigenvectors of the following two matrices.

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 3 \\ 3 & 3 & 20 \end{bmatrix} \qquad \text{and} \qquad B = \begin{bmatrix} -2 & -4 & 2 \\ -2 & 1 & 2 \\ 4 & 2 & 5 \end{bmatrix}.$$

36.2 Consider the matrix $A = \left[\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right]$. Show that there is no real number λ and nonzero vector $x \in \mathbb{R}^2$ such that $Ax = \lambda x$.

36.3 Find the eigenvalues of the matrix $A = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right] \in \mathbb{C}^{2\times 2}$ in terms of its entries.

36.4 For the matrix $A = \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$ with real entries, find its eigenvalues in terms of a, b, c, and d. Show that A has either two real eigenvalues or a pair of complex conjugate eigenvalues.

36.5 Prove that if every nonzero vector is an eigenvector of A, then A is a scalar multiple of the identity.

36.6 Let x and y be eigenvectors of a matrix A, and suppose $x+y$ is also an eigenvector of A. Prove that x and y correspond to the same eigenvalue.

36.7 Let A be an $n \times n$ matrix. Prove that the characteristic polynomial of A, $\det(\lambda I_n - A)$, is a degree-n monic polynomial, i.e., $\det(\lambda I_n - A) = \lambda^n + (\text{lower-order terms in } \lambda)$.

36.8 Are the eigenvalues of AB the products of the eigenvalues of A and B? What about the eigenvalues of $A + B$?

36.9 Find the eigenvalues and the associated eigenspaces of the following matrices:

$$A = \begin{bmatrix} -4 & -3 \\ 6 & 5 \end{bmatrix}, \qquad B = \begin{bmatrix} -4 & -3 & -3 \\ 0 & -1 & 0 \\ 6 & 6 & 5 \end{bmatrix}, \qquad \text{and} \qquad C = \begin{bmatrix} -1 & -1 & -2 \\ 8 & -11 & -8 \\ -10 & 11 & 7 \end{bmatrix}.$$

36.10 Let A and B be $n \times n$ matrices with $A + B = I_n$. Prove that x is an eigenvector of A if and only if it is an eigenvector of B.

36.11 Prove Proposition 36.24.

36.12 A binary relation $\sim$ on a set X is an **equivalence relation** if it satisfies the following for all $a, b, c \in X$:

(i) $a \sim a$,
(ii) $a \sim b$ if and only if $b \sim a$, and
(iii) if $a \sim b$ and $b \sim c$ then $a \sim c$.

Prove that the matrix similarity introduced in Definition 36.25 is an equivalence relation on the set of $n \times n$ matrices.

36.13 Prove that if two matrices A and B are similar, then their powers A^n and B^n are also similar.

36.14 Prove that similar matrices A and B do not necessarily share the same eigenvectors.

36.15 Prove that if A and B are similar, then $\dim(\text{eig}_A(\lambda)) = \dim(\text{eig}_B(\lambda))$ for every $\lambda \in \sigma(A)$.

36.16 Let A and B be matrices, with at least one of them invertible. Prove that AB and BA are similar, and conclude that AB and BA have the same eigenvalues.

The next problem addresses the case where neither A nor B is invertible.

36.17 Let A be an $m \times n$ matrix and let B be an $n \times m$. Verify the equality

$$\begin{bmatrix} I_m & 0 \\ B & I_n \end{bmatrix} \begin{bmatrix} AB & A \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & A \\ 0 & BA \end{bmatrix} \begin{bmatrix} I_m & 0 \\ B & I_n \end{bmatrix}.$$

Using this, show that AB and BA have the same nonzero eigenvalues. Finally, discuss what happens to the zero eigenvalues.

36.18 Consider a matrix A such that $\dim(\text{col}(A)) = p$. Prove that A has at most $p+1$ distinct eigenvalues.

36.19 Find the algebraic and geometric multiplicities of the eigenvalue $\lambda = 1$ of the following matrices:

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad \text{and} \quad D = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

36.20 Let $A \in \mathbb{C}^{n\times n}$ be a matrix with distinct eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k$. Suppose $\beta_1, \beta_2, \ldots, \beta_k$ are linearly independent sets of eigenvectors corresponding to $\lambda_1, \lambda_2, \ldots, \lambda_k$, respectively. Prove that $\beta_1 \cup \beta_2 \cup \cdots \cup \beta_k$ is a linearly independent set.

36.21 For the matrices A, B, and C from Problem 36.9, determine if they can be diagonalized. If they can, diagonalize them; otherwise, explain why diagonalization is not possible.

36.22 Consider the following linear transformations on $\mathbb{R}^2$. For each, find the real eigenvalues and eigenvectors, and determine whether the transformation is diagonalizable.

(a) Reflection across the horizontal axis.
(b) Counterclockwise rotation by $\frac{\pi}{2}$ radians.
(c) Counterclockwise rotation by π radians.

36.23 Consider the $n \times n$ matrix "of all 1s":

$$A = \begin{bmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{bmatrix}.$$

Determine if A is diagonalizable. If it is, find an invertible matrix S and diagonal matrix D such that $A = S^{-1}DS$.

36.24 Consider a matrix A with a characteristic polynomial $p(\lambda) = (\lambda+1)^3(\lambda-2)^2$.

(a) Determine the size of A.
(b) Find the eigenvalues of A and the possible dimensions of the associated eigenspaces.
(c) Find the determinant and the trace of A.
(d) If A is diagonalizable, find a diagonal matrix D that is similar to A.

36.25 (Left eigenvectors) Let $A = SDS^{-1}$ be a real diagonalizable matrix, where D is diagonal. Determine if A^T is diagonalizable. If it is, find its eigenvalues and a basis of eigenvectors (known as the **left eigenvectors** of A).

36.26 Prove that if two matrices can be diagonalized by the same matrix S, then they commute.

36.27 Prove that if a matrix A is diagonalizable, then any matrix similar to A is also diagonalizable.

36.28 Prove that an invertible matrix A is diagonalizable if and only if A^{-1} is diagonalizable.

36.29 Consider the matrices below and determine whether they are diagonalizable over the real numbers and/or over the complex numbers:

$$A = \begin{bmatrix} 3 & 1 \\ -2 & 5 \end{bmatrix}, \quad B = \begin{bmatrix} -5 & 9 \\ -4 & 7 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 1 & 1 \\ -2 & 3 & -2 \\ -1 & 0 & 1 \end{bmatrix}, \quad \text{and} \quad D = \begin{bmatrix} 0 & -1 & 1 \\ 0 & 2 & 0 \\ -2 & -1 & 3 \end{bmatrix}.$$

36.30 Are projection matrices diagonalizable?

Hint: We have already answered this question in Chapter 32.

36.31 Given two real $m \times n$ matrices A and B with full column rank, consider the orthogonal projection matrices $P_A = A(A^TA)^{-1}A^T$ and $P_B = B(B^TB)^{-1}B^T$ onto $\mathrm{col}(A)$ and $\mathrm{col}(B)$, respectively. Show that P_A and P_B are similar.

36.32 Let P be the orthogonal projection matrix onto a subspace $\mathcal{V} \subseteq \mathbb{R}^n$. Show that $\mathrm{trace}(P) = \dim(\mathcal{V})$.

36.33 Check that all the statements in Proposition 36.40 about the Jordan block (36.4) are true.

36.34 (The real Jordan canonical form) Given a complex matrix A, we defined its Jordan canonical form to be a block-diagonal matrix whose main-diagonal entries are the eigenvalues of A, which may themselves be complex numbers.

When A is real, there is an analogous decomposition, the *real Jordan canonical form*. Investigate this canonical form. Describe the possible blocks that can occur and explain how each block encodes either a real eigenvalue or a complex-conjugate eigenvalue pair A.

36.35 Show that the subspaces in Example 36.46 are invariant subspaces.

36.36 Determine all the invariant subspaces of the matrix $A = \left[\begin{smallmatrix} 2 & -5 \\ 1 & -2 \end{smallmatrix}\right]$ over the real and complex numbers.

36.37 Let $A \in \mathbb{C}^{n\times n}$, and let $\{x_1, \ldots, x_p\}$ be linearly independent eigenvectors of A with eigenvalues $\lambda_1, \ldots, \lambda_p$. In Proposition 36.49, we showed that the subspace $\mathcal{V} = \mathrm{span}(x_1, \ldots, x_p)$ is invariant under A, i.e.,

$$A(\mathcal{V}) \subseteq \mathcal{V}.$$

Prove that $A(\mathcal{V}) = \mathcal{V}$ if and only if $\lambda_1, \ldots, \lambda_p$ are all nonzero.

36.38 Let $\mathcal{V}$ and $\mathcal{W}$ be two invariant subspaces of a matrix A. Are the subspaces $\mathcal{V} \cup \mathcal{W}$ and $\mathcal{V} + \mathcal{W}$ invariant subspaces of A?

36.39 Let $A \in \mathbb{C}^{n\times n}$. Show that if $\mathcal{V}$ is an invariant subspace of A, then $\mathcal{V}^\perp$ is an invariant subspace of A^*.

36.40 Let A and B be two matrices such that $AB = BA$. Show that $\text{null}(B)$ and $\text{col}(B)$ are invariant subspaces of A. Show that the eigenspaces $\text{eig}_B(\lambda)$, for $\lambda \in \sigma(B)$, are also invariant subspaces of A.

36.41 Prove that every p-dimensional subspace of a diagonalizable matrix A is spanned by p eigenvectors of A.

36.42 Let A be a 3×3 matrix with three distinct eigenvalues λ_1, λ_2 and λ_3, and associated eigenvectors v_1, v_2 and v_3. Find all the invariant subspaces of A.

36.43 Find an invariant subspace of the matrix

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & -1 \end{bmatrix}$$

that is not spanned by the eigenvectors of A.

36.44 Consider the matrix

$$A = \begin{bmatrix} 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Find a real invariant subspace of dimension 2. Does A have a real invariant subspace of dimension 3?

Chapter 37
VIII-CORE TOPICS

As we recalled in the previous chapter, the eigenvalues of a matrix are the roots of its characteristic polynomial. So, solving the characteristic equation might seem like a natural way to compute them. But in practice, this approach is not a good idea. Even small changes in the polynomial's coefficients, such as those caused by round-off errors, can lead to large errors in the computed roots. A well-known example of this problem will be discussed in Section 37.4.

In this chapter, we introduce algorithms for computing eigenvalues and eigenvectors (in Section 37.4). Before that, we study the spectra and eigenspaces of certain special matrices. Their properties give useful insight into how the algorithms work and will be important in later chapters.

37.1 Eigenvalues and eigenvectors of special matrices

Determining the eigenvalues of a matrix directly from its entries is usually not possible. However, in some special cases, we can identify the eigenvalues by inspection.

Proposition 37.1. (Eigenvalues of triangular matrices)

The eigenvalues of an upper or lower triangular matrix are its diagonal entries.

Proof The result follows from the fact that $\lambda I_n - A$ is also triangular, and the determinant of a triangular matrix equals the product of its main diagonal entries. Thus, we have

$$\det(\lambda I_n - A) = \prod_{i=1}^{n} (\lambda - A_{ii}),$$

which shows that the roots of the characteristic polynomial are the diagonal entries A_{ii}, as claimed. □

Proposition 37.1 is the reason why triangular matrices are so important for finding eigenvalues. Many algorithms rely on transforming A into a triangular matrix using similarity transformations, which preserve the eigenvalues. Once in triangular form, the eigenvalues A can be read directly from the diagonal. But let us not get ahead

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_37

of ourselves. First, we will continue studying the eigenvalues and eigenvectors of special matrices.

We can generalize Proposition 37.1 to block triangular matrices. If a matrix A is block triangular, its eigenvalues are the combined eigenvalues of its diagonal blocks.

Proposition 37.2. (Eigenvalues of block triangular matrices)

Let A be a $k \times k$ block triangular matrix, either upper or lower triangular, that is, a matrix of the form

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1k} \\ 0 & A_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & A_{k-1,k} \\ 0 & \cdots & 0 & A_{kk} \end{bmatrix} \quad \text{or} \quad A = \begin{bmatrix} A_{11} & 0 & \cdots & 0 \\ A_{21} & A_{22} & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ A_{k1} & \cdots & A_{k,k-1} & A_{kk} \end{bmatrix},$$

where the block diagonal entries $A_{11}, \ldots, A_{kk}$ are all square matrices. Then, the eigenvalues of A are the union of the eigenvalues of its diagonal blocks, that is,

$$\sigma(A) = \sigma(A_{11}) \cup \sigma(A_{22}) \cup \cdots \cup \sigma(A_{kk}).$$

Proof The matrix $\lambda I - A$ is also block triangular, with diagonal blocks $\lambda I_{n_i} - A_{ii}$, where n_i is the size of the i-th diagonal block. The determinant of $\lambda I - A$ is the product of the determinants of these diagonal blocks:

$$\det(\lambda I - A) = \prod_{i=1}^{k} \det(\lambda I_{n_i} - A_{ii}).$$

Since λ is an eigenvalue of A if and only if $\det(\lambda I - A) = 0$, this happens precisely when $\det(\lambda I_{n_i} - A_{ii}) = 0$ for at least one i. Thus, the eigenvalues of A are exactly the eigenvalues of its diagonal blocks. □

The next proposition shows some important properties of the eigenvalues and eigenvectors of orthogonal matrices.

Proposition 37.3. (Eigenvalues and eigenvectors of an orthogonal matrix)

Let $Q \in \mathbb{R}^{n \times n}$ be an orthogonal matrix. Then,

(a) The eigenvalues of Q have modulus one, i.e., $|\lambda| = 1$ for all $\lambda \in \sigma(Q)$.
(b) Eigenvectors corresponding to distinct eigenvalues are orthogonal, that is, $x_1^* x_2 = 0$ if $x_1 \in \mathbb{C}^n$ and $x_2 \in \mathbb{C}^n$ are eigenvectors of Q with different eigenvalues.

Proof Let Q be an $n \times n$ orthogonal matrix, i.e., $Q^T Q = Q Q^T = I_n$. Let λ be an eigenvalue of Q, and let x be a corresponding eigenvector. We want to show that

$|\lambda| = 1$. Taking the 2-norm of both sides of $Qx = \lambda x$ and noting that orthogonal matrices preserve the 2-norm, we get

$$\|x\|_2 = \|Qx\|_2 = |\lambda| \, \|x\|_2.$$

Since $\|x\|_2 \neq 0$, it follows that $|\lambda| = 1$.

Now, let x_1 and x_2 be eigenvectors of Q corresponding to distinct eigenvalues λ_1 and λ_2, respectively. Then

$$x_1^* x_2 = (Qx_1)^*(Qx_2) = (\lambda_1 x_1)^*(\lambda_2 x_2) = \overline{\lambda_1}\lambda_2 x_1^* x_2,$$

where the first equality follows from the fact that orthogonal matrices preserve inner products. Rearranging gives

$$(1 - \overline{\lambda_1}\lambda_2)x_1^* x_2 = 0.$$

If $x_1^* x_2 \neq 0$, then $1 = \overline{\lambda_1}\lambda_2$. Thus,

$$\lambda_2 = \frac{1}{\overline{\lambda_1}} = \frac{\lambda_1}{\|\lambda_1\|^2} = \lambda_1,$$

contradicting the assumption that $\lambda_1 \neq \lambda_2$. Therefore, $x_1^* x_2 = 0$, showing that eigenvectors of Q corresponding to distinct eigenvalues are orthogonal. □

In Problem 37.5, you will generalize the result of Proposition 37.3 to unitary matrices.

Example 37.4. (Eigenvalues and eigenvectors of a 2×2 Givens rotator)

Consider the 2×2 Givens rotator

$$Q = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \in \mathbb{R}^{2\times 2},$$

where $0 \leqslant \theta < 2\pi$. Givens rotators are orthogonal, as we saw in Chapter 27. To find the eigenvalues, we solve the characteristic equation:

$$\det(\lambda I_2 - Q) = (\lambda - \cos(\theta))^2 + \sin^2(\theta) = \lambda^2 - 2\cos(\theta)\lambda + 1 = 0.$$

This simplifies to $\lambda = \cos(\theta) \pm i \sin(\theta)$, giving eigenvalues $\lambda_1 = \cos(\theta) + i\sin(\theta) = e^{i\theta}$ and $\lambda_2 = \cos(\theta) - i\sin(\theta) = e^{-i\theta}$. Notice that both eigenvalues have modulus 1 (i.e., $|\lambda_1| = |\lambda_2| = 1$) and form a conjugate pair ($\lambda_2 = \overline{\lambda_1}$). When $\theta \neq 0$ and $\theta \neq \pi$, the eigenspaces associated with these eigenvalues are

$$\mathrm{eig}_Q(\lambda_1) = \mathrm{span}\left(\begin{bmatrix} i \\ 1 \end{bmatrix}\right), \quad \text{and} \quad \mathrm{eig}_Q(\lambda_2) = \mathrm{span}\left(\begin{bmatrix} -i \\ 1 \end{bmatrix}\right).$$

These eigenspaces are orthogonal: for $x_1 \in \mathrm{eig}_Q(\lambda_1)$ and $x_2 \in \mathrm{eig}_Q(\lambda_2)$, we have $x_1^* x_2 = 0$. Also, the eigenvectors are not real.

When $\theta = 0$, the matrix $Q = I_2$, and when $\theta = \pi$, the matrix $Q = -I_2$. These matrices only have one eigenvalue, namely, 1 and -1, respectively. Moreover, all the eigenvectors are real.

The difference between the cases $\theta \notin \{0, \pi\}$ and $\theta \in \{0, \pi\}$ makes sense intuitively since Givens rotators represent rotations in the plane. A rotation changes the direction of any real vector unless the angle θ is 0 or π. Thus, no real vector keeps its direction after multiplication by Q when $\theta \notin \{0, \pi\}$.

Example 37.5. (Eigenvalues of a 3×3 rotation matrix)

Let Q be a real 3×3 orthogonal matrix ($QQ^T = I_3 = Q^TQ$) with $\det(Q) = 1$. Let us show that $\lambda = 1$ is an eigenvalue of Q. Using $\det(Q^T) = \det(Q) = 1$, we have

$$\det(I_3 - Q) = \det(Q^T)\det(I_3 - Q) = \det(Q^T - Q^TQ) = \det(Q^T - I_3).$$

Since $(Q^T - I_3)^T = Q - I_3$ and transposing a matrix does not change its determinant, we have

$$\det(I_3 - Q) = \det(Q^T - I_3) = \det(Q - I_3) = \det(-(I_3 - Q)) = -\det(I_3 - Q),$$

where the last equality follows from the odd size of Q and properties of determinants. This shows that $\det(I_3 - Q) = 0$. Therefore, $\lambda = 1$ is an eigenvalue of Q.

From Proposition 36.10, the other two eigenvalues of Q must form a conjugate pair $e^{\pm i\theta}$, for some angle $0 \leqslant \theta < 2\pi$. Hence, the spectrum of Q is $\sigma(Q) = \{1, e^{i\theta}, e^{-i\theta}\}$.

While real matrices can have complex eigenvalues, symmetric matrices always have real eigenvalues.

Proposition 37.6. (Eigenvalues and eigenvectors of a symmetric matrix)

Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix. Then:

(a) Every eigenvalue of A is real, so $\lambda \in \mathbb{R}$ for all $\lambda \in \sigma(A)$. Moreover, the eigenvectors of A can be chosen to have real entries.
(b) Eigenvectors corresponding to distinct eigenvalues are orthogonal, so $x_1^T x_2 = 0$ if $x_1 \in \mathbb{R}^n$ and $x_2 \in \mathbb{R}^n$ are eigenvectors of A with different eigenvalues.

Proof Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix, i.e., $A^T = A$. Suppose λ is an eigenvalue of A, and let x be a corresponding eigenvector. Then, $Ax = \lambda x$. Multiplying both sides of $Ax = \lambda x$ on the left by the conjugate transpose[1] of x, we obtain

[1] At this stage, we cannot assume that x is real since we do not know that λ is real; so we use the complex conjugate transpose.

$$x^*Ax = \lambda x^*x = \lambda \|x\|_2^2.$$

Taking the complex conjugate of both sides gives

$$\overline{\lambda}\|x\|_2^2 = (x^*Ax)^* = x^*A^*x = x^*Ax = \lambda\|x\|_2^2,$$

where the third equality follows from $A^* = A^T = A$. Since $\|x\|_2^2 \neq 0$ (because x is an eigenvector), we can divide through by $\|x\|_2^2$, which gives $\overline{\lambda} = \lambda$. Thus, λ is real.

The eigenvector x satisfies the equation $(A - \lambda I_n)x = 0$, where $A - \lambda I_n$ is a real matrix. Since the matrix is real, the solution space of this system is spanned by real vectors. Thus, we can always choose x to be real.

Let x_1 and x_2 be eigenvectors of A corresponding to distinct eigenvalues λ_1 and λ_2, respectively. Since eigenvalues are real, we can assume x_1 and x_2 are real. Then, we have

$$\lambda_1(x_2^Tx_1) = x_2^T(\lambda_1 x_1) = x_2^T(Ax_1) = (Ax_2)^Tx_1 = \lambda_2(x_2^Tx_1).$$

Rearranging gives

$$(\lambda_1 - \lambda_2)(x_2^Tx_1) = 0.$$

Since $\lambda_1 \neq \lambda_2$, it follows that $x_2^Tx_1 = 0$, so x_1 and x_2 are orthogonal. □

Recall a (real) orthogonal projection matrix is a square matrix that satisfies $P^2 = P$ and $P^T = P$. From this definition, we can deduce some properties of its eigenvalues and eigenvectors.

Proposition 37.7. (Eigenvalues and eigenvectors of orthogonal projection matrices)

Let $P \notin \{0, I_n\}$ be an orthogonal projection matrix. Then:

(a) The eigenvalues of P are 0 and 1, so $\sigma(P) = \{0, 1\}$. The eigenspaces (over the reals) are

$$\text{eig}_P(1) = \text{col}(P), \quad \text{eig}_P(0) = \text{null}(P).$$

(b) Eigenvectors corresponding to different eigenvalues are orthogonal: if $x_1 \in \mathbb{R}^n$ and $x_2 \in \mathbb{R}^n$ are eigenvectors of P with eigenvalues 1 and 0, respectively, then $x_1^Tx_2 = 0$.

Proof Let $P \in \mathbb{R}^{n\times n}$ be an orthogonal projection matrix that is neither the identity matrix nor the zero matrix. Since $P^2 = P$, the column space of P is

$$\text{col}(P) = \{Pv : v \in \mathbb{R}^n\} = \{x \in \mathbb{R}^n : Px = x\} = \text{eig}_P(1).$$

Also, by definition, the nullspace of P is given by:

$$\text{null}(P) = \{x \in \mathbb{R}^n \mid Px = 0\} = \text{eig}_P(0).$$

Since $P \notin \{0, I_n\}$, we have $1 \leqslant \dim(\text{col}(P)) < n$. This implies

$$\dim(\text{col}(P)) \geqslant 1 \quad \text{and} \quad \dim(\text{null}(P)) = n - \dim(\text{col}(P)) \geqslant 1.$$

Thus, both eigenspaces are nontrivial, and $\{0, 1\} \subseteq \sigma(P)$.

To show $\sigma(P) \subseteq \{0, 1\}$, let x be an eigenvector of P with eigenvalue λ. From $Px = \lambda x$, multiplying both sides by P, we get

$$\begin{aligned} P^2x &= \lambda Px && \text{(multiply by } P\text{)} \\ Px &= \lambda^2 x && \text{(use } P^2 = P \text{ and } Px = \lambda x\text{)} \\ \lambda x &= \lambda^2 x && \text{(use } Px = \lambda x\text{)}. \end{aligned}$$

Since $x \neq 0$, this gives $\lambda^2 - \lambda = 0$, so $\lambda = 0$ or $\lambda = 1$.

Since we have shown that $\{0, 1\} \subseteq \sigma(P)$ and $\sigma(P) \subseteq \{0, 1\}$, it follows that $\sigma(P) = \{0, 1\}$.

To prove orthogonality, note that eigenvectors with eigenvalue 1 belong to $\text{col}(P)$, and eigenvectors with eigenvalue 0 belong to $\text{null}(P)$. Since P is symmetric, Theorem 32.6 shows that $\text{null}(P)$ is the orthogonal complement of $\text{col}(P)$. Therefore, eigenvectors corresponding to distinct eigenvalues are orthogonal. Alternatively, we can use Proposition 37.6, as P is symmetric. □

Certain matrix operations change the eigenvalues of a matrix A in predictable ways, as shown in the next proposition.

Proposition 37.8. (Eigenvalues and eigenvectors of A^k and $p(A)$)

Let $A \in \mathbb{C}^{n\times n}$. If λ is an eigenvalue of A with eigenvector x, then λ^k is an eigenvalue of A^k with eigenvector x.

More generally, given a polynomial $p(x) = \sum_{i=0}^{d} a_i x^i$, define $p(A) := \sum_{i=0}^{d} a_i A^i$. If λ is an eigenvalue of A with eigenvector x, then $p(\lambda)$ is an eigenvalue of $p(A)$ with eigenvector x.

Proof Let λ be an eigenvalue of A with eigenvector x. We prove the result for A^k by induction on k.

When $k = 1$, the result is straightforward since $A^1 = A$, and by definition, $Ax = \lambda x$.

Assume λ^{k-1} is an eigenvalue of A^{k-1} with eigenvector x, so

$$A^{k-1}x = \lambda^{k-1}x.$$

Multiplying both sides by A, we get

$$A^k x = A(A^{k-1}x) = A(\lambda^{k-1}x) = \lambda^{k-1}(Ax) = \lambda^k x.$$

Thus, x is an eigenvector of A^k with eigenvalue λ^k.

For the polynomial case, let $p(x) = \sum_{i=0}^{d} a_i x^i$, and assume λ is an eigenvalue of A with eigenvector x. Then,

$$p(A)x = \sum_{i=0}^{d} a_i A^i x = \sum_{i=0}^{d} a_i \lambda^i x = p(\lambda)x,$$

where the second equality follows from the result for A^k. This shows that x is an eigenvector of $p(A)$ with eigenvalue $p(\lambda)$. □

The converse of Proposition 37.8 does not hold in general. If x is an eigenvector of A^k with eigenvalue λ^k, x is not necessarily an eigenvector of A with eigenvalue λ. Let us see an example.

Example 37.9 Consider the 2×2 Givens rotator for the angle $\theta = \pi/2$:

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

The matrix A has eigenvalues $\lambda_1 = i$ and $\lambda_2 = -i$, with no real eigenvectors (see Example 37.4). However,

$$A^2 = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix},$$

has eigenvalues $\lambda_1 = \lambda_2 = -1$, and every (real) nonzero 2-dimensional vector is an eigenvector of A^2.

The converse of the eigenvalue result in Proposition 37.8 does hold. If μ is an eigenvalue of A^k, then $\mu = \lambda^k$ for some eigenvalue λ of A. The proof uses more advanced tools, such as the Schur decomposition (introduced in Section 36.4) or the Jordan canonical form (from Section 36.3); see Problems 37.26 and 37.27.

Given a matrix A and a scalar μ, the matrix $A - \mu I_n$ is called a **diagonal shift** of A. The next proposition shows that shifting a matrix shifts its eigenvalues but leaves the eigenvectors unchanged. At first, this might seem like a special case of Proposition 37.8 with $p(x) = x - \mu$. However, the result here is stronger because it is an "if and only if" statement.

Proposition 37.10. (Eigenvalues and eigenvectors of a shifted matrix)

Let $A \in \mathbb{C}^{n \times n}$ and $\mu \in \mathbb{C}$. Then x is an eigenvector of A with eigenvalue λ if and only if x is an eigenvector of $A - \mu I_n$ with eigenvalue $\lambda - \mu$.

Proof The proof is straightforward:

$$\begin{aligned} &Ax = \lambda x && \text{if and only if} \\ &Ax - \mu x = \lambda x - \mu x && \text{if and only if} \\ &(A - \mu I_n)x = (\lambda - \mu)x. \end{aligned}$$

This proves the result. □

37.2 Powers of matrices

Matrix powers A^k have many practical uses. For example, in Chapter 39, we will use them to compute eigenvalues of A. In Chapter 52, they will play an important role in the study of Markov chains.

However, when n is large, computing A^k by repeatedly multiplying by A becomes expensive. Each matrix multiplication takes $O(n^3)$ operations, so the total cost grows quickly.

Additionally, just by examining the entries of A, it is hard to predict how A^k will behave as k increases. Will it converge to a specific matrix? If not, how quickly will its entries grow?

Let us first recall what it means for a sequence of matrices to converge.

Definition 37.11. (Convergent sequence of matrices)

Let $A_1, A_2, \ldots$ and L be $n \times p$ real matrices. We say (A_k) **converges to** L (or has limit L if, for every i and j,

$$\lim_{k\to\infty} (A_k)_{ij} = L_{ij}.$$

In other words, each entry of A_k converges to the corresponding entry of L. When this holds, we write

$$\lim_{k\to\infty} A_k = L.$$

Example 37.12 Consider the matrix

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1/3 \end{bmatrix}.$$

Then,

$$\lim_{k\to\infty} A^k = \begin{bmatrix} \lim_{k\to\infty} 1^k & \lim_{k\to\infty} 0 \\ \lim_{k\to\infty} 0 & \lim_{k\to\infty} (1/3)^k \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

By contrast, consider the matrix

$$B = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}.$$

It is clear that the sequence $(B^k)_{k=0}^{\infty}$ does not converge since $\lim_{k\to\infty} 2^k$ does not exist.

If A is diagonalizable, computing A^k or analyzing $\lim_{k\to\infty} A^k$ becomes much simpler, as we will see next.

37.2.1 Matrix powers when A is diagonalizable

When a matrix A is diagonalizable, we can write it as $A = SDS^{-1}$, where D is a diagonal matrix containing the eigenvalues of A, and S is an invertible matrix with columns as the corresponding eigenvectors. This factorization makes calculating powers of A much easier.

For example, we have

$$A^2 = (SDS^{-1})(SDS^{-1}) = SD(S^{-1}S)DS^{-1} = SD^2S^{-1}.$$

Using induction on k, we get

$$A^k = (SDS^{-1})^k = SD^kS^{-1}.$$

This expression reduces the computation of matrix powers to calculating the powers of the diagonal matrix D:

$$D^k = \begin{bmatrix} \lambda_1^k & 0 & \cdots & 0 \\ 0 & \lambda_2^k & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n^k \end{bmatrix},$$

where $\lambda_1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of A.

The eigenvalues of a diagonalizable matrix A help determine how quickly the entries of A grow or decay when raised to a power, as shown in the following examples.

Example 37.13. (Change in the entries of A^k as k grows)

Consider the matrix $A = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$. This matrix has two eigenvalues: $\lambda_1 = 3$ and $\lambda_2 = 1$, with eigenvectors $x_1 = \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$ and $\left[\begin{smallmatrix} 1 \\ -1 \end{smallmatrix}\right]$, respectively. Since the eigenvalues are distinct, Corollary 36.35 implies that A is diagonalizable.

We can write

$$A = SDS^{-1} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}.$$

Using this factorization, we can compute A^k:

$$A^k = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 3^k & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 3^k + 1 & 3^k - 1 \\ 3^k - 1 & 3^k + 1 \end{bmatrix}.$$

The largest eigenvalue, $\lambda_1 = 3$, determines the growth rate of A^k. The entries of A^k grow approximately like 3^k.

For large k, we can approximate A^k as

$$A^k \approx \frac{3^k}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

If a diagonalizable matrix A has all eigenvalues with magnitudes less than 1, the entries of A^k get closer to zero as k increases. In this case, we say A^k **approaches the zero matrix as k goes to infinity**, and we write

$$\lim_{k\to\infty} A^k = 0.$$

This follows from the formula $A^k = SD^kS^{-1}$. Since the eigenvalues of A are the entries of D, and all of them are less than 1 in magnitude, the entries of D^k approach zero, causing A^k to approach the zero matrix [2].

Example 37.14 Consider the matrix $A = \begin{bmatrix} 0 & 1/2 \\ -1/4 & 3/4 \end{bmatrix}$. The eigenvalues of A are $\lambda_1 = 1/4$ and $\lambda_2 = 1/2$, with corresponding eigenvectors $x_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $x_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Using these, we can diagonalize A as

$$A = SDS^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1/4 & 0 \\ 0 & 1/2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

With this diagonalization, we can compute A^k for any positive integer k:

$$\begin{aligned} A^k = SD^kS^{-1} &= \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} (1/4)^k & 0 \\ 0 & (1/2)^k \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 2/4^k - 1/2^k & -2/4^k + 1/2^{k-1} \\ 1/4^k - 1/2^k & -1/4^k + 1/2^{k-1} \end{bmatrix}. \end{aligned}$$

As k goes to infinity, the terms $(1/4)^k$ and $(1/2)^k$ approach zero, which makes all the entries of A^k approach zero as well.

Now, let us consider the case of a diagonalizable matrix A where all eigenvalues have magnitudes less than or equal to 1. In this case, as k goes to infinity, the entries of A^k approach finite values, and we say the limit

$$\lim_{k\to\infty} A^k$$

exists. This happens because of the formula $A^k = SD^kS^{-1}$. The behavior of A^k is controlled by the diagonal matrix D^k. If an eigenvalue of A is less than 1 in magnitude, its corresponding entry in D^k gets closer to 0 as k increases. If an eigenvalue has a magnitude of exactly 1, its corresponding entry stays constant. Together, this makes A^k converge to a matrix.

[2] This result can also be extended to matrices that are not diagonalizable, but proving it requires more advanced tools, like the Jordan canonical form (see Section 37.2.3).

Example 37.15 Consider the matrix $A = \left[\begin{smallmatrix} 0 & 1 \\ -1/2 & 3/2. \end{smallmatrix}\right]$. The eigenvalues of A are $\lambda_1 = 1/2$ and $\lambda_2 = 1$, and the associated eigenvectors are $x_1 = \left[\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right]$ and $x_2 = \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$, respectively. Using these eigenvectors, we can diagonalize A as

$$A = SDS^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1/2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

Using this diagonalization, we can compute A^k for any positive integer k as

$$\begin{aligned} A^k = SD^kS^{-1} &= \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} (1/2)^k & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 1/2^{k-1} - 1 & -1/2^{k-1} \\ 1/2^k - 1 & -1/2^k + 2 \end{bmatrix}. \end{aligned}$$

Taking the limit as k goes to infinity, we get

$$\lim_{k \to \infty} A^k = \begin{bmatrix} -1 & 2 \\ -1 & 2 \end{bmatrix}.$$

However, if even one eigenvalue has a magnitude greater than 1, the corresponding entry in D^k grows without bound, and so do some entries of A^k. In this case, we say that the limit $\lim_{k \to \infty} A^k$ does not exist.

37.2.2 Applying a diagonalizable matrix repeatedly to vectors

Suppose a matrix A is diagonalizable, so it can be written as $A = SDS^{-1}$, where $D = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$ is a diagonal matrix with the eigenvalues of A, and $S = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}$ is an invertible matrix whose columns are the eigenvectors of A.

When we multiply a vector v by A, we can rewrite it as

$$Av = (SDS^{-1})v = SD(S^{-1}v).$$

The vector $c = S^{-1}v = \begin{bmatrix} c_1 & c_2 & \cdots & c_n \end{bmatrix}^T$ gives the coordinates of v relative to the eigenvector basis of A. Using this, we can express v as

$$v = Sc = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = c_1x_1 + c_2x_2 + \cdots + c_nx_n.$$

Substituting back into the formula for Av, we get:

$$Av = A\left(c_1x_1 + c_2x_2 + \cdots + c_nx_n\right) = c_1\lambda_1x_1 + c_2\lambda_2x_2 + \cdots + c_n\lambda_nx_n,$$

Hence, Av is a linear combination of the eigenvectors of A, each scaled by its eigenvalue.

The eigenvalues of A^k are $\lambda_1^k, \ldots, \lambda^k$ with associated eigenvectors $x_1, \ldots, x_n$, respectively. Thus,

$$A^k v = A^k(c_1 x_1 + \cdots + c_n x_n) = c_1 \lambda_1^k x_1 + \cdots + c_n \lambda_n^k x_n. \tag{37.1}$$

This shows that $A^k v$ is a linear combination of the eigenvectors of A, with each scaled by its eigenvalue raised to the power k.

Example 37.16 Figure 37.1 shows how a 2×2 diagonalizable matrix A transforms a vector v. The left panel shows the eigenvectors x_1 and x_2 of A, with eigenvalues $\lambda_1 = 2$ and $\lambda_2 = 0.5$, along with the vector $v = x_1 + 1.5x_2$. In the middle panel, we see $Av = \lambda_1 x_1 + 1.5\lambda_2 x_2$, and in the right panel, we see $A^2 v = \lambda_1^2 x_1 + 1.5\lambda_2^2 x_2$. With each multiplication by A, the component of v along x_1 doubles, while the component along x_2 is halved. This makes the direction of $A^k v$ increasingly aligned with the eigenvector x_1.

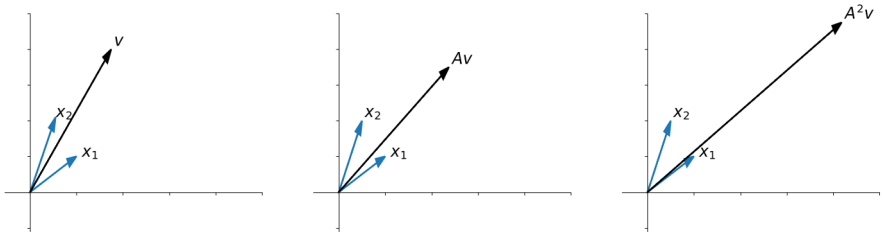


Fig. 37.1: The action of a diagonalizable matrix A on a vector v, showing how each multiplication stretches or shrinks v along A's eigenvectors.

Equation (37.1) shows that the behavior of $A^k v$ is determined by the eigenvalues and eigenvectors of A. In some cases, we can use this formula to predict how $A^k v$ behaves as k becomes very large. To make this idea more precise, we introduce the following definition.

Definition 37.17. (Dominant eigenvalue)

Let A be an $n \times n$ matrix with distinct eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_p$, ordered so that $|\lambda_1| \geqslant |\lambda_2| \geqslant \cdots \geqslant |\lambda_p|$. If $|\lambda_1| > |\lambda_2|$, we call λ_1 the **dominant eigenvalue** of A. The eigenvectors corresponding to λ_1 are called the **dominant eigenvectors** of A.

If the algebraic multiplicity of λ_1 is 1, λ_1 is said to be a **simple dominant eigenvalue**. If the algebraic multiplicity equals the geometric multiplicity of λ_1, then λ_1 is called a **regular dominant eigenvalue**.

If A has a simple dominant eigenvalue λ_1, the term $c_1\lambda_1^k x_1$ in $A^k v$ grows much faster than the terms involving the other eigenvalues as k increases. To understand this, we can express $A^k v$ as

$$\begin{aligned} A^k v =& c_1\lambda_1^k x_1 + c_2\lambda_2^k x_2 + \cdots + c_n\lambda_n^k x_n \\ =& \lambda_1^k \left(c_1 x_1 + c_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k x_2 + \cdots + c_n \left(\frac{\lambda_n}{\lambda_1}\right)^k x_n \right). \end{aligned}$$

As k increases, the terms involving $\frac{\lambda_i}{\lambda_1}$ for $i = 2, \ldots, n$ shrink to zero because $|\lambda_1| > |\lambda_i|$. For large k, this simplifies to

$$A^k v \approx c_1\lambda_1^k x_1.$$

Thus, as $k \to \infty$, the vector $A^k v$ becomes increasingly aligned with x_1. This is the same behavior illustrated in Example 37.16. Note that these observations hold for vectors whose coordinate $c_1 \neq 0$.

When λ_1 is a regular dominant eigenvalue, the direction of $A^k v$ approaches the dominant eigenspace, provided that v has a nonzero component in that eigenspace. In the simple dominant-eigenvalue case, this condition is $c_1 \neq 0$. More generally, when the dominant eigenspace has dimension greater than one, the component of v in that eigenspace, that is, the sum of all terms in the eigenvector expansion of v corresponding to the dominant eigenvalue, must be nonzero.

The fact that multiplying a vector by a diagonalizable matrix A makes it more parallel to the eigenvector corresponding to the dominant eigenvalue is the basis for several algorithms that compute eigenvalues and eigenvectors. One of the simplest and most commonly used methods is the power method, which we will discuss in Section 37.4.

37.2.3 Matrix powers when A is not diagonalizable

It has been easy to compute powers of a matrix A and to determine when $\lim_{k\to\infty} A^k$ exists if A is diagonalizable, but we know that not all square matrices are diagonalizable. What can we use in this case instead?

As we saw in Theorem 36.41, every square complex matrix is similar to a matrix in Jordan form. That is, for all $A \in \mathbb{C}^{n\times n}$, there exists an invertible matrix S and a Jordan matrix J such that $A = SJS^{-1}$. Using induction, it is easy to see that

$$A^k = SJ^kS^{-1}.$$

Recall that the Jordan canonical form J is a block diagonal matrix with upper bidiagonal blocks along the diagonal. So computing A^k reduces to computing powers of Jordan blocks, which is manageable in practice (see Problem 37.19). The real challenge is that finding the Jordan canonical form itself is not practical for large or general matrices. Later, in Section 37.3, we study a different canonical form, called the Schur form, that is more useful in numerical computations.

On the other hand, checking whether the limit $\lim_{k\to\infty} A^k$ exists is much easier. As in the diagonalizable case, the answer depends on the eigenvalues of A.

Theorem 37.18. (When the sequence of powers of a matrix is convergent)

Let A be a real square matrix. The limit $\lim_{k\to\infty} A^k$ exists if and only if one of the following holds:

(a) Every eigenvalue λ of A satisfies $|\lambda| < 1$; or
(b) 1 is an eigenvalue of A with the following properties: $|\lambda| < 1$ for every other eigenvalue and 1 is regular (i.e. the algebraic and geometric multiplicity of 1 are the same).

Proof We simplify the problem with a similarity transformation, which preserves eigenvalues. By the Jordan canonical form theorem (Theorem 36.41), there exists an invertible matrix S such that

$$A = SJS^{-1},$$

where J is the Jordan form of A. Raising both sides to the kth power and using induction gives

$$A^k = SJ^kS^{-1} \qquad (k = 1, 2, \ldots).$$

Because S and S^{-1} are constant, the limit

$$\lim_{k\to\infty} A^k = S\left(\lim_{k\to\infty} J^k\right)S^{-1}$$

exists if and only if $\lim_{k\to\infty} J^k$ exists.

Recall that the Jordan form J is block-diagonal: $J = \operatorname{diag}(J_1, \ldots, J_q)$. Each block J_i is a Jordan block of the form (all omitted entries are zero)

$$\begin{bmatrix} \lambda & 1 & & \\ & \lambda & \ddots & \\ & & \ddots & 1 \\ & & & \lambda \end{bmatrix},$$

where λ is an eigenvalue of A. Since $J^k = \operatorname{diag}(J_1^k, \ldots, J_q^k)$, the limit $\lim_{k\to\infty} J^k$ exists if and only if $\lim_{k\to\infty} J_i^k$ exists for every block. Therefore, we may focus on the behavior of a single Jordan block.

Let $J_r(\lambda)$ denote an $r \times r$ Jordan block with eigenvalue λ. We will show that $\lim_{k\to\infty} J_r(\lambda)^k$ exists exactly when either $|\lambda| < 1$ or $\lambda = 1$ and $r = 1$ (i.e., the eigenvalue 1 is *regular*).

One can show by induction (see Exercise 37.19) that for any $k \geqslant 1$

$$J_r(\lambda)^k = \begin{bmatrix} \lambda^k & \binom{k}{1}\lambda^{k-1} & \cdots & \binom{k}{r-1}\lambda^{k-r+1} \\ 0 & \lambda^k & \ddots & \vdots \\ \vdots & \vdots & \ddots & \binom{k}{1}\lambda^{k-1} \\ 0 & 0 & \cdots & \lambda^k \end{bmatrix}.$$

This formula leads to three cases we need to consider. If $|\lambda| < 1$, every entry tends to 0, so the block converges to the zero matrix. If $|\lambda| > 1$, the entries grow without bound, so the limit does not exist. The subtle case is $|\lambda| = 1$, which we examine next.

If $|\lambda| = 1$ but $\lambda \neq 1$, then λ^k always has modulus 1, yet its argument keeps rotating around the unit circle, each factor of λ adds the same non-zero angle. Because the argument never settles, λ^k does not converge as $k \to \infty$.

If $\lambda = 1$ and the block is larger than 1×1, the super-diagonal entries of $J_r(\lambda)^k$ are $\binom{k}{1}\lambda^{k-1} = k$. These grow without bound as $k \to \infty$, so the block, and therefore J^k, does not converge.

The block converges only in the special case $\lambda = 1$ with block size 1×1; in that situation $J_1(\lambda)^k$ is simply 1 for all k. □

Let us see two matrices that break the conditions in Theorem 37.18, and whose powers A^k therefore fail to converge.

Example 37.19 Consider the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix},$$

a single Jordan block. The eigenvalue 1 has algebraic multiplicity 2 but geometric multiplicity 1, so 1 is not regular. In fact,

$$A^k = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix},$$

which grows without bound as k increases. Hence $\lim_{k\to\infty} A^k$ does not exist.

Example 37.20 Consider the matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Its eigenvalues are 1 and -1, so it does not have a dominant eigenvalue. The matrix is diagonalizable with Jordan form

$$J = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Because $J^k = J$ when k is odd and $J^k = I_2$ when k is even, the sequence J^k, and thus A^k, oscillates and does not converge.

When A is diagonalizable and has a simple dominant eigenvalue λ_1, we saw that, for every vector v having a nonzero component in the dominant eigendirection, the direction of $A^k v$ approaches the line spanned by a dominant eigenvector x_1. This still holds even when A is not diagonalizable. However, proving this goes beyond the scope of this book.

37.2.4 Some applications of matrix powers

Beyond their role in algorithms for computing eigenvalues and eigenvectors, matrix powers have many other important uses, as mentioned earlier. Two important examples are their use in population dynamics and Markov processes. Next, we present two examples that illustrate these applications.

Example 37.21. (Predator-prey population model)

Imagine a forest where snakes prey on frogs. When there are more frogs, snakes have more food, causing their population to grow the next year. However, if the snake population increases, more frogs get eaten, leading to fewer frogs the following year.

Let x_n represent the snake population (in hundreds) and y_n the frog population (in tens of thousands) in year n. Suppose we can describe the relationship between the populations using the following equations

$$x_n = \frac{2}{5}x_{n-1} + \frac{3}{5}y_{n-1}, \tag{37.2}$$

$$y_n = -\frac{1}{5}x_{n-1} + \frac{6}{5}y_{n-1}. \tag{37.3}$$

If the initial populations are $x_0 = 2$ (hundreds of snakes) and $y_0 = 4$ (tens of thousands of frogs) in year 0, what will the populations be in year n? How will the populations of snakes and frogs evolve over time?

We can write the population evolution equations for snakes and frogs using matrix-vector multiplication as $v_n = Av_{n-1}$, where

$$A = \frac{1}{5}\begin{bmatrix} 2 & 3 \\ -1 & 6 \end{bmatrix}, \qquad \text{and} \qquad v_n = \begin{bmatrix} x_n \\ y_n \end{bmatrix}.$$

The matrix A, called the *transition matrix*, describes how the snake and frog populations change from one year to the next.

We can calculate the populations in year n using matrix multiplication:

$$
\begin{aligned}
v_1 &= Av_0, \\
v_2 &= Av_1 = A^2 v_0, \\
v_3 &= Av_2 = A^3 v_0, \\
&\vdots \\
v_n &= Av_{n-1} = A^n v_0,
\end{aligned}
$$

where $v_0 = \left[\begin{smallmatrix} 2 \\ 3 \end{smallmatrix}\right]$ gives the initial populations. We can use this matrix form to study the populations over time and understand their long-term behavior.

The transition matrix A has two distinct eigenvalues: $\lambda_1 = 1$ and $\lambda_2 = \frac{3}{5}$. The corresponding eigenvectors are $x_1 = \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$ and $\left[\begin{smallmatrix} 3 \\ 1 \end{smallmatrix}\right]$. Since A has distinct eigenvalues, it is diagonalizable. We can write

$$
A = SDS^{-1} = \begin{bmatrix} 1 & 3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 3/5 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 1 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & 3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 3/5 \end{bmatrix} \begin{bmatrix} -1/2 & 3/2 \\ 1/2 & -1/2 \end{bmatrix}.
$$

Using this factorization, we can compute A^n:

$$
A^n = \begin{bmatrix} 1 & 3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & (3/5)^n \end{bmatrix} \begin{bmatrix} -1/2 & 3/2 \\ 1/2 & -1/2 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} + \frac{3}{2}\left(\frac{3}{5}\right)^n & \frac{3}{2} - \frac{3}{2}\left(\frac{3}{5}\right)^n \\ -\frac{1}{2} + \frac{1}{2}\left(\frac{3}{5}\right)^n & \frac{3}{2} - \frac{1}{2}\left(\frac{3}{5}\right)^n \end{bmatrix}.
$$

The populations of snakes and frogs at year n are then

$$
v_n = A^n v_0 = \begin{bmatrix} -\frac{1}{2} + \frac{3}{2}\left(\frac{3}{5}\right)^n & \frac{3}{2} - \frac{3}{2}\left(\frac{3}{5}\right)^n \\ -\frac{1}{2} + \frac{1}{2}\left(\frac{3}{5}\right)^n & \frac{3}{2} - \frac{1}{2}\left(\frac{3}{5}\right)^n \end{bmatrix} \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 5 - 3\left(\frac{3}{5}\right)^n \\ 5 - \left(\frac{3}{5}\right)^n \end{bmatrix}.
$$

Figure 37.2 shows how the populations of snakes and frogs change over time. As n increases, both x_n and y_n approach 5. This means that, over time, if no conditions change, the snake population will stabilize at 500, and the frog population will stabilize at 50,000.

As time passes, the vector v_n becomes a multiple of the eigenvector x_1 of A, which corresponds to the dominant eigenvalue $\lambda = 1$. To understand why, let us write any vector $y = \left[\begin{smallmatrix} y_1 \\ y_2 \end{smallmatrix}\right]$ as a linear combination of the eigenvectors x_1 and x_2 of A:

$$
y = S(S^{-1}y) = \left(\frac{3y_2 - y_1}{2}\right) x_1 + \frac{y_1 - y_2}{2} x_2.
$$

Using this, we calculate $A^n y$:

$$
\begin{aligned}
A^n y &= \left(\frac{3y_2 - y_1}{2}\right) A^n x_1 + \frac{y_1 - y_2}{2} A^n x_2 \\
&= \left(\frac{3y_2 - y_1}{2}\right) x_1 + \frac{y_1 - y_2}{2} \left(\frac{3}{5}\right)^n x_2.
\end{aligned}
$$

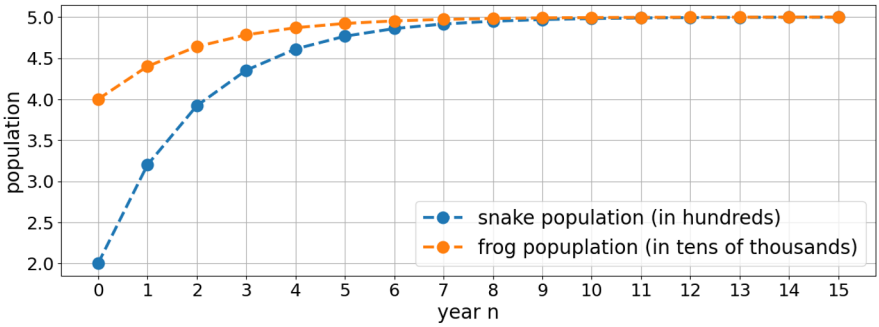


Fig. 37.2: Population growth of snakes and frogs over time in Example 37.21. The blue line shows the snake population, and the orange line shows the frog population. The initial populations in year 0 are 200 snakes and 40,000 frogs. Over time, the populations stabilize at 500 snakes and 50,000 frogs.

As n goes to infinity, the term involving x_2 vanishes because $\left(\frac{3}{5}\right)^n \to 0$, leaving

$$A^n y \xrightarrow[n\to\infty]{} \left(\frac{3y_2 - y_1}{2}\right) x_1.$$

In our example, the initial vector is $y = v_0 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$. Substituting v_0 into the formula gives

$$v_n = A^n v_0 \xrightarrow[n\to\infty]{} 5x_1.$$

This shows that as $n \to \infty$, the populations stabilize in the direction of x_1, scaled by a factor of 5.

Example 37.22. (Population growth model)

Let us consider a fish population divided into three age groups: young (age 0), middle-aged (age 1), and old (age 2). Each year, middle-aged fish produce 4 offspring, and old fish produce 8. For survival, three-quarters of the young fish make it to their second year, and one-third of the middle-aged fish survive to become old fish. After their third year, all old fish die, giving the fish a maximum lifespan of three years.

This type of model, which divides the population into discrete age groups, is called a Leslie model.

Suppose the initial population consists of 100 young fish, 60 middle-aged fish, and 20 old fish. We want to understand how the population evolves over time, considering the birth and death rates described above.

Let Y_n, M_n, and O_n represent the number of young, middle-aged, and old fish in year n. The population dynamics can be modeled by the following three equations

$$\begin{aligned} Y_n &= 4M_{n-1} + 8O_{n-1}, \\ M_n &= (3/4)Y_{n-1}, \quad \text{and} \\ O_n &= (1/3)M_{n-1}. \end{aligned}$$

We can also describe the population dynamics using matrix form:

$$\begin{bmatrix} Y_n \\ M_n \\ O_n \end{bmatrix} = \begin{bmatrix} 0 & 4 & 8 \\ 3/4 & 0 & 0 \\ 0 & 1/3 & 0 \end{bmatrix} \begin{bmatrix} Y_{n-1} \\ M_{n-1} \\ O_{n-1} \end{bmatrix}.$$

For example, the population after one year ($n = 1$) can be calculated as

$$\begin{bmatrix} Y_1 \\ M_1 \\ O_1 \end{bmatrix} = \begin{bmatrix} 0 & 4 & 8 \\ 3/4 & 0 & 0 \\ 0 & 1/3 & 0 \end{bmatrix} \begin{bmatrix} 100 \\ 60 \\ 20 \end{bmatrix} = \begin{bmatrix} 400 \\ 75 \\ 20 \end{bmatrix}.$$

In general, the population after n years can be calculated as

$$\begin{bmatrix} Y_n \\ M_n \\ O_n \end{bmatrix} = \begin{bmatrix} 0 & 4 & 8 \\ 3/4 & 0 & 0 \\ 0 & 1/3 & 0 \end{bmatrix}^n \begin{bmatrix} 100 \\ 60 \\ 20 \end{bmatrix}.$$

Figure 37.3 shows how the populations of young, middle-aged, and old fish change over time. In the long run, the fish population follows the behavior determined by the dominant eigenvalue of the transition matrix

$$A = \begin{bmatrix} 0 & 4 & 8 \\ 3/4 & 0 & 0 \\ 0 & 1/3 & 0 \end{bmatrix}.$$

This matrix has eigenvalues $\lambda_1 = 2$ and $\lambda_2 = \lambda_3 = -1$, but it is not diagonalizable. The dominant eigenvalue $\lambda_1 = 2$ corresponds to the eigenvector $x_1 = \begin{bmatrix} 16 & 6 & 1 \end{bmatrix}^T$. As n becomes large, the populations of young, middle-aged, and old fish stabilize into a fixed ratio of 16 : 6 : 1.

37.3 The Schur form and the spectral theorem for normal matrices

In Section 37.2 we saw how diagonalizing a matrix simplifies the computation of a matrix power A^k, and helps analyze the long-term behavior of A^k or $A^k v$ (for a given vector v) as k goes to infinity. But we know that not all matrices are diagonalizable. So, what other simple forms can we reduce a matrix to using similarity? The Jordan canonical form (see Section 36.3) is one option, but it is so sensitive to small perturbations and round-off errors that it is not very practical. Is there something else we could use? Even better, what if we could use a unitary matrix (orthogonal in the real case) for the similarity transformation? That way, instead of computing an inverse, we would only need to take the complex conjugate transpose.

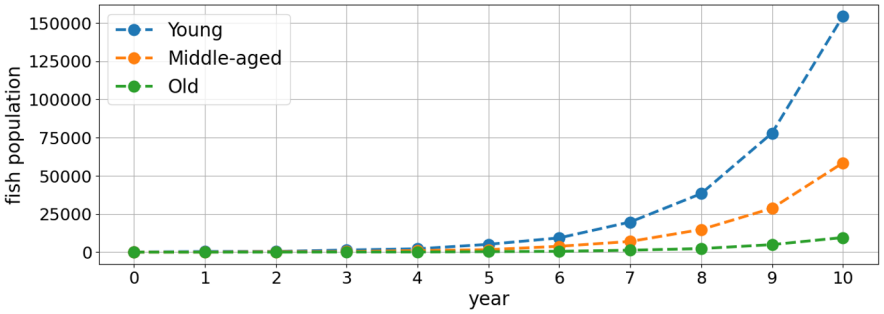


Fig. 37.3: Evolution of the fish population over 10 years. The initial population includes 100 young fish, 60 middle-aged fish, and 20 old fish. The graph shows how the population in each age group changes year by year.

In this section, we show that every matrix can be transformed into upper triangular form using a unitary similarity. Because similar matrices have the same eigenvalues, and the eigenvalues of a triangular matrix are just its diagonal entries (see Section 37.1), this transformation lets us read off the eigenvalues directly. This result is fundamental. In fact, the most widely used methods for computing eigenvalues are based on finding such a unitary similarity.

Theorem 37.23. (Schur factorization)

For any matrix $A \in \mathbb{C}^{n\times n}$, there exists a unitary matrix U (i.e., $UU^* = U^*U = I_n$) such that

$$T := U^*AU$$

is upper triangular. The matrix T is called the **Schur form of** A.

Proof We prove this result by induction on the size n of the matrix A.

When $n = 1$, any 1×1 matrix is already upper triangular. We can simply let $U = \begin{bmatrix}1\end{bmatrix}$, which is unitary.

Assume the theorem holds for all $(n-1)\times(n-1)$ matrices. Now consider an $n \times n$ matrix A. Every $n \times n$ matrix over the complex numbers has at least one eigenvalue. Let λ be an eigenvalue of A, and let x_1 be a corresponding eigenvector, normalized so that $\|x_1\|_2 = 1$.

Extend x_1 to an orthonormal basis $\{x_1, x_2, \ldots, x_n\}$ for $\mathbb{C}^n$, and construct the unitary matrix $U_1 := \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}$. Then, we have

$$U_1^* A U_1 = \begin{bmatrix} x_1^* \\ x_2^* \\ \vdots \\ x_n^* \end{bmatrix} A \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix} = \left[\begin{array}{c|ccc} x_1^* A x_1 & x_1^* A x_2 & \cdots & x_1^* A x_n \\ \hline x_2^* A x_1 & x_2^* A x_2 & \cdots & x_2^* A x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_n^* A x_1 & x_n^* A x_2 & \cdots & x_n^* A x_n \end{array}\right]$$

$$= \left[\begin{array}{c|ccc} \lambda & x_1^* A x_2 & \cdots & x_1^* A x_n \\ \hline 0 & x_2^* A x_2 & \cdots & x_2^* A x_n \\ \vdots & \vdots & \ddots & \vdots \\ 0 & x_n^* A x_2 & \cdots & x_n^* A x_n \end{array}\right] = \left[\begin{array}{c|c} \lambda & \star \\ \hline 0 & B \end{array}\right],$$

where λ is the eigenvalue of A, B is an $(n-1) \times (n-1)$ matrix, and $\star$ represents entries that are not relevant to the argument.

By the induction hypothesis, there exists a unitary matrix U_2 such that $U_2^* B U_2$ is upper triangular. Define the matrix

$$U := U_1 \begin{bmatrix} 1 & 0 \\ 0 & U_2 \end{bmatrix},$$

which is unitary because it is the product of two unitary matrices. Then,

$$U^* A U = \begin{bmatrix} 1 & 0 \\ 0 & U_2^* \end{bmatrix} \begin{bmatrix} \lambda & \star \\ 0 & B \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & U_2 \end{bmatrix} = \begin{bmatrix} \lambda & \star \\ 0 & U_2^* B U_2 \end{bmatrix}.$$

Since $U_2^* B U_2$ is upper triangular, $U^* A U$ is upper triangular as well. □

The Schur form of a matrix A has the eigenvalues of A along its main diagonal. In Section 37.4.4, we will introduce an algorithm that transforms A into a sequence of matrices $A_0, A_1, \ldots$ that eventually converges to the Schur form. This provides a practical method for computing the eigenvalues of A.

Now that we know every matrix can be transformed into upper triangular form using a unitary similarity, we ask: when is the Schur form actually diagonal? In other words, which matrices can be diagonalized by a unitary similarity transformation? To answer this, we begin with a definition.

Definition 37.24. (Normal matrices)

A matrix A is called **normal** if it satisfies $AA^* = A^*A$. In simpler terms, A is normal if it commutes with its conjugate transpose.

The family of normal matrices includes several important types of matrices. For example, it contains unitary ($U^{-1} = U^*$) and orthogonal ($Q^{-1} = Q^T$, with Q real) matrices; Hermitian ($A^* = A$) and symmetric ($A^T = A$, with A real) matrices. Notice that diagonal matrices and orthogonal projection matrices ($P^T = P$ and $P^2 = P$) are also normal since they are symmetric.

Normal matrices are the ones that can be diagonalized using a unitary similarity transformation.

Theorem 37.25. (The spectral theorem)

Let $A \in \mathbb{C}^{n\times n}$. Then A is normal if and only if there exists a unitary matrix U and a diagonal matrix D such that $D = U^*AU$.

Proof Assume A is normal. Let $T = U^*AU$ be the Schur form of A, where U is unitary and T is upper triangular. Since $AA^* = A^*A$, it follows that $TT^* = T^*T$. A normal triangular matrix must be diagonal (as shown in Problem 37.35), so T is diagonal. Thus, A can be diagonalized by a unitary similarity transformation.

Conversely, assume there exists a diagonal matrix D and a unitary matrix U such that $D = U^*AU$. Then,

$$AA^* = (UDU^*)(UD^*U^*) = UDD^*U^* = UD^*DU^* = (UD^*U^*)(UDU^*) = A^*A,$$

where we use the fact that diagonal matrices are normal. Hence, A is normal. □

Many matrices that arise in applications have real entries. However, real matrices can have complex eigenvalues. To handle this, we need an alternative to the Schur form. For real matrices, if we want to keep the factorization real, we must relax the requirement of an upper triangular form to allow for 2×2 blocks that represent complex conjugate pairs.

Theorem 37.26. (The real Schur factorization)

For any matrix $A \in \mathbb{R}^{n\times n}$, there exists an orthogonal matrix $Q \in \mathbb{R}^{n\times n}$ (i.e., $QQ^T = Q^TQ = I_n$) such that

$$T = Q^TAQ = \begin{bmatrix} T_{11} & T_{12} & \cdots & T_{1m} \\ 0 & T_{22} & \cdots & T_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & T_{mm} \end{bmatrix},$$

where each block T_{ii} is either a real number (for real eigenvalues of A) or a 2×2 matrix (for complex conjugate eigenvalues). This matrix T is called the **real Schur form** of A.

Proof See Problem 37.32. □

If a real matrix A has only real eigenvalues (no complex conjugate pairs), its real Schur form T is upper triangular. If A is also normal ($AA^T = A^TA$), then T is diagonal, and A can be diagonalized with an orthogonal similarity transformation.

One important example of real normal matrices with real eigenvalues is symmetric matrices (recall Proposition 37.6).

Theorem 37.27. (Spectral theorem for real symmetric matrices)

If A is a real symmetric matrix, there exists an orthogonal matrix Q and a diagonal matrix D such that $D = Q^T AQ$.

Theorem 37.27 is important because symmetric matrices will be central to many of the applications we will study in the coming chapters.

37.4 Numerical methods for computing eigenvalues and eigenvectors

In many applications, we need accurate computations of eigenvalues and eigenvectors. This section presents several numerical methods for these tasks, along with their advantages and disadvantages. We will cover more advanced techniques in Chapter 38.

Here, we focus on methods for general matrices. While specialized algorithms exist for certain classes, such as symmetric matrices, they are not the main focus of this book. One important exception is discussed in Section 37.4.6.

37.4.1 Using the characteristic polynomial: the naive way

The first method that comes to mind for finding eigenvalues is solving the characteristic equation, as the eigenvalues are the roots of the characteristic polynomial. However, this approach is not practical for large matrices. First, we can only solve polynomials of small degree analytically. For polynomials of degree higher than four, it is impossible to find all the roots using basic operations like addition, subtraction, multiplication, division, and kth roots. Second, finding the roots of a polynomial is an *ill-conditioned problem*. Small changes in the coefficients can cause large changes in the roots, as shown in Example 37.28. This is a concern in practice because matrix entries, and therefore the coefficients of the characteristic polynomial, are often not exact. They may come from noisy measurements or rounding errors due to floating-point arithmetic. Even if the matrix entries are exact, computing the coefficients of the characteristic polynomial can still introduce round-off errors.

Example 37.28. (The Wilkinson polynomial)

Consider the polynomial[3]

$$p(x) = \prod_{i=1}^{20} (x - i),$$

whose roots are clearly $\{1, 2, \ldots, 20\}$. Now, consider a slightly perturbed version of this polynomial:

$$q(x) = p(x) - 2^{-23} x^{19}.$$

[3] This polynomial is now called the "Wilkinson polynomial", named after Dr. James H. Wilkinson, a British mathematician known for his work in Numerical Analysis.

The polynomial $q(x)$ differs from $p(x)$ only in the coefficient of x^{19}, and the change is extremely small. Despite this tiny change, the roots of $q(x)$ can shift significantly [4]. Table 37.1 shows the roots of $q(x)$, correct to 9 decimal places.

Real roots	Imaginary roots
1	$10.095226615 \pm 0.643500904$i
2	$11.793633881 \pm 1.652329728$i
3	$13.992358137 \pm 2.518830070$i
4	$16.730737466 \pm 2.812624894$i
4.999999928	$19.502439400 \pm 1.940330347$i
6.000006944	
6.000697234	
8.007267603	
8.917250249	
20.846908101	

Table 37.1: Roots of the polynomial $\prod_{i=1}^{20}(x-i) - 2^{-23}x^{19}$.

The smaller roots are only slightly affected by the change to $p(x)$, but starting with the seventh root, the differences become much bigger. Also, the roots 11 through 20 now form five complex conjugate pairs, while all the roots of $p(x)$ were real. Other small changes to $p(x)$ can cause even bigger shifts in the roots. Try perturbing the coefficients of x^{15} or x^{16} and see what happens!

Solving the characteristic equation directly to compute eigenvalues is usually such a poor approach that we often take the opposite route. Instead, we find the roots of polynomials by treating them as eigenvalues of carefully constructed matrices, as we show next.

Polynomial root finding and the companion matrix

Consider a monic[5] polynomial $p(x)$ of degree d given by

$$p(x) = x^d + \sum_{i=0}^{d-1} c_i x^i.$$

Now, suppose we want to find the roots of $p(x)$. We will do this by building a matrix whose characteristic polynomial is $p(x)$. The roots of the polynomial will then be the eigenvalues of this matrix.

The **companion matrix** of the polynomial $p(x)$ is a $d \times d$ matrix with the polynomial coefficients $c_{d-1}, \cdots, c_1, c_0$ on the first row, ones on the first subdiagonal, and zeros in the remaining entries:

[4] In 1984, Wilkinson described the personal impact of this discovery, saying: "Speaking for myself I regard it as the most traumatic experience in my career as a numerical analyst."

[5] A monic polynomial has a leading coefficient of 1. You can make any polynomial monic by dividing all its terms by the leading coefficient, which does not change its roots.

$$C_p = \begin{bmatrix} -c_{d-1} & -c_{d-2} & \dots & -c_1 & -c_0 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix}. \tag{37.4}$$

The characteristic polynomial of C_p is given by (see Problem 37.15)

$$\det(\lambda I_d - C_p) = p(\lambda).$$

This means that the eigenvalues of C_p are precisely the roots of $p(x)$. Because of this connection, most numerical libraries use the companion matrix to efficiently compute polynomial roots.

Since we cannot always find the eigenvalues of a matrix A analytically (using exact formulas), we focus on numerical algorithms, which we group into two categories: direct methods and iterative methods[6].

1. **Direct methods** aim to compute all the eigenvalues and eigenvectors of a matrix. They work by gradually transforming the matrix into a simpler form, often through factorizations, until the eigenvalues can be easily identified.

   Examples includes:

   - **The QR algorithm**, which systematically reduces the matrix to upper triangular form while preserving its eigenvalues.
   - **The Jacobi method**, which is specifically designed for symmetric matrices and computes eigenvalues by repeatedly applying rotations.

   Advantages of direct methods:

   - They work well for matrices of small to medium size.
   - They compute all eigenvalues and eigenvectors of the matrix.

   Disadvantages of direct methods:

   - **Expensive**: They require $O(n^3)$ operations for an $n \times n$ matrix, making them computationally intensive for large matrices.
   - **Memory-intensive**: They need the entire matrix to be stored and manipulated, which makes them impractical for large matrices.

2. **Iterative methods** focus on finding a few eigenvalues (usually the largest or smallest) by gradually refining guesses through an iterative process. They start with an initial guess (for an eigenvalue or eigenvector) and improve it step by step,

[6] In other areas of Numerical Analysis, "direct" and "iterative" methods are defined differently. Direct methods solve a problem in a fixed number of steps, while iterative methods gradually improve an approximation through repeated steps, getting closer to the solution with each iteration.

In this chapter, even the "direct" methods are iterative under the broader definition, as they repeatedly apply steps to progressively approach the exact solution.

often using matrix-vector multiplication to get closer to the desired eigenvalues and eigenvectors.

Examples include:

- **Power iteration method**: A simple method for finding the largest eigenvalue and its eigenvector.
- **Shift-and-invert iteration method:** Finds eigenvalues near a target value (not necessarily the largest or smallest).
- **Rayleigh quotient iteration method**: A faster refinement of power iteration that improves estimates for both eigenvalues and eigenvectors.
- **Arnoldi process**: Often used to find a few eigenvalues of large, sparse matrices by approximating the original with a smaller matrix.
- **Lanczos process**: A version of the Arnoldi method designed for symmetric matrices.

Advantages of iterative methods:

- **Efficient for large matrices**: Fewer operations per step, making them practical for large, sparse matrices.
- **Memory-efficient**: Do not need to store the entire matrix. We only need to perform matrix-vector multiplications with it.
- **Flexible**: Can be adjusted to find specific eigenvalues (e.g., smallest, largest, or those near a target).

Disadvantages of iterative methods:

- **Convergence issues**: May not work well if the initial guess is poor.
- **Partial spectrum:** Usually used to find only a few eigenvalues, not all.

In this chapter, we cover a few of these algorithms: the power method, shift-and-invert, and the QR algorithm. In Chapter 38, we will discuss the Rayleigh quotient iteration, orthogonal iteration, and the Arnoldi process. As noted earlier, we will not cover algorithms designed specifically for symmetric matrices. If you are curious and want to learn more about these algorithms, you can check [15, 26, 51].

37.4.2 The power iteration method

We begin with the simplest method: the **power method**. This is an iterative algorithm for finding the dominant eigenvalue of a matrix A and an associated eigenvector.

The method starts with an initial vector v, typically chosen at random. At each step, we multiply by A, generating what is known as the Krylov sequence[7]:

$$v,\ Av,\ A^2v,\ A^3v,\ \ldots,\ A^kv.$$

If A has a simple dominant eigenvalue, we will see that A^kv gradually aligns with an eigenvector corresponding to that dominant eigenvalue as k increases.

[7] Named after Alexei Krylov, a Russian applied mathematician and naval engineer.

To understand why the power method works, we start by assuming that the matrix $A \in \mathbb{C}^{n\times n}$ is diagonalizable. Let $x_1, x_2, \ldots, x_n$ be a basis of eigenvectors with corresponding eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$, ordered by decreasing magnitude:

$$|\lambda_1| > |\lambda_2| \geqslant \cdots \geqslant |\lambda_n|,$$

so that λ_1 is a simple dominant eigenvalue. If A is not diagonalizable, a similar argument using the Jordan canonical form leads to the same result.

Suppose we start with a randomly chosen vector $v \in \mathbb{C}^n$. Since the eigenvectors form a basis for $\mathbb{C}^n$, we can write v as a linear combination of the eigenvectors:

$$v = \sum_{i=1}^{n} c_i x_i, \quad \text{where } c_i \in \mathbb{C}.$$

By Proposition 37.8, we can express $A^k v$ as

$$A^k v = \sum_{i=1}^{n} c_i A^k x_i = \sum_{i=1}^{n} c_i \lambda_i^k x_i.$$

Factoring out λ_1^k gives

$$A^k v = \lambda_1^k \left(c_1 x_1 + \sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1}\right)^k x_i \right).$$

Since λ_1 is a simple dominant eigenvalue of A, i.e., $|\lambda_1| > |\lambda_2|$, then $\left(\frac{\lambda_i}{\lambda_1}\right)^k$ approaches 0 as k increases, for all $i > 1$. This implies that, as the process continues, the contributions from all eigenvectors except x_1 become negligible. Thus, for large k, we have

$$A^k v \approx c_1 \lambda_1^k x_1,$$

where c_1 is the component of v in the direction of x_1. Therefore, as long as $c_1 \neq 0$, $A^k v$ will eventually point in the same direction as x_1, the eigenvector associated with the dominant eigenvalue λ_1.

Example 37.29 Consider the matrix $A = \left[\begin{smallmatrix} 1.25 & 0 \\ 0.25 & 0.75 \end{smallmatrix}\right]$. The eigenvalues of A are $\lambda_1 = 1.25$ and $\lambda_2 = 0.75$, with associated eigenvectors $x_1 = \left[\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right]$ and $x_2 = \left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right]$, respectively. These eigenvectors and their associated eigenspaces are shown in Figure 37.4.

Starting with the vector $v = x_1 + x_2 = \left[\begin{smallmatrix} 2 \\ 2 \end{smallmatrix}\right]$, the power method generates the sequence

$$A^k v = \lambda_1^k x_1 + \lambda_2^k x_2 = 1.25^k x_1 + 0.75^k x_2, \qquad (k = 1, 2, \ldots).$$

Each multiplication by A amplifies the component in the direction of x_1 by a factor of 1.25, while shrinking the component in the direction of x_2 by a factor of 0.75. As

a result, $A^k v$ gets closer to the eigenspace associated with the dominant eigenvalue λ_1 as k increases.

Figure 37.4 also shows the first four iterations of the power method.

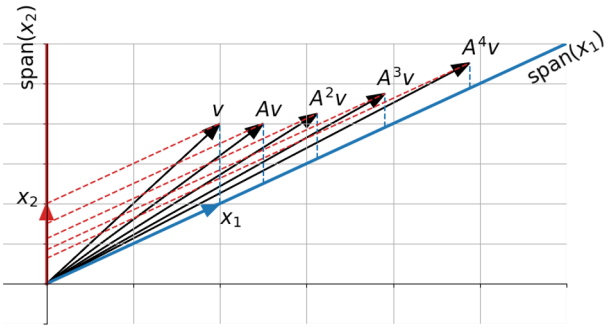


Fig. 37.4: Eigenvectors and power method iterations in Example 37.29.

The idea behind the power method is simple: if A has a simple dominant eigenvalue λ_1 and the starting vector v has a nonzero component in the corresponding eigendirection, then, as k increases, the direction of $A^k v$ approaches the line spanned by a dominant eigenvector x_1. However, the norm of $A^k v$ may grow too large or shrink too small, causing *overflow* or *underflow*[8]. To avoid this, the power method rescales the vector after each multiplication by A, usually so that it has unit norm. This controls the size of the iterates without changing their direction.

Algorithm 1 outlines the power method steps. The iteration continues until the norm of the residual is smaller than a tolerance or a maximum number of iterations is reached.

[8] Overflow occurs when a number is too large for the computer to represent, while underflow occurs when a number is too small to be distinguished from zero in the computer's arithmetic.

Algorithm 1 Power method

Inputs: An $n \times n$ matrix A with a simple dominant eigenvalue, a unit vector v_0 having a nonzero component in the dominant eigendirection, a tolerance $\epsilon > 0$, and a maximum number of iterations N.

for $k = 1, 2, \ldots, N$ **do**
  $w_k = Av_{k-1}$
  if $\|w_k\|_2 = 0$ **then**
    stop; the iteration cannot continue
  end if
  $v_k = \dfrac{w_k}{\|w_k\|_2}$
  $\lambda_k = v_k^* A v_k$
  $r_k = Av_k - \lambda_k v_k$
  if $\|r_k\|_2 \leqslant \epsilon$ **then**
    return λ_k, v_k
  end if
end for

Example 37.30 Consider the matrix $\left[\begin{smallmatrix} 7 & -6 \\ 3 & -2 \end{smallmatrix}\right]$. Its eigenvalues are $\lambda_1 = 4$ and $\lambda_2 = 1$ with associated eigenvectors $x_1 = \left[\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right]$ and $x_2 = \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$. To find an approximation of the dominant eigenvector and eigenvalue, we can use the power method. Let us choose the unit vector $v_0 = \left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right]$ as the starting vector. Then, the power method produces the following iterates:

$$w_1 = Av_0 = \begin{bmatrix} 7 \\ 3 \end{bmatrix} \quad \rightarrow \quad v_1 = \frac{w_1}{\|w_1\|_2} = 0.3939 \begin{bmatrix} 2.3333 \\ 1 \end{bmatrix}$$

$$w_2 = Av_1 = \begin{bmatrix} 4.0705 \\ 1.9696 \end{bmatrix} \quad \rightarrow \quad v_2 = \frac{w_2}{\|w_2\|_2} = 0.4356 \begin{bmatrix} 2.0667 \\ 1 \end{bmatrix}$$

$$w_3 = Av_2 = \begin{bmatrix} 3.6877 \\ 1.8294 \end{bmatrix} \quad \rightarrow \quad v_3 = \frac{w_3}{\|w_3\|_2} = 0.4444 \begin{bmatrix} 2.0159 \\ 1 \end{bmatrix}$$

$$w_4 = Av_3 = \begin{bmatrix} 3.6045 \\ 1.7987 \end{bmatrix} \quad \rightarrow \quad v_4 = \frac{w_4}{\|w_4\|_2} = 0.4465 \begin{bmatrix} 2.0039 \\ 1 \end{bmatrix}$$

$$w_5 = Av_4 = \begin{bmatrix} 3.5844 \\ 1.7913 \end{bmatrix} \quad \rightarrow \quad v_5 = \frac{w_5}{\|w_5\|_2} = 0.4470 \begin{bmatrix} 2.0010 \\ 1 \end{bmatrix}$$

$$w_6 = Av_5 = \begin{bmatrix} 3.5794 \\ 1.7895 \end{bmatrix} \quad \rightarrow \quad v_6 = \frac{w_6}{\|w_6\|_2} = 0.4472 \begin{bmatrix} 2.0002 \\ 1 \end{bmatrix}$$

Note that v_6 is a multiple of the vector $\left[\begin{smallmatrix} 2.0002 \\ 1 \end{smallmatrix}\right]$, which is close to the dominant eigenvector $x_1 = \left[\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right]$.

The power method approximates the dominant eigenvector x_1, but we often also need its corresponding eigenvalue λ_1. A way to estimate λ_1 is based on the fact that, when x is a good approximation of x_1, we expect $Ax \approx \lambda_1 x$. Multiplying both sides by x^*, the conjugate transpose of x, gives

$$x^*Ax \approx x^*\lambda_1 x = \lambda_1 \|x\|_2^2.$$

This suggests using $x^*Ax/\|x\|_2^2$ as an estimate of λ_1. The following proposition makes this idea precise.

Proposition 37.31. (The Rayleigh quotient)

Let $A \in \mathbb{C}^{n\times n}$, and let $v \in \mathbb{C}^n$ be nonzero. Then, the unique solution of the minimization problem

$$\min_{\lambda\in\mathbb{C}} \|Av - \lambda v\|_2^2$$

is the scalar given by

$$\lambda = \frac{v^*Av}{v^*v},$$

which is known as **the Rayleigh quotient**.

Proof The problem of minimizing $\|Av - \lambda v\|_2^2$ is a least squares problem, where Av plays the role of the "vector b", v plays the role of the "matrix A", and λ plays the role of the "unknown vector x". Since $v \neq 0$, this is a full rank least squares problem, and the unique solution satisfies the normal equations

$$v^*v\lambda = v^*Av.$$

Solving for λ gives $\lambda = v^*Av/v^*v$. □

Example 37.32 Let us revisit the matrix A from Example 37.30. After six iterations of the power method, we can estimate the dominant eigenvalue using the Rayleigh quotient $v_6^*Av_6/v_6^*v_6 = v_6^TAv_6$, which gives 4.0004. This is a good approximation of the exact value $\lambda_1 = 4$.

Measuring the accuracy of the approximate eigenvalue-eigenvector pair

When using the power method to approximate the dominant eigenvector and eigenvalue of a matrix A, it is important to monitor how quickly the iterates $v_1, v_2, \ldots$ converge and how accurate the estimated eigenvalue becomes. The difficulty is that the true eigenvalue and eigenvector are unknown.

To assess the quality of the current approximation (λ, v), we use an indirect measure called the **residual norm**:

$$\|Av - \lambda v\|_2.$$

This quantity tells us how close λ and v are to being an actual eigenvalue-eigenvector pair of A. Notice that the residual norm is zero precisely when $Av = \lambda v$, so v is an exact eigenvector and λ is its corresponding eigenvalue.

Our analysis of the power method assumed that A is diagonalizable, that there is a gap between the largest and second-largest eigenvalues (in other words, A has a simple dominant eigenvalue), and that the starting vector has a nonzero component in the direction of the dominant eigenvector. Now, let us see what happens when some of these assumptions are relaxed or removed.

Let us start by considering what happens when the starting vector has no component in the direction of the dominant eigenvector.

The power method almost always converges

The power method converges to the line spanned by x_1 if A has a simple dominant eigenvalue λ_1 and the starting vector v has a nonzero component c_1 in the direction of x_1. If v is chosen at random, this is almost always the case.

In the rare case where $c_1 = 0$, the iterates theoretically never approach the direction of x_1. But here is the surprising part: even when $c_1 = 0$, the method can still converge. Round-off errors introduced during the iterations usually create a small component in the direction of x_1, which is then amplified, allowing the method to eventually approximate the dominant eigenvector.

The next example shows this unexpected behavior.

Example 37.33 Consider the matrix

$$A = SDS^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

This matrix has eigenvalues $\lambda_1 = 1$ and $\lambda_2 = 0.5$ with associated eigenvectors $x_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $x_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

To see how the choice of starting vector affects the power method's convergence, let us test two initial vectors: a random vector v, and the eigenvector x_2, which lies entirely outside the direction of x_1. In this small example, we know the dominant eigenvector x_1, so we can directly measure how close the iterates v_k get to the subspace $\text{span}(x_1)$.

As we saw in Section 32.2, the distance from v_k to $\text{span}(x_1)$ is

$$\left\| v_k - \left(\frac{x_1^* v_k}{x_1^* x_1} \right) x_1 \right\|_2. \tag{37.5}$$

This is the distance between v_k and its orthogonal projection onto the eigenspace $\text{span}(x_1)$.

The left panel of Figure 37.5 shows how the distance to $\text{span}(x_1)$ (labeled as "error" in the figure) changes over the iterations for the two starting vectors. The right panel shows the same data on a logarithmic scale. With the random vector v as the starting guess, the power method converges quickly to the dominant eigenvector x_1. However, in theory, the power method should not converge to x_1 when starting

with x_2. This is indeed observed for the first $\sim$ 50 iterations, where the iterates remain in the subspace spanned by x_2. However, round-off errors introduce a small nonzero component in the direction of x_1, causing the iterates to eventually converge to the dominant eigenvector x_1.

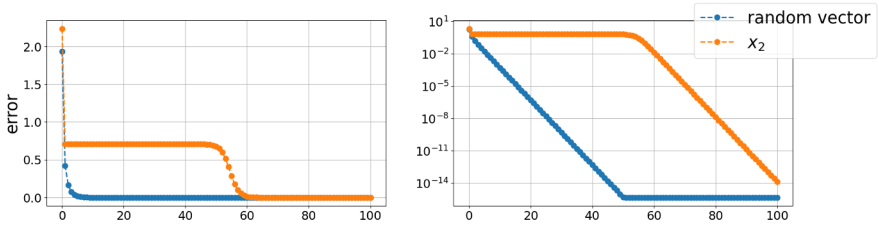


Fig. 37.5: Convergence of the power method for different starting vectors.

The power method almost always converges, but we also care about how fast it does. We want to know how quickly it reaches a good approximation of the eigenvector.

Convergence rate of the power method

Assume that A has a simple dominant eigenvalue λ_1, and let λ_2 be an eigenvalue of second-largest magnitude. The asymptotic convergence factor of the power method is

$$q = \left|\frac{\lambda_2}{\lambda_1}\right| < 1.$$

The normalized iterates become increasingly aligned with the dominant eigenvector. The misalignment decreases approximately like

$$\left|\frac{\lambda_2}{\lambda_1}\right|^k.$$

If q is close to 1, the power method converges slowly. If q is small, it converges rapidly. Thus, convergence is slow when the two largest eigenvalue magnitudes are close.

Example 37.34 Consider a matrix of the form

$$A = SDS^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}, \qquad \text{with } |\lambda_2| < 1,$$

which has eigenvalues $\lambda_1 = 1$ and λ_2 and associated eigenvectors $x_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $x_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. The matrix A has a simple dominant eigenvalue, so the power method will converge to the dominant eigenvector x_1.

Starting with a random vector, we measure the error after k steps, as in Example 37.33, using the distance from v_k to $\text{span}(x_1)$ given by (37.5).

The left panel of Figure 37.6 shows the error for $\lambda_2 = 0.9, 0.5, 0.2$. The right panel shows the same data on a logarithmic scale. Notice that the smaller the ratio $|\lambda_2/\lambda_1|$, the faster the convergence.

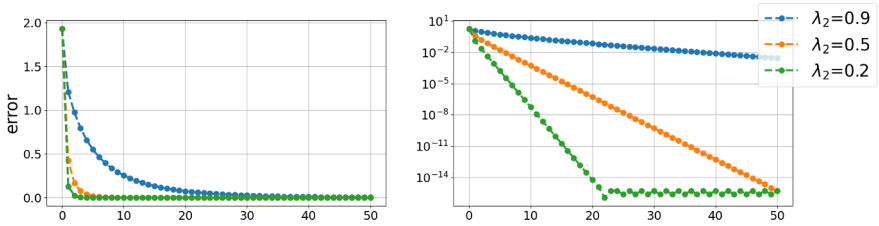


Fig. 37.6: Convergence of the power method for different values of the convergence factor $|\lambda_2/\lambda_1|$.

The power method without a dominant eigenvalue

If a matrix does not have a dominant eigenvalue, the power method might not converge. For example, real matrices may not have a dominant eigenvalue because complex eigenvalues come in conjugate pairs, λ and $\overline{\lambda}$. In these cases, the power method creates a sequence of vectors that oscillates between different directions instead of settling on one. The following example illustrates this behavior.

Example 37.35 Consider the matrix

$$A = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix},$$

which has eigenvalues $\lambda_1 = i$ and $\lambda_2 = -i$, both with modulus one. The corresponding eigenvectors are $x_1 = \begin{bmatrix} 1 \\ i \end{bmatrix}$ and $x_2 = \begin{bmatrix} 1 \\ -i \end{bmatrix}$. Although this matrix is diagonalizable (it has two distinct eigenvalues), it does not have a dominant eigenvalue. If we start the power method with $v_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{2}\begin{bmatrix} 1 \\ i \end{bmatrix} + \frac{1}{2}\begin{bmatrix} 1 \\ -i \end{bmatrix}$, the iterations produce the sequence of vectors

$$v_1 = Av_0 = \begin{bmatrix} 0 \\ -1 \end{bmatrix},$$
$$v_2 = Av_1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix},$$
$$v_3 = Av_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$
$$v_4 = Av_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \text{and}$$
$$v_5 = Av_4 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

Notice that $v_5 = v_1$, so the sequence v_1, v_2, v_3, v_4 keeps repeating. This means the sequence $v_1, v_2, \ldots$ produced by the power method does not have a limit. We conclude that the power method does not converge for this matrix A.

How can we tell if a matrix has a dominant eigenvalue and if the power method will work? One strategy is to run a few iterations and see if the vectors seem to converge. If they do but convergence is slow, the approximate eigenvalue from those iterations (even if not very accurate) can be used as a starting point for faster algorithms.

So far, we have assumed that A is diagonalizable and has a simple dominant eigenvalue to simplify the explanation of the power method. However, these assumptions can be relaxed.

Theorem 37.36. (Convergence of the power method)

Let $A \in \mathbb{C}^{n \times n}$ have distinct eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k$ with $|\lambda_1| > |\lambda_2| \geqslant \cdots \geqslant |\lambda_k|$ for some $k \leqslant n$. If λ_1 is regular, that is, its algebraic and geometric multiplicities are equal, then, for almost any starting vector v_0, the iterates of the power method converge to an eigenvector associated with λ_1.

An example will help illustrate this result, though we will not prove Theorem 37.36 in this book.

Example 37.37 Consider the matrices

$$A = \begin{bmatrix} 2 & 1 & -1 & 1 \\ 0 & 2 & -1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \text{and} \qquad B = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & -1 & 2 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Both matrices are upper triangular, so we can read their eigenvalues directly from the diagonals. The eigenvalues of A and B are the same: $\lambda_1 = 2$ (with algebraic multiplicity 2) and $\lambda_2 = 1$ (also with algebraic multiplicity 2).

For the matrix A, the algebraic multiplicity of the eigenvalue 2 is $\text{alg}_A(2) = 2$, but the geometric multiplicity is $\text{geom}_A(2) = 1$. This means A is not diagonalizable. For the matrix B, the algebraic and geometric multiplicities of the eigenvalue 2 are equal, $\text{alg}_B(2) = \text{geom}_B(2) = 2$, but for the eigenvalue 1, $\text{alg}_B(1) = 2 > \text{geom}_B(1) = 1$, so B is also not diagonalizable.

The eigenspaces of A and B associated with the eigenvalue 2 are

$$\text{eig}_A(2) = \text{span}\left\{\begin{bmatrix}1\\0\\0\\0\end{bmatrix}\right\}, \quad \text{and} \quad \text{eig}_B(2) = \text{span}\left\{\begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\\0\end{bmatrix}\right\}.$$

We run 50 iterations of the power method on both matrices A and B, starting from a random vector. Figure 37.7 shows the distance to the eigenspace of the eigenvalue 2, measured as the norm of the difference between the vector and its orthogonal projection onto that eigenspace. For the matrix B, the iterates v_k converge to an eigenvector associated with the eigenvalue 2. However, this does not happen for the matrix A, as shown by the blue dotted line in the graph.

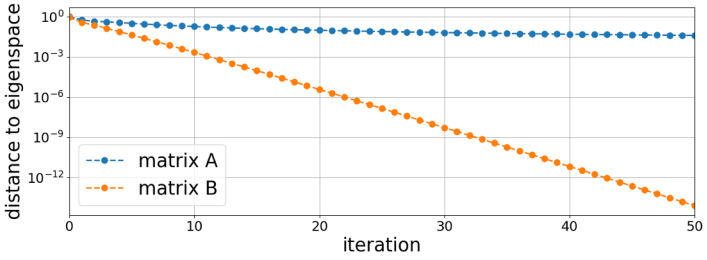


Fig. 37.7: The behavior of the power method for the non-diagonalizable matrices A and B defined in Example 37.37. The distance of v_k to the eigenspace associated with the eigenvalue 2 is plotted as a function of the number of iterations. While the power method does not converge for A, it converges to an eigenvector associated with the eigenvalue 2 for B.

In the previous example, the matrix B has a dominant eigenvalue that is regular but not simple. The power method converged to one specific eigenvector in the eigenspace $\text{eig}_B(2)$, though there are infinitely many. With a different starting vector, the method might have converged to a different eigenvector in the same eigenspace.

37.4.3 The shift-and-invert iteration method

The power method only gives the dominant eigenvalue and an associated eigenvector. But what if we want to find other eigenvalues and their corresponding eigenvectors? In this section, we introduce a variation called the **shift-and-invert iteration**

method. It can be used to find an eigenvalue of A closest to a chosen target $\mu \in \mathbb{C}$, along with a corresponding eigenvector. In particular, when $\mu = 0$ and A is invertible, the method targets an eigenvalue of A having smallest magnitude.

The shift-and-invert method is based on the following result.

Proposition 37.38. (Shift-and-invert)

Let $A \in \mathbb{C}^{n\times n}$ and $\mu \in \mathbb{C}$. Assume that μ is not an eigenvalue of A. Then, x is an eigenvector of A with eigenvalue λ if and only if x is an eigenvector of $(A - \mu I_n)^{-1}$ with eigenvalue $(\lambda - \mu)^{-1}$.

Proof Let $\lambda_1, \ldots, \lambda_n$ be the eigenvalues of A. By Proposition 37.10, the eigenvalues of $A - \mu I_n$ are $\lambda_1 - \mu, \ldots, \lambda_n - \mu$. Since μ is not an eigenvalue of A, the eigenvalues of $A - \mu I_n$ are all nonzero, and the matrix $A - \mu I_n$ is invertible by Proposition 37.10.

Then, by Propositions 37.10 and 36.24, we have

$$\begin{aligned} Ax &= \lambda x && \text{if and only if} \\ (A - \mu I_n)x &= (\lambda - \mu)x && \text{if and only if} \\ (A - \mu I_n)^{-1}x &= (\lambda - \mu)^{-1}x, \end{aligned}$$

which proves the proposition. □

If we assume A is diagonalizable, we can prove Proposition 37.38 in a simple and direct way. Let $A = SDS^{-1}$, where S is a nonsingular matrix and $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ is diagonal. Then,

$$(A - \mu I_n)^{-1} = S(D - \mu I_n)^{-1}S^{-1} = S \begin{bmatrix} \frac{1}{\lambda_1 - \mu} & & \\ & \ddots & \\ & & \frac{1}{\lambda_n - \mu} \end{bmatrix} S^{-1}.$$

This shows that the eigenvalues of A are "shifted and inverted", while the eigenvectors remain the same (since they come from the same matrix S). It also shows that $(A - \mu I_n)^{-1}$ is diagonalizable.

Any eigenvalue $\lambda \in \sigma(A)$ satisfying $|\lambda - \mu| = \min_{\nu\in\sigma(A)} |\nu - \mu|$ corresponds to an eigenvalue $\frac{1}{\lambda-\mu}$ of $(A - \mu I_n)^{-1}$ having largest magnitude.

Suppose that λ is the unique eigenvalue of A closest to μ, and that λ is simple (i.e., $\operatorname{alg}_A(\lambda) = \operatorname{geom}_A(\lambda) = 1$). Then $\frac{1}{\lambda-\mu}$ is the simple dominant eigenvalue of $(A - \mu I_n)^{-1}$. Provided that the starting vector has a nonzero component in the corresponding eigendirection, the power method can be applied to approximate its eigenvector. By Proposition 37.38, x will also approximate an eigenvector of A with eigenvalue λ. This is the basic idea behind the shift-and-invert iteration method.

Algorithm 2 shows the steps for the shift-and-invert method. Instead of directly computing $w_k = (A - \mu I_n)^{-1}v_{k-1}$, the algorithm solves the linear system $(A - \mu I_n)w_k = v_{k-1}$ to find w_k.

Algorithm 2 Shift-and-invert iteration

Inputs: A matrix $A \in \mathbb{C}^{n\times n}$, a shift $\mu \notin \sigma(A)$, a nonzero unit vector v_0, a tolerance $\varepsilon > 0$, and a maximum number of iterations N.

1: **for** $k = 1, 2, \ldots, N$ **do**
2: Solve

$$(A - \mu I_n) w_k = v_{k-1}$$

for w_k.
3: Normalize:

$$v_k = \frac{w_k}{\|w_k\|_2}.$$

4: Compute the eigenvalue approximation

$$\lambda_k = v_k^* A v_k.$$

5: **if** $\|Av_k - \lambda_k v_k\|_2 \leqslant \varepsilon$ **then**
6: return λ_k, v_k
7: **end if**
8: **end for**

When $\mu = 0$ and A is invertible, the shift-and-invert method is known as inverse iteration. It targets an eigenvalue of A having smallest magnitude and a corresponding eigenvector.

Remark 37.39. (Making the shift-and-invert method practical)

In Algorithm 2, we solve a linear system at each step. Factoring a dense $n \times n$ coefficient matrix from scratch would require about $O(n^3)$ operations at every iteration. However, since the matrix $A - \mu I_n$ remains fixed, we can factor it once using the LU factorization with partial pivoting before starting the iterations. This initial factorization costs about $O(n^3)$ operations.

Then, instead of solving the system from scratch each time, we use forward and backward substitution with the LU factors. These triangular solves require about $O(n^2)$ operations per iteration. This saves time by reusing the factorization throughout the iterations (recall Section 22.3.3).

The shift-and-invert method is especially useful for finding eigenvectors of a matrix A when we already have good approximations of its eigenvalues. For example, in Section 37.4.4, we will introduce an algorithm that computes all the eigenvalues of A at once. Once we have them, we can apply the shift-and-invert method to find the corresponding eigenvectors[9].

[9] There is a subtle issue: if μ is very close to an eigenvalue of A, then $A-\mu I_n$ can become nearly singular. Solving linear systems with nearly singular matrices can be tricky because they are highly sensitive to small round-off errors, which may lead to less accurate solutions of $(A-\mu I_n)w_{k-1} = v_{k-1}$. Still, this usually does not cause problems in practice when computing approximate eigenvectors. It is almost miraculous!

Example 37.40 Consider the matrix

$$A = \begin{bmatrix} 5 & 2 & 0 \\ 2 & 5 & 0 \\ -3 & 4 & 6 \end{bmatrix}.$$

Suppose you are given $\mu = 7.0005$ as a good approximation of an eigenvalue of A. In fact, you can verify that $\lambda = 7$ is indeed an eigenvalue of A with associated eigenvector $x = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T$.

We want to compute an eigenvector of A corresponding to this eigenvalue. To do this, we will apply the shift-and-invert method with $\mu = 7.0005$ and the starting vector $v_0 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^T$.

First, we solve the linear system $(A - \mu I_3)w_1 = v_0$ for the vector w_1, which gives

$$w_1 \approx \begin{bmatrix} -1000.1 & -999.9 & -998.6 \end{bmatrix}^T.$$

Next, we normalize w_1 to get

$$v_1 = \frac{w_1}{\|w_1\|_2} \approx \begin{bmatrix} -0.57779 & -0.57754 & -0.57682 \end{bmatrix}^T.$$

We then perform another shift-and-invert iteration by solving $(A - \mu I_3)w_2 = v_1$ for the vector w_2, which gives

$$w_2 \approx \begin{bmatrix} 1155.2 & 1155.2 & 1155.2 \end{bmatrix}^T.$$

Normalizing w_2, we obtain the approximate eigenvector and eigenvalue:

$$v_2 = \frac{w_2}{\|w_2\|_2} \approx \begin{bmatrix} 0.57735 & 0.57735 & 0.57735 \end{bmatrix}^T, \quad \text{and}$$
$$\lambda_2 \approx 7.0000002.$$

These are excellent approximations of the eigenvector $\frac{1}{\sqrt{3}} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T$ and its eigenvalue $\lambda = 7$.

In Section 38.3.1, we will introduce a variation of the shift-and-invert method that changes the shift at each iteration. This makes it converge much faster.

37.4.4 The QR algorithm

The best algorithms for finding all the eigenvalues of a matrix use an **eigenvalue-revealing factorization**. This means we factor the matrix in a way that makes the eigenvalues easy to read from one of the factors. For example, if A is diagonalizable, then in the factorization $A = SDS^{-1}$, the diagonal matrix D shows the eigenvalues. But as we have seen, not all matrices are diagonalizable. Another option is the Jordan canonical form (covered in Section 36.3). The problem is that the Jordan form is

very sensitive to small errors, like rounding, which can easily change it (see Example 36.44). That is why most modern algorithms avoid using it directly.

The QR algorithm solves these issues by computing the Schur factorization (or the real Schur factorization) of a matrix, as in Theorem 37.23. But how does this work in practice? How do we actually find the Schur form of a matrix?

Finding the Schur form of a matrix - the wrong way

Suppose we want to find an upper triangular matrix T and a unitary matrix Q such that $A = QTQ^*$, where $A \in \mathbb{C}^{n\times n}$. One idea might be to try directly triangularizing A using Householder reflectors[10] to zero out entries below the main diagonal. Householder reflectors are unitary, and the product of unitary matrices is also unitary.

For example, we could use a Householder reflector Q_1 to transform A so that the first column of Q_1A has zeros below the first entry:

$$Q_1A = \begin{bmatrix} \star & \star & \cdots & \star \\ 0 & \star & \cdots & \star \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \star & \cdots & \star \end{bmatrix}.$$

To make sure the transformation is a similarity (so the eigenvalues of A are preserved), we need to multiply Q_1A by Q_1^* on the right. However, multiplying by Q_1^* changes each column of Q_1A into a combination of all the columns. This process usually fills in the zeros we carefully introduced earlier, undoing our progress.

The right way to compute the Schur form of a matrix is not as obvious as the previous (and wrong) approach. It is called the QR method, and at first, it might feel like some kind of Linear Algebra magic. However, in Chapter 38, we will see that the QR method is actually just a clever reformulation of the subspace iteration method (a generalization of the power method).

The QR algorithm

Suppose we want to find all the eigenvalues of an $n \times n$ complex matrix A. The QR algorithm follows a simple idea. First, we compute a full QR factorization of A:

$$A = Q_0R_0.$$

Then, we form a new matrix by reversing the order of the factors:

$$A_1 := R_0Q_0.$$

Next, we factor A_1:

$$A_1 = Q_1R_1,$$

and repeat the process with $A_2 := R_1Q_1$, and so on.

---

[10] The complex version of the reflectors introduced in Chapter 27.

In short, the QR algorithm repeatedly factors a matrix as $A_k = Q_k R_k$, then sets $A_{k+1} := R_k Q_k$. Algorithm 3 shows the full procedure.

Algorithm 3 QR algorithm

Inputs:
An $n \times n$ matrix $A_0 = A$
while the algorithm has not converged **do**
Factor $A_k = Q_k R_k$.
Compute $A_{k+1} = R_k Q_k$.
end while

The next proposition shows that each matrix produced by the QR algorithm is unitarily similar to the previous one. This means all the iterates A_k have the same eigenvalues as the original matrix A.

Proposition 37.41. (Similarity of QR iterates)

The matrices $A, A_1, A_2, A_3, \ldots$ produced by the QR algorithm are similar to one another. As a result, the matrices $A_1, A_2, \ldots$, all have the same eigenvalues as the original matrix A.

Proof We need to show that A_{k+1} and A_k are unitarily similar for all $k \geqslant 0$. Since unitary similarity is transitive, this will imply that all iterates are unitarily similar to each other.

From the QR algorithm, we know $A_{k+1} = R_k Q_k$. Multiplying both sides on the left by Q_k, we get

$$Q_k A_{k+1} = Q_k R_k Q_k = A_k Q_k.$$

Rearranging, this gives

$$A_{k+1} = Q_k^* A_k Q_k.$$

Since Q_k is unitary, A_{k+1} and A_k are unitarily similar.

Because unitary similarities preserve eigenvalues, it follows that $A_1, A_2, A_3, \ldots$ all have the same eigenvalues as the original matrix A. □

The QR iterates are more than just unitarily similar to the original matrix. What is remarkable is that, for most matrices A, the iterates A_k actually converge to the Schur form of A. This means we can approximate the eigenvalues of A by looking at the diagonal entries of A_k as k increases. This convergence is guaranteed when the eigenvalues of A have distinct magnitudes[11], together with a technical condition on the eigenvector matrix.

[11] In practice, even when this assumption does not strictly hold, small rounding errors or numerical perturbations often break the symmetry enough to allow convergence. However, the convergence can be much slower in such cases.

Theorem 37.42. (Convergence of the QR algorithm)

Let $A \in \mathbb{C}^{n\times n}$ be diagonalizable, and let

$$A = X\Lambda X^{-1},$$

where

$$\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n), \qquad |\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|.$$

Suppose that every leading principal minor of X^{-1} is nonzero. Then the entries of A_k below the main diagonal tend to zero, while the diagonal entries tend to

$$\lambda_1, \lambda_2, \ldots, \lambda_n,$$

ordered by decreasing magnitude. Thus, as the iteration proceeds, the matrices A_k become upper triangular, with the eigenvalues of A on the diagonal.

The theorem does not necessarily imply that the entire sequence $\{A_k\}$ converges to one fixed upper triangular matrix. Some entries above the diagonal may continue to change because of sign or complex-phase oscillations.

Remark 37.43 The condition on the leading principal minors of X^{-1} is a technical assumption on the eigenvector matrix. Geometrically, for each $j = 1, \ldots, n-1$, it requires the coordinate subspace

$$\operatorname{span}\{e_1, \ldots, e_j\}$$

to intersect trivially with the invariant subspace

$$\operatorname{span}\{x_{j+1}, \ldots, x_n\},$$

where $x_1, \ldots, x_n$ are the eigenvectors of A ordered by decreasing eigenvalue magnitude. Equivalently, these two subspaces are complementary, so the first j coordinate directions are not contained in the invariant subspace associated with the smaller eigenvalues.

Remark 37.44 Recall that a square matrix admits an LU factorization without row exchanges if and only if all of its leading principal minors are nonzero. Thus, the assumption in the theorem is equivalent to requiring that the inverse eigenvector matrix X^{-1} have an LU factorization

$$X^{-1} = LU,$$

where L is unit lower triangular and U is upper triangular.

A full proof is beyond the scope of this chapter. However, Chapter 38 provides some intuition for why the QR algorithm works.

Not all matrices have eigenvalues with distinct magnitudes. For instance, Theorem 37.42 does not apply to real matrices with complex conjugate eigenvalues, since

these pairs have equal magnitude. As a result, the QR algorithm is not guaranteed to converge in every case.

Next, we present examples that illustrate Theorem 37.42, along with examples of matrices where the theorem's conditions fail to hold.

1. In the first example, we use a real matrix with three distinct real eigenvalues and show that the algorithm converges to an upper triangular matrix.
2. In the second example, we use a symmetric matrix with distinct eigenvalues. Here, the algorithm converges to a diagonal matrix.
3. In the third example, we use a matrix with imaginary eigenvalues and show convergence to the real Schur form, with a 2×2 diagonal block.
4. Finally, we use a real matrix with two real eigenvalues of the same magnitude and show that the QR algorithm does not converge.

Example 37.45 Consider the matrix

$$A = \begin{bmatrix} 5 & 2 & 0 \\ 2 & 5 & 0 \\ -3 & 4 & 6 \end{bmatrix}.$$

This matrix has eigenvalues $\lambda_1 = 7$, $\lambda_2 = 6$, and $\lambda_3 = 3$. We will apply the QR algorithm to A and see how it converges to its Schur form.

Figure 37.8 shows how the QR iterations converge. Figures (a), (b), and (c) plot the errors $|A_k[i,i] - \lambda_i|$ for $i = 1, 2, 3$. These are the distances between the diagonal entries of A_k and the eigenvalues of A, with the smallest eigenvalue $\lambda = 3$ converging first. Figure (d) shows the 2-norm of $\text{tril}(A_k)$, the strictly lower triangular part of A_k.

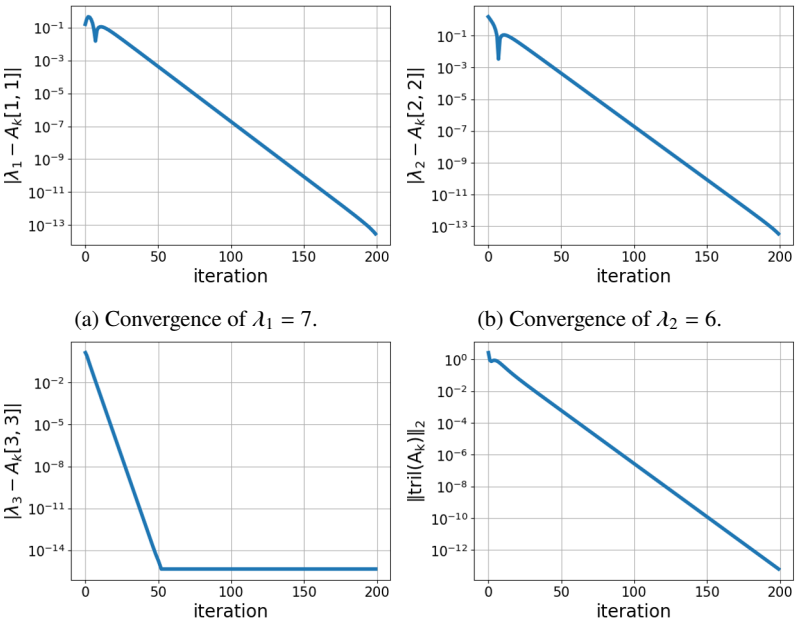
From the plots, we see that the diagonal entries of A_k are approaching the eigenvalues of A, while the entries of the strictly lower triangular part of A_k are all approaching zero. This confirms that the QR iterations A_k are converging to the Schur form of A.

After 50 iterations, for illustrative purposes, we obtain the matrix

$$A_{50} \approx \begin{bmatrix} 7.0005 & -0.7064 & -2.8549 \\ 0.0007 & 5.9995 & -4.0435 \\ -0.0000 & 0.0000 & 3.0000 \end{bmatrix}.$$

This matrix is nearly upper triangular, and its main diagonal entries are close approximations of the eigenvalues of A.

If A is symmetric and its eigenvalues have distinct magnitudes, the QR iteration converges to a diagonal matrix with the eigenvalues of A on the diagonal. This happens because the Schur form of a symmetric matrix is diagonal (see Theorem 41.3).



(a) Convergence of $\lambda_1 = 7$.

(b) Convergence of $\lambda_2 = 6$.

(c) Convergence of $\lambda_3 = 3$.

(d) Convergence of Schur form.

Fig. 37.8: Convergence of the QR algorithm for the matrix A in Example 37.45.

Example 37.46 Consider the symmetric matrix

$$A = \begin{bmatrix} 2 & 2 & 1 \\ 2 & 2 & -1 \\ 1 & -1 & 1 \end{bmatrix},$$

with eigenvalues $\lambda_1 = 4$, $\lambda_2 = 2$ and $\lambda_3 = -1$. After 10 iterations of the QR algorithm, we get

$$A_{10} \approx \begin{bmatrix} 4.00000 & 0.00113 & 0.00000 \\ 0.00113 & 1.99999 & -0.00414 \\ 0.00000 & -0.00414 & -0.99999 \end{bmatrix}.$$

This matrix is nearly diagonal, and the diagonal entries are already good approximations of A's eigenvalues.

After 20 iterations, the matrix has settled completely and becomes

$$A_{20} \approx \begin{bmatrix} 4 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

At this point, the QR algorithm has finished, and the eigenvalues of A are clearly shown on the main diagonal of A_{20}.

Example 37.47 Consider the real matrix

$$A = \begin{bmatrix} 2 & 9 & 0 & 2 \\ -1 & 2 & 1 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix},$$

with eigenvalues $\lambda_1 = 2 + 3i$, $\lambda_2 = 2 - 3i$, $\lambda_3 = 3$, and $\lambda_4 = -1$. Since $|\lambda_1| = |\lambda_2|$, Theorem 37.42 does not guarantee convergence of the QR algorithm. However, after 12 iterations of the QR algorithm, we obtain

$$A_{12} \approx \begin{bmatrix} 4.352 & 8.2354 & -0.7612 & -1.7703 \\ -1.7646 & -0.352 & -0.7727 & 0.8305 \\ 0 & 0 & 3 & -1 \\ 0 & 0 & 0 & -1 \end{bmatrix}.$$

In the top-left corner of the matrix, there is a 2×2 block:

$$\begin{bmatrix} 4.352 & 8.2354 \\ -1.7646 & -0.352 \end{bmatrix},$$

The eigenvalues of this block are good approximations of the imaginary eigenvalues $\lambda_1 = 2 + 3i$ and $\lambda_2 = 2 - 3i$ of A.

The QR algorithm works for most matrices, but for some, it does not converge, as shown in the next example.

Example 37.48 Consider the matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

which has eigenvalues $\lambda_1 = 1$ and $\lambda_2 = -1$.

A QR factorization of A is

$$A = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} = Q_0 R_0.$$

So, the next step in the QR algorithm gives

$$A_1 := R_0 Q_0 = A.$$

Thus, $A_0 = A_1 = A_2 = \cdots A_k$, so the QR algorithm does not converge to the Schur form of A in this case.

37.4.5 Practical considerations for the QR algorithm

Algorithm 3 has two main drawbacks. First, it is computationally expensive. Each step requires a QR factorization and a matrix multiplication, both taking $O(n^3)$ operations. Second, the algorithm converges slowly, or may not converge, when eigenvalues have the same or nearly the same magnitudes.

Making the QR algorithm practical is quite a journey, so we will not go into a full, detailed implementation here. Great books have been written on this topic [15, 26, 51]. However, in this section, we will introduce some ideas to improve the basic QR algorithm.

The first thing we will address is the computational cost of the QR algorithm steps.

How to reduce computational cost: The upper-Hessenberg reduction

We start by simplifying A into a form that is cheaper to work with. We use a unitary similarity transformation to preserve the eigenvalues of A. The new form must meet two requirements: Computing its QR factorization should be computationally cheap, and the form has to be preserved by the QR algorithm steps. Upper-Hessenberg form satisfies both requirements.

As we saw in Section 28.3.1, we can always transform A into an upper-Hessenberg matrix H using a unitary similarity transformation[12]:

$$H = QAQ^*.$$

So, before running the QR algorithm, we first reduce A to upper-Hessenberg form. This requires a bit of extra computational work at the start, but makes the overall process much more efficient, as we will see.

Let us start by proving that the QR algorithm preserves the upper-Hessenberg form at every step. In the proof, we use Givens rotations to compute the QR factorization of the Hessenberg matrix H. Although Householder reflections could also be used, Givens rotations are more efficient here since only one nonzero per column needs to be zeroed out.

Proposition 37.49. (The QR algorithm preserves the Hessenberg form)

If H is an $n \times n$ matrix in upper-Hessenberg form, then the QR iteration given by

[12] In Section 28.3.1, we showed how to reduce real matrices to upper Hessenberg form using orthogonal transformations. The same approach works for complex matrices using unitary transformations.

$$\begin{aligned} &\text{Factor} && H = QR, \\ &\text{Compute} && \widehat{H} := RQ \end{aligned}$$

produces an upper-Hessenberg matrix $\widehat{H}$.

Proof Without going too much into details, if H is upper-Hessenberg, we compute its QR factorization using a sequence of $n-1$ Givens rotators $Q_1, \ldots, Q_{n-1}$. Each rotator is applied from the left to reduce H to upper triangular form by zeroing out the subdiagonal entry below a diagonal entry. This gives

$$Q_{n-1} \cdots Q_1 H = R \quad \text{or equivalently} \quad H = \underbrace{Q_1^* \cdots Q_{n-1}^*}_{=:Q} R = QR.$$

To continue the QR iteration, we multiply Q and R in reverse order:

$$\widehat{H} = RQ_1^* \cdots Q_{n-1}^*.$$

Applying the Givens rotators from the right restores the Hessenberg form. (Try this with a small example to see how it works.) □

Reducing the matrix A to upper-Hessenberg form takes about $O(n^3)$ operations. While this is costly, it only needs to be done once. Afterward, all iterates H_k remain in upper-Hessenberg form, and each QR iteration takes roughly $6n^2$ operations (see Problem 37.45). The Hessenberg reduction saves a factor of $O(n)$ in computational cost!

Transforming A into upper-Hessenberg form H reduces the computational cost but does not resolve the slow convergence of the QR algorithm when eigenvalues have similar magnitudes.

To improve the convergence rate, we can incorporate shifts, similar to the shift-and-invert method. The key difference is that the shift can vary at each step.

QR algorithm with shifts

The shifted QR iteration modifies the basic QR algorithm by incorporating a shift at each step:

$$\begin{aligned} A_k - \mu_k I_n &= Q_k R_k \\ A_{k+1} &:= R_k Q_k + \mu_k I_n, \end{aligned} \tag{37.6}$$

where μ_k is the shift used at step k, and it can change at each iteration. The idea is simple: choosing shifts close to the eigenvalues helps the algorithm converge more quickly.

We will soon discuss how to choose good shifts. But first, let us show that, like the unshifted version, the shifted QR iteration keeps the iterates A_k unitarily similar to each other, so the eigenvalues remain unchanged at each step.

Proposition 37.50. (The shifted QR iterates are unitarily similar)

The matrices A_k and A_{k+1} in (37.6) are unitarily similar.

Proof Taking into account that $R_k = Q_k^*(A_k - \mu_k I_n)$, we have

$$\begin{aligned} A_{k+1} =& R_k Q_k + \mu_k I_n \\ =& Q_k^*(A_k - \mu_k I_n)Q_k + \mu_k I_n \\ =& Q_k^* A_k Q_k - \mu_k I_n + \mu_k I_n \\ =& Q_k^* A_k Q_k. \end{aligned}$$

Therefore, A_k and A_{k+1} are unitarily similar. □

Including the Hessenberg reduction and adding shifts to the QR algorithm gives us Algorithm 4.

Algorithm 4 Shifted Hessenberg QR algorithm

Inputs:
An $n \times n$ matrix A
Reduce A to upper-Hessenberg form H_0.
while the algorithm has not converged **do**
Choose a shift μ_k
Factor $H_k - \mu_k I_n = Q_k R_k$.
Set $H_{k+1} = R_k Q_k + \mu_k I_n$.
end while

Good shifts should be near an eigenvalue, but we do not know the eigenvalues! So, we need a way to estimate them. One approach is the Rayleigh quotient shift, which is based on the following idea. As the iterates H_k get closer to convergence, the entries at the bottom of the first subdiagonal tend to approach zero first. After a few iterations, we expect the matrix to be of the form

$$H_k = \begin{bmatrix} \times & \times & \cdots & \times & \times & \times \\ \times & \times & \cdots & \times & \times & \times \\ 0 & \times & \ddots & \vdots & \vdots & \vdots \\ \vdots & \ddots & \ddots & \times & \times & \times \\ 0 & \cdots & 0 & \times & \times & \times \\ 0 & \cdots & 0 & 0 & \epsilon & h_{n,n} \end{bmatrix},$$

where the value ϵ of the $(n, n-1)$ entry is small. If ϵ is sufficiently small, then the bottom-right entry $h_{n,n}$ will be a good approximation to an eigenvalue.

Definition 37.51. (The Rayleigh quotient shift)

At step k of the shifted Hessenberg QR algorithm, the **Rayleigh quotient shift** is defined as

$$\mu_k := e_n^T H_k e_n = H_k[n, n].$$

Notice that the Rayleigh quotient shift changes in each iteration step.

In the next example, we compare the QR algorithm with and without the Rayleigh shift.

Example 37.52 Consider the matrix

$$A = \begin{bmatrix} 5 & 2 & 0 \\ 2 & 5 & 0 \\ -3 & 4 & 6 \end{bmatrix},$$

with eigenvalues $\lambda_1 = 7$, $\lambda_2 = 6$, and $\lambda_3 = 3$.

We start by reducing A to upper-Hessenberg form H. Then, we apply 5 iterations of the QR algorithm without shifts. The resulting matrix is

$$H_5 = \begin{bmatrix} 6.6825 & -0.1448 & -1.6355 \\ -0.8825 & 6.5937 & -4.4646 \\ 0 & 0.2024 & 2.7238 \end{bmatrix}.$$

The bottom-right entry of H_k appears to be approaching the smallest eigenvalue, $\lambda_3 = 3$, but has not reached it yet.

From here, we will try two approaches. First, we will take five more steps of the unshifted QR algorithm, making it ten unshifted steps in total. Second, after the initial five unshifted steps, we will switch to the Rayleigh shift and run 5 iterations of the shifted QR algorithm.

Figure 37.9 shows the quantity $|3 - H_k[3, 3]|$ for both approaches. Notice that in both cases, $|3 - H_k[3, 3]|$ gets closer to zero, but the Rayleigh shift makes the convergence significantly faster!

In the previous example, we used the shifted QR algorithm to find a single eigenvalue. Now, let us see how to compute all of them.

Convergence of the QR algorithm and deflation

When we use the Rayleigh quotient shift, we expect the $(n, n-1)$ entry to slowly approach zero. However, as the QR algorithm runs and H_k gets closer to the Schur form of A, other subdiagonal entries may reach zero even faster. What should we do in that case? If any subdiagonal entry becomes zero, we can split the matrix into two smaller ones and work on each part separately. This works because the eigenvalues of an upper-Hessenberg matrix of the form

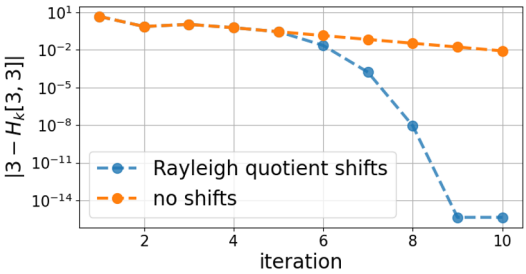


Fig. 37.9: Comparison of convergence between the unshifted QR algorithm and the QR algorithm with Rayleigh shift. The graph shows how the bottom-right entry of H_k approaches the smallest eigenvalue of A with each iteration.

$$H_k = \left[\begin{array}{ccc|ccc} \times & \cdots & \times & \times & \times & \cdots & \times \\ \times & \cdots & \times & \times & \times & \cdots & \times \\ & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ & & \times & \times & \times & \cdots & \times \\ \hline & & & 0 & \times & \cdots & \times \\ & & & & \times & \cdots & \times \\ & & & & & \ddots & \vdots \\ & & & & & & \times \end{array}\right] =: \left[\begin{array}{c|c} A_k & B_k \\ \hline 0 & C_k \end{array}\right]$$

are simply the combined eigenvalues of the two smaller Hessenberg matrices A_k and C_k. This step is called **deflation** and allows us to reduce the original problem into two smaller ones.

In practice, we do not wait for a subdiagonal entry to become exactly zero for deflation. Instead, we let the algorithm run until the entry is considered "small enough". A common criterion compares the subdiagonal entry to the entries above and to the right of it. We treat the subdiagonal $(j+1, j)$ entry as "numerically zero" if

$$|H_k[j+1, j]| \leqslant u \left(|H_k[j, j]| + |H_k[j+1, j+1]|\right),$$

where u is the unit roundoff, approximately 10^{-16} in double precision[13].

With deflation, we keep breaking the problem into smaller ones until we are left with 1×1 matrices (where the eigenvalue is just the single entry) and 2×2 matrices (where we can use the quadratic formula to find the two eigenvalues).

[13] **Unit roundoff** is the smallest positive number that makes $1 + u > 1$ in the computer's arithmetic. It reflects the limits of precision in floating-point calculations and helps us decide when very small numbers can be treated as zero.

Example 37.53 Consider the matrix

$$H = \begin{bmatrix} 5 & 2 & 0 & 1 \\ 1 & 4 & 3 & 0 \\ 0 & 2 & 3 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}.$$

Since H is already in Hessenberg form, we can skip the Hessenberg reduction step and directly apply the QR algorithm. After 5 iterations, the algorithm produces the matrix

$$H_5 = \begin{bmatrix} 6.77 & 1.15 & -0.31 & -0.54 \\ 0.09 & 4.58 & 0.37 & 0.13 \\ 0 & -0.782 & 0.33 & -0.39 \\ 0 & 0 & 0 & 2.315 \end{bmatrix}.$$

The $(4, 3)$ subdiagonal entry is now essentially zero (or small enough to treat as zero). This deflation shows that the bottom-right entry, 2.315, is one of the eigenvalues of H.

The remaining eigenvalues are those of the 3×3 submatrix $H_5[1:3, 1:3]$. We apply the QR algorithm to this smaller submatrix. After 4 iterations, we get

$$\begin{bmatrix} 6.81 & 1.11 & -0.069 \\ 0.57 & 4.47 & 1.16 \\ 0 & 0 & 0.397 \end{bmatrix}.$$

This gives us a second eigenvalue, 0.397.

The last two eigenvalues come from the 2×2 submatrix in rows and columns 1 and 2. We can find them using the quadratic formula, which gives 6.82 and 4.47.

The shifted QR algorithm with Rayleigh shifts does not always converge. A simple example is the matrix $A = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$, which has eigenvalues $\lambda_1 = 1$ and $\lambda_2 = -1$. In Example 37.48, we saw that the standard QR algorithm does not converge for this matrix–it simply produces the same matrix A at every iteration. Since the first Rayleigh shift is $A[2, 2] = 0$, the QR algorithm with Rayleigh shifts also fails to converge.

A trickier situation arises when trying to compute a pair of complex conjugate eigenvalues for a real matrix. With the Rayleigh shift strategy, the algorithm always produces real iterates and real shifts, which prevents it from using complex shifts close to the eigenvalues.

An alternative to the Rayleigh shift, and often a more popular approach, is the Wilkinson shift.

Definition 37.54. (The Wilkinson shift)

At step k of the QR iteration, consider the 2×2 bottom-right submatrix of H_k. The **Wilkinson shift** is the eigenvalue of this submatrix that is closest to $H_k[n, n]$. If the two eigenvalues are equally close, either one can be chosen.

The Wilkinson shift might sound like we are solving an eigenvalue problem just to find eigenvalues. But it only involves a 2×2 matrix, which is simple enough to do by hand.

Since the 2×2 bottom-right submatrix of H_k can have complex eigenvalues even for a real matrix, the Wilkinson shift makes it possible to converge to complex eigenvalues.

Example 37.55 Consider the matrix

$$A = \begin{bmatrix} -1 & 0 & 2 & 0 \\ -2 & 0 & 2 & 1 \\ -1 & 0 & 1 & 0 \\ -1 & -2 & 2 & 2 \end{bmatrix},$$

with eigenvalues $\lambda_1 = 1 + i, \lambda_2 = 1 - i, \lambda_3 = i$ and $\lambda_4 = -i$.

We start by reducing A to Hessenberg form H. Next, we apply the shifted QR algorithm, testing both Rayleigh shifts and Wilkinson shifts. With Rayleigh shifts, the convergence is painfully slow because the shifts are real and never close to an actual eigenvalue. After 50 iterations, however, the method finally converges to the Real Schur form:

$$H_{50} \approx \begin{bmatrix} 0.56 & -0.47 & -1.90 & -0.38 \\ 2.53 & 1.44 & 3.01 & -0.45 \\ 0 & 0 & -0.35 & 0.44 \\ 0 & 0 & -2.56 & 0.35 \end{bmatrix}.$$

With Wilkinson shifts, the algorithm converges much faster. After just five iterations, we obtain

$$\begin{bmatrix} 0.46 - 0.98i & -1.13 + 0.28i & -2.17 + 0.86i & -0.94 + 3.66i \\ -0.21 & 0.6 - 1.03i & 0.23 + 0.76i & -0.94 + 0.91i \\ 0 & 0.23 & 0.94 + 1.01i & 0.01 + 0.15i \\ 0 & 0 & 0 & i \end{bmatrix},$$

where one of the eigenvalues of A already appears in the bottom-right corner. At this point, we would continue by applying the QR algorithm to the upper-left 3×3 submatrix to find the rest of the eigenvalues.

If the original matrix is real, we often prefer to compute the real Schur form because complex arithmetic is usually more expensive. To avoid complex computations, it is better to use real shifts. But what if the matrix has complex conjugate eigenvalues? How can we target those eigenvalues while keeping all the arithmetic real? The answer is to take two steps at once using a pair of conjugate shifts. This approach is called the double shift strategy.

Double-shifted QR algorithm

The double-shifted QR iteration modifies the standard shifted QR algorithm by combining two shifts into a single iteration. Instead of performing one shift at a time, it takes two successive single-shift steps in each iteration:

$$\begin{aligned} A_k - \mu_k I_n &= Q_k R_k \\ B_k &= R_k Q_k + \mu_k I_n \\ B_k - \sigma_k I_n &= Q_{k+1} R_{k+1} \\ A_{k+1} &= R_{k+1} Q_{k+1} + \sigma_k I_n. \end{aligned}$$

With some algebraic manipulation, we can rewrite the double-shifted iteration as a single QR factorization and a similarity transformation:

$$\begin{aligned} &(A_k - \mu_k I_n)(A_k - \sigma_k I_n) = Q_k Q_{k+1} R_{k+1} R_k =: QR, \quad \text{and} \\ &A_{k+1} = Q_{k+1}^* Q_k^* A_k Q_k Q_{k+1} = Q^* A_k Q. \end{aligned}$$

The matrix $Q = Q_k Q_{k+1}$ is unitary because it is the product of two unitary matrices, and $R = R_{k+1} R_k$ is upper triangular because it is the product of two upper triangular matrices. This means we can take both shifts into account at once by

$$\begin{aligned} &\text{Factor} \quad (A_k - \mu_k I_n)(A_k - \sigma_k I_n) = QR, \\ &\text{Compute} \quad A_{k+1} = Q^* A_k Q. \end{aligned}$$

How does the double-shift strategy help maintain real arithmetic? When we use a pair of complex conjugate numbers, σ_k and $\bar{\sigma}_k$, as shifts, the product of the shifted matrices becomes

$$(A_k - \sigma_k I_n)(A_k - \bar{\sigma}_k I_n) = A_k^2 - (\sigma_k + \bar{\sigma}_k) A_k + \sigma_k \bar{\sigma}_k I_n.$$

Since $\sigma_k + \bar{\sigma}_k$ is twice the real part of σ_k and $\sigma_k \bar{\sigma}_k = |\sigma_k|^2$ is its modulus squared, the resulting matrix is

$$A_k^2 - 2\text{real}(\sigma_k) A_k + |\sigma_k|^2 I_n,$$

which is real when A_k is real. This allows us to perform all the computations using real arithmetic, even when the two shifts are imaginary.

Example 37.56 Let us revisit the matrix A from Example 37.55. We start by reducing A to Hessenberg form H. This time, we select shifts using the 2×2 submatrix in the bottom-right corner of H_k. If the eigenvalues of this submatrix are both real, we apply the Wilkinson shift. If the eigenvalues form a conjugate pair, we use both shifts and perform a double-shift step.

After just 10 iterations, we obtain the real Schur form

$$H_{10} \approx \begin{bmatrix} 0.89 & -2.61 & 3.4 & -0.19 \\ 0.39 & 1.11 & 1.07 & 0.53 \\ 0 & 0 & -0.26 & 0.41 \\ 0 & 0 & -2.59 & 0.26 \end{bmatrix}.$$

There is one tricky part with the double-shifted QR algorithm. Even if we start with a Hessenberg matrix H, it is not obvious how to perform the double-shift step in $O(n^2)$ operations. Why? Because at step k, we need to compute the matrix $H_k^2 - 2\text{real}(\sigma_k)H_k + |\sigma_k|^2 I_n$ and factor it, which is computationally expensive. Surprisingly, there is a way to handle this efficiently. Using a clever reformulation based on a result called the "implicit Q theorem", we can perform a double-shift step in $O(n^2)$ operations. But that is a story for another day!

37.4.6 The real symmetric QR algorithm

After all the complications we have seen with the QR algorithm, it is time for some good news: the QR algorithm works especially well for symmetric matrices.

First, we do not need to worry about complex eigenvalues. Real symmetric matrices always have real eigenvalues (see Proposition 37.6), so we can avoid complex shifts and skip the double-shift strategy entirely. Second, we can keep all computations real, replacing unitary similarities with orthogonal similarities.

The third advantage is that the Hessenberg reduction $H = Q^T AQ$ preserves the symmetry of A. And what does a symmetric Hessenberg matrix look like? It is tridiagonal! Even better, this tridiagonal structure is preserved throughout the QR iterations.

So, at each step k, the iterate has the form

$$H_k = \begin{bmatrix} a_1 & b_1 & & \\ b_1 & a_2 & \ddots & \\ & \ddots & \ddots & b_{n-1} \\ & & b_{n-1} & a_n \end{bmatrix}.$$

This means we do not need to store the full $n \times n$ matrix. It is enough to store just two vectors: one for the diagonal entries (a_i) and one for the off-diagonal entries (b_i).

The fourth advantage is that although the Rayleigh shift is always available, the Wilkinson shift is often a better choice for symmetric matrices because there is even an explicit formula for it.

Proposition 37.57. (Wilkinson shift formula for symmetric matrices)

Let

$$B = \begin{bmatrix} a_{n-1} & b_{n-1} \\ b_{n-1} & a_n \end{bmatrix}$$

denote the lower rightmost 2×2 submatrix of H_k. The Wilkinson shift at the kth step is given by

$$\mu_k = a_n - \frac{\text{sign}(\delta) b_{n-1}^2}{|\delta| + \sqrt{\delta^2 + b_{n-1}^2}},$$

where $\delta = \frac{a_{n-1} - a_n}{2}$. If $\delta = 0$, $\text{sign}(\delta)$ can be arbitrarily set equal to 1 or -1.

Proof See Problem 37.46. □

The symmetric QR algorithm is not only faster than the unsymmetric version because it works with a tridiagonal matrix instead of a Hessenberg one, but it also guarantees convergence [50]. With Wilkinson shifts, the $(n, n-1)$ entry converges to zero at a cubic rate[14]. In contrast, there is no general convergence theorem for the unsymmetric QR algorithm, and there are known examples where it fails to converge.

Example 37.58 Consider the symmetric matrix

$$A = \begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 2 \end{bmatrix}.$$

We apply the QR algorithm with Wilkinson shifts. Since A is already in Hessenberg form, we can skip the reduction step. Figure 37.10 shows how quickly $|H_k[4,3]|$ approaches zero as k increases. Notice how fast this happens! After just five iterations, the algorithm gives us the matrix

$$H_5 \approx \begin{bmatrix} 0.3926 & -0.1036 & 0 & 0 \\ -0.1036 & 1.4289 & -0.3549 & 0 \\ 0 & -0.3549 & 3.5604 & 0 \\ 0 & 0 & 0 & 2.618 \end{bmatrix}.$$

This reveals one of A's eigenvalues, 2.618, in the bottom-right corner.

Problems

37.1 Let $A = \left[\begin{smallmatrix} a & b \\ b & d \end{smallmatrix}\right]$ be a real symmetric 2×2 matrix. Find its eigenvalues in terms of a, b and d, and verify that both eigenvalues are real.

37.2 Find the eigenvalues and associated eigenvectors of the $n \times n$ permutation matrix

$$P = \begin{bmatrix} 0 & \cdots & 0 & 0 & 1 \\ 1 & & & & \\ & \ddots & & & \\ & & & 1 & \\ & & & & 1 \end{bmatrix},$$

[14] Cubic convergence means that the $(n, n-1)$ entry shrinks very quickly. Each step roughly reduces it by its cube.

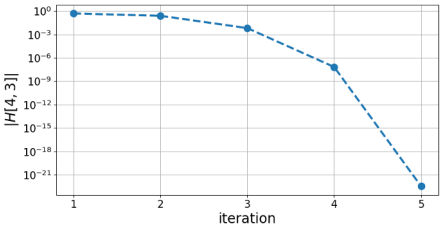


Fig. 37.10: Convergence of $|H_k[4,3]|$ to zero during the QR algorithm with Wilkinson shifts.

where all unspecified entries are zero.

37.3 An **antitriangular matrix** has the form

$$A = \begin{bmatrix} 0 & \cdots & 0 & a_{1n} \\ \vdots & \iddots & a_{2,n-1} & a_{2n} \\ 0 & \iddots & \iddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \quad \text{or} \quad A = \begin{bmatrix} a_1 & \cdots & a_{1,n-1} & a_{1n} \\ \vdots & \iddots & a_{2,n-1} & 0 \\ a_{n-1,1} & \iddots & \iddots & \vdots \\ a_{n1} & 0 & \cdots & 0 \end{bmatrix}.$$

Is it true that the eigenvalues of an antitriangular matrix are the entries $a_{1n}, a_{2,n-1}, \ldots, a_{n1}$ in its main antidiagonal?

37.4 In Proposition 37.1, we saw that the eigenvalues of a triangular matrix can be read directly from its diagonal. But what about the eigenvectors? Are they just as easy to compute?

37.5 Let $U \in \mathbb{C}^{n\times n}$ be a unitary matrix, i.e., $UU^* = I_n = U^*U$. Show that

(a) all the eigenvalues of U have magnitude 1, and
(b) eigenvectors associated to different eigenvalues are orthogonal.

37.6 An **involutory matrix** is a square matrix that satisfies $A^2 = I_n$ (i.e., A is its own inverse). Prove that its eigenvalues can only be 1 and -1.

37.7 A matrix A is **idempotent** if $A^2 = A$. Determine the possible eigenvalues of an idempotent matrix.

37.8 A matrix A is **nilpotent** if there is a positive integer k such that $A^k = 0$. Determine the possible eigenvalues of a nilpotent matrix.

37.9 Can you interpret geometrically the results from Example 37.5 regarding the eigenvalues of 3×3 rotation matrices?

37.10 Find the eigenvalues and eigenvectors of a 2×2 Householder reflector $Q = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ \sin(\theta) & -\cos(\theta) \end{bmatrix}$, and give a geometric interpretation of your results.

37.11 If the eigenvalues of an $n \times n$ matrix A are real and less than 1 (in magnitude), then show that $\det(I_n - A) > 0$.

37.12 Let A be an $n \times n$ matrix with entries in $\{0, 1\}$ and all eigenvalues positive. Prove that all eigenvalues of A are 1.

Hint: Use the Arithmetic Mean-Geometric Mean inequality to connect the trace and determinant.

37.13 Find the eigenvalues of a Householder reflector $Q = I_n - 2uu^T$, where $\|u\|_2 = 1$. Prove that Q is diagonalizable.

37.14 Prove that all Householder reflectors of the same size n are similar.

Hint: Householder reflectors are diagonalizable.

37.15 Consider a polynomial $p(x) = x^d + \sum_{i=0}^{d-1} c_i x^i$ and its companion matrix (37.4). Prove $\det(\lambda I_d - C_d) = p(\lambda)$.

37.16 Diagonalize

$$A = \begin{bmatrix} a & b-a \\ 0 & b \end{bmatrix},$$

and find a formula for A^k.

37.17 Let $A = \left[\begin{smallmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{smallmatrix}\right]$. Prove that A is diagonalizable and find a diagonalization for A^k for any positive integer k. Describe the behavior of A^k as k increases.

37.18 Consider the matrix

$$A = \begin{bmatrix} 0.6 & -0.4 & 0 \\ -0.4 & 0.6 & 0 \\ 0 & 0 & 0.5 \end{bmatrix}.$$

Find the limit of A^k as k goes to infinity.

37.19 Show that the nth power of the Jordan block $J_k(\mu)$ defined in (36.4) is given by

$$J_k(\mu)^n = \begin{bmatrix} \mu^n & \binom{n}{1}\mu^{n-1} & \binom{n}{2}\mu^{n-2} & \cdots & \binom{n}{k-1}\mu^{n-k+1} \\ & \mu^n & \binom{n}{1}\mu^{n-1} & \cdots & \binom{n}{k-2}\mu^{n-k+2} \\ & & \ddots & \ddots & \vdots \\ & & & \ddots & \vdots \\ & & & \mu^n & \binom{n}{1}\mu^{n-1} \\ & & & & \mu^n \end{bmatrix}.$$

37.20 Use the Jordan canonical form and Problem 37.19 to show that if the modulus of each eigenvalue of a matrix A is less than 1, then $A^k \to 0$ as k goes to infinity.

37.21 Consider an $n \times n$ matrix A with eigenvalues $\lambda_1, \ldots, \lambda_n$. Prove that $\text{trace}(A^k) = \lambda_1^k + \cdots + \lambda_n^k$.

Hint: Use either the Jordan canonical form of A (Theorem 36.41) or the Schur factorization of A (Theorem 37.23).

37.22 Four carbonated beverage companies, Bubbly Boar Beverages (BBB), Fizzy Flamingo Drinks (FFD), Sparkling Sloth Sodas (SSS), and Carbonated Camel Colas (CCC), operate in the city of Fizz, offering drinks like soda, sparkling water, and energy drinks. Their current market shares are as follows:

Company	Market share
Bubbly Boar Beverages	0.47
Fizzy Flamingo Drinks	0.10
Sparkling Sloth Sodas	0.04
Carbonated Camel Colas	0.39

The following table shows the fraction of customers that switch from one company to another in one year:

	$\mathbf{BBB}_{n-1}$	$\mathbf{FFD}_{n-1}$	$\mathbf{SSS}_{n-1}$	$\mathbf{CCC}_{n-1}$
$\mathbf{BBB}_n$	0.7	0.15	0.25	0.25
$\mathbf{FFD}_n$	0.2	0.75	0.1	0.15
$\mathbf{SSS}_n$	0.05	0.025	0.6	0.1
$\mathbf{CCC}_n$	0.05	0.075	0.05	0.5

The rows represent the company customers switch to, and the columns represent the company they switch from. For example, the entry 0.2 in the first column and second row indicates that 20% of BBB customers switch to FFD in one year.

Using this transition data, calculate and plot the market shares of the companies over time.

37.23 A social media platform has a collection of user accounts, each of which can be in one of two states: active or inactive. The states of the user accounts change at regular time intervals called time steps. Suppose that at the end of each time step, 20% of the inactive accounts become active, and 10% of the active accounts become inactive.

(a) Write down the transition matrix M for this system.
(b) Find the eigenvalues and associated eigenvectors of M.
(c) Show that the social media platform will eventually approach an equilibrium state, where the proportion of user accounts in each state becomes constant, and determine the equilibrium proportions.

37.24 Your friend Tom loves to procrastinate. At any given moment, he can be in one of three states: Studying, Watching Netflix, or Taking a Nap. The probabilities of transitioning between these states every 15 minutes are shown in the table below:

	Studying	Watching Netflix	Taking a nap
Studying	0.4	0.2	0.1
Watching Netflix	0.4	0.6	0.3
Taking a nap	0.2	0.2	0.6

Each column represents the current state, and each row represents the next state. For example, the probability of transitioning from Studying to Watching Netflix is the entry in the first column and second row, 0.4. The sum of each column is 1 because

the total probability of transitioning from any given state to all possible states is always 1.

Let v_n be the vector that contains the probabilities of being in the "Studying", "Watching Netflix", and "Taking a Nap" states at time step n, denoted by S_n, N_n, and T_n, respectively.

(a) If Tom starts the day studying for an exam, which vector represents the initial state?
(b) What are the probabilities that Tom will be studying, watching Netflix, or taking a nap 15 minutes later? 30 minutes later? 45 minutes later?
(c) Over time, what percentage of time does Tom spend studying, watching Netflix, and taking naps? You can solve this in one of two ways:

(c1) Use the fact that $\{1, 0.4, 0.2\}$ is the spectrum of the matrix

$$A = \begin{bmatrix} 0.4 & 0.2 & 0.1 \\ 0.4 & 0.6 & 0.3 \\ 0.2 & 0.2 & 0.6 \end{bmatrix}.$$

(c2) Graph the probability vectors for a moderate number of iterations and observe the vector they approach over time.

(d) Compare the vector obtained in the previous question with a dominant eigenvector of A.

37.25 Find by hand the Schur factorization of the matrices

$$A = \begin{bmatrix} 1 & 1 \\ -2 & -1 \end{bmatrix} \qquad \text{and} \qquad B = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 5 & 0 \\ 2 & 0 & 1 \end{bmatrix}.$$

37.26 Let $A \in \mathbb{C}^{n\times n}$, and let k be a positive integer.

(a) Prove that if $\lambda \in \sigma(A)$, then $\lambda^k \in \sigma(A^k)$.
(b) Use the Schur factorization theorem (Theorem 37.23) to prove that if $\mu \in \sigma(A^k)$, then there exists $\lambda \in \sigma(A)$ such that $\mu = \lambda^k$.
(c) Conclude that $\sigma(A^k) = \{\lambda^k : \lambda \in \sigma(A)\}$.

37.27 Let

$$p(z) = a_d z^d + a_{d-1} z^{d-1} + \cdots + a_1 z + a_0$$

be a polynomial, and let $A \in \mathbb{C}^{n\times n}$. Define

$$p(A) = a_d A^d + a_{d-1} A^{d-1} + \cdots + a_1 A + a_0 I_n.$$

(a) Prove that if $\lambda \in \sigma(A)$, then $p(\lambda) \in \sigma\big(p(A)\big)$.
(b) Use the Schur factorization theorem (Theorem 37.23) to prove that if $\mu \in \sigma\big(p(A)\big)$, then there exists an eigenvalue $\lambda \in \sigma(A)$ such that $\mu = p(\lambda)$.
(c) Conclude that

$$\sigma\big(p(A)\big) = p\big(\sigma(A)\big) = \{p(\lambda) : \lambda \in \sigma(A)\}.$$

37.28 The **Cayley-Hamilton theorem** states that every square matrix satisfies its characteristic equation $p_A(A) = 0$. Use the Schur factorization to prove the Cayley-Hamilton theorem.

37.29 (Every matrix is nearly diagonalizable) Let $A \in \mathbb{C}^{n\times n}$ be a non-diagonalizable matrix, and let $T = U^*AU$ be its Schur form. For any positive integer k, define the diagonal matrix $D_k = \frac{1}{k}\text{diag}(1, 2, \ldots, n)$.

(a) Find the eigenvalues of the matrix $A_k = A + UD_kU^*$ in terms of the eigenvalues of A.
(b) Show that for a large enough k, the matrix A_k has distinct eigenvalues and, thus, is diagonalizable.
(c) Show that $\|A - A_k\|_2 = \frac{n}{k}$.

Conclude that A is arbitrarily close to a diagonalizable matrix.

37.30 Compute by hand the real Schur factorization of the matrix

$$A = \begin{bmatrix} 1 & -2 & 2 \\ -1 & 1 & 1 \\ -2 & 0 & 3 \end{bmatrix}.$$

37.31 Let $A \in \mathbb{R}^{n\times n}$ have complex conjugate eigenvalues $\lambda = a + ib$ and $\bar{\lambda} = a - ib$, where $a, b \in \mathbb{R}$ and $b \neq 0$. Suppose $v = x + iy \in \mathbb{C}^n$ is an eigenvector of A corresponding to λ, where $x, y \in \mathbb{R}^n$. Prove that the subspace $\mathcal{S} = \text{span}(x, y)$ is a two-dimensional invariant subspace of A.

37.32 Using Problem 37.31, prove Theorem 37.26 by induction on n.

37.33 For $n \times n$ normal matrices A and B, is $A + B$ necessarily normal? What about AB?

37.34 Show that any matrix $A \in \mathbb{C}^{n\times n}$ can be written as the sum of a Hermitian matrix H and a skew-Hermitian matrix S. Then, show that A is normal if and only if H and S commute.

37.35 Consider a normal matrix A. Show that

$$\|A^*e_i\|_2^2 = \|Ae_i\|_2^2,$$

for $i = 1, \ldots, n$, and conclude that the ith row of A must have the same norm as the ith column. Use this fact to show that if A is, in addition, upper triangular, then it must be diagonal.

37.36 Prove that $\text{col}(A)$ is orthogonal to $\text{null}(A)$ for a normal matrix A.

37.37 Let A be a normal matrix. Show that A is Hermitian ($A^* = A$) if and only if the eigenvalues of A are real. Show that A is skew-Hermitian ($A^* = -A$) if and only if the eigenvalues of A are purely imaginary (i.e., have zero real part).

37.38 Consider an $n \times n$ real symmetric matrix A. If all eigenvalues of A are equal to 1, is A necessarily equal to the identity matrix? If all eigenvalues of A are equal to 0, is A necessarily equal to the zero matrix?

37.39 Consider the matrix $A = \left[\begin{smallmatrix} 5 & 8 \\ 8 & 5 \end{smallmatrix}\right]$. Find its eigenvalues and corresponding eigenvectors by hand. Starting with $q = \left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right]$, run several iterations of the normalized power method. At iteration k, compute the eigenvalue error $|\lambda_1 - q_k^T A q_k|$, where λ_1 is the dominant eigenvalue of A. Compare the decrease in the eigendirection error with $\left|\frac{\lambda_2}{\lambda_1}\right|^k$.

37.40 Repeat Problem 37.39 for the matrix $A = \left[\begin{smallmatrix} 5 & 1 \\ 1 & 5 \end{smallmatrix}\right]$. Compare the convergence behavior in the two problems.

37.41 Use the power method to compute the dominant eigenvalue and a corresponding eigenvector of the matrix

$$A = \begin{bmatrix} 2 & 3 & 2 \\ 10 & 3 & 4 \\ 3 & 6 & 1 \end{bmatrix}.$$

37.42 In Algorithm 1, we normalized w_k by its norm. But other normalization strategies are possible. Consider the following variation: For a nonzero vector v, define $\max(v)$ as the entry of v with the largest magnitude (if there is a tie, pick the first one). Starting with a nonzero vector v_0, consider the iteration

$$\begin{aligned} w_k &= A v_{k-1} \\ v_k &= w_k / \max(w_k). \end{aligned}$$

Show that if v_k converges to a dominant eigenvector, then $\max(w_k)$ converges to the corresponding dominant eigenvalue.

37.43 Write a program to find the smallest (in magnitude) eigenvalue and corresponding eigenvector of the $n \times n$ matrix

$$A = \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix}.$$

37.44 Consider the matrix

$$A = \begin{bmatrix} -150 & 334 & 778 \\ -89 & 195 & 464 \\ 5 & -10 & -27 \end{bmatrix},$$

which has eigenvalues $\lambda_1 = 17$, $\lambda_2 = 3$ and $\lambda_3 = -2$. Perform a hundred QR iterations on A. Plot the eigenvalue error $|A_k[i,i] - \lambda_i|$, for $i = 1, 2, 3$, where A_k denotes the matrix obtained after k iterations, and the norm of the strictly lower triangular part as a function of the iteration number.

37.45 Consider an $n \times n$ upper-Hessenberg matrix. Show how to use Givens rotators to perform a step of the Hessenberg QR algorithm in about $6n^2$ operations.

37.46 Prove Proposition 37.57.

37.47 Consider the matrix

$$A = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Perform several shifted QR iterations using the Rayleigh shift. Determine the eigenvalue to which $A_k[3,3]$ converges as k increases. Additionally, find the number of iterations required to ensure that $|A_k[3,2]| \leqslant 10^{-16} \cdot |A_k[3,3]|$.

37.48 An upper-Hessenberg matrix $H \in \mathbb{C}^{n\times n}$ is called **unreduced** if all subdiagonal entries are nonzero, that is, if $h_{i+1,i} \neq 0$, for $i = 1, \ldots, n-1$. Let H be an unreduced upper-Hessenberg matrix.

(a) Show that the first $n-1$ columns of H are linearly independent.
(b) If H is singular and $H = QR$ is a full QR decomposition of H, show that the last row of R is zero.

37.49 Let H be an unreduced upper-Hessenberg matrix (see Problem 37.48 for the definition of unreduced). If μ is an eigenvalue of H, show that one shifted QR iteration

$$\begin{aligned} &\text{Factor} && H - \mu I_n = QR, \\ &\text{Compute} && \widehat{H} = RQ + \mu I_n. \end{aligned}$$

produces a matrix of the form

$$\widehat{H} = \left[\begin{array}{c|c} \widehat{H}_1 & h \\ \hline 0 & \mu \end{array}\right].$$

Discuss how to interpret this result. Why is it necessary to assume H is unreduced? Do you think this condition is too restrictive for H?

37.50 Apply the QR algorithm with deflation to compute all the eigenvalues of the matrix

$$\begin{bmatrix} 6 & 2 & 1 & 1 \\ 2 & 5 & 1 & 1 \\ 1 & 1 & 4 & 1 \\ 1 & 1 & 1 & 3 \end{bmatrix}.$$

Chapter 38
VIII-SUPPLEMENTARY TOPICS

38.1 Localizing the eigenvalues of a matrix: Gershgorin's circle theorem

When deciding which algorithm to use for computing eigenvalues, it helps to know if a matrix has a dominant eigenvalue or to have a rough idea of where the eigenvalues are located. In this section, we introduce a well-known result that estimates the location of a matrix's eigenvalues using only its entries. This result provides approximate eigenvalues and useful insights, such as whether all eigenvalues are nonzero, positive or real.

Gershgorin's circle theorem[1] states that the eigenvalues of a square $n \times n$ matrix A lie within the union of n circles, where each circle's center and radius are computed from the entries of A in each row.

Theorem 38.1. (Gershgorin's circle theorem)

Let A be an $n \times n$ complex matrix, and let R_i be the sum of the moduli of the non-diagonal entries in the i-th row of A:

$$R_i := \sum_{j \neq i} |A_{ij}|.$$

Define the **Gershgorin circles** as the circular regions in the complex plane centered at the diagonal entries of A, with radii R_i:

$$\mathcal{G}_i(A) := \{z \in \mathbb{C} \mid |z - A_{ii}| \leqslant R_i\},$$

for $i = 1, \ldots, n$. The eigenvalues of A are contained in the union of the Gershgorin circles:

$$\sigma(A) \subseteq \mathcal{G}_1(A) \cup \mathcal{G}_2(A) \cup \cdots \cup \mathcal{G}_n(A).$$

Moreover, if k Gershgorin circles are disjoint from the others, then their union contains exactly k eigenvalues of A (counting multiplicities).

[1] Semyon Aranovich Gershgorin (1901-1933) was a Soviet mathematician known for his work in Numerical Analysis and Matrix Theory. He introduced Gershgorin's Circle Theorem in 1931.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_38

Proof Let λ be an eigenvalue of A. We will show that λ lies within at least one Gershgorin circle $\mathcal{G}_i(A)$.

Let x be an eigenvector of A corresponding to λ, i.e., $Ax = \lambda x$. Let p be the index such that $|x_p| = \max\{|x_1|, \ldots, |x_n|\}$. Since $x \neq 0$, we have $|x_p| > 0$.

Consider the p-th component of the equation $Ax = \lambda x$:

$$\lambda x_p = \sum_{j=1}^{n} A_{pj} x_j = A_{pp} x_p + \sum_{j \neq p} A_{pj} x_j.$$

Rearranging,

$$(\lambda - A_{pp}) x_p = \sum_{j \neq p} A_{pj} x_j.$$

Taking the modulus on both sides gives

$$|\lambda - A_{pp}|\,|x_p| = \left| \sum_{j \neq p} A_{pj} x_j \right| \leqslant \sum_{j \neq p} |A_{pj}|\,|x_j| \leqslant \sum_{j \neq p} |A_{pj}|\,|x_p| = R_p |x_p|.$$

Dividing through by $|x_p|$, we get

$$|\lambda - A_{pp}| \leqslant R_p,$$

which implies $\lambda \in \mathcal{G}_p(A)$.

The second claim follows from the fact that eigenvalues vary continuously with the entries of the matrix. A detailed proof of this result is beyond the scope of this book. □

We can use Gershgorin circles to infer properties of a matrix's eigenvalues. Let us see an example.

Example 38.2 Consider the matrix

$$A = \begin{bmatrix} 0 & 1 & -1 & 1 \\ 2 & 0 & 0 & 0 \\ 0 & 2 & 3 & 0 \\ 0 & 0 & -1 & 7 \end{bmatrix},$$

whose eigenvalues are approximately $\lambda_1 = -1.66$, $\lambda_2 = 6.98$, $\lambda_3 = 2.34 + 0.46i$ and $\lambda_4 = 2.34 - 0.46i$.

The Gershgorin circles for this matrix are

$$\begin{aligned} \mathcal{G}_1(A) &= \{z \in \mathbb{C} \mid |z| \leqslant 3\}, \\ \mathcal{G}_2(A) &= \{z \in \mathbb{C} \mid |z| \leqslant 2\}, \\ \mathcal{G}_3(A) &= \{z \in \mathbb{C} \mid |z - 3| \leqslant 2\}, \quad \text{and} \\ \mathcal{G}_4(A) &= \{z \in \mathbb{C} \mid |z - 7| \leqslant 1\}. \end{aligned}$$

These circles are shown in Figure 38.1, where the black dots represent the eigenvalues of A.

Even if we did not know the eigenvalues, Gershgorin's theorem already tells us something useful. The rightmost circle, $\mathcal{G}_4$, is disjoint from the others, so it must contain exactly one eigenvalue. Because A is real, that eigenvalue must also be real. If it were complex, its conjugate would have to lie in the same circle, which is impossible. So A has a real, simple, dominant eigenvalue, and we could use the power method to find it.

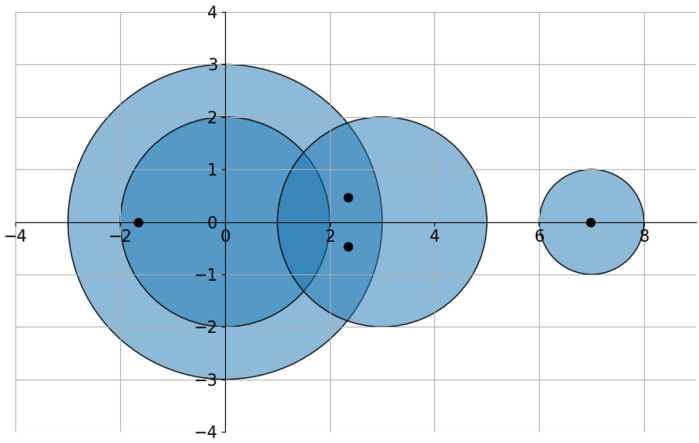


Fig. 38.1: Gershgorin circles and eigenvalues of the matrix A in Example 38.2.

We could just as well use the columns of A instead of the rows to define the Gershgorin circles. Why? Because A and A^T have the same eigenvalues, so Theorem 38.1 applies to both. This means the eigenvalues of A must lie in the intersection of the Gershgorin circles defined by its rows and those defined by its columns.

For the matrix A in Example 38.2, the circles from the rows and columns give the same result as using just the row disks. However, in some cases, we can combine the row and column circles to get a more precise estimate of the eigenvalues. Let us see an example where this gives a sharper result.

Example 38.3 Consider the matrix

$$A = \begin{bmatrix} 1 & 1 & 0 \\ -i/2 & 4 & 1/2 \\ i & 0 & 8 \end{bmatrix}.$$

The Gershgorin circles for A are

$$\mathcal{G}_1(A) = \{z \in \mathbb{C} : |z - 1| \leqslant 1\},$$
$$\mathcal{G}_2(A) = \{z \in \mathbb{C} : |z - 4| \leqslant 1\}, \text{and}$$
$$\mathcal{G}_3(A) = \{z \in \mathbb{C} : |z - 8| \leqslant 1\}.$$

These circles are disjoint, so each contains one eigenvalue of A.

Now consider the Gershgorin circles for A^T:

$$\mathcal{G}_1(A^T) = \{z \in \mathbb{C} : |z - 1| \leqslant 3/2\},$$
$$\mathcal{G}_2(A^T) = \{z \in \mathbb{C} : |z - 4| \leqslant 1\}, \text{and}$$
$$\mathcal{G}_3(A^T) = \{z \in \mathbb{C} : |z - 8| \leqslant 1/2\}.$$

These are also disjoint, so each contains one eigenvalue of A.

For the Gershgorin circles of A and A^T, we have $\mathcal{G}_1(A^T) \subset \mathcal{G}_1(A)$, $\mathcal{G}_2(A) = \mathcal{G}_2(A^T)$, and $\mathcal{G}_3(A) \subset \mathcal{G}_3(A^T)$. Since the eigenvalues of A lie in the intersection of the Gershgorin circles for A and A^T, we conclude

$$\sigma(A) \subseteq \mathcal{G}_1(A) \cup \mathcal{G}_2(A) \cup \mathcal{G}_3(A^T),$$

with each circle containing exactly one eigenvalue, as shown in Figure 38.2. So, when we add the circles from the columns, we replace a row circle with one that has half the radius. This gives us a better estimate for the eigenvalues.

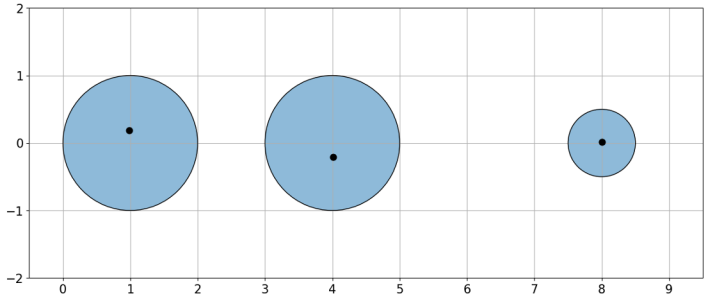


Fig. 38.2: Gershgorin circles and eigenvalues of the matrix A in Example 38.3.

In Chapter 36, we saw that similar matrices have the same eigenvalues, so we can apply Gershgorin's circle theorem to any matrix similar to A. In particular, we can use $D^{-1}AD$, where D is any diagonal matrix. The diagonal entries of A and $D^{-1}AD$ are the same, so the centers of the Gershgorin disks do not change. However, the radii of the disks can get smaller with a diagonal similarity.

Example 38.4 Consider the matrix

$$A = \begin{bmatrix} 3 & 0 & 2 \\ -4 & 8 & 2 \\ 0 & 1 & -4 \end{bmatrix},$$

with eigenvalues approximately 8.03, 3.22, and -4.25. The Gershgorin circles for A are

$$\begin{aligned} \mathcal{G}_1(A) &= \{z \in \mathbb{C} : |z - 3| \leqslant 2\}, \\ \mathcal{G}_2(A) &= \{z \in \mathbb{C} : |z - 8| \leqslant 6\}, \quad \text{and} \\ \mathcal{G}_3(A) &= \{z \in \mathbb{C} : |z + 4| \leqslant 1\}. \end{aligned}$$

These circles are shown in the left panel of Figure 38.3. Note that $\mathcal{G}_1(A)$ and $\mathcal{G}_2(A)$ overlap.

Is it possible to find a nonsingular matrix D so that the Gershgorin circles do not overlap, so that each eigenvalue is isolated in its own circle? It is! If we apply a diagonal similarity transformation with $D = \operatorname{diag}(1, d, 1)$, where $d > 0$, the transformed matrix is

$$D^{-1}AD = \begin{bmatrix} 3 & 0 & 2 \\ -4/d & 8 & 2/d \\ 0 & d & -4 \end{bmatrix}.$$

The Gershgorin circles for $D^{-1}AD$ are

$$\begin{aligned} \mathcal{G}_1(D^{-1}AD) &= \{z \in \mathbb{C} : |z - 3| \leqslant 2\}, \\ \mathcal{G}_2(D^{-1}AD) &= \left\{z \in \mathbb{C} : |z - 8| \leqslant \frac{6}{d}\right\}, \quad \text{and} \\ \mathcal{G}_3(D^{-1}AD) &= \{z \in \mathbb{C} : |z + 4| \leqslant d\}. \end{aligned}$$

Increasing d shrinks $\mathcal{G}_2(D^{-1}AD)$ but enlarges $\mathcal{G}_3(D^{-1}AD)$. For the circles to not overlap, we need:

$$5 < 8 - \frac{6}{d}, \quad \text{and} \quad -4 + d < 1.$$

These conditions are satisfied when $d \in (2, 5)$.

The right panel of Figure 38.3 shows the Gershgorin circles for $d = 2.25$. With this choice, the circles no longer overlap, and each eigenvalue lies in its own circle.

38.2 The Schur factorization and invariant subspaces

In Chapter 37, we showed that any complex matrix is unitarily similar to an upper triangular matrix, called its *Schur form* (Theorem 37.23). For a real matrix, a real

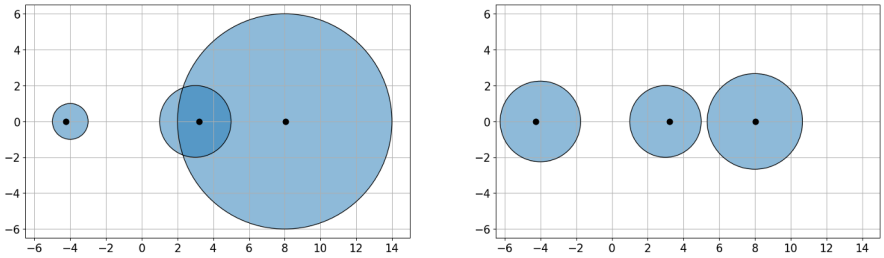


Fig. 38.3: Gershgorin circles and eigenvalues for A (left) and $D^{-1}AD$ with $D = \text{diag}(1, 2.25, 1)$ (right) in Example 38.4.

orthogonaly similarity generally produces a quasi-upper-triangular real Schur form with 1×1 and 2×2 diagonal blocks (Theorem 37.26).

Now we revisit that result from the perspective of invariant subspaces. This geometric point of view helps us understand the Schur factorization and also why the QR algorithm converges to it.

We introduced invariant subspaces in Chapter 36. Recall that an invariant subspace of a matrix A is a subspace $\mathcal{V}$ such that $A(\mathcal{V}) \subseteq \mathcal{V}$, where $A(\mathcal{V})$ denotes all vectors obtained by applying A to vectors in $\mathcal{V}$. So A maps the whole subspace back into itself. In particular, the span of a nonzero vector x is invariant under A if and only if x is an eigenvector of A. The next theorem generalizes this idea to subspaces of higher dimension.

Theorem 38.5. (Characterization of invariant subspaces)

Let $A \in \mathbb{C}^{n\times n}$ and $X = \begin{bmatrix} x_1 & \cdots & x_p \end{bmatrix}$ be an $n \times p$ matrix with linearly independent columns $x_1, \ldots, x_p$. Let $\mathcal{V} = \text{col}(X) = \text{span}(x_1, \ldots, x_p)$. Then, $\mathcal{V}$ is an invariant subspace of A if and only if there exists a $p \times p$ matrix B such that $AX = XB$.

Proof First, assume that $\mathcal{V}$ is an invariant subspace of A. This means that for each column x_i of X, the vector Ax_i is in $\mathcal{V}$, for $i = 1, \ldots, p$. Since $Ax_i \in \mathcal{V}$, we can write Ax_i as a linear combination of the columns of X:

$$Ax_i = \sum_{j=1}^{p} b_{ji} x_j, \quad i = 1, \ldots, p,$$

for some scalars b_{ji}. Putting this together for all columns of X, we have

$$\begin{aligned} AX = A\begin{bmatrix} x_1 & \cdots & x_p \end{bmatrix} &= \begin{bmatrix} Ax_1 & \cdots & Ax_p \end{bmatrix} \\ &= \begin{bmatrix} \sum_{j=1}^{p} b_{j1}x_j & \cdots & \sum_{j=1}^{p} b_{jp}x_j \end{bmatrix} \\ &= \begin{bmatrix} x_1 & \cdots & x_p \end{bmatrix} \begin{bmatrix} b_{11} & \cdots & b_{1p} \\ \vdots & \ddots & \vdots \\ b_{p1} & \cdots & b_{pp} \end{bmatrix} \\ &= XB, \end{aligned}$$

where B is the $p \times p$ matrix with entries b_{ji}.

Conversely, suppose $AX = XB$ for some $p \times p$ matrix B. Let $v \in \mathcal{V} = \text{col}(X)$. Then, $v = Xc$ for some $c \in \mathbb{C}^p$. Multiplying v by A, we get

$$Av = A(Xc) = (AX)c = (XB)c = X(Bc) \in \text{col}(X).$$

Thus, $\mathcal{V}$ is an invariant subspace of A. □

The next theorem connects the concept of eigenvalues and eigenvectors to the characterization of invariant subspaces in Theorem 38.5.

Theorem 38.6. (Eigenpairs and invariant subspaces)

Let $A \in \mathbb{C}^{n\times n}$, and suppose there exist a matrix $X \in \mathbb{C}^{n\times p}$ with full column rank and a matrix $B \in \mathbb{C}^{p\times p}$ such that $AX = XB$. Then

(a) The eigenvalues of B are also eigenvalues of A.
(b) If y is an eigenvector of B with eigenvalue λ, then Xy is an eigenvector of A with eigenvalue λ.

Proof Let y be an eigenvector of B with eigenvalue λ, and define $x := Xy$. Since X has full column rank and $y \neq 0$, $x \neq 0$. We have

$$Ax = A(Xy) = (AX)y = (XB)y = X(By) = X(\lambda y) = \lambda(Xy) = \lambda x.$$

Thus, x is an eigenvector of A with eigenvalue λ.

Since the above reasoning holds for any eigenvalue of B, all eigenvalues of B are also eigenvalues of A. □

The next theorem shows how a basis for an invariant subspace, through a similarity transformation, can put a matrix A into block upper-triangular form.

Theorem 38.7. (Block triangularization of a matrix)

Let $A \in \mathbb{C}^{n\times n}$, and suppose $AX = XB$ for some matrices $X \in \mathbb{C}^{n\times p}$ with full column rank and $B \in \mathbb{C}^{p\times p}$. Let $\{x_1, \ldots, x_p\}$ be the linearly independent columns of X, and extend them to a basis $\{x_1, \ldots, x_n\}$ for $\mathbb{C}^n$. Define the invertible matrix $S := \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}$. Then, there exist matrices $C \in \mathbb{C}^{p\times(n-p)}$ and $D \in \mathbb{C}^{(n-p)\times(n-p)}$ such that

$$S^{-1}AS = \begin{bmatrix} B & C \\ 0 & D \end{bmatrix}. \tag{38.1}$$

Proof Write S as

$$S = \begin{bmatrix} x_1 \cdots x_p \; x_{p+1} \cdots x_n \end{bmatrix} = \begin{bmatrix} X \; \widetilde{X} \end{bmatrix},$$

where $X = \begin{bmatrix} x_1 \cdots x_p \end{bmatrix}$ and $\widetilde{X} = \begin{bmatrix} x_{p+1} \cdots x_n \end{bmatrix}$. The matrix S is invertible by construction. We can write its inverse in block form as

$$S^{-1} = \begin{bmatrix} Y \\ \widetilde{Y} \end{bmatrix},$$

where Y consists of the first p rows of S^{-1}, and $\widetilde{Y}$ consists of the last $n - p$ rows. From $S^{-1}S = I_n$, we obtain

$$\begin{bmatrix} Y \\ \widetilde{Y} \end{bmatrix} \begin{bmatrix} X \; \widetilde{X} \end{bmatrix} = \begin{bmatrix} YX & Y\widetilde{X} \\ \widetilde{Y}X & \widetilde{Y}\widetilde{X} \end{bmatrix} = \begin{bmatrix} I_p & 0 \\ 0 & I_{n-p} \end{bmatrix}.$$

This gives $YX = I_p$ and $\widetilde{Y}X = 0$.

Now, compute $S^{-1}AS$:

$$S^{-1}AS = \begin{bmatrix} Y \\ \widetilde{Y} \end{bmatrix} A \begin{bmatrix} X \; \widetilde{X} \end{bmatrix} = \begin{bmatrix} YAX & YA\widetilde{X} \\ \widetilde{Y}AX & \widetilde{Y}A\widetilde{X} \end{bmatrix}.$$

Using $AX = XB$, we have $YAX = YXB = B$ and $\widetilde{Y}AX = \widetilde{Y}XB = 0$. Substituting these results, we get

$$S^{-1}AS = \begin{bmatrix} B & YA\widetilde{X} \\ 0 & \widetilde{Y}A\widetilde{X} \end{bmatrix}.$$

Define $C := YA\widetilde{X}$ and $D := \widetilde{Y}A\widetilde{X}$, and the block triangular form follows. □

The vectors $x_1, \ldots, x_p$ in the previous theorem form a basis for the invariant subspace $\mathrm{col}(X)$ of A. By extending this set to a full basis for $\mathbb{C}^n$, Theorem 38.7 shows how to construct a similarity transformation that reduces A to the upper triangular form (38.1). Because the matrix in (38.1) is similar to A, their eigenvalues are the same:

$$\sigma(A) = \sigma(S^{-1}AS) = \sigma(B) \cup \sigma(D).$$

This means we have split the eigenvalue problem for A into two smaller problems: one for B and one for D.

Example 38.8 Consider the matrix

$$A = \begin{bmatrix} 9/5 & -3/5 & -3/5 & 6/5 \\ -1/5 & 17/5 & -3/5 & -14/5 \\ 2/5 & 1/5 & 16/5 & -2/5 \\ -1/10 & 6/5 & -3/10 & -2/5 \end{bmatrix}.$$

The vectors $x_1 = \begin{bmatrix}-3 & 0 & 1 & 0\end{bmatrix}^T$ and $x_2 = \begin{bmatrix}0 & 2 & 0 & 1\end{bmatrix}^T$ are eigenvectors of A with eigenvalue 2. By Proposition 36.49, the subspace $\text{span}(x_1, x_2)$ is an invariant subspace of A. Now, let $x_3 = \begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix}^T$ and $x_4 = \begin{bmatrix}0 & 1 & 0 & 0\end{bmatrix}^T$. The set $\{x_1, x_2, x_3, x_4\}$ forms a basis for $\mathbb{C}^n$ that includes a basis for an invariant subspace of A. Define $S = \begin{bmatrix}x_1 & x_2 & x_3 & x_4\end{bmatrix}$. Then,

$$S^{-1}AS = \begin{bmatrix} 2 & 0 & 2/5 & 1/5 \\ 0 & 2 & -1/10 & 6/5 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

which is block upper triangular.

We can take the idea of transforming matrices into block-upper triangular form even further. With the right similarity transformation (change of basis), we can reduce any complex matrix to an upper triangular form. The following theorem identifies the basis required.

Theorem 38.9. (Upper triangularization through similarity)

Let $A \in \mathbb{C}^{n\times n}$, and let $\{x_1, \ldots, x_n\}$ be a basis for $\mathbb{C}^n$. Form the corresponding invertible matrix $S = \begin{bmatrix}x_1 & \cdots & x_n\end{bmatrix}$. The matrix $S^{-1}AS$ is upper triangular if and only if $\text{span}(x_1, \ldots, x_j)$ is an invariant subspace of A for every $j = 1, \ldots, n$.

Proof First, assume there is an invertible matrix S such that $T = S^{-1}AS$ is upper triangular. Let $x_1, x_2, \ldots, x_n$ be the columns of S. Since $AS = ST$, we have

$$Ax_i = T_{1i}x_1 + T_{2i}x_2 + \cdots + T_{ii}x_i \in \text{span}(x_1, \ldots, x_i),$$

for $i = 1, \ldots, n$. This implies that for each $j = 1, \ldots, n$, the subspace $\text{span}(x_1, \ldots, x_j)$ is an invariant subspace of A.

Now, assume $\text{span}(x_1, \ldots, x_j)$ is an invariant subspace of A for every $j = 1, \ldots, n$. Let $S = \begin{bmatrix}x_1 & \cdots & x_n\end{bmatrix}$. Since each $\text{span}(x_1, \ldots, x_j)$ is an invariant subspace of A, by Theorem 38.7, we know that

$$T = S^{-1}AS = \begin{array}{c} \begin{matrix} j & n-j \end{matrix} \\ \begin{bmatrix} T_1^{(j)} & T_2^{(j)} \\ 0 & T_3^{(j)} \end{bmatrix} \begin{matrix} j \\ n-j \end{matrix} \end{array}$$

is block upper triangular, for $j = 1, \ldots, n$, which implies that the matrix T is upper triangular. □

Now that we know the type of basis that transforms a matrix to upper-triangular form by similarity, does such a basis always exist? Yes! The Schur factorization

theorem (Theorem 37.23) guarantees that every matrix has such a basis, and it can even be chosen to be orthonormal.

38.3 More on algorithms for computing eigenvalues

In Chapter 37, we introduced three algorithms to compute eigenvalues and eigenvectors: the power method, the shift-and-invert method, and the QR algorithm. In this section, we will cover three more algorithms.

First, we let the shift in the shift-and-invert method change at each step. This leads to the Rayleigh quotient method. Next, we extend the power method to find several of the largest eigenvalues and their eigenspaces using the orthogonal iteration method. We will also see that the QR algorithm is just orthogonal iteration in disguise, which helps explain why it works. Finally, we introduce the Arnoldi method, which is designed to find a few eigenvalues of large, sparse matrices.

38.3.1 Updating the shift: The Rayleigh quotient iteration

In Section 37.4.3, we introduced the shift-and-invert method. It uses a fixed shift μ, chosen to be close to the eigenvalue we want to find. This allows the method to focus on eigenvalues other than the dominant one.

The Rayleigh quotient iteration[2] improves on this idea by updating the shift at every step. Given an initial guess v_0 for an eigenvector, the method refines the shift using the Rayleigh quotient:

$$\mu = \frac{v_k^* A v_k}{v_k^* v_k},$$

where v_k is the current approximation of the eigenvector.

This choice is not arbitrary: by Proposition 37.31, the Rayleigh quotient gives the best eigenvalue estimate we can get from v_k. So, at each step, we use the most accurate shift available to guide the iteration.

Algorithm 5 summarizes this method. At each step, the shift is updated using the Rayleigh quotient. The algorithm then solves the shifted system and normalizes the resulting vector to obtain a new approximation of the eigenvector. The process continues until the norm of the residual falls below a prescribed tolerance or the maximum number of iterations is reached.

[2] Named after Lord Rayleigh (John William Strutt, 1842–1919), a Nobel Prize-winning British physicist and mathematician.

Algorithm 5 Rayleigh quotient iteration

Inputs: A matrix $A \in \mathbb{C}^{n\times n}$, a unit vector $v_0 \in \mathbb{C}^n$, a tolerance $\epsilon > 0$, and a maximum number of iterations N.

1: Set
$$\mu_0 = v_0^* A v_0.$$
2: **for** $k = 1, 2, \ldots, N$ **do**
3: Solve
$$(A - \mu_{k-1} I_n) w_k = v_{k-1}.$$
4: Normalize
$$v_k = \frac{w_k}{\|w_k\|_2}.$$
5: Update the shift
$$\mu_k = v_k^* A v_k.$$
6: **if** $\|Av_k - \mu_k v_k\|_2 \leqslant \epsilon$ **then**
7: **return** μ_k, v_k
8: **end if**
9: **end for**

The Rayleigh quotient iteration often converges much faster than the shift-and-invert iteration. Its convergence is local: the initial vector must be sufficiently close to an eigenvector associated with a simple eigenvalue. For a general matrix, the local convergence is typically quadratic, so the number of correct digits roughly doubles at each iteration once the iterates are sufficiently close to the desired eigenpair. If A is Hermitian (or real symmetric), the local convergence is cubic, and the number of correct digits roughly triples at each iteration.

The main disadvantage is that the shift changes at every step. Consequently, we must solve a new linear system

$$(A - \mu_k I_n) w_{k+1} = v_k$$

at each iteration and, in general, cannot reuse the LU factorization from the preceding step. Thus, each iteration is more expensive than an iteration of the shift-and-invert method with a fixed shift. Nevertheless, because Rayleigh quotient iteration usually converges in only a few steps, it can still be very efficient in practice. A useful strategy is to begin with a few shift-and-invert iterations to obtain a good initial approximation and then switch to Rayleigh quotient iteration, as illustrated in the next example.

Example 38.10 Consider the matrix

$$A = \begin{bmatrix} 5 & 2 & 0 \\ 2 & 5 & 0 \\ -3 & 4 & 6 \end{bmatrix}.$$

The matrix A is invertible and has eigenvalues 7, 6, and 3. Hence, its eigenvalue of smallest magnitude is $\lambda = 3$, which in this example is also its smallest eigenvalue. An associated eigenvector is $x = \begin{bmatrix} 3 & -3 & 7 \end{bmatrix}^T$. We will compare how quickly the shift-and-invert method and the Rayleigh quotient iteration approximate this eigenpair.

We start with the shift-and-invert iteration. Since A is invertible, we may use the fixed shift $\mu = 0$. In this case, shift-and-invert targets the eigenvalue of A closest to zero, namely the smallest-magnitude eigenvalue $\lambda = 3$. We begin with a random unit vector v_0 and apply the method to approximate this eigenvalue and its eigenvector. Since both A and v_0 are real, all iterates will stay real.

The left panel of Figure 38.4 shows how the method converges for the eigenvalue error

$$|\lambda - v_k^T A v_k|,$$

where v_k is the unit vector at iteration k, and the Rayleigh shift $v_k^T A v_k$ is used as the approximation to the eigenvalue. The method takes about 50 iterations to reduce errors to the level of round-off accuracy[3].

Now let us see what happens when we use shift-and-invert with Rayleigh shifts. We begin with a random vector v_0 and run 5 iterations of the shift-and-invert method with a fixed shift $\mu = 0$. The resulting vector v_5 is then used to start the Rayleigh quotient iteration, where the first shift is calculated using the Rayleigh quotient.

The right panel of Figure 38.4 shows how the eigenvalue error decreases. This combined approach is very effective, taking only 9 iterations in total: 5 from shift-and-invert and 4 from Rayleigh quotient iteration, to reach round-off accuracy. That is a huge improvement compared to the 50 iterations needed with the basic shift-and-invert method alone!

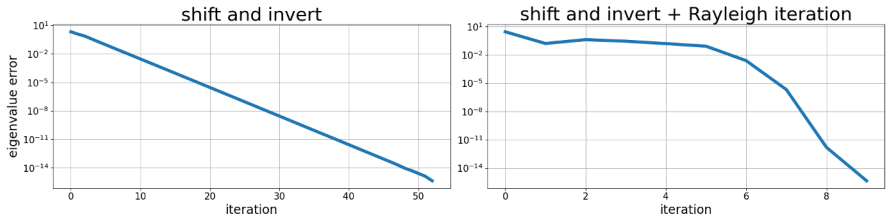


Fig. 38.4: Convergence of the shift-and-invert method (left) and the combined shift-and-invert with Rayleigh shifts (right) for approximating the smallest eigenvalue of the matrix A in Example 38.10.

38.3.2 Orthogonal iteration and the QR algorithm

In Chapter 37, we described how the QR algorithm works but did not explain why it converges to the Schur form of a matrix A when A has eigenvalues with distinct magnitudes. To build some intuition, we now show that the QR algorithm is essentially a special case of a more general method called orthogonal iteration.

[3] **Round-off errors** are small numerical inaccuracies that happen due to the limitations of representing real numbers in a computer, typically around 10^{-16} for double-precision arithmetic.

Orthogonal iteration is a natural extension of the power method. Instead of computing a single dominant eigenvector, it finds a subspace spanned by the eigenvectors corresponding to the largest eigenvalues (in magnitude).

The power method starts with a random nonzero unit vector v_0, which defines the initial subspace $\mathcal{V}_0 = \text{span}(v_0)$. By repeatedly multiplying by A, the subspace $\mathcal{V}_k = \text{span}(A^k v_0)$ approaches the eigenspace corresponding to the dominant eigenvalue.

Orthogonal iteration builds on this idea. Instead of a one-dimensional subspace, it starts with a p-dimensional subspace $\mathcal{V}_0$, where $p \leqslant n$. The method then repeatedly applies A to compute the subspaces

$$\mathcal{V}_k := A^k(\mathcal{V}_0) = \{A^k v \mid v \in \mathcal{V}_0\}.$$

As k increases, and under suitable conditions, these subspaces approach the one spanned by the eigenvectors associated with the p largest eigenvalues of A[4].

Just like in the power method we normalize $v_k = A^k v_0$ at each step to prevent overflow or underflow, in orthogonal iteration we avoid computing the subspace $\mathcal{V}_k = A^k(\mathcal{V}_0)$ directly by applying A^k to all vectors in $\mathcal{V}_0$. Instead, the method finds orthonormal bases for the subspaces $A^k(\mathcal{V}_0)$.

It starts with an $n \times p$ matrix Q_0 whose columns form an orthonormal basis for the initial subspace $\mathcal{V}_0$. At each step, it computes $W_k = AQ_{k-1}$, whose columns span the new subspace $\mathcal{V}_k$. Then, it performs a reduced QR factorization $W_k = Q_k R_k$ to obtain an orthonormal basis Q_k for $\mathcal{V}_k$. The matrix R_k is upper triangular and holds the coefficients relating W_k to Q_k.

Algorithm 6 shows the steps of orthogonal iteration.

Algorithm 6 Orthogonal iteration

Inputs:
An $n \times n$ matrix A, an $n \times p$ matrix Q_0 with orthonormal columns
while the algorithm has not converged **do**
$W_k = AQ_{k-1}$
$W_k = Q_k R_k$
end while

To understand why the columns of Q_k approach an orthonormal basis for the subspace spanned by the p eigenvectors corresponding to the largest eigenvalues of A, assume for simplicity that $A \in \mathbb{C}^{n \times n}$ is diagonalizable. Suppose the eigenvalues of A are ordered by decreasing magnitude:

$$|\lambda_1| \geqslant |\lambda_2| \geqslant \cdots \geqslant |\lambda_n| \geqslant 0.$$

For orthogonal iteration to work, we assume a gap in the spectrum: $|\lambda_p| > |\lambda_{p+1}|$.

Let $A = SDS^{-1}$, where S is the matrix of eigenvectors and D is diagonal with eigenvalues $\lambda_1, \ldots, \lambda_n$. We split D into two blocks:

$$D_1 := \text{diag}(\lambda_1, \ldots, \lambda_p) \quad \text{and} \quad D_2 := \text{diag}(\lambda_{p+1}, \ldots, \lambda_n),$$

[4] The distance between subspaces can be defined rigorously, but we skip those details here.

with D_1 holding the p largest eigenvalues and D_2 the rest.

We start with a random $n \times p$ matrix Q_0 whose columns form an orthonormal basis for some initial subspace $\mathcal{V}_0$. When we multiply Q_0 by A, we get

$$W_1 = AQ_0 = SDS^{-1}Q_0.$$

The product $S^{-1}Q_0$ gives the coordinates of the columns of Q_0 in the eigenvector basis $x_1, \ldots, x_n$. We split this matrix as

$$S^{-1}Q_0 = \begin{bmatrix} Y_0 \\ Z_0 \end{bmatrix},$$

where Y_0 is a $p \times p$ matrix giving the components along the dominant eigenvectors $x_1, \ldots, x_p$, and Z_0 gives the components along the remaining eigenvectors $x_{p+1}, \ldots, x_n$.

The crucial assumption is that Y_0 is invertible. This means that the initial subspace $\mathcal{V}_0$ has nontrivial components along each of the top p eigenvectors. If Y_0 were not invertible, then $\mathcal{V}_0$ would miss at least one direction in the span of $\{x_1, \ldots, x_p\}$. This is the natural generalization of the condition $c_1 \neq 0$ in the power method, which requires that the initial vector has a nonzero component in the dominant eigenvector's direction. Fortunately, if Q_0 is chosen randomly, Y_0 is almost always invertible.

Substituting into the expression for W_1, we get

$$W_1 = S \begin{bmatrix} D_1 & 0 \\ 0 & D_2 \end{bmatrix} \begin{bmatrix} Y_0 \\ Z_0 \end{bmatrix} = S \begin{bmatrix} D_1 Y_0 \\ D_2 Z_0 \end{bmatrix}.$$

Since D_1 is invertible and Y_0 is assumed invertible, the top block $D_1 Y_0$ is invertible, which implies W_1 has full column rank.

To compute an orthonormal basis for $\mathcal{V}_1 = \text{col}(W_1)$, we perform a reduced QR factorization: $W_1 = Q_1 R_1$, where Q_1 has orthonormal columns that span the same subspace as W_1.

So we have

$$\text{col}(Q_1) = \text{col}(W_1) = \text{col}(AQ_0) = A(\mathcal{V}_0).$$

Repeating the same steps iteratively gives

$$\text{col}(Q_k) = \text{col}(W_k) = \text{col}(A^k Q_0) = A^k(\mathcal{V}_0).$$

To study the behavior of this subspace, we expand

$$A^k Q_0 = SD^k S^{-1} Q_0 = S \begin{bmatrix} D_1^k & 0 \\ 0 & D_2^k \end{bmatrix} \begin{bmatrix} Y_0 \\ Z_0 \end{bmatrix}.$$

Now factor out λ_p^k from the diagonal matrices:

$$A^k Q_0 = \lambda_p^k S \begin{bmatrix} \left(\frac{D_1}{\lambda_p}\right)^k & 0 \\ 0 & \left(\frac{D_2}{\lambda_p}\right)^k \end{bmatrix} \begin{bmatrix} Y_0 \\ Z_0 \end{bmatrix}.$$

Since $|\lambda_p| > |\lambda_{p+1}|$, the entries of $\left(\frac{D_2}{\lambda_p}\right)^k$ decay to zero as k increases. So, $A^k Q_0$ approaches:

$$S \begin{bmatrix} D_1^k Y_0 \\ 0 \end{bmatrix} = \begin{bmatrix} x_1 & \cdots & x_p \end{bmatrix} D_1^k Y_0.$$

Because $D_1^k Y_0$ stays invertible, this shows that the columns of $A^k Q_0$ (and hence Q_k) approach the span of the first p eigenvectors of A, the ones associated with the eigenvalues of largest magnitude.

Remark 38.11. (Orthogonal iteration rate of convergence)

The matrix $(D_2/\lambda_p)^k$ approaches zero as $(\lambda_{p+1}/\lambda_p)^k$, so the larger the gap between λ_p and λ_{p+1}, the faster the orthogonal iteration will converge to an invariant subspace of A.

Orthogonal iteration does more than just approximate the span of the first p eigenvectors of A. A key insight is that it performs orthogonal iteration on multiple subspaces at once. To see this, focus on the first q columns of Q_k. These columns satisfy

$$\begin{aligned} W_k[:, 1:q] &= A Q_{k-1}[:, 1:q] && \text{(multiply by } A\text{)} \\ W_k[:, 1:q] &= Q_k[:, 1:q] R_k[1:q, 1:q] && \text{(reduced QR factorization),} \end{aligned}$$

so the first q columns of Q_k evolve just as they would in orthogonal iteration started from the first q columns of Q_0. In other words, the algorithm runs orthogonal iteration for each $q = 1, 2, \ldots, p$ in parallel, tracking all those subspaces at once inside a single procedure.

Suppose we strengthen our earlier assumption and require

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_p| > |\lambda_{p+1}|.$$

Then, the first q columns of Q_k will converge to a basis for the subspace $\text{span}(x_1, x_2, \ldots, x_q)$, corresponding to the q largest eigenvalues of A. Since this holds for each $q = 1, \ldots, p$, the columns of Q_k span an approximate nested sequence of subspaces:

$$\{\text{span}(x_1, \ldots, x_q) \mid q = 1, \ldots, p\}.$$

As a result, for large enough k, the matrix

$$A_k := Q_k^* A Q_k$$

will be nearly upper triangular, with the p largest eigenvalues of A (in decreasing magnitude) appearing along its diagonal.

Example 38.12 Consider the matrix

$$A = \begin{bmatrix} 5 & 2 & 0 \\ 2 & 5 & 0 \\ -3 & 4 & 6 \end{bmatrix}.$$

The two largest eigenvalues of A are $\lambda_1 = 7$ and $\lambda_2 = 6$, with associated eigenvectors $x_1 = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T$ and $x_2 = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T$, respectively.

Starting with a random 3×2 orthogonal matrix Q_0, we apply the orthogonal subspace method to A for 20 iterations. The results, rounded to three decimal places, are

$$Q_{20} = \begin{bmatrix} -0.579 & -0.407 \\ -0.579 & -0.407 \\ -0.575 & 0.818 \end{bmatrix} \qquad \text{and} \qquad A_{20} = \begin{bmatrix} 7.002 & 0.701 \\ 0 & 5.998 \end{bmatrix}.$$

The matrix A_{20} is close to upper triangular, with good approximations of the two largest eigenvalues of A on its diagonal. The first column of Q_{20} approximates $\begin{bmatrix} -1 & -1 & -1 \end{bmatrix}^T / \sqrt{3}$, an eigenvector associated with λ_1. Although the second column of Q_{20} is not close to an eigenvector of A, the two columns of Q_{20} are close to forming an orthonormal basis for

$$\operatorname{span}\left(\frac{1}{\sqrt{3}} \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}, \frac{1}{\sqrt{6}} \begin{bmatrix} -1 \\ -1 \\ 2 \end{bmatrix} \right) = \operatorname{span}(x_1, x_2).$$

Orthogonal iteration runs the algorithm simultaneously for $q = 1, 2, \ldots, p$, which suggests that if all eigenvalues have distinct magnitudes, we can compute the entire spectrum of a matrix by starting with an $n \times n$ unitary matrix as the initial Q_0. This is exactly what the QR algorithm does, as we will see in the next section.

38.3.3 From orthogonal iteration to the QR algorithm

Let $A \in \mathbb{C}^{n \times n}$ be a matrix with eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$, ordered by decreasing magnitude, and corresponding eigenvectors $x_1, x_2, \ldots, x_n$. Assume the eigenvalues have distinct magnitudes:

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|.$$

We now connect orthogonal iteration with the QR algorithm. Start with a unitary matrix $Q_0 \in \mathbb{C}^{n \times n}$. The orthogonal iteration method updates as follows:

$$AQ_k = Q_{k+1} R_{k+1}, \tag{38.2}$$

where Q_{k+1} is unitary and R_{k+1} is upper triangular. As we saw earlier, if the eigenvalues of A have distinct magnitudes, then Q_k converges to a unitary matrix Q such that $Q^* A Q$ is upper triangular. In other words, orthogonal iteration gives the Schur form $R = Q^* A Q$. To approximate this at step k, define

$$A_k = Q_k^* A Q_k. \tag{38.3}$$

As Q_k gets closer to Q, the matrix A_k gets closer to R.

Now comes the interesting part: the update rules in (38.2) and (38.3) actually define the QR algorithm. To see this, use (38.2) to write

$$A_k = Q_k^* Q_{k+1} R_{k+1} = \widetilde{Q}_{k+1} R_{k+1},$$

where $\widetilde{Q}_{k+1} = Q_k^* Q_{k+1}$ is unitary. This is a QR factorization of A_k.

To get the next iterate, reverse the factors:

$$\begin{aligned} R_{k+1}\widetilde{Q}_{k+1} &= \widetilde{Q}_{k+1}^* A_k \widetilde{Q}_{k+1} \\ &= Q_{k+1}^* Q_k (Q_k^* A Q_k)(Q_k^* Q_{k+1}) \\ &= Q_{k+1}^* (Q_k Q_k^*) A (Q_k Q_k^*) Q_{k+1} \\ &= Q_{k+1}^* A Q_{k+1} = A_{k+1}. \end{aligned}$$

So each step of the QR algorithm corresponds exactly to one step of orthogonal iteration, just expressed in a different form. We factor A_k as $A_k = \widetilde{Q}_{k+1} R_{k+1}$, then form $A_{k+1} = R_{k+1}\widetilde{Q}_{k+1}$, and repeat. This is precisely the QR algorithm.

38.4 The Arnoldi procedure

The Arnoldi process is an algorithm for finding a few eigenvalues of a matrix A, especially when A is large and sparse. It is well suited for this setting because it relies only on matrix-vector multiplications, which are fast and efficient when A is sparse.

Some of the methods we have studied, such as the power method and shift-and-invert, are designed to find one eigenvalue at a time. Others, like the QR algorithm, compute all the eigenvalues of a matrix. The Arnoldi method takes a different approach: it focuses on computing just a few eigenvalues, typically those with largest or smallest magnitude, without needing to find the full spectrum.

38.4.1 Partial Hessenberg reduction and the Arnoldi algorithm

In Section 28.3.1, we saw how to reduce a matrix A to upper-Hessenberg form. This means finding a unitary matrix Q and an upper-Hessenberg matrix H such that

$$H = Q^* A Q \quad \text{or, equivalently,} \quad AQ = QH.$$

As we discussed in Chapter 37, this reduction is the first step of the QR algorithm and helps simplify the computations that follow.

The Arnoldi process starts in a similar way but with an important difference: it builds the matrices Q and H one column at a time. When A is a large $n \times n$ matrix, computing the full Hessenberg reduction can be too costly. Instead, we pick a smaller number $m \ll n$ and compute only the first $m+1$ columns of Q, along with the top $(m+1) \times m$ block of H, which we denote by $H_{m+1,m} = H[1 : m+1, 1 : m]$. Later, we will see why this partial reduction is often enough. But first, let us show how the Arnoldi process computes Q and H recursively.

The Arnoldi relation

Let Q be an orthogonal matrix with columns $q_1, q_2, \ldots, q_n$, and let H be an upper-Hessenberg matrix such that

$$AQ = QH.$$

The first m columns of AQ are

$$A\begin{bmatrix} q_1 & q_2 & \cdots & q_m \end{bmatrix} = AQ_m.$$

On the other hand, the first m columns of QH are given by

$$QH[:, 1:m] = \begin{bmatrix} q_1 & q_2 & \cdots & q_m & q_{m+1} \end{bmatrix} H[1:m+1, 1:m] = Q_{m+1}H_{m+1,m},$$

where

$$H_{m+1,m} = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1m} \\ h_{21} & h_{22} & \cdots & h_{2m} \\ & \ddots & \ddots & \vdots \\ & & h_{m,m-1} & h_{mm} \\ & & & h_{m+1,m} \end{bmatrix} \in \mathbb{C}^{(m+1)\times m}.$$

This shows that computing the first m columns of $AQ = QH$ only requires the first $m+1$ columns of Q.

The following example illustrates why we only need the first $m+1$ columns of Q to compute the first m columns of QH

Example 38.13 Assume $n = 6$. Then,

$$QH = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 \end{bmatrix} \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{14} & h_{15} & h_{16} \\ h_{21} & h_{22} & h_{23} & h_{24} & h_{25} & h_{26} \\ 0 & h_{32} & h_{33} & h_{34} & h_{35} & h_{36} \\ 0 & 0 & h_{43} & h_{44} & h_{45} & h_{46} \\ 0 & 0 & 0 & h_{54} & h_{55} & h_{56} \\ 0 & 0 & 0 & 0 & h_{65} & h_{66} \end{bmatrix}.$$

Suppose we are only interested in the first $m = 3$ columns of QH:

$$QH[:, 1:3] = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 \end{bmatrix} \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ 0 & h_{32} & h_{33} \\ 0 & 0 & h_{43} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Because the last two rows of the submatrix are zero, the multiplication simplifies to

$$\begin{bmatrix} q_1 & q_2 & q_3 & q_4 \end{bmatrix} \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ 0 & h_{32} & h_{33} \\ 0 & 0 & h_{43} \end{bmatrix}.$$

By focusing on the first m columns of $AQ = QH$, we arrive at

$$AQ_m = Q_{m+1}H_{m+1,m}, \tag{38.4}$$

where $Q_{m+1} = \begin{bmatrix} Q_m & q_{m+1} \end{bmatrix}$ contains the first $m+1$ columns of Q, and $H_{m+1,m} = H[1 : m+1, 1 : m]$ consists of the first $m+1$ rows and first m columns of H. This identity is known as the **Arnoldi relation**.

To make this expression more useful later on, let us break it down further. Let H_m be the $m \times m$ matrix obtained by removing the last row of $H_{m+1,m}$. Then we can write

$$Q_{m+1}H_{m+1,m} = Q_m H_m + q_{m+1} \begin{bmatrix} 0 \cdots 0 & h_{m+1,m} \end{bmatrix}.$$

This second term is a rank-one update and can be written as $h_{m+1,m}\, q_{m+1} e_m^T$, where e_m is the m-th standard basis vector in $\mathbb{R}^m$. So the Arnoldi relation becomes

$$AQ_m = Q_m H_m + h_{m+1,m}\, q_{m+1} e_m^T. \tag{38.5}$$

The Arnoldi relation (38.4) allows us to compute the columns of Q and H recursively, one at a time. We start with a unit vector q_1, usually chosen at random.

The first column of the relation gives

$$Aq_1 = h_{11}q_1 + h_{21}q_2.$$

Taking the dot product with q_1 and using the orthonormality of q_1 and q_2, we find

$$h_{11} = q_1^* A q_1.$$

Rearranging, we isolate the component orthogonal to q_1:

$$h_{21}q_2 = Aq_1 - h_{11}q_1.$$

Taking norms, we get

$$h_{21} = \|Aq_1 - h_{11}q_1\|_2.$$

If $h_{21} \neq 0$, we normalize to get

$$q_2 = \frac{1}{h_{21}} \left(Aq_1 - h_{11}q_1 \right).$$

To find q_3, we use the second column of the Arnoldi relation:

$$Aq_2 = h_{12}q_1 + h_{22}q_2 + h_{32}q_3.$$

Taking the dot products with q_1 and q_2 gives

$$h_{12} = q_1^* A q_2 \quad \text{and} \quad h_{22} = q_2^* A q_2.$$

We then isolate the part orthogonal to q_1 and q_2:

$$h_{32}q_3 = Aq_2 - h_{12}q_1 - h_{22}q_2.$$

Again, we compute the norm:

$$h_{32} = \|Aq_2 - h_{12}q_1 - h_{22}q_2\|_2,$$

and if $h_{32} \neq 0$, we normalize:

$$q_3 = \frac{1}{h_{32}} \left(Aq_2 - h_{12}q_1 - h_{22}q_2\right).$$

This procedure continues in the same way for each subsequent column. In general, for the mth step we have

$$Aq_m = \sum_{j=1}^{m} h_{jm}q_j + h_{m+1,m}q_{m+1}.$$

Using the orthonormality of $q_1, \ldots, q_m$, each coefficient h_{jm} is computed as $q_j^* Aq_m$ for $j = 1, 2, \ldots, m$. Then, we obtain

$$h_{m+1,m}q_{m+1} = Aq_m - \sum_{j=1}^{m} h_{jm}q_j.$$

The value $h_{m+1,m}$ is given by

$$h_{m+1,m} = \left\|Aq_m - \sum_{j=1}^{m} h_{jm}q_j\right\|_2,$$

and, if $h_{m+1,m} \neq 0$, we obtain

$$q_{m+1} = \frac{1}{h_{m+1,m}} \left(Aq_m - \sum_{j=1}^{m} h_{jm}q_j\right).$$

In this way, the Arnoldi process builds the orthonormal basis $q_1, q_2, \ldots, q_{m+1}$ and fills in the corresponding entries of the matrix H. These steps make up the Arnoldi process, which is described in detail in Algorithm 7.

Starting from a single unit vector q_1, the algorithm produces an orthonormal sequence for any given m. The process continues as long as the subdiagonal entries $h_{i+1,i}$ are nonzero. If one of them becomes zero, the method stops. This is called a **breakdown**. Surprisingly, breakdown is not a failure but a good sign, as we will explain later.

Algorithm 7 Arnoldi process

Inputs:
An $n \times n$ matrix A, a unit vector $q_1 \in \mathbb{C}^n$
for $i = 1, 2, 3, \ldots, m$ **do**
  $w = Aq_i$
  for $j = 1, 2, \ldots, i$ **do**
    $h_{ji} = q_j^* w$
    $w = w - h_{ji} q_j$
  end for
  $h_{i+1,i} = \|w\|_2$
  $q_{i+1} = w / h_{i+1,i}$
end for

Example 38.14 Consider the 6×6 matrix

$$A = \begin{bmatrix} 7 & -3 & 1 & 4 & -2 & 5 \\ 2 & 6 & -4 & 3 & 7 & -1 \\ -1 & 2 & 5 & 0 & -3 & 4 \\ 3 & -6 & 2 & 8 & 1 & -5 \\ -4 & 3 & 5 & -2 & 9 & 6 \\ 7 & -7 & 4 & 1 & 3 & 10 \end{bmatrix}.$$

Our goal is to compute the first three columns of an orthogonal matrix Q and the first two columns of an upper-Hessenberg matrix H such that $A = QHQ^T$.

We start by choosing a unit vector q_1. For simplicity, we take $q_1 = e_1$, the first standard basis vector. Then,

$$Aq_1 = \begin{bmatrix} 7 & 2 & -1 & 3 & -4 & 5 \end{bmatrix}^T.$$

The nonzero entries in the first column of H are

$$h_{11} = q_1^T A q_1 = 7, \quad \text{and}$$
$$h_{21} = \|Aq_1 - h_{11} q_1\|_2 \approx 7.42.$$

We then obtain the second column of Q:

$$q_2 = \frac{1}{h_{21}}(Aq_1 - h_{11} q_1) \approx \begin{bmatrix} 0.00 & 0.27 & -0.13 & 0.40 & -0.54 & 0.67 \end{bmatrix}^T.$$

Next, we compute the third column of Q and the nonzero entries in the second column of H. First, we find

$$Aq_2 \approx \begin{bmatrix} 5.12 & -1.08 & 4.18 & -2.56 & -1.48 & 3.10 \end{bmatrix}^T.$$

Then,

$$
\begin{aligned}
h_{12} &= q_1^T A q_2 \approx 5.12, \\
h_{22} &= q_2^T A q_2 \approx 1.00, \quad \text{and} \\
h_{23} &= \|A q_2 - h_{12} q_1 - h_{22} q_2\|_2 \approx 6.00.
\end{aligned}
$$

Finally, we obtain the third column of Q:

$$
q_3 = \frac{1}{h_{23}}(A q_2 - h_{12} q_1 - h_{22} q_2) \approx \begin{bmatrix} 0.00 & -0.22 & 0.72 & -0.49 & -0.16 & 0.40 \end{bmatrix}^T.
$$

At the end of this process, we have constructed an orthonormal set $\{q_1, q_2, q_3\}$ and the corresponding entries of H. The matrix Q_3 containing the first three columns of Q and the corresponding upper-Hessenberg submatrix are approximately

$$
Q_3 \approx \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.27 & -0.22 \\ 0 & -0.13 & 0.72 \\ 0 & 0.40 & -0.49 \\ 0 & -0.54 & -0.16 \\ 0 & 0.67 & 0.40 \end{bmatrix} \quad \text{and} \quad H_{3,2} \approx \begin{bmatrix} 7.00 & 5.12 \\ 7.42 & 1 \\ 0 & 6.00 \end{bmatrix}.
$$

What if we run the Arnoldi process indefinitely? Will it ever stop?

Hessenberg reduction through Arnoldi

After $n-1$ steps, the Arnoldi process produces an orthonormal basis $\{q_1, q_2, \ldots, q_n\}$ for $\mathbb{C}^n$, assuming none of the subdiagonal entries $h_{i+1,i}$ are zero. At this point, the vector $A q_n$ must lie in the span of these basis vectors. That is,

$$
A q_n = (q_1^* A q_n) q_1 + \cdots + (q_n^* A q_n) q_n.
$$

So

$$
h_{n+1,n} = \left\| A q_n - \sum_{i=1}^{n} (q_i^* A q_n) q_i \right\|_2 = 0,
$$

and the process stops.

In this case, the Arnoldi relation (38.5) becomes

$$
A Q_n = Q_n H,
$$

where

$$
Q_n = \begin{bmatrix} q_1 & \cdots & q_n \end{bmatrix}
$$

is unitary, and H is an upper Hessenberg matrix unitarily similar to A.

In short, the Arnoldi process always breaks down after at most n steps. If no breakdown happens before that, the method reduces A to upper Hessenberg form

$$
H = Q_n^* A Q_n.
$$

This matrix H has the same eigenvalues as A. In practice, though, we rarely run Arnoldi for all n steps. Instead, we stop after computing a smaller number of columns of Q, based on how many eigenvalues we want[5].

38.4.2 Krylov subspaces

The Arnoldi process is closely related to the power method, as we will see next. To understand the connection, we first introduce an important type of subspace: the Krylov subspace.

Definition 38.15. (Krylov subspaces)

Let $A \in \mathbb{C}^{n\times n}$ and $v \in \mathbb{C}^n$. For any positive integer m, the **order-m Krylov subspace** is

$$\mathcal{K}_m(A, v) := \text{span}\{v, Av, A^2v, \ldots, A^{m-1}v\}.$$

The Krylov sequence $v, Av, A^2v, \ldots, A^{m-1}v$ used to build the Krylov subspace $\mathcal{K}_m(A, v)$ consists of precisely the same vectors generated by the power method. It is no surprise that this sequence and the associated Krylov subspace play an important role in solving eigenvalue problems.

Example 38.16 Consider the matrix and the vector

$$A = \begin{bmatrix} 2 & 3 & -1 \\ 1 & 1 & 2 \\ 0 & 2 & 3 \end{bmatrix} \qquad \text{and} \qquad v = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

The first four terms of the Krylov sequence associated with A and v are

$$\left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 7 \\ 3 \\ 2 \end{bmatrix}, \begin{bmatrix} 21 \\ 14 \\ 12 \end{bmatrix} \right\}.$$

Now, let us compute the Krylov subspaces $\mathcal{K}_m(A, v)$ for increasing values of m.

For $m = 1$, we obtain

$$\mathcal{K}_1(A, v) = \text{span} \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \right\},$$

which corresponds to the x_1-axis in $\mathbb{R}^3$.

For $m = 2$, we obtain

[5] If we want the full Hessenberg form, the method in Section 28.3.1, which uses Householder reflectors, is usually preferred because it is more efficient.

$$\mathcal{K}_2(A, v) = \text{span}\left\{\begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}2\\1\\0\end{bmatrix}\right\},$$

which spans the (x_1, x_2)-plane in $\mathbb{R}^3$.

For $m = 3$, we obtain

$$\mathcal{K}_3(A, v) = \text{span}\left\{\begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}2\\1\\0\end{bmatrix}, \begin{bmatrix}7\\3\\2\end{bmatrix}\right\} = \mathbb{R}^3.$$

Once the Krylov subspace spans the entire space $\mathbb{R}^3$, it cannot grow further. For all $m \geqslant 4$, we have

$$\mathcal{K}_m(A, v) = \mathbb{R}^3.$$

The previous example illustrates that Krylov subspaces associated with a matrix and a vector do not grow indefinitely. In that case, the process stopped once $\mathcal{K}_3(A, v) = \mathbb{R}^3$, since the subspace had already reached the entire space.

Now, let us show the connection between the sequence $\{q_1, q_2, \ldots, q_m\}$ of orthonormal vectors generated by the Arnoldi process and the Krylov subspaces.

Theorem 38.17. (Arnoldi iteration and Krylov subspace bases)

Let A be an $n \times n$ matrix, and let v be any nonzero vector. Set $q_1 = v/\|v\|_2$. If the Arnoldi process (Algorithm 7) proceeds without breakdown (i.e., $h_{i+1,i} \neq 0$ for $i = 1, \ldots, m$), it produces an orthonormal basis $\{q_1, q_2, \ldots, q_m\}$ for the Krylov subspace $\mathcal{K}_m(A, v)$. In other words,

$$\mathcal{K}_m(A, v) = \text{span}\{v, Av, \ldots, A^{m-1}v\} = \text{span}\{q_1, q_2, \ldots, q_m\}.$$

Proof We prove the theorem by induction on m.

The first Krylov subspace is $\mathcal{K}_1(A, v) = \text{span}\{v\}$. The Arnoldi process starts by setting $q_1 = v/\|v\|_2$. Since q_1 is a scalar multiple of v, it forms an orthonormal basis for $\mathcal{K}_1(A, v)$.

Assume that for some $i \geqslant 1$, the Arnoldi process has generated an orthonormal basis $\{q_1, q_2, \ldots, q_i\}$ for $\mathcal{K}_i(A, v) = \text{span}\{v, Av, \ldots, A^{i-1}v\}$. We will show that the next vector, q_{i+1}, extends this basis to a basis for $\mathcal{K}_{i+1}(A, v)$.

From the Arnoldi relation, we have

$$Aq_i = h_{1i}q_1 + h_{2i}q_2 + \cdots + h_{ii}q_i + h_{i+1,i}q_{i+1}.$$

The term $h_{1i}q_1 + h_{2i}q_2 + \cdots + h_{ii}q_i$ is in $\mathcal{K}_i(A, v)$ by the inductive hypothesis. On the other hand, $Aq_i \in \mathcal{K}_{i+1}(A, v)$ because q_i is a linear combination of $v, Av, \ldots, A^{i-1}v$ and thus Aq_i is a linear combination of $Av, A^2v, \ldots, A^iv$. It fol-

lows that

$$h_{i+1,i}q_{i+1} = Aq_i - \sum_{j=1}^{i} h_{ji}q_j$$

lies in $\mathcal{K}_{i+1}(A, v)$. Since $h_{i+1,i} \neq 0$ by assumption, we can divide by $h_{i+1,i}$ and conclude that

$$q_{i+1} = \frac{1}{h_{i+1,i}}\left(Aq_i - \sum_{j=1}^{i} h_{ji}q_j\right) \in \mathcal{K}_{i+1}(A, v).$$

The Arnoldi process also guarantees that q_{i+1} is orthogonal to $\{q_1, q_2, \ldots, q_i\}$. So the set $\{q_1, q_2, \ldots, q_{i+1}\}$ is orthonormal and lies in $\mathcal{K}_{i+1}(A, v)$. Since this Krylov subspace is generated by $i + 1$ vectors, its dimension is at most $i + 1$. Thus, the q_j's form an orthonormal basis for $\mathcal{K}_{i+1}(A, v)$. □

38.4.3 Ritz values and Ritz vectors

Now let us see how to approximate the eigenvalues of A using the Arnoldi process. We start with the Arnoldi relation (38.5). If $h_{m+1,m} = 0$ (that is, if Arnoldi breaks down), this relation becomes

$$AQ_m = Q_m H_m.$$

By Theorem 38.6, the columns of Q_m span an invariant subspace of A, the eigenvalues of H_m are also eigenvalues of A, and any eigenvector x of H_m gives an eigenvector of A as $v = Q_m x$.

In practice, though, $h_{m+1,m}$ is almost never exactly zero. When $h_{m+1,m}$ is small,[6] the eigenvalues of H_m, called **Ritz values**, serve as good approximations to some of the eigenvalues of A. Since H_m is much smaller than A, this makes the computation efficient. Likewise, the eigenvectors of H_m, called **Ritz vectors**, give approximate eigenvectors of A when we multiply them by Q_m.

Let us summarize the procedure for approximating the eigenvalues and eigenvectors of an $n \times n$ matrix A using the Arnoldi process.

Approximating eigenvalues and eigenvectors with Arnoldi

To approximate a few eigenvalues and eigenvectors of A:

1. Choose the number of Arnoldi iterations m, typically $m \ll n$.
2. Run the Arnoldi process (see Algorithm 7) to compute Q_m, H_m, $h_{m+1,m}$, and q_{m+1} so that the Arnoldi relation (38.5) holds.
3. Compute the eigenvalues (called **Ritz values**) and eigenvectors (called **Ritz vectors**) of H_m.
4. Use the Ritz values as approximations of A's eigenvalues, and multiply the Ritz vectors by Q_m to approximate the corresponding eigenvectors of A.

[6] As we will see in Theorem 38.20, good approximations are still possible even when $h_{m+1,m}$ is not very small.

Let us make a few observations about the procedure above. First, it reduces the large eigenvalue problem $Av = \lambda v$ to a much smaller one, $H_m x = \lambda x$. This smaller problem is easier to solve, especially since H_m is already in Hessenberg form, and we can use the QR algorithm from Section 37.4.4 to compute its eigenvalues.

Another way to view the approximation of an eigenvector of A by a vector of the form $Q_m x$ (a linear combination of the columns of Q_m) is to see it as searching for eigenvector approximations inside the Krylov subspace

$$\mathcal{K}_m(A, v) = \text{col}(Q_m).$$

But why would $\mathcal{K}_m(A, v)$ contain good approximations of A's eigenvectors? And if it does, which ones are being approximated?

To understand this, notice that the Arnoldi process builds its subspace using the same sequence of vectors as the power method: $v, Av, A^2v, \ldots, A^{m-1}v$. We know that this sequence tends to align with the dominant eigenvector associated with the largest eigenvalue of A. The power method uses only a scalar multiple of $A^{m-1}v$, so its approximation lies in the one-dimensional subspace $\text{span}(A^{m-1}v)$.

The Arnoldi process, in contrast, constructs the larger Krylov subspace $\mathcal{K}_m(A, v)$ from all these vectors. A subspace spanned by several vectors contains more information than just one vector. This means $\mathcal{K}_m(A, v)$ can be used to approximate not only the dominant eigenvector but also those associated with the next largest eigenvalues.

38.4.4 The Arnoldi process in action: numerical examples

Now that we have covered the theory behind the Arnoldi process, let us see it in action through a series of numerical examples that illustrate how to approximate eigenvalues and eigenvectors in practice.

For our first example, we use a small matrix ($n = 7$). However, with very small matrices the Arnoldi process does not have many iterations to gradually improve its approximations. In fact, if the process runs until the Hessenberg matrix is the same size as A, it essentially reconstructs the entire matrix and recovers its exact eigenvalues.

Example 38.18 Let us consider the 7×7 matrix

$$A = \begin{bmatrix} 3 & 2 & 0 & 0 & 0 & 1 & 0 \\ 2 & 5 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 4 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 6 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 7 & 2 & 1 \\ 1 & 0 & 0 & 0 & 2 & -3 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 8 \end{bmatrix}.$$

Table 38.1 shows the eigenvalues of A in the first column and the Ritz values obtained during each iteration of the Arnoldi process in the other columns. Each Ritz value is placed in the row corresponding to the eigenvalue it is closest to.

Eigenvalues	m=1	m=2	m=3	m=4	m=5	m=6	m=7
-3.55			-3.17	-3.52	-3.55	-3.55	-3.55
1.68		0.96		1.86	1.71	1.68	1.68
2.75	3.00		2.03		2.98	2.76	2.75
6.04		5.44		5.48	6.30	6.16	6.04
6.52			6.43	6.96			6.52
7.64						7.17	7.64
8.92					8.20	8.78	8.92

Table 38.1: Ritz values for increasing Arnoldi iterations compared with the exact eigenvalues.

Aside from the eigenvalue -3.55, the approximations to the eigenvalues of A are not very accurate until $m = 6$. At the final step ($m = 7$), the Hessenberg matrix H_7 is orthogonally similar to A, so it has the same eigenvalues.

The Arnoldi process works best on large matrices, where the Krylov subspace captures the dominant eigenvectors without needing the full matrix size. So let us now consider a few examples using larger matrices. For these examples, we will do a bit of matrix shopping and use two matrices from the "Matrix Market" [9].

Example 38.19 For the first example, we use the "add20" matrix, from the Matrix Market, which is of size 2395×2395. The dominant eigenvalues of this matrix are real and lie in the interval $[0.71, 0.73]$. We will approximate the five largest eigenvalues of A using the five largest Ritz values.

Figure 38.5 shows how the five Ritz values evolve over 35 Arnoldi steps, with a zoom-in of the last five steps. The Ritz values converge smoothly to the five largest eigenvalues of A.

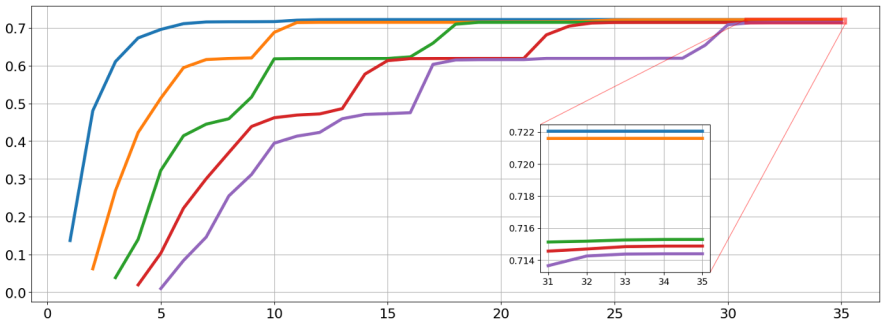


Fig. 38.5: Convergence of the five largest Ritz values to the dominant eigenvalues of the "add20" matrix over 35 Arnoldi steps.

In practice, we do not know the eigenvalues of A, since finding them is the goal. To check how good an approximate eigenpair (λ, v) is, we usually compute the residual $\|Av - \lambda v\|_2$.

With Arnoldi, we work with large matrices, so it is important to keep computations efficient. The good news is that the Arnoldi process gives us the residuals almost for free, as the next theorem shows.

Theorem 38.20. (Residual from the Arnoldi process)

Let $A \in \mathbb{C}^{n\times n}$, and let Q_m, q_{m+1}, H_m, and $h_{m+1,m}$ be obtained from the Arnoldi process, satisfying the Arnoldi relation (38.5). Suppose λ is an eigenvalue of H_m with associated eigenvector x. Define $v = Q_m x$. Then, the residual is given by

$$\|Av - \lambda v\|_2 = |h_{m+1,m}|\,|x_m|,$$

where x_m is the last entry of x.

Proof Let λ and x be an eigenvalue and eigenvector of H_m, respectively. Define $v := Q_m x$. Substituting v into the Arnoldi relation (38.5) gives

$$\begin{aligned} Av = A(Q_m x) &= Q_m(H_m x) + h_{m+1,m} q_{m+1} \left(e_m^T x\right) \\ &= \lambda Q_m x + h_{m+1,m} q_{m+1} \left(e_m^T x\right) \\ &= \lambda v + h_{m+1,m} q_{m+1} x_m. \end{aligned}$$

Taking the 2-norm of both sides and noting that q_{m+1} is a unit vector, we get

$$\|Av - \lambda v\|_2 = \|h_{m+1,m} q_{m+1} x_m\|_2 = |h_{m+1,m}|\,|x_m|,$$

the desired result. □

Theorem 38.20 explains what happens when the Arnoldi process breaks down, that is, when $h_{m+1,m} = 0$ at some iteration. In this case, the residual $\|Av - \lambda v\|_2$ is zero for every eigenvalue λ of H_m, which means that all the eigenvalues of H_m are exact eigenvalues of A. Moreover, the Krylov subspace $\mathcal{K}_m(A, v)$ becomes an invariant subspace of A (i.e., $A\,(\mathcal{K}_m(A, v)) \subseteq \mathcal{K}_m(A, v)$), and it remains unchanged for all subsequent iterations, that is, $\mathcal{K}_m(A, v) = \mathcal{K}_{m+1}(A, v) = \mathcal{K}_{m+2}(A, v) = \cdots$.

Example 38.21 For this example, we use the "west0479" matrix from the Matrix Market, which is a 479×479 matrix. Our goal is to approximate the eight eigenvalues of A with the largest magnitudes, along with their corresponding eigenvectors, by computing the eight largest Ritz values and their associated Ritz vectors.

In the left panel of Figure 38.6, the eigenvalues of A are shown together with the eight largest Ritz values obtained after 50 Arnoldi steps. Note that some of the

eigenvalues are complex. Visually, the Ritz values appear to closely match the eight eigenvalues of A with the largest magnitudes[7].

The right panel of Figure 38.6 displays the residuals $\|Av - \lambda v\|_2$ for these eight Ritz values and their corresponding approximate eigenvectors. After 50 Arnoldi steps, the residuals are nearly at the level of unit roundoff, indicating that these are excellent approximations of the eigenvalues and eigenvectors.

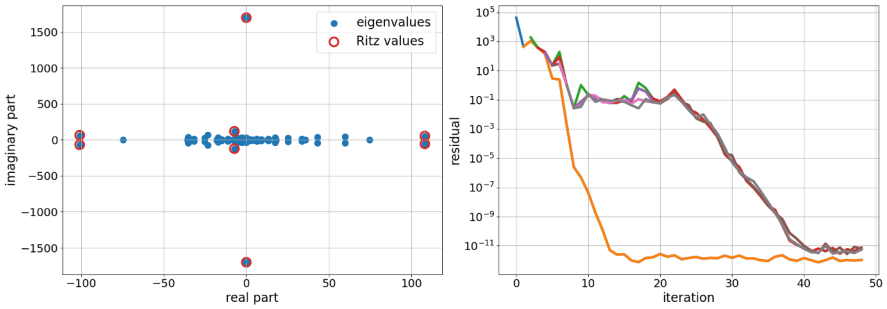


Fig. 38.6: Left: Eigenvalues of A and the eight largest Ritz values after 50 Arnoldi steps. Right: Residuals for the eight largest Ritz values and corresponding approximate eigenvectors.

Similar to the power method, if we are interested in interior eigenvalues (for example, the smallest ones or those closest to a specific target), we can replace A with $(A - \mu I_n)^{-1}$. Here, μ is the target value, and this transformation shifts the spectrum so that the eigenvalue of A closest to μ becomes the dominant eigenvalue of $(A - \mu I_n)^{-1}$. After computing a Ritz value λ of $(A - \mu I_n)^{-1}$, we recover the approximate eigenvalue of A using the formula

$$\frac{1}{\lambda} + \mu.$$

This technique is known as the **shift-and-invert Arnoldi process**.

Example 38.22 Let us revisit the "west0479" matrix from the Matrix Market. This time, we apply the Arnoldi process to $(A - \mu I_n)^{-1}$ to compute $m = 10$ Ritz values. We use two different shifts: $\mu = 0$ and $\mu = 0.5$. With $\mu = 0$, we target the ten smallest eigenvalues of A, while with $\mu = 0.5$, we target the ten eigenvalues closest to 0.5.

Figure 38.7 shows two zoomed-in views of the spectrum of A: one around $\mu = 0$ (left panel) and the other around $\mu = 0.5$ (right panel). The figure displays both the eigenvalues of A and the corresponding Ritz values.

[7] Be cautious when visually judging which eigenvalues are the largest. Differences in the horizontal and vertical axis scales can be misleading!

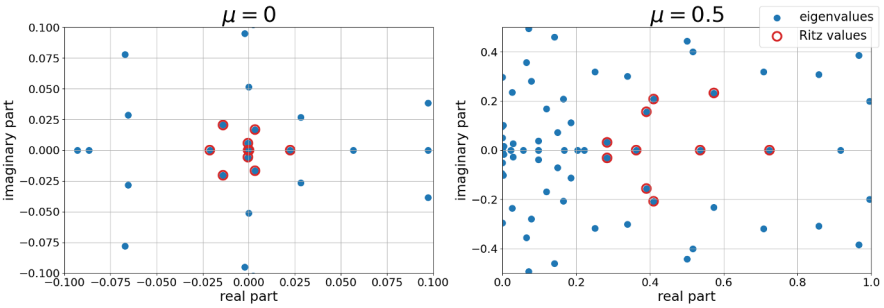


Fig. 38.7: Zoomed-in views of the spectrum of A around $\mu = 0$ (left) and $\mu = 0.5$ (right), showing eigenvalues and ten Ritz values computed using the shift-and-invert Arnoldi process.

Problems

38.1 Let $A \in \mathbb{C}^{n\times n}$, and let λ be an eigenvalue of A. Prove that for any submultiplicative norm $\|\cdot\|$, the following inequality holds:

$$|\lambda| \leqslant \|A\|.$$

38.2 Consider the matrix $A = \begin{bmatrix} 2 & -1 \\ 2 & 0 \end{bmatrix}$.

(a) Find the eigenvalues of A.
(b) Determine the Gershgorin circles of A.
(c) Verify that one of the circles does not contain an eigenvalue. Does this violate Gershgorin's theorem?

38.3 Apply Gershgorin's theorem to establish the existence of at least two real eigenvalues for the matrix

$$A = \begin{bmatrix} 1 & 0 & -2 & 0 \\ 0 & 12 & 0 & -4 \\ 1 & 0 & -1 & 0 \\ 0 & 5 & 0 & 0 \end{bmatrix}.$$

38.4 Consider the matrix

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & -6 & 0 \\ 0 & 1 & 9 \end{bmatrix}.$$

(a) Apply Gershgorin's circle theorem to determine a region that contains the eigenvalues of A.
(b) Can the matrix A have complex eigenvalues?
(c) Is the matrix A diagonalizable?

38.5 A matrix is said to be **(row) stochastic** if all its entries are nonnegative numbers and the sum of the entries in the ith row equals 1, for $i = 1, ..., n$. Show that, if λ is an eigenvalue of a stochastic matrix, then $|\lambda| \leqslant 1$.

38.6 Does every Gershgorin circle always contain exactly one eigenvalue?

38.7 Let $A \in \mathbb{R}^{n\times n}$. Illustrate with the matrix below that all the eigenvalues of A lie in the intersection of the union of the Gerschgorin disks of A and A^T.

$$A = \begin{bmatrix} 1 & 1/2 & 1/3 \\ 2 & 1 & 2/3 \\ 3 & 3/2 & 1 \end{bmatrix}.$$

38.8 Use the Gershgorin circles of A or A^T to show that the matrix

$$A = \begin{bmatrix} 2 & -1 & 1 & -1 \\ 1 & -3 & 1 & 1 \\ 0 & 1 & -5 & -1 \\ -1/2 & 0 & -1 & 4 \end{bmatrix}$$

is invertible.

38.9 Let $A \in \mathbb{R}^{n\times n}$ and let $d_1, \ldots, d_n$ be positive real numbers. Show that all the eigenvalues of A lie in the region

$$\bigcup_{i=1}^{n} \left\{ z \in \mathbb{C} \mid |z - A_{ii}| \leqslant \frac{1}{d_i} \sum_{j=1, j\neq i}^{n} d_j |A_{ij}| \right\}.$$

38.10 Consider the matrix

$$A = \begin{bmatrix} 7 & -16 & 8 \\ -16 & 7 & -8 \\ 8 & -8 & -5 \end{bmatrix}.$$

Use Gerschgorin theorem to say as much as you can about the location of the eigenvalues of A. Then, consider diagonal similarities of A and see if you can obtain any improvement in your location of the eigenvalues. Finally, compute the actual eigenvalues and comment on how well you did with the estimates.

38.11 Consider an $n \times n$ matrix A satisfying the condition $|A_{ii}| > \sum_{j\neq i} |A_{ij}|$ for $i = 1, \ldots, n$. Prove that A is invertible.

38.12 Let $A \in \mathbb{C}^{n\times n}$ with $n \geqslant 2$. Let S be a proper subset of $N = \{1, 2, \ldots, n\}$. Let

$$\mathcal{G}_S(A) := \bigcup_{i\in S} \mathcal{G}_i(A),$$

where $\mathcal{G}_i(A)$ denotes the Gershgorin disk of A associated with its ith row. If $\mathcal{G}_S(A) \cap \mathcal{G}_{N-S}(A) = \varnothing$, prove that $\mathcal{G}_S(A)$ contains exactly $|S|$ eigenvalues of A.

38.13 A matrix $A \in \mathbb{C}^{n\times n}$ is **strictly diagonally dominant** if $|A_{ii}| > \sum_{j\neq i} |A_{ij}|$ for all $i \in \{1, 2, ..., n\}$.

The Strict Diagonal Dominance Theorem says: If $A \in \mathbb{C}^{n\times n}$ is strictly diagonally dominant, then A is nonsingular.

Prove that the Gershgorin's Theorem and the Strict Diagonal Dominance Theorem are equivalent, that is, if you assume true one of them, then you can prove the other one and vice versa.

38.14 Let $A \in \mathbb{R}^{n\times n}$ and assume that

$$|A_{ii} - A_{jj}| \geqslant r_i(A) + r_j(A), \quad \text{for all } i \neq j,$$

where $r_i(A)$ and $r_j(A)$ denote, respectively, the radius of the ith and jth Gershgorin disks of A. Prove that all the eigenvalues of A are real.

38.15 Compute the eigenvalue nearest to 2 and the corresponding eigenvector of the matrix

$$A = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

First, use inverse iteration to solve the problem. Then, solve the same problem using the Rayleigh quotient iteration. Compare the convergence speed of both methods.

38.16 The Rayleigh quotient iteration is computationally expensive because solving a linear system in each step typically takes about n^3 operations. However, for certain types of matrices, the process can be more efficient. Show that an $n \times n$ matrix A in upper Hessenberg form can be factorized into LU form (with or without partial pivoting) in about n^2 operations.

38.17 Write a program (in MATLAB, Python, or any other language of your choice) that implements the orthogonal iteration algorithm. Use your program to compute the invariant subspace associated with the largest eigenvalue of a matrix A. Test your algorithm with the following matrices

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 6 & -2 & 1 \\ -2 & 5 & -3 \\ 1 & -3 & 7 \end{bmatrix}.$$

38.18 Suppose A is a matrix in upper Hessenberg form. How does the orthogonal iteration algorithm behave when applied to upper Hessenberg matrices? Does it retain the Hessenberg structure at each step?

38.19 In Chapter 37, we discussed using the QR algorithm with deflation to reduce the size of a matrix after computing an eigenvalue. Show that each deflation step in the QR algorithm corresponds to removing a direction from the subspace used in orthogonal iteration.

38.20 The goal of the QR algorithm with shifts is to improve rate of convergence. Show that this algorithm can be viewed as a generalization of orthogonal iteration with dynamic changes to the subspace.

38.21 Consider the matrix

$$A = \begin{bmatrix} 7 & -3 & 1 & 4 & -2 & 5 \\ 2 & 6 & -4 & 3 & 7 & -1 \\ -1 & 2 & 5 & 0 & -3 & 4 \\ 3 & -6 & 2 & 8 & 1 & -5 \\ -4 & 3 & 5 & -2 & 9 & 6 \\ 5 & -7 & 4 & 1 & 3 & 10 \end{bmatrix}.$$

Work through the Arnoldi process for $m = 4$, starting with $q_1 = e_1$ (the first standard basis vector), by hand.

38.22 Let $A \in \mathbb{C}^{n\times n}$ and $v \in \mathbb{C}^n$. Prove the following:

(a) $\mathcal{K}_m(A, v) \subseteq \mathcal{K}_{m+1}(A, v)$.
(b) $A\,(\mathcal{K}_m(A, v)) := \{Ax \mid x \in \mathcal{K}_m(A, v)\} \subseteq \mathcal{K}_{m+1}(A, v)$.

38.23 Let $A \in \mathbb{C}^{n\times n}$ and $v \in \mathbb{C}^n$. Suppose $v, Av, \ldots, A^{m-1}v$ are linearly independent, and let $q_1, q_2, \ldots, q_m$ be the orthonormal vectors generated by the Arnoldi process starting with $q_1 = v/\|v\|_2$. Prove that $h_{m+1,m} = 0$ if and only if $v, Av, \ldots, A^{m-1}v, A^m v$ are linearly dependent. Further, show that this holds if and only if the Krylov subspace $\mathcal{K}_m(A, v)$ is an invariant subspace of A.

38.24 Choose a moderately sized matrix from the Matrix Market [9] and use both the Arnoldi and shift-and-invert Arnoldi methods to compute a few eigenvalues of the matrix.

Chapter 39
VIII-FROM THE CLASSROOM TO REAL LIFE

39.1 The Harris method for corner detection

In Chapter 35, we saw that detecting corners in images is important. Corners are points where edges meet at different angles, like window edges or where two walls join. These points are special because they stand out and are easier to track across different images. That makes them useful for tasks like matching images, detecting objects, or aligning photos to create a panorama. In this section, we will see how the Harris method uses eigenvalues to find these key points.

Our goal is to turn the intuitive idea of a corner into a precise definition that a computer can use. But how can we define what a corner is?

Figure 39.1 gives us a visual starting point. We focus on a small square region in the image, called a **patch**:

- **Left image**: The patch sits on a flat, uniform area. No matter how we shift it (left, right, up, or down) the patch looks the same. Nothing changes.
- **Middle image**: The patch covers an edge. If we slide it along the edge (say, up or down), the patch still looks similar. But if we move it across the edge (left or right), the patch changes noticeably. So the appearance changes in some directions but not in others.
- **Right image**: The patch sits on a corner. Now, any small movement, no matter the direction, causes a clear change in what the patch contains.

To define a corner mathematically, we need a way to measure how much the pixel values change when we shift a small patch of the image.

We begin by introducing a **window function**:

$$w(x, y) := \begin{cases} 1, & \text{if the pixel } (x, y) \text{ is inside the patch,} \\ 0, & \text{otherwise.} \end{cases}$$

The variables (x, y) represent pixel coordinates in the image. The patch is simply the set of pixels where $w(x, y) = 1$, usually a square or rectangular region. This function tells us which pixels belong to the patch and which do not.

Next, let $I(x, y)$ be the grayscale intensity at pixel (x, y). This value tells us how bright or dark the pixel is. Now suppose we shift the patch by a small amount (u, v).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_39

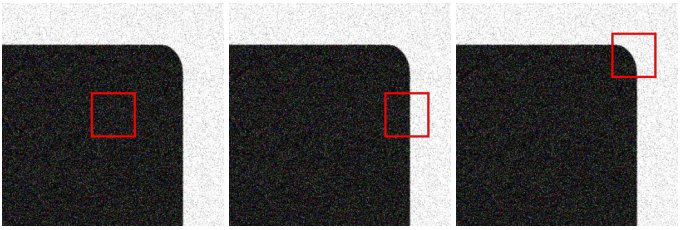


Fig. 39.1: Corners (right) are regions where changes happen in all directions. Edges (middle) have changes in some directions but stay the same in others. Flat (left) areas show no changes, no matter the direction.

Each pixel at (x, y) moves to $(x+u, y+v)$, and its intensity becomes $I(x+u, y+v)$. To measure how much the patch changes under this shift, we define:

$$E(u,v) := \sum_{x,y} w(x,y)\,(I(x+u,y+v) - I(x,y))^2\,.$$

This formula computes the total squared difference in intensity between the original patch and the shifted patch. In other words, $E(u,v)$ tells us how much the patch changes when moved by (u,v). If the region is flat (little intensity variation), the patch looks the same after the shift, so $E(u,v)$ is small. If the patch contains structure,like an edge or a corner, then shifting it will change it, and $E(u,v)$ will be larger.

Since we are interested in small shifts, we can approximate $I(x+u, y+v)$ using a Taylor expansion. Keeping only the first-order terms, we get

$$I(x+u,y+v) \approx I(x,y) + u\,I_x(x,y) + v\,I_y(x,y),$$

where $I_x(x,y)$ and $I_y(x,y)$ are the partial derivatives of the image intensity in the x- and y-directions. Substituting this into the expression for $E(u,v)$, we get

$$E(u,v) \approx \sum_{x,y} w(x,y)\left(u^2 I_x^2(x,y) + 2uv\,I_x(x,y)I_y(x,y) + v^2 I_y^2(x,y)\right).$$

We can express the approximation of $E(u,v)$ in compact matrix form:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix},$$

where the matrix M is defined as

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2(x,y) & I_x(x,y)I_y(x,y) \\ I_x(x,y)I_y(x,y) & I_y^2(x,y) \end{bmatrix}.$$

This expression for $E(u,v)$ is known as a *quadratic form*, a concept we will study in more detail in Chapter 47.

Approximating the partial derivatives I_x and I_y

In a digital image, the intensity function $I(x, y)$ is only defined at discrete points, so we cannot compute the partial derivatives I_x and I_y directly. Instead, we approximate them using numerical methods.

A simple approach is to use **forward differences**:

$$I_x \approx I(x+1, y) - I(x, y), \quad \text{and}$$
$$I_y \approx I(x, y+1) - I(x, y).$$

This method works, but it is not very accurate since it only uses two points for the calculation. A more accurate and widely used approach is the **Sobel method**, which considers eight neighboring points around a pixel at (x, y). It computes the partial derivatives as:

$$\begin{aligned} I_x \approx \frac{1}{8}(&I(x+1, y+1) - I(x-1, y+1) \\ &+ 2(I(x+1, y) - I(x-1, y)) + I(x+1, y-1) - I(x-1, y-1)) \quad \text{and} \\ I_y \approx \frac{1}{8}(&I(x+1, y+1) - I(x+1, y-1) \\ &+ 2(I(x, y+1) - I(x, y-1)) + I(x-1, y+1) - I(x-1, y-1)). \end{aligned}$$

We can now state the corner detection problem more precisely. We want to find points (x, y) where

$$\begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} \tag{39.1}$$

is large for all directions of shift, that is, for all unit vectors $\left[\begin{smallmatrix} u \\ v \end{smallmatrix}\right]$.

The matrix M is a 2×2 symmetric matrix. In Chapter 37, we saw that symmetric matrices can be diagonalized using an orthogonal matrix (Theorem 37.27). So we can write

$$M = Q \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} Q^T,$$

where Q is an orthogonal matrix whose columns are eigenvectors of M, and λ_1, λ_2 are the eigenvalues of M. The matrix M is also *positive semi-definite* (we will study this in Chapter 42). For now, this tells us that both eigenvalues are non-negative.

For any unit vector $\left[\begin{smallmatrix} u \\ v \end{smallmatrix}\right]$, we have

$$\begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} u' & v' \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} u' \\ v' \end{bmatrix} = (u')^2\lambda_1 + (v')^2\lambda_2,$$

where $\left[\begin{smallmatrix} u' \\ v' \end{smallmatrix}\right] = Q^T \left[\begin{smallmatrix} u \\ v \end{smallmatrix}\right]$ is also a unit vector, since orthogonal matrices preserve length. The value of this quadratic form depends on the eigenvalues λ_1 and λ_2. Assuming $\lambda_1 \geqslant \lambda_2$, the maximum occurs when $(u')^2 = 1$ and $(v')^2 = 0$, giving λ_1. The minimum occurs when $(u')^2 = 0$ and $(v')^2 = 1$, giving λ_2. So, for any direction $\left[\begin{smallmatrix} u \\ v \end{smallmatrix}\right]$, the quadratic form in (39.1) satisfies

$$\lambda_2 \leqslant \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} \leqslant \lambda_1.$$

We can now use the eigenvalues λ_1 and λ_2 of M to identify different types of regions in the image:

- **Corners**: Both λ_1 and λ_2 are large. The patch changes a lot in every direction.
- **Edges**: One eigenvalue is large and the other is small (for example, $\lambda_1 \gg \lambda_2$). The patch changes mostly in one direction.
- **Flat regions**: Both λ_1 and λ_2 are small. The patch barely changes in any direction.

We can test this eigenvalue corner detection method using the image in Figure 39.2. The image measures 300×300 pixels. To detect corners, edges, and flat regions, we use a small sliding window of size 5×5 pixels.

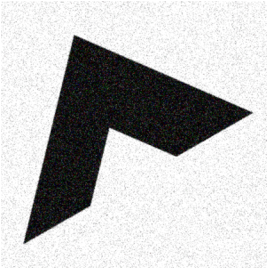


Fig. 39.2: Test image for corner detection. The image size is 300×300 pixels.

In the left panel of Figure 39.3, we plot the computed eigenvalues λ_1 and λ_2 for each window position. To classify each window as a corner, edge, or flat region, we use a threshold of 20:

1. If both eigenvalues are smaller than 20, the window is classified as a flat region.
2. If one eigenvalue is smaller than 20 while the other is larger, the window contains an edge.
3. If both eigenvalues are larger than 20, the window contains a corner.

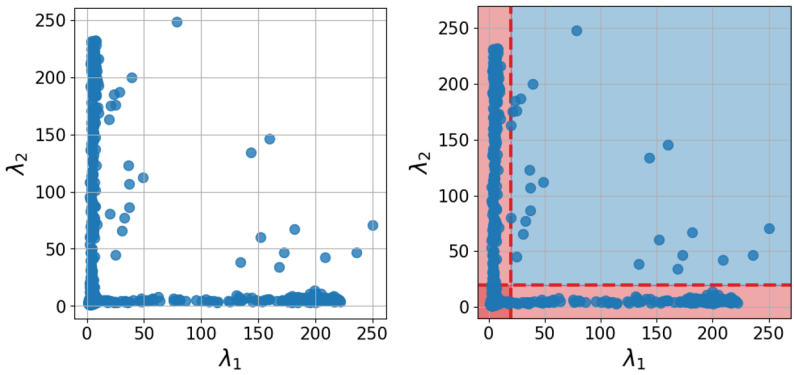
The right panel of Figure 39.3 illustrates this classification. The bottom-left region corresponds to flats, the two light-red regions represent edges, and the top-right region corresponds to corners.

Finally, Figure 39.4 shows the results of this classification. The left image highlights the centers of windows classified as corners, the center image highlights edges, and the right image highlights flats.

Now that we know how the eigenvalues of M reveal corners, edges, and flat regions, we can introduce the *Harris corner detection method*.

The Harris corner detection method

Instead of using the eigenvalues of M directly, the Harris method uses the *Harris response function* R, defined by



(a) Eigenvalues λ_1 and λ_2 for all windows. (b) Classification of regions based on eigenvalues.

Fig. 39.3: Computed eigenvalues and region classification for corner detection.

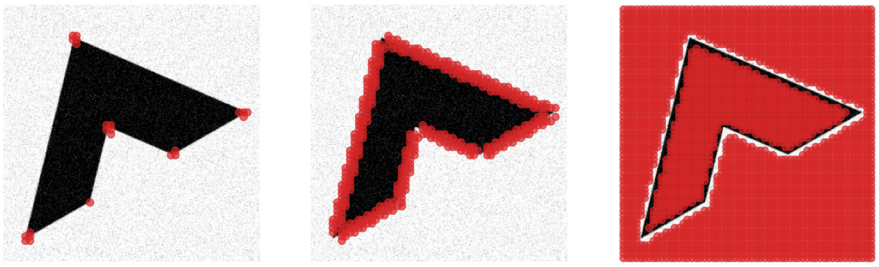


Fig. 39.4: Detected regions: Corners (left), edges (center), and flats (right).

$$R := \det(M) - k\,(\mathrm{trace}(M))^2\,,$$

where k is a small constant, typically between 0.04 and 0.06.

Why use this function? The response R reflects the same information as the eigenvalues λ_1 and λ_2 of M, but avoids computing them explicitly. Since $\det(M) = \lambda_1\lambda_2$ and $\mathrm{trace}(M) = \lambda_1 + \lambda_2$, we can rewrite R as

$$R = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2.$$

The Harris response condenses the behavior of the eigenvalues into a single number, which helps classify different regions in the image based on the sign and size of R:

- **Flat regions**: When both λ_1 and λ_2 are small, R is also small.
- **Edges**: If one eigenvalue is much larger than the other, R becomes negative.
- **Corners**: When both λ_1 and λ_2 are large, R is large and positive.

To detect corners with the Harris response, we choose a threshold value for R. Any point where R exceeds this threshold is marked as a corner. In Figure 39.5,

the left panel shows the boundary that separates corners from non-corners using a threshold of $R = 1000$. The right panel displays the corners identified by the Harris method.

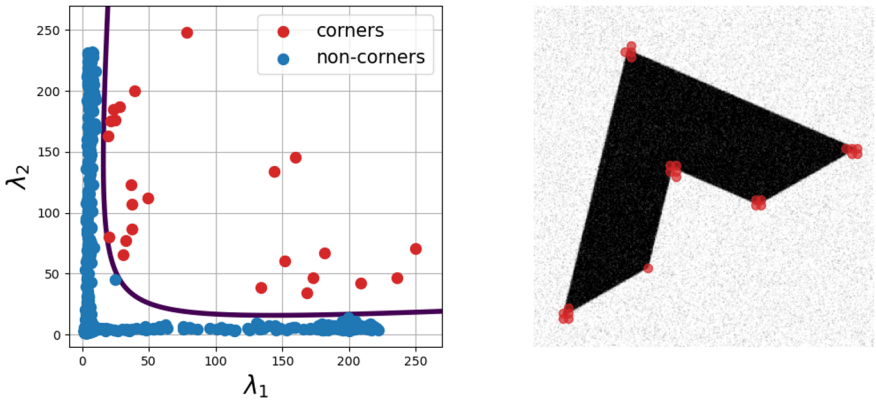


Fig. 39.5: The Harris corner detection method applied to the image in Figure 39.2. The left panel shows the boundary between corners and noncorners based on the threshold value $R = 1000$. The right panel shows the corners detected by the Harris method.

Problems

39.1 Consider the following 10×10 grayscale image. Pixel values range from 0 (black) to 255 (white):

$$\begin{bmatrix} 28 & 33 & 30 & 31 & 29 & 32 & 27 & 34 & 30 & 31 \\ 30 & 26 & 28 & 29 & 30 & 32 & 29 & 28 & 31 & 27 \\ 31 & 29 & 32 & 35 & 33 & 34 & 36 & 31 & 30 & 32 \\ 29 & 31 & 220 & 218 & 222 & 224 & 219 & 34 & 33 & 30 \\ 30 & 32 & 221 & 225 & 219 & 221 & 222 & 31 & 28 & 29 \\ 27 & 30 & 223 & 220 & 218 & 223 & 221 & 29 & 30 & 28 \\ 29 & 31 & 222 & 224 & 225 & 219 & 220 & 30 & 31 & 33 \\ 33 & 30 & 31 & 28 & 27 & 29 & 30 & 29 & 28 & 30 \\ 30 & 28 & 29 & 32 & 30 & 28 & 27 & 26 & 31 & 29 \\ 32 & 29 & 30 & 31 & 33 & 30 & 28 & 29 & 30 & 33 \end{bmatrix}$$

Use 2×2 patch windows to find potential corner pixels in the image.

39.2 For the following Harris matrix, calculate the eigenvalues and classify the type of feature (corner, edge, or flat region):

$$M = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}.$$

39.3 Implement the Harris corner detection algorithm in a programming language of your choice. Apply your implementation to an image and display the detected corners. Experiment with different values of the sensitivity parameter k and the threshold used for corner detection. How does changing k affect the number and location of the corners detected?

39.4 The Shi-Tomasi method is a variation of the Harris method where the response function is given by:

$$R = \min\{\lambda_1, \lambda_2\}.$$

If R is a greater than a threshold value, it is considered as a corner. Explain how this differs from the Harris corner response. Implement the Shi-Tomasi corner detection and compare the results with Harris corner detection on a given image. What differences do you observe in the detected corners?

39.5 The FAST (Features from Accelerated Segment Test) method detects corners by comparing pixel intensities around a candidate pixel. Read about the FAST algorithm and summarize how it differs from Harris and Shi-Tomasi.

39.6 Corners are often used in feature matching for image stitching. Investigate and explain the role of corner detection in this process and how corners help in finding correspondences between overlapping images.

39.7 Generate a noisy version of an image and apply Harris corner detection to both the original and noisy images. Compare the detected corners. How sensitive is the Harris method to noise?

39.8 Apply the Harris corner detector to an image of a natural scene (e.g., trees, mountains, rocks). Use an image processing library (such as OpenCV in Python) to load the image and run the Harris detector.

In natural scenes, regions with rich textures, such as foliage, bark, or rocky terrain, contain lots of local intensity variation. These regions tend to produce more detected corners. Are the corners concentrated in these textured areas?

Next, apply the detector to an image of a man-made scene (e.g., buildings, streets). Compare the number and distribution of detected corners with the results from the natural scene. Why does the detector behave differently in these two types of images?

Part IX
SYMMETRIC AND POSITIVE DEFINITE MATRICES

In this part, we introduce symmetric positive (semi)definite matrices, which extend the idea of positive (or nonnegative) numbers to matrices. We also introduce matrices that appear often in Machine Learning, like the graph Laplacian, the Hessian, and the covariance matrix, and study when they are positive (semi)definite. Finally, we see how these matrices are used in clustering, dimensionality reduction, and optimization.

Chapter 40
IX-WHY SHOULD WE CARE?

In this part of the book, we introduce symmetric positive definite and positive semidefinite matrices, which are common in many applications. We focus on two areas where they often appear: clustering and dimensionality reduction.

40.1 Clustering when k-means does not work

In Section 14.3, we introduced the k-means method for clustering. This method assumes that points in each cluster are spread evenly around the center, forming circles in 2D, spheres in 3D, or hyperspheres in higher dimensions. But real data often breaks these assumptions. The k-means method tends to struggle with elongated clusters, uneven cluster sizes, or irregular shapes, as shown in Figure 40.1, where each point is colored by its k-means cluster assignment.

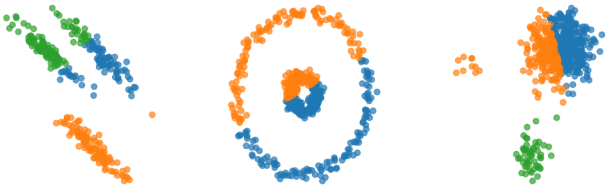


Fig. 40.1: Three scenarios where k-means produces unexpected results. In each example, the clusters identified by k-means are shown in distinct colors. From left to right, the figures illustrate elongated clusters, clusters with irregular shapes, and clusters of uneven sizes.

In the left panel of Figure 40.1, the blue and green dots are mixed, even though they belong to different clusters. The same problem appears in the other panels. In all three cases, k-means fails to find the correct clusters.

In Chapter 44, we introduce spectral clustering, a method that can handle data that is irregularly shaped or unevenly distributed, like the examples in Figure 40.1. This method uses the eigenvectors of the Laplacian matrix, a symmetric positive semidefinite matrix from graph theory.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_40

40.2 Dimensionality reduction and class separability

Dimensionality reduction maps data from a high-dimensional space to a lower-dimensional one (often $\mathbb{R}^2$ or $\mathbb{R}^3$) while keeping the characteristics that make the data useful. This process makes the data easier to visualize and analyze.

For example, consider the toy dataset in Figure 40.2, where different colors represent three well-separated classes. Now, suppose we want to project this dataset onto a one-dimensional subspace (a line through the origin) and still keep the classes well-separated. What criteria should we use to choose the best subspace for this projection?

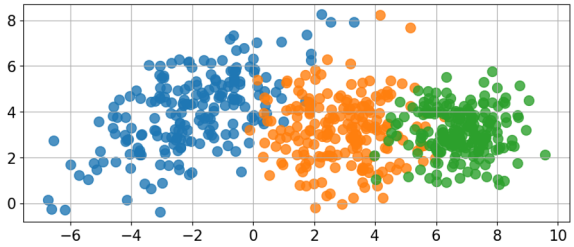


Fig. 40.2: Toy dataset with three well-separated classes represented by different colors.

At the top of Figure 40.3, we display the original colored dataset along with three lines, dashed, dotted, and dash-dotted. In the bottom panel, we show the results of orthogonally projecting the data points onto each of these lines, accompanied by histograms that illustrate the class distributions. The histograms clearly indicate that the dashed line (the third line) better preserves the original class separability. We identified this line using Linear Discriminant Analysis, a technique that relies on the eigenvectors of a symmetric positive definite matrix.

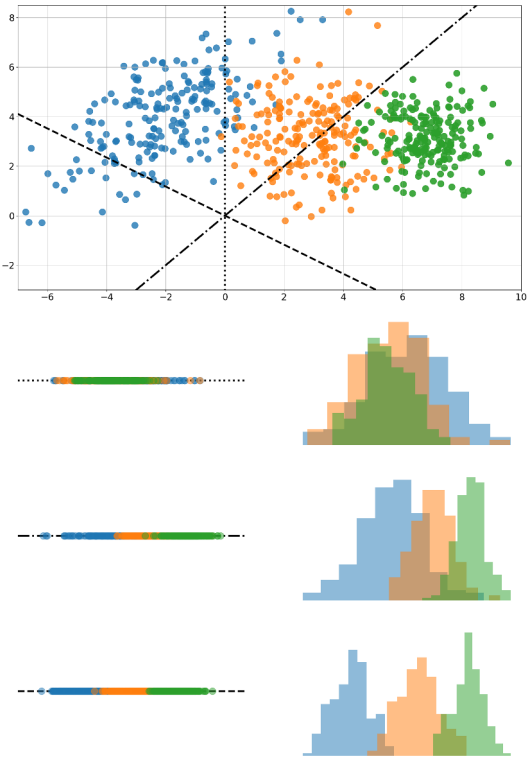


Fig. 40.3: Top: The original labeled dataset with three different lines passing through the origin. Bottom: The projected data points onto each line, shown alongside one-dimensional histograms to illustrate class separability.

Chapter 41
IX-WHAT YOU MUST HAVE LEARNED BEFORE

In this chapter, we review a few important ideas from earlier chapters that we will need in Chapter 42.

41.1 Symmetric matrices and the spectral theorem

Symmetric matrices are important in Machine Learning because of their useful properties and many applications. We start by recalling their definition.

Definition 41.1. (Symmetric Matrix)

A matrix $S \in \mathbb{R}^{n\times n}$ is **symmetric** if $S^T = S$.

Symmetric matrices possess several important properties. For example, the collection of all $n \times n$ real symmetric matrices forms a subspace of $\mathbb{R}^{n\times n}$ under the standard operations of matrix addition and scalar multiplication.

Proposition 41.2. (Closure under addition and scalar multiplication)

Let S_1 and S_2 be two real $n \times n$ symmetric matrices, and let $\alpha \in \mathbb{R}$. Then both $S_1 + S_2$ and αS_1 are symmetric matrices.

A fundamental property that we will often use is that real symmetric matrices have real eigenvalues and can be diagonalized by an orthogonal matrix. We proved these results in Chapter 37 and summarize them in the following theorem.

Theorem 41.3. (Spectral theorem for symmetric matrices)

For an $n \times n$ real matrix S, the following statements are equivalent:

(a) S is symmetric.
(b) S is orthogonally diagonalizable. That is, there exists a real orthogonal matrix Q and a real diagonal matrix $D = \text{diag}(\lambda_1, \ldots, \lambda_n)$ such that

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_41

$$S = QDQ^T.$$

(c) There exists an orthonormal basis for $\mathbb{R}^n$ consisting of eigenvectors of S.

Example 41.4 Consider the real symmetric matrix

$$S = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

The eigenvalues of S are $\lambda_1 = 1$ and $\lambda_2 = 3$, with corresponding eigenvectors

$$q_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \quad \text{and} \quad q_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

respectively. Note that the eigenvalues are real and the eigenvectors are both real and orthonormal. Since S is symmetric, it is orthogonally diagonalizable. For example, one possible diagonalization is given by

$$S = QDQ^T = \begin{bmatrix} -1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} -1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}.$$

41.2 Change of basis and the geometric interpretation of real symmetric matrices

The spectral decomposition $S = QDQ^T$ provides a way to understand the geometric picture of a symmetric matrix. To start, we first review the concept of a change of basis.

Change of basis

Consider an invertible matrix $P \in \mathbb{R}^{n \times n}$. Then, its columns $v_1, v_2, \ldots, v_n$ form a basis

$$\beta = \{v_1, v_2, \ldots, v_n\}$$

for $\mathbb{R}^n$.

We call P a **change-of-basis matrix** because, as a linear transformation, it changes the coordinates of any vector $x \in \mathbb{R}^n$ from the basis β to the standard basis. To see how this works, recall that any vector $x \in \mathbb{R}^n$ can be uniquely expressed as a linear combination of the basis vectors in β:

$$x = \beta_1 v_1 + \beta_2 v_2 + \cdots + \beta_n v_n,$$

where the scalars $\beta_1, \beta_2, \ldots, \beta_n$ are the coordinates of x in the basis β. We can group these coordinates into the column vector

$$[x]_\beta := \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_n \end{bmatrix}.$$

In matrix notation, the relationship between x and its coordinates in the basis β is given by

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_n \end{bmatrix} = P\,[x]_\beta.$$

Thus, P converts the coordinates of x from the basis β into its coordinates in the standard basis. Conversely, the inverse matrix P^{-1} converts coordinates in the standard basis back into the basis β:

$$P^{-1}x = [x]_\beta.$$

As we know, any matrix $A \in \mathbb{R}^{n\times n}$ defines a linear transformation $f : \mathbb{R}^n \to \mathbb{R}^n$ via $f(x) = Ax$. When A is diagonal, say $A = \operatorname{diag}(\alpha_1, \ldots, \alpha_n)$, f scales each coordinate of x by the corresponding diagonal entry:

$$f(x) = \begin{bmatrix} \alpha_1 & & & \\ & \alpha_2 & & \\ & & \ddots & \\ & & & \alpha_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} \alpha_1 x_1 \\ \alpha_2 x_2 \\ \vdots \\ \alpha_n x_n \end{bmatrix}.$$

What if A is not diagonal but is diagonalizable? In this case, we can write

$$A = PDP^{-1},$$

where P is invertible and D is diagonal. As we have discussed above, in the basis β defined by the columns of P, the coordinates of any vector x are given by $[x]_\beta = P^{-1}x$. Then,

$$[f(x)]_\beta = [Ax]_\beta = P^{-1}Ax = DP^{-1}x = D\,[x]_\beta,$$

which means that in the β-basis, f acts as a diagonal (scaling) operation.

Interpreting diagonalizable matrices as simple scaling operations in a different basis is especially neat for symmetric matrices. Using the spectral decomposition $S = QDQ^T$, we see that in the orthonormal eigenvector basis formed by the columns of Q, S scales each component by its corresponding eigenvalue.

Example 41.5 Consider the symmetric matrix

$$S = \begin{bmatrix} 4/5 & -3/5 \\ -3/5 & -4/5 \end{bmatrix}.$$

Our goal is to understand the geometric effect of $f(x) = Sx$ on $\mathbb{R}^2$. Since S is symmetric, it is orthogonally diagonalizable. In fact, we can write

$$S = \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ 3/\sqrt{10} & -1/\sqrt{10} \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ 3/\sqrt{10} & -1/\sqrt{10} \end{bmatrix} = QDQ^T.$$

Figure 41.1 shows that, in the orthonormal eigenvector basis given by the columns q_1, q_2 of Q, the transformation Sx becomes a simple scaling . In this basis, the component along the first eigenvector q_1 is multiplied by -1, while the component along the second eigenvector q_2 remains unchanged. Combined, these effects reflect the vector x across the line spanned by q_2.

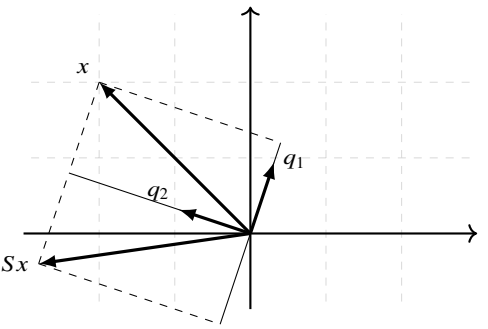


Fig. 41.1: Geometric effect of S: In the eigenvector basis $\{q_1, q_2\}$ (columns of Q), S scales each component of x, reflecting it across the line spanned by q_2.

There is an alternative geometric interpretation of symmetric matrices. To begin with, for a symmetric matrix $S \in \mathbb{R}^{n\times n}$ with the decomposition $S = QDQ^T$, we can choose Q as a proper orthogonal matrix (i.e., $\det(Q) = 1$). If Q turns out to be improper (with $\det(Q) = -1$), we can adjust it by multiplying one of its columns by -1. We illustrate this in the following example.

Example 41.6 Let us revisit Example 41.5. Notice that the orthogonal matrix Q is improper because $\det(Q) = -1$. However, we can adjust the factorization by writing it as

$$S = QDQ^T = \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ 3/\sqrt{10} & -1/\sqrt{10} \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ 3/\sqrt{10} & -1/\sqrt{10} \end{bmatrix}$$
$$= \begin{bmatrix} -1/\sqrt{10} & 3/\sqrt{10} \\ -3/\sqrt{10} & -1/\sqrt{10} \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -1/\sqrt{10} & -3/\sqrt{10} \\ 3/\sqrt{10} & -1/\sqrt{10} \end{bmatrix} =: \widehat{Q}D\widehat{Q}^T.$$

We just have multiplied the eigenvector in the first column of Q by -1. The new column is still an eigenvector, and now $\det(\widehat{Q}) = 1$.

For a symmetric matrix S, the diagonalization $S = QDQ^T$, where Q is a proper orthogonal (i.e., $\det(Q) = 1$) and D is diagonal, expresses the linear transformation $f(x) = Sx$ as a composition of three transformations, each with a simple geometric interpretation. The first transformation is $Q^T x$. Since Q (and therefore Q^T) is a proper orthogonal matrix, this step is a composition of Givens rotations. This transformation aligns the columns of Q, which are the eigenvectors of S, with the standard basis vectors. The second transformation is $D(Q^T x)$. The diagonal matrix D scales each component of the rotated vector $Q^T x$. The scaling factor for each component is given by the corresponding diagonal entry of D. The final transformation is $Q(DQ^T x)$. This step applies a second proper orthogonal transformation, the inverse of the first one, which returns the scaled vector to the original coordinate system.

Example 41.7 Consider the symmetric matrix

$$S = \begin{bmatrix} 4/5 & -3/5 \\ -3/5 & -4/5 \end{bmatrix}.$$

Our goal is to understand the geometric effect of $f(x) = Sx$ on $\mathbb{R}^2$. Since S is symmetric, it is orthogonally diagonalizable. We can write

$$S = \begin{bmatrix} 1/\sqrt{10} & -3/\sqrt{10} \\ 3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ -3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix} =: QDQ^T.$$

Figure 41.2 illustrates how the diagonalization $S = QDQ^T$ expresses the linear transformation $f(x) = Sx$ as a composition of three simple transformations. The first transformation, $Q^T x$, is a proper orthogonal transformation, which in $\mathbb{R}^2$ is a rotation. This rotation aligns the columns of $Q = \begin{bmatrix} q_1 & q_2 \end{bmatrix}$ with the x_1- and x_2-axes:

$$Q^T q_1 = \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ -3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix} \begin{bmatrix} 1/\sqrt{10} \\ 3/\sqrt{10} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{and}$$
$$Q^T q_2 = \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ -3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix} \begin{bmatrix} -3/\sqrt{10} \\ 1/\sqrt{10} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The second transformation, $D(Q^T x)$, applies a scaling. This scaling leaves the second component unchanged and flips the sign of the first component. The final transformation, $Q(DQ^T x)$, is another rotation. This rotation undoes the effect of the

initial rotation because Q is the inverse matrix of Q^T. Combining these steps, the transformation Sx reflects the vector x across the line spanned by q_2.

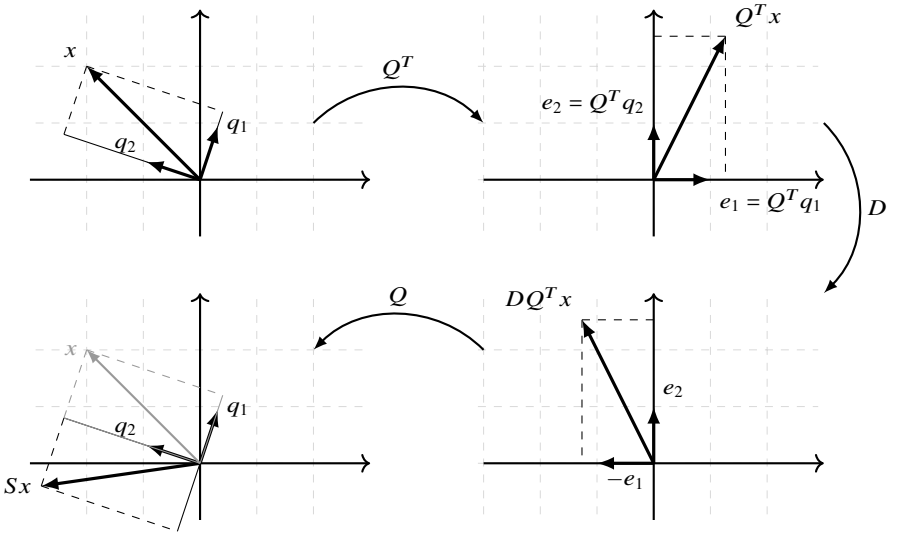


Fig. 41.2: Geometric interpretation of the diagonalization $S = QDQ^T$ as a sequence of rotations and scaling.

41.3 Projection decomposition of a symmetric matrix

If S is a symmetric matrix, then by the Spectral Theorem (Theorem 41.3), it can be written as $S = QDQ^T$ for some orthogonal matrix Q and some diagonal matrix $D = \text{diag}\,(\lambda_1, \lambda_2, \ldots, \lambda_n)$. We can write QD as

$$QD = \begin{bmatrix} \lambda_1 q_1 & \lambda_2 q_2 & \cdots & \lambda_n q_n \end{bmatrix},$$

where $q_1, q_2, \ldots, q_n$ are the columns of Q. Then, using the columns-times-rows matrix multiplication picture, we can express S as a linear combination of projection matrices:

$$S = QDQ^T = \lambda_1 q_1 q_1^T + \lambda_2 q_2 q_2^T + \cdots + \lambda_n q_n q_n^T.$$

Each $q_k q_k^T$ is the projection matrix onto the subspace spanned by q_k. Thus, any symmetric matrix S can be interpreted as a combination of stretches and compressions along n mutually orthogonal directions. These directions are defined by the eigenvectors of S. The eigenvalues $\lambda_1, \ldots, \lambda_n$ determine the magnitude of the stretching or compression along each direction.

Problems

41.1 Is the product of two symmetric matrices always symmetric? Prove your answer or provide a counterexample.

41.2 Let

$$S = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 0 & -1 \\ 3 & -1 & 4 \end{bmatrix}.$$

(a) Show that S is orthogonally diagonalizable by finding an orthonormal basis for $\mathbb{R}^3$ consisting of eigenvectors of S.
(b) Decompose S into a composition of a rotation, a scaling, and another rotation.

41.3 A complex square matrix A is called **Hermitian** if $A^* = A$, where A^* denotes the conjugate transpose of A. Determine whether the set of $n \times n$ Hermitian matrices forms a (complex) subspace of $\mathbb{C}^{n\times n}$ under the standard operations of addition and scalar multiplication.

41.4 Are all Hermitian matrices diagonalizable? Prove your answer or provide a counterexample. What about orthogonal or unitary diagonalizability?

41.5 In $\mathbb{R}^2$, what is the matrix representation of the orthogonal projection onto the horizontal axis?

41.6 Using a change of basis, derive a formula for the orthogonal projection onto the line $y = 3x$.

41.7 Let S be a symmetric matrix, and let λ_1 and λ_2 be two distinct eigenvalues of S. Prove that the eigenspaces corresponding to λ_1 and λ_2 are orthogonal. Specifically, show that $x_1^T x_2 = 0$ for any eigenvector x_1 associated with λ_1 and any eigenvector x_2 associated with λ_2.

41.8 Let S be an $n \times n$ real symmetric matrix. Use the Spectral Theorem to show that Sx represents an anisotropic[1] scaling of x along the directions of an orthonormal basis for $\mathbb{R}^n$.

[1] Anisotropic scaling refers to scaling that varies with direction. Unlike isotropic scaling, which applies the same factor in all directions, anisotropic scaling stretches or compresses differently depending on the direction.

Chapter 42
IX-CORE TOPICS

42.1 Symmetric matrices and quadratic forms

Symmetric matrices are important in Machine Learning because their connection to quadratic forms. A quadratic form is a polynomial that includes only terms of degree 2, with no linear or constant terms. The following examples illustrate this.

Example 42.1 For a single variable x, quadratic forms are polynomials of the form $f(x) = ax^2$, where a is a constant. If a quadratic polynomial includes linear or constant terms, such as $f(x) = ax^2 + bx + c$, it is no longer a quadratic form because it includes terms of degree less than 2.

Example 42.2 For two variables x_1 and x_2, a quadratic form can be written as $f(x_1, x_2) = ax_1^2 + bx_1x_2 + cx_2^2$, where a, b, and c are constants. This expression contains the quadratic terms ax_1^2 and cx_2^2, along with the cross term bx_1x_2. For example, $f(x_1, x_2) = 2x_1^2 - x_1x_2 + 3x_2^2$ is a quadratic form because all terms are of degree 2. If we add linear or constant terms, such as $f(x_1, x_2) = 2x_1^2 - x_1x_2 + 3x_2^2 + 4x_1 + 5$, the expression is no longer a quadratic form because it includes terms of lower degree.

We can now introduce the general definition of a quadratic form.

Definition 42.3. (Quadratic form)

A **quadratic form** is a function $f : \mathbb{R}^n \to \mathbb{R}$ of the form

$$f(x_1, x_2, \ldots, x_n) = \sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij} x_i x_j, \tag{42.1}$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_42

where x_i is the i-th component of x and $a_{ij} \in \mathbb{R}$ are constants.

Matrices give us a concise way to express quadratic forms. The result below illustrates this connection, and we omit the proof since it involves only simple calculations.

Proposition 42.4. (Matrix representation of quadratic forms)

The quadratic form (42.1) can be written as $f(x) = x^T Ax$, where $x = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}^T$ and $A = [a_{ij}] \in \mathbb{R}^{n \times n}$.

Example 42.5 Consider the quadratic form $f(x_1, x_2, x_3) = x_1^2 + 4x_3^2 + 4x_1x_2 + 6x_1x_3 - 2x_2x_3$ in three variables x_1, x_2, x_3. We can express f in matrix form as

$$f(x_1, x_2, x_3) = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 1 & 4 & 6 \\ 0 & 0 & -2 \\ 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

It is important to note that the matrix A used to represent f is not unique. For example, we can also write f as

$$f(x_1, x_2, x_3) = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 1 & 3 & 2 \\ 1 & 0 & -2 \\ 4 & 0 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

There are infinitely many ways to represent the quadratic form in (42.1) as $f(x) = x^T Ax$. However, if we require A to be symmetric, the matrix representation becomes unique. The following theorem states this result.

Theorem 42.6. (Unique symmetric representation of quadratic forms)

For the quadratic form in (42.1), there is a unique symmetric matrix $S \in \mathbb{R}^{n \times n}$ such that $f(x) = x^T Sx$.

Proof Consider the quadratic form in (42.1), which can be written as $f(x) = x^T Ax$, where $A = [a_{ij}]$. Define the symmetric matrix $S = \frac{A + A^T}{2}$. Since $x^T Ax$ is a scalar, we have $x^T Ax = (x^T Ax)^T = x^T A^T x$. Using this, we compute

$$f(x) = x^T Ax = \frac{1}{2}(x^T Ax + x^T A^T x) = x^T \left(\frac{A + A^T}{2} \right) x = x^T Sx.$$

Thus, there always exists a symmetric matrix S such that $f(x) = x^T Sx$. The uniqueness of the symmetric matrix is left as an exercise (see Problem 42.1). □

Example 42.7 We can represent the quadratic form from Example 42.5 using a symmetric matrix as

$$f(x_1, x_2, x_3) = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 2 & 0 & -1 \\ 3 & -1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

Since we can always choose a symmetric matrix to represent a quadratic form, we will assume from now on that the matrix in any quadratic form is symmetric and denote it by S.

Quadratic forms and change of basis

Recall from Chapter 41 that an invertible matrix P gives us a change of basis $x = Py$ (or, equivalently, $y = P^{-1}x$). This means we can convert a vector's coordinates from the basis defined by the columns of P to the standard basis in $\mathbb{R}^n$ and vice versa.

Now, when we apply the change of basis $x = Py$ to the quadratic form $f(x) = x^T Sx$, we get

$$f(x) = x^T Sx = (Py)^T S(Py) = y^T (P^T SP)y = g(y).$$

The new quadratic form $g(y)$ is expressed in the variables $y_1, \ldots, y_n$ and is associated with the symmetric matrix $P^T SP$.

This leads us to an important question: Can we choose a change-of-basis matrix P that simplifies a quadratic form? For instance, if we choose P so that $P^T SP$ is diagonal, then the quadratic form becomes

$$g(y) = y^T Dy = d_1 y_1^2 + d_2 y_2^2 + \cdots + d_n y_n^2,$$

eliminating all the cross-product terms $y_i y_j$ for $i \neq j$. So, can we always find such a P? The answer is yes! In fact, we can choose P to be an orthogonal matrix. We prove this result in Theorem 42.8, which follows directly from the Spectral Theorem for symmetric matrices (Theorem 41.3).

Theorem 42.8. (Orthogonal diagonalization of quadratic forms)

Let $f(x) = x^T Sx$ be a quadratic form with $S \in \mathbb{R}^{n \times n}$ symmetric. There exists an orthogonal matrix Q such that the change of basis $x = Qy$ transforms $f(x)$ into the simplified form $g(y) = y^T Dy$, where D is a diagonal matrix.

Proof By Theorem 41.3, we can write $S = QDQ^T$, where D is a diagonal matrix containing the eigenvalues of S, and Q is an orthogonal matrix whose columns form an orthonormal basis for $\mathbb{R}^n$ consisting of eigenvectors of S. Define the change of coordinates $x = Qy$. Then,

$$\begin{aligned} f(x) &= x^T Sx \\ &= (Qy)^T S(Qy) \\ &= y^T Q^T SQy \\ &= y^T Dy \\ &= \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2 = g(y), \end{aligned}$$

where $\lambda_1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of S. □

Example 42.9 Let us find a change of basis that diagonalizes the quadratic form $f(x) = 3x_1^2 + 4x_1x_2 + 3x_2^2$. First, we express $f(x)$ in matrix form

$$f(x) = \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

The symmetric matrix $S = \left[\begin{smallmatrix} 3 & 2 \\ 2 & 3 \end{smallmatrix}\right]$ has eigenvalues $\lambda_1 = 1$ and $\lambda_2 = 5$. Associated orthonormal eigenvectors are $q_1 = \frac{1}{\sqrt{2}} \left[\begin{smallmatrix} 1 \\ -1 \end{smallmatrix}\right]$ and $q_2 = \frac{1}{\sqrt{2}} \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$. Using these, we can orthogonally diagonalize S as $S = QDQ^T$, where

$$D = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix}, \qquad \text{and} \qquad Q = \begin{bmatrix} q_1 & q_2 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}.$$

The change of variables is defined by $x = Qy$, or equivalently, $y = Q^T x$. This gives the new variables:

$$y_1 = \frac{1}{\sqrt{2}} x_1 - \frac{1}{\sqrt{2}} x_2, \quad y_2 = \frac{1}{\sqrt{2}} x_1 + \frac{1}{\sqrt{2}} x_2.$$

In the new variables y_1 and y_2, the quadratic form $f(x)$ becomes $g(y) = y_1^2 + 5y_2^2$. Written in terms of the original variables, we have

$$f(x) = \left(\frac{1}{\sqrt{2}} x_1 - \frac{1}{\sqrt{2}} x_2 \right)^2 + 5 \left(\frac{1}{\sqrt{2}} x_1 + \frac{1}{\sqrt{2}} x_2 \right)^2.$$

42.1.1 Classifying quadratic forms

Quadratic forms in one variable, $f(x) = ax^2$, have a very limited set of possible shapes, as shown in Figure 42.1. When $a > 0$, the graph is a parabola opening

upwards. When $a < 0$, the graph is a parabola opening downwards. If $a = 0$, the graph is a flat line at zero.

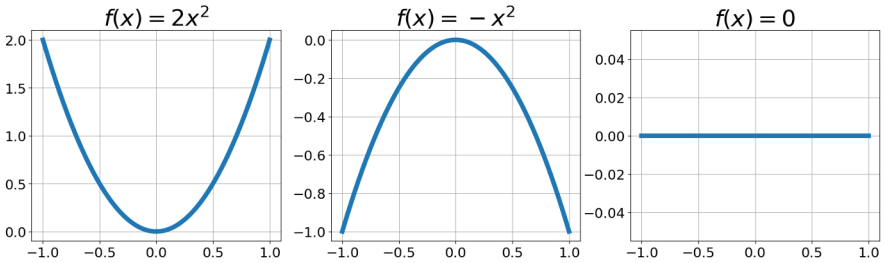


Fig. 42.1: Illustration of the possible shapes of one-variable quadratic forms.

In this section, we aim to classify quadratic forms in higher dimensions. To give you a visual understanding, Figure 42.2 displays the graphs of six two-variable quadratic forms $f(x_1, x_2) = ax_1^2 + bx_1x_2 + cx_2^2$. The first two graphs resemble paraboloids, the third looks like a saddle, the next two are flat along a line, and the last one is completely flat. Can you explain these shapes by examining their equations?

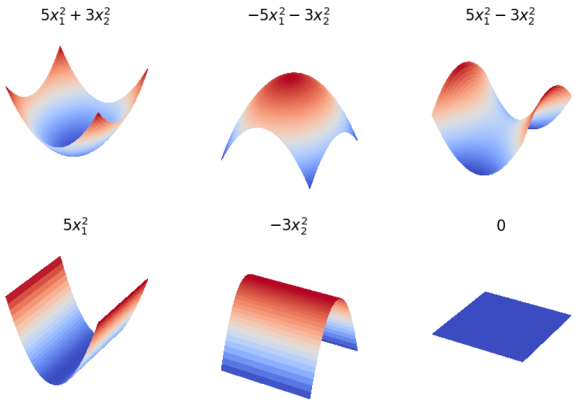


Fig. 42.2: Graphs of two-variable quadratic forms $f(x_1, x_2) = ax_1^2 + bx_1x_2 + cx_2^2$, illustrating different shapes.

For any quadratic form, plugging in $x = 0$ always gives zero. But to understand the shapes in Figure 42.2, we need to see how the form behaves for nonzero vectors. To do that, let us introduce a few definitions.

Definition 42.10. (Classification of quadratic forms)

Let S be a symmetric matrix. We call the quadratic form $f(x) = x^T Sx$

(i) **positive definite** if $f(x) > 0$ for all $x \neq 0$;
(ii) **positive semidefinite** if $f(x) \geqslant 0$ for all $x \neq 0$;
(iii) **negative definite** if $f(x) < 0$ for all $x \neq 0$;
(iv) **negative semidefinite** if $f(x) \leqslant 0$ for all $x \neq 0$;
(v) **indefinite** if $f(x)$ takes both positive and negative values.

Looking back at the graphs in Figure 42.2, we can classify the quadratic forms as follows:

- The first quadratic form is positive definite because $5x_1^2 + 3x_2^2 > 0$ for all nonzero $x = \left[\begin{smallmatrix} x_1 \\ x_2 \end{smallmatrix}\right]$.
- The second quadratic form is negative definite because $-5x_1^2 - 3x_2^2 < 0$ for all nonzero $x = \left[\begin{smallmatrix} x_1 \\ x_2 \end{smallmatrix}\right]$.
- The third quadratic form is indefinite. For example, $5x_1^2 - 3x_2^2 > 0$ when $x_1 \neq 0$ and $x_2 = 0$, but $5x_1^2 - 3x_2^2 < 0$ when $x_1 = 0$ and $x_2 \neq 0$.
- The fourth quadratic form is positive semidefinite because $5x_1^2 \geqslant 0$, and it equals zero for vectors of the form $x = \left[\begin{smallmatrix} 0 \\ x_2 \end{smallmatrix}\right]$.
- The fifth quadratic form is negative semidefinite because $-3x_2^2 \leqslant 0$, and it equals zero for all vectors of the form $x = \left[\begin{smallmatrix} x_1 \\ 0 \end{smallmatrix}\right]$.
- The last quadratic form, $f(x_1, x_2) = 0$, is a special case that is both positive semidefinite and negative semidefinite.

We can determine the definiteness of a quadratic form $f(x) = x^T Sx$ by checking the signs of the eigenvalues of the matrix S.

Theorem 42.11. (Eigenvalue characterization of quadratic forms)

Consider the quadratic form $f(x) = x^T Sx$, where $S \in \mathbb{R}^{n \times n}$ is a symmetric matrix. Let $\lambda_1, \ldots, \lambda_n$ be the eigenvalues of S, with corresponding orthonormal eigenvectors $q_1, \ldots, q_n$. Then

(a) f is positive definite if and only if $\lambda_i > 0$ for all $i = 1, \ldots, n$.
(b) f is positive semidefinite if and only if $\lambda_i \geqslant 0$ for all $i = 1, \ldots, n$.
(c) f is negative definite if and only if $\lambda_i < 0$ for all $i = 1, \ldots, n$.
(d) f is negative semidefinite if and only if $\lambda_i \leqslant 0$ for all $i = 1, \ldots, n$.
(e) f is indefinite if and only if S has both positive and negative eigenvalues.

Additionally:

(i) If $\lambda_i > 0$, f increases along the line spanned by the eigenvector q_i; that is, whether we move in the direction of q_i or $-q_i$, f increases.
(ii) If $\lambda_i < 0$, f decreases along the line spanned by the eigenvector q_i; that is, whether we move in the direction of q_i or $-q_i$, f decreases.
(iii) If $\lambda_i = 0$, f remains constant along the line spanned by the eigenvector q_i.

Proof We will prove parts (a) and (i), leaving the other parts as exercises (see Problem 42.8).

For part (a), assume S is positive definite, and let λ be an eigenvalue of S with a corresponding unit eigenvector q. Then,

$$\lambda = \lambda \|q\|_2^2 = \lambda q^T q = q^T(\lambda q) = q^T S q > 0.$$

Thus, all eigenvalues of a symmetric positive definite matrix are positive.

Conversely, assume all eigenvalues λ_i of S are positive. Let x be a nonzero vector.

Since S is symmetric, by the Spectral Theorem (Theorem 41.3, $S = QDQ^T$ for some orthogonal matrix Q and diagonal matrix D. Thus,

$$f(x) = x^T S x = x^T Q D Q^T x = (Q^T x)^T D (Q^T x) =: y^T D y = \sum_{i=1}^{n} \lambda_i y_i^2.$$

Since $\lambda_i > 0$, $y_i^2 \geqslant 0$, and at least one $y_i \neq 0$ (because $x \neq 0$), we have $f(x) > 0$ for any nonzero x. Therefore, the quadratic form f is positive definite.

For part (i), assume $\lambda_i > 0$ is an eigenvalue of S, and let x be a vector in the direction of the corresponding eigenvector q_i. We can write x as $x = tq_i$, where t is a scalar. Then,

$$\begin{aligned} f(x) = x^T S x &= (tq_i)^T S (tq_i) = t^2 q_i^T S q_i \\ &= t^2 q_i^T Q D Q^T q_i = t^2 e_i^T D e_i = t^2 \lambda_i. \end{aligned}$$

Since $\lambda_i > 0$ and $t^2 \geqslant 0$, $f(x)$ increases as $|t|$ increases. This shows that $f(x)$ grows along the line spanned by q_i. □

42.1.2 Constrained optimization of quadratic forms

As we discussed in Chapter 35 and will revisit in Chapter 44, many applications require optimizing a quadratic form over the unit sphere. We state this optimization problem as follows:

$$\begin{aligned} &\text{optimize} \quad x^T S x \\ &\text{subject to} \quad \|x\|_2 = 1, \end{aligned} \tag{42.2}$$

where S is a symmetric matrix, and "optimizing" means finding either a minimum or a maximum, depending on the application.

The following theorem uses the spectral decomposition $S = QDQ^T$ of a symmetric matrix S to solve (42.2).

Theorem 42.12. (The Rayleigh-Ritz theorem)

Let $S \in \mathbb{R}^{n \times n}$ be a symmetric matrix, and let $\lambda_1, \ldots, \lambda_n \in \mathbb{R}$ denote the eigenvalues of S, ordered such that $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_n$. Let $\{q_1, q_2, \ldots, q_n\}$ be an orthonormal basis of $\mathbb{R}^n$ consisting of eigenvectors of S, where q_i is the unit eigenvector associated with λ_i. The largest and smallest eigenvalues, λ_1 and λ_n, solve the following

constrained optimization problems:

$$\lambda_1 = \max_{\|x\|_2=1} x^T S x, \quad \text{and} \tag{42.3}$$

$$\lambda_n = \min_{\|x\|_2=1} x^T S x. \tag{42.4}$$

The maximum in (42.3) is attained when $x = q_1$, and the minimum in (42.4) is attained when $x = q_n$.

The k-th eigenvalue λ_k solves the following constrained optimization problems:

$$\lambda_k = \max_{\substack{\|x\|_2=1 \\ x \perp q_1,\ldots,q_{k-1}}} x^T S x, \quad \text{and} \tag{42.5}$$

$$\lambda_k = \min_{\substack{\|x\|_2=1 \\ x \perp q_{k+1},\ldots,q_n}} x^T S x. \tag{42.6}$$

The maximum in (42.5) and the minimum in (42.6) are attained when $x = q_k$.

Proof We will prove (42.3) and (42.5). We ask you to verify (42.4) and (42.6) in Problem 42.9.

Since S is a symmetric matrix, it is orthogonally diagonalizable. Thus, we can write $S = QDQ^T$, where $Q = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}$ is an orthogonal matrix, and $D = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$.

For any unit vector x, we have

$$x^T S x = x^T Q D Q^T x = (Q^T x)^T D (Q^T x) =: y^T D y = \sum_{i=1}^{n} \lambda_i y_i^2 \leqslant \lambda_1 \sum_{i=1}^{n} y_i^2 = \lambda_1,$$

where the last equality follows from the fact

$$\sum_{i=1}^{n} y_i^2 = \|y\|_2^2 = \|Q^T x\|_2^2 = \|x\|_2^2 = 1,$$

since orthogonal matrices preserve norms and x is a unit vector. The upper bound λ_1 is attained when $x = q_1$, since

$$q_1^T S q_1 = (q_1^T Q) D (Q^T q_1) = e_1^T D e_1 = \lambda_1.$$

This proves (42.3).

Now, consider a unit vector x that is orthogonal to $\{q_1, q_2, \ldots, q_{k-1}\}$. This implies

$$0 = q_j^T x = q_j^T Q y = e_j y = y_j, \quad \text{for } j = 1, \ldots, k-1.$$

Thus, we can write

$$x = Qy, \quad \text{where} \quad y = Q^T x = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ y_k \\ \vdots \\ y_n \end{bmatrix}.$$

Substituting this into $x^T Sx$, we get

$$x^T Sx = x^T QDQ^T x = y^T Dy = \sum_{i=k}^{n} \lambda_i y_i^2 \leqslant \lambda_k \sum_{i=k}^{n} y_i^2 = \lambda_k.$$

The upper bound λ_k is attained when $x = q_k$, since

$$q_k^T Sq_k = (q_k^T Q)S(Q^T q_k) = e_k^T De_k = \lambda_k, \quad \text{and } q_k \perp \{q_1, q_2, \ldots, q_{k-1}\}.$$

This proves (42.5). □

Remark 42.13. (Optimization via the Rayleigh quotient)

We can rewrite the constrained optimization problem (42.2) as the unconstrained (i.e., with no restrictions on x) optimization problem

$$\text{optimize} \quad \frac{x^T Sx}{x^T x}.$$

In Problem 42.10, you will show that optimizing the Rayleigh quotient $\frac{x^T Sx}{x^T x}$ is equivalent to optimizing the quadratic form $x^T Sx$ over all unit vectors (i.e., vectors with $x^T x = 1$).

Example 42.14 Consider the symmetric matrix $S = \left[\begin{smallmatrix} 3 & 1 \\ 1 & 3 \end{smallmatrix}\right]$. The eigenvalues of S are $\lambda_1 = 4$ and $\lambda_2 = 2$, with orthonormal eigenvectors $q_1 = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$ and $q_2 = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 \\ -1 \end{smallmatrix}\right]$.

Figure 42.3-(a) shows the unit circle, colored by the values of the quadratic form $f(x) = x^T Sx$ at different points x on the circle. The dashed lines mark the directions of q_1 and q_2. We can see that $f(x)$ reaches its maximum value of $\lambda_1 = 4$ at $x = \pm q_1$ and its minimum value of $\lambda_2 = 2$ at $x = \pm q_2$ on the unit circle.

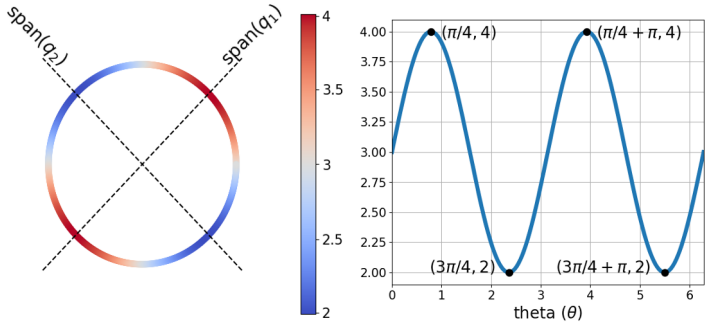
In $\mathbb{R}^2$, another way to visualize the quadratic form is to write unit vectors as

$$x = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}, \quad \theta \in [0, 2\pi).$$

Figure 42.3-(b) plots

$$f(\theta) = \begin{bmatrix} \cos(\theta) & \sin(\theta) \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}.$$

The plot marks the maxima and minima of $f(\theta)$. The function reaches its maximum at $\theta = \pi/4$ and $\theta = \pi/4 + \pi$, which correspond to the directions of q_1 and $-q_1$. Similarly, it reaches its minimum at $\theta = 3\pi/4$ and $\theta = 3\pi/4 + \pi$, which correspond to the directions of q_2 and $-q_2$.



(a) The unit circle colored according to the values of the quadratic form.

(b) The quadratic form values as a function of θ.

Fig. 42.3: Optimization of the quadratic form $\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ over the unit circle $x_1^2 + x_2^2 = 1$.

Example 42.15 Consider the symmetric matrix

$$S = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 2 \end{bmatrix}.$$

The eigenvalues of S are $\lambda_1 = 5$, $\lambda_2 = 2$, and $\lambda_3 = 0$, with the corresponding orthonormal eigenvectors

$$q_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad q_2 = \frac{1}{\sqrt{6}} \begin{bmatrix} 2 \\ -1 \\ -1 \end{bmatrix}, \quad \text{and} \quad q_3 = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}.$$

Figure 42.4-(a) shows the unit sphere in $\mathbb{R}^3$, with each point colored by the value of the quadratic form $f(x) = x^T S x$ for the corresponding unit vector x. The maximum value of $f(x)$ on the unit sphere is $\lambda_1 = 5$, achieved at the points $\pm q_1$. The minimum value is $\lambda_3 = 0$, which occurs at $\pm q_3$. Meanwhile, the points $\pm q_2$ are saddle points of $f(x)$; that is, the function increases in some directions and decreases in others around these points.

Another way to visualize a quadratic form in $\mathbb{R}^3$ is to express unit vectors in spherical coordinates:

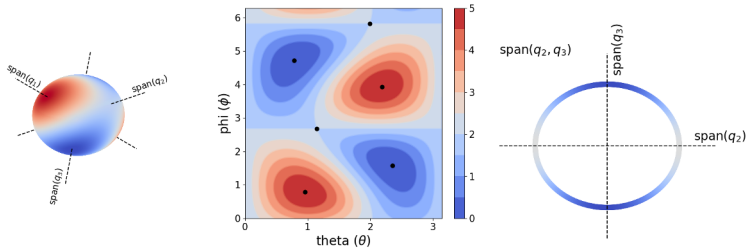
$$\begin{bmatrix} \sin(\theta)\cos(\phi) \\ \sin(\theta)\sin(\phi) \\ \cos(\theta) \end{bmatrix}, \quad \phi \in [0, 2\pi), \quad \text{and} \quad \theta \in [0, \pi].$$

Figure 42.4-(b) shows the contour plot of the function

$$f(\phi, \theta) = \begin{bmatrix} \sin(\theta)\cos(\phi), \ \sin(\theta)\sin(\phi), \ \cos(\theta) \end{bmatrix} \begin{bmatrix} 3 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 2 \end{bmatrix} \begin{bmatrix} \sin(\theta)\cos(\phi) \\ \sin(\theta)\sin(\phi) \\ \cos(\theta) \end{bmatrix}.$$

The plot marks the maxima, minima, and saddle points. From the contour plot, we see two maxima corresponding to the directions $\pm q_1$, two minima corresponding to the directions $\pm q_3$, and two saddle points corresponding to the directions $\pm q_2$.

Figure 42.4-(c) shows the intersection of the unit sphere with the plane $\text{span}(q_1)^{\perp} = \text{span}(q_2, q_3)$, colored by the values of $f(x)$. As predicted by (42.5), $f(x)$ reaches its maximum value $\lambda_2 = 2$ at $\pm q_2$ within this plane.



(a) The quadratic form on the unit sphere. (b) The quadratic form values as a function θ and ϕ. (c) The quadratic form restricted to $\text{span}\{q_2, q_3\}$.

Fig. 42.4: Optimization of the quadratic form $\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 3 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$ over the unit sphere $x_1^2 + x_2^2 + x_3^2 = 1$.

42.2 Positive (semi)definite matrices and their characterizations

In this section, we introduce positive (semi)definite matrices, study their properties, and give a few ways to characterize them.

We start with a definition based on their associated quadratic form.

Definition 42.16. (Positive (semi)definite matrices: energy-based definition)

Let S be a real $n \times n$ symmetric matrix.

(a) S is a **positive semidefinite matrix** if and only if $x^T S x \geqslant 0$ for all $x \in \mathbb{R}^n$.
(b) S is a **positive definite matrix** if and only if $x^T S x > 0$ for all nonzero $x \in \mathbb{R}^n$.

We can define negative (semi)definite and indefinite symmetric matrices in a similar way.

In physics, the energy of a system in state x is often written as $x^T Sx$ or $\frac{1}{2}x^T Sx$. This is why the definition of a positive (semi)definite matrix is often called the energy-based definition.

Example 42.17 The energy associated with the identity matrix is given by $x^T I_n x = x^T x = \|x\|_2^2$, which is positive for any nonzero x. Therefore, the identity matrix is positive definite.

Example 42.18. (Positive (semi)definite diagonal matrices)

An $n \times n$ diagonal matrix $D = \text{diag}(d_1, \ldots, d_n)$ is positive semidefinite if all $d_i \geqslant 0$, and positive definite if all $d_i > 0$. We will prove the definite case. The semidefinite case follows the same reasoning and is left to the reader.

We have

$$x^T Dx = \sum_{i=1}^{n} d_i x_i^2.$$

If $d_i > 0$ for all i, it is clear that $x^T Dx > 0$ for all nonzero x. Now, assume D is positive definite but $d_i \leqslant 0$ for some i. Then,

$$e_i^T De_i = d_i \leqslant 0,$$

which contradicts the definition of positive definiteness. Thus, $d_i > 0$ for all i.

Example 42.19 Consider the symmetric matrix $S = \left[\begin{smallmatrix} 2 & 3 \\ 3 & 5 \end{smallmatrix}\right]$. Evaluating the energy $x^T Sx$ gives $2x_1^2 + 6x_1x_2 + 5x_2^2$. To check if S is positive definite, we use the technique of completing the squares. We rewrite

$$\begin{aligned} 2x_1^2 + 6x_1x_2 + 5x_2^2 &= 2(x_1^2 + 3x_1x_2) + 5x_2^2 \\ &= 2\left(x_1^2 + 3x_1x_2 + \frac{9}{4}x_2^2\right) - \frac{9}{2}x_2^2 + 5x_2^2 \\ &= 2\left(x_1 + \frac{3}{2}x_2\right)^2 + \frac{1}{2}x_2^2. \end{aligned}$$

This expression shows that $x^T Sx$ is a sum of two squares, $\left(x_1 + \frac{3}{2}x_2\right)^2$ and x_2^2, each scaled by a positive coefficient. The two squares can vanish at the same time

only when $x = 0$. Therefore, $x^T Sx > 0$ for all nonzero x, which confirms that S is a positive definite matrix.

The "completing the square" method from the last example works for any matrix (see Problem 42.4 for the 2×2 case). But for larger matrices, the process gets tedious because you have to expand and handle many terms at once (see Problem 42.5). Later in this chapter and in Chapter 43, we will see other ways to check if a matrix is positive (semi)definite.

Sometimes the easiest way to check if a matrix is positive (semi)definite is to use the energy-based definition.

Proposition 42.20. (Sum of positive (semi)definite matrices)

The sum $S + T$ of two symmetric positive (semi)definite matrices $S, T \in \mathbb{R}^{n \times n}$ is also symmetric positive (semi)definite.

Proof We will prove the result for positive definite matrices; the argument for positive semidefinite matrices is similar.

Since S and T are symmetric, their sum $S + T$ is also symmetric. Now, consider any nonzero vector x. We have

$$x^T (S + T)x = x^T Sx + x^T Tx > 0,$$

because S and T are positive definite. This shows that $x^T (S+T)x > 0$ for all nonzero x, which implies $S + T$ is positive definite. □

Using induction, we can extend Proposition 42.20 to show that the sum $S_1 + S_2 + \cdots + S_k$ of k positive definite symmetric matrices $S_1, S_2, \ldots, S_k$ is also symmetric and positive definite.

There is an important matrix operation that preserves the positive (semi)definiteness of matrices.

Definition 42.21. (Congruent matrices and congruence transformation)

A matrix $A \in \mathbb{R}^{n \times n}$ is **congruent** to $B \in \mathbb{R}^{n \times n}$ if there exists an invertible matrix $P \in \mathbb{R}^{n \times n}$ such that $B = P^T AP$. The operation $B = P^T AP$ is called a **congruence transformation**.

Note that if A is congruent to B, then $B = P^T AP$. This implies $A = (P^T)^{-1} BP^{-1} = (P^{-1})^T BP^{-1}$. Thus, B is also congruent to A.

Theorem 42.22. (Preservation of positive (semi)definiteness under congruence)

Let P be an invertible matrix and S a symmetric matrix. Then, S is positive (semi)definite if and only if $T = P^T SP$ is positive (semi)definite.

Proof First, note that S is symmetric if and only if $T = P^T SP$ is symmetric.

Assume S is positive semidefinite, that is, $x^T Sx \geqslant 0$ for all $x \neq 0$. For any $x \neq 0$, the invertibility of P implies $Px \neq 0$. Then,

$$x^T Tx = x^T P^T SPx = (Px)^T S(Px) \geqslant 0,$$

since S is positive semidefinite.

Conversely, assume T is positive semidefinite. Since T is congruent to S, we can apply the same argument with P replaced by P^{-1} to show that S is positive semidefinite.

The argument for positive definite matrices is identical, except that $\geqslant$ is replaced by $>$ throughout the proof. □

42.2.1 Eigenvalue characterization

The Spectral Theorem for symmetric matrices (Theorem 41.3) tells us that any real symmetric matrix S is congruent to a diagonal matrix D with the eigenvalues of S on the diagonal. Since congruence preserves definiteness (Theorem 42.22), S is positive (semi)definite if and only if D is. As we saw in Example 42.18, a diagonal matrix is positive (semi)definite if and only if its diagonal entries are all positive (or nonnegative). This gives the following characterization of positive (semi)definite matrices.

Theorem 42.23. (Eigenvalue characterization of positive (semi)definite matrices)

A symmetric matrix S is positive semidefinite if and only if all its eigenvalues are nonnegative. It is positive definite if and only if all its eigenvalues are positive.

Theorem 42.23 provides a computational way to check if a symmetric matrix is positive (semi)definite: compute its eigenvalues and check that they are all positive (for positive definiteness) or nonnegative (for positive semidefiniteness). In the following sections, we introduce other characterizations that are often more efficient in practice.

An immediate consequence of Theorem 42.23 is the following result.

Corollary 42.24. (Invertibility and positive definiteness)

Every symmetric positive definite matrix $S \in \mathbb{R}^{n \times n}$ is invertible. Moreover, its inverse is also positive definite.

Proof Since S is symmetric, so is S^{-1}.

Let $\lambda_1, \ldots, \lambda_n$ be the eigenvalues of S. By Theorem 42.23, all $\lambda_i > 0$, so S is invertible by Proposition 36.24-(a).

By Proposition 36.24-(b), the eigenvalues of S^{-1} are $1/\lambda_1, \ldots, 1/\lambda_n$. Since each $\lambda_i > 0$, it follows that $1/\lambda_i > 0$ as well. So all eigenvalues of S^{-1} are positive, and Theorem 42.23 implies that S^{-1} is positive definite. □

42.2.2 Leading principal minor characterization

As we saw in Chapter 37, computing the eigenvalues of a matrix is not a trivial task. Can we determine whether a matrix is positive definite using only its entries? In Example 42.19, we used the "completing the square" method to check for positive (semi)definiteness. Problem 42.4 asks you to apply this to general 2×2 matrices, and Problem 42.5 extends the idea to 3×3 matrices. The process becomes more involved but leads to entry-based characterizations.

In this section, we introduce another technique that generalizes these characterizations to $n \times n$ matrices. We begin by reviewing some definitions and results. Some of these were introduced in Chapter 22, but we include them again for clarity.

Definition 42.25. (Principal submatrices and minors)

Let A be an $n \times n$ matrix. For any subset $\{i_1, \ldots, i_m\}$ of $\{1, 2, \ldots, n\}$, the submatrix formed by the rows and columns of A at positions $\{i_1, \ldots, i_m\}$ is called a **principal submatrix** of A. We denote this submatrix by $A^{\{i_1,\ldots,i_m\}}$. The determinant of a principal submatrix is called a **principal minor**.

Example 42.26 For a matrix A, the matrix itself and its diagonal entries are examples of principal submatrices.

Example 42.27 Consider the matrix

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}.$$

Selecting the subset of indices $\{1, 3\}$ gives the principal submatrix

$$A^{\{1,3\}} = \begin{bmatrix} a & c \\ g & i \end{bmatrix}.$$

The determinant of this principal submatrix is the associated principal minor

$$\det(A^{\{1,3\}}) = ai - cg.$$

If a matrix is symmetric and positive (semi)definite, then all its principal submatrices are too, as shown next.

Theorem 42.28. (Positive (semi)definiteness of principal submatrices)

If S is symmetric and positive (semi)definite, then every principal submatrix of S is also symmetric and positive (semi)definite.

Proof We prove the result for positive definite matrices. The proof for positive semidefinite matrices is identical, replacing the strict inequality > 0 with $\geqslant 0$.

Let S^B be a principal submatrix of S, where B is a subset of $\{1, 2, \ldots, n\}$. Since S^B is formed by selecting the same rows and columns from the symmetric matrix S, it is also symmetric.

Since S is positive definite, for any nonzero vector $x \in \mathbb{R}^n$, we have $x^T Sx > 0$. Now, take a vector $x \in \mathbb{R}^n$ with nonzero entries only in the positions indicated by B, and zeros elsewhere. Let x^B be the vector obtained by keeping only the components of x corresponding to B. Since $x^B \neq 0$, we have

$$(x^B)^T S^B x^B = x^T Sx > 0.$$

Thus, S^B is positive definite and symmetric. Since S^B was chosen arbitrarily, all principal submatrices of S are symmetric and positive definite. □

Example 42.29 Consider the matrix

$$S = \begin{bmatrix} -1 & 2 & 3 \\ 2 & 0 & 4 \\ 3 & 4 & 1 \end{bmatrix}.$$

Is this matrix positive definite? We can immediately see that it is not, because the principal submatrix

$$S^{\{1\}} = \begin{bmatrix} -1 \end{bmatrix}$$

is not positive definite.

In general, if a matrix S is positive definite, then by Theorem 42.28 every main diagonal entry must be positive, since these entries are the 1×1 principal minors.

Among the principal submatrices, a certain smaller subset plays an important role in characterizing positive definite matrices.

Definition 42.30. (Leading principal submatrices and minors)

Let A be an $n \times n$ matrix. A **leading principal submatrix** of A is a principal submatrix formed by selecting the first k rows and columns of A, where $k = 1, 2, \ldots, n$.

We denote this submatrix by A_k. The determinant of a leading principal submatrix is called a **leading principal minor**.

Example 42.31 Consider the matrix

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}.$$

The leading principal submatrices of A are

$$A_1 = \begin{bmatrix} a \end{bmatrix}, \qquad A_2 = \begin{bmatrix} a & b \\ d & e \end{bmatrix}, \qquad \text{and} \qquad A_3 = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}.$$

These submatrices are formed by taking the first one, two and three rows and columns of A, respectively.

Using colon notation, we can write the leading principal submatrix A_k as $A_k = A[1:k, 1:k]$.

We are now ready for another characterization of positive definite matrices based on leading principal minors. This is known as Sylvester's criterion[1].

Theorem 42.32. (Leading principal minor characterization of positive definite matrices)

A symmetric matrix $S \in \mathbb{R}^{n\times n}$ is positive definite if and only if all its leading principal minors are positive.

Proof Assume S is positive definite. We need to show that all its leading principal minors are positive. Since the leading principal submatrices of S are specific examples of principal submatrices, Theorem 42.28 implies that these submatrices are positive definite. By Theorem 42.23, all eigenvalues of the leading principal submatrices are positive. Since the determinant of a matrix is the product of its eigenvalues, all leading principal minors of S are positive.

Now assume all leading principal minors of S are positive. We prove that S is positive definite by induction on its size n.

For $n = 1$, the result is immediate. Assume the claim holds for all $(n-1)\times(n-1)$ matrices, and let S be an $n \times n$ matrix partitioned as

[1] Named after James Joseph Sylvester, a 19th-century English mathematician who made significant contributions to Matrix Theory.

$$S = \begin{bmatrix} S_{n-1} & v \\ v^T & \alpha \end{bmatrix},$$

where S_{n-1} is the $(n-1) \times (n-1)$ leading principal submatrix of S, v is a column vector, and α is a scalar.

Since the leading principal minors of S_{n-1} are also leading principal minors of S, they are positive. By the inductive hypothesis, S_{n-1} is positive definite and therefore invertible.

We can factor S as

$$S = \begin{bmatrix} I_{n-1} & 0 \\ v^T S_{n-1}^{-1} & 1 \end{bmatrix} \begin{bmatrix} S_{n-1} & 0 \\ 0 & \alpha - v^T S_{n-1}^{-1} v \end{bmatrix} \begin{bmatrix} I_{n-1} & S_{n-1}^{-1} v \\ 0 & 1 \end{bmatrix}.$$

Since S_{n-1} is symmetric, this is a congruence transformation. By Theorem 42.22, S is positive definite if and only if the block diagonal matrix $\operatorname{diag}(S_{n-1},\ \alpha - v^T S_{n-1}^{-1} v)$ is positive definite.

Let $x = \begin{bmatrix} y \\ x_n \end{bmatrix}$, where $y \in \mathbb{R}^{n-1}$ and $x_n \in \mathbb{R}$. Then

$$x^T \operatorname{diag}(S_{n-1},\ \alpha - v^T S_{n-1}^{-1} v) x = y^T S_{n-1} y + x_n^2 (\alpha - v^T S_{n-1}^{-1} v).$$

Since S_{n-1} is positive definite, $y^T S_{n-1} y \geqslant 0$, with strict inequality whenever $y \neq 0$. Moreover, $\det(S) = \det(S_{n-1}) \cdot (\alpha - v^T S_{n-1}^{-1} v) > 0$. Since $\det(S_{n-1}) > 0$, it follows that $\alpha - v^T S_{n-1}^{-1} v > 0$. Therefore, $x_n^2 \left(\alpha - v^T S_{n-1}^{-1} v\right) \geqslant 0$, with strict inequality whenever $x_n \neq 0$. Now let $x \neq 0$. If $y \neq 0$, the first term is positive. If $y = 0$, then $x_n \neq 0$, so the second term is positive. Hence

$$x^T \operatorname{diag}\left(S_{n-1}, \alpha - v^T S_{n-1}^{-1} v\right) x > 0$$

for every nonzero x. Thus, the block diagonal matrix is positive definite, and, by congruence, so is S.

For an $n \times n$ matrix, there are n leading principal minors. Theorem 42.32 uses n conditions on the entries to characterize positive definiteness. The following examples illustrate these conditions for 2×2 and 3×3 matrices. If you have worked on Problems 42.4 and 42.5, you will notice that you obtained the same conditions using the "completing the squares" method.

Example 42.33 Using Theorem 42.32, a 2×2 symmetric matrix $S = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$ is positive definite if and only if

$$\det(S_1) = \det\begin{bmatrix} a \end{bmatrix} = a > 0, \quad \text{and}$$
$$\det(S_2) = \det(S) = \det\begin{bmatrix} a & b \\ b & c \end{bmatrix} = ac - b^2 > 0.$$

Example 42.34 A 3×3 symmetric matrix

$$S = \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}$$

is positive definite if and only if

$$\det(S_1) = \det(\begin{bmatrix} a \end{bmatrix}) = a > 0,$$
$$\det(S_2) = \det\left(\begin{bmatrix} a & b \\ b & d \end{bmatrix}\right) = ad - b^2 > 0 \quad \text{and}$$
$$\det(S_3) = \det(S) = \det\left(\begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}\right) = adf + 2bec - c^2 d - e^2 a - b^2 f > 0.$$

There is an important difference between the eigenvalue characterization in Theorem 42.23 and the leading principal minor characterization in Theorem 42.32. While the eigenvalue characterization applies to both positive definite and positive semidefinite matrices, the leading principal minor characterization applies only to positive definite matrices. This raises a natural question: does Theorem 42.32 hold for symmetric positive semidefinite matrices if we replace the condition of leading principal minors being positive with being nonnegative? The following example shows that the answer is no.

Example 42.35 Consider the matrix

$$S = \begin{bmatrix} 0 & 0 \\ 0 & -3 \end{bmatrix}.$$

We compute

$$\det(S_1) = \det([0]) = 0 \geqslant 0,$$
$$\det(S_2) = \det(S) = 0 \geqslant 0.$$

Despite these nonnegative determinants, S is not positive semidefinite because it has a negative eigenvalue, namely -3.

For a symmetric matrix to be positive semidefinite it is necessary that all the principal minors are nonnegative, not just the principal ones. We will discuss this result in Chapter 43.

42.2.3 Pivot characterization

Theorem 42.32 shows that every positive definite matrix has nonsingular leading principal submatrices. This guarantees the existence of an LU factorization without row exchanges (see Theorem 22.10), so Gaussian elimination can proceed without pivoting.

In this section, we present another way to characterize positive definite matrices by looking at the signs of the pivots that appear during Gaussian elimination. These pivots are the diagonal entries of the upper triangular matrix U.

Theorem 42.36. (Pivot characterization of positive definite matrices)

A symmetric matrix S is positive definite if and only if all its pivots are positive.

Proof Assume S is positive definite. Then, by Theorems 22.10 and 42.32, S has an LU factorization. We will show that the pivots are positive using induction on the size n of the matrix.

For a 1×1 positive definite matrix S, we have $S > 0$. Its LU factorization is simply $S = 1 \cdot S$, with a positive pivot equal to S, which is positive.

Assume the result holds for symmetric matrices of size $n - 1$. Now consider an $n \times n$ positive definite matrix S, which we partition as

$$S = \begin{bmatrix} S_{n-1} & v \\ v^T & \alpha \end{bmatrix},$$

where S_{n-1} is the leading principal submatrix of size $n - 1$. By Theorem 42.28, S_{n-1} is positive definite. By the induction hypothesis, S_{n-1} has an LU factorization $S_{n-1} = L'U'$ where U' has positive diagonal entries (i.e., positive pivots).

To show that S has an LU factorization with positive pivots, we need to find vectors x, y, and a scalar $z > 0$ such that

$$S = \begin{bmatrix} L' & 0 \\ x^T & 1 \end{bmatrix} \begin{bmatrix} U' & y \\ 0 & z \end{bmatrix} = \begin{bmatrix} L'U' & L'y \\ x^T U' & x^T y + z \end{bmatrix}.$$

This gives the equations

$$v = L'y, \quad v^T = x^T U', \quad \text{and} \quad \alpha = x^T y + z.$$

Since L' is unit lower triangular, it is invertible, so $y = (L')^{-1}v$. Similarly, U' is upper triangular with positive diagonal entries, so it is invertible, and $x^T = v^T(U')^{-1}$. Finally, $z = \alpha - x^T y$.

The scalar z is the last pivot. To verify $z > 0$, note that $\det(S) > 0$ (since S is positive definite), and

$$\det(S) = \det(L') \det(U') z.$$

Since $\det(L') > 0$ (diagonal entries are 1) and $\det(U') > 0$ (pivots are positive), it follows that $z > 0$. Thus, S has an LU factorization with positive pivots.

Conversely, assume S has an LU factorization with positive pivots. Since S is symmetric, by Theorem 22.15, we can factor it as $S = LDL^T$, where $D = \text{diag}(d_1, \ldots, d_n)$ is a diagonal matrix with positive main diagonal entries. Define $D^{1/2} := \text{diag}(\sqrt{d_1}, \ldots, \sqrt{d_n})$. Then, we have

$$S = LD^{1/2}D^{1/2}L^T = (LD^{1/2})(LD^{1/2})^T.$$

This shows that S can be factored as $S = B^T B$, where $B := D^{1/2}L^T$ is invertible.

Now, for any $x \neq 0$, we have

$$x^T Sx = x^T B^T Bx = (Bx)^T (Bx) = \|Bx\|_2^2 \geqslant 0.$$

Thus, S is positive semidefinite. Furthermore, since B is invertible, $Bx = 0$ only when $x = 0$. For any $x \neq 0$, $Bx \neq 0$, which implies $\|Bx\|_2^2 > 0$. Therefore, S is positive definite[2]. □

Theorem 42.36 gives another practical way to check if a matrix is positive definite. Instead of computing eigenvalues, you can perform Gaussian elimination and check that all the pivots are positive. This approach is more efficient, since LU factorization is generally faster than computing eigenvalues.

Example 42.37 Consider the matrix

$$S = \begin{bmatrix} 1 & -2 & 3 \\ -2 & 5 & 4 \\ 3 & 4 & 1 \end{bmatrix}.$$

We will use Theorem 42.36 to determine whether S is positive definite. Gaussian elimination reduces S to

$$U = \begin{bmatrix} 1 & -2 & 3 \\ 0 & 1 & 10 \\ 0 & 0 & -108 \end{bmatrix}.$$

The pivots are 1, 1, and -108. Since one of the pivots is negative, S is not positive definite.

Theorem 42.36 applies only to positive definite matrices. What happens if one replaces positive pivots with nonnegative pivots for positive semidefinite matrices? Can an LU factorization even be guaranteed for positive semidefinite matrices? Section 43.4.3 addresses these questions.

[2] What we have established here is part of another characterization of positive (semi)definite matrices: they can be expressed in the form $S = B^T B$. We will discuss this further in the next section.

42.2.4 Gram characterization and the Cholesky factorization

In Chapter 17, we introduced Gram matrices: symmetric matrices of the form $S = B^T B$ for some (possibly rectangular) matrix B. The matrix B is called a Gram factor of S. Gram matrices give us another way to characterize positive (semi)definite matrices.

Theorem 42.38. (Gram characterization of positive (semi)definite matrices)

A symmetric matrix S is positive semidefinite if and only if there exists a matrix B such that $S = B^T B$. It is positive definite if and only if B has full column rank.

Proof Assume $S = B^T B$ for some matrix B. Clearly, S is symmetric. For any $x \neq 0$, we have

$$x^T Sx = x^T B^T Bx = (Bx)^T (Bx) = \|Bx\|_2^2 \geqslant 0.$$

Thus, S is positive semidefinite. The inequality $\geqslant 0$ arises because Bx may be zero for some nonzero x. However, if B has full column rank, then $Bx \neq 0$ for all $x \neq 0$, so $\|Bx\|_2^2 > 0$. In this case, $S = B^T B$ is positive definite.

Conversely, assume S is positive semidefinite. By the Spectral Theorem (Theorem 41.3), we can write $S = QDQ^T$, where Q is orthogonal and $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ contains the eigenvalues of S. By Theorem 42.23, all $\lambda_i \geqslant 0$.

Define $D^{1/2} := \operatorname{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_n})$ and let $B = D^{1/2} Q^T$. Then

$$S = QDQ^T = QD^{1/2} D^{1/2} Q^T = B^T B.$$

Thus, S is a Gram matrix with Gram factor $B := D^{1/2} Q^T$.

If S is positive definite, all eigenvalues $\lambda_i > 0$, so $D^{1/2}$ and $B = D^{1/2} Q^T$ are invertible. Since B is invertible, it has full column rank. □

The following example shows that if S is a Gram matrix and the columns of its Gram factor are linearly dependent, then S is positive semidefinite but not positive definite.

Example 42.39 Consider the matrix

$$S = B^T B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix}.$$

The matrix S is singular because its columns are linearly dependent. As a result, zero is an eigenvalue of S; so S is not positive definite. However, S is positive semidefinite since its eigenvalues (6, 0 and 0) are nonnegative.

Gram factorizations of positive (semi)definite matrices need not be unique. For any orthogonal matrix Q, you can change the basis by setting $C = QB$, which gives

an alternative Gram factorization

$$C^T C = (QB)^T (QB) = B^T (Q^T Q) B = B^T B = S.$$

If you require the Gram factors to be square, this orthogonal change of basis becomes the only degree of freedom (see Problem 42.36). Moreover, the size of the Gram factor is not unique either. For example, you can append B with a zero matrix to produce another Gram factorization:

$$S = B^T B = \begin{bmatrix} B^T & 0 \end{bmatrix} \begin{bmatrix} B \\ 0 \end{bmatrix}.$$

Among the many possible Gram factorizations of a positive definite matrix S, one is especially important: the **Cholesky factorization**. In this factorization, the Gram factor B is square, upper triangular, and has positive diagonal entries.

Theorem 42.40. (Cholesky factorization)

A symmetric matrix S is positive definite if and only if it has a unique Gram factorization in which the Gram factor is square, upper triangular with positive diagonal entries. This factorization is known as the **Cholesky factorization** of S.

Proof Assume S is symmetric positive definite. By Theorem 42.36, S has an LU factorization $S = LU$, where L is a lower triangular matrix, and U is an upper triangular matrix with positive diagonal entries. Since S is symmetric, we have $U = DL^T$ (from the LDL^T factorization in Theorem 22.15), where D is a diagonal matrix with positive entries (the diagonal entries of U).

We can rewrite the LU factorization as

$$S = LDL^T = L(D^{1/2}D^{1/2})L^T = (LD^{1/2})(LD^{1/2})^T = \widehat{L}\widehat{L}^T,$$

where $\widehat{L} = LD^{1/2}$ is a lower triangular matrix with positive diagonal entries. Thus, S has a Cholesky factorization.

To show the uniqueness of the Cholesky factorization, assume $L_1 L_1^T = L_2 L_2^T$ are two Cholesky factorizations of S. Since L_1 and L_2 are invertible, we have

$$L_2^{-1} L_1 = L_2^T (L_1^T)^{-1} = (L_1^{-1} L_2)^T.$$

Since $L_2^{-1} L_1$ is lower triangular and $(L_1^{-1} L_2)^T$is upper triangular, we have that $L_2^{-1} L_1$ is both lower and upper triangular, so it must be diagonal. Let $\widehat{D} = L_2^{-1} L_1$, where $\widehat{D}$ is a diagonal matrix. We then have $L_1 = L_2 \widehat{D}$, and

$$\widehat{D} = L_2^{-1} L_1 = (L_1^{-1} L_2)^T = (\widehat{D}^{-1})^T = \widehat{D}^{-1}.$$

Thus, the diagonal entries of $\widehat{D}$ are ± 1. Because L_1 and L_2 have positive diagonal entries, $\widehat{D} = I_n$, and $L_1 = L_2$. Thus, the Cholesky factorization is unique.

To prove the converse, assume $S = LL^T$, where L is lower triangular with positive diagonal entries. This is a Gram factorization with L^T as the Gram factor. Since L^T is invertible, Theorem 42.38 implies S is positive definite. □

The proof of Theorem 42.40 suggests a method for obtaining the Cholesky factorization. First, compute the LU factorization of the matrix. From there, derive an LDL^T factorization, and finally use it to determine the Cholesky factorization. Let us see an example.

Example 42.41 Consider the symmetric matrix

$$S = \begin{bmatrix} 5 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{bmatrix}.$$

Using Gaussian elimination, we find the LU factorization:

$$S = \begin{bmatrix} 1 & 0 & 0 \\ 2/5 & 1 & 0 \\ 2/5 & 1/21 & 1 \end{bmatrix} \begin{bmatrix} 5 & 2 & 2 \\ 0 & 21/5 & 1/5 \\ 0 & 0 & 4/21 \end{bmatrix}.$$

From the LU factorization, we obtain the LDL^T factorization:

$$S = \begin{bmatrix} 1 & 0 & 0 \\ 2/5 & 1 & 0 \\ 2/5 & 1/21 & 1 \end{bmatrix} \begin{bmatrix} 5 & 0 & 0 \\ 0 & 21/5 & 0 \\ 0 & 0 & 4/21 \end{bmatrix} \begin{bmatrix} 1 & 2/5 & 2/5 \\ 0 & 1 & 1/21 \\ 0 & 0 & 1 \end{bmatrix}.$$

Finally, we split the diagonal factor into two diagonal factors by taking the square root of the pivots to obtain the Cholesky factorization:

$$\begin{aligned} S &= \begin{bmatrix} 1 & 0 & 0 \\ 2/5 & 1 & 0 \\ 2/5 & 1/21 & 1 \end{bmatrix} \begin{bmatrix} \sqrt{5} & 0 & 0 \\ 0 & \sqrt{21/5} & 0 \\ 0 & 0 & \sqrt{4/21} \end{bmatrix} \begin{bmatrix} \sqrt{5} & 0 & 0 \\ 0 & \sqrt{21/5} & 0 \\ 0 & 0 & \sqrt{4/21} \end{bmatrix} \begin{bmatrix} 1 & 2/5 & 2/5 \\ 0 & 1 & 1/21 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \sqrt{5} & 0 & 0 \\ 2/\sqrt{5} & \sqrt{21/5} & 0 \\ 2/\sqrt{5} & 1/\sqrt{105} & \sqrt{4/21} \end{bmatrix} \begin{bmatrix} \sqrt{5} & 2/\sqrt{5} & 2/\sqrt{5} \\ 0 & \sqrt{21/5} & 1/\sqrt{105} \\ 0 & 0 & \sqrt{4/21} \end{bmatrix}. \end{aligned}$$

This is the Cholesky factorization of S.

In Theorem 42.40 (and Example 42.41), we derived the Cholesky factorization from the $S = LDL^T$ factorization obtained using Gaussian elimination. Another approach is to start from a Gram factorization $S = B^T B$ and apply the QR factorization to B (see Problem 42.39).

In Chapter 43, we introduce the *Cholesky algorithm*, a method built specifically for positive definite matrices. It is faster than LU, spectral, or QR decompositions. We

also extend the Cholesky factorization to positive semidefinite matrices by proving the existence of a *semi-Cholesky factorization*.

42.2.5 Square root characterization

We can use the concept of a matrix square root to characterize symmetric positive (semi)definite matrices.

Definition 42.42. (Matrix square root)

A matrix $B \in \mathbb{R}^{n\times n}$ is called a **square root** of $A \in \mathbb{R}^{n\times n}$ if it satisfies the equation $A = B^2$.

For a scalar $a \geqslant 0$, there are exactly two square roots, $\pm\sqrt{a}$, which coincide if $a = 0$. For matrices, however, the existence and uniqueness of square roots depend on the size and properties of the matrix A. When A has size $n > 1$, it may have no square roots, finitely many, or infinitely many.

A matrix can have multiple square roots. For instance, just like with scalars, if $A = B^2$ then we also have $A = (-B)^2$.

Example 42.43 Consider the matrix $A = \left[\begin{smallmatrix} 2 & 2 \\ 2 & 2 \end{smallmatrix}\right]$. You can verify that the following matrices are both square roots of A:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}.$$

In fact, these are the only square roots of A (see Problem 42.45).

Some matrices have no square root.

Example 42.44 The matrix $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$ has no square root (see Problem 42.43).

Diagonal matrices with nonnegative diagonal entries have especially simple square roots.

Example 42.45 Consider the matrix $A = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$, where $\lambda_i \geqslant 0$ for $i = 1, 2, \ldots, n$. The matrix $B = \operatorname{diag}(\pm\sqrt{\lambda_1}, \pm\sqrt{\lambda_2}, \ldots, \pm\sqrt{\lambda_n})$ is a square root of A for any choice of the signs of its diagonal entries.

A diagonal matrix may have square roots that are not diagonal matrices.

Example 42.46 The identity matrix $I_2 = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]$ has the following four diagonal square roots:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \text{and} \quad \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}.$$

But I_2 also has infinitely many non-diagonal square roots. Any matrix of the form

$$\begin{bmatrix} \pm\sqrt{1-bc} & b \\ c & \mp\sqrt{1-bc} \end{bmatrix},$$

where $b, c \in \mathbb{R}$ and $bc \leqslant 1$, is a square root of I_2. For instance, the following matrices are nondiagonal square roots of I_2:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad \text{and} \quad \begin{bmatrix} 4/5 & 3/5 \\ 3/5 & -4/5 \end{bmatrix}.$$

If the matrix S is positive semidefinite, it has a unique positive semidefinite square root, as the next theorem shows. This result extends the familiar property that every nonnegative number $a \geqslant 0$ has a unique nonnegative square root $\sqrt{a} \geqslant 0$.

Theorem 42.47. (Square root characterization of positive (semi)definite matrices)

A symmetric matrix S is positive semidefinite if and only if there exists a symmetric positive semidefinite matrix B such that $S = B^2$. This matrix B is unique and is called the **positive semidefinite square root of** S, denoted by $S^{1/2}$. Moreover, S is positive definite if and only if B is positive definite.

Proof Assume $S = B^2$ for some symmetric positive semidefinite matrix B. Since B is symmetric, we have $S = B^2 = B^T B$, which is a Gram matrix. By Theorem 42.38, S is positive semidefinite. If B is positive definite, then B is invertible, and thus S is a positive definite Gram matrix.

For the converse, assume S is positive semidefinite. By the Spectral Theorem (Theorem 41.3), $S = QDQ^T$, where Q is orthogonal and $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ contains the eigenvalues of S. Since S is positive semidefinite, all eigenvalues λ_i of S are nonnegative (Theorem 42.23).

Define $D^{1/2} := \operatorname{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_n})$ and $S^{1/2} := QD^{1/2}Q^T$. To check that $S^{1/2}$ is indeed a square root of S, compute

$$S^{1/2}S^{1/2} = QD^{1/2}Q^T QD^{1/2}Q^T = QD^{1/2}D^{1/2}Q^T = QDQ^T = S.$$

Since $D^{1/2}$ is a diagonal matrix with nonnegative entries, it is positive semidefinite (see Example 42.18). Because $S^{1/2} = QD^{1/2}Q^T$ is congruent to $D^{1/2}$, it is also

positive semidefinite by Theorem 42.22. If S is positive definite, then all entries on the diagonal of $D^{1/2}$ are positive, so both $D^{1/2}$ and $S^{1/2}$ are positive definite.

To prove the uniqueness[3] of the positive semidefinite square root, assume there exist two distinct positive semidefinite matrices B and C such that $B^2 = S = C^2$ but $B \neq C$. Consider the symmetric matrix $B - C$. Since $B \neq C$, the matrix $B - C$ is nonzero, so it has a nonzero eigenvalue λ with an eigenvector $x \neq 0$. Using $B^2 - C^2 = 0$, we can write

$$0 = x^T(B^2 - C^2)x = x^T[B(B - C) + (B - C)C]x.$$

Expanding, we get

$$0 = x^T B(B - C)x + x^T(B - C)Cx = \lambda(x^T Bx + x^T Cx).$$

Since $\lambda \neq 0$ and both B and C are positive semidefinite, $x^T Bx \geqslant 0$ and $x^T Cx \geqslant 0$. This forces $x^T Bx = 0$ and $x^T Cx = 0$. Thus

$$0 = x^T(B - C)x = \lambda\|x\|_2^2,$$

which is a contradiction since $\lambda \neq 0$ and $\|x\|_2^2 > 0$. Therefore, $B = C$. □

The proof of Theorem 42.47 also gives a way to compute the positive semidefinite square root of a matrix. Given the spectral decomposition $S = QDQ^T$, where Q is orthogonal and D is diagonal, the square root is $S^{1/2} = QD^{1/2}Q^T$.

42.3 Positive (semi)definite matrices in Machine Learning applications

In this section, we study the positive definiteness of three matrices that often appear in applications: the graph Laplacian, the Hessian, and the covariance matrix.

42.3.1 The graph Laplacian matrix and its properties

This section uses graph concepts from Chapter 3. You may want to review that chapter, especially the definition of the adjacency matrix (Definition 3.6).

We begin with the definition of the graph Laplacian for an undirected graph. All graphs are assumed to be weighted with positive weights. For unweighted graphs, we treat each edge as having weight one.

Definition 42.48. (Degree and graph Laplacian matrices)

Let G be an undirected weighted graph.

(a) The **degree of vertex** i, denoted by d_i, is the sum of the weights of edges adjacent to vertex i:

$$d_i := \sum_{j=1}^{n} w_{ij}, \quad \text{for } i = 1, 2, \ldots, n.$$

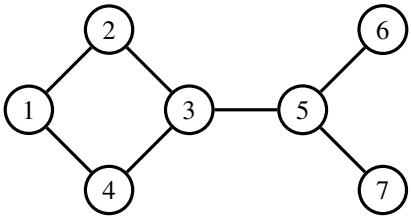
[3] This elementary proof of the uniqueness of the positive semidefinite square root is adapted from [34].

The **degree matrix** D of G is the $n \times n$ diagonal matrix $D := \operatorname{diag}(d_1, d_2, \ldots, d_n)$.

(b) The **graph Laplacian matrix** L of G is the matrix

$$L := D - A.$$

Example 42.49 Consider the following undirected graph.



Since the graph is unweighted, we assume that every edge has a weight of 1. The degree matrix D, adjacency matrix A, and graph Laplacian matrix $L = D - A$ for this graph are, respectively,

$$D = \begin{bmatrix} 2&0&0&0&0&0&0\\ 0&2&0&0&0&0&0\\ 0&0&3&0&0&0&0\\ 0&0&0&2&0&0&0\\ 0&0&0&0&3&0&0\\ 0&0&0&0&0&1&0\\ 0&0&0&0&0&0&1 \end{bmatrix} \quad A = \begin{bmatrix} 0&1&0&1&0&0&0\\ 1&0&1&0&0&0&0\\ 0&1&0&1&1&0&0\\ 1&0&1&0&0&0&0\\ 0&0&1&0&0&1&1\\ 0&0&0&0&1&0&0\\ 0&0&0&0&1&0&0 \end{bmatrix}, \quad \text{and} \quad L = \begin{bmatrix} 2&-1&0&-1&0&0&0\\ -1&2&-1&0&0&0&0\\ 0&-1&3&-1&-1&0&0\\ -1&0&-1&2&0&0&0\\ 0&0&-1&0&3&-1&-1\\ 0&0&0&0&-1&1&0\\ 0&0&0&0&-1&0&1 \end{bmatrix}.$$

The following theorem states the main properties of the graph Laplacian matrix that we will use in Chapter 44.

Theorem 42.50. (Properties of the graph Laplacian matrix)

The graph Laplacian matrix $L \in \mathbb{R}^{n \times n}$ of an undirected graph G satisfies the following properties:

(a) For any vector $x = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}^T$, we have

$$x^T L x = \frac{1}{2} \sum_{i,j=1}^{n} w_{ij}(x_i - x_j)^2. \tag{42.7}$$

(b) L is symmetric positive semidefinite.

(c) L is a singular matrix with the vector of all ones $e = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}^T$ as an eigenvector corresponding to the eigenvalue 0.

Proof Let d_i denote the degree of the ith vertex of the graph, and let A and D be the adjacency matrix and the degree matrix of G, respectively. For an arbitrary vector $x \in \mathbb{R}^n$, we have

$$x^T Lx = x^T (D - A)x = x^T Dx - x^T Ax.$$

Expanding each term, we get

$$x^T Dx = \sum_{i=1}^{n} d_i x_i^2, \quad \text{and} \quad x^T Ax = \sum_{i,j=1}^{n} w_{ij} x_i x_j.$$

Substituting these into the expression for $x^T Lx$, we obtain

$$\begin{aligned}
x^T Lx &= \sum_{i=1}^{n} d_i x_i^2 - \sum_{i,j=1}^{n} w_{ij} x_i x_j \\
&= \frac{1}{2}\left(\sum_{i=1}^{n} d_i x_i^2 - 2 \sum_{i,j=1}^{n} w_{ij} x_i x_j + \sum_{j=1}^{n} d_j x_j^2 \right) \\
&= \frac{1}{2}\left(\sum_{i,j=1}^{n} w_{ij} x_i^2 - 2 \sum_{i,j=1}^{n} w_{ij} x_i x_j + \sum_{i,j=1}^{n} w_{ij} x_j^2 \right) \\
&= \frac{1}{2} \sum_{i,j=1}^{n} w_{ij} \left(x_i^2 - 2x_i x_j + x_j^2 \right) \\
&= \frac{1}{2} \sum_{i,j=1}^{n} w_{ij} (x_i - x_j)^2.
\end{aligned}$$

To show that L is positive semidefinite, we need to prove that $x^T Lx \geqslant 0$ for any $x \in \mathbb{R}^n$. From part (a), $x^T Lx = \frac{1}{2} \sum_{i,j=1}^{n} w_{ij} (x_i - x_j)^2$. Since $w_{ij} \geqslant 0$ for all i, j, and $(x_i - x_j)^2$ is nonnegative, each term in the sum is nonnegative. Thus, $x^T Lx$ is a sum of nonnegative terms, and $x^T Lx \geqslant 0$ for all $x \in \mathbb{R}^n$.

Finally, observe that $De = Ae = \begin{bmatrix} d_1 & d_2 & \cdots & d_n \end{bmatrix}^T$, where e is the vector of all ones. Thus,

$$Le = (D - A)e = 0,$$

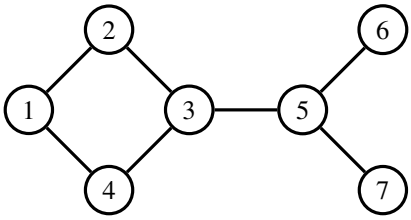
which shows that e is an eigenvector of L corresponding to the eigenvalue 0. Since L has a zero eigenvalue, it is singular. □

In the previous theorem, we saw that zero is always an eigenvalue of the graph Laplacian L. Since L is symmetric, it is diagonalizable, and the algebraic and geometric multiplicities of the zero eigenvalue are equal. Theorem 42.53 gives us the exact value of these multiplicities. First, we need some definitions.

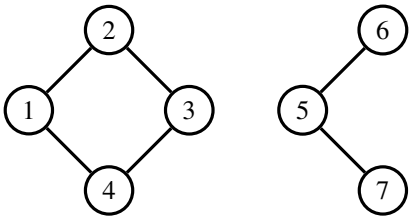
Definition 42.51. (Connected graph and connected components)

A graph G is **connected** if there exists a path between every pair of vertices in G. If G is not connected, a **connected component** of G is a maximal subgraph of G that is itself connected. In other words, a connected component is a subgraph that cannot include additional vertices from G without losing connectivity.

Example 42.52 The graph below is connected, so it has only one connected component.



The graph below is not connected and has two connected components.



Theorem 42.53. (Number of connected components)

For the graph Laplacian matrix $L \in \mathbb{R}^{n\times n}$ of an undirected graph G, the algebraic and geometric multiplicities of the eigenvalue zero are both equal to the number of connected components in G.

Proof Let k be the number of connected components of $G = (V, E)$, and let $G_1 = (V_1, E_1), G_2 = (V_2, E_2), \ldots, G_k = (V_k, E_k)$ be its connected components. Denote by s_i the number of vertices in G_i for $i = 1, \ldots, k$. Let L denote the graph Laplacian of G.

We first show that the geometric multiplicity of the zero eigenvalue of L is at least k by constructing k orthogonal eigenvectors corresponding to the eigenvalue 0.

After reordering the vertices of G so that the first s_1 vertices correspond to G_1, the next s_2 vertices correspond to G_2, and so on, the Laplacian matrix L takes the block diagonal form:

$$L = \begin{bmatrix} L_1 & & & \\ & L_2 & & \\ & & \ddots & \\ & & & L_k \end{bmatrix},$$

where $L_i \in \mathbb{R}^{s_i \times s_i}$ is the graph Laplacian of G_i for $i = 1, \ldots, k$.

Define the vector

$$q^{(i)} = \begin{bmatrix} 0_{s_1} \\ 0_{s_2} \\ \vdots \\ \mathbf{1}_{s_i} \\ \vdots \\ 0_{s_k} \end{bmatrix},$$

where $\mathbf{1}_{s_i}$ is a vector of length s_i with all entries equal to 1. The vectors $\{q^{(1)}, q^{(2)}, \ldots, q^{(k)}\}$ are orthogonal because $(q^{(i)})^T q^{(j)} = 0$ for $i \neq j$. From part (c) of Theorem 42.50, we know

$$Lq^{(i)} = \begin{bmatrix} 0_{s_1} \\ \vdots \\ L_i \mathbf{1}_{s_i} \\ \vdots \\ 0_{s_k} \end{bmatrix} = 0, \qquad \text{for each } i = 1, \ldots, k.$$

Thus, we have constructed k orthogonal (and therefore, linearly independent) eigenvectors of L corresponding to the eigenvalue 0. This shows that the geometric multiplicity of the eigenvalue 0 is at least k.

To complete the proof, we must show that any eigenvector u in the kernel of L lies in $\text{span}(q^{(1)}, q^{(2)}, \ldots, q^{(k)})$. Since $Lu = 0$, we have $u^T Lu = 0$. Using part (a) of Theorem 42.50 and the fact that G has k connected components, we get

$$0 = 2u^T Lu = \sum_{i,j=1}^{n} w_{ij}(u_i - u_j)^2 = 2 \sum_{\ell=1}^{k} \sum_{\{i,j\} \in E_\ell} w_{ij}(u_i - u_j)^2.$$

Since this is a sum of nonnegative terms that equals zero, each term must be zero. This implies $u_i = u_j$ for all $(i, j) \in E_\ell$, for $\ell = 1, \ldots, k$. Thus, u is constant within each connected component of G, and therefore we can write u as $u = \sum_{i=1}^{k} \alpha_i q^{(i)}$, for some scalars α_i, as we wanted to show.

Since L is symmetric, it is diagonalizable, and its geometric and algebraic multiplicities are equal. Therefore, the algebraic multiplicity of the eigenvalue 0 is k, completing the proof. □

There are several variations of the graph Laplacian matrix used in different contexts.

Definition 42.54. (Normalized graph Laplacian matrices)

Let D, A and L be, respectively, the degree, adjacency and graph Laplacian matrices of an undirected graph G.

(a) The **left normalized Laplacian**, also known as the random walk Laplacian, is defined as

$$L_{\text{left}} := D^{-1}L = D^{-1}(D - A) = I_n - D^{-1}A. \tag{42.8}$$

(b) The **right normalized Laplacian** is defined as

$$L_{\text{right}} := LD^{-1} = (D - A)D^{-1} = I_n - AD^{-1}. \tag{42.9}$$

(c) The **symmetrically normalized Laplacian** is defined as

$$L_{\text{sym}} := D^{-1/2}LD^{-1/2} = I - D^{-1/2}AD^{-1/2}, \tag{42.10}$$

where $D^{-1/2}$ denotes the inverse of $D^{1/2} = \text{diag}(d_1^{1/2}, \ldots, d_n^{1/2})$.

Of the three, the right normalized Laplacian is less commonly used.

Note that while the adjacency matrix A is symmetric, the left and right normalized Laplacian matrices usually are not. In contrast, the symmetrically normalized Laplacian is always symmetric.

The definitions of the three normalized Laplacian matrices assume that the degree matrix D is invertible, that is, the graph has no isolated vertices (vertices with degree zero). If the graph does include isolated vertices, making D singular, a common workaround is to replace the inverse of D with its pseudoinverse[4], or simply ignore the isolated vertices of the graph.

The next theorem summarizes the main properties of the left and symmetrically normalized graph Laplacians L_{left} and L_{sym}. The proofs are similar to those for the standard Laplacian in Theorem 42.50, so we skip them here. You can prove these results yourself in Problem 42.59.

Theorem 42.55. (Properties of the normalized graph Laplacian matrices)

Let L_{left} and L_{sym} be the left and symmetrically normalized graph Laplacian matrices of an undirected graph G. These matrices satisfy the following properties:

(a) For all vectors $x \in \mathbb{R}^n$, we have

$$x^T L_{\text{sym}} x = \frac{1}{2}\sum_{i,j=1}^{n} w_{ij}\left(\frac{x_i}{\sqrt{d_i}} - \frac{x_j}{\sqrt{d_j}}\right)^2. \tag{42.11}$$

(b) A vector v is an eigenvector of L_{left} with eigenvalue λ if and only if $D^{1/2}v$ is an eigenvector of L_{sym} with eigenvalue λ.

[4] We will introduce the pseudoinverse in Chapter 47.

(c) A vector v is an eigenvector of L_{left} with eigenvalue λ if and only if λ and v satisfy the generalized eigenvalue problem $Lv = \lambda Dv$, where L is the graph Laplacian matrix of G.
(d) L_{sym} is symmetric positive semidefinite.
(e) The vector e consisting of all ones is an eigenvector of L_{left} with eigenvalue $\lambda = 0$.
(f) L_{left} and L_{sym} have n non-negative eigenvalues $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_{n-1} \geqslant \lambda_n = 0$.

Like the graph Laplacian matrix, the eigenvalue $\lambda = 0$ for both the left and the symmetrically normalized Laplacian matrices, L_{left} and L_{sym}, has a multiplicity equal to the number of connected components in the graph G.

42.3.2 The Hessian matrix: minimization and convexity

Consider a function $f : \mathbb{R}^n \to \mathbb{R}$ that is twice differentiable with continuous second derivatives. We will refer to such functions as **smooth**. A point $x = a \in \mathbb{R}^n$ is a **critical point** of f if the gradient

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x) \\ \frac{\partial f}{\partial x_2}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{bmatrix}$$

vanishes at $x = a$, i.e., $\nabla f(a) = 0$. Here, $\frac{\partial f}{\partial x_i}$ denotes the partial derivative of f with respect to the variable x_i.

Example 42.56 Consider the function $f(x) = x_1^2 + x_2^2$. The gradient of f is

$$\nabla f(x) = \begin{bmatrix} 2x_1 \\ 2x_2 \end{bmatrix}.$$

It is clear that $x = \left[\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}\right]$ is a critical point of f because $\nabla f\left(\left[\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}\right]\right) = 0$.

An important question is determining the type of critical point a smooth function f has. Is it a local minimum, a local maximum, or a saddle point? Figure 42.5 illustrates different behaviors near a critical point of a function $f : \mathbb{R}^2 \to \mathbb{R}$: the first figure shows a local minimum, the second a local maximum, the third a saddle point, and the fourth a valley with infinitely many local minima.

To understand the behavior of a function f near a critical point, we need to analyze the function in a small neighborhood around it. For this, we use Taylor's theorem for functions of several variables, which lets us approximate a smooth function $f(x)$ as follows:

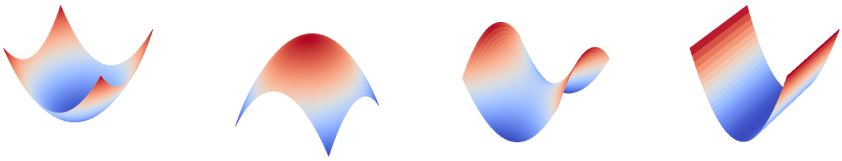


Fig. 42.5: Illustration of the behavior of a function $f : \mathbb{R}^2 \to \mathbb{R}$ near a critical point.

$$f(x) \approx f(a) + \nabla f(a)^T (x - a) + \frac{1}{2}(x - a)^T H(f, a)(x - a), \tag{42.12}$$

for points x near $x = a$. The matrix $H(f, a)$, called the **Hessian matrix** of f at $x = a$, is an $n \times n$ matrix containing all the second derivatives of f evaluated at the point $x = a$, that is,

$$H(f, a) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2}(a) & \frac{\partial^2 f}{\partial x_1 \partial x_2}(a) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(a) \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(a) & \frac{\partial^2 f}{\partial x_2^2}(a) & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n}(a) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(a) & \frac{\partial^2 f}{\partial x_n \partial x_2}(a) & \cdots & \frac{\partial^2 f}{\partial x_n^2}(a) \end{bmatrix}.$$

A notable result is that the Hessian matrix is symmetric since the mixed partial derivatives commute:

$$\frac{\partial^2 f}{\partial x_i \partial x_j}(a) = \frac{\partial^2 f}{\partial x_j \partial x_i}(a),$$

assuming the second partial derivatives exist and are continuous near $x = a$.

Let a be a critical point of f, so that $\nabla f(a) = 0$. Since f has continuous second derivatives, Taylor's theorem gives

$$f(a + h) = f(a) + \frac{1}{2} h^T H(f, a) h + r(h),$$

where

$$\frac{r(h)}{\|h\|_2^2} \longrightarrow 0 \qquad \text{as } h \to 0.$$

We can now classify the critical point using the definiteness of the Hessian.

1. If $H(f, a)$ is positive definite, let $m = \lambda_{\min}(H(f, a)) > 0$. Then, by Rayleigh-Ritz theorem,
$$h^T H(f, a) h \geqslant m \|h\|_2^2.$$
Since $r(h) = o(\|h\|_2^2)$, for all sufficiently small nonzero h,
$$f(a + h) - f(a) \geqslant \frac{m}{4} \|h\|_2^2 > 0.$$

Therefore, a is a **strict local minimum.**

2. If $H(f,a)$ is negative definite, an analogous argument shows that $f(a+h) - f(a) < 0$ for every sufficiently small nonzero h. Therefore, a is a **strict local maximum.**
3. If $H(f,a)$ is indefinite, there are unit vectors u and v such that $u^T H(f,a)u > 0$ and $v^T H(f,a)v < 0$. Consequently,

$$f(a+tu) - f(a) > 0 \quad \text{and} \quad f(a+tv) - f(a) < 0$$

for all sufficiently small nonzero t. Hence, a is a **saddle point.**
4. If $H(f,a)$ is positive semidefinite or negative semidefinite, the second-derivative test is inconclusive, and additional analysis is required (see Problem 42.63).

Example 42.57 Consider again the function $f(x) = x_1^2 + x_2^2$. The Hessian of f is

$$H(f,x) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix},$$

for all $x \in \mathbb{R}^2$. The point $a = \left[\begin{smallmatrix} 0 \\ 0 \end{smallmatrix}\right]$ is a critical point of f, and $H(f,a)$ is positive definite. Thus, a is a local minimum of f. In fact, since $f(x) = x_1^2 + x_2^2$ is a simple quadratic function with no other critical points, a is also the global minimum of f.

42.3.3 The covariance matrix

In this section, we introduce the covariance matrix, an important matrix in Statistics and Data Science. We will show that the covariance matrix is always positive semidefinite.

Boldface notation for vectors

In this section, we use bold letters, like $\mathbf{x}$, to represent vectors. We do this to distinguish them from scalars and avoid notational confusion. We only use this boldface notation in this section of the book.

When working with data in $\mathbb{R}^n$, it is helpful to understand the general shape or spread of the data points. A natural way to measure this spread is using the variance, which we introduce first.

Definition 42.58. (Variance)

The **variance** of a vector $\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}^T \in \mathbb{R}^n$ is given by

$$\operatorname{var}(\mathbf{x}) = \frac{\sum_{i=1}^n (x_i - \overline{x})^2}{n-1}, \quad \text{where} \quad \overline{x} = \frac{\sum_{i=1}^n x_i}{n}.$$

Roughly speaking, the square root of the variance of $\mathbf{x}$ measures how spread out or variable the entries $x_1, x_2, \ldots, x_n$ are around their mean value $\bar{x}$. If $\text{var}(\mathbf{x}) = 0$, all entries in $\mathbf{x}$ are identical. A larger variance indicates greater spread or variability among the entries.

Example 42.59 Consider the vector $\mathbf{x} = \begin{bmatrix} 1 & -1 & 1/2 & 3 & -1/2 & 3/2 \end{bmatrix}^T$. The mean is

$$\bar{x} = \frac{1 - 1 + 1/2 + 3 - 1/2 + 3/2}{6} = \frac{3}{4}.$$

The variance of $\mathbf{x}$ is

$$\text{var}(x) = \frac{1}{5}\left[(1 - 3/4)^2 + (-1 - 3/4)^2 + (1/2 - 3/4)^2 \right.$$
$$\left. +(3 - 3/4)^2 + (-1/2 - 3/4)^2 + (3/2 - 3/4)^2\right] = \frac{83}{40}.$$

Example 42.60 In Figure 42.6, we show a scatter plot of $n = 200$ data points in $\mathbb{R}^2$: $\begin{bmatrix} x_i \\ y_i \end{bmatrix}$ for $i = 1, \ldots, n$. Let $\mathbf{x} = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}^T$ and $\mathbf{y} = \begin{bmatrix} y_1 & \cdots & y_n \end{bmatrix}^T$. The variances of the two coordinates are approximately $\text{var}(\mathbf{x}) \approx 3.5$ and $\text{var}(\mathbf{y}) \approx 0.95$. As a general rule, three times the square root of the variance for each component typically covers most of the data points.

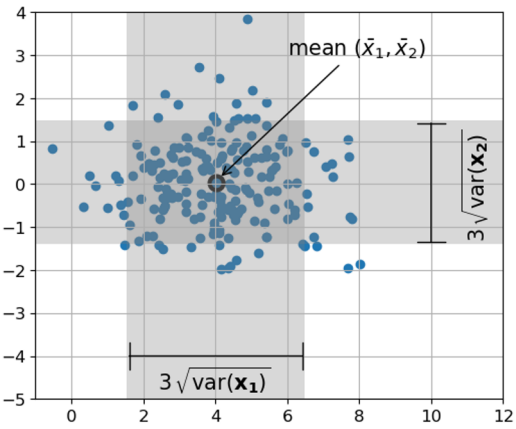


Fig. 42.6: Scatter plot of a two-dimensional dataset. The variances of $\mathbf{x_1}$ and $\mathbf{x_2}$ are approximately 3.5 and 0.95, with the spread roughly estimated by three times the square root of the variance.

Variances alone do not fully capture the data's "shape". Different datasets can have the same variances but very different patterns.

Example 42.61 Figure 42.7 shows three datasets that have the same means and variances in each coordinate but different overall shapes.

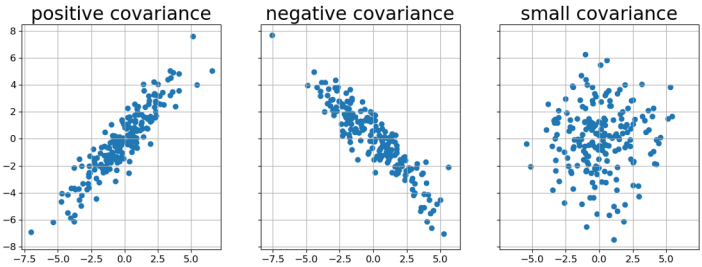


Fig. 42.7: Three datasets in $\mathbb{R}^2$ with identical means and variances but varying covariance.

To better understand the shape of the data, we need to consider the covariance between pairs of coordinates. Variance describes how one coordinate varies, while covariance describes how two coordinates vary together.

Definition 42.62. (Covariance)

Let $\mathbf{x} = \begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}^T$ and $\mathbf{y} = \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix}^T$ be two vectors in $\mathbb{R}^n$. The **covariance** of the vectors $\mathbf{x}$ and $\mathbf{y}$ is given by

$$\operatorname{cov}(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{n - 1},$$

where $\bar{x}$ and $\bar{y}$ are the means of $\mathbf{x}$ and $\mathbf{y}$, respectively.

Remark 42.63 The variance of a vector $\mathbf{x}$ equals the covariance of $\mathbf{x}$ with itself, i.e., $\operatorname{var}(\mathbf{x}) = \operatorname{cov}(\mathbf{x}, \mathbf{x})$.

Remark 42.64 Covariance is closely connected to Pearson's correlation coefficient r we introduced in Definition 12.58. We have

$$r = \frac{\operatorname{cov}(\mathbf{x}, \mathbf{y})}{\sqrt{\operatorname{var}(\mathbf{x}) \operatorname{var}(\mathbf{y})}}.$$

Example 42.65 Consider the three two-dimensional datasets in Table 42.1.

Dataset 1		Dataset 2		Dataset 3	
$\mathbf{x_1}$	$\mathbf{x_2}$	$\mathbf{x_1}$	$\mathbf{x_2}$	$\mathbf{x_1}$	$\mathbf{x_2}$
0.5	0.6	0.5	0.6	0.5	-0.62
-0.7	-1.2	-0.7	-0.8	-0.7	0.78
1.2	-1.0	1.2	1.3	1.2	-1.32
-1.1	-0.4	-1.1	-1.2	-1.1	1.18
0.2	1.3	0.2	0.3	0.2	-0.32
-0.5	-1.5	-0.5	-0.4	-0.5	0.38
0.9	-0.8	0.9	1.0	0.9	-1.02
-1.3	1.0	-1.3	-1.5	-1.3	1.48
1.5	0.3	1.5	1.6	1.5	-1.62
-0.9	1.6	-0.9	-1.0	-0.9	0.98

Table 42.1: Three datasets with identical means and variances but different covariances.

A quick calculation shows they have the same means, $\overline{\mathbf{x_1}} = -0.02$ and $\overline{\mathbf{x_2}} = -0.01$, and the same variances, $\text{var}(\mathbf{x_1}) = 1.026$ and $\text{var}(\mathbf{x_2}) = 1.24$. However, Figure 42.8 reveals they are quite different. What sets them apart is the covariance between $\mathbf{x_1}$ and $\mathbf{x_2}$: $\text{cov}(\mathbf{x_1}, \mathbf{x_2}) = -0.18$ for the first dataset, 1.13 for the second, and -1.13 for the third.

The magnitude of the covariance reflects the strength of the linear relationship between the variables. Larger values (in either direction) indicate stronger relationships, while values closer to zero suggest weaker ones. A positive covariance means the variables increase or decrease together. A negative covariance indicates that when one variable is high, the other is low.

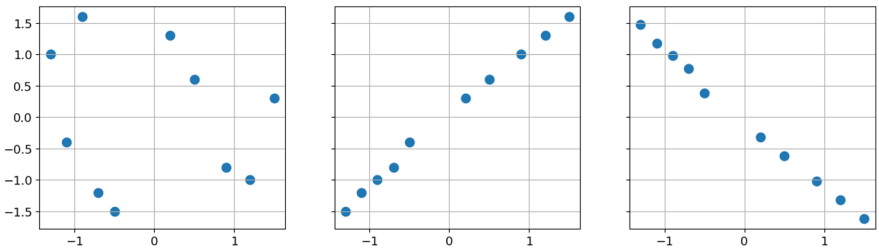


Fig. 42.8: Scatter plots of three datasets with identical means and variances but different covariances.

We are now ready to define the covariance matrix.

Definition 42.66. (Covariance Matrix)

Let $X = \begin{bmatrix} \mathbf{x_1} & \mathbf{x_2} & \cdots & \mathbf{x_d} \end{bmatrix} \in \mathbb{R}^{m \times d}$ be a matrix containing m data points (rows) with d features $\mathbf{x_1}, \mathbf{x_2}, \ldots, \mathbf{x_d}$ (columns). The **covariance matrix** C is a $d \times d$ symmetric matrix, whose (i, j)th entry is given by

$$C_{ij} = \text{cov}(\mathbf{x_i}, \mathbf{x_j}).$$

Alternatively, if $\overline{X} \in \mathbb{R}^{m \times d}$ is the matrix whose ith column is a vector of size m filled with the mean $\overline{x}_i$ of $\mathbf{x_i}$, the covariance matrix can be written as

$$C = \frac{1}{m-1}(X - \overline{X})^T (X - \overline{X}). \tag{42.13}$$

The diagonal entries of the covariance matrix correspond to the variances of the individual features, while the off-diagonal entries represent the covariances between pairs of distinct features.

Example 42.67 Consider m data points in $\mathbb{R}^2$: $\begin{bmatrix} x_i \\ y_i \end{bmatrix}$ for $i = 1, \ldots, m$. Let $\mathbf{x} = \begin{bmatrix} x_1 & \cdots & x_m \end{bmatrix}^T$ and $\mathbf{y} = \begin{bmatrix} y_1 & \cdots & y_m \end{bmatrix}^T$. The covariance matrix for $\mathbf{x}$ and $\mathbf{y}$ is given by

$$C = \begin{bmatrix} \text{var}(\mathbf{x}) & \text{cov}(\mathbf{x}, \mathbf{y}) \\ \text{cov}(\mathbf{x}, \mathbf{y}) & \text{var}(\mathbf{y}) \end{bmatrix}.$$

The diagonal entries are the variances of $\mathbf{x}$ and $\mathbf{y}$, while the off-diagonal entries are their covariance.

Example 42.68 Consider a dataset of $m = 5$ data points in $\mathbb{R}^3$, where each row of the matrix X represents a single data point. Each row represents the length (ℓ), width (w), and height (h) of a certain type of object:

$$X = \begin{bmatrix} 4 & 2 & 0.6 \\ 4.2 & 2.1 & 0.59 \\ 3.9 & 2 & 0.58 \\ 4.3 & 2.1 & 0.62 \\ 4.1 & 2.2 & 0.63 \end{bmatrix}.$$

To summarize this dataset, we calculate the mean vector $\begin{bmatrix} \overline{\ell} & \overline{w} & \overline{h} \end{bmatrix}^T$ and the covariance matrix. The means are

$$\overline{\ell} = \frac{\sum_{i=1}^5 \ell_i}{5} = 4.10, \quad \overline{w} = \frac{\sum_{i=1}^5 w_i}{5} = 2.08, \quad \overline{h} = \frac{\sum_{i=1}^5 h_i}{5} = 0.604.$$

The mean matrix is

$$\bar{X} = \begin{bmatrix} 4.1 & 2.08 & 0.604 \\ 4.1 & 2.08 & 0.604 \\ 4.1 & 2.08 & 0.604 \\ 4.1 & 2.08 & 0.604 \\ 4.1 & 2.08 & 0.604 \end{bmatrix}.$$

The covariance matrix is

$$C = \frac{1}{4}(X - \bar{X})^T (X - \bar{X}) = \begin{bmatrix} 0.025 & 0.0075 & 0.00175 \\ 0.0075 & 0.0070 & 0.00135 \\ 0.00175 & 0.00135 & 0.00043 \end{bmatrix}.$$

Note that C is symmetric, as expected for a covariance matrix.

The following theorem shows that the covariance matrix of any dataset is always symmetric and positive semidefinite.

Theorem 42.69. (Positive semidefiniteness of covariance matrices)

Let $X \in \mathbb{R}^{m \times d}$ be a matrix containing m data points associated with d features, and let C be its covariance matrix. The covariance matrix C is symmetric positive semidefinite.

Proof The covariance matrix C is symmetric because $\text{cov}(\mathbf{x}, \mathbf{y}) = \text{cov}(\mathbf{y}, \mathbf{x})$ for all vectors $\mathbf{x}$ and $\mathbf{y}$.

Additionally, from (42.13), C is a Gram matrix. By Theorem 42.38, all Gram matrices are positive semidefinite, so C is positive semidefinite as well. □

Problems

42.1 Let S and T be two $n \times n$ symmetric matrices. If $x^T Sx = x^T Tx$ for every $x \in \mathbb{R}^n$, prove that $S = T$.

42.2 Verify whether the following expressions are quadratic forms and represent them in the form $f(x) = x^T Sx$, where S is a symmetric matrix.

(a) $\|Ax\|_2^2$, where $A \in \mathbb{R}^{n \times n}$.
(b) $\sum_{i=1}^{n-1} (x_{i+1} - x_i)^2$.
(c) $\|Mx\|_2^2 - \|Nx\|_2^2$, where $M, N \in \mathbb{R}^{n \times n}$.

42.3 Use the method of "completing the squares" to check whether the following quadratic forms are positive definite or not.

(a) $2x_1^2 + 2x_1x_2 + 2x_2^2$.
(b) $x_1^2 - 6x_1x_2 + 4x_1x_3 + 8x_2^2 - 3x_3^2 - 6x_2x_3$.
(c) $x_1^2 + 6x_2^2 + 8x_3^2 - 4x_1x_2 - 6x_1x_3 - x_2x_3$.

42.4 Use the method of "completing the square" to show that the quadratic form $x^T Sx$, associated with the 2×2 symmetric matrix $S = \left[\begin{smallmatrix} a & c \\ c & d \end{smallmatrix}\right]$, is positive definite precisely when $a > 0$ and $\det(S) = ad - c^2 > 0$.

42.5 Consider a 3×3 symmetric matrix

$$S = \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}.$$

By completing the squares, write the quadratic form $x^T Sx$ as the sum of three squares. When is this quadratic form positive definite?

42.6 Determine the change of basis that diagonalizes the given quadratic forms and express the forms in terms of the new variables.

(a) $x_1^2 + 4x_1x_2 + x_2^2$.
(b) $x_1^2 - 2x_1x_2 + 2x_2x_3 + x_3^2$.
(c) $7x_1^2 + x_2^2 + x_3^2 + 8x_1x_2 + 8x_1x_3 - 16x_2x_3$.

42.7 Consider the quadratic form $f(x) = ax_1^2 + bx_1x_2 + cx_2^2$, where a, b and c are nonzero constant. Show that there is a counterclockwise rotation of the coordinate axes about the origin such that in the new coordinate system $f(x)$ has not cross terms.

42.8 Complete the proof of Theorem 42.11.

42.9 Let $S \in \mathbb{R}^{n \times n}$ be a symmetric matrix and let $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_n$ be the eigenvalues of S. Show that

(a) $\lambda_n = \min_{\|x\|_2 = 1} x^T Sx$.
(b) For $1 < k < n$,

$$\lambda_k = \min_{\substack{\|x\|_2 = 1 \\ x \perp q_n, \ldots, q_{k+1}}} x^T Sx.$$

42.10 Show that the following optimization problems are equivalent:

$$\begin{aligned} &\text{optimize} && x^T Sx \\ &\text{subjec to} && \|x\|_2 = 1, \end{aligned}$$

and

$$\text{optimize} \quad \frac{x^T Sx}{x^T x},$$

where "optimize" refers to either maximizing or minimizing.

42.11 Let S be a 3×3 symmetric matrix with eigenvalues $\lambda_1 > \lambda_2 > \lambda_3$ and corresponding orthonormal eigenvectors q_1, q_2, q_3. Consider the quadratic form

$$f(x) = x^T Sx,$$

where x lies on the unit sphere (i.e., $x^T x = 1$). We know that $f(x)$ attains its maximum value λ_1 at $\pm q_1$ and its minimum value λ_3 at $\pm q_3$. Show that $\pm q_2$ are saddle points of $f(x)$. In other words, show that in any neighborhood of $\pm q_2$, there exist directions along which $f(x)$ increases and directions along which $f(x)$ decreases.

42.12 What do positive semidefinite and positive definite means for 1×1 matrices?

42.13 Show that if A is positive definite so is A^T.

42.14 Show that the diagonal entries of a positive definite matrix are positive.

42.15 In Proposition 42.20, we established that the sum of two positive definite matrices is also positive definite. Now consider the following:

(a) What can you say about the sum of two positive semidefinite matrices?
(b) What can you say about the sum of a positive definite matrix and a positive semidefinite matrix?

42.16 If $A \in \mathbb{R}^{n \times n}$ is positive semidefinite, then show that $\alpha I_n + A$ is positive definite for any scalar $\alpha > 0$.

42.17 If A and B are positive definite, then what can you say about the positive definiteness of the following matrices:

(a) $A^T + A$
(b) AB
(c) A^2
(d) ABA
(e) $A^T BA$
(f) $A^T A$

42.18 If A and B are positive definite matrices, then show that the block diagonal matrix $M = \left[\begin{smallmatrix} A & 0 \\ 0 & B \end{smallmatrix}\right]$ is also positive definite.

42.19 Using the energy-based definition of positive definite matrices, prove the following statements:

(a) A positive definite matrix S is invertible.
(b) The inverse S^{-1} of a positive definite matrix S is also positive definite.

42.20 Let A be an $n \times n$ positive definite matrix, and consider the following $k \times k$ block matrix

$$M = \begin{bmatrix} 2A & A & \cdots & A \\ A & 2A & \ddots & \vdots \\ \vdots & \ddots & \ddots & A \\ A & \cdots & A & 2A \end{bmatrix}.$$

Show that M is positive definite.

Hint: If $x = \begin{bmatrix} x_1 & x_2 & \cdots & x_k \end{bmatrix}^T$, where $x_1, x_2, \ldots, x_k \in \mathbb{R}^n$ are arbitrary vectors, then show that $x^T M x = \sum_{i=1}^k x_i^T A x_i + \left(\sum_{i=1}^k x_i\right)^T A \left(\sum_{i=1}^k x_i\right)$.

42.21 Recall that the matrices AB and BA have the same nonzero eigenvalues (see Problem 36.17), is it true that if AB is positive definite, then BA is also positive definite?

42.22 Let S be an invertible and symmetric matrix. Show that S^2 is positive definite.

42.23 Show that a positive semidefinite matrix is positive definite if and only if it is invertible.

42.24 Let $x_1, x_2, \ldots, x_n$ be positive real numbers. Let A be the $n \times n$ symmetric matrix whose (i, j) entry is given by

$$a_{ij} = \frac{1}{x_i + x_j}.$$

This is called a **Cauchy matrix**. In this problem, you will show that a Cauchy matrix is positive semidefinite.

(a) Let $e \in \mathbb{R}^n$ denote the vector of all ones, and let $D = \operatorname{diag}(x_1, x_2, \ldots, x_n)$. Show that the Cauchy matrix A satisfies the equation

$$DA + AD = ee^T. \tag{42.14}$$

(b) Let λ be an eigenvalue of A with associated eigenvector x. Pre-multiply the equation (42.14) by x^T and post-multiply by x. Conclude that $\lambda \geqslant 0$.

42.25 Show that the powers A^k, for $k = 0, 1, 2, \ldots$, of a positive (semi)definite matrix A are positive (semi)definite.

42.26 Let A and B be two positive semidefinite matrices of the same size. Prove that AB is positive semidefinite if and only if AB is symmetric.

Hint: First, show that if AB is symmetric, then A and B must commute, i.e., $AB = BA$. Then, use the fact that commuting diagonalizable matrices can be simultaneously diagonalized: if A and B commute and each is diagonalizable, there is a single invertible matrix P such that both $P^{-1}AP$ and $P^{-1}BP$ are diagonal.

42.27 Prove that a positive definite matrix has positive determinant and positive trace.

42.28 Let A and B be two $n \times n$ symmetric positive semidefinite matrices. Prove that

$$\operatorname{trace}(AB) \geqslant 0.$$

Hint: Use the spectral decomposition of B and express the trace as a sum of products involving its eigenvectors.

42.29 Let $A \in \mathbb{R}^{n \times n}$ be a nonsingular symmetric matrix such that $A - I_n$ is positive definite. Prove that $I_n - A^{-1}$ is positive definite.

42.30 A matrix $A \in \mathbb{C}^{n\times n}$ is **diagonally dominant** if $|A_{ii}| \geqslant \sum_{j\neq i} |A_{ij}|$ for all $i \in \{1, 2, ..., n\}$. Show that a symmetric diagonally dominant matrix with nonnegative diagonal entries is positive semidefinite. What modification to diagonal dominance is needed to ensure positive definiteness?

Hint: Use Gershgorin's Circles Theorem 38.1.

42.31 Let $X \in \mathbb{R}^{n\times n}$. Show that $I_n - XX^T$ is positive definite if and only if $I_n - X^T X$ is positive definite.

Hint: Recall that XX^T and $X^T X$ has the same nonzero eigenvalues.

42.32 Show that $S = \left[\begin{smallmatrix} 3 & 4 \\ 4 & 5 \end{smallmatrix}\right]$ is not positive definite without computing its eigenvalues.

42.33 For what numbers b are the following matrix positive definite?

$$A = \begin{bmatrix} 2 & -1 & b \\ -1 & 2 & -1 \\ b & -1 & 2 \end{bmatrix} \qquad \text{and} \qquad B = \begin{bmatrix} 8 & -2 & b \\ -2 & 4 & -4 \\ b & -4 & 2 \end{bmatrix}.$$

42.34 Consider the following matrix

$$A = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix}.$$

Factor A as $A = LDL^T$, where L is unit lower triangular, and D is diagonal. Is A positive definite?

42.35 Consider the matrix

$$S = \begin{bmatrix} 2 & 1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}.$$

Use both the pivot characterization and the leading principal minor characterization to show that S is positive definite. Then, compare the values of the pivots of S with the quotients of consecutive leading principal minors. Is this just a coincidence?

42.36 Let S be a symmetric positive (semi)definite matrix, and suppose $S = B^T B = C^T C$, where both B and C are square matrices. Show that there exists an orthogonal matrix Q such that $C = QB$.

42.37 Let $S \in \mathbb{R}^{n\times n}$ be a positive semidefinite matrix, and let $B \in \mathbb{R}^{n\times m}$. Show that the matrix $T = B^T SB$ is positive semidefinite.

42.38 Show that a matrix of the form $B^T B + \alpha I_n$ is positive definite for any positive scalar α by finding a Gram factorization.

42.39 Let $S = B^T B$ be a positive definite matrix. Show that if $B = QR$ is the reduced QR factorization of the Gram factor B, then $L = R^T$ is the Cholesky factor of S.

42.40 Let A be a square matrix. Does A commute with all its square roots?

42.41 Show that any symmetric orthogonal matrix $\begin{bmatrix} \cos(\theta) & \sin(\theta) \\ \sin(\theta) & -\cos(\theta) \end{bmatrix}$ is a square root of the 2×2 identity matrix.

42.42 Consider the plane rotation matrix $Q = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$, with $0 \leqslant \theta < 2\pi$. Find the rotation matrix that is a square root of Q.

42.43 Let $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$. Prove that there is no matrix $B = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in \mathbb{R}^{2\times 2}$ such that $A = B^2$. Does A have a square root with complex-valued entries?

42.44 Find an explicit formula for a square root of a matrix of the form $\begin{bmatrix} a & 0 \\ b & c \end{bmatrix}$, where a, b, and c are real numbers.

42.45 Let A be an $n \times n$ diagonalizable matrix with n distinct eigenvalues $\lambda_1, \ldots, \lambda_n$. We know that we can write A as $A = VDV^{-1}$, where V is the matrix whose columns are eigenvectors of A, and D is the diagonal matrix whose diagonal elements are the corresponding n eigenvalues.

(a) Prove that B is a square root of A if and only if $B = SD_BS^{-1}$, where D_B is a square root of the diagonal matrix D.
(b) Show that D_B commutes with D. Deduce that D_B must be diagonal with diagonal elements equal to square roots of the diagonal elements of D.
(c) Since there are two possible choices for a square root of each diagonal element of D, conclude that A has at most 2^n square roots.

42.46 Find all the square roots of the matrix

$$A = \begin{bmatrix} 1 & 3 & -3 \\ 0 & 4 & 5 \\ 0 & 0 & 9 \end{bmatrix}.$$

42.47 Develop an algorithm to find a square root of an upper triangular matrix T. Then, explain how the Schur decomposition can be used to find a square root of any matrix A.

42.48 Explain how to compute the positive definite square root of a matrix of the form $A = I_n + vv^T$, where $v \in \mathbb{R}^n$ is a unit vector.

Hint: Find an orthogonal matrix that diagonalizes A.

42.49 Verify that the matrix $A = \begin{bmatrix} 41 & 12 \\ 12 & 34 \end{bmatrix}$ is positive definite, and find its unique positive definite square root.

42.50 Let A and B be positive definite matrices. Does $A^{1/2} = B^{1/2}$ imply that $A = B$?

42.51 Let A be a positive semidefinite matrix, and let $A^{1/2}$ be its positive semidefinite square root. Show that A and $A^{1/2}$ commute.

42.52 In this problem, you will derive a formula for a square root of a 2×2 matrix without explicitly computing its eigenvalues.

Let $A \in \mathbb{R}^{2\times 2}$ satisfy $\det(A) > 0$, and let B be a square root of A, so that $B^2 = A$. Recall from Problem 37.28 that B satisfies its characteristic polynomial:

$$B^2 - \operatorname{trace}(B)\, B + \det(B) I_2 = 0.$$

Since $A = B^2$, it follows that $\det(B)^2 = \det(A)$, so $\det(B) = \pm\sqrt{\det(A)}$. Let $r = \sqrt{\det(A)}$ denote the positive square root, and suppose that $\det(B) = r$. Then proceed as follows:

(a) Show that $\operatorname{trace}(B)B = A + r\, I_2$.
(b) Show that $\big(\operatorname{trace}(B)\big)^2 = \operatorname{trace}\big(\operatorname{trace}(B)\, B\big) = \operatorname{trace}(A) + 2r$.
(c) Provided that $\operatorname{trace}(A) + 2r > 0$, conclude that

$$B = \pm \frac{A + rI_2}{\sqrt{\operatorname{trace}(A) + 2r}}. \tag{42.15}$$

Show directly that both choices are square roots of A.
(d) Determine whether formula (42.15) applies when A is positive definite. Which choice of sign gives the unique positive definite square root of A?
(e) Repeat parts (a)–(c) under the assumption that $\det(B) = -r$. Show that, provided $\operatorname{trace}(A) - 2r > 0$,

$$B = \pm \frac{A - rI_2}{\sqrt{\operatorname{trace}(A) - 2r}}.$$

(f) If A is positive definite with distinct eigenvalues, interpret the four real square roots obtained from the two formulas in terms of the signs chosen for the square roots of the eigenvalues of A.

42.53 For a symmetric positive definite matrix $S \in \mathbb{R}^{n\times n}$, prove $(S^{1/2})^{-1} = (S^{-1})^{1/2}$.

Hint: Use the Spectral Theorem (Theorem 41.3).

42.54 If A and B are two positive definite matrices of the same size, then show that AB is similar to a positive definite matrix.

Hint: Write $A = B^2$, where B is the positive definite square root of A.

42.55 In this problem, the goal is to show that given a set of linearly independent vectors, it is possible to construct an orthonormal basis for the space they span by using the square root of their Gram matrix.

Consider a set of linearly independent vectors $b_1, b_2, \ldots, b_m$, with a Gram matrix $S = [S_{ij}] = [b_i^T b_j]$. Let $B = \begin{bmatrix} b_1 \cdots b_m \end{bmatrix}$ be the matrix formed by concatenating these vectors as columns. Show that the matrix $W = BS^{-1/2}$ has orthonormal columns and $\operatorname{col}(W) = \operatorname{col}(B)$, where $S^{-1/2}$ denotes the inverse of the positive definite square root of S.

42.56 If $S \in \mathbb{R}^{n\times n}$ is a positive definite matrix, and if $S = B^T B$ with $B \in \mathbb{R}^{n\times n}$, then show that $B = VS^{1/2}$, where V is an orthogonal matrix. If $S = B^T B = C^T C$ are two

Gram factorizations of S with $B, C \in \mathbb{R}^{n\times n}$, then conclude that $C = PB$ for some orthogonal matrix P.

Hint: Write B as $B = (BS^{-1/2})S^{1/2}$.

42.57 Consider the $n \times n$ matrix

$$A = \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & \ddots & & \\ & \ddots & \ddots & -1 & \\ & & -1 & 2 & \end{bmatrix}.$$

In this problem, you will prove that A is positive definite using three different approaches.

(a) Define the matrix B as an $n \times (n+1)$ matrix with entries on the main diagonal equal to 1, entries on the lower diagonal equal to -1, and all other entries equal to 0:

$$B = \begin{bmatrix} 1 & & & \\ -1 & 1 & & \\ & -1 & \ddots & \\ & & \ddots & 1 \\ & & & -1 \end{bmatrix}.$$

Prove that $A = B^T B$ and conclude that A is positive definite.

(b) Show that A is diagonally dominant. Use this result to prove that A is positive semidefinite (refer to Problem 42.30). Next, determine the null space of A and conclude that A is nonsingular. Finally, conclude that A is positive definite.

(c) Let $\det(A_k)$ denote the determinant of the $k \times k$ leading principal minor of A. Show that

$$\det(A_k) = 2\det(A_{k-1}) - \det(A_{k-2}) \qquad \text{for k} = 3, 4, \ldots, n.$$

Use this result to establish, possibly by induction, that $\det(A_k) = k + 1$ for $k = 1, \ldots, n$. Finally, conclude that A is positive definite.

42.58 In Chapter 6, we introduced the *standard dot product*dot product $x^T y$ between vectors $x, y \in \mathbb{R}^n$. Similar to the axiomatic definition of a vector norm, there is an analogous axiomatic definition of a dot product.

A **dot product** (or **inner product**) $\langle \cdot, \cdot \rangle$ on $\mathbb{R}^n$ satisfies the following properties for all $x, y, z \in \mathbb{R}^n$ and $c \in \mathbb{R}$:

i) **Linearity:** $\langle x + y, z\rangle = \langle x, z\rangle + \langle y, z\rangle$ and $\langle cx, y\rangle = c\langle x, y\rangle$,
(ii) **Symmetry:** $\langle x, y\rangle = \langle y, x\rangle$,
(iii) **Positive definiteness:** $\langle x, x\rangle > 0$ for $x \neq 0$ and $\langle x, x\rangle = 0$ if $x = 0$.

Show that $\langle \cdot, \cdot \rangle$ is a dot product on $\mathbb{R}^n$ if and only if there exists a symmetric positive definite matrix A such that

$$\langle x, y\rangle = y^T Ax \quad \text{for all } x, y \in \mathbb{R}^n.$$

Finally, interpret what it means if $A = B^T B$ for some matrix B in terms of the dot product.

42.59 Prove Theorem 42.55.

42.60 Let L be the Laplacian of a connected graph with some nodes labeled and the rest unlabeled. Reorder the nodes so that the labeled ones come first and the unlabeled ones second. In this ordering the Laplacian has the block form

$$L = \begin{bmatrix} L_{LL} & L_{LU} \\ L_{UL} & L_{UU} \end{bmatrix}.$$

Prove that the submatrix L_{UU} is symmetric positive definite if every unlabeled node is connected by a path to at least one labeled node.

42.61 Find the gradient and Hessian matrix of the following functions $f : \mathbb{R}^n \to \mathbb{R}$.

(a) $f(x) = x^T b$, where $x, b \in \mathbb{R}^n$.
(b) $f(x) = \|x\|_2^2$, where $x \in \mathbb{R}^n$.
(c) $f(x) = x^T Ax$, where $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$.
(d) $f(x) = x^T Ax + x^T b + c$, where $c \in \mathbb{R}$, $x, b \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$.

42.62 Find the critical points of the following functions and determine whether they are local minima, maxima, or saddle points.

(a) $f(x) = x_1^3 + 6x_1^2 + 3x_2^2 - 12x_1x_2 + 9x_1$.
(b) $f(x) = \cos(2x_1)\sin(x_2) + x_3^2$.

42.63 For each of the following functions, verify that $(0, 0)$ is a critical point and that the Hessian at $(0, 0)$ is positive semidefinite. Then determine whether the critical point is a local maximum, local minimum, or saddle point.

(a) $f(x) = x_1^2$
(b) $f(x) = x_1^2 - x_2^4$

Chapter 43
IX-SUPPLEMENTARY TOPICS

43.1 Geometric insights on positive definite quadratic forms: The Principal Axes Theorem

Positive definite quadratic forms have an elegant geometric interpretation in terms of ellipses (in two dimensions) or ellipsoids (in higher dimensions), a connection we establish in this section.

First, let us revisit the concept of an ellipse.

An **ellipse** is essentially a stretched or flattened circle (see Figure 43.1 for two examples). More formally, an ellipse is the set of points $x \in \mathbb{R}^2$ for which the sum of the distances to two fixed points, p_1 and p_2, remains constant:

$$d(x, p_1) + d(x, p_2) = \text{constant},$$

where $d(x, p) = \|x - p\|_2$ is the Euclidean distance between x and p. These fixed points are called the **foci**. Notice that a circle is a special case of an ellipse where $p_1 = p_2$.

An **ellipsoid** is simply the higher-dimensional analogue of an ellipse.

Every ellipse centered at the origin with its foci on the x_1-axis can be described by the equation

$$\frac{x_1^2}{a^2} + \frac{x_2^2}{b^2} = 1,$$

where $2a$ is the width and $2b$ is the height of the ellipse. The values a and b are called the lengths of the semi-major and semi-minor axes.

Example 43.1 Consider the equation

$$\frac{x_1^2}{9} + \frac{x_2^2}{4} = 1. \tag{43.1}$$

This describes an ellipse in $\mathbb{R}^2$ centered at the origin. The axes of the ellipse align with the x_1 and x_2 axes, with semi-axis lengths of 3 and 2, respectively. The left panel in Figure 43.1 shows this ellipse.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_43

Recognizing an ellipse from its equation is not always straightforward. Let us see an example.

Example 43.2 Consider the equation

$$\frac{13}{72}x_1^2 + \frac{10}{72}x_1x_2 + \frac{13}{72}x_2^2 = 1. \tag{43.2}$$

At first, it may not be clear that this equation describes an ellipse. Equation (43.2) is quadratic, and the cross-term $\frac{10}{72}x_1x_2$ makes it harder to tell whether it represents a line, parabola, hyperbola, or ellipse. However, as the right panel in Figure 43.1 shows, the set of points satisfying this equation does form an ellipse, although it is rotated relative to the x_1- and x_2-axes.

So, how can we recognize (43.2) as the equation of an ellipse? First, notice that the equation only involves quadratic terms in x_1 and x_2, which means the left-hand-side of (43.2) is a quadratic form.

We can rewrite the equation in matrix form as

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 13/72 & 5/72 \\ 5/72 & 13/72 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 1.$$

The symmetric matrix

$$S = \frac{1}{72}\begin{bmatrix} 13 & 5 \\ 5 & 13 \end{bmatrix}$$

is positive definite because both of its eigenvalues, $1/9$ and $1/4$, are positive. By the Spectral Theorem, we can write $S = QDQ^T$, where Q is an orthogonal matrix and D is a diagonal matrix with the eigenvalues of S.

Now, let us change variables by setting $x = Qy$ (or equivalently, $y = Q^Tx$). The vector y represents the coordinates of x in the orthogonal basis formed by the eigenvectors of S. With this change of variables, the quadratic form becomes

$$x^TSx = y^TDy = \frac{y_1^2}{9} + \frac{y_2^2}{4} = 1.$$

In the y-coordinates, it is now clear that this is the equation of an ellipse. The axes of the ellipse align with the eigenvectors q_1 and q_2 of S, as shown in the right panel of Figure 43.1.

The example above shows how choosing the right coordinate system can greatly simplify an equation. A similar idea applies in higher dimensions, as the next theorem shows.

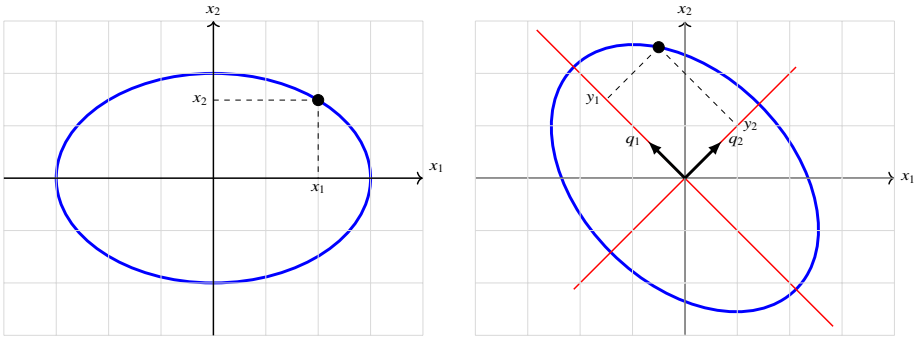


Fig. 43.1: Two ellipses in $\mathbb{R}^2$: one aligned with the axes (left) and one rotated (right).

Theorem 43.3. (The principal axes theorem)

Let $S \in \mathbb{R}^{n\times n}$ be a symmetric matrix. The set

$$\mathcal{E} = \{x \in \mathbb{R}^n \mid x^T Sx = 1\}$$

defines a non-degenerate[1] ellipsoid centered at the origin in $\mathbb{R}^n$ if and only if S is positive definite.

Furthermore, the ellipsoid's axes align with an orthonormal set of n eigenvectors of S, and the lengths of these axes are given by the reciprocals of the square roots of the corresponding eigenvalues.

Proof To prove this, recall the Spectral Theorem: any symmetric matrix S can be written as $S = QDQ^T$, where, Q is an orthogonal matrix whose columns are eigenvectors of S, and $D = \text{diag}(\lambda_1, \ldots, \lambda_n)$ is a diagonal matrix with the eigenvalues λ_i of S on the diagonal. Using this decomposition, we can rewrite the quadratic form $x^T Sx$ as

$$x^T Sx = x^T QDQ^T x = y^T Dy,$$

where $y := Q^T x$. In this transformed coordinate system y, the equation $x^T Sx = 1$ becomes

$$y^T Dy = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \cdots + \lambda_n y_n^2 = 1.$$

The equation above represents an ellipsoid if and only if all the eigenvalues λ_i are positive. Since S is positive definite if and only if all its eigenvalues are positive, we conclude that $\mathcal{E}$ defines an ellipsoid if and only if S is positive definite. In this case, the equation can be rewritten as

$$\sum_{i=1}^{n} \left(\frac{y_i}{1/\sqrt{\lambda_i}} \right)^2 = 1,$$

[1] "Non-degenerate" means the ellipsoid is full-dimensional in $\mathbb{R}^n$ and does not collapse into a lower-dimensional shape. For instance, an ellipse in three-dimensional space is considered degenerate.

which is the standard form of an ellipsoid in the y-coordinate system. The axes of the ellipsoid correspond to the eigenvectors of S, which are the columns of Q. The lengths of the axes are $1/\sqrt{\lambda_i}$. □

The following example illustrates the Principal Axes Theorem in $\mathbb{R}^2$ and $\mathbb{R}^3$.

Example 43.4 Figure 43.2-(a) shows an ellipse in $\mathbb{R}^2$ defined by the equation

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 1.$$

The ellipse's axes point in the direction of the eigenvectors of the matrix $S = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$. The eigenvectors are

$$q_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad q_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

The corresponding eigenvalues are $\lambda_1 = 3$ and $\lambda_2 = 1$, so the lengths of the ellipse's axes are $1/\sqrt{3}$ and 1.

Figure 43.2-(b) shows an ellipsoid in $\mathbb{R}^3$ defined by the equation

$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 1.$$

The axes of the ellipsoid align with the eigenvectors of the positive definite matrix

$$S = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}.$$

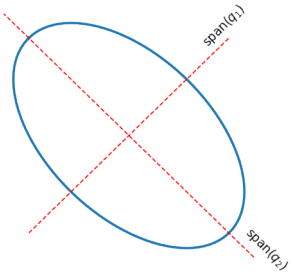
The eigenvectors are

$$q_1 = \frac{1}{2} \begin{bmatrix} -1 \\ \sqrt{2} \\ -1 \end{bmatrix}, \quad q_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \quad q_3 = \frac{1}{2} \begin{bmatrix} 1 \\ \sqrt{2} \\ 1 \end{bmatrix}.$$

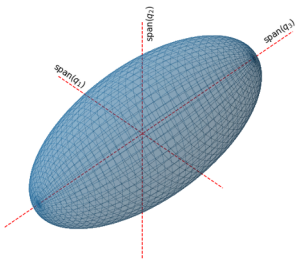
The eigenvalues are $\lambda_1 = 2 + \sqrt{2}$, $\lambda_2 = 2$, and $\lambda_3 = 2 - \sqrt{2}$. Thus, the lengths of the ellipsoid's axes are $1/\sqrt{2+\sqrt{2}}$, $1/\sqrt{2}$, and $1/\sqrt{2-\sqrt{2}}$, respectively.

43.2 Extensions of the Rayleigh-Ritz theorem

In this section, we study two variations of the Rayleigh-Ritz optimization problem. The first replaces the unit-circle constraint with an ellipsoid. The second replaces the single Rayleigh quotient with a sum of Rayleigh quotients.



(a) Two-dimensional ellipsoid (ellipse).



(b) Three-dimensional ellipsoid.

Fig. 43.2: Ellipsoids in $\mathbb{R}^2$ and $\mathbb{R}^3$.

43.2.1 Optimization of a quadratic form over an ellipsoid

In Chapter 44, we will see problems that can be written as the following constrained optimization problem:

$$\begin{aligned} &\text{optimize} \quad x^T S x, \\ &\text{subject to} \quad x^T A x = 1, \end{aligned} \tag{43.3}$$

where S is a symmetric matrix and A is a symmetric positive definite matrix. The goal of the optimization can be either minimize or maximize $x^T S x$.

By the Principal Axes theorem (Theorem 43.3), the constraint $x^T A x = 1$ describes an ellipsoid. In other words, (43.3) is asking us to maximize or minimize the quadratic form $x^T S x$ over the ellipsoid $x^T A x = 1$.

Let us transform the optimization problem in (43.3) into a familiar form (a problem like (42.2)). To do this, note that by Theorem 42.47, the symmetric positive definite matrix A has a unique positive definite square root $A^{1/2}$, which is both invertible and symmetric.

Using the change of variables $y := A^{1/2} x$, we can rewrite (43.3) as

$$\begin{aligned} &\text{optimize} \quad y^T \left(A^{-1/2} S A^{-1/2} \right) y, \\ &\text{subject to} \quad y^T y = 1, \end{aligned} \tag{43.4}$$

where $A^{-1/2}$ is the inverse of $A^{1/2}$. Since both $A^{-1/2}$ and S are symmetric, the matrix $A^{-1/2} S A^{-1/2}$ is also symmetric. Thus, the change of variables $y = A^{1/2} x$ transforms the original problem into one of optimizing a quadratic form over the unit sphere $y^T y = 1$.

Let $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_n$ denote the eigenvalues of $A^{-1/2} S A^{-1/2}$, and let $y_1, y_2, \ldots, y_n$ be corresponding orthonormal eigenvectors. By the Rayleigh-Ritz theorem, the quadratic form $y^T A^{-1/2} S A^{-1/2} y$ achieves its maximum at y_1 (with eigenvalue λ_1) and its minimum at y_n (with eigenvalue λ_n). Reversing the change of

variables, we obtain

$$\lambda_1 = \max_{x^T Ax=1} x^T Sx \quad \text{and} \quad \lambda_n = \min_{x^T Ax=1} x^T Sx.$$

These extrema are reached when $x = A^{-1/2} y_1$ and $x = A^{-1/2} y_n$, respectively.

Example 43.5 Consider the symmetric matrix $S = \begin{bmatrix} 5 & 2 \\ 2 & -1 \end{bmatrix}$ and the symmetric positive definite matrix $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Figure 43.3 illustrates the ellipse $x^T Ax = 1$, with the color gradient indicating the values of the quadratic form $x^T Sx$.

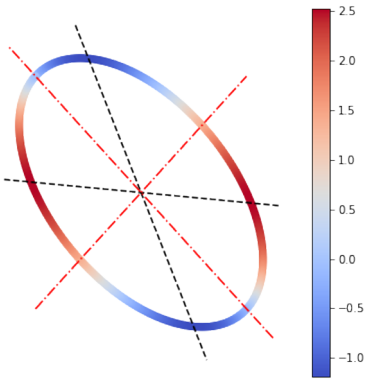


Fig. 43.3: Ellipse $x^T Ax = 1$ with a color gradient showing $x^T Sx$. Red dash-dotted lines are the ellipse's principal axes, and dashed black lines mark where $x^T Sx$ is maximized and minimized.

The eigenvalues of the symmetric matrix $A^{-1/2} S A^{-1/2}$ are approximately $\lambda_1 \approx 2.5$ and $\lambda_2 \approx -1.2$. These correspond to the maximum and minimum values of the quadratic form $x^T Sx$ over the ellipse.

The red dash-dotted lines in Figure 43.3 indicate the principal axes of the ellipse. The dashed black lines show the directions of the vectors $x_1 = A^{-1/2} y_1$ and $x_2 = A^{-1/2} y_2$, where y_1 and y_2 are eigenvectors associated with λ_1 and λ_2, respectively. These directions indicate where the quadratic form achieves its maximum and minimum values.

Generalized eigenvalue problems

A **generalized eigenvalue problem** asks for vectors x and scalars λ satisfying

$$Ax = \lambda Bx,$$

where A and B are $n \times n$ matrices. This generalizes the standard eigenvalue problem, which corresponds to $B = I_n$. Although solving generalized eigenvalue problems is beyond this book's scope, many numerical libraries offer built-in tools for them. Why mention this here? The optimization problem (43.3) can be reformulated as a generalized eigenvalue problem.

To see this, recall that by using the change of variables $y = A^{1/2}x$, we transformed (43.3) into the symmetric eigenvalue problem

$$A^{-1/2}SA^{-1/2}y = \lambda y.$$

If we pre-multiply both sides by $A^{1/2}$, we get

$$Sx = \lambda Ax,$$

which is a generalized eigenvalue problem. Multiplying both sides by A^{-1}, we can also write this as the standard eigenvalue problem

$$A^{-1}Sx = \lambda x.$$

We have three ways to solve (43.3): (i) solve the eigenvalue problem $A^{-1/2}SA^{-1/2}y = \lambda y$ and then set $x = A^{-1/2}y$; (ii) solve the eigenvalue problem $A^{-1}Sx = \lambda x$; or (iii) solve the generalized eigenvalue problem $Sx = \lambda Ax$. The best approach is to use the generalized eigenvalue problem, since most programming languages (like MATLAB and Python) have built-in functions for this that do not require computing matrix inverses or square roots.

43.2.2 Trace optimization problems

Recall that the **trace** of a matrix $A \in \mathbb{R}^{n\times n}$ is the sum of its diagonal entries. In Chapter 44, we will encounter an optimization problem involving the trace:

$$\begin{aligned} &\text{optimize} \quad \operatorname{trace}(X^T SX), \\ &\text{subject to} \quad X^T X = I_k,\ X \in \mathbb{R}^{n\times k}, \end{aligned} \tag{43.5}$$

where $S \in \mathbb{R}^{n\times n}$ is a symmetric matrix and the goal can be either maximization or minimization.

If we denote the columns of X by $x_1, x_2, \ldots, x_k$, then we can write

$$\operatorname{trace}(X^T SX) = x_1^T Sx_1 + x_2^T Sx_2 + \cdots + x_k^T Sx_k.$$

Thus, (43.5) asks us to maximize or minimize a sum of k Rayleigh quotients, with the condition that each vector x_i is orthogonal to the others. The optimization problem (43.5) becomes

$$\begin{aligned} &\text{optimize} \quad \sum_{i=1}^{k} x_i^T S x_i, \\ &\text{subject to} \quad X^T X = I_k,\ X = \begin{bmatrix} x_1 & \cdots & x_k \end{bmatrix} \in \mathbb{R}^{n \times k}. \end{aligned} \tag{43.6}$$

When $k = 1$, this reduces to the familiar problem of optimizing a quadratic form over the unit sphere that we solved in Theorem 42.12.

How do we solve (43.6)? A natural idea is to try a step-by-step iterative (or greedy) strategy: optimize each term in the sum individually, while maintaining the orthogonality constraints.

First, we solve

$$\text{optimize}_{\|x_1\|_2=1}\ x_1^T S x_1.$$

By the Rayleigh-Ritz theorem (Theorem 42.12), the solution x_1 is an eigenvector of S corresponding to either the largest or smallest eigenvalue, depending on whether we are maximizing or minimizing.

Next, we optimize the second term, adding the constraint that x_2 must be orthogonal to x_1:

$$\text{optimize}_{\substack{\|x_2\|_2=1 \\ x_1^T x_2=0}}\ x_2^T S x_2.$$

Again, the Rayleigh-Ritz theorem tells us that the solution x_2 is an eigenvector corresponding to the second largest or smallest eigenvalue of S.

We repeat this process for $x_3, x_4, \ldots, x_k$, requiring at each step that the next vector be orthogonal to all previously found vectors. By the Rayleigh-Ritz theorem, the vector x_i at each step is an eigenvector of S corresponding to its ith largest or smallest eigenvalue.

At the end of the process, we obtain a matrix $X = \begin{bmatrix} x_1 & \cdots & x_k \end{bmatrix}$, whose columns are eigenvectors associated with either the largest or smallest eigenvalues. This matrix X satisfies

$$\text{trace}(X^T S X) = \sum_{i=1}^{k} x_i^T S x_i = \begin{cases} \lambda_1 + \cdots + \lambda_k & \text{if maximizing} \\ \lambda_n + \cdots + \lambda_{n-k+1} & \text{if minimizing.} \end{cases}$$

In Theorem 43.6, we show that this iterative approach does indeed compute a solution to (43.6). This result, which is not immediately obvious, is due to Ky Fan [21], and our proof is based on his work.

Theorem 43.6. (Ky Fan's Theorem)

Let $\lambda_1, \lambda_2, \ldots, \lambda_n$ be the eigenvalues of a symmetric matrix $S \in \mathbb{R}^{n \times n}$, arranged in descending order, $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_n$. Then,

$$\max_{\substack{X \in \mathbb{R}^{n \times k} \\ X^T X = I_k}} \text{trace}(X^T S X) = \lambda_1 + \cdots + \lambda_k, \quad \text{and} \tag{43.7}$$

$$\min_{\substack{X \in \mathbb{R}^{n \times k} \\ X^T X = I_k}} \text{trace}(X^T S X) = \lambda_n + \cdots + \lambda_{n-k+1}. \tag{43.8}$$

The maximum in (43.7) is achieved when the columns of X form an orthonormal basis for the eigenspace corresponding to the k largest eigenvalues of S. Similarly, the minimum in (43.8) is achieved when the columns of X form an orthonormal basis for the eigenspace corresponding to the k smallest eigenvalues of S.

Proof We prove the result for the maximum in (43.7). The proof for the minimum in (43.8) is similar.

Let $X \in \mathbb{R}^{n\times k}$ have orthonormal columns $x_1, \ldots, x_k$. Let $q_1, \ldots, q_n$ be an orthonormal basis of eigenvectors of S, such that $Sq_i = \lambda_i q_i$.

Each column x_j can be written as a linear combination of the eigenvector basis,

$$x_j = \sum_{i=1}^{n} (q_i^T x_j) q_i.$$

Using this, we write $x_j^T S x_j$ as follows

$$\begin{aligned} x_j^T S x_j &= \sum_{i=1}^{n} \lambda_i (q_i^T x_j)^2 \\ &= \lambda_k \sum_{i=1}^{n} (q_i^T x_j)^2 + \sum_{i=1}^{k} (\lambda_i - \lambda_k)(q_i^T x_j)^2 + \sum_{i=k+1}^{n} (\lambda_i - \lambda_k)(q_i^T x_j)^2. \end{aligned}$$

Since $\sum_{i=1}^{n} (q_i^T x_j)^2 = \|x_j\|_2^2 = 1$, this simplifies to

$$x_j^T S x_j = \lambda_k + \sum_{i=1}^{k} (\lambda_i - \lambda_k)(q_i^T x_j)^2 + \sum_{i=k+1}^{n} (\lambda_i - \lambda_k)(q_i^T x_j)^2.$$

Because $\lambda_i - \lambda_k \geqslant 0$ for $i \leqslant k$ and $\lambda_i - \lambda_k \leqslant 0$ for $i > k$, we can bound $x_j^T S x_j$ as

$$x_j^T S x_j \leqslant \lambda_k + \sum_{i=1}^{k} (\lambda_i - \lambda_k)(q_i^T x_j)^2.$$

Summing over $j = 1, \ldots, k$, we get

$$\operatorname{trace}(X^T S X) = \sum_{j=1}^{k} x_j^T S x_j \leqslant \sum_{j=1}^{k} \lambda_k + \sum_{i=1}^{k} (\lambda_i - \lambda_k) \sum_{j=1}^{k} (q_i^T x_j)^2.$$

The orthonormality of $x_1, \ldots, x_k$ implies that $\sum_{j=1}^{k} (q_i^T x_j)^2 \leqslant \|q_i\|_2^2 = 1$ (see Problem 26.8). Thus,

$$\sum_{j=1}^{k} x_j^T S x_j \leqslant \sum_{j=1}^{k} \lambda_j.$$

Equality holds when $x_j = q_j$, for $j = 1, \ldots, k$, since in this case, $\sum_{j=1}^{k} x_j^T S x_j = \sum_{j=1}^{k} \lambda_j$. □

43.3 The Cholesky algorithm

As we saw in Section 42.2.4, any symmetric positive definite matrix S can be written as $S = LL^T$, where L is lower triangular with positive diagonal entries. This is the Cholesky factorization. In Example 42.41, we obtained it via three steps: LU, LDL^T, then Cholesky. With a small tweak to Gaussian elimination, you can get L directly. This direct method is the **Cholesky algorithm**, which we introduce next.

We begin with the following useful result.

Proposition 43.7. (Positive definiteness of the Schur complement)

Let

$$M = \begin{bmatrix} A & B \\ B^T & C \end{bmatrix}$$

be a symmetric 2×2 block matrix. If M is symmetric positive definite, then the matrix $C - B^T A^{-1} B$, called the **Schur complement** of the block A in M, is also symmetric positive definite.

Proof If M is positive definite, then by Theorem 42.28, its principal submatrix A is positive definite and, therefore, invertible (Corollary 42.24). This ensures that the Schur complement $C - B^T A^{-1} B$ is well-defined.

We can factor M as

$$\begin{bmatrix} A & B \\ B^T & C \end{bmatrix} = \underbrace{\begin{bmatrix} I & 0 \\ B^T A^{-1} & I \end{bmatrix}}_{=P^T} \begin{bmatrix} A & 0 \\ 0 & C - B^T A^{-1} B \end{bmatrix} \underbrace{\begin{bmatrix} I & A^{-1} B \\ 0 & I \end{bmatrix}}_{=:P}.$$

The matrix P is invertible. Therefore, the block diagonal matrix $\mathrm{diag}(A, C - B^T A^{-1} B)$ is congruent to the positive definite matrix M. By Theorem 42.22, $\mathrm{diag}(A, C - B^T A^{-1} B)$ is also positive definite. Since $C - B^T A^{-1} B$ is a principal submatrix of $\mathrm{diag}(A, C - B^T A^{-1} B)$, we conclude it is positive definite by Theorem 42.28. □

There are several ways to derive and implement the Cholesky factorization. Here, we describe a recursive approach that reduces computing the Cholesky factorization of an $n \times n$ positive definite matrix to computing the factorization for an $(n-1) \times (n-1)$ positive definite matrix.

The Cholesky algorithm

Consider a symmetric positive definite matrix $S \in \mathbb{R}^{n \times n}$, partitioned as

$$S = \begin{bmatrix} a & w^T \\ w & K \end{bmatrix},$$

where $a \in \mathbb{R}$ and K is a symmetric matrix of size $n - 1$. By Theorem 42.28, we know that $a > 0$ and K is positive definite.

We start with a Gaussian elimination step. Using lower-triangular operations on the left, we zero out the entries below a in the first column by subtracting multiples of the first row from the other rows. In matrix form:

$$\begin{bmatrix} 1 & 0 \\ -w/a & I_{n-1} \end{bmatrix} \begin{bmatrix} a & w^T \\ w & K \end{bmatrix} = \begin{bmatrix} a & w^T \\ 0 & K - ww^T/a \end{bmatrix}.$$

Normally, Gaussian elimination would continue column by column. But for Cholesky, we must keep symmetry. Next, we clear the first row to match the zeros in the first column by applying upper-triangular operations, subtracting multiples of the first column from the other columns. In matrix form:

$$\begin{bmatrix} a & w^T \\ 0 & K - ww^T/a \end{bmatrix} \begin{bmatrix} 1 & -w^T/a \\ 0 & I_{n-1} \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & K - ww^T/a \end{bmatrix}.$$

Notice that the upper-triangular factor here is the transpose of the lower-triangular factor we used earlier. This reflects the symmetry of S.

By combining the two triangular steps, we factor

$$S = \begin{bmatrix} a & w^T \\ w & K \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ w/a & I_{n-1} \end{bmatrix} \begin{bmatrix} a & 0 \\ 0 & K - ww^T/a \end{bmatrix} \begin{bmatrix} 1 & w^T/a \\ 0 & I_{n-1} \end{bmatrix}.$$

Since $a > 0$, write $a = (\sqrt{a})^2$ and absorb the square root into the triangular factors:

$$\begin{aligned} S = \begin{bmatrix} a & w^T \\ w & K \end{bmatrix} &= \begin{bmatrix} 1 & 0 \\ w/a & I_{n-1} \end{bmatrix} \begin{bmatrix} a & 0 \\ 0 & K - ww^T/a \end{bmatrix} \begin{bmatrix} 1 & w^T/a \\ 0 & I_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \\ w/a & I_{n-1} \end{bmatrix} \begin{bmatrix} (\sqrt{a})^2 & 0 \\ 0 & K - ww^T/a \end{bmatrix} \begin{bmatrix} 1 & w^T/a \\ 0 & I_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \sqrt{a} & 0 \\ w/\sqrt{a} & I_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & K - ww^T/a \end{bmatrix} \begin{bmatrix} \sqrt{a} & w^T/\sqrt{a} \\ 0 & I_{n-1} \end{bmatrix}. \end{aligned}$$

Proposition 43.7 says $K - \frac{ww^T}{a}$ is positive definite, so it has a Cholesky factorization

$$K - \tfrac{ww^T}{a} = L'(L')^T,$$

with L' lower triangular and positive diagonal entries. Substituting gives

$$\begin{aligned} S = \begin{bmatrix} a & w^T \\ w & K \end{bmatrix} &= \begin{bmatrix} \sqrt{a} & 0 \\ w/\sqrt{a} & I_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & L'(L')^T \end{bmatrix} \begin{bmatrix} \sqrt{a} & w^T/\sqrt{a} \\ 0 & I_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \sqrt{a} & 0 \\ w/\sqrt{a} & L' \end{bmatrix} \begin{bmatrix} \sqrt{a} & w^T/\sqrt{a} \\ 0 & (L')^T \end{bmatrix} \\ &=: LL^T. \end{aligned}$$

This is the Cholesky factorization of S. The first column of the Cholesky factor L is $\begin{bmatrix} \sqrt{a} \\ w/\sqrt{a} \end{bmatrix}$. Repeating this process on the Schur complement $K - ww^T/a$ and its smaller submatrices produces the remaining columns of L, yielding the full Cholesky factorization.

We can convert the discussion above into a simple algorithm. Let us illustrate this with an example.

Example 43.8 Consider the following symmetric positive definite matrix

$$S = \begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 1 \end{bmatrix}.$$

Our goal is to compute its Cholesky factorization.

We partition S as

$$S = \left[\begin{array}{c|cc} a & w^T \\ \hline w & K \end{array}\right] = \left[\begin{array}{c|cc} 25 & 15 & -5 \\ \hline 15 & 18 & 0 \\ -5 & 0 & 11 \end{array}\right].$$

The first column of the Cholesky factor L is

$$\begin{bmatrix} \sqrt{a} \\ w/\sqrt{a} \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \\ -1 \end{bmatrix}.$$

We can now write S as

$$\begin{aligned} S &= \begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{bmatrix} \\ &= \begin{bmatrix} \sqrt{a} & 0 \\ w/\sqrt{a} & I_2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & K - ww^T/a \end{bmatrix} \begin{bmatrix} \sqrt{a} & w^T/\sqrt{a} \\ 0 & I_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} 5 & 0 & 0 \\ 3 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 9 & 3 \\ 0 & 3 & 10 \end{bmatrix} \begin{bmatrix} 5 & 3 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

Next, we apply the same procedure to the smaller matrix

$$K - \frac{ww^T}{a} = \begin{bmatrix} 9 & 3 \\ 3 & 10 \end{bmatrix}.$$

This gives

$$\begin{bmatrix} 9 & 3 \\ 3 & 10 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 9 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix}.$$

Using the factorizations, the original matrix S can be written as

$$S = \begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{bmatrix} = \begin{bmatrix} 5 & 0 & 0 \\ 3 & 3 & 0 \\ -1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 9 \end{bmatrix} \begin{bmatrix} 5 & 3 & -1 \\ 0 & 3 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

Finally, the Cholesky factorization of the 1×1 matrix $\begin{bmatrix} 9 \end{bmatrix}$ is $\begin{bmatrix} 9 \end{bmatrix} = \begin{bmatrix} 3 \end{bmatrix} \begin{bmatrix} 3 \end{bmatrix}$. By combining all the intermediate factorizations, we arrive at the Cholesky factorization of the original matrix S:

$$S = \begin{bmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{bmatrix} = \begin{bmatrix} 5 & 0 & 0 \\ 3 & 3 & 0 \\ -1 & 1 & 3 \end{bmatrix} \begin{bmatrix} 5 & 3 & -1 \\ 0 & 3 & 1 \\ 0 & 0 & 3 \end{bmatrix}.$$

Note that while applying the Cholesky algorithm, we performed the following arithmetic computations:

1. Compute $\sqrt{25} = 5$ and $(1/5) \begin{bmatrix} 15 \\ -5 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$.
2. Compute $\begin{bmatrix} 18 & 0 \\ 0 & 11 \end{bmatrix} - \begin{bmatrix} 3 \\ -1 \end{bmatrix} \begin{bmatrix} 3 & -1 \end{bmatrix} = \begin{bmatrix} 9 & 3 \\ 3 & 10 \end{bmatrix}$.
3. Compute $\sqrt{9} = 3$ and $(1/3) \begin{bmatrix} 3 \end{bmatrix} = \begin{bmatrix} 1 \end{bmatrix}$.
4. Compute $10 - \begin{bmatrix} 1 \end{bmatrix} = 9$.
5. Compute $\sqrt{9} = 3$.

The numbers highlighted in blue correspond to the entries of the columns of the Cholesky factor

$$L = \begin{bmatrix} 5 & 0 & 0 \\ 3 & 3 & 0 \\ -1 & 1 & 3 \end{bmatrix}.$$

The steps in Example 43.8 replicate what a computer does to compute the Cholesky factorization of a positive definite matrix S. Because all intermediate matrices stay symmetric, we only store and update their lower triangular parts. That cuts the work to about half of an LU factorization.

For these reasons, the Cholesky algorithm is the preferred test for positive definiteness of a symmetric matrix: run the algorithm, and if it completes with all positive diagonal entries, S is positive definite; if it breaks down (a non-positive pivot appears), then S is not positive definite.

43.4 More characterizations of positive semidefinite matrices

In Chapter 42, we saw several ways to characterize positive definite and positive semidefinite matrices. Some of those apply only to positive definite matrices. In this section, we give two more characterizations for positive semidefinite matrices that extend the results for the definite case.

43.4.1 Principal minor characterization

In Theorem 42.32, we used leading principal minors to characterize positive definite matrices. Example 42.35 shows that this fails for positive semidefinite matrices. If instead we require every principal minor to be nonnegative, we recover a characterization for positive semidefinite matrices.

For the proof, we need the following result, which expands the determinant of $A + xI_n$ (with x a scalar) as a polynomial in x.

Proposition 43.9. (Determinant of $A + xI$)

Let A be an $n \times n$ real matrix and $x \in \mathbb{R}$. Then,

$$\det(A + xI_n) = x^n + \sum_{B \subseteq \{1,2,\ldots,n\},\ |B|>0} \det(A^B) x^{n-|B|},$$

where A^B denotes the principal submatrix of A containing the rows and columns indexed by B, and $|B|$ is the number of elements in B.

There is a lot to unpack in the previous proposition, so let us work through a small example to clarify it.

Example 43.10 Consider the 3×3 matrix

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}.$$

Proposition 43.9 shows how to express $\det(A + xI_3)$ as a polynomial in x. In this expansion, the term with x^{3-k} comes with a coefficient that is the sum of all principal minors of A of size k. In other words, the expansion is organized by the sizes of the principal minors.

For $k = 1$, the principal submatrices are the individual main diagonal elements:

$$\begin{aligned} \det(A^{\{1\}}) &= a, \\ \det(A^{\{2\}}) &= e, \quad \text{and} \\ \det(A^{\{3\}}) &= i. \end{aligned}$$

Their contribution to $\det(A + xI_3)$ is

$$(a + e + i)x^2.$$

For $k = 2$, the possible index sets B are $\{1, 2\}$, $\{1, 3\}$, and $\{2, 3\}$. The corresponding principal submatrices and their determinants are

$$\begin{aligned} A^{\{1,2\}} &= \begin{bmatrix} a & b \\ d & e \end{bmatrix}, & \det(A^{\{1,2\}}) &= ae - bd, \\ A^{\{1,3\}} &= \begin{bmatrix} a & c \\ g & i \end{bmatrix}, & \det(A^{\{1,3\}}) &= ai - cg, \quad \text{and} \\ A^{\{2,3\}} &= \begin{bmatrix} e & f \\ h & i \end{bmatrix}, & \det(A^{\{2,3\}}) &= ei - fh. \end{aligned}$$

Their combined contribution is

$$[(ae - bd) + (ai - cg) + (ei - fh)]\, x.$$

For $k = 3$, $B = \{1, 2, 3\}$ and the only submatrix is A itself. Its contribution is

$$\det(A) \cdot x^0 = \det(A).$$

Putting it all together, we have

$$\det(A + xI_3) = x^3 + (a + e + i)x^2 + \Big[(ae - bd) + (ai - cg) + (ei - fh)\Big]x + \det(A).$$

Now we are ready to characterize positive semidefinite matrices using their principal minors.

Theorem 43.11. (Principal minor characterization of positive semidefinite matrices)

A symmetric matrix $S \in \mathbb{R}^{n \times n}$ is positive semidefinite if and only if all its principal minors are nonnegative.

Proof We already showed in Theorem 42.28 that if S is positive semidefinite, then every principal minor of S is nonnegative. Now, we prove the converse.

Assume that every principal minor of S is nonnegative. Let T be any $k \times k$ principal submatrix of S and choose $\epsilon > 0$. By Proposition 43.9, we can write

$$\det(T + \epsilon I_k) = \epsilon^k + \sum_{i=1}^{k} s_i(T)\, \epsilon^{k-i},$$

where $s_i(T)$ denotes the sum of all principal minors of T of size i. Since each principal minor of T is also a principal minor of S and all such minors are nonnegative by assumption, we have $s_i(T) \geqslant 0$ for all i. Thus,

$$\det(T + \epsilon I_k) \geqslant \epsilon^k > 0.$$

This inequality tells us that every principal minor T of the matrix $S+\epsilon I_n$ is positive. In particular, its leading principal minors are all positive, which, by Theorem 42.32, implies that $S + \epsilon I_n$ is positive definite.

Now, because $S + \epsilon I_n$ is positive definite, all of its eigenvalues are positive. A fundamental fact in matrix analysis is that the eigenvalues of a matrix depend continuously on its entries[2]. As $\epsilon \to 0^+$, the matrix $S + \epsilon I_n$ converges to S and its eigenvalues converge to those of S. Since the limit of positive numbers cannot be negative, all eigenvalues of S are nonnegative. Therefore, by Theorem 42.23, S is positive semidefinite.

---

[2] We will not prove that here but encourage you to investigate this result on your own (see, for example, Horn and Johnson's *Matrix Analysis* [30]).

Example 43.12 The 2×2 symmetric matrix

$$S = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$$

is positive semidefinite if and only

$$\begin{aligned} a \geqslant 0, \quad c \geqslant 0 \qquad & (1x1 \text{ minors}) \\ ac - b^2 \geqslant 0 \qquad & (2x2 \text{ minor}) \end{aligned}$$

Example 43.13 The 3×3 symmetric matrix

$$S = \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}.$$

is positive semidefinite if and only if

$$\begin{aligned} a \geqslant 0, \quad d \geqslant 0, \quad f \geqslant 0 \qquad & (1x1 \text{ minors}) \\ ad - b^2 \geqslant 0, \quad af - c^2 \geqslant 0, \quad df - e^2 \geqslant 0 \qquad & (2x2 \text{ minors}) \\ \det(S) \geqslant 0 \qquad & (3x3 \text{ minor}) \end{aligned}$$

For an $n \times n$ symmetric matrix, there are $\binom{n}{k}$ principal minors of size k for each $k = 1, \ldots, n$. So the total number of nonempty principal minors is

$$\sum_{k=1}^{n} \binom{n}{k} = 2^n - 1.$$

This means checking positive semidefiniteness by testing all principal minors requires verifying $2^n - 1$ conditions. That makes Theorem 43.11 more of a theoretical tool than a practical one. By contrast, Theorem 42.32 needs only n checks to confirm positive definiteness.

43.4.2 Gram-matrix characterization and the semi-Cholesky factorization

In Section 42.2.4, we saw that symmetric positive (semi)definite matrices are exactly the Gram matrices. For positive definite matrices, the Gram factor has full column rank, and a particularly useful choice is the Cholesky factorization. We now show

that positive semidefinite matrices admit a similar factorization, called the **semi-Cholesky factorization**.

Theorem 43.14. (Semi-Cholesky factorization)

Let $S \in \mathbb{R}^{n\times n}$ be a symmetric matrix. Then, S is positive semidefinite if and only if there exists a lower triangular matrix L with nonnegative diagonal entries such that $S = LL^T$. This is called a **semi-Cholesky factorization** of S.

Proof Assume S is symmetric and has a semi-Cholesky factorization. Then, S is a Gram matrix and by Theorem 42.38, S is positive semidefinite.

Now assume that S is symmetric positive semidefinite. We prove the result by induction on the size n of S. The result is clearly true for $n = 1$.

Assume S is $n \times n$ and write

$$S = \begin{bmatrix} \alpha & v^T \\ v & B \end{bmatrix},$$

where B is $(n-1)\times(n-1)$. Since S is positive semidefinite,

$$\alpha = e_1^T S\, e_1 \geqslant 0.$$

We consider two cases depending on α.

Case I: $\alpha = 0$. For any $j > 1$, the 2×2 principal minor on rows and columns $\{1, j\}$ is

$$\det \begin{bmatrix} 0 & v_{j-1} \\ v_{j-1} & B_{j-1,j-1} \end{bmatrix} = -v_{j-1}^2.$$

If any $v_{j-1} \neq 0$, that minor is negative, contradicting Theorem 43.11, which requires all principal minors of a positive semidefinite matrix to be nonnegative. Hence $v = 0$.

By the inductive hypothesis,

$$B = L_1 L_1^T,$$

with L_1 lower triangular and nonnegative on its diagonal. Then

$$S = \begin{bmatrix} 0 & 0 \\ 0 & L_1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & L_1 \end{bmatrix}^T = LL^T, \quad \text{where} \quad L = \begin{bmatrix} 0 & 0 \\ 0 & L_1 \end{bmatrix}.$$

Case II: $\alpha \neq 0$. Let

$$C = B - \tfrac{1}{\alpha}\, v\, v^T$$

be the Schur complement of α in S. Since S is positive semidefinite, Proposition 43.7 implies C is also positive semidefinite. By the inductive hypothesis, C admits a semi-Cholesky factorization

$$C = L_1 L_1^T,$$

with L_1 lower triangular and nonnegative on the diagonal. Then

$$S = \begin{bmatrix} \sqrt{\alpha} & 0 \\ v/\sqrt{\alpha} & L_1 \end{bmatrix} \begin{bmatrix} \sqrt{\alpha} & 0 \\ v/\sqrt{\alpha} & L_1 \end{bmatrix}^T = LL^T, \quad \text{where} \quad L = \begin{bmatrix} \sqrt{\alpha} & 0 \\ v/\sqrt{\alpha} & L_1 \end{bmatrix}.$$

Example 43.15 Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 14 \end{bmatrix}.$$

Since the second row is twice the first, A is singular. Still, A is positive semidefinite because it admits the factorization

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 0 & 0 \\ 1 & 3 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 3 \\ 0 & 0 & 2 \end{bmatrix} = LL^T.$$

This semi-Cholesky factorization confirms that A is positive semidefinite.

43.4.3 Pivot characterization

Theorem 42.36 characterizes positive definite matrices by their Gaussian elimination pivots, that is, the diagonal entries of the U factor in an LU factorization. Can we extend this to positive semidefinite matrices by allowing nonnegative pivots? First, we must ask whether an LU factorization always exists for such matrices. The following example illustrates that this may be the case.

Example 43.16 Consider the symmetric matrix

$$S = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 12 \end{bmatrix}.$$

This matrix is positive semidefinite, since its eigenvalues are approximately 0, 0.9, and 16. To check for an LU factorization, we perform Gaussian elimination and obtain

$$S = LU = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

Notice that the U factor is singular, and its pivots, 1, 0, and 3, are all nonnegative.

Theorem 43.17 establishes two points. First, we can perform Gaussian elimination without pivoting on any positive semidefinite matrix. Second, a matrix is positive semidefinite exactly when all its Gaussian elimination pivots are nonnegative.

Theorem 43.17. (LU factorization of positive semidefinite matrices)

Let $S \in \mathbb{R}^{n \times n}$ be a symmetric matrix. Then S is positive semidefinite if and only if S has an LU factorization and all the pivots in that factorization are nonnegative.

Proof Assume S is a symmetric positive semidefinite matrix. We prove the result by induction on n. If $n = 1$, then $S \geqslant 0$ and $S = 1 \cdot S$ is an LU factorization of S. Moreover, the only pivot, S, is nonnegative.

Assume the result is true for matrices of size $n - 1$. Let S be a symmetric positive semidefinite matrix of size n, and let us partition S as follows:

$$S = \begin{bmatrix} \alpha & v^T \\ v & B \end{bmatrix}, \quad \alpha \in \mathbb{R}.$$

We have $\alpha \geqslant 0$ since S is positive semidefinite.

Case I: $\alpha = 0$. As in the proof of Theorem 43.14, we have $v = 0$. The submatrix B is then a principal submatrix of S, so it is symmetric and positive semidefinite. By the inductive hypothesis, B admits an LU factorization $B = L_1 U_1$ with nonnegative pivots. Hence

$$S = \begin{bmatrix} 1 & 0 \\ 0 & L_1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & U_1 \end{bmatrix} = LU.$$

The pivots of S are 0 (from the top-left entry) and the diagonal entries of U_1. All these values are nonnegative.

Case II: $\alpha \neq 0$. Write

$$S = \begin{bmatrix} 1 & 0 \\ v/\alpha & I_{n-1} \end{bmatrix} \begin{bmatrix} \alpha & v^T \\ 0 & B - \frac{v\,v^T}{\alpha} \end{bmatrix}.$$

The Schur complement $C = B - vv^T/\alpha$ is positive semidefinite (by Proposition 43.7). By the inductive hypothesis, C factors as $C = L_1 U_1$ with nonnegative pivots. Substituting gives

$$S = \begin{bmatrix} 1 & 0 \\ v/\alpha & L_1 \end{bmatrix} \begin{bmatrix} \alpha & v^T \\ 0 & U_1 \end{bmatrix},$$

an LU factorization of S. Its pivots are α and the diagonal entries of U_1, all of which are nonnegative.

Assume now that S is a symmetric matrix with an LU factorization $S = LU$ and such that the main diagonal entries of U are nonnegative real numbers. We first show that $S = LDL^T$, where D is a diagonal matrix whose main diagonal entries are those of U. We prove the result by induction on the size n of S.

If $n = 1$, then the result follows trivially since $S = 1 \cdot S = 1 \cdot S \cdot 1$.

Assume the result is true for symmetric matrices of size $n-1$. Let S be an $n \times n$ symmetric matrix and let $S = LU$ be an LU factorization of S. Let us partition L and U as follows:

$$L = \begin{bmatrix} 1 & 0 \\ v & L_1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} u_1 & w^T \\ 0 & U_1 \end{bmatrix}, \quad u_1 \in \mathbb{R}.$$

Since S is symmetric, we have

$$LU = \begin{bmatrix} u_1 & w^T \\ u_1 v & vw^T + L_1 U_1 \end{bmatrix} = \begin{bmatrix} u_1 & u_1 v^T \\ w & wv^T + (L_1 U_1)^T \end{bmatrix} = (LU)^T.$$

This implies that $w = u_1 v$, and

$$L_1 U_1 = (L_1 U_1)^T.$$

Note that $L_1 U_1$ is a symmetric matrix of size $n-1$. Thus, by the inductive hypothesis, $L_1 U_1 = L_1 D_1 L_1^T$, where D_1 is a diagonal matrix whose main diagonal entries are those of U_1. Hence,

$$U = \begin{bmatrix} u_1 & u_1 v^T \\ 0 & D_1 L_1^T \end{bmatrix} = \begin{bmatrix} u_1 & 0 \\ 0 & D_1 \end{bmatrix} \begin{bmatrix} 1 & v^T \\ 0 & L_1^T \end{bmatrix} =: DL^T.$$

Since the diagonal entries of U are nonnegative, we can write $D = D^{1/2} D^{1/2}$. Then

$$S = LDL^T = \left(L\, D^{1/2}\right)\left(L\, D^{1/2}\right)^T.$$

This expresses S as a Gram matrix, so by Theorem 42.38, S is positive semidefinite.□

Problems

43.1 Consider the equation

$$2x_1^2 + 2x_1 x_2 + 2x_2^2 = 1.$$

(a) Show that this equation represents an ellipse in $\mathbb{R}^2$.
(b) Determine the lengths of the ellipse's axes.
(c) Find an orthogonal transformation that rotates the ellipse so its axes are parallel to the x_1- and x_2-axes.

43.2 Determine whether the equation

$$x_1^2 + x_2^2 + 2x_3^2 + 4x_1 x_2 = 1.$$

represents an ellipsoid in $\mathbb{R}^3$.

43.3 In the spectral decomposition $S = QDQ^T$ of a symmetric matrix S, the matrix Q is not unique. How does this non-uniqueness impact the Principal Axes theorem, especially when an eigenvalue has multiplicity greater than 1.

43.4 What changes if the positive definite matrix in the Principal Axes theorem is replaced with a positive semidefinite matrix?

43.5 Suppose A and B be two $n \times n$ positive definite matrices.

(a) Show that $B - A$ is positive definite if and only if $I_n - A^{1/2}B^{-1}A^{1/2}$ is positive definite.

Hint: Show that the matrices $B - A$ and $I_n - B^{-1/2}AB^{-1/2}$ are congruent. Then use Problem 42.31.

(b) Show that $B - A$ is positive definite if and only if $A^{-1} - B^{-1}$ is positive definite.

Hint: Use part (a).

(c) Consider the following two sets (ellipsoids centered at the origin):

$$\epsilon_A = \{x \mid x^T A^{-1} x \leqslant 1\} \qquad \text{and} \qquad \epsilon_B = \{x \mid x^T B^{-1} x \leqslant 1\}.$$

Show that $B - A$ is positive definite if and only if $\epsilon_A \subseteq \epsilon_B$.

43.6 Use the Cholesky factorization $A = LL^T$ to convert the constrained optimization problem (43.3) into an unconstrained problem of the form (42.2).

43.7 Let A be a symmetric matrix, and let B be a positive definite matrix. Show how to use either the Cholesky factorization or the positive definite square root of B to transform the eigenvalue problem $(AB)x = \lambda x$ into one of the form $Sx = \lambda x$, where S is a symmetric matrix.

43.8 Consider the matrices

$$S = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \quad \text{and} \quad A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Solve the following optimization problem:

$$\begin{aligned} &\text{maximize} && x^T S x \\ &\text{subject to} && x^T A x = 1, \end{aligned}$$

43.9 Consider the matrix A from Problem 42.34. Find its Cholesky factorization using two different methods: convert the LDL^T factorization obtained in Problem 42.34 into a Cholesky factorization, and second, compute the Cholesky factorization directly using the Cholesky algorithm.

43.10 Apply the Cholesky algorithm to the matrix

$$S = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 7 \end{bmatrix}$$

to express it as a Gram matrix.

43.11 Use the Cholesky decomposition to solve the linear system of equations $Ax = b$, where

$$A = \begin{bmatrix} 10 & 2 & 3 \\ 2 & 8 & 1 \\ 3 & 1 & 6 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 14 \\ 10 \\ 15 \end{bmatrix}.$$

43.12 Given a symmetric positive definite matrix S with Cholesky factorization $S = LL^T$, suppose S is perturbed by a matrix ΔA so that $A + \Delta A$ remains positive definite. Investigate how the Cholesky factorization changes in response to ΔA. In particular, consider the case when ΔA is a rank-one matrix.

43.13 Consider the matrix

$$A = \begin{bmatrix} 4 & 1 & 0 & 0 \\ 1 & 3 & 1 & 0 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}.$$

Compute its Cholesky factorization and discuss the sparsity pattern of L. How does this pattern compare with the sparsity pattern of A?

43.14 In quadratic programming, one often needs to solve the optimization problem

$$\min_{x \in \mathbb{R}^n} \left(\frac{1}{2} x^T Q x - b^T x \right),$$

where $Q \in \mathbb{R}^{n \times n}$ is symmetric positive definite and $b \in \mathbb{R}^n$. Investigate how to express the solution in terms of the Cholesky decomposition of Q.

43.15 Consider the block matrix

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{12}^T & A_{22} \end{bmatrix}.$$

Derive the conditions under which A is positive definite and express its Cholesky factorization in terms of the block entries A_{11}, A_{12}, and A_{22}.

43.16 Turn Example 43.8 into an algorithm for computing the Cholesky factorization of a symmetric positive definite matrix. Then, test your algorithm on the matrix

$$S = \begin{bmatrix} 16 & 8 & 4 \\ 8 & 10 & 2 \\ 4 & 2 & 6 \end{bmatrix}.$$

43.17 For very large symmetric positive definite matrices, a parallel Cholesky algorithm can be faster than the standard version. Research and implement this parallel method, then compare its runtime against the standard Cholesky algorithm.

Chapter 44
IX-FROM THE CLASSROOM TO REAL LIFE

This chapter presents several Machine Learning applications of positive (semi)definite matrices. We cover problems such as clustering graph nodes, embedding graphs in low-dimensional spaces, and dimensionality reduction. For each application, we show how the eigenvalues and eigenvectors of positive (semi)definite matrices help solve the problem. Our goal is to build intuition rather than prove every detail. Where theory matters, we point to further reading.

44.1 Graph clustering methods

In Chapter 3, we showed that networks, whether social, communication, transportation, or document-based, can be represented as graphs. Nodes represent members of the network and edges represent connections between them.

In these graphs, communities are groups where members are more connected to each other than to the rest of the network. Our goal is to find these communities. For small networks, we can do this just by looking at the graph. But as networks get larger and more complex, we need computational methods.

We call these highly connected groups in a network **clusters**. Simply put, a **cluster in a graph** is a group of nodes that are closely connected to each other, with only a few connections to nodes outside the group. Figure 44.1 shows an example with 30 nodes split into two clusters of 15 nodes each.

To find clusters in a graph, we try to reorder the rows and columns of its adjacency matrix so that nodes in the same cluster appear next to each other. The goal is to make the matrix look as close as possible to block-diagonal form. In this form, the main diagonal blocks have many nonzero entries, representing dense connections within clusters, while the entries outside these blocks are mostly zeros. The question is how to find this reordering automatically.

44.1.1 Graph spectral partitioning and the Fiedler vector

We begin with the problem of finding clusters in graphs that have two clusters, like the one in Figure 44.1. We assume the graph is connected; otherwise, each connected component would already form a cluster.

Suppose we have a set of nodes V and a set of undirected edges E. Our goal is to split V into two disjoint sets, V_1 and V_2, with each set representing a cluster.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_44

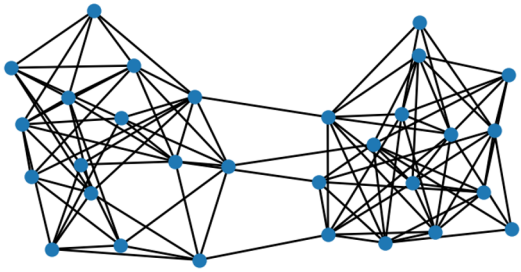


Fig. 44.1: An example of a graph with 30 nodes divided into two clusters, each containing 15 tightly connected nodes.

To find two clusters, we will use **spectral partitioning**, a method introduced by the Czech mathematician Miroslav Fiedler [22]. This approach is based on the positive semidefiniteness of the graph Laplacian L for a connected graph.

For simplicity, we assume the graph G is unweighted, so each edge has weight 1 and missing edges have weight 0. This makes the explanation easier, but spectral partitioning also works for weighted graphs. The only difference is how we build the Laplacian in the weighted case.

The main idea of spectral partitioning is to assign a real number y to each node in the graph. Nodes in one cluster get negative values, and nodes in the other cluster get positive values. This way, we can separate the two clusters.

To do this, we minimize the sum

$$\sum_{(i,j)\in E} (y_i - y_j)^2, \tag{44.1}$$

where the sum is over all pairs of connected nodes. But why do we want to minimize this? If two connected nodes i and j get values y_i and y_j with opposite signs, the difference $(y_i - y_j)$ will be large. So, minimizing this sum encourages connected nodes to have similar values. In other words, nodes in the same cluster (with the same sign) are pulled together.

In Theorem 42.50, we showed that for unweighted graphs, the sum (44.1) can be expressed as $y^T L y$, where $y = \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix}^T \in \mathbb{R}^n$, and L is the graph's Laplacian matrix. Since we only care about the signs of y's entries, we can normalize y without affecting the signs. Thus, the problem becomes

$$\min_{\|y\|_2=1} y^T L y. \tag{44.2}$$

Recall that the graph Laplacian matrix L is positive semidefinite, with its smallest eigenvalue $\lambda_n = 0$ and corresponding eigenvector being the all-ones vector e (Theorem 42.50). By the Rayleigh-Ritz theorem (Theorem 42.12), the unit vector y that minimizes $y^T L y$ is the unit eigenvector of L corresponding to $\lambda_n = 0$. This eigenvector is $y = \frac{1}{\sqrt{n}} e$, a vector whose entries are all equal to $1/\sqrt{n}$. In other words,

this solution collapses the entire graph into a single point. Hardly useful for finding clusters!

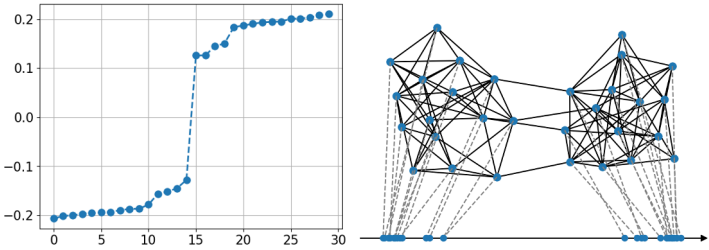
We need to adjust (44.2) to ensure that some nodes are mapped to positive values while others are mapped to negative values. The solution is adding the constraint $\sum_{i=1}^{n} y_i = 0$, which forces the y-values to balance around zero. Additionally, requiring y to be a unit vector prevents all y-values from being zero.

We can express the constraint $\sum_{i=1}^{n} y_i = 0$ as $y^T e = 0$, which means y must be orthogonal to the eigenvector e of the graph Laplacian L corresponding to the eigenvalue 0. With this constraint, our problem becomes

$$\min_{\substack{\|y\|_2^2=1 \\ y \perp e}} \sum_{(i,j)\in E} (y_i - y_j)^2 = \min_{\substack{\|y\|_2^2=1 \\ y \perp e}} y^T L y. \tag{44.3}$$

By the Raileigh-Ritz theorem (Theorem 42.12), (44.3) is solved when y is an eigenvector of L with the second smallest eigenvalue λ_{n-1} of the graph Laplacian L, which is nonzero since we are assuming that the graph is connected. If the eigenspace of L with eigenvalue λ_{n-1} (whose nonzero vectors are called **Fiedler vectors**) has dimension 1, then any of the Fiedler vectors can be used to produce the two clusters. Their entries are the real numbers that the nodes of the graph are mapped to. Positive and negative entries of the chosen Fiedler vector indicate the two clusters of nodes. It is clear that the obtained clusters are independent of which of the eigenvectors is chosen.

Example 44.1 Figure 44.2-(a) shows the entries of the chosen Fiedler vector (left panel) and how it maps the nodes of the graph in Figure 44.1 to the real line, separating the two clusters (right panel). Notice that the first 15 components of the Fiedler vector are negative and of similar magnitude, while the last 15 components are positive and of similar magnitude as well. Figure 44.2-(b) illustrates this mapping: one cluster is mapped to the positive real line, and the other to the negative real line.



(a) Entries of the Fiedler vector. (b) Graph nodes mapped into $\mathbb{R}$.

Fig. 44.2: Visualization of the Fiedler vector for the graph in Figure 44.1. Left: Entries of the Fiedler vector. Right: The mapping of the graph onto $\mathbb{R}$, where one cluster is on the positive real line and the other on the negative real line.

In this example, the negative entries of the Fiedler vector come first, followed by the positive ones. This happens because the nodes are labeled so that the first group

belongs to cluster 1 and the rest to cluster 2. In practice, though, this is not always the case. The entries can jump back and forth between negative and positive values.

Example 44.2 In this larger example, we generated a graph with $n = 80$ nodes split into two clusters: one with $n_1 = 50$ nodes and the other with $n_2 = 30$. We connected nodes within each cluster with a probability of 0.5, and connected nodes in different clusters with a probability of 0.01.

In the left panel of Figure 44.3, we show the graph's adjacency matrix. A blue dot at position (i, j) means nodes i and j are connected. Can you identify the two clusters just by looking at this matrix? It is not easy. The clusters are difficult to see from the adjacency matrix alone.

Fig. 44.3: Visualization of the adjacency matrix before and after reordering based on the Fiedler vector. The original adjacency matrix (left) shows no clear structure, while the reordered matrix (right) reveals two distinct clusters.

To reveal the two clusters, we compute a Fiedler vector. This vector contains both positive and negative entries, as shown in the left panel of Figure 44.4. In the right panel, we sort these entries in increasing order. The clear gap between the negative and positive values indicates the presence of two distinct clusters in the graph.

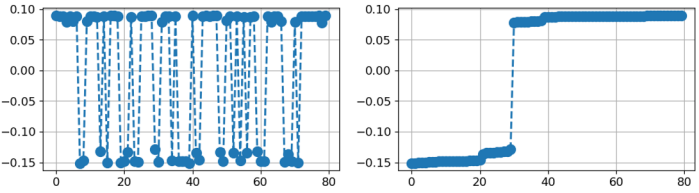


Fig. 44.4: Visualization of the Fiedler vector for a graph with two clusters: The left panel shows the entries of the Fiedler vector. The right panel sorts these entries, clearly showing a gap that indicates the two clusters.

Finally, we reorder the nodes of the graph according to the values in the Fiedler vector. Node 1 now has the smallest y value, node 2 the next smallest, and so on, up to node n with the largest y value. The right panel of Figure 44.3 shows the reordered adjacency matrix. Now, the matrix is almost block diagonal, and the two clusters are much easier to see.

In practice, the eigenspace of the Laplacian L for the second smallest eigenvalue λ_{n-1} may have dimension greater than one. In this situation, different choices of unit eigenvectors may produce different clusterings as we show in the next example.

Example 44.3. (Two different cuts from two Fiedler eigenvectors on C_6)

Consider the cycle graph C_6, a 6-vertex cycle where each vertex connects to two neighbors. Label the vertices 0, 1, 2, 3, 4, 5 in order. The eigenvalues of the Laplacian matrix of C_6 are

$$\lambda_k = 2 - 2\cos\left(\tfrac{2\pi k}{6}\right), \qquad k = 0, \ldots, 5.$$

The second-smallest eigenvalue is $\lambda_1 = \lambda_5 = 1$, with algebraic and geometric multiplicity $r = 2$.

An orthonormal basis for the eigenspace of $\lambda = 1$ is

$$u = \tfrac{1}{\sqrt{3}}\left[1 \ \tfrac{1}{2} \ -\tfrac{1}{2} \ -1 \ -\tfrac{1}{2} \ \tfrac{1}{2}\right], \qquad \text{and} \qquad v = \tfrac{1}{\sqrt{3}}\left[0 \ \tfrac{\sqrt{3}}{2} \ \tfrac{\sqrt{3}}{2} \ 0 \ -\tfrac{\sqrt{3}}{2} \ -\tfrac{\sqrt{3}}{2}\right].$$

Let us apply the simple *sign-thresholding* rule: place vertices with negative entries in one cluster and those with positive or zero entries in the other. For u, the signs of its entries are $+, +, -, -, -, +$. This yields the vertex partition

$$A_u = \{0, 1, 5\} \qquad \text{and} \qquad B_u = \{2, 3, 4\}.$$

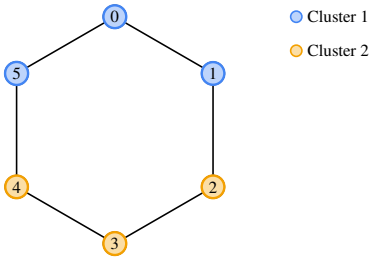
For v, the signs of its entries are $0, +, +, 0, -, -$. This produces the vertex partition

$$A_v = \{0, 1, 2, 3\} \qquad \text{and} \qquad B_v = \{4, 5\}.$$

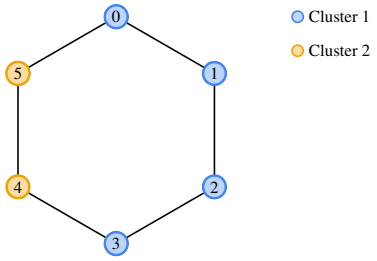
Thus, two valid eigenvectors from the same Fiedler eigenspace can give *different partitions* of the same graph. Figure 44.5 shows the two partitions from sign-thresholding the orthonormal eigenvectors u, v in the $\lambda_2 = 1$ eigenspace of the Laplacian of C_6.

Example 44.3 shows that when the Fiedler eigenvalue has multiplicity $r > 1$, the spectral clustering solution is not unique, and different eigenvectors can give different partitions of the vertices. But in this case, forcing a two-cluster partition on a cycle is not a good idea: C_n has no two well-connected subgraphs that are weakly connected to each other.

A natural question is whether there are graphs with two real clusters but with λ_2 having multiplicity larger than one. This happens in highly symmetric cases where



(a) Partition of the vertices from the Fiedler vector u.



(b) Partition of the vertices from the Fiedler vector v.

Fig. 44.5: When the eigenspace of the second smallest Laplacian eigenvalue has dimension greater than one, different choices of Fiedler vector can produce different vertex partitions.

several equally good 2-way cuts exist. In such cases, the multiplicity reflects this symmetry.

A graph can have more than two clusters, so a natural question is how to find the right number. The **eigengap heuristic** is a common method for this. We order the Laplacian eigenvalues $\lambda_1 \leqslant \lambda_2 \leqslant \ldots \leqslant \lambda_n$ and look for the largest gap $\lambda_{k+1} - \lambda_k$. The number k of eigenvalues before this gap is taken as the number of clusters. This rule works well in many cases, but it requires a careful interpretation in symmetric or noisy cases.

- Case 1 (evidence for k clusters). A block of k small eigenvalues, followed by a large gap, points to k clear clusters.
- Case 2 (single community). If the eigenvalues grow gradually with no clear early gap, the graph behaves like a single cluster, and any small-k partition of the vertices is arbitrary.

For the cycle, the spectrum grows smoothly with no clear early eigengap. The heuristic therefore places C_n in Case 2.

Remark 44.4. (Likelihood of multiple eigenvalues in real-life networks)

It is generally unlikely for the graph Laplacian of a large, complex real-life network to have non-trivial eigenvalues with an algebraic multiplicity greater than 1. However, it is not impossible and becomes significantly more likely in networks with a high degree of symmetry or specific regular structures.

When a graph has more than two clusters, we can still use spectral partitioning by applying it recursively. First, we divide the graph into two clusters. Then, we repeat the process within each cluster to check if further splits are possible. This approach is called **recursive spectral clustering**.

Example 44.5. (Recursive spectral clustering)

For this example, we generated a graph with $n = 80$ nodes and three clusters of sizes $n_1 = n_2 = 30$ and $n_3 = 20$. Nodes within the same cluster are connected with a probability of 0.5, while nodes between different clusters are connected with a probability of 0.01.

The left panel of Figure 44.6 shows the adjacency matrix of the graph, while the middle panel shows the adjacency matrix permuted based on the Fiedler vector sorting order. Spectral partitioning reveals two clusters in this step.

Next, we apply spectral partitioning to the top-left and bottom-right diagonal blocks of the permuted adjacency matrix and reorder them using their Fiedler vectors. The resulting adjacency matrix, displayed in the right panel, shows that the second cluster has been split into two subclusters, while the first cluster remains unchanged. With two applications of spectral partitioning, we successfully identify all three clusters.

Fig. 44.6: Adjacency matrix and reordered adjacency matrices using the Fiedler vector. The first spectral partitioning separates the graph into two clusters. Applying spectral partitioning again to one of these clusters shows that it actually contains two smaller clusters.

Like any method, recursive spectral partitioning can sometimes miss clusters, even in small or simple graphs.

Example 44.6 Consider the graph in Figure 44.7. If we apply spectral partitioning, we obtain cluster 1 containing vertices 0, 1, 2, and 3, and cluster 2 containing the rest. But it is clear that applying spectral clustering again to these two groups will not give us the three clusters that are so obvious in the graph.

In the next section, we introduce another algorithm for finding clusters in a graph. This method can sometimes detect clusters that spectral partitioning misses, as in Example 44.6.

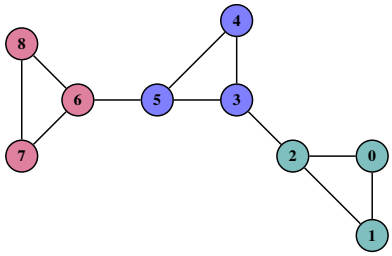


Fig. 44.7: Graph example where spectral partitioning, based on the Fiedler eigenvector, fails to correctly identify the three distinct clusters.

44.1.2 Spectral clustering

Spectral clustering extends spectral partitioning to find two or more clusters in graphs. At the end of this section, we will see how to use it with non-graph datasets that have challenging shapes, like those in Figure 44.8, where the k-means clustering method often fails.

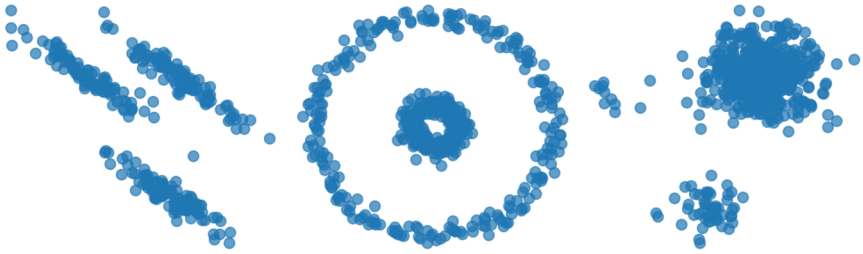


Fig. 44.8: Three datasets with challenging geometries for the k-means algorithm.

There are several variants of spectral clustering. If you want to learn more, [48] gives a good overview and explains the ideas behind these methods. The version we present here combines elements of two commonly used approaches: we use the unnormalized graph Laplacian $L = D - A$, as in unnormalized spectral clustering, and then normalize the rows of the eigenvector matrix before applying k-means, as is done in a normalized spectral-clustering variant. Rather than treating these steps simply as an algorithmic recipe, focus on building an intuitive understanding of how and why it works.

We have an undirected graph with m nodes, and our goal is to divide it into k clusters. Spectral clustering does this in the following steps:

1. Compute the graph's Laplacian matrix $L \in \mathbb{R}^{m \times m}$.
2. Find k unit eigenvectors $v_1, v_2, \ldots, v_k$ of L, corresponding to the k smallest eigenvalues. Form the matrix

$$Y = \begin{bmatrix} v_1 & v_2 & \cdots & v_k \end{bmatrix} \in \mathbb{R}^{m \times k}.$$

3. Normalize each row of Y so that it has unit 2-norm.
4. Treat the m rows of Y as points in $\mathbb{R}^k$ and cluster them using the k-means algorithm (described in Section 14.3).
5. Assign the i-th graph node to cluster j if the i-th row of Y belongs to cluster j.

Example 44.7 Let us revisit the graph in Example 44.6. The three smallest eigenvalues of its graph Laplacian are approximately 0, 0.2087, and 0.6972. Next, we find three unit eigenvectors corresponding to these eigenvalues and use them to form the matrix Y, where each column is one of the eigenvectors. Finally, we normalize the rows of Y to have unit 2-norm, resulting in the matrix Y_{norm}. Rounded to two decimal places, we have

$$Y = \begin{bmatrix} 0.33 & -0.43 & -0.29 \\ 0.33 & -0.43 & -0.29 \\ 0.33 & -0.34 & -0.09 \\ 0.33 & -0.09 & 0.38 \\ 0.33 & 0.00 & 0.59 \\ 0.33 & 0.09 & 0.38 \\ 0.33 & 0.34 & -0.09 \\ 0.33 & 0.43 & -0.29 \\ 0.33 & 0.43 & -0.29 \end{bmatrix} \quad \text{and} \quad Y_{\text{norm}} = \begin{bmatrix} 0.54 & -0.70 & -0.47 \\ 0.54 & -0.70 & -0.47 \\ 0.69 & -0.70 & -0.18 \\ 0.65 & -0.18 & 0.74 \\ 0.49 & 0.00 & 0.87 \\ 0.65 & 0.18 & 0.74 \\ 0.69 & 0.70 & -0.18 \\ 0.53 & 0.70 & -0.47 \\ 0.53 & 0.70 & -0.47 \end{bmatrix}. \tag{44.4}$$

We then apply the k-means algorithm with $k = 3$ to the rows of Y_{norm}. This gives us three clusters: $\{0, 1, 2\}$, $\{3, 4, 5\}$, and $\{6, 7, 8\}$. Thus, the first three nodes form one cluster, the next three form a second, and the last three form a third. This result agrees with what we can intuitively observe from the graph's structure in Figure 44.7.

Let us see how and why spectral clustering works. Suppose our graph has k clusters, with sizes $m_1, m_2, \ldots, m_k$ and $m_1 + \cdots + m_k = m$. To keep things simple, we will assume the nodes are ordered by cluster: the first m_1 nodes are in cluster 1, the next m_2 are in cluster 2, and so on.

To understand spectral clustering, it helps to start with an ideal case: a graph where the clusters are completely disconnected from each other. In this scenario, the graph has k connected components (one for each cluster), with no edges between them. The Laplacian matrix of such a graph, L_{ideal}, is block diagonal:

$$L_{\text{ideal}} = \text{diag}(L_1, L_2, \ldots, L_k),$$

where each block $L_i \in \mathbb{R}^{m_i \times m_i}$ is the Laplacian of the graph for cluster i. Since the graph has k connected components, the Laplacian L_{ideal} has k orthonormal eigenvectors corresponding to the eigenvalue 0 (Theorem 42.50). For any single connected component, the vector of all ones, $e = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}^T$, is an eigenvector with eigenvalue 0.

If we let e_{m_i} denote the vector of ones of length m_i for cluster i, the matrix containing the k eigenvectors corresponding to the k smallest eigenvalues (the zero eigenvalues) is

$$Y_{\text{ideal}} = \begin{bmatrix} \frac{e_{m_1}}{\sqrt{m_1}} & 0 & \cdots & 0 \\ 0 & \frac{e_{m_2}}{\sqrt{m_2}} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{e_{m_k}}{\sqrt{m_k}} \end{bmatrix} \in \mathbb{R}^{m \times k}.$$

Now, let us consider the rows of Y_{ideal} as points in $\mathbb{R}^k$. The first m_1 rows are identical and equal to $\left[1/\sqrt{m_1}\ 0 \cdots 0\right]$, representing cluster 1. Similarly, the next m_2 rows are all equal to $\left[0\ 1/\sqrt{m_2}\ 0 \cdots 0\right]$, representing cluster 2, and so on. This gives us k distinct points in $\mathbb{R}^k$, each corresponding to a cluster. These points are orthogonal, so clustering them with k-means is straightforward: the first m_1 nodes form one cluster, the next m_2 form another, and so on. So, in this ideal case, spectral clustering works perfectly to identify the clusters.

In general, the clusters are not completely disconnected, since some nodes belonging to different clusters may be connected. Let L_i denote the Laplacian matrix of the subgraph induced by cluster i, considering only edges whose two endpoints belong to that cluster. Also, let D_i^{out} be the diagonal matrix whose diagonal entries give the degrees contributed by edges connecting nodes in cluster i to nodes in other clusters. Then, after ordering the nodes according to their clusters, we can write the graph Laplacian as

$$L = \begin{bmatrix} L_1 + D_1^{\text{out}} & L_{12} & \cdots & L_{1k} \\ L_{21} & L_2 + D_2^{\text{out}} & \cdots & L_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ L_{k1} & L_{k2} & \cdots & L_k + D_k^{\text{out}} \end{bmatrix}.$$

Here, the off-diagonal blocks L_{ij} contain the contributions from connections between clusters i and j.

If there were no connections between different clusters, the Laplacian would instead be

$$L_{\text{ideal}} = \begin{bmatrix} L_1 & 0 & \cdots & 0 \\ 0 & L_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & L_k \end{bmatrix}.$$

Therefore,

$$L = L_{\text{ideal}} + E,$$

where

$$E = \begin{bmatrix} D_1^{\text{out}} & L_{12} & \cdots & L_{1k} \\ L_{21} & D_2^{\text{out}} & \cdots & L_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ L_{k1} & L_{k2} & \cdots & D_k^{\text{out}} \end{bmatrix}.$$

Thus, E captures all the effects of the connections between different clusters: both the off-diagonal entries corresponding to these connections and their contributions to the node degrees.

If the connections between different clusters are sufficiently weak, then E can be regarded as a small perturbation of L_{ideal}. More precisely, suppose that each cluster is connected, so that L_{ideal} has exactly k zero eigenvalues, and let $\gamma > 0$ denote its smallest positive eigenvalue. Standard perturbation results for symmetric matrices imply that, when $\|E\|_2$ is small relative to γ, the subspace spanned by the k eigenvectors of L corresponding to its k smallest eigenvalues is close to the nullspace of L_{ideal}.

Consequently, after a suitable orthogonal change of basis, there exists an orthogonal matrix $Q \in \mathbb{R}^{k\times k}$ such that

$$Y \approx Y_{\text{ideal}} Q = \begin{bmatrix} \frac{e_{m_1}}{\sqrt{m_1}} & 0 & 0 & \cdots & 0 \\ 0 & \frac{e_{m_2}}{\sqrt{m_2}} & 0 & \cdots & 0 \\ 0 & 0 & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \frac{e_{m_{k-1}}}{\sqrt{m_{k-1}}} & 0 \\ 0 & 0 & \cdots & 0 & \frac{e_{m_k}}{\sqrt{m_k}} \end{bmatrix} Q.$$

The matrix Q accounts for the fact that the eigenvectors computed from L need not align individually with the particular orthonormal basis chosen for the nullspace of L_{ideal}.

This implies that the first m_1 rows of Y are close to the point $\frac{1}{\sqrt{m_1}} e_1^T Q \in \mathbb{R}^k$, the next m_2 rows are close to $\frac{1}{\sqrt{m_2}} e_2^T Q \in \mathbb{R}^k$, and so on. Furthermore, these k sets of points are orthogonal because

$$e_i^T Q \left(e_j^T Q\right)^T = e_i^T Q Q^T e_j = e_i^T e_j = 0, \quad \text{when } i \neq j.$$

After the normalization step, the rows of Y_{norm} will form k tight clusters on the unit sphere, with approximately 90° separation between them.

Example 44.8 Returning to the graph in Example 44.6, Figure 44.9 displays a plot of the rows of the matrix Y_{norm} from (44.4). The blue dots represent these rows, while the black dot marks the origin. The dashed lines indicate the directions around which the three clusters are grouped. These directions look orthogonal, as expected based on our discussion.

If we have a numerical dataset given as a matrix X instead of a graph G, we can first turn the data into a graph using one of the methods from Section 13.4. Once we have the graph, we can use spectral clustering to find clusters. Spectral clustering on numerical datasets is particularly useful in cases where k-means struggles. Let us see some examples.

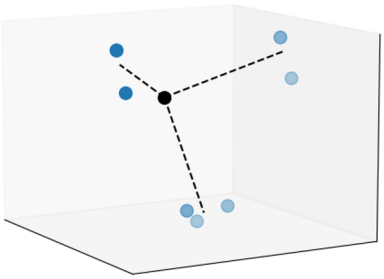


Fig. 44.9: Visualization of the rows of Y_{norm} as blue dots, where each group corresponds to a cluster in the original graph. The origin is marked by a black dot, and the dashed lines represent the directions of the clusters. The groups are orthogonally placed with respect to each other, making them easily separable by the k-means algorithm.

Example 44.9. (Spectral clustering for numerical datasets)

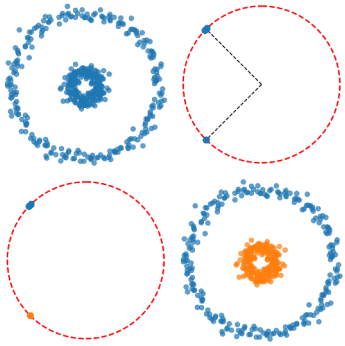
Figure 44.10 illustrates two examples where spectral clustering successfully identifies clusters. In both cases, the top-left panels show the original datasets: one with two clusters (an inner and an outer circle) in Figure 44.10a, and the other with three elongated clusters in Figure 44.10b. The top-right panels display the normalized rows of the matrix Y, and the bottom-left panels show the clusters identified by applying the k-means algorithm to these rows (different colors represent distinct clusters). Finally, the bottom-right panels reveal the assigned clusters for the original data points. Spectral clustering successfully separates the clusters in both datasets, even though k-means would fail in these cases.

44.2 Dimensionality reduction methods

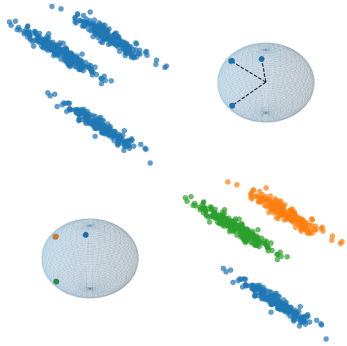
In the previous section, we discussed algorithms for clustering. Next, we introduce several methods for dimensionality reduction. We will begin with two techniques used in classification problems, and then cover a method for reducing dimensions in graphs.

44.2.1 Feature selection via class-separation variance

Recall that in a classification problem, we have a data matrix $X \in \mathbb{R}^{m \times n}$ and a target vector $y \in \mathbb{R}^m$. The target vector y gives the class or label for each row of X. The goal is to use the data in X to predict the labels in y so that we can use the resulting algorithm to classify data not in our data set.



(a) A dataset with two clusters (the inner and outer circles).



(b) A dataset with three elongated clusters.

Fig. 44.10: We apply spectral clustering to two datasets in $\mathbb{R}^2$: two concentric rings (panel a) and three elongated blobs (panel b). For each panel, we show four plots: Top-left: Original dataset. Top-right: Normalized rows of the eigenvector matrix Y. Bottom-left: Clustering of the normalized rows of Y using the k-means algorithm. Bottom-right: Final cluster assignments for the original data points.

Sometimes our data has many features, so X has many columns. This can make it harder to see how the classification method makes predictions. To address this issue, we try to reduce the number of features and work in a lower dimensional feature space.

So how can we reduce the number of features in a dataset? The first method we introduce evaluates each feature on its own and assigns it an *importance score*. A higher score means the feature is more useful for predicting the label. We keep only the features with the highest scores and use them for prediction.

The score relies on the ideas of between-class scatter and within-class scatter, which we define next.

Definition 44.10. (Between- and within-class scatter)

Let $X \in \mathbb{R}^{m\times n}$ be a data matrix, with rows as data points and columns as features. Each point has a label $y \in \{1,\dots,k\}$, so there are k different classes. For each feature $j \in \{1,\dots,n\}$ and class $\ell \in \{1,\dots,k\}$, define

$$\mu_{\ell j} := \frac{1}{m_\ell} \sum_{\{i:\, y_i=\ell\}} x_{ij}, \qquad \text{and} \qquad \mu_j := \frac{1}{m}\sum_{i=1}^{m} x_{ij},$$

where x_{ij} is the value of feature j for point i (that is, the (i,j) entry of the data matrix X), and m_ℓ is the number of points in class ℓ (so $m_1 + \cdots + m_k = m$).

The quantity $\mu_{\ell j}$ is the average value of feature j among data points belonging to class ℓ. The quantity μ_j is the overall mean of feature j, that is, it is the average value of that feature across the entire dataset.

For each feature j we first measure how tightly its values cluster *within* each class.

The **within-class variance** for feature j is

$$S_W(j) := \sum_{\ell=1}^{k} \sum_{\{i:\, y_i=\ell\}} (x_{ij} - \mu_{\ell j})^2.$$

For each class ℓ, the inner sum measures how far the values of feature j deviate from their class mean $\mu_{\ell j}$. A small $S_W(j)$ means feature j varies little within each class.

Next we measure how far the class means lie from the global mean. The **between-class variance** is

$$S_B(j) := \sum_{\ell=1}^{k} m_\ell \, (\mu_{\ell j} - \mu_j)^2.$$

It compares each class mean $\mu_{\ell j}$ with the overall mean μ_j, weighting by the class size m_ℓ. A large $S_B(j)$ means the class averages are well separated along feature j.

Example 44.11 Consider the dataset in Table 44.1 with two features, six data points, and two classes.

Point	Feature 1	Feature 2	Class
A	2	3	1
B	3	4	1
C	4	2	1
D	7	5	2
E	8	6	2
F	9	5	2

Table 44.1: Toy dataset for feature selection: two features, six points, two classes.

The overall means of the two features are

$$\mu_1 = \frac{2+3+4+7+8+9}{6} = \frac{11}{2}, \qquad \mu_2 = \frac{3+4+2+5+6+5}{6} = \frac{25}{6}.$$

The class means are

$$\mu_{11} = \frac{2+3+4}{3} = 3, \qquad \mu_{21} = \frac{7+8+9}{3} = 8,$$
$$\mu_{12} = \frac{3+4+2}{3} = 3, \qquad \mu_{22} = \frac{5+6+5}{3} = \frac{16}{3}.$$

The between-class variance for features 1 and 2 is

$$S_B(1) = 3(\mu_{11} - \mu_1)^2 + 3(\mu_{21} - \mu_1)^2 = 3\left(3 - \tfrac{11}{2}\right)^2 + 3\left(8 - \tfrac{11}{2}\right)^2 = \tfrac{75}{2},$$

$$S_B(2) = 3(\mu_{12} - \mu_2)^2 + 3(\mu_{22} - \mu_2)^2 = 3\left(3 - \tfrac{25}{6}\right)^2 + 3\left(\tfrac{16}{3} - \tfrac{25}{6}\right)^2 = \tfrac{49}{6}.$$

The within-class variance for features 1 and 2 is

$$S_W(1) = (x_{11} - \mu_{11})^2 + (x_{21} - \mu_{11})^2 + (x_{31} - \mu_{11})^2 + (x_{41} - \mu_{21})^2 + (x_{51} - \mu_{21})^2 + (x_{61} - \mu_{21})^2 = 4,$$

$$S_W(2) = (x_{12} - \mu_{12})^2 + (x_{22} - \mu_{12})^2 + (x_{32} - \mu_{12})^2 + (x_{42} - \mu_{22})^2 + (x_{52} - \mu_{22})^2 + (x_{62} - \mu_{22})^2 = \frac{8}{3}.$$

The previous example shows how to compute the within-class and between-class variances for labeled data, but it does not give much intuition about what those numbers actually mean. Figure 44.11 shows a graphical representation of the within-class and between-class variances for three one-dimensional features with three classes. Each value is colored by its class label, and the class means are μ_{1j}, μ_{2j}, and μ_{3j}, $j = 1, 2, 3$. The top panel shows feature values with low within-class variance and high between-class variance. Points in each class lie close to their class mean, and the three class means are well separated. The middle panel shows high within-class variance and high between-class variance. The class means are far apart, but the classes are spread out and overlap. The bottom panel shows low within-class variance and low between-class variance. Each class is tight, but the class means are close together.

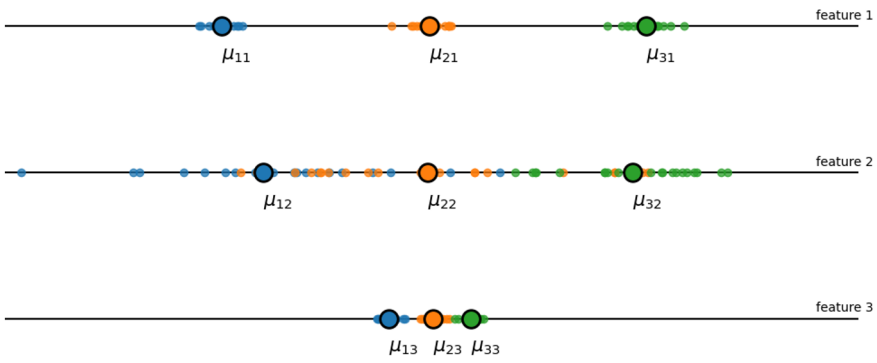


Fig. 44.11: Within- and between-class variances for three one-dimensional features with three classes. Top: low within-class, high between-class. Middle: high within-class, high between-class. Bottom: low within-class, low between-class.

If these three panels were three features in your dataset, which would you use to predict the class? Probably the top one. In the middle panel, the classes overlap, so the feature does not always predict the label. In the bottom panel, each class is tight, but the class means are close. A small change in the value can flip the label,

so the prediction is not reliable. A feature is most useful for classification when its within-class variance is small and its between-class variance is large.

We now formalize these ideas. Using the between-class and within-class variance, we define an **importance score** for each feature.

Definition 44.12. (Fisher and F-ANOVA scores)

Let $X \in \mathbb{R}^{m\times n}$ be a dataset with m data points, n features, and k classes. For each feature $j \in \{1,\dots,n\}$, its **Fisher score** is

$$S_j := \frac{S_B(j)}{S_W(j)},$$

and its **F-ANOVA score** is

$$F_j := \frac{S_B(j)/(k-1)}{S_W(j)/(m-k)} = S_j\,\frac{m-k}{k-1},$$

where $S_B(j)$ and $S_W(j)$ are the between-class and within-class variance of feature j.

A higher Fisher or F-ANOVA score means the feature separates the classes well and stays tight within each class. When m and k are fixed, F_j is a constant multiple of S_j, so both scores rank features the same way. The F-ANOVA score has a statistical interpretation, but we do not use it here.

Example 44.13 Let us revisit Example 44.11. The Fisher scores are

$$S_1 = \frac{75}{8} \approx 9.375, \qquad S_2 = \frac{49}{16} \approx 3.06.$$

The F-ANOVA scores are

$$F_1 = \frac{m-k}{k-1}\,S_1 = 4\cdot 9.375 = 37.5, \qquad F_2 = 4\cdot 3.0625 = 12.25.$$

Both scores rank feature 1 above feature 2, so feature 1 is the better choice for classification.

To close this section, let us look at a toy example from a real-world application

Example 44.14 Consider the data of five patients in a hospital in Table 44.2. For each patient, we record the following features:

1. **HbA1c (Hemoglobin A1c)**: a blood test that measures average blood sugar over the past 2 to 3 months.

2. **BMI (Body Mass Index)**: a number from height and weight that estimates body fat. Used to classify underweight, normal weight, overweight, or obese.
3. **Resting HR (Resting Heart Rate)**: beats per minute at rest. Lower values usually indicate better cardiovascular fitness.

Patient	Class	HbA1c	BMI	Resting HR
P_1	none	5.1	22	68
P_2	diab	9.0	35	82
P_3	pre	6.2	25	72
P_4	none	5.9	24	70
P_5	diab	8.1	30	78

Table 44.2: Patient data. Classes: none = no diabetes, pre = prediabetes, diab = diabetes.

Each patient also has a label: diabetes, prediabetes, or none. Let us compute the Fisher score for each of the three features to rank their relevance in predicting the label.

For feature HbA1c, we get

$$S_B \approx 18.6, \quad S_W \approx 0.32, \quad \text{Score(HbA1c)} \approx 58.$$

Similarly, for the BMI and Resting HR features, we obtain

$$\text{Score(BMI)} \approx 46 \quad \text{and} \quad \text{Score(Resting HR)} \approx 9.$$

These scores give the ranking

$$\text{HbA1c} > \text{BMI} \gg \text{Resting HR}.$$

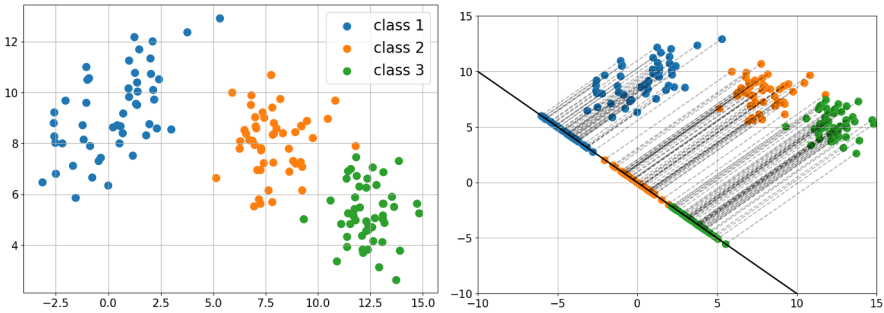
Thus, the levels of HbA1c and BMI are much more relevant than the resting heart rate for predicting a patient's diabetes status: no diabetes, prediabetes, or diabetes.

44.2.2 Linear discriminant analysis

Linear Discriminant Analysis (LDA), introduced by Ronald A. Fisher [23], is a supervised method for dimensionality reduction. It projects a multi-class dataset onto a lower-dimensional space, aiming to keep the classes well separated. LDA and the Fisher score are closely related. The Fisher score looks at each feature on its own, while LDA searches for combinations of features that maximize the Fisher score.

Suppose we have m data points in n dimensions, divided into k classes. Our goal is to reduce the data from n dimensions down to 1. We will discuss how to reduce to more than one dimension at the end of this section.

For example, consider the toy dataset on the left side of Figure 44.12, where $n = 2$ and $k = 3$. Each point is colored by its class label: 0, 1, or 2. You can see that points from the same class tend to form tight clusters.



(a) A toy dataset with three classes in $\mathbb{R}^2$.

(b) Data projected onto a line using LDA.

Fig. 44.12: Left: A toy dataset with three classes in $\mathbb{R}^2$, where each point is colored based on its class label. Right: The same dataset projected onto a line.

Our goal is to reduce this dataset from 2 dimensions to 1. In other words, we want to project the data points in $\mathbb{R}^2$ onto a line defined by a unit vector v, as shown on the right side of Figure 44.12.

We want each class to be tightly clustered after projection, and we want the class means to be as far apart as possible. To achieve this, LDA finds the vector v that maximizes the ratio of between-class variance to within-class variance for the projected data.

Before we go into the details, let us recall how to project a vector x onto a line defined by a unit vector v.

For any unit vector v, we have

$$\mathbb{R}^n = \text{span}(v) \oplus \text{span}(v)^\perp,$$

where $\text{span}(v)^\perp$ is the orthogonal complement of $\text{span}(v)$. Let $\{q_1, \ldots, q_{n-1}\}$ be an orthonormal basis for $\text{span}(v)^\perp$. Then, for any vector $x \in \mathbb{R}^n$, we can write

$$x = \alpha v + \sum_{i=1}^{n-1} \beta_i q_i$$

for some scalars $\alpha, \beta_1, \ldots, \beta_{n-1}$. Taking the dot product with v, we get

$$v^T x = \alpha v^T v,$$

since $v^T q_i = 0$ for all $i = 1, \ldots, n-1$. As v is a unit vector, $v^T v = \|v\|_2^2 = 1$, so we get $\alpha = v^T x$. Thus, the orthogonal projection of x onto $\text{span}(v)$ is given by $(v^T x)v$.

With an appropriate rotation, the points on span(v) can be mapped to the real line $\mathbb{R}$ by identifying $x = \alpha v$ with the scalar α. Thus, the projected point $(v^T x)v$ on span(v) corresponds to the scalar $v^T x$ on $\mathbb{R}$, as shown in Figure 44.13.

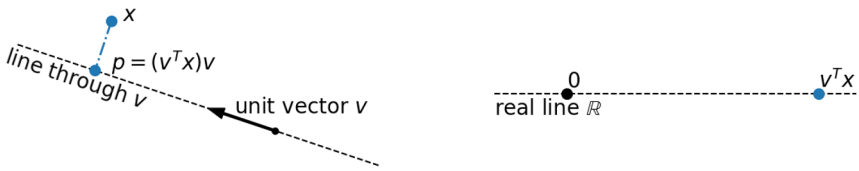


Fig. 44.13: Projection of points onto span(v) and their mapping to $\mathbb{R}$.

Now we are ready to compute the between-class variance and within-class variance we introduced in Section 44.2.1 for the projected data points.

Let us start with the mean vector and the class mean vectors:

$$M = \frac{1}{m}\sum_{i=1}^{m} x_i \quad \text{and} \quad M_\ell = \frac{1}{m_\ell}\sum_{\{i\,:\,y_i=\ell\}} x_i, \quad \text{for } \ell = 1, \ldots, k,$$

where m_ℓ is the number of points with label ℓ, $m = m_1 + \cdots + m_k$ is the total number of points, and k is the number of classes (for example, $k = 3$ in our case). The entries of M are the mean values of each feature for all data points. The entries of M_ℓ are the mean values of each feature for points with label ℓ.

Let us fix a unit vector v. When we project the data points onto span(v), we treat the projected points $(v^T x)v$ as the real numbers $v^T x$. The means of the projected points are

$$\mu = \frac{1}{m}\sum_{i=1}^{m} v^T x_i = v^T M \qquad \text{and} \qquad \mu_\ell = \frac{1}{m_\ell}\sum_{\{i\,:\,y_i=\ell\}} v^T x_i = v^T M_\ell.$$

In other words, the mean values of the scalars $v^T x_i$ are just the projections of the original means.

Now we can compute the between-class and within-class variances for the scalars $v^T x_i$ (see Definition 44.10).

The *between-class variance* is

$$\text{between-class variance} := \sum_{\ell=1}^{k} m_\ell\,(\mu_\ell - \mu)^2,$$

which measures how far the projected class means μ_ℓ are from the overall projected mean μ.

We can also write the between-class variance in matrix form as

$$\sum_{\ell=1}^{k} m_\ell\,(\mu_\ell-\mu)^2=\sum_{\ell=1}^{k} m_\ell\,(v^TM_\ell-v^TM)^2=\sum_{\ell=1}^{k} m_\ell\,(v^T(M_\ell-M))^2$$
$$=\sum_{\ell=1}^{k} m_\ell\,v^T(M_\ell-M)(M_\ell-M)^Tv$$
$$=v^T\left(\sum_{\ell=1}^{k} m_\ell\,(M_\ell-M)(M_\ell-M)^T\right)v=v^TS_B\,v,$$

where $S_B:=\sum_{\ell=1}^{k} m_\ell\,(M_\ell-M)(M_\ell-M)^T$ is called the **between-class scatter matrix**.

The *within-class variance* is

$$\text{within-class variance}:=\sum_{\ell=1}^{k}\sum_{\{i\,:\,y_i=\ell\}}(v^Tx_i-\mu_\ell)^2.$$

We can also write this in matrix form as

$$\sum_{\ell=1}^{k}\sum_{\{i\,:\,y_i=\ell\}}(v^Tx_i-\mu_\ell)^2=\sum_{\ell=1}^{k}\sum_{\{i\,:\,y_i=\ell\}}(v^Tx_i-v^TM_\ell)^2$$
$$=\sum_{\ell=1}^{k}\sum_{\{i\,:\,y_i=\ell\}}v^T(x_i-M_\ell)(x_i-M_\ell)^Tv$$
$$=v^T\left(\sum_{\ell=1}^{k}\sum_{\{i\,:\,y_i=\ell\}}(x_i-M_\ell)(x_i-M_\ell)^T\right)v=v^TS_W\,v,$$

where $S_W:=\sum_{\ell=1}^{k}\sum_{\{i\,:\,y_i=\ell\}}(x_i-M_\ell)(x_i-M_\ell)^T$ is called the **within-class scatter matrix**.

Figure 44.14 shows the ideas of within-class and between-class variance for the projected data. The between-class variance measures how far apart the class means are. The within-class variance measures how spread out the data points are within each class.

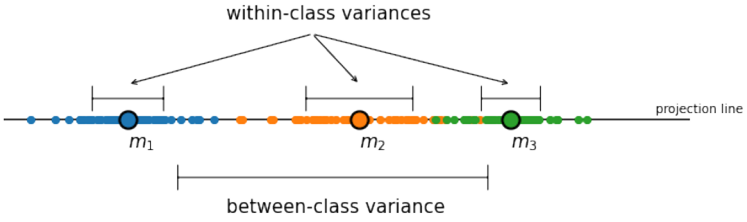


Fig. 44.14: Within-class and between-class variances of the projected dataset.

LDA looks for the direction that separates the class means as much as possible while keeping the points in each class close together. More precisely, it finds a vector $v \in \mathbb{R}^n$ that maximizes the *Fisher score* of the projected data. In other words, LDA solves the following optimization problem:

$$\max_{v \in \mathbb{R}^n} \frac{\text{between-class variance}}{\text{within-class variance}} = \max_{v \in \mathbb{R}^n} \frac{v^T S_B\, v}{v^T S_W\, v} = \max_{v^T S_W\, v = 1} v^T S_B\, v. \tag{44.5}$$

(For why these last two problems are the same, see Problem 44.9.) Both S_B and S_W are symmetric matrices. Moreover, S_W is always positive semidefinite because it is a Gram matrix. However, it is not necessarily positive definite. The following theorem characterizes when S_W is positive definite.

Theorem 44.15 *The within-class scatter matrix S_W is positive definite if and only if there is no nonzero direction v such that, after centering each class by its class mean, every class has zero variance in the direction v.*

Proof For any vector $v \in \mathbb{R}^n$, we have

$$\begin{aligned} v^T S_W v &= v^T \left(\sum_{\ell=1}^{k} \sum_{\{i : y_i = \ell\}} (x_i - M_\ell)(x_i - M_\ell)^T \right) v \\ &= \sum_{\ell=1}^{k} \sum_{\{i : y_i = \ell\}} v^T (x_i - M_\ell)(x_i - M_\ell)^T v \\ &= \sum_{\ell=1}^{k} \sum_{\{i : y_i = \ell\}} \left(v^T (x_i - M_\ell) \right)^2. \end{aligned}$$

Therefore $v^T S_W v \geqslant 0$ for every v, so S_W is positive semidefinite.

Moreover,

$$v^T S_W v = 0$$

if and only if

$$v^T (x_i - M_{y_i}) = 0$$

for every data point x_i. This means that, after centering each class by its own class mean, all centered data points have zero projection onto v. Equivalently, every class has zero variance in the direction v.

Thus $v^T S_W v > 0$ for every nonzero vector v if and only if there is no nonzero direction v along which every class has zero within-class variance. Therefore S_W is positive definite if and only if the within-class centered data have nonzero variance in every nonzero direction. □

The previous theorem shows that S_W is singular if and only if there is a direction in which the centered data have no variation. When the number of features is small compared with the number of data points, this is typically not the case. However, if the ambient dimension is too large, then such a direction must exist, regardless of the data.

Indeed, suppose that $n > m - k$. For each class ℓ, the centered vectors $x_i - M_\ell$ sum to zero:

$$\sum_{\{i:y_i=\ell\}} (x_i - M_\ell) = 0.$$

Therefore, the m_ℓ centered vectors from class ℓ span a subspace of dimension at most $m_\ell - 1$. Hence all within-class centered data vectors together span a subspace of dimension at most

$$\sum_{\ell=1}^{k} (m_\ell - 1) = m - k.$$

Thus

$$\operatorname{rank}(S_W) \leqslant m - k.$$

If $n > m - k$, then $\operatorname{rank}(S_W) < n$, so S_W is necessarily singular.

The optimization problem in (44.5) asks us to maximize a Rayleigh quotient, but instead of considering vectors on the unit sphere, we use vectors with $v^T S_W v = 1$. When S_W is positive definite, we can solve this optimization problem using a change of variables and the Rayleigh–Ritz theorem, as explained in Chapter 43. The maximum $\max_{v^T S_W v=1} v^T S_B v$ is reached when v is an eigenvector of $S_W^{-1} S_B$ with the largest eigenvalue. Equivalently, this happens when $v = S_W^{-1/2} y$, where y is a unit eigenvector of the symmetric matrix $S_W^{-1/2} S_B S_W^{-1/2}$ corresponding to its largest eigenvalue. The matrix $S_W^{-1/2}$ denotes the inverse of the positive definite square root of S_W.

Next, we show an example where we use LDA to reduce the data from two dimensions to one.

Example 44.16. (Online shopping)

You have launched an online clothing business. To help customers choose the right size without trying on clothes, you decide to use Machine Learning to recommend sizes based on their weight (in kilograms) and height (in centimeters). Table 44.3 shows data collected from previous customers.

Weight	Height	Size
52	155	S
50	160	S
55	160	S
58	165	S
60	160	M
65	166	M
70	168	M
68	171	M
75	175	L
75	180	L
80	187	L
90	190	L

Table 44.3: Customer data: weight, height, and recommended size.

The data is divided into three classes: S (Small), M (Medium), and L (Large). Predicting the right size for a new customer is a classification problem. To make this process simpler, we aim to find a simple formula based on weight and height to help predict the size.

We will create this formula by reducing the dataset from two dimensions to one. To do this, we will use LDA to project the data onto a line. This projection aims to clearly separate the three classes while keeping the points from each class close together.

The projection vector obtained from LDA, rounded to two decimal places, is

$$v = \begin{bmatrix} -0.99 \\ 0.11 \end{bmatrix}.$$

This corresponds to orthogonally projecting the data onto the line

$$\text{height} = -\frac{0.11}{0.99}\,\text{weight}.$$

To reduce the dimensionality from 2D to 1D, each data point is replaced by its projection coefficient on this line. For a data point (w, h), the transformation is given by

$$\begin{bmatrix} -0.99 & 0.11 \end{bmatrix} \begin{bmatrix} w \\ h \end{bmatrix} = -0.99w + 0.11h. \tag{44.6}$$

The left panel of Figure 44.15 shows the original two-dimensional data, while the right panel displays the data after projection. We can use the linear combination of "height" and "weight" from LDA (44.6) to predict the "size" variable in this dataset. For a new data point, we can apply any classification tool to this transformed representation. Even simple cutoff rules might work well.

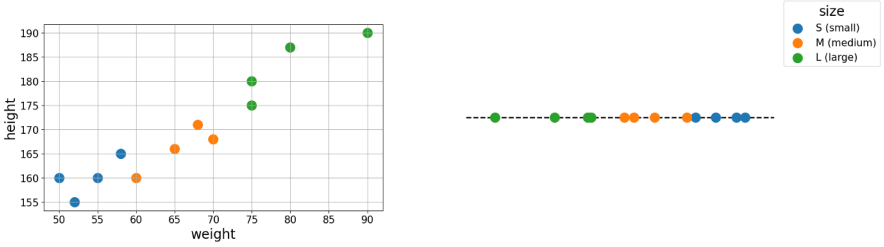


Fig. 44.15: Online shopping data (left), and its LDA projection (right).

If our data is in $\mathbb{R}^n$ and we want to project it onto a k-dimensional space (with $k < n$), we use the k eigenvectors $v_1, v_2, \ldots, v_k$ of $S_W^{-1} S_B$ with the largest eigenvalues. These directions maximize the ratio of between-class to within-class variance, in order from best to next best, and so on. We then project our data onto each of these directions to get the new lower-dimensional representation.

We illustrate this idea in the next example.

Example 44.17. (Visualizing iris flowers with LDA)

For this example, we will use the classic Iris flowers dataset.[1] It has measurements for 150 iris flowers from three species: setosa, versicolor, and virginica. Each flower is described by four features: *sepal length*, *sepal width*, *petal length*, and *petal width*.

Since the dataset is four-dimensional, we cannot visualize it directly. So, we will use LDA to reduce it to two dimensions. This lets us plot the data and see how the classes separate. Figure 44.16 shows the result. The three species form different regions in the plot. Setosas are clearly separated, while versicolors and virginicas have some overlap.

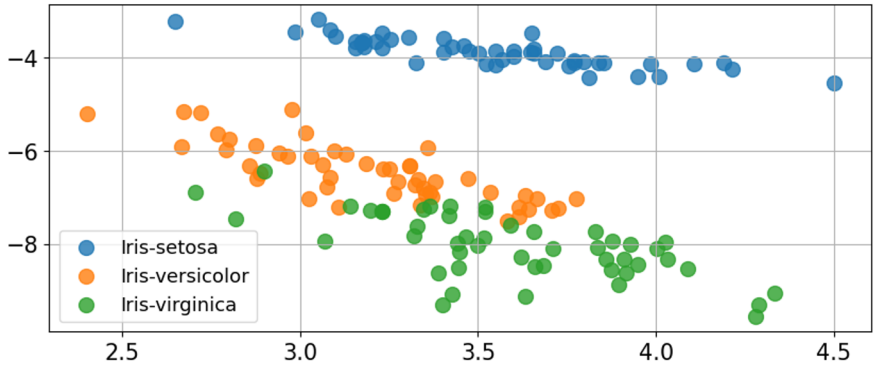


Fig. 44.16: Two-dimensional LDA projection of the Iris dataset.

The two directions that maximize the ratio of between-class to within-class variance found by LDA are, rounded to two decimal places,

$$v_1 = \begin{bmatrix} 0.24 \\ 0.74 \\ -0.25 \\ 0.57 \end{bmatrix} \quad \text{and} \quad v_2 = \begin{bmatrix} 0.29 \\ 0.37 \\ 0.72 \\ 0.51 \end{bmatrix}.$$

These give the following two linear combinations of features:

$$0.24 \cdot \text{sepal length} + 0.74 \cdot \text{sepal width} - 0.25 \cdot \text{petal length} + 0.57 \cdot \text{petal width},$$

and

$$0.29 \cdot \text{sepal length} + 0.37 \cdot \text{sepal width} + 0.72 \cdot \text{petal length} + 0.51 \cdot \text{petal width}.$$

We can use these combinations to help discriminate between the iris species.

[1] This dataset was first introduced by Fisher in [23]. It is one of the most famous datasets in machine learning and is easy to find online.

When you have labeled data with many features, choosing the right way to reduce dimensionality can really help your model. We have seen two common strategies: Linear Discriminant Analysis (LDA) and feature selection methods like the Fisher score or ANOVA F-statistic.

Fisher score vs LDA

LDA works by creating new features from combinations of your original ones, using the class labels to find directions that best separate the different classes. This approach is most helpful when you want to reduce a large number of features down to just a few, and your main goal is to improve class separation, not necessarily to know which original features matter most. Keep in mind that the new features from LDA are combinations of the originals, which can make interpretation more difficult.

On the other hand, feature selection methods such as the Fisher score focus on ranking your original features based on how well each one separates the classes. This way, you can keep only the most useful features, which makes your model simpler and easier to interpret, since you can see exactly which features are important. However, feature selection looks at each feature individually, so it might miss useful combinations that work well together.

For large datasets, it is common to use both methods: start with feature selection to remove features that are not helpful, then use LDA on the remaining ones. This way, you keep things simple and let LDA work with the most important features.

A few example scenarios can help make this clearer.

- **Gene expression data:** If you have gene expression data with thousands of genes, you might first use feature selection to filter down to the top 200 genes. After that, you can use LDA to further reduce the data and, for example, plot different cancer types in two dimensions.
- **Short survey:** For a short survey with only 20 questions, you probably do not need to drop any features. You can run LDA directly to find combinations of answers that best separate groups, such as "satisfied" versus "not satisfied" respondents.
- **Wearable fitness tracker:** Consider a wearable fitness tracker that records your heart rate every minute. This gives you 1,440 features (one for each minute in a day) for each data point. If you want to predict sleep quality, you can use LDA to find combinations of time periods that relate to how well you sleep. This might reveal, for example, that your afternoon activity or how relaxed you are in the morning has a big effect on your sleep.

44.2.3 Spectral embedding

Let us consider a weighted (or unweighted, where edges have weight 1 and non-edges have weight 0), undirected graph $G = (V, E, w)$ with n nodes labeled $1, 2, \ldots, n$. The main idea behind **spectral embedding** is to map the graph into $\mathbb{R}^k$, usually with small k. In this embedding, each node $i \in V$ is assigned a vector $y_i \in \mathbb{R}^k$. When $k = 2$ or $k = 3$, this makes it possible to visualize the graph.

The embedding should preserve the structure of the graph: if nodes i and j are connected by an edge with large weight w_{ij}, their vectors y_i and y_j in the embedding space should be close together. One way to achieve this is by minimizing

$$\sum_{i,j=1}^{n} w_{ij}\|y_i - y_j\|_2^2, \tag{44.7}$$

where $w_{ij} \geqslant 0$ are the edge weights.

We can rewrite (44.7) in matrix form. Let Y be an $n \times k$ matrix where the i-th row is y_i^T. Then,

$$\sum_{i,j=1}^{n} w_{ij}\|y_i - y_j\|_2^2 = 2\mathrm{trace}(Y^T LY),$$

where L is the Laplacian matrix of G.

If we minimize $\mathrm{trace}(Y^T LY)$ without any restrictions, we end up with the trivial solution $Y = 0$, which does not tell us anything about the graph. To avoid this, we need to add some constraints.

A common approach is to require the columns of Y to be orthogonal, so $Y^T Y = I_k$, and also to make them orthogonal to the all-ones vector e, so $e^T Y = 0$.

Why do we use these constraints? The first one, $Y^T Y = I_k$, makes sure that if we think of each column of Y as a different feature of the graph, these features are as linearly independent as possible. The second, $e^T Y = 0$, prevents any column from being almost constant, which would make it useless for distinguishing nodes. These constraints also make it possible to solve the problem using standard linear algebra techniques.

With these constraints, the optimization problem is

$$\min_{\substack{Y\in\mathbb{R}^{n\times k}\\ Y^TY=I_k,\ e^TY=0}} \mathrm{trace}(Y^T LY). \tag{44.8}$$

How do we solve this problem? Let us rewrite it in a more familiar form. Let $c_1, \ldots, c_k$ denote the k columns of Y. Since the trace is the sum of the diagonal entries, we can rewrite (44.8) as

$$\min_{\substack{Y\in\mathbb{R}^{n\times k}\\ c_i^Tc_j=\delta_{ij},\ e^Tc_i=0}} \sum_{i=1}^{k} c_i^T Lc_i. \tag{44.9}$$

This expression starts to look familiar. It is a sum of k Rayleigh quotients. One intuitive way to solve it is by a greedy, step-by-step approach: minimize each term in the sum one at a time while maintaining the orthogonality constraints.

First, we consider the minimization of the first term:

$$\min_{\|c_1\|_2=1,\ e^Tc_1=0} c_1^T Lc_1.$$

Since e is an eigenvector of L with the smallest eigenvalue, equal to 0, the Rayleigh-Ritz theorem (Theorem 42.12) tells us that the minimum is achieved when c_1 is the eigenvector corresponding to the second smallest eigenvalue of L.

Next, minimize the second term with the added constraint that c_2 is orthogonal to both c_1 and e:

$$\min_{\substack{\|c_2\|_2=1 \\ c_1^T c_2=0,\ e^T c_2=0}} c_2^T L c_2.$$

Again, by the Rayleigh-Ritz theorem, the minimum occurs when c_2 is the eigenvector corresponding to the third smallest eigenvalue of L.

We continue this process for $c_3, c_4, \ldots, c_k$, each time imposing that the new column is orthogonal to e and to all previously chosen columns. In general, the Rayleigh-Ritz theorem ensures that at each step c_i is the eigenvector corresponding to the $(i+1)$th smallest eigenvalue of L.

If you have read Chapter 43, this approach should seem familiar. It closely follows Ky Fan's theorem (Theorem 43.6), but with the added requirement that each column is also orthogonal to e. This extra constraint changes the result in a predictable way. The key point is that this greedy approach does indeed compute a solution to (44.8).

To summarize, the steps for spectral embedding are as follows:

1. Compute the Laplacian matrix L of the graph G.
2. Find the eigenvectors of L corresponding to its 2nd through $(k+1)$st smallest eigenvalues, and arrange them as the columns of a matrix Y.
3. Treat each row of Y as the k-dimensional embedding of a node in the graph.

Spectral embedding for dimensionality reduction of numerical data

Spectral embedding lets us map numerical data from high dimensions to a lower-dimensional space while keeping nearby points close together. Points that are close in the original space stay close in the reduced space. To do this, we first build a graph based on the distances between data points, using one of the methods from Section 13.4. After building the graph, we use spectral embedding to get the low-dimensional representation.

If we set $k = 2$, spectral embedding lets us "visualize" high-dimensional data by mapping it to two dimensions. Let us see how this works with the following two examples.

Example 44.18. (Visualizing handwritten digits)

In this example, we use spectral embedding to visualize a subset of the MNIST dataset [37] containing digits 3 and 8. Each image, originally a 784-dimensional vector, is mapped to a two-dimensional space.

We start by constructing a graph from the data points. Each point is treated as a node, and an edge connects two nodes if one is among the 20 nearest neighbors of the other. The weights of the edges are either 1 (for connected nodes) or 0 (otherwise).

Figure 44.17 shows the result. Blue points represent the digit "3" and red points represent the digit "8". Sample images are overlaid to show how similar digits group together. Digits that look alike end up close to each other in the plot. Within each cluster, the digit shapes change gradually as you move around. The overlapping region likely contains digits that are hard to read or written in a way that makes it difficult to tell if they are 3s or 8s.

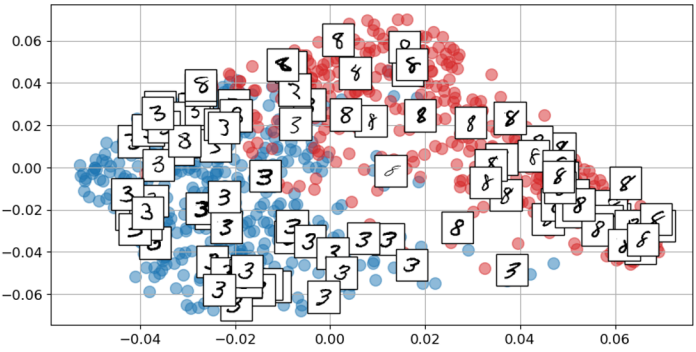


Fig. 44.17: Spectral embedding of MNIST digits "3" (blue) and "8" (red). Sample images illustrate how similar digits cluster in the 2D embedding.

Example 44.19. (Rubber ducks)

In this example, we use a dataset of 72 photos of a rubber duck, shown in Figure 44.18. Each photo is taken from a different angle around the duck. Specifically, the ith image is taken from an angle of $5(i-1)$ degrees, for $i = 1, \ldots, 72$.

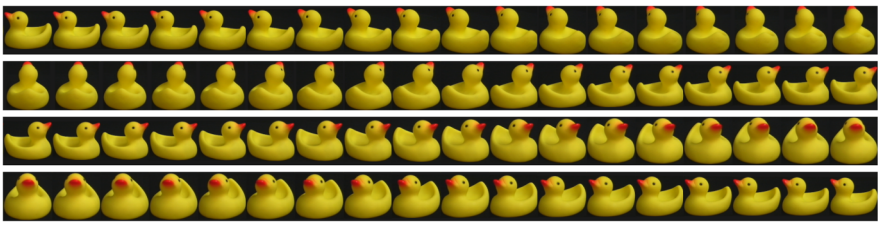
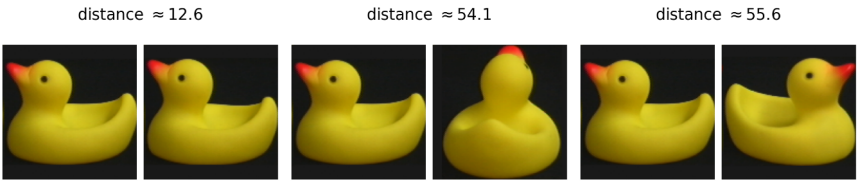


Fig. 44.18: Dataset of rubber duck photos taken from different angles.

Each photo is a 128×128 pixel RGB image, which we represent as a vector in $\mathbb{R}^{49152}$ ($49152 = 128 \times 128 \times 3$), as described in Chapter 3. Our goal is to use spectral embedding to reduce this high-dimensional dataset from $\mathbb{R}^{49152}$ down to $\mathbb{R}^2$.

Keep in mind that, in the original space, points that are close together represent photos taken from similar angles. Figure 44.19 illustrates this idea by showing the Euclidean distances between the first rubber duck image and the second, 20th, and 40th images. The second image, taken from almost the same angle, is much closer to the first than the 20th or 40th images, which come from very different viewpoints. This shows how changes in angle affect the distances between points in the high-dimensional space.



(a) First and second images. (b) First and 20th images. (c) First and 40th images.

Fig. 44.19: Distance from the first rubber duck image to the second, 20th and 40th images.

To illustrate this further, Figure 44.20 shows the distance matrix for the dataset. Each entry (i, j) in the matrix is color-coded based on the distance between the ith and jth rubber duck images. Blue shades indicate smaller distances, while red shades represent larger distances.

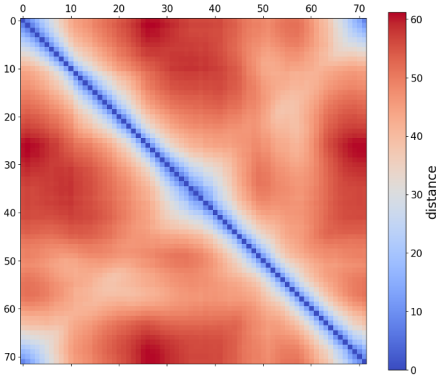


Fig. 44.20: Distance matrix for the rubber ducks dataset.

Before we can use spectral embedding, we need to build a graph from the dataset. We do this with a k-nearest neighbors method, using $k = 5$, and set the edge weights to 0 or 1.

Figure 44.21 shows the result after applying spectral embedding. For easier visualization, we display the actual duck images instead of just points, and show only a third of the data to avoid too much overlap. Spectral embedding arranges the images

based on the graph, so connected nodes end up close together. You can see this in how the ducks are placed in the plot. The sequential rotation of the photos is clearly captured in the two-dimensional space. This new representation nicely encodes the way the duck turns!

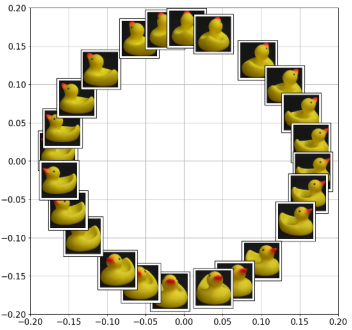


Fig. 44.21: Spectral embedding of the duck images.

44.2.4 Multidimensional scaling

Suppose you are given Table 44.4, which shows the road distances (in miles) between several cities in Montana. Now, imagine you need to recreate a map of Montana using only these distances. Is this even possible? Surprisingly, the answer is yes! This is exactly what Multidimensional Scaling (MDS) is designed for. With MDS, we can create a map that gives a good approximation of the actual locations of the cities.

	Billings	Missoula	Great Falls	Bozeman	Butte	Helena	Kalispell	Havre	Anaconda
Billings	0	345	218	144	228	241	433	248	249
Missoula	345	0	167	202	118	113	118	276	105
Great Falls	218	167	0	185	155	90	228	114	178
Bozeman	144	202	185	0	85	98	291	270	108
Butte	228	118	155	85	0	58	219	269	25
Helena	241	113	90	98	68	0	193	202	75
Kalispell	433	118	228	291	219	193	0	261	211
Havre	248	276	114	270	269	202	261	0	292
Anaconda	249	105	178	108	25	75	211	292	0

Table 44.4: Road distances (in miles) between cities in the US state of Montana.

Let Δ denote the matrix of road distances in Table 44.4. The MDS formulas below use squared distances. Thus, we form the matrix D defined by

$$D_{ij} := \Delta_{ij}^2.$$

Definition 44.20. (Euclidean distance matrix)

Given data points $x_1, x_2, \ldots, x_m \in \mathbb{R}^n$, the **Euclidean distance matrix** is an $m \times m$ matrix where each entry is the squared Euclidean distance between a pair of points:

$$D_{ij} = \|x_i - x_j\|_2^2.$$

Notice that the diagonal entries of a Euclidean distance matrix are always zero because they are the distance from each point to itself.

Example 44.21 Consider three points in $\mathbb{R}^4$:

$$x_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 0 \\ -1 \\ 1 \\ 0 \end{bmatrix}.$$

The Euclidean distance matrix for these points is

$$D = \begin{bmatrix} 0 & 2 & 2 \\ 2 & 0 & 6 \\ 2 & 6 & 0 \end{bmatrix}.$$

Definition 44.22. (The multidimensional scaling problem)

Given a Euclidean distance matrix $D \in \mathbb{R}^{m \times m}$, the goal of **multidimensional scaling (MDS)** is to reconstruct the vectors $x_i \in \mathbb{R}^n$ that were used to create D.

It is important to note that the solution to the MDS problem is not unique. If we find a set of vectors x_i that match the given distances, we can shift all the x_i by the same vector b, or apply an orthogonal transformation Qx_i (such as a rotation or reflection), and the pairwise distances between the points will not change.

This lack of uniqueness is not an issue because the distances only determine the relative positions of the points, not their exact coordinates. In fact, this flexibility can be helpful. By picking a convenient coordinate system, we can rewrite the MDS problem in a way that is easier to handle with Linear Algebra tools.

To solve the MDS problem, we start by finding an alternative formula for the Euclidean distance matrix D. Remember that the squared distance $D_{ij} = \|x_i - x_j\|_2^2$ can be written as

$$D_{ij} = (x_i - x_j)^T (x_i - x_j) = \|x_i\|_2^2 - 2x_i^T x_j + \|x_j\|_2^2.$$

Thus, we can express D as

$$D = \begin{bmatrix} \|x_1\|_2^2 & \|x_1\|_2^2 & \cdots & \|x_1\|_2^2 \\ \|x_2\|_2^2 & \|x_2\|_2^2 & \cdots & \|x_2\|_2^2 \\ \vdots & \vdots & \ddots & \vdots \\ \|x_m\|_2^2 & \|x_m\|_2^2 & \cdots & \|x_m\|_2^2 \end{bmatrix} - 2 \begin{bmatrix} x_1^T x_1 & x_1^T x_2 & \cdots & x_1^T x_m \\ x_2^T x_1 & x_2^T x_2 & \cdots & x_2^T x_m \\ \vdots & \vdots & \ddots & \vdots \\ x_m^T x_1 & x_m^T x_2 & \cdots & x_m^T x_m \end{bmatrix} + \begin{bmatrix} \|x_1\|_2^2 & \|x_2\|_2^2 & \cdots & \|x_m\|_2^2 \\ \|x_1\|_2^2 & \|x_2\|_2^2 & \cdots & \|x_m\|_2^2 \\ \vdots & \vdots & \ddots & \vdots \\ \|x_1\|_2^2 & \|x_2\|_2^2 & \cdots & \|x_m\|_2^2 \end{bmatrix}.$$

Let $X := \begin{bmatrix} x_1 & x_2 & \cdots & x_m \end{bmatrix}$ be the $n \times m$ matrix containing the original data points x_i as columns, and let $G := X^T X \in \mathbb{R}^{m \times m}$. Using e to denote the column vector of all ones, we can rewrite D as

$$D = \operatorname{diag}(G)\, e^T - 2G + e \operatorname{diag}(G)^T, \tag{44.10}$$

where $\operatorname{diag}(G) := \begin{bmatrix} \|x_1\|_2^2 & \|x_2\|_2^2 & \cdots & \|x_m\|_2^2 \end{bmatrix}^T$ is the vector containing the diagonal entries of G.

Here is the plan: first, solve for G from (44.10), and then find X using $G = X^T X$. However, the first step, solving for G, is not immediately obvious, given that G appears in all three terms of the equation. This is where we take advantage of the fact that the solution to the MDS problem is not unique.

To simplify the problem, let us assume $x_1 = 0$; in other words, we translate all the points so that x_1 is at the origin. Under this assumption, the main diagonal of G becomes

$$\operatorname{diag}(G) = \begin{bmatrix} \|x_1\|_2^2 \\ \|x_2\|_2^2 \\ \vdots \\ \|x_m\|_2^2 \end{bmatrix} = \begin{bmatrix} \|x_1 - x_1\|_2^2 \\ \|x_1 - x_2\|_2^2 \\ \vdots \\ \|x_1 - x_m\|_2^2 \end{bmatrix} =: d_1,$$

where d_1 is the first column of the Euclidean distance matrix D. Since D is part of the given data, $\operatorname{diag}(G)$ is now known. Substituting this into (44.10), we can solve for G as

$$G = -\frac{1}{2}\left(D - d_1\, e^T - e\, d_1^T\right).$$

The matrix $G = X^T X$ is symmetric and positive semidefinite since it is a Gram matrix. To recover the Gram factor X, we compute the eigenvalue decomposition of G:

$$G = Q \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_m) Q^T,$$

where $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_m \geqslant 0$ are the eigenvalues, and Q is an orthogonal matrix. If G is derived from a dataset in $\mathbb{R}^n$ with $n < m$, then G has rank at most n, so the smallest $m - n$ eigenvalues are zero ($\lambda_{n+1} = \cdots = \lambda_m = 0$). Thus, we can construct X as

$$X = \left[\operatorname{diag}(\sqrt{\lambda_1}, \sqrt{\lambda_2}, \ldots, \sqrt{\lambda_n})\ 0_{n \times (m-n)}\right] Q^T.$$

This gives us the reconstructed positions of the points in $\mathbb{R}^n$.

In practice, the given distances may not be exact Euclidean distances. They may contain measurement errors or noise, or they may come from a distance that is not exactly Euclidean, such as road distance in our example.

In this case, the matrix G reconstructed from D may no longer be positive semidefinite. In particular, some of the eigenvalues that would be zero for exact Euclidean data may become positive or negative.

Let

$$G = Q\,\mathrm{diag}(\lambda_1, \lambda_2, \ldots, \lambda_m)\,Q^T, \qquad \text{where} \qquad \lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_m.$$

To reconstruct the points in $\mathbb{R}^n$, we keep the n largest positive eigenvalues and their corresponding eigenvectors. Assuming $\lambda_1 \geqslant \cdots \geqslant \lambda_n > 0$, we define

$$\widehat{X} = \left[\mathrm{diag}(\sqrt{\lambda_1}, \sqrt{\lambda_2}, \ldots, \sqrt{\lambda_n})\ 0_{n\times(m-n)}\right] Q^T.$$

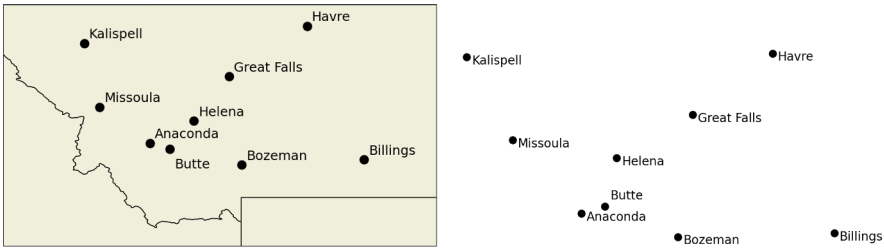
The columns of $\widehat{X}$ give an n-dimensional Euclidean representation of the points.

The negative eigenvalues measure, in part, the failure of the given distances to be exactly Euclidean. By keeping only the largest positive eigenvalues, we obtain a Euclidean approximation of the original distances.

Example 44.23. (Reconstructing Montana's map from city distances)

Let us return to the problem introduced at the start of this section. As a reminder, Table 44.4 provides road distances (in miles) between various cities in Montana. Our goal is to reconstruct the map of Montana using only the information in this table.

Figure 44.22-(a) displays the actual map of Montana, while Figure 44.22-(b) shows the city positions reconstructed using MDS. For clarity, we rotated the reconstructed configuration to align with the actual map.



(a) Map of the state of Montana. (b) Reconstructed positions.

Fig. 44.22: Actual map of Montana with selected cities, and reconstructed city positions using MDS.

The reconstructed map is reasonably accurate, though some city positions are not correct. For example, compare the locations of Anaconda and Butte in the actual map and the reconstructed one. This happens because the table gives road distances instead of straight-line distances, which can differ significantly due to the mountains

in the area. If we had exact straight-line Euclidean distances and retained two positive dimensions, the city locations could be reconstructed up to translation, rotation, and reflection. In practice, rounding and other measurement errors may still produce small discrepancies.

We can use Multidimensional Scaling (MDS) as a dimension-reduction technique to project high-dimensional data into lower dimensions while preserving relative distances between observations as closely as possible.

Suppose the data points $x_1, \ldots, x_m \in \mathbb{R}^n$ lie in, or close to, a k-dimensional affine subspace where $k < n$. MDS seeks points $\widehat{x}_1, \ldots, \widehat{x}_m \in \mathbb{R}^k$ that approximately preserve the pairwise squared distances $D_{ij} = \|x_i - x_j\|_2^2$ to the original points. This reduces the problem's dimensionality from n to k.

If the data points truly lie in a k-dimensional space, the matrix $G = X^T X$ will have rank at most k. Consequently, the $m - k$ smallest eigenvalues of G are zero ($\lambda_{k+1} = \cdots = \lambda_m = 0$). However, if the squared distances in the matrix D include noise, these trailing eigenvalues will not be exactly zero but will still be small. In either case, we can truncate X by setting the $m - k$ smallest eigenvalues to zero. A k-dimensional representation of the data can then be obtained as

$$\widehat{X} = \left[\operatorname{diag}(\lambda_1^{1/2}, \ldots, \lambda_k^{1/2})\ 0_{k\times(m-k)}\right] Q^T.$$

Example 44.24. (Using MDS for dimensionality reduction)

Figure 44.23 illustrates how MDS can reduce the dimensionality of a dataset from three to two. In Figure 44.23-(a), we see a three-dimensional dataset where all points lie close to a plane. Figure 44.23-(b) shows the two-dimensional representation reconstructed using MDS.

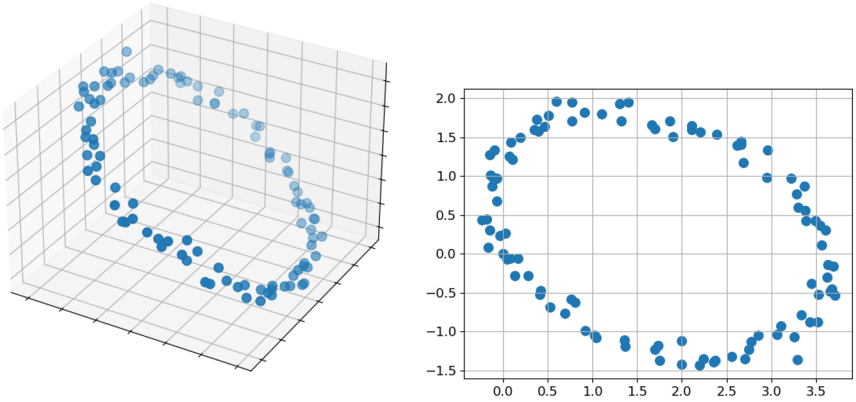
Problems

44.1 Let G be an unweighted, undirected graph. Suppose the second smallest eigenvalue of G's Laplacian is zero, and you want to find two clusters using the Fiedler vector. What would be your approach?

44.2 Consider the graph with the following adjacency matrix:

$$A = \begin{bmatrix} 0&1&1&0&0&0 \\ 1&0&1&0&0&0 \\ 1&1&0&1&0&0 \\ 0&0&1&0&1&1 \\ 0&0&0&1&0&1 \\ 0&0&0&1&1&0 \end{bmatrix}.$$

(a) Find the degree matrix D and then compute the Laplacian $L = D - A$.



(a) Original three dimensional dataset. (b) Two dimensional reconstruction.

Fig. 44.23: Dimensionality reduction using MDS: (a) Original three-dimensional dataset with points lying close to a plane. (b) Two-dimensional representation reconstructed with MDS.

(b) Determine the eigenvalues and eigenvectors of L. Use the Fiedler vector (the eigenvector corresponding to the second smallest eigenvalue) to partition the graph into two clusters, and identify which vertices belong to each cluster.
(c) Explain why this partition reflects the structure of the graph.

44.3 Consider the weighted adjacency matrix

$$A = \begin{bmatrix} 0 & 2 & 1 & 0 \\ 2 & 0 & 1 & 3 \\ 1 & 1 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}.$$

First, compute the Laplacian matrix for this graph. Then, use the spectral clustering algorithm to identify clusters in the graph.

44.4 Write a program in a programming language of your choice (such as Python or MATLAB) to implement spectral clustering. Your program should take an adjacency matrix A as input, compute the corresponding Laplacian matrix, and then determine the k smallest eigenvalues along with their eigenvectors. Next, use the rows of the eigenvector matrix to cluster the graph's vertices into k groups using k-means clustering. Test your implementation on the following graph with adjacency matrix

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}.$$

44.5 In Problem 9.8, we introduced Zachary's karate club dataset. This classic social network records friendships between members of a university karate club. Eventually, the club split into two groups. Use spectral clustering to identify these two communities in the network.

44.6 Navigate to the OpenML repository [3] and download the *Pima Indians Diabetes* dataset. The cohort has $m = 768$ adult female patients of Pima Indian heritage. Each record contains eight numeric features listed in Table 44.5.

Abbreviation	Feature description
`preg`	Number of pregnancies
`plas`	Plasma glucose concentration (2 h oral test)
`pres`	Diastolic blood pressure (mmHg)
`skin`	Triceps skinfold thickness (mm)
`insu`	2 h serum insulin (μU/mL)
`mass`	Body mass index (kg/m^2)
`pedi`	Diabetes pedigree function
`age`	Age (years)

Table 44.5: Features in the Pima Indians Diabetes dataset.

The goal is to predict whether a patient has diabetes using these features. For each feature, compute its Fisher score and rank the features by their importance for predicting the label.

44.7 Consider the following dataset with $m = 6$ points, $n = 2$ features, and $k = 2$ classes:

- Class 1: $(2, 3)$, $(3, 4)$, $(4, 5)$
- Class 2: $(6, 8)$, $(7, 9)$, $(8, 10)$

Manually compute the LDA projection vector for this data. You may use a computer to calculate the means and eigenvalues, but do not use a built-in LDA function.

44.8 Write a function in your preferred programming language that performs LDA. Choose a labeled dataset you like and test your function.

44.9 When studying the LDA technique, we claimed the following:

$$\max_{v} \frac{\text{between-class variance}}{\text{within-class variance}} = \max_{v} \frac{v^T S_B v}{v^T S_W v} = \max_{v^T S_W v = 1} v^T S_B v.$$

Prove the second equality.

44.10 (The eigenvalue zero in LDA) In LDA, we project the data onto $\text{span}(v)$, where v is an eigenvector of $S_W^{-1} S_B$. Show that if v is an eigenvector with eigenvalue 0, then projecting onto $\text{span}(v)$ results in zero between-class variance. In other words, this direction does not help separate the classes.

44.11 (Number of directions in LDA) In LDA, we find the directions that best separate the classes by taking the eigenvectors of $S_W^{-1} S_B$ associated with nonzero eigenvalues. How many such directions can there be?

Consider the between-class scatter matrix $S_B := \sum_{\ell=1}^{k} m_\ell \, (M_\ell - M)(M_\ell - M)^T$ defined in Section 44.2.2.

(a) Show that S_B can be written as a Gram matrix:

$$S_B = \left[\sqrt{m_1}(M_1 - M) \cdots \sqrt{m_k}(M_k - M)\right] \begin{bmatrix} \sqrt{m_1}(M_1 - M)^T \\ \vdots \\ \sqrt{m_k}(M_k - M)^T \end{bmatrix}$$

(b) Show that the columns of the Gram factor

$$\left[\sqrt{m_1}(M_1 - M) \cdots \sqrt{m_k}(M_k - M)\right],$$

are linearly dependent, and conclude that $\operatorname{rank}(S_B) \leqslant k - 1$.

(c) Conclude that LDA can find at most $k - 1$ directions.

44.12 (Two-class LDA) Consider a labeled dataset with two class labels ($k = 2$). Show that the direction that maximizes the ratio of between-class to within-class variance is

$$v = S_W^{-1}(M_1 - M_2)$$

where M_1 and M_2 are the mean vectors for each class, and S_W is the within-class scatter matrix.

44.13 Find the road distances between major cities in your country. Use MDS to reconstruct their relative positions on a map.

Part X
SINGULAR VALUE DECOMPOSITION

In this part, we introduce the Singular Value Decomposition (SVD) and its many applications. We begin with its definition and explaining its geometric interpretation. Next, we introduce the pseudoinverse, which extends the concept of matrix inversion to singular (even rectangular) matrices. We show its role in solving linear systems and least squares problems. Finally, we show how SVD is used for data analysis, compression, and matrix approximation. A key focus is Principal Component Analysis (PCA), where we show how SVD helps extract low-dimensional representations of high-dimensional data.

Chapter 45
X-WHY SHOULD WE CARE?

45.1 Data compression and low-rank approximations

Data matrices are often large, but they usually contain redundant information across their rows and columns. For example, in Chapter 19 we saw document-term matrices and rating matrices, and noted that their rows and columns often follow similar patterns due to shared topics or common user behaviors. In Chapter 17, we introduced the idea of a matrix's rank, the concept that captures the true, underlying dimensionality of the data.

Recall from Chapter 17 that, when a matrix $A \in \mathbb{R}^{m \times n}$ has rank r, we can factor it as

$$A = UV^T \qquad \text{or} \qquad A = UMV^T,$$

where $U \in \mathbb{R}^{m \times r}$, $M \in \mathbb{R}^{r \times r}$, and $V \in \mathbb{R}^{n \times r}$. We call such representations full rank factorizations because they explicitly reveal the rank structure of A. When r is much smaller than m or n, we say that A is low-rank. In these cases, full rank factorizations help reduce both storage and computational costs.

In practice, most matrices are not exactly low-rank. However, they can often be well-approximated by low-rank matrices. Instead of insisting on an exact equality, we aim for approximations

$$A \approx UV^T \qquad \text{or} \qquad A \approx UMV^T,$$

with the goal of finding a small rank r that still provides a good representation of the data.

Later, in Chapter 47, we will introduce singular values and the singular value decomposition (SVD). The singular values help us decide when a matrix can be effectively approximated by a lower-rank matrix, and the SVD gives us the best rank-r approximation possible.

Next, we present two examples where data matrices can be approximated with low-rank matrices.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_45

45.1.1 Image compression

Images are a great example of matrices that can sometimes be approximated by low-rank matrices. They have strong spatial correlations, adjacent pixels tend to be similar. They also contain dominant features like edges, as well as large areas with little variation (like skies or walls). These properties create redundancy that low-rank approximations can capture well.

Consider the grayscale image in Figure 45.1. The image is represented by a 600×1100 matrix A, where each element holds the intensity value of a pixel.

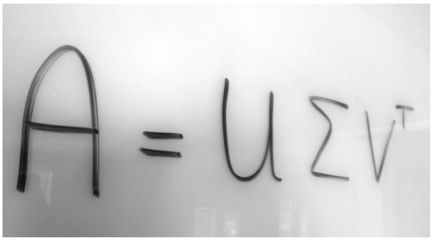


Fig. 45.1: A 600×1100 grayscale image.

Can we compress this image to save storage without sacrificing visual quality? First, notice that the image matrix has more columns than rows, which means that the columns are linearly dependent. So, we do not need to store every column. But what if we drop even some of the independent columns? Could we still closely approximate the original image?

Using the singular value decomposition (SVD), which will be introduced in Chapter 47, we can find the best rank-r approximation for any matrix. Figure 45.2 displays the rank-5 (top left), rank-10 (top right), rank-20 (bottom left), and rank-40 (bottom right) approximations of A.

Notice that the rank-40 version is nearly identical to the original image (at our book's resolution) while using only about 10% of the storage.

45.1.2 Storing weather data

Suppose you work for a weather-monitoring organization with n weather stations across the country, each recording average monthly temperatures for k years. This means the dataset would contain $12nk$ measurements in total.

For example, Figure 45.3 shows the monthly temperatures (in Celsius) for 37 towns in the western United States from January 2015 to December 2022. These towns include Seattle, Spokane, Portland, Boise, Salt Lake City, Las Vegas, Phoenix,

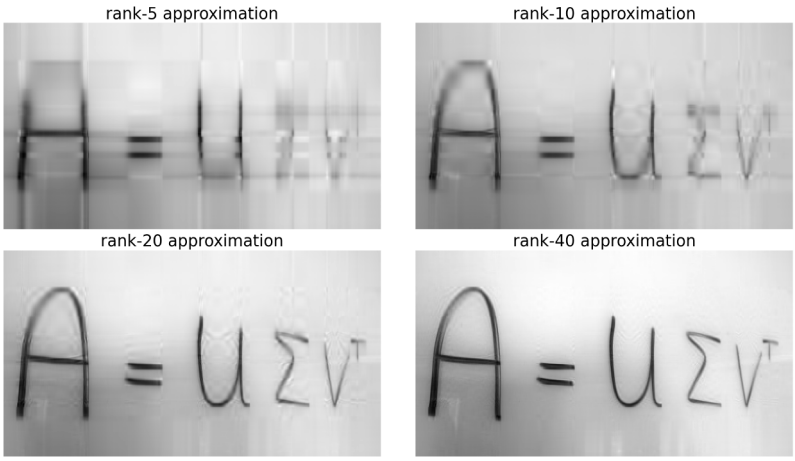


Fig. 45.2: Low-rank approximations of the image in Figure 45.1.

and San Francisco, among others.[1] In this dataset, each station records 96 values, one for each month over eight years.

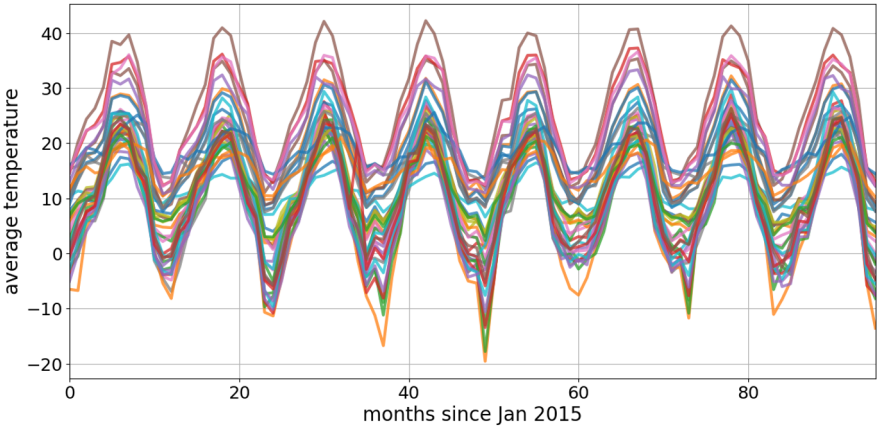


Fig. 45.3: Monthly average temperatures from January 2015 to December 2022 for 37 towns in the western US.

To analyze the data, we arrange it in a matrix X. Each row is a weather station and each column is a month. If there are n stations and $12k$ months of data (for k years),

[1] Complete list: Olympia, Seattle, Spokane, Yakima (Washington); Astoria, Eugene, Medford, Pendleton, Salem, Portland (Oregon); Boise, Lewiston, Pocatello (Idaho); Billings, Glasgow, Great Falls, Helena, Kalispell, Missoula (Montana); Cedar City, Salt Lake City (Utah); Elko, Ely, Las Vegas, Reno (Nevada); Tucson, Yuma, Phoenix (Arizona); and Bakersfield, Eureka, Fresno, Death Valley, Los Angeles, Modesto, Sacramento, San Diego, San Francisco (California).

then $X \in \mathbb{R}^{n \times 12k}$. The entry X_{ij} is the average temperature recorded by station i in month j. As we add stations or years, the matrix grows. Many stations record more often (daily or hourly), so these matrices can be very large.

Temperature data often admits a low-rank approximation because temperatures are strongly correlated across space and time. For example, if it is cold in Missoula (Montana), Seattle (Washington) is likely chilly too.

Figure 45.3 shows a roughly sinusoidal yearly cycle. A simple model is

$$X_{ij} \approx a_i + b_i \sin\left(\frac{2\pi(j-5)}{12}\right),$$

where a_i is the baseline for station i, b_i is the amplitude, and the shift $j - 5$ makes February the coldest month.

This model is simple, but it captures much of the structure. Instead of storing $12k$ monthly values per station, we store just a_i and b_i. In matrix terms, the data are well approximated by a rank-2 matrix.

This example shows a common idea in data science: high-dimensional data often has structure that allows effective low-rank approximations.

Chapter 46
X-WHAT YOU MUST HAVE LEARNED BEFORE

Before we introduce one of the most powerful tools in Linear Algebra, let us briefly review some key ideas from earlier chapters.

The four fundamental subspaces of a matrix

Recall that associated with any matrix $A \in \mathbb{R}^{m\times n}$, there are four important subspaces, two in $\mathbb{R}^m$ and two in $\mathbb{R}^n$. These are known as the *four fundamental subspaces* of A. They are:

Column space:	$\text{col}(A) = \{Ax \mid x \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$,
Row space:	$\text{col}(A^T) = \{A^T y \mid y \in \mathbb{R}^m\} \subseteq \mathbb{R}^n$,
(Right) nullspace:	$\text{null}(A) = \{x \in \mathbb{R}^n \mid Ax = 0\} \subseteq \mathbb{R}^n$, and
Left nullspace:	$\text{null}(A^T) = \{y \in \mathbb{R}^m \mid A^T y = 0\} \subseteq \mathbb{R}^m$.

Throughout this book, we have seen that the four fundamental subspaces are important to understanding linear systems, least squares problems, eigenvalues and eigenvectors, and factorizations.

The orthogonality of the four fundamental subspaces

We have also seen an important relationship between these subspaces: they form pairs that are orthogonal complements. For any $m \times n$ matrix A, we have

$$\text{null}(A) = \text{col}(A^T)^\perp \quad \text{and} \quad \text{null}(A^T) = \text{col}(A)^\perp.$$

These relationships allow us to express the spaces $\mathbb{R}^m$ and $\mathbb{R}^n$ as direct sums:

$$\mathbb{R}^m = \text{col}(A) \oplus \text{null}(A^T), \quad \text{and} \quad \mathbb{R}^n = \text{col}(A^T) \oplus \text{null}(A).$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_46

If you need a visual reminder, see Figure 27.2. Take a moment to look it over and check that you understand each idea it presents.

We will also rely on several key facts about symmetric positive semidefinite matrices. In particular, any such matrix can be written as a Gram matrix, so we begin by recalling Gram matrices.

The four fundamental subspaces of Gram matrices

A Gram matrix $S \in \mathbb{R}^{m\times m}$ is a symmetric matrix that can be written as $S = A^T A$ for some matrix $A \in \mathbb{R}^{n\times m}$, called a Gram factor. The four fundamental subspaces of $S = A^T A$ and those of A are closely connected:

$$\begin{aligned} &\text{Column space of } S: && \operatorname{col}(S) = \operatorname{col}(A^T), \quad \text{and} \\ &\text{Nullspace of } S: && \operatorname{null}(S) = \operatorname{null}(A). \end{aligned}$$

Since S is symmetric, its row space is the same as its column space, and its left nullspace is the same as its nullspace.

Gram matrices are symmetric and positive semidefinite (Theorem 42.38).

Symmetric positive semidefinite matrices and the spectral theorem

Every real symmetric matrix has real eigenvalues. A symmetric matrix is positive semidefinite exactly when all its eigenvalues are nonnegative. By the spectral theorem (Theorem 41.3), any symmetric matrix A can be orthogonally diagonalized:

$$A = QDQ^T,$$

where Q is orthogonal, D is diagonal, and the diagonal entries of D are the eigenvalues of A. For a Gram matrix, these eigenvalues are nonnegative. The columns of Q give an orthonormal basis for $\mathbb{R}^n$ of eigenvectors.

Orthogonal projection matrices

Let $\mathcal{W}$ be a subspace of $\mathbb{R}^n$. Then,

$$\mathbb{R}^n = \mathcal{W} \oplus \mathcal{W}^\perp,$$

that is, $\mathbb{R}^n$ is the direct sum of $\mathcal{W}$ and its orthogonal complement. This implies that every vector $x \in \mathbb{R}^n$ can be written in a unique way as

$$x = x_W + x_O \quad \text{with} \quad x_W \in \mathcal{W},\ x_O \in \mathcal{W}^\perp.$$

The orthogonal projection onto $\mathcal{W}$ is the linear transformation that takes $x \to x_W$ (recall Definition 31.1). Since this is a linear transformation between finite dimensional vector spaces, it can be expressed as a $x \to Px$ for some matrix P called the *orthogonal projection matrix* onto $\mathcal{W}$. If Q is a matrix whose columns form an orthogonal basis for $\mathcal{W}$, then $P = QQ^T$ (recall Theorem 32.1).

The orthogonal projection of a vector x onto a subspace $\mathcal{W}$ is the point in $\mathcal{W}$ closest to x. This property is what makes orthogonal projections important in approximating datasets by low-dimensional ones.

These ideas apply directly to a matrix A. The pairs $(\text{col}(A), \text{null}(A^T))$ and $(\text{col}(A^T), \text{null}(A))$ are orthogonal complements, so projection matrices onto these subspaces can be built in the same way.

Problems

46.1 Consider the 4×3 matrix

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 2 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}.$$

First, determine the four fundamental subspaces of A. Next, find an explicit way to write any vector $x \in \mathbb{R}^4$ as the sum of a vector from $\text{col}(A)$ and a vector from $\text{null}(A^T)$. Similarly, find an explicit method for writing any vector $y \in \mathbb{R}^3$ as the sum of a vector from $\text{col}(A^T)$ and a vector from $\text{null}(A)$.

46.2 Let $S = A^T A$ be a Gram matrix with spectral decomposition $S = QDQ^T$. Explain how to use Q and D to find orthonormal bases for $\text{col}(A^T)$ and $\text{null}(A)$.

Chapter 47
X-CORE TOPICS

47.1 The singular value decomposition

In Part IX, we learned a major advantage of working with positive semidefinite matrices: the Spectral Theorem (Theorem 41.3). This theorem tells us that any symmetric positive semidefinite matrix $S \in \mathbb{R}^{n\times n}$ can be written as

$$S = QDQ^T,$$

where Q is an orthogonal matrix and D is a diagonal matrix with nonnegative entries.

Wouldn't it be great if we could have a similar factorization for any matrix, regardless of its shape or symmetry? The Singular Value Decomposition (SVD) does just that. It generalizes the factorization $S = QDQ^T$ to all matrices.

Theorem 47.1. (Existence of singular value decompositions)

For any matrix $A \in \mathbb{R}^{m\times n}$ of rank r, there exist orthogonal matrices $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$, and a diagonal matrix $\Sigma_r = \operatorname{diag}(\sigma_1, \sigma_2, \ldots, \sigma_r)$ with $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_r > 0$, such that

$$A = U\Sigma V^T = U \underbrace{\begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix}}_{=\Sigma} V^T. \tag{47.1}$$

The factorization 47.1 is called **a singular value decomposition (SVD)** of A. The σ_i's are called the nonzero **singular values** of A, and the columns of U and V are referred to as **left and right singular vectors** of A, respectively.

If A is an $m \times n$ matrix, then the diagonal matrix

$$\Sigma = \begin{array}{c} \begin{matrix} r & n-r \end{matrix} \\ \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{matrix} r \\ m-r \end{matrix} \end{array} = \left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \ddots & & 0 \\ & & \sigma_r & \\ \hline & 0 & & 0 \end{array}\right],$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_47

from an SVD decomposition (47.1) of A, is an $m \times n$ matrix. Notice that Σ has the same dimensions as A and is not necessarily square.

When $r = \text{rank}(A) < p = \min\{m, n\}$, then we say that A has $p - r$ **additional zero singular values**, $\sigma_{r+1} = \cdots = \sigma_p = 0$. Therefore, an $m \times n$ matrix has $p = \min\{m, n\}$ singular values, which are the diagonal entries Σ.

Figure 47.1 shows the shape of Σ in three different cases: $m = n$ (left), $m > n$ (center), and $m < n$ (right). The shaded area marks the main diagonal of Σ where the $p = \min\{m, n\}$ singular values are located.

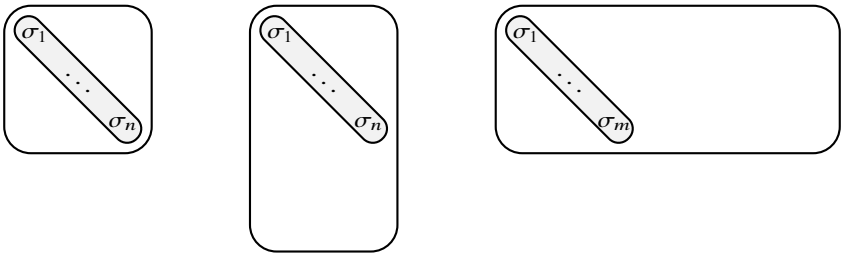


Fig. 47.1: Shape of the diagonal matrix Σ in a Singular Value Decomposition (SVD) for different matrix dimensions: square ($m = n$), tall ($m > n$), and wide ($m < n$). The shaded region represents the main diagonal of Σ, where the singular values are located.

We will prove Theorem 47.1 in Section 47.1.1, but first we build some intuition.

Theorem 47.1 tells us that any matrix A can be written as the product of three matrices: an orthogonal matrix, a "diagonal" matrix with nonnegative entries, and another orthogonal matrix. This is similar to the decomposition $A = QDQ^T$ we saw for positive semidefinite matrices, but here we need two orthogonal matrices, U and V, rather than a single Q and the diagonal matrix is not necessarily square.

The factorization $A = QDQ^T$ depends on finding an orthonormal basis of eigenvectors for A. This is not always possible: some square matrices are not diagonalizable, and even when they are, their eigenvectors may not be chosen to be orthogonal. Furthermore, for rectangular matrices the equation $Ax = \lambda x$ is undefined because x and Ax have different dimensions.

To prove Theorem 47.1, we take a different route. Instead of the eigendecomposition $A = QDQ^T$, we focus on the four fundamental subspaces of A.

The four fundamental subspaces: from QDQ^T to $U\Sigma V^T$

Let us first analyze what a QDQ^T factorization of a positive semidefinite matrix implies for its four fundamental spaces with an example.

Consider a 4×4 symmetric positive semidefinite matrix A with rank 2. We choose these dimensions and rank specifically so that we can visualize the four fundamental subspaces of A.

First, let us arrange the eigenvalues of A in decreasing order: $\lambda_1 \geqslant \lambda_2 \geqslant \lambda_3 \geqslant \lambda_4$. Since A has rank 2, only λ_1 and λ_2 are positive, while λ_3 and λ_4 are zero. Let

q_1, q_2, q_3, q_4 be associated orthonormal eigenvectors, so that

$$Aq_i = \lambda_i q_i \quad \text{and} \quad q_i^T q_j = \delta_{ij} \quad (i, j \in \{1, 2, 3, 4\}).$$

Now, let us examine the four fundamental subspaces of A, as shown in Figure 47.2.

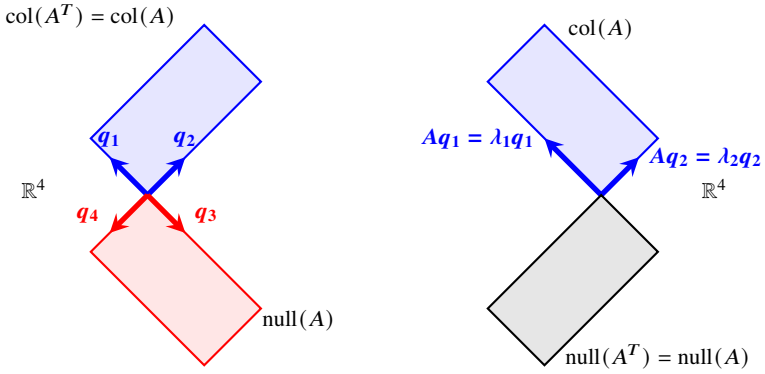


Fig. 47.2: The orthogonality of the four fundamental subspaces of a 4×4 symmetric positive semidefinite matrix A with rank 2.

Since A maps $\mathbb{R}^4$ to $\mathbb{R}^4$, all four of its fundamental subspaces lie within $\mathbb{R}^4$. Because A is symmetric, its row space and column space are the same. Likewise, its nullspace and left nullspace coincide. In particular, the eigenvectors q_1 and q_2 form an orthonormal basis for both the column and row spaces, while q_3 and q_4 do the same for the nullspace and left nullspace.

Moreover, A maps the orthonormal basis $\{q_1, q_2\}$ for the row space $\text{col}(A^T)$ to the orthogonal (but, in general, not orthonormal) basis $\{\lambda_1 q_1, \lambda_2 q_2\}$ for the column space $\text{col}(A)$.

In the next section we show how to extend the factorization $A = QDQ^T$ to non-symmetric matrices. The key is to find two orthonormal bases: $\{v_1, \ldots, v_r\}$ for the row space and $\{u_1, \ldots, u_r\}$ for the column space, with the property that

$$Av_i = \sigma_i u_i, \qquad \sigma_i > 0, \ i = 1, \ldots, r.$$

Figure 47.3 illustrates this idea[1]. Remarkably, such paired orthonormal bases always exist.

[1] Throughout this chapter we reuse diagrams similar to Figure 47.3. The relative sizes of the subspaces in these sketches do not reflect their exact dimensions; some regions are drawn larger simply to keep the picture clear.

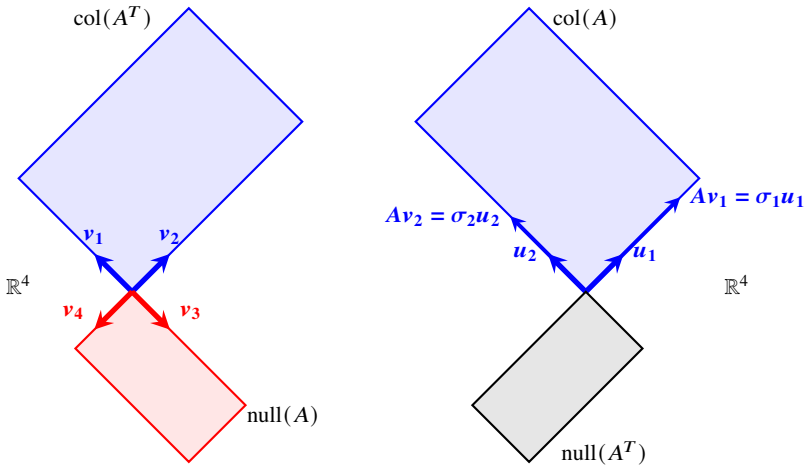


Fig. 47.3: Illustration of how a matrix A maps the orthonormal basis $\{v_1, v_2\}$ of the row space to an orthogonal basis $\{\sigma_1 u_1, \sigma_2 u_2\}$ of the column space.

47.1.1 Proof of Theorem 47.1: existence of the singular value decomposition

Let A be an $m \times n$ matrix with rank r. Recall that the row space and the column space of A both have dimension r. Our goal is to show that there exist orthonormal bases $\{v_1, \ldots, v_r\}$ for the row space of A and $\{u_1, \ldots, u_r\}$ for the column space of A, such that

$$Av_i = \sigma_i u_i, \quad \text{for } i = 1, 2, \ldots, r, \tag{47.2}$$

and for some positive scalar σ_i.

The first question is: how do we find an orthonormal basis for the row space of A? Fortunately, we already have a positive semidefinite matrix that shares the row space with A: the Gram matrix $A^T A \in \mathbb{R}^{n \times n}$ (see Theorem 17.45). Moreover, $A^T A$ and A have not only the same row space but also the same nullspace.

Applying the spectral theorem to the symmetric matrix $A^T A$ yields the eigendecomposition

$$A^T A = V D V^T,$$

where $V \in \mathbb{R}^{n \times n}$ is an orthogonal matrix whose columns $v_1, \ldots, v_n$ are linearly independent eigenvectors of $A^T A$, and D is a diagonal matrix containing the eigenvalues $\lambda_1, \ldots, \lambda_n \geqslant 0$ of $A^T A$. Since $\operatorname{rank}(A^T A) = \operatorname{rank}(A) = r$, the first r eigenvalues (assuming they are ordered in nonincreasing order) are positive, and the remaining $n - r$ are zero. Consequently, the first r columns $v_1, \ldots, v_r$ form an orthonormal basis for $\operatorname{col}(A^T A) = \operatorname{col}(A^T)$, while the last $n - r$ columns $v_{r+1}, \ldots, v_n$ form an orthonormal basis for $\operatorname{null}(A^T A) = \operatorname{null}(A)$.

For each positive eigenvalue λ_i (with $i = 1, \ldots, r$), we can write $\sigma_i = \sqrt{\lambda_i} > 0$. This yields the relationships

$$\begin{aligned} A^T A v_i &= \sigma_i^2 v_i, && \text{for } i = 1, \ldots, r, \quad \text{and} \\ A v_i &= 0, && \text{for } i = r+1, \ldots, n. \end{aligned} \tag{47.3}$$

Now that we have orthonormal bases for $\mathrm{col}(A^T)$ and $\mathrm{null}(A)$, our next step is to find bases for the column space $\mathrm{col}(A)$ and the left nullspace $\mathrm{null}(A^T)$. One idea is to use the spectral decomposition of the Gram matrix $AA^T \in \mathbb{R}^{m\times m}$, since A and AA^T share the same column space and left nullspace. However, this method does not guarantee that the bases for the column and row spaces are linked as in (47.2). Instead, we define the vectors

$$u_i := \frac{1}{\sigma_i} A v_i, \quad \text{for } i = 1, \ldots, r. \tag{47.4}$$

By construction, these vectors lie in the column space of A (since u_i is the image of $(1/\sigma_i)v_i$ under A) and satisfy (47.2). We only need to verify that they are orthonormal. To check this, we compute

$$u_j^T u_i = \frac{1}{\sigma_j \sigma_i} v_j^T A^T A v_i = \frac{\sigma_i^2}{\sigma_j \sigma_i} v_j^T v_i = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

where we used the fact that v_i is an eigenvector of $A^T A$ with eigenvalue $\lambda_i = \sigma_i^2$, and that the vectors $v_1, \ldots, v_r$ are orthonormal. This confirms that the u_i's form an orthonormal basis for $\mathrm{col}(A)$.

We now combine (47.3) and (47.4) into a single matrix equation

$$\begin{aligned} A \begin{bmatrix} v_1 & \cdots & v_r & v_{r+1} & \cdots & v_n \end{bmatrix} &= \begin{bmatrix} Av_1 & \cdots & Av_r & Av_{r+1} & \cdots & Av_n \end{bmatrix} \\ &= \begin{bmatrix} \sigma_1 u_1 & \cdots & \sigma_r u_r & 0 & \cdots & 0 \end{bmatrix} \\ &= \begin{bmatrix} u_1 & \cdots & u_r \end{bmatrix} \left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \ddots & & 0 \\ & & \sigma_r & \end{array}\right]. \end{aligned}$$

This is almost an SVD decomposition of A. Notice that $V = \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix}$ is an $n \times n$ orthogonal matrix. However, the matrix $\begin{bmatrix} u_1 & \cdots & u_r \end{bmatrix}$ has only r columns, so while its columns are orthonormal, it is not square (and hence not orthogonal). To complete the decomposition, we need $m - r$ additional columns to form a full orthogonal matrix.

We extend the orthonormal basis $\{u_1, \ldots, u_r\}$ for $\mathrm{col}(A)$ by choosing any orthonormal basis $\{u_{r+1}, \ldots, u_m\}$ for the left nullspace $\mathrm{null}(A^T)$. This gives us the $m \times m$ matrix

$$U = \begin{bmatrix} u_1 & \cdots & u_r & u_{r+1} & \cdots & u_m \end{bmatrix}.$$

Since $\mathrm{col}(A)$ and $\mathrm{null}(A^T)$ are orthogonal complementary subspaces, U is orthogonal. We can now write

$$A\begin{bmatrix} v_1 & \cdots & v_r & v_{r+1} & \cdots & v_n \end{bmatrix} = \begin{bmatrix} u_1 & \cdots & u_r \end{bmatrix} \left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \ddots & & 0 \\ & & \sigma_r & \end{array}\right]$$

$$= \begin{bmatrix} u_1 & \cdots & u_r & u_{r+1} & \cdots & u_m \end{bmatrix} \underbrace{\left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \ddots & & 0 \\ & & \sigma_r & \\ \hline & 0 & & 0 \end{array}\right]}_{=\Sigma}.$$

Thus, we have $AV = U\Sigma$, or equivalently, $A = U\Sigma V^T$. This establishes the existence of a singular value decomposition for any matrix A.

We can follow the steps from the SVD existence proof to manually compute the singular value decomposition for small matrices, as illustrated in the following example.

Example 47.2 Consider the matrix

$$A = \begin{bmatrix} 2 & -6 \\ 1 & -3 \\ 0 & 0 \end{bmatrix}.$$

The right singular vectors of A are eigenvectors of A^TA, and the singular values are the square roots of the eigenvalues of A^TA. We have

$$A^TA = \begin{bmatrix} 5 & -15 \\ -15 & 45 \end{bmatrix}.$$

The characteristic polynomial of A^TA is

$$p_A(\lambda) = \det(\lambda I_2 - A^TA) = (\lambda - 5)(\lambda - 45) - 225 = \lambda(\lambda - 50).$$

Thus, the eigenvalues of A^TA are $\lambda_1 = 50$ and $\lambda_2 = 0$. Associated unit eigenvectors are

$$v_1 = \frac{1}{\sqrt{10}}\begin{bmatrix} 1 \\ -3 \end{bmatrix} \quad \text{and} \quad v_2 = \frac{1}{\sqrt{10}}\begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

The vectors v_1 and v_2 are right singular vectors of A. The singular values are $\sigma_1 = \sqrt{\lambda_1} = \sqrt{50} = 5\sqrt{2}$ and $\sigma_2 = \sqrt{\lambda_2} = 0$. Since only one singular value is nonzero, $\operatorname{rank}(A) = 1$, and we have $\operatorname{col}(A^T) = \operatorname{span}(v_1)$ and $\operatorname{null}(A) = \operatorname{span}(v_2)$.

Next, we compute the left singular vectors. Using (47.4), the first left singular vector is

$$u_1 = \frac{Av_1}{\sigma_1} = \frac{1}{\sqrt{5}}\begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}.$$

To complete the orthonormal basis for $\mathbb{R}^3$, we extend $\{u_1\}$ by finding two additional orthonormal vectors u_2 and u_3. These vectors must be orthogonal to u_1 and to each

other. We choose

$$u_2 = \frac{1}{5}\begin{bmatrix} 1 \\ -2 \\ 0 \end{bmatrix} \quad \text{and} \quad u_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Thus, a singular value decomposition of A is

$$\begin{bmatrix} 2 & -6 \\ 1 & -3 \\ 0 & 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 2/\sqrt{5} & 1/\sqrt{5} & 0 \\ 1/\sqrt{5} & -2/\sqrt{5} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{=U} \underbrace{\begin{bmatrix} 5\sqrt{2} & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}}_{=\Sigma} \underbrace{\begin{bmatrix} 1/\sqrt{10} & -3/\sqrt{10} \\ 3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix}}_{=V^T}.$$

Remark 47.3 Notice that, if $A = U\Sigma V^T$ is an SVD of A, then

$$AA^T = U\Sigma V^T V\Sigma^T U^T = U\Sigma\Sigma^T U^T,$$

where $\Sigma\Sigma^T$ is a square matrix whose nonzero main diagonal entries are the squares of the nonzero singular values of A. The expression of AA^T in terms of U and Σ shows that the columns of U are eigenvectors of AA^T.

47.1.2 Properties of the singular value decomposition

Below is a theorem that summarizes important properties of the singular value decomposition. These were established during the proof of Theorem 47.1. For ease of reference, we list them here.

Theorem 47.4. (Properties of the singular value decomposition)

Let $A \in \mathbb{R}^{m\times n}$ with $\text{rank}(A) = r$, and let $A = U\Sigma V^T$ be a singular value decomposition of A. The following statements hold:

(a) The columns of V are eigenvectors of $A^T A$.
(b) The first r columns of V form an orthonormal basis for the row space $\text{col}(A^T)$.
(c) The last $n - r$ columns of V form an orthonormal basis for the nullspace $\text{null}(A)$.
(d) The columns of U are eigenvectors of AA^T.
(e) The first r columns of U form an orthonormal basis for the column space $\text{col}(A)$.
(f) The last $m - r$ columns of U form an orthonormal basis for the left nullspace $\text{null}(A^T)$.
(g) The nonzero eigenvalues of $A^T A$ and AA^T are equal to the squares of the nonzero singular values of A.

Since the SVD gives us orthonormal bases for all four fundamental subspaces, we can use it to construct orthogonal projection matrices onto each of these subspaces.

Theorem 47.5. (Orthogonal projection matrices from the SVD)

Let $A \in \mathbb{R}^{m\times n}$ with $\operatorname{rank}(A) = r$, and let $A = U\Sigma V^T$ be a singular value decomposition of A. Partition U and V as $U = \begin{bmatrix} U_1 & U_2 \end{bmatrix}$ and $V = \begin{bmatrix} V_1 & V_2 \end{bmatrix}$, where U_1 is $m \times r$ and V_1 is $n \times r$. Then:

(a) The matrices $U_1U_1^T$ and $U_2U_2^T$ are the orthogonal projection matrices onto the column space $\operatorname{col}(A)$ and the left nullspace $\operatorname{null}(A^T)$, respectively.
(b) The matrices $V_1V_1^T$ and $V_2V_2^T$ are the orthogonal projection matrices onto the row space $\operatorname{col}(A^T)$ and the nullspace $\operatorname{null}(A)$, respectively.

The SVD of a square matrix A offers an easy test for invertibility and, when A is invertible, a direct way to compute A^{-1}. The proof is left as Problem 47.6.

Theorem 47.6. (The SVD of the inverse)

Let $A \in \mathbb{R}^{n\times n}$, let $A = U\Sigma V^T$ be a singular value decomposition of A, and let $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_n \geqslant 0$ denote its singular values. The matrix A is invertible if and only if $\sigma_n > 0$. Moreover, if A is invertible, then a singular value decomposition of A^{-1} is given by

$$A^{-1} = V\Sigma^{-1}U^T \qquad \text{with} \qquad \Sigma^{-1} = \operatorname{diag}\left(\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \ldots, \frac{1}{\sigma_n}\right).$$

Theorem 47.6 shows that, when you invert a matrix, the singular values become the reciprocals of the original singular values, and the right and left singular vectors swap roles.

Recall that the determinant of a matrix is the product of its eigenvalues. Similarly, up to a sign, it is also the product of its singular values.

Theorem 47.7. (Singular values and the determinant)

Let $A \in \mathbb{R}^{n\times n}$ and let $A = U\Sigma V^T$ be a singular value decomposition of A. If $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_n \geqslant 0$ are the singular values of A, then

$$|\det(A)| = \sigma_1\sigma_2\cdots\sigma_n.$$

Proof From $A = U\Sigma V^T$, we have $\det(A) = \det(U)\det(\Sigma)\det(V^T)$. Since U and V are orthogonal, we have $\det(U), \det(V^T) = \pm 1$. Thus, it follows that $|\det(A)| = \det(\Sigma)$. As Σ is diagonal with singular values $\sigma_1, \ldots, \sigma_n$, $\det(\Sigma) = \sigma_1\sigma_2\cdots\sigma_n$. Hence, $|\det(A)| = \sigma_1\sigma_2\cdots\sigma_n$. □

Finally, we consider the special case of matrices A for which the matrices U and V in an SVD are equal, that is,

$$A = U\Sigma U^T,$$

with U orthogonal and Σ a diagonal matrix with nonnegative entries. This form forces several important properties on A.

First, both A and Σ must be square, since $U\Sigma U^T$ is defined only for square matrices. Second, A is symmetric because

$$A^T = (U\Sigma U^T)^T = U\Sigma U^T = A.$$

Third, $A = U\Sigma U^T$ is a similarity transformation, so the diagonal entries of Σ are the eigenvalues of A and are nonnegative. Therefore A is symmetric and positive semidefinite (see Theorem 42.23).

Conversely, if A is symmetric and positive semidefinite, the Spectral Theorem gives

$$A = QDQ^T,$$

with Q orthogonal and D diagonal with nonnegative entries. This factorization is precisely an SVD of A.

These observations lead to the following result.

Proposition 47.8. (SVD characterization of symmetric positive semidefinite matrices)

Let $A \in \mathbb{R}^{n\times n}$. The matrix A has an SVD of the form $A = U\Sigma U^T$ if and only if A is symmetric positive semidefinite.

47.1.3 When is the singular value decomposition unique?

Every matrix has an SVD, but is the factorization unique? In most cases the answer is no. As we noted in the proof of Theorem 47.1, the left and right singular vectors can often be chosen in more than one way, so different SVDs are possible. Below we explain where this freedom comes from.

Recall that the first r right singular vectors $v_1, \ldots, v_r$ of a matrix A form an orthonormal basis for the row space of A and are eigenvectors of the positive semidefinite matrix A^TA corresponding to its positive eigenvalues. If one of these eigenvalues has an algebraic multiplicity greater than one, the associated eigenspace is multidimensional, and we can choose an orthonormal basis for it in infinitely many different ways. Even when an eigenvalue is simple (has multiplicity one), its corresponding eigenvector is only determined up to a sign. Similarly, the last $n - r$ columns of V form an orthonormal basis for $\text{null}(A)$, and these can be chosen arbitrarily without significantly affecting the decomposition.

Once a right singular vector v_i corresponding to a nonzero singular value σ_i has been chosen, the matching left singular vector is determined by

$$u_i = \frac{Av_i}{\sigma_i}.$$

Thus, the left and right singular vectors cannot be chosen independently. Hence, the first r columns of U are uniquely determined by the relations above. However, the remaining $m - r$ columns of U form an orthonormal basis for the left nullspace $\text{null}(A^T)$ and can be chosen freely.

In contrast, the singular values (the diagonal entries of Σ) are uniquely determined. The nonzero singular values are the square roots of the positive eigenvalues of $A^T A$, and the number of zero singular values is exactly $\min\{m, n\} - r$, where r is the rank of A.

Putting it all together, the SVD of a matrix A is unique, up to the signs of the singular vectors, only in a very specific case. First, A must have trivial right and left nullspaces, which means it must be square and invertible. Second, each eigenspace of $A^T A$ must be one-dimensional, that is, the singular values must be distinct. In that case, the only remaining ambiguity is the sign of each singular vector.

The following theorem summarizes these observations.

Theorem 47.9. (Uniqueness of the SVD)

Let $A \in \mathbb{R}^{m \times n}$. Its singular values $\sigma_1, \ldots, \sigma_p$, where $p = \min\{m, n\}$, are uniquely determined. If A is square, invertible, and has distinct singular values, its left and right singular vectors $u_1, \ldots, u_n$ and $v_1, \ldots, v_n$ are uniquely determined, except for their signs.

Let us consider some examples where the SVD is not unique. We start with a matrix whose singular values are all equal.

Example 47.10 Consider the 3×3 identity matrix $I_3 = \text{diag}(1, 1, 1)$. Since $I_3^T I_3 = I_3$, its singular value is 1 (repeated three times). This means that for any orthogonal matrix U, the decomposition

$$I_3 = U I_3 U^T$$

is an SVD of I_3.

Now, let us consider an example of a matrix with distinct singular values but infinitely many possible SVD factorizations.

Example 47.11 Consider the matrix

$$A = \frac{1}{\sqrt{8}} \begin{bmatrix} 1 & 3 \\ 3 & 1 \\ 1 & 3 \\ 3 & 1 \end{bmatrix}.$$

The singular values of A are $\sigma_1 = 2$ and $\sigma_2 = 1$. Since A has full column rank, $\text{null}(A) = \{0\}$. This fixes the right singular vectors (columns of V) up to sign changes. The null space of A^T has dimension 2, so only the first two columns of

U are fixed; the last two can be any orthonormal basis for $\text{null}(A^T)$. As a result, A admits infinitely many SVDs.

Here are two distinct SVD factorizations of A:

$$A = \begin{bmatrix} 1/2 & 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 & -1/2 \\ 1/2 & 1/2 & -1/2 & -1/2 \\ 1/2 & -1/2 & -1/2 & 1/2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{2} & -1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}$$

$$= \begin{bmatrix} -1/2 & 1/2 & -1/\sqrt{2} & 0 \\ -1/2 & -1/2 & 0 & -1/\sqrt{2} \\ -1/2 & 1/2 & 1/\sqrt{2} & 0 \\ -1/2 & -1/2 & 0 & 1/\sqrt{2} \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} -1/\sqrt{2} & -1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}.$$

In the second factorization, we changed the signs of the first column of both U and V. Moreover, the last two columns of U are chosen differently in the two factorizations, although they both span the same subspace, $\text{null}(A^T)$.

47.1.4 The geometry of the singular value decomposition

Just as the spectral decomposition offers a geometric view of positive definite matrices (see Chapter 43), the SVD provides a geometric picture for any matrix.

The singular value decomposition reveals a not-so-obvious geometric fact: the image of the unit sphere under a linear transformation is an ellipsoid. The axes of this ellipsoid are aligned with the left singular vectors, and their lengths are determined by the singular values.

Figure 47.4 illustrates this idea using a 2×2 matrix A. The figure shows the image of the unit circle $\mathcal{B}$ under A, and it also displays how a unit vector x is transformed under the action of A into $y = Ax$. When we express y in the basis of left singular vectors $\{u_1, u_2\}$, its coordinates w_1 and w_2 satisfy the canonical ellipse equation

$$\frac{w_1^2}{\sigma_1^2} + \frac{w_2^2}{\sigma_2^2} = 1,$$

as we show in Theorem 47.12.

Next, we prove these observations in the special case when $A \in \mathbb{R}^{n \times n}$ is nonsingular. The singular and rectangular cases will be addressed later in Problem 47.13.

Theorem 47.12. (The image of the unit sphere under a nonsingular matrix)

Let $A \in \mathbb{R}^{n \times n}$ be a nonsingular matrix with singular values $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_n > 0$ and corresponding left singular vectors $u_1, u_2, \ldots, u_n$. The set

$$\mathcal{E} = \{Ax \mid \|x\|_2 = 1\}$$

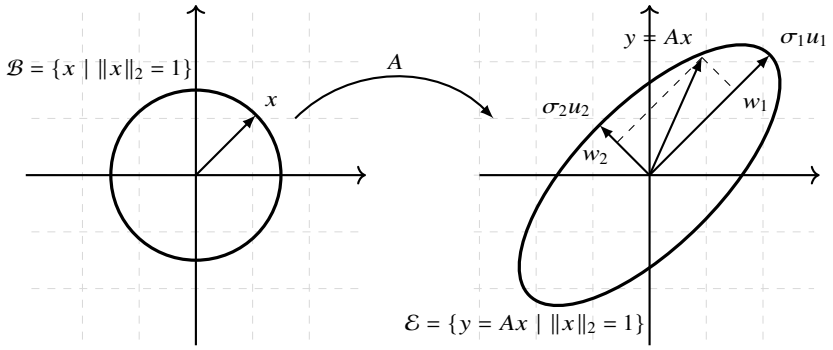


Fig. 47.4: The figure shows the image of the unit circle under a 2×2 matrix. The unit sphere transforms into an ellipse. The ellipse aligns with the left singular vectors and is scaled by the singular values.

is an n-dimensional (nondegenerate) ellipsoid. The principal axes of the ellipsoid are the vectors $\{\sigma_1 u_1, \sigma_2 u_2, \ldots, \sigma_n u_n\}$, with lengths $\sigma_1, \sigma_2, \ldots, \sigma_n$.

Proof Let $A = U\Sigma V^T$ be a singular value decomposition of A, where $\Sigma = \operatorname{diag}(\sigma_1, \sigma_2, \ldots, \sigma_n)$. Let $x \in \mathbb{R}^n$ be any unit vector, and define $y = Ax$. Let $w := U^T y$, where $U = \begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix}$ is the matrix of left singular vectors of A. Note that w is the vector of coordinates of the vector y in the basis U since $y = Uw$, a linear combination of the columns of U with coefficients the entries of w. We want to show that the entries $w_1, w_2, \ldots, w_n$ of w satisfy the canonical equation for an ellipsoid, that is,

$$\frac{w_1^2}{\sigma_1^2} + \frac{w_2^2}{\sigma_2^2} + \cdots + \frac{w_n^2}{\sigma_n^2} = 1.$$

Since x is a unit vector, we have

$$\begin{aligned} 1 = \|x\|_2^2 = \|A^{-1}Ax\|_2^2 &= \|A^{-1}y\|_2^2 \\ &= \|V\Sigma^{-1}(U^T y)\|_2^2 \\ &= \|\Sigma^{-1}w\|_2^2 \\ &= \frac{w_1^2}{\sigma_1^2} + \frac{w_2^2}{\sigma_2^2} + \cdots + \frac{w_n^2}{\sigma_n^2}, \end{aligned}$$

where the fifth equality follows from the fact that orthogonal matrices preserve the 2-norm. □

We can also use the singular value decomposition to interpret any linear transformation as a composition of simpler geometric transformations. When we decompose a matrix A as $A = U\Sigma V^T$, the linear transformation $x \mapsto Ax$ can be understood as the composition of three transformations:

1. **An isometry** $x \mapsto V^T x$.
2. **A scaling** $x \mapsto \Sigma x$, which stretches or compresses the coordinates of x by the singular values.
3. **Another isometry** $x \mapsto Ux$.

Remember that isometries preserve distances. In the linear context, an isometry usually means a rotation, a reflection, or a combination of both, depending on the sign of its determinant.

Figure 47.5 illustrates the composition of these transformations for a 2×2 non-singular matrix:

$$A = U\Sigma V^T = \begin{bmatrix} u_1 & u_2 \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix} \begin{bmatrix} v_1^T \\ v_2^T \end{bmatrix}, \quad \text{where } \sigma_1 \geqslant \sigma_2 > 0.$$

The first transformation, $x \mapsto V^T x$, maps the right singular vectors v_1 and v_2 to the canonical basis vectors e_1 and e_2:

$$V^T v_1 = \begin{bmatrix} v_1^T v_1 \\ v_2^T v_1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \text{and} \qquad V^T v_2 = \begin{bmatrix} v_1^T v_2 \\ v_2^T v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Since V^T is an isometry, it preserves distances and maps the unit circle onto itself. The second transformation, $x \mapsto \Sigma x$, stretches or compresses the canonical basis vectors e_1, e_2 by the singular values:

$$\Sigma e_1 = \sigma_1 e_1, \quad \Sigma e_2 = \sigma_2 e_2.$$

The transformation maps the unit circle onto an ellipse with axes parallel to e_1, e_2 and lengths determined by σ_1 and σ_2. The final transformation, $x \mapsto Ux$, maps the canonical basis vectors e_1, e_2 to the left singular vectors u_1, u_2:

$$Ue_1 = u_1, \quad Ue_2 = u_2.$$

Since U is also an isometry, it preserves the shape of the ellipse while rotating or reflecting it. The final ellipse aligns its axes with the directions of the left singular vectors u_1 and u_2, with lengths given by σ_1 and σ_2.

47.1.5 The reduced singular value decomposition

The reduced SVD is a compact version of the standard SVD, especially useful for rectangular or rank-deficient matrices. Like the reduced QR factorization, it removes the unnecessary rows and columns from the full decomposition.

Consider an $m \times n$ matrix A with rank r. The standard SVD expresses A as $A = U\Sigma V^T$, where

$$\Sigma = \begin{array}{c} \begin{matrix} r & n-r \end{matrix} \\ \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{matrix} r \\ m-r \end{matrix} \end{array} \qquad \text{and} \qquad \Sigma_r = \operatorname{diag}(\sigma_1, \sigma_2, \ldots, \sigma_r).$$

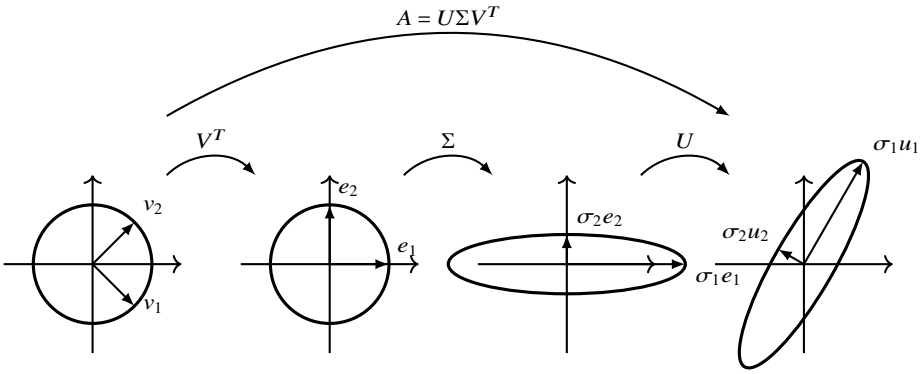


Fig. 47.5: The SVD decomposes the linear transformation $x \mapsto Ax$ into three simple geometric transformations: an isometry ($V^T x$), a scaling (Σx), and another isometry (Ux). These transformations map the unit circle into an ellipse, with axes aligned to the left singular vectors and lengths determined by the singular values.

We will refer to this factorization as a **full SVD** of A. The matrix Σ has dimensions $m \times n$, the same as A, but includes some zero rows and zero columns that are not essential.

The reduced SVD provides a more compact representation by removing these unnecessary rows and columns. To derive it, partition U and V as

$$U = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \quad \text{and} \quad V = \begin{bmatrix} V_1 & V_2 \end{bmatrix},$$

where U_1 and V_1 contain the first r columns of U and V, respectively. Using this partition, we can write A as

$$A = U\Sigma V^T = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix} = U_1 \Sigma_r V_1^T.$$

We will refer to $A = U_1 \Sigma_r V_1^T$ as a **reduced SVD** of A.

In this compact form, Σ_r is an $r \times r$ diagonal matrix containing only the nonzero singular values $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_r > 0$ of A. This makes Σ_r potentially much smaller than the full Σ. The matrix U_1 is $m \times r$, and its columns, the left singular vectors corresponding to the nonzero singular values, form an orthonormal basis for $\mathrm{col}(A)$. Similarly, V_1 is $n \times r$, and its columns form an orthonormal basis for $\mathrm{col}(A^T)$.

In the terminology of Chapter 17, a reduced SVD factorization $A = U_1 \Sigma_r V_1^T$ is a full rank factorization, with Σ_r as the mixing matrix.

Figure 47.6 illustrates the difference between the full SVD and the reduced SVD of a matrix A. The shaded areas indicate the (potentially) nonzero entries in the matrices.

Example 47.13 Let us revisit Example 47.2, where we calculated an SVD of a rank-1 matrix A. A reduced SVD for this matrix is given by:

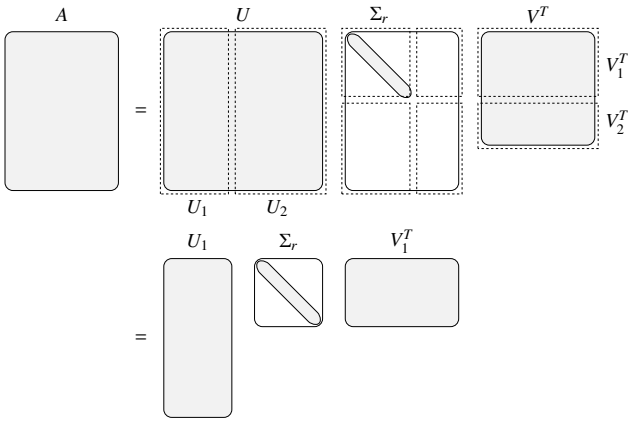


Fig. 47.6: Full (top) and reduced (bottom) singular value decomposition of a matrix A.

$$A = \begin{bmatrix} 2 & -6 \\ 1 & -3 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 2/\sqrt{5} \\ 1/\sqrt{5} \\ 0 \end{bmatrix} \begin{bmatrix} 5\sqrt{2} \end{bmatrix} \begin{bmatrix} 1/\sqrt{10} & -3/\sqrt{10} \end{bmatrix}.$$

You might wonder if removing the zero rows and columns from Σ makes the reduced SVD unique. Unfortunately, it does not. Even in the reduced form, if a matrix has repeated singular values, it can have multiple reduced SVDs. In Chapter 48, we will fully characterize all possible reduced SVDs of a matrix.

47.2 The singular value decomposition and optimization

Singular values and singular vectors, much like the eigenvalues and eigenvectors of symmetric matrices, arise as solutions to important optimization problems.

Theorem 47.14. (Optimization characterization of singular values)

Let $A \in \mathbb{R}^{m \times n}$, with $m \geqslant n$, and let $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_n \geqslant 0$ be its singular values, with corresponding right singular vectors $v_1, v_2, \ldots, v_n$. The largest and smallest singular values, σ_1 and σ_n, are solutions to the following constrained optimization problems:

$$\sigma_1 = \max_{\|x\|_2 = 1} \|Ax\|_2, \quad \text{and} \tag{47.5}$$

$$\sigma_n = \min_{\|x\|_2 = 1} \|Ax\|_2. \tag{47.6}$$

The maximum (47.5) is achieved when $x = v_1$, and the minimum (47.6) is achieved when $x = v_n$.

The kth singular value σ_k of A satisfies the following constrained optimization problems:

$$\sigma_k = \max_{\substack{\|x\|_2=1 \\ x \perp v_1,\ldots,v_{k-1}}} \|Ax\|_2, \quad \text{and} \tag{47.7}$$

$$\sigma_k = \min_{\substack{\|x\|_2=1 \\ x \perp v_{k+1},\ldots,v_n}} \|Ax\|_2. \tag{47.8}$$

The maximum (47.7) and minimum (47.8) are achieved when $x = v_k$.

Proof The results follow directly from the Rayleigh-Ritz characterization of eigenvalues for symmetric matrices (Theorem 42.12), noting that $\|Ax\|_2 = \sqrt{x^T A^T A x}$, and the singular values of A are the square roots of the eigenvalues of $A^T A$. □

We can provide a geometric interpretation of (47.5) and (47.6) as follows. The largest singular value, σ_1, represents the maximum stretch that the linear transformation A can apply to any unit vector, while the smallest singular value represents the minimum such stretch.

Recall from Chapter 12 that the induced matrix norm is defined as

$$\|A\| = \max_{\|x\|=1} \|Ax\|,$$

with respect to a given vector norm $\|\cdot\|$. In that chapter, we derived explicit formulas for $\|A\|_1$ and $\|A\|_\infty$, but we postponed the derivation of a formula for the 2-norm $\|A\|_2$, the most important of these norms. Now, using Theorem 47.14, we can express $\|A\|_2$ in terms of the largest singular value of A.

Corollary 47.15. (The 2-norm of a matrix and its inverse)

Let $A \in \mathbb{R}^{m\times n}$, let $p = \min\{m, n\}$, and let $\sigma_1 \geqslant \cdots \geqslant \sigma_p \geqslant 0$ be the singular values of A. Then, $\|A\|_2 = \sigma_1$. If A is invertible (hence, $p = m = n$), then $\|A^{-1}\|_2 = 1/\sigma_n$.

The matrix 2-norm is not the only norm we can express using singular values; the Frobenius norm is another important example.

Theorem 47.16. (Singular values and the Frobenius norm)

Let $A \in \mathbb{R}^{m\times n}$, and let $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_r > 0$ be the nonzero singular values of A. Then, the Frobenius norm of A

$$\|A\|_F = \sqrt{\sigma_1^2 + \sigma_2^2 + \cdots + \sigma_r^2}.$$

Proof Let $A = U_1 \Sigma_r V_1^T$ be a reduced SVD of A. The square of the Frobenius norm of A is given by

$$\begin{aligned}\|A\|_F^2 &= \operatorname{trace}(A^T A)\\ &= \operatorname{trace}((U_1\Sigma_r V_1^T)^T(U_1\Sigma_r V_1^T))\\ &= \operatorname{trace}(V_1\Sigma_r^T U_1^T U_1\Sigma_r V_1^T)\\ &= \operatorname{trace}(V_1\Sigma_r\Sigma_r V_1^T)\\ &= \operatorname{trace}(V_1^T V_1\Sigma_r^2)\\ &= \operatorname{trace}(\Sigma_r^2) = \sigma_1^2+\sigma_2^2+\cdots+\sigma_r^2.\end{aligned}$$

Here, we used the cyclic property of the trace ($\operatorname{trace}(AB) = \operatorname{trace}(BA)$) and the fact that Σ_r^2 is (square) diagonal with main diagonal entries $\sigma_1^2, \sigma_2^2, \ldots, \sigma_r^2$. □

The relationship between matrix norms and singular values goes beyond just the Frobenius and 2-norms. In fact, these two are part of a broader family known as the **Schatten norms** , all defined in terms of the singular values (see Problem 47.15). Among these, the most important are the Frobenius norm, the 2-norm, and the **nuclear norm** , which is defined as

$$\|A\|_* := \sigma_1 + \sigma_2 + \cdots + \sigma_r,$$

where $r = \operatorname{rank}(A)$.

47.3 The pseudoinverse matrix

One nice application of the SVD is that it lets us compute the pseudoinverse (also called the Moore-Penrose inverse), written as A^+. The pseudoinverse acts like an inverse for any matrix, even if the matrix is not square or is singular. What does this mean? We want AA^+ and A^+A to be as close as possible to the identity matrices I_m and I_n. In other words, for a linear map $L_A(x) = Ax$, we build another map $L_{A^+}(x) = A^+x$ so that $L_A \circ L_{A^+}$ and $L_{A^+} \circ L_A$ behave as much like the identity transformation as possible.

Suppose A is an $m \times n$ matrix, which defines a linear transformation $L_A : \mathbb{R}^n \to \mathbb{R}^m$. According to Theorem 27.12, we can decompose the domain and codomain of L_A as the direct sum of orthogonal subspaces:

$$\mathbb{R}^n = \operatorname{col}(A^T) \oplus \operatorname{null}(A) \quad \text{and} \quad \mathbb{R}^m = \operatorname{col}(A) \oplus \operatorname{null}(A^T).$$

A matrix A is singular when it has a nontrivial nullspace $\operatorname{null}(A)$ or left nullspace $\operatorname{null}(A^T)$. When $\operatorname{null}(A) \neq \{0\}$, the transformation L_A is not one-to-one. Similarly, if $\operatorname{null}(A^T) \neq \{0\}$, L_A is not onto.

However, if we restrict the domain of L_A to $\operatorname{col}(A^T)$ and its codomain to $\operatorname{col}(A)$, then L_A becomes invertible, as will be shown in Theorem 47.17. By restricting the domain to $\operatorname{col}(A^T)$ and the codomain to $\operatorname{col}(A)$, every vector $b \in \operatorname{col}(A)$ is the image of exactly one vector $x \in \operatorname{col}(A^T)$ under A, which allows us to "invert" the action of A. Figure 47.7 illustrates this.

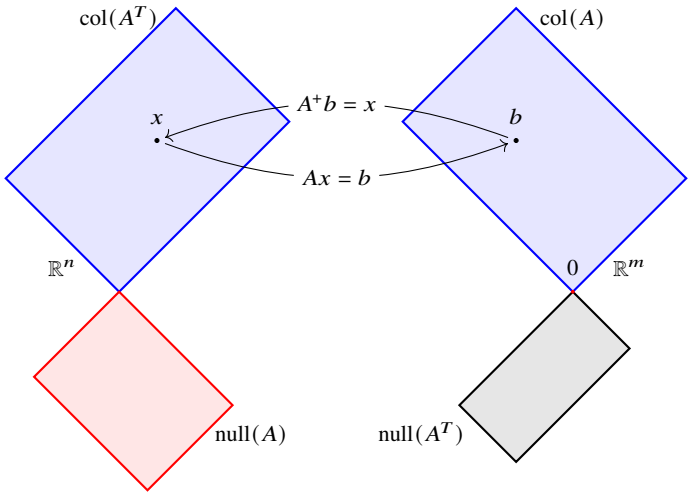


Fig. 47.7: The pseudoinverse matrix A^+ "inverts" A where that is possible.

Theorem 47.17. (Invertibility of $x \mapsto Ax$ from $\text{col}(A^T)$ to $\text{col}(A)$)

Let $A \in \mathbb{R}^{m\times n}$. The linear transformation $L_A : x \mapsto Ax$, when restricted from $\text{col}(A^T)$ to $\text{col}(A)$, is bijective and hence invertible. In other words, for every $b \in \text{col}(A)$, there is exactly one $x \in \text{col}(A^T)$ such that $Ax = b$.

Proof To prove bijectivity, we need to show that L_A is both onto and one-to-one.

We begin with onto. Let $b \in \text{col}(A)$. By definition, $b = Ax$ for some $x \in \mathbb{R}^n$. Since $\text{col}(A^T)$ and $\text{null}(A)$ are orthogonal complementary subspaces, we can write $x = x_r + x_n$, where $x_r \in \text{col}(A^T)$ and $x_n \in \text{null}(A)$. Because $Ax_n = 0$, we have $b = A(x_r + x_n) = Ax_r$, which shows that b is the image of a vector in $\text{col}(A^T)$. This proves that the restricted version of L_A is onto.

Since $\dim(\text{col}(A^T)) = \dim(\text{col}(A))$, the linear transformation L_A is bijective. □

Since the map $x \mapsto Ax$ is invertible when we restrict the domain to $\text{col}(A^T)$ and the codomain to $\text{col}(A)$, and the inverse of a linear transformation is also linear, there is a matrix A^+ that acts as an inverse on these subspaces. That is, A^+A acts as the identity on $\text{col}(A^T)$, and AA^+ acts as the identity on $\text{col}(A)$. This matrix A^+ is called the **pseudoinverse** of A. [2]

The pseudoinverse generalizes the concept of a standard matrix inverse. It was first introduced by mathematician Eliakim Moore in the 1920s [41] and later rediscovered by physicist Roger Penrose in the 1950s [44]. For this reason, A^+ is often referred to in the literature as the **Moore-Penrose inverse of** A.

[2] This definition does not uniquely determine A^+. We also need to describe how A^+ acts on the orthogonal complement of $\text{col}(A)$, which is $\text{null}(A^T)$. We will handle this in the next section.

47.3.1 Definition and characterizations of the pseudoinverse

In this section, we present three equivalent definitions of the pseudoinverse, each offering a different perspective on A^+. The first is a geometric perspective (Definition 47.18), which describes how A^+ acts on vectors. The second is a factorization perspective (Theorem 47.19), where the pseudoinverse is derived from a singular value decomposition of A. The third is an algebraic perspective (Theorem 47.21), based on a set of equations that A^+ satisfies.

In Chapter 48, we will introduce two additional characterizations. One is an optimization perspective (Theorem 48.8), which views A^+ as the solution to an optimization problem, and the other is a limit perspective (Theorem 48.9), where A^+ is defined as the limit of a matrix expression.

In the previous section, we have described how A^+ acts on vectors in $\text{col}(A)$. To fully define the linear transformation $L_{A^+} : \mathbb{R}^m \to \mathbb{R}^n$, we must also specify its action on vectors in $\text{null}(A^T)$.

Definition 47.18. (The pseudoinverse matrix)

Let $A \in \mathbb{R}^{m\times n}$. The **pseudoinverse** of A, denoted by A^+, is the unique $n \times m$ matrix that satisfies the following properties:

(i) $A^+(Ax) = x$, for any $x \in \text{col}(A^T)$; and
(ii) $A^+x = 0$, for any $x \in \text{null}(A^T)$.

Notice that condition (i) in the definition above corresponds exactly to what Figure 47.7 shows. When x is in the row space of A, the transformation Ax maps x into the column space of A, and A^+ reverses this process, recovering x.

Let us now verify that conditions (i) and (ii) uniquely define the matrix A^+ in Definition 47.18. We need to show that these conditions completely determine how A^+ acts on any vector $y \in \mathbb{R}^m$, thus proving that the pseudoinverse exists and is unique.

Take any vector $y \in \mathbb{R}^m$. Since $\text{col}(A)$ and $\text{null}(A^T)$ are orthogonal complementary subspaces, we can write

$$y = y_c + y_\ell,$$

where $y_c \in \text{col}(A)$ and $y_\ell \in \text{null}(A^T)$. By condition (ii), we have $A^+y_\ell = 0$.

For $y_c \in \text{col}(A)$, Theorem 47.17 guarantees that there exists a unique vector $x \in \text{col}(A^T)$ such that $Ax = y_c$. Then, by condition (i),

$$A^+y_c = A^+(Ax) = x.$$

Thus, combining these results, we obtain

$$A^+y = A^+y_c + A^+y_\ell = x,$$

as illustrated in Figure 47.8. Therefore, conditions (i) and (ii) fully determine the action of A^+ on any vector $y \in \mathbb{R}^m$.

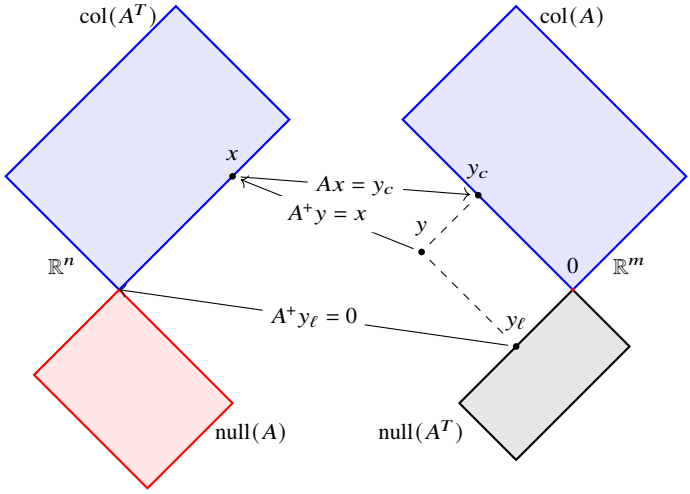


Fig. 47.8: The action of the pseudoinverse A^+ on an arbitrary vector $y \in \mathbb{R}^m$.

Conditions (i) and (ii) in Definition 47.18 uniquely characterize the pseudoinverse, but they do not offer a direct way to compute it. So, how can we determine the pseudoinverse of a given matrix? The following theorem shows how to use the singular value decomposition to compute A^+.

Theorem 47.19. (Pseudoinverse via the SVD)

Let $A \in \mathbb{R}^{m\times n}$ be a rank-r matrix with singular value decomposition

$$A = U\Sigma V^T = U\begin{bmatrix}\Sigma_r & 0\\ 0 & 0\end{bmatrix}V^T, \quad \text{where} \quad \Sigma_r = \operatorname{diag}(\sigma_1,\ldots,\sigma_r),$$

and $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_r > 0$ are the nonzero singular values of A. The pseudoinverse of A is the $n \times m$ matrix

$$A^+ = V\Sigma^+U^T = V\begin{bmatrix}\Sigma_r^{-1} & 0\\ 0 & 0\end{bmatrix}U^T, \quad \text{where} \quad \Sigma_r^{-1} = \operatorname{diag}\left(\frac{1}{\sigma_1},\ldots,\frac{1}{\sigma_r}\right).$$

For the special case $A = 0_{m\times n}$, the pseudoinverse is $A^+ = 0_{n\times m}$.

Proof We start by partitioning the orthogonal matrices U and V as $U = \begin{bmatrix}U_1 & U_2\end{bmatrix}$ and $V = \begin{bmatrix}V_1 & V_2\end{bmatrix}$, where U_1 and V_1 contain the first r columns of U and V, respectively. We will show that $A^+ = V\Sigma^+U^T$ satisfies the two conditions in Definition 47.18.

First, consider $x \in \operatorname{col}(A^T)$. Since the columns of V_1 form a basis for $\operatorname{col}(A^T)$, we can write $x = V_1 c$ for some vector $c \in \mathbb{R}^r$. Then

$$\begin{aligned} A^+Ax &= \begin{bmatrix} V_1 & V_2 \end{bmatrix} \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & 0 \end{bmatrix} U^T U \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix} V_1 c \\ &= \begin{bmatrix} V_1 & V_2 \end{bmatrix} \begin{bmatrix} I_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1^T V_1 c \\ V_2^T V_1 c \end{bmatrix} \\ &= \begin{bmatrix} V_1 & V_2 \end{bmatrix} \begin{bmatrix} c \\ 0 \end{bmatrix} \\ &= V_1 c = x. \end{aligned}$$

We used the orthonormality of U in the second equality (i.e. $U^TU = I_m$) and the orthonormality of V_1 (i.e., $V_1^TV_1 = I_r$) in the third step. Thus, A^+ satisfies condition (i) in Definition 47.18.

Next, let $x \in \text{null}(A^T)$. This implies that $V\Sigma^T U^T x = 0$. Since V is invertible, we have $\Sigma^T U^T x = 0$, or equivalently $\Sigma_r U_1^T x = 0$. Since Σ_r is invertible, it follows that $U_1^T x = 0$. We now show that $A^+x = 0$. We get

$$A^+x = V\Sigma^+U^Tx = V \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} U_1^T \\ U_2^T \end{bmatrix} x = V \begin{bmatrix} \Sigma_r^{-1} U_1^T x \\ 0 \end{bmatrix} = 0.$$

Thus, A^+ satisfies condition (ii) in Definition 47.18. □

Reduced SVD of A^+

From Theorem 47.19 we notice that, if $A = U_1\Sigma_r V_1^T$ is a reduced SVD of A, then $A^+ = V_1\Sigma_r^{-1}U_1^T$ is a reduced SVD of A^+.

Example 47.20 Consider the matrix A from Example 47.2, with the singular value decomposition

$$A = \begin{bmatrix} 2 & -6 \\ 1 & -3 \\ 0 & 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 2/\sqrt{5} & 1/\sqrt{5} & 0 \\ 1/\sqrt{5} & -2/\sqrt{5} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{=U} \underbrace{\begin{bmatrix} 5\sqrt{2} & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}}_{=\Sigma} \underbrace{\begin{bmatrix} 1/\sqrt{10} & -3/\sqrt{10} \\ 3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix}}_{=V^T}.$$

To compute A^+ from the SVD of A, we first find the pseudoinverse of Σ. For this purpose, transpose Σ, take the reciprocal of each nonzero singular value $(1/\sigma_i)$ and leave the zero singular values unchanged, that is,

$$\Sigma^+ = \begin{bmatrix} 1/(5\sqrt{2}) & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Now, we form A^+ using the formula $A^+ = V\Sigma^+U^T$:

$$A^+ = V\Sigma^+U^T = \begin{bmatrix} 1/\sqrt{10} & 3/\sqrt{10} \\ -3/\sqrt{10} & 1/\sqrt{10} \end{bmatrix} \begin{bmatrix} 1/(5\sqrt{2}) & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 2/\sqrt{5} & 1/\sqrt{5} & 0 \\ 1/\sqrt{5} & -2/\sqrt{5} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
$$= \frac{1}{50} \begin{bmatrix} 2 & 1 & 0 \\ -6 & -3 & 0 \end{bmatrix}.$$

Next, we show that the pseudoinverse can also be defined as the unique matrix that satisfies four equations. This was the original definition given by R. Penrose [44].

Theorem 47.21. (Algebraic characterization of the pseudoinverse)

Let $A \in \mathbb{R}^{m\times n}$. The pseudoinverse $A^+ \in \mathbb{R}^{n\times m}$ is the unique matrix that satisfies the following four equations:

(i) $AA^+A = A$, (ii) $A^+AA^+ = A^+$,
(iii) $(AA^+)^T = AA^+$, and (iv) $(A^+A)^T = A^+A$.

Proof Problem 47.16 outlines the steps to prove this result. □

Theorem 47.21 provides a simple way to check whether a matrix is the pseudoinverse of A or not. To verify a candidate A^+, we simply check if it satisfies the four conditions. In the next section, we will use this approach to determine the pseudoinverse of some special matrices.

Before that, let us see how Theorem 47.21 extends the standard definition of a matrix inverse. For an invertible matrix $A \in \mathbb{R}^{n\times n}$, the inverse A^{-1} is uniquely defined by

(i) $AA^{-1} = I_n$, (ii) $A^{-1}A = I_n$,
(iii) $(AA^{-1})^T = AA^{-1} = I_n$, (iv) $(A^{-1}A)^T = A^{-1}A = I_n$.

Conditions (iii) and (iv) in Theorem 47.21 generalize these same conditions for the usual inverse. Regarding conditions (i) and (ii), although AA^+ and A^+A are generally not the identity matrix, they act like the identity on the subspaces $\text{col}(A)$ and $\text{col}(A^T)$, respectively. Specifically, for every x,

$$(AA^+)Ax = Ax,$$

which corresponds to condition (i) in Theorem 47.21. Similarly,

$$(A^+A)A^Tx = A^Tx.$$

Later, Theorem 47.32 shows that $\text{col}(A^T) = \text{col}(A^+)$, which leads to condition (ii) in Theorem 47.21.

47.3.2 Pseudoinverse of some special matrices

For some matrices, computing the pseudoinverse is very simple. For instance, if A is the $m \times n$ zero matrix, then its pseudoinverse is the $n \times m$ zero matrix. In this section, we examine a few other types of matrices whose pseudoinverses are similarly easy to determine. We omit the proofs because you can easily verify that each candidate satisfies the four conditions in Theorem 47.21.

Let us begin with the case of diagonal matrices.

Proposition 47.22. (Pseudoinverse of a diagonal matrix)

The pseudoinverse of a diagonal matrix $A = \operatorname{diag}(d_1, \ldots, d_n)$ is the diagonal matrix $A^+ = \operatorname{diag}(c_1, \ldots, c_n)$, where

$$c_i = \begin{cases} 1/d_i & \text{if } d_i \neq 0, \\ 0 & \text{if } d_i = 0, \end{cases}$$

for $i = 1, \ldots, n$.

We can extend the result for diagonal matrices to block diagonal matrices.

Proposition 47.23. (Pseudoinverse of a block diagonal matrix)

The pseudoinverse of the block diagonal matrix

$$A = \begin{bmatrix} A_1 & & & \\ & A_2 & & \\ & & \ddots & \\ & & & A_k \end{bmatrix}$$

is the block diagonal matrix

$$A^+ = \begin{bmatrix} A_1^+ & & & \\ & A_2^+ & & \\ & & \ddots & \\ & & & A_k^+ \end{bmatrix}.$$

When A has full column rank or full row rank, we can derive a closed-form formula for its pseudoinverse.

Proposition 47.24. (Pseudoinverse of full-rank matrices)

Let $A \in \mathbb{R}^{m \times n}$.

(a) If A has full column rank, $A^T A$ is nonsingular, and

$$A^+ = (A^T A)^{-1} A^T,$$

so A^+ is a *left inverse* of A (i.e., $A^+ A = I_n$).

(b) If A has full row rank, AA^T is nonsingular, and

$$A^+ = A^T (AA^T)^{-1},$$

so A^+ is a *right inverse* of A (i.e. $AA^+ = I_m$).

(c) If A is square and invertible, then

$$A^+ = A^{-1}.$$

Example 47.25. (The pseudoinverse of a matrix with orthonormal columns)

If Q is a matrix with orthonormal columns, then it has full column rank and

$$Q^+ = (Q^T Q)^{-1} Q^T = Q^T.$$

Example 47.26. (The pseudoinverse of a nonzero vector)

Consider a nonzero (column) vector $x \in \mathbb{R}^n$. Notice that x is a full column matrix. By Proposition 47.24, we have

$$x^+ = \frac{x^T}{x^T x} = \frac{1}{\|x\|_2^2} x^T.$$

We can extend the formula from Example 47.26 to any rank-one matrix.

Proposition 47.27. (Pseudoinverse of a rank-one matrix)

Let $A \in \mathbb{R}^{m \times n}$ be a rank-one matrix. Let $u \in \mathbb{R}^m$ and $v \in \mathbb{R}^n$ be two nonzero vectors such that $A = uv^T$. Then, the pseudoinverse of A is

$$A^+ = \frac{1}{\operatorname{trace}(A^T A)} A^T = \frac{1}{\|u\|_2^2 \|v\|_2^2} vu^T.$$

47.3.3 Properties of the pseudoinverse

Let us begin by stating some fundamental properties of the pseudoinverse. Verifying these properties is left as an exercise (see Problem 47.17). You can use the characterizations from Section 47.3.1 to confirm them.

Theorem 47.28. (Elementary properties of the pseudoinverse)

Let $A \in \mathbb{R}^{m\times n}$. The pseudoinverse A^+ satisfies the following properties:

(a) $(A^+)^+ = A$.
(b) $(A^T)^+ = (A^+)^T$.
(c) For $\alpha \neq 0$, $(\alpha A)^+ = (1/\alpha)A^+$.
(d) $(A^TA)^+ = A^+(A^T)^+$ and $(AA^T)^+ = (A^T)^+A^+$.
(e) For any orthogonal matrices $Q \in \mathbb{R}^{m\times m}$ and $Z \in \mathbb{R}^{n\times n}$, we have $(QAZ^T)^+ = ZA^+Q^T$.
(f) $A^+ = A^T(AA^T)^+ = (A^TA)^+A^T$.

Properties (a)–(c) mirror the familiar properties of the matrix inverse: $(A^{-1})^{-1} = A$, $(A^T)^{-1} = (A^{-1})^T$, and $(\alpha A)^{-1} = \frac{1}{\alpha}A^{-1}$ for $\alpha \neq 0$.

Property (d) is a special case of $(AB)^{-1} = B^{-1}A^{-1}$ when $B = A^T$. The property $(AB)^+ = B^+A^+$ holds for some specific types of matrices (see Problem 47.32) but not for all. The following two examples illustrate this.

Example 47.29 Consider the matrices

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}.$$

First, note that $AB = B$. Additionally, B is a rank-one matrix, so using Proposition 47.27, we have

$$(AB)^+ = B^+ = \begin{bmatrix} 0.5 & 0 \\ 0.5 & 0 \end{bmatrix}.$$

For the matrix A, its pseudoinverse is $A^+ = A$. Therefore

$$B^+A^+ = \begin{bmatrix} 0.5 & 0 \\ 0.5 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \\ 0.5 & 0 \end{bmatrix}.$$

Thus, the property $(AB)^+ = B^+A^+$ holds for these two specific matrices.

Unfortunately, the property $(AB)^+ = B^+A^+$ does not hold for all matrices A and B, as the following example shows.

Example 47.30 Consider the matrices

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}.$$

First, note that $AB = A$. Since A is a rank-one matrix, we can use Proposition 47.27 to find

$$(AB)^+ = A^+ = \begin{bmatrix} 0.5 & 0 \\ 0.5 & 0 \end{bmatrix}.$$

Similarly, B is also a rank-one matrix, and we have

$$B^+ = \begin{bmatrix} 0 & 0.5 \\ 0 & 0.5 \end{bmatrix}.$$

Now, compute

$$B^+A^+ = \begin{bmatrix} 0 & 0.5 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.5 & 0 \\ 0.5 & 0 \end{bmatrix} = \begin{bmatrix} 0.25 & 0 \\ 0.25 & 0 \end{bmatrix}.$$

Clearly, $(AB)^+ \neq B^+A^+$.

Property (e) shows that the pseudoinverse interacts with orthogonal transformations in the same way as the standard inverse.

Property (f) generalizes Proposition 47.24 to arbitrary matrices. This result proves particularly useful when computing the pseudoinverse of AA^T or A^TA is easier than computing A^+ directly.

The following result establishes the relationships between the four fundamental subspaces of A^+ and A, respectively. An intuitive justification of this result is presented in Figures 47.9 and 47.10. In the left panel of Figure 47.9, we present the linear transformation $x \to Ax$. Both the domain and the codomain are expressed as the direct sum of two fundamental subspaces of A. In the right panel, we represent the transformation $y \to A^+y$ based on the geometric definition of A^+ (Definition 47.18). That is, A^+ can be seen as the "inverse" of A and we know how it acts on $\text{col}(A)$ and $\text{null}(A^T)$. The domain and the codomain of the linear transformation defined by A^+ can also be expressed in terms of the four fundamental subspaces of A^+ as shown in Figure 47.10. The direct sum decomposition of $\mathbb{R}^m$ and $\mathbb{R}^n$ in the right panel of Figures 47.9 and 47.10 are actually the same as we prove next.

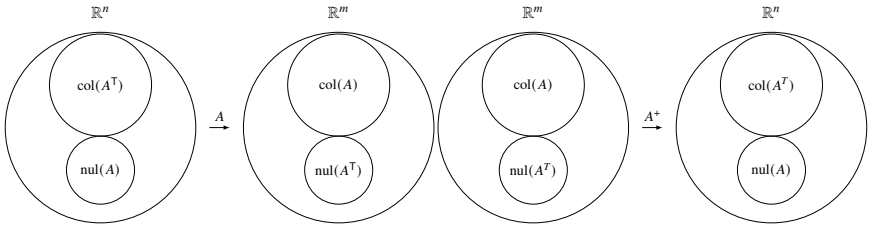


Fig. 47.9: A graphical representation of the transformation $x \to Ax$ and $y \to A^+y$ in terms of the four fundamental subspaces of A.

Proposition 47.31. (The four fundamental subspaces of A^+)

Let $A \in \mathbb{R}^{m \times n}$. Then,

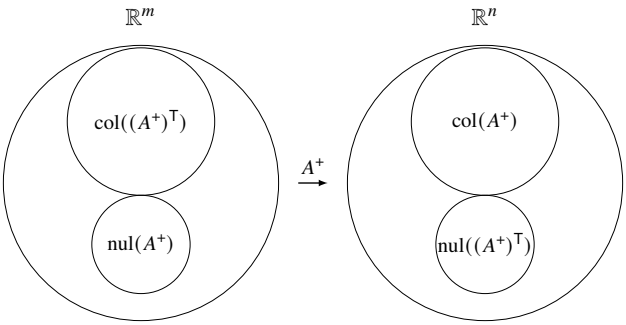


Fig. 47.10: A graphical representation of the transformation $y \to A^+y$ in terms of the four fundamental subspaces of A^+.

$$\begin{aligned} &\text{(i)} \quad \mathrm{col}(A^+) = \mathrm{col}(A^T), && \mathrm{null}(A^+) = \mathrm{null}(A^T), \\ &\text{(ii)} \quad \mathrm{col}((A^+)^T) = \mathrm{col}(A), && \mathrm{null}((A^+)^T) = \mathrm{null}(A). \end{aligned}$$

Proof Let us prove (i) first. By the (geometric) definition of the pseudoinverse, we have: for all $x \in \mathrm{null}(A^T)$, $A^+x = 0$, which implies $\mathrm{null}(A^T) \subseteq \mathrm{null}(A^+)$. Moreover, since A is invertible when restricted to $\mathrm{col}(A^T)$, we also have

$$\begin{aligned} \mathrm{col}(A^+) &= \{A^+x : x \in \mathbb{R}^m\} \\ &= \{A^+(x_r + x_n) : x_r \in \mathrm{col}(A),\ x_n \in \mathrm{null}(A^T)\} \\ &= \{A^+x_r : x_r \in \mathrm{col}(A)\} \subseteq \mathrm{col}(A^T). \end{aligned}$$

Next we show that the reverse inclusions also hold. Let $A = U_r\Sigma_r V_r^T$ be a reduced SVD of A. Then

$$A^+ = V_r\Sigma_r^+U_r^T = V_r\Sigma_r^{-1}U_r^T \quad \text{and} \quad A^T = V_r\Sigma_r U_r^T.$$

Assume $x \in \mathrm{null}(A^+)$. Then $V_r\Sigma_r^+U_r^Tx = 0$. Since V_r has full column rank and Σ_r is invertible, we conclude $U_r^Tx = 0$. Thus

$$A^Tx = V_r\Sigma_rU_r^Tx = 0.$$

This shows that $\mathrm{null}(A^+) \subseteq \mathrm{null}(A^T)$. Thus, $\mathrm{null}(A^+) = \mathrm{null}(A^T)$.

Now, assume $x \in \mathrm{col}(A^T)$. Then, there exists y such that $x = A^Ty = V_r(\Sigma_rU_r^Ty) = V_rz$, which implies $x \in \mathrm{col}(V_r)$. Since U_r has orthonormal columns, we can write

$$x = V_rz = V_r(\Sigma_r^{-1}U_r^TU_r\Sigma_r)z = A^+(U_r\Sigma_rz) \in \mathrm{col}(A^+).$$

Thus, $\mathrm{col}(A^+) = \mathrm{col}(A^T)$.

For (ii), apply (i) to the matrix A^T, noting that $(A^+)^T = (A^T)^+$. □

We can construct orthogonal projection matrices onto the four fundamental subspaces of a matrix using the pseudoinverse.

Theorem 47.32. (Orthogonal projections with the pseudoinverse)

Let $A \in \mathbb{R}^{m \times n}$. Then,

(a) A^+A is the orthogonal projection matrix onto $\text{col}(A^T)$, and $I_n - A^+A$ is the orthogonal projection matrix onto $\text{null}(A)$.
(b) AA^+ is the orthogonal projection matrix onto $\text{col}(A)$, and $I_m - AA^+$ is the orthogonal projection matrix onto $\text{null}(A^T)$.

Proof Let $\text{rank}(A) = r$, and consider the reduced SVD $A = U_r \Sigma_r V_r^T$ of A. Then, $A^+ = V_r \Sigma_r^{-1} U_r^T$, where U_r and V_r each have r orthonormal columns, and Σ_r is a diagonal matrix containing the nonzero singular values of A. We have

$$A^+A = V_r \Sigma_r^{-1} U_r^T U_r \Sigma_r V_r^T = V_r V_r^T.$$

Similarly,

$$AA^+ = U_r \Sigma_r V_r^T V_r \Sigma_r^{-1} U_r^T = U_r U_r^T.$$

From Theorem 47.5, $A^+A = V_r V_r^T$ is the orthogonal projection matrix onto $\text{col}(A^T)$, and $AA^+ = U_r U_r^T$ is the orthogonal projection matrix onto $\text{col}(A)$.

Since $\text{null}(A) = \text{col}(A^T)^\perp$ and $\text{null}(A^T) = \text{col}(A)^\perp$, the orthogonal projection matrices onto $\text{null}(A)$ and $\text{null}(A^T)$ are $I_n - A^+A$ and $I_m - AA^+$, respectively. □

47.3.4 Linear systems and least squares problems revisited: minimum norm solutions

The pseudoinverse provides a unified approach to solving both linear systems and least squares problems.

Consider a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$. The system $Ax = b$ has a solution if and only if $b \in \text{col}(A)$. If A has a nontrivial nullspace, then $Ax = b$ has infinitely many solutions, as often happens in underdetermined systems (where there are more unknowns than equations). Among these, the solution with the smallest Euclidean norm is particularly important. This is equivalent to solving the following optimization problem:

$$\begin{aligned} &\text{minimize} && \|x\|_2, \\ &\text{subject to} && Ax = b. \end{aligned}$$

This problem has a unique solution, which we can find using the pseudoinverse, as we show next.

Theorem 47.33. (The minimum norm solution of $Ax = b$)

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ such that $b \in \text{col}(A)$. The vector $x = A^+b$ is the unique solution to $Ax = b$ with the smallest 2-norm.

Proof Since $b \in \text{col}(A)$, there exists some vector x_0 such that $Ax_0 = b$. The vector $x = A^+b$ is also a solution to $Ax = b$ because

$$Ax = AA^+b = AA^+Ax_0 = Ax_0 = b,$$

where we used the property $AA^+A = A$ from Theorem 47.21. Thus, $x = A^+b$ satisfies the equation $Ax = b$.

Now, let us show that $x = A^+b$ has the smallest 2-norm among all solutions. Any other solution y must be of the form

$$y = x + n = A^+b + n, \quad n \in \text{null}(A),$$

which is a particular solution $x = A^+b$ plus a vector n in the null space of A. Since $A^+b \in \text{col}(A^T)$ (from Proposition 47.31) and $\text{null}(A) = \text{col}(A^T)^\perp$, the vectors x and n are orthogonal. Using the Pythagorean theorem, we find

$$\|y\|_2^2 = \|x + n\|_2^2 = \|x\|_2^2 + \|n\|_2^2 \geqslant \|x\|_2^2.$$

The equality $\|y\|_2 = \|x\|_2$ holds only if $n = 0$, which implies $y = x$.

Therefore, $x = A^+b$ is the unique solution with the smallest 2-norm. □

If b is not in $\text{col}(A)$, the equation $Ax = b$ has no solution. Instead, we solve the least squares problem $\min_x \|Ax - b\|_2$. In other words, we find the vector Ax in $\text{col}(A)$ that is closest to b. If A has a nontrivial nullspace, there are infinitely many least squares solutions, but the pseudoinverse finds the unique one with the smallest 2-norm.

Theorem 47.34. (Minimum norm least squares solution)

Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ such that $b \notin \text{col}(A)$. Then $x = A^+b$ is the unique solution to the least squares problem $\min_x \|Ax - b\|_2$ with the smallest 2-norm.

Proof By Theorem 32.11, the normal equations $A^TAx = A^Tb$ always have a solution, and the least squares solutions are precisely the solutions of these equations. Theorem 47.33 guarantees that the unique solution with the smallest norm to the normal equations is $x = (A^TA)^+A^Tb = A^+b$, where we use property (f) of Theorem 47.28. □

In Chapter 48, we will show how the minimum-norm least squares solution $x = A^+b$ relates to the regularized least squares solution introduced in Chapter 33.

Note the similarity between Theorems 47.33 and 47.34: in both cases, the minimum-norm solution is given by $x = A^+b$.

Problems

47.1 Find by hand the singular value decomposition of the following matrices

$$A = \begin{bmatrix} -2 & 2 \\ -1 & 1 \\ 2 & -2 \end{bmatrix}, \qquad B = \begin{bmatrix} -2 & 1 \\ 1 & 2 \\ 1 & 2 \end{bmatrix}, \qquad \text{and} \qquad C = \begin{bmatrix} 1 & -1 & 0 \\ 1 & -1 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}.$$

47.2 Find the singular value decomposition of the matrix

$$A = \begin{bmatrix} 4 & 0 & 0 & 0 & 3 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

47.3 What is the singular value decomposition of the rank-1 matrix $A = xy^T$, where $x \in \mathbb{R}^m$ and $y \in \mathbb{R}^n$ are nonzero vectors? What are the singular values of A?

47.4 What is the relationship between the eigenvalues of a symmetric matrix A and its singular values?

47.5 If $\sigma_1, \sigma_2, \ldots, \sigma_{\min\{m,n\}}$ are the singular values of $A \in \mathbb{R}^{m \times n}$, what are the singular values of cA, where $c \in \mathbb{R}$?

47.6 Prove Theorem 47.6.

47.7 Let $A \in \mathbb{R}^{m \times n}$ with $\operatorname{rank}(A) = r$. Show that the $(m+n) \times (m+n)$ symmetric matrix

$$M = \begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}$$

has nonzero eigenvalues $\{\pm\sigma_1, \pm\sigma_2, \ldots, \pm\sigma_r\}$, where $\sigma_1, \sigma_2, \ldots, \sigma_r$ are the nonzero singular values of A.

47.8 Let $A \in \mathbb{R}^{m \times n}$ have the factorization

$$A = UCV^T = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} C_1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix},$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices, $C_1 \in \mathbb{R}^{r \times r}$ is invertible, and $U_1 \in \mathbb{R}^{m \times r}$, $U_2 \in \mathbb{R}^{m \times (m-r)}$, $V_1 \in \mathbb{R}^{n \times r}$, $V_2 \in \mathbb{R}^{n \times (n-r)}$. Prove that

(a) $\operatorname{rank}(A) = r$;
(b) $\operatorname{col}(A) = \operatorname{col}(U_1)$ and $\operatorname{null}(A^T) = \operatorname{col}(U_2)$;
(c) $\operatorname{col}(A^T) = \operatorname{col}(V_1)$ and $\operatorname{null}(A) = \operatorname{col}(V_2)$.

47.9 Given the matrix and the vector

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 1 & -1 \\ 1 & 1 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix},$$

use the singular value decomposition of A to express b as $b = c + n$, where $c \in \operatorname{col}(A)$ and $n \in \operatorname{null}(A^T)$.

47.10 Using the singular value decomposition of a matrix A, explain how you would verify whether a vector b is in the column space of A.

47.11 Let $A \in \mathbb{R}^{m\times n}$ with $\operatorname{rank}(A) = r < \min\{m, n\}$. Use the SVD of A to show that for every $\epsilon > 0$ (no matter how small), there exists a full rank matrix B such that $\|A - B\|_2 \leqslant \epsilon$.

47.12 For each matrix in Problem 47.1, find at least two distinct SVDs.

47.13 Extend Theorem 47.12 to include singular and rectangular matrices. That is, show that an $m \times n$ matrix A maps the unit sphere in $\mathbb{R}^n$ to an ellipsoid, which may be degenerate.

47.14 Find the reduced SVD for each matrix in Problem 47.1.

47.15 Let $A \in \mathbb{R}^{m\times n}$, and let $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_p$ be its singular values, where $p = \min\{m, n\}$. Define the vector of singular values as

$$s_A := \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \vdots \\ \sigma_p \end{bmatrix}.$$

The **Schatten p-norm** of A is defined by $\|A\|_{p*} = \|s_A\|_p$, where $\|\cdot\|_p$ is the usual vector p-norm. Research and summarize the proof that the Schatten p-norm is indeed a matrix norm. In your summary, briefly explain how positive definiteness, homogeneity, and the triangle inequality are established.

47.16 (Proof of Theorem 47.21) Consider a matrix $A \in \mathbb{R}^{m\times n}$ with SVD $A = U\Sigma V^T = U \left[\begin{smallmatrix} \Sigma_1 & 0 \\ 0 & 0 \end{smallmatrix}\right] V^T$. Since U and V are orthogonal, and thus invertible, we can express the pseudoinverse of A as $A^+ = VMU^T$, where M is partitioned conformably with the partition of Σ:

$$A^+ = VMU^T = V \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} U^T$$

where M_{11}, M_{12}, M_{21}, and M_{22} are appropriately sized matrices. Show that

(a) $AA^+A = A$ implies $M_{11} = \Sigma_1^{-1}$,
(b) $(AA^+)^T = AA^+$ implies $M_{12} = 0$,
(c) $A^+AA^+ = A^+$ implies $M_{22} = 0$, and
(d) $(A^+A)^T = A^+A$ implies $M_{21} = 0$.

47.17 Prove Theorem 47.28.

47.18 Show that if A is symmetric, so is its pseudoinverse A^+.

47.19 Find the pseudoinverse of the following two block matrices

$$\begin{bmatrix} X & 0 \\ 0 & 0 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0 & X \\ 0 & 0 \end{bmatrix}$$

in terms of the pseudoinverse of the matrix X.

47.20 Let A be a square matrix such that $A^2 = 0$. Prove that $(A^+)^2 = 0$.

47.21 Verify the results in Propositions 47.22–47.27.

47.22 Find the pseudoinverse of the following matrices:

$$A = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, \quad \text{and} \quad E = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

47.23 Find the pseudoinverse of the matrix A in Problem 47.2.

47.24 If $\alpha \in \mathbb{R}$, show

$$\alpha^+ = \begin{cases} 1/\alpha & \text{if } \alpha \neq 0, \\ 0 & \text{if } \alpha = 0. \end{cases}$$

47.25 If $v \in \mathbb{R}^m$, show

$$v^+ = (v^T v)^+ v^T = \begin{cases} v^T/(v^T v) & \text{if } v \neq 0, \\ 0 & \text{if } v = 0. \end{cases}$$

47.26 Give an example to show that the pseudoinverse of a triangular matrix is not necessarily triangular.

47.27 The $k \times k$ Jordan block with eigenvalue 0 is given by

$$J_k(0) = \begin{bmatrix} 0 & 1 & & & \\ & 0 & 1 & & \\ & & \ddots & \ddots & \\ & & & 0 & 1 \\ & & & & 0 \end{bmatrix}.$$

Determine the pseudoinverse of $J_k(0)$.

47.28 (Pseudoinverse of rank-1 matrices) If $x, y \in \mathbb{R}^n$, show that $(xy^T)^+ = (x^T x)^+ (y^T y)^+ yx^T$.

47.29 (Pseudoinverse of normal matrices) If $A \in \mathbb{R}^{n \times n}$ is normal (i.e., $A^T A = AA^T$), show that you can compute A^+ by diagonalizing A, mapping all nonzero eigenvalues to their reciprocals, and mapping zero eigenvalues to zero.

47.30 (Pseudoinverse of projection matrices) If P is an orthogonal projection matrix (i.e., $P^2 = P$ and $P^T = P$), show that $P^+ = P$.

47.31 Let $A \in \mathbb{R}^{n \times n}$ be a normal matrix (i.e., $AA^T = A^T A$). Show that, for every positive integer k,

$$A^k A^+ = A^+ A^k \quad \text{and} \quad (A^k)^+ = (A^+)^k.$$

47.32 Show that $(AB)^+ = B^+ A^+$ in each of the following situations:

(a) A has orthonormal columns;

(b) B has orthonormal rows;
(c) A has full column rank and B has full row rank.

47.33 Show that $\text{rank}(A) = \text{rank}(A^+)$ for all $A \in \mathbb{R}^{m\times n}$.

47.34 Let A be an $n \times n$ matrix of rank $n - 1$.

(a) Show that $AA^+ = I_n - vv^T$, where v is an orthonormal basis for $\text{null}(A^T)$.
(b) Similarly, show that $A^+A = I_n - ww^T$, where w is an orthonormal basis for $\text{null}(A)$.
(c) Generalize these results for matrices of rank r.

47.35 Let B be an $n \times n$ matrix of rank r. Suppose the columns of C form a basis for $\text{null}(B^T)$, and the columns of D^T for a basis for $\text{null}(B)$. Prove that the matrix

$$A = \begin{bmatrix} B & C \\ D & 0 \end{bmatrix}$$

is nonsingular and that its inverse is given by

$$A^{-1} = \begin{bmatrix} B^+ & D^+ \\ C^+ & 0 \end{bmatrix}.$$

47.36 Let $A \in \mathbb{R}^{m\times n}$ and $b \in \mathbb{R}^m$. Show that the system $Ax = b$ has a unique solution if and only if $A^+A = I_n$ and $b \in \text{col}(A)$. Show also that, when these conditions hold, the unique solution is $x = A^+b$.

Chapter 48
X-SUPPLEMENTARY TOPICS

48.1 Characterization of all the reduced SVDs of a matrix

In Chapter 47, we introduced the reduced SVD. Starting from a full SVD of a matrix A,

$$A = U\Sigma V^T = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix},$$

we form a reduced SVD by dropping the zero rows and columns in Σ and the corresponding columns in U and V. This gives

$$A = U_1 \Sigma_r V_1^T,$$

where Σ_r is an invertible diagonal matrix containing the $r = \text{rank}(A)$ nonzero singular values of A.

A reduced SVD is more compact because it avoids computing the orthonormal bases for $\text{null}(A)$ and $\text{null}(A^T)$, eliminating the need for U_2 and V_2. As a result, the set of reduced SVDs is smaller than the set of full SVDs.

However, the reduced SVD still allows some flexibility in choosing the bases for $\text{col}(A)$ and $\text{col}(A^T)$ (i.e., U_1 and V_1). In this section, we describe all the possible reduced SVDs of a given matrix.

We begin by describing how different reduced SVDs of the same matrix relate to each other.

Theorem 48.1. (Relation between reduced SVDs of a matrix)

Let A be an $m \times n$ matrix with rank $r > 0$, and let $A = U_1 \Sigma_r V_1^T$ be a reduced SVD of A. Then $A = U_2 \Sigma_r V_2^T$ is also a reduced SVD of A if and only if there exists an orthogonal $r \times r$ matrix Q such that

$$U_2 = U_1 Q, \quad V_2 = V_1 Q, \quad \text{and} \quad Q\Sigma_r = \Sigma_r Q. \tag{48.1}$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_48

Proof Assume that $A = U_1\Sigma_r V_1^T = U_2\Sigma_r V_2^T$ are two reduced SVDs of A. First, we show that

$$\text{col}(U_1) = \text{col}(A) = \text{col}(U_2).$$

We prove the first equality; the proof of the second is identical. Using Proposition 17.9 and the fact that $A = U_1(\Sigma_r V_1^T)$, we have

$$\text{col}(A) \subseteq \text{col}(U_1).$$

To prove the reverse inclusion, note that $\Sigma_r V_1^T$ has full row rank with a right inverse $V_1\Sigma_r^{-1}$. Thus

$$U_1 = A(V_1\Sigma_r^{-1}).$$

Applying Proposition 17.9 again, we obtain

$$\text{col}(U_1) \subseteq \text{col}(A).$$

Since $\text{col}(U_1) = \text{col}(A) = \text{col}(U_2)$, the projection matrices onto these subspaces, $U_1U_1^T$ and $U_2U_2^T$ are equal:

$$U_1U_1^T = U_2U_2^T.$$

Define $Q := U_1^T U_2$. Then

$$U_2 = U_2 I_r = U_2U_2^T U_2 = U_1U_1^T U_2 = U_1Q.$$

Next, we show that Q is orthogonal:

$$Q^TQ = Q^TU_1^TU_1Q = (U_1Q)^T(U_1Q) = U_2^TU_2 = I_r.$$

Thus, Q is an orthogonal matrix. Using a similar argument, we can show that $V_2 = V_1\widehat{Q}$ for some orthogonal matrix $\widehat{Q}$.

Finally, we show that $Q = \widehat{Q}$ and that $Q\Sigma_r = \Sigma_r Q$. Notice that

$$U_1\Sigma_r V_1^T = U_2\Sigma_r V_2^T = U_1Q\Sigma_r\widehat{Q}^T V_1^T.$$

Since U_1 has full column rank and V_1^T has full row rank, they are left and right invertible, respectively. Multiplying the preceding equality on the left by U_1^T and on the right by V_1, and using $U_1^TU_1 = I_r$ and $V_1^TV_1 = I_r$, we obtain

$$\Sigma_r = Q\Sigma_r\widehat{Q}^T.$$

The proof that this equality implies $Q = \widehat{Q}$ is left as an exercise (see Problem 48.1). Therefore,

$$\Sigma_r = Q\Sigma_r Q^T.$$

Multiplying on the right by Q, we obtain $\Sigma_r Q = Q\Sigma_r$. Thus, Q commutes with Σ_r. Moreover, $U_2 = U_1Q$ and $V_2 = V_1Q$.

Assume that (48.1) holds for some orthogonal matrix Q. Because Q is orthogonal, both U_2 and V_2 have orthonormal columns. Also,

$$U_2\Sigma_r V_2^T = U_1 Q\Sigma_r (V_1 Q)^T = U_1 Q\Sigma_r Q^T V_1^T = U_1\Sigma_r QQ^T V_1^T = U_1\Sigma_r V_1^T = A.$$

So $U_2\Sigma_r V_2^T$ is a reduced SVD of A. □

Theorem 48.1 describes all reduced SVDs of a matrix A, though the description is not fully explicit. The main point is this: if $A = U\Sigma_r V^T$ is a reduced SVD, then any other reduced SVD can be written as

$$A = (UQ)\Sigma_r (VQ)^T,$$

where Q is an orthogonal matrix that commutes with the diagonal matrix Σ_r. This leads to a natural question: which orthogonal matrices commute with a diagonal matrix?

Theorem 48.2. (Orthogonal matrices that commute with Σ_r)

Let A be a matrix of rank r with d distinct positive singular values $\sigma_1 > \sigma_2 > \cdots > \sigma_d > 0$, and let $n_1, n_2, \ldots, n_d$ be their multiplicities (so $n_1 + n_2 + \cdots + n_d = r$). Let Σ_r be the diagonal matrix of the nonzero singular values of A. An orthogonal matrix $Q \in \mathbb{R}^{r\times r}$ commutes with Σ_r (i.e., $Q\Sigma_r = \Sigma_r Q$) if and only if Q is block diagonal:

$$Q = \begin{bmatrix} Q_{11} & 0 & \cdots & 0 \\ 0 & Q_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & Q_{dd} \end{bmatrix},$$

where each Q_{ii} is an $n_i \times n_i$ orthogonal matrix.

Proof Since Σ_r is a diagonal matrix with d distinct diagonal entries occurring with multiplicities $n_1, n_2, \ldots, n_d$, it takes the block form

$$\Sigma_r = \begin{bmatrix} \sigma_1 I_{n_1} & 0 & \cdots & 0 \\ 0 & \sigma_2 I_{n_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_d I_{n_d} \end{bmatrix}.$$

Assume first that Q is block diagonal, $Q = \operatorname{diag}(Q_{11}, \ldots, Q_{dd})$, where each $Q_{ii} \in \mathbb{R}^{n_i\times n_i}$ is orthogonal. Then, Q is orthogonal and

$$Q\Sigma_r = \operatorname{diag}(\sigma_1 Q_{11}, \ldots, \sigma_d Q_{dd}) = \Sigma_r Q.$$

Assume next $Q \in \mathbb{R}^{r\times r}$ is an orthogonal matrix satisfying $Q\Sigma_r = \Sigma_r Q$. Partition Q conformably with the block structure of Σ_r:

$$Q = \begin{bmatrix} Q_{11} & Q_{12} & \cdots & Q_{1d} \\ Q_{21} & Q_{22} & \cdots & Q_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ Q_{d1} & Q_{d2} & \cdots & Q_{dd} \end{bmatrix},$$

where each Q_{ij} is an $n_i \times n_j$ matrix. The commutative relation $Q\Sigma_r = \Sigma_r Q$ implies that, for all i, j,

$$Q_{ij}(\sigma_j I_{n_j}) = (\sigma_i I_{n_i})Q_{ij},$$

or equivalently, $\sigma_j Q_{ij} = \sigma_i Q_{ij}$. For $i \neq j$, since $\sigma_i \neq \sigma_j$, this equation forces $Q_{ij} = 0$. Therefore, Q is block diagonal:

$$Q = \begin{bmatrix} Q_{11} & 0 & \cdots & 0 \\ 0 & Q_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & Q_{dd} \end{bmatrix}.$$

Since Q is orthogonal, each diagonal block $Q_{ii} \in \mathbb{R}^{n_i \times n_i}$ must also be an orthogonal matrix. □

The block-diagonal structure of Q shows that, even though the reduced SVD is not unique, the subspaces spanned by the left and right singular vectors for each singular value are uniquely determined by the matrix A. In other words, while the individual singular vectors may differ among various reduced SVDs, the subspaces they span remain the same. These subspaces are precisely the eigenspaces of $A^T A$ and AA^T, respectively.

Example 48.3 Consider the following reduced SVD of the matrix A:

$$A = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \\ 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}.$$

Since A has a single distinct nonzero singular value (3) with multiplicity 2, any other reduced SVD of A has the form

$$A = \left(\begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \\ 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix} Q \right) \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} \left(Q^T \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix} \right),$$

where Q is any 2×2 orthogonal matrix.

48.2 The Polar decomposition and two distance problems

The eigenvalue behavior of square complex matrices reveals analogies to complex numbers. For example, symmetric positive semidefinite and positive definite matrices, characterized by nonnegative and positive eigenvalues (Theorem 42.23), act as matrix counterparts to nonnegative and positive real numbers. Similarly, orthogonal matrices, whose eigenvalues are constrained to the unit circle in the complex plane ($e^{i\theta}$, Proposition 37.3), are analogous to complex numbers of unit modulus.

This connection goes even further. Recall that any complex number $z = a + bi$ can be expressed in polar form as

$$z = r(\cos(\theta) + i\sin(\theta)) = re^{i\theta},$$

where $r \geqslant 0$ is the modulus of z and θ is the angle z makes with the real axis. If we think of r and $e^{i\theta}$ as 1×1 matrices, then r is a positive semidefinite matrix, while $e^{i\theta}$ is an orthogonal matrix. The polar decomposition extends this observation to $n \times n$ matrices, expressing any square matrix as the product of a positive semidefinite matrix and an orthogonal matrix.

Definition 48.4. (Polar decomposition)

For any matrix $A \in \mathbb{R}^{n\times n}$, the **polar decomposition** expresses it as

$$A = QS,$$

where Q is an orthogonal matrix and S is a symmetric positive semidefinite matrix.

The following theorem shows that every matrix has a polar decomposition, although the decomposition may not be unique.

Theorem 48.5. (Existence of polar decompositions)

For any $A \in \mathbb{R}^{n\times n}$, there exist a positive semidefinite matrix $S \in \mathbb{R}^{n\times n}$ and an orthogonal matrix $Q \in \mathbb{R}^{n\times n}$ such that $A = QS$. The positive semidefinite factor S is uniquely determined by A as the unique positive semidefinite square root of A^TA, i.e., $S = (A^TA)^{1/2}$. The orthogonal factor Q is unique if and only if A is nonsingular.

Proof Let $A = U\Sigma V^T$ be a singular value decomposition of A. We can rewrite A as

$$A = U(V^TV)\Sigma V^T = \underbrace{(UV^T)}_{=Q}\,\underbrace{(V\Sigma V^T)}_{=S} =: QS.$$

The matrix $Q = UV^T$ is orthogonal because it is the product of two orthogonal matrices. The matrix $S = V\Sigma V^T$ is symmetric, and its eigenvalues are the singular values of A, which are nonnegative. Therefore, S is positive semidefinite. This proves the existence of the polar decomposition.

Since S is symmetric and positive semidefinite, we have

$$A^T A = S^T Q^T Q S = S^T S = S^2,$$

which shows that S is the unique positive semidefinite square root of $A^T A$ (see Theorem 42.47). Thus, $S = (A^T A)^{1/2}$, and it is uniquely determined by A.

If A is nonsingular, S is also nonsingular because $S = Q^T A$, and Q is invertible. In this case, Q can be expressed as

$$Q = AS^{-1} = A(A^T A)^{-1/2}.$$

Hence, Q is uniquely determined by A.

If A is singular, then S is also singular. In this case, there is flexibility in how to choose Q, corresponding to the nontrivial left nullspace of S (see Problem 48.3). □

The proof of Theorem 48.5 shows how to construct a polar decomposition from an SVD. Conversely, you can also derive an SVD from a polar decomposition (see Problem 48.4).

Alternatively, you can factor $A \in \mathbb{R}^{n\times n}$ as $A = KQ$ via a reverse polar decomposition, where $Q \in \mathbb{R}^{n\times n}$ is orthogonal and $K \in \mathbb{R}^{n\times n}$ is symmetric positive semidefinite (see Problem 48.5).

Moreover, the idea of polar decomposition extends to non-square $m \times n$ matrices (with $m \geqslant n$) (see Problem 48.6).

The polar decomposition is especially useful because of its approximation properties. In particular, it helps solve two distance problems: finding the closest orthogonal matrix to a given matrix, and finding the closest positive semidefinite matrix.

Let us begin with finding the closest orthogonal matrix.

Theorem 48.6. (Closest orthogonal matrix)

Let $A \in \mathbb{R}^{n\times n}$ with polar decomposition $A = QS$. The orthogonal factor Q is a solution to the minimization problem

$$\min_{U^T U = I_n} \|A - U\|_F. \tag{48.2}$$

In other words, Q is a closest orthogonal matrix to A in the Frobenius norm.

Proof Let $S = VDV^T$ be a spectral decomposition of the positive semidefinite matrix S, where $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ with $\lambda_i \geqslant 0$. Let U be an orthogonal matrix. Then

$$\begin{aligned}
\|A - U\|_F^2 &= \|QS - U\|_F^2 \\
&= \|QVDV^T - U\|_F^2 \\
&= \|QV(D - V^T Q^T U V)V^T\|_F^2 \\
&=: \|D - Z\|_F^2,
\end{aligned}$$

where $Z := V^T Q^T UV$ is orthogonal because it is the product of orthogonal matrices. Notice that, in the last equality above we used the orthogonal invariance of the Frobenius norm.

Expanding $\|D - Z\|_F^2$, we get

$$\begin{aligned}\|D - Z\|_F^2 =& \text{trace}\left((D - Z)^T (D - Z)\right)\\ =& \text{trace}(D^T D) - \text{trace}(D^T Z) - \text{trace}(Z^T D) + \text{trace}(Z^T Z)\\ =& \text{trace}(D^T D) - 2\text{trace}(DZ) + n,\end{aligned}$$

where the last equality follows from the following facts: 1) $\text{trace}(D^T Z) = \text{trace}(Z^T D)$; 2) $D^T = D$ since D is diagonal; 3) Z is orthogonal and therefore, the main diagonal entries of $Z^T Z$ are all ones. The scalars $\text{trace}(D^T D)$ and n are constants that do not depend on Z. Therefore, minimizing $\|A - U\|_F^2$ is equivalent to maximizing $\text{trace}(DZ)$.

Writing $\text{trace}(DZ)$ explicitly, we obtain

$$\text{trace}(DZ) = \lambda_1 z_{11} + \lambda_2 z_{22} + \cdots + \lambda_n z_{nn}.$$

Since Z is orthogonal, for any column z_i of Z, we have

$$z_i^T z_i = 1.$$

Thus,

$$z_{ii}^2 \leqslant \sum z_{ij}^2 = z_i^T z_i = 1.$$

As $\lambda_i \geqslant 0$ and $|z_{ii}| \leqslant 1$, $\text{trace}(DZ)$ is maximized by setting $z_{ii} = 1$ for $i = 1, \ldots, n$, which forces $z_{ij} = 0$ for $i \neq j$. This makes $Z = I_n$. If $Z = I_n$, then $V^T Q^T UV = I_n$, which implies $U = Q$. Thus, Q minimizes $\|A - U\|_F^2$, as required. □

In Problem 48.10, we consider an application of Theorem 48.6 known as the orthogonal Procrustes problem. The goal is to find an orthogonal matrix Q that minimizes $\|X - QY\|_F$ for given matrices X and Y.

Now, let us find a positive semidefinite matrix that is closest to a given symmetric matrix.

Theorem 48.7. (Closest positive semidefinite matrix)

Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix with polar decomposition $A = QS$. The matrix $(A + S)/2$ is the unique positive semidefinite solution to the minimization problem

$$\min_{X \text{ is positive semidefinite}} \|A - X\|_F. \tag{48.3}$$

In other words, the matrix $(A + S)/2$ is the closest positive semidefinite matrix to A in the Frobenius norm.

Proof Let $A = VDV^T$ be a spectral decomposition of A where V is orthogonal and $D = \text{diag}(\lambda_1, \ldots, \lambda_n)$ contains the eigenvalues of A. This decomposition exists by

the spectral theorem (Theorem 41.3). Moreover, since A is symmetric, its eigenvalues are real.

Consider any positive semidefinite matrix X, and define $Y := V^T XV$, which is also positive semidefinite. Using the orthogonal invariance of the Frobenius norm, we have

$$\|A - X\|_F^2 = \|VDV^T - VYV^T\|_F^2 = \|D - Y\|_F^2.$$

Expanding this expression, we get

$$\|D - Y\|_F^2 = \sum_{i \neq j} y_{ij}^2 + \sum_{i=1}^{n} (\lambda_i - y_{ii})^2 \geqslant \sum_{i=1}^{n} (\lambda_i - y_{ii})^2.$$

For $\lambda_i \geqslant 0$, choosing $y_{ii} = \lambda_i$ minimizes the term $(\lambda_i - y_{ii})^2$. For $\lambda_i < 0$, since $y_{ii} \geqslant 0$ because Y is positive semidefinite, setting $y_{ii} = 0$ minimizes $(\lambda_i - y_{ii})^2 = \lambda_i^2 - 2y_{ii}\lambda_i + y_{ii}^2$. Thus

$$\|D - Y\|_F^2 \geqslant \sum_{\lambda_i < 0} \lambda_i^2.$$

This inequality holds for all positive semidefinite matrices X, so, in particular,

$$\|A - X\|_F^2 \geqslant \sum_{\lambda_i < 0} \lambda_i^2.$$

Next, we show that exactly one positive semidefinite matrix X achieves equality since, for equality to happen, the following conditions must hold: $y_{ij} = 0$ for $i \neq j$, which implies Y is diagonal; $y_{ii} = \lambda_i$ for $\lambda_i \geqslant 0$, minimizing $(\lambda_i - y_{ii})^2$; and $y_{ii} = 0$ for $\lambda_i < 0$, also minimizing $(\lambda_i - y_{ii})^2$. Thus, the unique positive semidefinite matrix X that minimizes $\|A - X\|_F^2$ is given by

$$X = V \operatorname{diag}(d_1, \ldots, d_n)\, V^T, \quad \text{where } d_i = \begin{cases} \lambda_i & \text{if } \lambda_i \geqslant 0, \\ 0 & \text{if } \lambda_i < 0. \end{cases}$$

Finally, the formula $X = (A+S)/2$ follows directly from $S = V \operatorname{diag}(|\lambda_1|, \ldots, |\lambda_n|)\, V^T$ (see Problem 48.7). □

The proof of Theorem 48.7 gives a simple way to find the closest positive semidefinite matrix to a symmetric matrix A: set all negative eigenvalues to zero in any eigen-decomposition of A.

Finding the closest positive semidefinite matrix is important in many areas, especially in Statistics. For example, covariance matrices may sometimes fail to be positive semidefinite due to numerical errors or incomplete data. Finding the nearest positive semidefinite matrix helps resolve these issues.

48.3 More characterizations of the pseudoinverse

In Chapter 47, we defined the pseudoinverse in three equivalent ways. Here, we present two additional characterizations.

One intuitive interpretation views the pseudoinverse A^+ as the matrix that makes both AA^+ and A^+A as close as possible to the identity matrix. This perspective

naturally extends the notion of an inverse to matrices that are singular or rectangular. The following theorem formalizes this idea.

Theorem 48.8. (Optimization characterization of the pseudoinverse)

Let $A \in \mathbb{R}^{m\times n}$. The pseudoinverse $A^+ \in \mathbb{R}^{n\times m}$ is the unique matrix that maps $\text{null}(A^T)$ to $\{0\}$ and satisfies

$$A^+ = \underset{X\in\mathbb{R}^{n\times m}}{\text{argmin}} \|I_m - AX\|_F \quad \text{and} \quad A^+ = \underset{X\in\mathbb{R}^{n\times m}}{\text{argmin}} \|I_n - XA\|_F.$$

Proof Let

$$A = U\Sigma V^T = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} V_1^T \\ V_2^T \end{bmatrix}$$

be an SVD of A, and write $X \in \mathbb{R}^{n\times m}$ as

$$X = VMU^T = V \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} U^T.$$

Using the invariance of the Frobenius norm under orthogonal transformations, we have

$$\begin{aligned} \|I_m - AX\|_F^2 &= \|I_m - U\Sigma V^T X\|_F^2 \\ &= \|I_m - \Sigma M\|_F^2 \\ &= \left\| \begin{bmatrix} I_r & 0 \\ 0 & I_{m-r} \end{bmatrix} - \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \right\|_F^2 \\ &= \left\| \begin{bmatrix} I_r - \Sigma_r M_{11} & \Sigma_r M_{12} \\ 0 & I_{m-r} \end{bmatrix} \right\|_F^2 \\ &= \|I_r - \Sigma_r M_{11}\|_F^2 + \|\Sigma_r M_{12}\|_F^2 + \|I_{m-r}\|_F^2. \end{aligned}$$

Thus, $\|I_m - AX\|_F$ is minimized when $M_{11} = \Sigma_r^{-1}$ and $M_{12} = 0$. Similarly, minimizing $\|I_n - XA\|_F$ gives the conditions $M_{11} = \Sigma_r^{-1}$ and $M_{21} = 0$. Together, these results imply that M must have the form

$$M = \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & M_{22} \end{bmatrix}.$$

Finally, the condition that X maps $\text{null}(A^T)$ to $\{0\}$ implies $M_{22} = 0$. To see this, let $v \in \text{null}(A^T)$. By Theorem 47.4, we know $v = U_2 x$ for some $x \in \mathbb{R}^{m-r}$. Then

$$\begin{aligned} 0 = Xv &= \begin{bmatrix} V_1 & V_2 \end{bmatrix} \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & M_{22} \end{bmatrix} \begin{bmatrix} U_1^T \\ U_2^T \end{bmatrix} U_2 x \\ &= \begin{bmatrix} V_1 & V_2 \end{bmatrix} \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & M_{22} \end{bmatrix} \begin{bmatrix} 0 \\ x \end{bmatrix} = V_2 M_{22} x. \end{aligned}$$

Since V_2 has linearly independent columns, we conclude $M_{22}x = 0$ for all $x \in \mathbb{R}^{m-r}$, which forces $M_{22} = 0$. □

Another way to characterize the pseudoinverse is by using limits. This approach connects the pseudoinverse to the regularized least squares problems discussed in Chapter 33.

Theorem 48.9. (Limit characterizations of the pseudoinverse)

Let $A \in \mathbb{R}^{m\times n}$. Then,

$$A^+ = \lim_{\delta\to 0}(A^TA + \delta^2 I_n)^{-1}A^T = \lim_{\delta\to 0} A^T(AA^T + \delta^2 I_m)^{-1} \tag{48.4}$$

These limits exist even when A^TA or AA^T are not invertible.

Proof We prove the first limit and leave the second as an exercise.

Let $A = U\begin{bmatrix}\Sigma_r & 0\\ 0 & 0\end{bmatrix}V^T$ be an SVD of A, where $\Sigma_r = \operatorname{diag}(\sigma_1,\dots,\sigma_r)$ contains the nonzero singular values of A. For any $\delta \neq 0$, we can write the symmetric matrix $A^TA + \delta^2 I_n$ as

$$\begin{aligned}
A^TA + \delta^2 I_n &= V\begin{bmatrix}\Sigma_r & 0\\ 0 & 0\end{bmatrix}U^TU\begin{bmatrix}\Sigma_r & 0\\ 0 & 0\end{bmatrix}V^T + \delta^2 I_n\\
&= V\begin{bmatrix}\Sigma_r^2 & 0\\ 0 & 0\end{bmatrix}V^T + \delta^2 VV^T\\
&= V\begin{bmatrix}\Sigma_r^2 + \delta^2 I_r & 0\\ 0 & \delta^2 I_{n-r}\end{bmatrix}V^T\\
&= V\operatorname{diag}\left(\sigma_1^2+\delta^2,\dots,\sigma_r^2+\delta^2,\delta^2,\dots,\delta^2\right)V^T.
\end{aligned}$$

Since $\delta^2 > 0$, $A^TA + \delta^2 I_n$ is invertible, and its inverse is

$$\begin{aligned}
(A^TA + \delta^2 I_n)^{-1} &= V\begin{bmatrix}(\Sigma_r^2 + \delta^2 I_r)^{-1} & 0\\ 0 & (\delta^2 I_{n-r})^{-1}\end{bmatrix}V^T\\
&= V\operatorname{diag}\left(\frac{1}{\sigma_1^2+\delta^2},\dots,\frac{1}{\sigma_r^2+\delta^2},\frac{1}{\delta^2},\dots,\frac{1}{\delta^2}\right)V^T.
\end{aligned}$$

Now, multiplying $(A^TA + \delta^2 I_n)^{-1}$ by A^T gives

$$\begin{aligned}
(A^TA + \delta^2 I_n)^{-1}A^T &= V\begin{bmatrix}(\Sigma_r^2 + \delta^2 I_r)^{-1} & 0\\ 0 & (1/\delta^2)I_{n-r}\end{bmatrix}V^TV\begin{bmatrix}\Sigma_r & 0\\ 0 & 0\end{bmatrix}U^T\\
&= V\begin{bmatrix}(\Sigma_r^2 + \delta^2 I_r)^{-1}\Sigma_r & 0\\ 0 & 0\end{bmatrix}U^T\\
&= V\operatorname{diag}\left(\frac{\sigma_1}{\sigma_1^2+\delta^2},\dots,\frac{\sigma_r}{\sigma_r^2+\delta^2},0,\dots,0\right)U^T.
\end{aligned}$$

Taking the limit as $\delta \to 0$, we obtain

$$\lim_{\delta \to 0} (A^T A + \delta^2 I_n)^{-1} A^T = V \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & 0 \end{bmatrix} U^T = A^+.$$

This completes the proof. □

Let us see how the limit characterization of the pseudoinverse connects with regularized least squares. In regularized least squares, we minimize the residual $\|Ax - b\|_2^2$ while also penalizing large values of $\|x\|_2$. This leads to the optimization problem

$$\min_x \left(\|Ax - b\|_2^2 + \lambda \|x\|_2^2 \right),$$

where $\lambda > 0$ controls the trade-off between fitting Ax to b and keeping $\|x\|_2$ small. This problem has a unique solution given by

$$x_\lambda = (A^T A + \lambda I_n)^{-1} A^T b.$$

Now, if we write $\lambda = \delta^2$ (since $\lambda > 0$) and let $\delta \to 0$ (or equivalently $\lambda \to 0^+$), Theorem 48.9 shows that

$$\lim_{\lambda \to 0^+} x_\lambda = \lim_{\delta \to 0} x_{\delta^2} = \lim_{\delta \to 0} (A^T A + \delta^2 I_n)^{-1} A^T b = A^+ b.$$

Thus, as the regularization parameter δ approaches zero, the regularized least squares solution x_λ converges to the minimum-norm least squares solution $A^+ b$ described in Theorem 47.34.

48.4 Algorithms for computing the SVD

The singular value decomposition (SVD) is a fundamental tool in Data Science and Machine Learning. It is not just a theoretical tool–it is incredibly practical. For example, one of the most reliable ways to determine the rank of a matrix is by counting how many of its singular values are above a chosen tolerance. The SVD also gives us the most accurate way to find orthonormal bases for the column space and nullspace of a matrix. While the QR factorization is faster for these tasks, it can sometimes be less accurate. Another important property of the SVD is that the largest singular value of a matrix A corresponds to its 2-norm, $\|A\|_2$.

A particularly useful application of the SVD is computing low-rank approximations of a matrix (Section 49.1). Additionally, the SVD is the mathematical foundation of Principal Component Analysis (PCA), a cornerstone technique in statistics and data analysis, which we will cover in Chapter 49.

Equally important is the conceptual clarity the SVD provides. Many problems in Linear Algebra become more intuitive when reframed with a simple question: *What happens if we apply the SVD?*

But how do we compute the SVD in practice? The most effective numerical methods for computing the SVD are closely related to algorithms used for finding eigenvalues of symmetric matrices. We will show this connection next.

Recall that, in theory, we can compute the SVD of a matrix A by finding the eigenvalues and eigenvectors of the symmetric matrix A^TA. Using this idea, a straightforward (but not ideal) approach to compute an SVD $A = U\Sigma V^T$ would involve the following steps: First, form the matrix $B = A^TA$. Next, compute the eigenvalues and an eigenvector matrix V of B. Then, construct the diagonal matrix Σ, with the same size as A, whose nonzero diagonal entries are the square roots of the eigenvalues of B. Finally, determine the left singular vectors U using the method described in Section 47.1.1.

While this method works in theory, it is not suitable for practical numerical computations. Forming A^TA and computing its eigenvalues can introduce inaccuracies, particularly when dealing with smaller singular values, as the following example demonstrates.

Example 48.10 The $(m+1) \times (n+1)$ Vandermonde matrix generated by the vector $x = \begin{bmatrix} x_0 & x_1 & \cdots & x_m \end{bmatrix}$ is defined as

$$V(x) = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \cdots & x_m^n \end{bmatrix}.$$

Consider a 15×7 Vandermonde matrix A generated by the vector $x = \begin{bmatrix} 1 & 1.5 & 2 & 2.5 & \cdots & 8 \end{bmatrix}$. We compute the singular values of A using the naive algorithm described earlier and compare them with the exact singular values. Table 48.1 shows the results, including the relative error for each singular value. The notation "number e$\pm$a" represents "number $\times$ $10^{\pm a}$".

Naive Algorithm	Exact Values	Relative Error
4.8391e+12	4.8391e+12	2.0180e-16
1.9761e+10	1.9761e+10	3.2817e-14
1.8568e+08	1.8568e+08	2.2595e-10
3.0515e+06	3.0514e+06	1.3121e-05
1.0336e+05	7.8771e+04	3.1213e-01
5.2243e+04	2.7761e+03	1.7819e+01
2.3856e+03	9.0291e+01	2.5421e+01

Table 48.1: Computed singular values using the naive approach, exact singular values, and relative errors.

The **relative error** measures how far the computed value is from the exact value, relative to the exact value itself. It is calculated as:

$$\text{Relative Error} = \frac{|\text{Computed Value} - \text{Exact Value}|}{|\text{Exact Value}|}.$$

From Table 48.1, we can see that the largest singular values are computed accurately, with very small relative errors. However, the smallest three singular values

have large relative errors, indicating that the naive algorithm fails to compute them correctly. In fact, the computed values for these singular values do not have any correct digits.

There is a more reliable way to compute the SVD that avoids forming A^TA, which, as we saw earlier, can lead to numerical inaccuracies. Instead, this approach transforms the problem into a larger symmetric eigenvalue problem. Most standard algorithms for computing the SVD are based on this idea.

Let $A \in \mathbb{R}^{m\times n}$ with $m \geqslant n$. (If $m < n$, we can simply work with A^T instead.) The key idea is to construct an augmented matrix

$$\tilde{A} := \begin{bmatrix} 0 & A^T \\ A & 0 \end{bmatrix} \in \mathbb{R}^{(m+n)\times(m+n)}. \tag{48.5}$$

The matrix (48.5) is symmetric, and its eigenvalues and eigenvectors are directly related to the singular values and singular vectors of A. To understand this connection, let $A = U\Sigma V^T$ be an SVD of A, and partition U as $U = \begin{bmatrix} U_1 & U_2 \end{bmatrix}$, where $U_1 \in \mathbb{R}^{m\times n}$. Then, A can be written as $A = U_1\Sigma_n V^T$, where Σ_n is the $n \times n$ diagonal matrix containing the singular values of A (both nonzero and zero). The augmented matrix (48.5) has the eigenvalue decomposition

$$\tilde{A} = \left(\frac{1}{\sqrt{2}}\begin{bmatrix} V & V & 0 \\ U_1 & -U_1 & \sqrt{2}U_2 \end{bmatrix}\right)\begin{bmatrix} \Sigma_n & 0 & 0 \\ 0 & -\Sigma_n & 0 \\ 0 & 0 & 0 \end{bmatrix}\left(\frac{1}{\sqrt{2}}\begin{bmatrix} V & V & 0 \\ U_1 & -U_1 & \sqrt{2}U_2 \end{bmatrix}\right)^T.$$

From this decomposition, we can observe two key properties. The positive eigenvalues of $\tilde{A}$ are exactly the singular values of A. The singular vectors of A (i.e., the columns of U and V) can be directly extracted from the eigenvectors of $\tilde{A}$.

Example 48.11 Let us revisit Example 48.10, but this time we compute the singular values of A by finding the positive eigenvalues of the augmented matrix (48.5). The results are shown in Table 48.2, where we compare the computed singular values with the exact values and include the relative error for each.

Positive Eigenvalues of $\tilde{A}$	Exact Values	Relative Error
4.8391e+12	4.8391e+12	1.4777e-16
1.9761e+10	1.9761e+10	5.3287e-14
1.8568e+08	1.8568e+08	9.9953e-12
3.0514e+06	3.0514e+06	3.1365e-11
7.8771e+04	7.8771e+04	1.0557e-09
2.7761e+03	2.7761e+03	2.3304e-07
9.0291e+01	9.0291e+01	7.7006e-07

Table 48.2: Computed singular values using the augmented matrix approach, exact singular values, and relative errors.

Compared to the results in Example 48.10, the accuracy of the computed singular values has improved significantly. In particular, the smallest singular values now have seven correct digits, whereas the naive algorithm failed to compute any correct digits for these values.

Although we could compute the SVD of A by forming the augmented matrix (48.5) and finding its eigenvalues and eigenvectors, practical algorithms use a more refined approach, taking advantage of its structure in clever ways. For one, we only need the nonnegative eigenvalues of (48.5), not all of them, so using an algorithm that computes all eigenvalues would be inefficient. Additionally, this method can still lead to accuracy issues (though less severe than when working with A^TA), especially when computing the singular vectors corresponding to the smallest singular values.

Algorithms for the SVD

We have seen how the SVD of a matrix is connected to the eigenvalues and eigenvectors of the symmetric matrices A^TA and $\left[\begin{smallmatrix} 0 & A^T \\ A & 0 \end{smallmatrix}\right]$. In practice, algorithms for computing the SVD are based on methods designed for symmetric matrices, but they are adapted to make the process faster and more reliable.

Most SVD algorithms follow two main steps:

1. **Reduce A to bidiagonal form B:** The first step is to transform A into a simpler, bidiagonal matrix B, that is, a matrix which has nonzeros only on its main diagonal and the diagonal above it. Similar to the Hessenberg reduction used in the QR algorithm, this simplification makes computing the SVD much easier, faster, and more memory-efficient. The bidiagonal reduction is done using orthogonal transformations, as we explained in Section 28.3.2. Mathematically, this step writes A as $A = U_1BV_1^T$, where U_1 and V_1 are orthogonal matrices.
2. **Compute the SVD of B:** Once A is in bidiagonal form B, the next step is to compute an SVD of B. This gives $B = U_2\Sigma V_2^T$, where Σ is a diagonal matrix of singular values.

Combining these two steps, we get

$$A = (U_1U_2)\Sigma(V_2V_1)^T.$$

which is an SVD of A, since the product of orthogonal matrices is also orthogonal.

The challenge now is to compute the SVD of B efficiently and accurately. Directly forming B^TB or the augmented matrix $\left[\begin{smallmatrix} 0 & B^T \\ B & 0 \end{smallmatrix}\right]$ can lead to numerical instability, especially for the smallest singular values. Instead, practical algorithms work directly on B or use clever transformations to avoid these issues.

Some traditional algorithms for computing the SVD of B include:

1. **QR iteration**: A variation of the QR algorithm applied implicitly to B^TB without actually forming the matrix.
2. **Divide-and-conquer**: A fast method that breaks the problem into smaller subproblems.

3. **Bisection and inverse iteration**: A technique for finding singular values with high precision.
4. **Jacobi's method**: An iterative method that diagonalizes the matrix step by step.

While we will not get into the details of these algorithms here, we encourage you to explore them further if you are curious about them!

In Machine Learning, where matrices are often very large, specialized algorithms are commonly used. Some of these include:

1. **Power iteration and block power iteration**: Used when only a few dominant singular values and vectors are needed. This is especially useful for dimensionality reduction.
2. **Randomized SVD**: A probabilistic method that approximates the SVD efficiently. It is widely used in recommendation systems and image compression.

These algorithms make it possible to compute the essential parts of the SVD, even for very large matrices, which is essential for many modern applications in Data Science and Machine Learning.

Problems

48.1 Let A, B, P, Q be $n \times n$ matrices. Suppose that P and Q are orthogonal matrices, while A and B are symmetric positive definite. Show that, if $QA = BP$, then $Q = P$.

48.2 Consider a matrix $A \in \mathbb{R}^{n\times n}$ with polar decomposition $A = QS$. Show that the spectrum of S, i.e., the set of eigenvalues of S, coincides with the singular values of A.

48.3 (Non-uniqueness of the polar decomposition) Consider a singular matrix $A \in \mathbb{R}^{n\times n}$ with a polar decomposition $A = QS$, where Q is orthogonal and S is positive semidefinite. This problem aims to show that there are infinitely many different polar decompositions for A.

Let $r = \operatorname{rank}(A) < n$, and let $\{z_1, \ldots, z_r\}$ be an orthonormal basis for the column space $\operatorname{col}(S)$. Extend this basis to an orthonormal basis $\{z_1, \ldots, z_r, z_{r+1}, \ldots, z_n\}$ of $\mathbb{R}^n$. As a result, $\{z_{r+1}, \ldots, z_n\}$ forms an orthonormal basis for the left nullspace $\operatorname{null}(S^T)$. Now, form the orthogonal matrix $Z = \begin{bmatrix} z_1 & \cdots & z_n \end{bmatrix} \in \mathbb{R}^{n\times n}$. For any orthogonal matrix $V \in \mathbb{R}^{(n-r)\times(n-r)}$, define the matrix

$$\widetilde{Q} = Q\, Z \operatorname{diag}(I_r, V) Z^T.$$

Show that $\widetilde{Q}$ is the orthogonal factor in a polar decomposition of A, i.e., $A = \widetilde{Q}S$.

Hint: Show that $Q(Sx) = \widetilde{Q}(Sx)$ for all $x \in \mathbb{R}^n$.

48.4 (SVD from the polar decomposition) Consider $A \in \mathbb{R}^{n\times n}$ with a polar decomposition $A = QS$, where Q is orthogonal and $S = V\Sigma V^T$ is a spectral decomposition of S, with V being orthogonal and Σ diagonal. Show that $A = (QV)\Sigma V^T = U\Sigma V^T$ is an SVD of A.

48.5 (Reversed polar decomposition) Given a matrix $A \in \mathbb{R}^{n\times n}$, show that we can factor it as $A = KQ$, where Q is orthogonal and K is positive semidefinite.

48.6 (Polar decomposition of tall and skinny matrices) Consider a tall and skinny matrix $A \in \mathbb{R}^{m\times n}$, where $m \geqslant n$. Show that there exists a matrix $Q \in \mathbb{R}^{m\times n}$ with orthonormal columns and a positive semidefinite matrix $S \in \mathbb{R}^{n\times n}$ such that $A = QS$. When is this polar decomposition unique?

Hint: Use the reduced SVD of A.

48.7 (Polar decomposition of symmetric matrices) Consider a symmetric matrix A with spectral decomposition $A = VDV^T$, where V is orthogonal ($V^TV = I_n = VV^T$), and $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ is a diagonal matrix with the eigenvalues of A arranged in descending order such that $|\lambda_1| \geqslant |\lambda_2| \geqslant \cdots \geqslant |\lambda_n| \geqslant 0$. Show that $S = V\operatorname{diag}(|\lambda_1|, \ldots, |\lambda_n|)V^T$ is the positive semidefinite factor in a polar decomposition of A. What is the orthogonal factor Q?

48.8 (Proof of Theorem 48.8) Consider a matrix $A \in \mathbb{R}^{m\times n}$ with SVD

$$A = U\Sigma V^T = U \left[\begin{smallmatrix} \Sigma_r & 0 \\ 0 & 0 \end{smallmatrix}\right] V^T.$$

Let $X \in \mathbb{R}^{n\times m}$ be written as

$$X = VMU^T = V \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} U^T,$$

where M is partitioned conformably with the partition of Σ, Find conditions on $M_{11}, M_{12}, M_{21}, M_{22}$ that minimize $\|I_m - AX\|_F^2$. Then, check that the pseudoinverse satisfies those conditions. Similarly, minimize $\|I_n - XA\|_F^2$ and investigate the corresponding conditions.

48.9 Prove the second equality in (48.4).

48.10 (The orthogonal Procrustes problem) In Greek mythology, Procrustes was notorious for forcing travelers to fit his bed, by either stretching them or cutting off their limbs. In the orthogonal Procrustes problem, we take a gentler approach: given two $m \times n$ matrices X and Y (think of X as the bed and Y as the traveler), we seek an orthogonal matrix Q that aligns Y with X (without any physical harm!)

There are two common formulations:

1. **Least squares criterion:**

$$\min_{Q^TQ=I_m} \|X - QY\|_F^2.$$

2. **Scalar product criterion:**

$$\max_{Q^TQ=I_m} \operatorname{trace}(X^TQY).$$

(a) Show that the two optimization problems above are equivalent by proving they have the same solution.

(b) Let $X, Y \in \mathbb{R}^{m\times n}$ and suppose $XY^T = QS$ is the polar decomposition of XY^T, where Q is orthogonal and S is positive semidefinite. Prove that Q is a solution to the orthogonal Procrustes problem.
(c) Explain how to solve the orthogonal Procrustes problem using the SVD of X^TY. In particular, show how to obtain a polar decomposition of X^TY from its SVD.
(d) Consider Figure 48.1: the left panel shows a 2D dataset X (resembling a ship on a calm sea), and the right panel shows a modified dataset Y (obtained by applying a rotation, reflection, and noise). Describe how you would use the orthogonal Procrustes problem to align Y with X.
(e) Research and discuss applications of the orthogonal Procrustes problem in Machine Learning.

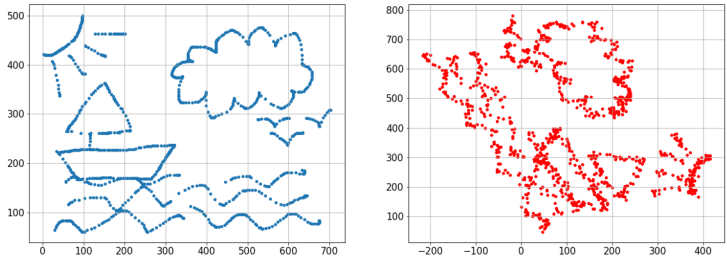


Fig. 48.1: Original 2-dimensional dataset X in the left panel. Modified dataset Y in the right panel, derived by applying rotation, reflection, and noise.

Chapter 49
X-FROM THE CLASSROOM TO REAL LIFE

The Singular Value Decomposition (SVD) is one of the most useful tools in matrix analysis and applied linear algebra. It is much more than just another matrix factorization. The SVD is central to many ideas and applications in mathematics, science, and engineering.

First, the SVD gives a reliable way to find the rank of any matrix, even if there are numerical errors or nearly dependent rows or columns. The singular values show the rank directly, so the SVD is a standard tool for finding the true dimension of a data set.

Besides rank, the SVD lets us extract orthonormal bases for all four fundamental subspaces of a matrix from the columns of U and V.

One of the most useful applications of the SVD is finding low-rank approximations of matrices. By truncating the SVD, we get the best possible lower-rank approximation of a matrix in a precise sense. This idea is used in data compression, image processing, and noise reduction.

Finally, the SVD is the basis for Principal Component Analysis (PCA), an important method in Statistics and Machine Learning. PCA uses the SVD to find the directions of greatest variance in data, which helps reduce dimensionality and find patterns in data.

In this chapter, we cover low-rank approximations and PCA, showing their theory and practical uses.

49.1 The best low-rank approximation

As we discussed in Chapter 45, data may appear high-dimensional but often contains a lot of redundant information. Even if a data matrix is full rank in the strict sense, it is often very close to being low-rank because the smallest singular values are due to noise or minor variations in the data. This leads to the idea of **low-rank approximation**. By replacing a matrix with one of lower rank, we remove unnecessary details, reduce storage needs, and often make the data easier to work with.

We need to address a few questions: How can we tell if a matrix can be well-approximated by one of lower rank? What rank should we choose, and how do we find it? Theorem 49.1 answers these. Singular values play a key role in measuring how close a matrix A is to a lower-rank approximation.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_49

Theorem 49.1. (Distance to lower-rank matrices and the best rank-k approximation)

Let $A \in \mathbb{R}^{m\times n}$ with $\operatorname{rank}(A) = r$, and let $\sigma_1 \geqslant \sigma_2 \geqslant \cdots \geqslant \sigma_r > 0$ be the nonzero singular values of A. Let $\|\cdot\|$ denote either the matrix 2-norm or the Frobenius norm. For any integer k such that $1 \leqslant k < r$, the distance from A to the closest rank-k matrix B is given by

$$\min_{\substack{B\in\mathbb{R}^{m\times n}\\ \operatorname{rank}(B)=k}} \|A - B\| = \begin{cases} \sigma_{k+1} & \text{for the 2-norm,} \\ \left(\sum_{i=k+1}^{r} \sigma_i^2\right)^{1/2} & \text{for the Frobenius-norm.} \end{cases} \tag{49.1}$$

Moreover, for a reduced singular value decomposition of A,

$$A = U_r \Sigma_r V_r^T = \begin{bmatrix} u_1 & \cdots & u_r \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix} \begin{bmatrix} v_1^T \\ \vdots \\ v_r^T \end{bmatrix},$$

the minimum in (49.1) is attained at the matrix

$$B = A_k := U_k \Sigma_k V_k^T = \begin{bmatrix} u_1 & \cdots & u_k \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_k \end{bmatrix} \begin{bmatrix} v_1^T \\ \vdots \\ v_k^T \end{bmatrix}.$$

Proof We prove the theorem for the 2-norm and leave the Frobenius norm case as an exercise.

Let B be any rank-k matrix. By definition, the 2-norm of $A - B$ is given by

$$\|A - B\|_2 = \max_{\|x\|_2=1} \|(A - B)x\|_2.$$

For any unit vector x, it follows that $\|A - B\|_2 \geqslant \|(A - B)x\|_2$. We will show that $\|A - B\|_2 \geqslant \sigma_{k+1}$ by finding a unit vector x such that $\|(A - B)x\|_2 \geqslant \sigma_{k+1}$.

Since $\operatorname{rank}(B) = k$, the nullspace of B has dimension $n - k$. Let $A = U\Sigma V^T$ be a singular value decomposition of A, with right singular vectors $v_1, v_2, \ldots, v_n$. Define the subspace $\mathcal{V}_k := \operatorname{span}(v_1, v_2, \ldots, v_{k+1})$. Since $\dim(\operatorname{null}(B)) + \dim(\mathcal{V}_k) > n$, the subspaces $\operatorname{null}(B)$ and $\mathcal{V}_k$ have a nonzero intersection.

Choose a unit vector $x \in \operatorname{null}(B) \cap \mathcal{V}_k$. Because $x \in \operatorname{null}(B)$, it follows that $Bx = 0$. Since $x \in \mathcal{V}_k$, we can write it as $x = \sum_{i=1}^{k+1} c_i v_i =: V_k c$ for some coefficients c_i.

Therefore, taking into account that V is an orthogonal matrix,

$$\begin{aligned}\|(A-B)x\|_2^2 = \|Ax\|_2^2 &= \|U\Sigma V^T x\|_2^2 \\ &\left\|\Sigma \begin{bmatrix} V_k^T \\ \widetilde{V}^T \end{bmatrix} V_k c\right\|_2^2 \\ &= \left\|\Sigma \begin{bmatrix} c \\ 0 \end{bmatrix}\right\|_2^2 \\ &= \sigma_1^2 c_1^2 + \cdots + \sigma_{k+1}^2 c_{k+1}^2 \\ &\geqslant \sigma_{k+1}^2 (c_1^2 + \cdots + c_{k+1}^2) = \sigma_{k+1}\|c\|_2^2 = \sigma_{k+1}^2 \|x\|_2^2.\end{aligned}$$

Since $\|x\|_2 = 1$, this implies $\|A - B\|_2 \geqslant \sigma_{k+1}$.

Next, we show that this distance is achievable. Consider a reduced singular value decomposition of A:

$$A = U_r \Sigma_r V_r^T,$$

where U_r and V_r contain the first r columns of U and V, respectively, and $\Sigma_r = \operatorname{diag}(\sigma_1, \ldots, \sigma_r)$. Define

$$A_k := U_k \Sigma_k V_k^T,$$

where U_k, V_k, and $\Sigma_k = \operatorname{diag}(\sigma_1, \ldots, \sigma_k)$ are the truncated versions containing the top k singular values along with their corresponding left and right singular vectors. Clearly, $\operatorname{rank}(A_k) = k$. Using the column-row definition of multiplication, we have

$$A - A_k = \sum_{i=1}^{r} \sigma_i u_i v_i^T - \sum_{i=1}^{k} \sigma_i u_i v_i^T = \sum_{i=k+1}^{r} \sigma_i u_i v_i^T.$$

It follows that the largest singular value of $A - A_k$ is σ_{k+1}. By Corollary 47.15, $\|A - A_k\|_2 = \sigma_{k+1}$.

Thus, we have shown that $\|A - B\|_2 \geqslant \sigma_{k+1}$ for any rank-k matrix B, and equality holds when $B = A_k$. Therefore, a closest rank-k matrix to A is A_k, and the distance is σ_{k+1}. □

Theorem 49.1 tells us that a matrix A can be well-approximated by a rank-k matrix when the $(k+1)$th singular value σ_{k+1} is small. It also shows how to build a best rank-k approximation: compute the SVD of A and retain only the top k singular values and their corresponding singular vectors.

Low-rank approximations reduce storage by representing large matrices more compactly. For an $m \times n$ matrix A, storing the full matrix requires mn entries. In contrast, a rank-k approximation $A_k = U_k \Sigma_k V_k^T$ of A, as illustrated in Figure 49.1, only requires storing the matrices U_k and V_k along with the k singular values in Σ_k. This totals $k(m+n+1)$ entries. When k is much smaller than $\min(m, n)$, the savings are substantial.

Example 49.2 Consider the matrix

$$A = \begin{bmatrix} 8 & 4 & 0 & 0 \\ 8 & 4 & 0 & 0 \\ 1 & -2 & 6 & 8 \\ -1 & 2 & 6 & 8 \end{bmatrix}.$$

A reduced SVD of A is given by

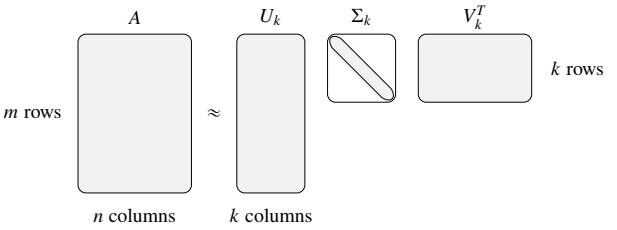


Fig. 49.1: Low-rank approximation: a more compact representation for large matrices.

$$A = \begin{bmatrix} 0 & 1/\sqrt{2} & 0 \\ 0 & 1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 0 & 1/\sqrt{2} \\ 1/\sqrt{2} & 0 & -1/\sqrt{2} \end{bmatrix} \begin{bmatrix} 10\sqrt{2} & 0 & 0 \\ 0 & 4\sqrt{10} & 0 \\ 0 & 0 & \sqrt{10} \end{bmatrix} \begin{bmatrix} 0 & 0 & 3/5 & 4/5 \\ 2/\sqrt{5} & 1/\sqrt{5} & 0 & 0 \\ 1/\sqrt{5} & -2/\sqrt{5} & 0 & 0 \end{bmatrix}.$$

The matrix A has rank equal to 3. Optimal rank-1 and a rank-2 approximations of A are, respectively,

$$A_1 = \begin{bmatrix} 0 \\ 0 \\ 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} \begin{bmatrix} 10\sqrt{2} \end{bmatrix} \begin{bmatrix} 0 & 0 & 3/5 & 4/5 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 6 & 8 \\ 0 & 0 & 6 & 8 \end{bmatrix}, \quad \text{and}$$

$$A_2 = \begin{bmatrix} 0 & 1/\sqrt{2} \\ 0 & 1/\sqrt{2} \\ 1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 0 \end{bmatrix} \begin{bmatrix} 10\sqrt{2} & 0 \\ 0 & 4\sqrt{10} \end{bmatrix} \begin{bmatrix} 0 & 0 & 3/5 & 4/5 \\ 2/\sqrt{5} & 1/\sqrt{5} & 0 & 0 \end{bmatrix} = \begin{bmatrix} 8 & 4 & 0 & 0 \\ 8 & 4 & 0 & 0 \\ 0 & 0 & 6 & 8 \\ 0 & 0 & 6 & 8 \end{bmatrix}.$$

What about the uniqueness of the best rank-k approximation in Theorem 49.1? Its uniqueness depends on both the norm used and the singular values of A.

For the 2-norm, the best rank-k approximation is never unique. For example, consider $A = \text{diag}(2, 1, \epsilon)$, where $0 < \epsilon < 1$. In this case, there are infinitely many best rank-2 approximations

$$A_2 = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad \text{and} \quad B_\ell = \begin{bmatrix} 2 - \epsilon/\ell & 0 & 0 \\ 0 & 1 - \epsilon/\ell & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \ell = 1, 2, 3, \ldots,$$

since $\|A - A_2\|_2 = \|A - B_\ell\|_2 = \sigma_3 = \epsilon$.

For the Frobenius norm, whether the best rank-k approximation is unique depends on the singular values. If $\sigma_{k+1} < \sigma_k$, then the best rank-k approximation is unique. However, if $\sigma_{k+1} = \sigma_k$, multiple best rank-k approximations may exist. For example, the 3×3 identity matrix I_3 has several best rank-2 approximations, including

$$A_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \text{and} \quad \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

How to choose the rank k for the approximation

When building a low-rank approximation for a matrix, we need to decide on the ideal rank k for the approximation. How should we choose k? Ideally, the singular values of A guide this decision. If the top few singular values are much larger than the rest, it makes sense to set k equal to the number of significant singular values.

However, in practice, this choice can be challenging because the singular values often decay gradually and noise in the data can obscure a clear cutoff. The goal is to keep k as small as possible while still achieving a sufficiently accurate approximation for your problem. What counts as "good enough" depends on the application. A common strategy is to **choose k so that the sum of the top k singular values makes up a large fraction of the total sum of all singular values**.

Ultimately, selecting k is a balance between accuracy of the approximation and efficiency for our specific task.

Example 49.3. (Fun with flags)

In this example, A is a 400×600 matrix whose entries are the pixel intensities of a black-and-white image of the Greek flag (top-left panel of Figure 49.2).

Can you guess the rank of A just from the image? The top-right panel, which shows the singular values of A in decreasing order, gives a clue: there is a sharp drop between the third and fourth singular values, with the fourth essentially zero. It is not exactly zero, likely due to noise in the pixel intensities. This means the rank of A is three.

The bottom panel shows the best rank-1, rank-2, and rank-3 approximations of A. Since A has rank 3, the rank-3 approximation matches the original flag exactly.

Example 49.4 The left panel of Figure 49.3 displays the singular values of the matrix representing the pixel intensities of the image in Figure 45.1 (introduced in Chapter 45). Notice the significant gap between the first and second singular values. After this gap, the singular values decrease steadily–initially rapidly, then more gradually. Unlike in Example 49.3, there isn't a clear cutoff point for a low-rank approximation.

To help guide the choice, the right panel of Figure 49.3 shows the cumulative fraction of the total sum of singular values captured by the leading singular values. When about 40 singular values are included, this cumulative fraction reaches the 0.9 threshold, suggesting a reasonable cutoff for constructing a low-rank approximation.

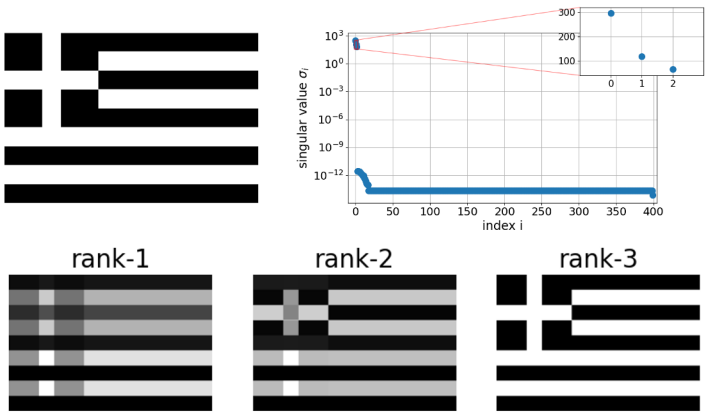


Fig. 49.2: Top-left: the original black-and-white flag. Top-right: the singular values of the matrix. Bottom: optimal rank-1, rank-2, and rank-3 approximations.

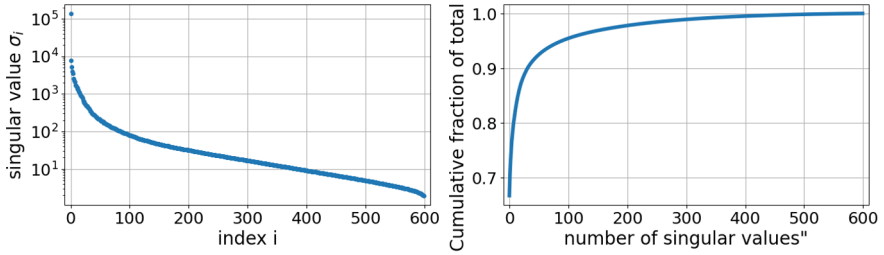


Fig. 49.3: The left panel shows the singular values in decreasing order. The right panel displays the cumulative fraction of the total sum of singular values.

49.2 Principal component analysis

Principal Component Analysis (PCA) is an important tool every data scientist should know. It reduces the number of features while keeping as much of the original information as possible, so the data can be well-approximated by linear combinations of the original features. This makes the data easier to work with, especially for large datasets.

The main applications of PCA are:

1. **Reducing dimensionality:** Instead of dealing with dozens or even hundreds of features, PCA creates a smaller set of new features, called principal components, that capture the main patterns in your data.
2. **Visualizing high-dimensional data:** By keeping just two or three principal components, PCA lets you plot complex data in 2D or 3D.
3. **Removing correlation:** The principal components are uncorrelated, since they are orthogonal to each other. This is particularly useful for many statistical models that work best with independent features.

4. **Noise reduction:** PCA focuses on the components that capture the most variance in your data, which often helps filter out less important, noisy details.

Like LDA, PCA looks for linear combinations of the original features that capture as much information as possible. However, they do this in different ways. LDA finds combinations that best separate the label classes, so it works with labeled data: a data matrix X and a label vector y. PCA, on the other hand, finds combinations that best reconstruct the data matrix X (in a sense we will explain shortly). This makes PCA an *unsupervised dimensionality reduction method*.

Let us see how Principal Component Analysis (PCA) can reduce the dimensionality of data, that is, the number of features for each data point. The main idea is that data in a high-dimensional space $\mathbb{R}^n$ can often be well approximated by points in a lower-dimensional subspace $\mathcal{V} \subset \mathbb{R}^n$. There is, however, a small complication: subspaces always pass through the origin, but our data may not be centered there.

Suppose we have m data points $x_1, x_2, \ldots, x_m \in \mathbb{R}^n$. If the mean

$$\mu = \frac{1}{m} \sum_{i=1}^{m} x_i$$

is not zero, then the data is not centered at the origin. PCA first shifts the data so its mean becomes zero. Instead of the original points x_i, we work with the centered points $x_i - \mu$, which satisfy

$$\frac{1}{m} \sum_{i=1}^{m} (x_i - \mu) = 0.$$

Once the data is centered, the goal is to find a subspace $\mathcal{V}$ so that each $x_i - \mu$ is well approximated by a vector in $\mathcal{V}$. Equivalently, each original point x_i is approximated as

$$x_i \approx \mu + v,$$

for some $v \in \mathcal{V}$. In other words, PCA approximates the data by points in an affine subspace, obtained by shifting $\mathcal{V}$ by μ. Figure 49.4 illustrates this idea.

As should be clear by now, the first goal is to find the low-dimensional subspace $\mathcal{V}$ from which we will obtain the approximated data. Assume $\mathcal{V}$ has dimension k (we will discuss how to choose k later) and is spanned by an orthonormal basis $\{v_1, \ldots, v_k\}$. Let V_k be the matrix with these basis vectors as columns:

$$V_k := \begin{bmatrix} v_1 & \cdots & v_k \end{bmatrix}.$$

From Chapter 32, the best way to approximate a centered data point $x_i - \mu$ with vectors in $\mathcal{V}$ is to take its orthogonal projection onto the subspace:

$$V_k V_k^T (x_i - \mu).$$

Thus, we approximate the original point x_i by centering it, projecting it, and then adding the mean back:

$$x_i \approx \widetilde{x}_i := \mu + V_k V_k^T (x_i - \mu) = \mu + V_k z_i. \tag{49.2}$$

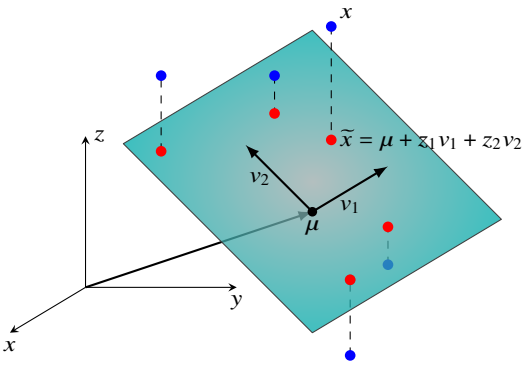


Fig. 49.4: Data points (blue) and their PCA approximations (red) using the 2-dimensional subspace $\mathcal{V}$, spanned by v_1 and v_2 shifted by μ. Each red point is the mean μ plus a vector in $\mathcal{V}$.

The vector $\widetilde{x}_i$ is the **reconstruction** of x_i. The vector

$$z_i := V_k^T (x_i - \mu) \in \mathbb{R}^k$$

is the **representation** of x_i and gives the coefficients of the linear combination of the columns of V_k that approximate the centered point $x_i - \mu$. Each direction v_j is a linear combination of the original features, and each z_i is a lower-dimensional version of the data, using only k features, each one a combination of the original n features.

But one important question remains: How do we choose the subspace $\mathcal{V}$ to approximate the data? Equivalently, how do we find the matrix V_k with orthonormal columns? Two common approaches are:

1. **Maximize variance:** Find the subspace that captures as much of the data's variability as possible.
2. **Minimize reconstruction error:** Find the subspace that lets us reconstruct the data as accurately as possible from its lower-dimensional form.

We start with the variance-maximization approach, the most common way to introduce PCA. Later, we will see that both approaches give the same result.

Before we start, note that variances and distances, which we use to measure reconstruction errors, depend on the scale of the features. For example, if one feature in our dataset is measured in thousands (like annual income in dollars) and another in single digits (like number of children), the first feature will dominate the computation. To avoid this, we can rescale the features, for instance by dividing each by its standard deviation, by its maximum value, or by another measure of its scale before applying PCA.

49.2.1 The first principal component: finding the line that maximizes data spread

Let us begin with a simple two-dimensional example. In Figure 49.5, we have a dataset with two features, centered around the origin. Notice that those features are correlated. If you know the value of one, you can make a good guess about the other. This correlation causes the data points to cluster along a narrow band, almost like they are arranged along a single line.

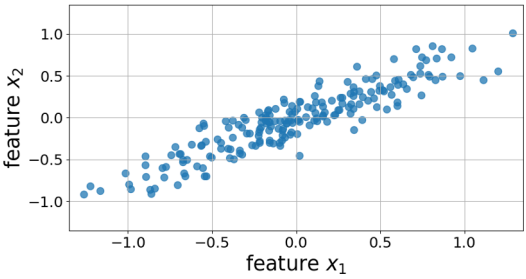


Fig. 49.5: A two-dimensional dataset with two correlated features.

PCA is useful in cases like this because it helps detect redundant features. If the same information can be captured with fewer variables, why keep them all? This becomes especially advantageous in high-dimensional datasets where visualization is not an option.

For instance, if the data in Figure 49.5 is nearly one-dimensional, how do we find the best one-dimensional representation? One natural approach is to orthogonally project the data onto a line. But, which line should we choose? Figure 49.6 illustrates the original data along with four possible lines for projection, and on the right, you can see how the data looks when projected onto each of these lines.

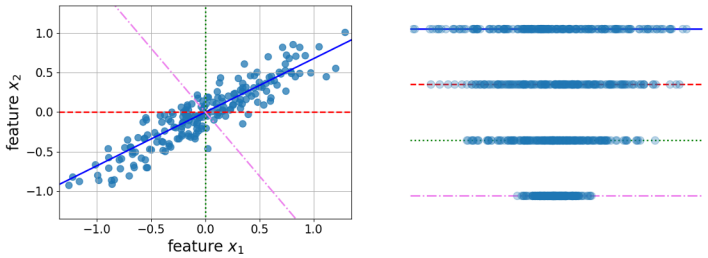


Fig. 49.6: Left: The original dataset with four candidate projection lines. Right: The result of projecting the data onto each line.

A natural choice is to select the line that preserves as much of the data's spread as possible. In our example, this is the solid blue line. This approach is the core idea

of PCA: to choose the line that maximizes the variance, or spread, of the projected data.

To formalize this idea, suppose we have m data points $x_1, x_2, \ldots, x_m \in \mathbb{R}^n$ with mean vector

$$\mu = \frac{1}{m} \sum_{i=1}^{m} x_i.$$

Our goal is to orthogonally project the centered[1] data points $x_i - \mu$ onto a line in the direction of a unit vector v. Recall from Chapter 32 that the orthogonal projection of a centered point $x_i - \mu$ onto the line spanned by v is given by

$$p_i := vv^T(x_i - \mu) =: v\, z_i.$$

The scalar z_i indicates how far along the direction of v the projection lies. Using this projection, we approximate the original data point x_i as

$$x_i \approx \widetilde{x}_i = \mu + vz_i.$$

Let us measure how much the approximations $\widetilde{x}_i$ deviate from their mean

$$\widetilde{\mu} = \frac{1}{m} \sum_{i=1}^{m} \widetilde{x}_i,$$

that is, let us determine the variance of the projected data. An important observation is that these approximations actually share the same mean as the original data. To see why, note that

$$\widetilde{\mu} = \frac{1}{m} \sum_{i=1}^{m} \widetilde{x}_i = \frac{1}{m} \sum_{i=1}^{m} \left(\mu + vv^T(x_i - \mu)\right) = \mu + \frac{vv^T}{m} \left(\sum_{i=1}^{m} x_i - m\mu\right) = \mu.$$

In other words, the reconstructed data remains centered at μ.

To quantify the variance of the data $\widetilde{x}_i$, we need to determine how far the points $\widetilde{x}_i$ are from their mean μ (or, equivalently, how much the distances $\|\widetilde{x}_i - \mu\|_2^2$ differ from zero). Consider the average squared distance

$$\frac{1}{m} \sum_{i=1}^{m} \|\widetilde{x}_i - \mu\|_2^2. \tag{49.3}$$

A larger value indicates that the approximations are more spread out along the line in the direction of v. Replacing the expression $\widetilde{x}_i = \mu + vv^T(x_i - \mu)$ in the average squared distance formula, we get

[1] For variance maximization, the choice of centering vector does not affect the result. Using the mean, however, lets us view the projections as approximations of the centered data points, as we will see in Section 49.2.7.

$$\begin{aligned}\frac{1}{m}\sum_{i=1}^{m}\|\widetilde{x_i}-\mu\|_2^2 &= \frac{1}{m}\sum_{i=1}^{m}\|v(v^T(x_i-\mu))\|_2^2 \\ &= \frac{1}{m}\sum_{i=1}^{m}(v^T(x_i-\mu))^2 = \frac{1}{m}\sum_{i=1}^{m}z_i^2,\end{aligned} \tag{49.4}$$

where we used the fact that v is a unit vector and $v^T(x_i-\mu)$ is a scalar. Thus, the variance of the projected data is the same as the variance of the representation of the data, as we show next.

The variance of the z_i coordinates

The scalar $z_i = v^T(x_i-\mu)$ is the coordinate of the projection of the centered data point $x_i-\mu$ onto the line spanned by v. The average squared distance we have computed (see equations (49.3)-(49.4)) is exactly the variance of these scalars z_i. To see why, first observe that the z_i's have a mean of zero:

$$\frac{1}{m}\sum_{i=1}^{m}z_i = \frac{1}{m}\sum_{i=1}^{m}v^T(x_i-\mu) = \frac{v^T}{m}\left(\sum_{i=1}^{m}x_i - m\mu\right) = 0.$$

Because the mean is zero, the variance of the z_i's is simply the average of their squares:

$$\frac{1}{m}\sum_{i=1}^{m}z_i^2,$$

which is exactly what is given in (49.4).

Figure 49.7 illustrates this idea. If we think of the line span(v) as the real number line $\mathbb{R}$, then projecting each point gives a single value z. Instead of working in the high-dimensional space $\mathbb{R}^n$, we represent each point by one coordinate. The variance of these values around zero is what we aim to maximize in PCA, which is why PCA is often described as finding the direction of maximum variance.

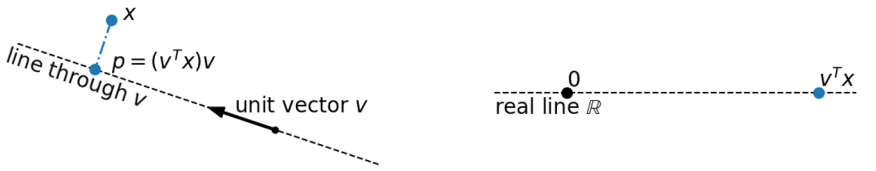


Fig. 49.7: Projection of multi-dimensional data onto a line, transforming high-dimensional points into one-dimensional coordinates.

To find the unit vector v that maximizes (49.4), we first rewrite that expression in a more convenient form, which turns the problem into a familiar maximization problem. We begin by arranging the centered data points $x_i-\mu$ as rows in an $m \times n$

matrix, which we call the **centered data matrix**:

$$\begin{bmatrix} x_1^T - \mu^T \\ \vdots \\ x_m^T - \mu^T \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

We can write this matrix more compactly as

$$\begin{bmatrix} x_1^T - \mu^T \\ \vdots \\ x_m^T - \mu^T \end{bmatrix} = \begin{bmatrix} x_1^T \\ \vdots \\ x_m^T \end{bmatrix} - \begin{bmatrix} \mu^T \\ \vdots \\ \mu^T \end{bmatrix} = A - e\mu^T,$$

where A is the original data matrix (with each x_i^T as its rows) and e is the $m \times 1$ column vector of ones.

Since $z_i = v^T(x_i - \mu)$ is a scalar, we can rewrite (49.4) as

$$\begin{aligned} \frac{1}{m}\sum_{i=1}^{m} z_i^2 &= \frac{1}{m}\sum_{i=1}^{m} \left(v^T x_i - v^T\mu\right)^2 \\ &= \frac{1}{m}\sum_{i=1}^{m} \left(v^T x_i - v^T\mu\right)\left(v^T x_i - v^T\mu\right)^T \\ &= \frac{1}{m}\sum_{i=1}^{m} \left(v^T x_i - v^T\mu\right)\left(x_i^T v - \mu^T v\right) \\ &= \frac{1}{m} v^T \left(\sum_{i=1}^{m} (x_i - \mu)(x_i - \mu)^T\right) v \end{aligned}$$

Using the columns-times-rows picture of matrix multiplication, we have

$$\sum_{i=1}^{m} (x_i - \mu)(x_i - \mu)^T = \left(A - e\mu^T\right)^T \left(A - e\mu^T\right).$$

Thus,

$$\frac{1}{m}\sum_{i=1}^{m} \left(v^T x_i - v^T\mu\right)^2 = \frac{1}{m} v^T \left(A - e\mu^T\right)^T \left(A - e\mu^T\right) v = \frac{1}{m}\left\|\left(A - e\mu^T\right) v\right\|_2^2.$$

This reformulation sets up our maximization problem: we want to find a unit vector v that maximizes

$$\frac{1}{m}\left\|\left(A - e\mu^T\right) v\right\|_2^2.$$

This is exactly the type of problem that the Singular Value Decomposition solves. According to Theorem 47.14, we have

$$\max_{\|v\|_2=1} \sum_{i=1}^{m} \|\widetilde{x_i} - \mu\|_2^2 = \max_{\|v\|_2=1} \sum_{i=1}^{m} (v^T x_i - v^T\mu)^2 = \max_{\|v\|_2=1} \|(A - e\mu^T)v\|_2^2 = \sigma_1^2,$$

where σ_1 is the largest singular value of the centered data matrix $A - e\mu^T$. The unit vector v_1 that attains this maximum, is a right singular vector corresponding to σ_1, and we call it the **first principal component**.

Example 49.5 Consider the dataset in Table 49.1, which contains four data points (rows), each of which has three features (columns). Notice that the third feature is approximately the average of the first two. The last row of Table 49.1 shows the mean value for each feature. Thus, $\mu^T = [-0.06, 0.5, 0.13]$.

	Feature 1	Feature 2	Feature 3
Point 1	-0.09	0.73	0.63
Point 2	-1.44	-0.66	-0.97
Point 3	-0.10	2.15	0.61
Point 4	1.39	-0.21	0.26
Mean	-0.06	0.5	0.13

Table 49.1: A dataset with four data points (rows) and three features (columns).

The centered data matrix for this dataset is given by

$$A - e\mu^T = \begin{bmatrix} -0.09 & 0.73 & 0.63 \\ -1.44 & -0.66 & -0.97 \\ -0.10 & 2.15 & 0.61 \\ 1.39 & -0.21 & 0.26 \end{bmatrix} - \begin{bmatrix} -0.06 & 0.50 & 0.13 \\ -0.06 & 0.50 & 0.13 \\ -0.06 & 0.50 & 0.13 \\ -0.06 & 0.50 & 0.13 \end{bmatrix} = \begin{bmatrix} \text{`} - 0.03 & 0.23 & 0.50 \\ -1.38 & -1.16 & -1.10 \\ -0.04 & 1.65 & 0.48 \\ 1.45 & -0.71 & 0.13 \end{bmatrix}.$$

Its largest singular value and corresponding right singular vector (first principal component) are

$$\sigma_1 = 2.53 \qquad \text{and} \qquad v_1 = \begin{bmatrix} 0.49 \\ 0.72 \\ 0.49 \end{bmatrix}.$$

The vector v_1 gives the direction of greatest variance in the data. Equivalently, the feature

$$0.49 \cdot \text{Feature 1} + 0.72 \cdot \text{Feature 2} + 0.49 \cdot \text{Feature 3}$$

is the linear combination of the original features with the largest variance.

In Figure 49.8, we have plotted the four data points, the line generated by the first principal component v_1, as well as the orthogonal projections of the data on the line.

The line spanned by the first principal component is a good one-dimensional subspace for projecting our data. But what if we want to capture more of the data's structure by projecting onto a higher-dimensional subspace? We can build such a subspace step by step, adding new directions that are orthogonal to those already chosen.

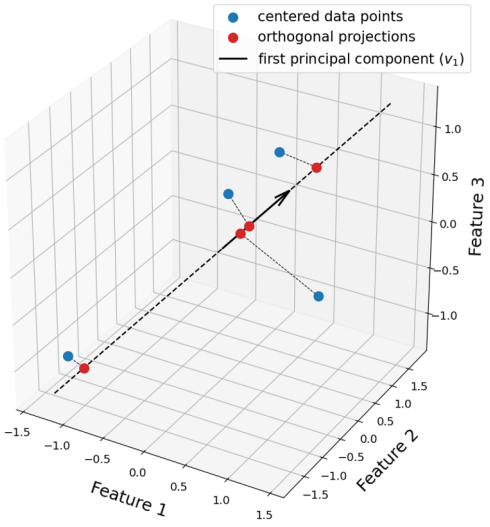


Fig. 49.8: Centered data points from Example 49.5, first principal component, and the projections of the centered data points.

49.2.2 Second and higher principal components

We have identified the first principal component by finding the direction along which the projected data has the largest variance. To find the second principal component, we use the same idea, with one extra condition: the new direction must be orthogonal to the first principal component v_1. In other words, among all directions perpendicular to v_1, we choose the one that maximizes the variance of the projected data. This leads to the optimization problem

$$\max_{\substack{\|v\|_2=1 \\ v\perp v_1}} \sum_{i=1}^{m} \|\widetilde{x}_i - \mu\|_2^2 = \max_{\substack{\|v\|_2=1 \\ v\perp v_1}} \left\| \left(A - e\mu^T\right) v \right\|_2^2 .$$

It follows from Theorem 47.14 that the maximum value is σ_2^2, where σ_2 is the second-largest singular value of the centered data matrix $A - e\mu^T$. The unit vector v_2 that attains this maximum is a right singular vector associated with σ_2, and we call it the **second principal component**.

We can keep repeating this process to find additional principal components. For the jth principal component, we search for a unit vector v_j that is orthogonal to the previously selected vectors $v_1, v_2, \ldots, v_{j-1}$ and that maximizes the variance of the projected data. In other words, we solve the optimization problem

$$\max_{\substack{\|v\|_2=1 \\ v\perp v_1, v_2, \ldots, v_{j-1}}} \sum_{i=1}^{m} \|\widetilde{x}_i - \mu\|_2^2 = \max_{\substack{\|v\|_2=1 \\ v\perp v_1, v_2, \ldots, v_{j-1}}} \left\| \left(A - e\mu^T\right) v \right\|_2^2 .$$

According to Theorem 47.14, the maximum value of this expression is σ_j^2, where σ_j is the jth singular value of the centered data matrix $A - e\mu^T$. The unit vector v_j that achieves this maximum is a corresponding right singular vector, and we refer to it as the **jth principal component**.

Example 49.6 Returning to Example 49.5, the second and third singular values, along with their corresponding right singular vectors, are

$$\sigma_2 = 1.94 \qquad \text{and} \qquad v_2 = \begin{bmatrix} 0.80 \\ -0.59 \\ 0.05 \end{bmatrix},$$

$$\sigma_3 = 0.42 \qquad \text{and} \qquad v_3 = \begin{bmatrix} -0.33 \\ -0.37 \\ 0.87 \end{bmatrix}.$$

The vector v_2 captures the second-largest variance in the data while remaining orthogonal to v_1.

Figure 49.9 shows the centered four data points, the subspace spanned by the first two principal components, and the orthogonal projections of those points.

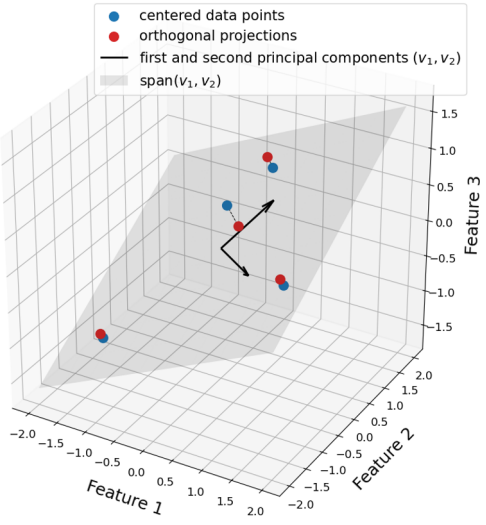


Fig. 49.9: Centered data points from Example 49.5, the first and second principal components with their span, and the projections of the centered data points.

The vector v_3 represents the direction with the least variance. Since the data is three-dimensional, there are no more principal components.

49.2.3 PCA representations and reconstructions

After identifying the first k principal components $v_1, v_2, \ldots, v_k$, we arrange them as the columns of a matrix

$$V_k = \begin{bmatrix} v_1 & v_2 & \cdots & v_k \end{bmatrix}.$$

We then compute the coordinates of the orthogonal projection of each centered data point $x_i - \mu$ onto the subspace $\text{span}\{v_1, \ldots, v_k\}$ with respect to the orthonormal basis $\{v_1, \ldots, v_n\}$:

$$z_i = \begin{bmatrix} v_1^T(x_i - \mu) \\ \vdots \\ v_k^T(x_i - \mu) \end{bmatrix} = V_k^T(x_i - \mu) \in \mathbb{R}^k.$$

These vectors z_i are called **representations** because they contain the coefficients of the linear combination of principal components that approximate the centered data point $x_i - \mu$.

Each z_i is a lower-dimensional version of the original data point, using only k features instead of n. We refer to these new features as the **principal component features** (PC-1, PC-2, ..., PC-k). The idea is that we can now use these principal component features in place of the original ones. Later, we will discuss how well this reduced representation captures the original data.

One important thing to note is that the principal component features are centered, that is, their mean is zero:

$$\frac{1}{m}\sum_{i=1}^{m} z_i = \frac{1}{m}\sum_{i=1}^{m} V_k^T(x_i - \mu) = \frac{1}{m}V_k^T\left(\sum_{i=1}^{m} x_i - m\mu\right) = 0.$$

If we stack the representations z_i as rows of a matrix, we obtain

$$Z_k := \begin{bmatrix} z_1^T \\ \vdots \\ z_m^T \end{bmatrix} = \begin{bmatrix} (x_1 - \mu)^T V_k \\ \vdots \\ (x_m - \mu)^T V_k \end{bmatrix} = (A - e\mu^T)V_k \in \mathbb{R}^{m \times k},$$

which is the data matrix in the principal component feature space. Each row of Z_k is a lower-dimensional representation of the corresponding original data point. In summary, PCA has reduced our original n-dimensional centered data matrix to a k-dimensional one.

Example 49.7 Returning to the data in Example 49.5, suppose we decide to keep only the first two principal components (we will discuss how to choose the number of components in Section 49.2.5). The matrix below contains the representations of our four data points, with each row corresponding to a data point.

$$Z_k = (A - e\mu^T)V_2 = \begin{bmatrix} ‘ -0.03 & 0.23 & 0.50 \\ -1.38 & -1.16 & -1.10 \\ -0.04 & 1.65 & 0.48 \\ 1.45 & -0.71 & 0.13 \end{bmatrix} \begin{bmatrix} 0.49 & 0.80 \\ 0.72 & -0.59 \\ 0.49 & 0.05 \end{bmatrix} = \begin{bmatrix} 0.39 & -0.13 \\ -2.06 & -0.48 \\ 1.40 & -0.98 \\ 0.27 & 1.59 \end{bmatrix}.$$

This gives a two-dimensional representation of our original three-dimensional data. Now, each data point is represented using just two features: PC-1 and PC-2. You can also verify that both have a mean of zero.

If needed, we can transform the k principal component features back into the original n-dimensional space by computing

$$\widetilde{x}_i = \mu + V_k z_i,$$

for each data point. These $\widetilde{x}_i$ are called the **reconstructions** of the original data points. Stacking them as rows in a matrix gives an approximation of the original data matrix:

$$A = \begin{bmatrix} x_1^T \\ \vdots \\ x_m^T \end{bmatrix} \approx \widetilde{A} := \begin{bmatrix} \widetilde{x}_1^T \\ \vdots \\ \widetilde{x}_m^T \end{bmatrix} = \begin{bmatrix} \mu^T \\ \vdots \\ \mu^T \end{bmatrix} + \begin{bmatrix} z_1^T V_k^T \\ \vdots \\ z_m^T V_k^T \end{bmatrix} = e\mu^T + Z_k V_k^T. \tag{49.5}$$

A useful property of these PCA reconstructions is that they share the same mean vector μ as the original data points,

$$\frac{1}{m}\sum_{i=1}^{m} \widetilde{x}_i = \frac{1}{m}\sum_{i=1}^{m} (\mu + V_k z_i) = \frac{1}{m}\left(m\mu + V_k \sum_{i=1}^{m} z_i \right) = \mu,$$

which follows from the fact that the principal component features are centered. Therefore, we can write

$$A - e\mu^T = \begin{bmatrix} x_1^T - \mu^T \\ \vdots \\ x_m^T - \mu^T \end{bmatrix} \approx \widetilde{A} - e\mu^T = \begin{bmatrix} \widetilde{x}_1^T - \mu^T \\ \vdots \\ \widetilde{x}_m^T - \mu^T \end{bmatrix} = Z_k V_k^T.$$

This shows that $Z_k V_k^T$ is a centered matrix that approximates the original centered data matrix.

Example 49.8 Continuing with Example 49.7, the reconstruction of the original data matrix using the first two principal components is

$$e\mu^T + Z_k V_k^T = \begin{bmatrix} -0.06 & 0.50 & 0.13 \\ -0.06 & 0.50 & 0.13 \\ -0.06 & 0.50 & 0.13 \\ -0.06 & 0.50 & 0.13 \end{bmatrix} + \begin{bmatrix} 0.39 & -0.13 \\ -2.06 & -0.48 \\ 1.40 & -0.98 \\ 0.27 & 1.59 \end{bmatrix} \begin{bmatrix} 0.49 & 0.72 & 0.49 \\ 0.80 & -0.59 & 0.05 \end{bmatrix}$$

$$= \begin{bmatrix} 0.02 & 0.86 & 0.31 \\ -1.46 & -0.69 & -0.91 \\ -0.16 & 2.08 & 0.77 \\ 1.35 & -0.25 & 0.35 \end{bmatrix}.$$

This reconstruction is close to the original data matrix in Table 49.1, but some information is lost because only two principal components are used.

49.2.4 Principal component analysis via the SVD

As we have seen, the principal components $v_1, v_2, \ldots$ are the right singular vectors of the centered data matrix $A - e\mu^T$ corresponding to its largest singular values. To obtain the first k principal components, we compute a singular value decomposition on the centered data matrix:

$$A - e\mu^T = \begin{bmatrix} x_1^T - \mu^T \\ x_2^T - \mu^T \\ \vdots \\ x_m^T - \mu^T \end{bmatrix} = U\Sigma V^T, \quad \text{where } V = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}.$$

The first k principal components of A are the first k columns of V, that is, $v_1, \ldots, v_k$. We continue to denote $V_k = \begin{bmatrix} v_1 & \cdots & v_k \end{bmatrix}$.

We now show that the SVD of the centered data matrix gives not only the principal components but also the PCA representations, features, and reconstructions of the data.

First, we can express the data matrix $A - e\mu^T$ in the principal component feature space as

$$Z_k = (A - e\mu^T)V_k = U_k\Sigma_k,$$

where $\Sigma_k = \operatorname{diag}(\sigma_1, \ldots, \sigma_k)$ contains the top k singular values of $A - e\mu^T$ and $U_k = \begin{bmatrix} u_1 & \cdots & u_k \end{bmatrix}$ holds the corresponding left singular vectors.

Second, we can express the PCA reconstruction of the data matrix A using the first k principal components as

$$\widetilde{A} = e\mu^T + Z_k V_k^T = e\mu^T + U_k\Sigma_k V_k^T.$$

PCA and best rank-k approximations

We can rewrite the PCA reconstruction of the data matrix $A - e\mu^T$ as

$$\widetilde{A} - e\mu^T = Z_k V_k^T = U_k\Sigma_k V_k^T,$$

which, by Theorem 49.1, shows that the centered PCA reconstruction of A is in fact a best rank-k approximation of the original centered data matrix.

49.2.5 Choosing the number of principal components: the total variance explained

Now that we know how to compute principal components, the next step is to decide how many principal components k to keep. We will start with a simple example to build our intuition, and then move on to a more detailed discussion on selecting the right k.

Example 49.9 Consider the dataset in Table 49.1 from Example 49.5. Our data has three features, and we want to understand how much variation exists along each one. One way to measure this variation is by computing the variance of each feature separately. If a feature has high variance, its values are more spread out; if it has low variance, they are more tightly clustered.

Rounding to two decimal places, the three feature variances are

$$\begin{aligned}
\sigma_1 &= \frac{1}{4}\Big((-0.09+0.06)^2 + (-1.44+0.06)^2 \\
&\qquad + (-0.1+0.06)^2 + (1.39+0.06)^2\Big) = 1.00, \\
\sigma_2 &= \frac{1}{4}\Big((0.73-0.5)^2 + (-0.66-0.5)^2 \\
&\qquad + (2.15-0.5)^2 + (-0.21-0.5)^2\Big) = 1.16, \quad \text{and} \\
\sigma_3 &= \frac{1}{4}\Big((0.63-0.13)^2 + (-0.97-0.13)^2 \\
&\qquad + (0.61-0.13)^2 + (0.26-0.13)^2\Big) = 0.42.
\end{aligned}$$

Now, if we add up the variances of all features, we get the **total variance**:

$$1.00 + 1.16 + 0.42 = 2.58.$$

To determine how much each feature contributes to the total variance, we divide each variance by the total variance. This gives us the percentage of the total variance that each feature accounts for. Table 49.2 summarizes these percentages, showing which features contribute most significantly to the overall variation.

	Feature 1	Feature 2	Feature 3	Total
Variance	1.00	1.16	0.42	2.58
% Total Variance	39%	45%	16%	100%

Table 49.2: Variance contribution of each feature in the dataset from Example 49.5.

Recall that in PCA, we transform our original data into a new set of features called principal component features. These components are ranked by the amount of variation they capture from the original data.

Example 49.10 Let us revisit the small dataset from Example 49.5. In Examples 49.5 and 49.6, we computed the three principal components. Since the original dataset had three features, there are no additional principal components. The representations of our data points in the new principal component space, given by the three principal components, are given by

$$Z_3 = (A - e\mu^T)V_3 = \begin{bmatrix} 0.39 & -0.14 & 0.35 \\ -2.07 & -0.47 & -0.07 \\ 1.4 & -0.98 & -0.18 \\ 0.28 & 1.59 & -0.1 \end{bmatrix}.$$

Each column of Z_3 corresponds to one principal component feature (PC-1, PC-2, and PC-3).

To understand how important each principal component feature is, we compute the variance for each one. Table 49.3 shows both the variance and the percentage of the total variance captured by each component.

	PC1	PC2	PC3	Total
Variance	1.60	0.94	0.04	2.58
% Total Variance	62%	36%	2%	100%

Table 49.3: Variance contribution of each principal component feature in the dataset from Example 49.5.

There are two important things to notice. First, the total variance is the same as the total variance of the original features. We have not lost any information, only redistributed it among the new principal component features. Second, the first principal component captures the most variance, followed by the second, and then the third, as explained in Section 49.2.2. The third principal component explains only 2% of the total variance and adds little new information. Since 98% of the total variance is captured by the first two components, we can drop the third without losing much detail. This is why the PCA reconstruction in Example 49.8 was a good approximation to the original data matrix.

The singular values of the centered data matrix quantify how much variance each principal component captures, as we show in the following theorem.

Theorem 49.11. (Total variance explained)

Consider a dataset $\{x_1, \ldots, x_m\} \subseteq \mathbb{R}^n$ arranged as the rows of the data matrix $A \in \mathbb{R}^{m \times n}$. Let $\mu = \frac{1}{m}\sum_{i=1}^{m} x_i$ be the mean vector of the dataset. Let $\sigma_1, \ldots, \sigma_r$ be

the r nonzero singular values of the centered data matrix $A - e\mu^T$, with corresponding left and right singular vectors $u_1, \ldots, u_r$ and $v_1, \ldots, v_r$. Then, the **total variance** of the dataset is given by

$$\text{total variance} = \frac{1}{m}\|A - e\mu^T\|_F^2 = \frac{1}{m}\sum_{j=1}^{r}\sigma_j^2.$$

The **variance explained by the jth principal component feature** (corresponding to v_j) and the **proportion of total variance explained by it** are, respectively,

$$\frac{1}{m}\sigma_j^2 \qquad \text{and} \qquad \frac{\sigma_j^2}{\sum_{i=1}^{r}\sigma_i^2}.$$

Proof Let us start by deriving the total variance formula. The variance of the jth feature (i.e., the jth column of the data matrix A) is given by

$$\text{var(feature } j) = \frac{1}{m}\sum_{i=1}^{m}(A_{ij} - \mu_j)^2 = \frac{1}{m}\sum_{i=1}^{m}(A - e\mu^T)_{ij}^2,$$

where μ_j denotes the mean of the jth column of A. Summing over all n features, we obtain the total variance:

$$\begin{aligned}\text{total variance} &= \sum_{j=1}^{n}\frac{1}{m}\sum_{i=1}^{m}(A - e\mu^T)_{ij}^2 \\ &= \frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{n}(A - e\mu^T)_{ij}^2 \\ &= \frac{1}{m}\|A - e\mu^T\|_F^2.\end{aligned}$$

By Theorem 47.16, the squared Frobenius norm of $A - e\mu^T$ equals the sum of the squares of its nonzero singular values. Substituting this into the total variance formula gives

$$\text{total variance} = \frac{1}{m}\sum_{j=1}^{r}\sigma_j^2,$$

as we wanted to show.

Now, let us prove the result about the variance explained by the jth principal component. The principal component representation of the data point x_i is

$$z_i^T = (x_i - \mu)^T V_k, \quad \text{where} \quad V_k = \begin{bmatrix} v_1 & \cdots & v_k \end{bmatrix},$$

for $i = 1, \ldots, m$. Stacking these for all m data points, we get the principal component representation matrix

$$Z_k = (A - e\mu^T)V_k.$$

Each column of $Z_k \in \mathbb{R}^{m\times k}$ corresponds to a principal component feature. Since these features all have mean zero, the variance of the jth principal component feature is

$$\text{var}(\text{PC}_j) = \frac{1}{m}\sum_{i=1}^{m} z_{ij}^2.$$

Using the reduced SVD $A - e\mu^T = U_r\Sigma_r V_r^T$, we can express the representation matrix as $Z_r = U_r\Sigma_r$, where $\Sigma_r = \text{diag}(\sigma_1, \ldots, \sigma_r)$ and $U_r = \begin{bmatrix} u_1 & \cdots & u_r \end{bmatrix}$. Thus, since the columns of U are orthonormal, the variance of the jth principal component is

$$\text{var}(\text{PC}_j) = \frac{1}{m}\sum_{i=1}^{m}(U_r\Sigma_r)_{ij}^2 = \frac{1}{m}\sum_{i=1}^{m}\sigma_j^2 U_r[i,j]^2 = \frac{\sigma_j^2}{m}\|U_r[:,j]\|_2^2 = \frac{\sigma_j^2}{m}.$$

Dividing by the total variance, we obtain the fraction of variance explained by the jth principal component:

$$\frac{\text{var}(\text{PC}_j)}{\text{total variance}} = \frac{\sigma_j^2}{\sigma_1^2 + \cdots + \sigma_r^2},$$

as we claimed. □

Example 49.12 The singular values of the centered data matrix for the dataset in Example 49.5 are $\sigma_1 = 2.53$, $\sigma_2 = 1.94$ and $\sigma_3 = 0.42$. Using Theorem 49.11, we compute the proportion of total variance explained by each principal component (rounded to two decimal places):

$$\begin{aligned}
\text{var}(\text{PC}_1) &= \frac{2.53^2}{2.53^2 + 1.94^2 + 0.42^2} = 0.62,\\
\text{var}(\text{PC}_2) &= \frac{1.94^2}{2.53^2 + 1.94^2 + 0.42^2} = 0.36, \quad \text{and}\\
\text{var}(\text{PC}_3) &= \frac{0.42^2}{2.53^2 + 1.94^2 + 0.42^2} = 0.02.
\end{aligned}$$

These values match the results from Example 49.9.

Returning to our original question, how do we choose the number of principal components k? The goal is to select k so that the variance in the discarded components is small enough to be considered negligible. What counts as "small" depends on the context and the level of detail you are willing to give up.

Example 49.13. (PCA with a 2D ellipse in 3D space)

For this example, we start by creating a dataset that lies on a 2D ellipse, as shown in the left panel of Figure 49.10. We then embed this dataset into 3D space by adding a z-component defined as $z = y + 5$. To make the data more interesting, we add some random noise to the z-values. This ensures the points no longer lie perfectly on a plane but remain close to one. The resulting 3D dataset is shown in the right panel of Figure 49.10.

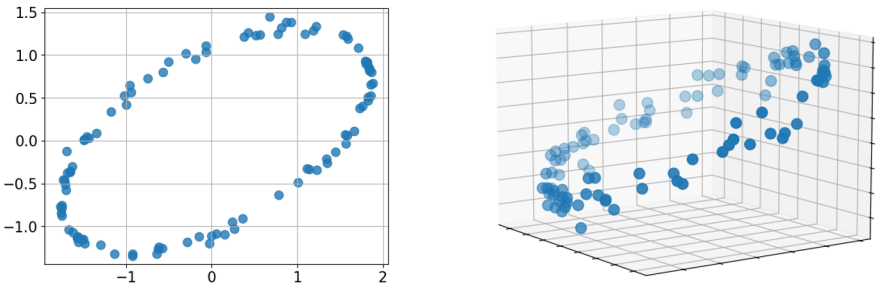


Fig. 49.10: Left: The original 2D dataset on an ellipse. Right: The dataset transformed into 3D space with added noise to the z-values.

For now, suppose we do not know that our 3D dataset $x_1, \ldots, x_m$ is almost two-dimensional. We will use PCA to discover this.

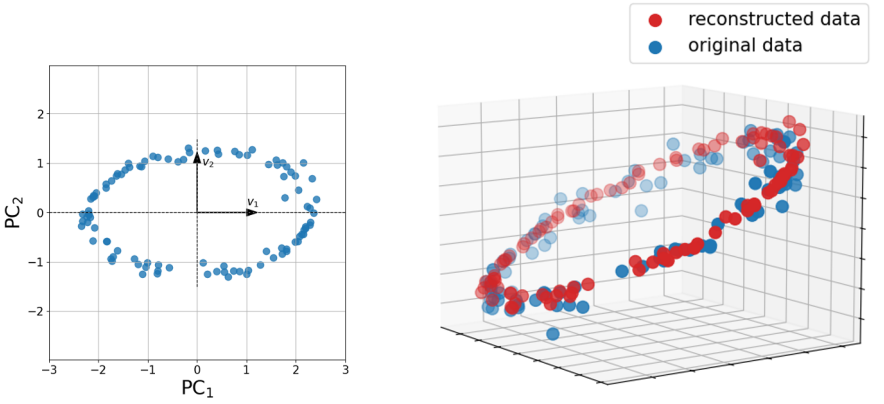
First, we compute the mean vector $\mu = \frac{1}{m}\sum_{i=1}^{m} x_i$ and subtract it from each data point to center the dataset. This gives us the centered matrix $A - e\mu^T$. Next, we compute an SVD of $A - e\mu^T$. Its singular values are $\sigma_1 = 16.2$, $\sigma_2 = 8.5$ and $\sigma_3 = 1.3$. From Theorem 49.11, the proportion of variance captured by each principal component feature is

$$\begin{aligned}
\text{var}(\text{PC}_1) &= \frac{16.2^2}{16.2^2 + 8.5^2 + 1.3^2} = 0.780, \\
\text{var}(\text{PC}_2) &= \frac{8.5^2}{16.2^2 + 8.5^2 + 1.3^2} = 0.215, \quad \text{and} \\
\text{var}(\text{PC}_3) &= \frac{1.3^2}{16.2^2 + 8.5^2 + 1.3^2} = 0.005.
\end{aligned}$$

The third principal component accounts for only 0.5% of the total variance, showing that the data is nearly two-dimensional. We therefore keep only the first two components, v_1 and v_2, and compute the 2D principal component representation of each point.

The left panel of Figure 49.11 shows the 2D principal component representations of the data points, which preserve the original elliptical shape of the dataset. The right panel compares the original 3D points (blue) with the reconstructed points (red),

now lying in a two-dimensional subspace. Even after reducing the dimensionality, the reconstructed points remain very close to the originals.



(a) 2D principal component representation of the dataset.

(b) Comparison of the original 3D dataset (blue) and the reconstructed points.

Fig. 49.11: PCA reduces the 3D dataset to a 2D subspace while preserving its elliptical shape.

49.2.6 Global vs. iterative principal components

In Section 49.2.2, we introduced an **iterative approach** for finding principal components. In this method, we determine one principal component at a time, starting with the first, and each new direction must be orthogonal to all previously selected directions.

More precisely, to find the first k principal components, $v_1, v_2, \ldots, v_k$, we solve the following sequence of optimization problems

$$\begin{aligned} v_1 &= \underset{\|v\|_2=1}{\operatorname{argmax}} \|(A - e\mu^T)v\|_2^2, \\ v_2 &= \underset{\substack{\|v\|_2=1 \\ v \perp v_1}}{\operatorname{argmax}} \|(A - e\mu^T)v\|_2^2, \\ &\vdots \\ v_k &= \underset{\substack{\|v\|_2=1 \\ v \perp v_1, v_2, \ldots, v_{k-1}}}{\operatorname{argmax}} \|(A - e\mu^T)v\|_2^2. \end{aligned} \tag{49.6}$$

At each step, we choose the direction that maximizes the spread of the projected data while remaining orthogonal to all previously chosen directions.

You might wonder why we find principal components one at a time. Why not determine them all at once? Instead of solving for each direction separately, could we compute the matrix $V_k = \begin{bmatrix} v_1 & \cdots & v_k \end{bmatrix}$ with orthonormal columns directly? Would that give a better reconstruction of the data?

Recall that if we have an orthonormal basis $v_1, \ldots, v_k$ for a subspace $\mathcal{V}$, we approximate each data point x_i with a vector in $\mathcal{V}$ using

$$x_i \approx \widetilde{x}_i = \mu + V_k z_i, \quad \text{where} \quad z_i = V_k^T (x_i - \mu).$$

These approximations maintain the same mean vector as the original data, because

$$\frac{1}{m} \sum_{i=1}^{m} \widetilde{x}_i = \frac{1}{m} \sum_{i=1}^{m} \left(\mu + V_k V_k^T (x_i - \mu) \right) = \mu + \frac{V_k V_k^T}{m} \left(\sum_{i=1}^{m} x_i - m\mu \right) = \mu.$$

To measure how spread out these approximations are around their mean, we use the average squared distance to μ:

$$\frac{1}{m} \sum_{i=1}^{m} \|\widetilde{x}_i - \mu\|_2^2 = \frac{1}{m} \sum_{i=1}^{m} \|V_k V_k^T (x_i - \mu)\|_2^2.$$

Our goal is to maximize this quantity. This leads to the optimization problem

$$\operatorname*{argmax}_{\substack{V_k \in \mathbb{R}^{n \times k} \\ V_k^T V_k = I_k}} \sum_{i=1}^{m} \|V_k V_k^T (x_i - \mu)\|_2^2. \tag{49.7}$$

We call this the **global approach** because it finds all k principal components $v_1, \ldots, v_k$ at once by directly computing the matrix V_k with orthonormal columns.

With some algebraic manipulation similar to what we did in Section 49.2.1, we can rewrite the optimization problem (49.7) as

$$\operatorname*{argmax}_{\substack{V_k \in \mathbb{R}^{n \times k} \\ V_k^T V_k = I_k}} \sum_{i=1}^{k} \|(A - e\mu^T) v_i\|_2^2. \tag{49.8}$$

Apart from the factor $1/m$, each term $\|(A - e\mu^T) v_i\|_2^2$ is the variance of the centered data projected onto the line spanned by the corresponding vector v_i. In other words, the global approach (49.8) seeks to maximize the total variance, that is, the sum of the variances along all k orthogonal directions.

Are the iterative and global approaches really different? In fact, they are not. Every solution from the iterative approach (49.6) also solves the global approach (49.8), as follows from Ky Fan's Theorem (Theorem 43.6). However, the reverse is not always true: the global approach may produce a matrix V_k whose columns do not individually satisfy the iterative optimization problems, though the total variance explained is the same.

49.2.7 The least reconstruction error

We have found the principal components by maximizing variance. There is another way to view PCA: instead of seeking the subspace that captures the most variance, we can look for the one that best approximates the data, in a sense we will define shortly. Interestingly, maximizing variance and minimizing reconstruction error lead to the same result. In this section, we explain why.

Recall that if we have an orthonormal basis $\{v_1, \ldots, v_k\}$ for a subspace $\mathcal{V}$, we approximate each data point x_i as

$$x_i \approx \widetilde{x_i} = \mu + V_k V_k^T (x_i - \mu),$$

where μ is the mean of the dataset $x_1, \ldots, x_m \in \mathbb{R}^n$ and $V_k = \begin{bmatrix} v_1 & \cdots & v_k \end{bmatrix}$.

How do we choose the subspace $\mathcal{V}$ or, equivalently, the matrix V_k with orthonormal columns? Previously, we selected V_k by maximizing the variance of the approximations $\widetilde{x_i}$. An alternative approach is to choose V_k so that the approximations $\widetilde{x_i}$ are as close as possible to the original data x_i. In other words, we can minimize the **total reconstruction squared error**:

$$\min_{\substack{V \in \mathbb{R}^{n \times k} \\ V^T V = I_k}} \sum_{i=1}^{m} \|x_i - \widetilde{x_i}\|_2^2.$$

But is this approach really different from maximizing variance? Not really. The reason comes down to the Pythagorean theorem. As shown in Figure 49.12, the original point x_i, its approximation $\widetilde{x_i}$, and the mean μ form a right triangle because of the properties of orthogonal projection. This means the vectors $x_i - \widetilde{x_i}$ and $\widetilde{x_i} - \mu$ are perpendicular, so by the Pythagorean theorem we have

$$\|x_i - \mu\|_2^2 = \|x_i - \widetilde{x_i}\|_2^2 + \|\widetilde{x_i} - \mu\|_2^2.$$

Summing over all data points yields

$$\sum_{i=1}^{m} \|x_i - \widetilde{x_i}\|_2^2 = \sum_{i=1}^{m} \left(\|x_i - \mu\|_2^2 - \|\widetilde{x_i} - \mu\|_2^2 \right).$$

Since $\sum_{i=1}^{m} \|x_i - \mu\|_2^2$ is fixed, minimizing the reconstruction error is equivalent to maximizing

$$\sum_{i=1}^{m} \|\widetilde{x_i} - \mu\|_2^2.$$

Apart from a constant factor, this is exactly the average squared distance from the mean, the same quantity that the global PCA approach maximizes.

In conclusion, whether we minimize the total reconstruction error or maximize the variance of the approximations, we obtain the same subspace $\mathcal{V}$, spanned by the top k principal components.

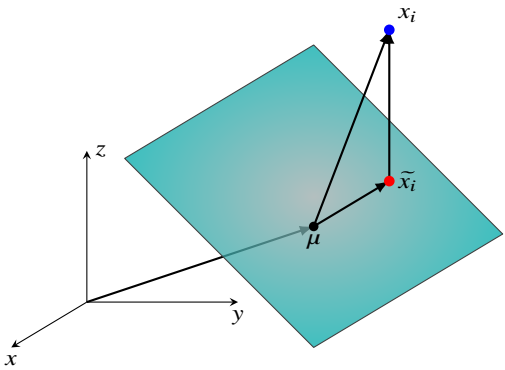


Fig. 49.12: Visualization of the reconstruction error in PCA: the original point x_i, its approximation $\widetilde{x}_i$, and the mean μ form a right triangle.

49.2.8 Practical applications of PCA

We will illustrate several practical uses of PCA. For our examples, we will use the MNIST dataset [37], which, as we have seen, consists of images of handwritten digits. For simplicity, we focus on a subset of the dataset that includes only the digits 3 and 9. Each image is $28 \times 28 = 784$ pixels, and we flatten each image into a 784-dimensional vector where each element represents the pixel intensity. These vectors (i.e., the handwritten digits) are then arranged as the rows of a data matrix A.

First, we apply PCA to our data matrix A. For different numbers k of principal components, we retain the first k components $\{v_1, \ldots, v_k\}$ and use them to reconstruct the original images (see Section 49.2.3 for details). After reconstruction, we reshape each vector back into its original 28×28 form.

Figure 49.13 shows the results for two digits: a nine (top) and a three (bottom). The left panel displays the original digit, and the panels to the right show the reconstructions for $k = 1, 2, 4, 8, 16, 32, 64, 128$, and 256 principal components.

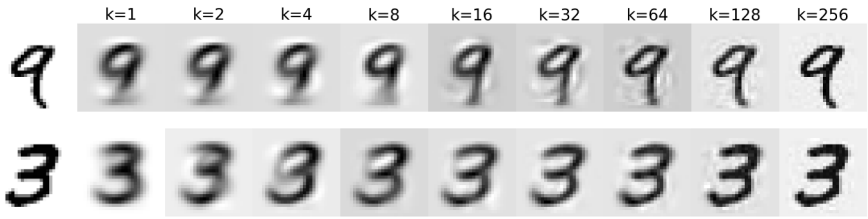


Fig. 49.13: PCA reconstruction of digits 9 (top) and 3 (bottom) using varying numbers of principal components.

With very few principal components, the reconstructions capture the overall shape of the digits but look very blurry. As k increases, more details emerge and the images

become clearer, with less quality loss. Even with just $k = 16$, the reconstructed image still closely resembles the original digit, despite minor imperfections like a slightly gray background or small distortions. This illustrates one of PCA's key applications: **data compression**. Once the mean image and the first 16 principal directions have been stored, each image can be represented by 16 PCA coordinates rather than 784 pixel values. The cost of storing the mean and the principal-component basis is shared across all images in the dataset.

Another application of PCA is **data visualization**. By using $k = 2$ (or sometimes $k = 3$) principal components, we can project our data into a lower-dimensional space that we can visualize. Figure 49.14 shows the result of applying PCA with two components to our dataset, with sample images overlaid to illustrate how the digits are arranged in this two-dimensional space.

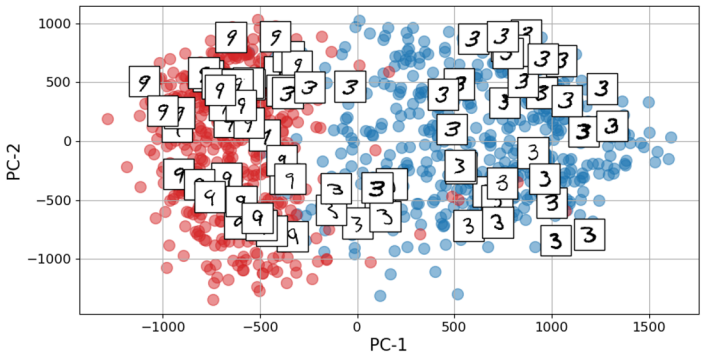


Fig. 49.14: Visualization of the dataset using PCA with two principal components.

Notice something interesting: in this two-dimensional space, the 9's and 3's are clearly separated, with most 9's clustering to the left and 3's to the right. This suggests that if our goal were to classify digits as either 3's or 9's, two principal components would be sufficient, perhaps even just one. In other words, we would not need all 784 pixel values; just two numbers would capture the essential differences. This highlights another key application of PCA: **reducing redundant information** for tasks like regression or classification.

We can further explore the principal component feature space by generating synthetic digits, new images that were not part of the original dataset. This process shows how the principal components capture the structure of the data.

Exploring the principal component space

After reducing the dataset to two principal components, we create a grid of points across this 2D space. Figure 49.15 shows this grid overlaid on the projections of the original data.

Most grid points do not correspond to actual digits in the original data. However, by applying the PCA reconstruction process, we can map these points back to

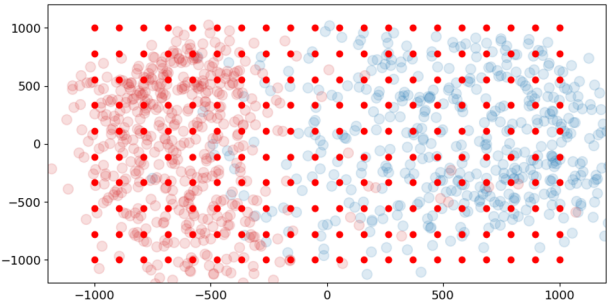


Fig. 49.15: A grid of points in the 2D principal component space, overlaid on the original data projections.

the original data space and generate new digit images. Figure 49.16 shows these reconstructed synthetic digits arranged according to their positions on the grid.

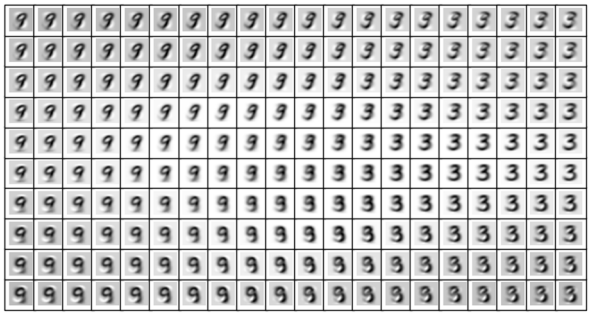


Fig. 49.16: Reconstructed synthetic digits arranged on the 2D PCA grid.

From Figure 49.16, we see that the first principal component (left-to-right) captures the "nineness" or "threeness" of the digits, while the second principal component (top-to-bottom) reflects the tilt, with digits at the top leaning right and those at the bottom leaning left.

49.2.9 Revisiting the weather data example

Let us revisit the monthly temperature time series from Chapter 45 (see Figure 45.3). Recall that this dataset contains monthly average temperatures for 37 US cities over 96 months (12 years). We previously suggested that this 96-dimensional dataset might be well-approximated by just two dimensions. Let us confirm this using PCA.

Figure 49.17 displays the 37 singular values in decreasing order. Notice that the first two singular values are much larger than the rest, which implies that most of the variance in the data is captured by just two principal components. The remaining singular values drop off sharply, contributing little to the overall variance.

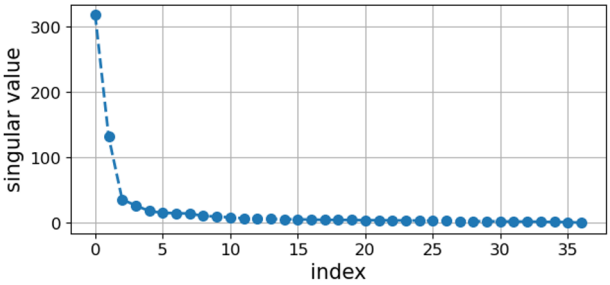


Fig. 49.17: Singular values of the monthly temperature dataset.

Figure 49.18 shows the data projected onto the subspace defined by the first two principal components.

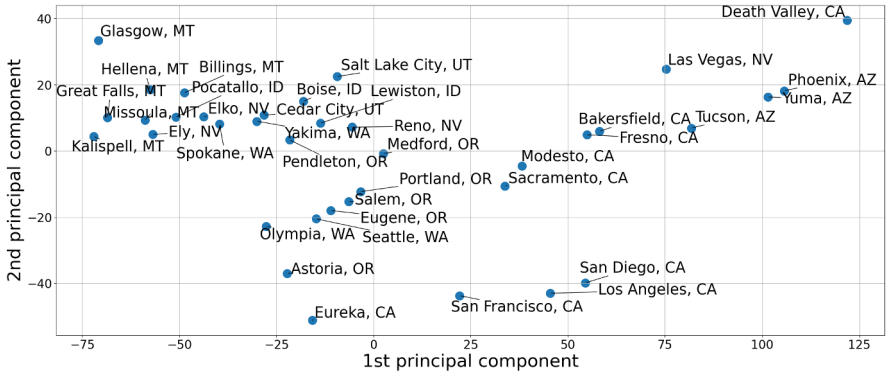


Fig. 49.18: Projection of the monthly temperature time series onto the first two principal components.

We can interpret these components as follows. The first principal component (horizontal axis) reflects the mean temperature–higher values correspond to higher average temperatures. For example, Figure 49.19 shows the temperature series for Kalispell (MT), Pendleton (OR), and Fresno (CA), which have first principal component values of approximately -72, -22, and 55, respectively, while their second principal component values are similar. A higher value along the first principal component corresponds to higher average temperatures.

The second principal component represents the amplitude of the temperature variations (i.e., the difference between the maximum and minimum temperatures), with higher values indicating greater variation. Figure 49.20 displays the temperature series for Eureka (CA), Seattle (WA), and Boise (ID). While these cities have similar first principal component values, their second principal component values differ markedly, at approximately -51 for Eureka, -20 for Seattle, and 15 for Boise. A larger second principal component indicates a greater temperature range.

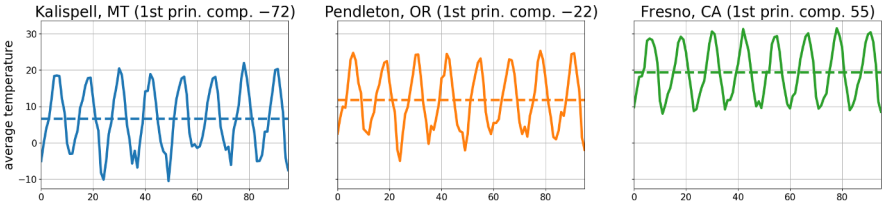


Fig. 49.19: Temperature time series for Kalispell (MT), Pendleton (OR), and Fresno (CA). The dashed line shows the mean temperature. Differences in the first principal component reflect variations in the mean temperature.

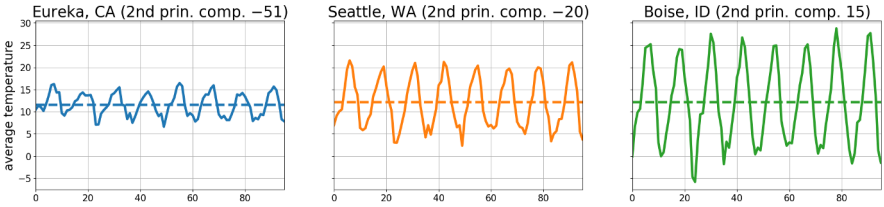


Fig. 49.20: Temperature time series for Eureka (CA), Seattle (WA), and Boise (ID). The dashed line shows the mean temperature. Variations in the second principal component reflect differences in temperature amplitude.

We can confirm these interpretations by examining the right singular vectors v_1 and v_2 in Figure 49.21. The entries of v_1 are nearly constant (around 0.1), so $v_1^T x_i$ is essentially a scaled average of the temperatures, consistent with its role in capturing the mean. In contrast, v_2 shows clear fluctuations, with peaks and troughs that align with seasonal extremes in the data, indicating that $v_2^T x_i$ captures temperature variation.

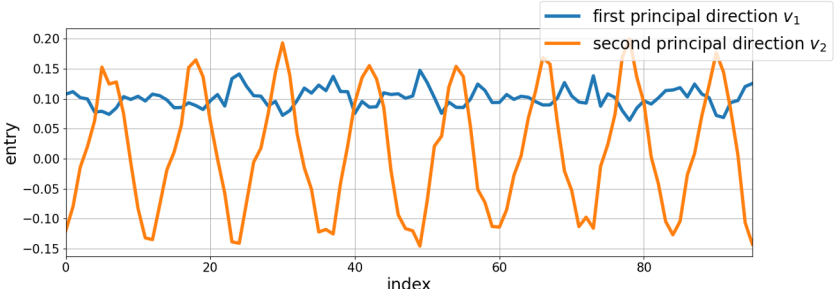


Fig. 49.21: Entries of the first two principal components v_1 and v_2 from the weather data example.

49.2.10 Principal component analysis vs. linear discriminant analysis

Some of the concepts discussed about PCA may remind you of our earlier discussion on linear discriminant analysis (LDA) in Chapter 44. Both PCA and LDA reduce dimensionality by projecting data into a lower-dimensional space, but they differ in their assumptions and objectives. Below, we summarize their key differences and when to use each method:

1. **Supervised vs. unsupervised:** PCA is an unsupervised method that treats the entire dataset as a single class, without using any label information. In contrast, LDA is supervised and relies on predefined class labels to guide the dimensionality reduction. In other words, PCA uses only the data matrix A, while LDA utilizes both A and the label vector y.
2. **Objective:** PCA seeks the directions of maximum variance in the data. LDA, on the other hand, aims to find the projection that best separates the different classes.

Because of these differences, PCA and LDA can produce very different projections, as the following example shows.

Example 49.14. (PCA vs. LDA)

Consider a toy dataset with two classes, as shown in Figure 49.22. Each class forms an elongated cluster of points with a small gap between them. We designed this dataset so that the direction of maximum variance (identified by PCA) differs from the direction that best separates the classes (identified by LDA).

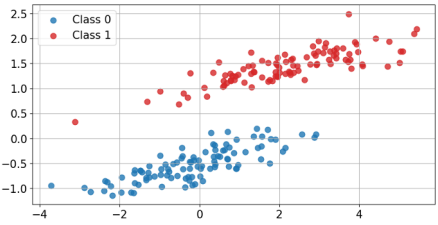


Fig. 49.22: A toy dataset with two elongated blobs, each representing a class.

Figure 49.23 shows the results of reducing the data to one dimension using both PCA and LDA. In the left panel, you will see the dataset with two overlaid directions: the dashed line represents the direction of maximum variance identified by PCA, while the dash-dotted line shows the direction that best separates the classes, as determined by LDA. Notice how these two directions are quite different. The right panel displays the projections: the top plot shows the PCA projection, where the classes are mixed together because PCA ignores the class labels, and the bottom plot shows the LDA projection, where the classes are clearly separated.

Figure 49.23 shows the outcomes of applying both PCA and LDA. The left panel displays the dataset along with the direction of maximum variance found by PCA (dashed line) and the direction maximizing class separation found by LDA (dash-dotted line). Notice that these directions are significantly different. The right panel

presents the projections: the top plot shows the PCA projection, where the two classes are mixed because PCA ignores the class labels; the bottom plot shows the LDA projection, where the classes are clearly separated.

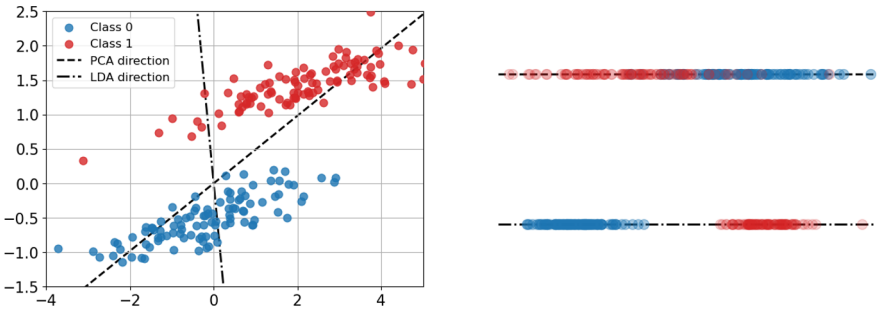


Fig. 49.23: Comparison of PCA and LDA on a toy dataset. Left: The dataset with the PCA direction (dashed line) and LDA direction (dash-dotted line). Right: Projections onto the PCA direction (top) and LDA direction (bottom), showing how PCA mixes the classes while LDA separates them.

Problems

49.1 (Fun with flags) Obtain grayscale images of several national flags (see examples in Figure 49.24). Identify which flags appear to be low rank, then create and display high-quality low-rank approximations for each. Finally, calculate the storage savings from these approximations and compare the results.

Fig. 49.24: Flags for experimenting with low-rank approximations.

49.2 Find a best rank-1 approximation to $A = \left[\begin{smallmatrix} 5 & -1 \\ 5 & 7 \end{smallmatrix}\right]$.

49.3 What is a best rank-k approximation of a diagonal matrix $A = \operatorname{diag}(a_1, a_2, \ldots, a_n)$?

49.4 (Approximating the column space of a matrix) In certain applications, given a matrix $A \in \mathbb{R}^{m \times n}$ and $k < \operatorname{rank}(A)$, the goal is to find a matrix $Q \in \mathbb{R}^{m \times k}$ with orthonormal columns such that

$$\operatorname{col}(A) \approx \operatorname{col}(Q).$$

One approach to finding such a matrix Q is to exploit the orthogonal projection property of QQ^T. The columns of QQ^TA are the orthogonal projections of the columns of A onto $\text{col}(Q)$. If $\text{col}(Q)$ provides a good approximation to $\text{col}(A)$, we would expect $A \approx QQ^TA$. Hence, one way to obtain an optimal matrix Q is by solving the optimization problem:

$$\min_{Q^TQ=I_k} \|A - QQ^TA\|_2. \tag{49.9}$$

To address this problem, consider the following:

(a) Show that $\min_{Q^TQ=I_k} \|A - QQ^TA\|_2 = \sigma_{k+1}$.
(b) Show that the minimum of (49.9) is attained by $Q = U_k = \begin{bmatrix} u_1 \cdots u_k \end{bmatrix}$, where $u_1, \ldots, u_k$ are the top k left singular vectors of A.

Hint: Consider the rank of QQ^TA.

49.5 Find the principal components of the data matrix

$$A = \begin{bmatrix} 1.1 & 0.9 & 1.0 & 0.0 & 0.0 \\ 2.1 & 2.2 & 1.9 & 0.0 & 0.1 \\ 1.9 & 1.9 & 2.0 & -0.2 & 0.1 \\ 0.2 & -0.1 & 0.0 & -1.0 & -1.0 \\ -0.1 & 0.0 & 0.0 & 1.1 & 0.9 \end{bmatrix}.$$

How many principal components are needed for a good approximation? How would you interpret these principal components?

49.6 Table 49.4 displays height, weight, and shoe-size measurements for a sample of students.

Data Points 1–10			Data Points 11–20		
Height	Weight	Shoe-size	Height	Weight	Shoe-size
68	160	12	72	176	12
66	215	9	75	173	12
75	183	12	69	183	10
71	175	11	70	169	11
67	188	12	65	185	10
65	201	8.5	64	210	8
67	167	9	68	177	9
75	171	13	73	203	11
72	151	11	70	200	8
74	193	12	76	186	12

Table 49.4: Individual measurements: first three columns for data points 1–10 and the next three for data points 11–20.

It is well known that height and shoe size are highly correlated. To study this relationship:

(a) Plot the data in 3D using height, weight, and shoe size for each student, and visually check whether the points lie close to a plane.
(b) Apply PCA to reduce the dimensionality from 3D to 2D.
(c) On your 3D plot, overlay the plane onto which the points are projected.

49.7 From the UCI Machine Learning Repository [19], download the *CDC Diabetes Health Indicators* dataset. This dataset includes healthcare statistics, lifestyle survey responses, and diabetes diagnoses.

Apply PCA to the dataset twice:

(a) without scaling the features,
(b) after scaling all features to the same scale.

Compare the results and discuss how scaling affects the principal components and the variance they capture.

Part XI
NONNEGATIVE MATRICES AND PERRON THEORY

In this part of the book, we introduce nonnegative matrices and the special case of positive matrices. We study their spectral properties through Perron-Frobenius theory and show how these ideas connect to real applications, such as Markov chains and the PageRank algorithm.

Chapter 50
XI-WHY SHOULD WE CARE?

50.1 Number of walks in a graph

As we explained in Chapter 7 and in several other sections, graphs are mathematical objects that represent relationships between elements of a set. These graphs can be either directed or undirected.

Example 50.1. (Graph of a social network)

Consider your social network on social media, that is, your group of "friends" along with the people in your friends' networks. We can represent this collection of people as a graph where each person is a node and an edge connects two nodes if those people are "friends". We assume that friendship is mutual; if person i is a friend of person j, then j is also a friend of i. Thus, the graph is undirected. See Figure 50.1 for an example of a graph representing a mock social network.

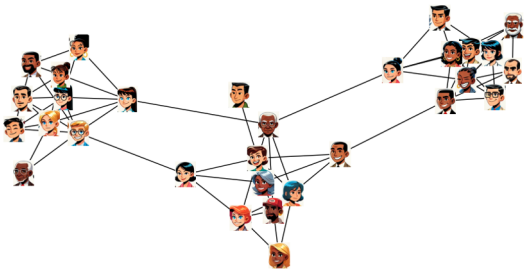


Fig. 50.1: Graph of a social network: nodes represent people, and edges represent friendships.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_50

Example 50.2. (Links in websites)

Consider the network of websites on the internet. We can represent their connections using a directed graph, where each node corresponds to a website, and there is a directed edge from node i to node j if website i contains a hyperlink to website j. Since links point from one website to another, the resulting graph is naturally directed. Figure 50.2 illustrates the link structure of one of the authors' personal website.

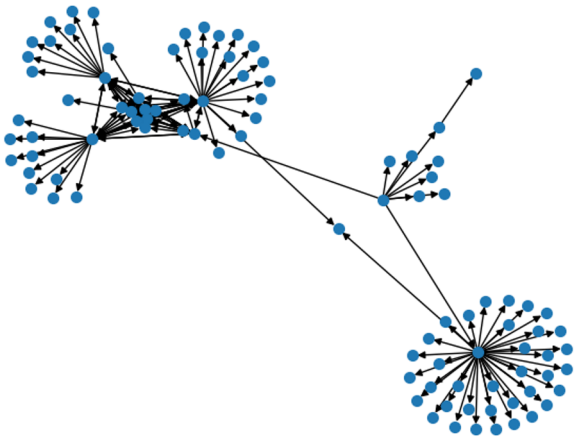


Fig. 50.2: Graph representing the link structure of the personal website of one of the authors: nodes represent websites, and directed edges indicate links from one site to another.

Recall that we can represent an unweighted graph using its adjacency matrix A. The entry $(A^k)_{ij}$ gives the number of walks of length k from node i to node j. More generally, for a weighted graph and the corresponding adjacency matrix A, $(A^k)_{ij}$ is the sum of the products of the edge weights along all walks of length k from node i to node j. Thus, matrix powers provide information about walks in both unweighted and weighted graphs. However, computing high powers of a matrix can be inefficient. Are there other ways to compute or estimate the number of walks?

Notice that every entry in an adjacency matrix A is nonnegative, and the same holds for all entries in its powers A^k. This part of the book focuses on matrices with this property, which we call **nonnegative matrices**. In Chapter 52, we will see that nonnegative matrices satisfying certain natural conditions always have a unique largest eigenvalue. This eigenvalue, along with its corresponding eigenvectors, plays a key role in estimating the total number of walks of a given length in a graph.

50.2 Page ranking for a web search engine

When you search the internet using a search engine like Google, the first step is to find all the web pages that contain the words in your query. But because the Web is so huge, the number of hits can be overwhelming. That is why we need a way to measure the "quality" of websites to rank the results and filter out the less useful ones.

Example 50.3 A Google search performed by the authors of this textbook with the query "university" returned top results such as the Wikipedia page for "University" and links to well-known universities like the University of California Santa Barbara (UCSB), University of Southern California, University of California Berkeley, University of California Los Angeles, University of Arizona, and University of California Davis. Notice that most of these are California institutions, and the search was done by a professor at UCSB. With an enormous number of pages potentially relevant to the query, it would have been nearly impossible to find useful information quickly without ranking the pages.

Google uses an algorithm to rank the web pages related to a query. The ranking is based on the Web's link structure. Google assigns a high rank to a page if it receives links from other high-ranking pages. This might seem a bit circular since the algorithm ranks pages based on rankings! Later in Chapter 54, we will show that this algorithm can be formulated mathematically as an eigenvalue equation for a particular nonnegative matrix.

50.3 Prediction of change of population

Imagine a region with $n \geqslant 2$ cities $C_1, \ldots, C_n$ where migration happens in a predictable way: every day at 7:00 am, a constant fraction a_{ij} of the population of city C_j moves to city C_i (for $i \neq j$), while the fraction a_{jj} of the population in city C_j remains there. For each source city C_j, these fractions must satisfy

$$\sum_{i=1}^{n} a_{ij} = 1.$$

This condition ensures that the total population is preserved.

Let $p_i^{(m)}$ be the population of city i on day m. Then, the population on the next day is given by

$$p_i^{(m+1)} = a_{i1}p_1^{(m)} + a_{i2}p_2^{(m)} + \cdots + a_{in}p_n^{(m)} \quad (i = 1, \ldots, n;\ m = 0, 1, 2, \ldots).$$

This equation means that on day $m + 1$, the population in city i comes from the portion of its own population that stayed plus the people who migrated from other cities. Notice that the term $a_{ij}p_j^{(m)}$ represents the fraction of people moving from city C_j to city C_i.

We can write this recurrence as a dot product:

$$p_i^{(m+1)} = \begin{bmatrix} a_{i1} \cdots a_{in} \end{bmatrix} \begin{bmatrix} p_1^{(m)} \\ \vdots \\ p_n^{(m)} \end{bmatrix}.$$

Defining the $n \times n$ **migration matrix** $A = [a_{ij}]$ and the **population vector**

$$p^{(m)} := \begin{bmatrix} p_1^{(m)} \cdots p_n^{(m)} \end{bmatrix}^T,$$

we obtain the matrix equation

$$p^{(m+1)} = Ap^{(m)} = A^{m+1}p^{(0)},$$

where $p^{(0)}$ is the initial population vector. Since the a_{ij} are fractions of people, every entry in A is nonnegative.

Government officials want to know how the population will be distributed far into the future to plan services and investments. Under suitable conditions on A, which we will study in later chapters, the sequence $(p^{(m)})$ converges, and its limit describes the long-term population distribution. The eigenvalues and eigenvectors of the nonnegative matrix A play an important role in determining whether this convergence occurs and in computing the limiting distribution.

We will explore the special properties of nonnegative matrices in this part of the book.

Chapter 51
XI-WHAT YOU MUST HAVE LEARNED BEFORE

51.1 Matrix norms and the submultiplicative property

Let us start by recalling the definition of a matrix norm. It has been a few chapters since we last saw this concept!

Definition 51.1. (Definition of a matrix norm)

For fixed positive integers m and n, a matrix norm $\|\cdot\|$ on $\mathbb{R}^{m\times n}$ is a function that assigns a real number to each matrix $A \in \mathbb{R}^{m\times n}$ and satisfies:

(a) **Positivity:** $\|A\| \geqslant 0$, and $\|A\| = 0$ if and only if A is the zero matrix.
(b) **Scalar multiplication:** $\|\alpha A\| = |\alpha|\|A\|$ for every scalar α.
(c) **Triangle inequality:** $\|A + B\| \leqslant \|A\| + \|B\|$.

Two common matrix norms

In this part of the book, we mainly use two matrix norms:

$$\text{The maximum absolute column sum:} \quad \|A\|_1 = \max_{1\leqslant j\leqslant n} \sum_{i=1}^{m} |A_{ij}|,$$

$$\text{The maximum absolute row sum:} \quad \|A\|_\infty = \max_{1\leqslant i\leqslant m} \sum_{j=1}^{n} |A_{ij}|,$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_51

These two norms share an important property.

Submultiplicative property

A matrix norm is submultiplicative if for any matrices A and B (with compatible dimensions for matrix multiplication),

$$\|AB\| \leqslant \|A\| \, \|B\|.$$

Both the $\|\cdot\|_1$ and $\|\cdot\|_\infty$ matrix norms satisfy the submultiplicative property.

If you would like to review matrix norms and their properties further, see Chapter 11 and Section 12.2.

51.2 Eigenvalues and eigenvectors

The main goal of this part of the book is to study the spectral properties of nonnegative matrices (matrices whose entries are all nonnegative). So let us review the definitions of eigenvalues and eigenvectors.

Definition 51.2. (Eigenvalues and eigenvectors)

An eigenvalue λ of a square matrix A is a scalar that satisfies

$$Ax = \lambda x,$$

for some nonzero vector $\mathbf{x}$. Such a vector is called an eigenvector corresponding to the eigenvalue λ.

Recall that for a real matrix, some or all of its eigenvalues (and corresponding eigenvectors) may be complex. However, if the matrix is symmetric, all its eigenvalues are guaranteed to be real.

Some transformations of a matrix produce new matrices that share the same eigenvalues.

Proposition 51.3. (Similar matrices)

If A and B are similar, i.e., $B = P^{-1}AP$ for some invertible matrix P, then A and B have the same eigenvalues.

Proposition 51.4. (Eigenvalues of the transpose)

The matrices A and A^T have the same eigenvalues, although their eigenvectors may be different.

As we have seen, in general, it is not easy to compute the eigenvalues of a matrix. However, having a good estimate of their magnitudes, or even a rough idea of where they are located, can be very useful in many applications.

The following result shows that the norm of A is an upper bound on how far its eigenvalues can lie from the origin.

Proposition 51.5. (Relation between eigenvalues and matrix norms)

For any eigenvalue λ of a square matrix A, and any submultiplicative matrix norm $\|\cdot\|$, we have

$$|\lambda| \leqslant \|A\|.$$

Proof Let x be an eigenvector of A with eigenvalue λ. From $Ax = \lambda x$, we get

$$\|A\|\|x\| \geqslant \|Ax\| = \|\lambda x\| = |\lambda|\,\|x\|.$$

Because $\|x\| \neq 0$, dividing both sides by $\|x\|$ yields $|\lambda| \leqslant \|A\|$. □

Gershgorin's circle theorem is another useful tool for estimating the location of a matrix's eigenvalues in the complex plane.

Theorem 51.6. (Gershgorin's circle theorem)

Every eigenvalue λ of an $n \times n$ complex matrix $A = [A_{ij}]$ lies within at least one of the Gershgorin disks defined by

$$\mathcal{G}_i(A) := \{z \in \mathbb{C} \mid |z - A_{ii}| \leqslant R_i\}, \quad \text{where } R_i = \sum_{j \neq i} |A_{ij}|.$$

Each disk $\mathcal{G}_i(A)$ is centered at the diagonal entry A_{ii} and has a radius equal to the sum of the modulus of the off-diagonal entries in that row.

If you would like to review more about eigenvalues and eigenvectors, see Chapter 36 and Chapter 37. For additional details on Gershgorin's Circle Theorem, refer to Section 38.1.

51.3 The Jordan canonical form

Since similar matrices share the same eigenvalues, it is often useful to find a matrix B similar to a given matrix A that has a structure revealing the spectral properties of A.

Proposition 51.7. (The Jordan canonical form)

Every square matrix A with complex entries is similar to a block-diagonal matrix called its Jordan canonical form. That is, A is similar to a matrix of the form

$$J = \begin{bmatrix} J_{m_1}(\lambda_1) & 0 & \cdots & 0 \\ 0 & J_{m_2}(\lambda_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & J_{m_k}(\lambda_k) \end{bmatrix},$$

where each main diagonal block, called a *Jordan block*, is given by

$$J_{m_i}(\lambda_i) = \begin{bmatrix} \lambda_i & 1 & 0 & \cdots & 0 \\ 0 & \lambda_i & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \ddots & 1 \\ 0 & 0 & 0 & \cdots & \lambda_i \end{bmatrix} \in \mathbb{C}^{m_i \times m_i}.$$

A matrix is diagonalizable precisely when all its Jordan blocks are 1×1.

Recall that the algebraic multiplicity of an eigenvalue is its multiplicity as a root of the characteristic polynomial, while the geometric multiplicity is the dimension of its eigenspace. The Jordan canonical form makes these multiplicities transparent: the algebraic multiplicity equals the sum of the sizes of all Jordan blocks for that eigenvalue, and the geometric multiplicity equals the number of such blocks.

Example 51.8 Consider the following matrix in Jordan canonical form:

$$J = \begin{bmatrix} J_2(0) & 0 & 0 \\ 0 & J_2(5) & 0 \\ 0 & 0 & J_1(5) \end{bmatrix} = \left[\begin{array}{cc|cc|c} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 5 & 1 & 0 \\ 0 & 0 & 0 & 5 & 0 \\ \hline 0 & 0 & 0 & 0 & 5 \end{array}\right].$$

There are three Jordan blocks in total. For the eigenvalue 0, there is one 2×2 Jordan block $J_2(0)$, so the algebraic multiplicity of 0 is 2 (the block size) and its geometric multiplicity is 1 (the number of blocks). For the eigenvalue 5, there are two Jordan blocks: a 2×2 block $J_2(5)$ and a 1×1 block $J_1(5)$. Thus, the algebraic multiplicity of 5 is $2 + 1 = 3$, while its geometric multiplicity is 2.

If you would like to review more about the Jordan canonical form and its related concepts, please see Section 36.3.

51.4 Graphs and the adjacency matrix

One important application of the spectral properties of nonnegative matrices is in the study of discrete-time Markov chains, which we will introduce in Chapter 52. As we will learn, these chains can be naturally represented using graphs. Let us review graphs and their adjacency matrices.

Definition 51.9. (Graph and adjacency matrix)

A graph G is defined by a set of vertices (or nodes) and a set of edges (connections between nodes). The adjacency matrix A_G of a weighted graph with n vertices is an $n \times n$ matrix where the entry $(A_G)ij$ is nonzero (1 for an unweighted graph) if there is an edge from vertex i to vertex j, and zero otherwise.

In our applications, the weighted graphs only use positive weights. Thus, the corresponding adjacency matrix has nonnegative entries.

For undirected graphs, the adjacency matrix is symmetric, which implies all its eigenvalues are real.

If you would like to review more about graphs and their associated matrices, see Section 3.4.

51.5 Limit of a sequence of matrices

What does it mean for a sequence of matrices to converge, and how can we tell if such a limit exists?

Definition 51.10. (Convergent sequence of matrices)

Let $A_1, A_2, \ldots$ and L be $n \times p$ real matrices. We say (A_m) **converges to** L (or has limit L) if, for every i and j,

$$\lim_{m\to\infty} (A_m)_{ij} = L_{ij}.$$

In other words, each entry of A_m converges to the corresponding entry of L. When this holds, we write

$$\lim_{m\to\infty} A_m = L.$$

A very interesting and useful sequence of matrices is formed by the powers of a matrix.

Theorem 51.11. (Convergent sequences of powers of a matrix)

Let A be a real square matrix. The limit $\lim_{m\to\infty} A^m$ exists if and only if one of the following holds:

(a) Every eigenvalue λ of A satisfies $|\lambda| < 1$; or
(b) The eigenvalue 1 is dominant and regular, that is, 1 is an eigenvalue, $|\lambda| < 1$ for every other eigenvalue, and 1 has equal algebraic and geometric multiplicities.

Let us see two matrices that break the conditions in Theorem 51.11, and whose powers A^m therefore fail to converge.

Example 51.12 Consider the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix},$$

a single Jordan block. The eigenvalue 1 has algebraic multiplicity 2 but geometric multiplicity 1, so 1 is not regular. In fact,

$$A^m = \begin{bmatrix} 1 & m \\ 0 & 1 \end{bmatrix},$$

which grows without bound. Hence $\lim_{m\to\infty} A^m$ does not exist.

Chapter 52
XI-CORE TOPICS

52.1 Nonnegative matrices: definition and properties

In this chapter, we will focus on nonnegative matrices. Let us begin by introducing their definition.

Definition 52.1. (Nonnegative and positive matrices)

An $m \times n$ real matrix A is called **nonnegative** if $A_{ij} \geqslant 0$ for all i, j. We denote that A is a nonnegative matrix by $A \geqslant 0$. Similarly, A is called **positive** if $A_{ij} > 0$ for all i, j. We denote that A is a positive matrix by $A > 0$.

Example 52.2 Consider the following matrices:

$$A = \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 3 \\ 1 & 1 \end{bmatrix}, \quad \text{and} \quad C = \begin{bmatrix} 2 & 3 \\ 1 & -1 \end{bmatrix}.$$

The matrix A is nonnegative (all entries are $\geqslant 0$) but not positive (because of the 0 in the bottom-left corner). The matrix B is both nonnegative and positive (all entries are > 0). The matrix C is neither nonnegative nor positive (because of the -1 in the bottom-right corner).

Remark 52.3. (Nonnegative and nonzero does not always imply positive)

For real numbers, if a number is nonnegative and not zero, it must be positive. But for matrices (bigger than 1×1), this is not always true. For example, the matrix A above is nonnegative and nonzero, but it is not positive because of the zero entry.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_52

There are some operations that preserve the nonnegativity of matrices.

Theorem 52.4. (Operations with nonnegative matrices)

Let $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{p\times q}$ be two nonnegative matrices, and let $\alpha \in \mathbb{R}$.

(a) If $\alpha \geqslant 0$, then $\alpha A \geqslant 0$.
(b) If $m = p$ and $n = q$, then $A + B \geqslant 0$.
(c) If $n = p$, then $AB \geqslant 0$.

Proof We will prove part (c). The other parts can be shown in a similar way.

Assume $n = p$. We need to show that $(AB)_{ij} \geqslant 0$ for all i, j when A and B are nonnegative. By the definition of matrix multiplication,

$$(AB)_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}.$$

Since A and B are nonnegative, $A_{ik} \geqslant 0$ and $B_{kj} \geqslant 0$ for all i, j, k. This implies each term $A_{ik}B_{kj}$ is also nonnegative (because the product of two nonnegative numbers is nonnegative). Adding up nonnegative numbers gives a nonnegative result, so $(AB)_{ij} \geqslant 0$, which completes the proof. □

If we combine parts (a) and (b), we get that any linear combination $\alpha A + \beta B$ of nonnegative matrices A and B, with $\alpha, \beta \geqslant 0$, is also nonnegative. Similarly, part (c) implies that any power A^k of a nonnegative matrix A stays nonnegative.

The same results hold if we replace nonnegative matrices and scalars with positive matrices and scalars.

We have seen how multiplying two nonnegative (or positive) matrices preserves nonnegativity (or positivity). Now, let us look at what happens when one of the matrices is a vector. The following proposition gives more specific results for this case.

Theorem 52.5. (Product of a matrix and a vector)

Let $A \in \mathbb{R}^{m\times n}$ and $x \in \mathbb{R}^n$.

(a) If $A > 0$, $x \geqslant 0$, and $x \neq 0$, then $Ax > 0$.
(b) If $A \geqslant 0$, $x > 0$, and $Ax = 0$, then $A = 0$.

Proof Let us start with part (a). Since $x \geqslant 0$ and $x \neq 0$, at least one entry of x is positive. Let us say the ℓth entry, x_ℓ, is positive, so $x_\ell > 0$. Then, for each row i of A, we have

$$(Ax)_i = \sum_{j=1}^{n} A_{ij}x_j \geqslant A_{i\ell}x_\ell > 0.$$

The first inequality holds because $A_{ij}x_j \geqslant 0$ for all i, j. The second inequality holds because $A_{ij} > 0$ for all i, j, and $x_\ell > 0$. This shows that $Ax > 0$.

Now, let us prove part (b). Assume $A \geqslant 0$, $x > 0$, and $Ax = 0$. For each row i, we have

$$0 = (Ax)_i = \sum_{j=1}^{n} A_{ij}x_j.$$

Since $A_{ij} \geqslant 0$ and $x_j > 0$ for all j, the only way this sum can be zero is if $A_{ij} = 0$ for all j. This holds for every row i, so A must be the zero matrix. □

52.1.1 Order in the set of matrices

On the real number line, the existence of nonnegative numbers allows the definition of the following order:

$$a \geqslant b \quad \text{if and only if} \quad a - b \geqslant 0.$$

We can extend this idea to matrices using the concept of nonnegative matrices.

Definition 52.6. (Order in the set of matrices)

Let A and B be two matrices of the same size. We say $A \geqslant B$ if $A_{ij} \geqslant B_{ij}$ for all i, j, or equivalently, if $A - B \geqslant 0$. Similarly, $A > B$ means $A_{ij} > B_{ij}$ for all i, j, or $A - B > 0$.

Example 52.7 Consider the matrices:

$$A = \begin{bmatrix} 2 & 3 \\ 0 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 3 \\ 0 & 3 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 3 \\ 1 & 3 \end{bmatrix}.$$

We have $A \geqslant B$ because

$$A - B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \geqslant 0.$$

However, neither $A \geqslant C$ nor $C \geqslant A$ holds. To see why, we compute

$$A - C = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \quad \text{and} \quad C - A = \begin{bmatrix} -1 & 0 \\ 1 & -1 \end{bmatrix}.$$

Both $A - C$ and $C - A$ have negative entries, so neither matrix is greater than or equal to the other.

The relation $\geqslant$ defined on the set of $m \times n$ matrices is an **order** because it satisfies the reflexive, antisymmetric, and transitive properties. These are generalizations of the basic properties of the usual order for real numbers, and we will prove them next. However, as shown in the previous example, this is not a **total order** because some pairs of matrices cannot be compared under this relation.

Theorem 52.8. (Properties of the ordering of matrices)

Let $A, B, C \in \mathbb{R}^{m\times n}$. Then,

(a) $A \geqslant A$ (reflexive property);
(b) If $A \geqslant B$ and $B \geqslant A$, then $A = B$ (antisymmetric property);
(c) If $A \geqslant B$ and $B \geqslant C$, then $A \geqslant C$ (transitive property).

Proof By Definition 52.6, $A \geqslant A$ if and only if $A - A = 0 \geqslant 0$, which is clearly true.

Assume $A \geqslant B$ and $B \geqslant A$. Then, for all $i \in \{1, \ldots, m\}$ and $j \in \{1, \ldots, n\}$, we have

$$A_{ij} \geqslant B_{ij} \quad \text{and} \quad B_{ij} \geqslant A_{ij}.$$

Since the order $\geqslant$ in $\mathbb{R}$ is antisymmetric, this implies $A_{ij} = B_{ij}$ for all i, j. Therefore, $A = B$.

The transitive property can be proven similarly by relying on the transitivity of the order for real numbers. □

Next, we will prove conditions under which the order is preserved by matrix addition and multiplication. These properties are similar to the familiar properties of the standard order on real numbers.

Theorem 52.9. (Operations that preserve the order)

Assume that A, B, C, D are matrices of appropriate size.

(a) If $A \geqslant B$ and $C \geqslant D$, then $A + C \geqslant B + D$.
(b) If $0 \leqslant A \leqslant B$ and $0 \leqslant C \leqslant D$, then $0 \leqslant AC \leqslant BD$.
(c) If $0 \leqslant A \leqslant B$, then $0 \leqslant A^m \leqslant B^m$ for all $m = 1, 2, \ldots$.

Proof For part (a), assume $A \geqslant B$ and $C \geqslant D$. By definition, we have $A - B \geqslant 0$ and $C - D \geqslant 0$. By Theorem 52.4, part (b), the sum of nonnegative matrices is nonnegative, so

$$(A - B) + (C - D) = (A + C) - (B + D) \geqslant 0.$$

This implies $A + C \geqslant B + D$.

For part (b), assume $0 \leqslant A \leqslant B$ and $0 \leqslant C \leqslant D$. We need to show $AC \geqslant 0$ and $AC \leqslant BD$. First, $AC \geqslant 0$ holds because the product of nonnegative matrices is nonnegative (Theorem 52.4, part (c)). Next, to show $AC \leqslant BD$, note that for each entry $(AC)_{ij}$, we have

$$(AC)_{ij} = \sum_{k=1}^{n} A_{ik}C_{kj} \leqslant \sum_{k=1}^{n} B_{ik}D_{kj} = (BD)_{ij}.$$

The inequality follows because, for real numbers, if $0 \leqslant a \leqslant b$ and $0 \leqslant c \leqslant d$, then $ac \leqslant bd$.

To prove part (c), we will use induction on m. We want to show that if $0 \leqslant A \leqslant B$, then $0 \leqslant A^m \leqslant B^m$ for all $m = 1, 2, \ldots$. For $m = 1$, the result holds by assumption. Now, assume $0 \leqslant A^m \leqslant B^m$ for some m. Since $0 \leqslant A \leqslant B$, applying part 2 gives $0 \leqslant A^{m+1} \leqslant B^{m+1}$. This completes the induction. □

We will finish this section with a property of positive matrices.

Proposition 52.10. (Product by a positive matrix preserves order)

Let $A \in \mathbb{R}^{n\times n}$ be a positive matrix, and let $u, v \in \mathbb{R}^n$ satisfy $u \geqslant v$ and $u \neq v$. Then, $Au > Av$.

Proof By Definition 52.6, showing $Au > Av$ is equivalent to showing $A(u-v) > 0$. For each $i = 1, 2, \ldots, n$, we have

$$(A(u - v))_i = \sum_{j=1}^{n} A_{ij}(u_j - v_j).$$

Since $A > 0$, all entries A_{ij} are positive. Also, $u_j - v_j \geqslant 0$ for all j, and since $u \neq v$, at least one $u_j - v_j$ is positive. Therefore,

$$(A(u - v))_i \geqslant \min\{A_{ij}\} \sum_{j=1}^{n} (u_j - v_j) > 0.$$

This shows that $A(u - v) > 0$, which proves the result. □

Example 52.11 Consider the matrix and vectors

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 2 \\ 2 & 1 & 2 \end{bmatrix}, \quad u = \begin{bmatrix} 3 \\ 1 \\ 2 \end{bmatrix}, \quad \text{and} \quad v = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}.$$

These satisfy the assumptions of Proposition 52.10. Computing the products, we get

$$Au = \begin{bmatrix} 11 \\ 8 \\ 11 \end{bmatrix} \quad \text{and} \quad Av = \begin{bmatrix} 7 \\ 5 \\ 7 \end{bmatrix}.$$

Clearly, $Au > Av$.

52.2 The spectral radius

The spectral radius plays a central role in the spectral theory of positive and nonnegative matrices.

Definition 52.12. (Spectral Radius)

Let $A \in \mathbb{R}^{n\times n}$. The **spectral radius** of A, denoted $\rho(A)$, is defined as

$$\rho(A) := \max\{|\lambda| \mid \lambda \in \sigma(A)\},$$

where $\sigma(A)$ is the set of all eigenvalues of A.

Example 52.13 Consider the matrix

$$A = \begin{bmatrix} -1 & 2 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & -2 \end{bmatrix}.$$

Since A is upper triangular, its eigenvalues are the diagonal entries. Thus, $\sigma(A) = \{-1, 2, -2\}$. The spectral radius is the largest absolute value of these eigenvalues, so

$$\rho(A) = \max\{|-1|, |2|, |-2|\} = 2.$$

The spectral radius is always a nonnegative number, even if the matrix has complex or imaginary eigenvalues, because it is defined as the maximum of the moduli of the eigenvalues. A nonzero matrix can still have a spectral radius of zero. In fact, the spectral radius is zero if and only if all the eigenvalues are zero.

In general, the spectral radius $\rho(A)$ is not necessarily an eigenvalue of A (though we will see later that this changes for positive and certain nonnegative matrices). In fact, the spectral radius is an eigenvalue of A if and only if the eigenvalue with the largest modulus is a nonnegative real number.

Geometrically, $\rho(A)$ is the radius of the smallest circle centered at the origin, called the **spectral circle**, that contains all of A's eigenvalues. Hence, the spectral radius is an upper bound on how far any eigenvalue can be from the origin, and twice this value is an upper bound on the distance between any two eigenvalues.

Example 52.14 In Figure 52.1, we have plotted the eigenvalues and the spectral circle of a 100×100 real matrix. In this example, the eigenvalues are uniformly distributed inside the spectral circle, but this is not the case in general.

In general, computing the spectral radius of a matrix is computationally expensive–remember, there is no simple formula for finding all eigenvalues of matrices larger than 4×4. A well-known result, called **Gelfand's formula**, provides a way to compute the spectral radius using the powers of the matrix. It states that for any matrix norm,

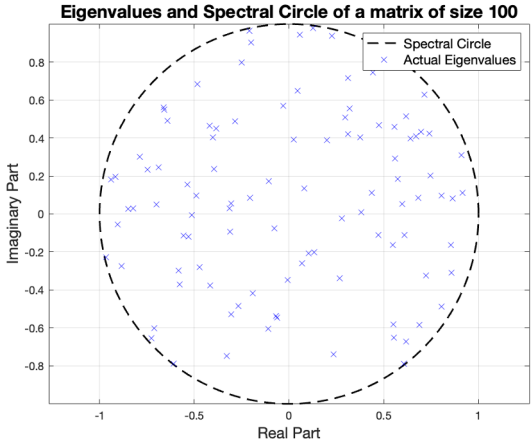


Fig. 52.1: The figure displays the eigenvalues of a real matrix of size 100 together with its spectral circle.

$$\rho(A) = \lim_{k\to\infty} (\|A^k\|)^{1/k}. \tag{52.1}$$

This means that, for large enough k, $\|A^k\|^{1/k}$ is a good approximation of $\rho(A)$. However, using this formula directly, which requires computing A^k for a full matrix, is an expensive approach. For this reason, it is often more useful to find upper bounds on the spectral radius that are easier to compute.

The spectral radius is always bounded above by any submultiplicative matrix norm.

Proposition 52.15. (Upper bound on spectral radius)

Let $A \in \mathbb{R}^{n\times n}$. For any submultiplicative matrix norm $\|\cdot\|$, we have

$$\rho(A) \leqslant \|A\|.$$

Proof Let λ be an eigenvalue of A. By Proposition 51.5, we know that $|\lambda| \leqslant \|A\|$. Taking the maximum of $|\lambda|$ over all eigenvalues of A gives $\rho(A) \leqslant \|A\|$, which proves the result. □

The 1-norm and infinity-norm provide easy-to-compute upper bounds for the spectral radius.

Proposition 52.16. (Some formulas for upper bound on spectral radius)

Let $A \in \mathbb{R}^{n\times n}$, and let

$$r_i := \sum_{j=1}^{n} |A_{ij}| \quad \text{and} \quad c_j := \sum_{i=1}^{n} |A_{ij}|$$

denote the ith absolute row sum and the jth absolute column sum of A, respectively. Then,

$$\rho(A) \leqslant \min\{\|A\|_\infty, \|A\|_1\} = \min\left\{\max_i\{r_i\}, \max_j\{c_j\}\right\}.$$

Proof By Proposition 52.15, the spectral radius is bounded above by both the 1-norm and the infinity norm of A. This gives the result. □

Example 52.17 Consider the matrix

$$A = \begin{bmatrix} 1 & 0 & 2 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Let us compute its absolute row and column sums. The absolute row sums are $r_1 = 3$, $r_2 = 6$, and $r_3 = 3$. The absolute column sums are $c_1 = 4$, $c_2 = 4$, and $c_3 = 4$. By Proposition 52.16, we have

$$\rho(A) \leqslant \min\{6, 4\} = 4.$$

This means the spectral radius of A is at most 4. In fact, it can be confirmed that $\sigma(A) = \{4, 1, 0\}$. Thus, in this particular case, the spectral radius equals the bound.

Notice that, in the previous example, the absolute row sums are all the same. In fact, since the matrix is nonnegative, the absolute row sums are just row sums. For nonnegative matrices with constant row or column sums, the inequality in Proposition 52.16 becomes an equality.

Proposition 52.18. (Spectral radius of matrices with constant row or column sum)

Let $A \in \mathbb{R}^{n\times n}$ be a nonnegative matrix.

(a) If A has constant row sums, then $\rho(A) = \|A\|_\infty$.
(b) If A has constant column sums, then $\rho(A) = \|A\|_1$.

Proof First, assume $A \geqslant 0$ has constant row sums. Then, the vector $x = \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix}^T$ is an eigenvector of A with eigenvalue $\|A\|_\infty$. By the definition of the spectral radius, $\|A\|_\infty \leqslant \rho(A)$. Combining this with Proposition 52.16, we get $\rho(A) = \|A\|_\infty$.

Next, assume A has constant column sums. In this case, A^T has constant row sums. Applying part (a) to A^T, we get $\|A\|_1 = \|A^T\|_\infty = \rho(A^T)$. Since A and A^T have

the same eigenvalues (and thus the same spectral radius), we have $\rho(A^T) = \rho(A)$. Therefore, $\|A\|_1 = \rho(A)$. □

Example 52.19 Consider the matrix

$$A = \begin{bmatrix} 5 & 1 \\ 2 & 4 \end{bmatrix} \geqslant 0.$$

This matrix has constant row sums equal to 6. You can check that the eigenvalues of A are 6 and 3. As expected from Proposition 52.18, the spectral radius is $\rho(A) = 6$.

Similar matrices share the same spectrum, so the spectral radius is preserved under similarity. We can use this fact to get more upper bounds on the spectral radius by applying a simple similarity transformation before using Proposition 52.16.

Proposition 52.20. (More upper bounds on spectral radius)

Let $A \in \mathbb{R}^{n\times n}$. For any positive vector $x \in \mathbb{R}^n$, we have

$$\rho(A) \leqslant \min\left\{ \max_i \left\{ x_i \sum_{j=1}^{n} \frac{|A_{ij}|}{x_j} \right\}, \max_j \left\{ \frac{1}{x_j} \sum_{i=1}^{n} |A_{ij}| x_i \right\} \right\}.$$

Proof Let x be a positive vector, and let P be the diagonal matrix with entries $x_1, x_2, \ldots, x_n$ on its main diagonal. Then, the (i, j)-entry of PAP^{-1} is

$$(PAP^{-1})_{ij} = x_i \frac{1}{x_j} A_{ij}.$$

Since A and PAP^{-1} have the same spectral radius (because they are similar matrices), the result follows by applying Proposition 52.16 to PAP^{-1}. □

Sometimes, we can use Proposition 52.20 to get better upper bounds on the spectral radius.

Example 52.21 Consider the matrix

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 2 & 4 & 5 \\ 0 & 1 & 3 \end{bmatrix}.$$

First, let us compute its row and column sums. The row sums are $r_1 = 4$, $r_2 = 11$, and $r_3 = 4$. The column sums are $c_1 = 3$, $c_2 = 5$, and $c_3 = 11$. By Proposition 52.16, we get $\rho(A) \leqslant 11$. Let us try to improve this bound using a diagonal similarity. If we choose

$$P = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 3 \end{bmatrix},$$

then

$$PAP^{-1} = \begin{bmatrix} 1 & 0 & 3 \\ \frac{2}{3} & 4 & \frac{5}{3} \\ 0 & 3 & 3 \end{bmatrix}.$$

Applying Proposition 52.16 again, we get

$$\rho(A) \leqslant \frac{19}{3} \approx 6.33.$$

This is a much better upper bound! In fact, the actual spectral radius is approximately $\rho(A) \approx 6.0379$.

For nonnegative matrices, we can also find easy-to-compute lower bounds on the spectral radius using row and column sums. But before we do that, we need the following result.

Proposition 52.22. (The spectral radius preserves order)

Let $A, B \in \mathbb{R}^{n\times n}$ and assume $0 \leqslant A \leqslant B$. Then, $0 \leqslant \rho(A) \leqslant \rho(B)$.

Proof Since $0 \leqslant A \leqslant B$, Theorem 52.9-(c) tells us that $0 \leqslant A^k \leqslant B^k$ for every $k = 1, 2, \ldots$. This implies $\|A^k\|_F \leqslant \|B^k\|_F$, where $\|\cdot\|_F$ is the Frobenius norm. Using Gelfand's formula (52.1), we get

$$\rho(A) = \lim_{k\to\infty} \|A^k\|_F^{1/k} \leqslant \lim_{k\to\infty} \|B^k\|_F^{1/k} = \rho(B).$$

This completes the proof. □

Now, let us return to finding lower bounds for the spectral radius of nonnegative matrices[1].

Proposition 52.23. (Bounds on spectral radius of nonnegative matrices)

Let $A \in \mathbb{R}^{n\times n}$ be a nonnegative matrix, and let

$$r_i := \sum_{j=1}^{n} A_{ij}, \quad c_j := \sum_{i=1}^{n} A_{ij}$$

denote the ith row sum and jth column sum of A. Then,

[1] The following two results are from [30], Chapter 8, which provides a detailed treatment of nonnegative matrices.

$$\max\left\{\min_i\{r_i\}, \min_j\{c_j\}\right\} \leqslant \rho(A) \leqslant \min\left\{\max_i\{r_i\}, \max_j\{c_j\}\right\}.$$

Proof We only need to prove the lower bound, since the upper bound was proven in Proposition 52.16 for arbitrary square matrices.

Let $\alpha := \min_i\{r_i\}$. We first prove that $\alpha \leqslant \rho(A)$.

If $\alpha = 0$, the result is immediate because $\rho(A) \geqslant 0$. Suppose, therefore, that $\alpha > 0$. Define $B = [B_{ij}]$ by

$$B_{ij} := \frac{\alpha}{r_i} A_{ij}.$$

Since $\alpha \leqslant r_i$, we have $0 \leqslant B_{ij} \leqslant A_{ij}$, and hence $0 \leqslant B \leqslant A$. Moreover, the ith row sum of B is

$$\sum_{j=1}^{n} B_{ij} = \frac{\alpha}{r_i} \sum_{j=1}^{n} A_{ij} = \alpha.$$

Thus, B has constant row sums equal to α. By Proposition 52.18, $\rho(B) = \alpha$. Since $0 \leqslant B \leqslant A$, Proposition 52.22 gives

$$\alpha = \rho(B) \leqslant \rho(A).$$

Applying the same argument to A^T, whose row sums are the column sums of A, gives

$$\min_j\{c_j\} \leqslant \rho(A^T) = \rho(A).$$

Therefore,

$$\max\left\{\min_i\{r_i\}, \min_j\{c_j\}\right\} \leqslant \rho(A).$$

Combining this inequality with the upper bound completes the proof. □

Example 52.24 Let us revisit the matrix A in Example 52.21. Using Proposition 52.23, we get

$$4 \leqslant \rho(A) \leqslant 11.$$

Similar to Proposition 52.20, we can use a diagonal similarity transformation on A to derive additional bounds on its spectral radius. We omit the proof since it follows along the same lines as that of Proposition 52.20.

Proposition 52.25. (More bounds on spectral radius of nonnegative matrices)

Let $A \in \mathbb{R}^{n\times n}$ be a nonnegative matrix. Then, for any positive vector $x \in \mathbb{R}^n$, the following bounds hold:

$$\min_i \left\{ x_i \sum_{j=1}^{n} \frac{A_{ij}}{x_j} \right\} \leqslant \rho(A) \leqslant \max_i \left\{ x_i \sum_{j=1}^{n} \frac{A_{ij}}{x_j} \right\},$$

and

$$\min_j \left\{ \frac{1}{x_j} \sum_{i=1}^{n} A_{ij} x_i \right\} \leqslant \rho(A) \leqslant \max_j \left\{ \frac{1}{x_j} \sum_{i=1}^{n} A_{ij} x_i \right\}.$$

Example 52.26 Consider the matrix A from Example 52.21. Earlier, using Proposition 52.16, we found the lower bound $4 \leqslant \rho(A)$. Now, let us try to improve this bound by applying a diagonal similarity transformation. If we choose $P = \operatorname{diag}(1, 3, 6)$, then

$$PAP^{-1} = \begin{bmatrix} 1 & 0 & \frac{1}{2} \\ 6 & 4 & \frac{5}{2} \\ 0 & 2 & 3 \end{bmatrix}.$$

Applying Proposition 52.16 to PAP^{-1}, we get the improved lower bound $6 \leqslant \rho(A)$. Combining this with the upper bound from Example 52.21, we conclude

$$6 \leqslant \rho(A) \leqslant \frac{19}{3} \approx 6.33.$$

This gives a very tight bound for the actual spectral radius $\rho(A) \approx 6.0379$.

We will end this section with a result that will be important for proving Perron's Theorem, the main result of this chapter. This result follows directly from Proposition 52.25[2].

Proposition 52.27. (Multiplying a nonnegative matrix by a positive vector)

Let $A \in \mathbb{R}^{n \times n}$ be a nonnegative matrix, and let $y \in \mathbb{R}^n$ be a positive vector. If $\alpha > 0$ and $\alpha y < Ay$, then $\alpha < \rho(A)$.

Proof Assume $\alpha > 0$ and $\alpha y < Ay$. Then, there exists $\alpha' > 0$ such that $(\alpha + \alpha')y \leqslant Ay$. Let $\widehat{\alpha} := \alpha + \alpha'$. For each $i \in \{1, 2, \ldots, n\}$, we have

$$\widehat{\alpha} y_i \leqslant \sum_{j=1}^{n} A_{ij} y_j,$$

where y_i is the ith component of y. Since $y > 0$, we can write $y_i = \frac{1}{x_i}$ for some $x_i > 0$. Substituting this into the inequality gives

[2] This result is also from [30]

$$\alpha < \widehat{\alpha} \leqslant x_i \sum_{j=1}^{n} \frac{A_{ij}}{x_j}, \quad i = 1, \ldots, n.$$

This implies

$$\alpha < \min_i \left\{ x_i \sum_{j=1}^{n} \frac{A_{ij}}{x_j} \right\} \leqslant \rho(A),$$

where the last inequality follows from Proposition 52.25. □

52.3 Spectral properties of square positive matrices

We begin with square positive matrices because the theory for this class of matrices is more elegant and straightforward. The main result we will discuss is Perron's Theorem, which captures the spectral properties of positive matrices.

We will use the 1-norm for vectors. Recall that for a vector $x \in \mathbb{R}^n$, its 1-norm is

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|.$$

Theorem 52.28. (Perron's Theorem)

Let $A \in \mathbb{R}^{n \times n}$ be a positive matrix. Then:

(a) $\rho(A) > 0$;
(b) $\rho(A)$ is an eigenvalue of A (called the **Perron root** of A);
(c) $\rho(A)$ is a simple eigenvalue of A, and hence its algebraic and geometric multiplicities are both 1;
(d) There exist positive eigenvectors of A associated with $\rho(A)$. Among these, there is a unique eigenvector p with $\|p\|_1 = 1$, called the **Perron vector**;
(e) Except for positive multiples of the Perron vector, A has no other nonnegative eigenvectors, no matter the eigenvalue;
(f) $\rho(A)$ is the only eigenvalue of A on its spectral circle. In other words, no other eigenvalue has a modulus equal to or larger than the Perron root.

The proof of Perron's Theorem is long and technical. Before presenting this proof, let us first illustrate the theorem with a couple of examples.

Example 52.29 Consider the positive matrix

$$A = \begin{bmatrix} 1 & 3 \\ 4 & 5 \end{bmatrix}.$$

Its characteristic polynomial is $\det(\lambda I_2 - A) = \lambda^2 - 6\lambda - 7 = (\lambda - 7)(\lambda + 1)$, so the eigenvalues are 7 and -1. This implies $\rho(A) = 7 > 0$, and 7 is the Perron root of A.

Notice that the eigenvalue 7 has algebraic multiplicity 1. Eigenvectors corresponding to 7 and -1 are of the form

$$\begin{bmatrix} a \\ 2a \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 3b \\ -2b \end{bmatrix},$$

respectively. Hence, the unique positive eigenvector for the eigenvalue 7, scaled so its 1-norm is one, is

$$\begin{bmatrix} 1/3 \\ 2/3 \end{bmatrix},$$

the Perron vector of A. No eigenvector corresponding to -1 is non-negative.

Example 52.30 Consider the positive matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

It is easy to verify that the eigenvalues of A, counting multiplicities, are $\{3, 0, 0\}$. Therefore, $\rho(A) = 3$; that is, the spectral radius of A is positive and is an eigenvalue of A. Moreover, this eigenvalue is simple, and a positive eigenvector with unit 1-norm is given by

$$u = \frac{1}{3}\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

This eigenvector is the Perron vector. The eigenspace associated with the zero eigenvalue is given by

$$\left\{ \begin{bmatrix} x \\ y \\ z \end{bmatrix} \mid x + y + z = 0 \right\} = \left\{ \begin{bmatrix} -y - z \\ y \\ z \end{bmatrix} \mid y, z \in \mathbb{R} \right\}.$$

Notice that if $y, z > 0$, then $-y - z < 0$; thus, no eigenvector associated with 0 is nonnegative.

Now we are ready to prove some parts of Perron's Theorem.

Proof (of Theorem 52.28) We will prove properties (a), (b), and (d).

First, by definition, $\rho(A) \geqslant 0$. Suppose, for contradiction, that $\rho(A) = 0$ for some positive matrix $A \in \mathbb{R}^{n \times n}$. This would mean that all eigenvalues of A are zero. Write $A = PJ_AP^{-1}$, where J_A is the Jordan canonical form of A. Since all eigenvalues are zero, J_A is an upper triangular matrix with zeros on its main diagonal, which implies that $J_A^k = 0$ for some positive integer k. Consequently, $A^k = PJ_A^kP^{-1} = 0$.

However, A^k is the product of positive matrices and must therefore be positive, a contradiction. Thus, we conclude that $\rho(A) > 0$.

Next, by the definition of spectral radius, there exists an eigenvalue λ of A such that $|\lambda| = \rho(A)$. Let u be an eigenvector corresponding to λ. We aim to show that $\rho(A)$ is an eigenvalue of A with a positive eigenvector.

Consider the vector $|u|$ obtained by taking the absolute value of each entry of u. Since A is positive, we have

$$A|u| = |A|\,|u| \geqslant |Au| = |\lambda u| = |\lambda|\,|u| = \rho(A)\,|u|,$$

where $|A|$ denotes the matrix whose entries are $|A_{ij}|$. Thus,

$$A|u| \geqslant \rho(A)\,|u|.$$

Our goal is to show that in fact $A|u| = \rho(A)\,|u|$. Suppose, for the sake of contradiction, that

$$A|u| > \rho(A)\,|u|.$$

Let $z := A|u|$. Since $|u|$ is nonzero and nonnegative, Theorem 52.5-(a) implies that $z > 0$. Then, by Proposition 52.10, we have

$$Az = A(A|u|) > \rho(A)(A|u|) = \rho(A)z.$$

Applying Proposition 52.27 then gives $\rho(A) > \rho(A)$, which is a contradiction. Therefore, we must have

$$A|u| = \rho(A)|u|.$$

We have shown that $\rho(A)$ is an eigenvalue of A with eigenvector $|u|$. To complete the proof, we must show that $|u| > 0$. Since A is positive, the vector $A|u|$ is positive, and therefore

$$|u| = \frac{A|u|}{\rho(A)}$$

is also positive. This shows that $\rho(A)$ is indeed an eigenvalue of A with a positive eigenvector.

At this point, we assume that part (c) has been established, that is, the Perron root has both algebraic and geometric multiplicity equal to one.

Finally, since $\rho(A)$ has geometric multiplicity one by part (c), the positive eigenvector corresponding to $\rho(A)$ is unique up to a scalar multiple. Normalizing this eigenvector with the 1-norm gives the unique Perron vector. □

52.4 Spectral properties of square nonnegative matrices

In this section, we relax the assumption that every entry of the matrix A is positive. Now, some entries may be zero while others remain positive, and we study how Perron's Theorem extends to these matrices.

One common way to generalize Perron's Theorem is by using a limiting argument. For example, we can add a small positive number ϵ to each zero entry of a nonnegative matrix, converting it into a positive matrix. After applying Perron's Theorem to this

perturbed matrix, we then take the limit as ϵ approaches zero to obtain the desired result for the original nonnegative matrix.

Theorem 52.31. (Perron's Theorem for nonnegative matrices)

Let $A \geqslant 0$ be an $n \times n$ matrix. Then:

(a) $\rho(A)$ is an eigenvalue of A (though $\rho(A) = 0$ is possible).
(b) A has a nonnegative eigenvector corresponding to the eigenvalue $\rho(A)$.

This is as far as Perron's Theorem can be generalized to nonnegative matrices without additional assumptions. While the spectral radius is still an eigenvalue of A, it might not be positive anymore, its algebraic multiplicity could be greater than one, and A might have other distinct eigenvalues with the same modulus. Additionally, we can guarantee the existence of a nonnegative eigenvector for $\rho(A)$, but we cannot guarantee any of the other eigenvector properties that hold for positive matrices.

The following examples illustrate how many of the properties in Perron's Theorem for positive matrices can fail for general nonnegative matrices.

Example 52.32 Consider the nonnegative matrix

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

The spectrum of A is $\sigma(A) = \{0\}$, so $\rho(A) = 0$. The spectral radius $\rho(A)$ has algebraic multiplicity 2, and any eigenvector associated with 0 has the form $\left[\begin{smallmatrix} x \\ 0 \end{smallmatrix}\right]$, which is not a positive vector.

Example 52.33 Now consider the nonnegative matrix

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

The spectrum of A is $\sigma(A) = \{1, -1\}$, so $\rho(A) = 1$. In this case, A has two distinct eigenvalues on the spectral circle.

Rather than accepting that nonnegative matrices had much more limited spectral properties than positive matrices, the German mathematician Georg Frobenius investigated further. He discovered that the limitations arise not simply from the presence of zeros, but from where those zeros are located in the matrix. Let us look at an example.

Example 52.34 Consider the nonnegative matrices

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

For A, we have $\sigma(A) = \{1, 1\}$. Thus, $\rho(A) = 1$ has algebraic multiplicity 2, and every eigenvector corresponding to 1 is of the form $\begin{bmatrix} 0 \\ y \end{bmatrix}$, which is not positive. In contrast, $\sigma(B) = \left\{(1 \pm \sqrt{5})/2\right\}$, so the spectral radius $\rho(B)$ has algebraic multiplicity 1, and there exist positive eigenvectors associated with this eigenvalue.

What makes these two nonnegative matrices so different? Frobenius discovered that the difference lies in a property called reducibility.

Definition 52.35. (Reducible and irreducible matrices)

An $n \times n$ matrix A is called **reducible** if there exists a permutation matrix P such that

$$P^T A P = \begin{bmatrix} X & Y \\ 0 & Z \end{bmatrix},$$

where X and Z are square matrices. The 1×1 zero matrix is also considered reducible. If no such permutation exists, A is called **irreducible**.

The matrix $P^T A P$ is called a **symmetric permutation** of A. This similarity transformation swaps rows and columns in the same way.

Remark 52.36. (Block lower triangular matrices are reducible)

Suppose there exists a permutation matrix P such that

$$P^T A P = \begin{bmatrix} X & 0 \\ Y & Z \end{bmatrix},$$

for some square matrices X and Z. Then the matrix A is *reducible*, since a simultaneous permutation of rows and columns can transform it into a block upper-triangular form.

To see this explicitly, let

$$P_2 = \begin{bmatrix} 0 & I \\ I & 0 \end{bmatrix},$$

which is itself a permutation matrix that swaps the block rows and columns. Then

$$P_2^T (P^T A P) P_2 = \begin{bmatrix} Z & Y \\ 0 & X \end{bmatrix}.$$

Hence,

$$(PP_2)^T A(PP_2) = \begin{bmatrix} Z & Y \\ 0 & X \end{bmatrix},$$

showing that A is similar to a block upper-triangular matrix via the permutation matrix PP_2. Since the product of permutation matrices is a permutation matrix, PP_2 is a valid permutation matrix, and thus A is reducible.

Example 52.37 Let us revisit Example 52.34. The matrix

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

is reducible because

$$P^T AP = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad \text{for} \quad P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

However, the matrix $B = \left[\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ is irreducible. To see why, note that there are only two permutation matrices of size 2, the identity matrix and the matrix P above, and neither will transform B into an upper block-triangular form.

Our examples suggest that irreducible nonnegative matrices have "better" spectral properties than reducible ones, much like positive matrices. Theorem 52.38 shows that irreducible nonnegative matrices retain most of the spectral properties of positive matrices.

Theorem 52.38. (Perron-Frobenius Theorem for irreducible nonnegative matrices)

Let $A \geqslant 0$ be an irreducible matrix. Then, the following properties hold:

(a) $\rho(A) > 0$.
(b) $\rho(A)$ is an eigenvalue of A.
(c) The algebraic multiplicity of $\rho(A)$ is 1.
(d) There exist positive eigenvectors corresponding to $\rho(A)$; moreover, there is a unique such eigenvector p normalized so that $\|p\|_1 = 1$, called the **Perron vector**.
(e) Aside from positive multiples of p, there are no other nonnegative eigenvectors of A, regardless of the eigenvalue.

The only property from Perron's Theorem for positive matrices that does not hold for irreducible nonnegative matrices is the spectral radius being the only eigenvalue on the spectral circle. However, for some irreducible nonnegative matrices, this property also holds. Let us give this special class of matrices a name.

Definition 52.39. (Primitive matrix)

A nonnegative matrix is said to be **primitive** if it is irreducible and it has exactly one eigenvalue on its spectral circle.

Example 52.40 Consider the nonnegative irreducible matrices

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

The spectra of these matrices are

$$\sigma(A) = \left\{ \frac{1 \pm \sqrt{5}}{2} \right\} \quad \text{and} \quad \sigma(B) = \left\{ 0, -\sqrt{2}, \sqrt{2} \right\}.$$

Since A has only one eigenvalue on the spectral circle and is irreducible (as shown in Example 52.37), $(1 + \sqrt{5})/2)$, it is primitive. However, B is not primitive because both $-\sqrt{2}$ and $\sqrt{2}$ lie on its spectral circle.

So, irreducibility and primitivity allow us to extend the spectral properties of positive matrices to nonnegative matrices. This makes it important to determine when a matrix is irreducible and primitive. Unfortunately, the definition of irreducibility is phrased negatively: "there is *no* permutation matrix that turns A into an upper block-triangular form", which in principle means testing all $n!$ permutation matrices for an $n \times n$ matrix, a hopeless task for even moderate sizes.

To avoid that brute-force approach, the next section presents several equivalent, computationally friendly ways to recognise irreducible and primitive non-negative matrices, using graph connectivity, matrix powers, and other easily checked criteria.

52.4.1 Characterization of irreducible and primitive nonnegative matrices

In this section, we introduce several ways to characterize irreducible matrices. One of these involves a directed graph constructed from the matrix.

There is a correspondence between nonnegative matrices and weighted directed graphs. Given a positively weighted directed graph $G = (V, E, w)$, we can form a nonnegative matrix (its adjacency matrix) that encodes the graph's structure. Conversely, if we have an $n \times n$ nonnegative matrix A, we can build a positively weighted directed graph $G_A = (V, E, w)$ where the node set is $V = \{1, 2, \ldots, n\}$. In this graph, there is a directed edge from node i to node j if and only if $A_{ij} \neq 0$, and the edge is given a weight equal to A_{ij}. The resulting graph is called the **directed graph (or digraph) associated with** A. In this graph, we allow self-loops, which appear when a diagonal entry of A is nonzero.

Example 52.41 In Figure 52.2, we show a nonnegative matrix along with its associated digraph. Each edge in the digraph is labeled with the corresponding matrix entry. For example, since the entry in position $(2, 3)$ of A is 3, the edge from node 2 to node 3 is assigned a weight of 3.

$$A = \begin{bmatrix} 5 & 2 & 1 \\ 1 & 0 & 3 \\ 4 & 1 & 0 \end{bmatrix}$$

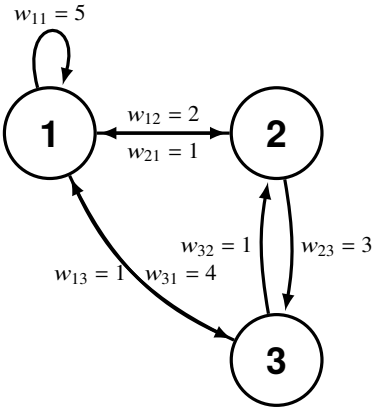


Fig. 52.2: A nonnegative matrix and its associated digraph.

For a directed graph G, the weighted adjacency matrix A tells us which nodes are directly connected: $A_{ij} \neq 0$ exactly when there is an edge from i to j. The next result shows how this idea extends to longer walks.

Proposition 52.42. (Existence of a walk of length k)

Let $A = [A_{ij}] \in \mathbb{R}^{n \times n}$ be a nonnegative matrix and G_A its associated digraph. For every positive integer k, the (i, j)th entry of A^k is positive if and only if there exists a walk $i = i_0, i_1, \ldots, i_k = j$ in G_A of length k with positive weight.

Proof By the definition of matrix multiplication, the (i, j)th entry of A^k is given by

$$(A^k)_{ij} = \sum_{i_1, \ldots, i_{k-1}=1}^{n} A_{ii_1} A_{i_1 i_2} \cdots A_{i_{k-1} j}. \tag{52.2}$$

Since $A \geqslant 0$, every term in this sum is nonnegative.

Suppose first that there exists a walk $i = i_0, i_1, \ldots, i_k = j$ whose weight is positive. The product $A_{i_0 i_1} A_{i_1 i_2} \cdots A_{i_{k-1} i_k}$ is then a positive term in (52.2). Therefore, $(A^k)_{ij} > 0$.

Conversely, suppose that $(A^k)_{ij} > 0$. Since every term in (52.2) is nonnegative, at least one term must be positive. Thus, for some indices $i_1, \ldots, i_{k-1}$,

$A_{ii_1}A_{i_1i_2}\cdots A_{i_{k-1}j} > 0$. Consequently, every factor in this product is positive, and $i, i_1, \ldots, i_{k-1}, j$ forms a walk of length k from i to j having positive weight. □

Now, let us define the type of digraph that we will use to characterize irreducible matrices.

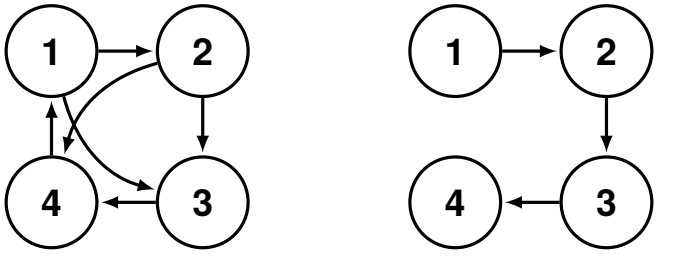
Definition 52.43. (Strongly connected digraph)

A directed graph G is **strongly connected** if, for any two nodes i and j (even when $i = j$), there is a directed walk that starts at i and ends at j.

Example 52.44 Figure 52.3 shows two digraphs. The one on the left is strongly connected; the one on the right is not.

In the right-hand graph node 4 has no outgoing edges, so there is no directed walk from 4 to 1, or to any other node, and the graph fails the strong-connectivity test.

In the left-hand graph every node can reach every other. Starting at node 1, for instance, we can go directly to node 2 ($1 \to 2$), directly to node 3 ($1 \to 3$), or reach node 4 via node 3 ($1 \to 3 \to 4$) or even by following a longer walk ($1 \to 2 \to 3 \to 4 \to 1$). A similar check for nodes 2, 3, and 4 confirms that all ordered pairs are linked, so this digraph is strongly connected.



Strongly connected digraph. Not strongly connected digraph.

Fig. 52.3: Two digraphs with four nodes: one strongly connected (left) and the other not (right).

The following proposition provides several equivalent characterizations of irreducible matrices.

Proposition 52.45. (Characterizations of irreducible matrices)

Let A be an $n \times n$ nonnegative matrix. The following conditions are equivalent:

(a) A is irreducible (no simultaneous row/column permutation can put it in block upper-triangular form).

(b) The digraph associated with A is strongly connected.
(c) For every pair of indices $i, j \in \{1, 2, \ldots, n\}$, there exists an integer k_{ij} such that $(A^{k_{ij}})_{ij} > 0$.
(d) $(I_n + A)^{n-1}$ is a positive matrix.

Proof First, we show that conditions (a) and (b) are equivalent.

Proof that (b) implies (a). We prove the contrapositive: if A is reducible, then its digraph is not strongly connected. Because A is reducible, there is a permutation matrix P such that

$$B := P^T AP = \begin{bmatrix} X & Y \\ 0 & Z \end{bmatrix}, \qquad Z \in \mathbb{R}^{(n-m)\times(n-m)}. \tag{52.3}$$

This permutation similarity simply relabels the vertices of the digraph; the edges themselves stay the same, so every connectivity property is preserved.

For any $k \geqslant 1$,

$$B^k = \begin{bmatrix} X^k & \star \\ 0 & Z^k \end{bmatrix}$$

Therefore, every power B^k has a zero block in the lower-left corner. Consequently no walk travels from the vertices indexed $m + 1, \ldots, n$ to those indexed $1, \ldots, m$. This implies G_B is not strongly connected, so G_A is not either.

Proof that (a) implies (b). Assume the digraph G_A of A is not strongly connected. Then, there are two distinct nodes, say i and j, such that there is no directed walk from i to j. Let

$$\mathcal{V}_1 := \{x : \text{there is a path from } i \text{ to } x\} \cup \{i\},$$

that is, the set of all nodes that can be reached by a directed walk starting from i, including i. Let $\mathcal{V}_2 := \{1, 2, \ldots, n\} \backslash \mathcal{V}_1$ be its complement. The set $\mathcal{V}_1$ is non-empty (since $i \in \mathcal{V}_1$) and $\mathcal{V}_2$ is nonempty (since $j \in \mathcal{V}_2$).

We claim that for all $x \in \mathcal{V}_1$ and $y \in \mathcal{V}_2$, there is no directed walk from x to y. Otherwise, since there is a walk from i to x, we could build a walk from i to y, which is a contradiction.

Assume that $\mathcal{V}_1$ contains n_1 nodes, and renumber the nodes so that $\mathcal{V}_1 = \{1, \ldots, n_1\}$ and $\mathcal{V}_2 = \{n_1 + 1, \ldots, n\}$. Hence, there is a permutation matrix P_1 such that

$$P_1^T AP_1 = \begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix},$$

which implies that A is reducible by Remark 52.36.

Proof that (b) and (c) are equivalent. The digraph G_A associated with A is strongly connected if and only if, for every pair of nodes i and j in G_A, there is a directed walk from i to j. Let k_{ij} be the length of such a walk. By Proposition 52.42, this is equivalent to the (i, j)-th entry of $A^{k_{ij}}$ being positive.

Proof that (d) implies (a). Assume, to get a contradiction, that A is reducible. Then, there is permutation matrix P that puts it in block upper-triangular form B as in (52.3). Hence, we can write

$$(I_n + A)^{n-1} = (PP^T + PBP^T)^{n-1} = \left(P(I_n + B)P^T\right)^{n-1} = P(I_n + B)^{n-1}P^T,$$

where the last equality follows because $P^T P = I_n$. Notice that

$$I_n + B = \begin{bmatrix} X + I & Y \\ 0 & Z + I \end{bmatrix}.$$

Using block matrix multiplication and induction, we can show that the lower-left block stays zero in every power of $I_n + B$, that is, for every $k \geqslant 1$,

$$(I_n + B)^k = \begin{bmatrix} \star & \star \\ 0 & \star \end{bmatrix}.$$

In particular $(I_n + B)^{n-1}$ contains a zero entry. Undoing the permutation cannot remove that zero, so $(I_n + A)^{n-1}$ is *not* positive. Hence the positivity of $(I_n + A)^{n-1}$ forces A to be irreducible.

Proof that (c) implies (d). Part (c) guarantees that for every pair (i, j) there exists a directed walk of some length k_{ij} from node i to node j. If the walk repeats a node, remove the cycle you just located; the remaining walk is still valid and strictly shorter. After repeatedly cutting out cycles you obtain a path, that is, a walk in which every vertex appears at most once, so its length is at most $n - 1$. Hence there is some $k_{ij} \leqslant n - 1$ with $(A^{k_{ij}})_{ij} > 0$.

Using the binomial expansion of $(I_n + A)^{n-1}$, we get

$$(I_n + A)^{n-1} = \sum_{k=0}^{n-1} \binom{n-1}{k} A^k.$$

For the entry (i, j) we have

$$\left[(I_n + A)^{n-1}\right]_{ij} = \sum_{k=0}^{n-1} \binom{n-1}{k} (A^k)_{ij}.$$

The term with $k = k_{ij}$ is strictly positive, and all the binomial coefficients $\binom{n-1}{k}$ are positive, so the whole sum is positive. Since this holds for every pair (i, j), every entry of $(I_n + A)^{n-1}$ is positive; that is, $(I_n + A)^{n-1} > 0$. □

For a small matrix you can often check irreducibility with part (b) of Proposition 52.45: draw the digraph of A and, at a glance, decide whether every node reaches every other. For moderate size matrices a drawing is impractical, and part (d) becomes the "practical" test: compute $(I_n + A)^{n-1}$ and see whether every entry is positive.

The exponent $n - 1$ is essential only as a worst-case bound. No irreducible $n \times n$ matrix needs a higher power, but some need exactly $n - 1$; it is the smallest exponent that works for every size-n case.

Let us illustrate Proposition 52.45 with some examples.

Example 52.46 Let B be the matrix from Example 52.34. Notice that

$$B^2 = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} > 0.$$

By Proposition 52.45-(c), this shows B is irreducible. We can also check irreducibility by noting that $I_2 + B > 0$. As an exercise, draw the digraph associated with B and verify that it is strongly connected.

Example 52.47 Consider the nonnegative matrices

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

You can see their respective digraphs in Figure 52.4. Notice that the graph on the left is strongly connected, while the graph on the right is not even connected. By Proposition 52.45, this implies A is irreducible, but B is not.

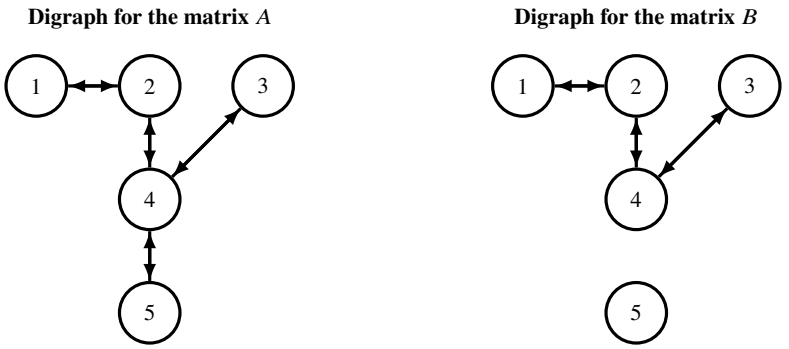


Fig. 52.4: Strongly connected graph (left) vs not connected graph (right).

Let us now present some characterizations of primitive matrices, that is, irreducible nonnegative matrices that have exactly one eigenvalue on their spectral circle.

Proposition 52.48. (Characterization of primitive matrices I)

A nonnegative matrix A is primitive if and only if some power of A is positive, that is, there exists a positive integer k such that $A^k > 0$. Moreover, if such a k exists, then $k \leqslant (n-1)^2 + 1$.

Proof Assume that A is a nonnegative matrix and that $A^k > 0$ for some positive integer k. This condition guarantees that, for every pair of nodes in the digraph G_A associated with A, there exists a walk of length k connecting them. By Proposition 52.45, this implies that A is irreducible, and by Theorem 52.38, the spectral radius $\rho(A) > 0$ is a simple eigenvalue of A.

Now, suppose for contradiction that A has another eigenvalue λ on the spectral circle, distinct from $\rho(A)$. Then we can write $\lambda = \rho(A)e^{i\theta}$ for some $\theta \in [0, 2\pi)$ with $\theta \neq 0$.

The matrix A^k has eigenvalues $\rho(A)^k$ and $\rho(A)^k e^{ik\theta}$, and the matrix A^{k+1} has eigenvalues $\rho(A)^{k+1}$ and $\rho(A)^{k+1}e^{i(k+1)\theta}$. But since A^k and A^{k+1} are positive matrices, Perron's Theorem (Theorem 52.28) for positive matrices tells us that $\rho(A)^k$ and $\rho(A)^{k+1}$ are the unique eigenvalues on their respective spectral circles. Thus, we must have

$$\rho(A)^k = \rho(A)^k e^{ik\theta} \quad \text{and} \quad \rho(A)^{k+1} = \rho(A)^{k+1} e^{i(k+1)\theta}.$$

Dividing both sides by the nonzero terms $\rho(A)^k$ and $\rho(A)^{k+1}$, respectively, gives $1 = e^{ik\theta} = e^{i(k+1)\theta}$, which forces $e^{i\theta} = 1$. This is a contradiction, since it implies $\theta = 0$ and hence $\lambda = \rho(A)$.

Now we prove the converse. Assume A is primitive. Let $\rho := \rho(A)$ be its spectral radius. By the Perron-Frobenius theorem, $\rho > 0$ is a simple eigenvalue.

Define

$$B := \tfrac{1}{\rho} A.$$

Then B is nonnegative and irreducible, with eigenvalues equal to those of A divided by ρ. In particular, $\rho(B) = 1$. Since A is primitive, B has only one eigenvalue on the unit circle, namely $\lambda = 1$.

If we show that $B^k > 0$ for some k, then $A^k = \rho^k B^k$ will also be positive, since $\rho^k > 0$. So it is enough to prove that $B^k > 0$ for some k.

Let $B = PJP^{-1}$ be the Jordan canonical form of B, with the eigenvalues in J ordered by size. The first column of P, call it x, is an eigenvector of B with eigenvalue 1. By the Perron-Frobenius theorem, there exists a strictly positive eigenvector for $\lambda = 1$. Since the eigenspace is one-dimensional, we can scale x so that it is this positive eigenvector.

The limit of $B^k = PJ^kP^{-1}$ is determined by the limit of J^k. To study this, we look at each Jordan block J_i. There are two possibilities for the blocks of B:

1. For any eigenvalue λ_i with $|\lambda_i| < 1$: The powers of the corresponding Jordan block, J_i^k, will converge to the zero matrix.
2. For the eigenvalue $\lambda_1 = 1$: The Perron-Frobenius theorem tells us this eigenvalue is simple (its algebraic multiplicity is 1). Since the size of a Jordan block depends on

the eigenvalue's multiplicity, a simple eigenvalue can only have a 1×1 block. Thus the block for $\lambda_1 = 1$ is $J_1 = [1]$, which remains the same as k goes to infinity.

As $k \to \infty$, the powers of the Jordan matrix J take a simple form. The block $J_1^k = [1]^k = [1]$, while all other blocks J_i^k go to zero. Thus,

$$\lim_{k\to\infty} J^k = J_\infty = \begin{pmatrix} 1 & 0 & \cdots \\ 0 & 0 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}.$$

It follows that the sequence B^k converges to a limit matrix L:

$$L = \lim_{k\to\infty} B^k = PJ_\infty P^{-1}.$$

Recall that the first column x of P is a positive eigenvector of B for $\lambda = 1$. Since $P^{-1}B = JP^{-1}$, the first row of P^{-1}, call it z^T, is a left eigenvector of B for $\lambda = 1$. By the Perron-Frobenius theorem, there is also a positive left eigenvector, say y^T. Because the left eigenspace is one-dimensional, we must have $z^T = cy^T$ for some constant c.

Since $P^{-1}P = I$, we get $z^T x = 1$. Thus $1 = z^T x = cy^T x$, which implies $c = 1/y^T x$. Multiplying out $PJ_\infty P^{-1}$, we get

$$L = \frac{xy^T}{y^T x} > 0,$$

since $x, y > 0$ makes xy^T positive and $y^T x$ a positive scalar.

Finally, since the sequence of matrices B^k converges to the positive matrix L, for each pair i, j, there exists an integer K_{ij} such that

$$(B^k)_{ij} > \frac{L_{ij}}{2} > 0$$

whenever $k \geqslant K_{ij}$. Since there are only finitely many entries, we may define $K := \max_{1\leqslant i,j\leqslant n} K_{ij}$. Then $(B^K)_{ij} > 0$ for every i, j, and therefore $B^K > 0$. Thus, B is primitive.

The fact that the smallest exponent k such that $A^k > 0$ satisfies $k \leqslant (n-1)^2 + 1$ was established by Wielandt in 1950 [52]. You can also find the statement and proof in modern texts, for example in [30]. □

Primitive matrices are, by definition, irreducible. For an irreducible matrix, its associated digraph guarantees that there is some walk connecting any two nodes, although the lengths of these walks may vary. In contrast, if A is primitive, there exists a fixed integer k such that a walk of length k exists between every pair of nodes.

Proposition 52.48 provides a computational test for primitivity: check whether any power A^k (with $k \leqslant (n-1)^2 + 1$) is positive.

Example 52.49 Consider the nonnegative matrices

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

Notice that

$$A^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} > 0.$$

Since some power of A is positive, A is primitive. On the other hand, for B we have, for $k = 1, 2, 3, \ldots,$

$$B^{2k-1} = \begin{bmatrix} 0 & 2^{k-1} & 2^{k-1} \\ 2^{k-1} & 0 & 0 \\ 2^{k-1} & 0 & 0 \end{bmatrix} \quad \text{and} \quad B^{2k} = \begin{bmatrix} 2^k & 0 & 0 \\ 0 & 2^{k-1} & 2^{k-1} \\ 0 & 2^{k-1} & 2^{k-1} \end{bmatrix}.$$

No power of B is positive since some entries remain zero. Thus, B is not primitive.

For a second characterization of primitive matrices, we first recall the concept of a cycle in a digraph, which is a closed path, that is, it starts and ends at the same vertex.

Primitive matrices can be characterized by analyzing the cycle lengths in their associated digraphs. Specifically, we define the **period of a digraph** as the greatest common divisor (gcd) of the lengths of all its cycles.

In particular, a digraph has period 1 if the greatest common divisor of the lengths of all its cycles is 1.

Proposition 52.50. (Characterization of primitive matrices II)

A nonnegative matrix A is primitive if and only if its associated digraph G_A is strongly connected and has period 1.

Proof Here we prove only the "only if" direction; the converse is more technical and therefore omitted.

Suppose $A \geqslant 0$ is primitive. By Proposition 52.48, there exists an integer k such that $A^k > 0$. Positivity of A^k implies $[A^k]_{ij} > 0$ for every i, j. Hence, by Proposition 52.45, the digraph of A is strongly connected.

Because $A^k > 0$, no column of A can be zero. Therefore, for every j, there is an index ℓ such that $A_{\ell j} > 0$, and hence, for every i

$$(A^{k+1})_{ij} = \sum_{\ell=1}^{n} (A^k)_{i\ell} A_{\ell j} > 0.$$

Thus, $A^{k+1} > 0$. In particular, $(A^k)_{ii} > 0$ and $(A^{k+1})_{ii} > 0$ for every i. Hence, there are closed walks from i to itself of lengths k and $k + 1$.

Let d be the greatest common divisor of the lengths of all cycles in G_A. Every closed walk can be decomposed into cycles, so its length is a sum of cycle lengths and is therefore divisible by d. Consequently, d divides both k and $k + 1$. Since $\gcd(k, k + 1) = 1$, we obtain $d = 1$. Thus, G_A has period 1. □

Example 52.51 In Figure 52.5, we show two digraphs, ignoring edge weights as they are not relevant here. Both digraphs are strongly connected. In the digraph on the left, although there are several cycles, we can identify at least two cycles, $A \to C \to D \to A$ (length 3) and $C \to B \to D \to A \to C$ (length 4), that have coprime lengths. Since the greatest common divisor of 3 and 4 is 1, the period of the digraph is 1, and its associated matrix is primitive. In contrast, in the digraph on the right, every cycle has a length that is a multiple of 2, giving it a period of 2. Therefore, its associated matrix is not primitive.

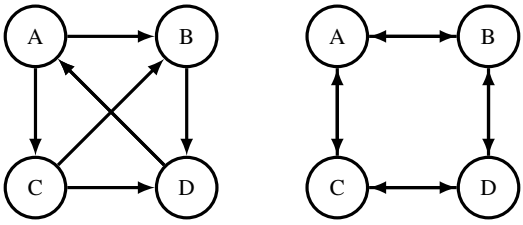


Fig. 52.5: Left: Digraph corresponding to a primitive matrix with cycles whose lengths have gcd 1. Right: Digraph corresponding to a non-primitive matrix where all cycles have lengths that are multiples of 2.

Another simple test for primitivity is given by the following result.

Proposition 52.52. (Sufficient condition for primitivity)

If a nonnegative irreducible matrix A has at least one positive diagonal entry, then A is primitive.

Example 52.53 Consider the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

As we have seen, A is nonnegative and irreducible, and it has a positive diagonal entry. Therefore, by the sufficient condition for primitivity, A is primitive.

Now, consider the matrix

$$B = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Although B is also nonnegative and irreducible, it is not primitive because the corresponding digraph has only a single cycle of length 2. In fact, you can verify that for all integers k, we have $B^{2k} = I_2$ and $B^{2k+1} = B$, so no power of B is positive. Note that all the diagonal entries of B are zero.

52.5 Stochastic matrices: a special class of nonnegative matrices

One of the most elegant applications of the Perron-Frobenius theory is its use in the study of finite Markov chains, which we will study later. This theory relies on a special type of nonnegative matrix, which we will introduce in this section.

Definition 52.54. (Row and column stochastic matrices)

A **row stochastic matrix** is a square nonnegative matrix whose row sums equal 1. A **column stochastic matrix** is a square nonnegative matrix whose column sums equal 1. If a matrix is both row and column stochastic, it is called **doubly stochastic**.

Note that a matrix is row stochastic if and only if its transpose is column stochastic, and vice versa.

Example 52.55 Permutation matrices, including the identity matrix, are both row and column stochastic. Therefore, they are doubly stochastic.

Example 52.56 Consider the matrix

$$A = \begin{bmatrix} 1/2 & 1/2 \\ 1/3 & 2/3 \end{bmatrix}$$

This matrix is row stochastic because the entries in each row add up to 1 (since $\frac{1}{2} + \frac{1}{2} = 1$ and $\frac{1}{3} + \frac{2}{3} = 1$). However, it is not column stochastic because the column sums are not 1 (for example, $\frac{1}{2} + \frac{1}{3} \neq 1$).

Let us study some properties of stochastic matrices. First, recall that for any square matrix A with eigenvalue λ, a nonzero vector x satisfying

$$Ax = \lambda x$$

is called a **right eigenvector** of A, while a nonzero vector y satisfying

$$y^T A = \lambda y^T$$

is called a **left eigenvector** of A corresponding to λ. Note that right eigenvectors are the ones we studied in Chapter 37, and left eigenvectors are simply the right eigenvectors of A^T.

Theorem 52.57. (Characterization of stochastic matrices)

A nonnegative matrix P is row (respectively, column) stochastic if and only if it has the eigenvalue 1 with a right (respectively, left) eigenvector $e = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}^T$. Moreover, if P is row (or column) stochastic, then its spectral radius is 1.

Proof Assume P is row stochastic, so each row sum is equal to 1. Then

$$Pe = e,$$

so e is a right eigenvector corresponding to the eigenvalue 1. Conversely, if $Pe = e$, then every row of P must sum to 1, implying that P is row stochastic.

Similarly, P is column stochastic if and only if P^T is row stochastic. That is, P is column stochastic if and only if

$$P^T e = e,$$

which is equivalent to $e^T P = e^T$. Thus, the characterization holds for column stochastic matrices as well.

Finally, Proposition 52.18 tells us that the spectral radius of P is 1. □

Next, we show that the property of being stochastic is preserved under multiplication.

Proposition 52.58. (Products of stochastic matrices)

Let P and Q be row (or column) stochastic matrices. Then, the product PQ is also row (or column) stochastic. In particular, P^k is row (or column) stochastic for all positive integers k.

Proof Assume first that P and Q are row stochastic. Since every row of Q sums to 1, we have $Qe = e$, and thus

$$PQe = P(Qe) = Pe = e.$$

This shows that PQ has 1 as an eigenvalue with associated right eigenvector e. By Theorem 52.57, PQ is row stochastic.

Now, if P and Q are column stochastic, note that a matrix is column stochastic if and only if its transpose is row stochastic. Since $(PQ)^T = Q^T P^T$ and both P^T and Q^T are row stochastic, their product is row stochastic. Therefore, PQ is column stochastic.

Finally, to prove that P^k is row stochastic for every positive integer k, we use induction. For $k = 1$, P is row stochastic by assumption. Now, assume that P^{k-1} is row stochastic. Then, since P is row stochastic,

$$P^k = P\,P^{k-1}$$

is the product of two row stochastic matrices, which we have already shown to be row stochastic. A similar argument applies to column stochastic matrices. □

In one of the applications of nonnegative matrices that we will discuss later, we often compute the product of a stochastic matrix with a probability vector. Recall that a vector $x \in \mathbb{R}^n$ is a **probability vector** if it is nonnegative (i.e. $x \geqslant 0$) and its entries sum to 1 ($\|x\|_1 = 1$). Equivalently, x is a probability vector if $x \geqslant 0$ and $x^T e = 1$, where $e = \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix}^T$.

Example 52.59 The vector $x = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}$ is a probability vector because its entries are nonnegative and add up to 1.

Proposition 52.60. (The product of a stochastic matrix and a probability vector)

Let x be a probability vector and let P be a row (or column) stochastic matrix. Then, $x^T P$ (if P is row stochastic) or Px (if P is column stochastic) is also a probability vector.

Proof Assume P is row stochastic. Since both x and P are nonnegative, the product $x^T P$ is a nonnegative vector. Also, because every row of P sums to 1, we have

$$x^T Pe = x^T e = 1,$$

where e is the vector of all ones. This shows that the entries of $x^T P$ are nonnegative and add up to 1, so $x^T P$ is a probability vector. The proof for column stochastic matrices follows similarly. □

In general, a stochastic matrix need not be irreducible or primitive; some are, and some are not. But the fact that each row (or each column) sums to 1 still forces strong spectral constraints.

Proposition 52.61. (Eigenvalues of stochastic matrices on the unit circle are regular)

Let A be a row (or column) stochastic matrix. Every eigenvalue of A that lies on the unit circle is regular, that is, its algebraic and geometric multiplicities are equal.

Proof For a proof, see Corollary 2 in the paper *On the Equality of Algebraic and Geometric Multiplicities of Matrix Eigenvalues* by J. Ding and N. H. Rhee in Applied Math Letters [17]. □

Example 52.62 The matrix

$$A_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

is clearly row stochastic. Its eigenvalues are 1 and -1, both lie on the unit circle and each has algebraic multiplicity 1, hence they are regular.

A simple example of a stochastic matrix that illustrates Proposition 52.61 is the identity matrix.

Example 52.63 For an $n \times n$ identity matrix I_n (with $n > 1$), the eigenvalue 1 has algebraic multiplicity n and geometric multiplicity n.

52.6 The theory of discrete time Markov chains

Before we give the formal definition of a discrete time Markov chain, let us begin with an example.

Example 52.64. (Recommender system model)

Imagine you run a small online business. When a customer visits your website, they can take one of several actions: browse products, add an item to their cart, make a purchase, or leave the site. Based on your observations, you have gathered the following data:

(i) If a customer is browsing, there is a 50% chance they will continue browsing after 15 minutes, a 30% chance they will add an item to the cart, and a 20% chance they will leave the site.
(ii) Once a customer adds an item to their cart, there is a 20% chance they will remain in the cart after 15 minutes, a 30% chance they will return to browsing, a 40% chance they will complete the purchase, and a 10% chance they will exit.
(iii) After a purchase, there is a 30% chance the customer will return to browsing and a 70% chance they will leave.
(iv) If a customer leaves the website, there is an 80% chance they will not return within the next 15 minutes and a 20% chance they will come back and start browsing again.

Figure 52.6 displays this information as a graph. In the graph, the nodes represent the four customer states: browsing, adding to the cart, purchasing, and exiting the

website. Directed edges connect the nodes, with each edge labeled by the probability of moving from one state to another. For clarity in the figure, we omit edges with zero probability.

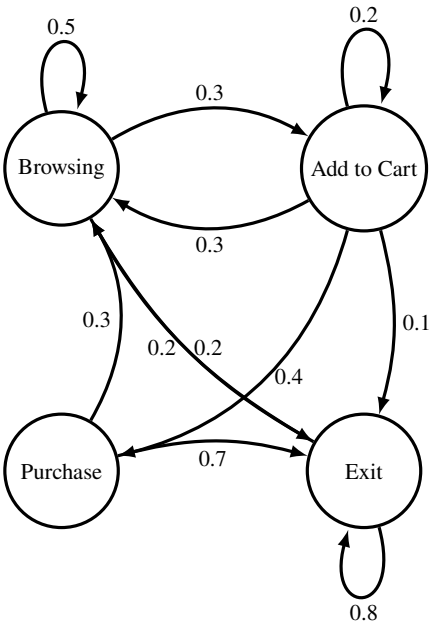


Fig. 52.6: Markov chain diagram illustrating the actions a customer can take in an online business–browsing, adding to cart, purchasing, and exiting–with the corresponding probabilities indicated on the edges.

The goal is to build a mathematical model of your customers' behavior–perhaps a more detailed version of the example above–that uses different states with associated probabilities to improve your business. For instance, you might use such a model to:

1. **Customer Retention:** Predict the likelihood that a customer will continue browsing, make a purchase, or exit your website, helping you develop strategies to keep them engaged.
2. **Churn Prediction:** Analyze transition probabilities to identify customers who are likely to leave, so you can target them with tailored marketing campaigns or special offers.
3. **Product Recommendations:** Understand your customers' behavior well enough to suggest products or discounts that match their current stage in the shopping process.

Let us create a mathematical model based on the graph we have drawn to help us make predictions. We define $\Omega = \{b, c, p, e\}$, where b stands for "browsing", c

for "add to cart", p for "purchase", and e for "exit". In other words, Ω is the set of possible states.

We record the transition probabilities in the following table:

Now\In 15 min.	b	c	p	e
b	0.5	0.3	0	0.2
c	0.3	0.2	0.4	0.1
p	0.3	0	0	0.7
e	0.2	0	0	0.8

We can represent these transition probabilities with the row stochastic matrix

$$P := \begin{bmatrix} 0.5 & 0.3 & 0 & 0.2 \\ 0.3 & 0.2 & 0.4 & 0.1 \\ 0.3 & 0 & 0 & 0.7 \\ 0.2 & 0 & 0 & 0.8 \end{bmatrix}.$$

Let $\pi_1^t, \pi_2^t, \pi_3^t$, and π_4^t denote the probabilities that at time t a customer is browsing, adding an item to the cart, purchasing, or exiting the site, respectively. We represent these probabilities with the vector

$$\pi_t := \begin{bmatrix} \pi_1^t & \pi_2^t & \pi_3^t & \pi_4^t \end{bmatrix}^T.$$

By the law of total probability, the chance that the customer has left the website 15 minutes later (at time $t + 1$) is

$$\pi_4^{t+1} = p_{14}\pi_1^t + p_{24}\pi_2^t + p_{34}\pi_3^t + p_{44}\pi_4^t,$$

where p_{i4} denotes the probability of transitioning from state i to state 4 (exiting the site). In other words, the probability of exiting at time $t + 1$ equals the sum of the probabilities of being in each state at time t multiplied by the probability of transitioning from that state to exit.

Similarly, we can compute the probabilities that 15 minutes after time t, the customer is still browsing, adding an item to the cart, or making a purchase.

In matrix form, we can summarize the four probabilities as

$$\pi_{t+1} = P^T \pi_t,$$

or equivalently,

$$\pi_{t+1}^T = \pi_t^T P,$$

if you prefer to express the conditions in terms of P instead of P^T. Here, $t + 1$ represents one period (15 minutes) after time t. Similarly, we have

$$\pi_t^T = \pi_{t-1}^T P.$$

By repeating this process, we obtain

$$\pi_{t+1}^T = \pi_{t-1}^T P^2 = \pi_{t-2}^T P^3 = \cdots = \pi_0^T P^{t+1},$$

where π_0 is the initial probability vector for the states (browsing, adding to cart, purchasing, and exiting) when we start our observations.

Since P is row stochastic, Proposition 52.58 tells us that P^{t+1} remains row stochastic for all $t \geqslant 1$. Furthermore, because π_0 is a probability vector, Proposition 52.60 ensures that π_{t+1} is also a probability vector.

We have now built a mathematical model that can help us answer some practical questions.

1. **Question:** What is the probability that a customer who is currently browsing will make a purchase in the next 30 minutes?

   Answer: The transition matrix P gives the probabilities for each 15-minute period. To find the probability after 30 minutes, we compute P^2. Look at the entry in position $(1, 3)$ of P^2; it equals 0.12. Thus, the probability that the customer browsing will be making a purchase after 30 minutes is 12%.
2. **Question:** How likely is it that a customer who has made a purchase will return directly to browsing?

   Answer: The transition probability from "Purchase" to "Browsing" is 30%, which might reflect a situation where a customer continues to explore products even after buying something. However, the probability of exiting is higher (70%), so most customers tend to leave after a purchase.
3. **Question:** Over the long term, if customer behavior remains stable, what proportion of customers do you expect to be in each state–browsing, adding to cart, purchasing, or exiting–at any given time?

   Answer: Assume a customer starts by browsing, so that $\pi_0^T = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$. Then, the state vector at time t is given by

   $$\pi_t^T = \pi_0^T P^t.$$

   As t becomes large,

   $$\lim_{t\to\infty} \pi_t^T = \pi_0^T \lim_{t\to\infty} P^t.$$

   Note that the digraph associated with P is strongly connected. Thus, P is irreducible. Moreover, $P_{11} = 0.5 > 0$, so the digraph contains a self-loop. Consequently, the digraph has period 1, and P is primitive. Since P is row stochastic, $\rho(P) = 1$. By the Perron–Frobenius theorem for primitive matrices, 1 is a simple, strictly dominant eigenvalue of P. Therefore, $\lim_{t\to\infty} P^t$ exists. If you compute a high power of P, you find that

   $$\lim_{t\to\infty} P^t \approx \begin{bmatrix} 0.3089 & 0.1158 & 0.0463 & 0.5290 \\ 0.3089 & 0.1158 & 0.0463 & 0.5290 \\ 0.3089 & 0.1158 & 0.0463 & 0.5290 \\ 0.3089 & 0.1158 & 0.0463 & 0.5290 \end{bmatrix}.$$

   Thus,

   $$\lim_{t\to\infty} \pi_t^T \approx \begin{bmatrix} 0.3089 & 0.1158 & 0.0463 & 0.5290 \end{bmatrix}.$$

In other words, over time, about 30.9% of customers will be browsing, 11.6% adding items to their cart, 4.6% making a purchase, and 52.9% exiting the site. Later, we will discuss methods to compute this vector without raising P to a high power. Notice that this result does not depend on the initial state since all rows of the limit matrix are identical.

52.6.1 The definition of Markov chains

We now formalize the theory behind the recommender system example we introduced earlier. The mathematical model in that example is called a Markov chain, a simple yet powerful tool for modeling random processes. In this section, our goal is to build an intuitive understanding of Markov chains without going into every technical detail. We aim to provide a clear conceptual explanation of how Markov chains work and why they are useful, even for readers without a strong background in statistics.

The state space

We start by considering a phenomenon or random experiment. For example, consider the daily weather in a city or the decisions made by an online buyer. The set S of possible outcomes when such a phenomenon is observed is called its **state space**. We will consider phenomena that produce finite state spaces.

Example 52.65 In the example about the online company that we described before, the state space consists of four states "browsing", "adding to the cart", "purchasing", "exiting the website", that can be represented by the state space $S = \{b, c, p, e\}$, where b represents "browsing", c represents "adding to the cart", p represents "purchasing", and e represents "exiting".

Example 52.66 Consider a weather model with three states: "rainy", "sunny", and "cloudy". In this case, the state space can be defined as $S = \{r, s, c\}$, where r represents "rainy", s represents "sunny", and c represents "cloudy".

Once we have defined the state space of the phenomenon we are studying, we assign a probability to each element in the state space.

The initial probability distribution

Given a finite state space $S = \{s_1, s_2, \ldots, s_k\}$ associated with a phenomenon, assume we have a **probability distribution** π^0 on S, that is, we have k real numbers $\{p_1, \ldots, p_k\}$ satisfying

(i) $0 \leqslant p_i \leqslant 1$, for $i \in \{1, \ldots, k\}$;

(ii) $\sum_{i=1}^{k} p_i = 1$.

We call p_i the probability of s_i and denote it by $p_i = \pi^0(s_i)$.

As mentioned earlier, our focus will be on **discrete-time** Markov chains. But what exactly do we mean by a "discrete-time process"?

Discrete-time Markov chains

A discrete-time process is one observed at fixed, regularly spaced intervals, evolving through a finite or countably infinite sequence of steps. For instance, in our weather model, each step could correspond to one day. We might observe the weather over a limited number of days or analyze its long-term behavior across an infinite sequence of days. In most practical applications, these steps represent uniform units of time.

Discrete-time Markov chains can evolve over a finite number of steps or an infinite sequence. We start with the finite case because it is simpler to understand and analyze; later, we will extend these ideas to infinite processes.

Suppose we have a phenomenon with a finite state space S and we observe it over n steps. We define the **sample space** as

$$\Omega_n := S^{n+1} = \{(x_0, x_1, \ldots, x_n) \mid x_i \in S\},$$

where the sequence $(x_0, x_1, \ldots, x_n)$ contains the states observed at times $0, 1, 2, \ldots, n$. For instance, in our weather model, Ω_n would consist of all possible sequences of weather conditions over $n + 1$ consecutive days.

Each subset of Ω_n is called an **event**. For example, in our weather model, an event might be "the second day is cloudy", which corresponds to all $(n + 1)$-tuples in Ω_n where the second entry is c.

Through observation or other techniques, it may be possible to calculate the so-called **transition probabilities**, which give the likelihood of moving from one state s_i to another state s_j in S. We denote this probability by $p(s_i, s_j)$. These probabilities must satisfy

(i) $p(s_i, s_j) \in [0, 1]$, and
(ii) for each state $s_i \in S$,

$$\sum_{s_j \in S} p(s_i, s_j) = 1.$$

Using the initial probability distribution π_0 and the transition probabilities, we define a probability distribution $\mathbb{P}$ on the power set of Ω_n. Let us start by assigning a probability to events consisting of one single element of Ω_n. For any $x = (x_0, x_1, \ldots, x_n) \in \Omega_n$, we define

$$\mathbb{P}(x) := \pi^0(x_0)p(x_0, x_1)p(x_1, x_2) \cdots p(x_{n-1}, x_n). \tag{52.4}$$

We will explain later why we have defined $\mathbb{P}(x)$ in this way. For now, we can say that this probability distribution works well for many physical phenomena and random experiments.

Now, given a subset $F \subseteq \Omega_n$, we define

$$\mathbb{P}(F) := \sum_{x \in F} \mathbb{P}(x).$$

It is easy to see that $\mathbb{P}(\Omega_n) = 1$.

Example 52.67 Suppose we observe the weather in our city for four consecutive days starting today. Assume that the initial probabilities of each possible weather state are

$$\pi^0(s) = 0.3, \quad \pi^0(r) = 0.2, \quad \text{and} \quad \pi^0(c) = 0.5.$$

Based on historical data, we have the transition probabilities collected in the following table, whose (i, j)-th entry gives the probability $p(s_i, s_j)$.

	s	r	c
s	0.6	0.2	0.2
r	0.3	0.5	0.2
c	0.1	0.2	0.7

For example, the probability that the weather sequence over the next four days is (c, r, r, s) is given by

$$\pi^0(c)p(c, r)p(r, r)p(r, s) = 0.5 \cdot 0.2 \cdot 0.5 \cdot 0.3 = 0.015.$$

Now, consider the event F that the weather is sunny for the last three days, regardless of the first day's weather. In other words, F consists of all sequences of the form (x_0, s, s, s). Thus,

$$\mathbb{P}(F) = \mathbb{P}(s, s, s, s) + \mathbb{P}(r, s, s, s) + \mathbb{P}(c, s, s, s).$$

Computing each term:

$$\begin{aligned}
&\mathbb{P}(s, s, s, s) = \pi^0(s)p(s, s)^3 = 0.3 \times 0.6^3, \\
&\mathbb{P}(r, s, s, s) = \pi^0(r)p(r, s)p(s, s)^2 = 0.2 \times 0.3 \times 0.6^2, \quad \text{and} \\
&\mathbb{P}(c, s, s, s) = \pi^0(c)p(c, s)p(s, s)^2 = 0.5 \times 0.1 \times 0.6^2.
\end{aligned}$$

Thus,

$$\mathbb{P}(F) = 0.3 \times 0.6^3 + 0.2 \times 0.3 \times 0.6^2 + 0.5 \times 0.1 \times 0.6^2 = 0.1044$$

We can represent the sequence of states by defining a sequence of functions. For each i, let

$$X_i : \Omega_n \to S, \quad X_i(x) = x_i.$$

That is, X_i extracts the ith state from the sequence $(x_0, x_1, \ldots, x_n) \in \Omega_n$. Each of these functions is called a **random variable** on Ω_n. In our example, the function X_i gives the weather on day i after the starting day.

Now consider the following question based on our weather example: What is the probability that day j is sunny, if we know the weather on days $0, 1, \ldots, j-1$? More generally, what is the probability that state s_i occurs at step j if we know the states of all previous steps? This leads us to the concept of **conditional probabilities**. Given two events A and B, the conditional probability of A given B is defined as

$$\mathbb{P}(A|B) := \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)},$$

which is the ratio of the probability that both A and B happen and the probability that B happens.

In our case, the conditional probability of $X_j = x_j$ given $X_0 = x_0$, $X_1 = x_1$, $\ldots$, $X_{j-1} = x_{j-1}$ is

$$\mathbb{P}(X_j = x_j \mid X_0 = x_0, \ldots, X_{j-1} = x_{j-1}) = \frac{\mathbb{P}(x_0, \ldots, x_{j-1}, x_j)}{\mathbb{P}(x_0, \ldots, x_{j-1})}.$$

Recall that we defined the probability of a sequence $x = (x_0, x_1, \ldots, x_n)$ in (52.4). Plugging in our definition, we obtain

$$\begin{aligned}\mathbb{P}(X_j = x_j \mid X_0 = x_0, \ldots, X_{j-1} = x_{j-1}) &= \frac{\mathbb{P}(x_0, \ldots, x_{j-1}, x_j)}{\mathbb{P}(x_0, \ldots, x_{j-1})} \\ &= \frac{\pi^0(x_0)p(x_0, x_1)\cdots p(x_{j-1}, x_j)}{\pi^0(x_0)p(x_0, x_1)\cdots p(x_{j-2}, x_{j-1})} \\ &= p(x_{j-1}, x_j).\end{aligned}$$

This is an interesting result! It shows that the probability of transitioning to x_j depends only on the previous state x_{j-1}, not on earlier states. In other words, we have

$$\mathbb{P}(X_j = x_j \mid X_0 = x_0, \ldots, X_{j-1} = x_{j-1}) = \mathbb{P}(X_j = x_j \mid X_{j-1} = x_{j-1}),$$

which is known as the **Markov property** (or Markov condition).

Note that the Markov property is a direct consequence of the way we defined $\mathbb{P}$ in (52.4).

It turns out that if we have a sample space S with an initial probability distribution π^0, a function $p : S \times S \to [0, 1]$ of transition probabilities, and a probability distribution $\mathbb{P}$ on the power set of Ω_n that satisfies the Markov property, then $\mathbb{P}(x)$ must be defined as in (52.4).

In examples such as the weather model, it is very reasonable to assume that tomorrow's weather depends only on today's conditions. With this in mind, we focus on phenomena with this property.

Now we are ready to define Markov chains of finite length.

Definition 52.68. (Markov chain of finite length)

Let $S = \{s_1, \ldots, s_k\}$. A collection of random variables $\{X_0, X_1, \ldots, X_n\}$ is called a **homogeneous Markov chain of length** n with state space S, transition probabilities p, and initial distribution π^0 if it satisfies the Markov property. That is, for every $j = 1, 2, \ldots, n$ and for all $x_0, x_1, \ldots, x_j \in S$,

$$\mathbb{P}(X_j = x_j \mid X_0 = x_0, \ldots, X_{j-1} = x_{j-1}) = p(x_{j-1}, x_j),$$

where $\mathbb{P}$ is as in (52.4).

The adjective "homogeneous" means that the transition probabilities remain constant over time. In other words, for any time t and any states s_j and s_i, we have that the probability

$$p_{ij} = \mathbb{P}(X_{t+1} = s_j \mid X_t = s_i)$$

does not depend on t.

Example 52.69 Consider the weather model with three states: "sunny", "rainy", and "cloudy" again. The table below contains the transition probabilities between these states.

	s	r	c
s	0.6	0.2	0.2
r	0.3	0.5	0.2
c	0.1	0.2	0.7

For example, the probability that tomorrow will be rainy given that today is sunny is given by the entry in position (1,2), namely, 0.2.

Define the random variable X_i as the weather on day i after a chosen starting date. Then, the sequence $\{X_0, X_1, \ldots, X_n\}$ is a Markov chain of length n for any positive integers n.

A good way of representing a Markov chain is through a matrix.

Definition 52.70. (The transition matrix)

For a Markov chain $\{X_0, X_1, X_2, \ldots, X_n\}$ with state space $\{s_1, \ldots, s_k\}$ and transition probabilities given by the function p, we construct a matrix P by setting

$$P_{ij} = \mathbb{P}(X_{t+1} = s_j | X_t = s_i) = p_{ij}$$

for all $t \in \{0, 1, \ldots, n-1\}$. We call P the **transition matrix** of the Markov chain.

Note that, to construct the transition matrix, we need to assume a certain ordering of the elements in the state space. Notice also that P is a nonnegative matrix, since

its entries are probabilities and therefore, $0 \leqslant p_{ij} \leqslant 1$ for all i, j. Moreover, each row of P represents the probability distribution of the next state given the current state, so the entries in each row add up to 1. In other words, P is a row stochastic matrix.

Example 52.71 Assuming the state space order is: sunny, rainy, cloudy, the following matrix is the transition matrix associated with the Markov chain in Example 52.69:

$$P = \begin{bmatrix} 0.6 & 0.2 & 0.2 \\ 0.3 & 0.5 & 0.2 \\ 0.1 & 0.2 & 0.7 \end{bmatrix}.$$

For instance, the entry $P(2, 3) = 0.2$ indicates that if the weather is rainy (state 2) at time t, then the probability of it being cloudy (state 3) at time $t + 1$ is 0.2. You can verify that P is row stochastic by checking that the entries in each row add up to 1.

We can also represent each Markov chain with a **transition diagram**. This diagram is a graph where each node represents a state, and we draw a directed edge from node i to node j if the transition probability $P_{ij} > 0$. We assign the weight P_{ij} to that edge.

Example 52.72 Figure 52.7 depicts the transition diagram for the Markov chain from Example 52.71. The weights on the directed edges are the transition probabilities.

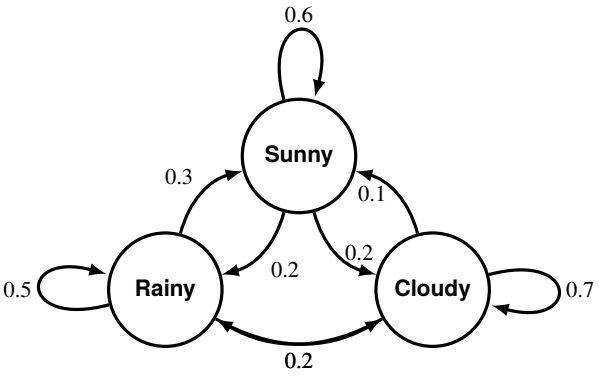


Fig. 52.7: Diagram representing the transition probabilities in a weather model with three states: sunny, rainy, and cloudy.

So far, we have focused on phenomena observed over a finite number of steps or within a limited time period. However, in practice, we may be interested in the long-term behavior of such a phenomenon. For example, in the weather model, we might wonder what fraction of days are sunny over a long period.

The main challenge in extending our finite Markov chain results to the infinite case is defining a probability distribution on the space of infinite sequences

$$\Omega_\infty := \{(x_0, x_1, x_2, \ldots) \mid x_i \in S\}.$$

We can think of Ω_∞ as $S^{\mathbb{N}_0}$, where $\mathbb{N}_0$ denotes the nonnegative integers.

As before, we define the random variable X_n to be the nth component of an infinite sequence $(x_0, x_1, x_2, \ldots)$. It turns out that there is a unique probability measure $\mathbb{P}$ on Ω_∞ that is consistent with the finite-dimensional distributions from the finite case.

These ideas are more advanced than the scope of this book, so we will not go into further details.

Definition 52.73. (Discrete-time Markov chain)

Let S be a finite set with initial probability distribution π^0 and transition probabilities p. Define $\Omega_\infty := S^{\mathbb{N}_0}$. A discrete-time Markov chain is a sequence of random variables $\{X_0, X_1, X_2, \ldots\}$, where each $X_t : \Omega_\infty \to S$ is a coordinate function and the chain satisfies the **Markov condition**. In other words, for all $t \geqslant 0$ and for every choice of states $i_0, i_1, \ldots, i_{t-1}, i, j \in S$, we have

$$\mathbb{P}(X_{t+1} = j \mid X_t = i, X_{t-1} = i_{t-1}, \ldots, X_0 = i_0) = \mathbb{P}(X_{t+1} = j \mid X_t = i),$$

where $\mathbb{P}$ is as in (52.4).

As in the finite case, we define the **transition matrix** P by setting

$$P_{ij} := \mathbb{P}(X_{t+1} = j \mid X_t = i) = p_{ij},$$

for each $i, j \in S$.

We can use the transition matrix of a Markov chain to compute the probability of each state at time t starting from the initial distribution π^0.

Definition 52.74. (The Probability distribution vector)

Let $(X_t)_{t=0}^{\infty}$ be a discrete-time Markov chain with state space $S = \{s_1, s_2, \ldots, s_k\}$. For each time t, define the vector

$$\pi^t := \begin{bmatrix} \pi_1^t \\ \pi_2^t \\ \vdots \\ \pi_k^t \end{bmatrix},$$

where $\pi_i^t = \mathbb{P}(X_t = s_i)$ is the probability that the chain is in state s_i at time t. We call π^t the **probability distribution vector** at time t or probability distribution vector for X_t.

When $t = 0$, the probability distribution vector π^0 represents the initial probability distribution over the state space S. Next, we show how to compute the distribution vector at any time t using the transition matrix and the initial distribution.

Theorem 52.75. (Computing the probability distribution vector)

Let P be the transition matrix of a Markov chain $(X_t)_{t=0}^{\infty}$ with n states, and let π^0 be the probability distribution vector for X_0. Then, the probability distribution vector at time t is given by

$$(\pi^t)^T = (\pi^0)^T P^t.$$

Proof The probability distribution vector $\pi^{t+1} = \begin{bmatrix} \pi_1^{t+1} & \cdots & \pi_n^{t+1} \end{bmatrix}^T$ for X_{t+1} is determined by

$$\pi_j^{t+1} = P_{1j}\pi_1^t + P_{2j}\pi_2^t + \cdots + P_{nj}\pi_n^t,$$

for each j. This equation states that the probability of being in state j at time $t+1$ equals the sum of the probabilities of transitioning from each state i at time t to state j at time $t+1$. In matrix form, we can write this as

$$(\pi^{t+1})^T = (\pi^t)^T P.$$

By applying this recurrence repeatedly, we obtain

$$\begin{aligned} (\pi^1)^T &= (\pi^0)^T P, \\ (\pi^2)^T &= (\pi^1)^T P = (\pi^0)^T P^2, \\ &\vdots \\ (\pi^t)^T &= (\pi^{t-1})^T P = (\pi^0)^T P^t. \end{aligned}$$

This proves the claim. □

Note that if e_i denotes the ith vector in the standard basis for $\mathbb{R}^n$, then

$$e_i^T P^t = \begin{bmatrix} P_{i1}^t & \cdots & P_{in}^t \end{bmatrix},$$

which tells us that P_{ij}^t is the probability that the chain is in state j at time t given that it started in state i at time 0. Thus, the (i, j)th entry of P gives the probability of moving from state i to state j in one step, and the (i, j)th entry of P^t gives the probability of moving from state i to state j in t steps.

Example 52.76 Consider the Markov chain from Example 52.69. Suppose that today's weather is rainy. What is the probability that it will be sunny in two days? To

answer this, we compute the $(2, 1)$ entry of P^2, which is 0.35. This means there is a 35% chance that the weather will be sunny two days from now.

52.6.2 Irreducible and aperiodic Markov chains

In applications, some particular types of Markov chains appear frequently. In the next section, we will study their long-term behavior—and we will do it by applying the nonnegative-matrix tools we have already developed.

Definition 52.77. (Irreducible Markov chain)

We call a Markov chain **irreducible** if its transition diagram is strongly connected. Equivalently, the Markov chain is irreducible if its transition matrix is irreducible.

Example 52.78 The transition diagram in Figure 52.7 is an example of an irreducible Markov chain. Clearly, if all the transition probabilities between any two nodes are positive, then the Markov chain is irreducible. In this case, the transition matrix is positive.

In a Markov chain, the (i, j) entry of P^k tells you the probability of moving from state i to state j in exactly k steps.

When $i = j$, $[P^k]_{ii}$ is the probability of returning to state i in k steps, so any k with $[P^k]_{ii} > 0$ is a possible return time to state i. Equivalently, the transition diagram contains a closed walk of length k from i to itself. Next, we consider all possible return times to define the period of a state.

Definition 52.79. (Period of a state)

The **period** of a state i in a Markov chain is the greatest common divisor of the set

$$\{t \in \mathbb{N} \mid P^t_{ii} > 0\},$$

where P^t_{ii} denotes the (i, i)th entry of P^t.

A state is called **aperiodic** if its period is 1.

A state is aperiodic if the greatest common divisor of cycle lengths is 1. In particular, if $P_{ii} > 0$, then there is a cycle of length 1, so the state is aperiodic.

Example 52.80 Since $P_{ii} > 0$ for each $i = 1, 2, 3$, the three states in the Markov chain shown in Figure 52.7 are aperiodic.

Example 52.81 Consider the Markov chain shown in Figure 52.8. Every state has period 3 because every closed walk has a length that is a multiple of 3, and for each node, there is a closed walk of length 3.

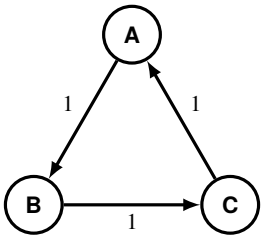


Fig. 52.8: Markov chain with states of period three.

Surprisingly, in an irreducible Markov chain every state has the same period.

Proposition 52.82. (Equal periods in an irreducible Markov chain)

If a Markov chain is irreducible, then all states have the same period.

Proof Because the Markov chain is irreducible, its transition diagram is strongly connected. This means that for any two states i and j, there exist integers t_1 and t_2 such that $P_{ij}^{t_1} > 0$ and $P_{ji}^{t_2} > 0$.

Let d_i and d_j denote the periods of states i and j, respectively. We show that $d_i \leqslant d_j$; by reversing the roles of i and j, we obtain $d_j \leqslant d_i$, so $d_i = d_j$.

Let c denote the number of states. Then, we have

$$P_{ii}^{t_1+t_2} = \sum_{k=1}^{c} P_{ik}^{t_1} P_{ki}^{t_2} \geqslant P_{ij}^{t_1} P_{ji}^{t_2} > 0.$$

Hence, d_i divides $t_1 + t_2$.

Now, let r be any positive integer for which $P_{jj}^{r} > 0$ (such an r exists because the chain is strongly connected). Then, using a similar argument, we obtain

$$P_{ii}^{t_1+t_2+r} \geqslant P_{ij}^{t_1} P_{jj}^{r} P_{ji}^{t_2} > 0.$$

Thus, d_i divides $t_1 + t_2 + r$. Since d_i divides both $t_1 + t_2$ and $t_1 + t_2 + r$, it must divide their difference, namely r. This implies that d_i is a common divisor of all values r

such that $P^r_{jj} > 0$. Hence $d_i \leqslant d_j$, since d_j is the greatest common divisor of those r's.

By symmetry (reversing the roles of i and j), we also have $d_j \leqslant d_i$. Therefore, $d_i = d_j$ for any two states i and j. □

Next, we introduce a special case of an irreducible Markov chain.

Definition 52.83. (Aperiodic Markov chain)

A Markov chain is **aperiodic** if it is irreducible and every state is aperiodic.

In an irreducible Markov chain, finding one aperiodic state guarantees that all states are aperiodic.

Example 52.84 The Markov chain shown in Figure 52.7 is aperiodic.

There is a simple way to test for aperiodicity, which follows immediately from Proposition 52.82.

Proposition 52.85. (Testing if a Markov chain is aperiodic)

Let P be the transition matrix of an irreducible Markov chain. If $P_{ii} > 0$ for some state i, then the Markov chain is aperiodic.

The next theorem shows that a Markov chain is aperiodic precisely when its transition matrix is primitive. This follows directly from Proposition 52.50.

Theorem 52.86. (Aperiodic Markov chain)

A Markov chain is aperiodic if and only if there exists a positive integer t such that $P^t > 0$, where P is the transition matrix of the Markov chain.

52.6.3 Limiting distributions

If we want to predict how a Markov chain behaves as t gets large, we have to look at how the distribution vector π^t changes as $t \to \infty$. Thus, we need to compute the limit of the sequence of vectors $(\pi^t)_{t=0}^{\infty}$, if it exists.

Since vectors are particular cases of matrices, we can use what we have learned before about convergence of sequences of matrices (see Chapter 51).

Example 52.87 Consider the vector

$$\pi = \begin{bmatrix} 1/2 \\ 1 \\ 1/3 \end{bmatrix}.$$

Then,

$$\lim_{t\to\infty} \pi^t = \begin{bmatrix} \lim_{t\to\infty}(1/2)^t \\ \lim_{t\to\infty} 1^t \\ \lim_{t\to\infty}(1/3)^t \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}.$$

By contrast, consider the vector

$$\pi = \begin{bmatrix} 2 \\ 1 \\ 1/3 \end{bmatrix}.$$

It is clear that the sequence $(\pi^t)_{t=0}^{\infty}$ does not converge since $\lim_{t\to\infty} 2^t$ does not exist.

In many cases the entries of π^t may be complicated functions of t, so studying π^t directly is cumbersome. Instead, recall from Theorem 52.75 that $(\pi^t)^T = (\pi^0)^T P^t$. Thus, we have

$$\lim_{t\to\infty} (\pi^t)^T = \lim_{t\to\infty} (\pi^0)^T P^t,$$

and deciding whether π^t converges is the same as deciding whether the sequence $(\pi^0)^T P^t$ converges.

The next example shows that this convergence can depend on the choice of the initial distribution π^0. In fact, when the limit exists, its value may vary with π^0.

Example 52.88 Consider the transition matrix

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

First, we start with the distribution $\pi^{(0)} = [1, 0, 0, 0]^T$. Multiplying by P leaves this vector unchanged, so the sequence is constant and therefore converges to

$$\lim_{t\to\infty} (\pi^{(0)})^T P^t = [1, 0, 0, 0].$$

Next, we choose a different initial distribution $\pi^{(0)} = [0, 1, 0, 0]^T$. Again the vector is unchanged by P, and the sequence converges to

$$\lim_{t\to\infty} (\pi^{(0)})^T P^t = [0, 1, 0, 0].$$

Finally, we take the initial distribution $\pi^{(0)} = [0, 0, 1, 0]^T$. In this case we find

$$(\pi^{(0)})^T P^t = \begin{cases} [0, 0, 1, 0], & t \text{ even}, \\ [0, 0, 0, 1], & t \text{ odd}, \end{cases}$$

so the sequence has no limit.

In many applications the initial distribution is fixed and beyond our control. Even if a Markov chain could converge to different limits for different initial distributions, we care only about the limit associated with the particular initial distribution we have. Example 52.104 illustrates this situation.

In many settings, though, we want the Markov chain's long-term behavior to be independent of the starting distribution π^0. In other words, the chain should eventually "forget" how it started. For example, the long-term fraction of sunny days in a weather model should be the same whether we start tracking the weather on a sunny day or a rainy one. This requirement motivates the next definition.

Definition 52.89. (Limiting probability distribution)

A Markov chain has a **limiting probability distribution** if, for every initial distribution π^0, the limit

$$(\pi^\infty)^T := \lim_{t\to\infty} (\pi^0)^T P^t$$

exists and is the same regardless of π^0.

Because the initial distribution π^0 is constant, we may pass the limit through this vector:

$$\lim_{t\to\infty} (\pi^0)^T P^t = (\pi^0)^T \left(\lim_{t\to\infty} P^t \right).$$

If the limit $\lim_{t\to\infty} P^t$ exists, then the limit of $(\pi^0)^T P^t$ exists for every starting distribution π^0. Conversely, if $(\pi^0)^T P^t$ converges for every π^0, then the rows of P^t converge, forcing P^t itself to converge. Hence a Markov chain has a limiting distribution exactly when the limit of the matrix powers P^t exists and the product

$$(\pi^0)^T \left(\lim_{t\to\infty} P^t \right)$$

is the same regardless of π^0. Notice that the existence of $\lim_{t\to\infty} P^t$ does not ensure the existence of a limiting distribution. Consider, for example, the matrix $P = I$. Then, $(\pi^0)^T \lim_{t\to\infty} I^t = (\pi^0)^T$.

Let us focus first on when $\lim_{t\to\infty} P^t$ exists. We start by collecting a few basic facts about transition matrices.

Properties of transition matrices

For any transition matrix P,

(i) P is non-negative and row stochastic; hence P^t is also row stochastic for every positive integer t.
(ii) The spectral radius of P is 1, and 1 is an eigenvalue of P.
(iii) The vector $e = \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix}^T$ is a positive eigenvector corresponding to the eigenvalue 1.
(iv) Every eigenvalue λ of P satisfies $|\lambda| \leqslant 1$.
(v) If P is irreducible, then 1 has algebraic multiplicity 1; if in addition P is primitive, all other eigenvalues satisfy $|\lambda| < 1$.

To determine when P^t converges as $t \to \infty$, we use a general result, Theorem 51.11, that tells us exactly when the sequence of powers of a square matrix converges. The statement relies on the notion of a regular eigenvalue introduced in Definition 36.20.

For our purposes, the matrix P is a transition matrix. A natural first question is whether $\lim_{t\to\infty} P^t$ always exists. The following example shows that this is not the case.

Example 52.90 Let

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

which is a transition matrix since it is nonnegative and row stochastic.

The eigenvalues of P are 1 and -1, so 1 is not dominant. The matrix is diagonalizable and $P = QDQ^{-1}$ with

$$D = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Hence, $P^t = QD^tQ^{-1}$. Because $D^t = D$ when t is odd and $D^t = I_2$ when t is even, the sequence D^t, and thus P^t, oscillates and does not converge.

For any row-stochastic matrix P (and therefore, for any transition matrix), we know that the spectral radius satisfies $\rho(P) = 1$, and 1 is an eigenvalue. Proposition 52.61 tells us that every eigenvalue on the unit circle is regular. Combining these facts with Theorem 51.11 gives the following result.

Corollary 52.91. (Convergence of powers of transition matrix)

Let P be a row stochastic matrix (or a transition matrix, in particular). Then, $\lim_{t\to\infty} P^t$ exists if and only if 1 is a dominant eigenvalue of P.

Example 52.92 Consider the transition matrix

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

whose transition diagram is shown in Figure 52.9.

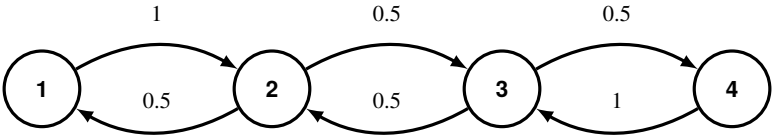


Fig. 52.9: Transition diagram for the Markov chain in Example 52.92.

The matrix P has four distinct eigenvalues: $\lambda \in \{1, -1, 0.5, -0.5\}$. However, 1 is not a dominant eigenvalue since -1 is also an eigenvalue of P and $|1| = |-1|$, violating the condition in Corollary 52.91.

Indeed, notice that P is diagonalizable (since all its eigenvalues are distinct) and $P = QDQ^{-1}$ for some nonsingular matrix Q and

$$D = \text{diag}(1, -1, 0.5, -0.5).$$

Thus, $P^t = QD^tQ^{-1}$, where

$$D^t = \text{diag}(1,\ (-1)^t,\ 0.5^t,\ (-0.5)^t).$$

Notice that the second diagonal entry oscillates between 1 and -1 as t increases. Consequently $\lim_{t\to\infty} D^t$ does not exist, and therefore $\lim_{t\to\infty} P^t$ does not exist either. So this Markov chain lacks a limiting probability distribution.

The previous example shows that some Markov chains lack a limiting probability distribution because $\lim_{t\to\infty} P^t$, where P is the corresponding transition matrix, does not exist. It is natural to ask when this limit is guaranteed to exist. The next result gives sufficient conditions for this to happen.

Corollary 52.93. (Limiting distribution of a primitive transition matrix)

Let P be the transition matrix of a Markov chain. If P is primitive, that is, the chain is irreducible and aperiodic, then $\lim_{t\to\infty} P^t$ exists.

Proof For a primitive matrix, Perron-Frobenius theory tells us that 1 is the unique dominant eigenvalue. Since P is row-stochastic, all other eigenvalues lie strictly inside the unit circle. By Corollary 52.91, this guarantees that $\lim_{t\to\infty} P^t$ exists. □

Recall that, for a primitive (aperiodic and irreducible) row stochastic matrix P, there is some positive integer k with $P^k > 0$. In other words, from any state x_i you can reach any state x_j in exactly k steps with positive probability.

Suppose that P is a primitive row-stochastic matrix. Let $\pi \in \mathbb{R}^n$ be the positive vector satisfying

$$\pi^T P = \pi^T \qquad \text{and} \qquad \pi^T e = 1.$$

Then

$$\lim_{t\to\infty} P^t = e\pi^T.$$

Consequently, for every $x_0 \in \mathbb{R}^n$,

$$\lim_{t\to\infty} P^t x_0 = e\left(\pi^T x_0\right).$$

Thus, the iterates converge to a scalar multiple of the Perron eigenvector e. If $\pi^T x_0 \neq 0$, the normalized iterates converge to the line spanned by e. In particular, if $x_0 \geqslant 0$ and $x_0 \neq 0$, then $\pi^T x_0 > 0$, and

$$\lim_{t\to\infty} \frac{P^t x_0}{\|P^t x_0\|_2} = \frac{e}{\|e\|_2}.$$

The converse of Corollary 52.93 does not hold, as the next example shows.

Example 52.94 Consider the transition matrix

$$P = \begin{bmatrix} 0.5 & 0.5 & 0 & 0 \\ 0.5 & 0 & 0.5 & 0 \\ 0 & 0.5 & 0 & 0.5 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

whose transition diagram appears in Figure 52.10.

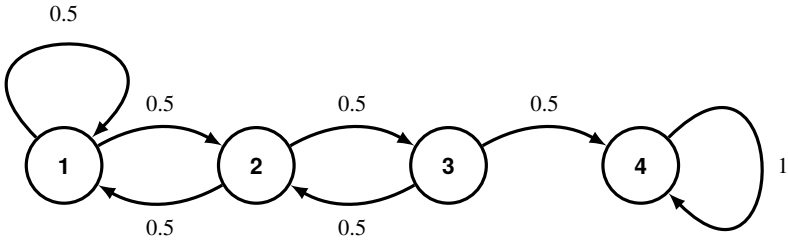


Fig. 52.10: Transition diagram for the Markov chain in Example 52.94.

The eigenvalues of P are approximately $1, 0.901, -0.6235, 0.2225$. Thus, 1 is the unique eigenvalue on the unit circle, and it is simple, so P^t converges as $t \to \infty$. Yet the matrix is clearly reducible (hence not primitive), so the converse of Corollary 52.93 fails.

A direct computation shows

$$\lim_{t\to\infty} P^t = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Consequently, for any initial distribution π^0,

$$(\pi^0)^T \lim_{t\to\infty} P^t = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix},$$

since $(\pi^0)^T e = 1$ because π^0 is a probability vector.

52.6.4 Stationary distributions

So far, we have given the necessary and sufficient conditions for the powers of a transition matrix to converge. Now we turn to the conditions for a limiting distribution to exist, where the limit is independent of the starting distribution.

Let us start with a new concept.

Definition 52.95. (Stationary (equilibrium) distribution)

A probability vector π that satisfies

$$\pi^T P = \pi^T$$

is called a **stationary distribution** (or **equilibrium distribution**) of the transition matrix P.

A stationary distribution is a probability vector that the transition matrix leaves unchanged. After the chain reaches this vector, each new step returns the same vector, so the state probabilities remain fixed.

Notice that a stationary distribution is a nonnegative left-eigenvector of the transition matrix P corresponding to the eigenvalue 1, or equivalently, a nonnegative (right) eigenvector of P^T with eigenvalue 1.

Because every row-stochastic matrix always has 1 as an eigenvalue, so does P^T. Hence, every finite-state Markov chain is guaranteed to possess at least one stationary distribution.

Proposition 52.96. (Existence and uniqueness of a stationary distribution)

Let P be the transition matrix of a (finite-state) Markov chain. There is always at least one stationary distribution π. If the eigenvalue 1 is simple for P (i.e. its algebraic multiplicity is one), then this stationary distribution is unique. In that case π is the Perron eigenvector of P^T.

Proof First note that $\pi^T P = \pi^T$ is equivalent to $P^T\pi = \pi$, that is, a stationary distribution is a right eigenvector of P^T with eigenvalue 1.

Because P^T is column-stochastic, its spectral radius is $\rho(P^T) = 1$. Perron–Frobenius theory for non-negative matrices guarantees that 1 is indeed an eigenvalue of P^T and that there is a non-negative eigenvector x with $P^T x = x$. Normalizing x to sum to one, $\pi = x/\|x\|_1$ gives a stationary distribution.

If 1 is a simple eigenvalue of P, and hence of P^T, then its eigenspace is one-dimensional. Therefore, every eigenvector of P^T corresponding to 1 is a nonzero scalar multiple of x, and every nonnegative eigenvector corresponding to 1 is a positive scalar multiple of x. After normalizing x so that its entries sum to 1, we obtain a unique stationary distribution. □

Example 52.97 For the positive transition matrix P of Example 52.71, an equilibrium state is a probability vector x satisfying $x^T P = x^T$ and $\|x\|_1 = 1$. Solving these conditions yields the unique stationary distribution

$$\pi = \begin{bmatrix} 11/35 \\ 2/7 \\ 2/5 \end{bmatrix}.$$

The following corollary follows immediately from Proposition 52.96 and Perron's Theorem for irreducible nonnegative matrices.

Corollary 52.98. (Uniqueness of the stationary distribution for an irreducible chain)

If P is an irreducible transition matrix, then it has exactly one stationary distribution.

The converse is false: a Markov chain may have a unique stationary distribution even when P is reducible, as Example 52.94 shows.

How are limiting and stationary distributions related? The following theorem shows that whenever a limiting distribution exists, it is necessarily stationary.

Theorem 52.99. (When the limit exists, it is stationary)

Let P be the transition matrix of a Markov chain. Suppose

$$\lim_{t\to\infty} (\pi^0)^T P^t =: x_0^T \quad \text{for some initial distribution } \pi^0$$

exists. Then, x_0 is stationary:

$$(x_0)^T P = (x_0)^T.$$

Proof Fix an arbitrary initial distribution π^0 and assume that

$$(x_0)^T := \lim_{t\to\infty} (\pi^0)^T P^t,$$

exists. If we right-multiply by P, we get

$$(x_0)^T P = \lim_{t\to\infty} (\pi^0)^T P^t P = \lim_{t\to\infty} (\pi^0)^T P^{t+1}.$$

Shifting the index $t \mapsto t+1$ leaves the limit unchanged, so the final expression equals $(x_0)^T$. Hence

$$(x_0)^T P = (x_0)^T,$$

and x_0 is a stationary distribution. □

We are nearly done. If $\lim_{t\to\infty} P^t$ exists, then $\lim_{t\to\infty} (\pi^0)^T P^t$ must be a stationary distribution for P. If we prove that P only has one stationary distribution, then $\lim_{t\to\infty} (\pi^0)^T P^t$ is independent of the initial distribution π^0 and, therefore, it is a limiting distribution.

The next theorem gives sufficient conditions under which a Markov chain both converges to a limit and has a single stationary distribution, so the two are the same. It also identifies that limit as the (normalized) Perron eigenvector of P and provides an explicit formula for $\lim_{t\to\infty} P^t$.

Theorem 52.100. (When the stationary distribution is the limiting distribution)

Let P be the transition matrix of a Markov chain (so P is row-stochastic). Suppose the eigenvalue 1 is dominant and simple. Let π be the Perron eigenvector of P^T. Then

$$\lim_{t\to\infty} P^t = e\,\pi^T, \qquad \text{where } e = [1, \ldots, 1]^T.$$

Consequently, for every initial distribution π^0,

$$\lim_{t\to\infty} (\pi^0)^T P^t = \pi^T.$$

Thus, π is both the unique stationary distribution for P and the limiting distribution of the Markov chain.

Proof Let Q be an invertible matrix such that $P = QJQ^{-1}$, where J is the Jordan form of P. Because 1 is the only eigenvalue on the unit circle and it is simple, J

splits into a 1×1 block $[1]$ and a block $\widetilde{J}$ whose eigenvalues satisfy $|\lambda| < 1$. Hence $J = \operatorname{diag}(1, \widetilde{J})$.

Since every eigenvalue of $\widetilde{J}$ lies strictly inside the unit circle, $\widetilde{J}^t \to 0$ as $t \to \infty$. Hence

$$\lim_{t\to\infty} J^t = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

and

$$B := \lim_{t\to\infty} P^t = Q \left(\lim_{t\to\infty} J^t \right) Q^{-1} = Q \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} Q^{-1},$$

so B has rank 1. Moreover, because each P^t is row-stochastic, we also have

$$Be = \lim_{t\to\infty} P^t e = \lim_{t\to\infty} e = e.$$

So B is itself row-stochastic.

Multiplying B by P gives

$$BP = \lim_{t\to\infty} P^{t+1} = B,$$

which shows that every row of B is a left-eigenvector of P with eigenvalue 1. The eigenvalue 1 is simple, so its left-eigenspace is one-dimensional. Therefore, each row of B must be a scalar multiple of the unique (normalized) Perron vector π^T. Because B is row–stochastic, all rows are also probability vectors, so they must coincide with π^T. Thus $B = e\,\pi^T$, proving the first claim.

Finally, for any initial distribution π^0, we have

$$\lim_{t\to\infty} (\pi^0)^T P^t = (\pi^0)^T \Big(\lim_{t\to\infty} P^t \Big) = (\pi^0)^T (e\,\pi^T) = [(\pi^0)^T e]\,\pi^T = 1 \cdot \pi^T = \pi^T,$$

because (π^0) is a probability vector. Hence every starting distribution converges to the same limit π. □

The next corollary follows directly from Perron-Frobenius theory for primitive matrices together with Theorem 52.100.

Corollary 52.101. (The limiting distribution of a primitive matrix)

Let P be a primitive (i.e., aperiodic) transition matrix of a Markov chain, and let π be the Perron eigenvector of P^T. Then

$$\lim_{t\to\infty} P^t = e\,\pi^T, \qquad \text{with } e = [1, \ldots, 1]^T,$$

and, for every initial probability distribution π^0,

$$\lim_{t\to\infty} (\pi^0)^T P^t = \pi^T.$$

Thus π is the unique stationary and limiting distribution of the Markov chain.

Example 52.102 Let us return to the three-state weather chain of Figure 52.7. In Example 52.97 we found the Perron left-eigenvector of its transition matrix:

$$\pi = \begin{bmatrix} 11/35 \\ 2/7 \\ 2/5 \end{bmatrix} \approx \begin{bmatrix} 0.3143 \\ 0.2857 \\ 0.4000 \end{bmatrix}.$$

Because the chain is irreducible and aperiodic, Theorem 52.100 tells us that this vector is the unique limiting distribution. Interpreting the entries, in the long term we expect about 31% of the days to be sunny, 29% to be rainy, and 40% to be cloudy, no matter what the weather was on the day we started observing.

Corollary 52.101 tells us that an aperiodic Markov chain has a limiting distribution that does not depend on the starting state. This limit equals the normalized Perron eigenvector of P^T and is the chain's unique stationary distribution. The converse does not hold: Example 52.94 gives a chain with a unique limiting distribution even though its transition matrix is not primitive.

Let us close the section with a few examples.

Example 52.103 (City-Suburb Migration Model) People move each year between the city and the surrounding suburbs while the total metropolitan population stays constant. Historical data give the annual transition matrix

$$P = \begin{bmatrix} 0.90 & 0.10 \\ 0.02 & 0.98 \end{bmatrix},$$

where rows represent current location (city, suburbs, respectively) and columns represent location next year. For instance, P_{12} says that a city resident has a 10% chance of living in the suburbs one year later.

Figure 52.11 shows the transition diagram for this Markov chain.

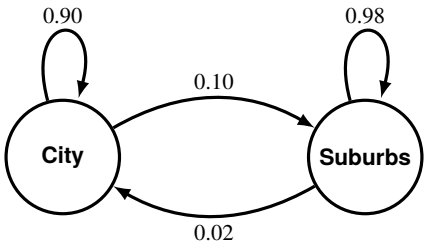


Fig. 52.11: Transition diagram for the probability of migrating from the city to the suburbs and vice versa.

Assume that on 1 Jan 2025, 70% of the population lives in the city and 30% in the suburbs, giving the initial vector $\pi^0 = [0.7,\ 0.3]^T$. What fractions of the population

should we expect for Jan 1, 2026 and Jan 1, 2027 to live in the city and the suburbs? After one year,

$$(\pi^0)^T P = \begin{bmatrix}0.7 & 0.3\end{bmatrix} \begin{bmatrix}0.90 & 0.10\\ 0.02 & 0.98\end{bmatrix} = \begin{bmatrix}0.636 & 0.364\end{bmatrix},$$

so roughly 63.6% of residents will be in the city and 36.4% in the suburbs by Jan 1, 2026. Applying P once more,

$$(\pi^0)^T P^2 = \begin{bmatrix}0.636 & 0.364\end{bmatrix} P = \begin{bmatrix}0.580 & 0.420\end{bmatrix}.$$

Thus, by Jan 1, 2027 about 58% will live in the city and 42% in the suburbs.

Note that the population living in the city decreased two years in a row. If these migration rates stay unchanged, will the city keep losing residents, or does the system settle to a stable split? In other words, what happens to the chain in the long run?

Since P is a positive matrix, it is primitive. Perron-Frobenius theory and Theorem 52.100 guarantee a unique limiting distribution π that satisfies $\pi^T P = \pi^T$. Solving $\pi^T P = \pi^T$ with $\pi_1 + \pi_2 = 1$ and $\pi_1, \pi_2 \geqslant 0$ gives

$$\pi = \begin{bmatrix}1/6\\ 5/6\end{bmatrix}.$$

Thus, if the migration pattern remains the same, the model predicts that in the long run roughly 16.7% of residents will live in the city and 83.3% in the suburbs, regardless of the starting proportions.

In the next example the transition matrix P is reducible. Although 1 is the only eigenvalue on the spectral circle, its geometric multiplicity is 2. Thus, P has several stationary distributions, and $\lim_{t\to\infty} P^t$ exists. However, the resulting long-run distribution may depend on the initial distribution. This does not create an issue in the example since the initial distribution is fixed by the context of the problem.

Example 52.104. (Graduation rates)

A local judo dojo runs a two-year beginners' course for children before they move on to competition training. To judge whether this program is profitable, the staff collect data on how pupils progress. Each child falls into one of four categories: Year 1 (Alevin), Year 2 (Benjamin), graduate (Master), or quitter. Parents may let a child repeat Year 1 or Year 2, so a pupil can remain an Alevin or Benjamin for more than one season.

The dojo's data reveal the following yearly pattern: 40% of Benjamins graduate and become Masters, 30% stay Benjamins for another season, and the remaining 30% leave the program. Among Alevins, 50% advance to Benjamin, 20% repeat Alevin, and 30% drop out. Once a child quits, they never return.

We can model the dojo's data with a four-state Markov chain whose states are Alevin, Benjamin, Master, and Quit. Figure 52.12 shows the corresponding transition diagram.

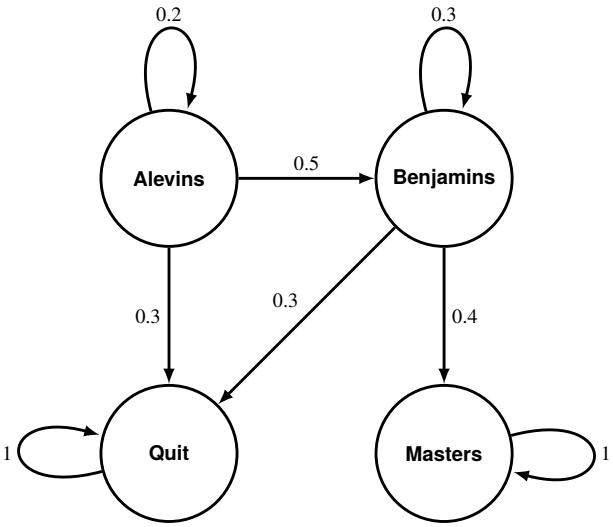


Fig. 52.12: Transition diagram for the Judo dojo 2-year program.

The dojo also knows the current makeup of its beginners' cohort: 50% are Alevins (state 1), 35% are Benjamins (state 2), and the remaining 15% have already quit; no one has graduated yet because the program began only last year.

Assuming the observed transition rates remain unchanged, the school wants to determine:

1. the percentages of this cohort who, by next autumn, will be Alevins, Benjamins, Masters, or quitters;
2. the same percentages two years from now;
3. the probability that a student in today's cohort will eventually graduate.

The transition matrix is

$$P = \begin{bmatrix} 0.20 & 0.50 & 0 & 0.30 \\ 0 & 0.30 & 0.40 & 0.30 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Notice the entries $P_{33} = 1$ and $P_{44} = 1$. A student who has reached the Master state remains a Master next year, and anyone who has quit remains out of the program; neither group can revert to Alevin or Benjamin. Thus we are assuming graduates and drop-outs stay in those states permanently.

The initial distribution vector is $\pi^0 = [0.5,\ 0.35,\ 0,\ 0.15]^T$, reflecting that no student has graduated yet.

To answer the first question, we multiply π^0 by the transition matrix:

$$(\pi^0)^T P = [0.1,\ 0.355,\ 0.14,\ 0.405].$$

Thus, by next autumn about 10% of today's students will still be Alevins, 35% will be Benjamins, 14% will have graduated, and 41% will have left the program.

For the second question, we compute $(\pi^0)^T P^2$:

$$(\pi^0)^T P^2 = [0.02,\ 0.1565,\ 0.282,\ 0.5415].$$

So two years from now we expect roughly 2% Alevins, 16% Benjamins, 28% Masters, and 54% quitters.

To answer the third question we need to compute

$$\lim_{t\to\infty} (\pi^0)^T P^t,$$

the long-term distribution of today's students. Although P is reducible, it is diagonalizable and has 1 as its only eigenvalue on the unit circle (algebraic multiplicity 2, geometric multiplicity 2). We can factor P as $P = QDQ^{-1}$ with

$$D = \operatorname{diag}(0.2,\ 0.3,\ 1,\ 1) \qquad \text{and} \qquad Q = \begin{bmatrix} 1 & 0.98 & 0.29 & 0.51 \\ 0 & 0.2 & 0.47 & 0.34 \\ 0 & 0 & 0.83 & 0 \\ 0 & 0 & 0 & 0.79 \end{bmatrix}$$

Because the first two diagonal entries lie inside the unit circle, D^t tends to

$$\lim_{t\to\infty} D^t = \begin{bmatrix} 0&0&0&0 \\ 0&0&0&0 \\ 0&0&1&0 \\ 0&0&0&1 \end{bmatrix}.$$

Consequently

$$\lim_{t\to\infty} P^t = Q \begin{bmatrix} 0&0&0&0 \\ 0&0&0&0 \\ 0&0&1&0 \\ 0&0&0&1 \end{bmatrix} Q^{-1} \approx \begin{bmatrix} 0&0&0.36&0.64 \\ 0&0&0.57&0.43 \\ 0&0&1&0 \\ 0&0&0&1 \end{bmatrix}.$$

Multiplying by the initial vector π^0 gives

$$\lim_{t\to\infty} (\pi^0)^T P^t \approx [0,\ 0,\ 0.379,\ 0.621].$$

Thus, about 38% of the current cohort will eventually graduate, while roughly 62% will leave before finishing, a plausible outcome for a group of very young students.

The eigenvalue 1 has multiplicity 2, so different initial vectors lead to different limits. That is fine here: we know the specific starting distribution π^0, so the limit we just computed is the one that matters.

Problems

52.1 Let $A, B \in \mathbb{R}^{m\times n}$ and let $\alpha \in \mathbb{R}$. Show that

(a) if $\alpha \geqslant 0$, then $\alpha A \geqslant 0$;
(b) $A + B \geqslant 0$ if $A, B \geqslant 0$.

52.2 Let $A > 0$ and $B > 0$ be matrices of appropriate size, and let $\alpha, \beta > 0$. Show that

(a) $\alpha A > 0$.
(b) $A + B > 0$.
(c) $\alpha A + \beta B > 0$.
(d) $AB > 0$.
(e) $A^m > 0$ for $m = 1, 2, \ldots$.

52.3 Show these basic order-preserving properties of nonnegative matrices:

(a) If $A \geqslant 0$ and $x, y \geqslant 0$ with $x \geqslant y \geqslant 0$, then $Ax \geqslant Ay$.
(b) If furthermore $A \geqslant 0$ has no zero rows and $x > y > 0$, then $Ax > Ay$.

52.4 Find the spectral radius of

$$A = \begin{bmatrix} 9 & -1 & 2 \\ -2 & 8 & 4 \\ 1 & 1 & 8 \end{bmatrix}.$$

Then, for at least three different matrix norms, plot $\|A^k\|^{1/k}$ as k increases.

52.5 Consider the matrix

$$\begin{bmatrix} 7 & -16 & 8 \\ -16 & 7 & -8 \\ 8 & -8 & -5 \end{bmatrix}.$$

Use Proposition 52.25 to estimate its spectral radius. Then compare your estimate with the actual spectral radius (you may use a computer to calculate it).

52.6 Let

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 2 & 4 & 5 \\ 0 & 1 & 3 \end{bmatrix}.$$

(i) Compute the row/column-sum bounds on $\rho(A)$.
(ii) Apply similarity with $P = \operatorname{diag}(3, 1, 3)$ and bound $\rho(A)$ using $\|PAP^{-1}\|_\infty$. Compare to the true value.

52.7 For the matrix A in Problem 52.6, pick $P = \operatorname{diag}(1, 3, 6)$ and use the diagonal-similarity bounds to produce a *lower* bound on $\rho(A)$. Compare to the true value.

52.8 Let $A \in \mathbb{R}^{n\times n}$ be a matrix and let P be an $n \times n$ nonsingular matrix. Show that $\rho(PAP^{-1}) = \rho(A)$.

52.9 Let $A \in \mathbb{R}^{m\times n}$. Show that $\|A\|_2 = \sqrt{\rho(A^TA)}$. If we further assume that A is square and symmetric, prove that $\|A\|_2 = \rho(A)$.

52.10 Let A and B be square matrices with $A > B > 0$. Show that $\rho(A) > \rho(B)$.

52.11 Let A and B be square matrices with $AB = BA$. Prove that $\rho(AB) \leqslant \rho(A)\,\rho(B)$.

Hint: Use Gelfand's formula (52.1) together with the submultiplicativity of the 2-norm.

52.12 What is the spectral radius of an orthogonal matrix Q (that is, a matrix with $Q^TQ = QQ^T = I_n$)? What is the spectral radius of a nilpotent matrix A (that is, a matrix with $A^p = 0$ for some p)?

52.13 Give an example of a nonzero matrix with spectral radius zero. Conclude that the spectral radius is not a matrix norm.

52.14 Let A be a real $n \times n$ matrix such that A is diagonally dominant with positive diagonal entries, i.e.,

$$A_{ii} > \sum_{j\neq i} |A_{ij}|, \quad \text{for all } i = 1, \ldots, n.$$

If $A \geqslant 0$, prove that $\rho(A) > 0$.

52.15 In Proposition 52.22 we showed that if $0 \leqslant A \leqslant B$, then $\rho(A) \leqslant \rho(B)$. Give a concrete example where $0 \leqslant A < B$ but $\rho(A) = \rho(B)$. Explain briefly why the inequality does not have to be strict.

52.16 (Perron's Theorem for 2×2 matrices) In this problem, you are going to prove Perron's theorem for 2×2 positive matrices without using the general $n \times n$ case. Let

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad a, b, c, d > 0.$$

Below, we propose some intermediate results to prove.

(a) Show that A has two distinct real eigenvalues.
(b) Assume $\lambda > 0$ is an eigenvalue of A with eigenvector $v = [v_1, v_2]^T$. Prove that $v_1 \neq 0$ and $v_2 \neq 0$.
(c) Deduce that A must have at least one positive eigenvalue.
(d) Finally, combine (a)-(c) to conclude that A's largest eigenvalue is simple and admits a strictly positive eigenvector.

52.17 (Collatz-Wielandt formula) Let $A > 0$ and let $\rho = \rho(A)$. For any $x > 0$, set

$$\alpha(x) = \max_i \frac{(Ax)_i}{x_i}, \qquad \beta(x) = \min_i \frac{(Ax)_i}{x_i}.$$

Then prove that for all $x > 0$,

$$\beta(x) \leqslant \rho \leqslant \alpha(x),$$

and

$$\rho = \min_{x>0} \alpha(x) = \max_{x>0} \beta(x).$$

Equality holds if and only if x is a Perron eigenvector of A.

Hint: A positive matrix A has a Perron vector $x > 0$ with $Ax = \rho x$. It also has a left Perron vector $y > 0$ with $y^T A = \rho\, y^T$.

52.18 Construct a 3×3 nonnegative matrix A such that: (i) A is reducible; (ii) $\rho(A) > 0$; and (iii) every eigenvector for $\rho(A)$ has at least one zero entry (hence is not strictly positive). Explain why this does not contradict Perron-Frobenius.

52.19 Let B be a nonnegative and primitive matrix whose spectral radius is 1.

(a) Prove that the limit $L := \lim_{k\to\infty} B^k$ exists.

Hint: Use the Perron-Frobenius Theorem (Theorem 52.38) and Theorem 37.18)

(b) Show that $L = \dfrac{xy^T}{y^T x}$, where x is a right eigenvector ($Bx = x$) and y^T is a left eigenvector ($y^T B = y^T$) for the eigenvalue 1. Interpret L as the projection onto the eigenspace associated with the eigenvalue $\lambda = 1$.

(c) Show that L is positive.

(d) Using the formal definition of a matrix limit, argue that there must exist some finite integer k for which the matrix B^k is also strictly positive.

Hint: The definition of a limit means that for any $\varepsilon > 0$, there exists an integer k_0 such that for all $k \geqslant k_0$, every entry satisfies $|(B^k)_{ij} - L_{ij}| < \varepsilon$. Let $\ell_{\min}$ be the smallest (and positive) entry in L. What happens if you choose $\varepsilon = \ell_{\min}/2$?

52.20 Let ϵ be any positive real number. Suppose A is an $n \times n$ irreducible nonnegative matrix. Show that $(\epsilon I + A)^{n-1} > 0$.

52.21 Show that, if A is an $n \times n$ irreducible nonnegative matrix with positive main diagonal entries, then $A^{n-1} > 0$.

52.22 Prove that if a nonnegative matrix A is reducible, then for any integer $k \geqslant 1$, A^k is also reducible.

52.23 Let

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

(a) Is A irreducible?

(b) Find the smallest k for which $(I + A)^k$ is positive (all entries > 0).

52.24 Show that an irreducible nonnegative matrix A with $\rho = \rho(A)$ is primitive if and only if

$$\lim_{k\to\infty} \left(\frac{A}{\rho}\right)^k$$

exists.

52.25 Give an example of a nonnegative, irreducible matrix A that is not primitive, and briefly explain why it fails to be primitive.

52.26 Let A and B be $n \times n$ nonnegative matrices with $A \leqslant B$ and $A \neq B$. Assume that A is primitive. Prove that $\rho(A) < \rho(B)$.

52.27 Consider the block matrix:

$$A = \begin{pmatrix} 0 & B \\ C & 0 \end{pmatrix}$$

where B and C are nonnegative matrices. Provide a necessary and sufficient condition on B and C for A to be irreducible.

52.28 This problem looks at how the structure of a matrix (reducible vs. irreducible) affects the stability of its eigenvectors.

Consider

$$A_\epsilon = \begin{bmatrix} 1 & \epsilon \\ 0 & 1 \end{bmatrix}, \qquad \epsilon \geqslant 0.$$

(a) Show that the eigenspace for $\rho(A_\epsilon)$ has dimension 1 when $\epsilon > 0$, but dimension 2 when $\epsilon = 0$.

Now look at a similar matrix that is always irreducible (in fact, positive):

$$B_\delta = \begin{bmatrix} 1 & 1 \\ \delta & 1 \end{bmatrix}, \qquad \delta > 0.$$

(b) Show that B_δ is **irreducible**.
(c) Find the eigenvalues of B_δ and show that the Perron root is $\rho(B_\delta) = 1 + \sqrt{\delta}$.
(d) Find a Perron eigenvector v_δ for $\rho(B_\delta)$. (You may set the first component equal to 1.)
(e) Compute $\lim_{\delta \to 0^+} v_\delta$ and show that this limit is a specific, well-defined vector.

Notice the contrast: for the *reducible* matrix A_ϵ, the eigenspace dimension "jumps" from 2 at $\epsilon = 0$ to 1 for $\epsilon > 0$. For the *irreducible* matrix B_δ, the Perron eigenvector varies smoothly and converges to a single limit as $\delta \to 0^+$. This shows that *irreducibility is what guarantees stability* of the Perron eigenvector.

52.29 (The power method converges to the Perron vector) Let A be a primitive matrix and let x be any nonnegative, nonzero vector. Show that

$$\lim_{k \to \infty} \frac{A^k x}{\|A^k x\|_1} = v,$$

where v is the Perron vector of A.

52.30 Let A be a primitive matrix with spectral radius $\rho(A)$ and let $v > 0$ be the corresponding eigenvector normalized so that $\|v\|_1 = 1$. Prove that there exist constants $C > 0$ and $0 < \lambda < 1$ such that for any vector $x > 0$,

$$\left\| \frac{A^k x}{\|A^k x\|_1} - v \right\|_1 \leqslant C\lambda^k \quad \text{for all } k \geqslant 1.$$

52.31 (Denseness of primitive matrices) Let A be an irreducible nonnegative matrix. Show that for every $\epsilon > 0$ there exists a nonnegative matrix E with $\|E\| < \epsilon$ (with respect to any matrix norm) such that $A + E$ is primitive.

52.32 Let A be an $n \times n$ nonnegative matrix. Prove that if A has a nonnegative eigenvector $v \geqslant 0$ for its spectral radius $\rho(A)$ with at least one zero entry, then A must be reducible.

52.33 Let A be an irreducible nonnegative matrix that is not primitive (i.e., the digraph G_A associated with A has period $d > 1$). Show that there exists a permutation matrix P such that

$$P^T A P = \begin{bmatrix} 0 & A_1 & 0 & \cdots & 0 \\ 0 & 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & A_{d-1} \\ A_d & 0 & 0 & \cdots & 0 \end{bmatrix},$$

where each A_i is a nonnegative block (possibly rectangular). Explain how this cyclic block form reflects the period d of G_A.

52.34 Construct an irreducible nonnegative matrix A with at least one zero entry, then form the positive matrix A_δ by adding a small $\delta > 0$ to each zero entry of A, and explain how its spectral radius $\rho(A_\delta)$ behaves relative to $\rho(A)$ as $\delta \to 0$.

52.35 Given $p \in \mathbb{R}^n$ with $p > 0$, and $\lambda > 0$, construct $A > 0$ such that p is the Perron vector of A and λ is its Perron root (i.e., $\lambda = \rho(A)$).

52.36 Let P be an $n \times n$ nonnegative, row-stochastic, irreducible matrix. Prove that $\operatorname{rank}(I_n - P) = n - 1$.

52.37 Let P be a row stochastic matrix. Let v be a vector such that $Pv = v$. Show that if P is irreducible, then v must be a constant vector (i.e., all its components are equal).

52.38 Let P be a doubly stochastic transition matrix (i.e., both the rows and columns sum to 1). Prove that the uniform distribution $u = \left(\frac{1}{n}, \frac{1}{n}, \ldots, \frac{1}{n}\right)^T$ is a stationary distribution of P. Under what additional conditions is u the unique stationary distribution?

52.39 A simple weather model uses a Markov chain with two states: **Sunny** (S) and **Rainy** (R). Suppose the transition matrix is

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix},$$

where the rows correspond to the current day's weather and the columns to the next day's weather.

(a) Find the limiting distribution for this weather model.
(b) Interpret the limiting distribution in terms of long-term weather patterns.

52.40 A retail store classifies customers into **Browsers** (B), **Buyers** (U) and an **Exiter** (E) with transition matrix

$$P = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.1 & 0.7 & 0.2 \\ 0 & 0 & 1 \end{bmatrix}.$$

(a) Determine the long-term behavior of customers starting as **Browsers**.
(b) What is the probability that a customer eventually becomes an **Exiter**?

52.41 A financial institution models borrowers' credit ratings as a Markov chain with three states, **Good** (G), **Average** (A), and **Poor** (P), and transition matrix

$$P = \begin{bmatrix} 0.85 & 0.10 & 0.05 \\ 0.15 & 0.70 & 0.15 \\ 0.10 & 0.20 & 0.70 \end{bmatrix}.$$

(a) Find the limiting distribution of this chain.
(b) Explain how the institution could use that distribution to evaluate long-term credit risk.

52.42 Consider a model for population migration among four regions with transition matrix

$$P = \begin{bmatrix} 0.7 & 0.1 & 0.1 & 0.1 \\ 0.2 & 0.6 & 0.1 & 0.1 \\ 0.1 & 0.2 & 0.6 & 0.1 \\ 0.1 & 0.1 & 0.2 & 0.6 \end{bmatrix}.$$

(a) Find the limiting distribution of this Markov chain.
(b) Explain what it tells us about the long-term population balance among the four regions.

52.43 A health department models disease spread with a Markov chain on three states: **Susceptible** (S), **Infected** (I), and **Recovered** (R). The transition matrix is

$$P = \begin{bmatrix} 0.9 & 0.1 & 0 \\ 0.2 & 0.5 & 0.3 \\ 0 & 0 & 1 \end{bmatrix}.$$

If everyone starts in state S, what is the probability that an individual eventually reaches state R?

52.44 A Markov chain models individuals' health with two states: **Healthy** (H) and **Infected** (I). Initially, the transition matrix is

$$P = \begin{bmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{bmatrix},$$

and after a vaccination campaign it becomes

$$Q = \begin{bmatrix} 0.98 & 0.02 \\ 0.05 & 0.95 \end{bmatrix}.$$

(a) Compute and compare the limiting distributions of P and Q.
(b) Discuss how the change in those distributions reflects the vaccination campaign's effectiveness.

52.45 A sociologist models social mobility (the movement of a person from one social status to another) with a Markov chain on three classes: **Low** (L), **Middle** (M), and **High** (H). The transition matrix is

$$P = \begin{bmatrix} 0.80 & 0.15 & 0.05 \\ 0.10 & 0.80 & 0.10 \\ 0.05 & 0.15 & 0.80 \end{bmatrix}.$$

(a) Compute the limiting distribution of this chain.
(b) Explain what the limiting distribution tells us about long-term social mobility trends.

52.46 For each of the following transition matrices P

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1/2 & 1/2 \\ 0 & 1/2 & 1/2 \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/5 & 3/10 & 1/2 \end{bmatrix},$$

determine whether

(a) the limit $\lim_{t\to\infty} P^t$ exists;
(b) the associated Markov chain has a unique stationary distribution;
(c) the associated Markov chain has a limiting distribution.

52.47 A simple model of a video game has three states: **Playing (P)**, **Paused (S)**, and **Game Over (G)**. The transitions between states happen every minute.

- From **Playing**, there is a 60% chance of continuing to play, a 20% chance of pausing, and a 20% chance of the game being over.
- From **Paused**, there is a 70% chance of resuming play and a 30% chance of remaining paused.
- **Game Over** is an absorbing state; once entered, it cannot be left.

(a) Write down the transition matrix for this Markov chain.
(b) Is the Markov chain irreducible? Explain why or why not.
(c) If a player starts in the "Playing" state, what is the probability that the game is not over after 2 minutes?

52.48 (Leslie matrix model) A certain species of bird lives at most three years. We divide the population into three age groups: 0-1 year, 1-2 years, and 2-3 years. The population from one year to the next can be described with the matrix

$$L = \begin{bmatrix} 0 & 1.5 & 0.8 \\ 0.5 & 0 & 0 \\ 0 & 0.7 & 0 \end{bmatrix}.$$

The entries show how many offspring each age group produces (first row) and how likely individuals are to survive to the next age group (sub-diagonal entries).

(a) Explain the meaning of the entry $L_{12} = 1.5$.
(b) If the initial population vector is $x_0 = \begin{bmatrix}100 & 80 & 50\end{bmatrix}^T$, what is the population distribution after two years?
(c) What is the long-term growth rate of this bird population?

52.49 Imagine a small social network with four individuals: Alex, Beth, Charles, and Diana. We can model their interactions using a directed graph, where an edge from person A to person B means that A follows B. The adjacency matrix for this network is:

$$A = \begin{bmatrix}0 & 1 & 1 & 0\\ 1 & 0 & 1 & 1\\ 0 & 1 & 0 & 1\\ 1 & 0 & 1 & 0\end{bmatrix},$$

where the rows and columns correspond to Alex, Beth, Charles, and Diana, respectively.

A simple way to measure the influence of a person in a network is to count how many walks of a certain length originate from them.

(a) Calculate A^2 and A^3.
(b) What does the entry $(A^2)_{13}$ represent in the context of this social network?
(c) Using the matrix $A+A^2$, determine who is the most influential person in the network, considering walks of length 1 and 2.

The Perron-Frobenius theorem can be applied to find the "eigenvector centrality" of each person, which is a more sophisticated measure of influence.

(d) Show that the matrix A is irreducible.
(e) Find the dominant eigenvalue of A and its corresponding eigenvector.
(f) Normalize the eigenvector so that its components sum to 1. This gives the eigenvector centrality for each person. Who is the most central person according to this measure?

52.50 (U.S. state-to-state migration) In this problem, you will use real data on migration within the United States to model population movement as a Markov chain. The goal is to predict the long-term distribution of the population across states. We will use the "Migration Flows Across U.S. States (2010–2022)" dataset from Kaggle. This dataset, based on U.S. Census Bureau data, reports state-to-state migration. Use the data from a single year (for example, 2022) to build your model.

(a) Construct a 51×51 matrix (for the 50 states plus the District of Columbia) where M_{ij} is the number of people who moved from state i to state j in that year. The diagonal entries M_{ii} represent the number of people who stayed in the same state. You can compute these by taking the total state population and subtracting the number of people who moved out.
(b) Turn M into a stochastic transition matrix by normalizing each row. Divide each entry in a row by the sum of that row. The resulting matrix P has entries P_{ij} giving the probability that a person from state i moves to state j in the next year. The matrix P is a *row-stochastic nonnegative matrix*.

(c) Is *P irreducible*? In this setting, what would it mean if the matrix were reducible?

Hint: Can someone eventually move from any state to any other state, perhaps over multiple years?

(d) Find the stationary distribution of the Markov chain with transition matrix P.

(e) The stationary distribution vector shows the long-term proportion of the U.S. population in each state if the migration patterns of your chosen year continue indefinitely. Which states are projected to have the largest populations in the long run? How does this compare with the current population distribution?

Chapter 53
XI-SUPPLEMENTARY TOPICS

53.1 Number of walks of length k in a graph

As we recalled in Chapter 50, for an undirected (weighted or unweighted) simple graph G, the adjacency matrix A_G records the connectivity between the nodes of G. Adjacency matrices of undirected graphs are symmetric, nonnegative matrices.

As we saw in Chapter 37, if $\lambda_1, \ldots, \lambda_n$ are the eigenvalues of an $n \times n$ matrix A, then $\lambda_1^k, \ldots, \lambda_n^k$ are the eigenvalues of A^k. The trace of a square matrix (the sum of its diagonal entries) equals the sum of its eigenvalues. For the adjacency matrix A_G of a simple undirected graph G, this gives:

1. The eigenvalues $\lambda_1, \ldots, \lambda_n$ of A_G sum to zero, since the main diagonal entries of A_G are all zero (simple graphs have no loops).
2. $\text{trace}(A_G^k) = \lambda_1^k + \cdots + \lambda_n^k$ counts the number of closed walks of length k in G. This is because the (i,i) entry of A_G^k counts the number of walks of length k starting and ending at node i (Theorem 9.3 in Chapter 9), and summing over i counts all such walks in the graph.

We have seen that the sum of the diagonal entries of A_G^k gives the number of closed walks of length k in G. Similarly, since $(A_G^k)_{ij}$ counts the number of walks of length k from node i to node j, summing over all i and j gives the total number of walks of length k (closed or not). If e is the vector of all ones, this total is

$$\text{number length-}k\text{ walks} = \sum_{i,j} (A_G^k)_{ij} = e^T A_G^k\, e.$$

When k is large, computing A_G^k directly is not practical. Instead, we will see how to estimate the number of walks of length k in a graph G with n nodes using the eigenvalues and eigenvectors of A_G. Our estimate assumes the graph is connected and non-bipartite, so that A_G is a primitive matrix and Perron's theorem applies.

An undirected graph is **connected** if there is a walk between any two nodes. For undirected graphs, this is equivalent to being strongly connected, although this term is usually reserved for directed graphs. In this case, the adjacency matrix is a nonnegative irreducible matrix (Proposition 52.45).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_53

A **bipartite** graph is one whose nodes can be split into two disjoint sets so that no two nodes within the same set are adjacent. Figure 53.1 shows a bipartite graph with five nodes. No two blue nodes are connected, and the same holds for the red nodes.

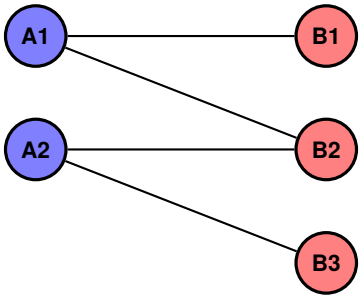


Fig. 53.1: A bipartite graph with five nodes. Blue nodes are not connected to each other, and the same is true for the red nodes.

Let us illustrate a property of bipartite graphs with an example.

Example 53.1. (Adjacency matrices of bipartite graphs are not primitive)

Consider the bipartite graph G in Figure 53.1. With the nodes ordered as A1, A2, B1, B2, B3, the adjacency matrix of this bipartite graph is

$$A_G = \left[\begin{array}{cc|ccc} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ \hline 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{array}\right] =: \begin{bmatrix} 0 & X \\ X^T & 0 \end{bmatrix}$$

Squaring and cubing gives

$$A_G^2 = \begin{bmatrix} XX^T & 0 \\ 0 & X^TX \end{bmatrix} \quad \text{and} \quad A_G^3 = \begin{bmatrix} 0 & XX^TX \\ X^TXX^T & 0 \end{bmatrix}.$$

By induction, even powers of A_G are always block diagonal like A_G^2, and odd powers have the form $\left[\begin{smallmatrix} 0 & Y \\ Y^T & 0 \end{smallmatrix}\right]$. If A_G were primitive, some power would be strictly positive, which never happens here. Thus, the adjacency matrix of any undirected, simple, bipartite graph is not primitive.

The observation from the previous example can be stated formally.

Lemma 53.2. (When the adjacency matrix of a connected graph is primitive)

Let G be a simple, connected graph with at least two nodes. The adjacency matrix of G is primitive if and only if G is non-bipartite.

Proof Let A_G be the adjacency matrix of a bipartite graph G. By the bipartite property, and with a suitable ordering of the nodes, A_G takes the block form $\left[\begin{smallmatrix} 0 & Y \\ Y^T & 0 \end{smallmatrix}\right]$ for some nonzero matrix Y. Induction shows that $(A_G^{2k+1})_{ii} = 0$ for all k and all i, so A_G cannot be primitive. If it were primitive, Proposition 52.48 would guarantee $A_G^t > 0$ for some t, which is not the case.

Now assume G is connected and non-bipartite. An undirected graph is bipartite if and only if all its cycles have even length (Problem 53.1). Since G is non-bipartite, it contains an odd cycle. Moreover, in the directed graph associated with A_G, every undirected edge of G gives rise to two oppositely directed edges, and hence to a directed cycle of length 2. Therefore, the associated digraph contains cycles whose lengths have greatest common divisor 1. By Proposition 52.50, A_G is primitive. □

Let us now show how to use the Perron root of the adjacency matrix of a simple, connected, non-bipartite graph to estimate the number of walks of length k.

Proposition 53.3. (Estimating the number of walks of fixed length)

Let G be a simple, connected, non-bipartite graph with at least two vertices, and let A_G be its adjacency matrix. Let $\lambda = \rho(A_G)$ be the Perron root of A_G (i.e., the largest eigenvalue in absolute value and the only one lying on the spectral circle). Let u be the unique positive eigenvector associated with λ, normalized to have unit length. If $e = [1, \ldots, 1]^T$ is the all-ones vector, set

$$c_{\max} := e^T u > 0.$$

Then, for large k, the total number of walks of length k in G satisfies

$$e^T A_G^k \, e \approx c_{\max}^2 \, \lambda^k,$$

and the relative error goes to zero as $k \to \infty$.

Proof Let $n \geqslant 2$ be the number of vertices of G. Since A_G is symmetric, by the Spectral Theorem (Theorem 41.3) it has real eigenvalues $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_n$, and an orthonormal basis of eigenvectors $\{u_1, \ldots, u_n\}$ for $\mathbb{R}^n$. Since G is connected and non-bipartite, A_G is primitive by Lemma 53.2. By the Perron-Frobenius theorem, $\lambda_1 = \rho(A_G)$, $\lambda_1 > |\lambda_i|$ for $i > 1$, and we can take $u_1 = u$ as the associated positive eigenvector.

We can write the all-ones vector e as

$$e = \sum_{i=1}^{n} c_i u_i,$$

where $c_i = e^T u_i$. In particular, $c_1 = e^T u_1 = c_{\max}$. The scalar $c_{\max} = e^T u$ is positive since every entry of the eigenvector u is positive.

The number of walks of length k in G is

$$
\begin{aligned}
e^T A_G^k e &= \left(\sum_{i=1}^{n} c_i u_i^T \right) A_G^k \left(\sum_{j=1}^{n} c_j u_j \right) \\
&= \left(\sum_{i=1}^{n} c_i u_i^T \right) \left(\sum_{j=1}^{n} c_j \lambda_j^k u_j \right) \\
&= \sum_{i=1}^{n} c_i^2 \lambda_i^k,
\end{aligned}
$$

where the second equality uses that u_j is an eigenvector of A_G^k with eigenvalue λ_j^k, and the third equality uses orthonormality of the eigenvectors: $u_i^T u_j = 0$ for $i \neq j$ and $u_i^T u_i = 1$.

Therefore,

$$
\lim_{k \to \infty} \frac{e^T A_G^k \, e}{\lambda_1^k} = \lim_{k \to \infty} \frac{\sum_{i=1}^{n} c_i^2 \lambda_i^k}{\lambda_1^k} = c_1^2 = c_{\max}^2.
$$

Hence, for large k, we have $e^T A_G^k \, e \approx c_{\max}^2 \, \lambda_1^k$. □

The next example illustrates Proposition 53.3. While the result is most useful for large matrices, we use a small one here for clarity.

Example 53.4 Consider the simple, undirected graph in Figure 53.2. It is not bipartite because it contains an odd cycle, for example BDC. We aim to estimate the number of walks of length 2, 3, and 4, and then compare these estimates with the exact counts.

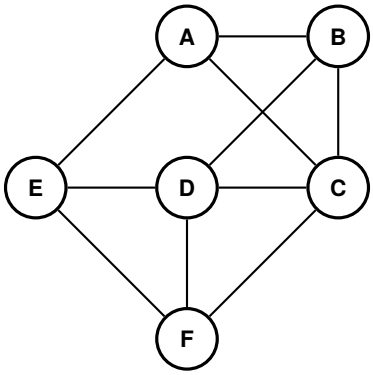


Fig. 53.2: A simple, non-bipartite, undirected graph with 6 nodes.

The adjacency matrix of the graph is

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}.$$

The eigenvalues of A are approximately

$$-2.247,\ -1.576,\ -0.555,\ 0.187,\ 0.802,\ 3.389,$$

so the spectral radius is $\rho(A) = 3.389$. A basis of eigenvectors of A is given by the columns of

$$V = \begin{bmatrix} -0.5211 & -0.0580 & 0.2319 & 0.6065 & -0.4179 & 0.3589 \\ 0.232 & 0.567 & -0.418 & -0.173 & -0.521 & 0.385 \\ 0.418 & -0.418 & 0.521 & -0.319 & -0.232 & 0.472 \\ -0.418 & -0.418 & -0.521 & -0.319 & 0.232 & 0.472 \\ 0.521 & -0.058 & -0.232 & 0.606 & 0.418 & 0.359 \\ -0.232 & 0.567 & 0.418 & -0.173 & 0.521 & 0.385 \end{bmatrix},$$

where the last column of V is the unit-length positive eigenvector corresponding to the Perron root $\rho(A)$.

In this eigenvector basis, the vector $e = [1, \ldots, 1]^T$ has approximate coordinates

$$0,\ 0.183,\ 0,\ 0.227,\ 0,\ 2.432,$$

so $c_{\max} = 2.432$.

The exact number of walks of length 2, 3, and 4 is given by

$$e^T A^2\, e, \quad e^T A^3\, e, \quad \text{and} \quad e^T A^4\, e,$$

while the estimates from Proposition 53.3 are

$$c_{\max}^2\,(3.389)^2, \quad c_{\max}^2\,(3.389)^3, \quad \text{and} \quad c_{\max}^2\,(3.389)^4.$$

The table below compares the exact and estimated values. For this small example, the estimates match the exact counts.

	Exact	Estimate
Walks of length 2	68	67.931
Walks of length 3	230	230.219
Walks of length 4	780	780.213

Problems

53.1 Show that an undirected graph is bipartite if and only if all its cycles have an even number of edges.

53.2 Consider the graph G with 5 vertices and edges

$$1-2, \quad 2-3, \quad 3-4, \quad 4-5, \quad 5-1, \quad 2-4.$$

(a) Find the exact number of walks of length 3.
(b) Use Proposition 53.3 to estimate this number and compare it with the exact value.

53.3 Let G be a connected, simple, non-bipartite graph with adjacency matrix A_G and Perron root $\lambda = \rho(A_G)$. Let v and w be the positive right and left unit eigenvectors associated with λ. Show that, for any vertices i and j, the number of walks of length k from i to j, given by the (i, j)-entry of A_G^k, satisfies

$$(A_G^k)_{ij} \approx \lambda^k \, v(i) \, w(j)$$

as $k \to \infty$.

53.4 Let G be a connected, non-bipartite graph with adjacency matrix A_G and Perron root $\lambda = \rho(A_G)$. Show that there exist positive constants C and D (which may depend on vertices i and j) such that, for all large k,

$$C \, \lambda^k \leqslant (A_G^k)_{ij} \leqslant D \, \lambda^k.$$

Explain why this means that the number of walks of length k grows at the rate λ^k.

53.5 Let G be a connected, simple, non-bipartite graph with adjacency matrix A_G. Show that

$$\frac{\max_{i,j}(A_G^k)_{ij}}{\min_{i,j}(A_G^k)_{ij}}$$

converges to a finite constant as $k \to \infty$.

Hint: Use Problems 53.3 and 53.4.

53.6 Let G be a connected, simple, non-bipartite graph with adjacency matrix A_G and Perron root $\lambda_1 = \rho(A_G)$. Let v_1 and w_1 be the right and left unit positive eigenvectors of A_G with eigenvalue λ_1. Give an upper bound for the error when approximating $(A_G^k)_{ij}$ by $\lambda_1^k \, v_1(i) \, w_1(j)$, in terms of the ratio $|\lambda_2/\lambda_1|^k$, where λ_2 is the second largest eigenvalue of A_G in modulus.

53.7 A graph is **d-regular** if every vertex has exactly d neighbors. Let G be a connected d-regular simple non-bipartite graph on n vertices with adjacency matrix A_G.

(a) Show that the Perron root of its adjacency matrix A_G is $\rho(A_G) = d$. What is the associated unit positive eigenvector?

(b) Show that the total number of walks of length k in G (i.e., $e^T A^k e$) satisfies

$$e^T A_G^k \, e \approx n \, d^k \qquad \text{as } k \to \infty.$$

53.8 In each of the following situations, explain why it might be useful to estimate the number of walks of a fixed length between pairs of nodes.

(a) A social network where nodes represent individuals and edges represent friendships.
(b) A network of webpages where nodes are pages and edges indicate hyperlinks.
(c) A community contact network where each edge represents a potential path for infection spread.
(d) A city transportation network where nodes are intersections and edges are roads.
(e) A communication network where nodes are routers and edges are direct communication links.
(f) A supply chain network where nodes represent suppliers, manufacturers, and retailers, and edges represent direct supply relationships.
(g) An ecological network where nodes are habitat patches and edges are migration corridors for animals.
(h) A financial network where nodes represent banks or financial institutions and edges represent significant financial exposures.
(i) A recommendation system where users and products are nodes in a graph (or in a bipartite graph projected onto one type of node).
(j) An internet routing network where nodes are routers and edges are direct physical or logical connections.

Chapter 54
XI-FROM THE CLASSROOM TO REAL LIFE

54.1 Page ranking for a web search engine

An online search begins with a simple but massive task: the engine instantly scans its index to find every single page matching your keywords. With common search terms, this can generate millions of potential results, creating a classic needle-in-a-haystack problem. To solve this, the search engine performs its most important function: ranking. It meticulously sorts the results to push the most relevant, high-quality pages to the top and bury the less useful ones.

Example 54.1 Consider a Google search for a broad term such as "university." The search may identify an enormous number of potentially relevant webpages, yet only a small selection appears on the first page of results. The ordering may depend on factors such as relevance, webpage quality, the user's location, and the context of the search. How does a search engine determine which pages should appear near the top?

A core part of Google's original search engine was PageRank, an algorithm created by co-founder Larry Page. Its core idea is to determine a webpage's importance by analyzing the links pointing to it. A page is considered important if it is linked to by other important pages.

This definition immediately presents a chicken-and-egg problem: how can you calculate a page's rank when it depends on the ranks of other pages that are not yet known? While this seems circular, the problem resolves elegantly through Linear Algebra. As we will see, it comes down to solving an eigenvalue problem for a matrix representing the entire web's link structure.

The influence of PageRank extends far beyond web search. It is a measure of node centrality used in all types of directed graphs, from ranking influential users on social media to analyzing biological networks.

PageRank is only one piece of today's search engines. It captures the idea of ranking pages by their links, but modern systems go much further. Google now uses hundreds of signals: the meaning and quality of the content, the user's intent and

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*,
Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6_54

location, how fast the site loads, and whether it works well on mobile. PageRank is best seen as the starting point, not the full story.

Our goal is to find a way to measure the "importance" of a webpage. To do this, we can model the entire web as a massive directed graph.

In this model:

- Each webpage is a node.
- A link from page i to page j is a directed edge from node i to node j.

We can label all webpages from 1 to n. For any given page i, we define its **outlinks** (O_i) as the set of pages it links to, and its **inlinks** (I_i) as the set of pages that link to it. Using this graph structure, we can now begin to explore different methods for calculating a page's importance.

Counting inlinks

A simple first approach is to rank a page by its popularity, that is, by counting its total number of inlinks. The more pages that link to it, the more "important" it seems.

However, this method has a critical flaw: it treats all links as equal endorsements. Intuitively, a link from a major news organization's homepage should carry far more weight than a link from an obscure personal blog, but this simple counting method fails to distinguish between them.

This technique, called **link farming**, makes a page seem important without real value, rewarding manipulation over quality. We need a better approach that considers both the number and quality of links.

Counting inlinks and their importance

The PageRank algorithm treats a link from page i to page j as a vote of confidence from i to j. But not all votes are equal. The vote from an important page is worth more than the vote from an obscure one.

Furthermore, a page's voting power (its own importance score) is split equally among all the pages it links to. If a page links to ten other pages, it passes on $1/10$th of its importance to each of them.

We capture this idea by defining the rank of page j, denoted as r_j, as follows:

$$r_j = \sum_{i \to j} \frac{r_i}{d_i}, \tag{54.1}$$

where d_i is the number of outlinks from page i (its out-degree) and the sum is taken over all pages i that link to page j.

In words: a page's importance is the sum of all the fractional importance it receives from the pages linking to it. If page i links to page j, then j inherits a fraction $(1/d_i)$ of page i's importance (r_i).

Notice that (54.1) is linear in the variables r_i. Since we have one such equation for every page on the web, finding all ranks means solving a huge system of linear equations. We will go through an example to see how this works.

Example 54.2 Imagine the entire web had only five webpages with links as shown in Figure 54.1. Each node represents a webpage, and a directed arrow from node i to node j means that page i links to page j.

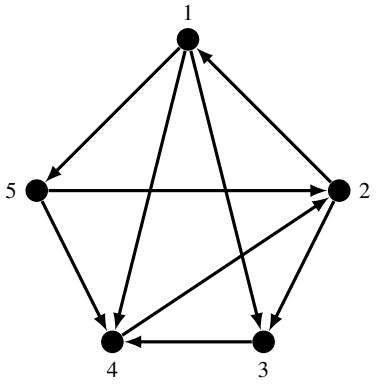


Fig. 54.1: A simple directed graph showing a small-scale example of the web.

In our model, we make one simplifying assumption: we count multiple links between two pages as a single link.

We now compute the score of each node using (54.1), where the score of a page j is the sum of the scores it receives from its inlinks. By examining the graph, we can write down the following equations:

$$
\begin{aligned}
r_1 &= \frac{r_2}{2} && \text{(Node 1 gets a link from node 2, which has 2 outlinks)}\\
r_2 &= \frac{r_4}{1} + \frac{r_5}{2} && \text{(Node 2 gets links from nodes 4 and 5)}\\
r_3 &= \frac{r_2}{2} + \frac{r_1}{3} && \text{(Node 3 gets links from nodes 1 and 2)}\\
r_4 &= \frac{r_3}{1} + \frac{r_1}{3} + \frac{r_5}{2} && \text{(Node 4 gets links from nodes 1, 3 and 5)}\\
r_5 &= \frac{r_1}{3} && \text{(Node 5 gets a link from node 1).}
\end{aligned}
$$

This gives a linear system of five equations with five unknowns (the scores we want to calculate):

$$\begin{aligned} r_1 - \frac{1}{2}r_2 &= 0, \\ r_2 - r_4 - \frac{1}{2}r_5 &= 0, \\ -\frac{1}{3}r_1 - \frac{1}{2}r_2 + r_3 &= 0, \\ -\frac{1}{3}r_1 - r_3 + r_4 - \frac{1}{2}r_5 &= 0, \\ -\frac{1}{3}r_1 + r_5 &= 0. \end{aligned} \tag{54.2}$$

This system has infinitely many nontrivial solutions, given by

$$r_1 = \frac{1}{2}t, \quad r_2 = t, \quad r_3 = \frac{2}{3}t, \quad r_4 = \frac{11}{12}t, \quad r_5 = \frac{1}{6}t, \quad t \in \mathbb{R}.$$

Thus, the set of solutions forms a one-dimensional space. Since we want a single score for each webpage, we add an extra condition to our system of equations. A common choice is to require that all scores r_i are positive and that their sum equals one:

$$r_1 + r_2 + r_3 + r_4 + r_5 = 1.$$

Solving this system gives the final importance scores:

$$r_1 = \frac{6}{39}, \; r_2 = \frac{12}{39}, \; r_3 = \frac{8}{39}, \; r_4 = \frac{11}{39}, \; r_5 = \frac{2}{39}.$$

This yields the final ranking, from most to least important:

$$\text{Node 2} \to \text{Node 4} \to \text{Node 3} \to \text{Node 1} \to \text{Node 5}.$$

Looking at the graph, this result makes intuitive sense. Node 2 is highly ranked because it receives links from two other nodes, including the full importance of node 4. Conversely, node 5 is the least important because it only receives a fraction of node 1's importance and has no other inlinks.

Let us revisit the results from our five-page web. We began with a system of equations and found an infinite set of solutions. By requiring all ranks to be positive and add up to one, we get a single ranking vector. This raises an important question: was this a lucky coincidence, or is such a unique, positive solution always guaranteed to exist for any web graph?

To answer this, we must reframe the problem. We will show that our system of equations is equivalent to an eigenvalue problem for a nonnegative matrix P called *link matrix* that we define next.

Definition 54.3. (Link matrix)

Let G be a directed graph with n nodes. For node i, let d_i be its **out-degree**, that is, the number of links going out from node i. The **link matrix** P is the $n \times n$ matrix whose (i, j) entry is defined by

$$P_{ij} = \begin{cases} \frac{1}{d_i}, & \text{if node } i \text{ links to node } j, \\ 0, & \text{otherwise.} \end{cases}$$

Notice that each row of the link matrix P describes how a single node distributes its importance. For any given row i, the entries are all positive or zero, and their sum is equal to one (unless a node has no outlinks, a problem we will address shortly).

Example 54.4 For the graph in Example 54.2, the link matrix is

$$P = \begin{bmatrix} 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix}.$$

Notice that in this example P is row stochastic.

Now we can connect our link matrix P to the original system of equations for the scores, r_j. Let us examine the jth entry of the vector resulting from the matrix-vector product $P^T r$, where $r = [r_1, r_2, \ldots, r_n]^T$ is the vector of scores.

$$(P^T r)_j = \sum_{i=1}^{n} (P^T)_{ji} r_i = \sum_{i=1}^{n} P_{ij} r_i.$$

By the definition of the link matrix, P_{ij} is $1/d_i$ if there is a link from node i to node j, and zero otherwise. Therefore, the sum simplifies to:

$$(P^T r)_j = \sum_{i \to j} \frac{1}{d_i} r_i.$$

This is precisely the right-hand side of our original system of equations. This means our entire system can be expressed by the single, elegant vector equation:

$$r = P^T r.$$

Hence, we have reframed our original problem as follows: finding the ranking vector r is equivalent to finding an eigenvector of the matrix P^T with eigenvalue 1.

Proposition 54.5. (Web scores as an eigenvalue problem)

Let G be a directed graph with n nodes and link matrix P. A nonzero ranking vector r is a solution to the system (54.1) if and only if $P^T r = r$, that is, if and only if r is an eigenvector of the matrix P^T with corresponding eigenvalue 1.

As we mentioned, the link matrix P in Example 54.4 is row-stochastic, that is, each row sums to one. By Theorem 52.57, 1 is an eigenvalue of P and therefore also of its transpose, P^T.

In our example, 1 is a simple eigenvalue, so its eigenspace is one-dimensional. Thus, after normalization, there is a unique ranking vector. In this case, this vector is also positive:

$$r = \begin{bmatrix} \frac{2}{13} \\ \frac{4}{13} \\ \frac{8}{39} \\ \frac{11}{39} \\ \frac{2}{39} \end{bmatrix} \approx \begin{bmatrix} 0.15 \\ 0.31 \\ 0.20 \\ 0.28 \\ 0.05 \end{bmatrix}.$$

This matches the ranking we found previously by solving the linear system.

There is another useful property of the link matrix in our example: 1 is its unique eigenvalue of largest absolute value. Thus, 1 is a dominant eigenvalue, and we can use the power method (from Chapter 37) to find the ranking vector r. Recall that this method works by repeatedly multiplying an initial guess vector by the matrix P^T. For huge matrices like the one for the web, this makes ranking practical. It removes the need to store the matrix. We only need to compute matrix-vector products.

Our simple example graph produced a matrix with a very convenient set of properties that give us a unique, computable ranking. To summarize, it had:

- A simple eigenvalue 1.
- A positive eigenvector associated with 1.
- No other eigenvalues with absolute value 1.

But do all link matrices share these ideal properties? In the next two examples, we will see what happens when they fail.

Example 54.6 shows a modified version of Example 54.2 in which link matrix P does not have 1 as an eigenvalue. The main problem is that one of the nodes has no outgoing links.

Example 54.6. (Dangling node)

Assume we remove the only outgoing link from node 4 in the graph in Example 54.2. Figure 54.2 shows the updated graph. Node 4 is now what is called a **dangling node**, a node with an out-degree of zero.

The new link matrix, P reflects this change in row 4:

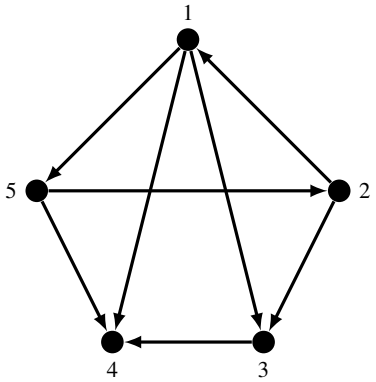


Fig. 54.2: Updated web graph after removing a link from node 4.

$$P = \begin{bmatrix} 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix}.$$

Notice that row 4 is now all zeros. This breaks a fundamental property of our original model: the matrix is no longer row-stochastic because not all rows sum to one.

This all-zero row acts like a "rank sink." Any importance passed to node 4 (from nodes 1, 3, and 5) is trapped there and vanishes from the network instead of being redistributed.

For this particular graph, importance eventually leaks out of the system through node 4. As a result, a direct calculation confirms that 1 is no longer an eigenvalue of P^T. Thus, our ranking equation $P^T r = r$ has no non-trivial solution.

So, what property does the link matrix in Example 54.2 have that guarantees 1 is an eigenvalue? The key property is row stochasticity.

The presence of a dangling node in Example 54.6 creates a row of all zeros in the link matrix, so the matrix is no longer row-stochastic. Therefore, we can no longer guarantee that 1 is an eigenvalue. In this particular example, as we have seen, it is not.

If there are no dangling nodes in the graph, the link matrix is row-stochastic, and 1 is guaranteed to be an eigenvalue. We state this formally in the following proposition.

Proposition 54.7. (Row-stochastic link matrix characterization)

The link matrix P of a directed graph G is row-stochastic if and only if G has no dangling nodes. In this case, 1 is an eigenvalue of P (and of P^T), the spectral radius is $\rho(P) = 1$, and every eigenvalue λ of P (and of P^T) satisfies $|\lambda| \leqslant 1$.

Proof The equivalence between P being row-stochastic and G having no dangling nodes follows directly from the definitions.

If a node i is a dangling node, its out-degree is $d_i = 0$, and the corresponding row in P consists entirely of zeros. This row sums to 0, so P is not row-stochastic. Conversely, if G has no dangling nodes, then every node i has an out-degree $d_i > 0$. The sum of the entries in row i is then

$$\sum_j P_{ij} = d_i(1/d_i) = 1,$$

which means that P is row-stochastic.

The statements regarding the eigenvalues and spectral radius are standard properties of all row-stochastic matrices, as established in Theorem 52.57. □

The next example shows that while a stochastic link matrix is guaranteed to have 1 as an eigenvalue, this eigenvalue is not always simple. When the eigenspace for $\lambda = 1$ has a dimension greater than one, there is no longer a unique eigenvector (up to scaling) to define the ranking. This raises a critical question: which one should we use?

Example 54.8. (Directed graph with disconnected components)

Consider the directed graph in Figure 54.3. It has two connected components: one with vertices 1, 2, and 5, and another with vertices 3 and 4.

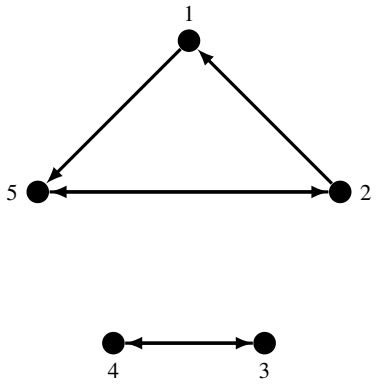


Fig. 54.3: A directed graph with two disconnected components.

The link matrix of the graph in Figure 54.3 is

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ \frac{1}{2} & 0 & 0 & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

A direct calculation shows that 1 is an eigenvalue of P with both algebraic and geometric multiplicity equal to 2. The corresponding eigenspace of P^T is

$$\text{eig}_{P^T}(1) = \text{span}\left\{ \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\}.$$

Even if we require positive rankings that sum to one, the ranking vector is not unique. The ranking is therefore not well defined.

The Perron-Frobenius theorem guarantees that, if a row-stochastic matrix is irreducible, then it has a simple eigenvalue of 1. However, this property is not guaranteed if the matrix is reducible.

Given its huge size, the graph of the internet inevitably has dangling nodes and is not strongly connected, which poses a problem: we can no longer be sure of the existence of a unique ranking vector.

So, how did Google solve this? The solution involves intentionally modifying the link matrix to give it the properties needed for the theory to apply. To guarantee the algorithm produces a single and positive ranking vector, the final matrix is engineered to be:

1. Row-stochastic, to ensure 1 is an eigenvalue.
2. Irreducible and primitive, so that the Perron-Frobenius theorem guarantees $\lambda = 1$ is a simple, dominant eigenvalue.

First, let us handle dangling nodes, that is, the zero rows in the link matrix. A common fix is to replace each zero row with $\frac{1}{n}e^T$, where e^T is the n-dimensional row vector of all ones. In other words, we connect every dangling node to all nodes in the graph.

If ℓ is the n-dimensional vector whose ith entry is 1 for every dangling node i and 0 otherwise, the new link matrix is

$$\widehat{P} := P + \frac{1}{n}\,\ell\, e^T. \tag{54.3}$$

This modified matrix $\widehat{P}$ is row-stochastic and thus has 1 as an eigenvalue.

Example 54.9. (Handling dangling nodes)

Let us revisit the graph in Figure 54.2, where node 4 is a dangling node. After connecting node 4 to all other nodes, the modified link matrix is

$$\widehat{P} = \begin{bmatrix} 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix} + \frac{1}{5} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \end{bmatrix}.$$

A direct calculation shows that 1 is an eigenvalue of $\widehat{P}^T$ with a positive eigenvector

$$r \approx \begin{bmatrix} 0.15 \\ 0.14 \\ 0.20 \\ 0.39 \\ 0.13 \end{bmatrix},$$

normalized so that its entries sum to one. This orders the nodes from most to least important: 4, 3, 1, 2, 5.

Modifying the link matrix as above does not guarantee that the eigenvalue one will be simple. Next, we show how to make the link matrix irreducible and primitive so that the Perron-Frobenius theorem applies, guaranteeing that 1 is a simple dominant eigenvalue. We define the revised link matrix as

$$G(\alpha) := \alpha \widehat{P} + (1 - \alpha) \frac{1}{n} e e^T, \tag{54.4}$$

where $0 \leqslant \alpha < 1$, $\widehat{P}$ is the modified link matrix from (54.3), and e is the vector of all ones (so ee^T is the matrix of all ones). The resulting matrix $G(\alpha)$ is called the **Google matrix**. It is a convex combination of two link matrices: $\widehat{P}$ and $\frac{1}{n} ee^T$. The parameter α, called the **damping factor**, controls the balance between the two. If α is close to 1, $G(\alpha)$ is close to the link matrix of the original graph (after fixing dangling nodes). If α is close to 0, $G(\alpha)$ is close to the link matrix of a fully connected graph, where every node links to all others, and the structure of the original graph is lost. In practice, a common choice is $\alpha = 0.85$.

The following theorem summarizes the key properties of the Google matrix.

Theorem 54.10. (Properties of the Google matrix)

Let G be a connected graph with n nodes and link matrix P. For $0 \leqslant \alpha < 1$, the Google matrix $G(\alpha)$ defined in (54.4) is row-stochastic, irreducible, and primitive.

Proof A nonnegative matrix A is row-stochastic when $Ae = e$, where e is the all-ones vector. For the Google matrix,

$$G(\alpha)\, e = \alpha\, \widehat{P}\, e + (1-\alpha)\frac{1}{n} e e^T e = \alpha e + (1-\alpha)e = e,$$

where we used that $\widehat{P}$ is row-stochastic by construction.

In the graph associated with $G(\alpha)$, every node links to every other node, so it is strongly connected and $G(\alpha)$ is irreducible. Moreover, $G(\alpha)$ has all positive entries, so it is also primitive. □

The Google matrix $G(\alpha)$ is positive and row-stochastic, so its transpose is positive and column-stochastic. By Theorems 52.57 and 52.28, $G(\alpha)^T$ has 1 as a simple dominant eigenvalue with a positive eigenvector.

We are now ready to define the node ranking first used by Google.

Definition 54.11. (PageRank)

Let G be a connected graph with n nodes and link matrix P. For $0 \leqslant \alpha < 1$, let $G(\alpha)$ be the Google matrix in (54.4). The **PageRank** vector r is the Perron vector of $G(\alpha)^T$. Its ith entry is the PageRank of node i.

Example 54.12. (PageRank)

Consider the graph in Figure 54.3. For $\alpha = 0.85$, its Google matrix is

$$G(0.85) = 0.85 \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ \frac{1}{2} & 0 & 0 & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} + \frac{1-0.85}{5} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.03 & 0.03 & 0.03 & 0.03 & 0.88 \\ 0.455 & 0.03 & 0.03 & 0.03 & 0.455 \\ 0.03 & 0.03 & 0.03 & 0.88 & 0.03 \\ 0.03 & 0.03 & 0.88 & 0.03 & 0.03 \\ 0.03 & 0.88 & 0.03 & 0.03 & 0.03 \end{bmatrix}.$$

The Perron vector of $G(0.85)^T$ is

$$r \approx \begin{bmatrix} 0.13 \\ 0.23 \\ 0.20 \\ 0.20 \\ 0.24 \end{bmatrix}.$$

This ranks the nodes from most to least important: 5, 2, 3, 4, 1 (with nodes 3 and 4 tied).

There is a natural Markov chain view of PageRank that explains how the Google matrix arises from the original link matrix.

Markov chain interpretation of PageRank

We can think of PageRank as the stationary, limiting distribution of a Markov chain whose transition matrix is $G(\alpha)$, the Google matrix of the graph.

The PageRank algorithm models a "random surfer" moving through the web. At every page, the surfer first makes a primary decision based on the damping factor, α. With probability α (typically 85%), they commit to following a hyperlink on the current page. With the remaining probability of $1 - \alpha$ (15%), they disregard the page's links and "teleport" to a completely random page on the web. If the surfer has committed to following a link, they then pick one of the available hyperlinks at random. However, if the page has no outgoing links (a "dangling node"), they must jump to a random page, since they have no other choice.

For the starting distribution of this Markov chain, we use

$$(\pi^0)^T = \frac{1}{n} e^T = \left[\tfrac{1}{n} \ \tfrac{1}{n} \ \cdots \ \tfrac{1}{n}\right],$$

where n is the total number of nodes (websites). This means the random surfer starts with equal probability $1/n$ at each website. In other words, they pick their first page completely at random.

At each step, the random surfer moves to another node based on the transition matrix. The distribution after each step is given by

$$(\pi^{t+1})^T = (\pi^t)^T G(\alpha), \tag{54.5}$$

where π^t is the probability distribution at step t. This vector shows the chance of the surfer being at each website after t steps.

Since the Google matrix $G(\alpha)$ is irreducible and primitive, Corollary 52.101 tells us that the Markov chain has a limiting distribution

$$\pi^T = \lim_{t \to \infty} (\pi^t)^T$$

that does not depend on the starting vector π^0. This limiting distribution is the unique stationary distribution,

$$\pi^T G(\alpha) = \pi^T,$$

or, equivalently, the Perron vector of $G(\alpha)^T$,

$$G(\alpha)^T \pi = \pi.$$

The iteration in (54.5) is just the Power method applied to $G(\alpha)^T$. This is actually the standard way PageRank is computed in practice. We do not need to normalize the vectors at each step, since they keep unit 1-norm throughout.

We know that 1 is a simple dominant eigenvalue, so the Power method converges to the PageRank vector. As we saw in Chapter 37, the convergence speed depends on the ratio between the largest and second largest eigenvalues in modulus. Here the

largest is 1. What about the second largest? It turns out we can estimate its modulus, as we show next.

In the next theorem, we assume the link matrix is row-stochastic, so there are no dangling nodes. If there are, we connect them to every node in the graph, as explained earlier.

Theorem 54.13. (Spectrum of the Google matrix)

Let G be a connected graph with n nodes and link matrix P. For $0 \leqslant \alpha < 1$, let $G(\alpha)$ be the Google matrix defined in (54.4). If $1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of P, then $1, \alpha\lambda_2, \ldots, \alpha\lambda_n$ are the eigenvalues of $G(\alpha)$.

Proof Since the link matrix P is row-stochastic, 1 is an eigenvalue with eigenvector e, the all-ones vector. Extend e to a basis of $\mathbb{R}^n$ and place these basis vectors as the columns of

$$S = \begin{bmatrix} e & X \end{bmatrix},$$

where X contains the remaining $n - 1$ columns. The matrix S is nonsingular. Write its inverse as

$$S^{-1} = \begin{bmatrix} y_1^T \\ Y^T \end{bmatrix}.$$

Then

$$S^{-1}S = \begin{bmatrix} y_1^T e & y_1^T X \\ Y^T e & Y^T X \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & I_{n-1} \end{bmatrix}.$$

From this, we get $y_1^T e = 1$ and $Y^T e = 0$.

Now consider the similarity transformation

$$S^{-1}PS = \begin{bmatrix} y_1^T Pe & y_1^T PX \\ Y^T Pe & Y^T PX \end{bmatrix} = \begin{bmatrix} y_1^T e & y_1^T PX \\ 0 & Y^T PX \end{bmatrix} = \begin{bmatrix} 1 & y_1^T PX \\ 0 & Y^T PX \end{bmatrix}.$$

Since similar matrices have the same eigenvalues and the eigenvalues of a block upper-triangular matrix is the union of the eigenvalues of the main diagonal blocks, the eigenvalues of the block $Y^T PX$ are $\lambda_2, \ldots, \lambda_n$.

Applying the same similarity transformation to the Google matrix $G(\alpha)$ gives

$$
\begin{aligned}
S^{-1}\left(\alpha P + (1-\alpha)\frac{1}{n}ee^T\right)S =& \alpha S^{-1}PS + (1-\alpha)\frac{1}{n}S^{-1}ee^TS \\
=& \alpha \begin{bmatrix} 1 & y_1^TPX \\ 0 & Y^TPX \end{bmatrix} + (1-\alpha)\frac{1}{n}\begin{bmatrix} y_1^Te \\ Y^Te \end{bmatrix}\begin{bmatrix} e^Te & e^TX \end{bmatrix} \\
=& \alpha \begin{bmatrix} 1 & y_1^TPX \\ 0 & Y^TPX \end{bmatrix} + (1-\alpha)\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 1 & \frac{1}{n}e^TX \end{bmatrix} \\
=& \alpha \begin{bmatrix} 1 & y_1^TPX \\ 0 & Y^TPX \end{bmatrix} + (1-\alpha)\begin{bmatrix} 1 & \frac{1}{n}e^TX \\ 0 & 0 \end{bmatrix} \\
=& \begin{bmatrix} 1 & \alpha\, y_1^TPX + (1-\alpha)\frac{1}{n}e^TX \\ 0 & \alpha\, Y^TPX \end{bmatrix}.
\end{aligned}
$$

This shows that the eigenvalues of $G(\alpha)$ are $\{1, \alpha\lambda_2, \ldots, \alpha\lambda_n\}$. □

Since the Google matrix is row-stochastic, its spectral radius is 1. If λ_2 denotes the eigenvalue of P with second largest magnitude, then $|\lambda_2| \leqslant 1$. From Theorem 54.13, the ratio between the largest and second largest eigenvalues of $G(\alpha)$ is at least $1/\alpha$ ($\alpha \neq 0$), where α is the damping parameter. This means we can control the convergence rate by choosing α conveniently. A smaller α leads to faster convergence, but it also reduces the influence of the web's actual hyperlink structure in ranking pages. In practice, we must choose α to balance speed and how well the ranking reflects the true link structure of the web. As we said, $\alpha = 0.85$ is the most common choice.

Why the Ranking from the Google Matrix is Reasonable

Since the Google matrix $G(\alpha)$ is a modification of the web's true link structure $\widehat{P}$, why should we trust the ranking it produces? The answer is that the modification is a form of **regularization**, a small adjustment that makes an ill-posed problem solvable without losing the essential information from the original system.

1. **The Problem.** For the raw transition matrix of the web graph, $\widehat{P}$, the web is often not strongly connected. It contains disconnected communities and dangling nodes (pages with no outgoing links). This causes the matrix to be **reducible**. Mathematically, this means the eigenvalue $\lambda = 1$ may not be simple. The corresponding eigenspace for the transpose matrix, $\widehat{P}^T$, can have a dimension greater than one:
$$E_1 = \{v \in \mathbb{R}^n \mid \widehat{P}^Tv = v\}, \quad (\dim(E_1) > 1).$$
This eigenspace E_1 contains an infinite number of potential stationary distributions. There is **no single, unique ranking vector**. The raw link structure alone is not enough to produce a definitive global ranking.
2. **The Solution.** The Google matrix introduces a "teleportation" term, creating a new matrix $G(\alpha)$:
$$G(\alpha) = \alpha\widehat{P} + (1-\alpha)\frac{1}{n}ee^T.$$
Here is why this is a reasonable fix and not just an arbitrary change:

- **It guarantees a *unique* answer.** For any damping factor $\alpha < 1$, the term $(1-\alpha)\frac{1}{n}ee^T$ is a matrix with all positive entries. This ensures that $G(\alpha)$ is **primitive** (irreducible and aperiodic). By the Perron-Frobenius theorem, a primitive matrix has a unique, positive stationary distribution, which we call the PageRank vector $\pi(\alpha)$. The problem is now well-posed.
- **It selects a particular stationary distribution**. As $\alpha \to 1^-$, the effect of the teleportation term decreases. Under the assumptions above and for a fixed teleportation vector, the PageRank vector has a limit

$$\lim_{\alpha \to 1^-} \pi(\alpha) = \pi_{\text{limit}},$$

where π_{limit} is a stationary distribution of $\widehat{P}$. Thus, teleportation provides a systematic way to select one stationary distribution when the stationary distribution of $\widehat{P}$ is not unique. The selected limit may depend on the teleportation vector and should therefore be viewed as a modeling choice rather than the uniquely correct member of the stationary eigenspace.
In practice, the damping factor is chosen to balance the influence of the link structure, numerical convergence, and sensitivity to changes in the network. The commonly used value $\alpha = 0.85$ is one practical choice, but it is not guaranteed to closely approximate the limit as $\alpha \to 1^-$ for every network.

In conclusion, a damping factor such as $\alpha = 0.85$ provides a practical balance between the influence of the original link structure and the computational stability introduced by teleportation. The teleportation term modifies the original transition model in a controlled way, so the resulting ranking reflects both the network's link structure and the chosen teleportation rule. However, $\pi(0.85)$ is not guaranteed to be close to the limiting vector as $\alpha \to 1^-$ for every network.

Problems

54.1 We said that dangling nodes and a reducible link matrix are unavoidable on the web. Why do dangling nodes appear in practice? What situations could make the link matrix reducible?

54.2 Consider a simplified model of web surfing where a user navigates among three websites according to the transition matrix:

$$P = \begin{bmatrix} 0.2 & 0.5 & 0.3 \\ 0.4 & 0.1 & 0.5 \\ 0.3 & 0.3 & 0.4 \end{bmatrix}.$$

(a) Find the stationary distribution of this Markov chain.
(b) Explain how this stationary distribution might be interpreted as a measure of the importance of each website.

54.3 Consider a web consisting of 4 pages with the following directed links:

Page 1 → Pages 2 and 3, Page 2 → Page 3,

Page 3 → Pages 1 and 4, Page 4 → Page 1.

Assume the damping factor is $\alpha = 0.85$. Formulate the corresponding Google matrix and compute the PageRank vector. List the pages in order of decreasing PageRank.

54.4 Consider the graph in Figure 54.4.

(a) Compute the PageRank vector for $\alpha = 0.85$. Rank the pages from highest to lowest.
(b) Study how the PageRank vector changes as α goes from 0.1 to 0.95. How does a larger or smaller α affect the scores and the page ranking?

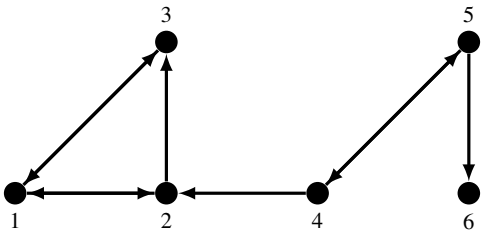


Fig. 54.4: Directed graph of links between six webpages.

54.5 The owner of page 1 (Figure 54.4) tries to boost its rank by creating many sites that link only to it. This trick is called a **link farm**. The modified graph is in Figure 54.5. Compute the PageRank vector with $\alpha = 0.85$. How does the ranking change? Did node 1 succeed?

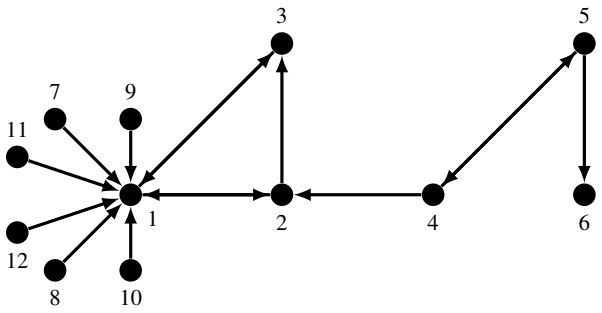


Fig. 54.5: Six-page web graph with a link farm.

54.6 For the graph in Figure 54.4, use the power method on the transpose of the Google matrix to compute the PageRank vector. Study how the convergence speed of the power method changes as the damping factor α varies.

54.7 PageRank is usually computed with the power method on the transpose of the Google matrix. Each step of the method only needs matrix-vector products, which would cost about n^2 operations if n is the number of webpages. Since the web has billions of pages, this is not feasible. But the link matrix P is extremely sparse, so we do not need to store it explicitly, and matrix-vector products are cheap. Show that the power method can be carried out using only vector-matrix products with the sparse P, without ever forming or storing the matrices $\widehat{P}$ or $G(\alpha)$.

54.8 (Personalized PageRank) Let $\widehat{P} \in \mathbb{R}^{n\times n}$ be a row-stochastic matrix, and let $e \in \mathbb{R}^n$ denote the vector whose entries are all equal to 1.

In the standard PageRank model, the random surfer teleports to each page with equal probability. More generally, let $v \in \mathbb{R}^n$ satisfy $v > 0$ and $e^T v = 1$. The vector v is called a *personalization vector*. For $0 \leqslant \alpha < 1$, define the personalized Google matrix by

$$G(\alpha, v) = \alpha\widehat{P} + (1-\alpha)ev^T.$$

(a) Show that $G(\alpha, v)$ is row stochastic.
(b) Show that $G(\alpha, v) > 0$. Conclude that $G(\alpha, v)$ is irreducible and primitive.
(c) Conclude that 1 is a simple dominant eigenvalue of $G(\alpha, v)$ and that $G(\alpha, v)^T$ has a unique positive eigenvector r, normalized so that $e^T r = 1$. Explain why r^T is the unique stationary distribution of $G(\alpha, v)$.
(d) Show that, for every initial distribution $\pi^{(0)}$,

$$\lim_{t\to\infty} \left(\pi^{(0)}\right)^T G(\alpha, v)^t = r^T.$$

(e) Explain how the entries of v influence the PageRank scores. What choice of v gives the standard Google matrix $G(\alpha)$?

54.9 For the graph in Figure 54.4, compute the PageRank vector using a non-uniform teleportation vector $v \neq \frac{1}{n}e$. Compare the result with the uniform case $v = \frac{1}{n}e$.

54.10 After fixing dangling nodes, PageRank forces the link matrix to be irreducible and primitive by connecting every node to every other node. This is a drastic change to the graph, so one might ask if there are gentler ways to achieve primitivity. You might come up with the idea of adding a "dummy" node to the web. This node connects to every other node, and every other node connects back to it. The modified matrix is

$$M(\alpha) = \begin{bmatrix} \alpha\widehat{P} & (1-\alpha)e \\ \frac{1}{n}e^T & 0 \end{bmatrix}, \quad \text{where} \quad \widehat{P} = P + \tfrac{1}{n}\ell e^T, \quad 0 < \alpha < 1.$$

(a) Show that $M(\alpha)$ is row stochastic, irreducible and primitive.
(b) Show that if $1, \lambda_2, \ldots, \lambda_n$ are the eigenvalues of $\widehat{P}$, then

$$\{\, 1,\ \alpha\lambda_2, \ldots, \alpha\lambda_n,\ \alpha - 1 \,\}$$

are the eigenvalues of $M(\alpha)$.
(c) Fix α. Let $r \in \mathbb{R}^n$ be the Perron vector of $G(\alpha)^T$, normalized so that $e^T r = 1$. Let

$$\widehat{r} = \begin{bmatrix} y \\ \gamma \end{bmatrix} \in \mathbb{R}^{n+1}$$

be the Perron vector of $M(\alpha)^T$, normalized so that $e^T y + \gamma = 1$. Show that y is proportional to r and that

$$r = \frac{y}{e^T y}.$$

Thus, after dropping the last entry of $\widehat{r}$ and renormalizing the remaining entries so that they sum to 1, we recover the PageRank vector r. In this sense, connecting every page to every other page and adding a single dummy node produce the same PageRank ranking.

54.11 (Linear system form of PageRank) Fix α and let r be the PageRank vector.

(a) Show that r satisfies

$$r^T (I_n - \alpha \widehat{P}) = \tfrac{1-\alpha}{n} e^T.$$

(b) Show that the matrix $(I_n - \alpha \widehat{P})$ is invertible.

54.12 A web crawler (or spider) is a program that visits webpages, follows their links, and collects information. Most programming languages, such as Python, have libraries that can do this. Use a crawler to build the link matrix of your university's domain. Compute the PageRank vector and list the top-ranked sites. Do the results match what you expected?

54.13 We introduced the PageRank vector as a way to rank webpages. But it can also be seen as another node centrality measure, like the ones studied earlier in this book. Compare PageRank with the other centrality measures we have seen. Outside of ranking webpages, in what settings could PageRank be useful for ranking the nodes of a graph?

54.14 (PageRank in real-world networks) The goal of this project is to compute PageRank centralities in a real-world directed network. Choose one of the following datasets:

1. OpenFlights Airport Network: Airports are nodes, and a directed edge from Airport A to Airport B means there is a flight route. Data available at OpenFlights.org Data Page. (You will need the files `routes.dat` and `airports.dat`.)
2. Stanford Web Pages Network: Web pages are nodes, and a directed edge from Page A to Page B means A links to B. Data available at SNAP Dataset Page: stanford-web.
3. Enron Email Network: Employees are nodes, and a directed edge from Person A to Person B means A sent at least one email to B. Data available at SNAP Dataset Page: enron-email.

Normalize the final PageRank vector so that its entries sum to 1. Then identify the top 15 nodes with the highest scores. What do these nodes represent? Do the results make sense given how importance is defined in your chosen network?

54.15 Eigenvector centrality measures the importance of nodes in a network. The idea is that a node is important if it receives links from other important nodes.

Let A be the adjacency matrix of the graph, where $A_{ij} = 1$ if there is an edge from node i to node j, and $A_{ij} = 0$ otherwise. Let x be the vector of centrality scores. The score of node i is proportional to the sum of the scores of the nodes that point to it:

$$x_i = \frac{1}{\lambda} \sum_j A_{ji} x_j.$$

In matrix form,

$$A^T x = \lambda x,$$

which is an eigenvalue–eigenvector equation.

In general, a matrix can have many eigenvalues and eigenvectors, but not every eigenvector is suitable as a centrality vector: its entries should be nonnegative. Eigenvector centrality for a directed graph can be defined using a nonnegative eigenvector of A^T associated with $\rho(A)$. If the graph is not strongly connected, some nodes may receive a centrality score of zero, and the centrality vector may not be unique. A zero score may occur because the chosen Perron eigenvector is supported on only part of the graph. If the centrality vector is not unique, the resulting ranking may depend on which nonnegative Perron eigenvector is selected.

If the graph is strongly connected, then A, and hence A^T, is irreducible. By the Perron–Frobenius theorem, $\rho(A)$ is a positive eigenvalue of A^T with a positive eigenvector, and this eigenvector is unique up to scaling. Thus, strong connectivity guarantees that every node receives a positive and unambiguous eigenvector-centrality score. Since eigenvectors are defined only up to scaling, we normalize the centrality vector so that its entries sum to 1.

If the graph is also aperiodic, then A is primitive, so $\rho(A)$ is a dominant eigenvalue. This additional property is useful when eigenvector centrality is computed using the power method.

(a) Compute the eigenvector centrality for the largest strongly connected component of one of the real-world networks from Problem 54.14. First, extract a largest strongly connected component of the original network and form the adjacency matrix A of the corresponding induced subgraph. Then compute the positive eigenvector of A^T associated with $\rho(A)$, and normalize it so that its entries sum to 1. Rank the nodes in that subgraph according to this measure.

(b) Compute an eigenvector-centrality vector for the entire original network. Find a nonnegative eigenvector of A^T associated with $\rho(A)$, and normalize it so that its entries sum to 1. Rank the nodes according to this centrality vector and report any nodes that receive a centrality score of zero. If there is more than one linearly independent nonnegative eigenvector associated with $\rho(A)$, choose one, state your choice, and explain how this nonuniqueness affects the interpretation of the centrality scores.

(c) Compute the PageRank vector for the entire original network and compare it with the eigenvector-centrality vector obtained in part (b). How do eigenvector centrality and PageRank differ? In particular, compare the uniqueness and positivity of the two centrality vectors, and discuss the treatment of nodes that receive zero eigenvector centrality.

References

[1] COVID-19 open research dataset challenge (cord-19), Kaggle. https://www.kaggle.com/datasets/allen-institute-for-ai/CORD-19-research-challenge. Accessed: 2024-04-01.

[2] Google trends. https://trends.google.com/trends/.

[3] Openml. https://www.openml.org. Accessed: 2025-08-04.

[4] Charu C. Aggarwal. *Recommender Systems*. Springer, New York, NY, 2016. ISBN 978-3-319-29658-0.

[5] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM–SIAM Symposium on Discrete Algorithms*, pages 1027–1035. Society for Industrial and Applied Mathematics, 2007. doi: 10.5555/1283383.1283494.

[6] Sheldon Axler. *Linear Algebra Done Right*. Springer, New York, 3rd edition, 2015. ISBN 978-3-319-11079-0.

[7] Adelchi Azzalini and Adrian W Bowman. A look at some data on the Old Faithful geyser. *Applied Statistics*, 39(3):357–365, 1990.

[8] Richard Bellman. On a routing problem. *Quarterly of applied mathematics*, 16(1):87–90, 1958.

[9] Ronald F. Boisvert, Roldan Pozo, and Karin Remington. Matrix Market: A web resource for test matrix collections. https://math.nist.gov/MatrixMarket/, 1996. Accessed: 2024-12-09.

[10] P. Bonacich. Factoring and weighting approaches to status scores and clique identification. *Journal of Mathematical Sociology*, 2(1):113–120, 1972.

[11] P. Bonacich. Power and centrality: A family of measures. *American Journal of Sociology*, 92(5):1170–1182, 1987.

[12] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, 2004. ISBN 978-0521833783.

[13] François Chollet. *Deep Learning with Python*. Manning Publications, Shelter Island, 2nd edition, 2021. ISBN 978-1617296864.

[14] Dean De Cock. Ames, Iowa: Alternative to the Boston Housing Data as an end of semester regression project. *Journal of Statistics Education*, 19(3):1–28, 2011.

[15] James W. Demmel. *Applied Numerical Linear Algebra*. SIAM, Philadelphia, PA, 1997. ISBN 978-0898713893.

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*, Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6

[16] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

[17] J. Ding and Noah H. Rhee. On the equality of algebraic and geometric multiplicities of matrix eigenvalues. *Applied Mathematics Letters*, 24(12):2211–2215, 2011.

[18] F. M. Dopico, C. R. Johnson, and J. M. Molera. Multiple LU factorizations of a singular matrix. *Linear Algebra and its Applications*, 419:24–36, 2006.

[19] Dheeru Dua and Casey Graff. UCI Machine Learning Repository. `https://archive.ics.uci.edu/ml`, 2019. Accessed: 2025-08-04.

[20] M. Ekin Eren, Nick Solovyev, Edward Raff, Charles Nicholas, and Ben Johnson. COVID-19 kaggle literature organization. In *Proceedings of the ACM Symposium on Document Engineering 2020*, pages 1–4, 2020. doi: 10.1145/3395027.3419591. Available at arXiv: `https://arxiv.org/abs/2008.13542`.

[21] Ky Fan. On a theorem of Weyl concerning eigenvalues of linear transformations I. *Proceedings of the National Academy of Sciences of the United States of America*, 35(11):652–655, 1949.

[22] Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973. URL `https://eudml.org/doc/12737`.

[23] Ronald A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.

[24] Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962.

[25] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Sebastopol, 3rd edition, 2022. ISBN 978-1098125974.

[26] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, 4th edition, 2013. ISBN 978-1421407944.

[27] J. Grossman. The Erdős Number Project. `https://sites.google.com/oakland.edu/grossman/home/the-erdoes-number-project`. Accessed: 2024-04-01.

[28] Richard W. Hamming. Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2):147–160, 1950.

[29] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer, New York, 2nd edition, 2009. ISBN 978-0387848570.

[30] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, Cambridge, UK, 2nd edition, 2012. ISBN 978-0521548236.

[31] Allison Marie Horst, Alison Presmanes Hill, and Kristen B Gorman. *palmerpenguins: Palmer Archipelago (Antarctica) penguin data*, 2020. URL `https://allisonhorst.github.io/palmerpenguins/`. R package version 0.1.0.

[32] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning: with Applications in Python*. Springer Texts in Statistics. Springer, New York, 2023. ISBN 978-3031387463.

[33] L. Katz. A new status index derived from sociometric analysis. *Psychometrika*, 18:39–43, 1953.

[34] Martin Koeber and Uwe Schäfer. The unique square root of a positive semidefinite matrix. *International Journal of Mathematical Education in Science and Technology*, 37(8):990–992, 2006. doi: 10.1080/00207390500285867.

[35] Ken Lang. The 20 newsgroups dataset. `http://qwone.com/~jason/20Newsgroups/`, 1995. Accessed: 2024-10-25.

[36] Cony M. Lau and Thomas L. Markham. LU factorizations. *Czechoslovak Mathematical Journal*, 29(4):546–550, 1979. doi: 10.21136/CMJ.1979.101635.

[37] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. Dataset available at `https://github.com/cvdfoundation/mnist`.

[38] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

[39] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. URL `https://arxiv.org/abs/1802.03426`.

[40] Wes McKinney. *Python for Data Analysis*. O'Reilly Media, Sebastopol, 3rd edition, 2022. ISBN 978-1098104030.

[41] E. H. Moore. On the reciprocal of the general algebraic matrix. *Bulletin of the American Mathematical Society*, 26:394–395, 1920.

[42] Pavel Okunev and Charles R. Johnson. Necessary and sufficient conditions for existence of the LU factorization of an arbitrary matrix, 2005. URL `https://arxiv.org/abs/math/0506382`.

[43] J. Padgett. `https://home.uchicago.edu/~jpadgett/`. Accessed: 2024-04-01.

[44] R. Penrose. A generalized inverse for matrices. *Proceedings of the Cambridge Philosophical Society*, 51:406–413, 1955.

[45] ProPublica. The NYPD files: Search thousands of civilian complaints against New York City police officers, 2020. URL `https://www.propublica.org/datastore/dataset/civilian-complaints-against-new-york-city-police-officers`. Accessed: 2024-06-24.

[46] Ross Quinlan. Auto MPG, UCI Machine Learning Repository. `https://archive.ics.uci.edu/dataset/9/auto+mpg`, 1993. Accessed: 2024-04-26.

[47] K. Sparck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.

[48] Ulrike von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007. doi: 10.1007/s11222-007-9033-z. URL `https://arxiv.org/abs/0711.0189`.

[49] Robert A. Wagner and Michael J. Fischer. The string-to-string correction problem. *Journal of the ACM*, 21(1):168–173, 1974. doi: 10.1145/321796.321811.

[50] Tai-Lin Wang. Convergence of the tridiagonal qr algorithm. *Linear Algebra and its Applications*, 322(1):1–17, 2001.

[51] David S. Watkins. *Fundamentals of Matrix Computations*. John Wiley & Sons, Hoboken, NJ, 3rd edition, 2010. ISBN 978-0470528334.

[52] Helmut Wielandt. Unzerlegbare, nicht-negative matrizen. *Mathematische Zeitschrift*, 52:642–648, 1950.

[53] Stephen J. Wright and Benjamin Recht. *Optimization for Data Analysis*. Cambridge University Press, Cambridge, 2022. ISBN 978-1316518984. doi: 10.1017/9781009004282.

[54] Wayne W. Zachary. An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4):452–473, 1977.

Index

A

addition 49
  properties of 50
adjacency matrix 95, 98, 967
affine space 467
algebraic multiplicity 539
angle between vectors 118, 356
Arnoldi algorithm 641, 648
  shift-and-invert 653
Arnoldi breakdown 644, 649, 652
Arnoldi relation 643

B

backward substitution 298, 299, 309
bag-of-words model 28
banded matrix 15
basis 213
basis extension lemma 214
best rank-k approximation 852
between-class scatter matrix 770
between-class variance 764
bidiagonal reduction 434
block
  entry 78
  matrix 78
  matrix addition 78
  matrix multiplication 80
  matrix transposition 79

C

canonical basis for $\mathbb{R}^n$ 117
Cauchy-Schwarz inequality 115
centrality 97
  closeness 194
  Katz 97
  PageRank 985
change of basis 674, 683
characteristic equation 539
characteristic polynomial 539
Cholesky algorithm 738
Cholesky factorization 703
classification problem 5, 107, 179, 762
closeness centrality 194
clustering problem 7, 182, 751
colon notation 13
color quantization 186
column space 231
column stochastic matrix 927
conformable
  for block multiplication 80
  for multiplication 54
congruence 693
conjugate 169
conjugate transpose 170, 537
convergent sequence of matrices 570
core unit lower triangular matrix 304
  properties of 305
cosine similarity 198, 200
covariance 717
covariance matrix 719
CUR factorization 267

D

data matrix 17
  centered 862
degree matrix 340, 708
determinant 294, 546, 806
diag 15
diagonal matrix 14, 58
diagonal shift 569
diagonalizable matrix 548
  characterization of 549, 552
diagonally dominant matrix 724
  strictly 656

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

M. I. Bueno Cachadina and J. Perez Alvaro, *Matrix Methods in Data Analysis*, Texts in Applied Mathematics 80, https://doi.org/10.1007/978-3-032-11314-6

digraph 23
dimension 213
  of a direct sum 217
dimensionality reduction 6, 7, 442, 762, 767, 777, 784, 856
  supervised 6
  unsupervised 7
direct sum of subspaces 215
  dimension of 217
directed graph 22
  period of 925
distance 136
  angular 141, 171
  Euclidean 137
  Hamming 143
  induced by a norm 137
  Jaccard 147, 156
  Levenshtein 144
  spherical 142
  Taxicab 137
document-term matrix 29, 32
dominant eigenvalue 574, 588
dominant eigenvector 574
dot product 52, 115, 727
  complex 537
doubly stochastic matrix 927

E

echelon form
  reduced-row 226
eigenpair 538
eigenspace 541
eigenvalue 538
  regular 545
eigenvector 538
  left 928
elementary
  matrix 219
  operation 218, 297
  permutation 221
  permutation matrix 318
  type I matrix 220
  type II matrix 221
  type III matrix 223, 301, 304
encoding 19
  one-hot 19
  ordinal 19
equivalent norms 124
Erdős number 45, 96

F

F-ANOVA score 766
factorization 76
  Cholesky 703
  CUR 267
  full-rank 238, 252, 255
  LU 301
  QR 391
  SVD 799
feature scaling 177
Fisher score 766, 771
forward substitution 300, 309
full column rank 245
full rank 245
full row rank 245
full-rank factorization 238, 252, 255
fundamental subspaces 242
  dimension of 247
  of Gram matrix 253
  orthogonal complements of 374
  orthogonal decompositions 374

G

Gaussian elimination 297, 307
Gaussian elimination with partial pivoting 321, 325
generalized eigenvalue problem 734
geometric multiplicity 543
GEPP 321, 325
Gershgorin's circle theorem 625
Givens rotator 382, 421
Google matrix 984
Gram matrices
  fundamental subspaces of 253
Gram matrix 253, 702
  invertibility of 254
Gram-Schmidt process 360, 362
  as orthogonal projections 475
  as triangular orthogonalization 419
  in matrix form 365
  modified 367
graph 22, 93
  connected 710
  connected component of 710
  directed 22
  Laplacian matrix of a 340
  notation for a 25
  regular 972
  simple 22
  undirected 22
  weighted 24

H

Haar basis 442
Haar discretization 447
Haar functions 442
Haar matrix 448
Haar recurrence 449

Haar transform
discrete version 450
Hadamard product 87, 93
Hermitian matrix 679
Hessenberg matrix 14
Hessenberg reduction 430, 646
Hessian matrix 714
Householder algorithm 402
Householder algorithm with column pivoting 408
Householder reflector 387, 421
Householder reflector theorem 389

I

identity matrix 13
inertia 183
information retrieval 7, 110, 197
invariant subspace 555
inverse matrix 56, 293, 806
eigenvalues of 546
left 59
one-sided 59
properties of 57
right 59
inverse matrix theorem 249
inverse of a product 57
invertible matrix 56
irreducible matrix 915
characterization of 919

J

Jaccard similarity 199
Jordan block 553
Jordan canonical form 554, 575

K

Katz centrality 97
kernel of a transformation 240
Kronecker product 88, 93
Krylov sequence 588, 647
Krylov subspace 647
Ky-Fan theorem 736

L

Laplacian matrix 340, 708
normalized 712
LDA 767
LDL^T factorization 314, 315
leading principal minor 697
leading principal submatrix 311, 696
least squares problem 478, 826
least norm solution 485
linearly constrained 491
minimum norm solution 827
regularized 489, 843
solution of 478
solution using pivoted QR factorization 482
solution using QR factorization for full column rank case 480
weighted 487
left inverse 822
left nullspace 238
linear combination 67, 69, 212
Linear Discriminant Analysis 767
linear independent vectors 212
linear isometry 376
linear model 514
linear system 59, 69, 291, 297, 308, 326, 826
least norm solution of 505
minimum norm solution 826
linear transformation 60, 233, 241
one-to-one 241
onto 233
link matrix 979
low-rank approximation 851
LU factorization 294, 301, 307, 747
existence of 311, 331
generalized 335
rectangular matrix 331
uniqueness of 313, 335
LU factorization with column pivoting 325

M

magnitude 537
Markov chain 938, 940
aperiodic 944
aperiodic state 942
homogeneous 938
initial probability distribution 934
irreducible 942
limiting probability distribution 946
period of a state 942
probability distribution vector 940
sample space 935
state space 934
stationary distribution 950
transition diagram 939
transition matrix 938
transition probabilities 935
Markov property 937
matrix 12
data 17
multiplication 52
by blocks 80
power 56, 95
product 52
of invertible matrices 57

sparse 14
tridiagonal 15
matrix square root 705
MNIST dataset 777
modulus 169
multipliers 306

N

nonnegative matrix 899
norm 114
axioms 114
distance induced by 137
Euclidean 115, 137
family of matrix norms 128, 130
Frobenius matrix norm 127, 814
induced matrix norm 129, 132
infinity 122
matrix p-norm 130
matrix 1-norm 130
matrix 2-norm 814
matrix infinity norm 130
mixed matrix p-norm 133
submultiplicative matrix norm 134
Taxicab 122, 137
vectorized matrix p-norm 127
normal equations 479
normal matrix 583
nuclear norm 815
nullspace 238
NumPy 40

O

Old Faithful dataset 183
one-hot encoding 19
one-to-one transformation 240
onto transformation 233
ordinal encoding 19
orthogonal complement 370
as a nullspace 371
dimension of 372
orthogonal complex vectors 367
orthogonal decomposition 373, 465
orthogonal iteration 637
orthogonal matrix 378, 421
2×2 381
characterization of 379
closest to a given matrix 838
decomposition theorem 422
determinant of 380
eigenvalues of 564
product of 380
proper 381
orthogonal projection 466
and least squares problems 478
the closest vector to a subspace 477
orthogonal projection matrix 469
characterization of 476
products of 483
properties of 475
sums of 483
orthogonal sets 370
orthogonal similarity 546
orthogonal vectors 356
orthonormal basis 358
orthonormal set 357
outer product 75
overfitting 500, 510, 516
overflow 590
overlap similarity 199

P

PageRank 985
Palmer Archipelago penguin data 107, 179
path 94
PCA 856
Pearson's correlation coefficient 155, 201, 717
permutation matrix 222, 318
properties of 319
Perron root 911, 969
Perron theorem 911, 914
Perron vector 911, 916
Perron-Frobenius theorem 916
pivot 307
PLU factorization 320, 325
polar decomposition 837
polynomial features 516
polynomial regression 462
positive definite matrix 691
Cholesky factorization of 703
diagonal 692
eigenvalue characterization of 694
Gram characterization of 702
inverse of 694
pivot characterization of 700
principal minor characterization of 697
square root characterization of 706
sum of 693
positive matrix 899
positive semidefinite
SVD characterization of 807
positive semidefinite matrix 661, 691
Cholesky characterization of 745
closest to a given matrix 839
eigenvalue characterization of 694
Gram characterization of 702
pivot characterization 747
principal minor characterization of 743
square root characterization of 706

power method 588
powers of a matrix 56, 95
primitive matrix 917
  characterization of 923, 925
principal axes theorem 731
principal component 863, 864
  features 866
  higher order 865
  reconstruction vector 858, 867
  representation vector 858, 866
  total variance explained 871
principal component analysis 856
  global approach 875
  iterative approach 864
  least reconstruction error approach 876
  variance maximization approach 859
  via the SVD 868
principal minor 695
principal submatrix 695
probability vector 122, 929
product of matrices 52
pseudodistance 171
pseudoinverse 816, 817
  algebraic characterization of 820
  limit characterization of 842
  optimization characterization of 841
  SVD formula of 818
  the four fundamental subspaces of 824

Q

QR algorithm 601, 640
  deflation of 610
  double-shifted 614
  shifted 608
  upper-Hessenberg reduction 607
QR factorization 391, 399
  and the Gram-Schmidt process 393
  of a full column rank matrix 392
  updating the 424
  with column pivoting 407, 410
quadratic form 660, 681
  indefinite 686
  negative definite 686
  negative semidefinite 686
  optimization over an ellipsoid of 733
  optimization over the unit circle of 687
  optimization over the unit circle of a sum of 735
  positive definite 686
  positive semidefinite 686

R

random variable 937
range of a transformation 233
rank 244, 245, 546
  of a product 250
  of a sum 249
rank-k
  characterization of 251
rank-nullity theorem 247
rank-one matrix 75
  characterization of 251
rating matrix 33
Rayleigh quotient 689, 776
Rayleigh quotient iteration 634
Rayleigh quotient shift 610, 634
Rayleigh-Ritz theorem 687, 688, 733, 736
recommender system 33, 199
reduced SVD 812
reducible matrix 915
regression 4, 207, 462
  generalized 511
  linear 209, 279
  local least squares 524
  multilinear 464, 514
  piecewise polynomial 501
  polynomial 463, 509
  regularized least squares 500
  weighted least squares 496
regular eigenvalue 545, 554
regular eigenvector 596
reinforcement learning 8
reverse triangle inequality 119
right inverse 822
Ritz value 649
Ritz vector 649
round-off errors 636
row equivalence 224
row space 231
row stochastic matrix 927
row-echelon form 226

S

scalar multiplication 49
scaling 177
Schatten norms 815
Schur complement 738
Schur factorization 582, 634
  real 584
semi-Cholesky factorization 745
Set
  size, union and intersection 146
shift-and-invert method 598
similar matrices 546
similarity 152
  cosine 152, 172, 198, 200
  distance-based 172
  Jaccard 155, 199
  overlap 157, 199

Pearson's correlation coefficient 155, 201
similarity transformation 546
singular matrix 56
singular value decomposition 799
full decomposition 812
geometry of 809
reduced 833
reduced decomposition 812
uniqueness of 808, 833
singular values 799
optimization characterization of 813
singular vectors 799
span 212
sparse matrix 14
spectral circle 904
spectral clustering 758
spectral embedding 775
spectral partitioning 752
spectral radius 904
Gelfand's formula 904
spectral theorem 584
spectral theorem for real symmetric matrices 585
square root of a matrix 705
stochastic matrix 927
product of 928
submatrix 13
submultiplicativity 134
subspace 211
basis of 213
dimension of 213
direct sum of 215
four fundamental 242
sum of 214
subspace dimension lemma 214
sum
of subspaces 214
supervised learning 3
SVD
full 812
reduced 812
reduced factorization 833
SVD factorization 799
symmetric matrix 314, 673, 682
symmetric permutation 915

T

tabular data 17
TF-IDF 30
trace 64, 128, 546, 735
transition matrix 578, 938
transpose 11, 50
of a product 54
properties of 50
triangle inequality 114, 121, 136, 148
reverse triangle inequality 119
triangular matrix 14, 58
the inverse of 65
tridiagonal matrix 15

U

underflow 590
undirected graph 22
unit
sphere 123
unit roundoff 611
unit vector 117
unitary matrix 381, 537
unitary similarity 546
unsupervised learning 6

V

variance 715
vector 11
vectorization 126

W

walk 94, 95
Wilkinson shift 612, 616
within-class scatter matrix 770
within-class variance 764

Z

zero matrix 13